ספר לימוד לתלמידי תיכון · מהדורת נובמבר 2026

מדעי הבינה המלאכותית

מלמידת מכונה ועד לרשתות עצביות

ספר הקורס המאוחד

חוברת יסוד • נושאי ליבה ביחידה 5 • למידה עמוקה • עבודת גמר

32 פרקים | 4 פרקי יסוד + 16 פרקי ליבה + 12 פרקי התמחות

הסבר מדורג • חישוב ידני • Python • תרשימים • משימות עצמאיות

מהדורת האיחוד משמרת את הפרקים המקוריים בשלמותם; רק דפי הפתיחה ותוכן העניינים הוחלפו.

הספר הוא חומר הוראה מקורי ואינו פרסום רשמי של משרד החינוך.

לפני שמתחילים — כיצד משתמשים בספר

הספר בנוי כסדרה של פרקים עצמאיים המתחברים למסלול אחד. מתחילים בהיכרות עם שפת Python ותכנות מונחה עצמים, מתקדמים להבנת נתונים וללמידת מכונה, ומסיימים ברשתות עצביות ובפרויקט גמר.

שלושת חלקי הלימוד

חלק	עיסוק מרכזי	מטרת הלמידה
א׳ — יסודות	Python, OOP, מבוא, EDA	הכנת התשתית המעשית לבניית מודלים
ב׳ — נושאי ליבה	וקטורים, סטטיסטיקה, קיבוץ, סיווג והערכה	שליטה מתמטית ויכולת נימוק לבגרות
ג׳ — התמחות	רגרסיה, עיבוד תמונה, רשתות עצביות ופרויקט	יישום מעשי, ניסוי והגנה על עבודת הגמר

איך לומדים כל פרק?

1. קוראים תחילה את סיפור הפתיחה ואת ההסבר המילולי. 2. בודקים את ההמחשה ומחשבים ביד. 3. מריצים את הקוד המודרך ומשווים תוצאות. 4. פותרים את התרגול העצמאי במחברת Colab. 5. כותבים מסקנה מנומקת ולא מסתפקים בפלט.

מקורות והערכה

ההסברים נכתבו כהתאמה לתלמידי תיכון בהשראת ספרי לימוד מרכזיים בתחום, ובהם Grokking Machine Learning, Mathematics for Machine Learning, Hands-On Machine Learning, An Introduction to Statistical Learning, StatQuest, Dive into Deep Learning, Understanding Deep Learning. התכנים, הגרפים והתרגילים המקוריים נותרו במקומם.

היקפי השעות במפות התכנית: 90 שעות לנושאי ליבה ו־450 שעות למסלול ההתמחות, שמתוכן 85 שעות מוקדשות לפרקי היסוד. היקפים אלה אינם מתורגמים אוטומטית למספר עמודי הספר. נדרשת התאמה למחוון ולסילבוס התקפים לשנת ההיבחנות, בטרם מציגים את הספר ככיסוי רשמי מלא.

על תלמידי הפרויקט לשמור גם קובצי נתונים, קוד, יומן ניסויים, תיעוד וקובצי המודל בנפרד מן הספר.

תוכן העניינים — חוברת יסוד ונושאי ליבה

מספרי העמודים מתייחסים למהדורת PDF המאוחדת, ולא לעימוד Word לאחר עריכה.

חלק	פרק	עמוד
יסוד	פרק 1 — Python	5
יסוד	פרק 2 — OOP	23
יסוד	פרק 3 — מבוא למדעי הנתונים ולמידת מכונה	41
יסוד	פרק 4 — חקר נתונים (EDA)	58
ליבה	ליבה 1.1 — וקטורים	72
ליבה	ליבה 1.2 — מרחק ודמיון בין וקטורים	96
ליבה	ליבה 2.1 — מדדי מרכז ופיזור	118
ליבה	ליבה 2.2 — התפלגות נורמלית	136
ליבה	ליבה 2.3 — תקנון ונרמול	154
ליבה	ליבה 2.4 — יחסים בין משתנים	170
ליבה	ליבה 2.5 — קריאת גרפים וייצוג נתונים	185
ליבה	ליבה 3.1 — מבוא ללמידה לא מונחית ולקיבוץ	199
ליבה	ליבה 3.2 — אלגוריתם K-Means	212
ליבה	ליבה 3.3 — הערכת איכות קיבוץ	226
ליבה	ליבה 4.1 — מבוא ללמידה מונחית	241
ליבה	ליבה 4.2 — Nearest Centroid	255
ליבה	ליבה 4.3 — KNN	269
ליבה	ליבה 4.4 — השוואת מסווגים מבוססי מרחק	284
ליבה	ליבה 5.1 — הערכת ביצועי מודלים	298
ליבה	ליבה 5.2 — ולידציה וחקר היפר־פרמטרים	311

תוכן העניינים — התמחות בלמידה עמוקה

פרקי ההתמחות 1–4 מופיעים בחלק היסוד בתחילת הספר. הרשימה כאן ממשיכה מפרק 5.

עמוד	פרק	מסלול
326	התמחות 5 — רגרסיה ליניארית	התמחות
342	התמחות 6 — עיבוד תמונה	התמחות
356	התמחות 7 — סיווג ו־KNN מתקדם	התמחות
374	התמחות 8 — מושגי יסוד בלמידת מכונה	התמחות
389	התמחות 9 — פרספטרון	התמחות
402	התמחות 10 — מיני־פרויקט	התמחות
416	התמחות 11 — מתמטיקה ללמידה עמוקה	התמחות
433	התמחות 12 — רשתות נוירונים עמוקות	התמחות
446	התמחות 13 — רשתות קונבולוציה (CNN)	התמחות
462	התמחות 14 — ניהול פרויקט במדעי הנתונים	התמחות
475	התמחות 15 — רשתות חוזרות (RNN)	התמחות
490	התמחות 16 — עבודת גמר	התמחות

מפת הספר — קישורים ישירים

בקובץ ה־PDF מופיעות סימניות צד לכל 32 הפרקים, בשלושה חלקים: יסודות, ליבה והתמחות. הן מאפשרות מעבר ישיר לכל פרק גם כאשר קוראים את הספר בטלפון או במחשב.

הכנה לבגרות ולפרויקט גמר

נושאי הליבה מספקים את הכלים המתמטיים, הסקת המסקנות והנימוקים הנדרשים ללמידה עיונית. פרקי ההתמחות מרחיבים אותם לניסויים, מימושים ופרויקטים. בפרקים עשויות להופיע הרחבות דידקטיות מעבר לסעיפי החובה.

כדי לשמור על כל התרשימים והנוסחאות ללא שינוי, עמודי הפרקים שולבו מן קובצי המקור; תוספות העריכה במהדורה זו נמצאות בעמודי הפתיחה בלבד.

חוברת היסוד לספר הקורס המאוחד

פרק 1 — היכרות חוזרת עם Python למפתח AI

מהקוד הראשון ועד לעיבוד נתונים, פונקציות ומערכים מספריים

מסלול התמחות למידת מכונה ולמידה עמוקה | כיתות י״א–י״ב | מסגרת סילבוס: 30 שעות

ספר תלמיד עצמאי • הסברים מלאים • דוגמאות מקוריות • קוד מודרך • תרגול עצמי



תהליך הלמידה בספר: לומדים לכתוב קוד כדי לשאול שאלות על נתונים — ורק בהמשך לאמן מודלים.

שאלה פותחת איך נכתוב תוכנית שמקבלת מדידות של מחשבים, מזהה מצב חריג ומכינה נתונים למחקר — בלי לאבד את היכולת להסביר כל שורה?

בפרק הזה דודו אחראי על מעבדת מחשבים בבית הספר. הוא אוסף טמפרטורות, שעות שימוש ונתוני תקלות. בתחילה הוא מחשב ממוצע ומסמן מדידות גבוהות. בהמשך הוא מעביר את הפעולות לפונקציות ומייצג את הנתונים במערכים. כל פעולה נועדה לפתור בעיה אמיתית, לא רק להדגים תחביר.

הפרק מבוסס על מפת תכנית ההתמחות שבמאגר הקורס שלנו: פרק 1 — היכרות חוזרת עם שפת תכנות, 30 שעות. חלוקת השעות הפנימית ומקרי החקר הם תכנון פדגוגי מקורי; אין בכך טענה שהפרטים מופיעים מילה במילה בסילבוס הרשמי.

מפת הדרך של הפרק

יחידת לימוד	מוקד	שעות מוצעות
1.1	סביבת Google Colab, קוד ונתונים	3
1.2	משתנים, טיפוסים, פעולות וחישובים	3
1.3	מחרוזות, קלט והדפסה קריאה	3
1.4	תנאים, ביטויים בוליאניים ולוגיקה	3
1.5	לולאות, סכימה וספירה	3
1.6	רשימות, אינדקסים והעתקת נתונים	3
1.7	מילונים ומבנה של רשומה	3
1.8	פונקציות, תכנון קוד ובדיקות	3
1.9	שגיאות, מודולים וקובצי נתונים	3
1.10	NumPy ופרויקט מסכם	3

בכל יחידה: בעיה קצרה, הסבר מושגי, דוגמה פתורה, קוד, טעויות נפוצות, משימות העברה וכרטיס יציאה. חלק מהמיומנויות מוכרות מיחידה 3; כאן מעמיקים בהן כדי להכין קוד יציב וקריא לפרויקט למידה עמוקה.

מפת הקשרים בין שלוש תכניות הלימוד

לאן ממשיכים	מה כבר נלמד / מה נלמד כאן	מסלול
הפעלת פונקציות וקוד עצמאי שאפשר להסביר בעל־פה	היכרות עם Colab, משתנים, רשימות, Pandas וניתוח ראשוני	יחידה 3 — כיתה י׳
הבנה מתמטית של פעולות NumPy ומרחקים	וקטורים, מטריצת מאפיינים, תיאור נתונים ואלגוריתמים	יחידה 5 — נושאי ליבה
OOP, EDA, טנזורים ומערכת פרויקט גמר	השלמת יסודות Python, קבצים, פונקציות ובדיקות	למידה עמוקה — פרק 1

מפת הקשרים היא תכנון דידקטי המתבסס על המסלולים במאגר; אין לראות בכל שורה דרישת חובה זהה בשלוש תכניות הלימודים.

מה נדרש לדעת לפני שמתחילים?

נדרשת היכרות בסיסית עם מושגי קלט ופלט, משתנה ופעולת חשבון. מי שכבר למד Python יוכל לבצע את מבדק הפתיחה ולעבוד מהר יותר בחלקי החזרה, אבל חשוב שלא לדלג על תרגול של פונקציות, רשימות ומבני נתונים.

מבדק פתיחה — ללא מחשב

1. מה ההבדל בין המספר 7 לבין המחרוזת "7"? איזו פעולה אפשר לבצע על כל אחד מהם?
2. אם ברשימה יש חמישה איברים, מהו האינדקס של האיבר האחרון?
3. מה פירוש הביטוי "הפונקציה מחזירה ערך"? במה הוא שונה מהדפסת ערך?
4. דודו קיבל 4 מדידות: 70, 82, 75, 89. כיצד יחשב ממוצע ומספר מדידות של 80 ומעלה?

מילון סימנים מוקדם

סימן	משמעות
=	השמה של ערך למשתנה
==	השוואת שני ערכים
:	תחילת בלוק כמו if, for או def
[]	רשימה, אינדקס או חיתוך
()	קריאה לפונקציה או קיבוץ ביטוי
None	היעדר ערך שימושי, ולא המספר אפס
shape	ממדי מערך NumPy; לא מופיע ברשימת Python רגילה

1.1 — סביבת עבודה, תאים וסדר הרצה

1 עובדים במחברת המורכבת מתאים. תא טקסט מאפשר לתאר את שאלת המחקר; תא Google Colab קוד מריץ הוראות Python ומציג פלט. היתרון הוא שניתן לשלב באותו מסמך הסבר, חישוב, טבלה וגרף. החיסרון הוא שתאים אינם תמיד רצים לפי סדר הופעתם: אם מריצים את תא 6 ואז חוזרים לתא 2, המשתנים בזיכרון עלולים להכיל ערכים שלא רואים כעת על המסך.

לכן הרגל מקצועי ראשון הוא הרצה מחדש מתחילת המחברת בסדר, ללא שימוש במשתנים שנוצרו במקרה בהרצה קודמת. בפרויקט גמר התלמיד צריך להוכיח שעמיתו או בוחן יכול לפתוח את המחברת ולשחזר את התוצאה.

דוגמה 1 — התא הראשון: שמירת נתונים והצגתם

```
# Notebook cell 1 — measurements
computer_name = "Dudu-PC-01"
temperature = 72.5
print(computer_name)
print(temperature)
```

בפקודה הראשונה הגדרנו משתנה שמחזיק טקסט. בשנייה הגדרנו משתנה שמחזיק מספר עשרוני. `print` אינו מאמן מודל ואינו משנה את הערכים: הוא רק מדפיס אותם כדי שנוכל לבדוק מה קרה.

לשאול לפני שמריצים איזה פלט יופיע? מה יקרה אם תא זה לא הורץ אבל תא אחר מנסה להשתמש ב־temperature? במקרה כזה תיתכן שגיאת NameError.

כיצד קוראים הודעת שגיאה?

כאשר שורה אינה פועלת, אין להעתיק מיד פתרון בלי להבין מה קרה. מתחילים מקריאת שורת השגיאה האחרונה: היא מתארת את סוג הבעיה. אחר כך מאתרים את השורה שאליה ההודעה מפנה ושואלים מה Python ציפה לקבל. שגיאה היא מידע על התנהגות התוכנית, לא רק כישלון של המתכנת.

קטע לדיון בלבד — מדוע סדר הרצה חשוב?

```
# This will fail if temperature was never defined:
# print(temperature + 5)
```

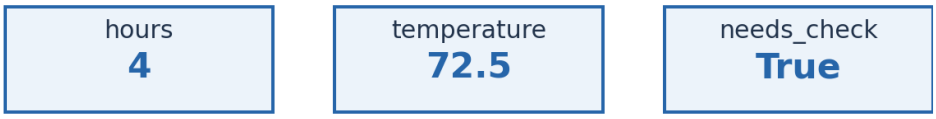
בדיקת הבנה

1. תארו שני מצבים שבהם מחברת נותנת פלט שונה בגלל סדר הרצת תאים.

2. כתבו תא שמגדיר שם מחשב, טמפרטורה ושעות שימוש; הדפיסו כל ערך בשורה משלו.

1.2 — משתנים, טיפוסים וחשבון מספרי

משתנה הוא שם שמאפשר לפנות לערך. ב־Python אין צורך להצהיר מראש על טיפוס המשתנה, אך לכל ערך יש טיפוס, והטיפוס קובע אילו פעולות אפשר לבצע עליו. מספר שלם (`int`) מתאים למשל למספר התקלות; מספר עשרוני (`float`) מתאים לטמפרטורה; מחרוזת (`str`) מתאימה לשם מחשב; ערך בוליאני (`bool`) מתאים לשאלה כן או לא.



Names refer to values; the value type matters

שלושה שמות משתנים והערכים שהם מפנים אליהם. הציור מדגיש שטיפוס הערך קובע את משמעותו.

טיפוס	דוגמה	משמעות בהקשר שלנו
int	faults = 3	מספר תקלות
float	temp = 72.5	טמפרטורה
str	name = "PC-01"	מזהה מחשב
bool	urgent = True	האם נדרש טיפול דחוף

דוגמה 2 — איך בודקים טיפוס?

```
faults = 3
temp = 72.5
name = "PC-01"
urgent = True

print(type(faults))
print(type(temp))
print(type(name))
print(type(urgent))
```

הסימן `=` מבצע השמה: לוקחים את התוצאה בצד ימין ומקשרים אותה לשם בצד שמאל. הוא אינו טענה מתמטית של שוויון. הביטוי `faults = faults + 1` תקין בתכנות: קודם מחשבים את מספר התקלות הקיים ועוד אחת, ואז מעדכנים את המשתנה.

דוגמה 3 — עדכון מצב

```
faults = 3
faults = faults + 1
print(faults)  # 4
```

אילו פעולות חשבון קיימות?

פעולה	דוגמה	תוצאה
+	7 + 2	9
-	7 - 2	5

פעולה	דוגמה	תוצאה
*	7 * 2	14
/	7 / 2	3.5
//	7 // 2	3
%	7 % 2	1
**	2 ** 3	8

שימו לב: `/` היא חלוקה רגילה ומחזירה מספר עשרוני גם כאשר החלוקה מדויקת, ואילו `//` היא חלוקת רצפה. עבור מספרים שליליים אין לפרש אותה תמיד כ״מחיקת החלק העשרוני״: למשל `-7 // 2` שווה -4. האופרטור `%` מספק שארית בהתאם לכללי Python ומשמש, בין היתר, לבדיקת זוגיות.

דוגמה פתורה: ממוצע ושליטה בסדר פעולות

דודו מדד שלוש טמפרטורות: 68, 82, 75. כדי לחשב ממוצע מחברים את הערכים ומחלקים במספרם. טעות נפוצה היא לכתוב `68 + 82 + 75 / 3`: לפי קדימות הפעולות רק המספר האחרון יחולק בשלוש.

mean = (68 + 82 + 75) / 3 = 75.0

דוגמה 4 — סוגריים מגנים על משמעות הנוסחה

```
t1, t2, t3 = 68, 82, 75
mean = (t1 + t2 + t3) / 3
print(mean)
```

למה זה חשוב ב־AI? אלגוריתם יוכל להשתמש בטמפרטורה ממוצעת בתור תכונה, אך תחילה עלינו לוודא שהחישוב מייצג את התכונה שהתכוונו אליה. מספר שהודפס ללא הודעת שגיאה עדיין עשוי להיות שגוי מבחינה מדעית.

תרגול מדורג

1. חשבו ביד ואז ב־Python את הממוצע של 60, 75, 90, 75.
2. הסבירו מה ההבדל בין `"5" + "2"` לבין `5 + 2`.
3. כתבו ביטוי המחזיר את מספר הדקות שנותרו לאחר חלוקת 137 דקות לשעות שלמות.
4. בדקו מה מחזיר `type(True)` ומדוע אין להשתמש ב־`True` כמדידה מספרית רגילה.

1.3 — מחרוזות, קלט והצגת תוצאות

הנתונים במערכת אינם רק מספרים. לפעמים צריך לייצג מזהה מחשב, שם כיתה, הודעת מצב או תאריך. כל אלה הם מחרוזות (`str`). מחרוזת היא רצף תווים: גם הספרות שבתוכה הן תווים ולא בהכרח מספרים שניתן לחבר ולממוצע. למשל `"72.5"` היא מחרוזת, בעוד `72.5` הוא מספר.

פונקציות כמו `float()` ו־`int()` מאפשרות להמיר טקסט לערכים מספריים כאשר הטקסט תקין. אם הוא מכיל מילה במקום מספר, ההמרה תיכשל; בהמשך נלמד כיצד לזהות ולטפל בכך.

דוגמה 5 — המרת טקסט ובדיקת תבנית

```
reading = "72.5"
value = float(reading)
print(value + 2)   # 74.5

computer = "PC-04"
print(computer.lower())
print(computer.startswith("PC-"))
```

פלט שקריא לאדם

הדפסת רשימת מספרים לבדה אינה מספרת לקורא מה המשמעות שלהם. לכן משתמשים במחרוזות מעוצבות (`f-string`), המאפשרות לשלב ערכים והסברים. הביטוי שבתוך הסוגריים המסולסלים מחושב לפני ההדפסה. ניתן לבקש שתי ספרות אחרי הנקודה כדי להציג ממוצע בצורה נוחה.

דוגמה 6 — תצוגה שאפשר להכניס לדוח

```
computer = "PC-04"
average = 75.333333
print(f"Computer: {computer}; average temperature: {average:.2f} C")
```

נבחין בין עיגול לצורך הצגה לבין שינוי הנתונים עצמם: הדפסת `75.33` אינה מוחקת את הדיוק של המשתנה `average`. חשוב לשמור את ערכי המקור, בייחוד לפני חישובי סטטיסטיקה או אימון מודל.

קלט מהמשתמש — מתי כן ומתי לא?

הפונקציה `input()` מאפשרת למשתמש להקליד ערך, אבל היא מחזירה מחרוזת. לכן לשם חישוב יש להמיר את הקלט. במחברות AI משתדלים לרוב להפריד בין קוד החישוב לבין קלט אינטראקטיבי, כדי שניתן יהיה להריץ את אותה מחברת באופן אוטומטי בלי להמתין למשתמש.

קוד לדיון — אינו מופעל אוטומטית במחברת

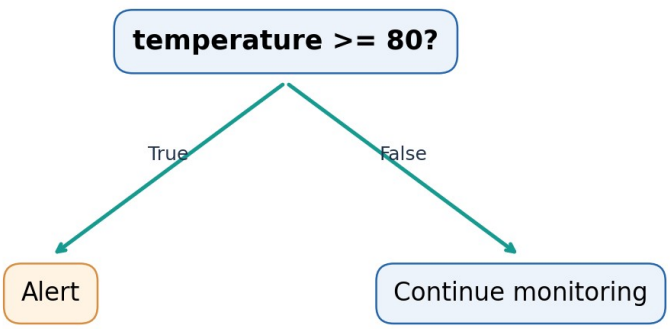
```
# Interactive example – run when you want to enter data:
# text = input("Temperature: ")
# temperature = float(text)
# print(temperature + 1)
```

בדיקת הבנה

1. כתבו שורת `f-string` המציגה מחשב וטמפרטורה עם ספרה אחת אחרי הנקודה.
2. הסבירו מדוע `int("4.5")` אינו מבצע אותה פעולה כמו `int(float("4.5"))`.
3. כתבו בדיקה שמזהה האם מזהה מחשב מתחיל באותיות `PC-`.

1.4 — תנאים ולוגיקה: מתיאור נתון להחלטה

מערכת לניטור מחשבים צריכה לעיתים לפעול אחרת לפי מצב המדידה. תנאי הוא ביטוי שמחזיר `True` או `False`. מבנה `if` מאפשר להריץ קוד רק כאשר התנאי מתקיים. החלק שמוזח פנימה שייך לבלוק התנאי; הזחה שגויה משנה את משמעות התוכנית או גורמת לשגיאה.



עץ החלטה פשוט: מדידה של 80 מעלות ומעלה מסומנת להתייחסות.

דוגמה 7 — החלטה בעזרת *if/else*

```
temperature = 82
if temperature >= 80:
    print("Temperature alert")
else:
    print("Temperature normal")
```

נוסחת ההחלטה אינה ״למידת מכונה״. היא כלל מפורש שנכתב על ידי המתכנת. ההבדל חשוב: בהמשך נלמד מודל שמנסה ללמוד כלל מנתוני עבר, אבל בשלב זה אנחנו עדיין כותבים את הכלל בעצמנו.

השוואות וצירופי תנאים

סימן	פירוש	דוגמה
==	שווה	temperature == 80
!=	שונה	faults != 0
>, >=	גדול / גדול או שווה	temperature >= 80
<, <=	קטן / קטן או שווה	faults <= 2
and	שני התנאים מתקיימים	temp > 80 and faults > 0
or	לפחות אחד מתקיים	temp > 90 or faults > 4
not	שלילת תנאי	not urgent

הביטוי `and` מחזיר אמת כאשר שני התנאים נכונים; `or` מחזיר אמת כאשר לפחות אחד נכון. בדוגמאות פשוטות הערכים הם בוליאניים, אך יש לזכור שב־Python אופרטורים אלה יכולים להחזיר גם אחד מהאופרנדים. כדי לא להקשות בתחילה נשתמש בביטויים שהשוואותיהם בוליאניות מפורשות.

דוגמה 8 — שלושה מצבים ולא רק שניים

```
temperature = 86
faults = 2
if temperature >= 80 and faults >= 2:
    status = "urgent"
elif temperature >= 80:
    status = "check temperature"
else:
    status = "normal"
print(status)
```

מקרה גבול דורש תשומת לב: האם 80 נחשב חריג או שהכלל חל רק מעל 80? תשובה כזו חייבת להופיע בהגדרת הבעיה. בקוד ההבדל הוא בין `>` לבין `>=`, אבל במערכת אמיתית מדובר בהחלטה עסקית או מדעית.

תרגול

1. עבור טמפרטורה 80, מה התוצאה של `temperature > 80` ומה של `temperature >= 80`?
2. כתבו תנאי שמזהה מחשב עם יותר משלוש תקלות או טמפרטורה של 90 ומעלה.
3. הסבירו מדוע כלל ידני בתנאי `if` אינו נחשב מודל שאומן על נתונים.

1.5 — לולאות, סכימה וספירה של נתונים

כאשר מתקבלות מאות מדידות, אי אפשר לכתוב פקודת חיבור נפרדת לכל מדידה. לולאה מאפשרת לבצע פעולה חוזרת לפי סדר על אוסף נתונים. נתחיל בלולאת `for` שמבקרת בכל מדידה, ולאחר מכן נכיר את `range` ואת לולאת `while` המתאימה למקרים שבהם ממשיכים עד שמתקיים תנאי עצירה.



Loop: visit each item and update a count or total

הלולאה עוברת לפי הסדר על ארבע מדידות; ערכים של 80 ומעלה מסומנים בצבע שונה.

דוגמה פתורה: סכום ומונה

דודו רוצה לחשב טמפרטורה ממוצעת וגם לדעת כמה מדידות הגיעו לסף 80. נצטרך שני משתנים: `total` יצבור את הסכום; `high_count` יספור מדידות חריגות. לפני תחילת הלולאה מאתחלים את שניהם באפס, כי עדיין לא עיבדנו אף מדידה.

דוגמה 9 — צבירה וספירה

```
temperatures = [68, 82, 76, 91]
total = 0
high_count = 0

for temperature in temperatures:
    total += temperature
    if temperature >= 80:
        high_count += 1

average = total / len(temperatures)
print(average)     # 79.25
print(high_count)  # 2
```

שלב	temperature	total לאחר עדכון	high_count
אתחול	—	0	0
1	6 8	6 8	0
2	8 2	150	1
3	76	226	1
4	9 1	3 1 7	2

טבלת המעקב חשובה יותר משינון התחביר: היא עוזרת להסביר מה נשאר נכון אחרי כל איטרציה. לאחר שני ערכים, למשל, `total` הוא סכום שני הערכים הראשונים ו־`high_count` הוא מספר הערכים שחצו את הסף מביניהם. הרעיון הזה יופיע בהמשך גם באלגוריתמים הלומדים מנתונים.

לולאה לפי אינדקס

דוגמה 10 — כאשר צריך גם את מיקום התצפית

```
for i in range(len(temperatures)):
    print(i, temperatures[i])
```

`range(len(...))` מחזיר אינדקסים מ־0 ועד אחד פחות ממספר האיברים. בהמשך, כאשר צריך לעבור על כמה מערכים מקבילים, חשוב לדעת מה מייצג האינדקס. אם לא צריך מיקום, נעדיף בדרך כלל מעבר ישיר על הערכים כדי לשמור על קוד פשוט.

מתי נשתמש ב־while?

דוגמה 11 — עם תנאי עצירה ברור *while*

```
index = 0
while index < len(temperatures):
    print(temperatures[index])
    index += 1
```

בלולאת `while` חובה לוודא שמשהו משתנה וכך מקדם אותנו לעבר תנאי העצירה. אם שכחנו `index += 1`, נקבל לולאה שאינה מסתיימת. ב־Colab אפשר לעצור את ההרצה, אבל עדיף ללמוד לזהות את הבעיה לפני שהיא מתרחשת.

זהירות: רשימה ריקה אם אין מדידות, `len(temperatures)` הוא אפס. חלוקה ב־0 תגרום ל־ZeroDivisionError. בהמשך נלמד טיפול בקלט חסר; כבר עכשיו חשוב להגדיר כיצד התוכנית צריכה להגיב.

תרגול העברה

1. חשבו ביד את הסכום ואת מספר המדידות מעל 85 ברשימה `[65, 86, 80, 92, 79]`.
2. כתבו לולאה המדפיסה רק מדידות בטווח 70–85 (כולל גבולות).
3. הסבירו מתי תעדיפו `for temperature in temperatures` ומתי תצטרכו `range(len(...))`.
4. תכננו שינוי שימנע חלוקה באפס במקרה של רשימה ריקה.

1.6 — רשימות, אינדקסים והעתקת נתונים

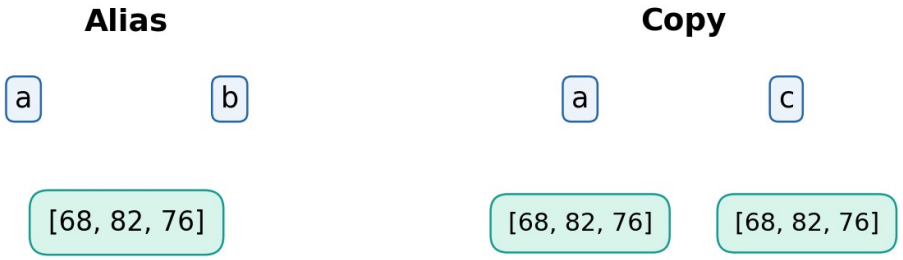
רשימה (`list`) מאפשרת לשמור כמה ערכים באוסף מסודר. האינדקסים מתחילים באפס, ולכן האיבר הראשון נמצא ב־`temperatures[0]`. רשימה ניתנת לשינוי: אפשר להוסיף איבר, להחליף איבר ולמחוק איבר. תכונה זו שימושית, אך גם דורשת זהירות כדי לא לשנות נתוני מקור בלי כוונה.

דוגמה 12 — גישה, חיתוך והוספה

```
temperatures = [68, 82, 76, 91]
print(temperatures[0])    # first value
print(temperatures[-1])   # last value
print(temperatures[1:3])  # values at indexes 1 and 2

temperatures.append(74)
print(len(temperatures))  # 5
```

בביטוי `[1:3]` הגבול השמאלי כלול והגבול הימני אינו כלול. לכן נקבל את האיברים במקומות 1 ו־2. חיתוך של רשימה יוצר רשימה חדשה; לעומת זאת, השמה `b = a` יוצרת שם נוסף לאותו אובייקט רשימה. ההבדל יכול ליצור טעויות שקשה לאתר בניסויי AI.



שני שמות יכולים להפנות לאותה רשימה; העתקה יוצרת רשימה עצמאית ברמה הראשונה.

דוגמה 13 — כינוי לעומת העתקה

```
original = [68, 82, 76]
alias = original
copy_of_values = original.copy()

alias[0] = 999
print(original)       # [999, 82, 76]
print(copy_of_values) # [68, 82, 76]
```

`copy()` יוצרת העתקה שטחית. עבור רשימה של מספרים זה מספיק בדרך כלל. אם הרשימה כוללת רשימות פנימיות, השינויים ברשימות הפנימיות עדיין עשויים להשפיע על שני המבנים. נרחיב על מבנים מקוננים רק לאחר שההבדל הבסיסי ברור.

משמעות ל־AI: לשמור נתוני מקור

נניח שתלמיד מחליף ברשימה את כל המדידות שמעל 80 בערך 80. זו פעולה שמתקנת לכאורה קלט חריג, אבל היא גם מוחקת מידע. אם לא נשמר עותק של נתוני המקור, קשה להבין מה בוצע וייתכן שנאמן בהמשך מודל על נתונים מעוותים. כל שלב בעיבוד נתונים צריך להיות מתועד ובעל הצדקה.

תרגול

1. הסבירו מה תחזיר הפקודה `temperatures[-2]` כאשר הרשימה כוללת ארבעה איברים.
2. כתבו קוד היוצר עותק של רשימת מדידות וממיר רק בעותק ערכים שליליים ל־0.
3. מדוע השורה `b = a` אינה מגינה על רשימת המקור?
4. מה יקרה אם ננסה לגשת ל־`temperatures[100]`? איך נמנעים מכך?

1.7 — מילונים, רשומות ומערכי נתונים פשוטים

כאשר לכל מחשב יש יותר ממאפיין אחד, לא נוח לשמור כמה רשימות מקבילות בלי הסבר. מילון (`dict`) שומר קשרים בין מפתחות לערכים, למשל `"name"` למזהה המחשב ו־`"temperature"` למדידה. רשימת מילונים מאפשרת לייצג אוסף רשומות, שכל אחת מהן מתארת תצפית אחת.

דוגמה 14 — תצפית כמילון

```
computer = {
    "name": "PC-01",
    "temperature": 72.5,
    "hours": 4,
    "faults": 1
}
print(computer["temperature"])
print(computer.get("location", "unknown"))
```

הגישה `computer["temperature"]` מניחה שהמפתח קיים; אם אינו קיים נקבל `KeyError`. לעומת זאת, `get` מאפשר להחזיר ערך חלופי כאשר המפתח חסר. אין לפרש ערך חסר כאפס באופן אוטומטי: אפס תקלות הוא נתון, ואילו ״אין לנו מידע על מספר תקלות״ הוא מצב אחר לגמרי.

דוגמה 15 — רשימה של תצפיות

```
computers = [
    {"name": "PC-01", "temperature": 72, "faults": 1},
    {"name": "PC-02", "temperature": 87, "faults": 3},
    {"name": "PC-03", "temperature": 76, "faults": 0}
]
```

```
for pc in computers:
    print(pc["name"], pc["temperature"])
```

מה ההבדל בין מפתח, מאפיין ותווית?

המפתח במילון הוא שם של שדה, למשל `temperature`. המאפיין (Feature) הוא ערך שמשתמשים בו כדי לתאר את התצפית בזמן החיזוי. התווית או משתנה המטרה (Target) היא התשובה שהמודל יתבקש לחזות, אם קיימת. בחירת תכונה אינה רק החלטה טכנית: אסור לכלול בין תכונות החיזוי מידע שלא יהיה זמין כאשר המערכת תידרש לפעול.

לדוגמה, אם רוצים לחזות האם מחשב יתקלקל מחר, מספר התקלות שיתרחשו בשבוע הבא אינו מאפיין לגיטימי של היום. בהמשך נקרא לבעיה של שימוש במידע עתידי בשם "זליגת נתונים" (Data Leakage).

היכרות שימושית — Set־ו Tuple

בנוסף לרשימה ולמילון יש שני מבנים נפוצים: `tuple` הוא רצף מסודר שאינו ניתן לשינוי לאחר יצירתו (אם כי אובייקטים משתנים בתוכו עשויים להשתנות), ו־`set` הוא אוסף ללא סדר אינדקסים קבוע וללא כפילויות. לא נעמיס כאן ניתוח מעמיק; התלמיד צריך לזהות אותם בקוד ולבחור מבנה נתונים מתאים.

דוגמה 16 — *tuple* לממדים ו־*set* לזיהוי ערכים ייחודיים

```
resolution = (8, 8)  # two dimensions
labels = ["ok", "alert", "ok", "check"]
unique_labels = set(labels)
print(resolution)
print(unique_labels)
```

תרגול

1. תכננו רשומת מחשב עם שדות `id`, `hours`, `temperature`, `status`.
2. הסבירו מדוע מידע חסר על תקלות אינו שקול לערך 0.
3. מה תהיה משמעות תווית `needs_repair` בפרויקט סיווג? מתי היא נודעת?
4. כתבו לולאה שמדפיסה את שמות המחשבים שהטמפרטורה שלהם 80 ומעלה.

1.8 — פונקציות: להפוך חישוב לפעולה ברורה ואמינה

תוכנית אמיתית מורכבת מכמה פעולות החוזרות במקומות שונים: בדיקת מדידות, חישוב ממוצע, יצירת סיכום, שמירת קובץ. אם נעתיק את אותן שורות שוב ושוב, כל תיקון יחייב אותנו למצוא את כל העותקים. פונקציה מאפשרת לתת לפעולה שם, להגדיר קלטים ולהחזיר תוצאה. זהו אחד המעברים החשובים מכתיבת פקודות בודדות לכתיבת מערכת שניתן לתחזק.



פונקציה מקבלת קלט ומחזירה תוצאה: אותו חישוב ניתן להפעלה על מדידות שונות.

דוגמה 17 — קלט, שם פעולה וערך מוחזר

```
def average_temperature(values):
    """Return the average of a non-empty sequence."""
    return sum(values) / len(values)

readings = [68, 82, 76]
result = average_temperature(readings)
print(result)
```

המילה `def` מגדירה פונקציה; `values` הוא פרמטר מקומי, שמקבל בזמן הקריאה את הרשימה שהעברנו; `return` מחזיר את התוצאה למקום שממנו קראנו לפונקציה. אם היינו כותבים `print` במקום `return`, הפונקציה הייתה מציגה ערך למסך אבל לא מאפשרת להשתמש בו בנוחות בחישובים הבאים.

הפרדה בין חישוב להצגה

דוגמה 18 — פונקציה בוליאנית וערך ברירת מחדל

```
def is_hot(temperature, threshold=80):
    return temperature >= threshold

print(is_hot(82))
print(is_hot(82, threshold=90))
```

הדוגמה גם מלמדת על ערך ברירת מחדל: אם המשתמש אינו מעביר סף, משתמשים ב־80; אם מעביר סף אחר, נקבל החלטה בהתאם אליו. חשוב לא לשנות ערכי סף סתם כדי להתאים תוצאה רצויה, אלא לקבוע אותם לפי הגדרת הבעיה ולתעד את ההחלטה.

בדיקות וקלט לא תקין

הפונקציה הקודמת לחלוקה בממוצע הניחה שהרשימה אינה ריקה. אפשר להפוך את ההנחה למפורשת ולשלוח הודעת שגיאה ברורה כאשר אין נתונים. לא תמיד צריך ״לתקן״ קלט לא תקין במקום; לעיתים נכון לעצור את החישוב כדי למנוע הפקת תוצאה מטעה.

דוגמה 19 — הגדרת תנאי תקינות לפונקציה

```
def average_temperature(values):
    if len(values) == 0:
        raise ValueError("Expected at least one temperature")
    return sum(values) / len(values)
```

משתנים שנוצרים בתוך הפונקציה אינם בהכרח זמינים מחוץ לה. זהו רעיון תחום ההכרה של משתנים (Scope): הוא מונע תלות מיותרת בין חלקי תוכנית. פרק התכנות מונחה העצמים יתבסס על היכולת הזאת להפריד אחריות בין רכיבים.

בדיקות עם assert

דוגמה 20 — שלוש בדיקות אוטומטיות קטנות

```
assert average_temperature([60, 80]) == 70
assert is_hot(80) is True
assert is_hot(79) is False
```

בדיקה טובה כוללת גם מקרה רגיל וגם מקרה גבול. עם זאת, `assert` איננו מנגנון אימות קלט למערכת ייצור: ניתן לבטל אותו בהרצות Python מסוימות. כאן הוא כלי לימודי מהיר לבדיקת נכונות של פונקציות. בהמשך נלמד לכתוב בדיקות מסודרות.

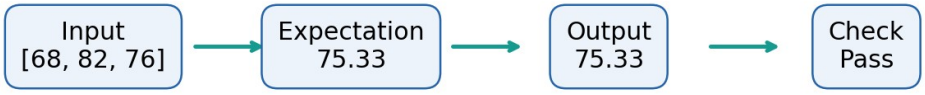
תרגול

1. כתבו פונקציה `count_hot(values, threshold=80)` המחזירה את מספר המדידות שעומדות בסף.
2. הסבירו מדוע פונקציה שמחזירה מספר עדיפה לעיתים על פונקציה שרק מדפיסה אותו.
3. הגדירו שלוש בדיקות: רשימה רגילה, רשימה שבה אף מדידה אינה חריגה ומקרה גבול בדיוק בסף.
4. תכננו מה הפונקציה אמורה לעשות כשהרשימה ריקה; נמקו את הבחירה.

1.9 — שגיאות, מודולים, קבצים ושחזור

תלמיד שמפתח מערכת AI חייב ללמוד לא רק לכתוב תוכנית שעובדת בדוגמה אחת, אלא גם לברר מדוע תוכנית אינה עובדת או מחזירה תוצאה חשודה. נהוג להבחין בין שגיאת תחביר (`SyntaxError`), שגיאת זמן ריצה כגון `IndexError` או `ValueError`, ושגיאת לוגיקה שבה הקוד רץ אך מחשב דבר אחר מהנדרש.

סוג הבעיה	דוגמה	דרך בדיקה
תחביר	חסר : אחרי if	קריאת שורת השגיאה ותיקון הסימן
זמן ריצה	גישה לאינדקס לא קיים	בדיקת אורך הרשימה והאינדקס
לוגיקה	ממוצע ללא סוגריים	חישוב ידני והשוואה לפלט
נתונים	טמפרטורה חסרה	בירור מדיניות ערכים חסרים



כל בדיקה משווה פלט שהתקבל לפלט צפוי, ולא רק בודקת שהקוד סיים לרוץ.

למה צריך import?

Python כוללת ספרייה תקנית עשירה, לצד ספריות חיצוניות. באמצעות `import` ניתן להשתמש בפונקציות שהוגדרו במודולים אחרים. `math` מספקת פעולות מתמטיות, `csv` קריאה של קבצי CSV, ו־`pathlib` טיפול בנתיבי קבצים. מודולים כגון `numpy` ו־`pandas` הם ספריות חיצוניות שמשמשות במיוחד במדעי הנתונים.

דוגמה 21 — מודולים מספרייה תקנית

```
from pathlib import Path
from math import sqrt

folder = Path(".")
print(folder.resolve())
print(sqrt(81))
```

קריאת CSV — לא להניח שכל קובץ זהה

קובץ CSV מייצג נתונים טבלאיים באמצעות טקסט. שורה אחת עשויה להכיל כותרות, והשורות הבאות ערכי תצפיות. אך אין להניח שהנתונים תקינים: ייתכנו שדות ריקים, מפרידים שונים ומספרים שנשמרו כטקסט. בפרק EDA נעמיק בבדיקות ובניקוי; כאן נלמד לזהות את המבנה ולקרוא קובץ בצורה בסיסית.

דוגמה 22 — יצירה וקריאה של *CSV* קטן

```
import csv
from pathlib import Path

path = Path("computers_demo.csv")
with path.open("w", newline="", encoding="utf-8") as f:
    writer = csv.writer(f)
    writer.writerow(["name", "temperature", "hours"])
    writer.writerow(["PC-01", 72, 4])
    writer.writerow(["PC-02", 87, 8])

with path.open("r", newline="", encoding="utf-8") as f:
    rows = list(csv.DictReader(f))
print(rows)
```

הערכים ש־`csv.DictReader` קוראת הם בדרך כלל מחרוזות, ולכן לפני חישוב ממוצע יש לבצע המרות ולבדוק תקינות. זוהי דוגמה נוספת להבחנה בין סוג הנתון כפי שנשמר בקובץ לבין המשמעות המספרית שלו. בפרויקט גמר יש לשמור מקור וגרסת קובץ ולא לדרוס אותו ללא צורך.

בדיקה מול מקרה מוכר

דוגמה 23 — התאמה בין תוצאה ידנית למימוש

```
def calc_average(values):
    if not values:
        raise ValueError("No values")
    return sum(values) / len(values)

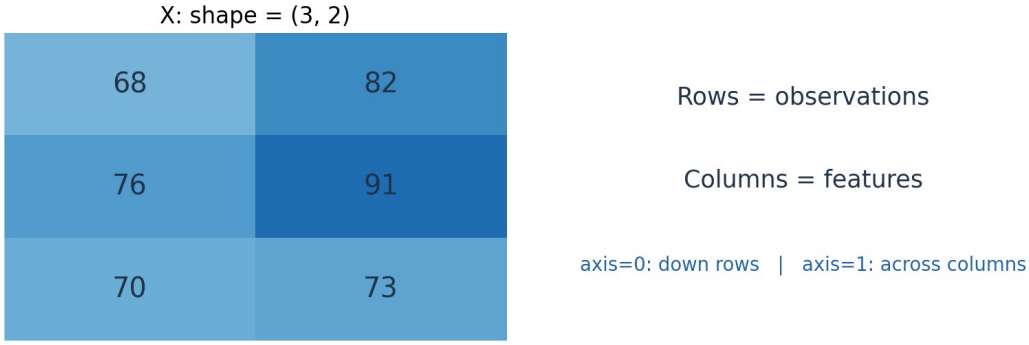
assert calc_average([68, 82, 76]) == 226 / 3
```

תרגול

1. תארו שגיאת לוגיקה שעלולה לעוות ממוצע גם כאשר אין הודעת שגיאה.
2. שנו את דוגמת ה־CSV כך שתכיל שלושה מחשבים ובדקו מה סוג הערכים שנקראו.
3. מדוע חשוב לרשום את מקור קובץ הנתונים ואת קוד ההכנה שלו בפרויקט AI?
4. תכננו בדיקה למקרה שבו קובץ אינו נמצא בתיקייה הרצויה.

1.10 — NumPy: מעבר מרשימות למערכים מספריים

רשימות Python נוחות לצעדים הראשונים, אך בעיבוד נתונים מספריים אנו עובדים לעיתים על אלפי ערכים, מטריצות ותמונות. NumPy מאפשרת לייצג מערכים רב־ממדיים בעלי טיפוס אחיד ולהפעיל עליהם פעולות וקטוריות. המשמעות היא לא רק כתיבה קצרה יותר: הצורה (`shape`) של המערך אומרת כמה תצפיות וכמה מאפיינים יש לנו.



מטריצה X בעלת שלוש שורות ושני מאפיינים. הצורה קובעת מה מייצג כל ציר.

דוגמה 24 — שלוש תצפיות ושני מאפיינים

```
import numpy as np

X = np.array([
    [68, 4],
    [82, 7],
    [76, 5]
], dtype=float)

print(X.shape)         # (3, 2)
print(X[:, 0])        # first feature for all computers
print(X.mean(axis=0)) # feature means
```

כאשר `axis=0` מפעילים חישוב לאורך שורות, כך שמתקבלת תוצאה לכל עמודה; ממוצע שתי העמודות בדוגמה הוא 75.33 לטמפרטורה ו־5.33 לשעות שימוש. כאשר `axis=1`, מתקבלת תוצאה לכל שורה. לא כדאי לחשב ממוצע של טמפרטורות ושעות יחד: היחידות שלהן שונות, והתוצאה אינה מדד בעל משמעות מוגדרת.

פעולות איבר־איבר ו־Broadcasting

דוגמה 25 — אותה פעולה על כל איבר

```
temperatures = np.array([68, 82, 76], dtype=float)
shifted = temperatures + 2
scaled = temperatures / 100
print(shifted)
print(scaled)
```

כאשר מוסיפים 2 למערך, NumPy מחילה את החיבור על כל איבר. לכן אפשר לחשוב על ההוראה כעל לולאה נסתרת, אך בביצוע המקצועי זו פעולה מערכית. Broadcasting מערכים בעלי צורות מתאימות יכולים להשתתף באותה פעולה בלי ליצור במפורש את כל העותקים. בפרקי הווקטורים ו־KNN שבספר הגדול נעמיק במנגנון ובתנאי ההתאמה בין ממדים.

דוגמה: הכנת מאפיינים לפני מודל

דוגמה 26 — מהו וקטור תכונות?

```
X = np.array([
    [68, 4],
    [82, 7],
    [76, 5]
], dtype=float)

feature_means = X.mean(axis=0)
print(feature_means)
print(X[0])
```

כל שורה ב־X מייצגת מחשב אחד וכל עמודה מאפיין. בעתיד נוסיף מערך `y` שבו נשמרות התוויות הידועות, אם מדובר בלמידה מונחית. חשוב לשמור תמיד על התאמה בין סדר השורות ב־X לבין סדר התוויות ב־y; ערבוב סדר אחד מהם ישבש את הלמידה גם אם הקוד רץ.

מיני־פרויקט מסכם: ניטור מעבדת המחשבים

דודו רוצה לקבל דוח קצר על מצב המעבדה. הוא צריך לוודא שיש נתונים, לחשב טמפרטורה ממוצעת, לספור מדידות שעברו סף, לכתוב פונקציה נפרדת לקבלת החלטה ולהכין מטריצת מאפיינים. זהו תהליך עיבוד נתונים מבוסס כללים; הוא עדיין אינו מודל למידת מכונה, משום שהמחשב לא למד את הכלל מתוך דוגמאות מסומנות.



Rule-based analysis is not yet a trained machine-learning model

תהליך מיני־הפרויקט: איסוף, בדיקה, סיכום, סימון והסבר התוצאה.

דוגמה 27 — מימוש מודרך של שלבי סיכום הנתונים

```
import numpy as np

readings = [68, 82, 76, 91]

def summarize(values, threshold=80):
    if len(values) == 0:
        raise ValueError("At least one reading is required")
    average = sum(values) / len(values)
    high_count = sum(value >= threshold for value in values)
    return average, high_count

average, high_count = summarize(readings)
print(f"Average: {average:.2f}; high readings: {high_count}")

X = np.array(readings, dtype=float).reshape(-1, 1)
print("Feature matrix shape:", X.shape)
```

`reshape(-1, 1)` משנה רשימה חד־ממדית למטריצה בעלת עמודה אחת. הסימון -1 מבקש מ־NumPy לחשב את מספר השורות הנדרש על פי כמות האיברים. עיבוד הנתונים כאן פשוט בכוונה, כדי שכל תלמיד יוכל להסביר אותו. בשלב מתקדם יותר נחליף את סף הכלל במודל שמאומן מנתונים, ונבדוק אותו על קלט שלא שימש לאימון.

משימת סיכום עצמאית — בלי קוד פתרון

במעבדת Colab המלווה את הפרק תבנו דוח למערכת ספרייה או למערכת ניטור אנרגיה על בסיס אותו רעיון, אך עם נתונים ושאלות אחרות. המחברת כוללת תאים להשלמת קוד, מקרים לבדיקה ושאלות לנימוק. אל תעתיקו את דוגמת דודו כפי שהיא: מטרת המשימה היא להוכיח שאתם יכולים להעביר את המושגים להקשר חדש.

שאלות חשיבה בסגנון בחינת ידע והגנה

1. דודו אומר ש־`X.shape == (4, 1)` פירושו ארבעה מאפיינים ותצפית אחת. האם הוא צודק? נמקו.

2. הסבירו כיצד ניתן לקבל תוכנית שרצה ללא שגיאות אך מפיקה תוצאה בלתי תקפה מבחינה מדעית.
3. מה ההבדל בין בדיקת כלל סף שנכתב ידנית לבין אימון מודל שלומד סף או גבול החלטה מנתונים?
4. כיצד הייתם מתעדים שינוי באופן חישוב הממוצע כדי לאפשר שחזור של ניסוי?

תכנית הוראה מוצעת ל־30 שעות

מפגש	מטרת ביצוע	תוצר אישי
1	מחברת Colab ומשתנים	מחברת נפתחת מתחילתה ומפיקה אותו פלט
2	טיפוסים וחישובים	ממוצע ויחידות מדידה מוסברים
3	מחרוזות וקלט	הצגת דוח קריא והמרת סוגי נתונים
4	תנאים	כלל החלטה ומקרי גבול
5	לולאות	סכימה וספירה עם טבלת מעקב
6	רשימות	גישה, חיתוך, העתקה ושמירת מקור
7	מילונים	רשומות בעלות שדות ברורים
8	פונקציות	פעולות עם return ובדיקות
9	שגיאות וקבצים	נקרא ונבדק CSV
10	NumPy ופרויקט	דוח מסכם ומטריצת מאפיינים

מילון מונחים מסכם

מונח	הסבר לתלמיד
Variable	שם שמתייחס לערך
Data type	הטיפוס שמגדיר אילו פעולות אפשריות על ערך
Condition	ביטוי שמוביל לענף פעולה
Loop	ביצוע חוזר של הוראות
List	אוסף מסודר וניתן לשינוי
Dictionary	אוסף של זוגות מפתח–ערך
Function	פעולה בעלת שם, פרמטרים ותוצאה אפשרית
Return	החזרת ערך אל קוד שקרא לפונקציה
Exception	אירוע שגיאה בזמן ריצה
Module	קובץ/יחידה המכילים קוד שניתן לייבא
NumPy array	מבנה מערך מספרי רב־ממדי
Shape	מספר האיברים לאורך כל ציר
Feature	מאפיין המשמש לתיאור תצפית
Target	משתנה המטרה שמנסים לחזות
Data leakage	שימוש במידע שאינו לגיטימי בזמן האימון או הבחינה

מקורות והקשר לספר הגדול

עוגן תכנית: מאגר Data-Science-ML-Course, מסלול ההתמחות, פרק 1 — היכרות חוזרת עם שפת תכנות (30 שעות). מקורות ההשראה להסברים הם ספרי יסוד מוכרים, אך הניסוח, הדוגמאות, האיורים וקוד התרגול בחוברת מקוריים.

Eric Matthes, Python Crash Course, 3rd ed., No Starch Press (2022) — https://nostarch.com/Python-crash-course-3rd-edition

Al Sweigart, Automate the Boring Stuff with Python, 3rd ed., No Starch Press (2025) — https://nostarch.com/automate-boring-stuff-python-3rd-edition

/Python Software Foundation, The Python Tutorial — https://docs.python.org/3/tutorial

מיפוי ההשלמות במסלול שלנו — https://github.com/samiabo75/Data-Science-ML-Course/tree/main/01-Python-Basics/03-Deep-Learning-Python

לאחר חוברת זו נמשיך ל־פרק 2: תכנות מונחה עצמים (30 שעות), לפרק 3: מבוא למדעי הנתונים וללמידת מכונה (10 שעות) ולפרק 4: חקר נתונים — EDA (15 שעות). לאחר השלמת ארבעתם נחבר אותם לחוברת יסוד אחת ונמקם אותה בתחילת ספר הקורס המאוחד. אין כאן כפילות של לימוד האלגוריתמים והמתמטיקה שכבר פותחו בספר הגדול.

בדיקת עצמאות לפני המשך האם אתם מסוגלים להסביר מה מייצגת כל שורה בטבלת הנתונים, להראות איך חושב כל מספר, לאתר טעות בקוד, ולהריץ את המחברת מחדש ללא התערבות? אם כן — מוכנים לפרק התכנות מונחה העצמים.

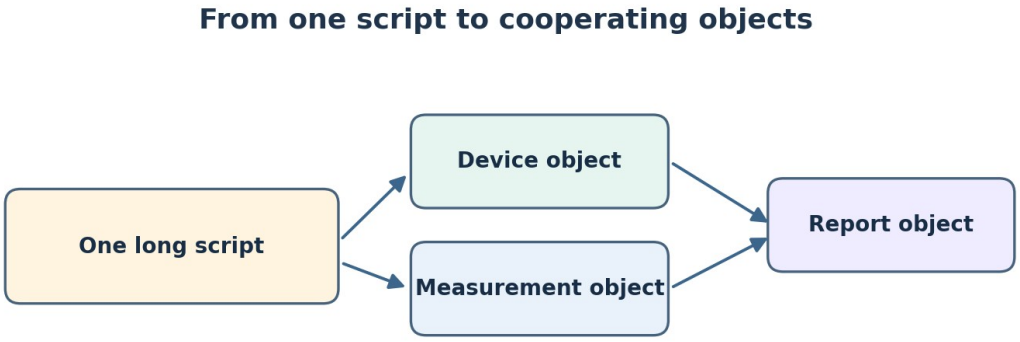
חוברת היסוד לספר הקורס המאוחד

פרק 2 — תכנות מונחה עצמים ב־Python

ממחלקה ועצם ועד למבנה של מערכת AI: הבנה, תכנון ומימוש

מסלול התמחות למידת מכונה ולמידה עמוקה | כיתות י״א–י״ב | מסגרת סילבוס: 30 שעות

מהדורת תלמיד • הסברים מדורגים • קוד עצמאי • המחשות מקוריות • תרגול והכנה לפרויקט



איור פתיחה: מעבר מסקריפט יחיד למספר עצמים בעלי תפקידים ברורים.

לפני שמתחילים — הסיפור שלנו

דודו אחראי על מעבדת המחשבים בבית הספר. עד היום הוא שמר את נתוני כל המחשבים ברשימות נפרדות: רשימת שמות, רשימת טמפרטורות ורשימת תקלות. כאשר נוספו מחשבים הוא התחיל להתבלבל: האם הטמפרטורה במקום השלישי שייכת למחשב השלישי בכל הרשימות? מה יקרה אם ימחק מחשב רק מרשימה אחת? כעת הוא רוצה לבנות תוכנה שאפשר להרחיב, לבדוק ולשלב בעתיד במערכת חיזוי תקלות מבוססת AI.

תכנות מונחה עצמים אינו אלגוריתם של למידת מכונה. זהו כלי לארגון תוכנה. הוא עוזר להפריד בין נתונים ופעולות, ליצור רכיבים שאפשר לבדוק בנפרד ולמנוע תלות מיותרת בין חלקי הפרויקט. בהמשך נשתמש באותן יכולות לכתיבת רכיבי קדם־עיבוד, מודלים ומערכות חיזוי.

מה תלמדו בפרק?

- להגדיר מחלקה, ליצור עצמים ולהבחין בין תכונה של עצם למשתנה רגיל.
- להסביר את התפקיד של self ושל __init__, ולכתוב פעולות שמבצעות חישוב אמיתי.
- להשתמש בהכלה ובהרכבה, ולהבחין ביניהן לבין ירושה ופולימורפיזם.
- להבין הפניה כפולה, העתקת עצמים, גישה מבוקרת לשדות וטיפול בקלט חריג.
- לבנות מערכת קטנה שבה רכיב נתונים ורכיב חיזוי פועלים יחד, ולבדוק את פעולתם.

תכנית הוראה מוצעת ל־30 שעות

תוצר תלמיד	נושאים ומיומנויות	מפגש (3 שעות)
מחלקת Device ראשונה	מהבעיה למחלקה, מצב והתנהגות	1
עצם עם חישוב ומצב	init__, self, get/set__ ופעולות	2
דוח על מספר מחשבים	פעולות פנימיות, רשימות עצמים	3
טבלת מעקב של עצמים	הפניות, העתקה וזהות	4
מערכת עצמים מקושרים	הכלה: Lab מכיל Device	5

תוצר תלמיד	נושאים ומיומנויות	מפגש (3 שעות)
	Measurement־ו	
מחלקה שמונעת מצב שגוי	כימוס, property ואימות נתונים	6
סוגי מחשבים ומימושים	ירושה, super ופולימורפיזם	7
קוד מאורגן וקובצי CSV	מודולים, קבצים וחריגות	8
Pipeline לימודי	רכיבי fit/transform/predict ובדיקות	9
SmartLab OOP	מיני־פרויקט, תיעוד והגנה בעל־פה	10

הקשר לשלוש תכניות הלימוד

דוגמה לשימוש	רמת הידע המכוונת	מסלול
קריאת נתוני מחשבים, סיכום והדפסתם	פונקציות, נתונים ומחברות Python בסיסיות	יחידה 3
עצם נתונים נושא וקטור מאפיינים	חישובים מתמטיים ואלגוריתמים (לא דרישת OOP עצמאית מזוהה)	נושאי ליבה — יחידה 5
Device, DataScaler, Predictor, Report	מחלקות, אחריות רכיבים, קוד מודלים ויישום תוכנה	התמחות למידה עמוקה

עוגן: במפת תכנית ההתמחות שבמאגר הקורס, פרק 2 — תכנות מונחה עצמים, 30 שעות. החלוקה למפגשים, דוגמת המעבדה וההקשרים ל־AI הם התאמה פדגוגית שלנו ולא חלוקה רשמית מאומתת של הסילבוס.

2.1 מדוע צריך תכנות מונחה עצמים?

נתאר שלושה מחשבים באמצעות שלוש רשימות: שמות, טמפרטורות ונתוני תקלות. כדי להפיק דוח על המחשב הראשון ניגש לאינדקס 0 בשלוש הרשימות. כל עוד הרשימות תמיד באותו אורך ובאותו סדר, התוכנית עובדת. אבל בפועל שינוי קטן — הוספה, מחיקה או מיון — עלול לשבור את ההתאמה ביניהן. הבעיה אינה שהרשימות הן כלי גרוע; הבעיה היא שאנחנו מייצגים ישות אחת באמצעות נתונים מפוזרים ללא הגדרה אחת שמאגדת אותם.

```
names = ["PC-01", "PC-02", "PC-03"]
temperatures = [58, 73, 64]
failures = [0, 2, 1]
for i in range(len(names)):
    print(names[i], temperatures[i], failures[i])
```

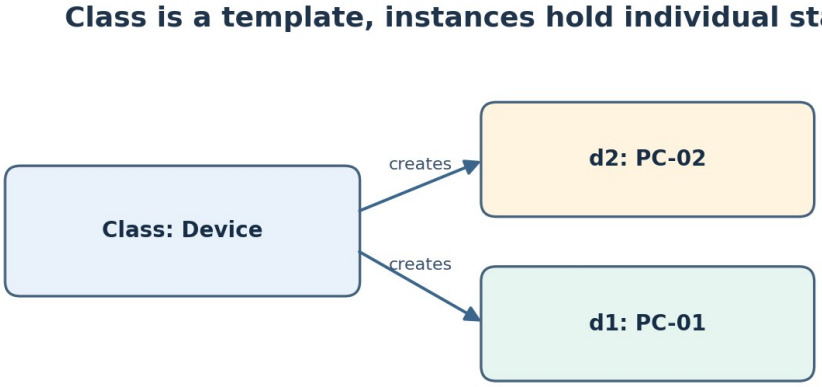
אפשר להחליף את שלוש הרשימות ברשימה של מילונים, וזה כבר שיפור. בתכנות מונחה עצמים עושים צעד נוסף: מגדירים מחלקה שמכילה את נתוני המחשב וגם פעולות שקשורות אליו. לדוגמה, במקום שקוד חיצוני יחליט בכל מקום אם הטמפרטורה חריגה, המחלקה עצמה יכולה להציע פעולה בשם needs_check. כך הכלל יוגדר פעם אחת ויוכל לשמש כמה חלקים בתוכנית.

☐ בדיקת הבנה: איזו בעיה עלולה להתרחש אם נוסיף שם של מחשב חדש אך נשכח להוסיף ערך לרשימת הטמפרטורות?

2.2 מחלקה, עצם, תכונות והתנהגות

מחלקה (Class) היא תיאור כללי של סוג עצמים; עצם (Object, או Instance) הוא מופע מסוים שנוצר על פי התיאור. אפשר לחשוב על מחלקה כתבנית להכנת כרטיסי מחשב: כל כרטיס מכיל אותם סוגי פרטים,

אבל הערכים בכל כרטיס שונים. התכונות (Attributes) מתארות את המצב של העצם; המתודות (Methods) מתארות מה הוא יודע לעשות.



איור 2.1: מחלקה אחת משמשת ליצירת כמה עצמים, ולכל עצם נתונים משלו.

```
class Device:
    def __init__(self, name, temperature):
        self.name = name
        self.temperature = temperature

    def needs_check(self):
        return self.temperature >= 70

computer_a = Device("PC-01", 58)
computer_b = Device("PC-02", 73)
print(computer_a.name, computer_a.needs_check())
print(computer_b.name, computer_b.needs_check())
```

כאשר מופיעה ההוראה Device("PC-01", 58), פייתון יוצרת מופע חדש ומפעילה את __init__ שלו. המופע שמתקבל נשמר במשתנה computer_a. שתי ההפניות computer_a ו־computer_b מצביעות על שני עצמים שונים, ולכן שינוי הטמפרטורה של אחד מהם אינו משנה את האחר.

דוגמה מהמעבדה	משמעות	רכיב בקוד
תבנית של כרטיס מחשב	הגדרה כללית של סוג העצם	class Device
מחשב מסוים	משתנה המחזיק הפניה לעצם	computer_a
שם המחשב	תכונה השייכת למופע	self.name
בדיקת סף טמפרטורה	פעולה של המופע	needs_check()

חשוב — הפעולה needs_check בדוגמה היא כלל תכנותי קבוע, ולא מודל שלמד מנתונים. בפרקים מתקדמים נחליף את הכלל הפשוט במודל שנאמן על נתוני אמת.

2.3 מה עושה __init__ ומה פירוש self?

הביטוי __init__ נקרא לעיתים "בנאי" בשפת ההוראה. ליתר דיוק, זוהי מתודת אתחול: היא מופעלת לאחר יצירת העצם ומגדירה את מצבו הראשוני. הפרמטר self מציין את המופע הנוכחי. כאשר קוראים computer_a.needs_check(), פייתון מעבירה את המופע בתור הפרמטר הראשון של המתודה — גם אם איננו כותבים אותו במפורש בעת הקריאה.

השוו בין name בלי self לבין self.name. השם name הוא פרמטר מקומי שקיים בזמן הפעלת __init__. לעומתו self.name נשמר כתכונה של המופע, ולכן אפשר לגשת אליו גם בהמשך, לאחר שהפעולה הסתיימה. ההבדל הוא כמו ההבדל בין פתק זמני לבין נתון ששמור בכרטיס של אותו מחשב.

```
class Device:
    def __init__(self, name, temperature=25):
        self.name = name
        self.temperature = temperature

    def description(self):
        return f"{self.name}: {self.temperature} C"

first = Device("PC-01")
second = Device("PC-02", 65)
print(first.description())
print(second.description())
```

בדוגמה הוגדר ערך ברירת מחדל temperature=25. לכן אפשר ליצור מחשב בלי למסור טמפרטורה, אך בכל זאת חייבים למסור שם. ערך ברירת מחדל צריך להתאים למשמעות הנתון; במערכת אמיתית עדיף לעיתים לציין שהמדידה אינה ידועה מאשר להמציא מדידה של 25 מעלות.

טבלת מעקב — יצירת שני עצמים

שלב	הוראה	התוצאה במצב העצמים
1	first = Device("PC-01")	first.name=PC-01, first.temperature=25
2	second = Device("PC-02",65)	second.name=PC-02, second.temperature=65
3	second.temperature = 72	רק second.temperature משתנה ל-72
4	first.description()	מחזירה את תיאור העצם הראשון

□ שאלת העברה: מה יקרה אם בתוך description נכתוב temperature במקום self.temperature? הסבירו לפי תחום ההכרה של המשתנים.

2.4 פעולות פנימיות: חישוב שמשתמש במצב העצם

מתודה אינה חייבת רק להדפיס נתון. היא יכולה לחשב תוצאה על סמך המצב של העצם או לעדכן אותו. נרצה למשל להוסיף קריאה חדשה של הטמפרטורה ולחשב את ההפרש מן המדידה הקודמת. כדי לעשות זאת נגדיר פעולה עם פרמטר חדש ונבחין בין ערך שהפעולה מחזירה לבין שינוי שהיא מבצעת במופע.

```
class Device:
    def __init__(self, name, temperature):
        self.name = name
        self.temperature = temperature

    def update_temperature(self, new_value):
        change = new_value - self.temperature
        self.temperature = new_value
        return change

pc = Device("PC-01", 60)
print(pc.update_temperature(68))
print(pc.temperature)
print(pc.update_temperature(65))
```

בקריאה הראשונה change הוא 8, והטמפרטורה נשמרת כ־68. בקריאה השנייה change הוא 3 −, כי הערך החדש קטן מן הקודם. חשוב לשים לב לסדר הפעולות: מחשבים את ההפרש לפני עדכון התכונה. אם נעדכן תחילה, החישוב ייתן תמיד אפס.

כיצד מתכננים מתודה?

לפני כתיבת כל מתודה מומלץ לענות על ארבע שאלות: מה היא מקבלת? מה היא משנה? מה היא מחזירה? ומה יקרה בקלט לא תקין? תכנון מראש מונע מתודות שמבצעות יותר מדי תפקידים ומקשה פחות על בדיקות היחידה.

שאלה	עבור **update_temperature**
קלט	טמפרטורה חדשה — new_value
מצב לפני	self.temperature
שינוי	עדכון self.temperature
פלט	ההפרש מן המדידה הקודמת

2.5 מספר עצמים — רשימת מחשבים ולולאות

מעבדה כוללת יותר ממחשב אחד. לכן ניצור רשימה של עצמי Device, נעבור עליהם בלולאה ונבנה דוח. המושג המרכזי הוא שהרשימה מחזיקה הפניות לעצמים; כל עצם אחראי לחשב האם הוא דורש בדיקה, ואילו הדוח אחראי לאסוף את התוצאות של כולם.

```
class Device:
    def __init__(self, name, temperature):
        self.name = name
        self.temperature = temperature

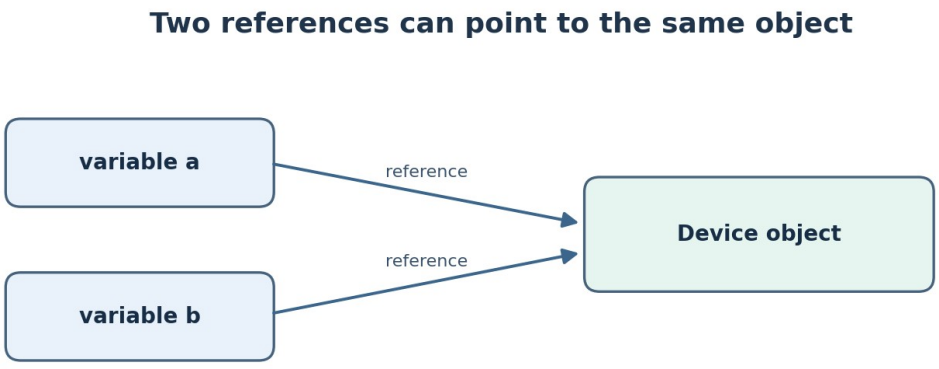
    def needs_check(self):
        return self.temperature >= 70

computers = [Device("PC-01", 61), Device("PC-02", 77),
             Device("PC-03", 72)]
for computer in computers:
    if computer.needs_check():
        print("CHECK:", computer.name)
```

בדוגמה הפעולה needs_check שייכת למחלקה Device, והלולאה רק מפעילה אותה. בעתיד נוכל להחליף את הכלל הפנימי בלי לשנות את לולאת הדוח. זו דוגמה ליתרון חשוב של OOP: מקומיות של שינוי.

2.6 זהות, הפניה כפולה והעתקה של עצמים

כאשר מבצעים b = a לא נוצרת אוטומטית העתקה של העצם. מתקבלת הפניה נוספת לאותו עצם, ולכן שינוי באמצעות b ייראה גם דרך a. זה שונה מעבודה עם מספרים פשוטים, שבהם המשתנה מקבל ערך, ולא עצם שניתן לשנות את שדותיו. כדי לבנות מערכות אמינות חשוב להבחין בין אותו עצם לבין עצם אחר בעל נתונים זהים.



איור 2.2: שני משתנים שונים מצביעים לאותו עצם בזיכרון.

```
class Device:
    def __init__(self, name):
        self.name = name

a = Device("PC-01")
b = a
b.name = "PC-99"
print(a.name)
print(a is b)

c = Device(a.name)
print(a is c)
print(a.name == c.name)
```

התוצאה a.name היא PC-99, מפני ששינוי דרך b שינה את אותו המופע. הביטוי a is b מחזיר True, כי זו אותה זהות. לעומת זאת c הוא מופע חדש, ולכן a is c מחזיר False גם אם לשניהם אותו שם. השוואת התכונות שלהם, לעומת זאת, יכולה להחזיר True. זהות אינה זהה לשוויון ערכים.

העתקה שטחית לעומת העתקה עמוקה

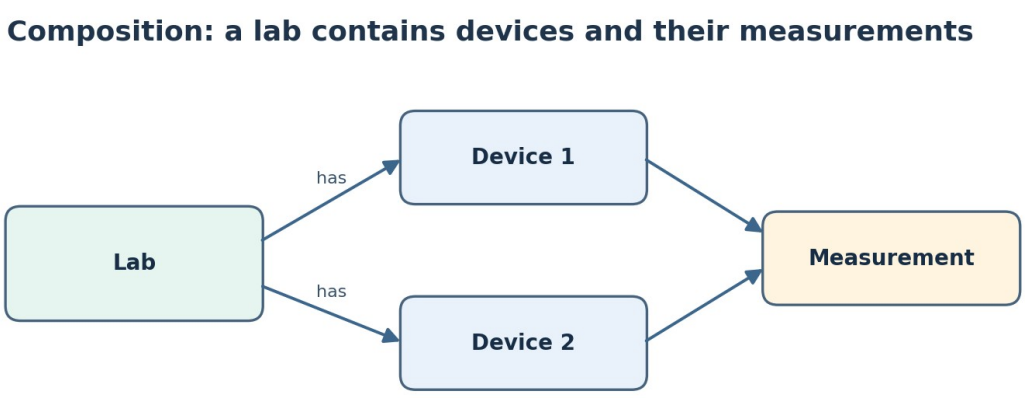
כאשר לעצם יש שדות פשוטים כמו מספר ומחרוזת, אפשר לעיתים ליצור עצם חדש באמצעות בנאי שמקבל את אותן תכונות. כאשר עצם מכיל רשימות או עצמים נוספים, העתקה מורכבת יותר: copy.copy מעתיק את האובייקט החיצוני, אבל בדרך כלל משאיר הפניות משותפות לעצמים פנימיים; copy.deepcopy מנסה להעתיק גם את הרכיבים המקוננים. אין צורך לשנן ספרייה זו, אך חשוב לדעת לשאול אם שני עצמים חולקים נתונים משותפים.

```
import copy
records = {"device": "PC-01", "temps": [60, 65]}
shallow = copy.copy(records)
deep = copy.deepcopy(records)
shallow["temps"].append(72)
print(records["temps"])
print(deep["temps"])
```

חשוב — עצם שנראה כעותק במסך אינו בהכרח בלתי תלוי. בפרויקט AI בעיות כאלה מופיעות בעת שינוי רשימות דוגמאות, הגדרות ניסוי או אובייקטים המשותפים למספר רכיבים.

2.7 הכלה: עצם שמכיל עצמים אחרים

בעולם האמיתי מחשב נמדד כמה פעמים, ומעבדה כוללת כמה מחשבים. לכן נכון ליצור מחלקת Measurement שמתארת מדידה, מחלקת Device שמחזיקה מדידות, ומחלקת Lab שמחזיקה מחשבים. זהו קשר של ״יש לו״ (has-a). הכלה מאפשרת לפרק מערכת ליחידות בעלות אחריות מוגדרת: מדידה שומרת תוצאה, מחשב אוסף מדידות, ומעבדה מפיקה דוח כללי.



איור 2.3: קשרי הכלה במערכת לניטור מעבדת מחשבים.

```
class Measurement:
    def __init__(self, value, minute):
        self.value = value
        self.minute = minute

class Device:
    def __init__(self, name):
        self.name = name
        self.measurements = []

    def add_measurement(self, value, minute):
        self.measurements.append(Measurement(value, minute))

    def average_temperature(self):
        if len(self.measurements) == 0:
            return None
        total = sum(m.value for m in self.measurements)
        return total / len(self.measurements)

a = Device("PC-01")
a.add_measurement(60, 0)
a.add_measurement(70, 10)
print(a.average_temperature())
```

נשים לב מדוע הרשימה נוצרת בתוך __init__ באמצעות self.measurements = []. כך כל מופע מקבל רשימה חדשה משלו. אם נגדיר רשימה משתנה ברמת המחלקה, או נשתמש ברשימה משתנה כערך ברירת מחדל, עלול להיווצר שיתוף לא רצוי בין עצמים. זו טעות נפוצה שכדאי לזהות כבר בשלב הלמידה.

הכלה לעומת ירושה

ביטוי עזר: Lab has Devices — המעבדה מכילה מחשבים, ולכן זה קשר הכלה. לעומת זאת Laptop is a Device — מחשב נייד הוא סוג של מחשב, ולכן ייתכן קשר ירושה. לפני כל קשר בין מחלקות שאלו: האם זהו רכיב שמצוי בתוך אחר, או סוג מיוחד של אותו דבר?

☐ שאלת חשיבה: האם רשת עצבית היא סוג של Dataset, או שהיא משתמשת ב־Dataset? נמקו לפי קשר is-a לעומת has-a.

2.8 כימוס, אימות נתונים ו־property

אם כל חלק בתוכנית יוכל להציב ערך שרירותי לטמפרטורה, אפשר לקבל נתונים לא הגיוניים. לכן נרצה שמחלקת Device תגדיר מקום אחד שבו בודקים קלט חדש. בפייתון נהוג לסמן שדות פנימיים בקו תחתון בתחילתם, למשל _temperature. זהו מוסכם בין מתכנתים ולא מנגנון אבטחה מוחלט; שמות עם קו תחתון הם עדיין נגישים מבחינה טכנית. המטרה היא שהקוד ישתמש בממשק ברור במקום לשנות את ייצוג הנתונים ישירות.

```
class Device:
    def __init__(self, name, temperature):
        self.name = name
        self.temperature = temperature

    @property
    def temperature(self):
        return self._temperature

    @temperature.setter
    def temperature(self, value):
        if not -30 <= value <= 120:
            raise ValueError("Temperature outside allowed range")
        self._temperature = value

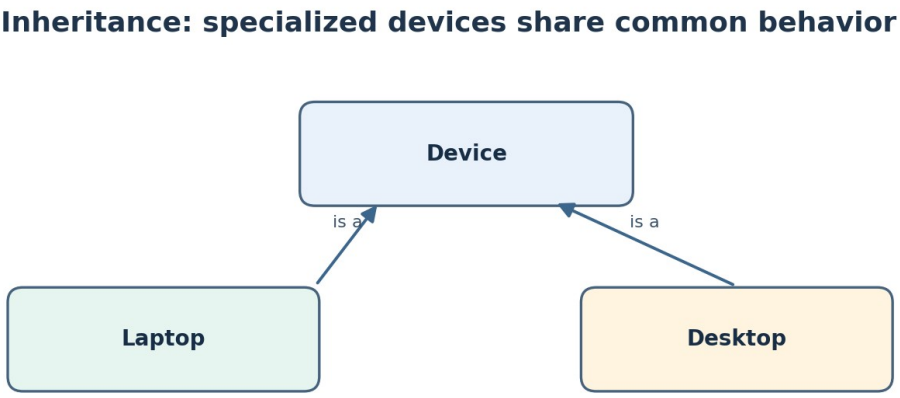
pc = Device("PC-01", 65)
pc.temperature = 71
print(pc.temperature)
```

הקוד בודק שהטמפרטורה בטווח שנבחר לדוגמה. מדובר בכלל אימות לצורכי הוראה, לא בטווח תקין רפואי או תקן של יצרן מחשבים. אם ננסה להציב 999 תיזרק חריגה מסוג ValueError. עדיף שהנתונים יידחו בשלב הקליטה מאשר שהתוכנית תמשיך לבצע חיזויים על ערכים שאין להם משמעות.

דגש — @property מאפשרת לקרוא תכונה כאילו הייתה שדה רגיל, אבל לבצע מאחורי הקלעים פעולה. זו אפשרות שימושית, לא חובה לכל שדה; מחלקה פשוטה אינה צריכה להפוך כל נתון ל־property ללא סיבה.

2.9 ירושה, super ופולימורפיזם

ירושה מאפשרת ליצור מחלקה מיוחדת שמבוססת על מחלקה כללית. אם לכל המחשבים יש שם וטמפרטורה, אבל למחשב נייד יש גם אחוז סוללה, אפשר לתאר Laptop כ־Device עם מידע נוסף. המחלקה היורשת מקבלת את המתודות והמאפיינים של מחלקת הבסיס, ויכולה להוסיף או לשנות התנהגות. חשוב לא להשתמש בירושה רק כדי לקצר כמה שורות קוד: קשר הירושה צריך להיות הגיוני מבחינת משמעות העצמים.



איור 2.4: *Laptop* ו־*Desktop* הם שני סוגים של *Device*.

```
class Device:
    def __init__(self, name):
        self.name = name

    def status(self):
        return "Active"

class Laptop(Device):
    def __init__(self, name, battery):
        super().__init__(name)
        self.battery = battery

    def status(self):
        if self.battery < 20:
            return "Charge needed"
        return super().status()

computers = [Device("PC-01"), Laptop("LAP-02", 12)]
for computer in computers:
    print(computer.name, computer.status())
```

הקריאה super().__init__(name) מפעילה את מתודת האתחול של מחלקת הבסיס. כך נשמרת הגדרת השם במקום אחד. ב־Laptop דרסנו (override) את המתודה status: בקריאה על מחשב נייד בעל סוללה חלשה יוחזר מסר אחר; בקריאה על Device רגיל יוחזר Active. זו דוגמה לפולימורפיזם: אותו שם מתודה מוביל להתנהגות מתאימה למופע המסוים.

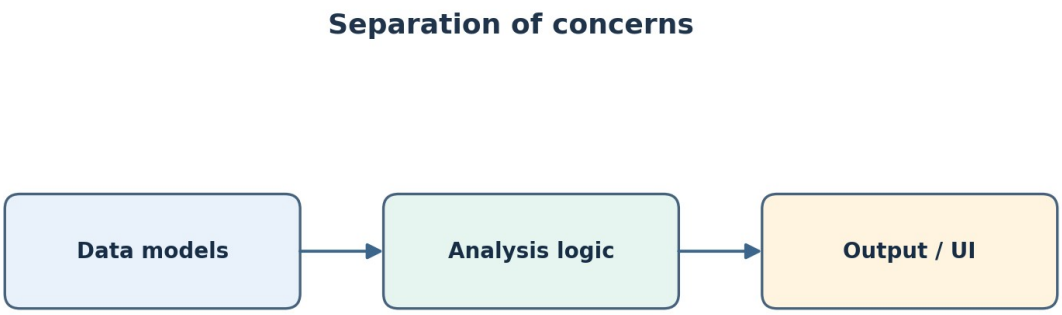
פולימורפיזם ללא קשר ירושה ישיר

פייתון מאפשרת גם כתיבת קוד שעובד עם כל עצם שמציע מתודה מתאימה, ללא חובה ששניהם יירשו ממחלקת בסיס. דרך חשיבה זו מכונה לעיתים duck typing. בספר שלנו נשתמש תחילה בירושה כי היא קלה להמחשה, ובהמשך נכיר ממשק של רכיבי AI שממממשים fit או predict בדרכים שונות.

☐ שאלת הבנה: מדוע שינוי של המתודה status במחלקת Laptop אינו מחייב שינוי בלולאה שעוברת על computers?

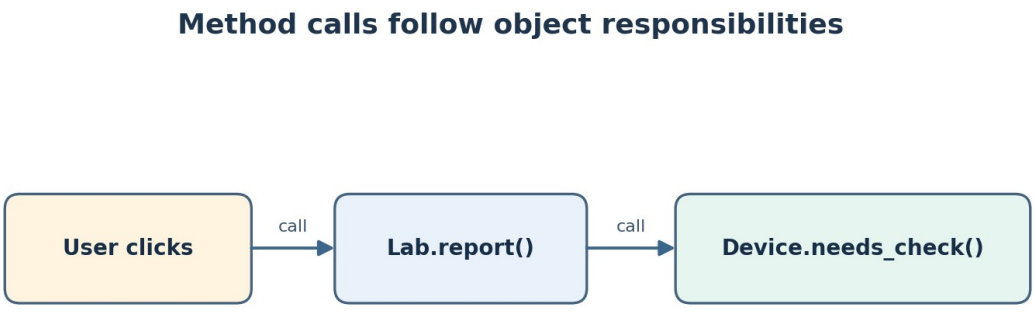
2.10 תרשים מחלקות ותכנון אחריות

לפני שמתחילים לכתוב פרויקט גדול, כדאי לשרטט אילו מחלקות דרושות, מי מחזיקה נתונים, מי מבצעת חישוב ומי מציגה תוצאה. כלל פשוט הוא "אחריות אחת ברורה לכל מחלקה". אין הכוונה שלמחלקה מותרת רק מתודה אחת; הכוונה שאפשר לתאר את מטרתה במשפט ברור. אם מחלקת Device גם קוראת CSV, גם מאמנת רשת וגם מציירת גרף, היא מבצעת תפקידים שונים מאוד וקשה לבדוק אותה.



איור 2.5: מומלץ להפריד בין מודלים של נתונים, לוגיקת ניתוח וממשק משתמש.

מחלקה	אחריות	פעולה אפשרית
Device	שמירת נתוני מחשב	add_measurement
Lab	ניהול אוסף מחשבים	add_device
DataScaler	לימוד טווחי מאפיינים והמרתם	fit / transform
Predictor	הפקת תחזית מדוגמה	predict
Report	הצגת תוצאות ומסקנות	build_summary



איור 2.6: כל מתודה מסייעת לאחרת בלי לבצע את כל העבודה בעצמה.

2.11 מודולים, קבצים ו־import

במחברת אחת אפשר להחזיק את כל הקוד, אבל בפרויקט אמיתי זו בדרך כלל בחירה לא נוחה. מודול בפייתון הוא קובץ py. המכיל הגדרות ומשפטי קוד. חלוקת התוכנית לקבצים מאפשרת לתקן רכיב בלי לחפש אותו בתוך מאות שורות ולבצע שימוש חוזר במחלקות שונות. בסביבת Colab אפשר להתחיל בתאים נפרדים, ובהמשך לעבור לקובצי .py.

שם קובץ	מה יכיל
device.py	המחלקות Device ו־Measurement
lab.py	המחלקה Lab והפעולות לניהול מחשבים
analysis.py	פעולות חישוב וסיכום נתונים
main.py	יצירת עצמים, הפעלה והצגת תוצאות

```
# file: device.py
class Device:
    def __init__(self, name):
        self.name = name

# file: main.py (save as a separate file)
# from device import Device
# computer = Device("PC-01")
# print(computer.name)
```

הדוגמה מציגה שני קבצים לצורכי תכנון, ולכן קטע main.py מובא כהערות כדי שניתן יהיה להריץ את התא בפני עצמו במחברת. כאשר עובדים בקבצים אמיתיים יש להסיר את סימני ההערה ולהריץ את main.py בסביבה שמכילה גם device.py. חשוב לא ליצור קובץ אישי בשם זהה לספרייה סטנדרטית כמו csv.py או math.py, כדי להימנע מהתנגשות בייבוא.

2.12 טיפול בטעויות — לא מתעלמים מקלט בעייתי

קוד אמין מבדיל בין מצב שאפשר לטפל בו לבין שגיאת תכנות שיש לתקן. לדוגמה, משתמש עשוי להקליד מחרוזת במקום טמפרטורה; אפשר לבקש ממנו להזין שוב. לעומת זאת, אם קראנו למתודה שאינה קיימת, אין טעם להסתיר את התקלה ולהמשיך כאילו דבר לא קרה. המנגנון try/except נועד לטפל במצבים צפויים ומוגדרים, ולא לכסות כל חריגה באופן גורף.

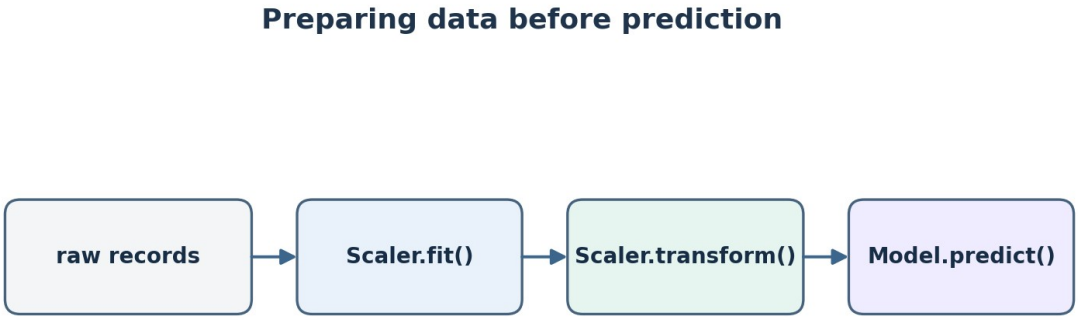
```
raw = "71"
try:
    temperature = float(raw)
    if not -30 <= temperature <= 120:
        print("Value outside the chosen range")
    else:
        print("Accepted:", temperature)
except ValueError:
    print("Enter a numeric temperature")
```

במערכת למידת מכונה יש לבצע בדיקות גם על אורך וקטור המאפיינים, על סדר העמודות ועל ערכים חסרים. מודל שמקבל ארבע תכונות בסדר מסוים אינו יכול לקבל בשקט שלוש תכונות או סדר אחר. עקרון זה נבנה בפרקים הבאים על הכנת נתונים ובדיקת מודלים.

2.13 OOP במערכת AI: הפרדה בין fit, transform ו־predict

נרצה כעת לראות את הקשר הישיר בין תכנות מונחה עצמים למערכות AI. בשפת העבודה של scikit-learn מקובל שרכיבי עיבוד נתונים מציעים פעולת fit הלומדת נתונים הדרושים להמרה, ופעולת transform הממירה דוגמאות באמצעות הערכים שנלמדו. מסווג מציע fit לאימון ו־predict לחיזוי. כאן נכתוב מחלקת MinMaxScaler לימודית קטנה כדי להבין את הרעיון, אך בפרויקט אמיתי נעדיף רכיבים מקצועיים שנבדקו היטב.

- מהדורת תלמיד • Python ב־OOP



איור 2.7: הרכיבים השונים משתתפים בשלבי הכנת הנתונים והחיזוי.

```
class SimpleMinMax:
    def __init__(self):
        self.minimum = None
        self.maximum = None

    def fit(self, values):
        if len(values) == 0:
            raise ValueError("Empty training data")
        self.minimum = min(values)
        self.maximum = max(values)
        return self

    def transform(self, values):
        if self.minimum is None:
            raise ValueError("Call fit before transform")
        span = self.maximum - self.minimum
        if span == 0:
            return [0.0 for _ in values]
        return [(v-self.minimum)/span for v in values]

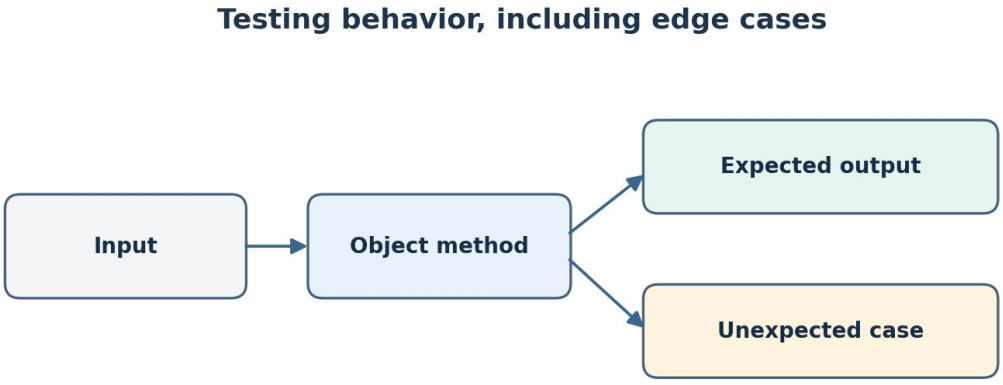
scaler = SimpleMinMax()
scaler.fit([40, 60, 80])
print(scaler.transform([50, 90]))
```

הפעולה fit לומדת מן האימון מינימום 40 ומקסימום 80. לאחר מכן transform ממירה 50 לערך 0.25 ואת 90 לערך 1.25. האחרון גדול מ־1 — וזה תקין, מפני שדוגמה חדשה יכולה להיות מחוץ לטווח האימון. אסור להתאים את המינימום והמקסימום מחדש על נתוני Test: פעולה כזו תחשוף למערכת מידע שלא אמור היה להיות זמין בזמן האימון.

הבחנה מרכזית — מחלקת SimpleMinMax היא הדגמה לימודית של עקרון התאמת טרנספורמציה. היא אינה מחליפה את בדיקות הקלט, הטיפול בחסרים וההתנהגות המלאה של MinMaxScaler המקצועית.

2.14 בדיקות יחידה ואיכות קוד

מחלקה טובה אינה רק מחלקה שנראית מסודרת. צריך להראות שהמתודות שלה פועלות נכון על קלט רגיל וגם במקרי קצה. בדיקת יחידה מפעילה רכיב קטן ומוודאת שהתוצאה מתאימה למה שציפינו. כאשר מתכננים בדיקות מראש, ההסבר של המתודות נעשה מדויק יותר: התלמיד חייב לחשוב מה פירוש ״נכון״ לפני שהוא רואה את הפלט.



איור 2.8: בדיקה משווה תוצאה בפועל לתוצאה צפויה, כולל מצבים חריגים.

```
scaler = SimpleMinMax()
scaler.fit([10, 20, 30])
result = scaler.transform([10, 15, 30])
assert result == [0.0, 0.25, 1.0]
assert scaler.transform([40]) == [1.5]
print("Tests passed")
```

הפקודה assert בודקת תנאי; אם הוא אינו מתקיים, התוכנית תעלה AssertionError. בדיקות מסוג זה שימושיות ללמידה ולבדיקות קטנות. במערכת גדולה אפשר להשתמש גם במסגרת בדיקות כמו pytest. חובה לא להסתפק במבחן אחד: יש לבדוק רשימת אימון ריקה, מאפיין קבוע, ערכי קצה, וכמות מאפיינים לא צפויה.

בדיקה חזותית של שינוי מצב

בנוסף לבדיקות חישוביות, חשוב לוודא שהממשק הציבורי של העצם לא מאפשר מצב סותר. לדוגמה, Device לא אמור להחזיק ממוצע טמפרטורה שנשמר במשתנה אחד בזמן שרשימת המדידות מתעדכנת במקום אחר. לעיתים עדיף לחשב את הממוצע מתוך הרשימה בכל קריאה, ולא לשמור עותק נוסף שדורש סנכרון. זו דוגמה לחשיבה הנדסית המתאימה במיוחד לתחום הנתונים.

2.15 מיני־פרויקט מסכם: SmartLab OOP

במיני־פרויקט דודו מבקש לבנות מערכת לניהול מדידות ממחשבי המעבדה. המערכת אינה צריכה לאמן מודל עמוק — זה יגיע בהמשך — אבל המבנה שלה צריך לאפשר להחליף בעתיד את כלל הסף הפשוט במסווג למידת מכונה. לכן נפריד בין אחסון המדידות, ניהול המעבדה, והפקת החלטת החיזוי.

רכיב	דרישה בסיסית	הרחבה אפשרית
Measurement	שמור ערך וזמן מדידה	תקינות טווח
Device	שמור שם ורשימת מדידות	חישוב מינימום, מקסימום וממוצע
Lab	הוסף מחשבים; הצג דוח	חיפוש לפי מזהה
ThresholdPredictor	החזר 0/1 לפי סף מוגדר	סף נפרד לסוג מחשב
Report	הצג דוח קריא	ייצוא CSV

```
class ThresholdPredictor:
    def __init__(self, threshold):
        self.threshold = threshold

    def predict(self, value):
        return int(value >= self.threshold)

predictor = ThresholdPredictor(70)
print(predictor.predict(65))
print(predictor.predict(74))
```

החלפנו כלל if שנכתב ישירות בתוך הדוח בעצם Predictor בעל מתודה predict. זו עדיין אינה למידת מכונה: הסף נקבע בידי המתכנת ואינו נלמד מנתונים. אבל המבנה כבר דומה לממשק של מודל אמיתי, כך שבהמשך נוכל להשתמש במחלקה שמאמנת פרמטרים ובוחנת נתונים חדשים.

מבדק קבלה למיני־פרויקט

- ☐ המערכת מציגה דוח עבור שלושה מחשבים, שלכל אחד מספר שונה של מדידות.
- ☐ המערכת אינה קורסת כאשר אין מדידות; היא מחזירה ערך מוסכם ומוסבר במקום ממוצע מדומה.
- ☐ מחלקת הניבוי מקבלת קלט אחד ומחזירה החלטה עקבית, בלי להדפיס מידע שלא התבקשה להדפיס.
- ☐ התלמיד יכול להסביר באילו מקומות יש הכלה ובאילו מקומות, אם בכלל, יש צורך בירושה.
- ☐ כל רכיב נבדק לפחות פעם אחת באמצעות קלט רגיל ופעם אחת באמצעות מקרה קצה.

העמקה 2.15.1 — משתנה של מחלקה לעומת משתנה של מופע

עד כה לכל מחשב הייתה טמפרטורה משלו: self.temperature. אבל לפעמים צריך נתון משותף לכל המחלקה, למשל מספר המופעים שנוצרו. במקרה כזה אפשר להגדיר משתנה מחלקה בראש המחלקה, מחוץ לכל המתודות. ההבחנה חשובה מאוד: משתנה מופע מתאר עצם אחד; משתנה מחלקה שייך למחלקה עצמה, אף שניתן לראות אותו גם דרך מופעים מסוימים.

```
class Device:
    created_count = 0

    def __init__(self, name):
        self.name = name
        Device.created_count += 1

a = Device("PC-01")
b = Device("PC-02")
print(Device.created_count)
print(a.name, b.name)
```

הספירה היא 2 מפני שמתודת האתחול הופעלה פעמיים. לעומת זאת a.name ו־b.name שונים, משום שכל שם נשמר במופע המתאים. אם נגדיר measurements = [] כמשתנה מחלקה, כל המופעים עלולים לחלוק אותה רשימה. מבחינה לשונית שתי הגישות נראות דומות; מבחינת התנהגות הן שונות לגמרי.

מתי להשתמש?	משמעות	מקום ההגדרה
מדידה של מחשב מסוים	מצב נפרד לכל מופע	self.temperature בתוך __init__
ספירת מופעים לימודית	נתון משותף למחלקה	created_count בגוף המחלקה
היסטוריית מדידות נפרדת	רשימה חדשה לכל מופע	self.measurements = [] בתוך __init__
בדרך כלל לא מתאים להיסטוריה אישית	רשימה משותפת בין מופעים שלא הסתירו אותה	measurements = [] בגוף המחלקה

כדי לבחון את התוצאה, נסו לפני ההרצה לנבא: אם למחשב הראשון נוספה מדידה, האם היא אמורה להופיע במחשב השני? אם התשובה בפועל חיובית בלי שהתכוונו לכך, ככל הנראה נשמרה רשימה

משותפת במקום רשימת מופע. זהו באג חשוב במיוחד כאשר עובדים עם מערכי נתונים של כמה משתמשים או ניסויים.

העמקה 2.15.2 — מעקב ידני אחרי רצף הפעלת מתודות

היכולת החשובה ביותר במבחן על עצמים אינה לזכור מילים באנגלית, אלא לדעת לעקוב אחר מצב העצם. נציג מעקב מלא ללא מחשב: בתחילה דודו יוצר מחשב ללא מדידות, לאחר מכן מוסיף שתי מדידות, שומר הפניה נוספת ולבסוף מוסיף מדידה דרך ההפניה השנייה. נעקוב גם אחר הערך המוחזר וגם אחר השינוי בזיכרון.

```
class Device:
    def __init__(self, name):
        self.name = name
        self.values = []

    def add(self, value):
        self.values.append(value)

    def average(self):
        if not self.values:
            return None
        return sum(self.values) / len(self.values)

first = Device("PC-01")
first.add(60)
first.add(80)
second = first
second.add(70)
print(first.values)
print(first.average())
```

חשוב במיוחד: ההוראה second = first אינה יוצרת עותק חדש. לכן הוספת 70 דרך second משנה את הרשימה שמוחזקת באותו עצם שאליו מתייחס גם first. תוצאת הממוצע תהיה 70, אבל התלמיד צריך להראות כיצד הגיע אליה ולא להסתפק בכך שהמחשב הדפיס אותה.

שלב	אילו נתונים נשמרים בעצם?	מה מחזירה **average**?
מיד אחרי first = Device(…)	[]	None
לאחר first.add(60)	[60]	60
לאחר first.add(80)	[60,80]	70
לאחר second = first	[60,80]	70
לאחר second.add(70)	[60,80,70]	70

שימו לב גם למקרה שבו אין מדידות. בתוכנית שלנו מחזירים None ולא אפס. החזרת אפס תיצור רושם שממוצע הטמפרטורות הוא 0 מעלות, בעוד שבפועל אין נתונים כלל. בחירת ערך החזרה מתאים היא חלק מתכנון ממשק המחלקה, ולא רק שאלה של תחביר.

העמקה 2.15.3 — תכנון לפני קוד: איך מחלקים מערכת לאחריות?

תלמידים רבים מתחילים לכתוב מחלקות לפי שמות העצמים שהם רואים בסיפור, אבל שוכחים לתכנן אילו פעולות כל מחלקה אמורה לבצע. נתרגל תהליך של ארבעה צעדים: ראשית מזהים את הישויות המרכזיות; שנית מנסחים את אחריותה של כל ישות; שלישית בוחרים איזה מידע צריך להיות שמור בה; ולבסוף מגדירים מתודות שיאפשרו לרכיבים אחרים להשתמש בה בלי לדעת את כל פרטי המימוש.

צעד	שאלת תכנון	החלטה בדוגמה
1 — זיהוי	מהן הישויות שקיימות בסיפור?	מחשב, מדידה, מעבדה, מסווג
2 — אחריות	מה כל אחד צריך לדעת לעשות?	Device שומר מדידות; Lab מנהלת מחשבים

צעד	שאלת תכנון	החלטה בדוגמה
3 — מידע	מהו מצב העצם ומה מחושב מחדש?	רשימת מדידות נשמרת; ממוצע מחושב
4 — ממשק	אילו פעולות חלקים אחרים יפעילו?	add, average, predict, summary

נבחן שתי הצעות לתכנון. בהצעה הראשונה Device מדפיס את כל הדוח ומאמן גם מודל. בהצעה השנייה Device שומר מדידות, Report מטפל בתצוגה ו־Predictor מחזיר תחזית. ההצעה השנייה נוחה יותר להרחבה, אך גם היא אינה ערובה לאיכות — חלוקה למחלקות קטנות מדי עלולה לפזר את הלוגיקה ולהקשות על ההבנה. המטרה היא חלוקה הגיונית לפי אחריות, לא מספר גדול ככל האפשר של מחלקות.

בפרויקט למידת מכונה נוכל בעתיד להחליף את Predictor במודל CNN או ברגרסיה מבלי לשכתב את המחלקה ששומרת את נתוני המחשב. כאשר ממשק predict ברור ונשאר יציב, שינוי המימוש הפנימי מצריך פחות שינויים במקומות אחרים. זהו שימוש מעשי חשוב בעקרון ההפרדה בין הממשק לבין אופן ביצוע העבודה.

העמקה 2.15.4 — כתיבת קוד קריא עם טיפוסים ותיעוד

ב־Python אפשר להוסיף רמזי טיפוסים (Type Hints) כדי לתאר מה מתודה מקבלת ומה היא מחזירה. לדוגמה, אפשר לציין שהמדידה היא מספר עשרוני והפעולה מחזירה אמת או שקר. רמזי הטיפוסים אינם מבצעים אכיפה מלאה בזמן ריצה; יש להמשיך לבדוק קלט בפועל כאשר זה חשוב. יתרונם העיקרי בספר שלנו הוא שהם הופכים את הממשק לקל יותר להבנה ומאפשרים לכלי פיתוח להתריע על חלק מהטעויות.

```
class ThresholdPredictor:
    def __init__(self, threshold: float):
        self.threshold = threshold

    def predict(self, value: float) -> bool:
        """Return True when the value reaches the threshold."""
        return value >= self.threshold

predictor = ThresholdPredictor(70.0)
print(predictor.predict(75.0))
```

השורה בתוך שלוש המרכאות היא Docstring — תיאור קצר של הפעולה. ההסבר אינו מתאר כיצד מחשבים באמצעות כל שורת קוד, אלא מה הפעולה מבטיחה למשתמש בה. כאשר תלמיד יפתח בעתיד מודל באמצעות PyTorch, תיעוד כזה יסייע לו להסביר את תפקיד רכיבי המערכת בפני בוחן ויקל על בדיקת הקוד.

העמקה 2.15.5 — תכנית עבודה למיני־פרויקט בארבע אבני דרך

כדי לאפשר למידה עצמאית ולא רק העתקת דוגמה, מומלץ לפתח את SmartLab בהדרגה. בכל אבן דרך נדרשת הוכחה שהרכיב הקודם עובד לפני שמוסיפים עוד חלק. כך קל יותר לגלות היכן נוצרה טעות, והתלמיד נדרש להסביר בכל שלב מה חדש ומה עדיין חסר.

אבן דרך	מה בונים?	בדיקת הצלחה
א׳ — נתונים	Measurement ו־Device עם מדידות	לכל עצם נשמרת היסטוריה עצמאית
ב׳ — ניהול	Lab שמכילה כמה מחשבים	אפשר להוסיף, לאתר ולספור מחשבים
ג׳ — חיזוי	ThresholdPredictor בעל predict	סיווג עקבי לפי סף מוגדר
ד׳ — הנדסה	Report, בדיקות ותיעוד	המערכת פועלת מחדש

בדיקת הצלחה	מה בונים?	אבן דרך
ומסבירה תוצאות		

הערכת הביצוע אינה נשענת רק על מספר הפקודות שהתלמיד כתב. נבדוק אם יש חלוקת אחריות נכונה, אם האובייקטים אינם משתפים מידע בטעות, אם קיימות בדיקות למצבי קצה ואם ניתן להסביר כיצד יעבור הקוד לשימוש במודל נלמד. חשוב להדגיש: הקוד המצורף לספר מדגים עקרונות; הגשת המיני־פרויקט דורשת מהתלמיד לכתוב פתרון משלו ולתעד החלטות.

2.16 טעויות נפוצות — כיצד מזהים ומתקנים?

דרך חשיבה לתיקון	למה זו בעיה?	הטעות
שאלו היכן המידע צריך להישמר	הפרמטר המקומי אינו בהכרח קיים במתודה אחרת	כתיבת name במקום self.name
צור רשימה חדשה בתוך __init__	מדידות של מחשב אחד מופיעות אצל אחר	שימוש ברשימה משותפת לכל העצמים
צרו מופע חדש או העתיקו במודע	שתי הפניות לאותו עצם	כדי להעתיק b = a
בדקו משמעות: הוא סוג של או מכיל?	המחלקות אינן מייצגות יחס is-a	ירושה במקום הכלה
הפרידו חישוב מהצגה	קשה לבדוק ולעשות שימוש חוזר	כל המחלקה מדפיסה וגם מחשבת
התאימו עיבוד על אימון בלבד	דליפת מידע בהכנת נתונים	fit על כל הנתונים לפני Test
טפלו רק בשגיאות צפויות	באגים מתגלים מאוחר מדי	except הסתרת כל חריגה עם כללי

2.17 תרגול מדורג — ללא פתרונות

רמה א׳ — זיהוי והבנת מושגים

☐ הסבירו במילים שלכם את ההבדל בין מחלקה, מופע, תכונה ומתודה. תנו דוגמה חדשה שאינה מחשב.

☐ עבור Device("PC-07", 68), כתבו מה מתרחש בעת יצירת המופע ואיזה מידע נשמר בו.

☐ בתוך __init__ מופיע גם name וגם self.name. מדוע צריך את שניהם?

☐ בתוכנית b = a ולאחר מכן "b.name = "X: מה יהיה a.name? נמקו בלי להריץ קוד.

רמה ב׳ — חישוב, מעקב ותכנון

☐ בנו מחלקת Sensor המקבלת שם וערך מדידה ומחזירה True כאשר המדידה חריגה לפי סף שנמסר.

☐ בנו טבלת מעקב לשלוש קריאות ל־update_temperature; בכל שלב ציינו מצב קודם, קלט, פלט ומצב חדש.

☐ צרו מחלקת Lab שמכילה לפחות שלושה Device, וכתבו פעולה המונה כמה מהם דורשים בדיקה.

☐ כתבו בדיקות עבור SimpleMinMax, כולל דוגמה שבה כל ערכי האימון זהים ודוגמה חדשה החורגת מטווח האימון.

רמה ג׳ — חשיבה של מפתח AI

☐ סרטטו תרשים מחלקות עבור מערכת שמזהה תקלות על סמך מדידות: אילו רכיבים הם נתונים ואילו מבצעים חיזוי?

☐ השוו שתי הצעות: מחלקה אחת של 250 שורות לעומת ארבע מחלקות בעלות אחריות ברורה. הציגו יתרונות ומגבלות לשתי החלופות.

☐ דונו מדוע לא נכון לקרוא לכללים מבוססי threshold בשם מודל למידת מכונה אם הסף לא נאמד מנתונים.

☐ הציעו שני מקרי בדיקה שיגלו באגים בהכנת נתונים לפני predict, והסבירו מה יקרה אם לא נבדוק אותם.

2.18 שאלת הערכה מסכמת בסגנון בחינה

מערכת SmartLab מכילה אוסף של עצמי Device. כל Device שומר שם, רשימת מדידות וטמפרטורה אחרונה. המחלקה Lab מפיקה דוח על כל המחשבים, והמחלקה ThresholdPredictor מסווגת את המחשבים לפי סף. לא כל קטע קוד המופיע בהמשך הוא מימוש של למידת מכונה; בחלק מן הסעיפים מדובר בסיווג לפי כלל קבוע.

סעיף א׳: הסבירו את הקשר בין Lab ל־Device ואת הקשר בין Device ל־Measurement. בחרו עבור כל קשר הכלה או ירושה ונמקו. סעיף ב׳: נתון שלמשתנים a ו־b יש אותה הפניה ל־Device. לאחר הוספת מדידה דרך b, כיצד ישתנה הממוצע המחושב דרך a? סעיף ג׳: תכננו מתודה שמחזירה את הממוצע בלי לשנות את מצב העצם. סעיף ד׳: הסבירו כיצד אפשר להחליף את ThresholdPredictor במודל שנלמד מנתונים תוך שמירה על ממשק predict. סעיף ה׳: הציעו שני ניסויי בדיקה ואיום אחד על אמינות המערכת.

בעת פתרון נדרשת דרך חשיבה ולא רק תשובה סופית. כתבו אילו הנחות קיבלתם, מתי מתבצע שינוי מצב ומה ניתן להסיק מן הפלט. השאלה מקורית בסגנון הערכה לימודית, ולא שאלה רשמית משאלון בגרות.

2.19 סיכום מושגים ודף נוסחאות

מושג	פירוש קצר
Class	תבנית המגדירה נתונים ומתודות
Instance/Object	מופע יחיד שנוצר ממחלקה
Attribute	נתון השייך לעצם
Method	פעולה של עצם
self	הפניה למופע שעליו פועלת המתודה
__init__	מתודת אתחול בעת יצירת מופע
Encapsulation	ארגון מידע וגישה מבוקרת אליו
Composition	עצם המכיל עצמים אחרים
Inheritance	מחלקה מיוחדת היורשת התנהגות ממחלקת בסיס
Polymorphism	אותו ממשק מפעיל התנהגויות שונות
Identity	האם שתי הפניות מצביעות לאותו עצם
fit/transform	למידת פרמטרי הכנה והמרת נתונים
predict	הפקת תחזית ממידע קלט
Unit test	בדיקה של רכיב תוכנה קטן

אין נוסחה מתמטית יחידה ל־OOP: ההצלחה נמדדת ביכולת לתכנן אחריות, להסביר מצב של עצמים, לבצע בדיקות ולהשתמש בקוד באופן אמין. כאשר נרצה לבצע חישוב מספרי, נוכל ליישם אותו בתוך מתודה תוך שמירה על ההפרדה בין נתונים ללוגיקה.

2.20 מקורות והמשך הדרך

עוגן בתכנית: מפת התמחות למידת מכונה ולמידה עמוקה — פרק 2, תכנות מונחה עצמים, 30 שעות: מסלול . https://github.com/samiabo75/Data-Science-ML-Course/tree/main/03-Deep-Learning-5-Units ההכנה המלווה: https://github.com/samiabo75/Data-Science-ML-Course/tree/main/01-Python-Basics/ 03-Deep-Learning-Python

מקורות מקצועיים: The Python Tutorial — Classes (docs.python.org/3/tutorial/classes.html); Python Data Model (docs.python.org/3/reference/datamodel.html); Python documentation — copy, dataclasses; Effective Python — Brett Slatkin, מהדורות מעודכנות; Hands-On Machine Learning with Scikit-Learn and PyTorch — Aurélien Géron, O’Reilly 2025. הספרים והתיעוד משמשים להעמקה מושגית ולהשראה פדגוגית; דוגמאות SmartLab, ההסברים והתרגילים נכתבו באופן מקורי.

הפרק הבא בחוברת היסוד: פרק 3 — מבוא למדעי הנתונים ולמידת מכונה (10 שעות). בו נחבר את מחלקות הנתונים והתכנות אל שאלת מחקר, מאגר נתונים, מודל, אימון וחיזוי.

דף סיום אישי — האם אני מוכן לפרק הבא?

לפני המעבר למבוא למדעי הנתונים, דרגו את שליטתכם בכל מיומנות. בחרו דוגמת קוד משלכם שמוכיחה לפחות שני סעיפים. אם אחת המיומנויות אינה ברורה, חזרו לדוגמה ולתרגול המתאימים; עדיף לזהות פער עכשיו מאשר באמצע פרויקט גדול.

מיומנות נבדקת	שולט/ת	דורש/ת תרגול
אני יוצר מחלקה ומסביר את תפקיד __init__ ו־self	☐	☐
אני מבדיל בין משתנה מופע למשתנה מחלקה	☐	☐
אני יודע להסביר הפניה כפולה והעתקת עצם	☐	☐
אני מזהה קשר הכלה לעומת קשר ירושה	☐	☐
אני מממש מתודה שמשנה מצב ומחזירה ערך	☐	☐
אני מתכנן מחלקה בעלת אחריות מוגדרת	☐	☐
אני בודק קוד בקלט רגיל ובמקרה קצה	☐	☐
אני מבדיל בין כלל קבוע לבין מודל שלמד מנתונים	☐	☐

משימת רפלקציה קצרה: כתבו שני משפטים על החלטת תכנון אחת שתחסוך לכם עבודה במיזם AI בעתיד. הוסיפו דוגמה אחת לבאג שזיהיתם בעצמכם, כיצד מצאתם אותו ומה שיניתם כדי לתקן. המטרה אינה לכתוב תשובה ״יפה״ אלא להראות למידה עצמאית מתוך התנסות.

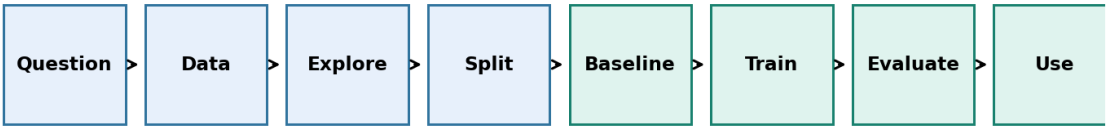
חוברת היסוד לספר הקורס המאוחד

פרק 3 — מבוא למדעי הנתונים וללמידת מכונה

כיצד הופכים נתונים לשאלה מחקרית, למודל, לתחזית ולהחלטה אחראית?

התמחות למידת מכונה ולמידה עמוקה, 5 יחידות לימוד | כיתות י״א–י״ב | מסגרת הפרק בתכנית: 10 שעות

מהדורת תלמיד · הסברים מלאים · דוגמאות מקוריות · חישובים ידניים · Python ו־Colab · משימות עצמאיות



מפת הפרק: לא מתחילים מבחירת אלגוריתם, אלא מהגדרת בעיה שאפשר לבדוק.

פתח דבר — מדוע אנחנו צריכים פרק מבוא?

בפרק הראשון למדנו לכתוב תוכניות ב־Python. בפרק השני למדנו כיצד לארגן קוד באמצעות מחלקות ועצמים. כעת נעמוד בפני שאלה חדשה: כיצד מחשב יכול להשתמש בדוגמאות מן העבר כדי לקבל תחזית לגבי מקרה שטרם פגש? זו כבר אינה רק שאלה בתכנות. זו שאלה של נתונים, מדידה, הנחות, ניסוי והערכת תוצאות.

המטרה אינה להכיר כמה שיותר אלגוריתמים בתוך עשר שעות. המטרה היא להבין את המילים, את דרך החשיבה ואת סדר הפעולות שילוו אותנו בכל פרקי ההמשך: רגרסיה, KNN, פרספטרון, CNN, רשתות חוזרות ופרויקט הגמר. לכן נשתמש בסיפור אחד שחוזר לאורך הפרק: דודו בונה מערכת המסייעת למעבדת המחשבים בבית הספר לקבוע אילו מחשבים כדאי לבדוק קודם.

מה תוכלו לעשות בסיום הפרק?

- להבחין בין נתון בודד, רשומה, מאפיין, תווית, מערך נתונים ומודל.
- להסביר את ההבדל בין AI, למידת מכונה ולמידה עמוקה, בלי להניח שכל מערכת חכמה לומדת.
- לזהות האם בעיה מתאימה לסיווג, לרגרסיה או לקיבוץ, ולנסח שאלת חקר ניתנת לבדיקה.
- להגדיר את מטריצת המאפיינים X ואת משתנה המטרה y, ולהסביר מדוע אסור להשתמש במידע שאינו זמין בזמן החיזוי.
- לקרוא קוד אימון בסיסי, להסביר את תפקידי fit, predict ו־score, ולהשוות למודל בסיס.
- להציג תוצאה מספרית בזהירות: מה נמדד, באילו נתונים, ומה איננו יכולים להסיק.

חלוקת שעות מוצעת לפרק

שעות	תוכן ועבודה	יחידת לימוד
2	הקשר לעולם ה־AI, שאלת חקר, טבלת תצפיות	א — בעיה ונתונים
2	סוגי למידה, X/y, סיווג מול רגרסיה וקיבוץ	ב — שפת המודלים

שעות	תוכן ועבודה	יחידת לימוד
2	הכנת נתונים, חלוקה, קו בסיס, טעויות והטיות	ג — ניסוי הוגן
2	חישוב ידני, קוד Colab, מדד הצלחה	ד — מודל ראשון
2	ניתוח מקרה, שאלות דמויות בחינה ותחילת פרויקט	ה — העברה והערכה

עוגן בתכנית: פרק 3 — "מבוא למדעי הנתונים ולמידת מכונה", 10 שעות, לפי מפת מסלול ההתמחות במאגר הקורס. החלוקה למפגשים והדוגמאות הן תכנון דידקטי שלנו. פרטי החובה המדויקים כפופים למסמך הרשמי התקף.

3.1 הסיפור: דודו רוצה למנוע השבתה של מחשבים

במעבדת המחשבים יש מחשבים רבים. חלקם פועלים באופן תקין, ואחרים נוטים להיתקע בדיוק כשמתחיל מבחן. דודו רוצה לקבל בכל בוקר רשימה של מחשבים שכדאי לבדוק לפני השיעורים. נניח שהוא מסוגל למדוד לכל מחשב טמפרטורה ממוצעת, מספר שגיאות בשבוע ושעות שימוש ביום. בנוסף, בסיום בדיקה מקצועית נרשמת תווית: האם המחשב נזקק לטיפול דחוף או לא.

חשוב להבדיל בין שתי פעולות. הראשונה היא כלל שכתב מתכנת: "אם הטמפרטורה גבוהה מ־70, הצג אזהרה". הכלל מבצע החלטה על פי סף קבוע מראש. הפעולה השנייה היא למידה מדוגמאות: המחשב מקבל נתוני עבר ותוצאות בדיקה, ומנסה לגלות קשר שעוזר לנבא טיפול דחוף במקרה חדש. בשלב זה עדיין אין הבטחה שהלמידה תהיה טובה יותר מכלל ידני.

ID	Temp C	Errors	Hours	Label y
PC-01	52	1	5	0
PC-02	67	2	8	1
PC-03	58	1	6	0
PC-04	74	4	10	1

X = three measurements; y = target label

דוגמת רשומות: שלוש עמודות מדידה ותווית אחת. הנתונים בספר מומצאים לצורכי הוראה בלבד.

3.2 נתונים, תצפיות, מאפיינים ותוויות

נתון אינו עדיין ידע

המספר 67, כשהוא עומד לבדו, כמעט אינו אומר דבר. האם זה גיל, מחיר או טמפרטורה? כדי להפוך אותו לנתון שימושי צריך לדעת מה נמדד, באיזו יחידה, מתי ובאיזו דרך. אם 67 הוא טמפרטורת המעבד בצֶלְסִיוּס שנמדדה אתמול, כבר אפשר לחקור אותו. משמעות הנתון נובעת מן ההקשר ומאיכות המדידה, לא מן המספר לבדו.

המונחים שנשתמש בהם

דוגמה מן המעבדה	הסבר לתלמיד	מונח
טבלת 12 מחשבים	אוסף דוגמאות עם מבנה ידוע	מערך נתונים — Dataset

מונח	הסבר לתלמיד	דוגמה מן המעבדה
תצפית — Observation / Sample	מקרה אחד שעליו נאספו נתונים	מחשב PC-04
מאפיין — Feature	פרט קלט שיש לנו בזמן הניבוי	טמפרטורה, מספר שגיאות
מטרה או תווית — Target / Label	מה שאנו מבקשים לנבא	האם נדרש טיפול דחוף?
מודל — Model	מנגנון שחושב קשר בין קלט לתוצאה	מסווג שמפיק 0 או 1
תחזית — Prediction	הפלט שמופק עבור מקרה חדש	"נדרש טיפול"

עמודת מזהה כגון PC-04 מסייעת לנו לעקוב אחר הרשומה, אך לרוב אינה תכונה טובה לחיזוי: היא לא מתארת את מצבו של המחשב. יתרה מכך, מזהה שמרמז על מקור הנתונים עלול ליצור דפוס מלאכותי. בחירת מאפיינים דורשת הבנה של הבעיה, לא רק בחירה של כל העמודות המספריות.

ייצוג מתמטי: X ו־y

נסמן את כל מאפייני הקלט באות X ואת הערך שאנחנו רוצים לנבא באות y. אם יש 12 מחשבים ושלושה מאפיינים לכל אחד, X תהיה מטריצה בעלת 12 שורות ו־3 עמודות. לעומתה y יהיה וקטור בן 12 תוויות. חייבת להיות התאמה מלאה: התווית במקום השביעי שייכת לתצפית השביעית בדיוק.

X.shape = (12, 3) y.shape = (12,)

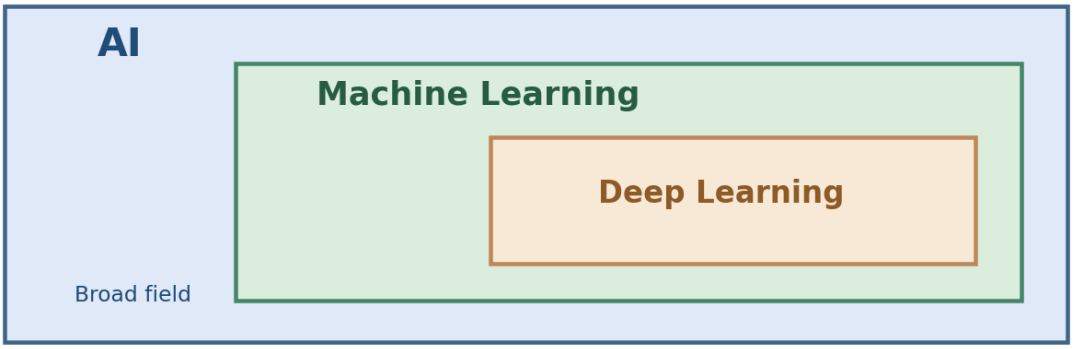
הסוגריים אינם מציינים שני משתנים: מדובר ב־shape, תיאור הממדים של מערך. כאשר עובדים עם pandas, שמות העמודות מסייעים לנו להבין מה מייצג כל רכיב. כאשר עוברים ל־NumPy, אין להניח שמערכת התכנות תזכור את משמעות היחידות עבורנו.

```
import pandas as pd

data = {"temperature": [52, 67, 58, 74],
        "errors": [1, 2, 1, 4],
        "hours": [5, 8, 6, 10],
        "urgent": [0, 1, 0, 1]}
df = pd.DataFrame(data)
X = df[["temperature", "errors", "hours"]]
y = df["urgent"]
print(X.shape, y.shape)
```

בדוגמה זו אנחנו משתמשים בארבע רשומות בלבד כדי לראות את המבנה. אין כאן מספיק דוגמאות כדי להוכיח שמודל יהיה שימושי בעולם האמיתי. כאשר יהיו לנו יותר נתונים, אותו מבנה קוד ימשיך לפעול.

3.3 כיצד AI, למידת מכונה ולמידה עמוקה קשורות זו לזו?



שלוש רמות של רעיונות: למידה עמוקה היא משפחה של שיטות בלמידת מכונה, והיא חלק מעולם ה־AI.

בינה מלאכותית (AI) היא שם רחב לתחום שמפתח מערכות המבצעות משימות הדורשות יכולות כגון הסקה, זיהוי דפוסים, תכנון או עבודה בשפה. חלק מן המערכות משתמשות בכללים ובחיפוש, ולא בכל מערכת כזו מתקיימת למידה מנתונים.

למידת מכונה (Machine Learning) מתמקדת בשיטות שמשתמשות בדוגמאות כדי להתאים מודל. לעיתים ההתאמה היא בחירת מקדמים ברגרסיה; לעיתים היא חיפוש קבוצות דומות; ולעיתים היא אימון מיליוני משקולות ברשת עצבית. המשמעות המשותפת היא שהמערכת מסתמכת על מבנה שנתגלה בנתונים ולא רק על אוסף כללים שנוסח מראש עבור כל מקרה.

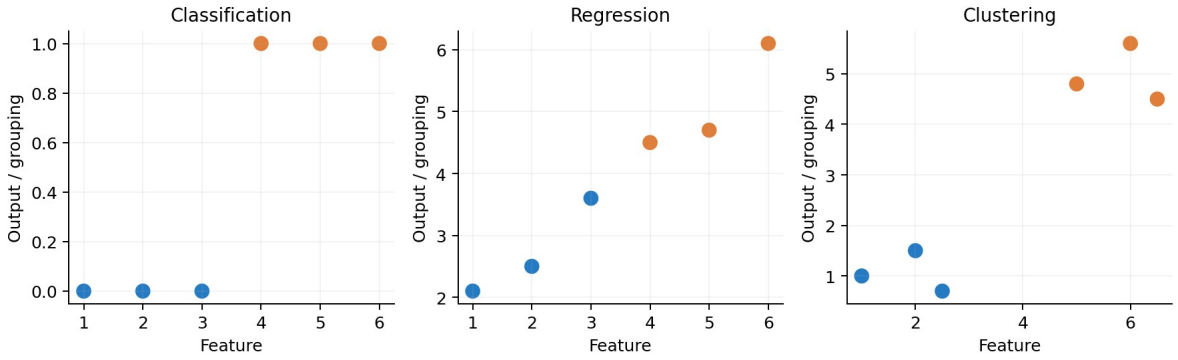
למידה עמוקה (Deep Learning) היא קבוצת שיטות המבוססת על רשתות עצביות עם כמה שכבות עיבוד. במקרים רבים היא יעילה לתמונות, צליל וטקסט, אך זה אינו אומר שהיא הבחירה המתאימה לכל בעיה. במיוחד כשיש מעט נתונים טבלאיים, מודל פשוט עשוי להיות קל יותר להסבר ואף מדויק יותר.

> שאלת הבנה: האם "תוכנה שמשתמשת ב־if" היא בהכרח מערכת AI? לא. האם כל מודל למידת מכונה הוא רשת עצבית? לא. האם תוצאת מודל היא אמת ודאית? גם לא.

3.4 מהי בכלל המשימה שלנו?

סיווג, רגרסיה וקיבוץ

לפני בחירת אלגוריתם צריך להגדיר מהו הפלט המבוקש. בסיווג (Classification) מבקשים לנבא קטגוריה — לדוגמה "טיפול דחוף" או "אין טיפול דחוף". ברגרסיה (Regression) מבקשים לנבא ערך כמותי — למשל מספר הדקות המשוער עד לסיום תיקון. בקיבוץ (Clustering) לא מספקים למערכת מראש תווית טיפול, אלא מבקשים ממנה לזהות דמיון בין רשומות.



שלוש משימות שונות: סיווג למחלקות, חיזוי ערך מספרי, וגילוי קבוצות ללא תוויות ידועות.

שאלת חקר	סוג משימה	הסיבה
האם מחשב זקוק לטיפול דחוף?	סיווג	התשובה היא קטגוריה
כמה דקות יימשך התיקון?	רגרסיה	התשובה היא מספר כמותי
אילו מחשבים פועלים בדפוסים דומים?	קיבוץ	אין תווית תשובה מראש
האם הציון עולה עם שעות למידה?	ניתוח קשרים	זו שאלת קשר; לא כל שאלה דורשת תחזית

יש להיזהר מן הטעות "אם התוויות כתובות במספרים, זו רגרסיה". התוויות 0 ו־1 בדוגמת טיפול הן שמות קוד למחלקות ולא כמות שיש לחזות. לעומת זאת ציון 85 או זמן תיקון 120 דקות הוא משתנה כמותי.

למידה מונחית, לא־מונחית ולמידת חיזוק

למידה מונחית (Supervised Learning) משתמשת בדוגמאות שיש להן תוצאה ידועה: X ו־y. כך אפשר ללמד מסווג או רגרסור. למידה לא־מונחית (Unsupervised Learning) מחפשת מבנה בנתונים ללא תוויות המטרה, למשל קבוצות בעלות דפוס שימוש דומה. למידת חיזוק (Reinforcement Learning) עוסקת בסוכן המקבל משוב על פעולות שביצע ומנסה לשפר את מדיניות הפעולה לאורך זמן. בפרק זה נכיר את ההבדלים באופן מושגי; את שיטות הסיווג והקיבוץ נלמד באופן שיטתי בפרקים המתקדמים.

3.5 לנסח שאלת חקר שהמחשב יכול לענות עליה

"אני רוצה לבנות AI חכם למחשבים" אינו ניסוח מחקרי: לא ברור מהו הקלט, מהי התשובה הרצויה ואיך נבדוק הצלחה. לעומתו "האם ניתן לנבא אם מחשב יזדקק לטיפול דחוף בשבוע הקרוב על סמך נתונים שהיו זמינים עד סוף השבוע הקודם?" הוא ניסוח שאפשר לבדוק. הוא קובע זמן חיזוי, מאפיינים אפשריים ותווית עתידית.

חמש שאלות לפני כתיבת קוד

- מהו המקרה היחיד שאנחנו מנתחים — מחשב, יום עבודה, תלמיד או תמונה?
- מהי התוצאה שנרצה לנבא או התופעה שנרצה לתאר?
- מאילו מקורות יגיעו הנתונים, והאם מותר להשתמש בהם?
- אילו מאפיינים באמת זמינים בזמן שבו המערכת אמורה לפעול?
- כיצד נדע שהפתרון מועיל בהשוואה לדרך פשוטה יותר?

דוגמת דליפת מידע: אם התווית היא "זקוק לטיפול בשבוע הבא", אסור לתת למודל מאפיין בשם "מספר תיקונים שבוצעו בשבוע הבא". המידע הזה מופיע רק לאחר הזמן שבו תופק התחזית. מודל שיקבל אותו אולי יציג ציון נפלא בבדיקה, אך לא יוכל להשתמש בו באמת.

3.6 מחקר נתונים אינו מתחיל באימון מודל

בדיקת איכות של מערך הנתונים

לפני אימון צריך להבין אם נתוני המקור אמינים. ייתכן שמדידות הטמפרטורה נלקחו מחיישנים שונים; ייתכן שעמודת שעות שימוש מכילה ערכים חסרים; וייתכן שמחשב אחד מופיע בטבלה כמה פעמים. בלי בדיקה אנחנו עלולים ללמד את המודל דפוסים הנובעים משגיאות הקלדה או מכפל רשומות.

לכל עמודה נבדוק: סוג הנתון, יחידת המידה, טווח אפשרי, מספר חסרים, חריגים ומקור. בנוסף נבדוק אם המחלקות מאוזנות, אם תוויות ניתנו באופן עקבי, ואם הנתונים מייצגים גם את הסיטואציה שבה ישתמשו במודל בעתיד.

```
print(df.head())
print(df.info())
print(df.isna().sum())
print(df["urgent"].value_counts())
```

אלה פעולות סקירה ולא תהליך למידה. הן עוזרות לנו לשאול שאלות על הנתונים. בפרק הבא — EDA — נעמיק בגרפים, סטטיסטיקה ובדיקה של קשרים בין מאפיינים. כאן נתרכז במשמעות השאלות ובצורך לתעד את ממצאי הבדיקה.

מדוע הטיות בנתונים חשובות?

דמיינו שמרבית נתוני המעבדה הגיעו ממחשבים ישנים, ובחודש הבא מתקינים מחשבים חדשים. מודל שהתאים את עצמו לדוגמאות הישנות עלול להצליח פחות בסביבה החדשה. גם אם קוד האימון תקין מבחינה טכנית, איכות התחזית תלויה במידת ההתאמה בין נתוני האימון לנתונים שהמודל יפגוש בפועל.

3.7 מאוסף דוגמאות לניסוי הוגן



Illustration only: proportions depend on dataset and evaluation plan

המחשה עקרונית של חלוקת נתונים. אין יחס אחיד שמתאים לכל מערך, ובנתוני זמן נדרשת חלוקה לפי סדר הזמן.

אם נשאל את המודל על אותן דוגמאות ששימשו לבנייתו, נקבל תשובה לשאלה "עד כמה המודל מתאים למה שכבר ראה?". אבל המטרה שלנו היא לבדוק נתונים חדשים. לכן נהוג להפריד בין Train, שממנו לומדים; Validation, שבעזרתו בוחרים בין גרסאות מודל; ו־Test, המיועד להערכה סופית לאחר סיום הבחירות.

אפשר גם להשתמש ב־Cross-Validation על קבוצת האימון במקום לשמור מראש קבוצת Validation קבועה. התלמיד אינו צריך לזכור כאן כל פרוטוקול אפשרי, אבל עליו להבין כלל עקרוני: אסור להשתמש בנתוני Test כדי לבחור מאפיינים, לכוונן פרמטרים או להחליט איזה מודל לפרסם.

מהי זליגת מידע (Data Leakage)?

זליגת מידע נוצרת כאשר תהליך האימון או קבלת ההחלטות משתמש במידע שאמור להיות מחוץ להישג ידו של המודל. דוגמה נוספת היא חישוב ממוצע לכל מערך הנתונים לפני חלוקה, ואז שימוש בממוצע הזה כדי לתקנן את נתוני האימון. במקרה זה נתוני הבדיקה כבר השפיעו על ההכנה, אפילו אם לא הוכנסו ישירות לפעולת fit של המסווג.

בפרקים המתקדמים נשתמש ב־Pipeline שמבטיח כי פעולות הכנה מתאימות את עצמן רק לנתוני האימון בכל פיצול. זו לא רק המלצה תכנותית: זו דרך להגן על אמינות המסקנה המדעית.

> בדיקת הבנה: אם בחרנו ערך k ב־KNN אחרי שבדקנו עשרה ערכים על Test, האם Test עדיין מספק הערכה בלתי תלויה? לא. אפשר ללמוד מהניסוי, אך אין להציג את אותו Test כהערכה סופית בלתי תלויה.

3.8 קודם קו בסיס, אחר כך מודל מסובך

קו בסיס (Baseline) הוא פתרון פשוט שמשמש נקודת השוואה. במקרה של חיזוי האם מחשב יזדקק לטיפול דחוף, אפשר להתחיל ממודל שתמיד מנבא את המחלקה הנפוצה ביותר. זו אינה שיטת חיזוי מתוחכמת, אבל היא עוזרת לברר האם המודל החדש מוסיף ערך מעבר לניחוש פשוט.

חישוב ידני: מדוע Accuracy של 90% עלול להטעות?

נניח שנבדקו 100 מחשבים. 90 לא היו זקוקים לטיפול דחוף, ו־10 כן. מודל שמנבא לכל המחשבים "לא דרוש טיפול" מסווג נכון 90 מקרים מתוך 100. לכאורה הוא מדויק ב־90%, אך הוא החמיץ את כל 10 המחשבים שבאמת נזקקו לטיפול דחוף.

$$\text{Accuracy} = 90 / 100 = 0.90 = 90\%$$

$$\text{Recall (urgent)} = 0 / 10 = 0$$

כאשר מחלקה אחת נפוצה מאוד, מדד הדיוק לבדו עלול להציג תמונה מעודדת אך מטעה.

המסקנה איננה ש־Accuracy חסר ערך. המסקנה היא שיש לבחור מדדים לפי הבעיה ולפי מחיר הטעות. במערכת בטיחותית החמצת מקרה חמור עשויה להיות יקרה במיוחד. בנושא זה נעמיק בפרק הערכת המודלים שכבר נכתב בספר המאוחד.

3.9 מודל ראשון: מחישוב ידני להרצת קוד

דוגמה ידנית קטנה — שש תחזיות

נניח שיש לנו שש דוגמאות בדיקה. התוויות האמיתיות והתחזיות של מודל ראשוני הן אלה:

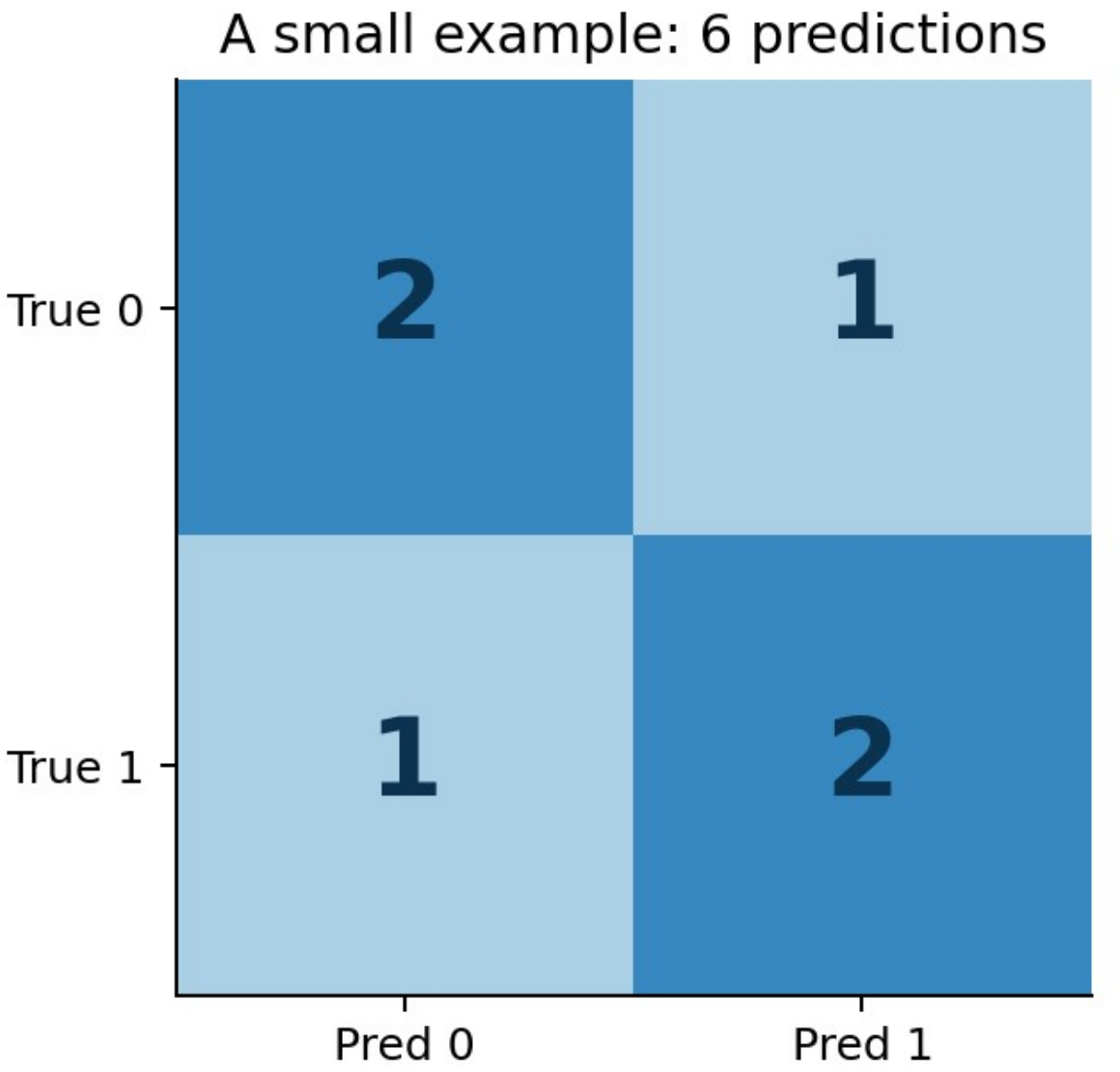
תצפית	תווית אמיתית y	תחזית $\hat{y}$	נכון?
1	0	0	כן
2	0	1	לא
3	1	1	כן
4	1	0	לא
5	0	0	כן
6	1	1	כן

ארבע תחזיות נכונות ושתי תחזיות שגויות. לכן Accuracy הוא ארבע שישיות, כלומר כשני שלישים. כעת נבנה מטריצת בלבול: שני מקרים של 0 זוהו נכון, שני מקרים של 1 זוהו נכון, מקרה אחד של 0 סווג בטעות כ־1, ומקרה אחד של 1 סווג בטעות כ־0.

$$\text{Accuracy} = (2 + 2) / 6 = 4/6 \approx 0.667$$

$$\text{Precision}(1) = 2 / (2 + 1) = 2/3$$

$$\text{Recall}(1) = 2 / (2 + 1) = 2/3$$



מטריצת בלבול עבור שש הדוגמאות: השורה היא אמת, העמודה היא תחזית.

אין להסיק משש דוגמאות זעירות שהמודל יהיה מדויק בעתיד. התרגיל נועד ללמד את משמעות המדד ואת הדרך לבדוק חיזוי. בדוגמה אמיתית נגדיר מראש כיצד מדגמים את נתוני הבדיקה, ומהי רמת אי־הוודאות של האומדן.

מודל ראשון עם נתוני Iris

במקום לאמן מסווג על ארבעת המחשבים שבטבלה, נשתמש כעת במאגר Iris המובנה בספריית scikit-learn. כל תצפית היא פרח, ובכל פרח ארבע מדידות. תווית המטרה מציינת את קבוצת הפרח. נשתמש בשיטה מוכרת, KNN, בלי להיכנס כרגע לכל המתמטיקה שלה: הוא בוחן דוגמאות אימון קרובות ומנבא בהתאם לתוויות שלהן. נלמד אותו לעומק בפרקים המתקדמים.

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.dummy import DummyClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)
```

```
baseline = DummyClassifier(strategy="most_frequent")
baseline.fit(X_train, y_train)
model = KNeighborsClassifier(n_neighbors=3)
model.fit(X_train, y_train)

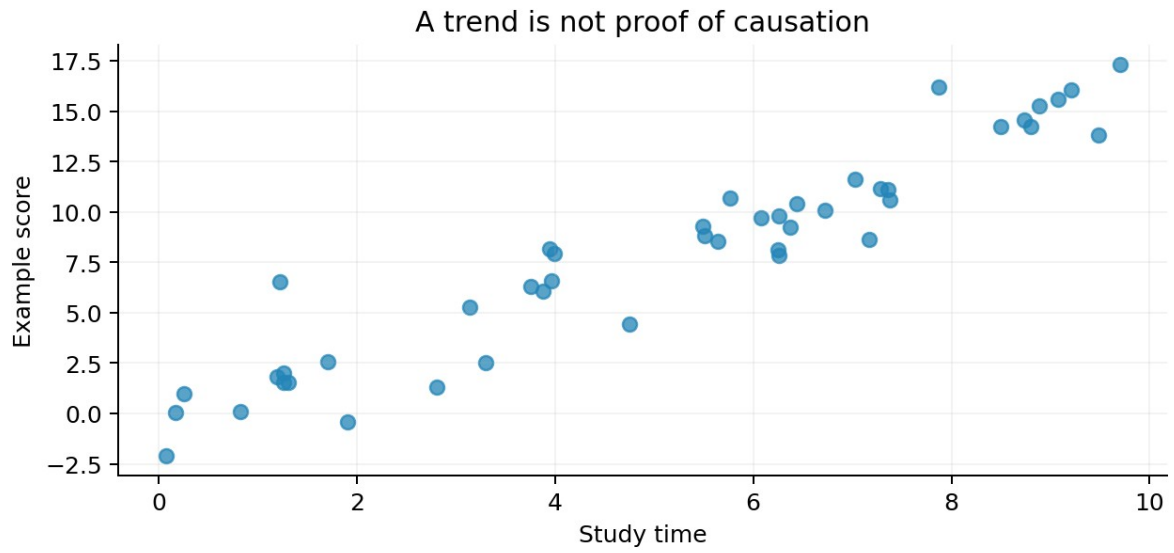
print("Baseline:", accuracy_score(y_test, baseline.predict(X_test)))
print("KNN:", accuracy_score(y_test, model.predict(X_test)))
```

בקריאת הקוד חשוב להבין את הרצף: load_iris טוענת נתונים; train_test_split מחלקת אותם; DummyClassifier מספק קו בסיס; fit משתמש בדוגמאות אימון; predict מפיק תחזיות; ו־accuracy_score משווה אותן לתוויות האמיתיות. גם אם נקבל ציון גבוה, זו תוצאה של חלוקה אחת, ולא הבטחה לביצועים בכל מצב.

חמש שאלות שכל תלמיד חייב לענות עליהן לאחר הרצה

- מה כל שורה ב־X מייצגת, וכמה מאפיינים יש בה?
- היכן בקוד בוצעה חלוקת הנתונים, ומה מטרתה?
- איזו פקודה "לומדת" מן הדוגמאות, ואיזו פקודה מחזירה תחזיות?
- איזה מודל שימש קו בסיס, ולמה כדאי להשוות אליו?
- מה אפשר להסיק ממדד הדיוק — ומה עדיין לא אפשר להסיק?

3.10 מתאם, סיבתיות והקשר האנושי



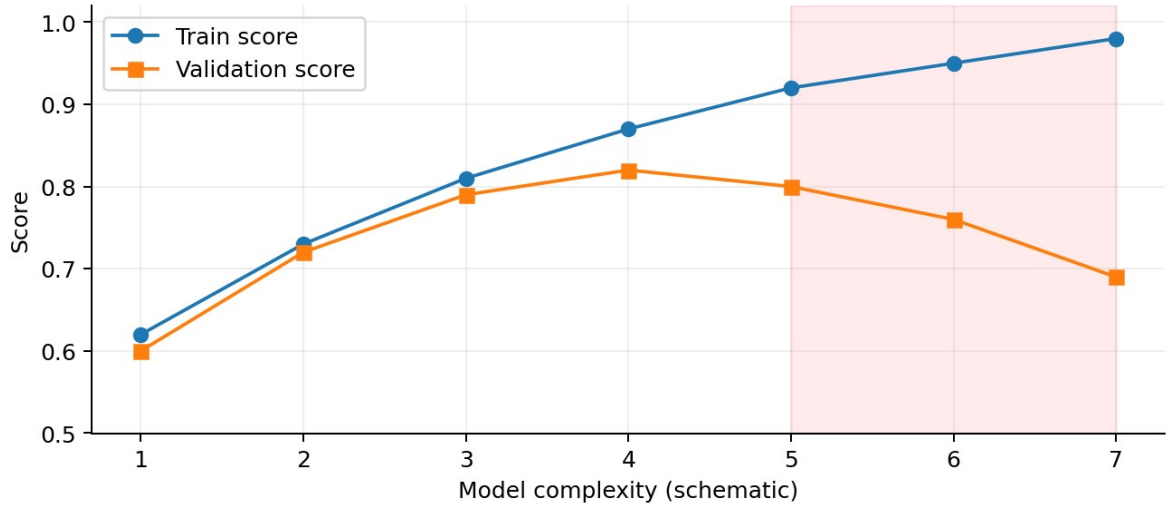
תרשים פיזור של נתונים מדומים: קו מגמה אינו הוכחה שהמשתנה האחד גרם לשני.

גם כאשר נזהה קשר בין מאפיין לתוצאה, אין להסיק אוטומטית קשר של סיבה ותוצאה. למשל, אפשר למצוא קשר בין שעות שימוש במחשב לבין מספר התקלות. אבל ייתכן שמחשבים ישנים גם נמצאים בשימוש רב יותר וגם מתקלקלים לעיתים קרובות. בלי לבחון את גורמי הרקע, הקשר שנמצא לא אומר ששעות השימוש לבדן גרמו לתקלה.

כאשר עוברים למערכות העוסקות באנשים, חובת הזהירות גדלה. מודל לחיזוי ציונים, לזיהוי קושי לימודי או למיון בקשות עשוי להשפיע על הזדמנויות של אנשים. לכן יש להתייחס לפרטיות, לאיכות תוויות, לייצוג הוגן של קבוצות וליכולת לערער על החלטה. ההבנה הזאת היא חלק מהמקצועיות של מפתח AI, לא נושא נפרד שאפשר לדלג עליו.

3.11 כיצד מזהים התאמת יתר?

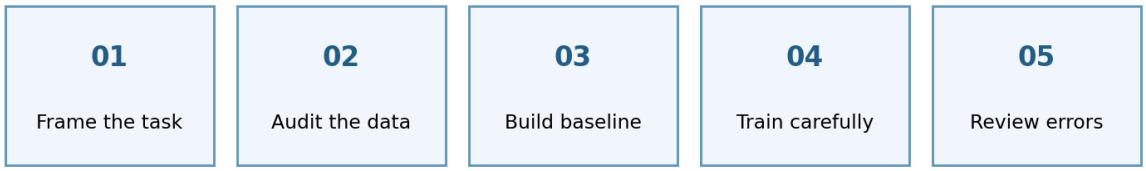
נניח שמודל מורכב מצליח לזכור כל פרט קטן בנתוני האימון. לפעמים הוא אף מצליח במבחן על דוגמאות אלו באופן מושלם, אבל נכשל כאשר מגיעות תצפיות חדשות. תופעה זו מכונה התאמת יתר (Overfitting). המטרה אינה לקבל את ציון האימון הגבוה ביותר, אלא לבחור מודל שמצליח להכליל.



עקומות סכמטיות: ציון האימון יכול להמשיך לעלות גם כאשר ציון הוולידציה מתחיל לרדת.

לכן פרק זה לא מסתיים בהדפסת Accuracy בלבד. בכל ניסוי נדרש לתעד את חלוקת הנתונים, את החלטות הפיתוח, את מגבלות המדידה ואת השגיאות שנמצאו. בהמשך הספר נראה כיצד להשתמש בוולידציה, ברגולריזציה ובעצירה מוקדמת, אך כאן חשוב להבין קודם את הבעיה.

3.12 יומן מדעי: מה על תלמיד להגיש בסוף ניסוי?



A research notebook should explain each decision, not just show a final score.

הסבר ניסוי מקצועי הוא סיפור החלטות: שאלה, נתונים, קו בסיס, מודל, בדיקת שגיאות.

מה כותבים	רכיב בדוח
מה התופעה שאותה מנסים לחזות או להבין?	שאלת חקר
מאין הנתונים, מהו היקפם ומהן מגבלותיהם?	מקור הנתונים
מה מייצגת רשומה? מה זמינות התכונות בעת ניבוי?	תצפיות ומאפיינים
כיצד נמנעה זליגת מידע?	חלוקה והכנה
מהי נקודת ההשוואה הפשוטה?	קו בסיס
איזו שיטה הופעלה, על אילו נתונים ואילו	מודל וניסוי

מה כותבים	רכיב בדוח
הגדרות נבחרו?	
מהו המדד, מהן השגיאות ומה עדיין לא ידוע?	תוצאה ומגבלה

3.12א חקר מקרה מפורט — מה באמת אומרת טבלת מחשבים?

נחזור למאגר שמופיע במחברת התלמיד. יש בו 12 מחשבים, ולכל מחשב שלוש מדידות ותווית טיפול. נניח שנבדקו שישה מחשבים שדרשו טיפול ושישה מחשבים שלא דרשו טיפול. מה אפשר להגיד על מערך כזה? ראשית, בדוגמה שלנו אין חוסר איזון בין המחלקות. אבל אין להסיק מכך שהנתונים מייצגים את בית הספר כולו: ייתכן שבחרנו בכוונה מספר שווה מכל קבוצה כדי להקל על התרגול. אם כך, שכיחות הצורך בטיפול בעולם האמיתי עשויה להיות אחרת לגמרי.

כעת נשאל על המאפיינים. טמפרטורה של 74 מעלות עשויה להיות משמעותית רק כאשר יודעים היכן נמדדה ובאילו תנאי עומס. גם מספר שגיאות 4 אינו מספר שאפשר לפרש לבד: האם מדובר בשגיאות מערכת חמורות, בניסיונות התחברות שגויים, או באירועים רגילים? לכן לפני אימון מודל אנחנו חייבים לכתוב "מילון נתונים" שבו לכל מאפיין הגדרה, יחידת מדידה ותנאי איסוף.

מאפיין	הגדרה ברורה שנדרשת בפרויקט	בדיקת איכות מוצעת
temperature	ממוצע מדידת חיישן המעבד במשך יום, במעלות צלזיוס	איתור קריאות חסרות או ערכים בלתי סבירים
errors	מספר שגיאות מערכת מסוג מוגדר במשך שבעה ימים	וידוא שכל המחשבים נספרו באותו חלון זמן
hours	ממוצע שעות שימוש יומי באותו שבוע	בדיקת ערכי 0 וערכים מעל 24
urgent	קביעה של גורם מקצועי לפי כלל אחיד	בדיקת עקביות בהגדרת "דחוף"

נניח שהמטרה היא תחזית למצב בשבוע הבא. במקרה כזה גם חלון הזמן של המאפיינים צריך להסתיים לפני תחילת השבוע הבא. אם בטעות נצרף מידע שנרשם לאחר תחילת התקלה, המודל עשוי לזהות את תוצאות התקלה במקום את סימני האזהרה שקדמו לה. כעת אפשר לראות מדוע הגדרת שאלת המחקר קובעת גם את אופן איסוף הנתונים, ולא רק את שם האלגוריתם.

> מסקנת ביניים: טבלה מסודרת אינה בהכרח מערך נתונים טוב. צריך לברר כיצד נוצרה כל שורה, כיצד נמדדה כל תכונה, ומתי התקבלה התווית.

3.12ב חישוב ידני של מודל רגרסיה וקו בסיס

עד כה חישבנו מדדי סיווג. כדי להבין כיצד אותן פעולות מחקריות משמשות גם ברגרסיה, נדמיין מודל שמנבא את משך תיקון המחשב בדקות. נציג ארבעה מקרים שלא שימשו לאימון. משכי התיקון האמיתיים הם 30, 40, 35 ו־45 דקות. תחזיות המודל הן 32, 39, 30 ו־42 דקות. כל תחזית היא ניסיון לנבא גודל מספרי, ולכן איננו מחשבים כאן Accuracy של סיווג.

שגיאה מוחלטת	תחזית	זמן אמיתי	מקרה
2	32	30	א
1	39	40	ב
5	30	35	ג
3	42	45	ד

שגיאה מוחלטת	תחזית	זמן אמיתי	מקרה

כדי לחשב MAE — ממוצע השגיאה המוחלטת — מחברים את כל ההפרשים המוחלטים ומחלקים במספר המקרים. מקבלים סכום 11 על ארבעה מקרים, כלומר 2.75 דקות. מדוע משתמשים בערך מוחלט? משום ששגיאה של מינוס חמש דקות אינה אמורה "לבטל" שגיאה של פלוס חמש דקות: שתיהן טעויות בחיזוי, למרות הכיוון השונה.

$$MAE = (2 + 1 + 5 + 3) / 4 = 2.75 \text{ minutes}$$

אבל עדיין אין לנו תשובה לשאלה אם המודל מועיל. נניח שבנתוני האימון זמן התיקון הממוצע היה 38 דקות. קו בסיס פשוט מנבא תמיד 38 דקות, ולכן על ארבעת המקרים החדשים נקבל שגיאות של 8, 2, 3 ו־7 דקות. הממוצע הוא 5 דקות. בדוגמה זו המודל המורכב משפר את MAE לעומת קו הבסיס. השוואה הוגנת מחייבת להשתמש באותם מקרי בדיקה ובאותו מדד.

$$\text{Baseline MAE} = (8 + 2 + 3 + 7) / 4 = 5 \text{ minutes}$$

שימו לב שהממוצע 38 התקבל מנתוני האימון בלבד. אילו היינו מחשבים את ממוצע זמני התיקון האמיתיים של קבוצת הבדיקה, היינו מאפשרים לתוצאות הבדיקה להשפיע על קו הבסיס. זו דוגמה נוספת לכך שלעיתים זליגת מידע מתרחשת בתוך חישוב שנראה תמים ביותר.

3.12ג אותה רשומה פעמיים — מדוע חלוקה אקראית אינה תמיד נכונה?

נניח שמחשב PC-07 נמדד בכל יום במשך שנה. אם נחלק באופן אקראי את כל המדידות, ייתכן שחלק מימי המחשב ישמשו לאימון ואחרים לבדיקה. במקרה זה המודל עלול לזהות תכונות קבועות של המחשב עצמו, ולכן ציון הבדיקה לא ישקף היטב את היכולת להתמודד עם מחשב חדש לגמרי. אם שאלת המחקר היא "כיצד ננבא מחשבים שלא הופיעו באימון?" כדאי לשקול חלוקה לפי מזהה מחשב או קבוצת מכשירים.

לעומת זאת, אם המטרה היא לחזות את השבוע הבא עבור אותם מחשבים שכבר קיימים, חשוב לשמור על הסדר הכרונולוגי. בשני המקרים צריך להתאים את שיטת ההערכה לשאלה המחקרית. אין חלוקת Train/Test אחת שהיא תמיד נכונה. תלמיד שמסביר את ההבדל בין שתי השאלות האלה מפגין הבנה של המחקר, ולא רק של פקודת Python.

טעות אפשרית	חלוקה הגיונית	מטרת הפרויקט
אותו מחשב גם באימון וגם בבדיקה	הפרדה לפי מזהה מחשב	חיזוי עבור מחשבים חדשים
שימוש בנתונים מן העתיד בזמן האימון	אימון על עבר ובדיקה על עתיד	חיזוי השבוע הבא
אי־שמירה על ייצוג המחלקות כאשר הן נדירות	חלוקה אקראית מבוקרת כשיש הצדקה	חיזוי על דוגמאות עצמאיות

3.12ד כיצד קוראים פלט של תוכנית בצורה מדעית?

נניח שקוד ההדגמה מדפיס שתי שורות: Baseline accuracy ואחריה KNN accuracy. לפני שמכריזים שהמודל השני "טוב יותר", עונים על סדרת שאלות. האם שני המודלים נבדקו על אותה קבוצת בדיקה? האם השתמשנו באותה הגדרת הצלחה? האם מספר הדוגמאות מספק כדי לקבל אומדן יציב? האם אותן קבוצות פרחים מופיעות באופן דומה בנתוני הבדיקה?

לאחר מכן מסבירים את התוצאה במילים מדויקות. ניסוח מקצועי הוא למשל: "בחלוקת הבדיקה הזאת, ובמדד Accuracy, המודל השיג תוצאה גבוהה מקו הבסיס. יש צורך בהערכה נוספת על דוגמאות אחרות כדי לקבוע עד כמה ההבדל יציב." לעומת זאת המשפט "המודל הבין את הפרחים ומעכשיו תמיד יצליח" חורג ממה שהניסוי מוכיח.

דרך טובה לבדוק הבנה היא להפוך את השאלה: מה ישתנה אם נחליף רק את random_state? החלוקה עשויה להשתנות, ולכן ייתכן שנקבל ציון אחר. מה ישתנה אם נשאיר את החלוקה קבועה ונחליף את המודל? כאן ההשוואה מתמקדת בשיטת החיזוי, אם יתר התנאים נשמרים. מה ישתנה אם נוסיף מאפיין שנמדד ביחידות גדולות מאוד? מודל המבוסס על מרחקים עשוי להגיב אחרת, ולכן נצטרך לבדוק את קנה המידה בפרקים הבאים.

3.12ה חיבור בין Python, OOP והמשך הלמידה העמוקה

בפרקי Python למדנו לכתוב פונקציות שמקבלות נתונים ומחזירות תוצאה. בפרק OOP למדנו להגדיר מחלקות בעלות אחריות ברורה. עכשיו אפשר להבין מדוע ספריית scikit-learn משתמשת בעצמים: אנו יוצרים אובייקט של מסווג, מאמנים אותו, ואז מבקשים ממנו תחזית. הפעולות fit ו־predict אינן מילים קסומות; הן שמות של פעולות בממשק תוכנתי, והן מחברות בין ארגון תוכנה לבין תהליך של למידת מכונה.

מאוחר יותר, בפרקי הרשתות העצביות, נשתמש ברעיון דומה אך עם חישובים מורכבים יותר: נגדיר ארכיטקטורה, נעביר נתונים קדימה, נחשב שגיאה, נעדכן משקולות ונעריך את התוצאה. מושגי היסוד שנלמד כאן — נתון, מאפיין, תווית, אימון, בדיקה ומדד — אינם נעלמים כאשר המודל הופך עמוק. הם נשארים הבסיס שעליו נבנה הפרויקט כולו.

3.13 תרגול מדורג — מן הידע להבנה עצמאית

רמה א — הבנת מושגים

- כתבו במילים שלכם מה ההבדל בין תצפית, מאפיין ומשתנה מטרה. הוסיפו דוגמה שאינה קשורה למחשבים.
- בחרו איזו משימה היא סיווג ואיזו רגרסיה: מחיר רכב, סוג ציפור, זמן משלוח, האם אימייל הוא דואר זבל.
- טבלה מכילה 200 תלמידים וחמישה מאפייני קלט. כתבו את גודל X ואת אורך y והסבירו את משמעותם.
- הסבירו מדוע עמודת מזהה אינה בהכרח מאפיין בעל ערך חיזויי.

רמה ב — חישוב, ניתוח קוד ופרשנות

- בדקו את שש התחזיות שבסעיף 3.9. חשבו מחדש Accuracy, Precision ו־Recall והסבירו איזו טעות מסוכנת יותר במערכת לתחזוקת מחשבים.
- במערך של 100 דוגמאות יש 90 דוגמאות ממחלקה אחת. הסבירו מדוע דיוק של 90% אינו בהכרח הצלחה.
- בקוד Iris מצאו את השורות של חלוקה, אימון, חיזוי והערכה. הסבירו מה עשויה לגרום השמטת random_state.
- נתונים נורמלו לפני החלוקה ל־Train ול־Test. האם יש כאן סיכון לזליגה? הסבירו בלי להשתמש רק במילה "אסור".

רמה ג — חשיבה והכנה לפרויקט גמר

- דודו מציע לחזות תקלה בשבוע הבא בעזרת "מספר תיקונים בשבוע הבא". נסחו במדויק מה שגוי במאפיין, וכתבו שני מאפיינים חלופיים.
- שני מודלים מציגים 92% ו־89% Accuracy. איזה מהם נבחר? נסחו שלוש שאלות שצריך לברר לפני החלטה.
- תכננו מחקר מקורי של 6–8 שורות: שאלת מחקר, סוג משימה, יחידת תצפית, שלושה מאפיינים, יעד ומדד הערכה.
- ציירו תרשים זרימה המציג את שלבי פרויקט AI, וכתבו מעל כל שלב סכנה אפשרית אחת ודרך לצמצם אותה.

3.14 משימת סיכום — כרטיס פרויקט ראשון

בחרו בעיה קטנה ומדידה מתוך עולם בית הספר, הספורט, התחבורה או המחשבים. הימנעו בשלב זה מנתונים אישיים רגישים. הציגו כרטיס פרויקט בן עמוד אחד: שם הפרויקט, מהי ההחלטה או התחזית, מיהו המשתמש, מהי תצפית, אילו מאפיינים ייאספו, מי יספק את תווית המטרה, מהו קו בסיס, וכיצד תיבחן הצלחה על נתונים שלא שימשו לאימון.

אין חובה לאמן עדיין מודל מורכב. תלמיד שמסוגל להגדיר היטב בעיה ולהסביר כיצד תיבדק כבר עשה צעד מקצועי משמעותי לקראת פרויקט הגמר.

3.15 מילון מושגים מסכם

מושג	הגדרה קצרה
Dataset	אוסף רשומות לצורך תיאור, חקר או למידה
Feature	מאפיין קלט המייצג מידע על תצפית
Target/Label	הערך שאותו מבקשים לנבא בלמידה מונחית
Model	מנגנון הממפה קלט לתחזית או לתוצאה
Train	דוגמאות המשמשות להתאמת המודל
Validation	דוגמאות לבחירת מודל ולכוונון תהליך
Test	נתונים להערכה סופית אחרי קבלת החלטות הפיתוח
Baseline	פתרון פשוט להשוואה
Generalization	היכולת להצליח על נתונים חדשים
Overfitting	התאמה חזקה מדי לפרטי האימון
Data Leakage	שימוש במידע שאינו אמור להיות זמין בתהליך
EDA	חקר נתונים לפני ובמהלך תהליך בניית המודל

3.16 קשרי המשך לשלוש תכניות הלימוד

היכן זה ממשיך	מה כבר למדנו / מה מוסיפים	מסלול
ספר יחידה 3 — EDA, סיווג, הערכה ופרויקט	תצפיות, מאפיינים, גרפים, KNN, מדדים וחקר	יחידה 3 — יסוד
פרקי נושאי ליבה 1–5 בספר	העמקת הסברים מתמטיים,	יחידה 5 — נושאי ליבה

היכן זה ממשיך	מה כבר למדנו / מה מוסיפים	מסלול
המאוחד	מרחקים, סטטיסטיקה ואלגוריתמים	
פרקים 4–16 בהמשך ספר ההתמחות	מחקר נתונים, מימוש, בחירה, תיעוד ואימון רשתות	התמחות Deep Learning

מעמד ההקשרים: מטרת פרק 3 בהתמחות אומתה לפי מפת הפרקים במאגר. ההפניות ליחידה 3 וליחידה 5 הן גשרים פדגוגיים ולא קביעה שכל אחד מן המושגים הוא דרישת בחינה חדשה במסלול האחר.

3.17 מקורות והכוונה לקריאה נוספת

- מפת תכנית ההתמחות במאגר הקורס: github.com/samiabo75/Data-Science-ML-Course — פרק 3, מבוא למדעי הנתונים ולמידת מכונה (10 שעות).
- Aurélien Géron, Hands-On Machine Learning with Scikit-Learn and PyTorch — גישת העבודה על נתונים, אימון והערכה.
- Luis G. Serrano, Grokking Machine Learning — הסברים אינטואיטיביים והתקדמות מדוגמאות למודלים.
- James et al., An Introduction to Statistical Learning with Applications in Python — הבנת קשר בין נתונים, מודל, דיוק והכללה.
- Google Machine Learning Crash Course: developers.google.com/machine-learning/crash-course
- scikit-learn, Common pitfalls: scikit-learn.org/stable/common_pitfalls.html
- התמונות, הנתונים המדומים, התרגילים והסיפורים בפרק נוצרו עבור תלמידי הקורס; אין בהם העתקה של קטעי ספר או תרשימים מוגנים.

3.18 דף משוב עצמי — מוכנות לקראת פרק 4 (EDA)

הפרק הבא יעמיק בבדיקת איכות נתונים, בסטטיסטיקה ובגרפים. לפני המעבר אליו מומלץ לענות על שאלות הבדיקה שלהלן ללא הסתכלות במחברת. כל שורה שבה אינכם בטוחים היא סימן לנושא שחשוב לחזור עליו. מדובר בכלי ללמידה עצמית, לא בציון.

מצב אישי	מה אני מסוגל להסביר ללא עזרה?	מיומנות
☐ יודע ☐ לתרגל	אני יכול לנסח שאלת חקר שיש לה קלט, יעד ומדד הצלחה.	הגדרת בעיה
☐ יודע ☐ לתרגל	אני יכול לזהות X, y ואת משמעות shape.	תצפיות ומאפיינים
☐ יודע ☐ לתרגל	אני מבחין בין סיווג, רגרסיה וקיבוץ ונותן דוגמה.	סוגי למידה
☐ יודע ☐ לתרגל	אני מסביר את ההפרדה בין Train, Validation ו־Test.	אימון והערכה
☐ יודע ☐ לתרגל	אני יודע לחשב מדד פשוט ולהסביר למה משווים אליו.	קו בסיס
☐ יודע ☐ לתרגל	אני יודע למה צריך לבדוק חסרים, יחידות וחריגים.	איכות נתונים
☐ יודע ☐ לתרגל	אני מסוגל לכתוב תוצאה ומגבלה בלי להבטיח דיוק בעתיד.	פרשנות

משימת כרטיס יציאה: בחרו אחת מן הבעיות שבהן עסקנו, והסבירו בחמש עד שבע שורות מה יידרש כדי להפוך אותה לפרויקט AI מקצועי. בתשובתכם השתמשו לפחות במושגים תצפית, מאפיין, תווית, קו בסיס, אימון ובדיקה. יש לנסח את התשובה במילים שלכם, לא רק להעתיק הגדרה מן הטבלה.

> נקודת המעבר לפרק 4: שם נלמד להפוך את הבדיקה הראשונית של מערך הנתונים לחקר שיטתי — זיהוי שאלות, תיאורים סטטיסטיים, גרפים, איכות נתונים ומסקנות מבוססות. ההחלטה אם לאמן מודל תתקבל רק אחרי שנבין מה הנתונים יכולים לספר לנו.

פרק 4 — חקר נתונים (EDA)

לשאול, לחקור, לבדוק ולנמק: כיצד הופכים טבלת נתונים להבנה שמקדמת מודל AI?

ספר הלימוד המאוחד — חוברת יסוד, פרק רביעי | התמחות למידת מכונה ולמידה עמוקה | 15 שעות לימוד



איור — המסלול החשיבתי בחקר נתונים: מתחילים בשאלה ולא בגרף.

הסיפור: דודו מרכז מעבדת מחשבים בבית הספר. לפני שהוא מפתח מודל שיחזה תקלות, הוא מקבל טבלה עם נתוני מחשבים: גיל, טמפרטורה, מספר תקלות וזמן תיקון. מנהל המעבדה שואל: ״איזה חדר סובל מהתחממות גבוהה יותר, והאם גיל המחשב קשור לתקלות?״

התשובות אינן מצויות בפקודת Python אחת. על דודו לבדוק מה כל שורה מייצגת, אילו ערכים חסרים, האם קיימות מדידות חריגות, איזה גרף מתאים לשאלה ומה ניתן להסיק בלי לחרוג מן הנתונים.

מטרות למידה והידע הנדרש

בסיום הפרק תוכלו: לטעון ולבדוק DataFrame; להסביר תצפית, מאפיין ומשתנה מטרה; לחקור מבנה וטיפוסים; לסנן ולמיין; לזהות חסרים וכפילויות; לחשב מדדי מרכז ופיזור; להשתמש ב־groupby; לשרטט גרפים; לזהות קשרים וחריגים; ולהציג ממצא, מגבלה ושאלה להמשך.

ידע קודם: משתנים, רשימות, פונקציות, יסודות NumPy ומושגי הממוצע והחציון מפרקי היסוד הקודמים. כל קטעי הקוד המודרכים נועדו להרצה ברצף באותה מחברת Colab.

תוצר	מטרת ביצוע	חלוקת 15 שעות מוצעת
כרטיס מאגר וסקירה	שאלה, מבנה נתונים, CSV ו־DataFrame	שעות 1–3
יומן איכות	סינון, מיון, איכות נתונים וחסרים	שעות 4–6
טבלה מנומקת	סטטיסטיקה, groupby והשוואת קבוצות	שעות 7–9
שלוש המחשות ופירוש	גרפים, קשרים וערכים חריגים	שעות 10–12
דוח קצר ומעבדת תלמיד	דוח EDA, מגבלות והכנה ל־ML	שעות 13–15

4.1 מהו EDA ומדוע הוא קודם לאימון מודל?

EDA הוא קיצור של Exploratory Data Analysis — חקר נתונים ראשוני. מטרתו להבין את מבנה המאגר, איכות המדידות, דפוסים וקשרים אפשריים, וגם לגלות שגיאות ושאלות שלא ידענו לשאול לפני שפתחנו את הנתונים. חקר נתונים אינו שלב קוסמטי של ״בניית גרפים יפים״; הוא חלק מתהליך קבלת החלטות מבוסס ראיות.

יש הבדל בין ממצא לבין פירוש. ״בחדר ב׳ ממוצע הטמפרטורה גבוה יותר״ הוא משפט תיאורי שניתן לבדוק בטבלה. ״בחדר ב׳ מערכת המיזוג מקולקלת״ הוא הסבר סיבתי אפשרי, אך לא הוכחה. ייתכנו הבדלים בגיל המחשבים, בשעת המדידה, בתנאי העומס או במד תקול.

שאלת חקר	נתונים שצריך	מה לא ניתן לקבוע לבדו
איזה חדר חם יותר?	טמפרטורה וחדר	מדוע הטמפרטורה שונה
האם גיל קשור לתקלות?	גיל ומספר תקלות	שגיל בהכרח גורם לכל תקלה
כמה מחשבים דורשים טיפול?	הגדרת ״דורש טיפול״ ומדידות	שההגדרה תתאים לעולם האמיתי

תרגול 1 | נסחו שאלה שאפשר לענות עליה מנתוני המעבדה, ושאלה שאי אפשר לענות עליה בלי נתון נוסף. הסבירו מדוע.

4.2 היכרות עם המאגר: תצפית, מאפיין וטבלת DataFrame

במאגר שלנו, כל שורה מתארת מחשב אחד. זו יחידת התצפית. כל עמודה היא תכונה: למשל גיל המחשב בשנים או טמפרטורת עבודה במעלות צלזיוס. זיהוי מחשב (`computer_id`) הוא מזהה ולא בהכרח מאפיין המתאים לאימון מודל. בעת חיזוי זמן תיקון, `repair_hours` עשוי להיות משתנה המטרה; בעת חיזוי תקלה אפשר שהיעד יהיה שדה אחר.

חשוב לשמור על יחידות: 55 מעלות צלזיוס אינן שקולות ל־55 דקות. מחשב מייצג עצם בעולם האמיתי, אך טבלת הנתונים היא ייצוג חלקי שלו, ולא תיאור של כל מה שמשפיע עליו.

computer_id	room	age_years	temp_c	failures
101	A	1	49.0	0
102	A	2	51.0	0
103	A	3	53.0	1
104	A	4	55.0	1
105	A	5	57.0	1
106	A	6	60.0	2

איור — חמש עמודות מתוך המאגר: שורה אחת מתארת מחשב אחד; לעמודות יחידות ומשמעויות שונות.

```
import pandas as pd
import numpy as np

data = {
   "computer_id": list(range(101, 113)),
   "room": ["A"] * 6 + ["B"] * 6,
   "age_years": [1, 2, 3, 4, 5, 6, 2, 4, 3, 7, 8, 1],
   "temp_c": [49, 51, 53, 55, 57, 60, 50, 58, 54, 91, None, 48],
   "failures": [0, 0, 1, 1, 1, 2, 0, 1, 1, 3, 2, 0],
   "repair_hours": [1, 1, 2, 2, 3, 4, 1, 2, 3, 8, 5, 1]
}
df = pd.DataFrame(data)
print(df.shape)
print(df.head())
```

שורת `data` מגדירה מילון: לכל שם עמודה מוצמדת רשימת ערכים באורך זהה. `pd.DataFrame` הופכת את המילון לטבלה. `shape` מחזירה זוג: מספר שורות ומספר עמודות — כאן 12 שורות ו־6 עמודות. `head()` מציגה תצוגה מקדימה; היא אינה משנה את הנתונים.

```
df.to_csv("computers_demo.csv", index=False)
loaded = pd.read_csv("computers_demo.csv")
print(loaded.head(3))
```

קובץ CSV הוא קובץ טקסט פשוט לייצוג טבלה. הכתיבה `index=False` מונעת שמירת אינדקס Pandas כעמודה נוספת. `read_csv` קוראת אותו בחזרה. בקובץ חיצוני חשוב לתעד מקור, תאריך, רישוי והגדרות של כל עמודה; כאן המאגר סינתטי ונוצר ללמידה בלבד.

4.3 סקירה ראשונית: head, info, describe ו־dtypes

לפני בניית גרף, כדאי להתבונן במאגר בכמה רמות. `head` עונה ״מה רואים ברשומות הראשונות?״; `shape` עונה ״מהו גודל הטבלה?״; `dtypes` עונה ״באיזה טיפוס נשמר כל שדה?״; `info` מסכמת טיפוסים ומספר ערכים שאינם חסרים; `describe` מסכמת ערכים מספריים, בדרך כלל בלי למחוק חסרים.

שימו לב: שדה גיל עשוי להיות מסוג `int64`, ואילו שדה טמפרטורה עם `None` ייקלט בדוגמה כסוג מספרי עשרוני (`float64`), משום ש־NaN משתלב בטיפוס זה. טיפוס שנשמר במחשב אינו בהכרח מפרש נכון את משמעות השדה — מיקוד ביקורתי צריך להתייחס גם לשמות וליחידות.

```
print(df.columns.tolist())
print(df.dtypes)
df.info()
print(df.select_dtypes(include="number").describe())
print(df["room"].value_counts())
```

המספר `count` בעמודת `temp_c` ב־`describe` יהיה 11, אף שבמאגר 12 מחשבים. הסיבה היא שקיימת מדידה חסרה אחת. זו הבחנה יסודית: 0 הוא ערך מספרי, אך חוסר מדידה הוא סימון של היעדר מידע.

זהירות	שאלה שהיא עוזרת לברר	פעולה
לא בהכרח מייצגות את כל המאגר	איך נראות חמש השורות הראשונות?	head(5)
הטיפוס לא מבטיח משמעות נכונה	איזה טיפוס וכמה ערכים זמינים?	info()
סיכומים אינם חושפים כל דפוס	ממוצע, חציון, פיזור וטווח	describe()
קטגוריות חסרות עשויות לא להיספר כברירת מחדל	כמה מחשבים בכל חדר?	value_counts()

תרגול 2 | אם יש 12 רשומות ורק 11 טמפרטורות ידועות, האם נכון לומר שהטמפרטורה החסרה היא אפס? נמקו.

4.4 בחירת נתונים, סינון ומיון: לשאול את הטבלה שאלה

השוואת קבוצות דורשת בחירה מודעת של שורות ועמודות. `df["temp_c"]` מחזירה Series, כלומר עמודה אחת; `df[["room", "temp_c"]]` מחזירה DataFrame, כלומר טבלה של שתי עמודות. ההבדל חשוב כאשר פעולות בהמשך מצפות לטבלה.

תנאי בוליאני ב־Pandas מחזיר עבור כל שורה אמת או שקר. בעזרת המסכה בוחרים את הרשומות המתאימות. אם יש כמה תנאים, נכתוב כל תנאי בסוגריים ונשתמש ב־`&` (וגם) או `|` (או) — ולא ב־`and` או `or` הפועלים על ערכי אמת יחידים.

```
temperatures = df["temp_c"]
room_and_temp = df[["room", "temp_c"]]
room_b = df.loc[df["room"] == "B", ["computer_id", "temp_c"]]
needs_review = df.loc[
   (df["temp_c"] > 60) | (df["failures"] >= 3),
   ["computer_id", "temp_c", "failures"]
]
print(needs_review)
print(df.sort_values("failures", ascending=False).head(4))
```

`loc` בוחרת לפי תוויות ושמות עמודות; `iloc` לפי מיקומים מספריים. `sort_values` מסדרת רשומות להצגה ואינה משנה את הטבלה המקורית ללא השמה או שימוש מפורש באפשרות מתאימה. סינון שורות ל״מחשבים שדורשים בדיקה״ הוא כלל פשוט, ולא מודל AI מאומן.

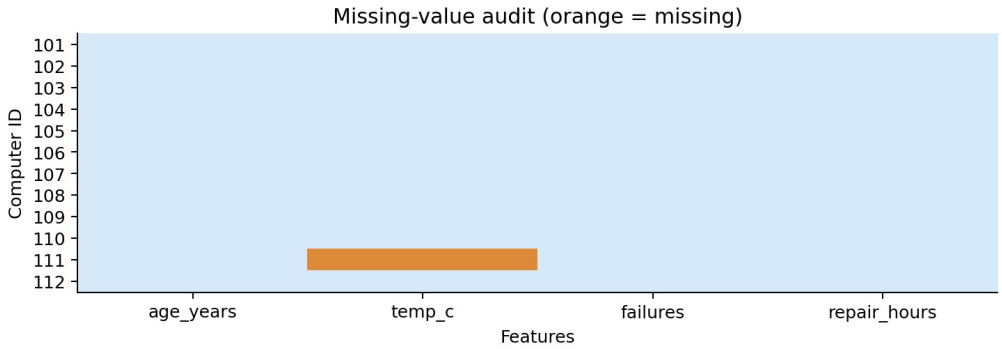
תרגול 3 | בנו מסכה לבחירת מחשבים שגילם ארבע שנים לפחות ושנרשמה אצלם לפחות תקלה אחת. לפני הרצה נסו לחזות כמה שורות יתקבלו.

4.5 איכות נתונים: ערכים חסרים, כפילויות, טווחים ושגיאות מדידה

נתוני העולם אינם תמיד שלמים. חוסר יכול להיווצר ממד שלא עבד, רישום שדולג או בעיית העברה. כפילות עשויה להופיע כאשר אותה רשומה נשמרה פעמיים. ייתכן גם ערך מחוץ לטווח סביר, כגון טמפרטורה גבוהה במיוחד. EDA צריכה להציג את הבעיה ולתעד את ההחלטה, לא לתקן בשקט.

נבדוק קודם את שיעור החסרים בכל עמודה. מאגר קטן בעל חסר אחד עלול לאבד חלק ניכר מהמידע אם נמחק שורות ללא הבחנה. גם מילוי חסרים בממוצע עלול לעוות את הפיזור. צריך לשאול האם החוסר אקראי, קשור לחדר מסוים או נובע מכשל מדידה.

```
missing_counts = df.isna().sum()
missing_percent = 100 * df.isna().mean()
print(pd.DataFrame({"missing": missing_counts,
                    "percent": missing_percent.round(1)}))
print("Exact duplicate rows:", df.duplicated().sum())
print("Duplicate computer IDs:", df.duplicated(subset=["computer_id"]).sum())
```



איור — כתום מסמן מדידה חסרה. גילוי מקום החוסר חשוב יותר ממחיקה אוטומטית.

הפקודה `duplicated()` בודקת שורות זהות, ואילו `duplicated(subset=["computer_id"])` בודקת חזרה על מזהה מחשב. בשני המקרים נקבל אפס בדוגמה, אך אם יופיע אותו מזהה פעמיים, נצטרך לבדוק האם מדובר בכפילות או בשתי מדידות בזמנים שונים. במקרה האחרון יש להוסיף זמן מדידה למפתח המתאר את הרשומה.

```
print(df.loc[df["temp_c"].isna(), ["computer_id", "room"]])
print(df.loc[df["temp_c"] > 75, ["computer_id", "temp_c"]])
clean_view = df.dropna(subset=["temp_c"]).copy()
print("Rows with known temperature:", len(clean_view))
```

בדוגמה נשמרות בטבלה המקורית 12 הרשומות. `clean_view` הוא עותק זמני המיועד לניתוח טמפרטורות בלבד, הכולל 11 רשומות. אין בכך הצדקה למחוק את המחשב בעל המדידה החסרה מכל יתר הניתוחים. אי אפשר ללמוד מטמפרטורה חסרה על זמן התיקון או על גיל המחשב.

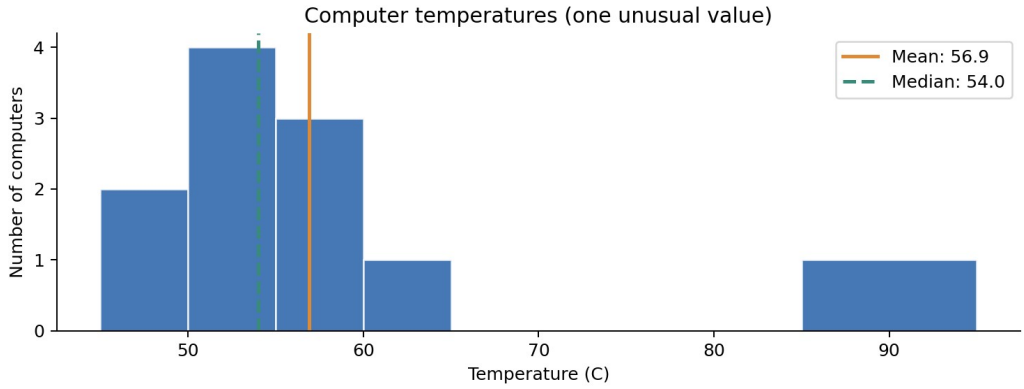
תרגול 4 | הציעו שתי אפשרויות לטיפול בטמפרטורה החסרה וציינו יתרון ומגבלה של כל אחת. החליטו מה תעשו בדוח **EDA**, בלי לפגוע במקור.

4.6 מדדי מרכז ופיזור: מה מסתתר מאחורי הממוצע?

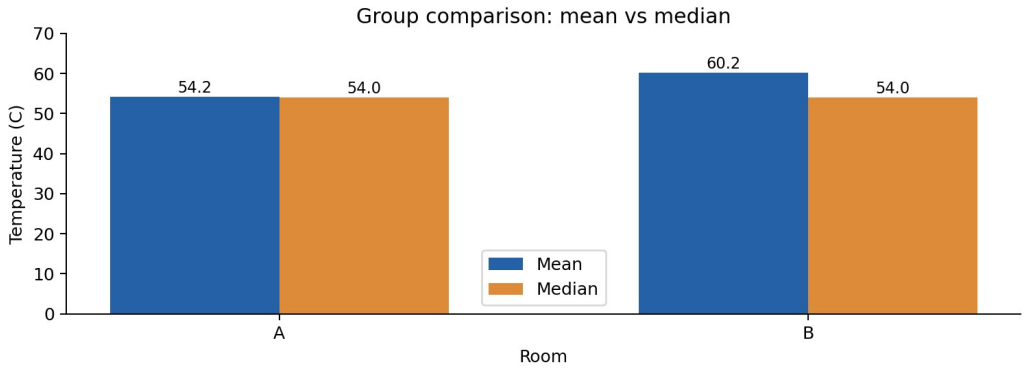
נחשב את טמפרטורת העבודה הממוצעת של החדרים. בחדר A הנתונים הם 49, 51, 53, 55, 57, 60. סכומם 325 ולכן הממוצע הוא 325 חלקי 6, כלומר כ־54.17 מעלות. בחדר B ידועות חמש מדידות בלבד: 50, 58, 54, 91, 48. סכומן 301 ולכן הממוצע הוא 301 חלקי 5, כלומר 60.2 מעלות.

אם מסתפקים בממוצע בלבד, עלולים לקבוע שחדר B חם בהרבה. אולם יש בו מדידה חריגה של 91 מעלות. החציון של חמש המדידות בחדר B הוא 54 מעלות — נמוך מהממוצע. לכן יש להציג גם חציון, התפלגות והחריג, ולשאול אם המדידה נכונה.

```
summary = df.groupby("room")["temp_c"].agg(
   count="count", mean="mean", median="median",
   std="std", minimum="min", maximum="max"
)
print(summary.round(2))
```



איור — היסטוגרמה של הטמפרטורות ומיקומי הממוצע והחציון. הערך החריג משפיע על הממוצע.



איור — ממוצע מול חציון לפי חדר. תמיד יש לבדוק מה מסתתר מאחורי ההבדל.

סטיית תקן מתארת פיזור סביב הממוצע. כברירת מחדל, `Series.std()` של Pandas מחשבת סטיית תקן מדגמית באמצעות חלוקה ב־n−1; לעומת זאת `np.std` כברירת מחדל מחלקת ב־n. אין להציג את שתי התוצאות כטעות — צריך לבחור הגדרה מתאימה ולציין אותה.

רבעונים וטווח בין־רבעוני (IQR) מסייעים לבדוק ערכים חריגים. ב־Pandas הפונקציה `quantile` משתמשת כברירת מחדל באינטרפולציה ליניארית; שיטות חלוקה ידניות אחרות עשויות לתת ערכי רבעונים מעט שונים במדגם קטן. חשוב להסביר מהי השיטה שנבחרה, ולא לערבב שיטות באמצע פתרון.

```
known = df["temp_c"].dropna()
q1 = known.quantile(0.25)
q3 = known.quantile(0.75)
iqr = q3 - q1
upper_fence = q3 + 1.5 * iqr
print(f"Q1={q1}, Q3={q3}, IQR={iqr}, upper={upper_fence}")
print(known[known > upper_fence])
```

בדוגמה זו Pandas מחזירה בקירוב Q1=50.5, Q3=57.5 וגבול עליון 68. לכן הערך 91 מסומן כחריג לפי כלל IQR×1.5. סימון חריג הוא אות לחקירה נוספת — לא הוכחה שהמדידה שגויה ולא הוראה למחוק אותה.

תרגול 5 | הסבירו מדוע לחדר **B** ממוצע גבוה יותר אף שהחציון בו דומה לזה של חדר **A**. איזו בדיקה הייתם עורכים לפני כתיבת המלצה להנהלת בית הספר?

4.7 GroupBy: להשוות בין קבוצות בלי לאבד את ההקשר

פעולת `groupby` מחלקת את הרשומות לפי ערך משותף, מפעילה פעולה על כל קבוצה ומחזירה סיכום. הרעיון נקרא לעיתים Split–Apply–Combine: מחלקים, מחשבים, מחברים את התוצאות. למשל, לכל חדר נחשב את מספר המחשבים, הממוצע של מספר התקלות וסך שעות התיקון.

חשוב להבחין בין `size`, הסופרת את כל השורות בקבוצה, לבין `count`, הסופרת רק ערכים שאינם חסרים בעמודה מסוימת. במקרה שלנו בחדר B יש שישה מחשבים, אך רק חמש מדידות טמפרטורה זמינות.

```
room_report = df.groupby("room").agg(
    computers=("computer_id", "size"),
    known_temperatures=("temp_c", "count"),
    average_failures=("failures", "mean"),
    total_repair_hours=("repair_hours", "sum")
)
print(room_report.round(2))
```

נניח שחדר A מכיל 6 מחשבים וחדר B גם 6. אם נגלה שבחדר B סך שעות התיקון גבוה יותר, נצטרך לבדוק האם התקלות קשות יותר, המחשבים ישנים יותר, או שתועדו אחרת. טבלת קיבוץ מספקת תיאור, לא תשובה סיבתית. בניתוח אמיתי נרצה להוסיף תאריכים, שעות שימוש ומאפיינים נוספים.

תרגול 6 | האם מספר המחשבים בחדר בהכרח שווה למספר טמפרטורות ידועות? תנו דוגמה מתוך הנתונים והסבירו את משמעותה.

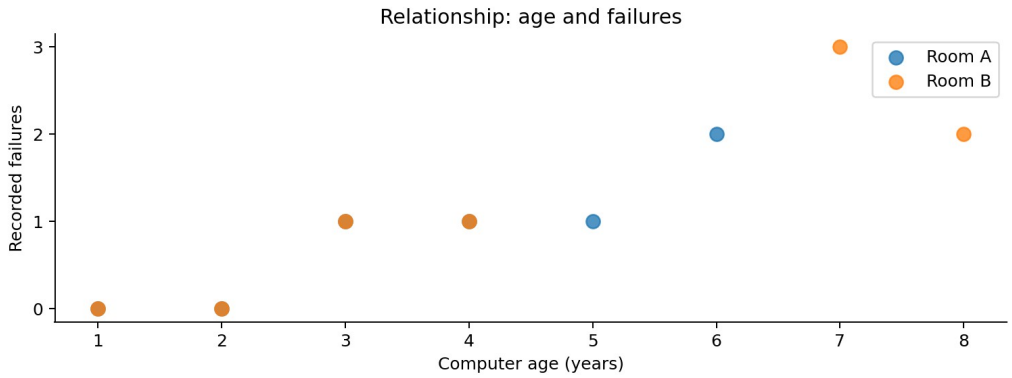
4.8 בחירת גרף לפי השאלה ולא לפי ההרגל

תרשים עמודות מתאים להשוואת קטגוריות — למשל מספר מחשבים בכל חדר. היסטוגרמה מתארת התפלגות של משתנה מספרי, כגון טמפרטורה. תרשים פיזור מתאים לשני משתנים מספריים, כגון גיל ומספר תקלות. תרשים קופסה מסכם חציון, רבעונים וערכים חריגים אפשריים.

לפני גרף נשאל: איזה משתנה מוצג בכל ציר? מה היחידות? האם יש חסרים? האם גודל המדגם מספיק? האם הצירים התחילו בנקודה שאינה אפס באופן שמגזים את ההבדל? מהי השאלה שהגרף אמור לעזור לענות עליה?

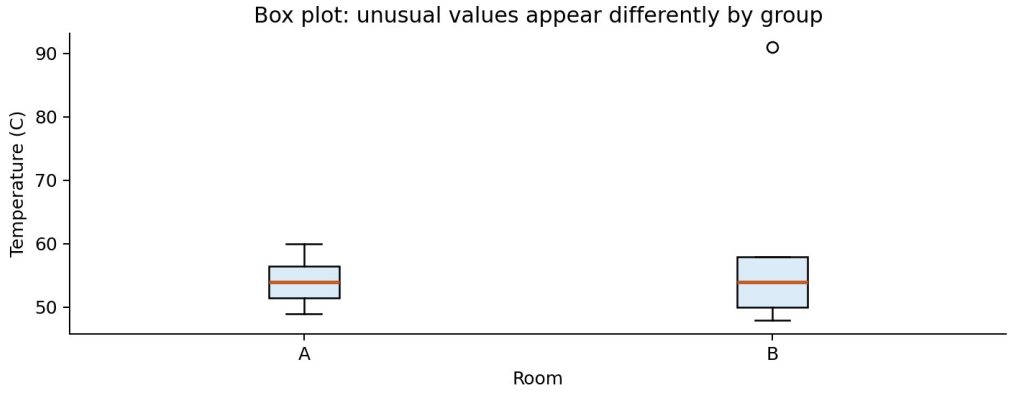
```
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(6, 4))
ax.scatter(df["age_years"], df["failures"])
ax.set_xlabel("Computer age (years)")
ax.set_ylabel("Recorded failures")
ax.set_title("Age and failures")
plt.show()
```



איור — תרשים פיזור: קשר אפשרי בין גיל לתקלות; אין להסיק סיבתיות מתרשים.

```
fig, ax = plt.subplots(figsize=(6, 4))
room_a = df.loc[df["room"] == "A", "temp_c"].dropna()
room_b = df.loc[df["room"] == "B", "temp_c"].dropna()
ax.boxplot([room_a, room_b], labels=["A", "B"])
ax.set_ylabel("Temperature (C)")
ax.set_title("Temperatures by room")
plt.show()
```



איור — תרשים קופסה מציג את מאפייני ההתפלגות גם כאשר הממוצע מטעה.

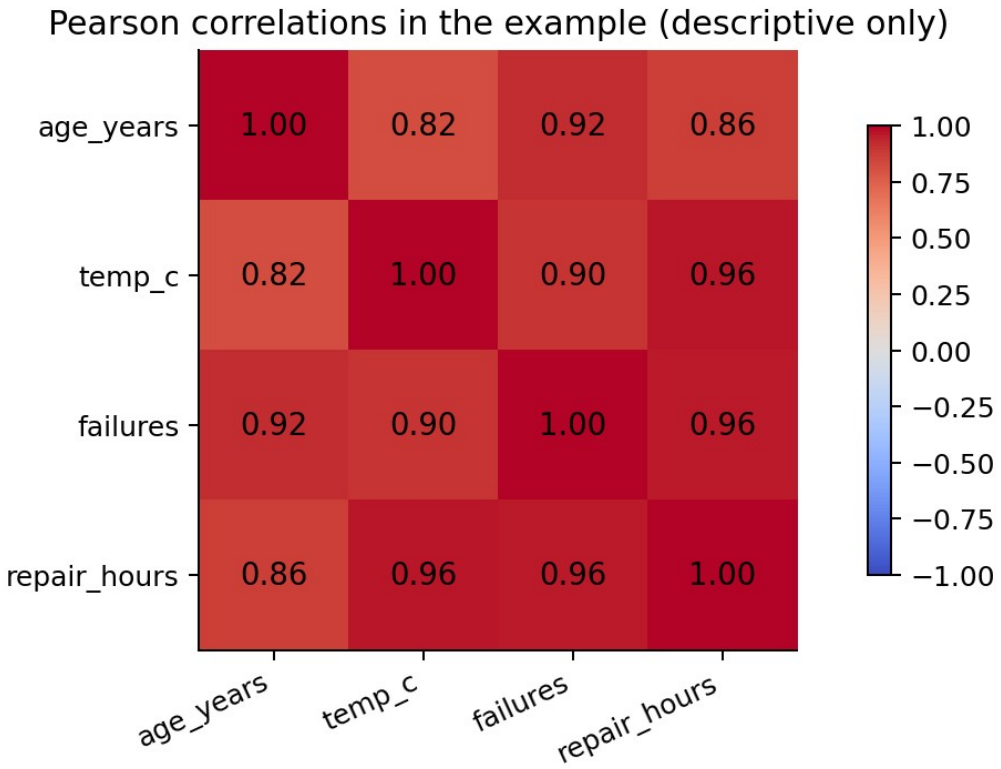
מטרת המחקר	הגרף המתאים	למה?
השוואת מספר מחשבים לפי חדר	עמודות	קטגוריות ומניינים
התפלגות הטמפרטורות	היסטוגרמה	התפלגות של משתנה מספרי
קשר גיל–תקלות	פיזור	שני משתנים מספריים
חריגים לפי חדר	קופסה	חציון, רבעונים וחריגים

4.9 קשר בין משתנים: מתאם אינו הסבר סיבתי

מתאם פירסון מודד עוצמה וכיוון של קשר ליניארי בין שני משתנים כמותיים, כאשר הוא מוגדר. ערכו בין מינוס אחת לאחת. ערך גבוה וחיובי מצביע על קשר ליניארי חיובי במדגם, אך אינו מוכיח שמשתנה אחד גורם לשינוי בשני. קשר יכול לנבוע ממשתנה שלישי, ממבנה הדגימה או מצירוף נסיבות.

במאגר שלנו ייתכן קשר חיובי בין מספר התקלות לשעות התיקון, בין השאר משום ששני המשתנים מתארים היבטים של בעיות במחשבים. עדיין ייתכן שזמן התיקון מושפע מחלקי חילוף, זמינות טכנאי ומועד התקלה — נתונים שאינם נמצאים בטבלה.

```
numeric = df[["age_years", "temp_c", "failures", "repair_hours"]]
correlations = numeric.corr(numeric_only=True)
print(correlations.round(2))
# The correlation matrix is descriptive, not a causal model.
```



איור — מפת חום של מתאמי פירסון מתוך הנתונים הסינתטיים בלבד.

קיימת מגבלה נוספת: ערכי המתאם מחושבים על זוגות זמינים של מדידות, ולכן מספר הרשומות המעורבות בכל תא במטריצה עשוי להשתנות כשיש חסרים. כמו כן, ערך חריג יכול להשפיע מאוד על פירסון; במידת הצורך יש לבדוק גם תרשים פיזור ומדד חלופי כגון Spearman. איננו מפעילים חישוב נוסף רק כדי לקבל מספר גבוה — מטרתנו להבין את השאלה.

תרגול 7 | כתבו שני משפטים: משפט אחד שמתאר ממצא שיכול להתקבל מתרשים פיזור, ומשפט אחד שמציג הסבר אפשרי שעדיין טעון בדיקה.

4.10 מה־EDA למודל AI: הבחירה והבדיקה חייבות להיות הוגנות

אם EDA מיועדת לתיאור מאגר בלבד, אפשר לסקור את הנתונים במלואם — בתנאי שלא נטען בהמשך שהערכת המודל עליהם בלתי תלויה. אך כאשר חקר הנתונים משמש לבחירת מאפיינים, זיהוי משתנים חשובים, קביעת ספי ניקוי או התאמת טרנספורמציות למודל, יש לשמור מראש קבוצת בדיקה עצמאית.

לכן בפרויקט למידת מכונה נפריד תחילה בין נתוני אימון לבדיקה (לפי מבנה הבעיה: חלוקה אקראית, לפי זמן או לפי קבוצות). החלטות שמכוונות את המודל נלמדות מנתוני האימון בלבד, או מתהליך ולידציה פנימי. לעולם אין להשתמש בקבוצת Test שוב ושוב כדי לבחור מודל ואז לדווח על הציון כאילו זו הערכה עצמאית.



איור — נתוני בדיקה נפרדים אינם משמשים לבחירת מאפיינים או להתאמת שלבי עיבוד.

```
from sklearn.model_selection import train_test_split

train_df, test_df = train_test_split(
    df, test_size=0.25, random_state=42
)
# Exploratory choices for modeling are made on train_df.
print("Train:", len(train_df), "Test:", len(test_df))
```

החלוקה כאן נועדה להמחיש רעיון בלבד: המאגר מכיל 12 מחשבים סינתטיים, מעט מדי כדי להסיק על מודל אמיתי. בפרויקט עם נתוני זמן או כמה מדידות לאותו מחשב נצטרך חלוקה מתאימה, המונעת מעבר מידע בין אימון לבדיקה. למשל, שתי מדידות של אותו מחשב עלולות להיות תלויות זו בזו, ולכן חלוקה אקראית של שורות אינה תמיד מספיקה.

תרגול 8 | מדוע אסור לחשב ממוצע לצורך מילוי חסרים על כל נתוני הפרויקט ואז להפריד לאימון ולבדיקה? הסבירו במונחי מידע שהיה זמין בזמן האימון.

4.11 חקר מקרה מסכם — דוח EDA שלא מסתפק בהדפסת גרפים

דודו נדרש להגיש להנהלת בית הספר דוח קצר. הוא לא כותב ״חדר B חם יותר ולכן צריך להחליף מזגן״. במקום זאת הוא מתאר את מקור הנתונים, טבלת המאפיינים, היקף החסרים, המדדים שנבחרו והחשש שמדידה אחת משפיעה על הממוצע. הוא מציג כמה הסברים אפשריים ומציע מדידות נוספות.

דוח מקצועי כולל לפחות ארבעה מרכיבים: מה בדקנו; מה ראינו בפועל; מה מגביל את הפרשנות; ומה נדרש כדי לקבל החלטה טובה יותר. כמו כן יש לשמור קוד שניתן להרצה מחדש, לציין אילו שורות נכללו בחישובים ולשמור עותק של נתוני המקור.

מרכיב בדוח	דוגמה מנוסחת
שאלת המחקר	האם התפלגות הטמפרטורות שונה בין חדרים A ו־B?
ממצא	הממוצע בחדר B הוא 60.2 מעלות, לעומת כ־54.17 בחדר A.
מגבלה	בחדר B חסרה מדידה אחת וקיימת מדידה גבוהה במיוחד של 91.
המשך בדיקה	בדיקת חיישן, שעות שימוש ורישום מדידות חוזרות לפני קביעה סיבתית.

כאשר עובדים עם תלמידים, נתוני בריאות או ציונים, חשוב במיוחד לשקול פרטיות, הוגנות והשפעה על אנשים. דוגמאות הספר סינתטיות; בעבודה עם מידע אמיתי יש לפעול לפי מדיניות בית הספר, רישיונות השימוש והוראות החוק הרלוונטיות.

4.12 מטלה מסכמת לתלמיד — חוקר נתונים עצמאי

במחברת התרגול המצורפת תעבדו על מאגר שונה מן הדוגמה שבספר: נתוני השאלת ציוד בית־ספרי שנוצרו לצורכי לימוד. תגדירו שאלת חקר, תבדקו את מבנה הטבלה ואת איכות הנתונים, תבצעו לפחות שני סיכומי קבוצות, תיצרו שלושה גרפים ותכתבו מסקנה מסויגת. קטעי הקוד החסרים במחברת הם מטלות לביצוע — לא בעיה בקובץ.

תרגול 9 | הגישו דוח קצר ובו כרטיס מאגר, שתי שאלות חקר, טבלת איכות נתונים, השוואת שתי קבוצות, שלושה גרפים עם פרשנות, מסקנה מוגבלת והמלצה להמשך איסוף נתונים.

4.13 ממוצע של ממוצעים: מדוע חשוב לדעת מה גודל כל קבוצה?

נניח שמנהלת המעבדה משווה בין שתי קבוצות תלמידים. בקבוצה הראשונה יש שני מחשבים, שממוצע זמן התיקון שלהם הוא 4 שעות. בקבוצה השנייה יש שמונה מחשבים, שממוצע זמן התיקון שלהם הוא 2 שעות. אם נחשב (4+2)/2 נקבל 3 שעות — אבל זהו ממוצע הממוצעים של הקבוצות, המייחס משקל שווה לשתי קבוצות בגדלים שונים. הוא אינו ממוצע זמן התיקון של כל עשרת המחשבים.

כדי לחשב את הממוצע המשוקלל לפי מספר הרשומות, נכפיל את ממוצע הקבוצה במספר המחשבים שבה: בקבוצה הראשונה סך השעות הוא 2×4=8; בשנייה 8×2=16. ביחד 24 שעות לעשרה מחשבים, ולכן ממוצע כלל המחשבים הוא 24/10=2.4 שעות.

כלומר, כשמחברים נתונים מקבוצות שונות יש לשאול באיזה מדד משתמשים, ומהו המשקל הראוי לכל קבוצה. ממוצע הממוצעים יכול להיות מוצדק אם השאלה היא ״מהו ממוצע הביצועים בין חדרים כשכל חדר מקבל משקל שווה?״ — אך אינו תשובה לשאלה ״מהו ממוצע כלל המחשבים?״.

קבוצה	מספר מחשבים	ממוצע שעות למחשב	סך שעות
חדר 1	2	4	8
חדר 2	8	2	16
ביחד	10	2.4 (משוקלל)	24

```
small_groups = pd.DataFrame({
    "group": ["A", "B"],
    "computers": [2, 8],
    "mean_hours": [4.0, 2.0]
})
weighted_mean = (
    (small_groups["computers"] * small_groups["mean_hours"]).sum()
    / small_groups["computers"].sum()
)
print(weighted_mean)  # 2.4 hours
```

תרגול 10 | כתבו דוגמה שבה ממוצע פשוט של ממוצעי שתי קבוצות יהיה נמוך מהממוצע המשוקלל. הסבירו איזו שאלה כל חישוב עונה עליה.

4.14 טיפול בחסרים: לא כל חסר דורש את אותו הפתרון

יש כמה דרכים אפשריות לטיפול בחסר: להשאיר אותו כחסר בזמן התיאור; לא לכלול את הרשומה בחישוב המתייחס לעמודה החסרה; למלא ערך לפי שיטת השלמה שהוגדרה מראש; או לחזור למקור הנתונים כדי לתקן את האיסוף. אין פתרון אחד שנכון לכל מאגר.

אם חסרות טמפרטורות דווקא כאשר המחשבים חמים מאוד והחיישן קורס, מחיקה של אותן מדידות עלולה לגרום להערכת חסר של הטמפרטורה הממוצעת. אם החסרים קשורים לחדר, לשעה או לגיל המחשב, גם

ההשוואה בין קבוצות עשויה להשתנות. לכן יש לבדוק היכן מופיעים החסרים ולשקול את מנגנון ההיעדרות, ולא רק לספור אותם.

בתהליך פיתוח מודל חשוב להבחין בין סיכום תיאורי לבין השלמת נתונים לצורך חיזוי. למשל, השלמה בחציון שנלמד מנתוני האימון יכולה להיות חלק מ־Pipeline; את אותו חציון מפעילים על קבוצת הבדיקה בלי לחשב אותו מחדש. חציון שחושב גם על הבדיקה חושף מידע שלא אמור להשפיע על ההחלטות בזמן אימון.

```
# Descriptive comparison only: keep the original intact.
analysis_a = df.copy()
analysis_b = df.copy()
analysis_b["temp_c"] = analysis_b["temp_c"].fillna(
   analysis_b["temp_c"].median()
)
print("Original known temperatures:", df["temp_c"].count())
print("Filled series count:", analysis_b["temp_c"].count())
# Do not use full-data imputation when preparing an independent ML test.
```

ההדגמה האחרונה ממלאת את המדידה החסרה לצורך השוואה לימודית בלבד, ולא לצורך הערכה של מודל. שימו לב שהעמודה החדשה כוללת ערך שהוסק מן הנתונים הקיימים, ולא מדידה אמיתית. לכן יש לסמן בתיעוד מה הושלם, מה נמדד בפועל ובאיזו שיטה הושלם.

4.15 ייצוג נתונים וגרפים: לזהות גם הצגה מטעה

כאשר מנהלת רואה תרשים עמודות שבו ערכי הטמפרטורה מתחילים בציר האנכי ב־50 מעלות, הבדל של חמש מעלות עשוי להיראות כמו פי שניים. לעיתים חיתוך ציר לגיטימי לצורך הצגת שינוי קטן, אך הוא מחייב סימון ברור של הטווח. בתרשימי עמודות של כמויות עדיף בדרך כלל להתחיל מאפס; בתרשימי קו המתארים מדידה לאורך זמן ייתכן שטווח מצומצם מועיל, בתנאי שמציגים אותו במפורש.

חשוב גם להבחין בין מספרים מוחלטים לאחוזים. בכיתה אחת היו שתי תקלות מתוך עשרה מחשבים ובכיתה אחרת שלוש תקלות מתוך חמישים. מספר התקלות גדול יותר בשנייה, אך שיעור התקלות דווקא נמוך יותר. כשעובדים עם קבוצות בגדלים שונים יש להציג לפעמים גם מונה וגם מכנה.

קבוצה	תקלות	מספר מחשבים	שיעור תקלות
קבוצה א׳	2	10	20%
קבוצה ב׳	3	50	6%

הטבלה מדגימה עיקרון של שיעורים: היחס 2/10 גדול מהיחס 3/50. אבל גם שיעור אינו בהכרח השוואה הוגנת אם משכי שימוש שונים, אם למחשבים בוצעו טיפולים שונים או אם אופן רישום התקלה אינו אחיד. השוואה אמינה דורשת הגדרות שוות ובדיקה של גורמים מערפלים.

תרגול 11 | מנהל אמר: ״בחדר ב׳ היו יותר תקלות, ולכן הוא מתוחזק פחות טוב״. הסבירו מה חסר בטענה, ואילו שני מדדים נוספים כדאי לבקש.

4.16 מילון נתונים (Data Dictionary): לדעת מה פירוש כל עמודה

טבלה בלי מילון שדות דומה למפת דרכים בלי שמות הרחובות. לפני חקר הנתונים יש להבין מי יצר את השדה, מה היחידות שלו, אילו ערכים חוקיים ומה משמעות החסר. לדוגמה, `age_years` מתאר את גיל המחשב בשנים; אם אחד המחשבים חדש וחלקי שנה עוגלו למטה, גיל 0 אינו בהכרח טעות.

ננסח מילון נתונים בסיסי למאגר התרגול, ונשמור אותו לצד ה־CSV והמחברת. אם בהמשך נשנה את סדר העמודות או ניצור עמודה חדשה, נעדכן גם את המילון. תיעוד כזה מצמצם שגיאות כאשר כמה תלמידים עובדים יחד על אותו פרויקט.

שדה	טיפוס ומשמעות	יחידה או תחום	אזהרת פירוש

computer_id	מזהה ייחודי למחשב	מזהה, לא מדידה	מספר גדול אינו מחשב חזק יותר
room	מיקום פיזי	A או B	קטגוריה, לא סדר מספרי
age_years	גיל המחשב	שנים	ייתכן עיגול
temp_c	מדידת חום	מעלות צלזיוס	חסר אחד; מדידה 91 דורשת בדיקה
failures	מספר תקלות מתועדות	ספירה	תלוי במשך זמן המעקב
repair_hours	סך שעות תיקון	שעות	עשוי להיות יעד של מודל חיזוי

```
data_dictionary = pd.DataFrame([
    ["computer_id", "identifier", "not a numeric feature"],
    ["room", "category", "A or B"],
    ["age_years", "numeric", "years"],
    ["temp_c", "numeric", "degrees Celsius"],
    ["failures", "count", "recorded cases"],
    ["repair_hours", "numeric", "hours"]
], columns=["field", "kind", "description"])
data_dictionary.to_csv("computers_dictionary.csv", index=False)
```

4.17 כיצד כותבים דוח EDA כראיה ולא כטענה כללית?

דוח טוב אינו אוסף של תרשימים. לכל תרשים צריכים להיות כותרת ותיאור שאלה, צירים ויחידות, ממצא מדויק, פירוש מוגבל והצעה להמשך. אפשר להשתמש במבנה הבא: ״בדקנו ... באמצעות ...; ראינו כי ...; יש להביא בחשבון ש ...; לפני הסקת מסקנה על ... נדרשת בדיקה של ...״.

למשל: ״בדקנו את הטמפרטורה בחדרים A ו־B בעזרת ממוצע, חציון ותרשים קופסה. ממוצע חדר B היה 60.2 מעלות לעומת כ־54.17 בחדר A, אך בחדר B נרשמה מדידה אחת של 91 מעלות ומדידה חסרה נוספת. לכן נדרש בירור מקור המדידות והרחבת תקופת האיסוף לפני קביעה שחדר אחד חם יותר באופן עקבי״.

ממצא אינו חייב לתמוך בהשערה המקורית. אם הגרף אינו מראה קשר ברור, מסקנה מתאימה היא ״בנתונים הקיימים לא מצאנו עדות מספקת לקשר ברור״, בתוספת מגבלות הדגימה. אין לשנות מאפיינים או מדדים שוב ושוב עד שמתקבלת תוצאה ״יפה״ ואז להציג זאת כניסוי בלתי תלוי.

תרגול 12 | נסחו פסקת ממצא חדשה מתוך אחד הגרפים בחוברת. פסקתכם צריכה לכלול נתון מספרי, פירוש, מגבלה ופעולת המשך אפשרית.

שאלות הבנה והעברה — שלוש רמות

משימה	רמה
פרשו את ההבדל בין 12 תצפיות ל־11 טמפרטורות ידועות. מהן היחידות?	א — בסיס
הבחינו בין Series ל־DataFrame ובין loc ל־iloc.	א — בסיס
חשבו ביד את הממוצע בחדר B והסבירו את	ב — חישוב

משימה	רמה
השפעת 91.	
בחרו גרף מתאים להשוואת התפלגויות ונמקו.	ב — ניתוח
תלמיד מחק את כל השורות החסרות. אילו הטיות עלולות להיווצר?	ג — ביקורת
כיצד הייתם מבצעים EDA בלי לחשוף קבוצת Test לבחירת מאפיינים?	ג — מחקר

בדיקת מוכנות לפרויקט גמר

לפני המעבר לפרק הרגרסיה הליניארית התלמיד צריך להראות שהוא יודע להגדיר משתנה מטרה, להסביר מהו מקור הנתונים, להפעיל בדיקות חסרים וכפילויות, לחשב סיכומים ו־groupby, לבחור גרף לפי סוג משתנה, לפרש מתאם בזהירות ולשמור על הערכת Test עצמאית. קוד שרץ ללא הסבר אינו מספיק.

מפת הקשרים בין התכניות

שימוש בהמשך	מוקד ההוראה	מסלול
שאלת חקר ותלקיט אישי	היכרות עם DataFrame, גרפים, סיכומים ומסקנות	יחידה 3 — מדעי הנתונים
העמקה מתמטית ופירוש	מדדי מרכז ופיזור, יחסים בין משתנים, קריאת גרפים	יחידה 5 — נושאי ליבה
בחירת מאפיינים, ניסויים ופרויקט גמר	תהליך EDA עצמאי לקראת פיתוח מודל	התמחות למידה עמוקה — פרק 4

מילון מושגים

משמעות	מושג
אוסף תצפיות ומאפיינים שנאספו או נוצרו למטרה מסוימת	Dataset
יחידת תצפית — במקרה שלנו מחשב	Observation
מאפיין המתאר תצפית	Feature
משתנה שאנחנו מנסים לחזות במודל מונחה	Target
טבלה דו־ממדית ב־Pandas	DataFrame
עמודה או סדרת ערכים עם אינדקס	Series
ערך חסר, שאינו זהה לאפס	Missing value
חלוקת רשומות לקבוצות וחישוב סיכומים	GroupBy
תצפית חריגה ביחס להתפלגות או לכלל שנבחר	Outlier
שימוש לא תקין במידע שלא אמור להיות זמין בזמן בניית המודל	Data Leakage

מקורות והערות עריכה

עוגן תכנית: מפת התמחות למידת מכונה ולמידה עמוקה במאגר GitHub של הקורס, פרק 4 — חקר נתונים, 15 שעות. חלוקת השעות, הדוגמאות והעומק המפורט הם הצעה דידקטית שלנו; יש לאמת כל חובת בחינה מול הנוסח הרשמי המעודכן.

חומרי הוראה משלימים במאגר: סיכום ההשתלמות על Pandas, Iris ו־Penguins, והנחיות המורה למפגש CSV. מקורות מקצועיים: An Introduction to Statistical Learning with Applications in Python; Hands-On Machine Learning; Pandas User Guide; scikit-learn — Common Pitfalls. הדוגמאות והאיורים בחוברת מקוריים; אין הם העתקה מספרי המקור.

תיעוד מקצועי: https://pandas.pydata.org/docs/user_guide/ ; https://pandas.pydata.org/docs/user_guide/groupby.html ; https://scikit-learn.org/stable/common_pitfalls.html

דף עזר — איזו פקודה מתאימה לשאלה שלי?

שאלה	פקודת **Pandas** מומלצת
כמה רשומות ועמודות יש?	`df.shape`
אילו טיפוסים וחסרים יש?	`df.info(); df.isna().sum()`
מהי שכיחות קטגוריה?	`df["room"].value_counts()`
איך בוחרים שורות ועמודות?	`df.loc[mask, ["col1", "col2"]]`
איך מסכמים לפי חדר?	`df.groupby("room").agg(...)`
איך מוצאים חציון או רבעון?	`df["temp_c"].median(); .quantile(0.25)`
מה הקשר הליניארי בין מאפיינים?	`df[["x", "y"]].corr()`
כיצד שומרים תוצאות בלי אינדקס?	`df.to_csv("file.csv", index=False)`

זכרו: בחירת הפקודה היא רק התחלה. בכל משימת חקר יש להסביר גם את משמעות הפלט, את היחידות ואת מגבלת המסקנה.

מדעי הבינה המלאכותית

מנתונים למודלים ולרשתות עצביות

פרק 1.1

וקטורים — השפה המספרית של הנתונים

איך מתארים תלמיד, מוצר או תמונה כאוסף מסודר של מספרים — ולמה זה חשוב ללמידת מכונה?

היקף הפרק	שייכות בתכנית	קשר לפרקים הבאים
כשמונה עד עשר שעות כהצעה דידקטית	יחידה 5 — פרק 1, סעיף 1.1 (לפי מפת הסילבוס הקיימת)	מרחק ודמיון, רגרסיה, פרספטרון, רשת עצבית

מהדורת תלמיד מורחבת. כל הדוגמאות, הטבלאות, האיורים, הקוד ותרגילי הפרק נכתבו מחדש ומותאמים לתלמידי תיכון.

הערת התאמה: מיקום הסעיף נקבע לפי מפת תכנית הליבה הקיימת במאגר. התאמה מלאה לגרסה העדכנית ולמחוון הבחינה תאושר רק אחרי הצלבה של נוסח המסמכים הרשמיים. נושאי העשרה מסומנים במפורש.

מפת הלמידה בפרק

נלמד רעיון אחד בכל פעם, נבדוק אותו בחישוב ידני, ורק לאחר מכן נבצע אותו במחשב.

חלק	מה נלמד	מה נוכל לעשות
1	למה בכלל צריכים וקטורים?	להמיר תיאור מילולי לנתונים מספריים
2	רכיבים, ממד וסדר	לקרוא ולכתוב וקטור במדויק
3	ממערך תכונות למטריצה	לזהות דוגמאות ומאפיינים
4	ייצוג גיאומטרי	להבין נקודה, חץ וכיוון
5	חיבור, חיסור וכפל בסקלר	לחשב ולהסביר מה השתנה
6	נורמות $L1$ ו־$L2$	למדוד גודל ולפרש את משמעותו
7	וקטור יחידה	להבחין בין גודל לבין כיוון
8	מכפלה פנימית	לחשב סכום משוקלל של מאפיינים
9	NumPy ומודל ליניארי	לממש את המתמטיקה בקוד
10	שגיאות, ניסויים ותרגול	לבדוק הבנה ולהעביר ידע למקרה חדש

מה כבר צריך לדעת?

אין צורך בידע מוקדם בלמידה עמוקה. יספיק להכיר פעולות חשבון, חזקות ושורש ריבועי, זוגות סדורים במערכת צירים, משתנים ורשימות בסיסיות ב־Python. כאשר נזדקק לכלי חדש, נסביר אותו לפני השימוש בו.

מטרות למידה — בסיום הפרק התלמיד יוכל

- לזהות מאפיין (Feature) של דוגמה ולבחור סדר רכיבים קבוע.
- להגדיר וקטור ב־$\mathbb{R}^n$ ולהבחין בין וקטור, סקלר, נקודה ומטריצה.
- לבצע חיבור, חיסור וכפל בסקלר ולפרש את התוצאה בהקשר של נתונים.
- לחשב נורמת $L1$ ונורמת $L2$, ולהסביר את ההבדל ביניהן.
- לנרמל וקטור לא־אפס לאורך 1 ולהסביר מתי הפעולה אינה מוגדרת.
- לחשב מכפלה פנימית ולהסביר מדוע היא שימושית במודל עם משקולות.
- לממש כל פעולה באמצעות NumPy תוך השוואה לתוצאה הידנית.
- לזהות טעות בתיאור התכונות, בממדים, ביחידות המדידה או בקוד.

- להסביר כיצד הרעיונות בפרק מתחברים ל־KNN, לרגרסיה ולנוירון.

1. סיפור הפתיחה: איך מחשב מכיר את דודו?

דודו בונה אפליקציה שממליצה לתלמידים על חוגים וסרטונים לימודיים. כאשר תלמיד נכנס למערכת, האפליקציה אינה יכולה להסתפק במשפט כללי כגון: "התלמיד אוהב מדע ומתעניין גם בספורט". כדי לבצע חישובים, להשוות בין תלמידים וללמוד דפוסים, נרצה לתאר אותם באמצעות מאפיינים מספריים בעלי משמעות ברורה.

לצורך הדוגמה, דודו בוחר שני מאפיינים שאפשר למדוד: כמה שעות בשבוע צפה כל תלמיד בתכנים מדעיים, וכמה שעות צפה בתכני ספורט. הוא אוסף ארבע רשומות. הנתונים מומצאים לצורך ההוראה; הם אינם מעידים על העדפותיהם האמיתיות של תלמידים.

תלמיד	שעות מדע	שעות ספורט
א׳	3	4
ב׳	6	8
ג׳	4	3
ד׳	0	5

נבדוק את תלמיד א׳: הוא צפה שלוש שעות במדע וארבע שעות בספורט. במקום לכתוב את שני הנתונים בכל פעם מחדש, נוכל לרכז אותם יחד, באותו סדר, בתוך מבנה מתמטי קצר:

$$A = (3, 4)$$

זהו וקטור. הסוגריים מסמנים שמדובר בזוג רכיבים מסודר. המספר הראשון הוא שעות המדע והמספר השני הוא שעות הספורט. אין כאן עדיין חיזוי או למידת מכונה: רק ייצוג של נתונים. עם זאת, זוהי אבן יסוד חיונית לבניית מודל.

מדוע דווקא שני מאפיינים? שני רכיבים מאפשרים לנו לצייר את הנתונים במישור ולראות אותם בעיניים. במערכת אמיתית יכולים להיות חמישה, חמישים או אלפי מאפיינים. המתמטיקה שנלמד בהמשך פועלת גם כאשר אי־אפשר לצייר את כל הממדים.

עצור וחשוב: האם הווקטור $(4, 3)$ אומר שתלמיד א׳ "אוהב מדע" יותר מספורט? לא בהכרח. שעות צפייה הן מדידה של פעילות, ולא הוכחה להעדפה או לאיכות הלמידה. הבחירה במאפיינים חייבת להתאים לשאלת המחקר.

2 . מהו וקטור? הסבר מהאינטואיציה להגדרה

2.1. רכיב, סדר וממד

וקטור הוא אוסף סדור של מספרים. המילה "סדור" חשובה מאוד: לכל מקום בווקטור יש משמעות קבועה. אם הרכיב הראשון מייצג שעות מדע, הוא חייב לייצג שעות מדע בכל הרשומות. לא מחליפים סדר בין תלמידים רק מפני שהמספרים נוחים יותר.

$$x = (x_1, x_2, \ldots, x_n)$$

הסימון x הוא שם הווקטור. הסימונים x_1, x_2 וכן הלאה הם הרכיבים שלו. האות n היא מספר הרכיבים, המכונה ממד הווקטור. לדוגמה, (4 , 3) הוא וקטור דו־ממדי. הווקטור (2 , 4 , 3) הוא וקטור תלת־ממדי.

$$x \in \mathbb{R}^n$$

הכתיב הזה אומר: "הווקטור x שייך למרחב של וקטורים בעלי n רכיבים שהם מספרים ממשיים". הסימן $\mathbb{R}$ מייצג את קבוצת המספרים הממשיים, ולכן הרכיבים יכולים להיות גם מספרים שליליים, עשרוניים או אפס — כל עוד יש להם משמעות מתאימה בבעיה.

2.2. איך בונים מילון מאפיינים?

לפני שיוצרים וקטורים, כותבים מילון מאפיינים. זהו מסמך קטן שמגדיר אילו נתונים יופיעו בכל רכיב, באילו יחידות הם נמדדים, ואילו ערכים תקינים עבורם. מילון כזה מונע חלק גדול מהטעויות בפרויקטי AI.

מיקום ברכיב	שם המאפיין	יחידות	דוגמה
1	science_hours	שעות בשבוע	3
2	sport_hours	שעות בשבוע	4

אם נוסף מאפיין שלישי, למשל שעות קריאה, נחליט מראש שיהיה תמיד הרכיב השלישי. בשלב זה נקבל $x=(3,4,2)$, ולא נצטרך לשנות את משמעות שני הרכיבים הראשונים.

2.3. וקטור לעומת סקלר

סקלר הוא מספר יחיד. למשל 5 הוא סקלר, ואילו (4 , 3) הוא וקטור בעל שני רכיבים. לעיתים תוצאת פעולה על וקטורים תהיה סקלר, ולעיתים וקטור חדש. חשוב ללמוד לצפות מראש איזה סוג תוצאה נקבל.

ביטוי	סוג	כיצד קוראים אותו
5	סקלר	מספר יחיד
(4 , 3)	וקטור	שני רכיבים בסדר קבוע
((4 , 3),(6 , 8))	אוסף שני וקטורים	שתי רשומות, כל אחת בעלת

ביטוי	סוג	כיצד קוראים אותו
		שני רכיבים

2.4 מהו וקטור האפס?

וקטור האפס הוא וקטור שכל רכיביו שווים לאפס. בממד שני נסמן אותו (0 , 0), ובממד שלושה (0 , 0 , 0). הוא שימושי בפעולות חשבון: חיבור וקטור האפס אינו משנה את הווקטור. בהמשך נראה שאי־אפשר לנרמל את וקטור האפס באמצעות חלוקה באורכו, מפני שאורכו אפס.

$$(3, 4) + (0, 0) = (3, 4)$$

בדיקת הבנה 1: שישה תלמידים מתוארים כל אחד בארבע תכונות. כמה ממדים יש לכל וקטור? כמה תלמידים יש לנו? למה חשוב לא להתבלבל בין שני המספרים?

3 . מרשומה אחת למטריצת מאפיינים

בלמידת מכונה כמעט לעולם איננו מסתפקים בדוגמה אחת. רוצים לחקור אוסף דוגמאות, למשל עשרות תלמידים או אלפי תמונות. לכל דוגמה יש וקטור מאפיינים משלה. כאשר מסדרים את הווקטורים בשורות של טבלה מספרית, מתקבלת מטריצה.

	Science	Sport
A	3	4
B	6	8
C	4	3
D	0	5

איור 1.1 — ארבע רשומות ושני מאפיינים. כל שורה היא תלמיד וכל עמודה היא סוג מאפיין.

$$X = [[3, 4], [6, 8], [4, 3], [0, 5]]$$

$$\text{shape}(X) = (4, 2)$$

למטריצה X יש ארבע שורות, מפני שיש ארבעה תלמידים. יש לה שתי עמודות, מפני שבחרנו שני מאפיינים. הביטוי shape(X) מציין את ממדי המטריצה: תחילה מספר השורות ואחר כך מספר העמודות. סדר זה אינו מתהפך בגלל כיוון הכתיבה בעברית.

שלב 1. בוחרים את רשומת תלמיד א׳: (3 , 4).

שלב 2 . מוסיפים בשורה חדשה את רשומת תלמיד ב׳: (6 , 8).

שלב 3 . ממשיכים באותו סדר מאפיינים עבור תלמידים ג׳ וד׳.

שלב 4. סופרים ארבע רשומות ושתי תכונות — לכן גודל המטריצה הוא 4 × 2.

1.3. מימוש ראשון ב־NumPy

NumPy היא ספריית Python המיועדת לפעולות יעילות על מערכים מספריים. אנחנו יכולים ליצור בה מערך חד־ממדי עבור וקטור ומערך דו־ממדי עבור מטריצה. תחילה נכתוב את הנתונים באופן שהקוד משקף בדיוק את הטבלה שלנו.

```
import numpy as np

A = np.array([3, 4])
X = np.array([
    [3, 4],
    [6, 8],
    [4, 3],
    [0, 5]
])

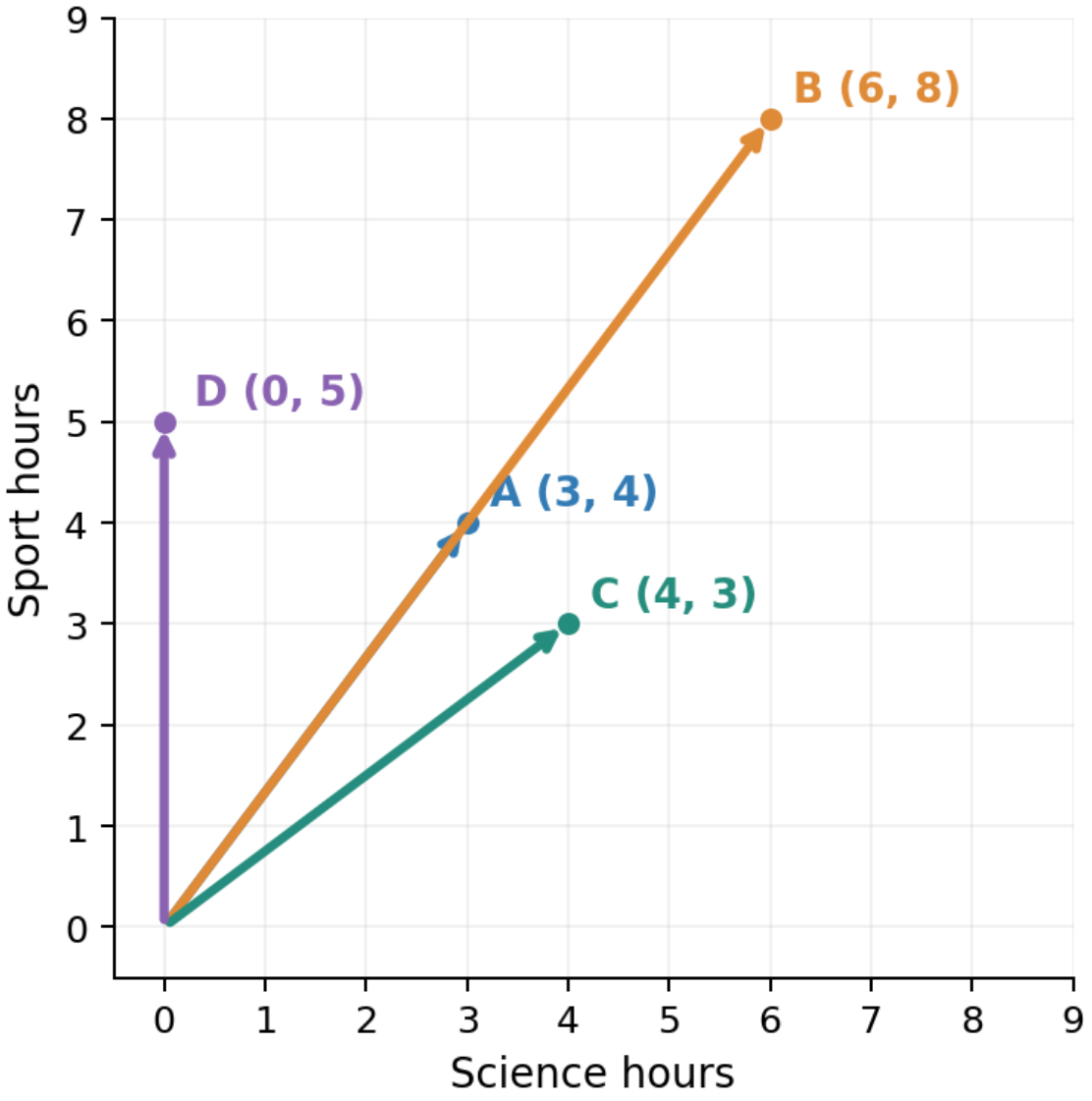
print(A.shape)   # (2,)
print(X.shape)   # (4, 2)
print(X[0])      # [3 4]
print(X[0, 1])   # 4
```

המחשב מתחיל לספור אינדקסים מאפס: X[0] הוא התלמיד הראשון, ו־X[0,1] הוא הרכיב השני של השורה הראשונה. לכן מתקבל הערך 4. חשוב להבחין בין הכתיב המתמטי "רכיב ראשון" לבין אינדקס 0 בקוד.

דגש ללמידת מכונה: במשימה של למידה מונחית נשמור בדרך כלל את התכונות במטריצה X ואת התוויות או ערכי היעד בווקטור נפרד y. לדוגמה, אם רוצים לחזות בחירת חוג, הבחירה היא יעד אפשרי — ולא מאפיין שמכניסים אוטומטית לקלט.

4. איך מציירים וקטור? נקודה, חץ וכיוון

בממד שני נוכל להשתמש במערכת צירים. הציר האופקי מייצג שעות מדע, והציר האנכי מייצג שעות ספורט. תלמיד א׳ מיוצג בנקודה (4, 3): הולכים שלוש יחידות ימינה וארבע יחידות למעלה. אפשר גם לצייר חץ שמתחיל בראשית הצירים ומסתיים בנקודה זו.



איור 1.2 — ארבעה וקטורים באותו מרחב תכונות. כיוון וגודל הם מאפיינים גיאומטריים של החצים.

הנקודה והחץ משתמשים באותם שני רכיבים, אך מדגישים רעיונות שונים. הנקודה עונה על השאלה "היכן נמצא הפרופיל במישור התכונות?". החץ עוזר לנו לחשוב על כיוון ועל גודל. עבור נתונים כמו שעות צפייה, אין כאן "תנועה אמיתית"; זהו ייצוג חזותי למספרים.

תלמיד א' ו־ב' מונחים על אותו כיוון מן הראשית: B=(6,8) הוא פי שניים מ־A=(3,4). לכן היחס בין שני המאפיינים אצלם זהה, אף על פי שתלמיד ב' צפה כפול שעות בכל תחום. בפרק המרחקים נלמד כיצד ניתן למדוד הן הבדלי גודל והן דמיון כיווני.

מגבלה של הגרף: כשהווקטור מכיל ארבעים מאפיינים, אין לנו דרך פשוטה לצייר אותו כנקודה במישור רגיל. ובכל זאת, אפשר לחבר, לחסר, לנרמל ולהכפיל וקטורים בדיוק לפי אותם כללים מתמטיים.

4.1 העשרה — תיאור וקטור באמצעות אורך וזווית

עד כה תיארנו את הווקטור במישור באמצעות שני רכיבים: כמה הולכים ימינה וכמה למעלה. אפשר לתאר אותו גם באמצעות אורך r והזווית θ שהוא יוצר עם הכיוון החיובי של הציר האופקי. דרך זו נקראת ייצוג פולרי. נלמד אותה כהרחבה המקשרת בין כיוון, נורמה ודמיון קוסינוס בפרק הבא.

עבור $A=(3,4)$, האורך הוא $r=5$. הזווית מתאימה למשולש שבו הניצב האופקי הוא 3 והאנכי הוא 4. לכן $\tan(\theta)=4/3$ והזווית היא בקירוב 53.13 מעלות. אין צורך לשנן את המספר; חשוב להבין שזו דרך נוספת לתאר את אותו חץ.

$$r = \sqrt{x^2+y^2} \quad ; \quad x = r\cos(\theta) \quad ; \quad y = r\sin(\theta)$$

```
x, y = 3.0, 4.0
angle_deg = np.degrees(np.arctan2(y, x))
print(round(angle_deg, 2))  # 53.13
```

> מעמד תת־הסעיף: זהו גשר גיאומטרי והרחבה לימודית. נשתמש בו רק לאחר שהתלמידים שולטים ברכיבים ובנורמת $L2$, ונבדיל בינו לבין הדרישות הפורמליות שאומתו.

5. פעולות בסיסיות על וקטורים

5.1 חיבור וקטורים — פעולה רכיב אחר רכיב

כדי לחבר שני וקטורים, מחברים את הרכיב הראשון של הראשון לרכיב הראשון של השני, את הרכיב השני לשני, וכן הלאה. חייב להיות מספר רכיבים זהה. כאשר מפרשים תוצאה על נתונים, חשוב גם שהרכיבים המתאימים ייצגו את אותם מאפיינים ובאותן יחידות.

$$\mathbf{a} + \mathbf{b} = (a_1+b_1,\ a_2+b_2,\ \ldots,\ a_n+b_n)$$

נניח שדודו מבקש לסכם שעות פעילות של שתי קבוצות באותו שבוע. הוא מסכם שני וקטורים: $A=(3,4)$ ו־$C=(4,3)$.

שלב 1. רכיב המדע: $3 + 4 = 7$.

שלב 2. רכיב הספורט: $4 + 3 = 7$.

שלב 3. כותבים את התוצאה לפי הסדר המקורי: $(7,\ 7)$.

$$(3,\ 4) + (4,\ 3) = (7,\ 7)$$

שימו לב: התוצאה יכולה לתאר סכום שעות של שני תלמידים, אך היא אינה בהכרח "תלמיד חדש". המתמטיקה נותנת לנו וקטור תקין; כדי לפרש אותו, חייבים לחזור להקשר שבו נשאלה השאלה.

5.2. חיסור וקטורים — זיהוי השוני

בחיסור וקטורים, מחסרים בכל רכיב את הערך השני מן הראשון. לדוגמה, נשווה את תלמיד ג׳ לתלמיד א׳: $C-A=(4-3,3-4)=(1,-1)$.

$$C - A = (4-3,\ 3-4) = (1,\ -1)$$

הרכיב הראשון אומר שתלמיד ג׳ צפה שעה אחת יותר במדע. הרכיב השני אומר שהוא צפה שעה אחת פחות בספורט. סימן שלילי אינו ״טעות״; הוא מציין כיוון של שינוי או הפרש. זהו הבדל חשוב בין נתון שלילי לבין נתון שגוי.

5.3. כפל בסקלר — הגדלה או הקטנה של כל הרכיבים

סקלר הוא מספר יחיד, למשל 2, 0.5 או -1. כפל וקטור בסקלר מכפיל כל אחד מרכיביו באותו מספר. אם כל שעות הצפייה מוכפלות פי שניים, הווקטור גדל אך היחסים בין רכיביו אינם משתנים.

$$k \cdot (a_1,\ a_2) = (k \cdot a_1,\ k \cdot a_2)$$

$$2 \cdot (3,\ 4) = (6,\ 8)$$

כפל ב־0.5 נותן $(1.5,2)$ — מחצית מכל רכיב. כפל ב־0 מחזיר וקטור אפס. כפל בסקלר שלילי הופך את כיוון החץ במישור; בהקשר של שעות צפייה ערכים שליליים אינם מתארים זמן צפייה רגיל, אך מבחינה מתמטית הפעולה תקינה.

5.4. בדיקה עם NumPy

```
A = np.array([3, 4])
C = np.array([4, 3])

print(A + C)     # [7 7]
print(C - A)     # [ 1 -1]
print(2 * A)     # [6 8]
print(0.5 * A)   # [1.5 2. ]
```

התוצאות זהות לחישוב הידני. זאת דרך עבודה נכונה בלמידת מכונה: קודם מבינים את הפעולה ומשחזרים אותה על נתונים קטנים; אחר כך נותנים לספרייה לבצע אותה במהירות על נתונים גדולים יותר.

5.5. מתי אסור לחבר?

אי־אפשר לחבר רכיב־רכיב וקטור בן שני רכיבים לווקטור בן שלושה רכיבים באותה פעולה רגילה. נוסף על כך, גם שני וקטורים בעלי אותו אורך מבחינת מספר רכיבים עלולים להיות בלתי־מתאימים מבחינת המשמעות: (גובה בס״מ, גיל בשנים) אינו אותו סוג נתונים כמו (גיל בשנים, גובה בס״מ). יש לתאם סדר ויחידות לפני החישוב.

> שאלת חשיבה: אם נחליף בטעות את סדר העמודות רק אצל תלמיד אחד, האם המחשב יודיע בהכרח שיש שגיאה? לא. רוב החישובים עדיין יעבדו, אך תתקבל תוצאה שאין לה פרשנות נכונה.

6 . כמה גדול הווקטור? נורמה ואורך

לאחר שלמדנו לחבר ולחסר, עולה שאלה חדשה: האם אפשר לתאר את הגודל של וקטור בעזרת מספר יחיד? כן. משפחה של כלים מתמטיים הקרויים נורמות מספקת דרכים למדוד את גודלו. נלמד שתי נורמות חשובות: L1 ו־L2. מרחק בין שני וקטורים יופיע בהרחבה בפרק 1.2; כאן אנחנו מחשבים את גודלו של וקטור יחיד.

6.1. נורמת L1 — סכום ערכים מוחלטים

ב־L1 לוקחים את הערך המוחלט של כל רכיב ומחברים. ערך מוחלט הוא המרחק של מספר מאפס על ציר המספרים, ולכן הוא אינו שלילי. למשל הערך המוחלט של −3 הוא 3.

$$\|x\|_1 = |x_1| + |x_2| + \dots + |x_n|$$

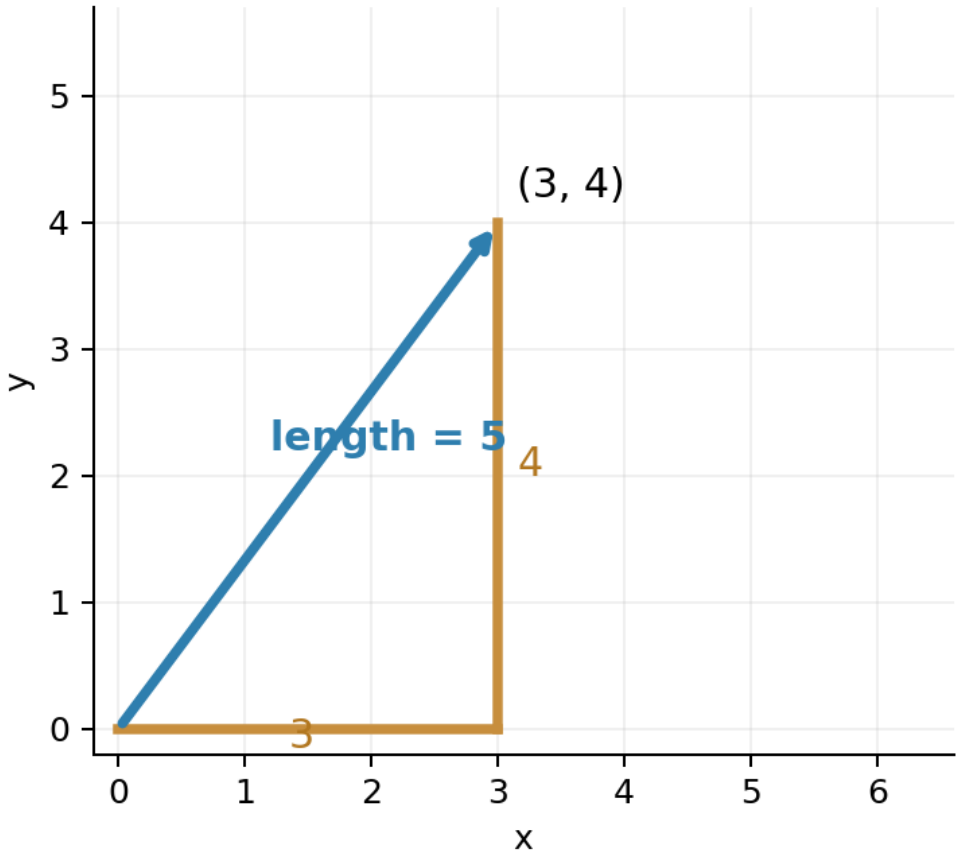
ל־A=(3,4), נורמת L1 היא 4 + 3 = 7. במקרה שלנו שני הרכיבים הם שעות, ולכן התוצאה היא סך שעות הצפייה בשני התחומים.

$$\|A\|_1 = |3| + |4| = 7$$

אבל אם נבחר וקטור (גובה בסנטימטרים, גיל בשנים), אין משמעות טבעית לסכום שני הרכיבים כפי שהם. לכן התוצאה המתמטית אינה מבטיחה פרשנות מדעית שימושית.

6.2. נורמת L2 — אורך אוקלידי

נורמת L2 נותנת את אורך החץ במרחב. בממד שני היא נשענת על משפט פיתגורס: אם רכיבי הווקטור הם שתי ניצבות במשולש ישר־זווית, אורך החץ הוא אורך היתר.



איור 1.3 — הרכיבים 3 ו־4 יוצרים משולש ישר־זווית. אורך הווקטור הוא 5.

$$\|x\|_2 = \sqrt{(x_1^2 + x_2^2 + \ldots + x_n^2)}$$

נחשב את אורך הווקטור A=(3,4) לאט ובשלבים, כדי להבין מה עושה כל חלק בנוסחה:

שלב 1. מעלים כל רכיב בריבוע: 3 = 9 ו־ 4 ² =16. כך כל תרומה לגודל נעשית לא־שלילית.

שלב 2. מחברים את ריבועי הרכיבים: 6 1 + 9 = 2 5.

שלב 3. מוציאים שורש ריבועי כדי לחזור ליחידות המקוריות: √ 5 2 = 5.

$$\|A\|_2 = \sqrt{(3^2 + 4^2)} = \sqrt{25} = 5$$

לתלמיד ב', B=(6,8), נקבל √(4 6 + 6 3)=√100=10. הוא צפה פי שניים בכל תחום, ולכן גם אורך הווקטור שלו גדול פי שניים.

6.3. למה L1 ו־L2 אינן נותנות תמיד אותה תוצאה?

נורמת L1 מסכמת את הערכים המוחלטים. נורמת L2 מחברת ריבועים ואז מוציאה שורש. אלה שתי דרכים שונות להגדיר גודל, ולכן הן מדגישות היבטים שונים של הנתונים. נורמת L2 רגישה יותר לרכיבים גדולים, בשל פעולת הריבוע.

וקטור	נורמת L1	נורמת L2
(3 , 4)	7	5

וקטור	נורמת L1	נורמת L2
(6 , 8)	14	10
(5 , 0)	5	5
(3 , 4 −)	7	5

למרות ההבדל, שתי הנורמות מחזירות תמיד ערך לא־שלילי, ורק וקטור האפס מקבל נורמה אפס. אם מכפילים וקטור פי 3 , הנורמה שלו מוכפלת פי 3 ; ואם מכפילים במינוס 3 , הנורמה מוכפלת בערך המוחלט 3 .

$$\|k \cdot x\| = |k| \cdot \|x\|$$

6.4. מחשבים ב־NumPy

```
A = np.array([3, 4])

l1 = np.sum(np.abs(A))
l2 = np.linalg.norm(A)

print(l1)  # 7
print(l2)  # 5.0
```

np.abs מחזירה ערכים מוחלטים רכיב־רכיב; np.sum מסכמת אותם. np.linalg.norm מחזירה כברירת מחדל את נורמת L2 של וקטור חד־ממדי. מומלץ תמיד לכתוב הערה קצרה בקוד המסבירה איזו נורמה בחרנו ולמה.

7 . וקטור יחידה — להפריד בין גודל לכיוון

נניח שדודו רוצה לזהות תלמידים שיש להם יחס דומה בין תחומי הצפייה, גם כאשר אחד מהם צופה הרבה יותר שעות. A=(3,4) ו־B=(6,8) שונים בכמות שעות הצפייה, אך שניהם מקיימים אותו יחס בין מדע לספורט. אפשר לייצג את הכיוון המשותף באמצעות וקטור שאורכו 1 — וקטור יחידה.

נרמול לפי נורמת L2 פירושו לחלק כל רכיב באורך הווקטור. התנאי החשוב הוא שהווקטור לא יהיה וקטור האפס, משום שאסור לחלק באפס.

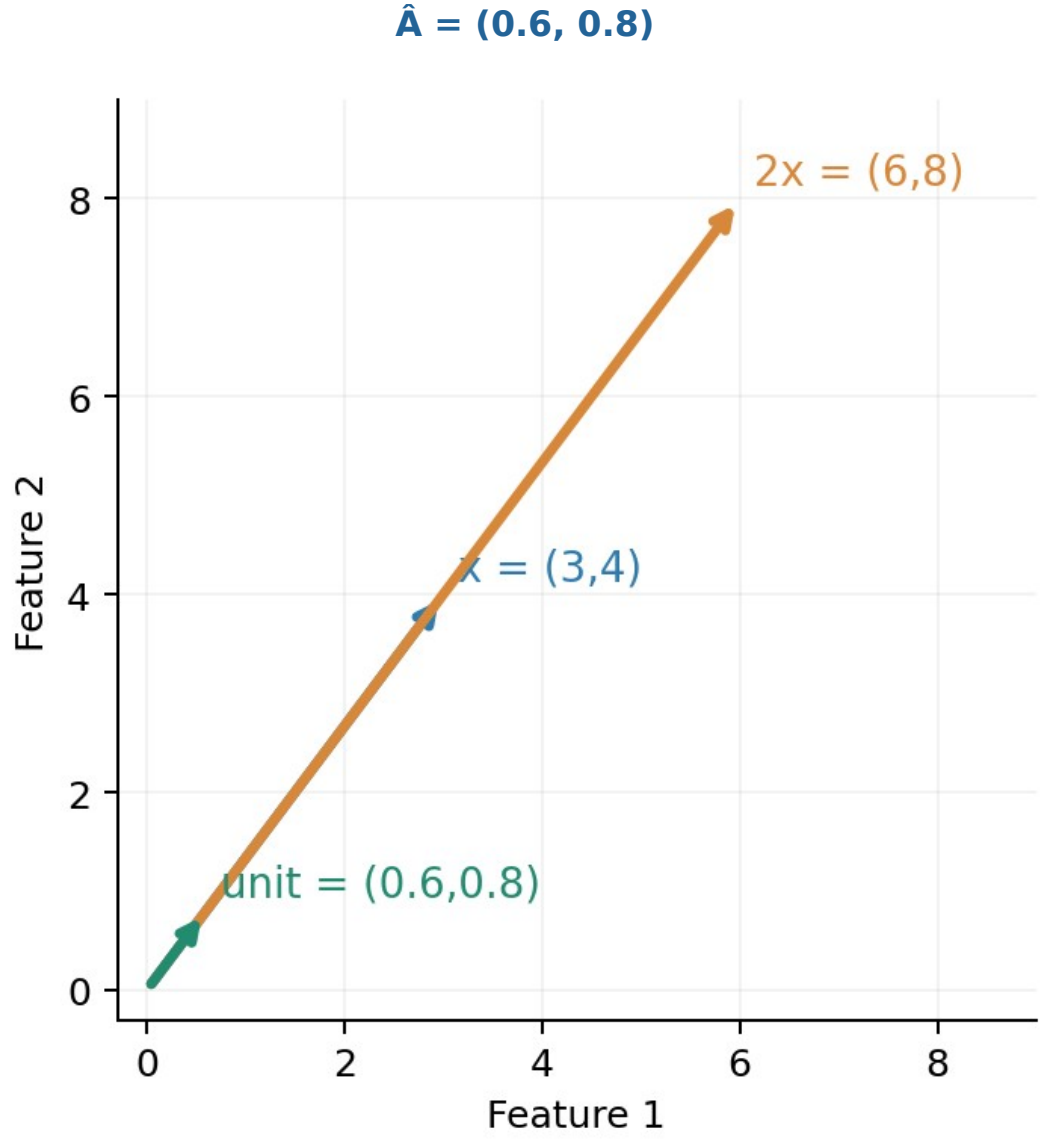
$$\hat{x} = x / \|x\|_2 \quad (x \neq 0)$$

שלב 1. מתחילים ב־A=(3,4).

שלב 2 . כבר חישבנו ש־‖A‖₂=5.

שלב 3 . מחלקים כל רכיב ב־ 5 : (3 / 5 , 4 / 5)=(0 . 6 , 0 . 8).

שלב 4 . בודקים: √(0 . 6 + 0 . 8)=√(0 . 3 6 + 0 . 6 4)=1.



איור 1.4 — הכפלה ב־ 2 משנה את האורך; חלוקה באורך יוצרת וקטור יחידה באותו כיוון.

אם נבצע אותה פעולה על $B=(6,8)$, נחלק ב־10 ונקבל (0.6 , 0.8). לכן שני הווקטורים השונים הופכים לאותו וקטור יחידה. איבדנו את המידע על הכמות הכוללת, אבל שמרנו את יחס הרכיבים ואת הכיוון.

1.7. מימוש בטוח ב־Python

```
A = np.array([3.0, 4.0])
length = np.linalg.norm(A)

if length > 0:
    unit_A = A / length
    print(unit_A)   # [0.6 0.8]
else:
    print('Zero vector: no unit direction')
```

בקוד זה אנחנו משתמשים בערכים עשרוניים כדי שפעולת החלוקה תחזיר תוצאה עשרונית. התנאי length > 0 מגן מפני חלוקה באפס. בפרויקט אמיתי יש לכתוב גם בדיקות לטיפוסי נתונים ולערכים חסרים.

7.2. לא לבלבל בין נרמול לאורך 1 לבין תקנון

נרמול L2 משנה כל וקטור בנפרד כך שאורכו יהיה 1. תקנון Z-score, שנלמד בפרק סטטיסטיקה, מחסר ממוצע של תכונה ומחלק בסטיית התקן שלה. אלה שתי פעולות שונות, בעלות מטרות שונות. עוד נלמד כיצד מתאימים טרנספורמציות לנתוני האימון בלבד כדי למנוע זליגת נתונים.

מה הפסדנו? לאחר נרמול, A ו־B נראים זהים. אם כמות שעות הצפייה חשובה לשאלת החקר, נרמול זה עשוי דווקא להסתיר מידע חשוב. לא משתמשים בפעולה רק מפני שהיא קיימת בספרייה; בוחרים אותה לפי מטרת המודל.

8 . מכפלה פנימית — כיצד וקטור הופך לציון יחיד?

עד כה הפקנו וקטורים מתוך פעולות על וקטורים, או חישבנו את גודלם. כעת נלמד פעולה מרכזית מאוד בלמידת מכונה: מכפלה פנימית. היא מחברת את ההשפעה של כמה מאפיינים למספר יחיד.

נגדיר שני וקטורים באותו ממד: x הוא וקטור המאפיינים של תלמיד, ו־w הוא וקטור משקולות. כל משקל קובע כמה המאפיין המתאים יתרום לחישוב. המכפלה הפנימית מכפילה רכיב מול רכיב, ואז מחברת את כל המכפלות.

$$w \cdot x = w_1x_1 + w_2x_2 + \ldots + w_nx_n$$

8.1. חישוב ידני מלא

נתייחס שוב ל־A=(3,4). דודו מגדיר — רק לצורך הדגמה — שמשקל שעות המדע הוא 2 ומשקל שעות הספורט הוא 1. לכן w=(2,1).

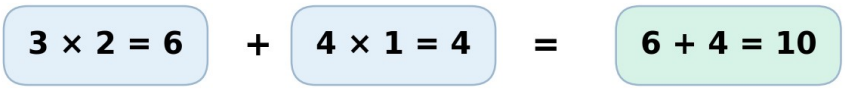
מאפיין	ערך x	משקל w	התרומה למכפלה
מדע	3	2	2 × 3 = 6
ספורט	4	1	1× 4 = 4

שלב 1. מכפילים כל ערך במשקל המתאים לו: 2 × 3 = 6 , ואחר כך 1× 4 =4 .

שלב 2 . מחברים את המכפלות: 6 + 4 =10.

שלב 3 . המספר 10 הוא התוצאה של המכפלה הפנימית. זהו סקלר — לא וקטור.

$$w \cdot A = (2,1) \cdot (3,4) = 2\times3 + 1\times4 = 10$$



איור 1.5 — מכפלה פנימית: מכפילים רכיבים מתאימים ומסכמים לתוצאה אחת.

למה דווקא המשקולות (1 , 2)? הן נבחרו באופן שרירותי כדי שנוכל לבצע חישוב פשוט. הן עדיין לא נלמדו מתוך דוגמאות. כאשר מאמנים מודל, תהליך הלמידה עשוי להתאים את המשקולות כדי לשפר חיזוי על נתונים, בכפוף להגדרת מודל ופונקציית שגיאה.

8.2. מה קורה כאשר מחליפים את סדר הרכיבים?

אם בטעות נכתוב את הנתונים בסדר הפוך ונשאיר את המשקולות ללא שינוי, נקבל x=(4,3), והמכפלה תהיה 2 × 1 + 4 × 3 = 11. התוצאה השתנתה אף על פי ששני המספרים המקוריים נשארו. לכן הסדר בווקטור אינו עניין אסתטי: הוא חלק מן המשמעות של המודל.

8.3. מכפלה פנימית איננה כפל רגיל של מערכים

ב־NumPy, הביטוי w * x מחזיר בדרך כלל את המכפלות של הרכיבים זה מול זה — כלומר מערך. הפעולה np.dot(w,x) מחברת גם את המכפלות ומחזירה מספר יחיד במקרה של שני וקטורים חד־ממדיים.

```
x = np.array([3, 4])
w = np.array([2, 1])

print(w * x)        # [6 4]
print(np.dot(w, x)) # 10
print(w @ x)        # 10
```

זהו מקום שבו תלמידים רבים טועים: שתי השורות הראשונות משתמשות באותם נתונים, אבל מחזירות סוג תוצאה אחר. לפני שמריצים קוד, נבא האם הפלט יהיה מערך או מספר.

8.4. מה המחשב עושה בתוך np.dot?

כדי לראות שמאחורי הפקודה מסתתר אותו חישוב ידני, נכתוב לולאה שעוברת על הזוגות המתאימים. zip(w,x) יוצר זוגות של רכיבים מאותו מיקום; בכל צעד נכפיל את שני המספרים ונוסיף לתוצאה המצטברת. כך נוכל להסביר את הפעולה גם ללא ספרייה מיוחדת.

```
total = 0
for wi, xi in zip(w, x):
    total = total + wi * xi
print(total)  # 10
```

זו אינה דרך חלופית לבחור משקולות או לאמן מודל: זו דרך מפורטת לחשב מכפלה פנימית. כאשר נבין היטב את הלולאה, נעדיף בדרך כלל את הפקודה הספרייתית הקצרה והיעילה.

1.2 קשר גיאומטרי — הכנה לפרק 8.5.

למכפלה הפנימית יש גם פירוש שקשור לזווית בין וקטורים. בהמשך נראה שהיא קשורה לאורך כל וקטור ולקוסינוס של הזווית ביניהם. לא נדרש כרגע לחשב זוויות; החשוב הוא לזכור שאותה פעולה יכולה לתאר גם סכום משוקלל של מאפיינים וגם קשר גיאומטרי בין כיוונים.

נקודת בקרה: תוצאת מכפלה פנימית יכולה להיות אפס, חיובית או שלילית, בהתאם לרכיבים. אם המשקולות שליליות, חלק מן המאפיינים עשויים להקטין את הציון במקום להגדיל אותו. אין להסיק מן הציון לבדו על הסתברות.

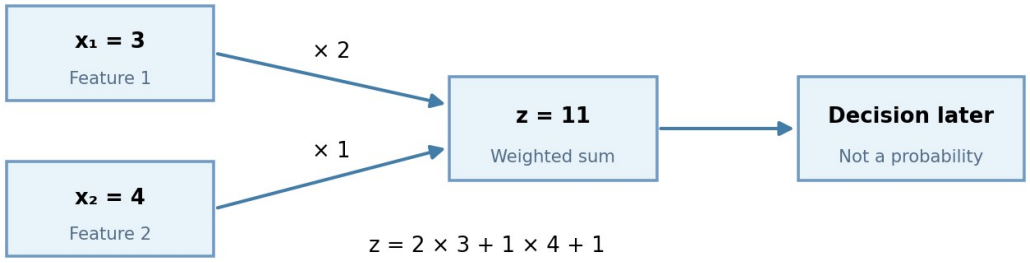
9 . מהו הקשר בין מכפלה פנימית לנוירון?

נוירון מלאכותי בסיסי מתחיל לא פעם בחישוב ליניארי: הוא מקבל וקטור מאפיינים x, משקולות w והטיה b. המכפלה הפנימית מרכזת את תרומת המאפיינים, וההטיה מוסיפה מספר קבוע. לאחר מכן יכולה לפעול פונקציית הפעלה; את סוגי הפונקציות ואת תהליך האימון נלמד בפרקים מאוחרים יותר.

$$z = w \cdot x + b$$

למשל כאשר $x=(3,4)$, $w=(2,1)$ וההטיה $b=1$, נקבל:

$$z = 2\times3 + 1\times4 + 1 = 11$$



איור 1.6 — מעבר מתכונות ומשקולות לסכום משוקלל עם הטיה. ההחלטה או ההסתברות מגיעות רק בשלבים נוספים.

המספר 11 הוא ציון של פונקציה ליניארית. הוא איננו הסתברות, מפני שהסתברות חייבת להיות בין 0 ל־1, והוא גם איננו בהכרח חיזוי של מערכת מאומנת. בספר נחזור לאותו מבנה כאשר נלמד רגרסיה ליניארית, פרספטרון ורשתות עצביות עמוקות.

1.9. דוגמה נגדית: משקולות לא מבטיחות למידה

אם דודו בחר בעצמו את המשקולות, הוא כתב כלל חישוב, אך המערכת עדיין לא למדה מדוגמאות. כדי לדבר על למידת מכונה נצטרך להגדיר מטרה, דוגמאות אימון, מדד שגיאה ותהליך שמעדכן פרמטרים או מתאים מודל. זו הבחנה יסודית בין חישוב מתמטי לבין למידה מתוך נתונים.

1.10. מעבדת הבנה מודרכת ב־Google Colab

כעת נבדוק שכל רעיון שחישבנו ידנית מופיע גם בקוד. נבנה תוכנית קצרה, וננסה להסביר את התוצאה לפני הרצת כל תא.

1.10.1. טעינת נתונים ובדיקת צורה

```
import numpy as np

X = np.array([
    [3, 4],
    [6, 8],
    [4, 3],
    [0, 5]
], dtype=float)

print('shape:', X.shape)
print('first student:', X[0])
print('science column:', X[:, 0])
```

הסימון X[:,0] פירושו: כל השורות בעמודה הראשונה. התוצאה תהיה שעות המדע של ארבעת התלמידים. כך אפשר לשלוף מאפיין אחד מכל הדוגמאות ולנתח אותו. את הניתוח הסטטיסטי המלא נבצע בפרק ייעודי.

1.10.2. השוואת חישוב ידני לפלט

```
A = X[0]
B = X[1]

print('A+B:', A + B)
print('B-A:', B - A)
print('2*A:', 2 * A)
print('L1(A):', np.abs(A).sum())
print('L2(A):', np.linalg.norm(A))
```

לפני ההרצה כתבו במחברת מה אתם מצפים לקבל. אם הפלט אינו תואם את התחזית, אל תתקנו מיד את הקוד. בדקו קודם אם טעיתם בפעולת החשבון, בבחירת השורה או בפירוש המאפיינים.

1.10.3. הדגמת משקולות והטיה

```
w = np.array([2.0, 1.0])
b = 1.0
```

```
for i in range(X.shape[0]):
    score = np.dot(w, X[i]) + b
    print('student', i, 'score', score)
```

הלולאה עוברת על כל שורה במטריצה. בכל איטרציה X[i] הוא וקטור המאפיינים של תלמיד אחד. המכפלה הפנימית מחשבת את סכום התרומות, ולאחריה מתווספת ההטיה.

10.4. השוואה למימוש וקטורי

```
scores = X @ w + b
print(scores)
```

הביטוי X @ w מפעיל את אותו חישוב על כל השורות של X. הפלט הוא וקטור ציונים, אחד לכל תלמיד. לא למדנו כאן אלגוריתם חדש אלא השתמשנו בדרך תמציתית לביצוע חישוב שכבר הבנו. זו דוגמה לכוח של חישוב וקטורי ב־NumPy.

10.5. בדיקות שגיאה פשוטות

```
bad_x = np.array([3, 4, 2])

if bad_x.shape[0] == w.shape[0]:
    print(np.dot(w, bad_x))
else:
    print('Feature count does not match weight count')
```

בדיקת התאמה בין מספר המאפיינים למספר המשקולות יכולה למנוע שגיאת ממדים. היא אינה בודקת האם סדר המאפיינים נכון — את זה נצטרך להבטיח באמצעות מילון התכונות ותהליך הכנת הנתונים.

11. התבוננות ביקורתית: מתי פעולה מתמטית נכונה עלולה להטעות?

11.1. יחידות וקנה מידה

נניח שנוסיף מאפיין של עלות מנוי בחודש. השעות נמדדות במספרים קטנים, ואילו המחיר עשוי להיות 0 5 2 שקלים. בחישוב מרחק או נורמה, רכיב המחיר עלול לשלוט בתוצאה פשוט מפני שסולם המספרים גדול יותר. הבעיה אינה בחוקי המתמטיקה אלא באופן שבו תיארנו את המציאות.

בהמשך נלמד כיצד מבצעים תקנון ונרמול של תכונות לפי הצורך, וכיצד נמנעים מזליגת מידע ממאגר הבדיקה לתהליך האימון. בשלב זה חשוב לזהות את הבעיה ולנמק מדוע אין לשלב יחידות שונות ללא חשיבה על המשמעות שלהן.

11.2. נתונים חסרים

אם תלמיד לא דיווח על שעות הספורט, אין פירוש הדבר בהכרח שצפה אפס שעות. אפס הוא ערך מספרי בעל משמעות; חוסר נתון הוא מצב אחר. החלפת ערך חסר באפס בלי בדיקה עלולה לשבש את כל החישובים. הכנת נתונים היא שלב חיוני בפיתוח AI ולא תיקון טכני קטן.

11.3. הטיה בתכונות ובהחלטות

משקולת גדולה למאפיין מסוים עשויה להשפיע מאוד על תוצאה. אם נגדיר משקל גבוה לשעות צפייה רק משום שנוח למדוד אותן, ייתכן שנפגע בתלמידים שלומדים בדרכים אחרות. כדי לפתח מערכת אחראית יש לבחון מה כל מאפיין מייצג, למי הוא מתאים ומהן מגבלותיו.

11.4. אין להסיק סיבתיות מציון

גם אם מערכת מוצאת שציון חישובי גבוה קשור להצלחה בחוג, אין זה מוכיח שהגדלת שעות הצפייה תגרום להצלחה. ייתכנו הסברים אחרים, כגון ניסיון קודם או עניין אישי. למדען נתונים חשוב להבחין בין תיאור קשר לבין טענה על גורם ותוצאה.

12. טעויות נפוצות — מתקנים מתוך ההיגיון

טעות	למה זו טעות	דרך בדיקה
וקטור בעל ארבע שורות ושתי עמודות הוא 4-ממדי	ארבע השורות הן ארבע דוגמאות; לכל שורה שני מאפיינים	הסתכלו על X.shape ועל X[0]
חיבור (4, 3) ו־(1, 5, 2)	מספרי הרכיבים שונים	ספרו רכיבים לפני הפעולה
w*x זהה תמיד ל־np.dot(w,x)	כפל איבר־איבר מחזיר מערך; dot מסכם	נבאו מהו סוג התוצאה
אפשר לחלק את (0, 0) באורך שלו	האורך 0 ולכן נקבל חלוקה באפס	בדיקת norm לפני חלוקה
L2-normalization זהה ל־Z-score	האחת מנרמלת כל וקטור, השנייה מתקננת תכונה	הגדירו מה משתנה בתהליך
ציון ליניארי 11 הוא הסתברות	הוא אינו מוגבל לתחום 0–1	z, זהו לא הסתברות
מותר להחליף סדר מאפיינים אצל תלמיד אחד	המשקולות יוצמדו למאפיינים לא נכונים	מילון תכונות קבוע

13. תרגול עצמאי מדורג — ללא פתרונות

לפני כל תרגיל: נסו להסביר במילים איזו פעולה נדרשת. כתבו חישוב ידני, ורק אחריו בדקו ב־Python. חלק מן השאלות מיועדות לדיון ולא לפתרון מספרי יחיד.

13.1 רמה א׳ — שליטה במושגים

- א1. כתבו את הווקטור המתאים לתלמיד שצופה 2 שעות מדע ו־5 שעות ספורט. ציינו מה מייצג כל רכיב.
- א 2. כמה ממדים יש לווקטור (4, 0, 7, 2)? האם הרכיב השלישי חייב להיות חיובי?
- א 3. בחרו שלושה מאפיינים לחנות ספרים. כתבו מילון מאפיינים ווקטור לדוגמה עבור ספר אחד.
- א 4. למטריצה בעלת 2 שורות ו־4 עמודות: כתבו את shape ואת משמעות כל אחד משני המספרים.
- א 5. הבחינו בין סקלר, וקטור ומטריצה בדוגמאות: 12; (3, 2); מערך של 6 × 4.

13.2 רמה ב׳ — חישוב והסבר

- ב1. נתונים $u=(2,5)$, $v=(4,1)$. חשבו $u+v$, $u-v$ ו־$3u$. כתבו גם את משמעות התוצאה כאשר הרכיבים הם שעות.
- ב 2. חשבו את נורמת L1 ואת נורמת L2 של (8, 6). בדקו באיזו נורמה תקבלו סכום רכיבים.
- ב 3. חשבו את נורמת L2 של (−4, 3). הסבירו מדוע קיבלתם את אותה תוצאה כמו עבור (4, 3).
- ב 4. מצאו וקטור יחידה בכיוון (2, 1, 5). הראו את בדיקת האורך.
- ב 5. נתונים $x=(2,4,1)$ ו־$w=(3,-1,2)$. חשבו $w\cdot x$ והסבירו כיצד המשקל השלילי משפיע על התוצאה.
- ב 6. הסבירו מדוע (4, 3) ו־(8, 6) שונים זה מזה אף על פי שנרמול L2 ייתן לשניהם אותו וקטור.

13.3 רמה ג׳ — ניתוח ושיפוט מדעי

- ג1. צוות פיתוח משתמש בווקטור (גיל, ציון, גובה). כיצד עלול הסדר של המאפיינים להוביל לטעות שקטה?
- ג 2. הציעו שני וקטורים בעלי אותה נורמת L2 אך כיוונים שונים. האם פירושם בהכרח דומה?
- ג 3. לציון מועמדים נקבע $w=(10,1)$ ללא נתוני אימון. האם זו מערכת שלמדה מדוגמאות? נמקו.
- ג 4. הסבירו מדוע לא נכון להחליף ערך חסר של שעות צפייה באפס באופן אוטומטי.
- ג 5. כאשר מייצגים תמונה בצבעים מספריים, מהו מאפיין ומהו וקטור? אילו בעיות עשויות להתעורר בגלל מספר הממדים הגדול?
- ג 6. תארו מקרה שבו נרמול לאורך 1 דווקא יפגע במידע שהמודל צריך.

13.4. משימות Python — השלמות מכוונות

בתאים הבאים מופיעים מקומות להשלמה. סימני השאלה הם משימות לתלמיד, ולא שגיאת כתיב או טקסט חסר בספר. השלימו אותם במחברת התרגול האישית.

```
import numpy as np

u = np.array([2.0, 5.0])
v = np.array([4.0, 1.0])

sum_vectors = ?
diff_vectors = ?
l2_u = ?
print(sum_vectors, diff_vectors, l2_u)


w = np.array([3.0, -1.0, 2.0])
x = np.array([2.0, 4.0, 1.0])

score = ?
print(score)
```

לאחר שהשלמתם את התאים, בדקו מקרים נוספים, כולל וקטור אפס, וקטור בעל שלושה ממדים ומשקולת שלילית. אל תסתפקו בהרצה אחת שהצליחה.

14. שאלת הערכה מקורית בסגנון בגרות

במערכת התאמה לקורסים כל תלמיד מיוצג באמצעות שני נתונים: מספר שעות תרגול מעשי ומספר שעות לימוד עיוני בשבוע. תלמיד א׳ מיוצג בווקטור $A=(3,4)$, ותלמיד ב׳ בווקטור $B=(0,5)$. וקטור המשקולות הוא $w=(2,1)$ וההטיה היא $b=1$.

- סעיף א׳: ציינו מה מייצג כל רכיב ומהו הממד של הווקטורים A, B ו־w.
- סעיף ב׳: חשבו את נורמת $L2$ של A ושל B. הציגו במפורש נוסחה, הצבה ותוצאה.
- סעיף ג׳: חשבו את $w\cdot A$ ואת $w\cdot B$. הציגו טבלת תרומות של שני המאפיינים.
- סעיף ד׳: חשבו את הציון $z=w\cdot x+b$ לכל אחד מן התלמידים והסבירו אם מדובר בהסתברות.
- סעיף ה׳: תלמיד טוען שאם שני וקטורים קיבלו אותו אורך, הם חייבים לייצג אותו פרופיל. קבעו אם הוא צודק ונמקו בדוגמה.
- סעיף ו׳: כתבו קטע NumPy שמחשב את התוצאות ובודק שהמספרים תואמים את חישוביכם הידניים.

שאלת הערכה מקורית: זוהי שאלה שנכתבה לספר לצורך אימון חשיבה וחישוב; היא אינה מועתקת משאלון בגרות ואינה מוצגת כמחוון רשמי. פתרון מלא ומפתח בדיקה יישמרו בחוברת המורה הנפרדת.

1.5 . היישום הראשון בפרויקט הגמר

כבר בפרק הראשון כדאי לחשוב כמו מפתח מערכת, ולא רק כמו תלמיד שפותר תרגיל. בחרו רעיון לפרויקט AI: המלצת סרטים, ניתוח תמונות, מערכת לסיווג מוצרים, חיזוי צריכה או ניתוח מדידות סביבה. אינכם צריכים עדיין לאמן מודל. המטרה היא לתאר את הנתונים באופן מסודר ונכון.

- נסחו בשתי שורות את הבעיה: מה המערכת אמורה להציע, לחזות או לזהות?
- בחרו שלושה עד חמישה מאפיינים שאפשר למדוד. הסבירו את משמעות כל מאפיין, את יחידותיו ואת טווח הערכים הצפוי.
- כתבו וקטור לדוגמה אחת ובנו מטריצת דוגמה בת שלוש רשומות לפחות.
- ציינו אילו נתונים עלולים להיות חסרים וכיצד תתעדו אותם בלי להפוך כל חוסר לאפס.
- כתבו פסקה שמסבירה איזה מידע הייצוג שבחרתם שומר, ואיזה מידע הוא עלול להסתיר.

תוצר התלמיד בסיום השלב: מילון מאפיינים, טבלת נתונים קצרה, וקטור אחד, מטריצה ומסמך החלטות ראשוני. בהמשך נחזור למסמכים האלה כאשר נלמד הכנת נתונים, בחירת מודל והערכה.

1.6 . סיכום: מפת המושגים של הפרק

מושג	במילים פשוטות	דוגמה
Feature	תכונה מדידה של דוגמה	שעות ספורט
Vector	רשימה סדורה של רכיבים	$(3, 4)$
Dimension	מספר הרכיבים בווקטור	2
Scalar	מספר יחיד	5
Matrix	טבלה מספרית של שורות ועמודות	2×4
L1 Norm	סכום ערכים מוחלטים	$\|(3, 4)\|_1 = 7$
L2 Norm	אורך וקטור אוקלידי	$\|(3, 4)\|_2 = 5$
Unit Vector	וקטור באורך 1	$(0.6, 0.8)$
Dot Product	סכום מכפלות רכיבים מתאימים	$(2, 1) \cdot (3, 4) = 10$
Weight	מקדם המשפיע על תרומת מאפיין	למדעים 2
Bias	מספר קבוע המתווסף לסכום המשוקלל	$b=1$

דף נוסחאות מרוכז — לקריאה ולתרגול

$$x = (x_1, x_2, \ldots, x_n) \in \mathbb{R}^n$$

$$a + b = (a_1+b_1, \ldots, a_n+b_n)$$

$$k \cdot x = (kx_1, kx_2, \ldots, kx_n)$$

$$\|x\|_1 = \Sigma\, |x_i|$$

$$\|x\|_2 = \sqrt{(\Sigma\, x_i^2)}$$

$$\hat{x} = x / \|x\|_2 \quad (x \neq 0)$$

$$w \cdot x = \Sigma\, w_i x_i$$

$$z = w \cdot x + b$$

כל נוסחה מחייבת גם פירוש: מהו הקלט, מהו סוג הפלט, אילו תנאים חייבים להתקיים ומה משמעות התוצאה בהקשר הנתונים. לדעת להעתיק נוסחה אין פירושו לדעת להשתמש בה.

כרטיס יציאה — חמש שאלות קצרות

- מה ההבדל בין וקטור דו־ממדי לבין מטריצה בעלת שתי עמודות?
- מדוע חשוב לקבוע מראש סדר מאפיינים?
- מהו ההבדל בין נורמת L1 לנורמת L2?
- מתי אסור לנרמל וקטור לפי אורכו?
- כיצד מכפלה פנימית הופכת מספר מאפיינים לציון אחד?

מה נלמד בפרק 1.2?

בפרק הבא ננסה לענות על שאלה שעדיין נשארה פתוחה: כיצד קובעים אם שני תלמידים או שני מוצרים דומים זה לזה? נשתמש בנורמות שכבר למדנו כדי לחשב מרחקים בין וקטורים, ונכיר דמיון קוסינוס ואת ההבדל בין "קרוב במספרים" לבין "דומה בכיוון". כך נוכל להבין טוב יותר כיצד אלגוריתמים כמו KNN ומערכות המלצה משתמשים בייצוג מספרי.

מקורות לבניית ההסבר ולהעמקה

הפרק נכתב מחדש בעברית כיחידה מקורית. הנוסחאות הן כלים מתמטיים סטנדרטיים. מקורות ההשראה וההעמקה אינם מחליפים את תכנית הלימודים של משרד החינוך, ואינם מוצגים כציטוטים מתוך הספרים.

- Deisenroth, M. P., Faisal, A. A., & Ong, C. S. (2020). Mathematics for Machine Learning. Cambridge University Press. mml-book.github.io
- Serrano, L. G. (2021). Grokking Machine Learning. Manning Publications.

- Starmer, J. The StatQuest Illustrated Guide to Machine Learning.
- Géron, A. (2025). Hands-On Machine Learning with Scikit-Learn and PyTorch. O’Reilly Media.
- Zhang, A., Lipton, Z. C., Li, M., & Smola, A. J. Dive into Deep Learning. d2l.ai

- מפת יחידה 5 במאגר Data-Science-ML-Course, פרק 1.1 — וקטורים ב־$\mathbb{R}^n$; מסלול למידה עמוקה — הבסיס למודלים ולרשתות.
- משרד החינוך — תכנית התמחות בלמידת מכונה ולמידה עמוקה והוראות ההערכה המתעדכנות לפי שנת לימודים.

סוף פרק 1.1 — וקטורים. קובץ זה מיועד להשתלב בעתיד בספר השלם בלי לשנות את מספרי הסעיפים, הגדרות המושגים ומילון המאפיינים.

מדעי הבינה המלאכותית

מנתונים למודלים ולרשתות עצביות

פרק 1.2

מרחק ודמיון בין וקטורים

כיצד מחשב יודע מי דומה למי — ומה ההבדל בין מרחק, כיוון וגודל?

תחום	מיקום בתכנית	יישום בהמשך
בסיס מתמטי ללמידת מכונה	יחידה 5 — פרק 1, סעיף 1.2	KNN, Nearest Centroid, מערכות המלצה וייצוגי נתונים

מהדורת תלמיד עצמאית ומורחבת. הסברים, תרשימים, חישובים, תרגילים וקוד מקוריים שנכתבו לצורכי הוראה בעברית.

היקף הוראה מוצע: 6 – 8 שעות הכוללות הסבר, התנסות, תרגול ומבדק. אין זה היקף חובה רשמי לכל סעיף.

קשר לפרק הקודם: הפרק מניח היכרות עם וקטור, רכיבים, חיסור וקטורים, נורמת L1/L2, מכפלה פנימית ו־NumPy.

הערת התאמה: הנושאים נבנו מול מפת סעיף 1.2 וחומרי ההשתלמות במאגר. הנוסח המחייב והיקף החובה המדויק ייקבעו מול סילבוס משרד החינוך המעודכן.

מפת הלמידה

נתקדם משאלה פשוטה — איזה שני תלמידים דומים יותר? — אל כלים מתמטיים שבהם משתמשים אלגוריתמים אמיתיים.

תחנה	הרעיון החדש	התוצר
1	מרחק בין דוגמאות	הבחנה בין מאפיין לבין ערך
2	מרחק אוקלידי $L2$	חישוב והמחשת קו ישר
3	מרחק מנהטן $L1$	חישוב והמחשת מסלול צירי
4	בחירת מדד מרחק	פירוש מדעי של תוצאה
5	הרחבה למרחב R^n	חישוב ידני בתלת־ממד
6	מימוש ב־Python ו־NumPy	קוד שקול לנוסחה
7	מכפלה פנימית, זווית וקוסינוס	הבחנה בין כיוון לגודל
8	מערכת המלצה	דירוג דוגמאות
9	מרחק ב־KNN וקנה מידה	הסבר בחירת שכנים
10	מעבדה ותרגול עצמאי	תשובה מנומקת ללא פתרון מוכן

מטרות למידה

- לחשב מרחק מנהטן ומרחק אוקלידי בין שני וקטורים באותו ממד.
- להסביר מהי יחידת המדידה של המרחק ומה קורה כשמשנים קנה מידה.
- להגדיר מכפלה פנימית ולחשב דמיון קוסינוס בין שני וקטורים שאינם אפס.
- להבחין בין דמיון קוסינוס, מרחק קוסינוס, מרחק אוקלידי ומכפלה פנימית.
- לנתח הבדל בין שני משתמשים בעלי אותו דפוס העדפות אך כמויות שונות.
- לממש חישובים בשתי דרכים: קוד עם לולאה וקוד באמצעות NumPy.
- להסביר את השימוש במרחק ב־KNN ואת השימוש בדמיון במערכת המלצה.
- לזהות שימוש לא תקין: אורכים שונים, וקטור אפס, יחידות לא תואמות או התאמת קנה מידה לנתוני בדיקה.

שאלת פתיחה דודו ומיה אוהבים את אותם סוגי סרטים, אך דודו צופה פי שניים זמן. האם הם קרובים? האם הם דומים? האם שתי השאלות חייבות לקבל אותה תשובה?

1. מדוע בכלל צריך למדוד מרחק בין נתונים?

בספר הקודם למדנו כיצד לתאר משתמש באמצעות וקטור. לדוגמה, וקטור $(4, 3)$ יכול לייצג שלוש שעות צפייה במדע וארבע שעות צפייה בספורט. זהו ייצוג מסודר: הרכיב הראשון מתאר תמיד מדע, והרכיב השני תמיד ספורט. אבל היכולת לייצג משתמש אינה מספיקה כדי להמליץ לו על סרטון. נצטרך לדעת כיצד להשוות בינו לבין משתמשים אחרים.

נניח שמערכת המלצות מקבלת פרופיל של תלמיד חדש. היא יכולה לאתר תלמידים בעלי פרופילים דומים, לבדוק אילו סרטונים אהבו ולהציע את הסרטונים גם לתלמיד החדש. כדי לבצע את ההשוואה בצורה עקבית, אנחנו זקוקים לכלל חישוב — מדד מרחק או מדד דמיון.

מרחק קטן מרמז, כאשר בחרנו מאפיינים וקנה מידה מתאימים, שהדוגמאות קרובות זו לזו במרחב המאפיינים. לעומת זאת, דמיון גבוה מרמז על קרבה לפי ההגדרה המסוימת של מדד הדמיון. אין מדד יחיד שנכון לכל בעיה: הבחירה תלויה במה חשוב לנו להשוות.

תלמיד	שעות מדע	שעות ספורט	וקטור
דודו	3	4	$(4, 3)$
מיה	6	8	$(8, 6)$
אורי	3	5	$(5, 3)$

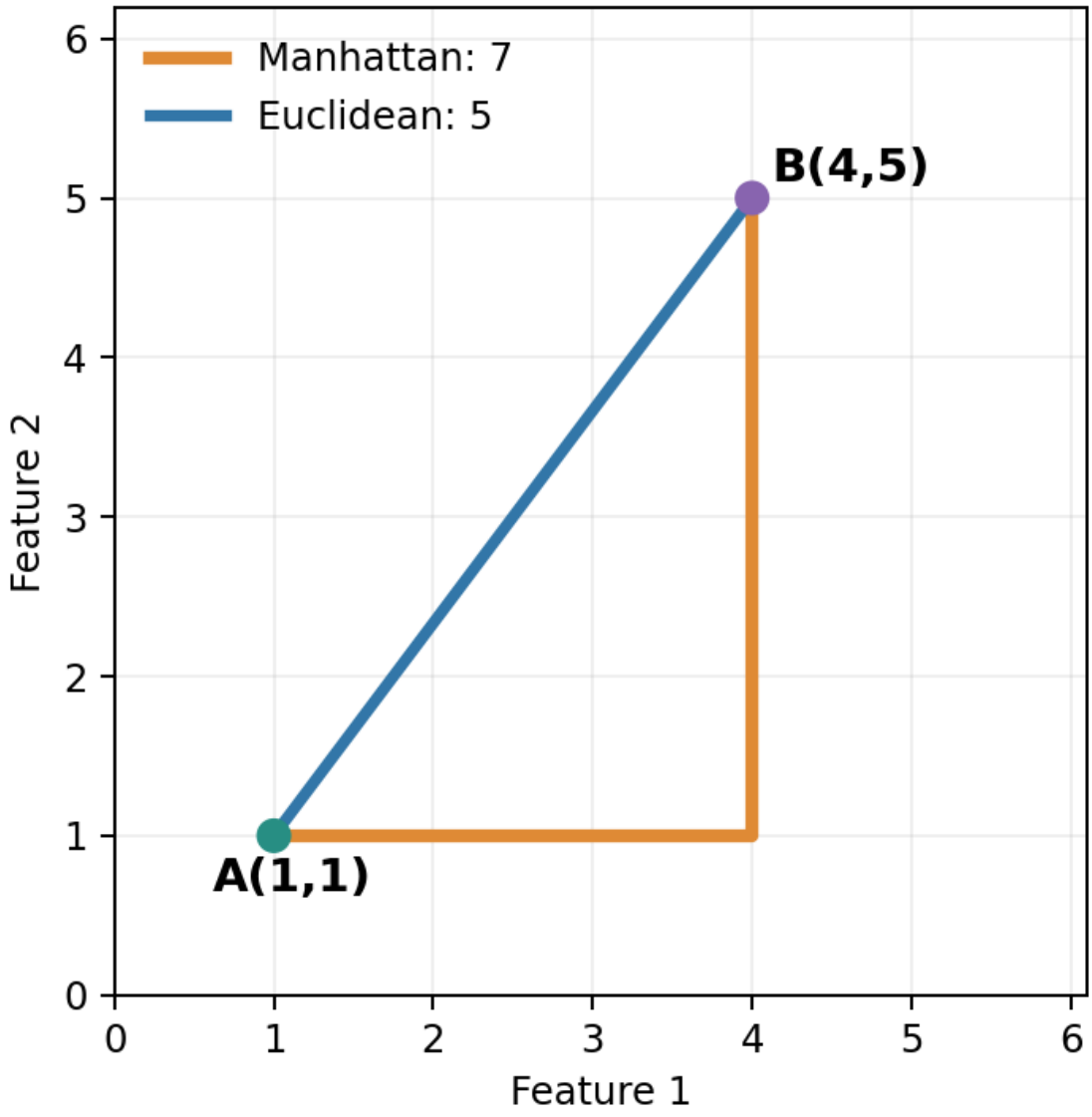
כבר כעת אפשר לזהות שני סוגים של קרבה: אורי דומה לדודו בכמויות שעות הצפייה, בעוד מיה שומרת על היחס בין מדע לספורט בדיוק. בהמשך נראה שמרחק אוקלידי יבחר במי שקרוב יותר בערכים, בעוד דמיון קוסינוס יכול להבליט דווקא את הדמיון ביחסים.

חשוב להבין שני משתמשים יכולים להיות קרובים לפי מדד אחד אך פחות דומים לפי מדד אחר. זו אינה סתירה; כל מדד עונה על שאלה אחרת.

2. מהו מרחק אוקלידי?

המרחק האוקלידי הוא אורכו של הקטע הישר המחבר שתי נקודות במרחב.

נפתח בדוגמה גיאומטרית מוכרת: שני בתים נמצאים על מפה. בית אחד מסומן בנקודה $A=(1,1)$ ובית שני בנקודה $B=(4,5)$. אם נוכל לנוע בקו ישר בין הבתים, נרצה למצוא את אורכו של הקטע שמחבר ביניהם. מכיוון שאנחנו יודעים לחשב מרחק במשולש ישר־זווית, נוכל להיעזר במשפט פיתגורס.



איור 1 — הקטע הכחול הוא המרחק האוקלידי; המסלול הכתום בנוי מתנועה לאורך הצירים.

שלב 1 — מחשבים הפרש אופקי: הקואורדינטה הראשונה משתנה מ־1 ל־4, ולכן ההפרש הוא 3.

שלב 2 — מחשבים הפרש אנכי: הקואורדינטה השנייה משתנה מ־1 ל־5, ולכן ההפרש הוא 4.

שלב 3 — מפעילים את משפט פיתגורס: מעלים כל הפרש בריבוע, מחברים את הריבועים ומוציאים שורש.

$$d_2(A,B) = \sqrt{[(4-1)^2 + (5-1)^2]}$$

$$= \sqrt{(3^2 + 4^2)} = \sqrt{25} = 5$$

קיבלנו מרחק של 5 יחידות ציר. אם כל יחידה במפה מייצגת קילומטר, המרחק הוא 5 קילומטרים. אם הצירים מייצגים שעות לימוד וציונים, לא מדובר במרחק פיזי: זו יחידה מתמטית שנגזרת מקני המידה של התכונות.

2.1 הנוסחה הכללית — שני ממדים

$$d_2(A,B) = \sqrt{[(a_1-b_1)^2 + (a_2-b_2)^2]}$$

הסימון d_2 מתייחס למרחק לפי נורמת L2. האותיות a ו־b הן רכיבי הווקטורים. מכיוון שכל הפרש מועלה בריבוע, אין חשיבות לשאלה אם מחסרים A−B או B−A: המרחק נשאר אותו מרחק. הריבוע גם מבטיח שהסכום לא יהיה שלילי.

מדוע צריך שורש? בלי השורש היינו מקבלים את ריבוע המרחק, ולא את המרחק עצמו. בריבוע המרחק משתמשים לעיתים בחישובים אחרים, אך אם השאלה דורשת L2 יש להוציא שורש.

בדיקת היגיון כשהנקודות זהות, כל הפרש הוא אפס ולכן המרחק הוא אפס. אם אחת מהנקודות מתרחקת, המרחק בדרך כלל גדל; אך גודל השינוי תלוי בכיוון.

3. מרחק מנהטן — כאשר נעים לאורך הצירים

כעת נניח שנוסעים בתוך עיר שבה הרחובות בנויים כרשת של רחובות אופקיים ואנכיים. לא תמיד אפשר לחצות בניין בקו ישר, ולכן המסלול מורכב מצעדים לאורך הצירים. מרחק מנהטן מחבר את גודלם של כל ההפרשים, בלי לשלב אותם דרך פיתגורס.

באותה דוגמה, כדי להגיע מ־$A=(1,1)$ ל־$B=(4,5)$ אפשר לנוע שלוש יחידות ימינה וארבע יחידות למעלה. אורך המסלול הכולל יהיה 7 יחידות. אם נלך קודם למעלה ואז ימינה נקבל אותו אורך; סדר הצעדים אינו משנה את הסכום.

$$d_1(A,B) = |4-1| + |5-1| = 3 + 4 = 7$$

3.1 מדוע מופיעים סימני ערך מוחלט?

אם נחליף את כיוון התנועה, הפרש הקואורדינטות עשוי להיות שלילי. למשל, מ־4 ל־1 ההפרש הוא 3−, אבל עדיין עברנו שלוש יחידות. ערך מוחלט מודד את גודל ההפרש בלי להתחשב בסימנו. לכן מרחק מנהטן לעולם אינו שלילי.

$$d_1(A,B) = |a_1-b_1| + |a_2-b_2|$$

במתמטיקה משתמשים בסימון L1 כי מדובר בסכום של הערכים המוחלטים. מרחק אוקלידי נקרא L2 מפני שהוא מבוסס על ריבועי ההפרשים ושורש סכומם.

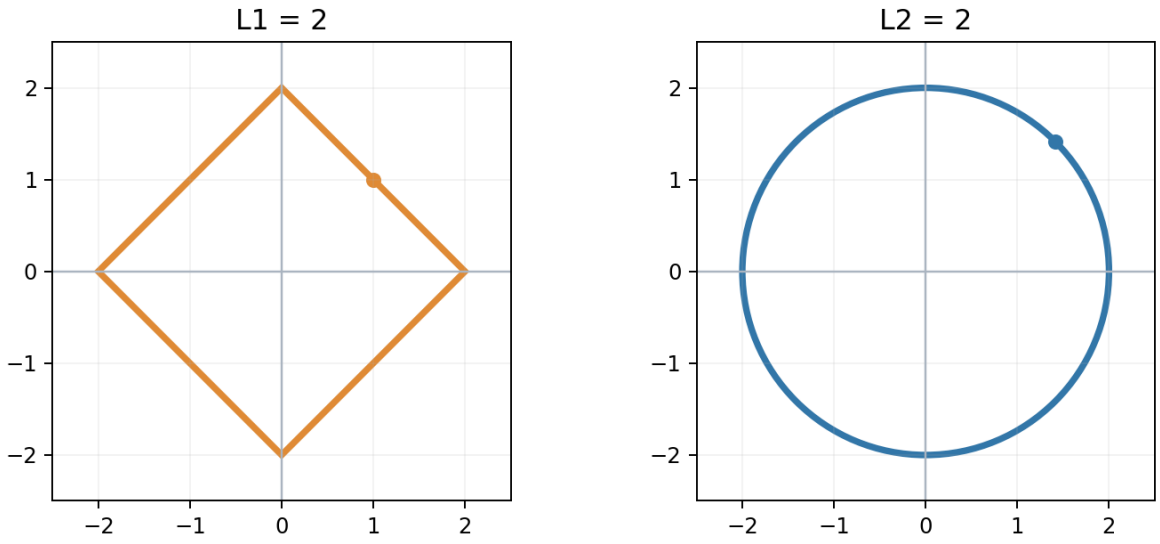
שאלת חשיבה במערכת ניווט בעיר משובצת רחובות, האם מרחק מנהטן הוא בהכרח זמן הנסיעה? לא. הוא מתאר אורך לפי מודל פשוט; פקקים, מהירות וכבישים חסומים אינם מיוצגים בחישוב.

4 . מה ההבדל האמיתי בין L1 ל־L2?

בדוגמה שלנו L1 היה 7 ואילו L2 היה 5 . האם פירוש הדבר ש־L2 טוב יותר? לא. אין מדד נכון באופן אוניברסלי. L1 מודד סכום של שינויים מוחלטים בכל רכיב. L2 מעניק משקל גדול יותר לשינויים גדולים ברכיב בודד, מפני שהם מועלים בריבוע לפני החיבור.

נבחן שתי אפשרויות לשינוי בווקטור: בהשוואה הראשונה ההפרש בין הדוגמאות הוא (4 , 0). בהשוואה השנייה ההפרש הוא (2,2). בשתי ההשוואות מתקבל L1=4. לעומת זאת, ב־L2 נקבל 4 עבור (4 , 0), אך √ 8 ≈ 2.83 עבור (2,2). כלומר, ב־L2 שינוי גדול ומרוכז ברכיב אחד נחשב משמעותי יותר משני שינויים קטנים יותר.

הפרש הרכיבים	מרחק L1	מרחק L2	מה לומדים?
(4 , 0)	4	4	כל השינוי ברכיב יחיד
(2,2)	4	2.83≈	השינוי מתחלק בין שני רכיבים
(3 , 4)	7	5	שני המדדים שונים



איור 2 — כל הנקודות על המעוין בעלות מרחק L1=2 מהמרכז; כל הנקודות על המעגל בעלות מרחק L2=2.

מעוין ומעגל הם דרך חזותית לראות שהגדרת "אותו מרחק" תלויה במדד. אנו מציגים כאן את שתי הצורות לצורך הבנה; איננו דורשים בפרק זה הוכחה גיאומטרית של הצורות.

4.1 האם L1 תמיד גדול או שווה ל־L2?

באותו מרחב ובאותם רכיבים מתקיים L1≥L2. אפשר לבדוק זאת בדוגמאות המספריות. להוכחה פורמלית יש כלים מתמטיים מתקדמים יותר, שאינם נדרשים להבנת השימוש הבסיסי במדדים כאן. חשוב יותר שהתלמיד יוכל לפרש את ההבדל לפי נתוני הבעיה.

5 . מרחק בין וקטורים במרחב תלת־ממדי וב־$\mathbb{R}^n$

עד עכשיו השתמשנו רק בשני מאפיינים מפני שקל לצייר שתי נקודות על דף. בעולם ה־AI משתמשים לעיתים בעשרות, מאות ואף אלפי מאפיינים. המחשב לא חייב לצייר את המרחב כדי למדוד בו מרחק: הוא פשוט עובר על הרכיבים המתאימים ומבצע את אותה פעולה.

$$d_1(u,v) = \sum_{i=1}^{n} |u_i - v_i|$$

$$d_2(u,v) = \sqrt{\sum_{i=1}^{n} (u_i - v_i)^2}$$

האות היוונית Σ פירושה סכום. המספר n הוא מספר הרכיבים. האינדקס i נע מהרכיב הראשון עד הרכיב האחרון. הסימן u_i מייצג את הרכיב במקום i של הווקטור u; v_i הוא הרכיב המתאים של v. זוג הרכיבים חייב לתאר את אותה תכונה.

5.1 דוגמה פתורה בתלת־ממד

ניקח שתי דוגמאות מספריות כפי שהופיעו בחומרי ההשתלמות של סעיף 1.2. לצורך החישוב אין חשיבות לשמות התכונות; ההנחה היא שכל רכיב עומד מול רכיב בעל משמעות זהה.

$$u = (-1,\ 3,\ 2)\ ;\quad v = (1,\ -4,\ -2)$$

מחסרים רכיב מול רכיב: $-1-1=-2$; $3-(-4)=7$; $2-(-2)=4$. לכן ההפרשים הם $(-2,7,4)$. כעת מבצעים שני חישובים שונים — אחד עם ערכים מוחלטים ואחד עם ריבועים.

Coordinate	u	v	u - v	Absolute	Square
1	−1	1	−2	2	4
2	3	−4	7	7	49
3	2	−2	4	4	16
Total				13	69

איור 3 — טבלת מעקב לחישוב מדורג בתלת־ממד: אותם הפרשים משמשים בשני מדדי המרחק.

$$L1 = |-2| + |7| + |4| = 13$$

$$L2 = \sqrt{(-2)^2 + 7^2 + 4^2} = \sqrt{69} \approx 8.31$$

כדאי לבדוק את התוצאה לפני פתיחת מחשב: המרחק האוקלידי צריך להיות לפחות 7 , מפני שיש כבר הפרש של 7 ברכיב השני. הוא גם לא יעלה על 13. התוצאה 1 3 . 8 עומדת בשתי הבדיקות.

5.2 בדיקות תקינות של מדד מרחק

אי־שליליות: מרחק אינו שלילי, כי הוא נבנה מסכומים של ערכים מוחלטים או ריבועים.

זהות: מרחק של וקטור מעצמו הוא אפס. בשני המדדים שלנו מרחק אפס בין שני וקטורים בעלי אותו ממד מציין שהם שווים.

סימטריה: המרחק מ־u ל־v שווה למרחק מ־v ל־u.

אי־שוויון המשולש: הדרך הישירה בין שתי נקודות אינה ארוכה מסכום שני מקטעים שעוברים דרך נקודה שלישית. אפשר להמחיש אותו על מפה; הוכחה כללית אינה נדרשת כאן.

6. כיצד מממשים את הנוסחה ב־Python?

אחרי שהבנו את החישוב ביד, נוכל לכתוב אותו בפייתון. חשוב לזכור שקוד אינו מחליף את ההסבר המתמטי. אם נריץ פונקציה בלי להבין מה היא מחשבת, יהיה קשה לזהות טעות במאפיינים, בקנה המידה או בקלט.

6.1 מימוש מדורג בלולאת for

```
u = [-1, 3, 2]
v = [1, -4, -2]

l1_sum = 0
l2_sum = 0

for i in range(len(u)):
    difference = u[i] - v[i]
    l1_sum += abs(difference)
    l2_sum += difference ** 2

l2_distance = l2_sum ** 0.5
print(l1_sum)
print(l2_distance)
```

הלולאה מבצעת שלושה צעדים, אחד לכל תכונה. בכל צעד המשתנה difference מחזיק את ההפרש בין שני הרכיבים. המונה l1_sum צובר ערכים מוחלטים, ואילו l2_sum צובר ריבועים. רק לאחר סיום הלולאה מוציאים שורש כדי לקבל מרחק אוקלידי.

שימו לב: כאשר שני הווקטורים אינם באותו אורך, הקוד הזה אינו פתרון תקין לבעיה. בפרק נשתמש בווקטורים בעלי אותו מספר רכיבים, והקלט ייבדק לפני ההשוואה ביישומים אמיתיים.

6.2 אותו חישוב באמצעות NumPy

```
import numpy as np

u = np.array([-1, 3, 2])
v = np.array([1, -4, -2])

difference = u - v
l1 = np.sum(np.abs(difference))
```

```
l2 = np.linalg.norm(difference)

print(difference)
print(l1)
print(round(l2, 2))
```

NumPy מבצעת את חיסור המערכים רכיב־רכיב, ולכן אפשר לכתוב u−v בלי לולאה מפורשת. הפקודה np.abs מחזירה את הערך המוחלט של כל רכיב, והפקודה np.sum מחברת אותם. הפקודה np.linalg.norm מחשבת כברירת מחדל את נורמת L2 של וקטור חד־ממדי.

> העיקרון שלנו חשב ביד → נבא תוצאה → כתוב קוד → בדוק שהפלט מתאים → הסבר מה משמעות התוצאה בעולם האמיתי.

7 . מרחק אינו זהה לדמיון — נחזור לדודו ולמיה

נניח שדודו צופה שלוש שעות במדע וארבע בספורט, ומיה צופה שש שעות במדע ושמונה בספורט. מיה צופה בדיוק פי שניים מדודו בכל תחום, ולכן דפוס ההעדפות שלהם זהה. ובכל זאת, מבחינת כמות הצפייה, הם אינם נמצאים באותה נקודה במרחב.

$$D = (3,4) \; ; \; M = (6,8)$$

$$d_2(D,M) = \sqrt{[(3-6)^2 + (4-8)^2]} = 5$$

כעת נוסיף את אורי: $O=(3,5)$. המרחק מדודו לאורי הוא 1 בלבד, ולכן לפי מרחק אוקלידי אורי קרוב יותר לדודו מאשר מיה. אבל דפוס החלוקה של שעות הצפייה אצל מיה מדויק יותר: יחס של 4 : 3 , בדיוק כמו אצל דודו.

הדיון הזה מוביל אותנו למדד אחר. במקום לשאול כמה רחוקות הנקודות, נשאל האם הווקטורים מצביעים לאותו כיוון. דמיון קוסינוס נועד לענות על שאלה מן הסוג הזה.

8 . מהי מכפלה פנימית ומה הקשר שלה לזווית?

בפרק 1.1 למדנו שמכפלה פנימית היא סכום מכפלות של רכיבים מתאימים. נחזור על הרעיון כיוון שהוא משמש לבניית דמיון הקוסינוס. עבור u=(a,b) ו־v=(c,d), מכפילים את שני הרכיבים הראשונים, אחר כך את שני הרכיבים השניים, ולבסוף מחברים.

$$u \cdot v = ac + bd$$

לדוגמה, עבור u=(3,4) ו־v=(6,8) נקבל $3 \cdot 4 + 6 \cdot 8 = 50$. התוצאה היא מספר יחיד, המכונה סקלר; היא אינה וקטור חדש. במרחב בעל n ממדים נחבר n מכפלות.

$$u \cdot v = \sum_{i=1}^{n} u_i v_i$$

למכפלה הפנימית יש גם משמעות גיאומטרית: היא שווה למכפלת אורכי הווקטורים בקוסינוס הזווית שביניהם. לכן, אם אנו יודעים את המכפלה הפנימית ואת האורכים, אפשר להעריך האם שני הווקטורים מצביעים בכיוון דומה.

$$u \cdot v = ||u||_2 \cdot ||v||_2 \cdot \cos(\theta)$$

הסימון θ (תטה) מייצג את הזווית בין הווקטורים כאשר משרטטים אותם מאותה נקודת מוצא. אם הזווית אפס, הווקטורים פונים לאותו כיוון. אם הזווית ישרה, אין להם רכיב משותף בכיוון. אם הזווית קהה מאוד, הם פונים לכיוונים מנוגדים יותר.

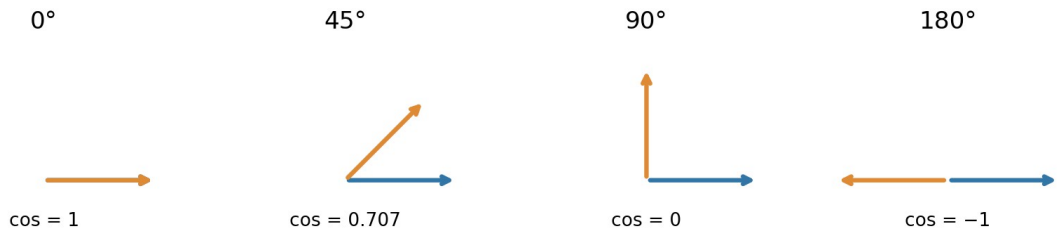
9 . דמיון קוסינוס — כיצד מפרידים כיוון מגודל?

Cosine Similarity: דמיון בין כיווני וקטורים, תוך התעלמות מהאורך שלהם כאשר מכפילים במקדם חיובי.

נבודד את קוסינוס הזווית מהמשוואה שבסעיף הקודם. מחלקים את המכפלה הפנימית במכפלת אורכי שני הווקטורים. חילוק זה מבטל את השפעת הגודל הכולל של הווקטורים, כל עוד הם אינם וקטורי אפס.

$$\cos(u,v) = (u \cdot v) / (||u||_2 \cdot ||v||_2)$$

הערך המתקבל נמצא בתחום שבין -1 ל־1 עבור וקטורים ממשיים שאינם אפס. כאשר כל המאפיינים אינם שליליים, כפי שקורה בדוגמאות של שעות צפייה או דירוגים חיוביים, נקבל ערכים בין 0 ל־1.



איור 4 — זוויות בין וקטורים והערכים האופייניים של הקוסינוס: זהות כיוון, זווית 45°, ניצבות וכיוונים הפוכים.

ערך הקוסינוס	פרשנות גיאומטרית	הערה
1	אותו כיוון בדיוק	גם אם האורכים שונים
קרוב ל־1	כיוונים דומים	אין פירושו שהתכונות זהות בערכן
0	זווית של 90°	ניצבות, בהקשר של המרחב
-1	כיוונים מנוגדים בדיוק	אפשרי עם רכיבים שליליים

9.1 חישוב ידני — אותה העדפה, יותר שעות

$$u=(3,4)\ ;\ v=(6,8)$$

מונה: המכפלה הפנימית היא 3 · 4 + 6 · 8 = 50.

מכנה — נורמת u: שורש (3² + 4²) שווה 5.

מכנה — נורמת v: שורש (8² + 6²) שווה 10.

$$\cos(u,v) = 50 / (5 \cdot 10) = 1$$

קיבלנו דמיון מרבי: שני הווקטורים מצביעים לאותו כיוון. יחד עם זאת, המרחק האוקלידי ביניהם הוא 5. לכן התוצאה אינה אומרת ששעות הצפייה זהות. היא אומרת שהיחס בין שני תחומי הצפייה זהה.

9.2 דוגמה מתוך רמת המפגש בתלת־ממד

בחומרי ההשתלמות הופיעה גם דוגמה בעלת שלושה רכיבים. נחשב אותה בהדרגה כדי לוודא שנוכל להפעיל את הנוסחה גם ללא תרשים דו־ממדי.

$$u=(2,1,-3)\ ;\ v=(-3,4,-5)$$

$$u \cdot v = (2)(-3) + (1)(4) + (-3)(-5) = 13$$

$$\|u\|_2 = \sqrt{14}\ ;\ \|v\|_2 = \sqrt{50}$$

$$\cos(u,v) = 13 / \sqrt{700} \approx 0.491$$

הקוסינוס חיובי, ולכן הזווית חדה, אך הערך אינו קרוב במיוחד ל־1. מכאן שהכיוונים מגלים דמיון חלקי בלבד. ללא הקשר למאפיינים, אי אפשר להסיק אם זו התאמה טובה לצורך מערכת מסוימת.

10. דמיון קוסינוס מול מרחק קוסינוס

לעיתים מערכת או ספרייה אינם רוצים מדד דמיון אלא מדד אי־דמיון, שבו מספר קטן מעיד על קרבה גדולה יותר. דרך נפוצה להפוך דמיון קוסינוס לאי־דמיון היא לחסר אותו מ־1. למדד הזה קוראים Cosine Distance, אף שחשוב לזכור שאינו מקיים תמיד את כל תכונות המרחק המתמטי.

$$\text{cosine_distance}(u,v) = 1 - \text{cosine_similarity}(u,v)$$

דמיון קוסינוס	מרחק קוסינוס	משמעות
1	0	כיוון זהה
0.8	0.2	דמיון בכיוון גבוה
0	1	כיוונים ניצבים
−1	2	כיוונים הפוכים

השם "מרחק קוסינוס" נפוץ בשימוש מעשי, אך זה אינו אותו מרחק אוקלידי, והוא אינו בהכרח מטריקה המקיימת את אי־שוויון המשולש. להעמקה: שלושה וקטורים בזוויות 0°, 60° ו־120° נותנים מרחק קוסינוס של 0.5, 0.5 ו־1.5, ולכן המעבר דרך הווקטור האמצעי עשוי להניב סכום קטן מן המרחק הישיר. זו הרחבה מתמטית ולא דרישת בסיס לתלמיד מתחיל.

10.1 מתי אי אפשר לחשב דמיון קוסינוס?

אם אחד הווקטורים הוא וקטור אפס, אורכו שווה לאפס. מכיוון שהנוסחה מחלקת במכפלת האורכים, המכנה יהיה אפס והחישוב אינו מוגדר. אי אפשר להחליט אוטומטית שווקטור אפס דומה או לא דומה: יש לבחור כיצד המערכת מטפלת במצב כזה בהתאם למשמעות הנתונים.

אזהרת קלט אם למשתמש חדש אין עדיין העדפות, וקטור האפס יכול לייצג "אין מידע" ולא "אינו אוהב שום דבר". מערכת המלצות חייבת להבחין בין האפשרויות.

11. דוגמה יישומית מלאה — המלצת סרטים

דודו רוצה להמליץ על סרטים לפי תחומי עניין: פעולה, רומנטיקה וקומדיה. כל משתמש מיוצג בשלושה מספרים, וכל סרט מקבל שלושה מספרים באותו סדר. הערכים הם ציונים מומצאים לצורך התנסות, לא נתוני משתמשים אמיתיים. הבחירה בסדר קבוע חיונית: אם נחליף בין עמודות, המכפלה הפנימית תאבד את משמעותה.

סוג	פעולה	רומנטיקה	קומדיה
משתמש 1	4	1	1
משתמש 2	1	3	3
סרט A	5	1	1
סרט B	1	4	4
סרט C	3	3	3

נחשב ציון התאמה באמצעות מכפלה פנימית. שימו לב: ציון כזה משלב גם את גודל וקטור המשתמש והסרט, ולכן אינו מדד דמיון טהור של כיוון.

$$\text{score}(u, \text{film}) = u \cdot \text{film}$$

11.1 דירוג עבור משתמש 1

A: סרט A: $4\cdot5 + 1\cdot1 + 1\cdot1 = 22$

B: סרט B: $4\cdot1 + 1\cdot4 + 1\cdot4 = 12$

C: סרט C: $4\cdot3 + 1\cdot3 + 1\cdot3 = 18$

על פי המכפלה הפנימית, מערכת ההמלצות תדרג את A ראשון, אחריו C ולבסוף B. זה הגיוני משום שמשתמש 1 מתעניין מאוד בסרטי פעולה, וסרט A מקבל ציון גבוה ביותר בתחום זה.

11.2 דירוג עבור משתמש 2

A: סרט A: $1\cdot5 + 3\cdot1 + 3\cdot1 = 11$

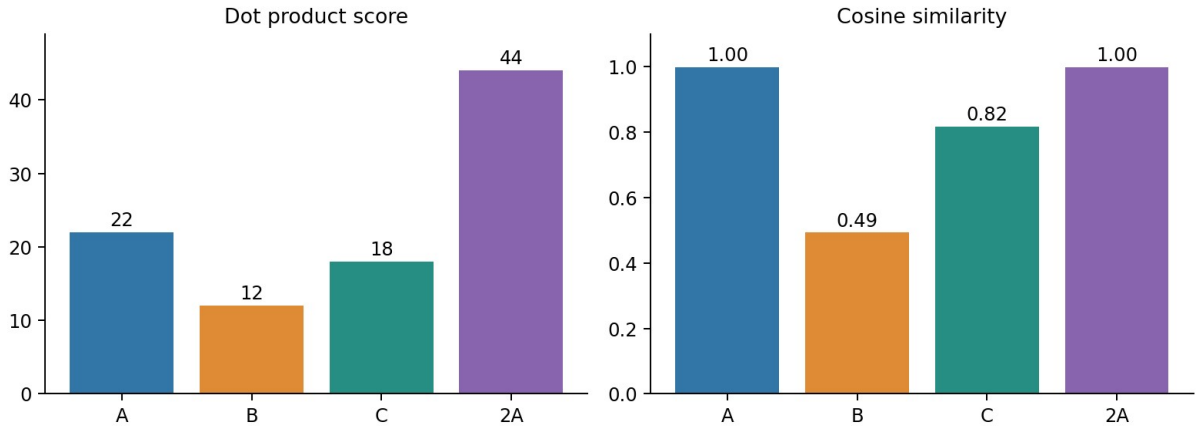
סרט B: $1\cdot1 + 3\cdot4 + 3\cdot4 = 25$.

סרט C: $1\cdot3 + 3\cdot3 + 3\cdot3 = 21$.

כאן סרט B מדורג ראשון, מפני שהמשתמש השני מעדיף רומנטיקה וקומדיה. עד כה קיבלנו מערכת המלצה פשוטה, ללא אימון של מודל. בהמשך הספר נלמד מתי מערכת כזאת מספיקה ומתי יש צורך במודל שנלמד מנתונים.

11.3 מה קורה אם מגדילים את כל רכיבי הסרט?

ניצור סרט נוסף $2A = (10, 2, 2)$, כלומר נכפיל את כל רכיבי A בשתיים. המכפלה הפנימית עם משתמש 1 תגדל מ־22 ל־ 4 4 . אולם דמיון הקוסינוס יישאר בדיוק כפי שהיה לסרט A, משום שהכיוון אינו משתנה כאשר מכפילים וקטור במקדם חיובי.



איור 5 — דירוגים לפי מכפלה פנימית מול דמיון קוסינוס. 2A הוא אותו פרופיל של A בכמות כפולה.

האיור מדגים מדוע חשוב לדעת מה המודל מודד. אם המטרה היא לזהות התאמה בפרופורציות של תחומי עניין, קוסינוס עשוי להתאים יותר. אם לגודל הציונים יש משמעות אמיתית, למכפלה הפנימית יש יתרון אפשרי. אין להסיק ש־2A הוא סרט פופולרי יותר; לא מדדנו כאן פופולריות.

11.4 קוד NumPy למערכת ההמלצות

```
import numpy as np

user = np.array([4, 1, 1], dtype=float)
movies = {
    "A": np.array([5, 1, 1], dtype=float),
    "B": np.array([1, 4, 4], dtype=float),
    "C": np.array([3, 3, 3], dtype=float)
}

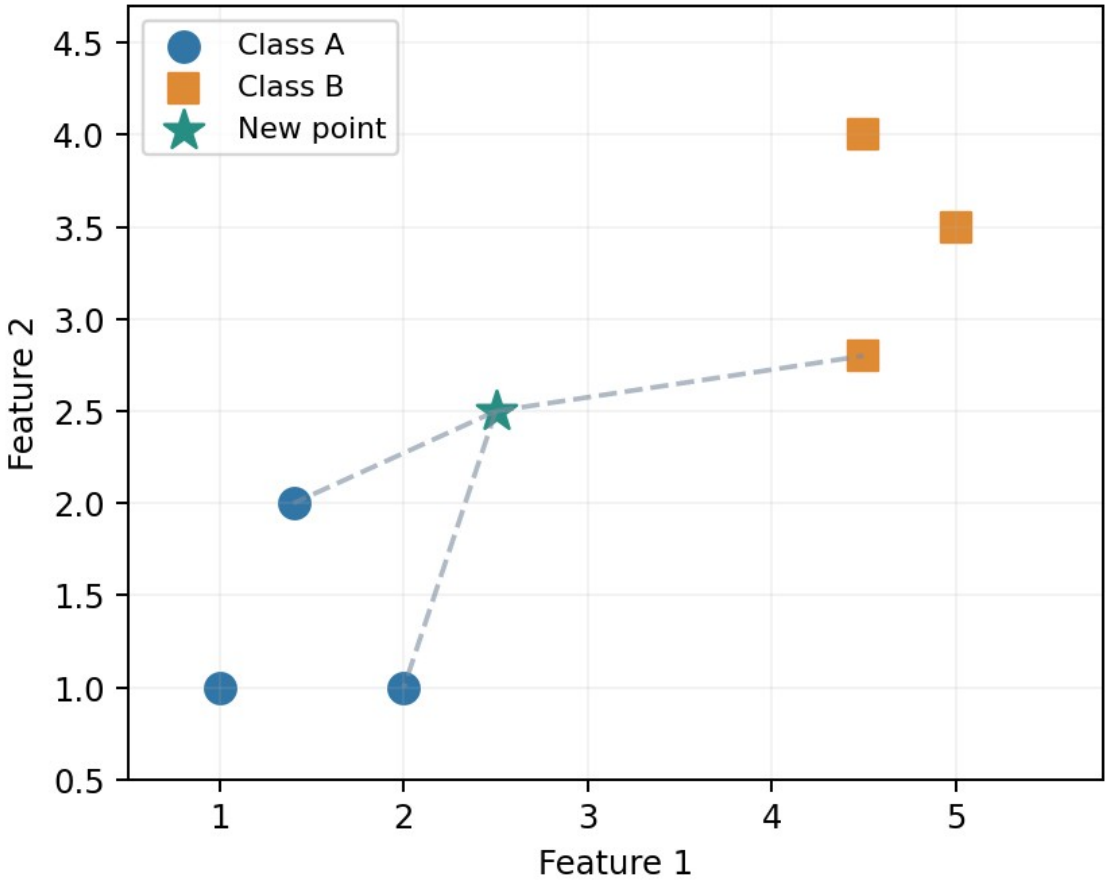
for name, film in movies.items():
    dot_score = np.dot(user, film)
    cosine = dot_score / (
        np.linalg.norm(user) * np.linalg.norm(film)
    )
    print(name, round(dot_score, 2), round(cosine, 3))
```

כאן השתמשנו במילון כדי לשמור שם ווקטור לכל סרט. הלולאה עוברת על הסרטים, מחשבת מכפלה פנימית, מחשבת דמיון קוסינוס ומדפיסה את שתי התוצאות בנפרד. זו הדגמה עם קלטים קטנים; פרויקט אמיתי יצטרך גם מנגנון איסוף העדפות, טיפול בחוסרים והערכת טיב המלצות.

1.2.2 כיצד מתייחסים לדמיון בתוך KNN?

KNN, ראשי תיבות של K-Nearest Neighbors, הוא אלגוריתם שמסווג דוגמה חדשה לפי השכנים הקרובים אליה בנתוני אימון. בשלב זה לא נפתח עדיין את כל האלגוריתם. נתמקד רק בהבנה שמאחורי המילה "קרובים": צריך לבחור כיצד מודדים מרחק בין שתי דוגמאות.

דמיינו נתונים של שני סוגי משחקים, שכל משחק מתואר על ידי שני מאפיינים מספריים. נקודה חדשה מגיעה למערכת. אם נבחר $k=3$, נמדוד את המרחק מהנקודה החדשה לדוגמאות האימון, נמיין את המרחקים ונבחן את שלושת השכנים הקרובים. כאשר מדובר בסיווג, אפשר לבחור במחלקה הנפוצה בקרב השכנים לפי כלל הכרעה שהוגדר מראש.



איור 6 — נקודה חדשה (כוכב) וקווי מרחק לשלושה שכנים; המחלקות הן דוגמה מומצאת להמחשה.

לפי האיור, השכנים הקרובים ביותר אינם בהכרח כל הדוגמאות השייכות למחלקה שנראית גדולה יותר על הדף. יש למדוד ולא רק להסתכל. אם נשנה את המדד, או את סולם התכונות, ייתכן שסדר השכנים ישתנה. לכן בחירת מדד מרחק היא גם החלטה מקצועית.

גשר ליחידה 3 ביחידה 3 פגשנו את KNN בדרך יישומית. כעת אנחנו מבינים לעומק מה נעשה

בשלב המרחק, וכיצד בחירת הנוסחה משנה את פעולת האלגוריתם.

1.3 מדוע קנה המידה של התכונות חשוב כל כך?

נבחן דוגמה של שני מאפיינים: מספר שעות תרגול בשבוע וניקוד מצטבר במשחק. מאפיין אחד נע בטווח קטן, והאחר נע בטווח של אלפים. אם נחסר את שניהם בלי להתחשב ביחידות, ההפרש בניקוד עלול לשלוט בחישוב המרחק, גם אם השאלה המקצועית שלנו נותנת חשיבות רבה לשעות התרגול.

ניקח נקודת שאילתה N=(3,1000) ושתי נקודות קיימות P=(1,1000), Q=(4,1005). תחילה נחשב מרחקים ביחידות המקוריות.

$$d_2(N,P) = \sqrt{[(3-1)^2+(1000-1000)^2]} = 2$$

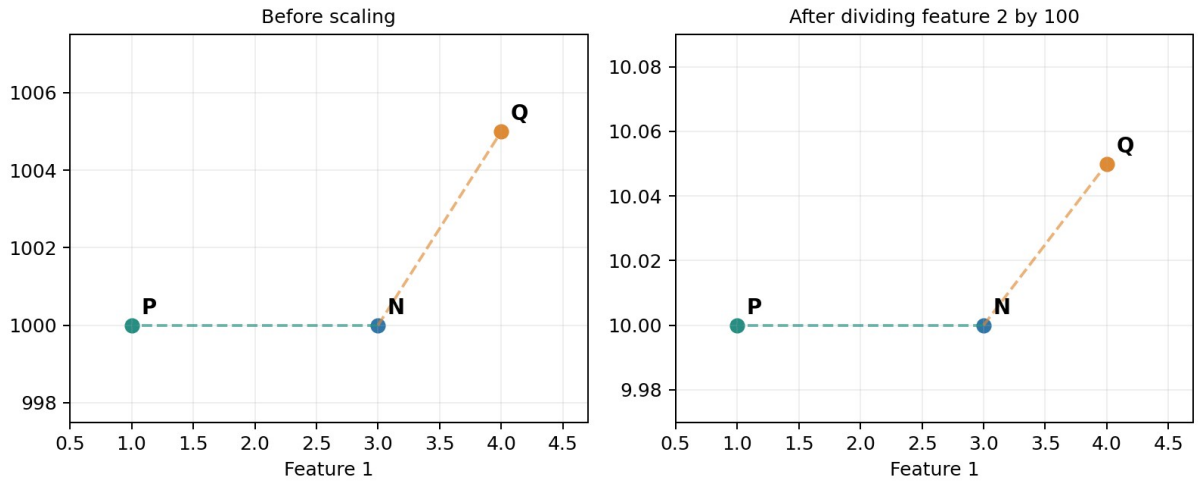
$$d_2(N,Q) = \sqrt{[(3-4)^2+(1000-1005)^2]} = \sqrt{26} \approx 5.10$$

לפי המדידה המקורית, P קרובה יותר. עכשיו נציג לצורך המחשה את המאפיין השני ביחידות של מאות נקודות, כלומר נחלק כל ערך שלו ב־100. זוהי המרת יחידות שהומצאה לצורך ההסבר; אין להניח שחלוקה ב־100 היא שיטת תקנון מומלצת לכל מאגר.

$$N' = (3,10) \; ; \; P' = (1,10) \; ; \; Q' = (4,10.05)$$

$$d_2(N',P') = 2 \; ; \; d_2(N',Q') = \sqrt{1.0025} \approx 1.001$$

כעת Q קרובה יותר! אף נקודה לא "השתנתה בעולם". שינינו רק את קנה המידה שבו תיארנו את מאפיין הניקוד, וכתוצאה מכך שינינו את סדר השכנים. המשמעות היא שהבחירה בייצוג המספרי משפיעה על החלטות של אלגוריתמים מבוססי מרחק.



איור 7 — אותו מידע לפני ואחרי המרת יחידות של תכונה אחת. סדר הקרבה יכול להשתנות.

13.1 מה נלמד בהמשך על תקנון?

בפרק הסטטיסטיקה והכנת הנתונים נלמד Standardization ו־Normalization בצורה מלאה. ביישום של למידת מכונה עם חלוקת Train/Test, מחשבים את הפרמטרים של התקנון רק על נתוני האימון, ואז משתמשים באותם פרמטרים להמרת נתוני הבדיקה. התאמת התקנון על כל הנתונים לפני החלוקה עלולה לגרום לדליפת מידע (Data Leakage).

זכרו המטרה של תקנון אינה להפוך כל שתי תכונות לחשובות באותה מידה באופן אוטומטי. צריך להבין את משמעות הנתונים ואת השאלה שהמודל אמור לפתור.

14. השוואה מסכמת — מתי להשתמש במה?

כלי	מה מחשבים?	מתי שימושי?	מה לא להסיק?
L1	סכום ערכים מוחלטים	השוואת סך שינויים ברכיבים	שאורך דרך אמיתי תמיד שווה ל־L1
L2	שורש סכום ריבועים	קרבה גיאומטרית, KNN	שכל המאפיינים בעלי קנה מידה מתאים
Dot Product	סכום מכפלות רכיבים	ציון משוקלל, מודל ליניארי	שהציון מבוסס רק על כיוון
Cosine Similarity	מכפלה פנימית מחולקת בנורמות	השוואת דפוסי העדפות	שהווקטורים שווים בגודלם
Cosine Distance	1 פחות דמיון קוסינוס	דירוג אי־דמיון בכיוון	שהוא בהכרח מטריקה מתמטית

כלי המתמטיקה האלה שונים, אך משלימים זה את זה. הבחירה בהם אינה עניין של שינון נוסחה בלבד. קודם צריך להחליט מהו פירוש המילה "דומה" עבור הבעיה שאנחנו מנסים לפתור. בהמלצות על סרטים ייתכן שחשוב דפוס העדפות; בסיווג רפואי ייתכן שקרבה של מדידות מסוימות חשובה יותר, אך אסור לבחור מדד בלי להבין את הקשר והמשמעות של כל תכונה.

14.1 אילו טעויות נרצה למנוע?

חיבור רכיבים שלא באותו סדר: אם במשתמש מדע נמצא בעמודה הראשונה ובסרט ספורט בעמודה הראשונה, התוצאה אינה משווה תכונות תואמות.

שכחת השורש ב־**L2**: כך מקבלים מרחק בריבוע; זה לא המספר שהתבקשתם לחשב.

בלבול בין כפל רכיבי למכפלה פנימית: u*v ב־NumPy מחזיר וקטור רכיבי; np.dot או @ מחזירים סקלר עבור שני וקטורים חד־ממדיים.

חישוב קוסינוס עם וקטור אפס: המכנה אפס; הנוסחה אינה מוגדרת.

התעלמות מקנה מידה: תכונה בעלת ערכים גדולים יכולה לשלוט במרחק.

מסקנה חזקה מדי: דמיון וקטורי גבוה לא מוכיח דמיון בין בני אדם, סיבתיות או הצלחה של מודל בעולם האמיתי.

1.5 התנסות ב־Excel — מחשבים ידנית בגיליון

בחומרי ההשתלמות של פרק 1.2 נכללה גם התנסות בExcel. הגיליון עוזר לראות שכל נוסחה מתייחסת לתאים ולרכיבים מתאימים, ולא רק למספר אחד בסוף החישוב. אפשר להקליד שתי רשומות בגיליון ולאחר מכן להשוות לתוצאה מפייתון.

עמודה	תוכן
A	מספר תכונה (1,2,3)
B	רכיבי הווקטור u
C	רכיבי הווקטור v
D	הפרש: =B2-C2
E	ערך מוחלט: =ABS(D2)
F	ריבוע ההפרש: =D2^2
G	מכפלה: =B2*C2

נעתיק את הנוסחאות לשורות 2–4. סכום העמודה E יחזיר L1; השורש של סכום עמודה F יחזיר L2; סכום עמודה G יחזיר מכפלה פנימית. עבור דמיון הקוסינוס נצטרך לחשב בנפרד גם את נורמת u וגם את נורמת v.

תרגול עצמי: צרו גיליון שבו 10 וקטורים מושווים ל־ref_vec קבוע בן שלושה רכיבים. עבור כל אחד חשבו L1, L2 וקוסינוס. הסבירו אם סדר המועמדים זהה בשלושת המדדים.

1.6 מעבדה מודרכת — קולאב: מימוש שתי גישות

מטרת המעבדה היא להבין מה מבצעת כל שורת קוד, ולא רק לקבל פלט נכון.

ראשית נוודא שאנחנו יודעים לחשב את ההפרש בין שני וקטורים, את L1, את L2 ואת המכפלה הפנימית. לאחר מכן נחשב דמיון קוסינוס רק אם שתי הנורמות שונות מאפס. הקוד הבא הוא הדגמת מורה פתורה; המעבדה העצמאית בהמשך אינה כוללת פתרונות.

```
import numpy as np

u = np.array([2., 1., -3.])
v = np.array([-3., 4., -5.])

difference = u - v
l1 = np.abs(difference).sum()
l2 = np.linalg.norm(difference)
dot = np.dot(u, v)
```

```
norm_u = np.linalg.norm(u)
norm_v = np.linalg.norm(v)
if norm_u == 0 or norm_v == 0:
    print("Cosine is undefined for a zero vector")
else:
    cosine = dot / (norm_u * norm_v)
    print(l1, l2, dot, cosine)
```

אתם אמורים לקבל מכפלה פנימית 13 ודמיון קוסינוס בקירוב 1 9 4 0. מרחק L1 ו־L2 בדוגמה זו שונים מן הדוגמה הקודמת, כי הווקטורים שונים. אם הפלט אינו תואם לחישוב הידני שלכם, עברו על סדר הרכיבים וודאו שהנוסחה מקבלת את המספרים הנכונים.

16.1 חקר קצר — מה ישתנה אחרי הכפלה?

הכפילו את הווקטור v במספר חיובי, למשל 3. לפני הרצה נסו לנבא אילו גדלים ישתנו ואילו יישארו זהים. בדקו את המכפלה הפנימית, את הנורמות, את דמיון הקוסינוס ואת המרחק האוקלידי. כעת נסו להכפיל במספר שלילי ולתאר מה קורה לכיוון הווקטור.

הערה חשובה כפל שני וקטורים באותו מספר חיובי שומר על דמיון הקוסינוס ביניהם, אך בדרך כלל משנה את המרחק האוקלידי באותו יחס. כפל וקטור אחד במקדם שלילי הופך את הכיוון שלו ומשנה את סימן הקוסינוס.

17. תרגול עצמאי — רמה א׳: הבנת המושגים

1. הסבירו במילים שלכם מדוע מרחק אוקלידי ומרחק מנהטן עשויים לתת תוצאות שונות לאותו זוג נקודות.

2. נתונים $A=(0,0)$, $B=(3,4)$. חשבו את L1 ואת L2. הציגו כל שלב ויחידה מתאימה.

3. כתבו דוגמה לשני וקטורים שונים בעלי דמיון קוסינוס 1, והסבירו כיצד ייתכן שהם שונים.

4. הסבירו מדוע דמיון קוסינוס אינו מוגדר עבור וקטור אפס.

5. תלמיד טוען: ״אם שני וקטורים קרובים בקוסינוס, אז בוודאות המרחק האוקלידי ביניהם קטן״. האם הוא צודק? הציגו נימוק או דוגמה נגדית.

6. מה תפקיד סימן הערך המוחלט במרחק מנהטן? מה תפקיד הריבוע במרחק האוקלידי?

18. תרגול עצמאי — רמה ב׳: חישובים והסקת מסקנות

7. נתונים $u=(2,-1,4)$, $v=(-1,3,4)$. חשבו את הפרש הווקטורים, L1, L2 ואת המכפלה הפנימית.

8. נתונים $a=(1,1)$, $b=(2,2)$, $c=(1,-1)$. חשבו את דמיון הקוסינוס של a עם b ושל a עם c; הסבירו את הערכים על פי הזווית.

9. הוכיחו באמצעות חישוב שדמיון הקוסינוס בין (2,3) ל־(6 , 4) שווה 1, אף על פי שהמרחק האוקלידי ביניהם אינו אפס.

10. משתמש מיוצג באמצעות (2 , 1 , 3) ושני סרטים באמצעות (1 , 1 , 4) ו־(1,4,4). דרגו לפי מכפלה פנימית, ואז בדקו אם הדירוג נשאר לפי דמיון קוסינוס.

11. מצאו זוג וקטורים דו־ממדיים שמרחק מנהטן ביניהם הוא 6 והמרחק האוקלידי שונה מ־ 6 . הסבירו מה בחרתם.

12. בנו טבלת מעקב לחישוב L2 עם שלושה רכיבים. בכל שורה כתבו הפרש, ריבוע הפרש, סכום מצטבר ותוצאת הסיום.

19 . תרגול עצמאי — רמה ג׳: שאלות חקר

13. צרו שני מאגרי נתונים קטנים עם שתי תכונות. באיזה מאגר המרחק האוקלידי יושפע במיוחד מתכונה אחת? הציגו דוגמה מספרית והסבירו.

14. כתבו פונקציית Python שמקבלת שני וקטורים בני אותו אורך ומחזירה מרחק מנהטן. אל תשתמשו ב־np.linalg.norm. השוו ל־NumPy.

15. בחרו שלושה משתמשים ושלושה פריטי תוכן. החליטו אילו מאפיינים נמדוד, הסבירו למה בחרתם בהם והשוו דירוג לפי Dot Product ולפי Cosine Similarity.

16. ב־KNN, מדוע לא נכון להחליט מהו k המיטבי מתוך תוצאות הבדיקה הסופית? כתבו תשובה המבדילה בין אימון, ולידציה ובדיקה.

17. אילו הטיות עלולות להיווצר אם מערכת ממליצה על סרטים רק לפי מספר צפיות, בלי לשאול מה המשתמש אהב בפועל?

18. בחרו דוגמה שבה דמיון קוסינוס אינו מתאים להחלטה הרצויה, והציגו מדד חלופי עם נימוק.

20 . שאלות מקוריות בסגנון בגרות

השאלות הבאות נכתבו לצורכי תרגול. הן אינן שאלות מועתקות מבחינות הבגרות ואינן מעידות לבדן על נוסח מחייב של הבחינה.

שאלה 1 — מדידת דמיון בין תלמידים

מערכת AI משווה תלמידים לפי מספר שעות תרגול במדעים, בתכנות ובמתמטיקה. התלמיד דודו מיוצג באמצעות (2,4,3), והתלמידה מיה מיוצגת באמצעות (6 , 8 , 4).

א. כתבו את הנוסחה הכללית למרחק אוקלידי במרחב בעל שלושה רכיבים. ב. חשבו את המרחק בין התלמידים. ג. חשבו את דמיון הקוסינוס. ד. הסבירו מדוע הממצאים שהתקבלו אינם סותרים זה את זה. ה. הציגו מגבלה של השוואת תלמידים על בסיס שלושת המאפיינים בלבד.

שאלה 2 — הבנת קוד וניתוח שגיאה

נתונה תכנית המחשבת דמיון בין משתמש לסרט. הוכנס קוד שבו הוחלף בטעות np.dot(u,v) בביטוי u*v, ובמקום לחבר את הרכיבים התלמיד מדפיס את התוצאה כפי שהיא.

א. הסבירו מה יחזיר הביטוי u*v עבור מערכי NumPy חד־ממדיים. ב. כתבו כיצד יש לתקן את המונה של נוסחת הקוסינוס. ג. הסבירו באיזה מצב ייכשל החישוב גם לאחר התיקון. ד. מה יקרה לדמיון הקוסינוס אם נגדיל את כל דירוגי הסרט פי שלושה? נמקו מתמטית.

שאלה 3 — שינוי סדר השכנים

למודל KNN ניתנו שתי תכונות מספריות. לאחר המרת יחידות של התכונה השנייה סדר שלושת השכנים הקרובים השתנה. א. האם הדבר אפשרי? ב. הסבירו באמצעות נוסחת L2 כיצד זה קורה. ג. הציעו דרך לבדוק את השפעת השינוי על ביצועי המודל בלי להשתמש בנתוני הבדיקה הסופיים לבחירת ההגדרה.

1 2 . מעבדת תלמיד — קוד להשלמה

המעבדה הבאה מיועדת לתלמיד. סימני שאלה מסמנים מקום להשלמת קוד, כפי שמקובל במחברות שלנו. אין להסיר אותם כאשר מכינים מחברת עצמאית לתלמיד. הפתרונות נשמרים בנפרד בגרסת המורה.

```
import numpy as np

u = np.array([3., 1., 2.])
v = np.array([1., 4., 4.])

# Task 1: elementwise difference
difference = ???

# Task 2: Manhattan distance
l1 = ???

# Task 3: Euclidean distance
l2 = ???

# Task 4: dot product
dot = ???

# Task 5: cosine similarity
cosine = ???

print(l1, l2, dot, cosine)
```

לפני השלמת הקוד חשבו את התוצאות על דף. לאחר מכן מלאו את החסר והריצו את התכנית. אם ההשוואה תיכשל, הסבירו תחילה מה עלול להיות לא תקין במקום לשנות את הקוד באקראי.

משימת הרחבה: החליפו אחד הווקטורים ב־(0 , 0 , 0). כיצד תזהו את הבעיה ותדפיסו הודעת שגיאה מובנת בלי לבצע חלוקה באפס?

משימת חקר: הגדירו סרט D חדש עם אותו כיוון כמו v אך גודל כפול. השוו את Dot Product ואת Cosine Similarity ונמקו את ההבדל.

2.2 כרטיס יציאה וסיכום מושגים

כדי לוודא הבנה, נסו לענות בכתב על השאלות הקצרות הבאות בלי לחזור לדוגמאות הפתורות:

- מהי השאלה שמרחק L2 עונה עליה?
- מה מחשב מרחק L1?
- מדוע מכפלה פנימית אינה מדד דמיון שמבטל גודל?
- מה פירוש דמיון קוסינוס השווה 1?
- מדוע שינוי קנה מידה עשוי לשנות את שכני KNN?
- מתי נוסחת דמיון קוסינוס אינה מוגדרת?

מושג	הגדרה קצרה	מילת מפתח
מרחק אוקלידי	שורש סכום ריבועי ההפרשים	L2
מרחק מנהטן	סכום ערכי ההפרשים המוחלטים	L1
מכפלה פנימית	סכום מכפלות רכיבים מתאימים	Dot
נורמה	מדד גודל של וקטור	Norm
דמיון קוסינוס	מדד של דמיון בכיוון	Cosine
מרחק קוסינוס	1 פחות דמיון קוסינוס	$\cos - 1$
קנה מידה	גודל מספרי ויחידות של מאפיין	Scale
שכן קרוב	דוגמת אימון במרחק קטן	KNN

2.3 חיבור הפרק להמשך הספר

הכלים שלמדנו אינם עומדים לבדם. כשנגיע ל־KNN נשתמש במרחק כדי לבחור שכנים. ב־Nearest Centroid נשווה דוגמאות למרכזי מחלקות. ב־K-Means נשתמש במרחקים כדי לשייך נקודות למרכזים ולעדכן אותם. בהמשך, ביישומים של למידה עמוקה, מודלים מסוימים יפיקו ייצוגים מספריים (Embeddings), ואז נוכל להשתמש בדמיון קוסינוס כדי להשוות בין הייצוגים. לכן כדאי להבין היטב כבר עכשיו מה כל נוסחה מודדת.

בהמשך, כאשר נשווה ביצועי מודלים, נלמד שדמיון בין וקטורים אינו מדד יחיד לאיכות מערכת. מערכת המלצות זקוקה לבדיקת איכות מול משתמשים או נתוני הערכה מתאימים, ומודל סיווג זקוק למדדים כגון Precision ו־Recall בהתאם לבעיה. הנוסחאות שבפרק הן כלי עבודה בתוך תהליך גדול יותר.

התמונה הגדולה נתונים → ייצוג כווקטורים → בחירת מדד מרחק או דמיון → אלגוריתם שמקבל החלטה → הערכה של ההחלטה. שינוי בכל אחד משלבי התהליך יכול לשנות את תוצאת המערכת.

2.4 מקורות ועקרונות העריכה

הפרק נכתב באופן מקורי בעברית על פי הרצף ומושגי סעיף 1.2 בחומרי ההשתלמות ובמפת נושאי הליבה. דוגמאות ה־L1/L2 בתלת־ממד ומערכת המלצת הסרטים משתמשות בערכים המופיעים בסיכומי ההשתלמות, בצירוף ניסוח, פירוק חישובי, המחשות וקוד חדשים. התרגילים, האיורים וההסברים האחרים הם עיבוד מקורי לצורכי הוראה.

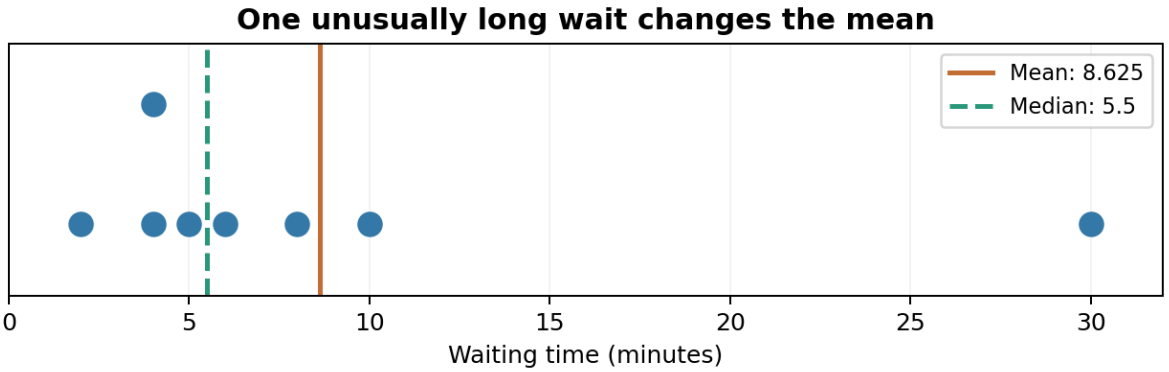
- משרד החינוך — נושאי ליבה בבינה מלאכותית, סעיפים 1.1–1.2, בכפוף לאימות הגרסה העדכנית. מפת המאגר: 02 - Unit5-AI-Core/01-Linear-Algebra/README.md.
- חומרי השתלמות וסיכום סעיף 1.2 במאגר: 02 - Unit5-AI-Core/02-Mathematical-Foundations/Vector-Distance-Similarity/.
- Deisenroth, Faisal & Ong — Mathematics for Machine Learning: הבסיס המתמטי למרחבים, נורמות ומכפלה פנימית. https://mml-book.github.io/
- Serrano — Grokking Machine Learning: השראה להסבר אינטואיטיבי ולדוגמאות מדורגות.
- Starmer — The StatQuest Illustrated Guide to Machine Learning: השראה לפירוק חזותי של רעיונות.
- Géron — Hands-On Machine Learning with Scikit-Learn and PyTorch (2025): הקשר לבחירת מודל, הכנת נתונים והערכתו.
- Zhang et al. — Dive into Deep Learning: העיקרון של חיבור הסבר מתמטי, קוד וניסוי. https://d2l.ai/.

אין להציג את תרגילי הפרק כשאלות בגרות מקוריות. ההתאמה המחייבת לבגרות ולמחוון הפרויקט תיבדק מול מסמכי משרד החינוך התקפים בעת הכנת המהדורה הסופית של הספר המאוחד.

סדרת ספרי מדעי הבינה המלאכותית

פרק 2.1 — מדדי מרכז ופיזור

כיצד מספרים מעטים יכולים לתאר מאגר נתונים שלם — ומה הם עלולים להסתיר?



איור — ממוצע וחציון של זמני המתנה: אותו מאגר, שתי נקודות מבט שונות.

ספר תלמיד מורחב • עברית מדורגת • נוסחאות וחישובים ידניים • Python ו־Colab • חיבור ל־AI

שיוך ומעמד

יחידה 5 — נושאי ליבה בבינה מלאכותית, פרק 2.1: מדדי מרכוז ופיזור. חומר ההשתלמות הקיים במאגר הוא עוגן למיפוי. שילוב הדוגמאות, הגרפים וניסויי ה־AI הוא עיבוד הוראתי מקורי; יש לאמת את דרישות הבחינה במפרט התקף לשנת הבחינה.

מהדורת תלמיד: התרגולים העצמאיים אינם כוללים פתרונות. החישובים המלאים בפרק הם דוגמאות לימודיות, ולא פתרונות למשימות התלמיד.

לפני שמתחילים — מה נדע לעשות בסוף הפרק?

נלמד להפוך רשימת מדידות לסיפור מספרי שאפשר להסביר. נבחין בין התוצאה שהמחשב חישב לבין המשמעות שלה בעולם האמיתי. חשוב לא פחות: נזהה מתי מדד שנראה מרשים עלול להטעות אותנו.

- לזהות תצפית, משתנה, ערך, מדגם ואוכלוסייה, ולהבחין בין משתנה מספרי למשתנה קטגוריאלי.
- לבנות טבלת שכיחויות ולהסביר הבדל בין רשימת הערכים לבין תדירות הופעתם.
- לחשב ולהסביר ממוצע, חציון ושכיח — ולבחור מדד מתאים בהתאם לשאלה ולצורת הנתונים.
- לחשב טווח, רבעונים, IQR, שונות וסטיית תקן; לפרש את התוצאה ולא רק לבצע חשבון.
- להבין הבדל בין שונות אוכלוסייה לשונות מדגם ובין הפרמטרים ddof=0 ו־ddof=1.
- להשתמש ב־NumPy, Pandas ובגרפים לאימות חישובים ולהסקת מסקנות.
- להסביר כיצד המדדים משמשים בתיאור נתונים, בתקנון, בזיהוי חריגים ובהכנת נתונים למודל.

ידע קודם קצר

נניח שאתם מכירים רשימות בפייתון, אינדקסים, חיבור וחילוק, חיסור מספרים שליליים, שורש ריבועי ואת מושג הווקטור מפרק 1.1. אם שורש ריבועי או חזקה אינם מוכרים, חזרו לדוגמה: $4^2=16$ ו־$\sqrt{16}=4$.

בדיקת כניסה

ברשימה [4, 7, 7, 10] — כמה מדידות יש? איזה מספר מופיע פעמיים? מהו סכום כל המדידות? האם אפשר לתאר את כל הרשימה באמצעות מספר אחד בלי לאבד מידע?

מפת הדרך בפרק

תחנת למידה	השאלה שמניעה אותה
איסוף והצגה	מה בדיוק מדדנו, וכמה פעמים קיבלנו כל ערך?
מדדי מרכז	איפה נמצא הערך הטיפוסי?
מדדי פיזור	כמה ערכים שונים זה מזה?
חריגים ורבעונים	האם תוצאה קיצונית משפיעה על הסיפור?
Python ו־AI	איך מחשבים נכון ומשתמשים במידע במודל?

1. הסיפור המרכזי — דודו בודק את זמן ההמתנה בקפיטריה

דודו לומד במגמת הבינה המלאכותית. התלמידים מתלוננים על תור ארוך בקפיטריה, ומנהל בית הספר מבקש תשובה מדויקת: ״כמה זמן תלמיד מחכה בדרך כלל?״. המנהל אינו רוצה רק מספר; הוא רוצה להחליט אם צריך לפתוח עמדת שירות נוספת.

דודו נעמד ליד הקפיטריה ורושם את זמן ההמתנה של שמונה תלמידים בדקות. הנתונים שאסף, בסדר עולה, הם:

$$2, 4, 4, 5, 6, 8, 10, 30$$

כבר במבט ראשון רואים שרוב התלמידים המתינו זמן קצר יחסית, אך אחד המתין 30 דקות. האם אותו תלמיד צריך להשפיע על ההחלטה? התשובה אינה "כן" או "לא" באופן אוטומטי. עלינו לבחור מדד שמתאים לשאלה.

מחשבה לפני חישוב

אם נרצה להציג למנהל רק מספר אחד, באיזה מספר נבחר? האם אנחנו מתארים זמן המתנה טיפוסי, או את העלות הכוללת של זמן ההמתנה? אלה שאלות שונות; לכן גם המדדים עשויים להיות שונים.

2. מילון הנתונים: מה בדיוק נמדד?

כל תלמיד שנמדד הוא תצפית (Observation). זמן ההמתנה הוא משתנה (Variable), והמספר שנרשם עבור תלמיד מסוים הוא ערך. היחידה היא דקות. ציון יחידת המדידה אינו פרט שולי: ממוצע של 8 דקות אינו שווה לממוצע של 8 שניות.

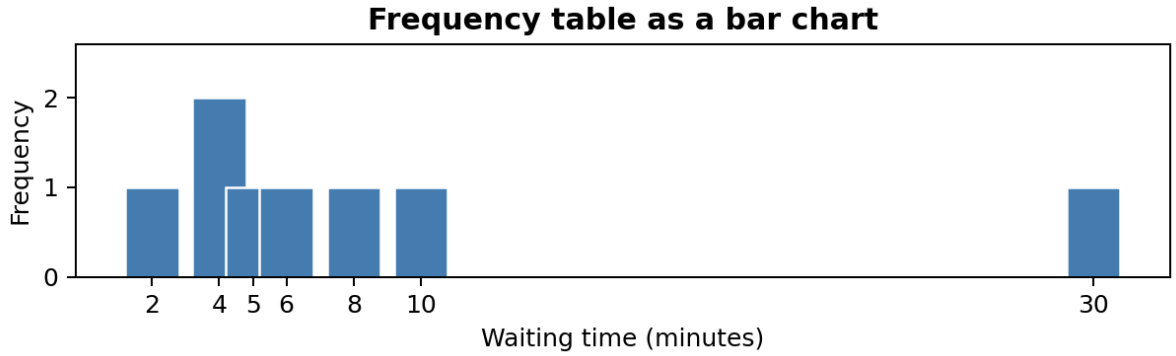
בדוגמה זו זמן ההמתנה הוא משתנה כמותי. מספר התלמידים הכולל בבית הספר הוא האוכלוסייה שאולי נרצה להסיק עליה, ואילו שמונה התלמידים שנמדדו הם מדגם. אם אלה כל שמונת התלמידים שהגיעו לקפיטריה באותו יום ובאותה שעה, אפשר לראות בהם אוכלוסייה מוגדרת של אותה קבוצה. ההבחנה תלויה בשאלת המחקר, לא רק במספר הנתונים.

מונח	פירוש בדוגמה	למה זה חשוב?
תצפית	תלמיד אחד שנמדד	כל תצפית היא רשומה במאגר
משתנה	זמן המתנה בדקות	אומר מה משמעות המספרים
מדגם	שמונת התלמידים שנבדקו	מאפשר להעריך אוכלוסייה רחבה יותר
אוכלוסייה	כל קבוצת התלמידים המעניינת אותנו	מגדירה על מי רוצים להסיק
יחידת מידה	דקות	נדרשת לפרשנות נכונה

3. מהי שכיחות ומדוע מתחילים בה?

לפני שנחשב ממוצע, נרצה לדעת האם יש ערכים חוזרים. שכיחות של ערך היא מספר הפעמים שהוא מופיע במאגר. למשל זמן המתנה של 4 דקות מופיע פעמיים ולכן שכיחותו 2. הזמן 30 מופיע פעם אחת בלבד.

זמן המתנה (דקות)	שכיחות
2	1
4	2
5	1
6	1
8	1
10	1
30	1
סך הכול	8



איור — ייצוג טבלת השכיחויות: עמודה גבוהה יותר מעידה על הופעות רבות יותר של אותו ערך.

טבלת שכיחויות עוזרת לזהות ערכים נפוצים ולוודא שלא שכחנו מדידות. סכום השכיחויות חייב להיות שווה למספר התצפיות. עבור מדידות של גדלים רציפים, כמו גובה או משקל, אפשר לקבץ ערכים לטווחים ולהציג היסטוגרמה. אין להתבלבל בין היסטוגרמה של טווחים רציפים לבין תרשים עמודות של קטגוריות.

4. ממוצע חשבוני — שוויון מחדש של סכום הנתונים

נניח שמחברים את כל זמני ההמתנה ומחלקים אותם באופן שווה בין כל התלמידים. המספר שכל תלמיד היה מקבל הוא הממוצע החשבוני. זוהי דרך טובה להבין את הרעיון: הממוצע שומר על הסכום הכולל, אך אינו מספר בהכרח כמה חיכה תלמיד כלשהו בפועל.

$$\text{mean} = (x_1 + x_2 + \cdots + x_n) / n$$

הסימון x_i מייצג מדידה אחת; האות n מייצגת את מספר המדידות; הסימן Σ, סכימה, פירושו לחבר את כל הערכים. במתמטיקה נוהגים לסמן ממוצע מדגם ב־$\bar{x}$ וממוצע אוכלוסייה ב־μ.

4.1 חישוב ידני מלא של זמן ההמתנה

שלב ראשון — מחברים את שמונה הערכים:

$$2 + 4 + 4 + 5 + 6 + 8 + 10 + 30 = 69$$

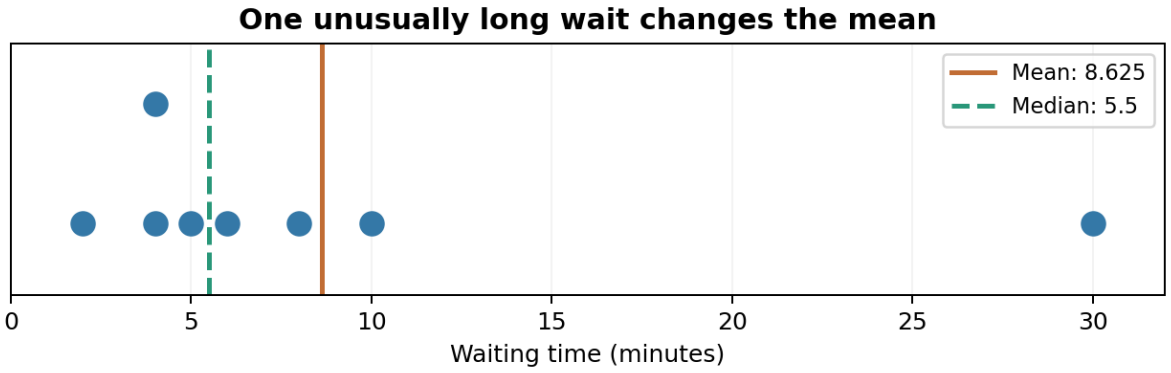
שלב שני — מוודאים שמספר התצפיות הוא 8. שלב שלישי — מחלקים את הסכום במספר התצפיות:

$$\bar{x} = 69 / 8 = 8.625 \text{ minutes}$$

פירוש: אילו היה זמן ההמתנה הכולל מתחלק שווה בשווה בין שמונה התלמידים, כל אחד היה "מקבל" 8.625 דקות. במציאות, אף תלמיד לא המתין בדיוק 8.625 דקות. הממוצע הוא תיאור מתמטי של הנתונים ולא תצפית חדשה.

4.2 האם הממוצע מתאר את רוב התלמידים?

שימו לב: שבעה תלמידים המתינו 10 דקות או פחות, אך הממוצע הוא 8.625 דקות — קרוב לחלק העליון של רוב הזמנים. הסיבה היא שהערך 30 מושך את הממוצע כלפי מעלה. לכן תמיד נשאל: האם יש ערך קיצוני? האם ההתפלגות סימטרית?



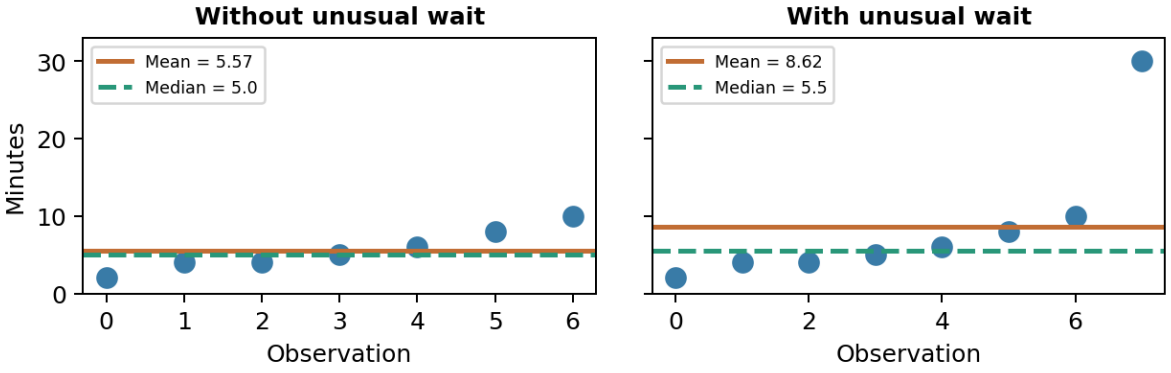
איור — קו כתום: הממוצע. קו ירוק מקווקו: החציון. המדדים אינם מתלכדים בנתונים לא סימטריים.

5. ממוצע וערכים חריגים — ניסוי מחשבתי

נבחן מה היה קורה אילו התלמיד שהמתין 30 דקות לא נכלל במדגם. כעת סכום הזמנים הוא 39 דקות, ומספר המדידות הוא 7. הממוצע החדש הוא כ־5.57 דקות. החציון החדש, כפי שנלמד מיד, הוא 5 דקות.

$$39 / 7 \approx 5.57 \text{ minutes}$$

שינוי של תצפית אחת בלבד הגדיל את הממוצע מכ־5.57 דקות ל־8.625 דקות. המסקנה אינה שיש למחוק תמיד תוצאות חריגות. אולי ההמתנה הארוכה חושפת תקלה חשובה! המסקנה הנכונה היא שממוצע לבדו לא מספר את כל הסיפור.



איור — השוואה בין מאגר עם זמן המתנה חריג לבין מאגר בלעדיו.

כלל לבחירת מדד

כאשר השאלה קשורה לסכום הכולל — למשל כמה זמן כל התלמידים איבדו במצטבר — הממוצע עשוי להיות חשוב מאוד. כאשר השאלה היא "מה בדרך כלל קורה לתלמיד יחיד?", חציון עשוי לתאר טוב יותר נתונים בעלי זנב ארוך.

6. חציון — נקודת האמצע לאחר מיון

החציון (Median) הוא הערך האמצעי ברשימה לאחר שמסדרים אותה מן הקטן לגדול. הוא מחלק את רשימת התצפיות המסודרת לשני חצאים מבחינת מספר התצפיות. בניגוד לממוצע, הוא מושפע הרבה פחות מהגדלה קיצונית של הערך הגבוה ביותר.

6.1 מספר אי־זוגי של תצפיות

בדוגמה שבה זמני ההמתנה הם שתיים, ארבע, חמש, שמונה ושלושים דקות, יש חמישה ערכים. הערך השלישי הוא האמצעי ולכן החציון הוא 5. מיקומו ברשימה הממוינת הוא המקום השלישי, משום שמוסיפים אחת למספר הערכים ומחלקים בשתיים.

6.2 מספר זוגי של תצפיות

בדוגמת הקפיטריה יש שמונה ערכים. אין ערך יחיד שנמצא בדיוק במרכז; שני הערכים האמצעיים נמצאים במקומות הרביעי והחמישי: 5 ו־6. כאשר מספר התצפיות זוגי, החציון הוא הממוצע של שני הערכים האלה.

$$\text{median} = (5 + 6) / 2 = 5.5 \text{ minutes}$$

חשוב להבין למה צריך למיין קודם: אם נקח את האמצע לפי סדר הכניסה של התלמידים, נקבל מספר שתלוי בסדר ההגעה ולא בהתפלגות זמני ההמתנה. החציון מתאר מיקום ברשימה ממוינת, לא סדר מקרי במאגר.

6.3 מתי החציון מתאים יותר?

מחירי דירות, זמני המתנה ומשכורות יכולים להכיל ערכים גבוהים במיוחד. במקרים כאלה חציון עשוי לתאר תוצאה טיפוסית באופן יציב יותר. לעומת זאת, אם רוצים לחשב הוצאה כוללת, לא ניתן להסתפק בחציון משום שהוא אינו שומר על סכום הערכים.

הבחנה עמוקה

אם נחליף את 30 ב־300, החציון יישאר 5.5, כל עוד סדר הערכים האחרים אינו משתנה. הממוצע, לעומת זאת, יגדל מאוד. עמידות לשינוי קיצוני היא תכונה חשובה של חציון.

7. שכיח — הערך שחוזר בתדירות הגבוהה ביותר

השכיח (Mode) הוא הערך שהשכיחות שלו היא הגבוהה ביותר. בדוגמת הקפיטריה הערך 4 מופיע פעמיים, וכל האחרים מופיעים פעם אחת. לכן השכיח הוא 4 דקות.

השכיח מתאים גם לנתונים קטגוריאליים: אם תלמידים מצביעים עבור ארוחה מועדפת — פיצה, סלט, פיצה, פסטה — השכיח הוא "פיצה". אי אפשר לחשב ממוצע חשבוני של שמות מאכלים, אך אפשר לזהות את הקטגוריה הנפוצה ביותר.

יכולים להיות שני ערכים שכיחים באותה מידה, או יותר. למשל בסדרה שבה מופיעים פעמיים המספר 2, פעמיים המספר 5 ופעם אחת המספר 7, קיימים שני שכיחים: 2 ו־5. בסדרה שבה כל ערך מופיע פעם אחת אין שכיח יחיד ומובהק. חשוב לא לומר שקיים תמיד שכיח יחיד.

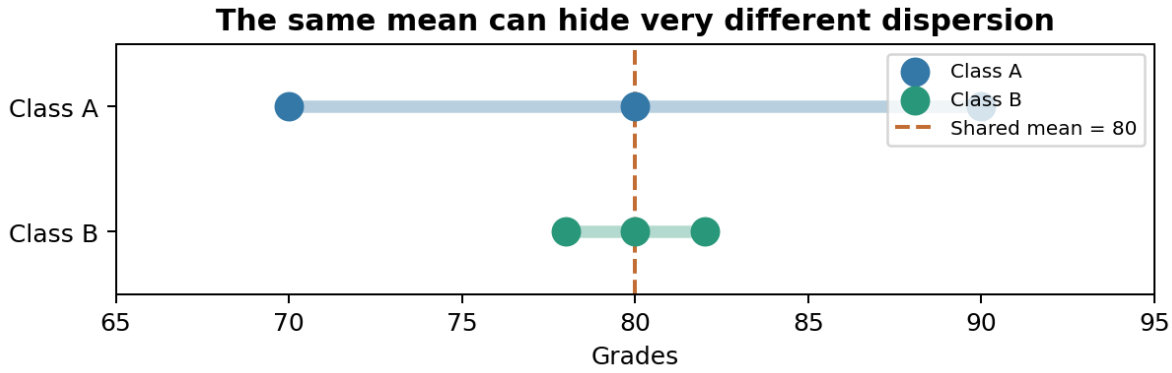
8. השוואה: ממוצע, חציון ושכיח

מדד	מה מודדים?	יתרון מרכזי	מגבלה
ממוצע	חלוקת סכום הערכים במספרם	משתמש בכל הערכים; מתאים לחישובי סכום	רגיש לערכים קיצוניים
חציון	המיקום האמצעי ברשימה ממוינת	עמיד יחסית לערכים קיצוניים	אינו משמר את סכום המדידות
שכיח	הערך שמופיע הכי הרבה	מתאים גם לנתונים קטגוריאליים	עשוי להיות לא יחיד או לא מייצג

כדי לבחור מדד, אל תשאלו רק "איזו נוסחה למדנו?" אלא "מה בדיוק רוצים להחליט?". למשל, לפיתוח שירות חדש בקפיטריה נרצה לדעת גם את החציון וגם את שיעור התלמידים שמחכים יותר מ־10 דקות. מדד מרכז בודד אינו מספיק תמיד לקבלת החלטה.

9. מהו פיזור, ולמה ממוצע אחד אינו מספיק?

נניח שבשתי כיתות ממוצע הציונים הוא 80. האם נכון לומר שהכיתות דומות? לא בהכרח. בכיתה א׳ הציונים הם 70, 80, 90; בכיתה ב׳ הציונים הם 78, 80, 82. הממוצע זהה, אך ציוני כיתה א׳ רחוקים יותר מן המרכז.



איור — שני אוספי ציונים עם ממוצע זהה ופיזור שונה.

מדדי פיזור עונים על השאלה "עד כמה הנתונים מפוזרים?". בלי פיזור לא נוכל להבחין בין קבוצה יציבה יחסית לבין קבוצה שיש בה הבדלים גדולים, גם כאשר הממוצעים שווים.

10. טווח — מדד הפיזור הפשוט ביותר

$$\text{Range} = \text{maximum} - \text{minimum}$$

בדוגמת הקפיטריה, ההמתנה הארוכה ביותר היא 30 דקות והקצרה ביותר 2 דקות, לכן הטווח הוא 28 דקות. בכיתה א׳ הטווח הוא 20 נקודות, ובכיתה ב׳ הטווח הוא 4 נקודות.

הטווח קל לחישוב ולהסבר, אך הוא תלוי רק בשני ערכים. גם אם ששת זמני ההמתנה האחרים ישתנו מאוד, הטווח לא ישתנה כל עוד המינימום והמקסימום נשארים זהים. לכן הוא מתאים כמדד ראשוני, לא כתיאור מלא של הפיזור.

11. רבעונים — חלוקת הנתונים לארבעה חלקים

רבעונים עוזרים לנו להבין היכן נמצאת המחצית האמצעית של התצפיות. Q1 הוא הרבעון התחתון, Q2 הוא החציון ו־Q3 הוא הרבעון העליון. הרעיון הוא לחלק את סדרת הנתונים המסודרת לארבעה חלקים בקירוב מבחינת מספר התצפיות.

11.1 חישוב ידני לפי שיטת חצאי המדגם

נשתמש במפורש בשיטה שבה מחשבים חציון של המחצית התחתונה וחציון של המחצית העליונה לאחר מיון. עבור שמונה זמני ההמתנה, החצי התחתון מורכב מהערכים 2, 4, 4, 5; החצי העליון מורכב מהערכים 6, 8, 10, 30. בכל אחד מן החצאים נחשב חציון בנפרד.

$$Q1 = (4 + 4) / 2 = 4$$

$$Q2 = (5 + 6) / 2 = 5.5$$

$$Q3 = (8 + 10) / 2 = 9$$

כאשר מספר הנתונים אי־זוגי, יש יותר משיטת חישוב אחת. בספר הזה נציין תמיד באיזו שיטה אנו משתמשים, ובחישובים ידניים של שיטת חצאי המדגם נשמיט את החציון המרכזי לפני יצירת שני החצאים. חשוב לא לערבב בין שיטות במהלך פתרון.

11.2 מדוע NumPy או Excel עשויים לתת תשובה אחרת?

תוכנות מסוימות מחשבות אחוזונים באמצעות אינטרפולציה בין ערכים. לכן הפקודה np.percentile עשויה לתת לרבעון העליון ערך של 8.5 עבור אותה רשימה, בעוד שבשיטת חצאי המדגם שלמדנו קיבלנו 9. שתי התוצאות נובעות מהגדרות חישוב שונות, ואין לסמן אחת מהן כשגויה בלי לציין את השיטה שנדרשה בשאלה.

דגש חשוב לבגרות

בחישוב רבעונים יש לכתוב את שיטת החישוב, ובפרט כאשר מספר התצפיות אי־זוגי או כאשר משווים לתוצאה ממחשב. שאלת בגרות יכולה להגדיר במפורש שיטה — ואז יש לפעול לפיה.

12. IQR — הטווח הבין־רבעוני

$$IQR = Q3 - Q1$$

הטווח הבין־רבעוני מודד את רוחב המחצית המרכזית של הנתונים, בין הרבעון התחתון לרבעון העליון. במקרה שלנו: IQR=9−4=5 דקות. הוא מתייחס לחלק המרכזי של הנתונים, ולכן פחות רגיש למקסימום חריג מאשר הטווח הרגיל.

מה פירוש המספר 5? הוא אינו אומר שחצי מהתלמידים המתינו בדיוק 5 דקות; הוא אומר שהמרחק בין שני הרבעונים הוא 5 דקות. הסבר מילולי מדויק חשוב לא פחות מן החישוב.

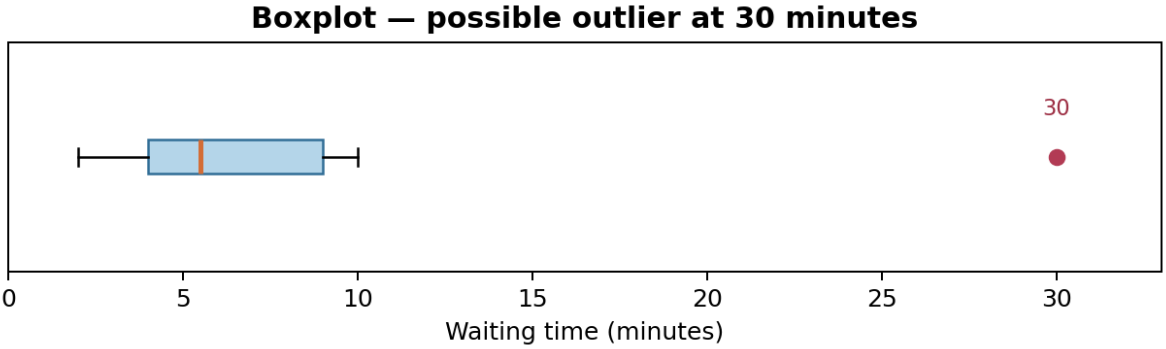
13. ערכים חריגים — כלל IQR×1.5

דרך נפוצה לסמן ערכים שראויים לבדיקה היא לחשב גבול תחתון ועליון באמצעות IQR. הערכים שנמצאים מחוץ לגבולות מסומנים כחריגים אפשריים. זהו כלל עבודה סטטיסטי, ולא הוכחה שמדובר בנתון שגוי.

$$\text{Lower fence} = Q1 - 1.5 \times IQR$$

$$\text{Upper fence} = Q3 + 1.5 \times IQR$$

בדוגמת הקפיטריה: הגבול התחתון הוא מינוס 3.5, והגבול העליון הוא 16.5. לכן הערך 30 נמצא מעל הגבול העליון ומסומן לבדיקה. לא נסיר אותו רק מפני שהוא חריג: אולי הייתה תקלה, ואולי המדידה בוצעה ביום עמוס במיוחד.



איור — תרשים קופסה מציג את מרכז הנתונים, הרבעונים והערך החריג האפשרי.

14. סטייה מן הממוצע — מה מודדים באמת?

כדי לקבל מדד שמשתמש בכל התצפיות, נבחן עד כמה כל תצפית רחוקה מהממוצע. הפרש כזה נקרא סטייה מן הממוצע. בכיתה א׳ הממוצע הוא 80 והציונים הם 70,80,90. הסטיות הן −10, 0 ו־10.

אם נחבר את הסטיות נקבל 0, משום שסטיות חיוביות ושליליות מבטלות זו את זו. לכן סכום סטיות פשוט אינו מדד טוב לפיזור. צריך דרך למנוע את הביטול.

15. שונות — ממוצע ריבועי הסטיות

אחת הדרכים להתמודד עם ביטול סטיות היא להעלות כל סטייה בריבוע. כעת כל ריבוע אינו שלילי, והפרש גדול מקבל משקל גדול יותר. ממוצע ריבועי הסטיות נקרא שונות (Variance).

$$\text{Population variance: } \sigma^2 = \Sigma(x_i - \mu)^2 / N$$

האות μ מייצגת את ממוצע האוכלוסייה, N את מספר הערכים באוכלוסייה, והביטוי $(x_i-\mu)^2$ הוא ריבוע ההפרש בין ערך יחיד לממוצע.

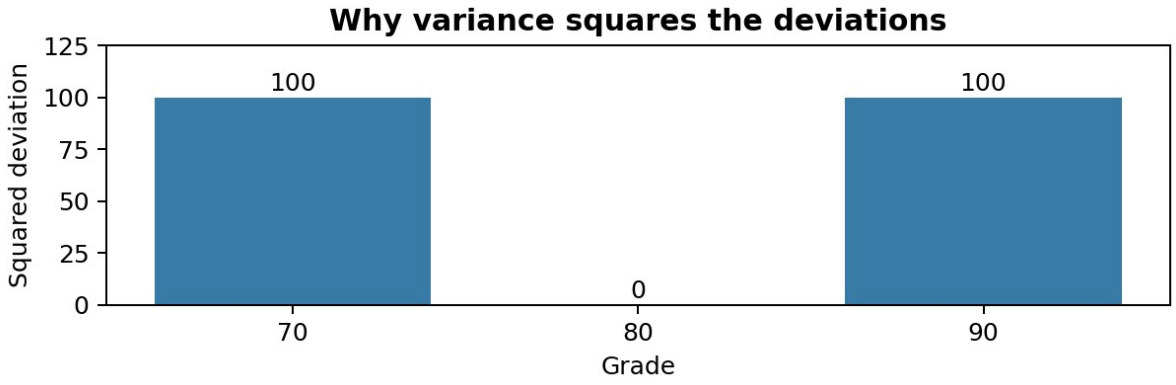
15.1 דוגמה פתורה — שונות אוכלוסייה

נתייחס לשלושת הציונים 70, 80, 90 כאוכלוסייה כולה. ממוצעם הוא 80. עכשיו נחשב סטייה, נעלה בריבוע ונחבר:

ציון	סטייה מן הממוצע	ריבוע הסטייה
70	70−80= −10	100
80	80−80=0	0
90	90−80=10	100
סה"כ		200

$$\sigma^2 = (100 + 0 + 100) / 3 = 200/3 \approx 66.67$$

השונות היא כ־66.67 נקודות־ציון בריבוע. מאחר שהיחידות הועלו בריבוע, שונות פחות נוחה לפירוש יומיומי. לכן נלמד מיד גם סטיית תקן.



איור — ריבועי הסטיות של שלושת הציונים. הסטיות הגדולות מקבלות ריבוע גדול.

16. שונות מדגם — מדוע לפעמים מחלקים ב־n−1?

אם שלושת הציונים הם רק מדגם מתוך אוכלוסייה גדולה ואנו רוצים לאמוד את שונות האוכלוסייה על סמך המדגם, מקובל להשתמש בשונות מדגמית לא מוטה. במקום לחלק ב־n מחלקים ב־n−1. הסיבה היא שחישבנו את סטיות המדגם סביב ממוצע שנאמד מאותם נתונים, ולכן אבדת דרגת חופש אחת.

$$\text{Sample variance: } s^2 = \Sigma(x_i - \bar{x})^2 / (n - 1)$$

בדוגמת 70, 80, 90: סכום ריבועי הסטיות נשאר 200, אבל כעת מחלקים ב־2 ולא ב־3:

$$s^2 = 200 / (3 - 1) = 100$$

חשוב להבהיר: לא מדובר בשתי תשובות לאותה שאלה בדיוק. שונות האוכלוסייה של שלושת הערכים היא 66.67; שונות המדגם המשמשת לאמידת שונות אוכלוסייה רחבה היא 100. הבחירה תלויה במטרת החישוב ובמה שהשאלה מגדירה.

מצב	מכנה	פקודת NumPy
כל הערכים באוכלוסייה המוגדרת	N	np.var(data, ddof=0)
מדגם המשמש לאמידת שונות אוכלוסייה	n−1	np.var(data, ddof=1)

שגיאה נפוצה

לכתוב "סטיית תקן" אך להחזיר שונות. למשל תוצאה של np.var אינה סטיית תקן. בנוסף, שונות מדגם עם n=1 אינה מוגדרת בנוסחה עם n−1; אי אפשר לחלק באפס.

17. סטיית תקן — לחזור ליחידות המקוריות

סטיית תקן (Standard Deviation) היא השורש הריבועי של השונות. היתרון: התוצאה חוזרת ליחידות המקוריות של המשתנה, ולכן קל יותר לפרש אותה בהשוואה לשונות.

$$\text{Population SD: } \sigma = \sqrt{(\sigma^2)}$$

$$\text{Sample SD: } s = \sqrt{(s^2)}$$

עבור ציוני כיתה א׳, סטיית התקן של האוכלוסייה היא בקירוב 8.16 נקודות. אם נתייחס אל אותם ציונים כמדגם, סטיית התקן המדגמית היא 10 נקודות. אין לפרש זאת כאילו כל ציון רחוק מן הממוצע בדיוק 8.16 או 10 נקודות; סטיית תקן היא מדד כולל לפיזור.

18. דוגמת השוואה — שתי כיתות, אותו ממוצע

כעת נחשב גם את הפיזור של כיתה ב׳: 78,80,82. הממוצע הוא 80. הסטיות הן מינוס שתיים, אפס ושתיים; ריבועי הסטיות הם ארבע, אפס וארבע. לכן שונות האוכלוסייה היא 8/3≈2.67 וסטיית התקן √(8/3)≈1.63.

מדד	כיתה א׳: 70,80,90	כיתה ב׳: 78,80,82
ממוצע	80	80
טווח	20	4
שונות אוכלוסייה	66.67	2.67

מדד	כיתה א׳: 70,80,90	כיתה ב׳: 78,80,82
סטיית תקן אוכלוסייה	8.16≈	1.63≈

המסקנה: שתי הכיתות אינן זהות מבחינת הפיזור. בכיתה א׳ ההבדלים בין התלמידים גדולים יותר. במערכת AI, שתי קבוצות עם ממוצע דומה עשויות לדרוש טיפול שונה בגלל הפיזור.

19. מדדי פיזור יחסיים — הרחבה מקצועית

לפעמים רוצים להשוות פיזור של שתי תכונות עם ממוצעים שונים מאוד. אחד המדדים האפשריים הוא מקדם השונות (CV), שהוא היחס בין סטיית התקן לערך המוחלט של הממוצע. זהו נושא העשרה: אין להשתמש במדד בלי לבדוק שהסולם מתאים ושהממוצע אינו אפס או קרוב מאוד לאפס.

$$CV = \sigma / |\mu|, \quad \mu \neq 0$$

לדוגמה, אם הממוצע הוא 80 והסטייה היא 8, נקבל CV=0.1 או 10%. אם הממוצע קרוב לאפס, היחס עלול להיות גדול ולא יציב. במיוחד בטמפרטורות בסולמות כמו צלזיוס אין לפרש CV באופן נאיבי משום שנקודת האפס אינה מוחלטת.

20. מימוש ב־Python — מן החישוב הידני אל הספרייה

מימוש טוב אינו רק הפעלה של np.mean. מתחילים בבדיקה שהערכים נמצאים במבנה מתאים ושמבינים את משמעות כל פעולה. לאחר מכן משווים את התוצאה לחישוב הידני. כך אפשר לאתר שגיאות הקלדה, שיטת רבעונים שונה או בחירה לא נכונה בין מדגם לאוכלוסייה.

```
import numpy as np

wait = np.array([2, 4, 4, 5, 6, 8, 10, 30])
print("mean:", np.mean(wait))
print("median:", np.median(wait))
print("population variance:", np.var(wait, ddof=0))
print("sample variance:", np.var(wait, ddof=1))
print("population std:", np.std(wait, ddof=0))
```

הפקודות np.var ו־np.std משתמשות כברירת מחדל ב־ddof=0. משמעות ddof היא מספר דרגות החופש שמפחיתים מן המכנה, ולכן ddof=1 מוביל לחלוקה ב־n−1. אין לשנות אותו ללא הסבר.

20.1 חישוב ידני בלולאת for

כדי להבין מה עושה הספרייה, נרשום קודם מימוש בעזרת לולאה. הוא ארוך יותר, אך כל שורה מקבילה לשלב בחישוב המתמטי.

```
data = [70, 80, 90]
mean = sum(data) / len(data)
squared_sum = 0

for i in range(len(data)):
    difference = data[i] - mean
    squared_sum += difference ** 2

population_var = squared_sum / len(data)
print(population_var)
```

תחילה מחשבים ממוצע; אחר כך מבקרים כל ערך, מוצאים את ההפרש מן הממוצע, מעלים בריבוע ומצטברים. בסיום מחלקים במספר התצפיות. גרסת המדגם תשתמש ב־len(data)−1, בתנאי שיש לפחות שתי תצפיות.

20.2 טבלת שכיחויות ב־Pandas

```
import pandas as pd

wait = pd.Series([2, 4, 4, 5, 6, 8, 10, 30])
print(wait.value_counts().sort_index())
print(wait.describe())
```

הפקודה value_counts סופרת מופעים של כל ערך, ו־describe נותנת סיכום סטטיסטי מהיר. שימו לב: ב־Pandas, סטיית התקן בתוצאת describe מחושבת בדרך כלל כסטיית תקן מדגמית (ddof=1), בעוד NumPy משתמש כברירת מחדל ב־ddof=0. מכאן נובע הצורך לדעת איזו הגדרה בחרה הספרייה.

21. עקרונות להצגת נתונים בגרפים

לפני שמציגים גרף, נחליט איזו שאלה הוא עונה עליה. תרשים שכיחויות מראה מה נפוץ; Boxplot מציג חציון ורבעונים; גרף נקודות מציג את המדידות עצמן. שרטוט יפה אינו תחליף לבחירת סוג הגרף המתאים.

היסטוגרמה מתאימה למדידות מספריות המחולקות לתחומים, אבל בחירת רוחב התחומים יכולה להשפיע על המראה. תרשים עמודות מתאים במיוחד לקטגוריות או לשכיחויות של ערכים בדידים. עבור מדגם קטן, כדאי לעיתים להציג את כל הנקודות כדי שהתלמיד יוכל לבדוק מה מסתתר מאחורי המדדים.

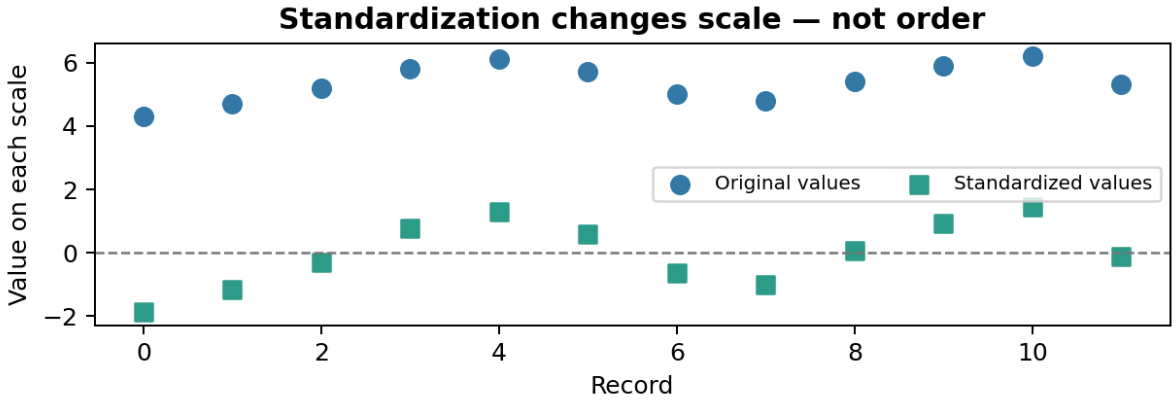
22. איך סטטיסטיקה נכנסת ללמידת מכונה?

מערכת למידת מכונה אינה מקבלת "הבנה" באופן אוטומטי ברגע שמזינים אליה טבלה. לפני האימון אנחנו חוקרים את מאגר הנתונים: האם יש ערכים חסרים? האם אחת התכונות נמדדת במספרים גדולים בהרבה מהאחרות? האם יש חריגים? האם המידע מייצג את הבעיה?

למשל, במודל KNN המשתמש במרחק אוקלידי, תכונה הנמדדת באלפי שקלים עשויה לשלוט על תכונה שנמדדת בסולם 1–5. לכן לעיתים מבצעים תקנון לפי ממוצע וסטיית תקן: כל ערך חדש הופך להפרש שלו מן הממוצע, מחולק בסטיית התקן.

$$z = (x - \mu) / \sigma, \quad \sigma > 0$$

תקנון אינו הופך בהכרח את התפלגות הנתונים לנורמלית. הוא משנה את מיקום וקנה המידה של התכונה. אם סטיית התקן אפס, כל הערכים זהים ולא ניתן לבצע חלוקה רגילה בסטיית התקן; נדרשת החלטת טיפול מתאימה.



איור — לפני תקנון ואחריו: סדר הרשומות נשאר, אך המיקום וקנה המידה המספרי משתנים.

22.1 מלכודת חשובה: זליגת נתונים

כאשר מאמנים מודל ובודקים אותו על נתוני Test, אין לחשב ממוצע וסטיית תקן מתוך כל המאגר מראש. אחרת נתוני הבדיקה השפיעו על פעולת ההכנה אף שהם היו אמורים להישאר לא מוכרים למודל. זו דוגמה לזליגת נתונים (Data Leakage).

הסדר הנכון: מחלקים ל־Train ול־Test; מחשבים פרמטרי תקנון מנתוני Train בלבד; משתמשים באותם פרמטרים כדי לשנות גם את נתוני Test. בשלב מאוחר יותר של הספר נלמד לעשות זאת באמצעות Pipeline של scikit-learn.

23. דוגמה מסכמת — מה כדאי לדווח למנהל?

נחזור לשמונת זמני ההמתנה. קיבלנו ממוצע 8.625 דקות, חציון 5.5 דקות, שכיח 4 דקות, טווח 28 דקות, רבעון תחתון 4 דקות, רבעון עליון 9 דקות ו־IQR של 5 דקות. הערך 30 סומן כחריג אפשרי לפי כלל IQR×1.5.

מה כדאי לכתוב בדוח? למשל: "במדגם קטן של שמונה תלמידים, זמן ההמתנה החציוני היה 5.5 דקות, אך נמדדה גם המתנה חריגה של 30 דקות. מומלץ לאסוף עוד מדידות בשעות ובימים שונים לפני קבלת החלטה על שינוי שירות." אין לומר מכאן שזמן ההמתנה הממוצע של כל תלמידי בית הספר הוא בהכרח 8.625 דקות.

מיומנות של חוקר נתונים

תיאור מקצועי כולל ארבעה חלקים: מה מדדנו; מה מצאנו; מה אפשר להסיק; ומה אי אפשר להסיק עדיין. ההבחנה בין ממצא למסקנה היא בסיס לבגרות ולהגנה על פרויקט הגמר.

24. שגיאות נפוצות והדרך לתקן אותן

שגיאה	למה היא שגויה?	איך מתקנים?
חישוב חציון ללא מיון	האמצע תלוי בסדר ההקלדה	ממיינים ואז מחפשים את המרכז

שגיאה	למה היא שגויה?	איך מתקנים?
ממוצע של קטגוריות טקסט	אין משמעות לסכום ״כחול״ ו״ירוק״	משתמשים בשכיחויות או בשכיח
חילוק ב־n−1 בלי להגדיר מדגם	אין התאמה לשאלת החישוב	מגדירים אוכלוסייה או מדגם
הצגת שונות כסטיית תקן	אלה יחידות שונות וחישובים שונים	מוציאים שורש כדי לחשב סטיית תקן
טענה שערך חריג הוא טעות	הוא עשוי להיות אירוע אמיתי	בודקים מקור והקשר
השוואת np.percentile לשיטת חצאים כאילו הן זהות	אלה הגדרות רבעונים שונות	מציינים את שיטת החישוב
תקנון לפני פיצול Train/Test	זליגת מידע לקבוצת הבדיקה	מחשבים על Train בלבד

25. תרגול עצמאי — שלוש רמות

רמה א׳ — הבנת המושגים

- 1. בחנות מחשבים נמדדו מחירים: [2000, 2400, 2400, 2600, 3000]. חשבו ממוצע, חציון ושכיח. הסבירו במילים מה כל אחד מספר.
- 2. הגדירו במילים תצפית, משתנה ויחידת מידה עבור נתוני שעות שינה של תלמידים.
- 3. הסבירו מדוע אי אפשר לחשב ממוצע של צבעי חולצות, אך כן אפשר למצוא שכיח.
- 4. מדוע חציון של [2, 4, 6, 100] אינו 6? הסבירו את שיטת החישוב.

רמה ב׳ — חישוב ופרשנות

- 5. נתוני צעדים יומיים (באלפים): [3, 4, 4, 6, 8, 9]. חשבו טווח, חציון, Q1, Q3 ו־IQR לפי שיטת חצאי המדגם. ציינו את שיטתכם.
- 6. שתי קבוצות קיבלו ציונים [4, 6, 8] ו־[5, 6, 7]. חשבו ממוצע וסטיית תקן של האוכלוסייה עבור כל קבוצה. מי מפוזרת יותר?
- 7. עבור הנתונים [10, 12, 14], חשבו שונות אוכלוסייה ושונות מדגם. הסבירו מדוע קיבלתם שתי תוצאות שונות.
- 8. סדרת הזמנים [5, 6, 7, 8, 50] מכילה ערך גבוה במיוחד. איזה מדד תציגו למי ששואל על זמן טיפוסי? נמקו.

רמה ג׳ — חשיבה של חוקר AI

- 9. תלמיד הציג גרף של ממוצעי שתי כיתות בלבד והסיק שאין ביניהן הבדל. הסבירו מה חסר בדוח והציעו דרך בדיקה משופרת.

- 10. מודל KNN משתמש במאפייני "גובה בסנטימטרים" ו"הכנסה בשקלים". הסבירו מדוע המדדים הסטטיסטיים חשובים לפני הפעלתו.
- 11. בדקו מה משתנה ומה אינו משתנה אם מוסיפים 10 לכל הערכים: ממוצע, חציון, טווח, שונות וסטיית תקן. הציגו נימוק, לא רק תשובה.
- 12. מה קורה לכל אחד מן המדדים אם מכפילים את כל הערכים ב־3? הבחינו בין מדדי מרכז, שונות וסטיית תקן.
- 13. הסבירו מדוע אי אפשר להבטיח שהשכיח הוא הערך "הטיפוסי" ביותר בכל מאגר. תנו דוגמה נגדית.

26. שאלת הערכה מקורית בסגנון בגרות

מערכת בית ספרית מודדת זמני הגעה של אוטובוס, בדקות: [3, 5, 5, 7, 8, 10, 12, 22]. המערכת אמורה להציג להורים זמן המתנה צפוי ולסמן מקרים חריגים. ענו על כל הסעיפים, הציגו חישוב ונמקו.

- א. בנו טבלת שכיחויות, וחשבו ממוצע, חציון ושכיח.
- ב. חשבו Q1, Q3 ו־IQR לפי שיטת חצאי המדגם, ובדקו האם 22 מסומן כחריג לפי IQR×1.5.
- ג. הסבירו באיזה מדד מרכז הייתם משתמשים כדי לתאר זמן "טיפוסי" ומדוע.
- ד. תכננו קוד Python שמחשב שונות וסטיית תקן של אוכלוסיית המדידות. אפשר להיעזר ב־NumPy.
- ה. הסבירו כיצד איסוף המדידות מעוד יום עשוי לשנות את המסקנה, ומדוע אין להבטיח שמדגם של שמונה מדידות מייצג את כל השנה.

הערה לתלמיד

זוהי שאלת הערכה מקורית שנכתבה לצורכי הוראה בהשראת מיומנויות נושאי הליבה. אין להציג אותה כשאלה רשמית ממועד בגרות מסוים.

27. מעבדת Colab — מעבר מחישוב ידני לחקר נתונים

לפרק מצורפת מחברת נפרדת שאפשר לפתוח ב־Google Colab. היא כוללת משימות השלמת קוד ולא פתרונות. מבנה המעבדה: טעינת נתונים, טבלת שכיחויות, חישוב ממוצע וחציון, רבעונים לפי שיטה מפורשת, שונות וסטיית תקן, שני גרפים, ניתוח קבוצה והשלכת שינוי קנה מידה.

כלל העבודה: לפני כל תא חישוב, כתבו תחזית מספרית או מילולית; לאחר הרצת הקוד הסבירו אם התוצאה הגיונית. אל תסתפקו בהעתקת פלט.

28. דף נוסחאות מסכם

מושג	נוסחה או כלל	פירוש
ממוצע	$\bar{x} = \Sigma x_i / n$	חלוקה שווה של הסכום
חציון	הערך האמצעי לאחר מיון	מרכז לפי מיקום
טווח	max − min	מרחק בין קצוות
IQR	Q3 − Q1	רוחב המחצית האמצעית
שונות אוכלוסייה	$\sigma^2 = \Sigma(x_i-\mu)^2/N$	ממוצע ריבועי הסטיות
שונות מדגם	$s^2 = \Sigma(x_i-\bar{x})^2/(n-1)$	אומדן שונות האוכלוסייה
סטיית תקן	$\sqrt{\text{variance}}$	פיזור ביחידות המקור
תקנון — גשר לפרק הבא	$z = (x-\mu)/\sigma$	מרחק יחסי מהממוצע ביחידות σ

29. רשימת בדיקה עצמית

- □ אני יכול להסביר מדוע לממוצע ולחציון עשויות להיות תוצאות שונות באותו מאגר.
- □ אני יכול לחשב רבעונים ולציין באיזו שיטת חישוב השתמשתי.
- □ אני יודע להבדיל בין שונות וסטיית תקן ובין אוכלוסייה למדגם.
- □ אני יכול לבדוק תוצאה ב־Python ולזהות את משמעות ddof.
- □ אני יכול להסביר מתי ערך חריג הוא חשוב ולא בהכרח שגוי.
- □ אני יכול לתאר קשר בין מדדי פיזור לבין הכנת נתונים למודל AI.

30. מקורות והמשך לימוד

עוגן התכנית: נושאי ליבה בבינה מלאכותית — פרק 2.1 ״מדדי מרכוז ופיזור״, על פי מפת התכנים וחומרי ההשתלמות השמורים במאגר Data-Science-ML-Course. סיכומי ההשתלמות כוללים טבלאות שכיחויות, ממוצע, חציון, שכיח, רבעונים, שונות וסטיית תקן, וכן הבחנה בין חישובי NumPy ל־Excel.

- Deisenroth, Faisal & Ong — Mathematics for Machine Learning (Cambridge University Press): רקע מתמטי לסטטיסטיקה וייצוגים.
- James et al. — An Introduction to Statistical Learning with Applications in Python (Springer): עקרונות חקר נתונים והערכת מודלים.
- Josh Starmer — The StatQuest Illustrated Guide to Machine Learning: השראה להמחשה מדורגת של רעיונות סטטיסטיים.

- Aurélien Géron — Hands-On Machine Learning: קשר לתהליך עבודה עם נתונים ולהכנה נכונה לאימון.

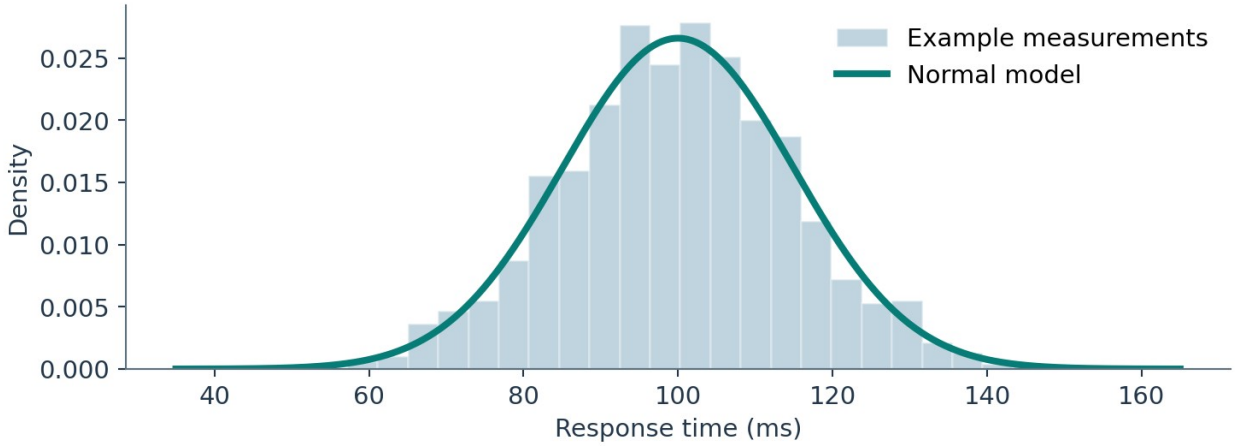
הדוגמאות, הניסוחים, האיורים, השאלות והקוד נכתבו כיצירה מקורית לתלמידי תיכון. ספרי המקור משמשים רקע מקצועי ומתודולוגי; אין כאן העתקה של פרקים או תרשימים מספרים מסחריים.

מה נלמד בפרק 2.2?

התפלגות נורמלית, ציון תקן z, הכלל 68–95–99.7, PDF ו־CDF, והקשר בין סטטיסטיקה תיאורית להסתברות. מדדי המרכז והפיזור שלמדנו הם הידע הקודם הדרוש לכך.

פרק 2.2 — התפלגות נורמלית

מתי אפשר לחזות הסתברות מנתונים — ומתי צורת הפעמון עלולה להטעות אותנו?



איור — מדידות לדוגמה ועקומה נורמלית: המודל מסכם את דפוס המדידות, אך אינו מוכיח שכך מתנהגים נתוני אמת.

ספר תלמיד מורחב • הסבר מדורג • חישובים ידניים • גרפים • Python • שאלות בסגנון בגרות

נושאי ליבה בבינה מלאכותית — יחידה 5 | גשר ללמידת מכונה, לחקר נתונים ולפרויקט הגמר

הערה לתלמיד כל המספרים בפרק המשמשים להמחשה הם נתונים מומצאים או נתוני סימולציה. אנו נבדוק בכל פעם אם הנחת ההתפלגות הנורמלית מוצדקת, ולא נניח שכל מאגר מספרי הוא נורמלי.

לפני שמתחילים — מטרות הלמידה

בפרק הקודם למדנו לתאר נתונים באמצעות ממוצע, חציון, שונות וסטיית תקן. כעת נשאל שאלה חדשה: האם אפשר להעריך מה הסיכוי שערך חדש יהיה גבוה או נמוך מסף מסוים? לשם כך לא די לדעת את הממוצע ואת הפיזור; אנו צריכים גם מודל לצורת ההתפלגות. ההתפלגות הנורמלית היא אחד המודלים החשובים, אך היא אינה מתאימה לכל מצב.

- להבחין בין תוצאה בדידה לתוצאה רציפה ולתאר מה מייצג גרף הסתברות.
- להסביר את המשמעות של הממוצע μ ושל סטיית התקן σ בהתפלגות נורמלית.
- לחשב ציון תקן Z, לפרש אותו ולהשוות תוצאות שנמדדו בסולמות שונים.
- להשתמש בכלל 68–95–99.7 ולזהות מתי מדובר בקירוב בלבד.
- להבחין בין צפיפות PDF לבין הסתברות, ובין PDF לבין פונקציית ההתפלגות המצטברת CDF.
- לחשב הסתברות משמאל לסף, מימין לסף ובין שני ערכים בעזרת טבלה ו־Python.
- להסביר באופן אינטואיטיבי את משפט הגבול המרכזי ואת שגיאת התקן של ממוצע מדגם.
- לזהות מגבלות של מודל נורמלי ולנסח מסקנה אחראית בהקשר של פרויקט AI.

מפת הפרק והידע הקודם

למה זה חשוב ל־AI	מה נלמד	אשכול לימוד
הבנת אי־ודאות ומודלים	תוצאות בדידות לעומת Plinko, רציפות	הסתברות ותוצאות
תיאור תכונות ומדידות	צורת העקומה והכלל μ, σ, האמפירי	המודל הנורמלי
פרשנות ציונים, ספים ותוצאות	Z-score, PDF, CDF, תחומים וזנבות	תקנון והסתברות
אמינות אומדנים וניסויים	CLT, ממוצעי מדגמים, שגיאת תקן	הסקה ודגימה
שימוש נכון בידע במערכת AI	Python, ביקורת נתונים, שאלות ופרויקט	יישום והערכה

ידע קודם: ממוצע ושונות מפרק 2.1, שורש ריבועי, הצבה בנוסחה, קריאת צירים בגרף, רשימות ו־NumPy בסיסי. אין צורך לדעת לחשב אינטגרלים כדי להבין את החלק המרכזי בפרק.

1. הסיפור המרכזי — דודו מודד זמני תגובה במשחק

דודו בונה יישומון אימון לתלמידי מגמת AI. בכל סבב מופיעה צורה על המסך, והתלמיד צריך ללחוץ כאשר הוא מזהה אותה. דודו מודד זמן תגובה במילישניות. לאחר ניסויים רבים הוא רוצה לענות על שלוש שאלות שונות: מהו זמן תגובה טיפוסי? עד כמה יש שונות בין התגובות? ומה הסיכוי שבניסוי הבא זמן התגובה יהיה ארוך מ־130 מילישניות?

שתי השאלות הראשונות מזכירות את פרק 2.1: ממוצע וסטיית תקן. השאלה השלישית שונה: היא שואלת על הסתברות של תוצאה עתידית. כדי לענות עליה דודו צריך לדעת, או לפחות להניח בזהירות, כיצד מתפלגים זמני התגובה. נניח לצורך הלמידה שמודל נורמלי עם ממוצע 100 וסטיית תקן 15 מתאים למדידות. רק תחת הנחה זו נוכל להשתמש בכלים שנלמד כדי לחשב הסתברויות.

$$X \sim N(100, 15^2)$$

הסימון אינו קובע שכך מתנהגים תמיד זמני תגובה של בני אדם. הוא מתאר הנחה מתמטית על המשתנה X. בעולם האמיתי זמני תגובה מוגבלים מלמטה ולעיתים אסימטריים, ולכן צריך לבדוק את איכות ההתאמה בעזרת נתוני אמת.

נקודת חשיבה אם שתי קבוצות מאופיינות באותו ממוצע ובאותה סטיית תקן, האם בהכרח יש להן אותה התפלגות? לא. הממוצע והפיזור אינם מספרים לנו לבדם אם ההתפלגות סימטרית, מוטה או מורכבת מכמה קבוצות.

2. חזרה קצרה: מרכז, פיזור והבדל בין מדגם לאוכלוסייה

הממוצע מתאר את מרכז הנתונים וסטיית התקן מתארת פיזור אופייני של הערכים סביבו. אם המרחקים מהממוצע קטנים בדרך כלל, סטיית התקן קטנה. אם הערכים פזורים במרחקים גדולים מן המרכז, סטיית התקן גדולה יותר. שונות היא ריבוע סטיית התקן, ולכן היחידות שלה שונות: אם מודדים שניות, השונות נמדדת בשניות בריבוע.

Population variance: $\sigma^2 = \Sigma(x_i - \mu)^2 / N$

Population SD: $\sigma = \sqrt{[\Sigma(x_i - \mu)^2 / N]}$

בספר זה נשתמש בדרך כלל ב־μ וב־σ לתיאור התפלגות תאורטית או פרמטרים של אוכלוסייה, וב־$\bar{x}$ וב־s לתיאור נתונים של מדגם. חשוב להבדיל גם בין התפלגות של תצפיות בודדות לבין התפלגות של ממוצעי מדגמים; נחזור לכך במשפט הגבול המרכזי.

3. לפני הפעמון: מה ההבדל בין משתנה בדיד למשתנה רציף?

משתנה מקרי (Random Variable) הוא כלל שמקשר תוצאה של ניסוי אקראי למספר. נניח שדודו זורק מטבע ארבע פעמים ומונה כמה פעמים יצא עץ. מספר הפעמים יכול להיות רק 0, 1, 2, 3 או 4. אין אפשרות לקבל 2.4 פעמים עץ. זהו משתנה מקרי בדיד.

לעומתו, זמן תגובה נחשב בדרך כלל למשתנה רציף במודל המתמטי: מבחינה תאורטית הוא עשוי לקבל כל ערך בתחום מתאים, גם 101.3 או 101.37 מילישניות. מכשיר מדידה מציג מספר סופי של ספרות, אך מודל רציף מאפשר לנו לתאר טווחים קטנים מאוד של ערכים.

רציף	בדיד	מאפיין
זמן תגובה או גובה	מספר פניות ימינה ב־Plinko	דוגמה
טווח ערכים רציף במודל	קבוצה ניתנת למנייה	ערכים אפשריים
מחשבים הסתברות של תחום	אפשר לחשב $P(X=k)$	הסתברות
עקומת צפיפות	עמודות נפרדות	המחשה

אל תתבלבלו גם משתנה רציף יכול להיות מוגבל לתחום מסוים. למשל זמן אינו שלילי. העובדה שמודל נורמלי מאפשר מתמטית ערכים שליליים היא אחת הסיבות שהוא עשוי להיות קירוב לא מתאים במקרים מסוימים.

4. Plinko — איך פניות קטנות יוצרות צורה דמוית פעמון?

דמיינו לוח מסמרים שבו כדור נופל מלמעלה. בכל מפגש עם מסמר הוא פונה שמאלה או ימינה. בדגם מתמטי פשוט ננית שכל פנייה היא בלתי תלויה באחרות, והסיכוי לפנייה ימינה הוא חצי. הכדור יכול להגיע לתאים שונים בתחתית הלוח. היכן צפויים להגיע רוב הכדורים?

ניקח תחילה ארבע פניות בלבד. נסמן K כמספר הפניות ימינה. כדי להגיע למיקום המרכזי הכדור צריך לפנות בדיוק פעמיים ימינה ופעמיים שמאלה, ולא משנה באיזה סדר. קיימות שש דרכים לסדר שתי פניות ימינה בארבע החלטות, ולכן למיקום המרכזי יש יותר מסלולים מאשר למיקום הקיצוני שדורש ארבע פניות באותו כיוון.

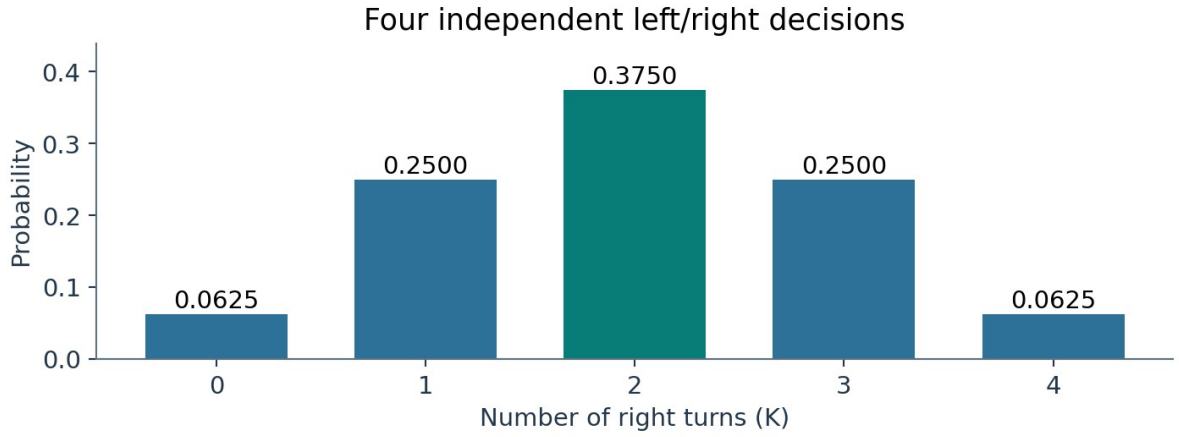
$$P(K = k) = C(n,k) \cdot p^k \cdot (1-p)^{(n-k)}$$

כאן n הוא מספר הפניות, k מספר הפניות ימינה, p ההסתברות לפנות ימינה בכל החלטה, ו־$C(n,k)$ מספר הדרכים לבחור באילו החלטות פונים ימינה. זו נוסחת ההתפלגות הבינומית, והיא משמשת כאן כגשר אינטואיטיבי לנושא הנורמלי; אין חובה בפרק זה לזכור את כל נוסחת הבינומית אם היא אינה חלק מדרישות ההערכה שלנו.

4.1 חישוב ידני: בדיוק שתי פניות ימינה

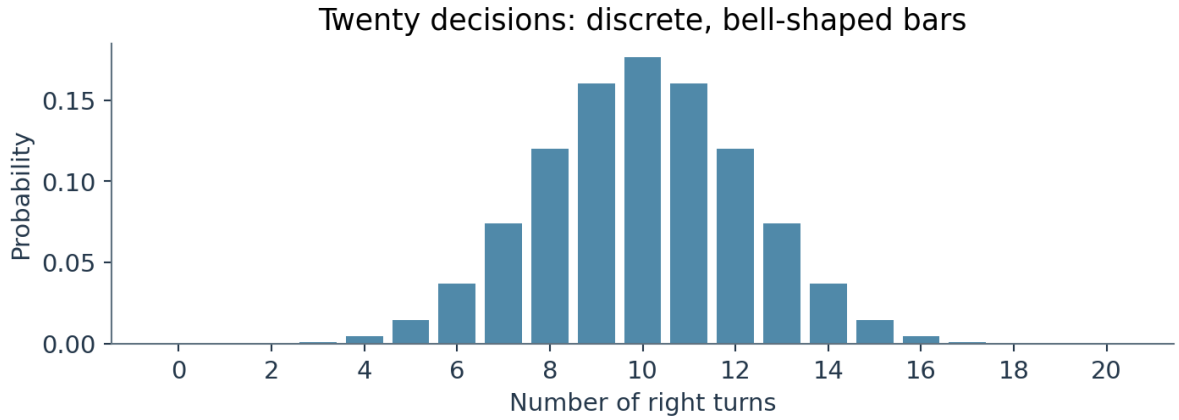
שלב ראשון: בארבע פניות יש $2^4 = 16$ מסלולים אפשריים אם כל ההחלטות בעלות הסתברות שווה. שלב שני: בוחרים אילו שתי פניות יהיו ימינה. מספר הדרכים הוא 6. שלב שלישי: כל מסלול יחיד הוא בעל הסתברות 1/16. שלב רביעי: מחברים את ההסתברויות של ששת המסלולים המתאימים.

$$P(K = 2) = C(4,2) \cdot (1/2)^4 = 6/16 = 0.375$$



איור — בארבע פניות יש חמש תוצאות אפשריות. ההסתברות להגיע למרכז גדולה מההסתברות להגיע לקצה.

כאשר מגדילים את מספר הפניות, מספר המיקומים האפשריים גדל, והעמודות מקבלות צורה דמוית פעמון. אבל שימו לב: התוצאה עדיין בדידה, והעמודות אינן מתחלפות אוטומטית בהתפלגות נורמלית. במצבים מתאימים ההתפלגות הבינומית יכולה להתקרב להתפלגות נורמלית לאחר תקנון מתאים.



איור — עשרים פניות: ההתפלגות עדיין בדידה, אבל צורתה מזכירה יותר את עקומת הפעמון.

```
from math import comb
n, p = 4, 0.5
k = 2
probability = comb(n, k) * p**k * (1-p)**(n-k)
print(probability)    # 0.375
```

5. מהי התפלגות נורמלית?

התפלגות נורמלית היא מודל הסתברות רציף, סימטרי ובעל עקומה בצורת פעמון. שיא העקומה נמצא במרכז, ושני הצדדים יורדים בהדרגה ככל שמתרחקים מן המרכז. המשמעות אינה שרוב הערכים חייבים להיות זהים; המשמעות היא שערכים הקרובים למרכז צפויים להיות נפוצים יותר, וערכים רחוקים ממנו פחות נפוצים לפי המודל.

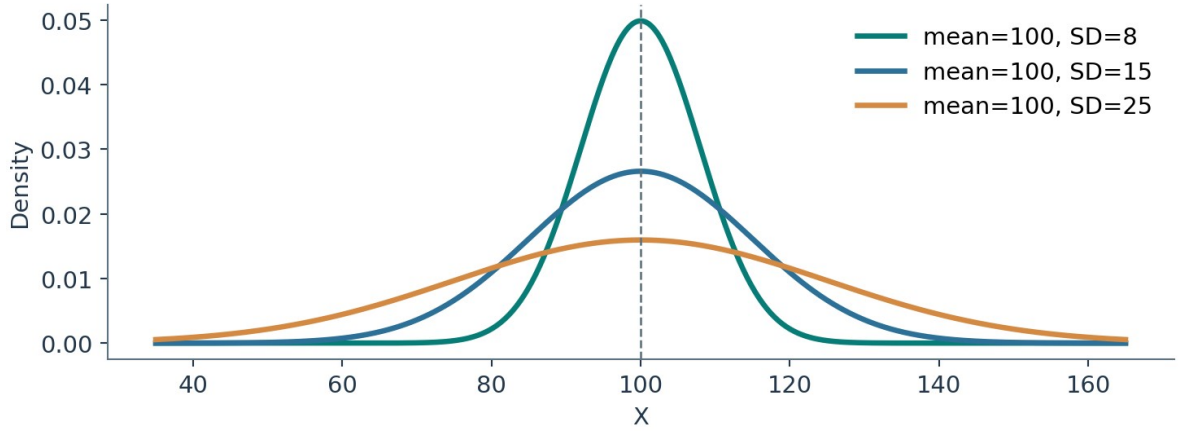
במודל נורמלי הממוצע, החציון והשכיח נמצאים באותה נקודה. אולם עובדה זו לבדה אינה מספיקה כדי להוכיח נורמליות. למשל אפשר לבנות התפלגות סימטרית עם שני שיאים שלא תהיה נורמלית. לכן אנו בוחנים גם את צורת הנתונים ואת ההקשר שבו נאספו.

5.1 שני הפרמטרים שמגדירים את העקומה

הפרמטר μ (מיו) הוא הממוצע ומגדיר את המיקום של מרכז העקומה. הגדלת μ מזיזה את העקומה ימינה בלי לשנות את צורתה אם σ נשאר קבוע. הפרמטר σ (סיגמא) הוא סטיית התקן ומגדיר את רוחב העקומה. σ קטן יוצר עקומה צרה וגבוהה יותר; σ גדול יוצר עקומה רחבה ונמוכה יותר. בכל המקרים השטח הכולל שמתחת לעקומה הוא 1.

$$X \sim N(\mu, \sigma^2) \quad \text{with } \sigma > 0$$

שימו לב לסימון: בתוך N מופיעה השונות σ^2, ולא סטיית התקן σ. לעומת זאת, כשמזינים את הפרמטר scale לפונקציות של scipy.stats.norm, נותנים את סטיית התקן σ עצמה.



איור — שלוש עקומות בעלות אותו ממוצע וסטיות תקן שונות: פיזור רחב יותר פירושו עקומה שטוחה יותר.

5.2 דוגמה מחיי התלמידים

נניח שבשתי קבוצות ממוצע זמן התגובה הוא 100 מילישניות. בקבוצה א׳ סטיית התקן 8, ובקבוצה ב׳ סטיית התקן 25. בשתי הקבוצות המרכז זהה, אבל בקבוצה ב׳ צפויים ערכים המפוזרים על טווח רחב בהרבה. אם אנו מפתחים משחק שדורש תגובה יציבה, חשוב לנו לדעת לא רק את זמן התגובה הממוצע אלא גם את פיזורו.

שאלת הבנה אם נזיז את כל המדידות בקבוצה למעלה ב־20 יחידות, מה יקרה לממוצע ומה יקרה לסטיית התקן? נסו לנמק בלי לחשב. התשובה נובעת מן ההבדל בין מיקום המרכז לבין מרחקים יחסיים מן המרכז.

6. ציון תקן Z — לדבר באותה שפה על ציונים שונים

דודו קיבל 85 במבחן מתמטיקה ו־90 במבחן מדעי המחשב. באיזה מקצוע הוא הצליח יחסית לקבוצה? לכאורה 90 גבוה מ־85, אבל ייתכן שבמדעי המחשב כולם קיבלו ציונים גבוהים ואילו במתמטיקה הציון 85 נמצא הרבה מעל הממוצע. לכן אי אפשר להשוות כראוי לפני שמתחשבים במרכז ובפיזור של כל מבחן.

ציון תקן Z מתאר בכמה סטיות תקן נמצא ערך מסוים מעל הממוצע או מתחתיו. הוא חסר יחידות: הוא אינו מדוד בנקודות, שקלים או מילישניות, אלא במספר סטיות תקן.

$$z = (x - \mu) / \sigma \quad (\sigma > 0)$$

בדוגמת זמני התגובה	משמעות	סימון
130 מילישניות	המדידה שאנו רוצים לתקנן	x

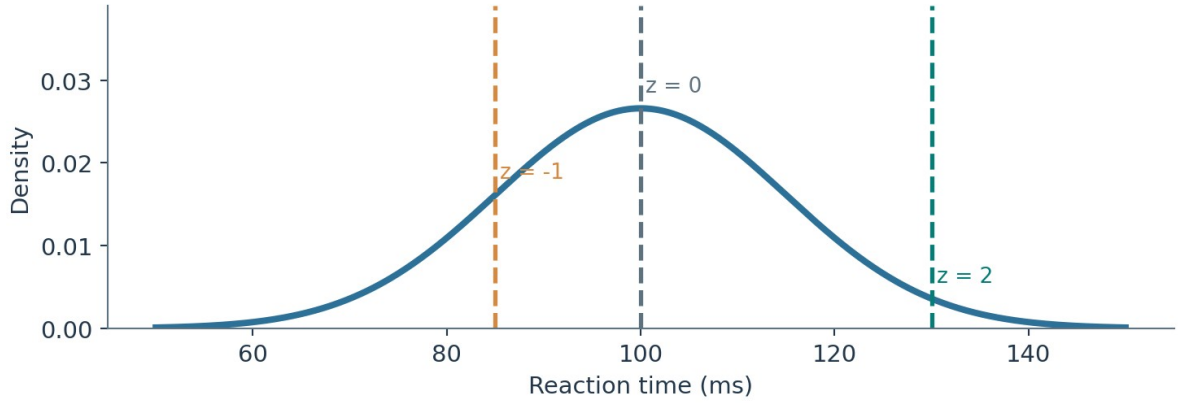
μ	ממוצע ההתפלגות	100 מילישניות
σ	סטיית התקן	15 מילישניות
z	מרחק מהממוצע ביחידות סטיית תקן	2

6.1 חישוב ידני מלא

נחשב את ציון התקן של זמן תגובה 130 מילישניות כאשר הממוצע 100 וסטיית התקן 15. שלב ראשון: נחשב בכמה המדידה גבוהה מהממוצע — 30 מילישניות. שלב שני: נחלק את ההפרש ב־15 מילישניות, כי כל סטיית תקן מייצגת קפיצה של 15 מילישניות. נקבל 2. שלב שלישי: ננסח פירוש מילולי.

$$z = (130 - 100) / 15 = 30 / 15 = 2$$

פירוש: זמן תגובה של 130 מילישניות נמצא שתי סטיות תקן מעל הממוצע במודל שהוגדר. הוא איטי יותר מהממוצע, משום שבמשימה זו ערך זמן גבוה פירושו תגובה איטית יותר. ערך Z חיובי אינו בהכרח ״טוב״, וערך Z שלילי אינו בהכרח ״רע״ — צריך להבין מה המשתנה מודד.



איור — הסימונים z=-1, z=0 ו־z=2 ממוקמים על ציר הזמן ומאפשרים לפרש מרחקים מהממוצע.

6.2 חישוב הפוך: משחזרים ערך מתוך ציון תקן

לעיתים אנחנו מכירים את ציון התקן ורוצים לדעת מה היה הערך המקורי. נסדר מחדש את הנוסחה: נכפיל את z בסטיית התקן ונוסיף את הממוצע. למשל, אם z=-1 במודל זמני התגובה, נקבל זמן תגובה של 85 מילישניות.

$$x = \mu + z \cdot \sigma \quad \rightarrow \quad x = 100 + (-1) \cdot 15 = 85$$

6.3 השוואה בין שני מבחנים

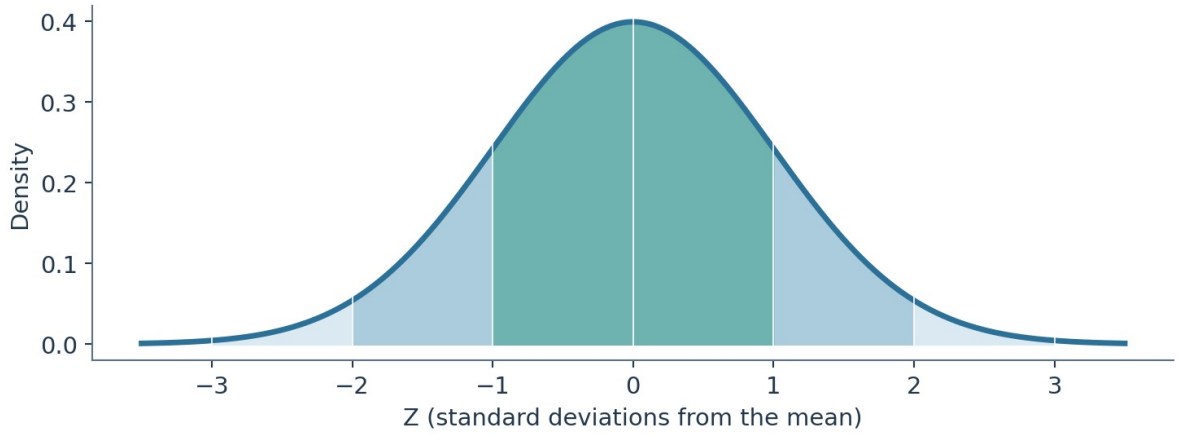
במתמטיקה דודו קיבל 85, ממוצע הכיתה 70 וסטיית התקן 10. לכן Z במתמטיקה הוא 1.5. במדעי המחשב קיבל 90, ממוצע הכיתה 85 וסטיית התקן 10. לכן Z במדעי המחשב הוא 0.5. למרות הציון הגולמי הגבוה במדעי המחשב, במתמטיקה דודו רחוק יותר מעל הממוצע של כיתתו. זו מסקנה יחסית, שאינה מתייחסת לקושי של שני המבחנים או לאיכותם.

Math: $z = (85-70)/10 = 1.5$ | CS: $z = (90-85)/10 = 0.5$

הבחנה חשובה אפשר לחשב Z גם עבור נתונים שאינם מתפלגים נורמלית. אבל כדי להמיר Z להסתברות באמצעות טבלת ההתפלגות הנורמלית, נדרשת ההנחה המתאימה על ההתפלגות.

7. כלל 68–95–99.7 — קירוב שימושי לזכירה ולהבנה

בכל התפלגות נורמלית, בקירוב 68% מן הערכים נמצאים עד סטיית תקן אחת לכל צד של הממוצע; כ־95% עד שתי סטיות תקן; וכ־99.7% עד שלוש סטיות תקן. זהו כלל אמפירי המתאר שטחים מתחת לעקומה נורמלית — לא כלל התקף לכל מאגר נתונים.



איור — האזורים סביב המרכז: בין −1 ל־1, בין −2 ל־2 ובין −3 ל־3 סטיות תקן.

תחום תקני	אחוז בקירוב	תחום כש־μ=100 ו־σ=15
$Z \le 1$ ≥ −1	68%	85 עד 115
$Z \le 2$ ≥ −2	95%	70 עד 130
$Z \le 3$ ≥ −3	99.7%	55 עד 145

למשל, לפי הכלל, כ־95% מזמני התגובה במודל שלנו נמצאים בין 70 ל־130 מילישניות. אין להסיק מכך ש־95% מכל קבוצת תלמידים בעולם יגיבו בטווח הזה; הטווח מתייחס להתפלגות ההיפותטית עם הפרמטרים שבחרנו.

7.1 מתי הכלל אינו מספיק?

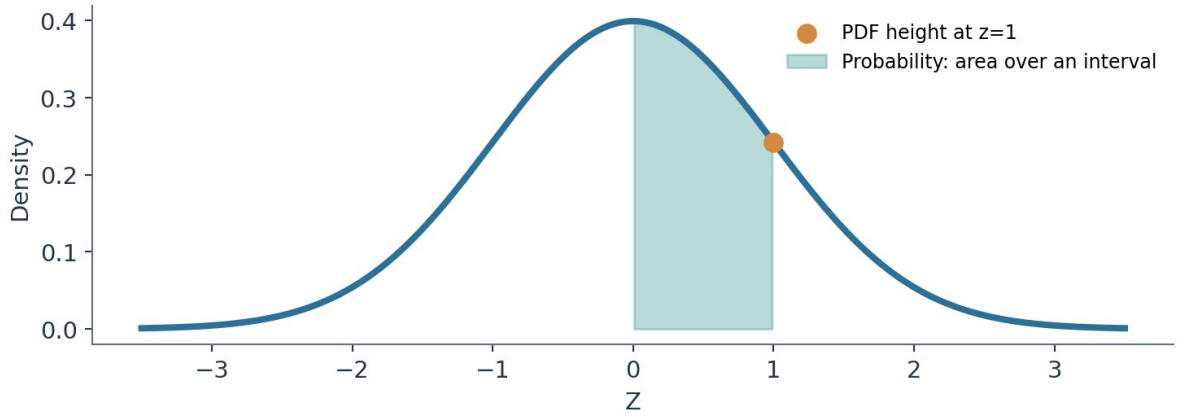
אם מבקשים לחשב הסתברות מתחת לערך שנמצא ב־1.3 סטיות תקן מעל הממוצע, כלל 68–95–99.7 אינו נותן תשובה ישירה. הוא גם אינו מספיק כדי להבחין במדויק בין צד ימין לשמאל סביב סף כלשהו. במקרים כאלה נשתמש בפונקציית ההתפלגות המצטברת או בטבלת Z מתאימה.

8. מהו PDF? צפיפות אינה הסתברות של נקודה

במשתנה רציף קשה לשאול ״מה ההסתברות שזמן התגובה יהיה בדיוק 100 מילישניות?״. לפי מודל הסתברות רציף, ההסתברות לערך יחיד ומדויק היא אפס. לעומת זאת, אפשר לשאול ״מה ההסתברות לזמן תגובה בין 95 ל־105 מילישניות?״ והתשובה תהיה חיובית. בדומה לכך, בקו מספרים יש אינסוף נקודות בתוך קטע, ולכן נקודה אחת אינה תופסת אורך.

PDF הם ראשי תיבות של Probability Density Function — פונקציית צפיפות הסתברות. גובה העקומה בנקודה מסוימת מתאר צפיפות, ולא הסתברות לאותה נקודה. כדי לחשב הסתברות לקטע יש לחשב את השטח שמתחת לעקומה לאורך הקטע. הסכום של כל השטח מתחת לעקומה על פני כל הערכים האפשריים שווה 1.

$$P(a \le X \le b) = \text{area under } f(x) \text{ from } a \text{ to } b$$



איור — הנקודה הכתומה מייצגת גובה צפיפות. האזור הירקרק מייצג הסתברות של תחום.

8.1 נוסחת הצפיפות הנורמלית והמשמעות של רכיביה

$f(x) = [1 / (\sigma\sqrt{(2\pi)})] \cdot \exp[-(x-\mu)^2 / (2\sigma^2)]$

הנוסחה אולי נראית עמוסה, ולכן נפענח אותה במקום לשנן אותה בלי משמעות. הביטוי $x-\mu$ הוא המרחק החתום מן הממוצע. העלאה בריבוע מבטיחה שמרחק שלילי וחיובי באותו גודל יתרמו באופן זהה לצפיפות — הסבר לסימטריה. החלוקה ב־$2\sigma^2$ מתאימה את קצב הירידה מן המרכז לפיזור. הפונקציה המעריכית exp גורמת לצפיפות לרדת ככל שמתרחקים מן המרכז. המקדם שלפני exp מבטיח שהשטח הכולל יהיה 1.

אין צורך לחשב את כל הביטוי הזה ביד בכל שאלה. עליכם להבין מה תפקידו, מהם הפרמטרים שלו, וכיצד עוברים ממנו להסתברות. כאשר נדרש ערך מספרי מדויק נשתמש בטבלה או בספריית Python.

8.2 למה PDF עשוי להיות גדול מ־1?

המספר שעל הציר האנכי הוא צפיפות ביחידות הפוכות ליחידות של X, ולכן אינו חייב להיות בין 0 ל־1. כאשר ההתפלגות צרה מאוד, גובה העקומה עשוי לעבור את 1; למרות זאת השטח הכולל תמיד שווה 1. זה שונה מהסתברות, שתמיד נמצאת בתחום שבין 0 ל־1.

Example: $\sigma = 0.1 \rightarrow f(\mu) = 1 / (0.1\sqrt{(2\pi)}) \approx 3.989$

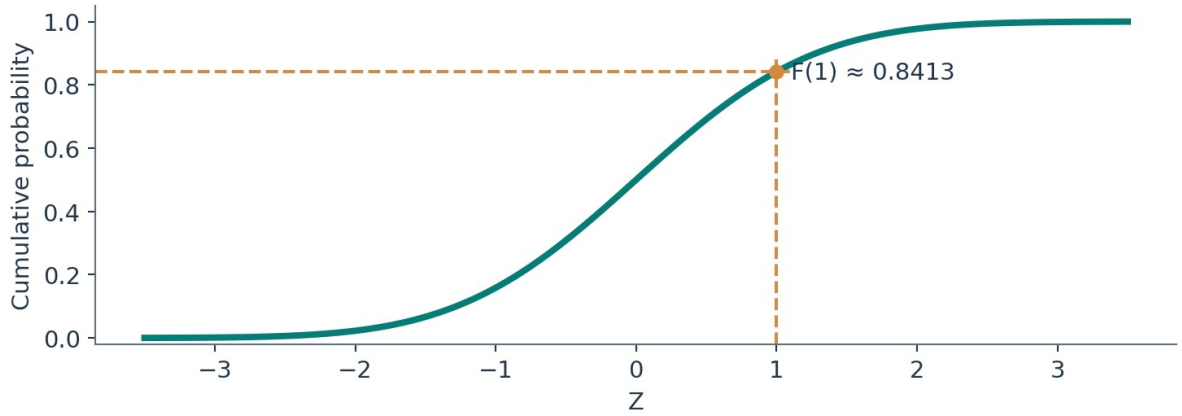
אין כאן סתירה: העקומה צרה מאוד, ולכן היא יכולה להיות גבוהה ועדיין להקיף שטח כולל של 1.

9. מהו CDF? הסתברות מצטברת עד סף

דודו רוצה לדעת איזה חלק מזמני התגובה צפויים להיות עד 115 מילישניות. במקום להתבונן בגובה הצפיפות מעל 115, אנחנו סוכמים את כל השטח שמשמאל לסף זה. בדיוק לכך נועדה פונקציית ההתפלגות המצטברת — Cumulative Distribution Function, או CDF.

$F(a) = P(X \leq a)$

ככל שנבחר סף a גדול יותר, אנחנו כוללים יותר ערכים אפשריים; לכן CDF לעולם אינה יורדת כאשר a גדל. בקצה השמאלי הרחוק היא מתקרבת ל־0 ובקצה הימני הרחוק היא מתקרבת ל־1. בנורמלית התקנית מקובל לסמן את ה־CDF באות הגדולה Φ (פי).



איור — ה־CDF מחזירה את ההסתברות המצטברת שמשמאל לסף; עבור Z=1 היא בקירוב 0.8413.

9.1 חישוב ידני באמצעות טבלת Z

נניח ש־Z מתפלג נורמלית תקנית. בטבלה המצטברת נמצא כי Φ(1)≈0.8413. פירוש התוצאה הוא שכ־84.13% מן הערכים נמצאים משמאל ל־Z=1. בעזרת סימטריה של העקומה נמצא כי Φ(−1)≈0.1587, וכי בדיוק מחצית השטח נמצאת משמאל ל־0.

$$\Phi(1) \approx 0.8413 \quad | \quad \Phi(-1) \approx 0.1587 \quad | \quad \Phi(0) = 0.5$$

שימו לב להבדל בין הסימון ≈ לבין הסימון =. חלק מן הערכים הם קירובים מעוגלים מטבלה או מחישוב, ולכן עדיף לכתוב ≈ כאשר מציגים אותם ארבע ספרות אחרי הנקודה.

```
from scipy.stats import norm
print(norm.cdf(1))       # approx. 0.8413
print(norm.cdf(-1))      # approx. 0.1587
```

10. שלושה סוגי שאלות הסתברות

רוב שאלות ההסתברות בפרק נפתרות על ידי הבנת הצורה בגרף, תרגום השאלה לשפה מתמטית, תקנון אם צריך, ואז שימוש ב־CDF. במקום לזכור כמה נוסחאות ללא קשר ביניהן, נלמד לזהות את שלושת המקרים היסודיים: הסתברות משמאל לסף, מימין לסף, ובין שני ספים.

10.1 הסתברות מתחת לסף

נניח שאנחנו רוצים לדעת מה ההסתברות לזמן תגובה שאינו עולה על 115 מילישניות. במודל שלנו 115 גבוה מהממוצע ב־15 מילישניות, כלומר בסטיית תקן אחת. לכן נחשב Z=1, ונסתמך על הערך המצטבר של הנורמלית התקנית.

$$P(X \le 115) = P(Z \le 1) = \Phi(1) \approx 0.8413$$

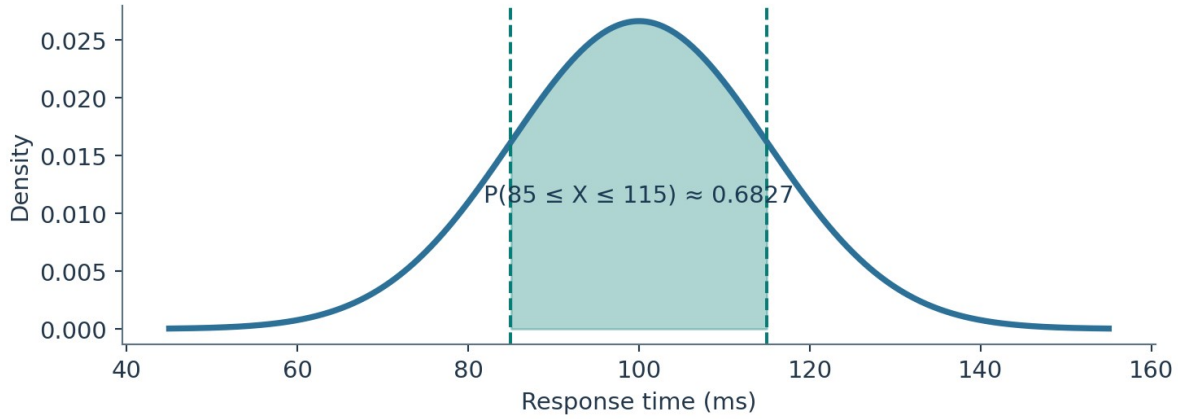
פירוש: אם המודל הנורמלי תקף, כ־84.13% מן התגובות צפויות להיות לא ארוכות מ־115 מילישניות. זו טענה הסתברותית על המודל, לא התחייבות שכל סדרת ניסויים קטנה תניב בדיוק את האחוז הזה.

10.2 הסתברות בין שני ספים

נרצה לחשב את ההסתברות לזמן תגובה בין 85 ל־115. תחילה נהפוך כל סף לציון תקן: 85 הוא סטיית תקן אחת מתחת לממוצע ולכן Z=−1; 115 הוא סטיית תקן אחת מעל הממוצע ולכן Z=1. כדי לקבל את השטח שביניהם נחסיר מן השטח שמשמאל ל־115 את השטח שמשמאל ל־85.

$$P(85 \le X \le 115) = \Phi(1) - \Phi(-1)$$

$$\approx 0.8413 - 0.1587 = 0.6826 \quad (\text{actual} \approx 0.6827)$$



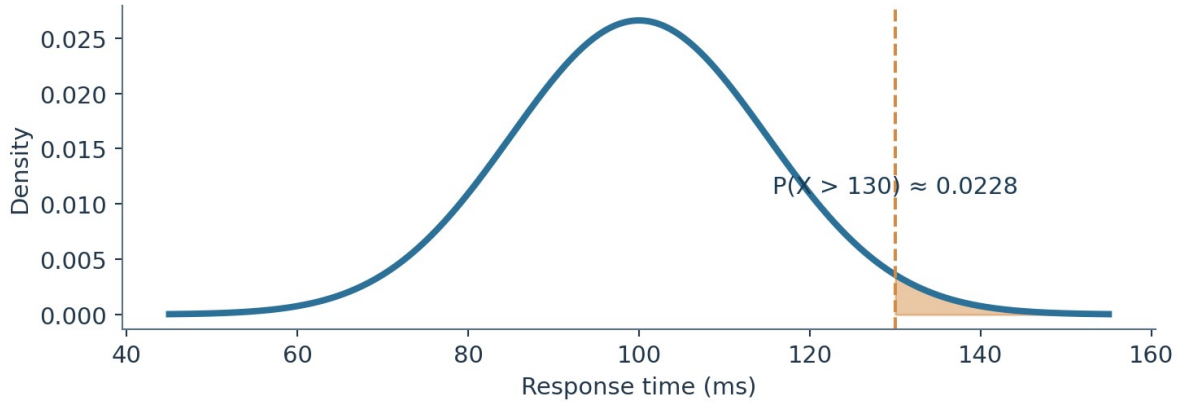
איור — השטח הצבוע הוא בין שני הספים, ולא השטח שמשמאל או מימין לכל העקומה.

הבדל של מאית אחוז בין חישוב מטבלה מעוגלת לבין ערך ספרייה מדויק יותר הוא טבעי. לכן בעת השוואת תשובות נציין את רמת הדיוק שהתבקשה ולא נסיק שקיימת טעות רק בגלל עיגול ביניים.

10.3 הסתברות מעל סף

עכשיו דודו רוצה לדעת מה הסיכוי לזמן תגובה ארוך מ־130 מילישניות. ערך זה נמצא שתי סטיות תקן מעל הממוצע, ולכן $Z=2$. ה־CDF נותנת את ההסתברות להיות משמאל לסף; אנחנו צריכים את השטח שנשאר מימין, ולכן נחסיר מ־1.

$$P(X > 130) = 1 - \Phi(2) \approx 1 - 0.97725 = 0.02275$$



איור — שטח הזנב הימני אחרי 130 מילישניות: בקירוב 2.28% מן המודל הנורמלי.

```
from scipy.stats import norm
mu, sigma = 100, 15
left = norm.cdf(115, loc=mu, scale=sigma)
between = norm.cdf(115,mu,sigma)-norm.cdf(85,mu,sigma)
right = norm.sf(130, loc=mu, scale=sigma)
print(left, between, right)
```

הפעולה norm.sf מחזירה את ההסתברות בזנב הימני (Survival Function). מבחינה מתמטית זו אותה הסתברות כמו $1-\text{CDF}$, אך בחישובים של הסתברויות קטנות מאוד היא עשויה להיות מדויקת יותר מספרית.

11. דוגמת ההשתלמות — תוחלת 2.5 וסטיית תקן 1.5

בחומר ההשתלמות מופיעה דוגמה נוספת עם ממוצע 2.5 וסטיית תקן 1.5. חשוב להתרגל לעבור בין סיפורים שונים בלי לשנות את שיטת הפתרון. נחשב את ההסתברות לקבל ערך עד 3, ואת ההסתברות לקבל ערך בין 3 ל־4. המספרים בתרגיל מתארים משתנה מופשט, ללא יחידות מסוימות.

$X \sim N(2.5, 1.5^2)$

ראשית, עבור הערך 3, נחשב $Z=(3-2.5)/1.5\approx 0.3333$. כיוון שהערך גבוה רק בשליש סטיית תקן מן הממוצע, מצופה שההסתברות משמאל לו תהיה קצת מעל חצי. החישוב הספרייתי אכן נותן בקירוב 0.63056.

$P(X \le 3) = \text{norm.cdf}(3, \text{loc}=2.5, \text{scale}=1.5) \approx 0.63056$

כדי לחשב הסתברות בין 3 ל־4, נחשב את השטח המצטבר עד 4 ונחסיר ממנו את השטח עד 3. התוצאה המתקבלת היא בקירוב 0.21079. שימו לב שלא מחסרים את שני ערכי הספים עצמם; אנו מחסרים הסתברויות מצטברות.

$P(3 \le X \le 4) = F(4) - F(3) \approx 0.21079$

```
from scipy.stats import norm
mu, sigma = 2.5, 1.5
p_left = norm.cdf(3, loc=mu, scale=sigma)
p_between = norm.cdf(4,mu,sigma) - norm.cdf(3,mu,sigma)
print(round(p_left, 5), round(p_between, 5))
```

12. אחוזונים: לחשב סף לפי ההסתברות (העמקה)

עד עכשיו ידענו ערך x וחיפשנו את ההסתברות להיות מתחתיו. לפעמים השאלה הפוכה: מהו הזמן שמתחתיו נמצאים, לפי המודל, 90% מן התגובות? זהו אחוזון 90. כדי למצוא אותו מחפשים את ערך הסף שמתאים להסתברות מצטברת של 0.9.

ב־Python הפונקציה norm.ppf היא הפעולה ההפוכה של CDF: מקבלים הסתברות מצטברת ומחזירים ערך. עבור המודל שלנו אחוזון 90 הוא בערך 119.22 מילישניות. פירושו ש־90% מן הערכים מתחת לסף זה לפי המודל, וכ־10% מעליו.

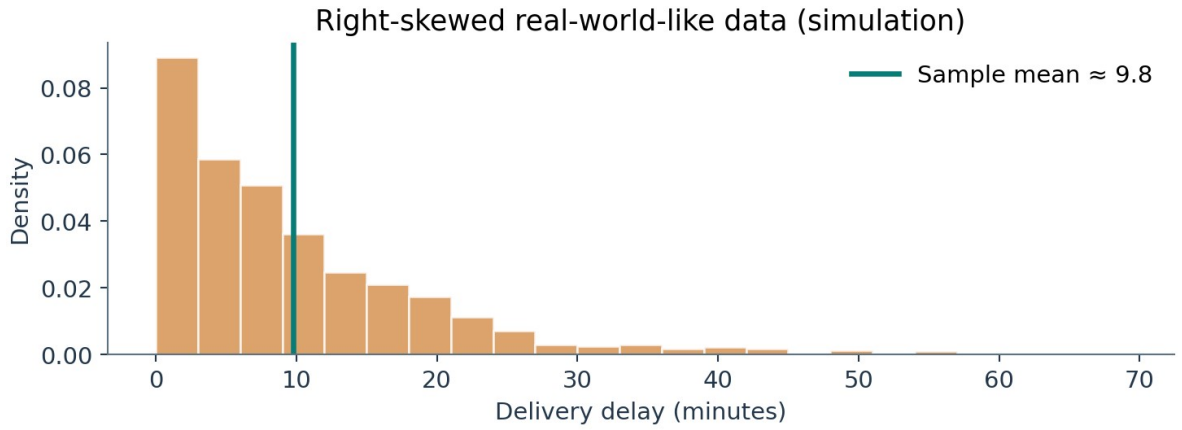
$F(x_{90}) = 0.90 \quad \rightarrow \quad x_{90} \approx 119.22 \text{ ms}$

```
from scipy.stats import norm
print(norm.ppf(0.90, loc=100, scale=15))
```

מעמד הנושא אחוזונים ושימוש ב־ppf מובאים כאן כהעמקה יישומית המסייעת לתכנון ספים בפרויקטים, ולא כקביעה שהם דרישת חובה בבחינת יחידה 5.

13. האם כל הנתונים בעולם האמיתי מתפלגים נורמלית?

בהחלט לא. ההנחה הנורמלית מתאימה לעיתים למדידות המושפעות מצירוף של גורמים קטנים רבים, אך נתוני אמת יכולים להיות מוטים, מחולקים לקבוצות, חסומים מלמטה, או מושפעים מאירועים נדירים. למשל זמני משלוחים מכילים בדרך כלל הרבה משלוחים רגילים ומעט משלוחים שהתעכבו מאוד. התפלגות כזו עשויה להיות מוטה לימין ולא סימטרית.



איור — נתוני סימולציה של זמני איחור: רוב המדידות קטנות ומעטן גדולות. זו צורה אסימטרית שאינה נורמלית.

גם מחירי דירות, מספר צפיות בסרטון, מספר כניסות לאתר ומספר תקלות אינם חייבים להיות נורמליים. אם נטעה בבחירת המודל, עלולות להתקבל הסתברויות לא אמינות. לכן לפני שימוש ב־CDF נורמלי חשוב להתבונן בהיסטוגרמה, לבדוק חריגים, להבין את היחידות ולשקול אם נתוני המאגר מייצגים אוכלוסייה אחידה או תערובת של קבוצות.

13.1 האם Z-score הופך נתונים לנורמליים?

לא. תקנון באמצעות החסרת ממוצע וחלוקה בסטיית תקן משנה את המיקום ואת קנה המידה של הנתונים, אך אינו משנה בדרך כלל את צורת ההתפלגות הבסיסית. נתונים מוטים יישארו מוטים גם לאחר תקנון. חשוב להפריד בין פעולה חשבונית של תקנון לבין הנחה הסתברותית על התפלגות.

$z_i = (x_i - \text{mean_train}) / \text{std_train}$

בלמידת מכונה יש עיקרון נוסף: אם משתמשים בתקנון לפני אימון מודל, את הממוצע ואת סטיית התקן יש להתאים על נתוני האימון בלבד. לאחר מכן משתמשים באותם פרמטרים כדי לתקנן את נתוני הבדיקה. אחרת מידע מן הבדיקה עלול להשפיע על האימון ולהטות את ההערכה. את המימוש המפורט נלמד בפרק 2.3.

14. משפט הגבול המרכזי — מדוע ממוצעים מתנהגים אחרת?

דודו מודד כמה דקות נדרשו למשלוחים להגיע לבית הספר. נניח שזמני ההמתנה הבודדים אינם נורמליים: רובם קצרים, אך לעיתים יש עיכובים ארוכים. אם נמדוד משלוח יחיד, נקבל ערך מן ההתפלגות המוטה. אם נאסוף הרבה מדגמים עצמאיים, ובכל מדגם נחשב את ממוצע הזמנים, התפלגות הממוצעים תתנהג אחרת.

משפט הגבול המרכזי אומר שבתנאים מקובלים — למשל תצפיות עצמאיות ובעלות אותה התפלגות עם שונות סופית — התפלגות ממוצע המדגם מתקרבת לנורמלית ככל שגודל המדגם גדל, לאחר תקנון מתאים. אין פירושו שהמדידות הבודדות הופכות נורמליות או שכל מדגם קטן יספק קירוב טוב.

14.1 מהו שגיאת תקן (Standard Error)?

סטיית התקן σ מתארת את הפיזור של מדידה בודדת באוכלוסייה. שגיאת התקן של ממוצע המדגם (SE) מתארת עד כמה ממוצעים ממדגמים שונים צפויים להשתנות סביב ממוצע האוכלוסייה. כאשר המדידות בלתי תלויות ובעלות שונות סופית, הנוסחה הבאה קובעת את הפיזור של ממוצע מדגם בגודל n:

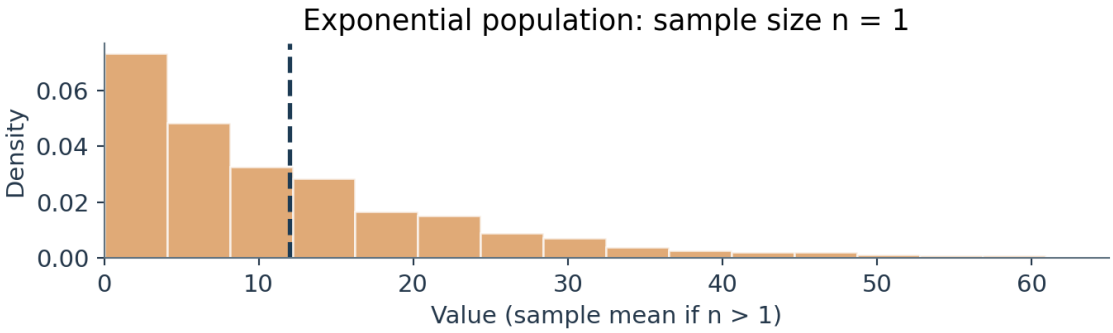
$SE(\bar{x}) = \sigma / \sqrt{n}$

אם סטיית התקן של זמן המתנה בודד היא 12 דקות, במדגם של 9 משלוחים שגיאת התקן תהיה 4 דקות. במדגם של 36 משלוחים היא תהיה 2 דקות. כלומר הגדלת מספר התצפיות פי ארבעה הקטינה את שגיאת התקן פי שניים, לא פי ארבעה.

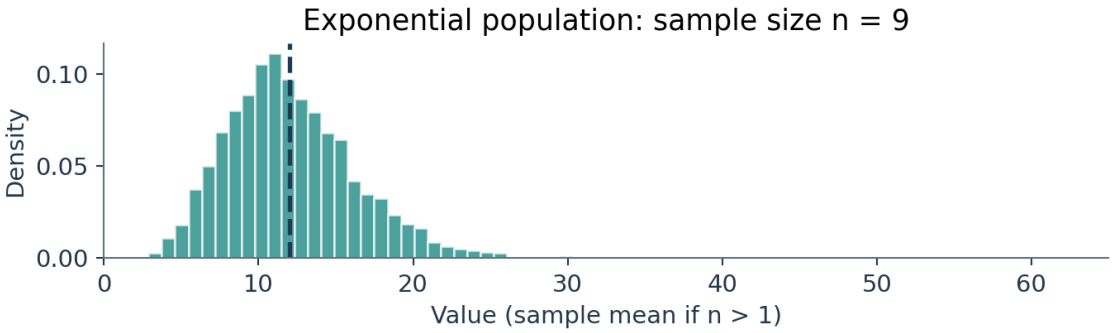
$n = 9$: $SE = 12/\sqrt{9} = 4$ | $n = 36$: $SE = 12/\sqrt{36} = 2$

14.2 סימולציה חזותית — שלושה גדלי מדגם

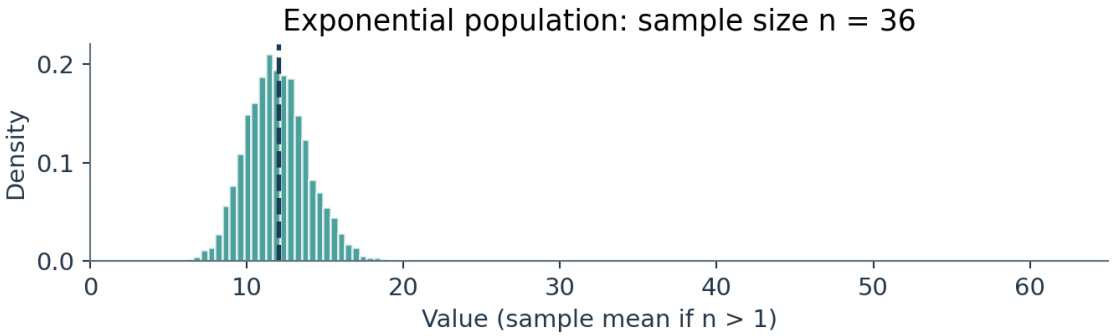
באיורים הבאים נוצרו באופן מלאכותי מדידות בודדות מהתפלגות מעריכית בעלת ממוצע 12. עבור כל גודל מדגם נלקחו שוב ושוב מדגמים עצמאיים ונחשב ממוצע כל מדגם. כאשר n=1 הגרף מוטה מאוד. כאשר n=9 גרף הממוצעים כבר סימטרי יותר, וכאשר n=36 הוא מרוכז יותר ליד 12 ונראה קרוב יותר לצורת פעמון.



איור — מדידה בודדת: התפלגות מקורית אסימטרית מאוד.



איור — ממוצעים של מדגמים בגודל 9: פיזור קטן יותר וצורה סימטרית יותר.



איור — ממוצעים של מדגמים בגודל 36: ריכוז חזק יותר סביב ממוצע האוכלוסייה.

```
import numpy as np
import matplotlib.pyplot as plt
rng = np.random.default_rng(42)
for n in [1, 9, 36]:
    samples = rng.exponential(scale=12, size=(1000, n))
    sample_means = samples.mean(axis=1)
    plt.hist(sample_means, bins=30)
    plt.title(f"Means of samples, n={n}")
```

```
    plt.show()
```

זהירות מדעית ההתקרבות לנורמלית אינה מבטיחה שכל נתון חדש יתנהג נורמלית. המשפט עוסק בהתפלגות ממוצעים של מדגמים חוזרים תחת תנאים מתאימים. כמו כן, כאשר קיימות תלות חזקה או שונות אינסופית, ניסוח המשפט הפשוט אינו חל כפי שהוא.

15. מה לומדים מכאן על AI ועל פרויקטים אמיתיים?

סטטיסטיקה אינה שלב טכני לפני אימון מודל; היא חלק מהבנת הבעיה. כאשר מפתח AI בוחר מאפיינים, הוא צריך לשאול באילו יחידות הם נמדדים, מהו תחום הערכים, עד כמה הם מפוזרים והאם קיימים חריגים. הנחת מודל לא מתאימה עלולה להוביל להחלטה שגויה גם כשהקוד תקין.

מה חובה לבדוק לפני שימוש	איך הפרק מסייע	יישום
האם זמני התגובה סימטריים והאם יש חסם תחתון	חישוב אחוזונים וספים	זמני תגובה במשחק
הטיה ימינה, ימים עמוסים, קבוצות שונות	אמידת סיכוי לאיחור	חיזוי זמני משלוח
חריג אינו בהכרח נתון שגוי	זיהוי Z גבוה או נמוך	איתור חריגים
להימנע מזליגת מידע	תקנון תכונות לפי נתוני אימון	הכנת נתונים ל־KNN
איך נבחרו המדגמים והאם הם עצמאיים	שגיאת תקן של ממוצע מדדים	ניסוי מודל AI

15.1 למה Z=3 אינו ״מחיקה אוטומטית״?

ערך Z של 3 הוא רחוק מן המרכז, אבל ההחלטה אם הוא חריג בלתי רצוי תלויה בשאלה: האם הוא שגיאת הקלדה, אירוע אמיתי, או מצב חשוב שהמערכת דווקא חייבת לזהות? במערכת לזיהוי תקלות, מקרים נדירים עשויים להיות המקרים החשובים ביותר. לכן החלטה על סינון ערכים מחייבת בדיקה והצדקה ולא רק כלל מספרי.

16. מימוש ב־Python — מהגרף אל התוצאה

נכתוב תכנית אחת שמחזיקה את הפרמטרים μ ו־σ, יוצרת עקומת PDF, מחשבת הסתברויות בשתי דרכים ומבצעת בדיקת היגיון. נעדיף פקודות שקופות וקריאות. לכל תוצאה מספרית נצרף פירוש מילולי ולא נסתפק ב־print של מספר.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import norm

mu = 100
sigma = 15
x = np.linspace(50, 150, 500)
y = norm.pdf(x, loc=mu, scale=sigma)

plt.plot(x, y, label="Normal density")
plt.axvline(mu, linestyle="--", label="Mean")
plt.xlabel("Response time (ms)")
plt.ylabel("Density")
plt.legend()
plt.show()
```

```
p_left = norm.cdf(115, loc=mu, scale=sigma)
p_right = norm.sf(130, loc=mu, scale=sigma)
print("P(X <= 115) =", round(p_left, 4))
print("P(X > 130) =", round(p_right, 4))
```

בדקו את סבירות התוצאות: הסף 115 גבוה מהממוצע, ולכן ההסתברות להיות מתחתיו חייבת להיות גדולה מחצי. הסף 130 גבוה בשתי סטיות תקן, ולכן ההסתברות להיות מעליו צפויה להיות קטנה יחסית. בדיקת היגיון לפני קבלה של פלט היא הרגל חיוני למפתח AI.

17. טעויות שכיחות ואיך נמנעים מהן

טעות נפוצה	למה זו טעות	מה עושים במקום
מחליפים בין σ לבין σ^2	הפונקציה צריכה סטיית תקן, לא שונות	בודקים את פרטי הסימון ואת loc/scale
מפרשים norm.pdf כ־P(X=x)	גובה אינו שטח במשתנה רציף	מחשבים CDF או הפרש CDF
מחברים הסתברויות מצטברות של שני ספים	השטחים חופפים	לחישוב תחום מחסרים F(b)−F(a)
משתמשים ב־CDF לסף ימני בלי להשלים ל־1	CDF סופרת משמאל	משתמשים ב־CDF−1 או norm.sf
מחשבים Z בכיוון הפוך	סימן המרחק משתנה	כותבים תמיד x−μ לפני החלוקה
טוענים שתקנון יוצר נורמליות	תקנון אינו משנה את צורת ההתפלגות	בודקים היסטוגרמה והנחות
מניחים שכל ממוצע בודד נורמלי	CLT עוסק בהתפלגות ממוצעי מדגמים	מבחינים בין X לבין $\bar{x}$
משתמשים בנתוני Test לתקנון	זו זליגת מידע	מתאימים פרמטרים על Train בלבד

18. מילון מושגים: בואו נסכם בשפה ברורה

מושג	פירוש לתלמיד
התפלגות נורמלית	מודל רציף, סימטרי, בצורת פעמון, המוגדר על ידי μ ו־σ.
ממוצע μ	מרכז ההתפלגות.
סטיית תקן σ	מדד הפיזור של הערכים סביב המרכז.
Z-score	מספר סטיות התקן של ערך מן הממוצע.
PDF	פונקציית צפיפות: השטח תחתיה לאורך קטע הוא הסתברות.
CDF	ההסתברות המצטברת להיות קטן או שווה לסף נתון.
Survival Function	ההסתברות להיות מעל סף — הזנב הימני.
אחוזון	סף שמתחתיו נמצא אחוז מסוים מן ההתפלגות.
CLT	תחת תנאים מתאימים, התפלגות ממוצעי מדגמים מתקרבת לנורמלית.

סטיית התקן של התפלגות ממוצע המדגם.	SE

19. תרגול עצמאי — רמה א׳: יסודות והבנה

הנחיה: כתבו לצד כל תשובה הסבר במילים. שימוש במחשבון מותר, אך התחילו בשאלה מה צפוי לקבל בקירוב. התרגילים הבאים חדשים ואינם הפתרונות לדוגמאות המודרכות בפרק.

1. נתון $X \sim N(60, 5^2)$. הסבירו מה משמעותם של 60 ושל 5. האם 25 הוא סטיית התקן או השונות?

2. לכל תלמיד בקבוצה נמדד זמן ריצה. הסבירו האם זמן ריצה הוא משתנה בדיד או רציף במודל המתמטי.

3. שרטטו שתי עקומות נורמליות בעלות ממוצע זהה וסטיות תקן 4 ו־12. איזו עקומה רחבה יותר? נמקו.

4. התפלגות נורמלית עם $\mu=50$ ו־$\sigma=10$: מהו Z של הערך 70? מהו Z של הערך 40? פרשו את הסימנים.

5. הסבירו מדוע PDF בגובה 0.08 בנקודה מסוימת אינו אומר שההסתברות לקבל בדיוק את הנקודה היא 8%.

6. לפי כלל 68–95–99.7, באיזה תחום נמצאים בקירוב 95% מהערכים כאשר $\mu=20$ ו־$\sigma=3$?

20. תרגול עצמאי — רמה ב׳: חישוב, גרפים ופירוש

7. זמן ההמתנה של שירות מסוים מתואר בקירוב על ידי $X \sim N(45, 6^2)$. חשבו Z של 54 והסבירו את התוצאה.

8. עבור $Z \sim N(0,1)$, נתון $\Phi(0.5)=0.6915$ ו־$\Phi(1.5)=0.9332$. חשבו את $P(0.5<Z<1.5)$. צרפו שרטוט של השטח.

9. בהתפלגות $N(75, 8^2)$, חשבו את ההסתברות לציון גדול מ־83 בעזרת טבלת Z. ציינו מדוע מחסרים מ־1.

10. ציירו גרף PDF וסמנו את השטח המתאים ל־$P(30<X<50)$ כאשר $\mu=40$ ו־$\sigma=10$. האם התוצאה קרובה יותר ל־68% או ל־95%?

11. תלמיד א׳ קיבל 92 במבחן שממוצעו 80 וסטיית התקן שלו 6. תלמיד ב׳ קיבל 85 במבחן שממוצעו 70 וסטיית התקן 10. מי נמצא גבוה יותר ביחס לכיתתו?

12. נתונים מגיעים ממאגר מחירי מכוניות משומשות. הסבירו מדוע נדרש להציג היסטוגרמה לפני חישוב הסתברות על סמך התפלגות נורמלית.

21. תרגול עצמאי — רמה ג׳: בגרות, ביקורת וחשיבה

13. במערכת AI סף לזיהוי חריג נקבע כאשר $|Z|>2$. תנו מקרה שבו כדאי לשמור ערך שסומן כחריג, ומקרה שבו נדרש לתקן או להסיר אותו. נמקו.

14. מפתח מחשב `norm.pdf(100, loc=100, scale=0.1)` ומקבל מספר גדול מ־1. האם זה מעיד על טעות בחישוב? הסבירו מתוך משמעות צפיפות לעומת הסתברות.

15. דודו טוען: ״אחרי שהפעלתי StandardScaler על נתוני האימון, הנתונים מתפלגים נורמלית ולכן אפשר להשתמש בטבלת Z״. האם הטענה נכונה? הסבירו.

16. ידוע שסטיית התקן באוכלוסייה היא 18. חשבו SE לממוצע מדגם עבור $n=9$ ועבור $n=81$. הסבירו מדוע שגיאת התקן לא קטנה פי תשעה.

17. נתון קוד שמחשב p = norm.cdf(140,loc=110,scale=15). מה בדיוק מתאר p? כיצד תשנו את הקוד כדי לחשב את ההסתברות ל־X>140?

18. הסבירו כיצד יכול להיות שממוצעי מדגמים גדולים מתפלגים בקירוב נורמלית גם כאשר תצפיות האוכלוסייה עצמן מתפלגות באופן אסימטרי.

22. משימת חקר מקשרת — מהנתונים לסיפור אמין

במשימה זו תהיו צוות שמפתח מערכת לניטור זמני המתנה למוקד שירות בית־ספרי. מטרתכם להחליט האם סף התרעה מסוים הגיוני. אספו או צרו נתוני סימולציה, תעדו בבירור מה מקורם, והציגו היסטוגרמה של זמני ההמתנה. חשבו ממוצע וסטיית תקן והסבירו אם התפלגות נורמלית היא קירוב סביר. אם תבחרו להשתמש במודל נורמלי, נסחו את ההנחה במפורש והשוו בין הסתברות מחושבת לבין שיעור בפועל במדגם.

התוצר יכלול מחברת Colab עובדת, שני גרפים, טבלת השוואה, פירוש של לפחות שתי הסתברויות ומסקנה מנומקת: מתי ניתן לסמוך על מודל נורמלי ובאיזה מצב עדיף שלא. התלמידים יבחרו בעצמם סף התרעה ולא יקבלו אותו מן המורה.

רכיב עבודה	ראיה שהבנתם
נתונים ומקור	כתוב מה נמדד, ביחידות אילו והאם מדובר בדוגמה אמיתית או בסימולציה.
EDA	היסטוגרמה, מרכז, פיזור והתייחסות להטיה ולחריגים.
מודל	נוסחה $X \sim N(\mu,\sigma^2)$, חישוב ידני אחד ופלט Python.
בדיקת היגיון	השוואת תוצאה תאורטית עם שיעור אמפירי והסבר לפער.
פרשנות	פסקה קצרה המזהירה מפני מסקנה שאינה נתמכת בנתונים.

23. מפת נוסחאות מסכמת

מה בודקים?	נוסחה/כלי	מה רוצים לחשב?
σ חיובית; פירוש סימן z	$z = (x-\mu)/\sigma$	תקנון ערך
ממירים בלי להחליף כיוון	$x = \mu + z\sigma$	חזרה ליחידות מקור
רק תחת מודל נורמלי	99.7% / 95% / 68%	כלל אצבע
גובה אינו הסתברות	PDF $f(x)$	צפיפות
cdf	$F(a)=P(X\le a)$	משמאל לסף
sf	$P(X>a)=1-F(a)$	מימין לסף
b גדול מ־a	$F(b)-F(a)$	בין שני ספים
q בין 0 ל־1	$\text{ppf}(q)$	אחוזון (העשרה)
ממוצעי מדגמים, לא תצפית בודדת	$SE=\sigma/\sqrt{n}$	שגיאת תקן

24. עוגני מקור, התאמה לתכנית והמשך הספר

פרק זה ממוקם ביחידה 5, נושאי ליבה בבינה מלאכותית, סעיף 2.2 — התפלגות נורמלית, לאחר סעיף 2.1 מדדי מרכוז ופיזור. נושאי הליבה בהשתלמות כוללים ממוצע וסטיית תקן של התפלגות נורמלית, ציון תקן, כלל 68–95–99.7, PDF/CDF ותרגולי הסתברות. Plinko, משפט הגבול המרכזי, בדיקות הנחות והקשרים לפרויקט מובאים בספר כגשרים והעמקות פדגוגיות לפי חומרי ההשתלמות וההרחבות שלנו. רמת החובה המדויקת של כל תת־נושא בבגרות טעונה הצלבה עם הנוסח הרשמי העדכני של התכנית ומפרט הבחינה.

עוגני פיתוח מתוך המאגר שלנו: Unit5-AI-Core/03-Statistics/02-Normal-Distribution/Meeting--02 Summary.md והמסמך Didactic-Recommendations.md; בנוסף מסמך START-HERE-12- LESSONS.md מסביר את רצף Plinko, Z ו־CLT. התיאורים, המספרים, הקוד, הניסויים והתרשימים בפרק זה נכתבו מחדש כתוכן מקורי לתלמידי תיכון.

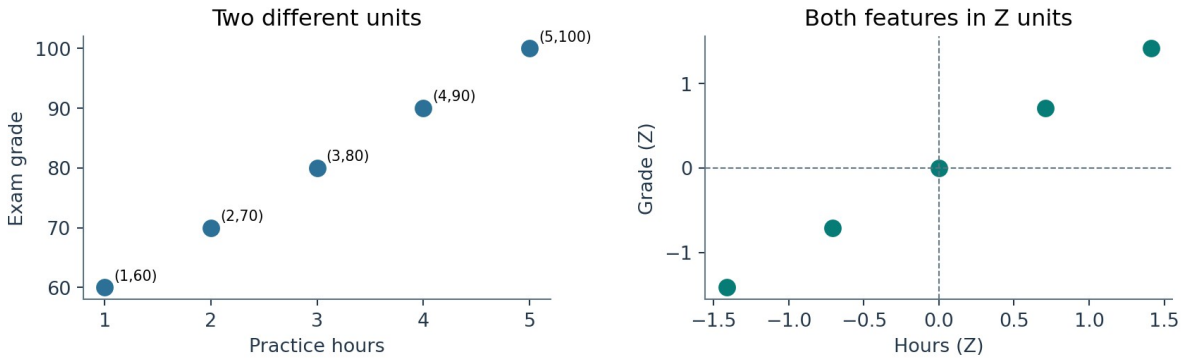
מקורות מקצועיים מומלצים להעמקת המורה

- Starmer, J., The StatQuest Illustrated Guide to Machine Learning — השראה להמחשה, פירוק מושגים וקריאת גרפים.
- Deisenroth, M. P., Faisal, A. A., & Ong, C. S., Mathematics for Machine Learning — עיגון מתמטי בהסתברות ובשיטות חישוב.
- James, G., Witten, D., Hastie, T., Tibshirani, R., & Taylor, J., An Introduction to Statistical Learning with Applications in Python — הקשר בין סטטיסטיקה למודלים ולניתוח נתונים.
- Géron, A., Hands-On Machine Learning — גישה מעשית לתהליך למידת מכונה, הכנת נתונים ובדיקת הנחות.
- Zhang, A., Lipton, Z. C., Li, M., & Smola, A. J., Dive into Deep Learning — מעבר מנוסחאות לסימולציות ותכנות.

כעת אנחנו מוכנים לפרק 2.3 — תקנון ונרמול: נלמד כיצד נתונים שעשויים להיות ביחידות וקני מידה שונים משפיעים על מודלים, וכיצד מכינים אותם נכון בלי לגרום לזליגת מידע. שימו לב: Z-score שהכרנו כאן הוא גשר מתמטי, אך מימוש תקנון באימון דורש הבנה נוספת של הפרדה בין Train ל־Test.

פרק 2.3 — תקנון ונרמול נתונים

איך משווים בין נתונים ביחידות שונות, ולמה פעולה פשוטה יכולה לשנות את החלטת מודל למידת המכונה?



איור — שתי תכונות באותן דוגמאות: לפני התקנון ולאחריו. היחידות משתנות, אך הקשרים בין הדוגמאות נשמרים.

חוברת תלמיד מורחבת • הסברים מדורגים • דוגמאות פתורות • חישוב ידני • Python • גרפים ותרגול

ספר קורס משולב: נושאי ליבה בבינה מלאכותית ולמידה עמוקה | פרק נפרד הניתן לשילוב בכרך השלם

לפני שמתחילים פרק זה ממשיך את פרקים 1.1–1.2 (וקטורים ומרחקים) ואת פרקים 2.1–2.2 (מרכז, פיזור והתפלגות נורמלית). כל הדוגמאות המספריות הן דוגמאות לימוד מקוריות, ולא מדידות של תלמידים אמיתיים.

מטרות הפרק

עד סוף הפרק תדעו להסביר לא רק כיצד מפעילים נוסחה, אלא גם מה היא עושה לנתונים, אילו תכונות היא משמרת, כיצד היא עשויה להשפיע על אלגוריתם ומתי יש להפעיל אותה בזהירות. בתחילת הדרך נחשב ביד, לאחר מכן נשחזר את החישוב בקוד ורק בסוף נשתמש בכלי למידת מכונה מקצועיים.

- להסביר מדוע מרחק אוקלידי מושפע מיחידות המדידה ומטווחי הערכים של מאפיינים שונים.
- לחשב תקנון Z-score בעזרת ממוצע וסטיית תקן ולפרש ערכים שליליים, חיוביים ואפס.
- לחשב Min-Max Scaling בתחום [0,1], כולל נקודה חדשה מחוץ לטווח שנצפה באימון.
- להבחין בין תקנון, נרמול Min-Max, נרמול וקטור ליחידת אורך לבין המושג הכללי Feature Scaling.
- לבחון את השפעת הנתונים החריגים, המאפיינים הקבועים והסולמות השונים על שיטת ההמרה.
- לבצע fit על נתוני אימון בלבד ולהפעיל transform באותו אופן על נתוני בדיקה ועל נתונים חדשים.
- להסביר את הקשר ל־KNN, למערכות המלצה, למודלים ליניאריים ולרשתות עצביות.
- לנתח שגיאות קוד וניסויים, להגן על בחירת דרך ההכנה ולבצע מעבדה עצמאית.

מפת הפרק

שלב	נושא	תוצר
א׳	הבעיה: קני מידה ביחידות שונות	ניבוי השפעה על מרחקים
ב׳	תקנון Z-score	חישוב ידני וקריאת תוצאה
ג׳	Min-Max Scaling	השוואת נוסחאות ותוצאות
ד׳	חריגים, קבועים ונורמה	בחירה מושכלת של שיטה
ה׳	Data Leakage־ו KNN	ניתוח שינוי החלטה
ו׳	NumPy, Pandas, scikit-learn	מימוש, תרגול ובקרת למידה

1. סיפור הפתיחה: דודו מפתח מערכת התאמה לתלמידים

דודו רוצה לפתח מערכת המסייעת לתלמידים לבחור מסלול העשרה. לכל תלמיד נמדדים שני מאפיינים: מספר שעות התרגול בשבוע וציון במבחן הכניסה. אלו שני נתונים מספריים, ולכן אפשר לייצג כל תלמיד בווקטור של שני רכיבים. בהמשך אפשר להשוות בין תלמידים בעזרת מרחק אוקלידי, כפי שלמדנו בפרק 1.2.

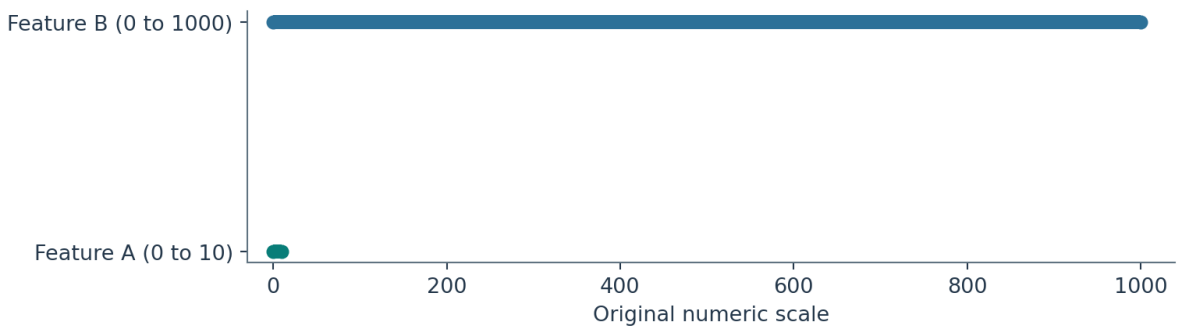
תלמיד	שעות תרגול בשבוע	ציון במבחן
א׳	2	60
ב׳	4	80
ג׳	5	90

לכאורה, שני המאפיינים שווים בחשיבותם. בפועל, אם נחשב מרחק ישירות, הבדל של 20 נקודות בציון יגדיל את המרחק הרבה יותר מהבדל של שתי שעות תרגול. אבל האם 20 נקודות הן תמיד חשובות יותר משעתיים? כדי לענות על השאלה צריך להבין מה כל מאפיין מודד, מהו הטווח שלו ומה התפקיד שלו במודל.

$$d(A, B) = \sqrt{[(4 - 2)^2 + (80 - 60)^2]} = \sqrt{404} \approx 20.10$$

בדוגמה הזו ההפרש בציון תורם 400 ל־404 שמתחת לשורש, ואילו הפרש השעות תורם רק 4. כלומר, כמעט כל המרחק נקבע לפי הציונים. אנו רואים כאן בעיה שנובעת מהמספרים, לא בהכרח מהמשמעות החינוכית של הנתונים.

שאלת פתיחה אם נחליף את שעות התרגול מדיווח בשעות לדיווח בדקות, כל מספר שעות יוכפל ב־60. האם פתאום תלמידים נעשו שונים יותר מבחינה לימודית? כמובן שלא. אך המרחק המחושב עלול להשתנות באופן ניכר, ואף סדר השכנים עלול להשתנות.



איור — הדגמה של שני סולמות שונים. גודל מספרי גדול אינו בהכרח חשיבות גדולה יותר.

2. למה בכלל מבצעים שינוי קנה מידה?

באלגוריתמים מסוימים נמדד מרחק בין וקטורי נתונים או מחושבת מכפלה פנימית של מאפיינים ומשקולות. כאשר מאפיין אחד נמדד ביחידות גדולות ומאפיין אחר ביחידות קטנות, ערכיו של המאפיין הראשון עלולים לשלוט בחישוב. שינוי קנה מידה (Feature Scaling) הוא משפחה של שיטות הממירות את המספרים לייצוג חדש שנוח יותר להשוואה ולמידול.

השינוי אינו תהליך קסם המשפר כל מודל, ואינו מוכיח שהנתונים איכותיים. מטרתו להעמיד את המאפיינים על סולמות מתאימים למשימה, תוך שמירה על הקשר לתהליך המדידה המקורי. חשוב במיוחד לזכור שלא כל אלגוריתם רגיש לקנה מידה באותה מידה: KNN ו־K-Means תלויים במרחקים, ולעומת זאת עצי החלטה רגילים בדרך כלל פחות רגישים לשינוי ליניארי עולה של הסולם.

לשיטות שינוי קנה מידה יש גם השלכות על פרשנות: לאחר ההמרה ערך 1 עשוי להיות סטיית תקן מעל הממוצע, או הערך המרבי שנמדד באימון, לפי השיטה. לכן התלמיד חייב לציין בכל חישוב איזו שיטה בוצעה ומה משמעות התוצאה.

3. תקנון Z-score: כמה סטיות תקן אנחנו רחוקים מהממוצע?

בפרק 2.2 פגשנו את Z-score לצורך השוואת מדידה לממוצע ולסטיית תקן. כעת נשתמש באותה נוסחה גם כדי לשנות את סולם הנתונים לפני אימון מודל. בניגוד להמרת יחידות שרירותית, כאן אנחנו מודדים את מיקום הערך ביחידות של פיזור הנתונים.

$$z = (x - \mu) / \sigma \qquad (\sigma > 0)$$

פירוק הסימנים: x הוא הערך המקורי של תצפית מסוימת. האות μ היא הממוצע של המאפיין בנתוני האימון, והאות σ היא סטיית התקן שחושבה על אותם נתונים. תחילה מחסרים את הממוצע, כדי לדעת אם הערך נמצא מעל המרכז או מתחתיו. אחר כך מחלקים בסטיית התקן, כדי לדעת עד כמה המרחק מן המרכז גדול ביחס לפיזור האופייני.

דוגמה פתורה 1 — תקנון ציונים

נניח כי בקבוצת האימון ממוצע הציונים הוא 70 וסטיית התקן היא 10. דודו רוצה לתקנן ציון 85. תחילה מחשבים את ההפרש מן הממוצע: 85 פחות 70 שווה 15. לאחר מכן מחלקים את ההפרש ב־10, שהיא סטיית התקן. התוצאה היא 1.5.

$$z = (85 - 70) / 10 = 15 / 10 = 1.5$$

המשמעות: הציון 85 נמצא 1.5 סטיות תקן מעל ממוצע קבוצת האימון. זו תשובה על מיקום יחסי; היא אינה אומרת שהציון הוא 1.5 נקודות, ואינה אומרת שמדובר בהסתברות של 1.5. אילו הציון היה 60, היינו מקבלים $z = -1$, כלומר סטיית תקן אחת מתחת לממוצע.

ציון	חישוב z	פירוש
60	$(60-70)/10 = -1$	סטיית תקן אחת מתחת לממוצע
70	$(70-70)/10 = 0$	בדיוק בממוצע
85	$(85-70)/10 = 1.5$	1.5 סטיות תקן מעל הממוצע
90	$(90-70)/10 = 2$	שתי סטיות תקן מעל הממוצע

דוגמה פתורה 2 — מחשבים את הממוצע וסטיית התקן בעצמנו

ניקח חמישה ערכים: 10, 20, 30, 40 ו־50. הממוצע הוא 30. ההפרשים מהממוצע הם $-20, -10, 0, 10, 20$. ריבועי ההפרשים הם 400, 100, 0, 100 ו־400. סכומם 1000, ולכן שונות האוכלוסייה של חמש התצפיות היא 200 וסטיית התקן היא שורש 200, בקירוב 14.142.

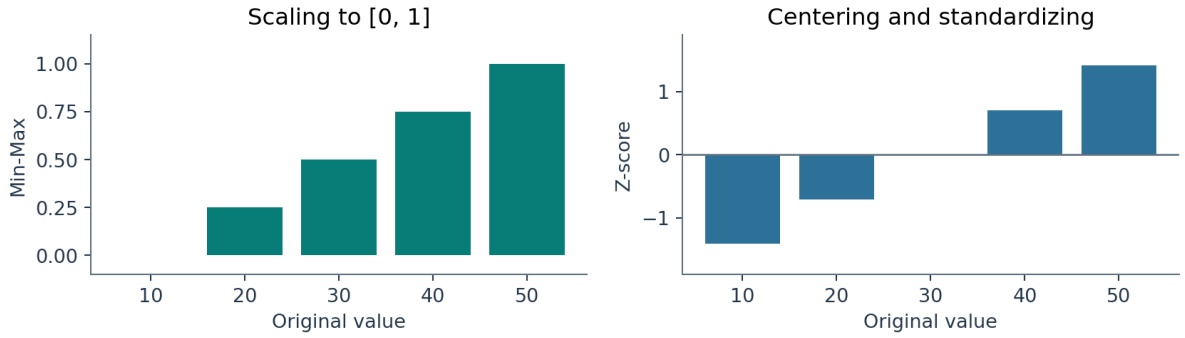
$\mu = (10 + 20 + 30 + 40 + 50) / 5 = 30$

$\sigma^2 = (400 + 100 + 0 + 100 + 400) / 5 = 200$

$\sigma = \sqrt{200} \approx 14.142$

$z(50) = (50 - 30) / 14.142 \approx 1.414$

לכן ערך 50 נמצא כ־1.414 סטיות תקן מעל הממוצע. כאשר נתקנן את כל חמש התצפיות לפי הממוצע וסטיית התקן של אותה קבוצה, נקבל ערכים שהממוצע שלהם אפס וסטיית התקן שלהם אחת, בהתאם לאותה הגדרת שונות. ב־scikit-learn ברירת המחדל של StandardScaler משתמשת בשונות עם חלוקה ב־n (בדומה ל־ddof=0 ב־NumPy).



איור — אותו אוסף נתונים בשתי שיטות. בתקנון Z האפס הוא הממוצע; ב־Min-Max האפס הוא ערך המינימום באימון.

חשוב להבין תקנון Z-score אינו הופך התפלגות אסימטרית להתפלגות נורמלית. הוא משנה את המרכז ואת יחידת המדידה, ולא את צורת ההתפלגות היסודית. אין צורך להניח נורמליות כדי לחשב Z-score.

4. כתיבה מתמטית כללית — מאפיין אחד ומטריצת מאפיינים

במערכת AI אמיתית יש בדרך כלל הרבה שורות והרבה עמודות. כל שורה היא דוגמה, וכל עמודה היא מאפיין. לדוגמה: מספר שעות תרגול, ציון, מספר משימות שהושלמו. אין מחסרים ממוצע כללי של כל המספרים יחד. לכל עמודת מאפיין מחשבים ממוצע וסטיית תקן משלה באמצעות נתוני האימון.

$z_{ij} = (x_{ij} - \mu_j) / \sigma_j$

הסימון x_{ij} הוא הערך בשורה i ובעמודה j. הסימון μ_j הוא ממוצע המאפיין שמופיע בעמודה j, ואילו σ_j היא סטיית התקן של אותה עמודה. כאשר מחשבים את z_{ij}, אנחנו משווים את התלמיד לתלמידים אחרים באותו מאפיין, ולא משווים ישירות שעות לציונים.

שני מאפיינים יכולים להיות בעלי ערכים שונים מאוד מבחינה מספרית, אך לאחר תקנון כל עמודה תהיה ממורכזת סביב אפס. זאת אחת הסיבות שהשלב הזה שימושי במיוחד לפני חישובי מרחקים ואופטימיזציה.

5. נרמול Min-Max: מיקום בתוך טווח האימון

שיטה שנייה היא Min-Max Scaling. כאן השאלה אינה כמה סטיות תקן הערך רחוק מן הממוצע, אלא היכן הוא נמצא ביחס למינימום ולמקסימום שנמצאו בנתוני האימון. הצורה הנפוצה ממירה את התחום שבין מינימום למקסימום לערכים בין אפס לאחת.

$x_{scaled} = (x - x_{min}) / (x_{max} - x_{min})$

פירוק הנוסחה: מחסרים את הערך הקטן ביותר שנמדד בקבוצת האימון, וכך הופכים את נקודת ההתחלה של הסולם לאפס. לאחר מכן מחלקים ברוחב הטווח, כלומר בהפרש בין הערך המרבי לערך המזערי של

קבוצת האימון. כאשר הערך המקורי נמצא בדיוק במינימום, התוצאה אפס; כאשר הוא בדיוק במקסימום, התוצאה אחת.

דוגמה פתורה 3 — זמן תרגול

נניח שבקבוצת האימון זמן התרגול הקצר ביותר היה 20 דקות והארוך ביותר היה 60 דקות. נרצה להמיר זמן תרגול של 35 דקות. ראשית נחסר 20 ונקבל 15. רוחב הטווח הוא 60 פחות 20, כלומר 40. לכן נקבל 15 חלקי 40, שהם 0.375.

x_scaled = (35 − 20) / (60 − 20) = 15 / 40 = 0.375

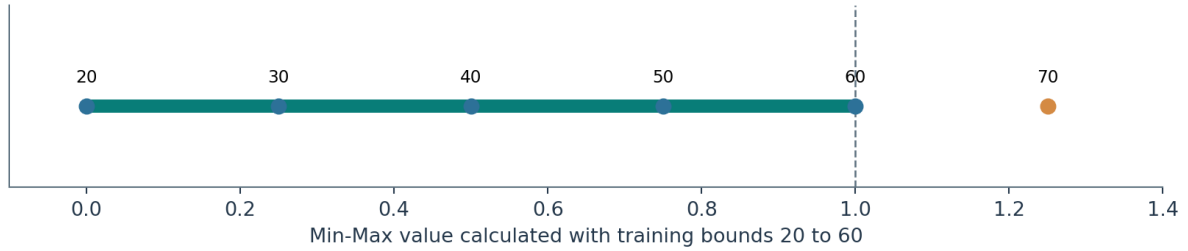
התוצאה 0.375 מלמדת שהערך נמצא ב־37.5% מהדרך בין שני גבולות האימון. אין להסיק מכאן שתלמיד השלים 37.5% מהמשימות שלו או שקיבל הסתברות של 37.5%; זהו רק מיקום על סולם חדש.

משמעות	נרמול לפי 20–60	זמן בדקות
מינימום האימון	0	20
רבע הדרך	0.25	30
אמצע הטווח	0.50	40
שלושה רבעים	0.75	50
מקסימום האימון	1.00	60

דוגמה פתורה 4 — ומה אם מגיע תלמיד חדש עם 70 דקות?

הערכים 20 ו־60 נקבעו לפי נתוני האימון בלבד. כאשר מגיע תלמיד חדש עם 70 דקות תרגול, נציב גם אותו באותה נוסחה. התוצאה תהיה 1.25, למרות שהיא גדולה מ־1. הדבר תקין: הנרמול אינו מחייב שכל ערך עתידי יישאר בין אפס לאחת.

x_scaled(70) = (70 − 20) / 40 = 1.25



איור — דוגמת אימון 20–60 דקות. נתון חדש 70 דקות מקבל 1.25; אין לחשב את המינימום והמקסימום מחדש כדי לכפות עליו את הטווח.

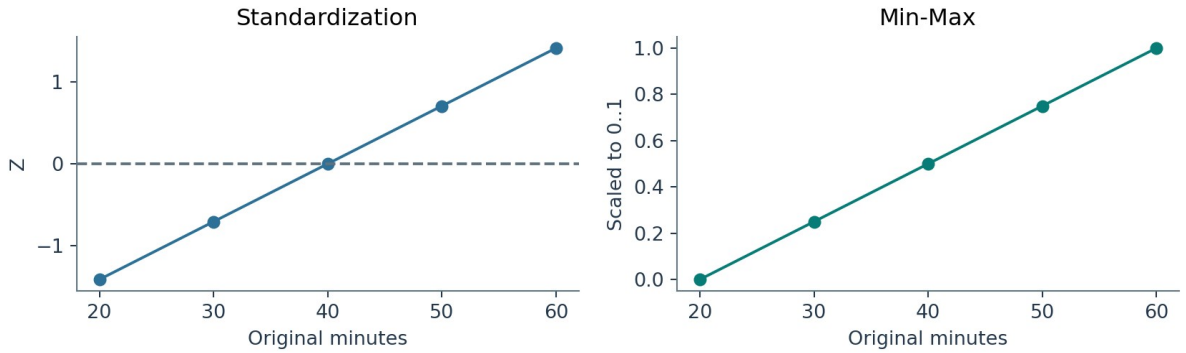
אפשר להשתמש במקרים מסוימים באפשרות clipping כדי להגביל ערכי פלט לטווח שנבחר, אך פעולה כזו גורמת לאובדן מידע: 70 ו־700 עשויים לקבל אותו ערך עליון. לכן אין להפעיל הגבלה באופן אוטומטי בלי להבין את מטרת המודל.

6. השוואה מדויקת בין Z-score לבין Min-Max

שתי השיטות מחליפות את ערכי המאפיין בערכים חדשים, אך הן אינן עונות על אותה שאלה. בתקנון Z בוחנים מרחק מהממוצע ביחידות סטיית תקן. בנרמול Min-Max בוחנים מיקום בתוך הטווח של נתוני האימון. הבחירה תלויה במודל, בנתונים ובמטרת הניסוי. אין כלל שלפיו שיטה אחת תמיד עדיפה.

מאפיין להשוואה	Z-score	Min-Max
נוסחה	σ/(x−μ)	(max−min)/(x−min)

ערך מרכזי	הממוצע מקבל 0	אמצע הטווח מקבל 0.5
טווח פלט	אין טווח קבוע	על נתוני האימון [0,1]
מושפע מערכים חריגים?	כן — ממוצע וסטיית תקן משתנים	כן — מינימום/מקסימום רגישים מאוד
ערך חדש מחוץ לטווח	עשוי להיות גדול בערך מוחלט	עשוי להיות קטן מ־0 או גדול מ־1
צריך הנחת נורמליות?	לא	לא

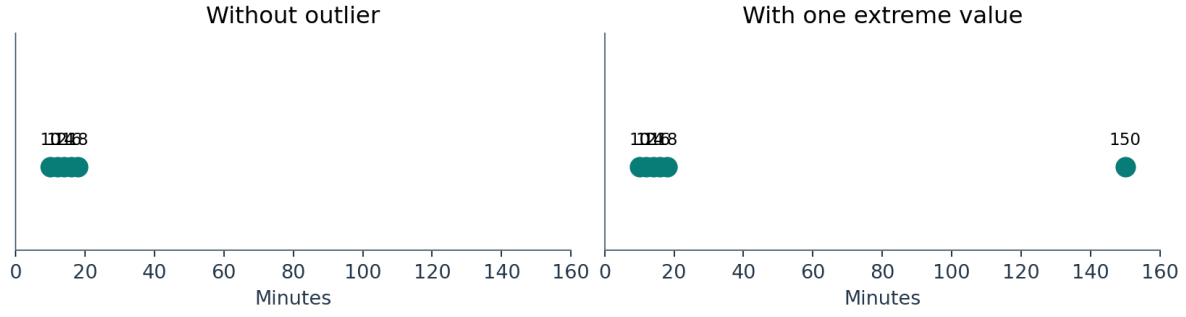


איור — שתי דרכים שונות לייצג אותם ערכים. בחרו שיטה לפי המשמעות והמטרה, לא רק לפי גרף יפה.

בדיקת הבנה מישהו טוען ש־Z-score מקבל תמיד ערכים בין 3− ל־3. מדוע הטענה שגויה? ערך חריג עשוי להימצא גם 5 או 10 סטיות תקן מהממוצע. גם ההתפלגות הנורמלית עצמה אינה מוגבלת לתחום הזה.

7. למה לא תמיד כדאי להשתמש בממוצע, במקסימום או במינימום?

נניח שחמישה תלמידים תרגלו במשך 10, 12, 14, 16 ו־18 דקות. הטווח הוא שמונה דקות והממוצע הוא 14. אם נוסיף תלמיד אחד שתרגל 150 דקות, משתנה מאוד המקסימום, וגם הממוצע וסטיית התקן משתנים. למרות שרוב התלמידים נשארו בדיוק אותם תלמידים עם אותם זמני תרגול, הערכים המנורמלים שלהם יידחסו לסביבה קטנה קרוב לאפס.

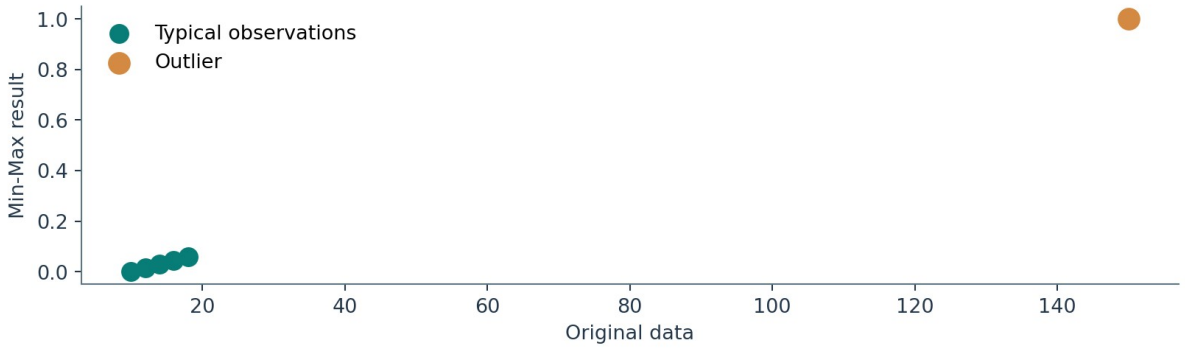


איור — כמעט כל הנתונים נשארים קרובים, אבל ערך קיצוני אחד מרחיב באופן ניכר את הסולם הכללי.

Min-Max(18) without outlier = (18−10)/(18−10) = 1

Min-Max(18) with outlier = (18−10)/(150−10) ≈ 0.0571

זו אותה תצפית, 18 דקות, אך הערך המנורמל השתנה מ־1 לכ־0.0571 בגלל הוספת תצפית חריגה. מכאן שלא נכון לטעון ש־Min-Max מעלים ערכים חריגים. לעיתים קרובות הוא דווקא חושף את הרגישות שלהם במלוא עוצמתה.



איור — דחיסת תצפיות רגילות לאחר נרמול כשקיים ערך קיצוני מאוד.

אפשר לשקול בשלב מתקדם שיטות חסינות יותר, כגון RobustScaler המשתמש בחציון ובטווח בין־רבעוני. אך גם כאן יש להבין תחילה מהו סוג המודל, מהי משמעות החריג ומהי ההשפעה של בחירת הכלי. לעיתים ערך חריג הוא טעות הזנה, ולעיתים הוא המקרה החשוב ביותר שהמערכת נועדה לזהות.

8. ומה עושים אם לכל התלמידים יש אותו ערך?

נניח שכל תלמידי קבוצת האימון השלימו בדיוק חמש משימות. הממוצע הוא 5 וסטיית התקן היא אפס. בנוסחת Z-score הרגילה אי אפשר לחלק באפס. באופן דומה, בנוסחת Min-Max, אם המינימום והמקסימום שניהם 5, המכנה הוא אפס. לכן אין להציב בנוסחאות בצורה עיוורת.

Constant feature: $x_{min} = x_{max}$ and $\sigma = 0$

מבחינה מושגית, מאפיין קבוע אינו מסייע להבחין בין דוגמאות בקבוצת האימון. ספריות מקצועיות מטפלות במקרה הזה בהגדרות מוגדרות, ולא על ידי חלוקה באפס. אפשר גם לשקול להסיר מאפיין קבוע לאחר בחינת משמעותו. אם הוא עשוי להשתנות בעתיד, יש לבחון את ההחלטה לפי הקשר הפרויקט.

חשוב בקוד בחישוב ידני יש לציין שהנוסחה אינה מוגדרת כאשר המכנה אפס. אין להמציא מספר כאילו חילקנו באפס. במעבדה נבדוק כיצד התנהגות זו מטופלת בספרייה שבחרנו.

9. נרמול של וקטור לאורך 1 — רעיון שונה!

בפרק 1.1 למדנו את אורך הווקטור, או הנורמה שלו. בתחום הבינה המלאכותית המילה נרמול (Normalization) יכולה לציין גם שינוי של וקטור שלם כך שאורכו יהיה 1, ולאו דווקא שינוי של עמודה לטווח [0,1]. חשוב להבחין בין המושגים, משום שהם משמשים מטרות שונות.

$$v_{unit} = v / ||v||_2 \quad \text{for} \quad ||v||_2 > 0$$

דוגמה פתורה 5 — אותו כיוון, אורך חדש

נתון הווקטור v=(3,4). אורכו הוא 5. נחלק כל רכיב ב־5 ונקבל (0.6,0.8). אורך הווקטור החדש הוא 1, כלומר כיוון הווקטור לא השתנה, אך גודלו השתנה.

$$||(3,4)||_2 = \sqrt{3^2+4^2} = 5$$

$$(3,4)/5 = (0.6,0.8) \qquad ||(0.6,0.8)||_2 = 1$$

הטיפול הזה נפוץ כאשר רוצים להשוות בעיקר כיוון של וקטורי ייצוג, למשל במערכות המלצה ובהשוואת מסמכים. הוא שונה מתקנון Z-score של עמודות, והוא אינו מוגדר עבור וקטור האפס ללא כלל טיפול מיוחד.

10. האם שינוי קנה מידה באמת יכול לשנות את השכן הקרוב ביותר?

זהו הגשר החשוב ביותר בין פרק 1.2 לבין פרק 2.3. ב־KNN משווים דוגמה חדשה לדוגמאות האימון לפי מרחק. מכיוון שהמרחק תלוי ביחידות, שינוי קנה המידה של המאפיינים עשוי לשנות את זהות השכן הקרוב ביותר. נבחן זאת במספרים, ולא נסתפק בטענה תאורטית.

נניח שדודו משווה שתי מועמדויות A ו־B למועמדות חדשה Q לפי שני מדדים. המאפיין הראשון נמדד בסולם 0–10, והמאפיין השני בסולם 0–1000. המינימום והמקסימום נקבעו מתוך קבוצת האימון: (0,0) ו־(10,1000).

דוגמה	מאפיין 1	מאפיין 2
Q — חדשה	2	100
A — אימון	3	110
B — אימון	6	101

נחשב תחילה מרחק אוקלידי לפני שינוי קנה המידה. עבור A הפרשי הרכיבים הם 1 ו־10. עבור B ההפרשים הם 4 ו־1. מכיוון שההפרש של 10 גדול מאוד על הסולם השני, A נראית רחוקה יותר מ־B.

$$d(Q,A) = \sqrt{[(3-2)^2+(110-100)^2]} = \sqrt{101} \approx 10.05$$

$$d(Q,B) = \sqrt{[(6-2)^2+(101-100)^2]} = \sqrt{17} \approx 4.12$$

לפני התקנון, B היא השכנה הקרובה יותר. כעת נחלק את המאפיין הראשון ב־10 ואת המאפיין השני ב־1000. אלה בדיוק פעולות Min-Max עבור טווחי האימון שנבחרו. כך נקבל:

$$Q_{scaled} = (0.2, 0.100)$$

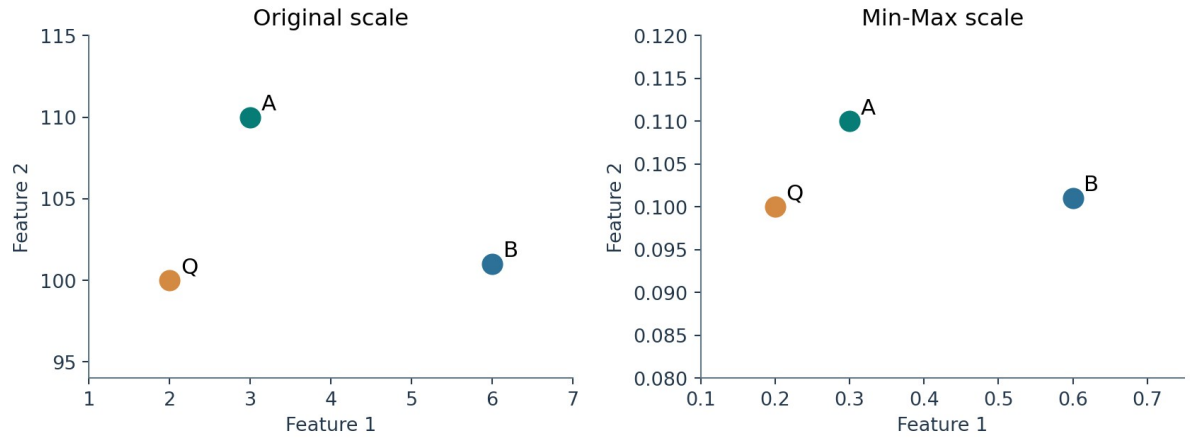
$$A_{scaled} = (0.3, 0.110)$$

$$B_{scaled} = (0.6, 0.101)$$

$$d(Q,A) = \sqrt{[(0.3-0.2)^2+(0.110-0.100)^2]} \approx 0.1005$$

$$d(Q,B) = \sqrt{[(0.6-0.2)^2+(0.101-0.100)^2]} \approx 0.4000$$

לאחר השינוי A היא השכנה הקרובה יותר. שום אדם לא שינה את הנתונים שלו. מה שהשתנה הוא האופן שבו האלגוריתם מודד מרחק. זו אחת הסיבות ששיטת ההכנה של הנתונים היא חלק מהגדרת הניסוי ולא רק פעולה טכנית לפני המודל.



איור — דוגמת KNN: לפני שינוי קנה מידה B קרובה יותר; לאחריו A קרובה יותר. השוו את שני חישובי המרחק.

בפרויקט אמיתי לא בוחרים את שיטת שינוי קנה המידה על סמך דוגמה אחת בלבד. בודקים כמה אפשרויות על נתוני אימון ואימות, ומדווחים על מדד הבדיקה הסופי בנתוני Test שלא שימשו לבחירה.

11. מהי זליגת נתונים (Data Leakage) וכיצד נמנעים ממנה?

זליגת נתונים מתרחשת כאשר מידע שלא אמור להיות זמין בזמן בניית המודל משפיע על תהליך האימון או בחירת הפתרון. אחת הדוגמאות הנפוצות היא חישוב ממוצע, סטיית תקן, מינימום או מקסימום על כל המאגר — כולל נתוני בדיקה — ורק לאחר מכן חלוקה לקבוצות. במקרה כזה המודל אמנם לא ראה ישירות את תוויות ה־Test, אך הכנת המאפיינים שלו כבר הושפעה מההתפלגות של הדוגמאות האלה.

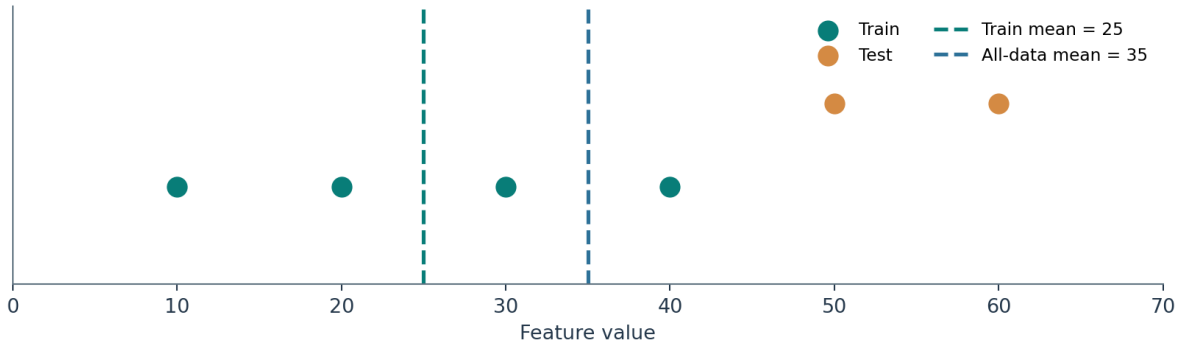
ניקח ארבעה ערכי אימון: 10, 20, 30, 40; ושני ערכי בדיקה: 50, 60. ממוצע האימון הוא 25 וסטיית התקן שלו היא שורש 125, בקירוב 11.18. כדי לתקנן את הערך 50 בקבוצת הבדיקה נשתמש דווקא בשני המספרים שהופקו מהאימון.

Train mean: μ_train = (10+20+30+40)/4 = 25

Train SD: σ_train = √125 ≈ 11.180

Test value x=50: z = (50−25)/11.180 ≈ 2.236

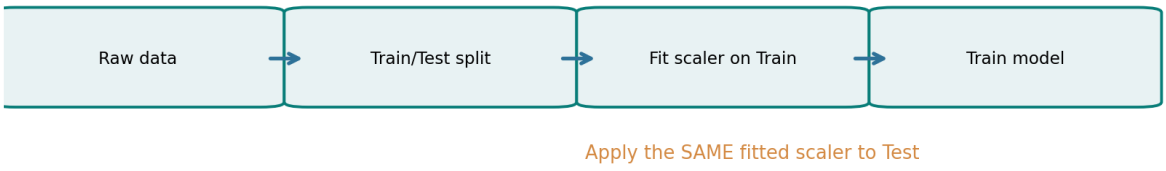
אם נחשב בטעות את הממוצע והפיזור על כל ששת הערכים, הממוצע יהיה 35 והתוצאה עבור 50 תהיה אחרת. זו אינה אותה טרנספורמציה, והמספרים החדשים כוללים מידע מהנתונים שהיינו אמורים לשמור לבדיקת הכללה.



איור — קבוצת אימון וקבוצת בדיקה. מרכז קבוצת האימון שונה ממרכז כל הנתונים; בשלב האימון משתמשים רק בראשון.

הסדר הנכון בעבודה

ראשית מפרידים בין Train לבין Test. לאחר מכן מחשבים את פרמטרי שינוי קנה המידה באמצעות Train בלבד — זהו fit. מפעילים את השינוי על Train — זהו transform. על Test ועל כל דוגמה חדשה מפעילים רק transform עם אותם פרמטרים, בלי לחשב אותם מחדש.



איור — הסדר המומלץ: הפרדת קבוצות, התאמת scaler על האימון, הכנת האימון ואימון המודל. את הבדיקה מכינים באותו scaler קיים.

ניסוח מדויק גם בעת Cross-Validation צריך ללמוד את פרמטרי ההמרה מחדש בתוך כל קיפול אימון

בלבד. שימוש ב־Pipeline של scikit-learn מסייע ליישם זאת בצורה תקינה.

12. מימוש הדרגתי ב־Python וב־NumPy

לפני השימוש בספרייה מקצועית נכתוב את החישוב באופן גלוי. כך נוכל להשוות כל שורה בקוד לשלב המתמטי שעשינו ביד. שימו לב שכדי לחשב Z-score עבור תצפיות חדשות אין להשתמש בממוצע שלהן. נחשב את הפרמטרים מקבוצת האימון ונשמור אותם.

```
import numpy as np

train = np.array([10., 20., 30., 40., 50.])
new_value = 60.

mean_train = np.mean(train)
std_train = np.std(train, ddof=0)

z_train = (train - mean_train) / std_train
z_new = (new_value - mean_train) / std_train

print("Mean:", mean_train)
print("SD:", std_train)
print("Train Z:", z_train)
print("New value Z:", z_new)
```

הפונקציה np.mean מחשבת ממוצע, ו־np.std מחשבת סטיית תקן. הפרמטר ddof=0 מציין את החישוב עם חלוקה ב־n. המשתנה z_train הוא מערך שבו כל איבר תוקנן לפי אותו ממוצע ואותה סטיית תקן. המשתנה z_new הוא ערך בודד שהומר באמצעות פרמטרי האימון, ולא על סמך נתוני בדיקה חדשים.

מימוש Min-Max באותה רוח

הפעם נחשב תחילה את המינימום והמקסימום של האימון, ואז נשתמש בהם כדי להמיר גם ערכים שכבר נמצאים באימון וגם ערך חדש. בתרגול נבדוק מה קורה אם הערך החדש גדול מהמקסימום שנצפה.

```
import numpy as np

train = np.array([20., 30., 40., 50., 60.])
new_value = 70.

min_train = np.min(train)
max_train = np.max(train)
train_scaled = (train - min_train) / (max_train - min_train)
new_scaled = (new_value - min_train) / (max_train - min_train)

print("Train scaled:", train_scaled)
print("New scaled:", new_scaled)
```

תוצאה מעל 1 אינה שגיאה בפני עצמה. היא מלמדת שהערך החדש נמצא מעל המקסימום של קבוצת האימון. אם נקבל אזהרת חלוקה באפס, עלינו לחזור ולבדוק שהמאפיין אינו קבוע.

13. מ־Pandas ל־scikit-learn: היישום המקצועי

במערכות למידת מכונה נשתמש לרוב ב־DataFrame שיש בו כמה מאפיינים. לדוגמה, נמדוד שעות תרגול, ציון ומספר משימות שהושלמו. במקום לחשב את ההמרה ידנית בכל פעם, אפשר להשתמש ב־StandardScaler או ב־MinMaxScaler מתוך scikit-learn. כל כלי לומד את פרמטרי ההמרה ב־fit, ואז משתמש בהם ב־transform.

```
import pandas as pd
from sklearn.preprocessing import StandardScaler, MinMaxScaler

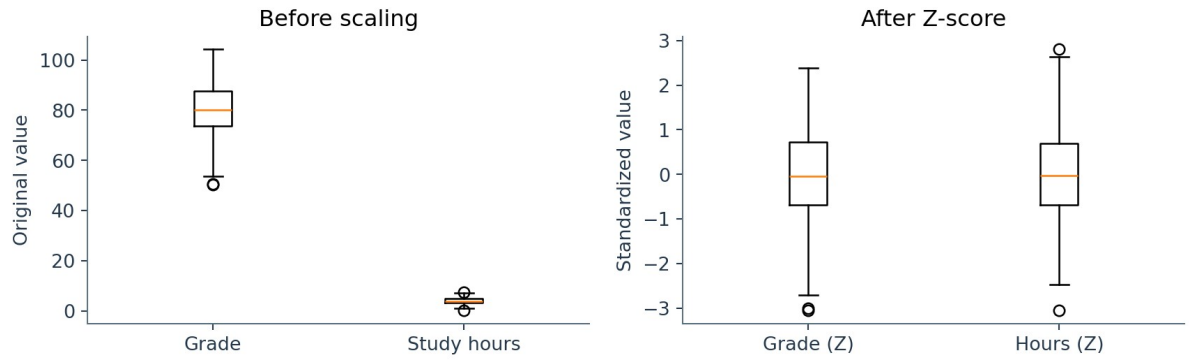
train_df = pd.DataFrame({
    "study_hours": [1, 2, 3, 4, 5],
    "grade": [60, 70, 80, 90, 100]
})

standard = StandardScaler()
scaled_standard = standard.fit_transform(train_df)

minmax = MinMaxScaler()
scaled_minmax = minmax.fit_transform(train_df)

print("Z-score results:")
print(scaled_standard)
print("Min-Max results:")
print(scaled_minmax)
```

בפעם הראשונה שאנו רואים fit_transform קל לחשוב שמדובר בפקודה אחת שאין צורך להבין. למעשה היא מקצרת שני שלבים: התאמת פרמטרים לפי קבוצת האימון והמרת אותה קבוצה. על קבוצה חדשה לא משתמשים שוב ב־fit_transform; משתמשים ב־transform בלבד.



איור — התפלגות שתי תכונות לפני תקנון ואחריו. הגרפים מדגימים ערכים אקראיים בסולמות שונים, לא נתונים אמיתיים.

14. מודל KNN עם Pipeline: הכנה ומודל באותו תהליך

בשלב הבא משלבים את שינוי קנה המידה עם אלגוריתם KNN. השילוב בתוך Pipeline חשוב במיוחד משום שהוא שומר על סדר פעולות ברור: קודם מתאימים את ההמרה לפי הנתונים המותרים, ואחר כך מאמנים את המסווג. כאשר מבקשים חיזוי, אותה המרה מיושמת על הקלט החדש.

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

iris = load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.25,
    random_state=42, stratify=iris.target
)

model = make_pipeline(
    StandardScaler(),
    KNeighborsClassifier(n_neighbors=3)
```

```
)
model.fit(X_train, y_train)
predictions = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, predictions))
```

קוד זה אינו פותר את התרגילים של התלמיד. הוא מדגים את המבנה המקצועי של התהליך באמצעות מאגר Iris מוכר. במעבדה התלמידים יתבקשו לבצע ניתוח נפרד, לבדוק שתי שיטות הכנה ולהסביר את ההבדלים. אין לפרש את Accuracy שהתקבל בהרצה יחידה כהוכחה שמודל אחד עדיף בכל מצב.

15. מתי Scaling חשוב, ומתי פחות?

השאלה אינה אם צריך תמיד להפעיל scaler, אלא האם הגדרת המודל שלנו רגישה לגודל המספרי של המאפיינים. ב־KNN, K-Means ואלגוריתמים נוספים המבוססים על מרחקים, שינוי קנה המידה עשוי לשנות תוצאות באופן דרמטי. גם מודלים המבוססים על אופטימיזציה מספרית יכולים להרוויח מקנה מידה מתאים, משום שהוא משפיע על מהירות והתנהגות תהליך האימון.

במודלים מבוססי עצים, החלפת סולם כל מאפיין בהמרה ליניארית עולה אינה משנה בדרך כלל את סדר הערכים בתוך אותו מאפיין, ולכן עץ החלטה אינו זקוק בדרך כלל לתקנון לשם יצירת הפיצולים הרגילים. עם זאת, גם כאן צריך לבדוק את המימוש, את עיבוד הנתונים ואת מכלול מערכת הלמידה; אין להסיק שלעולם אין צורך בהכנה.

סוג מודל	רגישות אופיינית לשינוי קנה מידה	מה בודקים
KNN / K-Means	גבוהה — המרחקים משתנים	שכנים, מרכזי קבוצות, ביצועים
רגרסיה עם רגולריזציה	יכולה להיות משמעותית	מקדמים, ענישה, התכנסות
רשת עצבית	לעיתים חשובה לאופטימיזציה	יציבות, קצב למידה ותוצאות
עץ החלטה רגיל	לרוב נמוכה להמרות עולות	התנהגות המימוש והנתונים

16. הקשר ללמידה עמוקה: כיצד הנתונים מגיעים לנוירון?

בתחילת הספר למדנו לייצג תלמיד בווקטור תכונות x ולהכפיל אותו במשקולות w. בשלב מתקדם נבנה רשת עצבית שמבצעת הרבה פעולות כאלה. אם חלק מהקלטים גדולים מאוד וחלקם קטנים מאוד, תהליך האימון והגרדיאנטים עשויים להתנהג באופן פחות נוח. תקנון קלטים הוא אחד הכלים החשובים בהכנת נתונים לרשת.

$$z = w \cdot x + b$$

אין צורך ללמוד בפרק זה את כל פרטי Backpropagation. חשוב כבר עכשיו להבין את העיקרון: שינוי המספרים בקלט משנה את המיקום שלהם במרחב התכונות, ולעיתים משנה את יעילות הלמידה. בהמשך נבחין גם בין תקנון קלטים לבין שכבות נרמול בתוך רשת עצבית, כגון Batch Normalization, שהן טכניקות אחרות ואין לבלבל ביניהן.

17. שתי טעויות קלאסיות — ניתוח והסבר

טעות א׳ — מחשבים סטיית תקן חדשה עבור כל תלמיד

נניח שהגענו לתלמיד חדש עם וקטור [4, 90]. תלמיד אחד אינו קבוצת אימון, וגם אין היגיון לחשב ממנו ממוצע וסטיית תקן עבור כל מאפיין. לכן חייבים לשמור את הפרמטרים שנלמדו מנתוני האימון ולהפעיל אותם על התלמיד החדש.

טעות ב׳ — חושבים שכל ערך Z בין 1− ל־1 הוא הסתברות

Z-score הוא מספר סטיות תקן. הוא יכול להיות שלילי או חיובי ואפילו גדול מ־3 בערכו המוחלט. הוא אינו הסתברות. אם רוצים לחשב הסתברות מתוך Z, צריך להוסיף הנחה מתאימה על ההתפלגות, למשל מודל נורמלי, ולהשתמש בכלי הסתברות מתאים. כאן אנחנו משתמשים ב־Z ככלי לשינוי קנה מידה, לא כחישוב הסתברות.

18. שאלות חשיבה ותרגול — רמה בסיסית

- א. קבוצת אימון מכילה ציונים 50, 60, 70, 80, 90. חשבו את הממוצע, את שונות האוכלוסייה ואת סטיית התקן. חשבו Z לציון 80 ופרשו במילים.
- ב. מאפיין מקבל בקבוצת האימון מינימום 8 ומקסימום 28. מצאו את הערך המנורמל של 18 ושל 33. האם אחד מהם חייב להיות שגוי?
- ג. הסבירו במילים שלכם מה ההבדל בין ערך מתוקנן 0 לבין ערך מנורמל Min-Max השווה ל־0.
- ד. תלמיד טוען שבשתי השיטות כל התוצאות נמצאות בין אפס לאחת. תקנו את הטענה בעזרת דוגמה מספרית.
- ה. מה קורה לנוסחאות כאשר כל ערכי המאפיין זהים? מדוע אין להציב ללא בדיקה?

19. שאלות יישום — רמת ביניים

- ו. עבור הערכים [10,20,30,40,50] חשבו Z-score לכל ערך. בדקו שממוצע הערכים המתוקננים הוא אפס (עד כדי שגיאת עיגול).
- ז. דודו מקבל קבוצת Train עם ערכים [20,30,40,50] וקבוצת Test עם [60,70]. כתבו את הנוסחאות שיידרשו כדי לתקנן את Test נכון. הסבירו מדוע לא מחשבים ממוצע חדש לפי Test.
- ח. בדקו באופן ידני כיצד משתנה מרחק Euclidean בין שתי נקודות אם מכפילים מאפיין אחד ב־100. הסבירו אם זו תכונה רצויה.
- ט. השוו את אותו אוסף נתונים עם ערך חריג, לפני הוספת החריג ואחריה. איזו שיטה רגישה למינימום ולמקסימום, ואיזו רגישה לממוצע ולסטיית תקן?
- י. ב־KNN מתקבל Accuracy שונה עם ובלי Scaling. תארו ניסוי תקין המאפשר להשוות את שתי האפשרויות ללא Data Leakage.

20. שאלות בסגנון בגרות — רמת ניתוח והסקה

השאלות הבאות מקוריות ונכתבו לצורך הכנה לרמת החשיבה הנדרשת בבגרות; הן אינן משוכפלות משאלון רשמי. לכל שאלה יש לנמק כל צעד ולהבחין בין חישוב לבין מסקנה.

שאלה 1 — מסווג מרחקים (20 נקודות מוצעות): נתון מאפיין A בטווח 0–10 ומאפיין B בטווח 0–1000. נקודת שאילתה Q=(2,100), ושתי נקודות אימון A=(3,110), B=(6,101). א. חשבו את המרחקים במקור (6 נק׳). ב. בצעו Min-Max לפי גבולות האימון (6 נק׳). ג. חשבו מרחקים חדשים (4 נק׳). ד. הסבירו את שינוי השכן הקרוב ביותר (4 נק׳).

שאלה 2 — הכנת נתונים והכללה (20 נקודות מוצעות): תלמיד חילק נתונים ל־Train ו־Test רק לאחר שקרא StandardScaler.fit_transform על כל המאגר. א. מהי הבעיה (6 נק׳)? ב. סדרו את שלבי התהליך כראוי (6 נק׳). ג. כתבו שלד קוד מתאים ל־Pipeline (4 נק׳). ד. הסבירו למה אין להכריע מהמודל הטוב ביותר על סמך Test שוב ושוב (4 נק׳).

שאלה 3 — נימוק מקצועי (15 נקודות מוצעות): נאמר שנתון מתוקנן Z=2.5. א. פרשו את הערך (5 נק׳). ב. האם פירוש זה מחייב שהתכונה נורמלית (5 נק׳)? ג. מדוע אי אפשר להסיק ממנו ישירות הסתברות של 2.5% (5 נק׳)?

21. מעבדת Colab — חוקר נתונים לפני אימון מודל

המעבדה הנלווית לקובץ הספר מיועדת להתנסות עצמאית. היא כוללת משימות הדרגתיות: חישוב בסיסי ביד וב־NumPy, תרגיל Min-Max, דוגמה של תלמיד חדש מחוץ לטווח האימון, הדגמת שינוי שכן קרוב ותרגול Pipeline. בכל שלב יש מקום להשלמת קוד ולהסבר במילים. המחברת אינה מכילה את פתרונות התלמיד; את פתרונות המורה נשמור במהדורה נפרדת.

דרך העבודה: תחילה כותבים תחזית מספרית ומנמקים אותה. אחר כך משלימים את התא החסר. רק לאחר מכן מריצים את המחברת ומשווים לתחזית. אם התקבל ערך שונה, עוצרים כדי למצוא אם הבעיה היא בחשבון, בנוסחה, בהגדרת קבוצת האימון או בקוד עצמו.

22. הכנה לפרויקט גמר — יומן החלטות מקצועי

לכל פרויקט AI מומלץ לשמור יומן ניסויים המסביר מה שונה בין גרסאות המודל. כאשר משנים את שיטת תקנון הנתונים, אין לומר רק ״שיפרנו את Accuracy״. עלינו לציין באיזה מאגר עבדנו, איזה חלוקה ביצענו, מה הייתה שיטת התקנון, מה היו מדדי ההערכה, האם אותם נתוני בדיקה נשמרו, ומדוע בחרנו בשיטה הסופית.

שאלה שיש לתעד	רכיב ביומן
אילו מאפיינים נשמרו ומה משמעות היחידות?	נתונים
כיצד בוצעה הפרדת Train / Validation / Test?	חלוקה
איזה scaler נלמד, על אילו דוגמאות ומדוע?	המרה
איזה אלגוריתם, אילו פרמטרים ומה היה קו הבסיס?	מודל
אילו מדדים התקבלו ומה ניתן להסיק מהם?	הערכה
מה עלול להשתנות כשיופיעו נתונים חדשים?	מגבלות

23. בדיקת שליטה עצמית לפני שעוברים לפרק הבא

- ☐ אני מסוגל להסביר מדוע מספר גדול יותר במאפיין אינו הופך אותו אוטומטית לחשוב יותר.
- ☐ אני יודע לפרק את נוסחאות Z-score ו־Min-Max ולהציב בהן נתונים בלי להתבלבל בין ממוצע למינימום.
- ☐ אני יכול להסביר מה פירוש Z שלילי ומדוע אין זה מספר הסתברותי.
- ☐ אני מסוגל לחשב ערך Min-Max עבור תצפית חדשה שמחוץ לטווח האימון.
- ☐ אני מבין מדוע אפשר ש־KNN יבחר שכן אחר לאחר שינוי קנה המידה.
- ☐ אני מסוגל להסביר מדוע StandardScaler.fit נעשה רק על Train.
- ☐ אני יודע להבחין בין Min-Max Scaling, תקנון Z-score ונרמול וקטור ליחידת אורך.
- ☐ אני יודע לכתוב מסקנה זהירה כאשר תוצאה משתנה בעקבות בחירת שיטת הכנה.

כרטיס יציאה תלמיד חדש קיבל ציון 95. האם צריך לחשב מחדש את הממוצע וסטיית התקן של כל

תלמידי האימון? הסבירו מדוע, וציינו איזו פעולה נבצע בקוד לפני החיזוי.

24. מילון מושגים

מושג	הסבר פשוט
Feature Scaling	שינוי הסולם של מאפיינים כדי לאפשר השוואה ועבודה נוחה למודל.
Z-score / Standardization	מרחק מן הממוצע ביחידות סטיית תקן.
Min-Max Scaling	מיקום הערך ביחס למינימום ולמקסימום של קבוצת האימון.
Normalization to unit length	חלוקת וקטור בנורמה שלו כך שאורכו יהיה 1.
fit	למידת פרמטרי ההמרה מתוך קבוצת האימון.
transform	הפעלת ההמרה באמצעות הפרמטרים שכבר נלמדו.
Data Leakage	שימוש במידע לא מורשה בתהליך האימון או בחירת המודל.
Pipeline	שרשרת פעולות הכנה ומודל הפועלות בסדר קבוע.
Outlier	ערך בולט או חריג ביחס לשאר הנתונים, המחייב בדיקה בהקשר.

25. סיכום הפרק והקשר להמשך

כאשר אנו בונים מודל למידת מכונה, הנתונים אינם נכנסים אליו תמיד כפי שנאספו. לפעמים עלינו לשנות את הסולם שלהם כדי שהמודל יוכל להשוות בין המאפיינים באופן מתאים. בפרק למדנו שתי שיטות מרכזיות: Z-score, המודד את המרחק היחסי מהממוצע, ו־Min-Max Scaling, הממיר נתונים לפי הטווח של קבוצת האימון. למדנו גם שנרמול וקטור ליחידת אורך הוא תהליך אחר, ושעלינו להימנע מבלבול ביניהם.

מעבר לחישוב עצמו, למדנו עיקרון מקצועי מרכזי: כל פרמטר של הכנת הנתונים נלמד רק מתוך קבוצת האימון המותרת. אותה המרה משמשת אחר כך את נתוני האימות, הבדיקה והנתונים החדשים. הדגמנו כיצד שינוי קנה מידה עשוי לשנות את החלטת KNN, ומדוע אין להסיק מכך שתקנון מסוג מסוים הוא תמיד הפתרון הטוב ביותר.

בפרק הבא נבחן יחסים בין משתנים: נלמד להבחין בין קשר חיובי לשלילי, לקרוא גרפי פיזור, לחשב מתאם, להבין כיצד קשרים מסייעים לחקר נתונים ומדוע מתאם אינו מוכיח סיבתיות. הידע הזה יעזור לנו בהמשך לבחור מאפיינים לרגרסיה ולמודלים אחרים.

מקורות ועקרונות ההתאמה

עוגן הסילבוס: יחידה 5, פרק 2 — בסיס סטטיסטי לחקר נתונים, סעיף 2.3 תקנון ונרמול. נושאי 2.1, 2.2 ו־1.2 משמשים ידע קודם. מפת הפרקים כפי שנשמרה במאגר Data-Science-ML-Course של צוות ההוראה; יש לוודא התאמה סופית לגרסת התכנית הרשמית התקפה לשנת הבחינה.

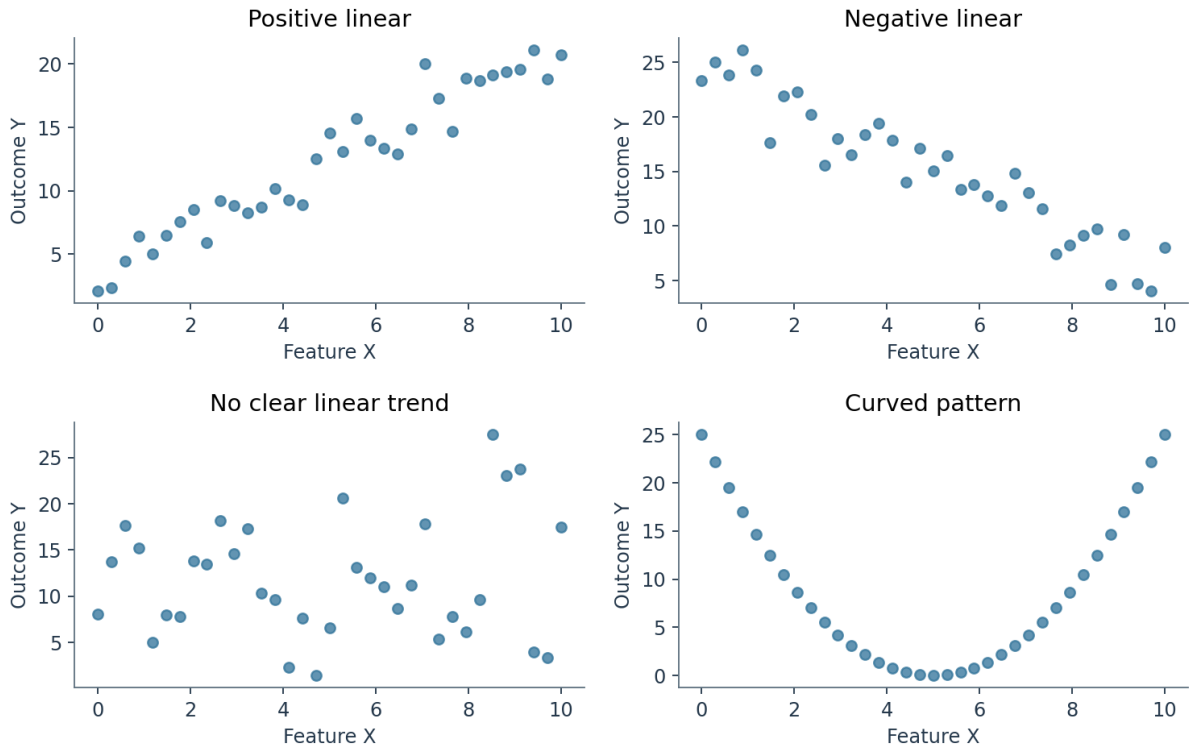
מקורות השראה מקצועיים — לא העתקת טקסט: Mathematics for Machine Learning (Deisenroth, Faisal & Ong); Hands-On Machine Learning (Géron); An Introduction to Statistical Learning with

Applications in Python; The StatQuest Illustrated Guide to Machine Learning; Grokking Machine Learning (Serrano). התוכן, הדוגמאות, התרשימים והתרגילים בפרק נכתבו במיוחד לתלמידי תיכון בעברית.

תיעוד למורה: ההקשרים ל־Data Leakage, Pipeline, בחירת scaler ביחס לאלגוריתם, טיפול בחריגים ובערכים קבועים הם העמקה מקצועית של הספר. הם אינם מוצגים כהעתק של ניסוח דרישה מחייבת בתכנית משרד החינוך, כל עוד לא אומת הנוסח הרשמי המדויק.

פרק 2.4 — יחסים בין משתנים

מה אפשר ללמוד משני משתנים שנעים יחד, וכיצד נמנעים ממסקנות שגויות בניתוח נתונים ובּAI?



איור — ארבע צורות קשר שונות. תרשים פיזור נותן לנו מידע שמקדם מתאם אחד לבדו אינו מסוגל להעביר.

חוברת תלמיד מורחבת • הסברים בעברית • דוגמאות פתורות • חישוב ידני • Python • מעבדה עצמאית

ספר הקורס המאוחד: נושאי ליבה בבינה מלאכותית ולמידה עמוקה | קובץ פרק עצמאי

על נתוני הפרק כל הנתונים המופיעים בטבלאות ובגרפים הם דוגמאות לימוד מקוריות או נתונים מסומלצים. אין להסיק מהם מסקנות על תלמידים אמיתיים. המטרה היא ללמוד דרך מדעית וזהירה לחשוב על קשרים.

מטרות הלמידה והידע הקודם

עד סוף הפרק לא נסתפק בשאלה ״כמה שווה המתאם?״. נלמד קודם לזהות מה מציגות הנקודות בגרף, אילו שאלות אפשר לענות עליהן באמצעותן, ואילו שאלות דורשות מידע נוסף. רק לאחר שנבין את משמעות הקשר, נחשב אותו ביד ובּPython ונבדוק כיצד הוא משתלב בתכנון מודלים של למידת מכונה.

- לזהות במציאות זוגות משתנים ולהגדיר נכון משתנה מסביר ומשתנה נבדק, בלי להניח מראש סיבתיות.
- לקרוא תרשים פיזור ולהבחין בין קשר חיובי, קשר שלילי, קשר לאּליניארי והיעדר קשר ליניארי.
- לחשב שונות משותפת (Covariance) באמצעות טבלת סטיות מלאה ולהבחין בין חישוב של מדגם לחישוב של אוכלוסייה.

- לחשב את מתאם פירסון r על נתונים קטנים, ולפרש את הסימן, הגודל והתחום שבין 1− ל־1.
- להבין מדוע מתאם אינו מעיד לבדו על סיבתיות, וכיצד חריגים, ערבוב קבוצות ומאפיינים מיותרים משפיעים על המסקנה.
- לשחזר את החישוב עם NumPy ו־Pandas, לבנות גרפים ולתעד מסקנה מבוססת־נתונים.
- ליצור גשר למודל רגרסיה, לבחירת מאפיינים ולפרויקט גמר שבו בודקים השערות באופן מבוקר.

ידע קודם: ממוצע, שונות וסטיית תקן מפרק 2.1; קריאת צירי גרף; חיבור, חיסור ושורש ריבועי; טבלת DataFrame ו־NumPy בסיסי. אין צורך להכיר מראש את הרגרסיה הליניארית.

חלק	נלמד	מה התלמיד יפיק
א׳	הבנת קשרים באמצעות גרפים	תיאור מילולי מנומק
ב׳	שונות משותפת ומתאם פירסון	חישוב ידני מדויק
ג׳	מגבלות: חריגים, אי־ליניאריות וסיבתיות	מסקנה ביקורתית
ד׳	NumPy, Pandas וקריאת מטריצת מתאמים	מימוש ובדיקת תוצאה
ה׳	תרגול בגרות ומעבדת חקר	תוצר עצמאי וראיות להבנה

1. סיפור הפתיחה: האם יותר שעות תרגול משפרות ציונים?

דודו בונה מערכת שתמליץ לתלמידי מגמת AI כיצד לתכנן את זמן הלמידה לקראת המבחן. הוא מתחיל משני נתונים פשוטים: כמה שעות תרגל כל תלמיד בשבוע ומה היה הציון שלו במבחן קצר. הרעיון נשמע הגיוני: מי שמתרגל יותר אולי יקבל ציון גבוה יותר. אבל כיצד בודקים את הרעיון מבלי לנחש?

דודו אוסף נתונים מארבעה תלמידים. המספרים קטנים בכוונה כדי שיהיה אפשר לחשב הכול ביד. כל שורה מתארת תלמיד אחד, וכל זוג ערכים שייך לאותו תלמיד — אי אפשר למיין את עמודת השעות בנפרד ולשמור על אותה התאמה לציון.

תלמיד	x — שעות תרגול	y — ציון קצר
א׳	1	2
ב׳	2	4
ג׳	3	4
ד׳	4	6

בדוגמה זו מספרי הציון נבחרו בקנה מידה קטן כדי לפשט את החישוב. הם אינם ציונים מלאים בין 0 ל־100. כאשר מסתכלים על הטבלה, רואים שבאופן כללי תלמידים שתרגלו יותר קיבלו ערכים גבוהים יותר. זו השערה על קשר בין שני משתנים — אבל עדיין לא הוכחה ששעות התרגול לבדן גרמו לציון.

שאלת פתיחה דמיינו שתלמיד חזק מאוד במתמטיקה מתרגל מעט ובכל זאת מצליח, ותלמיד אחר מתרגל זמן רב מפני שהנושא קשה לו. האם מספר שעות התרגול לבדו מסביר את כל ההבדלים? עלינו לזכור שיש גורמים נוספים.

2. מהו משתנה? מהו זוג תצפיות?

משתנה הוא מאפיין שיכול לקבל ערכים שונים בין תצפיות. בדוגמה שלנו X הוא מספר שעות התרגול ו־Y הוא ציון המבחן. כל תלמיד מייצג זוג מדידות (X,Y), למשל התלמיד השלישי הוא (3,4). כדי לבחון קשר בין משתנים חשוב לשמור על ההתאמה הנכונה בין הערכים: שני המספרים בזוג מתייחסים לאותה תצפית.

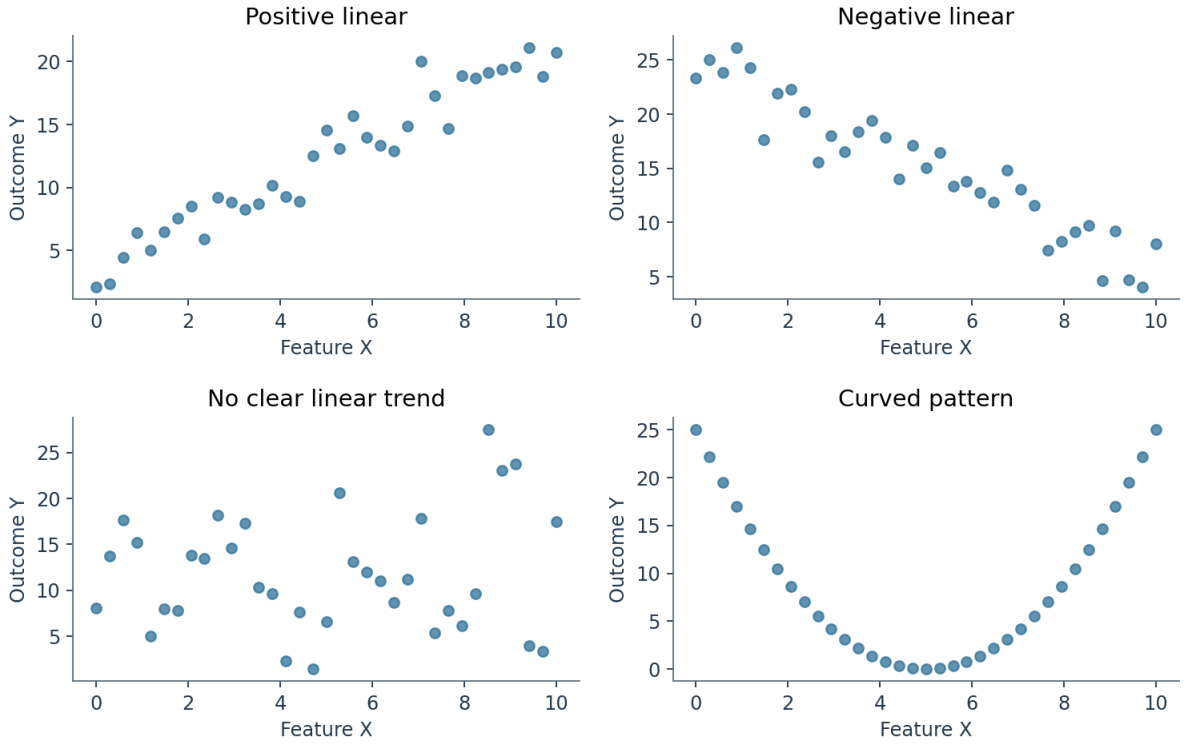
בהקשר של למידת מכונה מקובל לקרוא ל־X מאפיין (Feature) ול־Y משתנה מטרה (Target) כאשר מגדירים בעיית חיזוי. אבל בפרק סטטיסטי אפשר לבחון קשרים גם בלי לקבוע מי הגורם ומי התוצאה; עצם הכינוי ״משתנה מסביר״ אינו מוכיח שהוא הסיבה לשינוי האחר.

בדוגמה שלנו	משמעות	מונח
תלמיד מסוים	רשומה אחת	תצפית
שעות תרגול	משתנה ראשון	X
ציון מבחן	משתנה שני	Y
(3,4)	שני ערכים שייכים לאותה תצפית	זוג נתונים
עלייה כללית בציון כששעות עולות	דפוס שינוי משותף	קשר

3. תרשים פיזור: לראות את הקשר לפני שמחשבים

תרשים פיזור (Scatter Plot) מציג כל תצפית כנקודה במישור. הערך על ציר X קובע את מיקומה האופקי, והערך על ציר Y קובע את מיקומה האנכי. היתרון הוא שאנחנו רואים את כל הזוגות באותו הזמן: מגמה, פיזור, חריגים, קבוצות שונות וקשרים שאינם קווים ישרים.

כדי לצייר תרשים כזה מתחילים בבחירת המשתנים, כותבים שם יחידות לכל ציר ומסמנים נקודה אחת לכל תצפית. לאחר מכן שואלים שלוש שאלות: לאיזה כיוון נעות הנקודות בדרך כלל? עד כמה הן צמודות לקו מדומיין? והאם הצורה שלהן בכלל מתאימה לקו ישר? רק אז מחליטים איזה מדד לחשב.



איור — קשר חיובי, שלילי, היעדר מגמה ליניארית וקשר מעוקל. המדדים המספריים צריכים להיקרא יחד עם התרשים.

3.1 קשר חיובי, קשר שלילי וקשר שאינו ליניארי

בקשר חיובי, ערכים גבוהים יותר של X נוטים להופיע לצד ערכים גבוהים יותר של Y. בקשר שלילי, ערכים גבוהים יותר של X נוטים להופיע לצד ערכים נמוכים יותר של Y. כאשר הנקודות מפוזרות בלי מגמה ישרה נוכל לומר שלא זוהה קשר ליניארי ברור, אך לא להסיק מכך שאין שום קשר. לפעמים הקשר מעוקל, מדרגתי או משתנה בין תת־קבוצות.

דוגמה לקשר שלילי אפשרי היא מספר התקלות של מחשב ביחס למספר שעות שימוש מאז התחזוקה האחרונה — רק אם הנתונים בפועל מראים זאת; אין כלל אוניברסלי שמבטיח את הכיוון. צורה מעוקלת יכולה להופיע כאשר תוספת תרגול מועילה בהתחלה, אך אחרי שעות רבות התועלת השולית פוחתת.

4. משותף או מנוגד? מממוצעים למכפלות סטייה

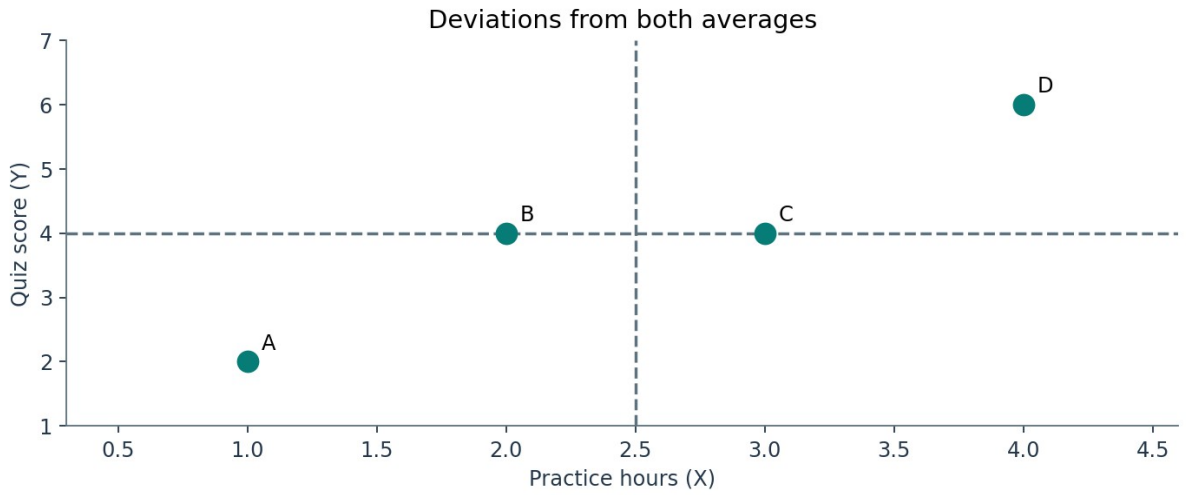
לפני שנכיר נוסחה חדשה נחזור לממוצע שלמדנו בפרק 2.1. מחשבים ממוצע נפרד לכל משתנה. אחר כך בודקים אם כל תצפית נמצאת מעל הממוצע שלה או מתחתיו בכל אחד משני המשתנים. כאשר שני המשתנים נמצאים מעל הממוצע, מכפלת הסטיות חיובית. גם כאשר שניהם מתחת לממוצע, מכפלת הסטיות חיובית. לעומת זאת, כאשר אחד מעל והשני מתחת, המכפלה שלילית.

$\bar{x} = (1 + 2 + 3 + 4) / 4 = 2.5$

$\bar{y} = (2 + 4 + 4 + 6) / 4 = 4$

נרשום טבלת מעקב מלאה: עבור כל תלמיד נחסר את ממוצע שעות התרגול מהשעות שלו ואת ממוצע הציונים מהציון שלו, ואז נכפיל את שתי הסטיות. את הפעולה הזו כדאי לבצע לפני השימוש בפונקציה מוכנה, כי היא מסבירה מאין נובע סימן הקשר.

תלמיד	X	Y	X−2.5	Y−4	מכפלה
א׳	1	2	−1.5	−2	3
ב׳	2	4	−0.5	0	0
ג׳	3	4	0.5	0	0
ד׳	4	6	1.5	2	3
סכום	10	16	0	0	6



איור — קווי המקווקו הם ממוצעי X ו־Y. נקודות שנמצאות יחד בצדדים תואמים תורמות מכפלה חיובית.

סכום המכפלות הוא 6, כלומר בדוגמה שלנו הכיוונים המשותפים גוברים על הכיוונים המנוגדים. זהו עדיין סכום שתלוי ביחידות המדידה. השלב הבא הוא להפוך אותו למדד שמסכם את הדפוס.

5. שונות משותפת — Covariance

שונות משותפת מודדת את הכיוון הכללי שבו שני משתנים משתנים יחד ביחס לממוצעים שלהם. היא מבוססת על סכום מכפלות הסטייה שראינו. כאשר השונות המשותפת חיובית, ערכים מעל הממוצע של אחד המשתנים נוטים להופיע לצד ערכים מעל הממוצע של השני; כשהיא שלילית נוטים לראות מצבים הפוכים. גודל המספר אינו מדד אוניברסלי לעוצמת הקשר, כי הוא תלוי בסולם וביחידות הנתונים.

$$\mathrm{Cov}_{pop}(X,Y) = \Sigma[(x_i-\mu_x)(y_i-\mu_y)] / N$$

$$\mathrm{Cov}_{sample}(X,Y) = \Sigma[(x_i-\bar{x})(y_i-\bar{y})] / (n-1)$$

נוסחת האוכלוסייה מחלקת במספר התצפיות N כאשר האוסף כולו מוגדר כאוכלוסייה שאנחנו מתארים. נוסחת השונות המשותפת המדגמית מחלקת ב־$n-1$ כשאנחנו רוצים לאמוד את השונות המשותפת באוכלוסייה על סמך מדגם בתנאים המקובלים. ההבדל זהה ברוחו להבדל בין שונות אוכלוסייה לשונות מדגם מפרק 2.1.

דוגמה פתורה 1 — חישוב שונות משותפת

סכום מכפלות הסטייה בטבלה שלנו הוא 6. אם ארבעת התלמידים הם כל הקבוצה שאנו מתארים, נחלק ב־4 ונקבל שונות משותפת של 1.5. אם נתייחס אליהם כאל מדגם שמטרתו לאמוד שונות משותפת באוכלוסייה רחבה יותר, נחלק ב־3 ונקבל 2. הסימן נשאר חיובי בשתי השיטות; רק המקדם השתנה.

$$\mathrm{Cov}_{pop} = 6 / 4 = 1.5$$

$$\mathrm{Cov}_{sample} = 6 / (4-1) = 2$$

במקרה שלנו היחידות של השונות המשותפת הן שעות תרגול כפול יחידות ציון. מכיוון שאלו יחידות מורכבות, קשה להשוות ישירות בין שונות משותפת של שעות וציונים לבין שונות משותפת של גובה ומשקל. לכן נצטרך מדד יחסי וחסר יחידות.

שגיאה נפוצה שונות משותפת שווה אפס אינה הוכחה לעצמאות סטטיסטית. ייתכן קשר לא־ליניארי גם כאשר מכפלות הסטייה החיוביות והשליליות מבטלות זו את זו.

6. מתאם פירסון — Pearson correlation

מתאם פירסון הוא מדד שמתאר את הכיוון ואת העוצמה של הקשר הליניארי בין שני משתנים כמותיים. בניגוד לשונות המשותפת, מתאם פירסון אינו תלוי ביחידות המדידה שלו. הוא מתקבל כאשר מחלקים את השונות המשותפת במכפלת סטיות התקן של שני המשתנים, כל עוד אף אחת מסטיות התקן אינה אפס.

$$r = \Sigma[(x_i-\bar{x})(y_i-\bar{y})] / \sqrt{[\Sigma(x_i-\bar{x})^2 \cdot \Sigma(y_i-\bar{y})^2]}$$

הנוסחה נראית ארוכה, אך היא מורכבת משלושה חישובים מוכרים: (1) סכום מכפלות הסטיות שמספר לנו אם התנועה משותפת או מנוגדת; (2) סכום ריבועי הסטיות של X, שמספר לנו עד כמה X מפוזר; (3) סכום ריבועי הסטיות של Y. השורש במכנה מכניס את שני סוגי הפיזור לאותו חישוב ומנרמל את המדד לטווח שבין -1 ל־1.

נקבל תוצאה קרובה ל־1 כאשר הנקודות צמודות לקו ישר עולה, תוצאה קרובה ל־-1 כאשר הן צמודות לקו ישר יורד ותוצאה קרובה ל־0 כאשר אין קשר ליניארי ברור. ערך 0 אינו שולל קשר מעוקל; ערך 0.9 אינו מוכיח סיבתיות.

ערך r	פירוש מתמטי	מה לא מסיקים ממנו
1	קשר ליניארי חיובי מושלם	לא מוכיח גורם ותוצאה

ערך r	פירוש מתמטי	מה לא מסיקים ממנו
0.8	קשר ליניארי חיובי חזק במדגם זה	לא קובע איכות מודל לבדו
0	אין קשר ליניארי במדד זה	לא שולל קשר לא־ליניארי
−0.8	קשר ליניארי שלילי חזק במדגם זה	לא קובע שהמשתנה גורם לירידה
−1	קשר ליניארי שלילי מושלם	לא מעיד על כל נקודה מחוץ למדגם

7. חישוב ידני מלא של מתאם פירסון

נשתמש שוב בארבעת התלמידים. כבר חישבנו את הממוצעים ואת סכום מכפלות הסטיות, ששווה 6. כדי להשלים את פירסון חסרים לנו רק סכומי ריבועי הסטיות של X ושל Y. נחשב אותם בתשומת לב בלי לדלג על שלבים.

תלמיד	X−2.5	Y−4	(X−2.5)²	(Y−4)²	מכפלה
א׳	−1.5	−2	2.25	4	3
ב׳	−0.5	0	0.25	0	0
ג׳	0.5	0	0.25	0	0
ד׳	1.5	2	2.25	4	3
סכום			5	8	6

$S_{xx} = 2.25 + 0.25 + 0.25 + 2.25 = 5$

$S_{yy} = 4 + 0 + 0 + 4 = 8$

$S_{xy} = 3 + 0 + 0 + 3 = 6$

$r = 6 / \sqrt{(5 \times 8)} = 6 / \sqrt{40} \approx 0.9487$

המתאם חיובי וגבוה מאוד, כלומר ארבע הנקודות מסודרות קרוב לקו ישר עולה. שימו לב שמדובר בארבע תצפיות בלבד; כאשר מספר התצפיות קטן, צריך להיזהר במיוחד בהכללה לקבוצות גדולות יותר. מטרת החישוב היא להבין את המדד, ולא להוכיח חוק טבע על למידה.

דוגמה פתורה 2 — קשר שלילי מושלם

נניח ש־X הוא 1, 2, 3, 4 ו־Y הוא 8, 6, 4, 2. בכל תוספת של יחידה אחת ל־X, משתנה Y יורד בשתי יחידות. כל הנקודות נמצאות בדיוק על ישר יורד ולכן מתאם פירסון הוא 1−. הסימן מלמד על כיוון ולא על איכות, וכך גם קשר שלילי מושלם אינו מוכיח סיבתיות.

$X = [1, 2, 3, 4]\ ;\ \ Y = [8, 6, 4, 2]\ ;\ \ r = -1$

8. לא תמיד קיים מתאם: משתנה קבוע

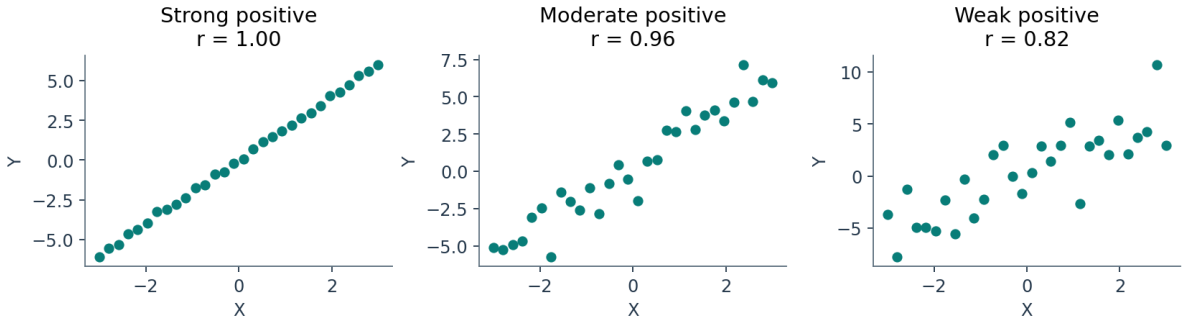
נניח שכל התלמידים תרגלו בדיוק שעתיים, כך שערכי X הם 2, 2, 2 ו־2. כל הסטיות מהממוצע של X שוות אפס, ולכן גם סכום הריבועים במכנה שווה אפס. במקרה כזה מתאם פירסון אינו מוגדר. חשוב לא לכתוב שהתוצאה אפס: אפס הוא ערך תקף של מתאם במקרים אחרים, ואילו כאן אין דרך לחשב מתאם מן הנתונים.

If $S_{xx} = 0$ or $S_{yy} = 0$, Pearson r is undefined.

כאשר נתקל בפלט NaN של NumPy או Pandas לגבי מאפיין קבוע, נבדוק קודם את השונות של אותו מאפיין במקום להחליף את הערך באפס בצורה אוטומטית. החלטה כזו חשובה במיוחד לפני בניית מטריצת מתאמים לפרויקט.

9. עוצמה וכיוון אינם אותו דבר

הסימן של r מלמד על הכיוון: חיובי או שלילי. הגודל המוחלט |r| מלמד על קרבה לקשר ליניארי מושלם. לדוגמה, r=−0.9 הוא קשר ליניארי שלילי חזק יותר מבחינת |r| מאשר r=0.4 שהוא קשר חיובי מתון. המספר אינו מדד ישיר לחשיבות חברתית, לימודית או עסקית של הממצא.

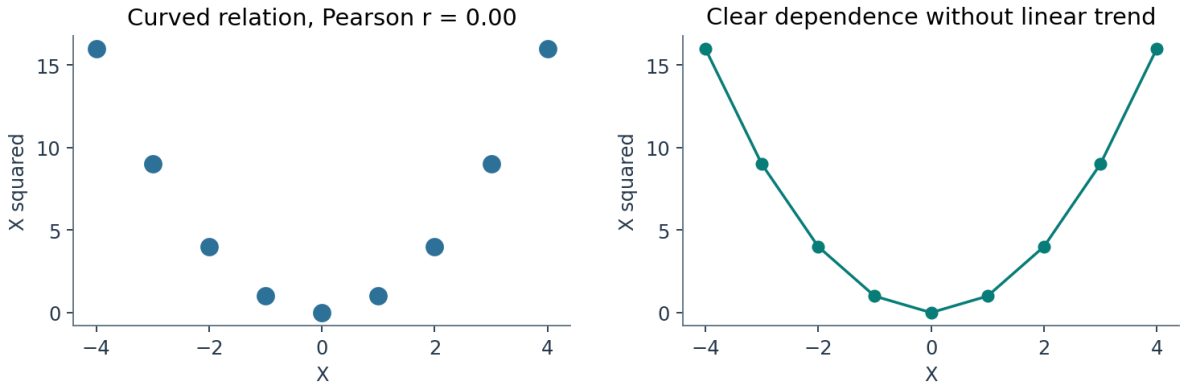


איור — שלושה ענני נקודות בעלי כיוון חיובי. ככל שהפיזור סביב המגמה גדול יותר, ערך המתאם קטן לרוב.

אין סולם אוניברסלי קשיח שקובע מה נחשב מתאם חלש או חזק בכל תחום. בעולם שבו טעות קטנה חשובה מאוד, גם קשר מתון עשוי להיות שימושי. בתחומים אחרים מתאם גבוה עשוי לנבוע ממדגם מוטה או ממשתנה שמדליף את התוצאה.

10. מדוע צריך לבדוק את התרשים גם כאשר r=0?

ניקח את X בערכים 4− עד 4, ונגדיר $Y=X^2$. אין ספק ש־Y תלוי ב־X: ברגע שיודעים את X אפשר לחשב בדיוק את Y. ובכל זאת, מתאם פירסון של הנתונים הסימטריים האלה הוא אפס, מכיוון שהעלייה בצד הימני מאוזנת על ידי הירידה בצד השמאלי מבחינת הקשר הליניארי.



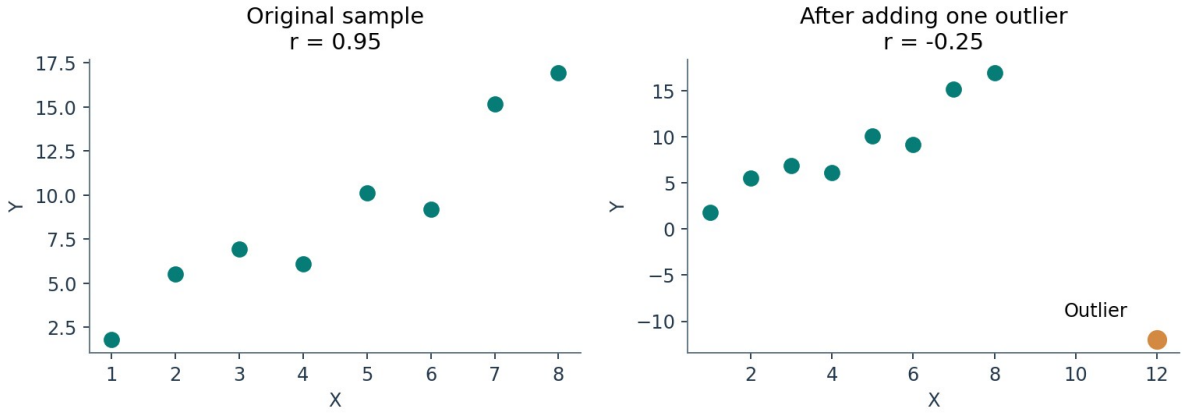
איור — התוצאה $Y=X^2$ יוצרת קשר מעוקל מושלם מבחינה פונקציונלית, אך המתאם הליניארי r שווה אפס.

$$X = [-2, -1, 0, 1, 2] ; Y = [4, 1, 0, 1, 4] ; r = 0$$

מכאן המשפט החשוב: ״אין מתאם ליניארי״ אינו שקול ל־״אין קשר״. כאשר בונים מודל AI, בדיקת r בלבד עלולה לפספס מבנה לא־ליניארי חשוב. לכן בודקים גם תרשימים, ידע בתחום ומודלים שמסוגלים ללמוד יחסים מורכבים.

11. איך ערך חריג יכול לשנות את הסיפור?

ערך חריג (Outlier) הוא תצפית שרחוקה מדפוס הנתונים המקובל. אם נוסיף תצפית קיצונית אחת לענן נקודות, היא עלולה למשוך את ממוצעי המשתנים ואת המתאם, ולעיתים אף להפוך את סימנו. לכן לפני פרשנות מתאם צריך לבדוק את התרשים ולברר האם התצפית קיצונית בגלל שגיאת מדידה, מקרה אמיתי חריג או קבוצת אוכלוסייה אחרת.

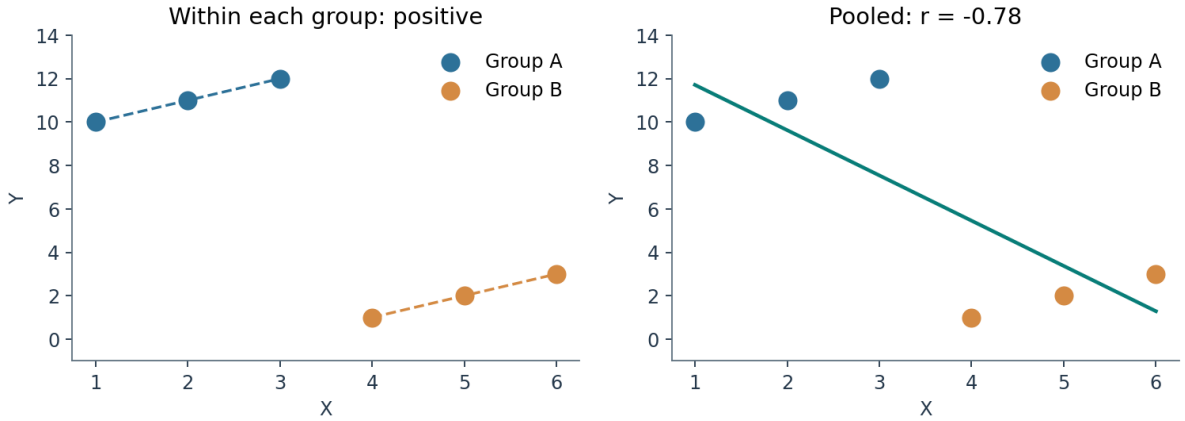


איור — דוגמה מסומלצת: תצפית אחת עשויה להשפיע על אומדן המתאם באופן חריף. אין למחוק חריג אוטומטית.

בפרויקט גמר מתעדים מה עשינו עם חריגים ומדוע. כדאי לחשב את המדד פעם אחת עם התצפית ופעם שנייה בלעדיה לצורך בדיקת רגישות, אך אין להסתיר את הגרסה שאינה נוחה לתוצאה המבוקשת. החלטה על סינון צריכה להיקבע לפי כללים מקצועיים, לא כדי ״לשפר״ מתאם.

12. ערבוב קבוצות — מתי הכיוון הכללי מטעה?

דמיינו שני בתי ספר שונים, שבהם בכל בית ספר מספר שעות תרגול גבוה יותר קשור לעלייה בציון. ובכל זאת, אם בית הספר השני משתמש במבחן קשה הרבה יותר, הערבוב של כל התלמידים באותו תרשים עשוי ליצור מגמה שלילית כללית. זו דוגמה לרעיון המוכר בשם פרדוקס סימפסון: מגמות בתת־קבוצות עשויות להיראות אחרת כאשר מאחדים את הקבוצות.



איור — בתוך כל קבוצה המגמה חיובית, אך כאשר מערבבים את הקבוצות עשויה להופיע מגמה שלילית כללית.

הלקח אינו שצריך תמיד לפצל את הנתונים, אלא שחייבים להבין כיצד נאספו. לפני שמדווחים מתאם על מאגר שיש בו קבוצות שונות (בתי ספר, כיתות, אזורים, שנות מדידה), יש לבחון האם משתנה קיבוץ חשוב משנה את הפרשנות. זהו חלק מחקר הנתונים האיכותי לפני אימון מודל.

13. מתאם אינו סיבתיות

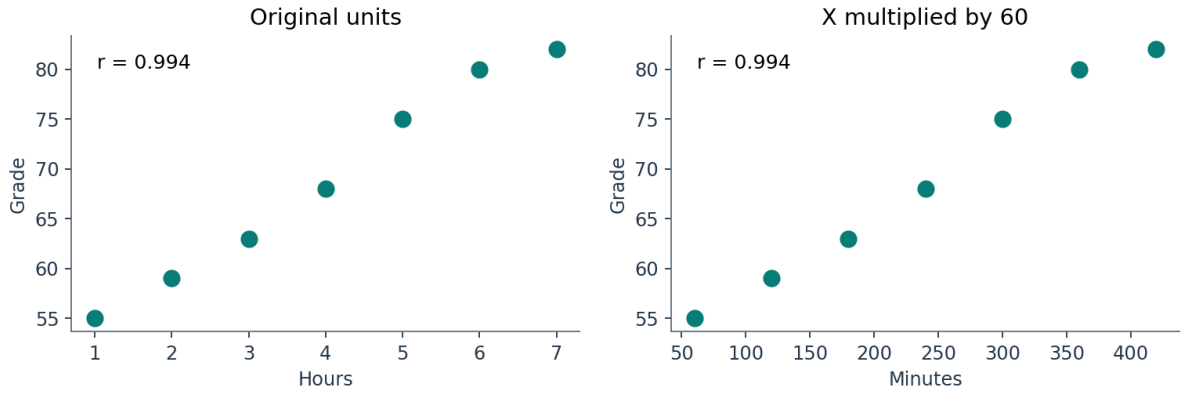
נניח שבחודשי הקיץ נמדדות גם מכירות גבוהות יותר של גלידות וגם עלייה במספר הנכנסים לים. שני המדדים עשויים לעלות יחד, אך אין להסיק שמכירת גלידה גורמת למספר הנכנסים לים לעלות. ייתכן שמזג האוויר החם משפיע על שניהם. גורם שלישי המשפיע על שני המשתנים נקרא משתנה מערפל או Confounder.

גם בדוגמת שעות התרגול והציונים לא נוכל לטעון בוודאות שהתרגול לבדו גרם לשיפור. ייתכן שיש קשר לרמת ידע קודמת, למוטיבציה, לסוג המבחן, לעזרה פרטית או למאפיינים נוספים. כדי לטעון סיבתיות נדרשים תכנון מחקר מתאים, ניסויים מבוקרים או שיטות הסקה סיבתית עם הנחות מפורשות — ולא רק חישוב של r.

מה צריך לעשות?	האם מותר להסיק ממתאם בלבד?	טענה
לציין גודל מדגם וגרף	כן, עם פירוש מדויק	שני המשתנים משתנים יחד במדגם
לבדוק תכנון מחקר וגורמים נוספים	לא	X גורם לשינוי ב־Y
לבדוק מודל ונתוני Test	לא בהכרח	אפשר לחזות Y היטב באמצעות X
לבדוק ייצוגיות והכללה	לא	ממצא חשוב לכל האוכלוסיות

14. שינוי יחידות לא משנה את מתאם פירסון

נניח שהשעות נמדדות מעכשיו בדקות. מכפילים את כל ערכי X ב־60, אבל לא משנים את זהות התלמידים או את ציוניהם. שונות משותפת ביחידות המקור גדלה בהתאם, אך מתאם פירסון נשאר זהה. זו אחת הסיבות שמתאם שימושי להשוואת זוגות משתנים שנמדדים ביחידות שונות.



איור — אותן נקודות מוצגות בשתי יחידות. המספרים על ציר X משתנים, אך הדירוג היחסי והמתאם נשארים זהים.

$$corr(X,Y) = corr(60X,Y) \quad \text{if } 60 > 0$$

כאשר כופלים את אחד המשתנים במקדם שלילי, כיוון הקשר מתהפך ולכן סימן המתאם מתהפך. הוספת קבוע לאחד המשתנים אינה משנה את מתאם פירסון. אין פירוש הדבר שכל המרות נתונים משמרות מתאם: למשל העלאה בריבוע יכולה לשנות באופן משמעותי את הקשר הליניארי.

15. איך כל זה מתחבר לרגרסיה ליניארית?

בפרק הלמידה העמוקה נלמד רגרסיה ליניארית, שבה מנסים להתאים ישר כדי לחזות Y מתוך X. המתאם עוזר לתאר את מידת הקשר הליניארי, אך אינו מספק לבדו תחזית ל־Y. כדי לחזות צריך לחשב גם את השיפוע ואת נקודת החיתוך של הישר, ובהמשך להעריך כמה טוב הוא עובד על נתונים שלא שימשו לאימון.

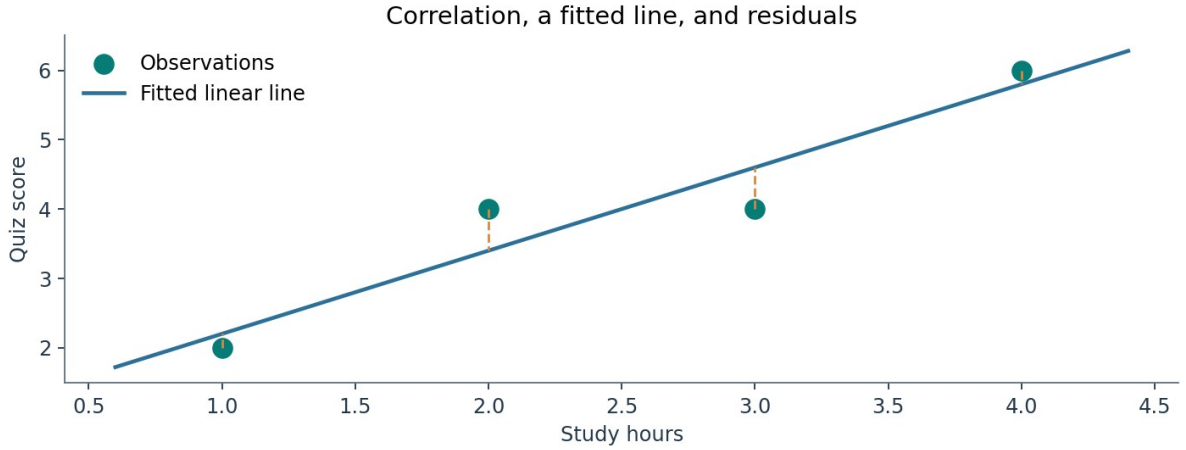
$$\hat{y} = b_0 + b_1 x$$

$$b_1 = S_{xy} / S_{xx} = 6 / 5 = 1.2$$

$$b_0 = \bar{y} - b_1 \bar{x} = 4 - 1.2 \times 2.5 = 1$$

$$\hat{y} = 1 + 1.2x$$

בדוגמה שלנו, כאשר תלמיד מתרגל 3 שעות, הישר חוזה ציון של $1.2+1\times3=4.6$. הציון הנמדד של התלמיד השלישי הוא 4, ולכן השארית (Residual) שלו היא $4-4.6=-0.6$. זה ממחיש את ההבדל בין תיאור קשר לבין מודל חיזוי: גם קשר ליניארי חזק אינו אומר שהמודל עובר דרך כל נקודה.



איור — קו רגרסיה שמותאם לארבע הנקודות. הקטעים המקווקווים מציינים שאריות, שיילמדו לעומק בהמשך.

מבט קדימה ברגרסיה ליניארית פשוטה עם חיתוך, ערך R^2 על נתוני האימון שווה ל־r^2 עבור אותו זוג משתנים, בתנאים הרגילים. זו הערת העמקה בלבד: אין להסיק מכך שהמודל יצליח על נתוני בדיקה שלא נראו.

16. יישום ב־Python: מן הטבלה לחישוב מתאם

לאחר שחישבנו ביד, נוכל לבדוק את עצמנו באמצעות Python. חשוב לשמר את סדר התצפיות: הערך הראשון של X מתאים לערך הראשון של Y, וכן הלאה. אם נשנה את סדר אחת הרשימות בלבד, נשווה זוגות שאינם שייכים זה לזה ונקבל תוצאה מטעה, גם אם הקוד פועל ללא שגיאה.

```
import numpy as np

x = np.array([1, 2, 3, 4], dtype=float)
y = np.array([2, 4, 4, 6], dtype=float)

x_mean = x.mean()
y_mean = y.mean()

sxy = np.sum((x - x_mean) * (y - y_mean))
sxx = np.sum((x - x_mean) ** 2)
syy = np.sum((y - y_mean) ** 2)

r_manual = sxy / np.sqrt(sxx * syy)
r_numpy = np.corrcoef(x, y)[0, 1]

print(round(r_manual, 4))
print(round(r_numpy, 4))
```

השורה x.mean() מחשבת ממוצע, הפעולות x-x_mean ו־y-y_mean מחשבות סטיות ממוצעים, ו־np.sum מחברת את הערכים אחרי פעולה רכיבית. np.corrcoef מחזירה מטריצת מתאמים, ולכן [0,1] בוחרת את התא שמבטא את הקשר בין המשתנה הראשון לשני. תוצאה הנמצאת באלכסון מתארת מתאם של משתנה עם עצמו, ולכן עבור משתנה שאינו קבוע היא 1.

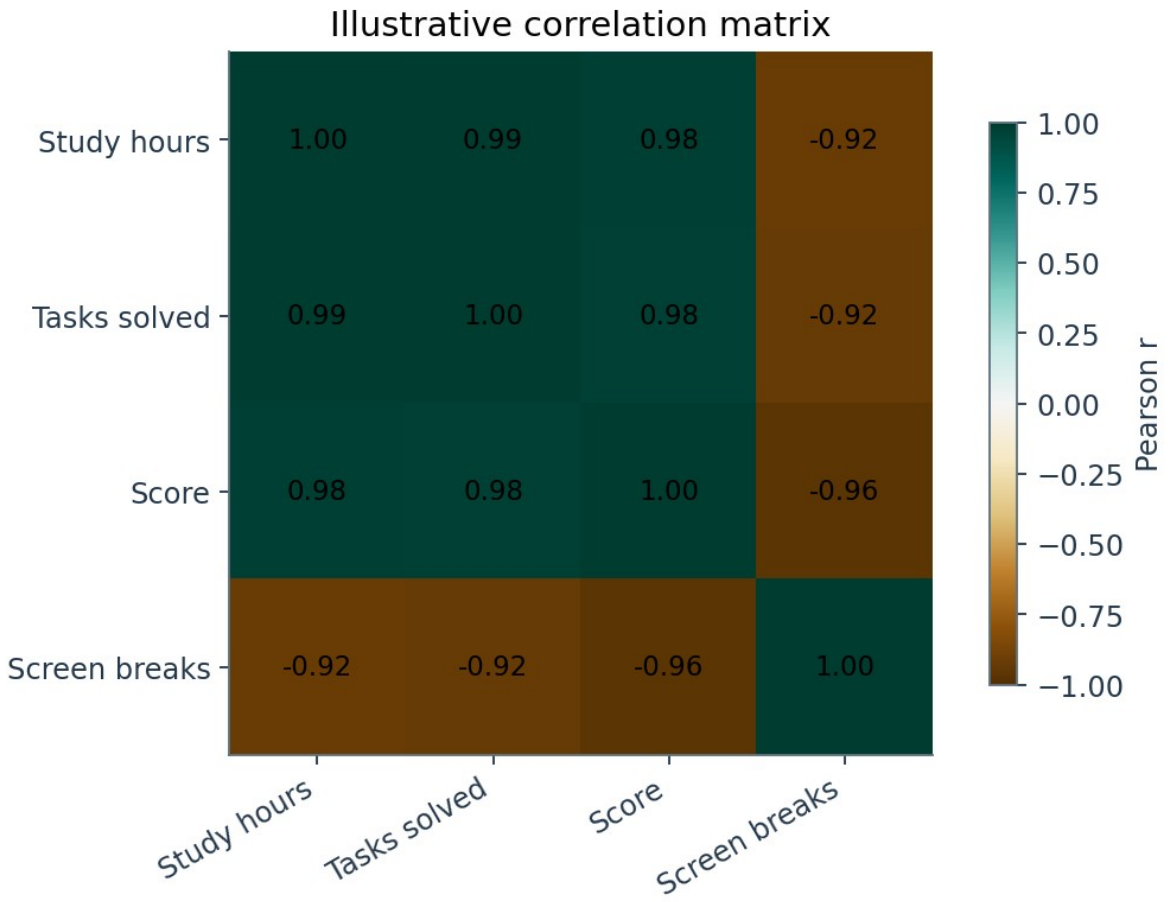
17. DataFrame ו־Pandas: ניתוח מספר משתנים

בפרויקטים אמיתיים יש בדרך כלל יותר משתי עמודות מספריות. Pandas מאפשרת לחשב מטריצת מתאמים — טבלה שכל תא בה משווה בין זוג עמודות. זוהי דרך נוחה לזהות מגמות ראשוניות, אך היא אינה מחליפה בחינת תרשימי פיזור ואת איכות הנתונים.

```
import pandas as pd

students = pd.DataFrame({
    "hours": [1, 2, 3, 4],
    "score": [2, 4, 4, 6]
})

print(students)
print(students.corr(numeric_only=True))
print(students["hours"].corr(students["score"]))
```



איור — מטריצת מתאמים על נתוני דוגמה נוספים; צבע ומספר מייצגים את כיוון הקשר הליניארי בין כל זוג תכונות.

לפני שימוש במטריצה צריך לוודא שמדובר בתכונות כמותיות בעלות משמעות מתאימה. שימו לב גם לערכים חסרים: Pandas יכולה לחשב כל זוג עמודות מתוך שורות בעלות ערכים תקינים לזוג הזה, ולכן שתי משבצות במטריצה אינן בהכרח מבוססות על אותו מספר תצפיות. במחקר מסודר מדווחים כמה זוגות תקינים השתתפו בכל חישוב.

18. בניית תרשים פיזור בקוד

אפשר לקבל מספר מתאם מבלי לראות את הנתונים, אבל מומלץ תמיד להפיק גם גרף. נתחיל מפקודות Matplotlib פשוטות. הפקודה scatter מציירת נקודה לכל זוג ערכים, xlabel ו־ylabel מציגות את משמעות הצירים, ו־grid מסייע לקרוא מיקום בלי להפוך את הגרף לעמוס.

```
import matplotlib.pyplot as plt

plt.scatter(students["hours"], students["score"])
plt.xlabel("Practice hours")
plt.ylabel("Quiz score")
plt.title("Practice and score: exploratory data")
plt.grid(alpha=0.2)
plt.show()
```

לאחר הציור יש לנסח מה באמת רואים: האם יש מגמה? האם היא נראית ישרה? האם נמצאו נקודות יוצאות דופן? כדאי להימנע ממשפטים מוחלטים כמו ״ככל שמתרגלים יותר, הציון תמיד עולה״, מפני שלעיתים קיימת מגמה כללית לצד מקרים בודדים שסותרים אותה.

19. שאלות ביקורת לפני שמכניסים מאפיין למודל AI

מתאם יכול לעזור בשלב חקר הנתונים, אבל הוא אינו מבחן אוטומטי לבחירת מאפיינים. מאפיין עם מתאם נמוך ליעד עשוי להיות חשוב עבור מודל לא־ליניארי, ושני מאפיינים בעלי מתאם גבוה ביניהם עשויים לספק מידע דומה שמקשה על פרשנות של חלק מהמודלים. חשוב עוד יותר לבדוק אם המאפיין יהיה זמין בזמן החיזוי: אסור להשתמש בטעות בעמודה שנוצרה מתוך התשובה שאנחנו מנסים לחזות.

- האם כל זוג ערכים מתאר אותה תצפית, ללא שיבוש סדר או כפילויות?
- האם אנו מודדים שני משתנים כמותיים, או משתמשים במספרים שרק מקודדים קטגוריות?
- האם התמונה בגרף מתאימה לקשר ליניארי או מציגה מבנה אחר?
- האם קבוצות משנה, שנתונים או בתי ספר שונים משנים את המגמה?
- האם יש חריגים, ערכים חסרים או מדידות חשודות שמשפיעות על התוצאה?
- האם המשתנה יהיה זמין ברגע שבו המודל ישתמש בו, והאם יש סכנת זליגת מידע?
- האם בחרנו והערכנו את המודל באמצעות הפרדה תקינה בין Train לבין Test?

20. תרגול מדורג — רמה א׳: הבנה וחישוב בסיסי

פתרו כל שאלה תחילה ללא מחשב. הראו שלבי חישוב והסבירו במילים מהי משמעות התוצאה. אפשר להיעזר במחשבון לצורך שורש ריבועי רק לאחר הצגת הנוסחה וההצבה.

- א1. תנו דוגמה מקורית לקשר חיובי ולדוגמה לקשר שלילי. האם הדוגמאות שבחרתם מבטיחות סיבתיות? נמקו.
- א2. נתונים X=[1,2,3] ו־Y=[3,6,9]. ציירו את הנקודות וקבעו את כיוון הקשר.
- א3. נתונים X=[1,2,3] ו־Y=[9,6,3]. האם מצופה מתאם חיובי או שלילי? הסבירו בלי לחשב.
- א4. מדוע יש לחשב ממוצע נפרד לכל משתנה לפני חישוב שונות משותפת?
- א5. חשבו שונות משותפת של אוכלוסייה ושל מדגם עבור X=[1,2,3], Y=[2,3,5].
- א6. מדוע מתאם של משתנה קבוע אינו מוגדר? כתבו את המקום בנוסחה שבו נוצרת הבעיה.

21. תרגול מדורג — רמה ב׳: ניתוח והסקה

- ב1. עבור X=[2,4,6,8] ו־Y=[9,7,5,3], חשבו מתאם פירסון בעזרת טבלת סטיות מלאה.
- ב2. תלמיד קיבל r=0.12 עבור קשר בין שעות תרגול לציון. האם מותר לקבוע שאין כל קשר ביניהם? הציעו שתי בדיקות נוספות.
- ב3. חוקר מציג מתאם בין מספר ספרים שנרכשו לבין הישגים במבחן וטוען שרכישת הספרים גרמה לשיפור. הסבירו מה חסר בטענה.
- ב4. מדוע שינוי שעות לדקות אינו משנה את פירסון, ואילו מעבר בין קבוצות תלמידים יכול לשנות אותו?
- ב5. תלמיד מפעיל df.corr() בלי להציג גרף. תארו דוגמה שבה המספר לבדו עלול להטעות.
- ב6. במאגר קיימים Missing Values בשתי עמודות. הסבירו כיצד הדבר עשוי להשפיע על מספר הזוגות שבהם מחשבים מתאם.

22. תרגול מדורג — רמה ג׳: שאלות בסגנון בגרות

השאלות הבאות מקוריות ונכתבו בסגנון הערכה עיונית. הן אינן שאלות בגרות רשמיות ואינן מציגות מחוון ניקוד רשמי. בכל תשובה נדרש חישוב מדויק וגם נימוק מילולי.

שאלה 1 — מתאם והנחת קשר ליניארי

נתונים X=[1,2,3,4] ו־Y=[2,4,4,6]. א. חשבו את שני הממוצעים. ב. השלימו את טבלת סטיות הממוצע. ג. חשבו את השונות המשותפת המדגמית. ד. חשבו את מתאם פירסון. ה. הסבירו מה ניתן להסיק ומה לא ניתן להסיק מן התוצאה. ו. תנו דוגמה לאופן שבו ערך חריג עשוי לשנות את המסקנה.

שאלה 2 — קשר לא־ליניארי ובחירת מודל

חוקרת בודקת X=[−2,−1,0,1,2] ו־Y=[4,1,0,1,4]. א. ציירו את הנקודות. ב. הסבירו מדוע אין זה נכון לפרש r=0 כהיעדר קשר בין המשתנים. ג. האם ישר פשוט צפוי לייצג היטב את הקשר? נמקו. ד. הציעו רעיון להמשך חקר בלי להניח שסיבתיות כבר הוכחה.

שאלה 3 — יחידות מדידה וניהול פרויקט

צוות AI מודד זמן בשעות ומחשב מתאם עם הצלחה במבחן. בהמשך מודדים את אותו זמן בדקות ומקבלים מתאם זהה. א. הסבירו מדוע התוצאה מתקבלת. ב. האם פירוש השיפוע ברגרסיה ישתנה? ג. איזה תיעוד על הצוות לצרף כדי שמפתח אחר יוכל לשחזר את החישוב?

23. אתגר פרויקט גמר: מחקר קצר על נתונים בעלי משמעות

בנו תהליך חקר קטן לפי נתונים אמתיים שיש לכם רשות להשתמש בהם, או לפי מאגר ציבורי שמתאים לגיל ולתחום. אפשר לבדוק קשר בין מחיר מחשב לבין גודל זיכרון, בין שטח דירה למחיר, בין מאפייני מוצר למכירות או בין מדדים פיזיים שונים של צמחים. הימנעו מאיסוף פרטים אישיים על תלמידים ללא צורך ואישור מתאים.

התוצר יכיל הגדרה של שני המשתנים ויחידותיהם, טבלת מקור מסודרת, תרשים פיזור, חישוב או בדיקת מתאם, חקירת חריגים, דיון בגורם שלישי אפשרי, מסקנה מוגבלת לראיות והצעה למודל חיזוי מתאים. אין צורך בשלב זה לאמן רשת עצבית. איכות החקר נמדדת ביכולת להסביר החלטות ולהכיר במגבלות.

שאלת בדיקה	מה צריך להגיש	שלב תיעוד
האם המדדים ברורים?	שאלה מדויקת על שני משתנים	שאלת חקר
האם הנתונים אמינים?	מקור, היקף ויחידות	מקור הנתונים
האם יש חריגים או קבוצות?	תרשים פיזור ותיאור	חקר חזותי
האם הקשר נראה ליניארי?	r ופירושו	מדד מספרי
האם נמנענו מסיבתיות ללא ראיה?	לפחות שני הסברים חלופיים	מגבלות

24. מילון מושגים לסיכום

מושג	משמעות קצרה
Scatter Plot	תרשים נקודות: כל נקודה מייצגת זוג ערכים של תצפית.
Covariance	מדד לכיוון ההשתנות המשותפת; תלוי ביחידות המדידה.

מושג	משמעות קצרה
Pearson r	מדד חסר יחידות לעוצמה ולכיוון של קשר ליניארי.
Outlier	תצפית חריגה העלולה להשפיע על חישובים והסקה.
Nonlinear	קשר שאינו מתואר היטב באמצעות קו ישר.
Confounder	משתנה נוסף שעשוי להסביר חלק מן הקשר שנצפה.
Correlation matrix	טבלה המציגה מתאם בין זוגות משתנים כמותיים.
Residual	הפרש בין תוצאה נמדדת לחיזוי של מודל.
Data leakage	מצב שבו מידע שאינו אמור להיות זמין בזמן חיזוי נכנס לתהליך האימון או בחירת המודל.

25. בדיקת מוכנות להמשך

לפני המעבר לפרק 2.5 — קריאת גרפים, בקשו מעצמכם לענות בעל־פה: כיצד יודעים אם קשר הוא חיובי? מדוע חשוב להסתכל על צורת הפיזור? איך בונים טבלת סטיות? במה שונים Covariance ו־Pearson? מה פירוש מתאם שלילי חזק? מתי r=0 אינו מעיד על היעדר קשר? כיצד אפשר לגלות גורם שלישי? ומה ההבדל בין תיאור קשר לבין חיזוי או סיבתיות?

מי שמסוגל לענות על שאלות אלה, לבצע חישוב ביד, לשחזרו ב־Python ולנמק מגבלה אמיתית של הנתונים — מוכן להמשך. מי שזקוק לחיזוק יחזור לטבלת הסטיות בפרק 7 ויבדוק מחדש כל סימן לפני הפעלת הספרייה.

מקורות ופירוט התאמה

עוגן ההוראה: יחידה 5, פרק 2 — בסיס סטטיסטי לחקר נתונים, סעיף 2.4 ״יחסים בין משתנים״, לפי מפת הסילבוס השמורה במאגר Data-Science-ML-Course. ההסברים, המספרים, התרגילים והאיורים בפרק זה נכתבו במקור לצורכי הוראת תלמידי תיכון. עומק הרגרסיה, פרדוקס סימפסון וסוגיות שימוש בפרויקט מסומנים כהעמקה דידקטית ואינם מוצגים כאן כחובת בחינה שאומתה מתוך הנוסח הרשמי האחרון.

השראה מקצועית: An Introduction to Statistical Learning with Applications in Python; Mathematics for Machine Learning; The StatQuest Illustrated Guide to Machine Learning; Hands-On Machine Learning; תיעוד SciPy ו־Pandas. ספרי המקור אינם מתורגמים או מועתקים לפרק; השימוש בהם הוא להבנת הרעיונות והוראתם.

המשך רצף הלמידה: 2.5 — קריאת גרפים וביקורת ייצוגים; בהמשך: מודלים מבוססי מרחק, רגרסיה, הערכת מודלים ובניית פרויקט גמר.

פרק 2.5 — קריאת גרפים וייצוג נתונים

איך הופכים נתונים לתמונה נכונה, מזהים הטעיה ומסיקים מסקנות שאפשר להצדיק?



Start from the question, not from the graph style

איור: לכל שאלת מחקר מתאים כלי חזותי אחר. בחירת הגרף מתחילה מסוג הנתונים ומהשאלה.

ספר הקורס המאוחד: נושאי ליבה בבינה מלאכותית ולמידה עמוקה | חוברת עצמאית לתלמיד

בפרק זה נלמד לקרוא גרפים בזהירות, לבנות אותם ב־Python, לזהות מתי ייצוג חזותי מסתיר מידע ולהסביר כיצד קריאה נכונה של נתונים משפיעה על בניית מודלים של AI. כל המספרים בדוגמאות הם נתונים מומצאים לצורכי הוראה, אלא אם צוין אחרת.

מפת הפרק, מטרות וידע קודם

בפרקים הקודמים למדנו על ממוצע, חציון, פיזור, תקנון, שונות משותפת ומתאם. כעת נעמיק בשאלה אחרת: כיצד מציגים את התוצאות כך שניתן להבין אותן באופן מדויק? תוכנת מחשב יכולה להפיק תרשים יפה מאוד גם כאשר התרשים מציג מסקנה מטעה. מדען נתונים אינו מסתפק ביופי; הוא בודק את משמעות הצירים, את סוג המשתנים, את גודל המדגם ואת המגבלות של הנתונים.

- לזהות משתנים קטגוריאליים, כמותיים בדידים וכמותיים רציפים, ולהתאים גרף לשאלת המחקר.
- לקרוא ולהשוות תרשימי עמודות, היסטוגרמות, תרשימי קופסה, קו, פיזור, עוגה ומפות חום.
- לחשב ביד שכיחות יחסית, ממוצע משוקלל, רבעונים וגבולות חריגים ולפרש את הקשר לגרפים.
- לזהות הטעיה הקשורה לצירים, לטווחים, לקבוצות לא מאוזנות, לנתונים חסרים ולבחירת מדגם.
- לבנות גרפים בעזרת Pandas ו־Matplotlib, ולהפריד בין ממצא, פרשנות והשערה.

ידע קודם: DataFrame, ממוצע, חציון, רבעונים, שונות, סטיית תקן ומתאם פירסון. אין צורך להכיר ספריית גרפים מתקדמת מעבר ל־Matplotlib.

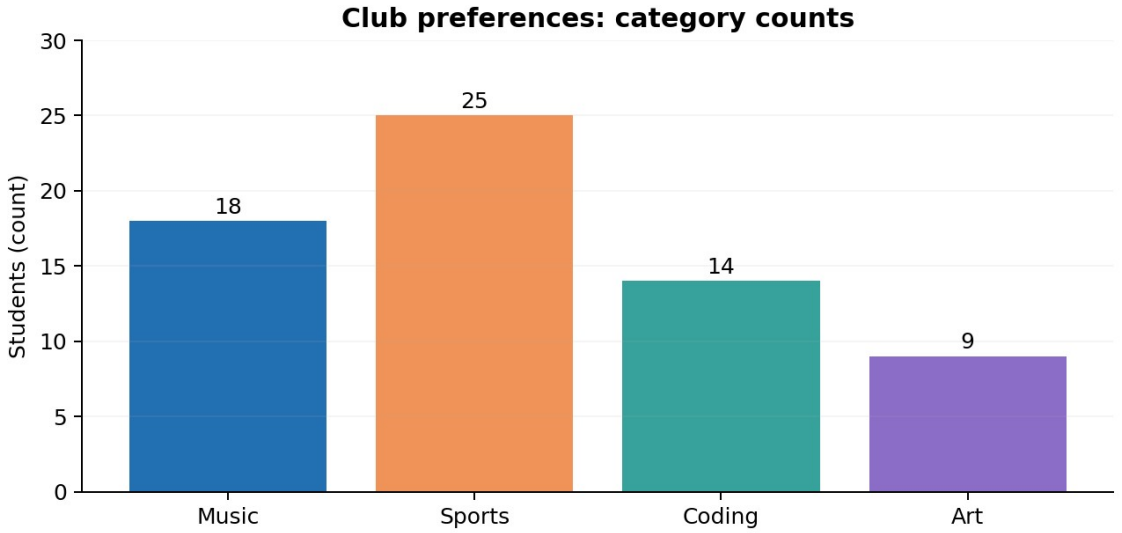
שאלת מחקר	סוג המידע	ייצוג מתאים
כמה תלמידים בחרו בכל חוג?	קטגוריאלי — מספר תלמידים	עמודות
כיצד מתפלגים זמני ההמתנה?	כמותי — מדידות רבות	היסטוגרמה / קופסה
האם מספר הפניות גדל משבוע לשבוע?	סדרת זמן	קו
האם שעות תרגול קשורות לציון?	שני משתנים כמותיים	פיזור

ייצוג מתאים	סוג המידע	שאלת מחקר
מפת חום של מתאמים	מספר זוגות מאפיינים כמותיים	אילו מאפיינים קשורים ביניהם?

1. סיפור פתיחה — לוח הבקרה של דודו

דודו עוזר להנהלת בית הספר לשפר את השירות בקפיטריה. ביום אחד הגיעו 66 תלמידים להירשם לחוגים, ובמקביל נאספו זמני המתנה בקפיטריה. המנהלת מבקשת לדעת לאילו חוגים יש ביקוש גבוה, כמה זמן מחכים בדרך כלל, האם זמני ההמתנה גדלים במהלך השבוע והאם תלמידים שמתאמנים יותר מקבלים ציונים גבוהים יותר. ארבע השאלות נשמעות דומות, אך הן עוסקות בסוגים שונים של נתונים. אי אפשר לענות על כולן באמצעות אותו גרף.

למשל, מספר נרשמים לחוג הוא ספירה לפי קטגוריה. זמן המתנה הוא מדידה מספרית; ניתן לבחון את צורת ההתפלגות שלו. מספר פניות בכל יום הוא נתון שתלוי בסדר הכרונולוגי. שעות תרגול וציון הם זוג מדידות של אותה תצפית. הבחנה זו היא הצעד הראשון בפתרון מדעי: לפני הפקת גרף אנחנו מגדירים מה כל רשומה מייצגת ומה בדיוק אנו מבקשים לגלות.



איור: דוגמאות לכמות נרשמים לפי חוג: יש לקרוא את הציר האנכי כ״מספר תלמידים״, לא כציון או אחוז.

עצירה לחשיבה

מה הייתם מציירים כדי לברר האם חוג ספורט מבוקש יותר מחוג מוזיקה? מדוע תרשים קו אינו הבחירה הראשונה? כתבו משפט המתחיל ב״כל עמודה מייצגת...״.

2. מהו גרף טוב, ומדוע לא מתחילים מהצבעים?

גרף טוב הוא ייצוג חזותי המסייע לענות על שאלה מוגדרת, בלי לשנות את משמעות המידע. עליו להציג כותרת שמבהירה מה נבדק, שם לכל ציר, יחידות מדידה, סולם מספרי הגיוני והסבר לסימנים או לצבעים. אם חסר אחד המרכיבים האלה, הקורא עלול לזהות מגמה שאינה קיימת או לייחס משמעות שאינה מוצדקת.

נניח שמופיע גרף שבו ציר X נקרא ״קבוצה״ וציר Y אינו מסומן. הגובה של עמודה הוא 18. האם מדובר ב־18 תלמידים, ב־18 דקות או ב־18 אחוזים? בלי יחידות לא ניתן לנסח מסקנה מדעית. גם כאשר היחידות מופיעות, יש לבדוק אם הנתונים הם ספירה מוחלטת או שיעור יחסי. קבוצה של 20 מתוך 40 אינה שקולה לקבוצה של 20 מתוך 200.

בדיקת גרף בשלוש שאלות

הסתכלו על כל גרף ושאלו: (א) מהי יחידת התצפית? (ב) מה מוצג בכל ציר ומהו הסולם? (ג) איזו מסקנה נתמכת, ואיזו מסקנה דורשת נתונים נוספים?

3. תרשים עמודות — השוואה בין קטגוריות

תרשים עמודות (Bar Chart) מתאים במיוחד להשוואה בין קבוצות מובחנות, כגון מגמות לימוד, סוגי מוצרים או מחלקות. כל עמודה מייצגת קטגוריה, והגובה שלה מייצג ספירה, סכום, ממוצע או שיעור — לפי מה שכתוב בכותרת הציר. העמודות מופרדות משום שהקטגוריות אינן בהכרח ערכים רציפים על ציר מספרי.

כדי לפרש את גרף החוגים נשווה את הגובה של העמודות: 25 תלמידים בחרו ספורט, לעומת 18 במוזיקה. ההפרש הוא 7 תלמידים. אם רוצים לבטא חלק מתוך כלל הנרשמים, מחלקים ב־66. שיעור הבוחרים בספורט הוא בקירוב 37.9%. אין להסיק מכך ש״37.9% מכלל תלמידי בית הספר״ בחרו ספורט, אלא אם ידוע ש־66 הנרשמים מייצגים את כלל תלמידי בית הספר.

$$\text{Relative frequency (\%)} = (\text{category count} / \text{total count}) \times 100$$

$$(25 / 66) \times 100 \approx 37.88\%$$

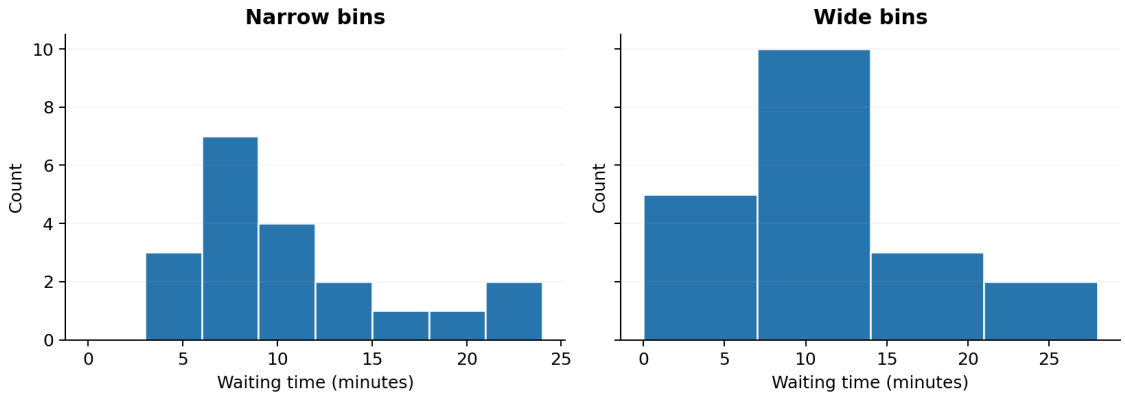
באופן רגיל כדאי להתחיל את הציר האנכי של עמודות מאפס, משום שהאורך הגרפי הוא בסיס להשוואת גודל. אם ציר עמודות מקוצץ, הבדלים קטנים עלולים להיראות דרמטיים. כאשר מציגים שינוי לאורך זמן בקו, לעומת זאת, נקודת אפס אינה תמיד הכרחית; הבחירה תלויה במה שמבקשים להראות.

```
import matplotlib.pyplot as plt
clubs = ["Music", "Sports", "Coding", "Art"]
counts = [18, 25, 14, 9]
plt.bar(clubs, counts)
plt.ylabel("Students (count)")
plt.title("Club preferences")
plt.ylim(bottom=0)
plt.show()
```

4. היסטוגרמה — לא עמודות רגילות

כאשר מודדים זמן המתנה של כל תלמיד, הגרף אינו מציג שמות של קטגוריות אלא מספרים על ציר רציף. היסטוגרמה מחלקת את טווח המדידה למחלקות (Bins) ובודקת כמה תצפיות נפלו בכל מחלקה. למשל, כמה תלמידים המתינו בין 0 ל־3 דקות, בין 3 ל־6 דקות וכן הלאה. בניגוד לתרשים עמודות קטגוריאלי, כאן סדר המחלקות נקבע על פי ערכן המספרי, והעמודות צמודות בדרך כלל.

לרוחב המחלקה יש משמעות. מחלקות צרות חושפות פירוט רב אבל גם תנודות מקריות; מחלקות רחבות מבליטות תמונה כללית ועלולות להסתיר שתי קבוצות שונות. לכן שינוי במספר המחלקות יכול לשנות את צורת ההיסטוגרמה בלי ששינינו ולו תצפית אחת. לפני שמתארים את ההתפלגות כנורמלית, מוטה או דו־שיאית, כדאי לבדוק אם הצורה יציבה בכמה חלוקות סבירות.



איור: אותם 20 זמני המתנה, עם חלוקה שונה למחלקות. הנתונים לא השתנו, אך המראה משתנה.

חשוב להבחין בין היסטוגרמת שכיחויות לבין היסטוגרמת צפיפות. כאשר רוחבי המחלקות אינם שווים, גובה העמודה לבדו אינו מייצג שכיחות; השטח שלה חייב להתאים לתדירות או להסתברות. הקשר הזה חשוב במיוחד כהכנה לפרק ההתפלגות הנורמלית שבו הסתברות מיוצגת על ידי שטח מתחת לעקומת צפיפות.

```
import numpy as np
waiting = np.array([4, 5, 5, 6, 6, 7, 7, 7, 8, 8,
                    9, 10, 11, 11, 12, 14, 17, 19, 22, 24])
plt.hist(waiting, bins=8, edgecolor="white")
plt.xlabel("Waiting time (minutes)")
plt.ylabel("Count")
plt.show()
```

שאלת חשיבה

אם היסטוגרמה נראית בעלת שיא אחד כאשר משתמשים ב־5 מחלקות ובעלת שני שיאים כאשר משתמשים ב־20 מחלקות — האם הוכחנו שהנתונים מגיעים משתי אוכלוסיות? נמקו.

5. תרשים קופסה — חמישה ערכי סיכום והבנת פיזור

תרשים קופסה (Boxplot) מאפשר להשוות התפלגויות באמצעות חמישה ערכי סיכום: מינימום, רבעון ראשון, חציון, רבעון שלישי ומקסימום. בתרשימים נפוצים קצות השפמים אינם בהכרח המינימום והמקסימום, ולכן צריך לברר כיצד תוכנת השרטוט בנתה אותם. ב־Matplotlib ברירת המחדל המקובלת מבוססת על שפמים המגיעים לערכים הקיצוניים הנמצאים בתוך תחום 1.5×IQR; ערכים שמעבר עשויים להופיע כנקודות נפרדות.

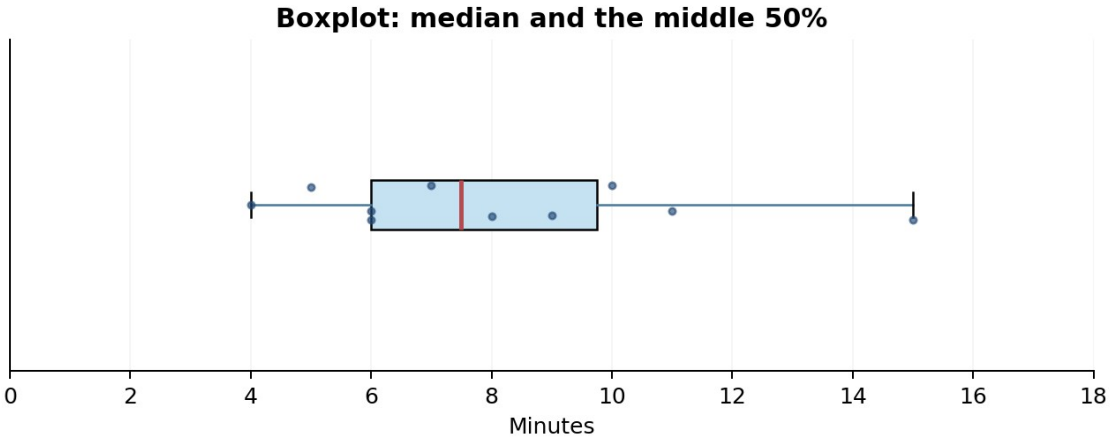
נשתמש בזמני המתנה ממוינים: 4, 5, 6, 6, 7, 8, 9, 10, 11, 15. החציון הוא ממוצע הערכים החמישי והשישי ולכן 7.5. בשיטת חצאי המדגם שלמדנו בפרק 2.1, החציון של חמשת הערכים התחתונים הוא 6 והחציון של חמשת הערכים העליונים הוא 10. הטווח הבין־רבעוני הוא 4 דקות.

$$\text{Median} = (7 + 8) / 2 = 7.5$$

$$Q1 = 6 \; ; \; Q3 = 10 \; ; \; IQR = Q3 - Q1 = 4$$

$$\text{Lower fence} = 6 - 1.5\times4 = 0 \; ; \; \text{Upper fence} = 10 + 1.5\times4 = 16$$

לפי כלל זה אין בדוגמה ערך מחוץ לגבולות 0–16. אילו הופיעה המתנה של 22 דקות, הייתה מסומנת כחריגה לצורך בדיקה. חריג אינו בהכרח נתון שגוי: הוא עשוי לייצג יום עמוס, תקלה, שינוי בתהליך או תופעה חשובה בפרויקט.

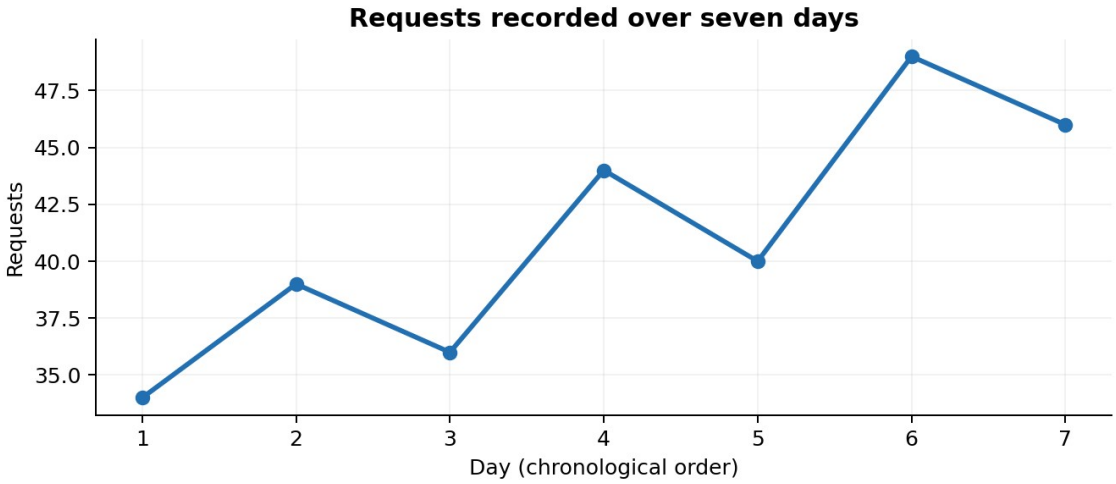


איור: תיבה המייצגת את 50% המרכזיים של הנתונים וסמן חציון. הנקודות מסייעות לראות את המדידות עצמן.

החישוב ביד מכוון לשיטת רבעונים אחת במפורש; קיימות שיטות נוספות בספריות שונות. לכן כאשר משווים תוצאה של Python לחישוב על הלוח, בודקים תחילה ששתי השיטות מגדירות את הרבעונים באותו אופן.

6. תרשים קו — סדר זמנים הוא חלק מן המידע

תרשים קו (Line Chart) מחבר מדידות שמתקדמות על פני ציר בעל סדר משמעותי, בעיקר זמן. הוא מתאים למשל למספר פניות למוקד בכל יום או לשינוי ממוצע השגיאה לאורך אפוקים באימון רשת עצבית. הקו אינו רק עיצוב: הוא מרמז על רציפות וקשר בין נקודות עוקבות. משום כך אין לחבר באופן אוטומטי שמות חוגים בקו; אין יחס של ״בין לבין״ בין חוג ספורט לחוג אמנות.



איור: מספר פניות לפי יום. סדר הימים אמיתי ולכן קו מסייע לראות מגמה ותנודות.

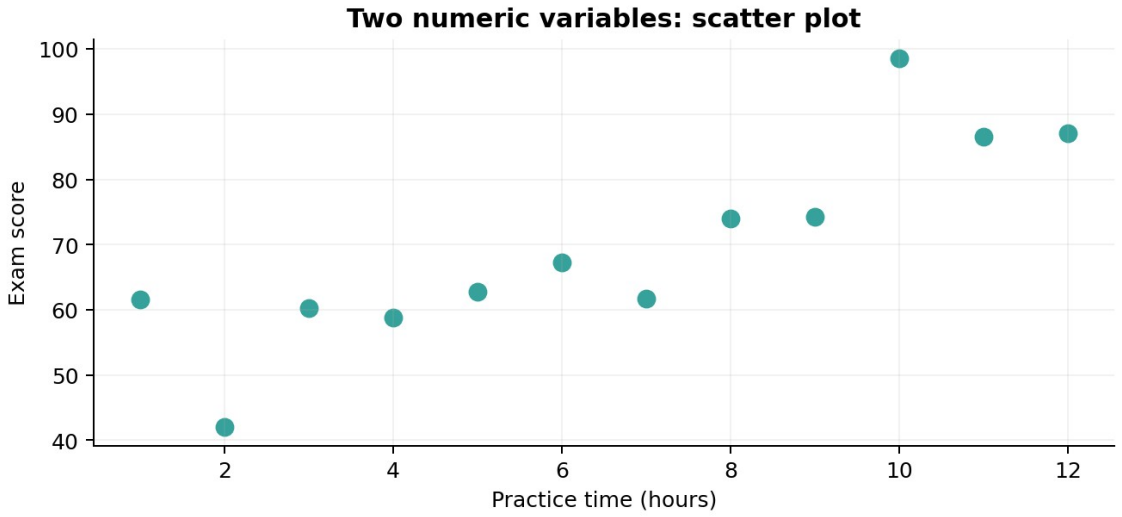
נניח שמספר הפניות בשבעת הימים הוא 34, 39, 36, 44, 40, 49, 46. בין היום הראשון לשביעי יש גידול של 12 פניות; עם זאת, אין גידול בכל יום. הניסוח ״מספר הפניות גדל בכל יום״ שגוי, בעוד ״המגמה הכוללת עולה למרות תנודות יומיות״ מתאים יותר לנתונים.

$$\text{Change} = \text{Last} - \text{First} = 46 - 34 = 12 \text{ requests}$$

אם חסרים נתונים ביום מסוים, אין להחליף אותם באפס בלי הצדקה. אפס מציין מדידה שבה לא היו פניות; ערך חסר מציין שלא ידוע כמה פניות היו. הבחנה זו משנה את צורת הגרף ואת המסקנות.

7. תרשים פיזור — האם שני משתנים משתנים יחד?

בתרשים פיזור (Scatter Plot) כל נקודה היא תצפית אחת עם שני ערכים מספריים. לדוגמה, לתלמיד מסוים יש 4 שעות תרגול וציון 67, ולכן הנקודה שלו ממוקמת בהתאם בשני הצירים. כאשר הנקודות נוטות לעלות משמאל לימין, יש רמז לקשר חיובי; כאשר הן יורדות — קשר שלילי. הפרק הקודם הראה כיצד מחשבים מתאם פירסון, אבל כאן חשוב להדגיש: לפני חישוב המדד קוראים את הנקודות ומזהים חריגים, קבוצות ודפוסים לא ליניאריים.



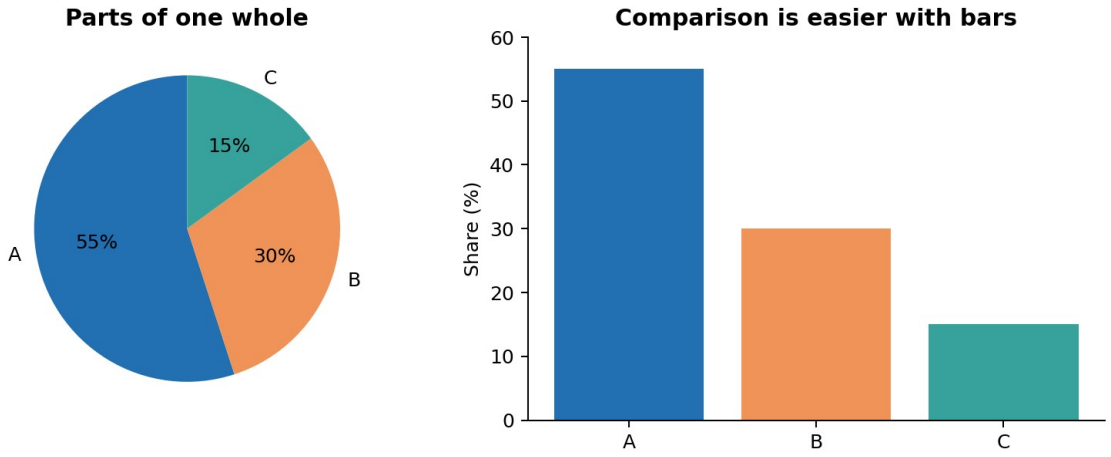
איור: פיזור דו־ממדי של שעות תרגול וציונים. כל נקודה מייצגת תלמיד אחד.

אין לחבר את נקודות התלמידים בקו לפי סדר הרשומות בקובץ; לרשומות שונות אין בהכרח רצף טבעי. גם אין להסיק שהוספת שעת תרגול גורמת בוודאות לעלייה מסוימת בציון. ייתכן שמאפיינים אחרים, כגון ידע קודם, קשורים לשני המשתנים. לשם חיזוי נלמד בהמשך רגרסיה, אך גם אז חיזוי טוב אינו הוכחה לסיבתיות.

```
import matplotlib.pyplot as plt
hours = [1, 2, 3, 4, 5, 6]
grades = [56, 59, 65, 67, 75, 74]
plt.scatter(hours, grades)
plt.xlabel("Practice hours")
plt.ylabel("Grade")
plt.title("Practice and achievement")
plt.show()
```

8. תרשים עוגה — שימוש מוגבל, לא פתרון לכל השוואה

תרשים עוגה מחלק שלם אחד לחלקים. הוא יכול לעזור כאשר מספר הקטגוריות קטן, כל קטגוריה בלעדית והחלקים מסתכמים ב־100%. למשל, התפלגות אמצעי הגעה לבית הספר: 55% אוטובוס, 30% הליכה, 15% רכב. תרשים עוגה אינו מתאים אם כל תלמיד יכול לבחור יותר מאפשרות אחת, משום שאז הסכומים עשויים לעלות על 100%.

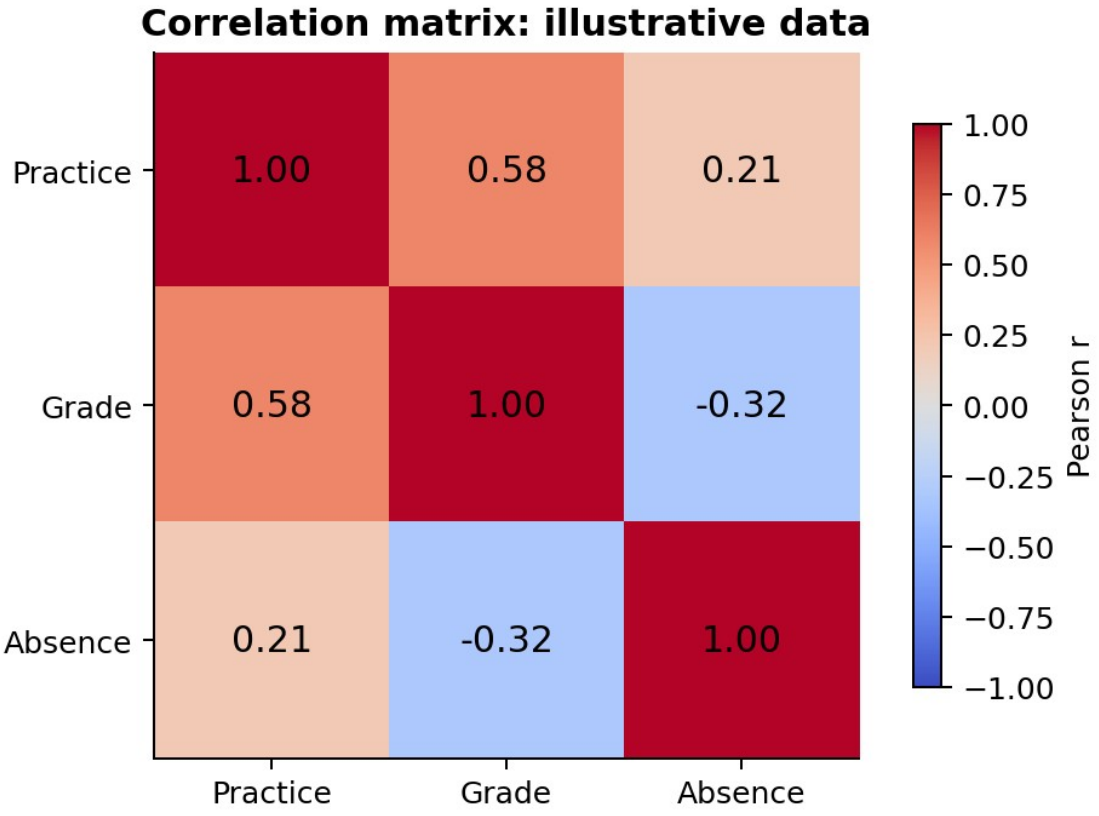


איור: אותם חלקים של שלם; תרשים עמודות מאפשר לעיתים השוואה מספרית מדויקת יותר.

כאשר ההבדלים בין הקבוצות קטנים, השוואת זוויות או שטחי פרוסות קשה יותר מהשוואת גובה עמודות. לכן כדאי לשאול לא רק אם אפשר להציג עוגה, אלא אם היא עוזרת לקורא להבין את הממצא. אם המטרה היא להשוות בין עשר קטגוריות, בדרך כלל עמודות יהיו בחירה טובה יותר.

9. מפת חום — קריאה של מטריצת קשרים

מפת חום (Heatmap) מציגה מטריצה שהערכים שלה מסומנים באמצעות צבע. למשל, מטריצת מתאמים בין שעות תרגול, ציון והיעדרויות מאפשרת לראות במהירות אילו זוגות מאפיינים קשורים חיובית או שלילית. כדי שהצבע לא יטעה, הסקאלה חייבת להיות מפורשת ואחידה. במתאם פירסון היא בדרך כלל נעה בין −1 ל־1, עם נקודת אמצע אפס.

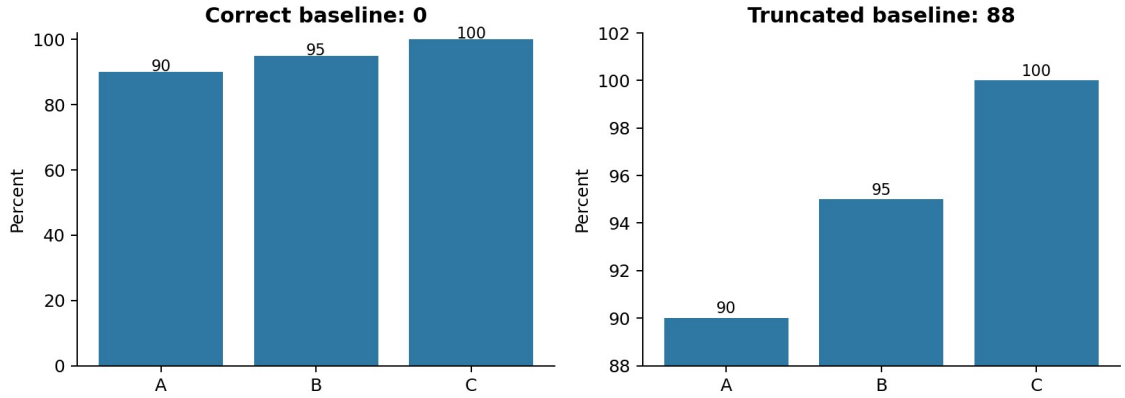


איור: מפת מתאמים להמחשה; הערכים מומצאים ואינם תוצאה של מחקר על תלמידים אמיתיים.

נניח שבתא המפגש של שעות תרגול וציון מופיע 0.58. זו אינדיקציה לקשר ליניארי חיובי בינוני בנתונים המוצגים; היא אינה אומרת ש־58% מהתלמידים יצליחו או שהתרגול מסביר 58% מן השונות. חשוב להיזהר במיוחד מאותה טעות בעת קריאת מדדי AI. גם ערכי האלכסון 1.00 אינם תגלית: כל משתנה מתואם באופן מושלם עם עצמו.

10. הטעיה חזותית — כאשר ציר משתנה, הסיפור משתנה

קנה המידה בגרף משפיע על הרושם שנוצר. נניח ששלוש כיתות הצליחו במשימה בשיעורים של 90%, 95% ו־100%. כאשר ציר העמודות מתחיל מאפס, רואים שלוש הצלחות גבוהות יחסית. כאשר אותו ציר מתחיל ב־88%, הפער בין העמודות נראה גדול מאוד — אף שההפרשים נשארו 5 ו־10 נקודות אחוז בלבד.

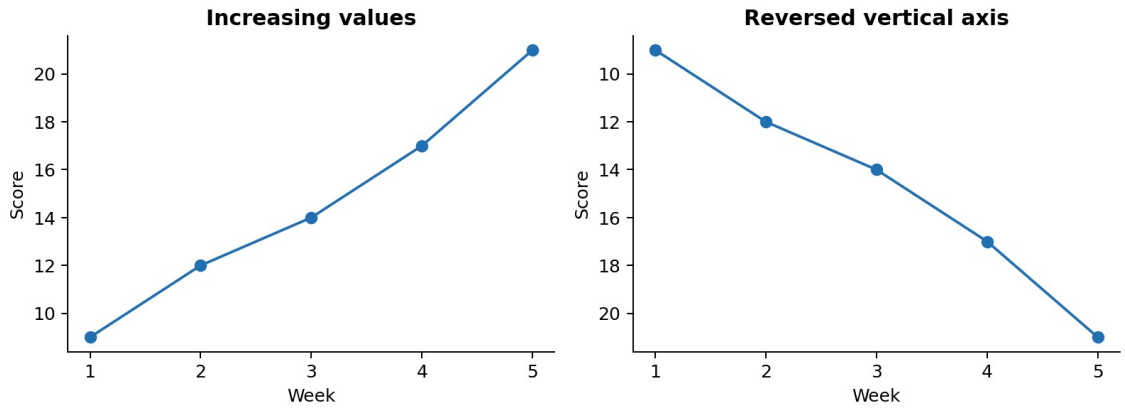


איור: אותם נתונים מוצגים בשני סולמות. ציור העמודות הימני מדגיש הפרשים קטנים באמצעות קיצוץ ציר.

הבעיה אינה שהמספרים לא נכונים. כל אחת משלוש העמודות עדיין מתוארת בערך הנכון. הבעיה היא שהייצוג עלול לגרום לקורא להגזים במשמעות הפער. יש להבחין גם בין שינוי של 5 נקודות אחוז לבין עלייה יחסית של 5%. למשל, מעבר מ־90% ל־95% הוא עלייה של 5 נקודות אחוז, אך העלייה היחסית היא כ־5.56% מהערך המקורי.

$$\text{Percentage points: } 95 - 90 = 5$$

$$\text{Relative change: } (95 - 90) / 90 \times 100 \approx 5.56\%$$



איור: היפוך ציר אנכי יכול לגרום לעלייה מספרית להיראות כירידה — תמיד בודקים את כיוון הציר.

שאלה בסגנון בגרות — חשיבה ביקורתית

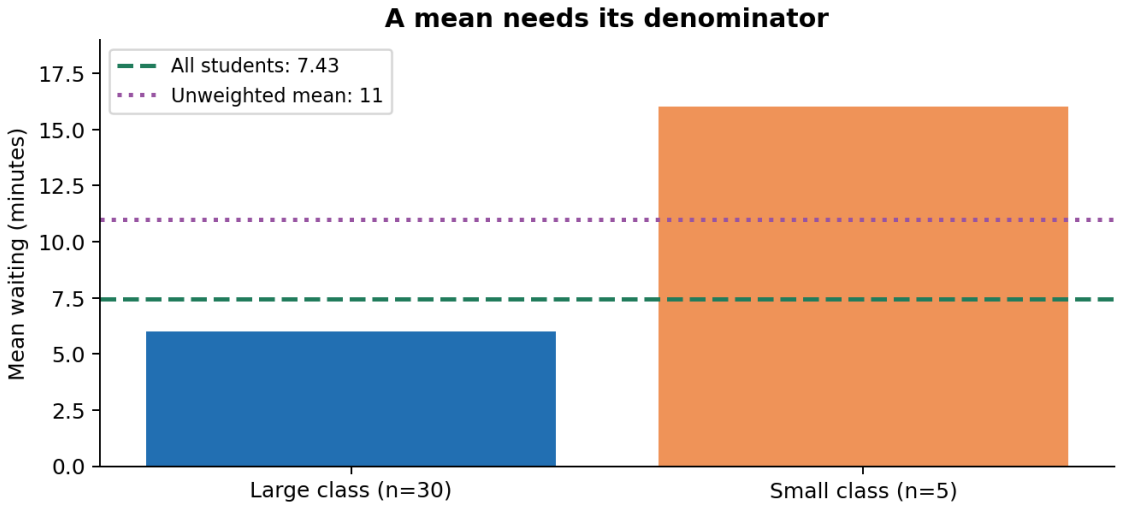
נתון גרף עמודות של שיעורי הצלחה 88%, 92%, 94%, שהציר האנכי שלו מתחיל ב־85%. הציעו שני תיקונים לעיצוב, ונסחו מסקנה שלא תטעה את הקורא.

11. חישובי קבוצות — ממוצע של ממוצעים עלול להטעות

דודו משווה בין שתי כיתות. בכיתה א׳ 30 תלמידים וזמן ההמתנה הממוצע הוא 6 דקות. בכיתה ב׳ 5 תלמידים וזמן ההמתנה הממוצע הוא 16 דקות. אם נחשב ממוצע של שני הממוצעים, נקבל 11 דקות. אולם זה אינו הממוצע של 35 התלמידים: הוא מעניק משקל שווה לכיתה בת 30 תלמידים ולכיתה בת 5 תלמידים.

$$\text{Unweighted mean of group means} = (6 + 16) / 2 = 11$$

$$\text{Weighted overall mean} = (30\times6 + 5\times16) / (30 + 5) = 260/35 \approx 7.43$$



איור: כאשר הקבוצות שונות בגודלן, יש לשקלל את גודל הקבוצה בחישוב הממוצע הכולל.

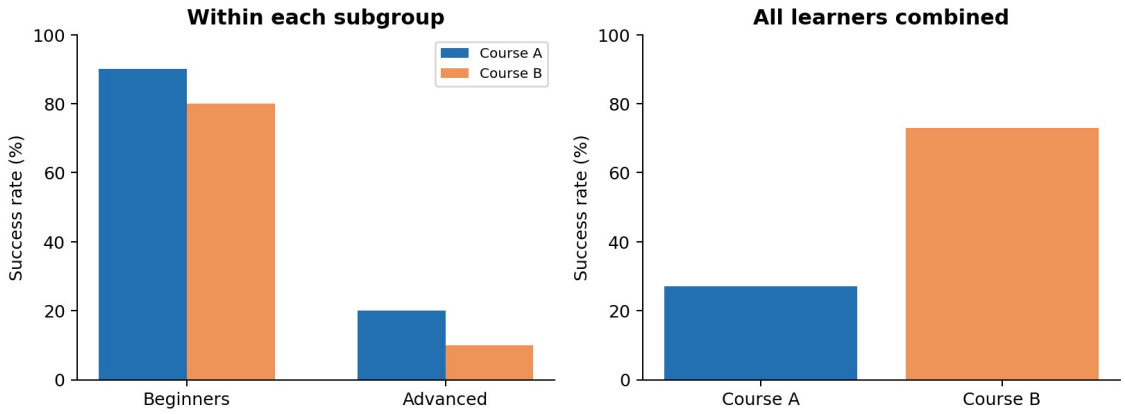
בדוחות AI עלולים להופיע ממוצעי דיוק לפי קבוצות, מדינות, ימים או סוגי משתמשים. לפני שמאחדים אותם יש לברר את המכנה ואת המטרה: האם אנו רוצים לתת משקל שווה לכל אדם, לכל קבוצה או לכל ניסוי? כל בחירה עונה על שאלה אחרת, ולכן חייבים לציין אותה במפורש.

12. פרדוקס סימפסון — מדוע מסקנה על כלל הנתונים עלולה להתהפך?

נניח שמשווים שני קורסים, A ו־B, עבור תלמידים מתחילים ומתקדמים. בקבוצת המתחילים קורס A מצליח ב־9 מתוך 10 (90%), ואילו B ב־72 מתוך 90 (80%). בקרב מתקדמים קורס A מצליח ב־18 מתוך 90 (20%) ו־B ב־1 מתוך 10 (10%). בתוך כל אחת משתי הקבוצות שיעור ההצלחה של A גבוה מזה של B.

אולם בחיבור כלל התלמידים מתקבלת תמונה הפוכה: ל־A יש 27 הצלחות מתוך 100 (27%), ול־B יש 73 הצלחות מתוך 100 (73%). אין כאן טעות בחשבון. ההבדל נובע מכך שמספר המתחילים והמתקדמים שונה מאוד בין הקורסים. מכאן נלמד שגרף כולל יכול להסתיר מאפיין רקע בעל השפעה חזקה.

קבוצה	קורס **A**	קורס **B**
מתחילים	9 מתוך 10 — 90%	72 מתוך 90 — 80%
מתקדמים	18 מתוך 90 — 20%	1 מתוך 10 — 10%
הכול יחד	27 מתוך 100 — 27%	73 מתוך 100 — 73%

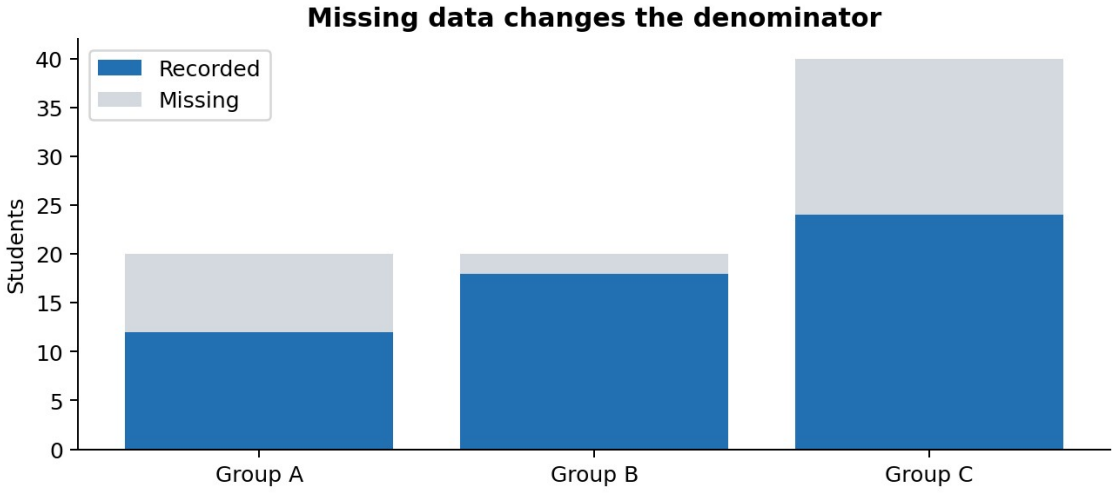


איור: השוואה בתוך קבוצות לעומת השוואה כוללת; ערבוב של קבוצות בגדלים שונים משנה את המסר.

זהו תרחיש מלאכותי שנועד ללמד חשיבה ביקורתית. לפני שנמליץ על קורס או מודל, עלינו להבין מי נכלל בכל קבוצה ומדוע. לא די להשוות אחוז כולל כאשר הרכב האוכלוסיות שונה. בפרויקטים מתקדמים זו הסיבה לבדיקת ביצועי מודל לפי תתי־קבוצות רלוונטיות, ולא רק לפי ממוצע כללי אחד.

13. נתונים חסרים — מי אינו מופיע בגרף?

חלק מהגרפים מציגים רק את הרשומות שעבורן יש נתון. נניח שבקבוצה א׳ נרשמו ציונים ל־12 מתוך 20 תלמידים, ובקבוצה ב׳ ל־18 מתוך 20. אם משווים ממוצעי ציונים בלי לציין את שיעור החסרים, אפשר ליצור רושם שגוי. ייתכן שהתלמידים שחסר עבורם מידע שונים מן האחרים באופן שיטתי. אין לשער אוטומטית שהמידע החסר הוא אפס או שהחוסר אקראי.



איור: מספר הרשומות שנצפו לעומת הרשומות החסרות; הפער יכול לשנות את אמינות המסקנה.

$$\text{Completeness (\%)} = \text{known values} / \text{expected values} \times 100$$

$$\text{Group A: } 12 / 20 \times 100 = 60\% \text{ ; Group B: } 18 / 20 \times 100 = 90\%$$

בפרויקט ML, בדיקת חסרים היא חלק מחקר הנתונים. ההחלטה כיצד להשלים או להסיר ערכים צריכה להתבצע ללא שימוש בלתי הוגן במידע ממבחן הבדיקה. כאשר מחליטים על אסטרטגיית השלמה, מתאימים אותה על נתוני האימון ומיישמים על נתוני הבדיקה את אותם כללים.

14. מתי גרף מספר את כל הסיפור, ומתי צריך מידע נוסף?

תרשים הוא תצוגה חלקית של נתונים. עמודות יכולות להראות את ממוצע הציונים של שתי כיתות, אבל לא את הפיזור בתוך כל כיתה. תרשים קופסה מציג חציון ורבעונים, אך אינו מגלה תמיד פיזור דו־שיאי. תרשים פיזור מראה יחסים בין שני משתנים, אבל אינו מפריד באופן אוטומטי בין קשר לסיבתיות. לכן לעיתים נצטרך להציג יחד שני גרפים משלימים: היסטוגרמה לצד קופסה, או פיזור לצד טבלת מקדמי מתאם.

חשוב גם לבדוק מהו מספר התצפיות. הבדל גדול למראה בין שתי קבוצות עם שלוש תצפיות בכל אחת מחייב זהירות רבה יותר מהבדל דומה בין מדגמים גדולים ואיכותיים. אי־ודאות יכולה להיות מוצגת באמצעות רווחי סמך או פיזור נקודות, אך יש להסביר מה חושב בפועל; אין להוסיף פסי שגיאה דקורטיביים ללא משמעות סטטיסטית.

15. מן הנתונים אל הגרף ב־Pandas וב־Matplotlib

Python מאפשר לשחזר את התהליך באופן שקוף: מתחילים מטבלה, בודקים את סוגי העמודות ואת יחידות המדידה, מסכמים לפי השאלה ורק אחר כך מציירים. חשוב לא להניח שעמודה הכוללת ספרות מייצגת בהכרח כמות. מספר טלפון, מזהה תלמיד או מיקוד הם מזהים, לא מדידות שאותן נכון למצע.

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.DataFrame({
    "club": ["Music", "Sports", "Music", "Art", "Sports", "Coding"],
    "wait_min": [5, 11, 6, 8, 12, 7]
})

print(df.info())
print(df["club"].value_counts())
print(df["wait_min"].describe())

counts = df["club"].value_counts()
counts.plot(kind="bar")
plt.ylabel("Students")
plt.tight_layout()
plt.show()
```

שימו לב: הפקודה value_counts סופרת תדירויות של קטגוריות, ואילו describe מסכמת משתנה כמותי. החלפתן ללא מחשבה על סוג המשתנה עלולה להוביל לפרשנות שגויה. כמו כן, אין די בהרצת הקוד. בכל תרשים על התלמיד לנסח במילים: מה מציג הגרף, מהי המסקנה הנתמכת, ומהו מידע שעדיין חסר כדי להגיע להחלטה.

16. כיצד בוחרים גרף? עץ החלטה מחשבתי

לפני יצירת כל תרשים שואלים מהו התוצר הרצוי. אם רוצים להשוות קטגוריות — נבחר בדרך כלל תרשים עמודות. אם רוצים לזהות את צורת ההתפלגות של משתנה כמותי — נבחר היסטוגרמה, תרשים קופסה או שניהם. אם רוצים לראות שינוי לאורך זמן — תרשים קו. אם רוצים לבדוק קשר בין שתי מדידות — תרשים פיזור. אם רוצים להשוות מתאמים רבים — מטריצת מתאמים ומפת חום. לאחר הבחירה יש לוודא שכותרות, יחידות וגדלי מדגם מופיעים בצורה שקופה.

אין כלל שמכריח לבחור גרף אחד בלבד. בפרויקט גמר טוב נציג מספר מצומצם של גרפים משלימים, כאשר לכל אחד תפקיד מחקרי. שלושה גרפים שמסבירים את הבעיה חשובים יותר מעשרים גרפים שנוצרו רק משום שקיימת פונקציה שמציירת אותם.

17. תרגול מודרך — מצאו את המסר ואת הטעות

תרגיל 1: מספרים לעומת שיעורים

שתי כיתות משתתפות בפרויקט: בכיתה א׳ 24 תלמידים ומתוכם 12 השלימו אותו; בכיתה ב׳ 15 ומתוכם 10 השלימו. חשבו לכל כיתה את שיעור ההשלמה, בחרו גרף הוגן להשוואת השיעורים ונסחו מסקנה אחת שונה ממסקנה המבוססת רק על מספרי המשלימים.

תרגיל 2: היסטוגרמה ורוחב מחלקה

ניתנו זמני המתנה: 3, 3, 4, 5, 5, 6, 7, 8, 8, 9, 10, 12, 15, 20. הציעו שתי חלוקות למחלקות, וכתבו האם ניתן לקבוע שיש שני שיאים באופן חד־משמעי.

תרגיל 3: קופסה וחריגים

עבור 2, 4, 4, 5, 6, 8, 10, 30 חשבו חציון, רבעונים בשיטת חצאי המדגם, IQR וגבולות חריגים. האם 30 מסומן כחריג לפי כלל 1.5×IQR? הסבירו מדוע לא מוחקים אותו אוטומטית.

תרגיל 4: מתאם ללא סיבתיות

תרשים פיזור מראה קשר חיובי בין מספר ספרים בבית לציון בבחינה. כתבו שתי אפשרויות למשתנה נוסף שעשוי להשפיע על הקשר, והציעו בדיקה שתעזור לברר את הנושא.

תרגיל 5: סיכום קבוצות

בקבוצה א׳ 40 לקוחות והממוצע הוא 3 פניות; בקבוצה ב׳ 10 לקוחות והממוצע הוא 11 פניות. חשבו ממוצע משוקלל והשוו לממוצע הפשוט של שני הממוצעים. איזה מבין השניים מתאים לשאלה: כמה פניות היו לכל לקוח בממוצע?

18. מעבדה מסכמת — חקר לוח בקרה בית־ספרי

במחברת Google Colab המצורפת תעבדו על מאגר מומצא של תלמידים. כל רשומה מייצגת תלמיד אחד וכוללת חוג, זמן המתנה, מספר שעות תרגול וציון. תחילה תזהו את סוגי המשתנים, תחשבו שכיחויות וממוצעים, תבחרו גרפים מתאימים ותתעדו את המסקנות. בהמשך תבדקו כיצד שינוי הסולם, בחירת רוחב מחלקות וערכים חסרים משנים את ההצגה.

אין להסתפק בהעתקת דוגמה מוכנה. בכל משימה כתבו לפני הקוד השערה קצרה, הריצו את הניסוי, ואז תארו במילים האם התוצאה תמכה בהשערה ומהי מגבלת הנתונים. חלק מהמשימות ידרשו לכתוב קוד; אחרות יתמקדו דווקא בפרשנות ובביקורת של הגרף.

משימה	מוצר נדרש	מיומנות
A	תרשים עמודות ושורת מסקנה	קטגוריות ושכיחות
B	היסטוגרמות עם שני רוחבי מחלקות	קריאת התפלגות
C	תרשים קופסה וזיהוי חריגים	פיזור ורבעונים
D	תרשים פיזור	קשר בין שני משתנים
E	השוואה בין שתי קבוצות	ספירה מול שיעור
F	ביקורת גרף מטעה	ציר וסולם
G	בדיקת חסרים	איכות נתונים
H	סיכום חקר קצר	טענה, ראיה, מגבלה

19. שאלות מקוריות בסגנון בגרות

השאלות שלהלן נכתבו במיוחד לפרק ומדמות דרישות של חישוב, ניתוח נתונים והסקת מסקנות. אלה אינן שאלות שהועתקו מבחינות בגרות רשמיות.

שאלה 1 — בחירת ייצוג (12 נקודות)

חברת משלוחים שומרת לכל משלוח סוג רכב, זמן הגעה, מרחק נסיעה ותאריך. א. ציינו סוג של כל משתנה. ב. בחרו גרף להערכת פיזור זמן ההגעה. ג. בחרו גרף לבדיקת הקשר בין מרחק לזמן. ד. הסבירו מדוע תרשים עוגה אינו מתאים לבדיקת הקשר בין מרחק לזמן.

שאלה 2 — חישוב ופרשנות (16 נקודות)

בקבוצה א׳ נצפו 8 זמני המתנה בממוצע 5 דקות, ובקבוצה ב׳ נצפו 12 זמני המתנה בממוצע 10 דקות. א. חשבו את הממוצע של כלל התצפיות. ב. תלמיד חישב 7.5 דקות. הסבירו את הטעות. ג. הציעו שני גרפים משלימים שיאפשרו להעריך גם את הפיזור בתוך הקבוצות, ולא רק ממוצעים.

שאלה 3 — גרף מטעה (14 נקודות)

דוח מציג שיעורי דיוק של מודלים A ו־B: 93% ו־95%. הציר האנכי מתחיל ב־92%. א. הסבירו כיצד סולם הציר משפיע על הרושם. ב. חשבו הפרש בנקודות אחוז. ג. הסבירו מדוע אי אפשר לקבוע איזה מודל עדיף בלי לדעת על אילו נתונים נבדקו המודלים ומהן הטעויות שהם מבצעים.

שאלה 4 — נתונים חסרים (18 נקודות)

בדוח בית־ספרי מוצג ממוצע ציונים של תלמידים, אך ל־25% מהם חסר ציון. א. הסבירו מדוע הממוצע של הרשומות הקיימות עלול להטעות. ב. הציעו תרשים שמראה את שיעור החסרים בכל כיתה. ג. תארו כיצד תתעדו את מגבלת הנתונים בדוח מחקר.

20. גשר לפרויקט הגמר בלמידת מכונה וללמידה עמוקה

בחקר נתונים לקראת מודל AI יש להשתמש בגרפים כדי להבין את מאגר הנתונים ולגלות בעיות לפני האימון. עם זאת, רצוי להבחין בין נתוני אימון לבין נתונים שהוחזקו לבדיקת המודל. תהליך הכנה או בחירת מאפיינים שנקבע בעזרת תוצאות הבדיקה עלול ליצור אופטימיות מופרזת בהערכת הביצועים. יש לתעד מה נבדק בכל שלב ולהשתמש בנתוני האימון לצורך התאמת טרנספורמציות ומודלים.

בפרויקט לסיווג תמונות נרצה לדעת כמה דוגמאות יש לכל מחלקה, האם יש מחלקה קטנה במיוחד ומהי איכות התמונות. בפרויקט לחיזוי מחירי בתים נבחן התפלגות מחירים, קשרים בין מאפיינים, ערכים חריגים וחסרים. בפרויקט רשת עצבית נצייר בהמשך גרפי Loss ו־Accuracy לפי Epoch ונבחן האם שגיאת האימון ממשיכה לרדת כאשר שגיאת הוולידציה כבר עולה. כך קריאת גרפים הופכת לכלי מקצועי להבנת אימון מודלים, ולא רק לתרגיל בעיצוב.

דוח חקר איכותי יכיל לכל גרף את השאלה, מקור הנתונים, מספר התצפיות, הצירים והיחידות, ממצא נתמך וגבול פרשנות. אין די במשפט ״רואים קשר חזק״. יש לומר בין אילו משתנים, באיזה מדגם, מהו סוג הקשר ומה עוד נדרש לבדוק לפני קבלת החלטה.

21. מילון מושגים מסכם

מושג	הסבר לתלמיד
Bar chart	השוואה בין קבוצות או קטגוריות באמצעות עמודות.

מושג	הסבר לתלמיד
Histogram	תדירות או צפיפות של ערכים מספריים לפי מחלקות.
Boxplot	סיכום חזותי של חציון, רבעונים ופיזור.
Scatter	נקודה לכל תצפית עם שני משתנים כמותיים.
Line chart	הצגת מגמה לאורך ציר בעל סדר משמעותי, בעיקר זמן.
Heatmap	מטריצת ערכים שמיוצגת באמצעות צבע וסולם.
Bin	קטע ערכים בהיסטוגרמה; רוחב המחלקות משפיע על הצורה.
IQR	הטווח שבין הרבעון הראשון לשלישי.
Missing values	ערכים שלא נמדדו או לא נרשמו; אינם בהכרח אפס.
Weighted mean	ממוצע המעניק לכל קבוצה משקל בהתאם לגודלה או למשקל מוגדר.
Data leakage	שימוש במידע לא מתאים משלב הבדיקה בזמן הכנת המודל או התאמתו.
Simpson’s paradox	מצב שבו קשר בתוך קבוצות יכול להתהפך אחרי איחוד קבוצות בהרכבים שונים.

22. בדיקת שליטה לפני מעבר לפרק הבא

סמנו לעצמכם האם אתם מסוגלים לבצע כל אחת מן הפעולות בלי להיעזר בפתרון. אם אתם מתקשים בפעולה מסוימת, חזרו לחלק המתאים ונסו תרגיל חדש: לבחור גרף לפי סוג הנתונים; להסביר מדוע עמודות אינן היסטוגרמה; לחשב שכיחות יחסית; לפרש חציון ורבעונים בקופסה; לקרוא גרף עם ציר מקוצץ; לחשב ממוצע משוקלל; לזהות חסרים; ולהבחין בין תיאור גרפי לבין הוכחת קשר סיבתי.

כרטיס יציאה: בחרו גרף אחד שיצרתם. כתבו שלוש שורות נפרדות: ״הנתונים מראים...״, ״אפשר לשער ש...״, ״כדי לבדוק זאת יש צורך ב...״. ההפרדה הזאת תלווה אתכם בהמשך הספר, בבחינה העיונית ובהצגת פרויקט הגמר.

23. מקורות והמשך

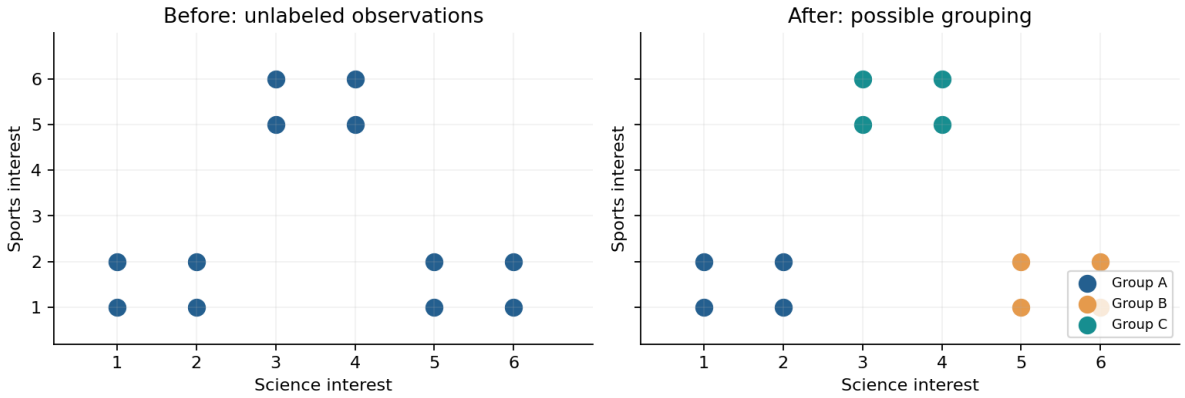
עוגן תכנית הלימודים: יחידה 5 — נושאי ליבה בבינה מלאכותית, סעיף 2.5 ״קריאת גרפים״, לפי מפת הפרקים השמורה במאגר הקורס. החומר המורחב כאן, כולל הדוגמאות, האיורים והשאלות, הוא התאמה פדגוגית מקורית. יש לאמת את היקף חובת הבחינה מול נוסח התכנית הרשמית התקפה.

מקורות להעמקה ולמתודולוגיה: An Introduction to Statistical Learning with Applications in Python (James, Witten, Hastie, Tibshirani & Taylor); The StatQuest Illustrated Guide to Machine Learning (Josh Starmer); Hands-On Machine Learning (Aurélien Géron); Mathematics for Machine Learning (Deisenroth, Faisal & Ong). אלה מקורות רעיוניים והוראתיים; הדוגמאות והאיורים בחוברת נוצרו במיוחד לספר.

הפרק הבא: שער 3 — מבוא ללמידה לא מונחית ולקיבוץ נתונים (Clustering). נשתמש בידע על התפלגויות, מרחקים וייצוגים חזותיים כדי להבין כיצד אלגוריתם מגלה קבוצות בלי לקבל תוויות מוכנות.

פרק 3.1 — מבוא ללמידה לא מונחית ולקיבוץ נתונים

איך מחשב מזהה דפוסים וקבוצות כאשר איש אינו מוסר לו מראש את התשובה הנכונה?



איור: אותן 12 תצפיות לפני קיבוץ ואחרי הצעה לחלוקה אפשרית. הצבעים בצד ימין הם פירוש שנוסף לאחר ניתוח הנתונים — לא תוויות אמת.

חוברת תלמיד עצמאית | חיבור לפרקים 1.1–2.5 | הכנה לפרק 3.2: K-Means

מטרת הפרק אינה רק להכיר את המילה Clustering, אלא להבין איזו שאלה אפשר לשאול על נתונים ללא תוויות, מתי חלוקה לקבוצות מועילה, מדוע היא תלויה בבחירת מאפיינים ומרחקים, ומה לא ניתן להסיק מקבוצה שנמצאה. נלמד באמצעות סיפור מעולם בית הספר, תרשימים, חישובים ידניים, Python ותרגול עצמאי.

מפת הפרק: מטרות, ידע קודם ותוצרי למידה

לפני פרק זה למדנו לייצג רשומה כווקטור, למדוד מרחק, לחשב סטטיסטיקה ולקרוא תרשימים. כעת נשתמש בכל אלה כדי לבחון שאלה חדשה: כאשר אין עמודת תשובה ידועה, האם אפשר לגלות בתוך הנתונים מבנה שחוזר על עצמו? התשובה היא שלעתים כן, אך יש צורך בבחירת מאפיינים, בהגדרת דמיון ובהערכה ביקורתית של מה שהתגלה.

- בסיום הפרק התלמיד יסביר, במילים שלו, מהי למידה לא מונחית וכיצד היא שונה מלמידה מונחית ומחלוקה ידנית לפי כלל.
- התלמיד יזהה תצפיות ומאפיינים, יבנה מטריצת X ללא עמודת יעד y, ויציג נתונים בגרף דו־ממדי.
- התלמיד יחשב מרחק בין שתי דוגמאות, ינמק כיצד שינוי קנה מידה או מאפיין עשוי לשנות קבוצות, וידגים זאת ב־Python.
- התלמיד יזהה דפוסים, רצפים, נתונים חריגים ומקרים שבהם אין הצדקה להניח שיש קבוצות נפרדות.
- התלמיד ינסח שאלת חקר לפרויקט קיבוץ, יציע בדיקות איכות ויזהיר מפני מסקנות לא מבוססות על אנשים.

משמעות בפרק החדש	ידע קודם
כל שורה היא תצפית; המרחק נמדד בין שורות.	וקטור ומטריצת מאפיינים
הגדרת ״דומות״ עשויה להשתנות לפי מדד.	L1, L2 ודמיון קוסינוס
הבנת מבנה ופיזור של התצפיות.	ממוצע, פיזור וחריגים
מניעת השתלטות של מאפיין בעל טווח מספרי גדול.	תקנון ונרמול
בדיקה חזותית ביקורתית לפני בחירת אלגוריתם.	תרשימי פיזור ו־EDA

1. סיפור פתיחה — דודו רוצה להציע חוגים לתלמידים

דודו מסייע לרכזת החוגים בבית הספר. בעבר בית הספר שלח לכל התלמידים אותו שאלון והציע להם את אותם החוגים. השנה הרכזת רוצה להבין האם קיימים דפוסי עניין שונים, כדי לתכנן פעילויות מגוונות. היא אינה יודעת מראש לכמה קבוצות יחולקו התלמידים, ואינה רוצה לקבוע שתלמיד מסוים ״שייך לקבוצת המדע״ רק על סמך ציון יחיד. במקום זאת היא אוספת שתי תשובות: עד כמה כל תלמיד מתעניין במדע ועד כמה הוא מתעניין בספורט — בסולם מ־1 עד 6.

שימו לב להבדל: בטבלה אין עמודה בשם ״סוג תלמיד״ או ״חוג נכון״. אלו שאלות שטרם נענו. כל תלמיד מיוצג באמצעות זוג מספרים בלבד. המחשב יכול לבדוק מי נמצא קרוב למי במישור, אך הוא לא מקבל תווית נכונה שאפשר להשוות אליה את ניחושיו.

מספר תלמיד	עניין במדע	עניין בספורט
1	1	1
2	1	2
3	2	1
4	2	2
5	5	1
6	5	2
7	6	1
8	6	2
9	3	5
10	3	6
11	4	5
12	4	6

הנתונים בדוגמה מומצאים לצורכי הוראה. הם אינם נתונים אמיתיים של תלמידים, ואינם משמשים לקביעת יכולות או אישיות. בהמשך נראה מדוע הקביעה הזו חשובה: אלגוריתם עשוי לקבץ אנשים לפי שני מדדים בלבד, אבל לא ניתן להסיק מכך מי הם או מה מתאים להם באופן מוחלט.

2. מהן תצפית, מאפיין, מאגר נתונים ומטריצת X?

תצפית (Observation) היא רשומה אחת המתארת מקרה שנחקר. בסיפור שלנו זו שורה של תלמיד אחד. מאפיין (Feature) הוא תכונה שנמדדת עבור אותה תצפית, למשל רמת עניין במדע. המאגר (Dataset) הוא אוסף כל התצפיות. כאשר מסדרים את המאפיינים המספריים בטבלה, אפשר לחשוב עליהם כמטריצה X, שבה כל שורה היא תצפית וכל עמודה היא מאפיין.

$$X = [\,[1,1], [1,2], [2,1], \ldots, [4,6]\,] \quad \text{shape}(X) = (12, 2)$$

ממדי המטריצה הם 12×2: יש 12 תלמידים ושני מאפיינים. אין לבלבל בין מספר התלמידים לבין ממד הווקטור. כל תלמיד הוא וקטור דו־ממדי. אם נוסיף מאפיין שלישי, למשל עניין במוזיקה, וקטור התלמיד יהיה תלת־ממדי, ומטריצת X תהיה בגודל 12×3. המרחק בין תצפיות עשוי להשתנות משום שמרחב המאפיינים השתנה.

```
import numpy as np
X = np.array([[1,1],[1,2],[2,1],[2,2],
              [5,1],[5,2],[6,1],[6,2],
              [3,5],[3,6],[4,5],[4,6]])
print(X.shape)      # 12 rows, 2 features
print(X[0])        # first observation
```

בדיקת הבנה 1

אם נוסיף עוד שמונה תלמידים אך לא נשנה את מספר השאלות בשאלון, מה יהיה גודל מטריצת X? אם במקום זאת נוסיף שתי שאלות חדשות לכל 12 התלמידים, מה יהיה גודלה? נמקו במילים ולא רק במספרים.

3. למידה מונחית לעומת למידה לא מונחית

בלמידה מונחית (Supervised Learning) יש נתוני קלט ולרוב גם תשובה רצויה עבור כל דוגמת אימון. במערכת לחיזוי ציון, למשל, נרשום שעות למידה ולצדן את הציון שנמדד בפועל. המודל לומד קשר בין התכונות לבין משתנה המטרה, ואז נבדק אם הוא מסוגל לחזות תשובות חדשות. בלמידה לא מונחית (Unsupervised Learning) אין במערך האימון תוויות יעד ידועות לסוג המשימה שאנו מבצעים. האלגוריתם מחפש תבניות, מבנה, דמיון או ייצוג מועיל בתוך X בלבד.

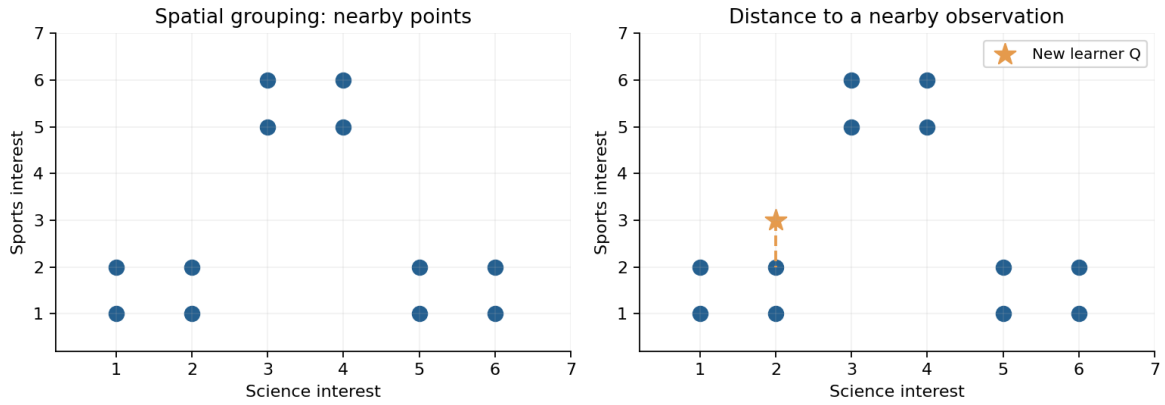
אין פירוש הדבר שהמחשב ״מלמד את עצמו ללא כל הדרכה״. האדם בוחר מה למדוד, כיצד להכין את הנתונים, באיזה אלגוריתם להשתמש, איזה מרחק משמעותי ומהי מטרת הניתוח. בהיעדר תשובות אמת ידועות קשה לעתים להחליט אם החלוקה שנמצאה טובה. זו אחת הסיבות החשובות ללימוד חשיבה ביקורתית לצד הנוסחאות.

למידה לא מונחית	למידה מונחית	שאלה
בדרך כלל X ללא y למשימת הקיבוץ	X וגם y באימון	מה נתון?
לגלות מבנה, דמיון או קבוצות	לחזות ערך או מחלקה ידועה	מה רוצים?
גילוי דפוסים בסוגי הודעות	חיזוי אם הודעה היא ספאם	דוגמה
בדיקות מבנה, יציבות ופירוש מקצועי	השוואה לתוויות אמת ב־Test	איך בודקים?

קיימות גם שיטות נוספות, כמו למידה חצי־מונחית, למידה עצמית (Self-supervised) ולמידת חיזוק. הן אינן זהות ללמידה לא מונחית, ונזכיר אותן רק כדי למנוע בלבול בשמות. בפרק זה נתמקד בקיבוץ תצפיות ללא תווית יעד מוכנה.

4. אז מהו קיבוץ נתונים — Clustering?

קיבוץ נתונים הוא ניסיון לחלק תצפיות לקבוצות כך שבתוך כל קבוצה יהיה דמיון מסוים, ובין קבוצות יהיו הבדלים שניתן להסביר. זו הגדרה אינטואיטיבית, לא כלל יחיד שמתאים לכל האלגוריתמים. מה שנחשב לקבוצה טובה תלוי בשאלה, בבחירת המאפיינים, באופן מדידת הדמיון ובצורת הנתונים. לעתים קבוצה תהיה ענן צפוף של נקודות, ולעתים היא תהיה רצועה מעוקלת או אזור בעל צפיפות גבוהה.



איור: שמאל: הנקודות במרחב המאפיינים. ימין: תצפית חדשה Q ושכנה סמוכה; קרבה גיאומטרית היא כלי ניתוח, לא תשובה מוכנה.

נסתכל על הגרף: אפשר לזהות תצפיות בעלות עניין נמוך יחסית בשני התחומים, תצפיות עם עניין גבוה במדע ונמוך בספורט, ותצפיות עם עניין גבוה בספורט. למרות המראה המסודר, זו פרשנות מסוימת של שתי תכונות. תלמיד עם עניין מדעי 3 ועניין בספורט 4 היה עשוי להיות בין אזורי הקיבוץ, וההחלטה לגביו הייתה פחות ברורה.

חשוב מאוד: קבוצה שמקבלת את המספר 0 אינה ״קבוצה 0״ בעלת משמעות טבעית. מספרי הקבוצות הם מזהים טכניים שנבחרו על ידי התוכנה. אלגוריתם אחר עשוי לקרוא לאותה קבוצה 2. רק לאחר ניתוח הנתונים נוכל להציע שם תיאורי, למשל ״עניין גבוה בספורט״, וגם אז יש להבהיר שזה תיאור על סמך המאפיינים שנמדדו בלבד.

5. מדוע קרבה בין נקודות חשובה — ומתי היא מטעה?

בפרק 1.2 למדנו למדוד מרחק בין וקטורים. נשתמש באותו רעיון, לא כאלגוריתם קיבוץ מלא אלא ככלי להבנת הדמיון בין שתי תצפיות. עבור וקטורים דו־ממדיים $A=(a_1,a_2)$ ו־$B=(b_1,b_2)$, המרחק האוקלידי מחושב בעזרת הפרשים בריבוע ושורש. אם שני המאפיינים נמדדים באותה יחידה ובטווח דומה, מרחק קטן עשוי להעיד על דפוס עניין דומה. אם היחידות אינן שוות, יש לבדוק תחילה את סולם המדידה.

$$d_2(A,B) = \sqrt{[(a_1 - b_1)^2 + (a_2 - b_2)^2]}$$

נחשב את המרחק בין התלמיד $P=(1,2)$ לבין התלמיד $R=(2,2)$. במאפיין המדע ההפרש הוא יחידה אחת; במאפיין הספורט אין הפרש. לכן המרחק הוא 1. כעת נחשב עבור $Q=(3,5)$: ההפרש במדע הוא 2 ובספורט 3, ולכן המרחק הוא $\sqrt{13}$, כלומר בקירוב 3.606. המספרים מראים ש־R קרוב יותר ל־P מאשר Q, אך אינם מוכיחים ששני התלמידים יהיו באותה קבוצה בכל אלגוריתם.

$$d_2(P,R) = \sqrt{[(1-2)^2 + (2-2)^2]} = 1$$

$$d_2(P,Q) = \sqrt{[(1-3)^2 + (2-5)^2]} = \sqrt{13} \approx 3.606$$

מרחק מנהטן מחבר את הערכים המוחלטים של הפרשי הרכיבים. בדוגמה השנייה הוא יהיה 3+2=5. לכן שני מדדי מרחק יכולים להחזיר מספרים שונים ואף לשנות את סדר הקרבה בין כמה תצפיות. בחירת מדד המרחק היא החלטה של מי שבונה את מערכת ה־AI, ולא עובדה שהנתונים קובעים לבדם.

$$d_1(P,Q) = |1-3| + |2-5| = 5$$

```
import numpy as np
P = np.array([1., 2.]); R = np.array([2., 2.])
Q = np.array([3., 5.])
print(np.linalg.norm(P - R))
print(np.linalg.norm(P - Q))
print(np.abs(P - Q).sum())
```

בדיקת הבנה 2

שני תלמידים קיבלו אותו ציון במדע אך בציוני ספורט שונים מאוד. האם הם אמורים להיחשב דומים? התייחסו לשאלת המחקר: התאמה לחוג מדע לעומת התאמה לחוג שמשלב מדע וספורט.

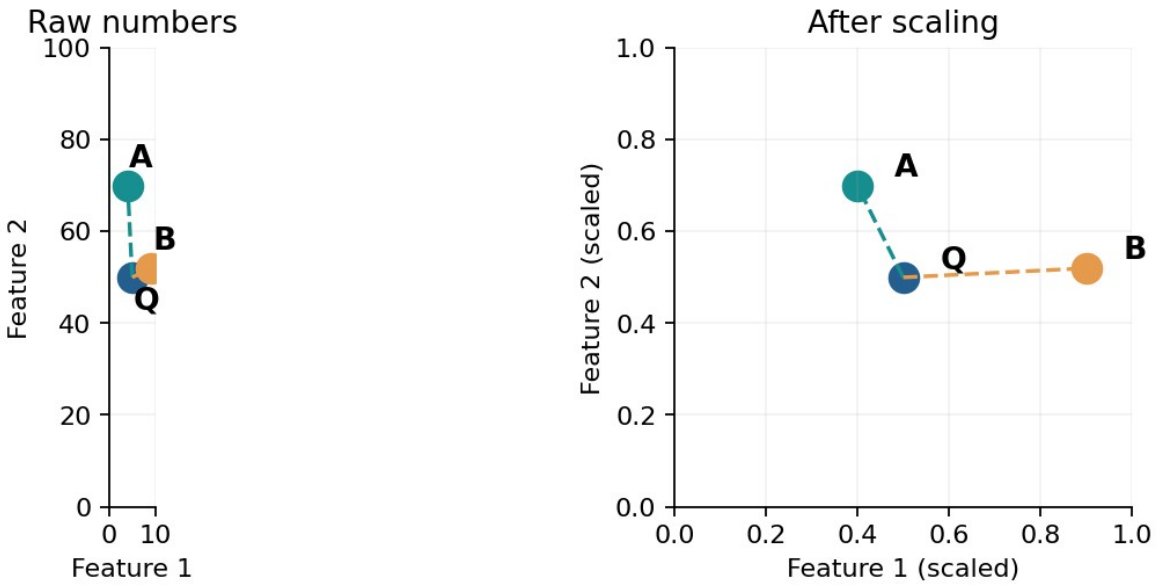
6. קנה המידה של המאפיינים עשוי לשנות את התוצאה

נניח שבפרויקט אחר אנו משווים שני מאפיינים: מספר משימות שהושלמו, בין 0 ל־10, וזמן פעילות במערכת, בין 0 ל־100 דקות. אם נחבר את הפרשי שני המאפיינים ישירות, זמן הפעילות עשוי לשלוט בחישוב רק מפני שהמספרים שלו גדולים יותר. לכן תלמיד שדומה מאוד מבחינת מספר משימות עשוי להיראות רחוק מבחינה מתמטית בגלל הפרש של כמה עשרות דקות.

לדוגמה נתייחס לתצפית $Q=(5,50)$, לתצפית $A=(4,70)$ ולתצפית $B=(9,52)$. ללא שינוי קנה מידה, B קרובה בהרבה ל־Q משום שהפרש הזמנים הוא 2 דקות בלבד, אף על פי שההבדל במספר המשימות הוא 4. אם ננרמל את שני המאפיינים לטווח 0–1 לפי סולמות קבועים 0–10 ו־0–100, A תהיה קרובה יותר. כלומר בחירת הסולם שינתה את המשמעות המעשית של ״דומה״.

$$\text{Raw: } d(Q,A)=\sqrt{(1^2+20^2)}\approx 20.025 \quad ; \quad d(Q,B)=\sqrt{(4^2+2^2)}\approx 4.472$$

$$\text{Scaled: } d(Q,A)=\sqrt{(0.1^2+0.2^2)}\approx 0.224 \quad ; \quad d(Q,B)=\sqrt{(0.4^2+0.02^2)}\approx 0.400$$

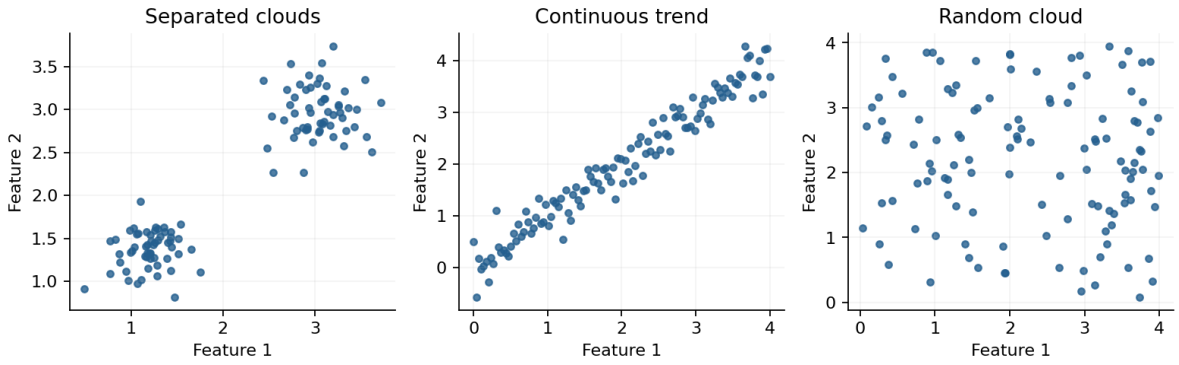


איור: לפני שינוי קנה מידה B קרוב יותר ל־Q; לאחר שינוי היחידות A קרוב יותר. זו תוצאה של בחירת ייצוג, לא שינוי בתצפיות.

למדנו תקנון ונרמול בפרק 2.3, וכעת אנחנו מבינים מדוע הם קריטיים לקיבוץ המבוסס על מרחק. כאשר המטרה היא לבנות מודל שייבחן על נתונים עתידיים, יש להתאים את פעולות ההכנה על נתוני האימון בלבד וליישם אותן באותם פרמטרים על נתונים חדשים. השימוש בכל המאגר לפני חלוקת אימון ובדיקה עלול ליצור זליגת מידע, בהתאם למטרת ההערכה. מצד שני, חקר תיאורי של מאגר שלם אינו זהה לניסוי שמודד הכללה — ההחלטה תלויה בשאלת העבודה.

7. האם בכל מאגר יש קבוצות טבעיות?

לא. לפעמים הנתונים יוצרים עננים נפרדים שקל להצביע עליהם, אך לפעמים הם יוצרים רצף הדרגתי או ענן אחד מפוזר. גם כאשר אלגוריתם מצליח להפיק שלוש קבוצות, אין בכך ראיה שקיימות במציאות שלוש אוכלוסיות נפרדות. אפשר תמיד להמציא גבולות בתוך מרחב, אבל ההצדקה לגבולות היא עניין אחר. בשלב החקר כדאי לשאול האם ההפרדה חזותית, האם היא יציבה, ואיזה הסבר מקצועי עשוי להיות לה.

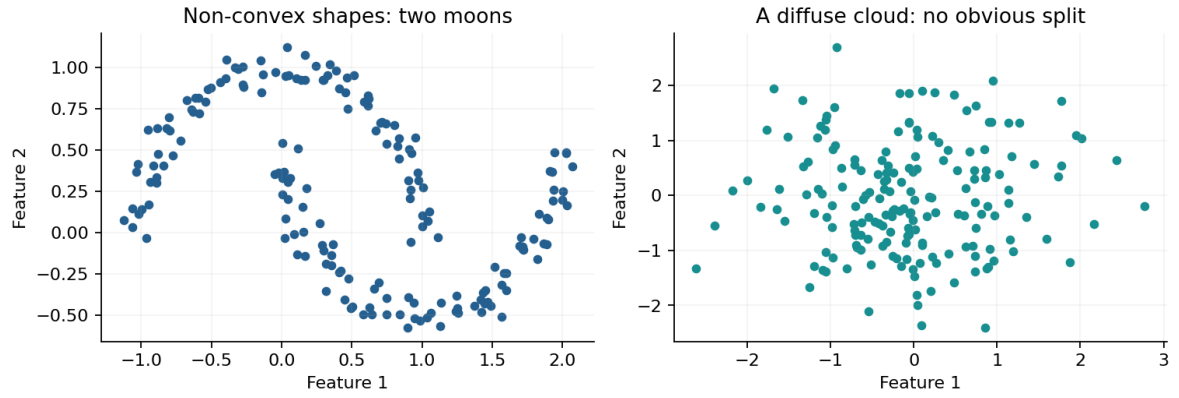


איור: שלוש צורות אפשריות של נתונים: עננים מופרדים, מגמה רציפה וענן ללא חלוקה ברורה. ״מציאת קבוצות״ אינה תמיד המסקנה הנכונה.

הגרף הראשון מרמז על שתי קבוצות, והשני דווקא על קשר רציף בין התכונות. אם נכפה שתי קבוצות על הגרף השני, נקבל גבול שרירותי יחסית שלא בהכרח מייצג תופעה טבעית. אם נבקש להציג את הפתרון לבוחן, עלינו להסביר כיצד הבחירה מתאימה לשאלה המקצועית, לא רק להצביע על צבעים שונים בגרף.

8. צורת הקבוצות — לא כל אלגוריתם מגלה אותו מבנה

יש מערכי נתונים שבהם קבוצות נראות כמו עננים עגולים, ויש מערכים שבהם נקודות מסתדרות בצורת חצי סהר, טבעת או מסלול מעוקל. השאלה מהו ״קיבוץ נכון״ תלויה גם במודל שבחרנו. אלגוריתם המעדיף קבוצות קומפקטיות סביב מרכז יתקשה לעתים לתאר מבנה מעוקל; שיטות המתמקדות בצפיפות יכולות להתאים למצבים אחרים. אין צורך ללמוד כעת את המנגנון של כל אלגוריתם. נדרש להבין שצורת התפלגות הנתונים היא חלק מהשיקולים בבחירתו.

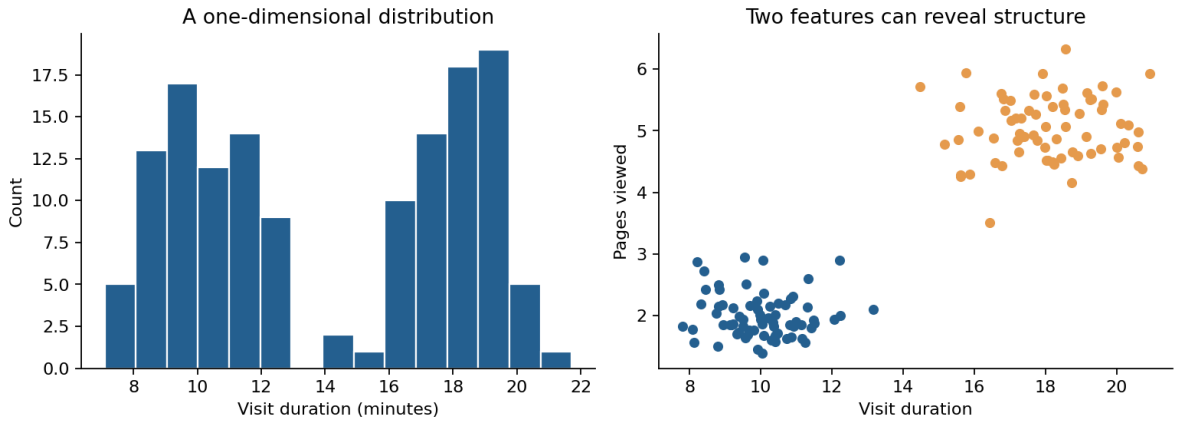


איור: תבנית של שני חצאי סהר לעומת ענן מפוזר. הדוגמה משמשת להמחשת חשיבות מבנה הנתונים; היא אינה מבחן איכות של אלגוריתם מסוים.

בפרק הבא נלמד את K-Means לעומק ונבין מדוע בחירת מספר קבוצות ומיקום מרכזיהן חשובות לו. כאן מספיק לדעת שהוא דוגמה לשיטת Clustering; עדיין איננו מבצעים חישובי עדכון מרכזים או מלמדים את נוסחת פונקציית המטרה שלו.

9. בחירת מאפיינים היא חלק מהחשיבה המדעית

נניח שיש לנו נתוני ביקורים באתר, ובהם זמן שהייה ומספר עמודים שנפתחו. שימוש בזמן שהייה בלבד עשוי להציג שתי פסגות בהיסטוגרמה, אך אינו מסביר האם המשתמשים ביצעו פעולות שונות. הוספת מספר עמודים שנפתחו עשויה להראות שני דפוסי שימוש. מנגד, מאפיין לא רלוונטי יכול להוסיף רעש ולפגוע במבנה שניתן היה לזהות קודם.



איור: ייצוג חד־ממדי לעומת ייצוג דו־ממדי של נתונים מדומים. מידע נוסף יכול להועיל — אם הוא רלוונטי לשאלה.

עלינו להסביר מדוע בחרנו בכל מאפיין. האם אפשר למדוד אותו עבור דוגמה חדשה? האם הוא מספק מידע עצמאי? האם הוא נוצר אחרי האירוע שאותו אנחנו מנסים להבין, ולכן עלול לספק מידע שלא יהיה זמין בזמן אמת? האם הוא כולל מידע אישי שאיננו זקוקים לו? אלה שאלות מקצועיות, דידקטיות ואתיות כאחד.

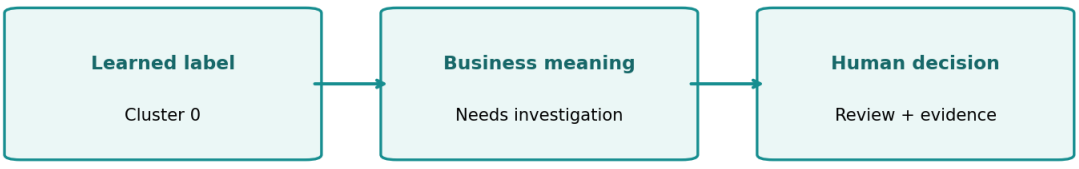
10. חלוקה אוטומטית לקבוצות לעומת חלוקה לפי כלל ידני

כאשר אנחנו מחלקים תלמידים לפי הכלל ״כל מי שמעל 4 במדע נמצא בקבוצת מדע״, ביצענו סינון לפי תנאי. לא גילינו קבוצה באמצעות אלגוריתם קיבוץ. הכלל הוגדר לפני בחינת הנתונים והמשמעות שלו נקבעה מראש. קיבוץ לא מונחה, לעומת זאת, מחפש תבניות לפי מאפיינים ומדד דמיון, ובדרך כלל מחליט על גבולות הקבוצות מתוך הנתונים או לפי מודל שנבחר.

שתי הפעולות יכולות להיות שימושיות: סינון לפי כלל מתאים כאשר ההנהלה קבעה קריטריון מפורש, וקיבוץ מתאים כאשר אנחנו רוצים לחקור מבנה לא ידוע. הטעות היא לקרוא לכל חלוקה של נתונים ״למידת מכונה״. ההבחנה תשמש אותנו גם בבגרות, כאשר נידרש להציע שיטה ולנמק מה היא עושה.

11. איך מפרשים קבוצה אחרי שהאלגוריתם מצא אותה?

לאחר שהאלגוריתם החזיר תוויות טכניות, עלינו לבדוק מה מאפיין כל קבוצה: ממוצעים, טווחים, גדלים, התפלגויות ודוגמאות מייצגות. אפשר למשל לגלות שקבוצה מסוימת מכילה תצפיות רבות עם מספר גבוה של עמודים שנצפו וזמן שהייה בינוני. זו מסקנה תיאורית הנתמכת בנתונים. לא נוכל לקבוע כי ״אלו תלמידים חרוצים״ או כי ״הם בהכרח מתאימים לחוג תכנות״. פרשנות כזאת דורשת מידע נוסף ועשויה להיות לא הוגנת.

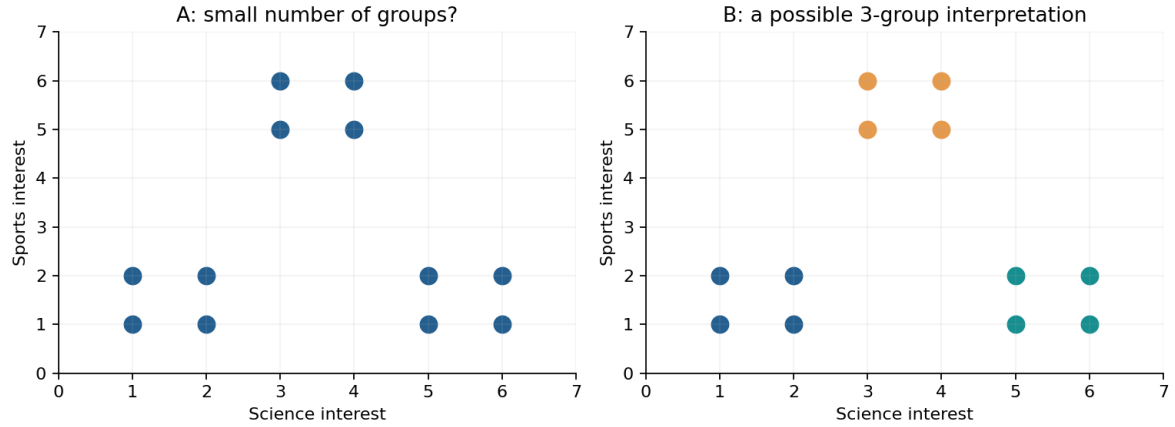


איור: מספר קבוצה הוא מזהה טכני. שם ותיאור מקצועיים דורשים בדיקה, ולפעולה מעשית דרוש גם שיקול אנושי.

לעתים גודל הקבוצה חשוב לא פחות מהממוצע שלה. קבוצה הכוללת שתי תצפיות בלבד עשויה להיות חריגה, אך גם עשויה לייצג מדגם קטן שלא מאפשר הכללה. בקבוצה גדולה הממוצע עשוי להסתיר מגוון פנימי. לכן נדרוש לפחות טבלת גודל קבוצה, מדדים לכל מאפיין וגרף מתאים לפני הצעת פעולה.

12. מהיכן מגיע מספר הקבוצות?

מובן לתלמיד לשאול: האם יש תמיד שתי קבוצות, או שלוש, או יותר? לא. במצבים מסוימים אנחנו קובעים מראש מספר קבוצות לפי הצורך העסקי או לפי אלגוריתם מסוים; במצבים אחרים האלגוריתם עשוי לזהות מספר קבוצות באופן שונה. המספר אינו ידוע רק מפני שיש לנו 12 תצפיות, ואינו נקבע אוטומטית לפי מספר המאפיינים. גם בגרף שלנו ניתן להציע חלוקה לשלוש קבוצות, אך אפשר להעלות שאלות אחרות שמובילות לחלוקה שונה.

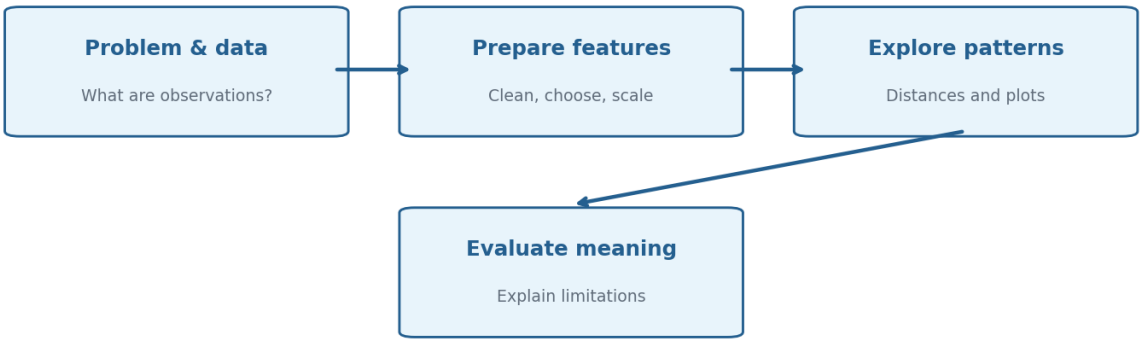


איור: אפשר לצבוע את הנקודות לשלוש קבוצות, אך הצבעים אינם מעידים שהחלוקה היא ״התשובה הנכונה״.

כאשר נלמד K-Means בפרק 3.2 נבדוק כיצד האלגוריתם מקבל מספר קבוצות מבוקש, כיצד משייכים תצפיות למרכזים וכיצד משפרים את מיקום המרכזים. בפרק 3.3 נלמד כיצד להעריך את הקיבוץ ולבחור בין חלופות. אין צורך לערבב את כל המושגים בשיעור המבוא.

13. מבט ראשון על תהליך עבודה מקצועי

אפשר לחשוב על תהליך קיבוץ כסדרה של החלטות הניתנות להסבר: מנסחים שאלה, מזהים מהי תצפית, בוחרים מאפיינים, בודקים ערכים חסרים וחריגים, קובעים כיצד למדוד דמיון, בוחנים נתונים בגרפים ורק לאחר מכן מריצים אלגוריתם. אחרי שקיבלנו קבוצות, בודקים האם הן יציבות ומועילות והאם יש להן משמעות מקצועית. אם לא — חוזרים לבחירת המאפיינים או אפילו לשאלת המחקר.



איור: מחזור עבודה מקוצר לקיבוץ נתונים: בעיה → הכנת נתונים → חקר מבנה → הערכה ופרשנות.

אין כאן הבטחה שנמצא קבוצות משמעותיות. מדע נתונים כולל גם מצבים שבהם המסקנה היא ״לא זיהינו חלוקה מבוססת״. זו מסקנה לגיטימית, ולעתים נכונה יותר ממצגת מרשימה עם צבעים שאין מאחוריהם משמעות. כאשר תלמיד עובד על פרויקט גמר, הוא צריך לשמור תיעוד של החלטות הכנה, גרסאות נתונים, ניסויים, מגבלות והסברים לתוצאה.

14. ערכים חסרים, חריגים ונתונים לא מספריים

לפני כל חישוב מרחק יש לשאול האם חסרים ערכי מדידה. אם תלמיד לא השיב לשאלה על עניין בספורט, לא נכון להציב עבורו 0 בלי מחשבה — 0 עשוי לציין ״אין עניין״ בעוד שערך חסר מציין ״לא ידוע״. אפשר להשלים ערכים, לסמן חסרים, להסיר תצפיות בתנאים מתאימים או להשתמש בשיטות אחרות. ההחלטה חייבת להיות מוצדקת ולהתאים להקשר.

נתון חריג יכול להגדיל מרחקים באופן משמעותי, בעיקר במרחק אוקלידי. ייתכן שהוא שגיאת הקלדה וייתכן שהוא מקרה חשוב. באופן דומה, מאפיינים קטגוריאליים כמו ״סוג חוג״ אינם ערכים מספריים רגילים: המרת שמות הקבוצות ל־1, 2 ו־3 אינה הופכת את ההפרש המספרי ביניהם למדד דמיון אמיתי. יש לבחור ייצוג ומדד המתאימים לטיפוס המידע.

15. מבט ל־Python — רואים מבנה לפני שבוחרים אלגוריתם

בתחילת עבודה נריץ קוד קצר שיוצר מטריצה ומשרטט את התצפיות. בשלב זה הקוד אינו מאמן מודל ולא מחשב קבוצות. הוא מסייע לנו לתאר את הנתונים ולהציע השערות. זוהי דוגמה להבדל בין חקר נתונים (EDA) לבין הרצת אלגוריתם למידת מכונה.

```
import numpy as np
import matplotlib.pyplot as plt
X = np.array([[1,1],[1,2],[2,1],[2,2],
              [5,1],[5,2],[6,1],[6,2],
              [3,5],[3,6],[4,5],[4,6]])
plt.scatter(X[:, 0], X[:, 1], s=80)
plt.xlabel("Science interest")
plt.ylabel("Sports interest")
plt.title("Unlabeled observations")
plt.grid(alpha=0.2)
plt.show()
```

אם תרצו להציע גבולות או צבעים בגרף, כתבו במפורש שזו פרשנות לצורכי חקר ולא תוצאת אימון אמיתית. אין להציג סימון ידני של צבעים כהוכחה שאלגוריתם קיבוץ כבר למד את הקבוצות. בפרק הבא נעבור מחיפוש חזותי לחשיבה אלגוריתמית: כיצד תוכנה מבצעת שיוכים במקום שאנחנו נצבע נקודות לפי העין.

16. האם Clustering הוא אותו הדבר כמו Classification?

לא. בסיווג (Classification) אנו מנסים לנבא אחת ממחלקות היעד שהוגדרו מראש, למשל האם דואר מסוים הוא ספאם או לא. התוויות הן חלק מהבעיה. בקיבוץ (Clustering) אין למשימה תוויות אמת מוכנות; קבוצות נוצרות מתוך דפוסים במאפיינים. רק לאחר מכן נוכל לבדוק אם הן תואמות קטגוריות אנושיות קיימות, אם בכלל. ההתאמה אינה מובטחת.

נניח שבית ספר כבר הגדיר קבוצות של ״מתחילים״ ו״מתקדמים״ לפי מבחן מיון, ומבקשים מהמודל לחזות לאיזו קבוצה שייך תלמיד חדש: זו בעיית סיווג. אם אין מבחן מיון ואנו מבקשים לבחון אילו דפוסי למידה קיימים בנתוני תרגול: זו בעיה לא מונחית. שימוש בטעות במילה ״סיווג״ בשני המצבים מסתיר הבדל מהותי בין למידה מתוך תוויות לבין גילוי מבנה.

מה מתקבל?	מה המחשב מקבל?	פעולה
קבוצות לפי כלל אנושי	תנאי ידוע מראש	סינון לפי כלל
תחזית תווית לדוגמה חדשה	דוגמאות עם תוויות y באימון	סיווג
שיוך לקבוצות שהוגדרו על פי נתונים ושיטה	מאפיינים X ללא תווית קבוצה ידועה	קיבוץ

17. כיצד בודקים אם קיבוץ שנמצא מועיל?

בניגוד לבעיה שבה לכל תצפית יש תווית אמת ידועה, בקיבוץ קשה לעתים לחשב "אחוז תשובות נכונות". לכן נבחן כמה שאלות משלימות: האם נקודות בתוך קבוצה אכן דומות לפי המאפיינים שנבחרו? האם קבוצות שונות נבדלות זו מזו? האם התוצאה יציבה אם נשנה מעט את הנתונים או את הגדרות השיטה? האם אפשר להסביר את הקבוצות במונחים ענייניים? והאם החלוקה מסייעת לקבל החלטה מועילה, הוגנת וזהירה?

בפרק 3.3 נלמד מדדים פורמליים לבדיקת איכות הקיבוץ, כגון מדדים המבוססים על פיזור פנימי והפרדה. כרגע חשוב יותר להבין שמדד מספרי יחיד אינו מבטיח "אמת": אפילו קיבוץ בעל קבוצות צפופות עשוי לשקף מאפיין לא רלוונטי, ולהחמיץ את הבעיה שהמשתמש התכוון לפתור.

18. שאלות של אחריות, פרטיות והוגנות

כשמקבצים נתונים של משתמשים, תלמידים או לקוחות, יש להיזהר במיוחד מהסקת מסקנות אישיות על סמך מאפיינים מועטים. אלגוריתם אינו יודע מה משמעותם החברתית או החינוכית של הנתונים. חלוקה לפי שעות כניסה למערכת אינה הוכחה למוטיבציה או ליכולת. נתונים עשויים לשקף נגישות שונה לאינטרנט, שעות עבודה בבית או גורמים אחרים שלא נמדדו.

בפרויקט תלמידים נעדיף להשתמש בנתונים מדומים או בנתונים ציבוריים מתאימים, להימנע מפרטים מזהים ולתעד את מטרת העבודה. אין להשתמש בקבוצות שנמצאו כדי לקבוע החלטות משמעותיות על תלמידים בלי בדיקות, הקשר, הצדקה מקצועית ופיקוח אנושי. כך לומדים לא רק להפעיל מודל אלא גם לשאול האם ראוי להשתמש בו.

19. שלוש דוגמאות יישומיות בעולם ה־AI

דוגמה א׳ — חנות מקוונת

חנות רוצה לחקור דפוסי קנייה. כל תצפית היא לקוח, וכל מאפיין הוא מדידה שנאספה באופן תקין, כמו מספר רכישות ותדירות ביקורים. קיבוץ עשוי לסייע בהבנת תבניות פעילות לצורך הצעת ממשק נוח יותר. אך אם נמצאה קבוצה עם מספר רכישות נמוך, אין להסיק שכל חבריה אינם מעוניינים במוצרים: ייתכן שהם לקוחות חדשים, או שהנתונים מתעדים רק חלון זמן קצר.

דוגמה ב׳ — חקר תמונות

בתמונה אפשר לייצג כל פיקסל באמצעות ערכי צבע, למשל RGB. אפשר לחקור האם קיימות קבוצות של גוונים דומים. זה שימוש בסיסי בקיבוץ בעולם עיבוד התמונה, אך קרבת צבעים אינה שקולה להבנת משמעות התמונה. שתי נקודות כחולות יכולות להשתייך לאובייקטים שונים, ושטח של אובייקט אחד יכול להכיל גוונים שונים. ההבחנה הזו חשובה לקראת יחידות CNN.

דוגמה ג׳ — ניתוח תוצאות מעבדה מדעית

מדענים עשויים לאסוף מדידות רבות מניסויים ולחפש קבוצות בעלות דפוסי מדידה דומים. קיבוץ יכול להציע השערות מחקריות חדשות, אך אינו מוכיח את ההשערות. כדי להכריע האם מדובר בסוגי תופעה שונים יידרשו מחקר נוסף, בקרה של הניסוי וידע מתחום הדעת.

20. התנסות ראשונה — בניית ״כרטיס מאגר״

לפני הפעלת מודל נבנה תיאור מסודר של הנתונים. כרטיס המאגר יכלול מהי תצפית, מהי יחידת כל מאפיין, מהו טווח הערכים הצפוי, כמה תצפיות קיימות, האם יש ערכים חסרים, האם יש חשש להטיה וכיצד ייבדקו תוצאות הניתוח. בעבודה זו חשוב במיוחד לתעד אילו שדות לא נכללו במודל ולמה. הסרת מאפיין אינה כישלון; לעתים היא החלטה נכונה שמפחיתה רעש ומשפרת את המשמעות של התוצאה.

דוגמה מתוך סיפור דודו	פריט בכרטיס המאגר
תלמיד אחד (בדוגמה מומצאת)	יחידת תצפית
עניין במדע, עניין בספורט	מאפיינים
ציונים מספריים 1–6	סולמות
האם יש דפוסי עניין שונים?	שאלת חקר
אין תווית קבוצה נכונה ואין מדדי הצלחה עתידיים	מידע שאינו קיים
דפוסי עניין אינם קובעים התאמה אישית מוחלטת	מגבלה

21. תרגול מדורג — רמה א׳: הבנת מושגים

תרגיל 1 — זיהוי בעיה

במוסד חינוכי נאספו נתוני שעות תרגול ומספר תרגילים שהושלמו. אין נתון של קבוצת תלמיד מוכנה. הסבירו מהי תצפית, מהם המאפיינים והאם מתאים לשאול שאלת קיבוץ. נסחו גם שאלה שאי אפשר לענות עליה בעזרת הנתונים האלה בלבד.

תרגיל 2 — סוגי למידה

עבור כל אחת משלוש הבעיות נמקו אם מדובר בסיווג, קיבוץ או סינון לפי כלל: (א) קביעת קבוצת לימוד לפי ציון סף 80; (ב) חיזוי קבוצת לימוד מתוך דוגמאות מסומנות; (ג) חיפוש דפוסים בקרב תלמידים ללא תוויות.

תרגיל 3 — מטריצה

במאגר יש 30 תצפיות ו־5 מאפיינים. כתבו את ממדי X. האם קיים בהכרח y? הסבירו מה מייצגת העמודה השלישית, ומדוע אין לקבוע מראש איזו קבוצה תתקבל רק לפי מספר העמודות.

22. תרגול מדורג — רמה ב׳: חישוב והסקה

תרגיל 4 — מרחקים

נתונות התצפיות $A=(2,1)$, $B=(5,5)$, $C=(2,4)$. חשבו ידנית את המרחק האוקלידי ואת מרחק מנהטן בין A ל־B ובין A ל־C. איזו תצפית קרובה יותר ל־A לפי כל מדד? האם מכאן נובע בהכרח שיוך קבוצתי זהה בכל אלגוריתם?

תרגיל 5 — סולם המדידה

תצפית $Q=(5{,}50)$, $A=(4{,}70)$, $B=(9{,}52)$. חשבו מרחקים לפני ואחרי חלוקה של המאפיין הראשון ב־10 ושל השני ב־100. הציגו את התוצאות והסבירו מדוע התהפך סדר הקרבה, מבלי להשתמש בביטוי ״הנתונים השתנו״.

תרגיל 6 — שני מאגרים

במאגר ראשון הנקודות יוצרות שלושה עננים מופרדים. במאגר שני הנקודות מפוזרות ברצף אחד לאורך ישר. האם מוצדק לצבוע אוטומטית שלוש קבוצות בשני המקרים? כתבו טענה, ראיה חזותית אפשרית ומגבלה אחת.

תרגיל 7 — קוד והבנה

בקוד Python מופיעים X.shape ו־X[:, 0]. הסבירו מה תחזיר כל פקודה עבור מטריצה בגודל 12×2. כתבו מדוע השימוש ב־plt.scatter אינו שקול לאימון אלגוריתם קיבוץ.

23. תרגול מדורג — רמה ג׳: שאלות בסגנון בגרות ופרויקט גמר

שאלת הערכה מקורית 1 — התאמה לשיטה

חברה משתמשת במאפיינים ״הכנסה חודשית״ ו״מספר ביקורים בחודש״ כדי לקבץ לקוחות. הסבירו איזו בעיה עלולה להיווצר כאשר מפעילים מרחק אוקלידי ללא תקנון. הציעו פעולת הכנה וכתבו כיצד נוודא שלא השתמשנו בטעות בנתונים העתידיים בתהליך הערכת המודל.

שאלת הערכה מקורית 2 — ביקורת מקצועית

תלמיד הפעיל אלגוריתם וקיבל ארבע קבוצות. הוא כתב בדוח: ״הוכח שקיימים ארבעה סוגי לקוחות טבעיים, ושקבוצה 0 היא הלקוחות הטובים ביותר״. הסבירו לפחות שלוש בעיות במשפט זה ונסחו מסקנה זהירה יותר שאפשר להצדיק בבדיקות נתונים.

שאלת הערכה מקורית 3 — ניסוי מתוכנן

תכננו ניסוי השוואה בין שתי בחירות מאפיינים עבור אותו מאגר: רק זמן ביקור, ולעומתו זמן ביקור ומספר עמודים. כתבו מה יישמר קבוע, איזה גרף תכינו בכל גרסה, כיצד תתעדו את הניסוי ואיזו תוצאה תגרום לכם להטיל ספק בתועלת הקיבוץ.

משימת הכנה לפרויקט גמר

בחרו תרחיש של חנות, בית ספר, שירות דיגיטלי או תמונות. נסחו שאלת חקר אחת לקיבוץ, תארו שלושה מאפיינים אפשריים, יחידות מדידה, מידע חסר אפשרי, בחירת גרף ראשונית, סיכון לפרטיות ודרך לבחון אם לקיבוץ תהיה משמעות. אין להפעיל עדיין K-Means; זו משימת אפיון איכותית.

24. מעבדת Google Colab עצמאית — מה צריך להגיש?

המחברת המצורפת לפרק מלווה את ספר התלמיד ומכילה 12 משימות: טעינת מטריצה, בדיקת ממדים, תרשימי פיזור, חישוב מרחקים, בחירת מאפיינים, השוואה לפני ואחרי שינוי קנה מידה, תיאור השערות ומגבלות, והכנת כרטיס מאגר. התא הראשון כולל קוד התחלתי עובד, אך רוב המשימות מיועדות להשלמה בידי התלמידים. אין במחברת פתרונות מלאים למשימות.

התוצר שמגישים אינו רק גרף או קוד. ליד כל תרגיל יש להוסיף משפט פרשנות: מה נמצא, איזו מסקנה מותרת ומה עוד לא ידוע. מטרתנו להרגיל את התלמיד לכתוב בדוח פרויקט לא רק מה הפעולה החזירה, אלא מדוע השתמש בה ומה משמעותה.

25. טעויות נפוצות — ודאי שתוכלו לזהות אותן

ההבחנה המדויקת	אמירה שגויה
אפשר לגלות מבנה ב־X גם ללא תווית יעד.	״אם אין y, אין שום למידה״
המספר תלוי בשיטה ובשאלת המחקר; דורש הצדקה.	״שלוש קבוצות הן האמת של הנתונים״
תלוי באלגוריתם, במאפיינים ובסביבה של הנקודות.	״שתי נקודות קרובות תמיד באותה קבוצה״
חלוקה לפי תנאי ידני שונה מחיפוש דפוס לא מונחה.	״כל חלוקה של טבלה היא Clustering״

ההבחנה המדויקת	אמירה שגויה
מזהה טכני אינו דרגת איכות או משמעות מקצועית.	״מספר קבוצה 0 אומר שהקבוצה חלשה״
נתוני עניין מצומצמים אינם הוכחה להתאמה אישית.	״המודל הוכיח מי מתאים לחוג״

26. סיכום — מה התלמיד צריך לדעת עכשיו?

למידה לא מונחית מאפשרת לחקור נתונים כאשר אין תשובות יעד מוכנות. בקיבוץ אנו מנסים למצוא דפוסים של דמיון ושוני, אך התוצאה אינה מגיעה עם פירוש מובנה או עם הוכחה שהיא חלוקה ״נכונה״. לפני שמריצים אלגוריתם יש להבין את התצפיות, את המאפיינים, את סולמות המדידה ואת הצורה של הנתונים. אחרי שמריצים אותו יש לבדוק אם הקבוצות יציבות, רלוונטיות ומבוססות. לכן גם בפרק שהוא לכאורה ״ללא תוויות״, האחריות האנושית להגדרת השאלה ולפירוש התוצאה גדולה מאוד.

בפרק 3.2 נלמד כיצד K-Means יוצר קבוצות באופן חישובי, כיצד נקבעים מרכזים, כיצד מתבצע שיוך התצפיות ומה קורה בכל צעד של האלגוריתם. בפרק 3.3 נלמד להשוות חלופות ולהעריך באופן שיטתי את איכות החלוקה. כעת כבר יש לנו את מושגי הבסיס הדרושים כדי להבין מה האלגוריתם עושה — ובעיקר מה הוא אינו יודע.

מפת קשר לספר המאוחד ולתכנית הלימודים

מעמד	קשר לתוכן כאן	מסלול / פרק
ידע קודם והעמקה ספירלית	שימוש בווקטורי מאפיינים, בתרשימי פיזור ובמרחק	יחידה 3 — EDA, Pandas, KNN
עוגן הסילבוס במאגר	מבוא ליישומי למידה לא מונחית וקיבוץ	נושאי ליבה — סעיף 3.1
יילמד במלואו בפרק הבא	K-Means: הצגת הבעיה בלבד	נושאי ליבה — סעיף 3.2
יילמד במלואו בהמשך	הערכת קיבוץ: שאלות איכות ראשוניות	נושאי ליבה — סעיף 3.3
הרחבה יישומית לפרויקט גמר	החלטות אפיון, הכנת נתונים ופרשנות תוצאות	התמחות למידה עמוקה — פרויקט

הערת התאמה: שיוך סעיף 3.1 נסמך על מפת נושאי הליבה שבמאגר הקורס. עד לאימות סופי מול קובץ הסילבוס הרשמי המעודכן, אין לראות ברשימת ההרחבות בפרק קביעה שכל פרט הוא חומר חובה לבגרות. הדוגמאות, הנתונים והתרשימים כאן מקוריים לצורכי הוראה.

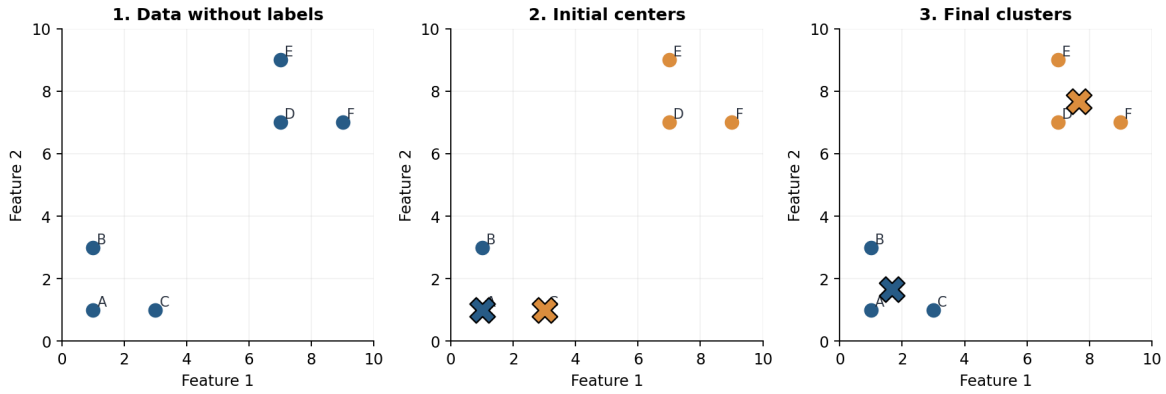
מקורות והמשך קריאה

בסיס הסילבוס: מאגר Data-Science-ML-Course, יחידה 5, פרק 3 — Clustering, תתי־סעיפים 3.1–3.3: https://github.com/samiabo75/Data-Science-ML-Course/tree/main/02-Unit5-AI-Core/03-Clustering

לגישה אינטואיטיבית: Luis G. Serrano, Grokking Machine Learning. לגישה יישומית: Aurélien Géron, Hands-On Machine Learning. להבנה סטטיסטית של קיבוץ ומבנה נתונים: James et al., An Introduction to Statistical Learning with Applications in Python. למתמטיקה של וקטורים ומרחקים: Deisenroth, Faisal & Ong, Mathematics for Machine Learning. אלה מקורות השראה ללימוד; ההסברים והתרגילים בפרק נכתבו בניסוח מקורי.

פרק 3.2 — האלגוריתם K-Means

כיצד מחשב מוצא קבוצות, מזיז מרכזים ולומד מבנה מתוך נתונים ללא תוויות?



איור: שלוש תמונות של אותו מאגר: נקודות ללא תוויות, מרכזים התחלתיים וחלוקה סופית. הצבעים אינם תשובת אמת.

מהדורת תלמיד מלאה • חישוב ידני דו־איטרטיבי • 10 המחשות • Python ו־scikit-learn • תרגילים מקוריים בסגנון בגרות • מעבדה עצמאית

בפרק הקודם למדנו שקיבוץ נתונים אינו זהה לסיווג לפי תוויות ידועות. כעת נבנה את התהליך המתמטי המדויק של K-Means. נשאל בכל שלב: מה המחשב יודע? מה הוא מחשב? מדוע המרכזים משתנים? כיצד יודעים שנעצרנו? ומה בעצם אפשר להסיק מהקבוצות שהתגלו?

מטרות הלמידה וידע קודם

לפני שמתחילים, חשוב לדעת לחשב ממוצע, מרחק אוקלידי, וקטור ומטריצה; לפרש תרשים פיזור; ולהבין מדוע שינוי קנה מידה משפיע על מרחקים. אין צורך לדעת את K-Means מראש. נבנה את האלגוריתם מהתחלה באמצעות שש נקודות בלבד.

- להסביר מהו K, מהו מרכז קבוצה (Centroid), ומה ההבדל בין שיוך נקודה לקבוצה לבין עדכון מרכז.
- לחשב ידנית מרחקים, לשייך נקודות למרכז הקרוב, ולחשב מרכז חדש כממוצע רכיבי הנקודות ששויכו אליו.
- לעקוב אחרי שתי איטרציות מלאות, ולזהות האם השיוכים והמרכזים השתנו.
- להסביר את מטרת מזעור סכום ריבועי המרחקים (WCSS / inertia), בלי לבלבל אותה עם דיוק סיווג.
- לממש את השלבים ב־NumPy ולהפעיל KMeans באמצעות scikit-learn; לפרש centers, labels ו־inertia.
- לבקר בחירת K, אתחול, רגישות לחריגים, לקנה מידה ולצורת קבוצות; ולהציע שימוש זהיר בפרויקט AI.

ידע מהפרקים הקודמים	מה נשתמש בו כעת
פרק 1.1 — וקטורים	כל תצפית היא וקטור; ממוצע רכיבי נותן מרכז של קבוצה.
פרק 1.2 — מרחקים	נקודה משויכת למרכז בעל המרחק האוקלידי

מה נשתמש בו כעת	ידע מהפרקים הקודמים
הקטן ביותר.	
הממוצע וקנה המידה חשובים לחישוב המרכזים.	פרק 2.1 — מרכז ופיזור
קנה המידה עשוי לשנות את השיוכים ואת התוצאה.	פרק 2.3 — תקנון
בוחנים תרשים ומפרשים קבוצות בלי להמציא משמעות.	פרק 2.5 — גרפים

1. סיפור פתיחה — דודו בונה מערכת לתכנון חוגי העשרה

דודו מרכז מיזם חוגי העשרה. בית הספר אסף מתלמידים שני ציוני עניין, במדע ובספורט, בטווח 0–10. אין ברשותו עמודה המציינת "הקבוצה הנכונה" לכל תלמיד. דודו מקווה לזהות קבוצות של תחומי עניין כדי להציע מגוון פעילויות. זו משימת למידה לא מונחית: המודל מחפש מבנה בתצפיות עצמן, לא מנסה לחקות תשובה ידועה מראש.

נתחיל במדגם מצומצם של שש תצפיות מומצאות. בחרנו מספרים קטנים במכוון, כדי שכל תלמיד יוכל לעקוב אחר האלגוריתם בעזרת דף, עיפרון ומחשבון. בהמשך נפעיל אותה מתודה על נתונים רבים יותר באמצעות Python.

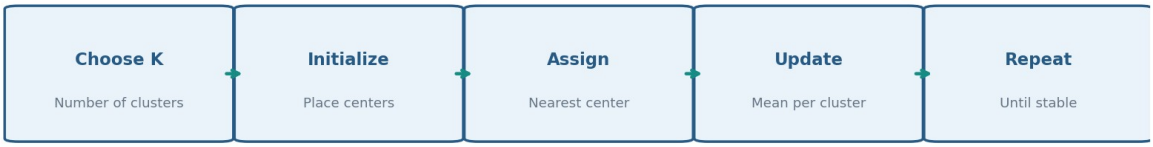
תלמיד	עניין במדע (x)	עניין בספורט (y)	ייצוג וקטורי
A	1	1	(1 ,1)
B	1	3	(3 ,1)
C	3	1	(1 ,3)
D	7	7	(7 ,7)
E	7	9	(9 ,7)
F	9	7	(7 ,9)

הטבלה אינה קובעת אילו חוגים מתאימים לתלמידים מסוימים. היא רק מדגימה אוסף של שני מאפיינים מספריים. חשוב לשמור על ההבחנה: הבדל מספרי בשאלון אינו בהכרח הבדל אמיתי ביכולת, בזהות או בהעדפה היציבה של אדם.

2. מהו K-Means, ומה פירוש השם?

K-Means הוא אלגוריתם נפוץ לקיבוץ נתונים, כלומר לחלוקה של תצפיות לקבוצות לפי דמיון במרחב המאפיינים. האות K מציינת את מספר הקבוצות שאנו מבקשים מן האלגוריתם למצוא. המילה Means מזכירה כיצד מחושבים המרכזים: כל מרכז הוא וקטור הממוצע של הנקודות השייכות לקבוצה שלו. לפיכך, K אינו "מספר הקבוצות שהמחשב מוכיח שקיימות בטבע" — הוא בחירה שמבצע החוקר לפני הרצה.

למשל, אם נגדיר $K=2$, האלגוריתם יחפש שני מרכזים וישייך כל תצפית לאחד מהם. אם נגדיר $K=3$, הוא יחפש שלושה מרכזים. חלוקה היא תמיד תוצאה של אלגוריתם, נתונים, בחירת K ומדד מרחק. תוצאה מסוימת יכולה להיות מועילה, אך אין לראות בה אמת מוחלטת.



איור: חמש הפעולות של K-Means: בוחרים K, מאתחלים מרכזים, משייכים נקודות, מעדכנים ממוצעים וחוזרים עד עצירה.

3. מושגי יסוד: נקודה, מרכז, מרחק ושיוך

נקודה היא וקטור של מאפיינים. בדוגמה שלנו $A=(1,1)$, ולכן התלמיד A מיוצג באמצעות שני מספרים. מרכז קבוצה (Centroid) הוא נקודה ממוצעת; הוא אינו חייב להיות תלמיד אמיתי או אחת התצפיות המקוריות. למשל, המרכז $(1.67,1.67)$ מייצג את ממוצע הקבוצה, גם אם אין תלמיד עם הערכים האלה.

שיוך (Assignment) הוא הבחירה לאיזה מרכז שייכת כל נקודה באיטרציה הנוכחית. בשלב זה לא מזיזים את הנקודות עצמן — רק בוחרים איזה מרכז קרוב ביותר אליהן. בשלב הבא מחשבים מחדש את המרכזים לפי נקודות הקבוצה. האלגוריתם מחליף שוב ושוב בין שתי הפעולות האלה.

$$d(x,c) = \sqrt{[(x_1-c_1)^2 + (x_2-c_2)^2]}$$

נוסחת המרחק האוקלידי מוכרת מפרק 1.2. מבחינת בחירת המרכז הקרוב, אפשר להשוות גם את ריבועי המרחקים ללא שורש: מאחר שהשורש הריבועי הוא פונקציה עולה לערכים אי־שליליים, המרכז שמזער את d הוא גם המרכז שמזער את d^2. זוהי דרך לחסוך חישוב בלי לשנות את החלטת השיוך.

$$d^2(x,c) = (x_1-c_1)^2 + (x_2-c_2)^2$$

בדיקת הבנה: אם מרחקים לשני מרכזים הם $\sqrt{5}$ ו־$\sqrt{20}$, איזה מרכז קרוב יותר? אין צורך לחשב את ערכי השורשים העשרוניים. חשוב להסביר מדוע גם השוואת 5 מול 20 מספיקה.

4. איטרציה ראשונה — בוחרים מרכזים התחלתיים

כדי שנוכל לחשב הכול ידנית, נגדיר $K=2$ ונבחר בכוונה שני מרכזים קרובים בצד השמאלי של המישור: $C_1=(1,1)$ ו־$C_2=(3,1)$. בחירה זו אינה אידאלית — וזה בדיוק מה שנרצה להבין. אם מתחילים במרכזים לא טובים, האלגוריתם יצטרך להזיז אותם במהלך האימון.

$$C_1^{(0)} = (1,1) \qquad C_2^{(0)} = (3,1)$$

הסימון העליון (0) מציין את המיקום ההתחלתי, לפני עדכון כלשהו. המספר אינו חזקה. בסעיפים הבאים נשתמש ב־(1) וב־(2) לציון מרכזים לאחר העדכון הראשון והשני.

5. איטרציה ראשונה — שלב השיוך למרכז הקרוב

נחשב לכל תלמיד את ריבוע המרחק אל שני המרכזים. לדוגמה, עבור $B=(1,3)$: המרחק בריבוע ל־C_1 הוא $(1-1)^2+(3-1)^2=4$; המרחק בריבוע ל־C_2 הוא $(1-3)^2+(3-1)^2=8$. לכן B משויך למרכז הראשון. עבור $D=(7,7)$, המרחקים בריבוע הם 72 ו־52, ולכן D משויך למרכז השני. אין כאן תוויות אמת; זו בחירת קרבה בלבד.

תצפית	אל C_1 d^2	אל C_2 d^2	השיוך
A (1,1)	0	4	קבוצה 1

תצפית	d^2 אל c_1	d^2 אל c_2	השיוך
B (1,3)	4	8	קבוצה 1
C (3,1)	4	0	קבוצה 2
D (7,7)	72	52	קבוצה 2
E (7,9)	100	80	קבוצה 2
F (9,7)	100	72	קבוצה 2

כעת קבוצה 1 מכילה את A ו־B, ואילו קבוצה 2 מכילה את C, D, E ו־F. התבוננו בנקודה C: היא נמצאת למעשה קרוב יותר לאזור השמאלי של הנתונים, אך בתחילת הריצה משויכת לקבוצה השנייה בגלל המרכז שנבחר. בהמשך נראה שהשיוך הזה יכול להשתנות.

6. איטרציה ראשונה — מחשבים מחדש מרכזים

בשלב העדכון מחליפים כל מרכז בממוצע הרכיבי של הנקודות ששויכו אליו. כלומר, מחשבים ממוצע נפרד לעמודת x וממוצע נפרד לעמודת y. אין בוחרים בהכרח אחת הנקודות המקוריות; מרכז חדש יכול להיות בין הנקודות.

$$C_j(\text{new}) = (1 / |S_j|) \cdot \Sigma[x \in S_j]\ x$$

כאן S_j היא קבוצת התצפיות ששויכה למרכז מספר j, ו־$|S_j|$ הוא מספר התצפיות בקבוצה. הסכימה מתבצעת על הווקטורים רכיב־רכיב. לכן הנוסחה מחזירה וקטור חדש באותו ממד כמו הנתונים.

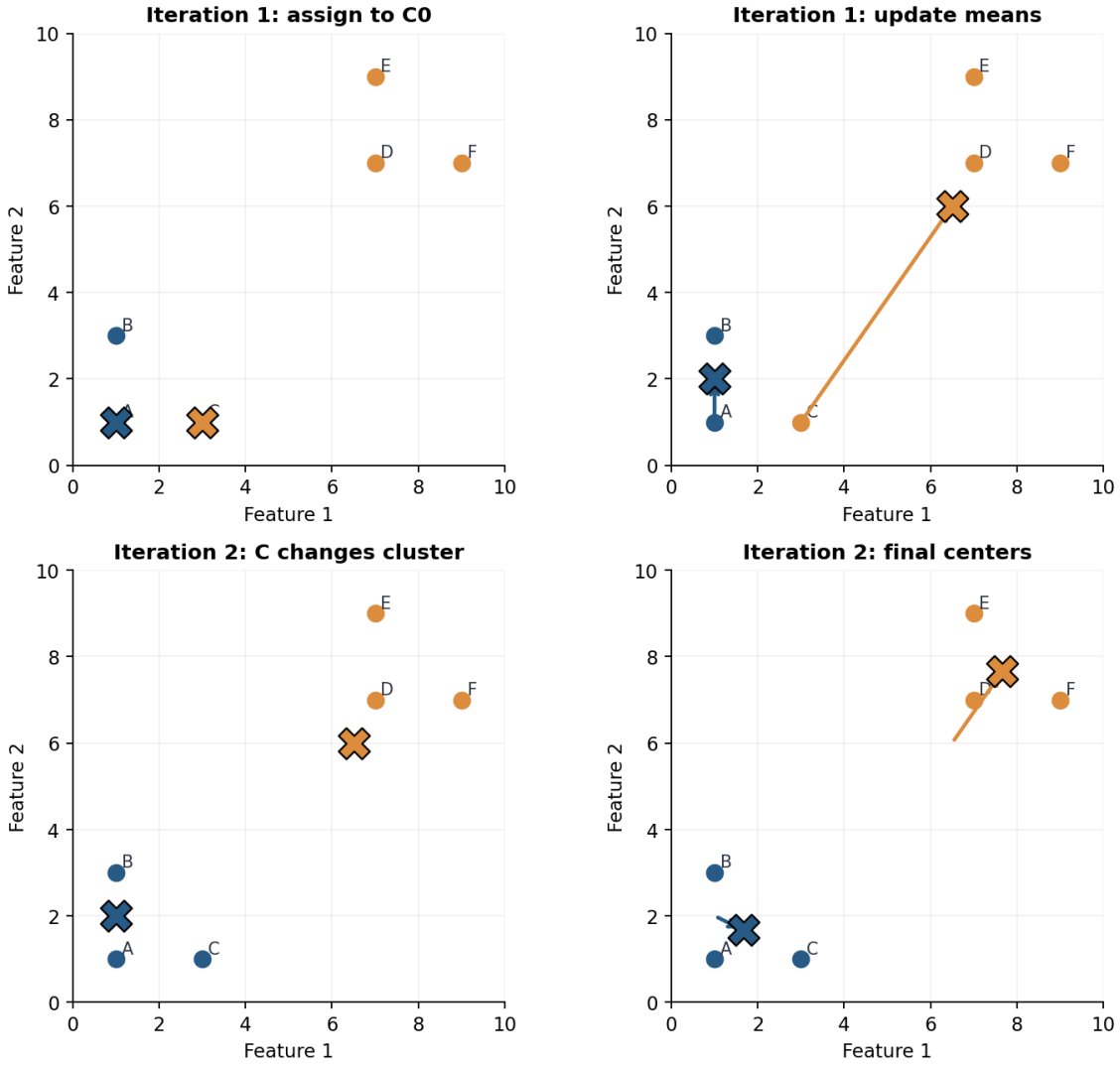
קבוצה 1 כוללת שתי נקודות, $A=(1,1)$ ו־$B=(1,3)$. ממוצע רכיבי x הוא $(1+1)/2=1$, וממוצע רכיבי y הוא $(1+3)/2=2$. לפיכך המרכז נע מהנקודה (1,1) אל (1,2).

$$C_1^{(1)} = ((1+1)/2,\ (1+3)/2) = (1,2)$$

קבוצה 2 כוללת ארבע נקודות: C, D, E ו־F. סכום רכיבי x הוא $9+7+7+3=26$, ולכן הממוצע $26/4=6.5$. סכום רכיבי y הוא $7+9+7+1=24$, ולכן הממוצע $24/4=6$. המרכז החדש הוא (6.5,6).

$$C_2^{(1)} = ((3+7+7+9)/4,\ (1+7+9+7)/4) = (6.5,6)$$

שימו לב לשינוי הגדול במרכז השני. הוא נע משום שרוב הנקודות ששויכו אליו נמצאות רחוק מהמרכז ההתחלתי. כעת הגיוני לבדוק אם כל הנקודות עדיין משויכות למרכז הקרוב ביותר — התשובה אינה מובנת מאליה.



איור: ארבעת מצבי התהליך: שיוך ראשון, עדכון ראשון, שיוך מחדש ועדכון שני. סימן x מציין מרכז קבוצה.

7. איטרציה שנייה — בודקים מחדש את כל השיוכים

אחרי העדכון הראשון, המרכזים הם $C_1=(1,2)$ ו־$C_2=(6.5,6)$. נחשב שוב את המרחקים בריבוע. עבור $C=(3,1)$, המרחק בריבוע למרכז הראשון הוא $(1-1)^2+(3-2)^2=5$, בעוד המרחק בריבוע למרכז השני הוא $(1-6)^2+(3-6.5)^2=37.25$. לכן התצפית C עוברת מקבוצה 2 לקבוצה 1.

תצפית	d^2 אל (1,2)	d^2 אל (6.5,6)	שיוך מעודכן
A	1	55.25	קבוצה 1
B	1	39.25	קבוצה 1
C	5	37.25	קבוצה 1
D	61	1.25	קבוצה 2
E	85	9.25	קבוצה 2
F	89	7.25	קבוצה 2

הפעם כל הנקודות A,B,C משויכות לקבוצה הראשונה, וכל הנקודות D,E,F משויכות לשנייה. זהו רגע חשוב להבנת התהליך: האלגוריתם אינו ״מתחייב״ לשיוך שנקבע באיטרציה הראשונה. הוא מעדכן החלטות בעקבות שינוי המרכזים.

8. איטרציה שנייה — המרכזים מתעדכנים שוב

קבוצה 1 כוללת A=(1,1), B=(1,3), C=(3,1). סכום רכיבי x הוא 5, וסכום רכיבי y גם הוא 5. לכן שני ממוצעי הקבוצה שווים 5/3, כלומר כ־1.67.

$$C_1^{(2)} = (5/3,\ 5/3) \approx (1.67, 1.67)$$

קבוצה 2 כוללת D=(7,7), E=(7,9), F=(9,7). סכום כל עמודה הוא 23, ולכן כל רכיב במרכז החדש שווה 23/3, כ־7.67.

$$C_2^{(2)} = (23/3,\ 23/3) \approx (7.67, 7.67)$$

אחרי עדכון זה נבצע בדיקת שיוך נוספת. כל אחת משלוש הנקודות השמאליות נשארת בקבוצה הראשונה, וכל אחת משלוש הנקודות הימניות נשארת בקבוצה השנייה. מאחר שהשיוכים לא השתנו, גם הממוצעים לא ישתנו בעדכון הבא. הדוגמה הגיעה למצב יציב.

9. כיצד יודעים שהאלגוריתם השתפר? פונקציית המטרה WCSS

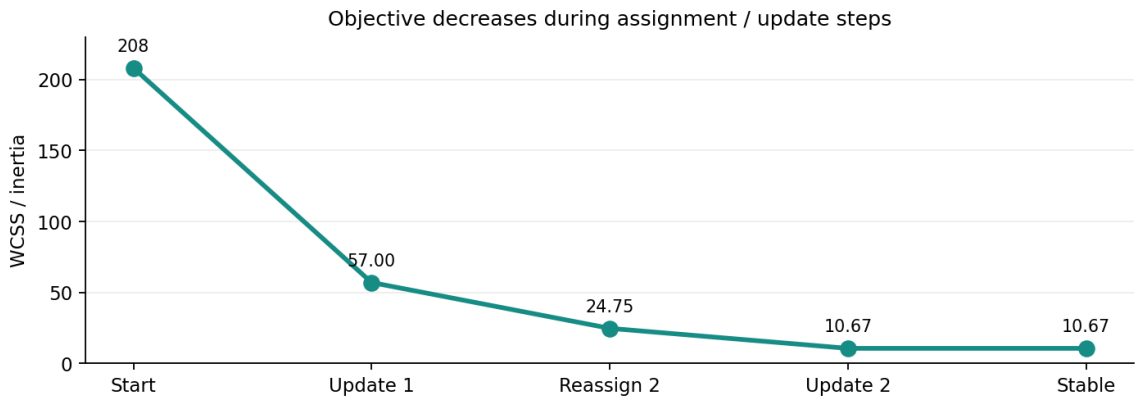
העובדה שהמרכזים זזים אינה מבטיחה לבדה שפתרנו בעיה שימושית. כדי להבין מה האלגוריתם מנסה לעשות, נגדיר מדד מתמטי: סכום ריבועי המרחקים של כל התצפיות מן המרכז שאליו שויכו. מדד זה מכונה Within-Cluster Sum of Squares, בקיצור WCSS. בספריית scikit-learn תראו גם את המונח inertia.

$$J = \Sigma[j=1..K]\ \Sigma[x \in S_j]\ \|x - C_j\|_2^2$$

הנוסחה היא סכום בתוך סכום: בוחרים קבוצה אחת, מחברים את ריבועי המרחקים של הנקודות למרכזה, ואז מחברים על פני כל K הקבוצות. הערך J לעולם אינו שלילי. ערך קטן יותר פירושו שהנקודות קרובות יותר למרכזיהן במובן הספציפי הזה; הוא אינו בהכרח הוכחה שהחלוקה מועילה או הוגנת.

בדוגמה שלנו, עם המרכזים ההתחלתיים והשייכים שנקבעו בתחילת הריצה, סכום המרחקים בריבוע הוא 72+80+52+0+4+0=208. לאחר עדכון המרכזים, בלי לשנות את השיוכים, הסכום יורד ל־57. שיוך מחודש למרכזים המעודכנים מוריד את הסכום ל־24.75, ואחרי העדכון השני הסכום יורד ל־32/3≈10.67.

שלב	סכום ריבועי המרחקים = J	מה השתנה?
התחלה לפני עדכון	208	מרכזים התחלתיים
אחר עדכון 1	57	המרכזים זזו אל ממוצעי הקבוצות
אחר שיוך 2	24.75	עברה לקבוצה 1 C
אחר עדכון 2	10.6667	עודכנו שני הממוצעים
בדיקת יציבות	10.6667	השיוכים נשארו ללא שינוי



איור: סכום ריבועי המרחקים בדוגמה שלנו קטן אחרי שלבי השיוך והעדכון, עד שמגיע למצב יציב.

מדוע הממוצע מתאים דווקא ל־WCSS? אפשר להראות שבקבוצה נתונה, הממוצע ממזער את סכום ריבועי המרחקים מהנקודות. אינטואיטיבית, אם מרכז נמצא רחוק מדי מחלק מהנקודות, הזזתו לעבר הממוצע מפחיתה את סכום הסטיות הריבועיות. זו הסיבה המתמטית למילה Means בשם האלגוריתם. הממוצע אינו בהכרח בחירה מיטבית עבור מטרת מרחק אחרת, למשל סכום מרחקי מנהטן.

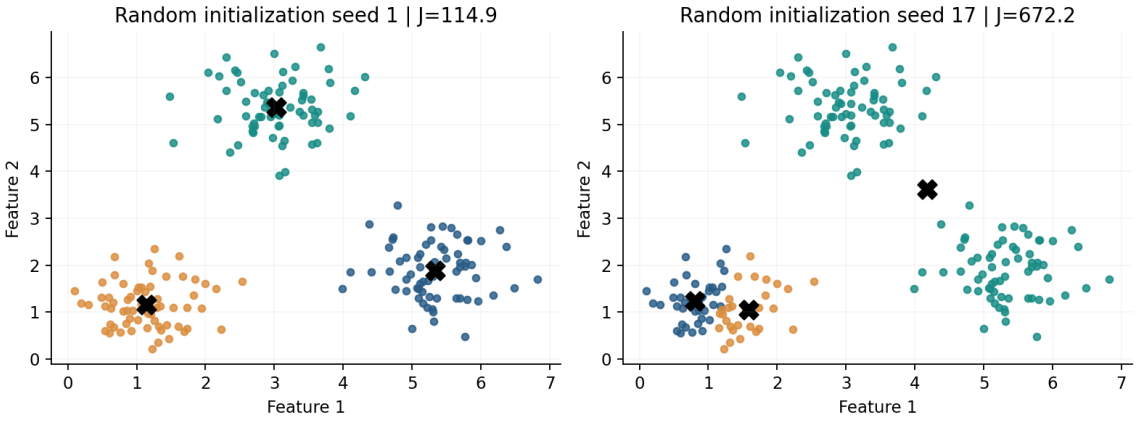
10. מדוע K-Means עוצר, ומה בדיוק פירוש התכנסות?

K-Means חוזר על שני צעדים: שיוך לפי המרחק למרכזים הקיימים, ועדכון המרכזים לממוצעים של קבוצות התצפיות. כל צעד אינו מגדיל את J כאשר עובדים עם ריבוע המרחק האוקלידי ובוחרים שיוכים באופן עקבי. מאחר שיש מספר סופי של אפשרויות שיוך עבור מאגר סופי, האלגוריתם יגיע למצב שבו לא משתנה עוד השיוך, או ייעצר לפי תנאי עצירה מעשי כמו מספר איטרציות מרבי או שינוי מרכזים קטן מסף שהוגדר.

חשוב לדייק: התכנסות אינה הבטחה שמצאנו את החלוקה הטובה ביותר מבין כל החלוקות האפשריות. האלגוריתם יכול להגיע לפתרון מקומי. תוצאה תלויה לעיתים במרכזי הפתיחה, במספר הקבוצות, במאפיינים ובאופן תקנונם. לכן חשוב להשוות מספר הרצות, לנמק את K ולבדוק את התוצאה בהקשר של שאלת המחקר.

11. אתחול המרכזים: מדוע תוצאה יכולה להשתנות?

בדוגמה הידנית בחרנו מרכזים במפורש. במאגרים גדולים לא תמיד יודעים היכן כדאי להתחיל. אפשר לבחור מרכזים באקראי, אך לפעמים שני מרכזים ייפלו באותו אזור ויהיה קשה להגיע לחלוקה טובה. שיטת k-means++ בוחרת מרכזים התחלתיים מפוזרים יותר בצורה הסתברותית, והיא ברירת המחדל המקובלת ב־scikit-learn. גם איתה אין הבטחה למציאת מינימום גלובלי בכל בעיה.



איור: הרצות שונות של אתחול אקראי עשויות להוביל לפתרונות בעלי פונקציית מטרה שונה. זו הדגמה על נתונים סינתטיים.

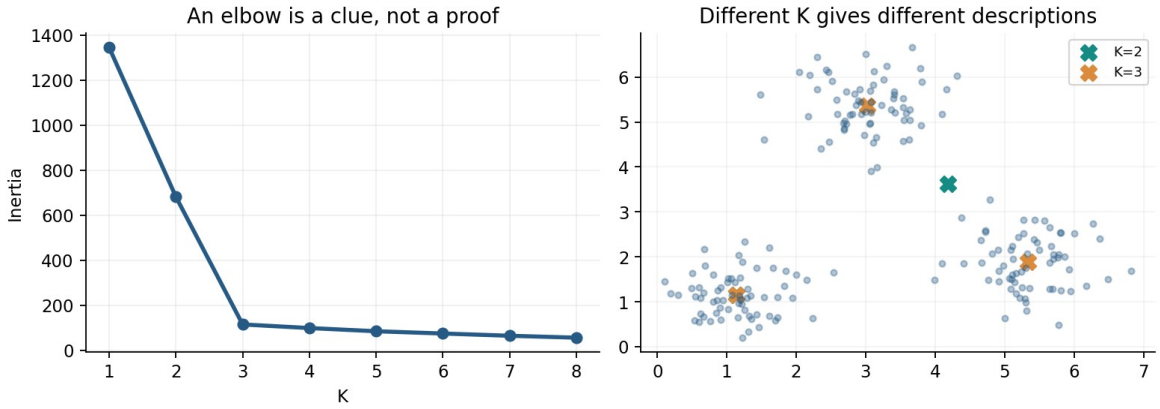
הפרמטר random_state קובע את מקור האקראיות כך שאפשר לשחזר את הניסוי; הוא אינו עושה את המודל ״חכם יותר״. הפרמטר n_init קובע כמה אתחולים ננסה, והספרייה בוחרת תוצאה בעלת inertia נמוכה יותר. במימושים שונים ובגרסאות שונות של הספרייה ערכי ברירת המחדל עשויים להשתנות, ולכן בחוברת נכתוב את n_init במפורש.

12. בחירת K — כמה קבוצות יש?

אחת השאלות המרכזיות היא מה צריך להיות K. נוכל לנסות K=2, K=3 וכן הלאה. ככל שמגדילים את מספר הקבוצות, בדרך כלל ניתן להקטין את WCSS, משום שליותר מרכזים יש גמישות רבה יותר להיות

קרובים לנקודות. אם נגדיר K כמספר הנקודות, כל נקודה יכולה להיות מרכז בפני עצמה ו־WCSS יהיה אפס — אך החלוקה תהיה חסרת תועלת למחקר ברוב המקרים.

מכאן שלא נכון לבחור K רק מפני שהתקבל J הקטן ביותר. אפשר לשרטט את J כפונקציה של K ולחפש נקודת "מרפק", שבה השיפור מתמתן. אבל המרפק עשוי להיות לא ברור, ותוצאה חזותית אינה הוכחה שהקבוצות אכן משמעותיות. בפרק 3.3 נלמד מדדי הערכה נוספים, כגון Silhouette, ונבדוק גם היגיון מקצועי, יציבות וקשרים לשאלת המחקר.

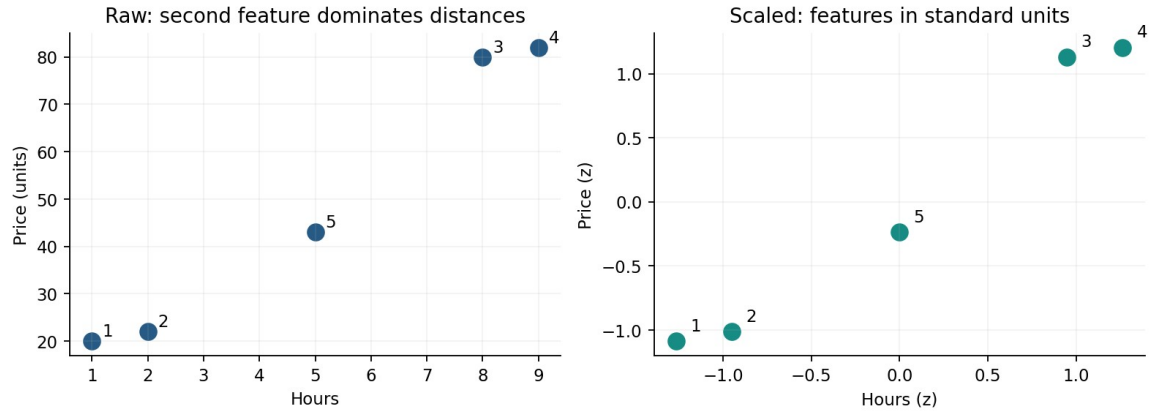


איור: משמאל: ירידת WCSS ככל שגדל K. מימין: לאותם נתונים אפשר להתאים מספרים שונים של מרכזים.

13. השפעת קנה המידה — הקשר לפרק 2.3

מרחק אוקלידי רגיש ליחידות. אם מאפיין אחד נמדד בטווח 1–10 ומאפיין אחר בטווח 100–10000, ייתכן שהשני ישלוט כמעט לבדו בחישוב המרחקים. האלגוריתם יכול להחזיר חלוקה שונה לחלוטין אחרי תקנון. שינוי התוצאה אינו תקלה; הוא מעיד שבחירת קנה המידה היא חלק מהגדרת השאלה המתמטית.

נניח שתי תכונות: שעות פעילות ומחיר. הבדל של 1 בשעות לעומת הבדל של 100 במחיר תורם לריבוע המרחק 1 לעומת 10000. לפני שמחשבים, צריך להחליט אם שתי התכונות צריכות לקבל משקל דומה או אם יש הצדקה מקצועית להעדיף אחת. StandardScaler יכול לשנות את קנה המידה, אבל אינו הופך את הקיבוץ ל"אובייקטיבי" או מבטל את הצורך בבחירה מודעת.



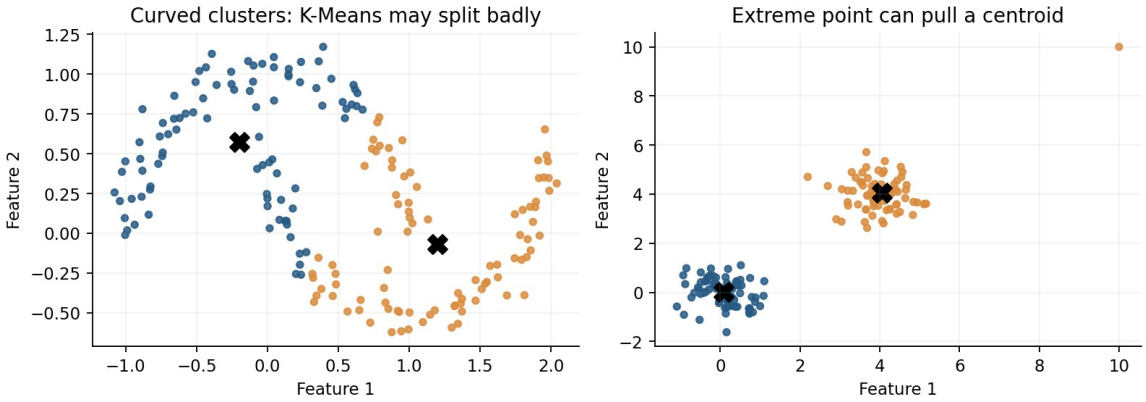
איור: אותן חמש תצפיות מוצגות במספרים המקוריים ואחרי תקנון. המרחקים והמרכזים תלויים בסולם הצירים.

14. מתי K-Means מתקשה? צורות, חריגים ומשמעות

K-Means מתאים במיוחד למצבים שבהם הקבוצות יחסית קומפקטיות במרחק אוקלידי, ולא גדולות מאוד בצפיפות ובפיזור באופן שונה. הוא אינו פועל היטב בכל דפוס. אם יש קבוצות מעוקלות כגון שני חצאי ירח,

קיבוץ לפי קרבה למרכז עלול לחתוך את הצורות באופן שאינו תואם את מבנה הנתונים. גם קבוצה קטנה מאוד לצד קבוצה גדולה או צפופה יכולה להיות מאתגרת.

ערך חריג רחוק מאוד מושך את הממוצע מפני שהממוצע רגיש לערכים קיצוניים, וריבוע המרחק מעניק להם משקל גדול. לכן לפני בחירת K-Means יש לבדוק תרשימי פיזור, מאפיינים חסרים, חריגים ויחידות מדידה. אין למחוק חריג אוטומטית; אולי זו התצפית החשובה ביותר לצורך המחקר.



איור: שתי מגבלות מרכזיות: קבוצות מעוקלות וערך חריג שמושך מרכז קבוצה. הסימנים השחורים מציינים מרכזים.

15. K-Means אינו מסווג ומספרי הקבוצות אינם שמות אמיתיים

נניח שהמודל החזיר עבור תלמיד מסוים cluster=0. אין משמעות טבעית למספר 0: בהרצה אחרת אותה קבוצה יכולה לקבל את המספר 1. המספר הוא מזהה פנימי בלבד. בניגוד ללמידה מונחית, כאן אין תשובת אמת מוגדרת מראש להשוואה. לכן אין לפרש cluster=0 כ״מצליח״ ו־cluster=1 כ״מתקשה״ ללא מחקר עצמאי, הגדרות מקצועיות ומודעות למגבלות.

גם אם קבוצות מתאימות חזותית לשני אזורים, נדרש שלב אנושי: בדיקת מאפיינים, התפלגויות, דוגמאות מייצגות ומידת היציבות. כשעובדים עם אנשים יש להימנע מהסקה על אישיות או יכולות מתוך כמה מדדים, לשמור פרטיות, לבדוק הטיות ולא להשתמש בקבוצה כהחלטה גורלית ללא בקרה מקצועית.

16. נממש את החישוב הידני ב־NumPy — בלי ספריית ML

לפני שמשתמשים ב־KMeans המוכן, נתרגם את הרעיון לארבע שורות מרכזיות: נחשב מרחקים בריבוע בין כל נקודה לכל מרכז, נבחר לכל נקודה את המרכז הקרוב, נחשב ממוצע עבור כל קבוצה ונחזור. בדוגמה הקטנה אין קבוצות ריקות ולכן הקוד פשוט. במימוש כללי יותר חייבים לטפל גם במקרה שבו לא שויכה שום נקודה לאחד המרכזים.

```
import numpy as np

X = np.array([[1,1], [1,3], [3,1],
              [7,7], [7,9], [9,7]], dtype=float)
centers = np.array([[1,1], [3,1]], dtype=float)

for iteration in range(10):
    squared = ((X[:, None, :] - centers[None, :, :]) ** 2).sum(axis=2)
    labels = squared.argmin(axis=1)
    updated = np.array([X[labels == j].mean(axis=0) for j in range(2)])
    print(iteration + 1, updated, labels.tolist())
    if np.allclose(centers, updated):
        break
    centers = updated
```

נסביר את התחביר: X[:, None, :] מוסיף ציר עזר לכל נקודה כדי שנוכל להשוות אותה בבת אחת לכל המרכזים. לאחר חיסור הווקטורים, מעלים בריבוע ומסכמים לאורך ציר המאפיינים. מטריצת squared

שהתקבלה מכילה שש שורות ושתי עמודות: בכל שורה שני ריבועי מרחקים. argmin(axis=1) מחזיר את מספר העמודה בעלת הערך הקטן ביותר, כלומר מספר המרכז הקרוב.

שורת updated יוצרת בכל פעם מרכז אחד כממוצע של כל הנקודות שקיבלו את מספר הקבוצה j. הבדיקה allclose מאפשרת לעצור כאשר המרכזים כמעט לא השתנו. בקוד זה ההנחה היא ששתי הקבוצות אינן ריקות. זו דוגמה לימודית, לא מימוש תעשייתי לכל סוגי הנתונים.

17. מימוש מקצועי בספריית scikit-learn

לאחר שהבנו את הפעולה באופן ידני, נעבור לכלי מקצועי. הספרייה מממשת קיבוץ בצורה יעילה יותר ומאפשרת אתחולים חוזרים, בחירת K וקבלת מדדי תוצאה. מומלץ להתחיל בדיוק באותו המאגר כדי לוודא שהפקודות אינן ״קסם״: הן מבצעות את אותו עיקרון מתמטי של שיוך ועדכון.

```
import numpy as np
from sklearn.cluster import KMeans

X = np.array([[1,1], [1,3], [3,1],
              [7,7], [7,9], [9,7]], dtype=float)

model = KMeans(n_clusters=2, random_state=42, n_init=10)
labels = model.fit_predict(X)

print("centers:\n", model.cluster_centers_)
print("labels:", labels)
print("inertia:", model.inertia_)
```

הפעולה fit מתאימה את המרכזים לנתונים. הפעולה fit_predict מחזירה את מזהי הקבוצות של תצפיות האימון. cluster_centers_ מחזיר מטריצה שבה כל שורה היא מרכז קבוצה שנלמד. labels_ מכיל את מזהי הקבוצות של נתוני האימון, ו־inertia_ הוא סכום ריבועי המרחקים שלהן אל המרכזים. סדר המרכזים ומספרי הקבוצות עשויים להשתנות; יש להשוות לפי מיקום המרכזים, לא לפי מזהה 0 או 1.

אפשר לשייך גם נקודה חדשה באמצעות model.predict([[2,2]]). במקרה כזה K-Means אינו מאמן מחדש את המרכזים: הוא משייך את הנקודה החדשה אל המרכז הקיים הקרוב. אם מבקשים לשנות את המרכזים לפי כל הנתונים החדשים, נדרש תהליך התאמה מחדש או שיטת אימון מתאימה.

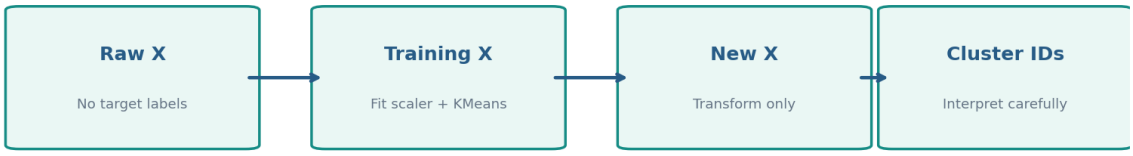
18. הכנת נתונים נכונה — מה קורה ב־Train וב־Test?

בפרויקטים של למידת מכונה מקובל לבדוק אם פתרון עובד גם על נתונים חדשים. אף שקיבוץ הוא למידה לא מונחית, גם בו חשוב להבחין בין התאמה של פעולות עיבוד ואלגוריתם לנתונים לבין יישום של התוצאה על תצפיות חדשות. אם מחלקים את הנתונים לצורכי בדיקת הכללה, יש ללמוד את ממוצעי התקנון ואת המרכזים רק על קבוצת האימון, ולהחיל אותם על קבוצת הבדיקה בלי להתאים אותם מחדש.

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans

pipeline = make_pipeline(
    StandardScaler(),
    KMeans(n_clusters=2, n_init=10, random_state=42)
)
pipeline.fit(X_train)       # learn scaler + cluster centers on train
new_labels = pipeline.predict(X_test)  # apply without re-fitting
```

הקוד מראה תבנית כללית; המשתנים X_train ו־X_test נוצרים בשלב פיצול הנתונים בפרויקט ואינם מוגדרים בדוגמה הקטנה למעלה. המטרה כאן היא להבין את עקרון מניעת זליגת המידע, לא לדלג על שלב הכנת הנתונים. הערכת קיבוץ אינה מחייבת תמיד חלוקה קבועה ל־Train/Test, אך כשמנסים לטעון שהקבוצות מועילות לנתונים חדשים או לשילוב במערכת חיזוי, צריך תכנון הערכה מתאים.



איור: בתהליך יישומי אפשר ללמוד את התקנון ואת מרכזי הקיבוץ בנתוני אימון, ולהחיל את אותו תהליך על נתונים חדשים.

19. פרויקט משולב — הצעת קבוצות שימוש למערכת המלצות

נניח שדודו בונה אפליקציה שמציעה לתלמידים פעילות העשרה. הוא מודד משך שימוש באפליקציה, מספר פעילויות שנצפו ומספר משימות שהושלמו. אין עמודת ״סוג משתמש״ ידועה מראש. הוא יכול לבדוק אם K-Means יוצר קבוצות של דפוסי שימוש, אך לפני האימון חייב להגדיר שאלת חקר: האם קיימים דפוסי שימוש בעלי יציבות וערך לצורך שיפור ממשק האפליקציה?

לאחר קיבוץ, יש לבדוק מספר פריטים בכל קבוצה, פיזור, מאפיינים טיפוסיים ודוגמאות שנראות לא מתאימות. את הקבוצות לא מציגים למשתמשים כקביעות על יכולת או אישיות, אלא כאפשרויות עזר שנבדקות. אם מציעים פעילויות, יש לתת למשתמש אפשרות לבחור אחרת ולבדוק אם ההמלצות אכן מועילות.

יומן תכנון לפרויקט יכלול: מקור הנתונים והרשאות השימוש, משמעות כל עמודה, ניקוי וטיפול בחסרים, שיטת שינוי קנה המידה, ערכי K שנבדקו, הגדרת random_state ו־n_init, תוצאה ונימוק מדוע החלוקה שימושית או אינה שימושית. כך נבנה הרגל עבודה שילווה את פרויקט הגמר עד להצגה בפני בוחן.

20. תרגול מדורג — רמה א׳: הבנה וחישוב בסיסי

א1. הבנת מונחים

נסחו במילים שלכם את משמעות המושגים תצפית, מאפיין, K, centroid, assignment ו־iteration. הסבירו מדוע מספר הקבוצה אינו תווית אמת.

א2. מרחקים

נתונות הנקודות P=(2,3), Q=(5,7) ומרכזים C_1=(1,3), C_2=(6,6). חשבו מרחק אוקלידי של P ושל Q לכל מרכז, והכריעו לאיזו קבוצה כל נקודה תשויך.

א3. עדכון מרכז

נתונות שלוש נקודות בקבוצה אחת: (2,2), (4,2), (3,5). חשבו את המרכז החדש באופן ידני ופירשו את שני רכיביו.

א4. קריאת גרף

הביטו באיור המציג ארבעה שלבים של K-Means. תארו מה השתנה בין שני השלבים העליונים ומה השתנה בין שני השלבים התחתונים.

21. תרגול מדורג — רמה ב׳: ניתוח אלגוריתם

ב1. איטרציה מלאה

נתונות ארבע נקודות (0,0), (0,2), (6,6), (6,8), עם K=2 ומרכזים התחלתיים (0,0) ו־(6,8). בצעו שיוך, עדכון מרכזים, שיוך נוסף ובדיקת עצירה. הסבירו כל החלטה.

ב2. השוואת פונקציות מטרה

הסבירו מדוע עדכון מרכז כממוצע מתאים למזעור סכום ריבועי המרחקים. האם היה נכון להחליף את המרחק האוקלידי בריבוע שלו רק בשלב השיוך? נמקו.

ב3. קנה מידה

לשני לקוחות מאפיינים: גיל והוצאה שנתית. האם הייתם מריצים K-Means על הערכים המקוריים? תארו בדיקה אחת לפני קיבוץ, ואת הסיכון בשימוש בתקנון ללא הצדקה מקצועית.

ב4. בחירת K

הסבירו מדוע inertia תמיד יכולה לרדת כשמוסיפים מרכזים, ובכל זאת אין לבחור K השווה למספר התצפיות.

22. תרגול מדורג — רמה ג׳: חשיבה ביקורתית והקשר ל־AI

ג1. איתור כשל

חוקר אימן K-Means על ביקורי אתר וקבע ש״קבוצה 0 היא קבוצת המשתמשים המצליחים״. הצביעו על לפחות שלוש בעיות אפשריות במסקנה זו והציעו בדיקות נדרשות.

ג2. בחירה בין מודלים

בקובץ נתונים שתי קבוצות נראות כשני חצאי ירח. הסבירו מדוע K-Means עשוי להיכשל בתיאורן, ומה הייתם בוחנים לפני בחירת אלגוריתם אחר.

ג3. תכנון ניסוי

כתבו מתווה ניסוי שמשווה K=2,K=3,K=4, תוך שמירה על אותה הכנת נתונים ועל שקיפות לגבי מקריות האתחול. אילו פרטים תתעדו ולמה?

ג4. תכנון רכיב פרויקט

הציעו תכונת אפליקציה שבה קיבוץ נתונים יכול לעזור, הגדירו מאפיינים אפשריים והסבירו כיצד תמנעו הסקת מסקנות לא הוגנת על משתמשים.

23. שאלות מקוריות במבנה בגרות — ללא פתרון בחוברת התלמיד

השאלות הבאות נכתבו במיוחד לספר הזה כתרגול בסגנון בחינה. הן אינן שאלות רשמיות של משרד החינוך, ומטרתן לבחון הבנה, חישוב, נימוק וניתוח טעויות — לא רק זיכרון של פקודות.

שאלה 1 — מרכזים ונקודות (20 נקודות מוצעות)

נתונות חמש נקודות: A=(0,0), B=(0,2), C=(4,4), D=(4,6), E=(6,4). נבחר K=2 עם מרכזים התחלתיים (0,0) ו־(4,4). (א) חשבו עבור כל נקודה את ריבוע המרחק לשני המרכזים (8 נקודות). (ב) שייכו כל נקודה והסבירו את החלטתכם (4 נקודות). (ג) חשבו שני מרכזים חדשים (6 נקודות). (ד) הסבירו כיצד תבדקו אם נדרשת איטרציה נוספת (2 נקודות).

שאלה 2 — בדיקת קוד (20 נקודות מוצעות)

ניתן קטע קוד שבו המרחקים מחושבים כ־((2**(X[:,None,:]-centers[None,:,:])).sum(axis=2). הסבירו מה משמעות צירי X, centers והתוצאה; מה מחזיר argmin; ומה יקרה אם ישונה axis.

שאלה 3 — בחירת K ותקנון (20 נקודות מוצעות)

באותו מאגר שני מאפיינים: גיל בטווח 15–18 והוצאה חודשית בטווח 100–3000. הרצת K-Means עם K=2 נתנה שיוכים שונים לפני StandardScaler ואחריו. (א) נמקו מדוע תוצאה כזו אפשרית (8 נקודות). (ב) הציגו יתרון אחד ומגבלה אחת של התקנון (6 נקודות). (ג) הציעו שתי בדיקות המראות אם החלוקה מועילה לשאלת המחקר (6 נקודות).

שאלה 4 — ביקורת תוצאה (20 נקודות מוצעות)

בדוח נכתב: ״הרצנו K-Means עם K=5 והשגנו דיוק של 96% כי inertia ירדה״. הסבירו מה שגוי במשפט, מה מודדת inertia ומה יידרש כדי להעריך את הקבוצות באופן משמעותי.

24. טעויות נפוצות — כיצד נמנע אותן?

טעות נפוצה	מדוע היא שגויה / מה עושים במקום
מניחים שמספר הקבוצה הוא מזהה טכני	מספר הקבוצה הוא תווית אמת; צריך לפרש תוצאות בנפרד.
מחשבים את המרכז כחציון ללא הנמקה	K-Means במרחק אוקלידי בריבוע מעדכן מרכז כממוצע רכיבי.
מדלגים על שלב השיוך מחדש	שיוך יכול להשתנות אחרי תזוזת מרכזים; יש לחשב שוב.
בוחרים K לפי inertia נמוכה בלבד	inertia יורדת בדרך כלל עם הגדלת K; צריך גם פרשנות ומדדים נוספים.
משווים מספרי cluster בין שתי ריצות	מזהי הקבוצות שרירותיים; משווים לפי מרכזים ומאפיינים.
מתעלמים מקנה מידה ומערכים חריגים	מרחק וממוצע רגישים לסקאלה ולחריגים; צריך לבדוק ולנמק.
מתייחסים להתכנסות כפתרון מושלם	K-Means עשוי להתכנס לפתרון מקומי ולהחמיץ מבנה שאינו כדורי.
מתייחסים ל־inertia כאחוז הצלחה	זה סכום ריבועי מרחקים, לא Accuracy ולא ציון אחוזי.

25. סיכום הפרק — מה אתם יודעים עכשיו?

K-Means מחפש K מרכזים ומחלק את הנתונים לפי קרבת כל נקודה למרכז. מתחילים ממספר מרכזים ואתחול כלשהו. לאחר שיוך הנקודות למרכזים הקרובים, מחשבים לכל קבוצה מרכז חדש כממוצע רכיבי התצפיות. חוזרים על הפעולות עד שהתהליך מתייצב או מתקיים תנאי עצירה.

האלגוריתם שואף להקטין את WCSS, אך התאמה מתמטית טובה אינה מבטיחה פרשנות נכונה. לפני השימוש ב־K-Means צריך לבחון מאפיינים, תקנון, חריגים, צורת קבוצות, בחירת K ומגבלות היישום. במחקר על אנשים, פירוש אחראי ופרטיות חשובים לא פחות מהקוד.

מה היא מתארת	נוסחה
ריבוע המרחק האוקלידי בין נקודה למרכז.	$d^2(x,c) = \Sigma_i (x_i - c_i)^2$
עדכון מרכז קבוצה לממוצע רכיבי הנקודות.	$C_j = (1/\lvert S_j \rvert)\Sigma_{x \in S_j} x$

מה היא מתארת	נוסחה
סכום ריבועי המרחקים בתוך הקבוצות.	$J = \Sigma_j \Sigma_{x \in S_j} \|x - C_j\|^2$
n תצפיות, p מאפיינים מספריים.	$shape(X) = (n,p)$

לפני שממשיכים: בדקו שאתם מסוגלים לבצע ביד איטרציה אחת, להסביר למה מרכז מתעדכן, לחשב J לקבוצה אחת, לפרש cluster_centers_ ו־inertia_ ולהציג סיבה אחת שבגללה התוצאה עשויה להיות מטעה.

26. הגשר לפרק 3.3 — כיצד בודקים קיבוץ?

אחרי שמצאנו קבוצות נותרת השאלה החשובה: האם הקיבוץ טוב? האם $K=2$ מתאים יותר מ־$K=3$? האם הקבוצות באמת מובחנות? האם הן יציבות כאשר משנים קצת את הנתונים או את נקודת ההתחלה? בפרק 3.3 נעסוק בהערכת קיבוץ, במדדי איכות כגון WCSS ו־Silhouette, ובשילוב חוות דעת מקצועית עם תוצאות חישוביות. נבדוק גם מצב שבו לקיבוץ מתקבל מספר יפה אך אין לו משמעות מעשית.

27. מקורות וייחוס

עוגן התכנית: יחידה 5, פרק 3 — קיבוץ נתונים, סעיף 3.2 K-Means, לפי מפת התכנית וחומרי ההוראה שבמאגר Data-Science-ML-Course. ניסוח דרישות החובה וההתאמה לשאלון הסופי מחייבים הצלבה עם מסמך משרד החינוך התקף; הדוגמאות המספריות, הגרפים והשאלות כאן מקוריים.

ספרי עיון להעמקה: Mathematics for Machine Learning (Deisenroth, Faisal & Ong); An Introduction to Statistical Learning with Applications in Python (James ואחרים); Hands-On Machine Learning (Géron); The StatQuest Illustrated Guide to Machine Learning (Starmer). כל הדוגמאות, הטבלאות והאיורים כאן מקוריים.

חומר מקדים בספר שלנו: 1.1 וקטורים; 1.2 מרחק ודמיון; 2.1 מרכז ופיזור; 2.3 תקנון ונרמול; 2.5 קריאת גרפים; 3.1 מבוא ללמידה לא מונחית. המשך: 3.3 הערכת קיבוץ. קשר למסלול למידה עמוקה: חקר נתונים והכנת ייצוגי מאפיינים במסגרת פיתוח מודלים ופרויקטים; קיבוץ אינו מחליף את אימון הרשת העצבית הנדרש במסלול ההתמחות.

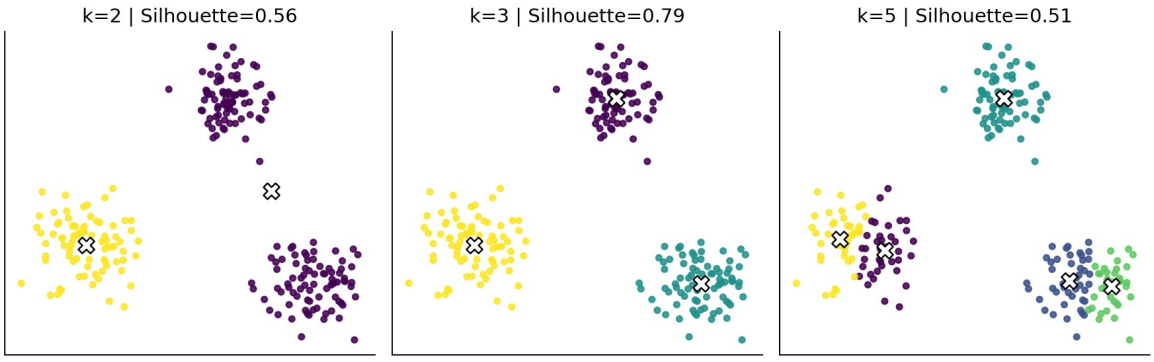
ספר הלימוד המאוחד לבינה מלאכותית

פרק 3.3

כיצד יודעים שקיבוץ נתונים הוא טוב?

Clustering Evaluation — הערכת קיבוץ נתונים

מהדורת תלמיד מורחבת | מבוססת על סעיף 3.3 במפת נושאי הליבה של יחידה 5 | הרחבות מובחנות להכנה לפרויקט גמר



השוואה חזותית: אותו מערך נתונים מחולק למספרי קבוצות שונים. המספר לבדו אינו מלמד על איכות ההחלטה.

שאלת הפתיחה של הפרק: אם אלגוריתם חילק 240 תלמידים לשלוש קבוצות, כיצד נוכל לבדוק האם החלוקה טובה — כאשר איננו יודעים מראש מהי ״התשובה הנכונה״?

הפרק נכתב כתוכן לימודי עצמאי בעברית. הנוסחאות והעקרונות המדעיים מבוססים על מקורות מקובלים; כל נתוני ההדגמה, התרגילים והתרשימים נבנו עבור הספר.

מה נלמד בפרק?

בפרק 3.2 למדנו כיצד K-Means מחלק תצפיות לקבוצות ומחשב מרכז לכל קבוצה. כאן נמשיך לשאלה חשובה לא פחות: מהי ראיה לכך שהחלוקה שהתקבלה שימושית? במערכת אמת אין די בכך שהקוד הסתיים ללא שגיאה. עלינו לבדוק מה המדד מודד, כיצד הוא משתנה כאשר מספר הקבוצות משתנה, והאם המספרים תואמים למטרה שלשמה נאספו הנתונים.

הוכחה ללמידה	יכולת בסיום הפרק
חישוב ידני של מרחקים בריבוע ממרכזי קבוצות	**לחשב WCSS**
להבחין בין שיפור מתמטי לבחירה סבירה של k	**לפרש עקומת מרפק**
ממוצע מרחק בתוך קבוצה, ממוצע לקבוצה אחרת ומקדם	**לחשב Silhouette לנקודה**
ניתוח גודל קבוצות, חריגים, יציבות והקשר עסקי	**להעריך השוואות**
השוואת 2–6 קבוצות עם seed ותיעוד מדדים	**לכתוב ניסוי Python**
המלצה עם עדויות, הסתייגויות ומגבלות	**לנסח מסקנה זהירה**

ידע קודם — מה נחזור לפני שמתחילים?

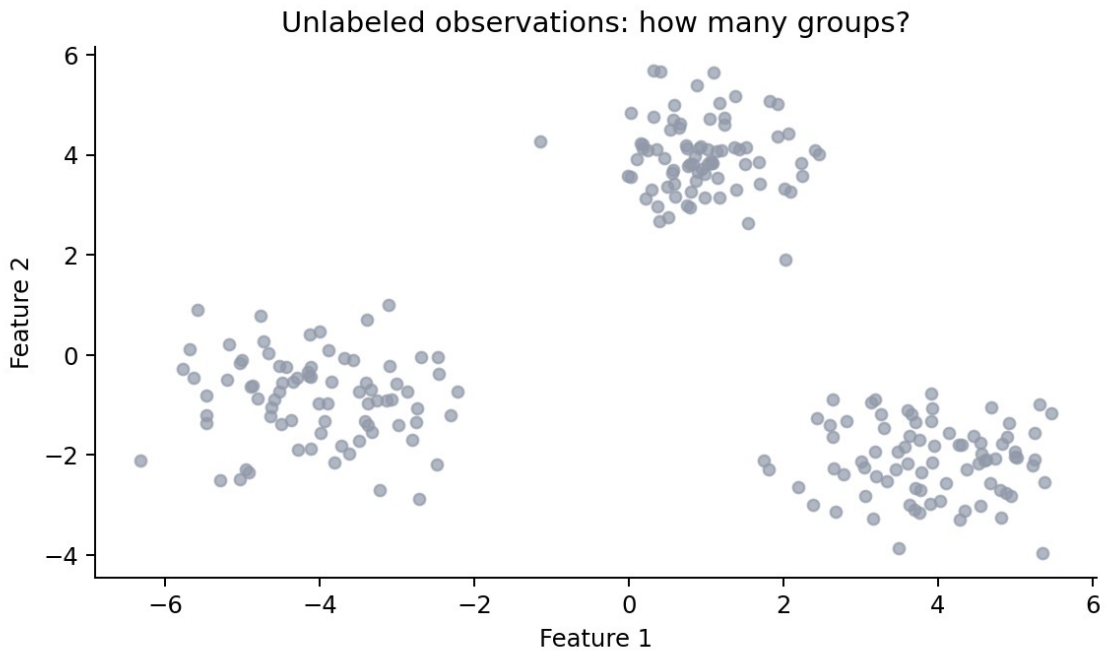
התלמיד צריך להכיר וקטורים, מרחק אוקלידי, ממוצע רכיבי, תקנון ו־K-Means. אין צורך ללמוד נגזרות או לבצע חישוב סמלי מורכב. בכל זאת, נשתמש בסימן סכימה Σ כדי לרשום בקיצור סכום של כמה חישובים פשוטים.

חשבו והסבירו: אם לקבוצה יש נקודות 1, 2, 3, מה המרכז שלה? מהו המרחק של הנקודה 3 מהמרכז?

1. הבעיה: קיבוץ אינו חיזוי עם תשובה ידועה

דודו מפתח אתר המציע לתלמידים פעילויות לאחר הלימודים. לכל תלמיד הוא מודד שני מאפיינים: שעות השתתפות בפעילות יצירתית ושעות השתתפות בספורט. K-Means יכול לחלק את התלמידים לקבוצות לפי דמיון. אולם אין במאגר עמודה שאומרת ״קבוצה נכונה״. לכן לא נוכל לחשב שיעור הצלחה כמו בסיווג רגיל, שבו משווים ניבוי לתווית אמיתית.

במקום זאת נשתמש במדדים פנימיים (Internal Validation): הם בוחנים את הסידור הגיאומטרי של הנקודות ואת הקשרים בין קבוצות. מדדים אלה חשובים, אך אינם מחליפים שאלה חינוכית או עסקית: האם החלוקה מסייעת לתכנן פעילויות מותאמות?



נתונים ללא תוויות: אפשר לזהות בעין עננים, אבל לפני חישוב איננו יודעים איזו חלוקה תהיה המתאימה ביותר.

החלוקה הרצויה עשויה להיות תלויה במטרה. עבור חלוקת כיתות, גודל קבוצות מאוזן חשוב. עבור זיהוי צרכים נדירים, קבוצה קטנה עשויה להיות דווקא החשובה ביותר. לכן שום מדד יחיד אינו מספיק.

2. מהי קבוצה טובה? שלושה מבחנים

באינטואיציה, קבוצה טובה מאגדת נקודות דומות זו לזו, שונה ככל האפשר מקבוצות אחרות, ומתאימה למשימה האמיתית. שתי התכונות הראשונות ניתנות למדידה מספרית, ואילו השלישית מחייבת פרשנות אנושית של הנתונים והקשרם.

- לכידות (Cohesion): כמה קרובות הנקודות בתוך אותה קבוצה?
- הפרדה (Separation): כמה רחוקות הקבוצות זו מזו?
- שימושיות ויציבות: האם הקבוצה בעלת משמעות? האם החלוקה דומה גם כאשר משנים מעט את הנתונים או את נקודת האתחול?

מדד WCSS מתמקד בעיקר בלכידות. מדד Silhouette בוחן גם את הקרבה בתוך הקבוצה וגם את המרחק מקבוצות אחרות. בהמשך נראה מדוע הם עלולים להמליץ על מספר שונה של קבוצות, ומדוע אין בכך סתירה.

3. מדד ראשון — WCSS ו־Inertia

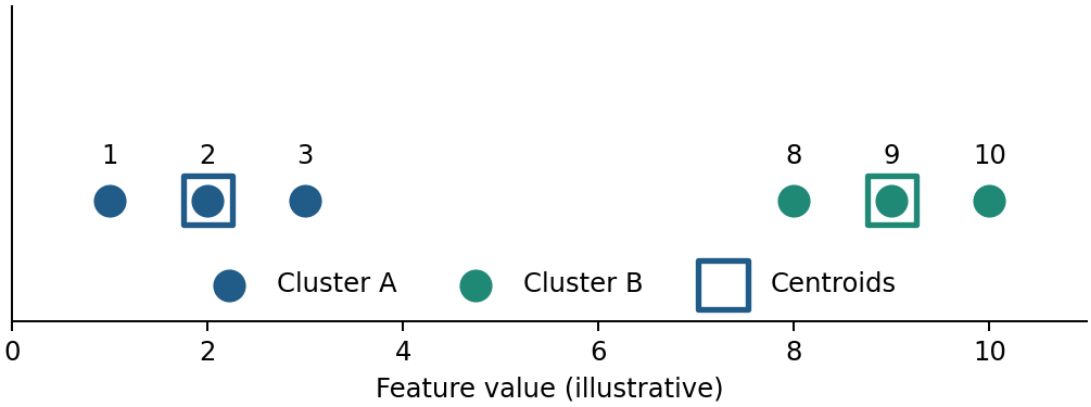
WCSS הוא קיצור של Within-Cluster Sum of Squares: סכום ריבועי המרחקים של כל תצפית ממרכז הקבוצה שלה. ב־K-Means עם מרחק אוקלידי זהו גודל מרכזי שהאלגוריתם מבקש לצמצם. בספריית scikit-learn נקבל אותו לאחר אימון מתוך המאפיין inertia_.

$$\mathbf{WCSS = \Sigma_j \Sigma_{x \in C_j} \|x - \mu_j\|_2^2}$$

פירוש הסימנים: C_j היא הקבוצה שמספרה j; x היא תצפית בקבוצה; μ_j הוא המרכז של אותה קבוצה; $\|x-\mu_j\|_2$ הוא המרחק האוקלידי למרכז. מרבעים את המרחק כדי לקבל ערך לא־שלילי ולהעניק משקל גדול יותר למרחקים גדולים, ומחברים על כל הנקודות בכל הקבוצות.

3.1 חישוב ידני של WCSS — שש תצפיות

נתחיל ממאגר חד־ממדי פשוט, כדי להבין כל פעולה בלי להסתבך עם קואורדינטות רבות. שש המדידות הממוינות הן 1, 2, 3, 8, 9, 10. נניח שקבוצת A היא 1, 2, 3 וקבוצת B היא 8, 9, 10. מרכז A הוא 2, ומרכז B הוא 9.



שש תצפיות ושני מרכזים: הריבועים החלולים מסמנים את המרכזים 2 ו־9.

תצפית	מרכז הקבוצה	מרחק	מרחק בריבוע
1	2	1	1
2	2	0	0
3	2	1	1
8	9	1	1
9	9	0	0
10	9	1	1

$$\mathbf{WCSS = 1 + 0 + 1 + 1 + 0 + 1 = 4}$$

המספר 4 אינו ״אחוז הצלחה״. זוהי כמות בסכום יחידות המאפיין בריבוע. אילו מדדנו את אותו מאפיין ביחידות גדולות פי 10, ערך WCSS היה גדל פי 100. לכן אי אפשר להשוות ללא מחשבה ערכי WCSS שחושבו על מאגרים או קני מידה שונים.

חשבו והסבירו: אם מעבירים את התצפית 3 לקבוצה המרוחקת, האם WCSS בהכרח יקטן? הסבירו מה צריך לחשב מחדש לפני שקובעים.

4. מדוע WCSS יורד ככל שמגדילים את k?

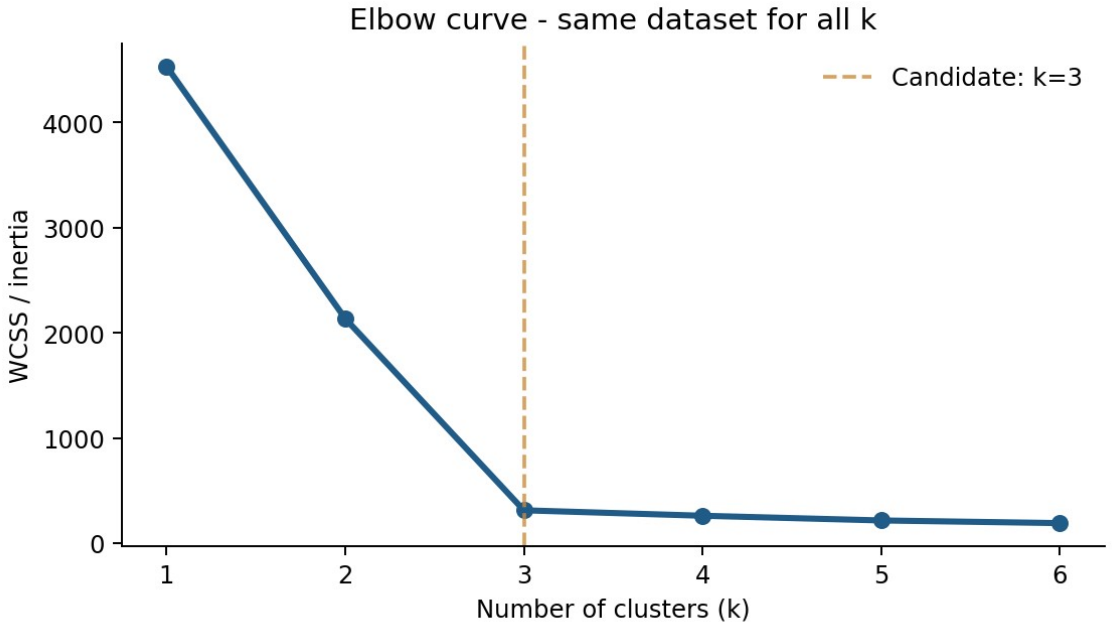
כאשר מספר הקבוצות k גדל, אנו נותנים למודל יותר מרכזים. הוא יכול להצמיד כל נקודה למרכז קרוב יותר או להשאיר חלוקה שאינה טובה פחות. לכן, אם מחפשים את הפתרון המיטבי עבור כל k, ה־WCSS המינימלי אינו גדל כאשר k גדל. בהרצות מעשיות של K-Means עשויות להיות סטיות קלות בגלל אתחולים מקומיים או הגדרות שונות.

במקרה הקיצוני שבו לכל אחת מ־n הנקודות מרכז משלה, אפשר להגיע ל־WCSS אפס — אך לא למדנו כמעט דבר מעניין על הקבוצות. התוצאה הזו מלמדת מדוע ״בחר את ה־WCSS הקטן ביותר״ היא המלצה שגויה כאשר גם מספר הקבוצות משתנה.

בדוגמת שש התצפיות, אם נגדיר k=6, לכל תצפית תהיה קבוצה משלה ו־WCSS=0. האם שש קבוצות שימושיות יותר משתי קבוצות? לא בהכרח. זה בדיוק המחיר של מודל מורכב מדי עבור המטרה.

5. שיטת המרפק (Elbow Method)

שיטת המרפק בוחנת את האיזון בין איכות ההתאמה לבין מספר הקבוצות. מריצים K-Means עבור מספר ערכי k, מתעדים את WCSS ומציגים על גרף. מחפשים אזור שבו עד נקודה מסוימת נרשמת ירידה חדה, ומאותה נקודה הירידה נעשית מתונה. האזור הזה מכונה ״מרפק״.



גרף מרפק המבוסס על נתונים סימולטיביים. הקו המקווקו מציין מועמד אפשרי; אין כלל הקובע שכל גרף יכיל מרפק ברור.

חשוב: שיטת המרפק אינה נוסחה שמחזירה תמיד תשובה חד־משמעית. לעתים העקומה חלקה, ולעתים מתגלים כמה ״מרפקים״ סבירים. הבחירה חייבת להתחשב גם בגודל הקבוצות, בהפרדה ביניהן ובשאלה לשמה נבנה המודל.

5.1 איך קוראים את הגרף באופן מקצועי?

- בציר האופקי מוצג k — מספר הקבוצות; בציר האנכי מופיע WCSS.
- בוחנים מה התועלת בהוספת קבוצה אחת: כמה פחת WCSS, ולא רק מהו ערכו הנמוך ביותר.
- מחפשים שינוי בקצב הירידה. תוספת של קבוצות רבות תמורת שיפור קטן עלולה להיות מיותרת.
- בודקים את החלוקה בפועל: האם קבוצה אחת מכילה רק תצפית בודדת? האם האשכולות צפופים או חופפים?

חשבו והסבירו: תארו מצב שבו דווקא מספר קבוצות גדול יותר מהמרפק עשוי להיות מוצדק מבחינת המשתמש במערכת.

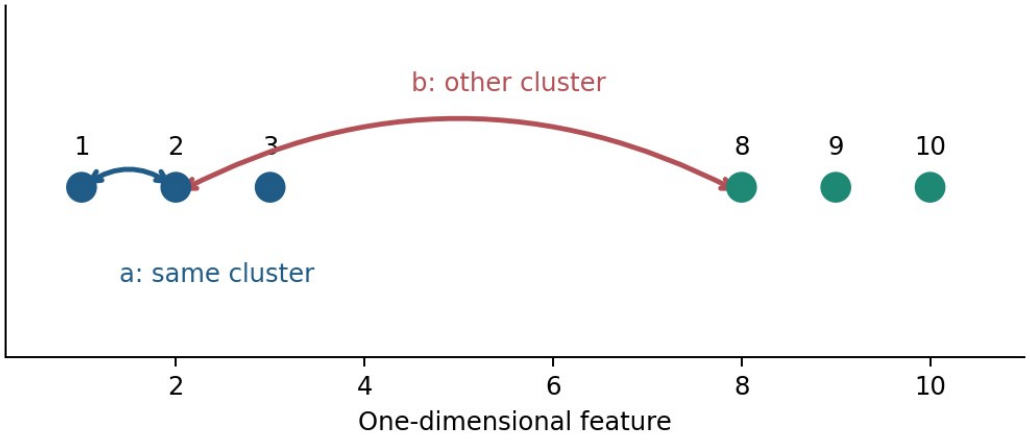
6. מדד Silhouette — שתי שאלות לגבי כל תצפית

מדד Silhouette נותן לכל תצפית ציון המתאר את מידת ההתאמה שלה לקבוצה שאליה שובצה, ביחס לקבוצה חלופית. במקום להסתכל רק על המרחק למרכז, הוא משווה ממוצעי מרחקים בין תצפיות. בכך הוא מוסיף מידע ש־WCSS לבדו אינו מספק.

נגדיר a(x) כממוצע המרחקים מן התצפית x אל שאר התצפיות באותה קבוצה, ללא התצפית עצמה. נגדיר b(x) כממוצע המרחקים של x אל התצפיות בקבוצה אחרת הקרובה אליה ביותר במובן זה. כשיש יותר משתי קבוצות מחשבים לכל קבוצה חלופית ממוצע ובוחרים את הקטן ביותר.

$$s(x) = [b(x) - a(x)] / \max\{a(x), b(x)\}$$

כאן s(x) הוא מקדם הצללית של התצפית. אם a קטן מאוד ו־b גדול מאוד, התצפית נמצאת קרוב לחבריה ורחוק מאחרים: s חיובי וגבוה. אם a ≈ b, התצפית עומדת כמעט על גבול בין קבוצות: s קרוב לאפס. אם a>b, היא קרובה בממוצע יותר לקבוצה אחרת: s שלילי.



ההבדל בין a, הקרבה לחברים באותה קבוצה, לבין b, הקרבה הממוצעת לקבוצה חלופית.

7. חישוב Silhouette ביד — דוגמה מלאה

נמשיך עם אותן שש תצפיות כדי לשמור על רצף ולמנוע בלבול. הקבוצות הן A={1,2,3}, B={8,9,10}. נתבונן בתצפית x=2 שנמצאת בקבוצה A. כל המרחקים כאן הם מרחקים מוחלטים על ציר אחד.

שלב 1 — מחשבים ממוצע מרחקים לחברי אותה קבוצה: המרחק בין 2 ל־1 הוא 1 והמרחק בין 2 ל־3 הוא 1. איננו כוללים את המרחק בין 2 לעצמה.

$$a(2) = (|2-1| + |2-3|) / 2 = (1+1)/2 = 1$$

שלב 2 — מחשבים ממוצע מרחקים לקבוצה האחרת: המרחקים בין 2 לבין 8, 9, 10 הם 6, 7, 8. ממוצעם הוא 7.

$$b(2) = (6+7+8) / 3 = 7$$

שלב 3 — מציבים בנוסחת הצללית. במכנה נבחר את הגדול מבין 1 ובין 7, כלומר 7. לכן התוצאה 6/7.

$$s(2) = (7-1) / \max(1,7) = 6/7 \approx 0.8571$$

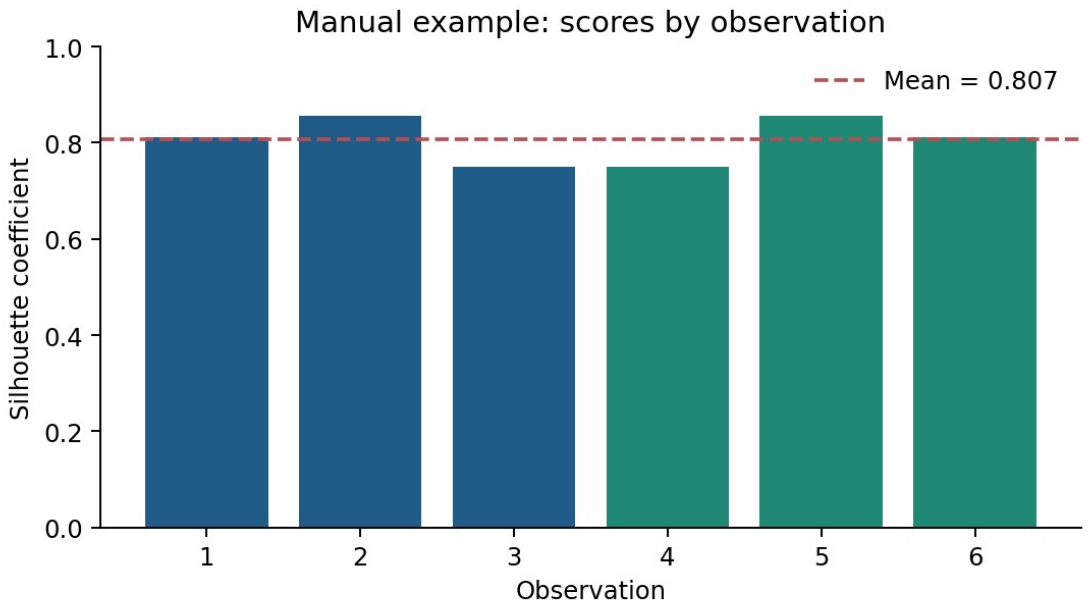
פירוש: התצפית 2 נמצאת בתוך קבוצה צפופה יחסית, ורחוקה בממוצע מן הקבוצה האחרת. זהו ערך חיובי גבוה, אך לא ״הסתברות של 85.71%״ ולא שיעור הצלחה. הוא מדד מבני המתאר את איכות השיוך של נקודה מסוימת.

7.1 אותה נוסחה — כל שש התצפיות

x	a(x)	b(x)	s(x) בקירוב
1	1.5	8	0.8125
2	1	7	0.8571
3	1.5	6	0.7500
8	1.5	6	0.7500
9	1	7	0.8571
10	1.5	8	0.8125

mean(s) ≈ 0.8065

ציון הצללית הכללי הוא ממוצע מקדמי הצללית של כלל התצפיות. בדוגמה זו הוא בקירוב 0.8065. יתרונו בכך שהוא נותן סיכום אחד, אך החיסרון הוא שממוצע עלול להסתיר קבוצה קטנה עם שיוכים גרועים. לכן נבחן גם ערכי נקודות וקבוצות בנפרד.



מקדם *Silhouette* של כל אחת משש התצפיות והקו של ממוצע הקיבוץ.

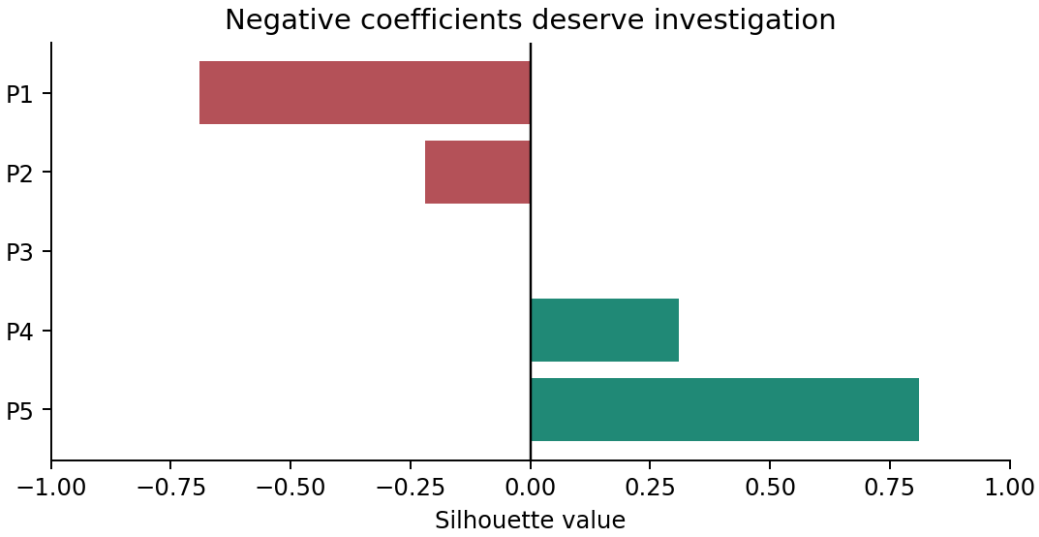
חשבו והסבירו: למה בבדיקה של תצפית 2 אין לכלול אותה בחישוב $a(2)$? מה יקרה לערך a אם נוסיף בטעות מרחק אפס?

8. מה המשמעות של מקדם שלילי?

נבחן תצפית $x=8$ ששובצה בטעות לקבוצה $A=\{1,2,8\}$, כאשר הקבוצה החלופית $B=\{9,10,11\}$. המרחקים שלה לחברי קבוצתה הם 7 ו־6, לכן $a=6.5$. המרחקים שלה לקבוצה האחרת הם 1, 2, 3, ולכן $b=2$.

s(8) = (2−6.5) / 6.5 ≈ −0.6923

מקדם שלילי הוא סימן אזהרה: הנקודה קרובה בממוצע לחברי קבוצה אחרת יותר מאשר לחברי הקבוצה שאליה שויכה. עם זאת, הוא אינו מוכיח שהנתון שגוי או שיש למחוק אותו. ייתכן שקבוצות חופפות, ייתכן שנבחר מרחק לא מתאים, וייתכן שהבעיה כלל אינה מתאימה לקיבוץ מבוסס מרחק.



מחוונים שליליים מחייבים בדיקה; הם אינם הוראת מחיקה אוטומטית.

9. גבולות המדד והמצבים המיוחדים

המקדם של תצפית נמצא בתחום שבין מינוס אחת לאחת. ערך גבוה מעיד על הפרדה טובה ביחס לממוצעי המרחקים; אפס מעיד על קרבה דומה לקבוצה הנוכחית ולחלופית; ערך שלילי עשוי להצביע על שיוך בעייתי. אבל גם הערך 1 אינו הבטחה שהחלוקה מייצגת ״קבוצות אמיתיות״.

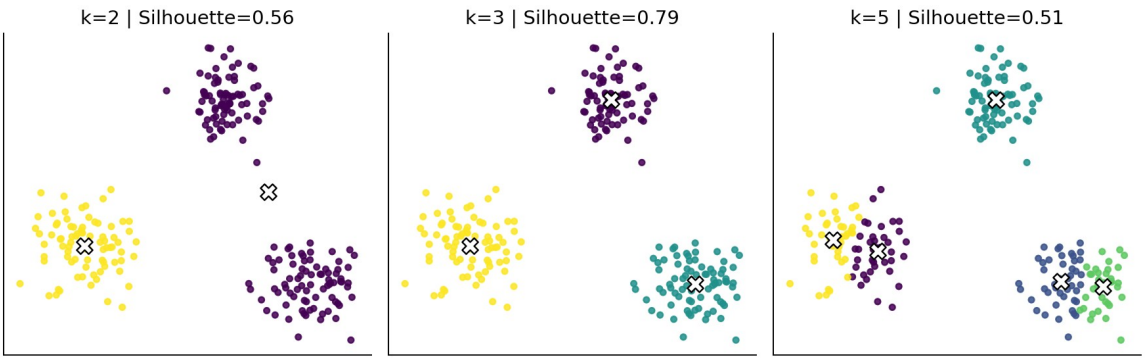
אי אפשר לחשב Silhouette עבור $k=1$, משום שאין קבוצה חלופית שממנה אפשר לחשב b. גם כשהקיבוץ יוצר לכל אחת מן הנקודות קבוצה משלה, לא מתקיים התנאי הדרוש לחישוב מדד ממוצע רגיל. ב־scikit-learn המדד מוגדר כאשר מספר הקבוצות השונות הוא בין 2 לבין מספר הדוגמאות פחות אחת.

לתצפית שנמצאת לבדה בקבוצתה, a אינו מוגדר כממוצע על נקודות אחרות. המימוש המקובל בספריית scikit-learn מגדיר את ציון הצללית שלה כ־0. יש לזכור את ההסכמה הזו כשמנתחים קבוצות קטנות מאוד.

חשבו והסבירו: אם $k=1$, מדוע אפשר עדיין לחשב WCSS אבל אי אפשר לחשב Silhouette?

10. כיצד בוחרים k בעזרת Silhouette?

מריצים את האלגוריתם עבור כמה ערכי k שמקיימים את תנאי החישוב. עבור כל k מחשבים ציון Silhouette ממוצע ומשווים. לעתים הערך הגבוה ביותר הוא מועמד טוב, אך אין להפסיק בו את הניתוח: כדאי לבדוק גם האם כל קבוצה משמעותית, האם קיימות קבוצות זעירות והאם השינויים באתחול יוצרים תוצאות שונות.



השוואת מספר קבוצות באותו מאגר: הצבעים הם מזהי קבוצות פנימיים בלבד, וה־X מציין מרכז.

בחירה ב־k יכולה להיות שונה בין שיטת המרפק לבין Silhouette, מפני שהראשונה מודדת צמצום פיזור סביב מרכזים והשנייה משלבת לכידות והפרדה. אין מדד שמחזיק בכל המידע הרלוונטי, ולכן דרושה החלטה מנומקת.

11. טבלת השוואה — מה כל מדד אומר?

מאפיין	WCSS / Inertia	Silhouette
מה נמדד?	סכום ריבועי מרחקים למרכזים	לכידות יחסית להפרדה מהקבוצה החלופית
כיוון מועדף	נמוך יותר עבור k קבוע	גבוה יותר בדרך כלל
האם אפשר $k=1$?	כן	לא
מה קורה כשמגדילים k?	המינימום אינו עולה	עשוי לעלות או לרדת
השוואה בין מאגרים?	לא ישירות בגלל יחידות וקנה מידה	גם היא דורשת זהירות לגבי מרחק, צפיפות והתפלגות
סיכון פרשני	k גדול מדי יכול להקטין מאוד WCSS	ממוצע גבוה עשוי להסתיר קבוצות קטנות או צורות לא כדוריות

התלמיד נדרש להסביר בכל שאלה מה המדד מחשב ומה הוא אינו מחשב. שימוש בשם של ספרייה או במספר פלט ללא פרשנות אינו מהווה מסקנה מדעית.

12. פירוש גרף Silhouette

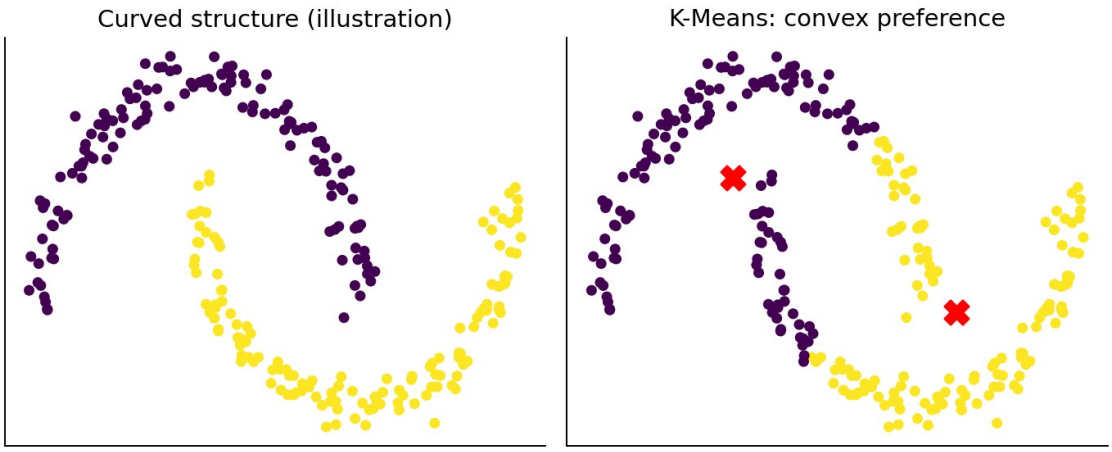
מלבד ממוצע המדד, ניתן להציג תרשים המפרט את מקדמי הצללית של כל תצפית, מסודרים בתוך קבוצות. תרשים כזה יכול לגלות קבוצה שבה רוב הנקודות מקבלות מקדמים נמוכים או שליליים, גם כאשר הממוצע הכללי נראה סביר. בעת קריאה יש לבחון את מספר התצפיות בכל קבוצה ואת רוחב פסי הצללית.

נקודה שלילית בודדת אינה תמיד בעיה. לעומת זאת קבוצה שלמה בעלת ציונים נמוכים או שליליים מחייבת בחינה של מספר הקבוצות, בחירת המאפיינים וסולם המרחק. כך ההערכה עוברת מן השאלה ״מה יצא בפלט?״ לשאלה ״מה התוצאה אומרת על הנתונים?״

13. איכות הקיבוץ אינה זהה ליופיו של הגרף

קיבוץ שמופיע יפה בגרף דו־ממדי עשוי להיות מטעה כאשר הנתונים האמיתיים כוללים עשרות תכונות. בחירת שני מאפיינים בלבד להצגה אינה מציגה את כל המרחב. גם השלכת נתונים למישור באמצעות שיטת צמצום ממדיות יכולה לשנות את צורת הקרבה החזותית. לכן נחשב את המדדים על ייצוג הנתונים שעליו אומן המודל, ונשתמש בגרף כהמחשה נוספת בלבד.

מנגד, מדדים המבוססים על מרחק אינם בהכרח מתאימים לצורות קבוצות שונות. למשל, שתי קבוצות בצורת ירח עלולות להיראות מובחנות לעין, אך K-Means נוטה ליצור חלוקה סביב מרכזים, ולא בהכרח לעקוב אחרי צורה מעוקלת.



דוגמה שבה מבנה מעוקל ומבנה סביב מרכז אינם אותו דבר. זו המחשה למגבלות גיאומטריות של *K-Means*.

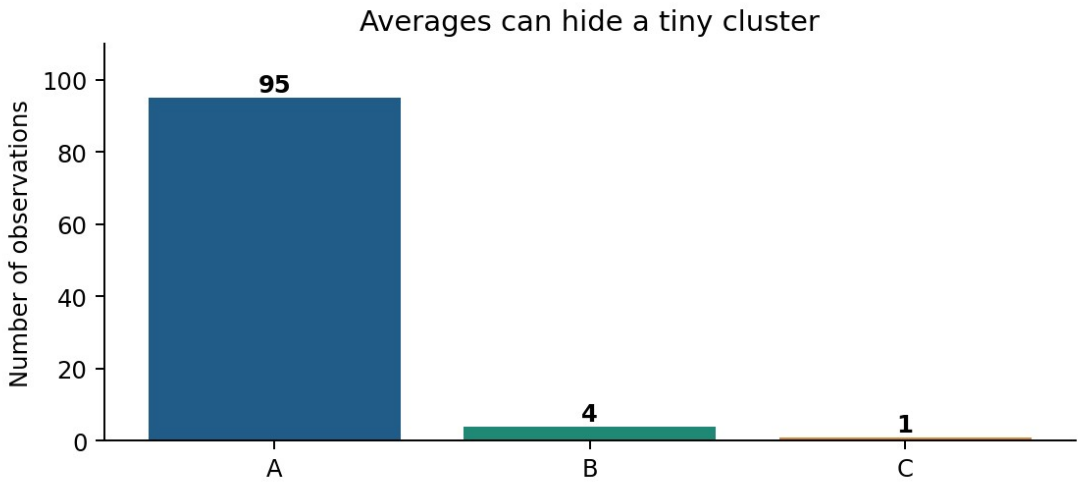
14. מספר הקבוצות אינו תחליף לשאלה מקצועית

אם דודו מקים שלוש קבוצות פעילות לאחר בית הספר, ייתכן שיש לו שלושה חדרים פנויים ולכן הוא יעדיף $k=3$ גם אם $k=4$ נותן מדד מעט גבוה יותר. הבחירה אינה ״התעלמות מהמתמטיקה״, אלא שילוב של תוצאה סטטיסטית עם אילוצים אמיתיים. עליו לתעד שהבחירה נובעת ממגבלת מקום ולא לייחס אותה למסקנה של האלגוריתם.

כמו כן, אסור להסיק ש״קבוצה 0״ היא קבוצת ״התלמידים החלשים״ רק מפני שמרכז הקבוצה נמוך באחד המאפיינים. שמות הקבוצות הם מזהים טכניים, והמשמעות החברתית או החינוכית חייבת להיבדק בזהירות ובאחריות.

15. קבוצות קטנות, חריגים ויציבות

ממוצע יכול להסתיר חלוקה לא מאוזנת. נניח שאלגוריתם יצר שלוש קבוצות בגדלים 95, 4 ו־1. ציון כללי טוב אינו פוטר אותנו מלבדוק את התצפית הבודדת. לפני שמתכננים המלצה אוטומטית נברר האם מדובר בנתון חריג, שגיאת מדידה או תת־קבוצה נדירה בעלת חשיבות.



גם כאשר יש שלוש קבוצות, גדלים שונים מאוד מחייבים בדיקה נפרדת.

גם ההתחלה משפיעה: K-Means יכול להתכנס לפתרונות שונים לפי בחירת מרכזים התחלתיים. לכן בהשוואת k יש להשתמש בפרוטוקול קבוע, למשל נתונים מעובדים זהים, מספר אתחולים מספק ו־random_state לצורך שחזור; אין משמעות לאיכות מדד אם שינינו במקביל את הנתונים ואת שיטת החישוב בלי לציין זאת.

16. תפקיד התקנון — קשר ישיר לפרק 2.3

כאשר תכונה אחת נעה בטווח של אלפי יחידות ותכונה אחרת בטווח של ערכים בודדים, מרחק אוקלידי עלול להיות מושפע בעיקר מהתכונה הראשונה. שינוי קנה המידה משנה גם את המרכזים, גם את WCSS וגם את Silhouette. אין להשוות אותם כאילו מדובר באותה בעיה מתמטית. לפעמים שינוי קנה מידה מוצדק ולפעמים יחידות המדידה המקוריות חשובות מקצועית — והחלטה זו חייבת להיות מנומקת.

בפרויקט שבו קיימים Train ו־Test לצורך משימה מונחית בהמשך, יש להתאים את התקנון על נתוני האימון בלבד. בפרויקט קיבוץ חקרני ללא מבחן עתידי, נגדיר במפורש מהי יחידת הניתוח ונשמור את כל הגדרות העיבוד לצורך שחזור.

17. מימוש ב־Python — מדדים ופלט

כעת לאחר שהבנו את ההיגיון המספרי נציג קוד מקצועי. הקוד שלפניכם הוא דוגמת הסבר פתורה בספר; המחברת העצמאית המצורפת מכילה מאגר אחר ומשימות שהתלמיד משלים בעצמו.

```
import numpy as np
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

X = np.array([[1.], [2.], [3.], [8.], [9.], [10.]])
model = KMeans(n_clusters=2, n_init=15, random_state=42)
labels = model.fit_predict(X)

print("Centers:", model.cluster_centers_.ravel())
print("WCSS:", model.inertia_)
print("Silhouette:", round(silhouette_score(X, labels), 4))
```

במימוש זה K-Means מאומן על שישה ערכים חד־ממדיים; fit_predict מחזירה מזהה קבוצה לכל תצפית; cluster_centers_ מחזירה את המרכזים; inertia_ היא WCSS; והפונקציה silhouette_score מקבלת את

הנתונים ואת שיוכי הקבוצות. סדר המספרים של הקבוצות עשוי להתחלף בין ריצות בלי לשנות את איכות החלוקה.

חשוב להבחין בין silhouette_score שמחזירה ממוצע לבין silhouette_samples שמחזירה ציון נפרד לכל תצפית. בפרויקט עם הרבה נקודות חישוב זוגות מרחקים עשוי להיות יקר; אפשר להשתמש בפרמטר sample_size בהנחה שמדווחים שבוצעה הערכה מדגמית.

18. השוואת k בניסוי חוזר

בניסוי מסודר משווים מספר קבוצות באותו מאגר מעובד, ובשיטה אחידה. אין לחשב Silhouette עבור k=1 או עבור k השווה למספר הדוגמאות. ראוי לעיין גם בגודל הקבוצות ובתרשימי הפיזור כדי שלא להסתפק במספר יחיד.

```
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

for k in range(2, 6):
    model = KMeans(n_clusters=k, n_init=15, random_state=42)
    labels = model.fit_predict(X)
    print(k, round(model.inertia_, 2),
          round(silhouette_score(X, labels), 3))
```

הקוד מתאים למסגרת רעיונית, אך במאגר שיש בו שש תצפיות בלבד בחירה של חמש קבוצות יוצרת קבוצות בודדות ואינה בהכרח משמעותית. בחקר אמיתי נבחר טווח k לפי מספר התצפיות, צפיפות הנתונים והמשימה, ולא רק לפי האפשרות שהפקודה תחזיר תוצאה.

19. מדדים נוספים — הרחבה מקצועית ולא תחליף לליבה

בעבודה מתקדמת פוגשים גם את מדד Calinski–Harabasz, המעדיף יחס גבוה של פיזור בין קבוצות לפיזור בתוך קבוצות, ואת Davies–Bouldin, שבו ערך נמוך יותר מצביע בדרך כלל על קבוצות נפרדות וצפופות יותר. איננו דורשים כאן חישוב ידני שלהם. מטרת ההיכרות היא להבין שמדדים שונים מנסחים הגדרות שונות של קבוצה ״טובה״.

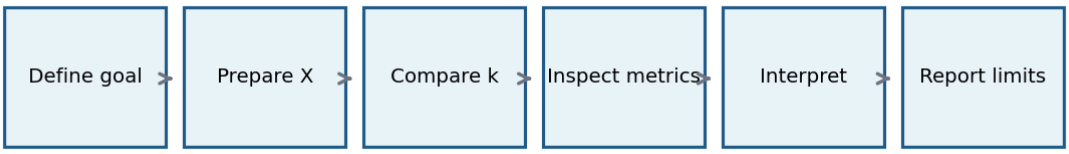
אין לפרש מספרים של מדדים שונים כאילו הם באותו סולם. למשל אין להשוות 0.6 ב־Silhouette ל־450 במדד Calinski–Harabasz ולקבוע שהשני ״טוב פי 750״. לכל מדד משמעות, נוסחה, טווח ומגבלות משלו.

```
from sklearn.metrics import calinski_harabasz_score
from sklearn.metrics import davies_bouldin_score

# Example after fitting a valid clustering model:
# print(calinski_harabasz_score(X, labels))
# print(davies_bouldin_score(X, labels))
```

20. כיצד מסכמים ניסוי בצורה מדעית?

בפרויקט גמר על התלמיד לנסח מה ניסה, מה מדד ומה משמעות המדדים. לא די בגרף מרפק או בתמונה צבעונית. התלמיד צריך להציג את אופן הכנת הנתונים, המאפיינים שנבחרו, ערכי k שנבדקו, מדדי ההשוואה, נימוק הבחירה, דיון במגבלות והמלצה להמשך.



רצף עבודה מדעי: הגדרת מטרה, בחינת הנתונים, ניסוי, מדדים, פרשנות ודיווח מגבלות.

20.1 דוגמה לניסוח מסקנה זהירה

״בדקנו כמה חלוקות של תלמידים לפי שני מאפייני פעילות. בשיטת המרפק נראה ששלוש קבוצות הן אפשרות סבירה, וגם ערך Silhouette הממוצע היה גבוה יחסית לחלופות שבדקנו. אולם לפני שנמליץ על פעילויות נבדוק את גודל הקבוצות ואת מאפייניהן, ונערוך בדיקה חוזרת על נתונים נוספים.״

ניסוח כזה עדיף על הטענה ״המודל מצא את שלוש הקבוצות האמיתיות״, משום שהמדדים מודדים מבנה מספרי ולא משמעות חינוכית מוכחת.

21. שאלות לדוגמה בסגנון בגרות — רמה מדורגת

רמה א׳ — ידע ומיומנות

חשבו והסבירו: נתונות הקבוצות $A=\{0,2,4\}$ ו־$B=\{10,12\}$. חשבו את מרכזי הקבוצות ואת WCSS עבור החלוקה הנתונה. הציגו כל שלב.

חשבו והסבירו: הסבירו במילים שלכם מדוע הערך הנמוך ביותר של WCSS אינו כלל מספק לבחירת k.

חשבו והסבירו: מהי משמעות ערכי Silhouette הקרובים ל־1, ל־0 ול־−1?

רמה ב׳ — הבנה וניתוח

חשבו והסבירו: תלמיד השווה קיבוצים עם $k=2,3,4$ וקיבל ערכי WCSS של 70, 42, 33. מה אפשר ללמוד? מה אי אפשר לקבוע בלי מידע נוסף?

חשבו והסבירו: עבור x מסוים מתקבל $a=5$ ו־$b=3$. חשבו את $s(x)$ ופירשו את סימנו.

חשבו והסבירו: מדוע חיוני לדווח על המאפיינים ועל שיטת התקנון יחד עם תוצאות קיבוץ?

רמה ג׳ — חשיבה ביקורתית ופרויקט

חשבו והסבירו: שני ניסויי K-Means נתנו אותו Silhouette ממוצע, אך בניסוי הראשון אחת הקבוצות מכילה נקודה אחת בלבד. כיצד תבדקו איזה ניסוי מתאים יותר לשימוש?

חשבו והסבירו: הציעו נימוק לבחירה ב־$k=4$ גם כאשר Silhouette ממוצע של $k=3$ מעט גבוה יותר. מה חייב להופיע בדוח כדי שהבחירה תהיה שקופה?

חשבו והסבירו: קיבוץ הצליח ליצור קבוצות צפופות אך נתן המלצות פעילות לא מועילות. הסבירו כיצד ייתכן דבר כזה.

22. טעויות נפוצות — נזהה ונמנע

מה חשוב להבין	טעות
WCSS אינו אחוז אלא סכום של ריבועי מרחקים	**WCSS קטן = אחוז הצלחה גבוה**

מה חשוב להבין	טעות
הגדלת k מקטינה את המינימום ולכן דרושה פשרה	**נבחר תמיד k עם WCSS מזערי**
זהו מדד יחס מרחקים בתחום [−1,1]	**Silhouette הוא הסתברות**
הוא מצביע על צורך בבדיקה, לא על שגיאה מוכחת	**מקדם שלילי מחייב מחיקת נתון**
מספרי קבוצות הם תוויות שרירותיות	**קבוצה 0 = קבוצה בעלת שם מהותי**
חייבים לבדוק קבוצות קטנות וערכי נקודות	**ציון ממוצע מייצג את כולם**
קנה המידה משנה את ערכי המרחק בריבוע	**מותר להשוות WCSS בין יחידות שונות**
גרף עשוי להסתיר ממדים או לעוות את תחושת המרחק	**קבוצות נראות ברורות בגרף דו־ממדי**

23. מילון מושגים

מושג	פירוש קצר
Cluster	קבוצה של תצפיות שהאלגוריתם שייך יחד
Centroid	וקטור המרכז של קבוצה
Cohesion	לכידות: קרבת חברי קבוצה זה לזה
Separation	הפרדה בין קבוצות
WCSS / Inertia	סכום ריבועי מרחקים מהמרכזים
Elbow Method	חיפוש התמתנות בקצב ירידת WCSS
Silhouette	מדד איכות שיוך לפי מרחקים בתוך קבוצה וביחס לחלופית
k	מספר הקבוצות שנקבע לניסוי
Internal Validation	הערכת קיבוץ ללא שימוש בתוויות אמת
random_state	בקרה על רכיב האקראיות לשחזור הניסוי
Feature Scaling	שינוי קנה מידה של מאפיינים

24. המשך חיבור לספר ולפרויקט הגמר

בפרק 3.1 הכרנו למידה לא מונחית; בפרק 3.2 בנינו קיבוץ באמצעות K-Means; ובפרק הנוכחי למדנו לבדוק את איכותו. בהמשך הספר נעבור ללמידה מונחית, שבה קיימת תווית יעד ואפשר להשתמש במדדי סיווג. ההבחנה בין המדדים חשובה מאוד: ציון Silhouette גבוה אינו אומר שמודל יודע לחזות תוויות בהצלחה.

במסלול הלמידה העמוקה חשוב לדעת להעריך גם קבוצות תכונות או ייצוגים נלמדים, אך השימוש בהם אינו הופך את מדדי הקיבוץ לאותה הערכה כמו Accuracy של רשת עצבית. בפרויקט גמר אנו בוחרים מדד על פי הבעיה והמודל.

25. כרטיס יציאה — ללא מחשב

חשבו והסבירו: במשפט אחד: מה ההבדל בין WCSS ובין Silhouette?

חשבו והסבירו: מדוע Silhouette אינו מוגדר לקיבוץ בעל קבוצה אחת?

חשבו והסבירו: ציינו מגבלה אחת של שיטת המרפק ומגבלה אחת של Silhouette.

חשבו והסבירו: כתבו משפט אחד שמדווח על תוצאה ומבהיר גם את מגבלתה.

26. מקורות והבחנה בין חובה להעמקה

עוגן התוכן: תכנית נושאי ליבה בבינה מלאכותית, פרק 3 — קיבוץ נתונים, סעיף 3.3 ״הערכת קיבוץ נתונים״, לפי מפת הסילבוס השמורה במאגר הקורס. לפני הצגת פרטי בחינה כמחייבים, יש להשוות לנוסח הרשמי התקף לאותה שנת לימודים. שיטת המרפק, Silhouette, הרחבות ביקורתיות ומדדים נוספים מופיעים כאן ברמת פירוט פדגוגית שנקבעה לספר; אין להסיק שכל רכיב הרחבה הוא דרישת חובה של משרד החינוך.

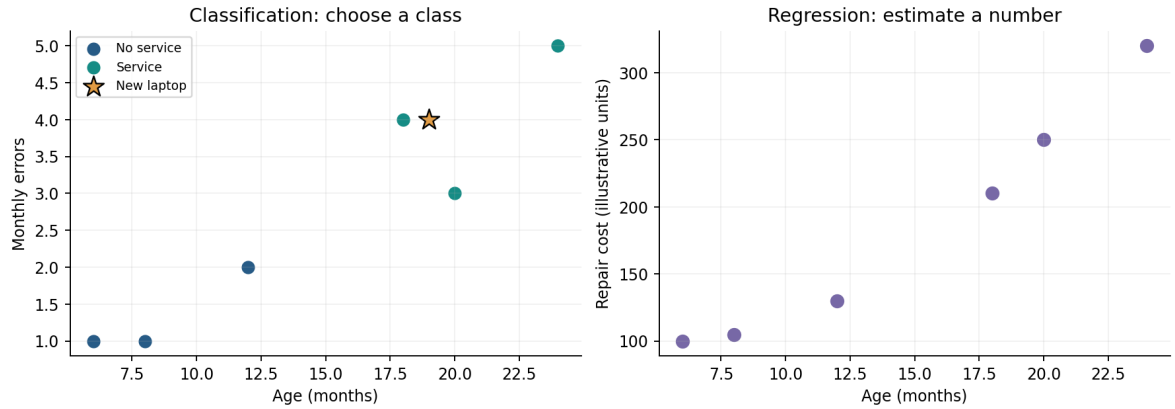
מקורות מקצועיים לעיון ולתכנון פדגוגי: An Introduction to Statistical Learning with Applications in Python (2023), פרק למידה לא מונחית; Hands-On Machine Learning, Aurélien Géron, פרקים העוסקים ב־Clustering; Mathematics for Machine Learning, Deisenroth, Faisal & Ong (2020), יסודות גיאומטריים ומתמטיים; StatQuest, Josh Starmer, גישת המחשה ופירוק הדרגתי; תיעוד scikit-learn הרשמי ל־Clustering ול־Silhouette Analysis.

קישורים: https://www.statlearning.com/ | https://scikit-learn.org/stable/modules/clustering.html | https://scikit-learn.org/stable/auto_examples/cluster/plot_kmeans_silhouette_analysis.html

כל ההסברים, החישובים המודגמים, הגרפים והתרגילים בספר זה נוסחו ונוצרו מחדש לצורכי הוראה. חישובי הדוגמה מאומתים מול NumPy ו־scikit-learn; ניסויי התלמיד נשארו פתוחים לעבודה עצמאית.

פרק 4.1 — מבוא ללמידה מונחית

איך מחשב לומד מדוגמאות שיש להן תשובה ידועה, ומתי אפשר לסמוך על התחזית שלו?



איור: שתי שאלות המבוססות על נתונים: האם תידרש בדיקת מחשב (סיווג), ומה תהיה עלות טיפול משוערת (רגרסיה). כל הנתונים בדוגמה מומצאים.

מהדורת תלמיד • הגדרות והסברים מדורגים • חישובים ידניים • 10 המחשות • NumPy / Pandas / scikit-learn • שאלות מקוריות לבגרות • מעבדה עצמאית

עד כה עסקנו בוקטורים, במרחקים, בסטטיסטיקה, בגרפים ובקיבוץ נתונים ללא תוויות. בפרק זה נעבור לשאלה שונה: מה קורה כאשר לצד כל דוגמה יש גם תשובה שאנחנו רוצים שהמחשב ילמד לחזות? כאן מתחילה הלמידה המונחית. נעבור בהדרגה מסיפור פשוט למונחים המדויקים, ואז נבדוק כיצד בונים תהליך למידה שאפשר להעריך בצורה אמינה.

מטרות הלמידה והידע הנדרש

ידע קודם: טבלת DataFrame, וקטור ומטריצה, מרחק אוקלידי, קריאת תרשים פיזור, ממוצע וסטיית תקן ופעולות Python בסיסיות. אין צורך להכיר עדיין את Nearest Centroid או KNN; הם יילמדו בפרקים הבאים.

- להבחין בין למידה מונחית, למידה לא מונחית ותכנות על פי כללים קבועים.
- לזהות תצפית, מאפיין, תווית, משתנה מטרה, X ו־y; ולבדוק אם מאפיין זמין בזמן החיזוי.
- להבחין בין סיווג, כולל סיווג בינארי ורב־מחלקתי, לבין רגרסיה המנבאת ערך מספרי.
- לתאר תהליך fitpredictevaluate ולהפריד בין Train, Validation ו־Test.
- לחשב ידנית תחזית על סמך דוגמה קרובה, ולפרש טעות, Accuracy ו־MAE ברמה מושגית.
- להבין הכללה, התאמת־יתר, התאמת־חסר, קו בסיס וזליגת מידע.
- להפעיל מודל בסיסי במחברת Colab, לבקר את התוצאה ולנסח מגבלה הנוגעת לפרויקט גמר.

לשם מה נשתמש בו עכשיו	מה למדנו
שורה בטבלה היא וקטור; מודל עשוי להשוות בינו לבין דוגמאות עם תווית.	פרקים 1.1–1.2 — וקטורים ומרחקים
יש להתאים את פעולת קנה המידה על נתוני אימון בלבד.	פרק 2.3 — תקנון ונרמול

לשם מה נשתמש בו עכשיו	מה למדנו
נבדוק אם למאפיינים יש קשר אפשרי למטרה, בלי לבלבל קשר עם סיבתיות.	פרקים 2.4–2.5 — קשרים וגרפים
בלמידה לא מונחית אין y ידוע; כעת y מלווה את נתוני האימון.	פרקים 3.1–3.3 — קיבוץ

1. סיפור פתיחה — דודו מנהל את מעבדת המחשבים

במעבדת המחשבים של בית הספר יש מחשבים ניידים בגילים שונים. דודו מנסה להיערך מראש לעומס התיקונים. הוא מחזיק רישום של מחשבים שנבדקו בעבר: גיל המחשב בחודשים, מספר התקלות שדווחו בחודש שלפני הבדיקה, והאם בסופו של דבר נדרש טיפול טכני. כל שורה מתארת מחשב שהיה מוכר בעבר; התוצאה שהתבררה לאחר בדיקה משמשת תווית.

דודו אינו רוצה לכתוב כלל נוקשה כגון "כל מחשב מעל 18 חודשים חייב טיפול". ייתכן שמחשב חדש יחסית חווה תקלות רבות, וייתכן שמחשב ותיק פועל היטב. לכן הוא רוצה להשתמש בדוגמאות קודמות כדי ללמוד קשר בין מאפיינים לבין תוצאה. זו בעיית סיווג בינארי — התוצאה היא אחת משתי אפשרויות: נדרש טיפול או לא נדרש טיפול.

מחשב	גיל (חודשים)	תקלות בחודש הקודם	האם נדרש טיפול?
A	6	1	לא (0)
B	12	2	לא (0)
C	18	4	כן (1)
D	24	5	כן (1)
E	8	1	לא (0)
F	20	3	כן (1)

המספרים מומצאים להוראה, ואינם מדגם מייצג. איננו בונים עליהם מערכת תחזית אמינה לבית ספר אמיתי. יתרונם הוא שאפשר לחשב ולבדוק כל שלב ביד לפני שעוברים למאגר גדול יותר.

2. שלוש גישות שונות לפתרון בעיה

בתכנות רגיל המפתח כותב במפורש את הכלל: אם תנאי מסוים מתקיים — בצע פעולה. המחשב מפעיל את הכלל אך אינו משנה אותו מתוך הדוגמאות. בלמידה מונחית אנו מספקים דוגמאות וערכי מטרה ידועים, והאלגוריתם מתאים מודל שמנסה לחזות את המטרה מדוגמאות חדשות. בלמידה לא מונחית אין תוויות מטרה בזמן האימון; מחפשים מבנה, קבוצות או ייצוגים בתוך המאפיינים.

דוגמה	מה מתקבל?	מה מספקים?	שיטה
אם נרשמו 5 תקלות, הצג התראה.	פלט שקובע המפתח	חוקי if/else	כללים כתובים
חיזוי האם יידרש טיפול.	מודל לחיזוי תווית	מאפיינים + תווית ידועה	למידה מונחית
חלוקת מחשבים לקבוצות שימוש.	קבוצות או מבנה נתונים	מאפיינים ללא תווית	למידה לא מונחית

נקודה עדינה: גם במערכת מונחית המפתח מחליט מהי בעיית החיזוי, אילו נתונים לאסוף ואיך למדוד הצלחה. האלגוריתם אינו "מחליט בעצמו" מהו הדבר החשוב לחזות. אחריות הגדרת הבעיה ובדיקת התוצאות נשארת בידי בני אדם.

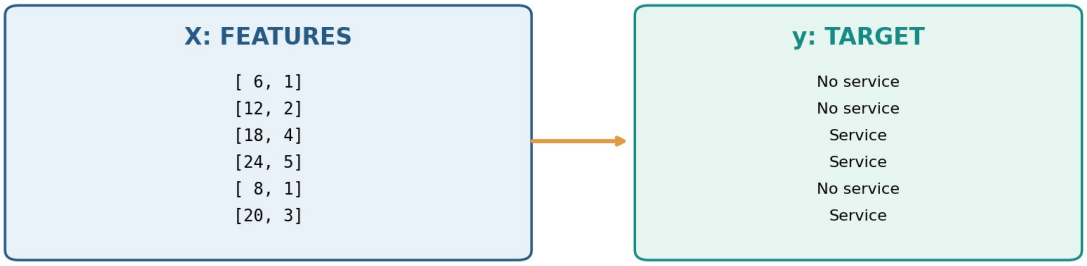
3. מהו המידע שהמודל מקבל? X, y, מאפיין ותווית

תצפית (Observation / Sample) היא דוגמה אחת, למשל מחשב A. מאפיין (Feature) הוא משתנה הזמין לפני קבלת ההחלטה, כגון גיל המחשב או מספר התקלות המדווחות. משתנה המטרה (Target) הוא

התוצאה שאנו מבקשים לחזות; בדוגמה זו הוא האם נדרש טיפול. כאשר התוצאה היא מחלקה, מקובל לכנות אותה גם Label — תווית.

אנו מסדרים את כל המאפיינים במטריצה X. כל שורה היא מחשב, וכל עמודה היא מאפיין. את התוויות שומרים בווקטור y בעל ערך אחד לכל שורה. בנתונים הקטנים שלנו ל־X יש שש שורות ושתי עמודות, ואילו ל־y יש שישה ערכים.

$$X \in \mathbb{R}^{6\times 2} \qquad y = [0, 0, 1, 1, 0, 1]$$



איור: אותן שש תצפיות, אבל שני תפקידים שונים: X משמש לתיאור המחשבים; y הוא התוצאה הידועה המלווה כל תצפית באימון.

מבחינת עיצוב הקוד, סדר השורות ב־X וב־y חייב להיות תואם. אם נשנה את סדר התוויות בלי לשנות בהתאם את סדר המאפיינים, נלמד קשרים שגויים. גם זהו סוג של שגיאת נתונים, ולא תקלה של הספרייה.

```
import numpy as np

X = np.array([[6, 1], [12, 2], [18, 4],
              [24, 5], [8, 1], [20, 3]])
y = np.array([0, 0, 1, 1, 0, 1])

print(X.shape)  # (6, 2)
print(y.shape)  # (6,)
```

4. סיווג לעומת רגרסיה — שתי שאלות שונות

סיווג (Classification) הוא חיזוי של מחלקה, כלומר ערך מתוך קבוצת אפשרויות. כאשר יש שתי אפשרויות בלבד מדובר בסיווג בינארי, כמו כן/לא; כאשר יש שלוש אפשרויות או יותר, למשל סוג תקלה: סוללה, מסך או מקלדת, מדובר בסיווג רב־מחלקתי. מספרים 0 ו־1 המייצגים מחלקות אינם כמו מדידה מספרית: הפער ביניהם אינו ״הפרש של יחידה״ במובן כמותי רגיל.

רגרסיה (Regression) מבקשת לחזות כמות מספרית, לדוגמה משך טיפול משוער בדקות או עלות תיקון בשקלים. שתי בעיות עשויות להשתמש באותם מאפיינים, אבל המטרה, המודל והדרך למדוד שגיאה יהיו שונים. אין להסיק שהמילה Regression בהכרח פירושה ״קו ישר״; רגרסיה היא סוג המשימה, וקיימים מודלים ליניאריים ולא־ליניאריים רבים.

השאלה	סוג המשימה	דוגמה ל־y	מדד מתאים בהמשך
האם נדרש טיפול?	סיווג בינארי	0 או 1	Accuracy, Recall ועוד
מהי קטגוריית התקלה?	סיווג רב־מחלקתי	סוללה / מסך / תוכנה	מטריצת בלבול ועוד
כמה דקות יימשך התיקון?	רגרסיה	35, 52, 120	MAE, MSE ועוד

שאלת הבנה: ״מה הסיכוי שתתרחש תקלה?״ היא אמנם שאלה שהתשובה לה מספרית בין 0 ל־1, אך היא מתייחסת להסתברות של מחלקה. יש להבחין בין חיזוי מחלקה, חיזוי הסתברות למחלקה וחיזוי גודל רציף כגון עלות.

5. מודל, למידה, אימון וחיזוי

מודל (Model) הוא מנגנון שמקבל קלט ומפיק תחזית. אנו יכולים לחשוב עליו כפונקציה: מכניסים וקטור תכונות x, ומקבלים תוצאה חזויה $\hat{y}$. בשלב האימון (Training), אלגוריתם הלמידה משתמש בדוגמאות הידועות כדי לקבוע פרמטרים או לשמור מידע על הדוגמאות, לפי סוג המודל. בשלב החיזוי (Prediction), המודל שהותאם מקבל מאפיינים של דוגמה חדשה ומחזיר תחזית.

$$\hat{y} = f(X\ ;\ \theta)$$

הסימון θ מייצג את מה שנלמד מן הנתונים, למשל מקדמים של מודל ליניארי. אין חובה שלכל אלגוריתם יהיה אותו סוג פרמטרים. מודל שכן קרוב מבוסס במידה רבה על שמירת דוגמאות האימון והשוואתן, ואילו מודל רגרסיה ליניארית מתאים מקדמים מספריים.

בספריית scikit-learn מקובל להשתמש ב־fit כדי להתאים מודל לנתוני האימון, ב־predict כדי להפיק תחזיות, ובפעולות הערכה כדי להשוות את התחזית לערכים האמיתיים. לכן חשוב להבחין בין "הרצתי predict" לבין "למדתי מודל": predict משתמש במודל שכבר עבר fit.



איור: תהליך עבודה טיפוסי: ניסוח שאלה, יצירת X ו־y, חלוקה, התאמה, חיזוי והערכת שגיאות.

6. חישוב ידני ראשון — חיזוי באמצעות הדוגמה הקרובה

לפני שנכיר את KNN באופן מלא בפרק 4.3, נשתמש בכלל פשוט כדי להמחיש איך תוויות מוכרות יכולות לסייע לחיזוי: נשווה מחשב חדש לדוגמאות האימון ונבחר את התווית של הדוגמה הקרובה ביותר. זהו מקרה פשוט של שיטת שכן קרוב אחד. המטרה כאן אינה ללמוד את האלגוריתם כולו, אלא להבין מדוע לאימון דרושות גם תוויות.

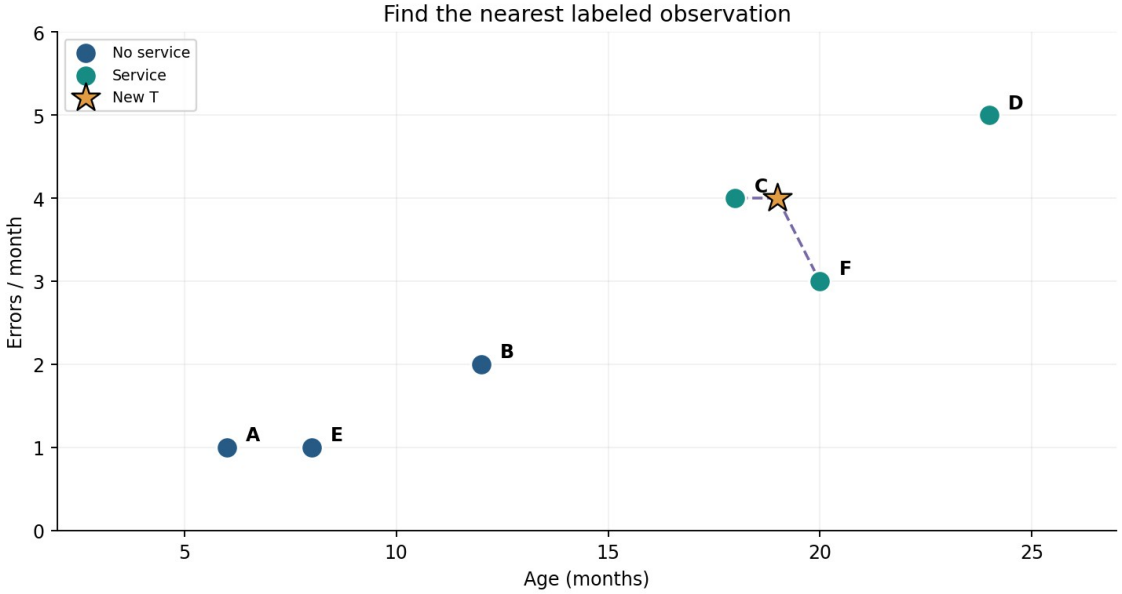
המחשב החדש T בן 19 חודשים ונרשמו לו 4 תקלות בחודש האחרון. לכן וקטור המאפיינים הוא $T=(19,4)$. נחשב מרחק אוקלידי בינו לבין כל דוגמה. כדי להקל על החישוב נשווה תחילה את ריבועי המרחקים.

$$d^2(T,A) = (19-6)^2 + (4-1)^2 = 178$$

$$d^2(T,C) = (19-18)^2 + (4-4)^2 = 1$$

$$d^2(T,F) = (19-20)^2 + (4-3)^2 = 2$$

מחשב אימון	מרחק בריבוע מ־T	התווית שלו
A	178	לא
B	53	לא
C	1	כן
D	26	כן
E	130	לא
F	2	כן



איור: המחשב החדש T מסומן בכוכב. התצפית הקרובה אליו ביותר היא C, שנושאת תווית ״נדרש טיפול״. המרחק אינו הופך את התחזית לוודאית.

לכן לפי הכלל הפשוט נחזה עבור T תווית 1, כלומר ״נדרש טיפול״. אבל אין לנו עדיין תשובת אמת לגבי T, ולכן איננו יודעים אם התחזית נכונה. ייתכן שגם יש גורמים חשובים שאינם מיוצגים בשני המאפיינים, כגון סוג החומרה או היסטוריית תיקונים. בנוסף, אם נמדוד גיל בשנים במקום בחודשים, המרחקים עשויים להשתנות; לכן נלמד בהמשך כיצד מניחים קנה מידה מתאים.

7. למה מחלקים את הנתונים ל־Train, Validation ו־Test?

דמיינו תלמיד שמתרגל עשר שאלות ומקבל עליהן פתרונות מלאים. אם ניתן לו שוב בדיוק אותן שאלות, ציון גבוה אינו מוכיח שידע לפתור שאלה חדשה. כך גם מודל: הצלחה על דוגמאות ששימשו לאימון אינה מעידה בהכרח על יכולת להכליל לדוגמאות שלא נראו קודם. לכן עלינו לשמור מידע להערכה נפרדת.

Train הוא אוסף הדוגמאות שעליו מתאימים את המודל. Validation משמש לבחירת הגדרות ולשיפור תהליך העבודה בלי לחשוף את סט הבדיקה הסופי. Test משמש לבדיקה סופית של המודל שנבחר, לאחר שנקבעו ההגדרות. בתרגילים פשוטים נתחיל לעיתים רק בחלוקת Train/Test, ובהמשך נרחיב לשיטות של אימות צולב (Cross-Validation).



איור: שלוש קבוצות נתונים בעלות תפקידים נפרדים. האיור סכמטי: היחסים המדויקים נקבעים לפי גודל המאגר וסוג הבעיה.

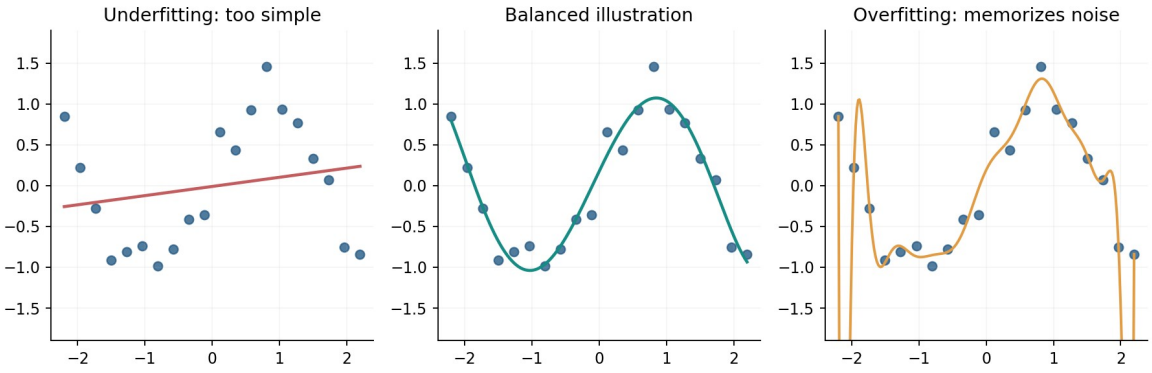
למשל, מתוך 100 מחשבים ניתן להקצות 80 לאימון ו־20 לבדיקה, אם אין אילוצים נוספים. אולם אין כלל שלפיו תמיד חייבים בדיוק 80/20. במאגר קטן כדאי להיזהר משיעור מדדים לא יציב; בנתונים על פני זמן יש להימנע מחלוקה שמאפשרת לעתיד לדלוף לעבר; ובתצפיות חוזרות מאותו מחשב צריך לדאוג שלא יופיע מידע כמעט זהה בשתי הקבוצות.

בסיווג, לעיתים נרצה לשמר בקירוב את שיעור המחלקות בכל קבוצה. פעולה זו נקראת Stratification, ומוגדרת למשל בפרמטר stratify=y. היא אינה פותרת בעיות של זליגת מידע או של תלות בין תצפיות, אך עשויה להפחית את הסיכון לכך שקבוצת הבדיקה תכיל רק מחלקה אחת.

8. מהי הכללה (Generalization)?

הכללה היא יכולתו של מודל להפיק תחזיות מועילות על נתונים חדשים מאותה בעיה, בתנאים המתאימים. אין די בכך שהמודל זוכר היטב את נתוני האימון. המטרה היא ללמוד קשרים שעשויים להמשיך להתקיים גם בדוגמאות שלא נראו בזמן ההתאמה. לכן מדד האימון ומדד הבדיקה ממלאים תפקידים שונים.

אם המודל פשוט מדי, הוא עלול להחמיץ קשרים שקיימים בנתונים — מצב המכונה התאמת־חסר (Underfitting). אם הוא מורכב מדי ומתאים את עצמו גם לרעש או לפרטים מקריים במדגם, הוא עלול להתאים לנתוני האימון באופן מצוין, אך להיכשל על חדשים — התאמת־יתר (Overfitting). המצב הרצוי אינו "המודל המורכב ביותר" אלא איזון שנבדק על מידע שלא היה זמין בעת האימון.



איור: הדגמה סכמטית של התאמת־חסר, מורכבות סבירה והתאמת־יתר. לא לכל בעיה יש "עקומת אמת" ידועה.

גם ביצוע טוב על Test יחיד אינו הבטחה שכל תחזית בעתיד תהיה מדויקת. אם סוגי המחשבים, אופן הדיווח על תקלות או דפוסי השימוש משתנים, גם הנתונים עלולים להשתנות. בפרויקט גמר עלינו לתאר מאילו נתונים למדנו, באילו תנאים נבדקנו ומהן המגבלות.

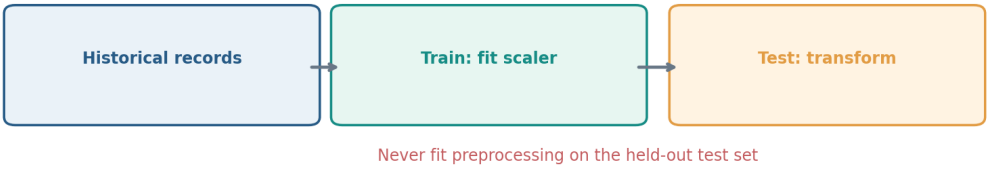
9. Data Leakage — כשהתשובה מתחבאת בקלט

זליגת מידע (Data Leakage) מתרחשת כאשר תהליך האימון מקבל מידע שלא היה אמור להיות זמין לו בזמן חיזוי אמיתי או כאשר מידע מסט הבדיקה משפיע על בחירת המודל והכנת הנתונים. התוצאה עלולה להיות ציון גבוה באופן מלאכותי. זו אחת הסיבות לכך שפרויקט נראה מרשים בניסוי אחד, אך נכשל כאשר משתמש חדש מפעיל אותו.

דוגמה ראשונה: אסור לכלול במאפיינים את העמודה "טכנאי החליט לבצע תיקון" כאשר המטרה היא לחזות לפני הבדיקה האם יש צורך בתיקון. הנתון מתקבל אחרי מועד החיזוי ולכן הוא רמז ישיר לתשובה. דוגמה שנייה: אם מחשבים ממוצע וסטיית תקן של כל המאגר לפני חלוקת Train/Test, נתוני הבדיקה כבר השפיעו על טרנספורמציה שנלמדה במהלך ההכנה.



איור: מאפיינים מותרים חייבים להיות זמינים ברגע שבו המודל אמור לספק תחזית. החלטת הטכנאי מאוחרת מדי.



איור: התאמת התהליך על Train בלבד, ולא על סט הבדיקה. לאחר מכן משתמשים באותה טרנספורמציה על הנתונים החדשים.

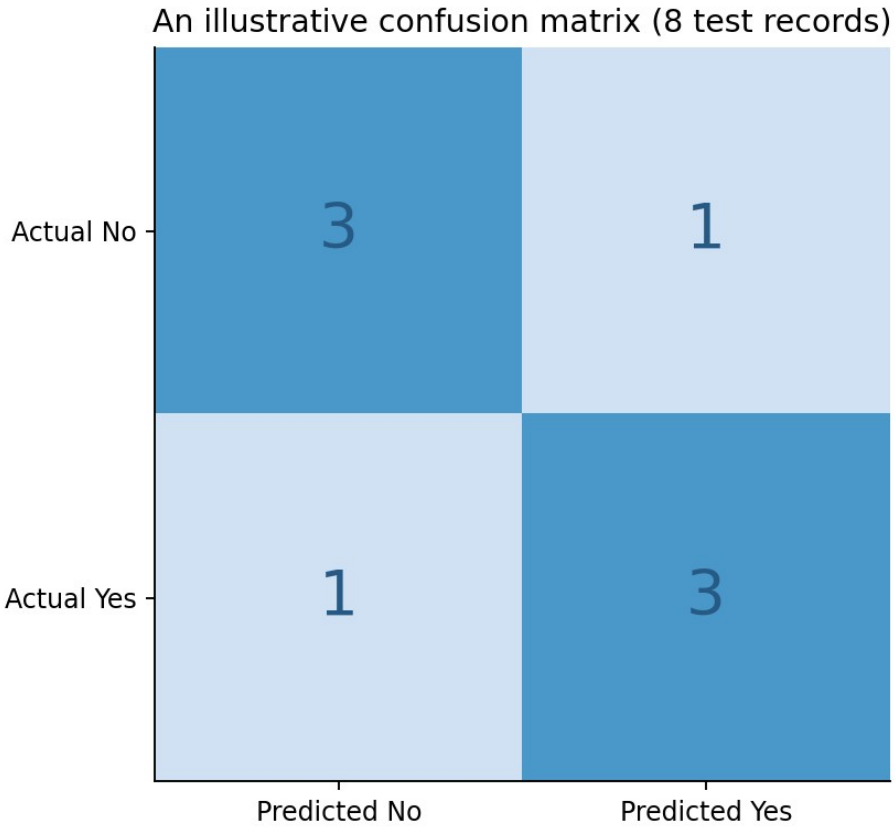
הפתרון המעשי המקובל הוא להפריד לפני ההתאמה בין אימון לבדיקה ולהשתמש ב־Pipeline שבו שלבי העיבוד והמודל נלמדים יחד על נתוני האימון בלבד. בפרק 2.3 כבר ראינו את כלל fit על Train ו־transform על Test. עכשיו אנו מבינים מדוע הכלל קשור ישירות לאמינות הערכת הביצועים.

10. איך בודקים תחזיות סיווג?

נניח ששמרנו בצד שמונה מחשבים לבדיקת מודל. עבור כל מחשב יש תווית אמת שהתבררה בבדיקה, והמודל מפיק תחזית. השוואת שתי העמודות מאפשרת לספור כמה תחזיות היו נכונות. Accuracy — שיעור הדיוק הכולל — מוגדר כמספר התחזיות הנכונות חלקי כלל דוגמאות הבדיקה.

Accuracy = (מספר התחזיות הנכונות) / (מספר דוגמאות הבדיקה)

נניח שמתוך שמונה דוגמאות התקבלו שש תחזיות נכונות ושתי שגויות. במקרה זה Accuracy הוא 6/8 = 0.75, כלומר 75%. זו דוגמת חישוב לצורך הבנת המדד; בפרק 5.1 נלמד מדדים נוספים ונראה באילו מקרים Accuracy אינו מספק.

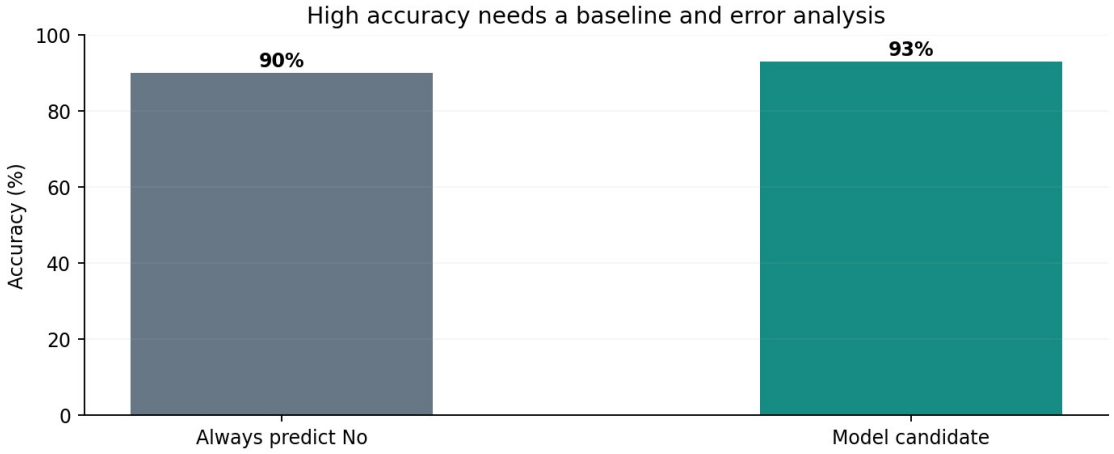


איור: מטריצת בלבול לדוגמה: שלוש תצפיות ״לא״ סווגו נכון, שלוש תצפיות ״כן״ סווגו נכון, ושתי תצפיות סווגו לא נכון.

חשוב לשאול לא רק כמה טעויות יש, אלא איזה סוג של טעויות. במקרה שלנו, חיזוי ״לא צריך טיפול״ למחשב שמצריך טיפול עלול להיות יקר או מסוכן יותר מחיזוי ״צריך טיפול״ למחשב תקין. לכן בחירת מדד ההערכה תלויה במשמעות ההחלטה, ולא רק במה שנוח לחשב.

11. קו בסיס (Baseline) — מול מה משווים את המודל?

קו בסיס הוא פתרון פשוט שמאפשר לבדוק אם המודל שלנו מוסיף ערך. אם 90 מתוך 100 מחשבים במאגר אינם דורשים טיפול, שיטה שמנחשת תמיד ״לא צריך טיפול״ תקבל 90% Accuracy אף שלא למדה שום דבר על מצב המחשבים. מודל מתקדם שמקבל 93% Accuracy אולי טוב יותר בממוצע, אך נצטרך לבדוק מה קרה למחלקה החשובה ולסוגי השגיאות.



איור: דיוק גבוה יכול להתקבל גם ממנבא טריוויאלי כאשר המחלקות אינן מאוזנות. שיפור קטן אינו מספיק ללא ניתוח סוג הטעות.

בהערכת מודל מחפשים אפוא שלוש תשובות: מהי הביצועיות של קו הבסיס, מהי הביצועיות של המודל על נתונים שלא שימשו לאימון, והאם השיפור רלוונטי לבעיה המעשית? בפרויקט גמר מומלץ לתעד את קו הבסיס לפני שמחליפים אלגוריתמים או מוסיפים מורכבות.

12. כיצד מודדים שגיאה ברגרסיה?

ברגרסיה אנו חוזים מספר. לכן אין משמעות להערכה אם ״המודל בחר מחלקה נכונה״. במקום זאת מודדים בכמה התחזית התרחקה מן הערך האמיתי. המדד הפשוט ביותר להסבר הוא MAE — ממוצע השגיאות המוחלטות. מחשבים לכל דוגמה את הערך המוחלט של ההפרש בין הערך האמיתי לתחזית, וממוצעים את התוצאות.

$$\mathrm{MAE} = (1/n) \cdot \Sigma\, |y_i - \hat{y}_i|$$

נניח שזמני תיקון אמיתיים היו 30, 40 ו־70 דקות, והמודל חזה 35, 35 ו־60 דקות. השגיאות המוחלטות הן $|35-30|=5$, $|40-35|=5$, $|60-70|=10$. לכן MAE הוא $(10+5+5)/3 = 20/3 \approx 6.67$ דקות. כדאי לפרש את היחידות: טעות ממוצעת של כ־6.67 דקות, ולא 6.67 אחוזים.

טיפול	זמן אמיתי	תחזית	שגיאה מוחלטת
1	30	35	5
2	40	35	5
3	70	60	10
ממוצע	—	—	$\approx$ 6.67

MAE הוא מדד אחד בלבד. בהמשך נלמד מדדים אחרים כגון MSE ו־R^2, ונראה מדוע מודל עם שגיאה נמוכה במדד אחד אינו בהכרח המתאים ביותר לכל שימוש.

13. ממושגים למימוש: נתוני Iris ומודל ראשון ב־scikit-learn

כדי לאמן מודל באופן סביר נשתמש כעת במאגר מוכר הכולל 150 דוגמאות של פרחי אירוס, עם ארבע מדידות מספריות ותווית מין הפרח. זהו מאגר לימודי סטנדרטי בספרייה עצמה. הוא משמש כאן לתרגול תהליך העבודה, ולא להבטחת איכות של מערכת סיווג צמחים בעולם האמיתי.

הדוגמה משתמשת בשכן קרוב באמצעות ספרייה מקצועית, רק כדי להדגים את זרימת העבודה. ההסבר המלא של מנגנון Nearest Centroid ו־KNN, לרבות בחירת K וניתוח שגיאות, יופיע בפרקים 4.2 ו־4.3. כרגע אנו רוצים להבין את תפקידי הפקודות ואת מנגנון חלוקת הנתונים.

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

iris = load_iris()
X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

model = make_pipeline(StandardScaler(),
                      KNeighborsClassifier(n_neighbors=3))
model.fit(X_train, y_train)
predictions = model.predict(X_test)
print("Train:", X_train.shape, "Test:", X_test.shape)
print("Test accuracy:", accuracy_score(y_test, predictions))
```

נסביר את הקוד בשפה פשוטה: load_iris מספק נתונים שכבר יש להם תוויות; train_test_split מחלק את הנתונים; make_pipeline מגדיר שתקנון ואלגוריתם הסיווג יופעלו ברצף; fit מתאים את שלבי התהליך

לנתוני האימון; predict מפיק תחזיות למאפיינים של דוגמאות הבדיקה; accuracy_score משווה תחזיות לתשובות האמת, שהמודל לא ראה בעת האימון.

random_state קובע את החלוקה האקראית כך שנוכל לשחזר את אותה בדיקה. הוא אינו ״משפר את הדיוק״ בפני עצמו. stratify=y מסייע לשמור על יחס מיני הפרחים בחלוקת הנתונים. גם לאחר הרצה תקינה יש לבדוק מטריצת בלבול, גודל המדגם ומגבלות ההכללה.

14. איך בוחרים מאפיינים טובים?

מאפיין שימושי הוא נתון שניתן למדוד באופן אמין לפני החיזוי, שיש סיבה מקצועית להאמין שהוא עשוי לסייע, ושניתן לפרש אותו בהקשר של הבעיה. יותר מאפיינים אינם בהכרח טובים יותר: יש מאפיינים כפולים, שדות עם ערכים חסרים ומאפיינים שעלולים לחשוף את התשובה. בכל פרויקט נרשום עבור כל מאפיין מה משמעותו, באילו יחידות נמדד, באיזה זמן הוא זמין ומהם הקשיים באיסופו.

הסבר	מותר בזמן החיזוי?	מאפיין אפשרי
ידוע לפני שהטכנאי בדק את המחשב.	כן, בדרך כלל	גיל המחשב בחודשים
הנתון חייב להיות מתוארך ולא להסתמך על העתיד.	כן, אם נאסף לפני ההחלטה	מספר תקלות שדווחו עד אתמול
יתברר רק לאחר ביצוע הטיפול.	לא	משך הטיפול שבוצע בהמשך
כמעט זהה למטרה שאנו מנסים לחזות.	לא	החלטת הטכנאי בבדיקה הנוכחית
מזהה יכול ליצור זיכרון או הטיה בלי משמעות כללית.	לעיתים לא רצוי	מספר מזהה פנימי

15. שאלות ביקורת לפני שבוטחים בתחזית

מודל יכול להחזיר תחזית מספרית מסודרת גם כאשר שאלת החקר אינה מוגדרת היטב. לכן לפני שימוש בו נברר: מהי בדיוק המטרה? מי קבע את תוויות האמת ובאיזו עקביות? האם יש דוגמאות מייצגות של כל סוגי המחשבים? האם נתוני הבדיקה מופרדים בצורה הוגנת? ומה יקרה אם המודל יטעה?

יש לבחון במיוחד מקרים שבהם תוצאת המודל משפיעה על בני אדם. אין להסיק מקבוצה סטטיסטית תכונה אישית של תלמיד, ולא להשתמש במודל שמחליט אוטומטית על תלמידים ללא בקרה אנושית. במהלך הפרויקט נתעד את מקור הנתונים, דרך איסופם, מגבלות המודל והחלטת האדם המשתמש בתחזית.

16. טבלת סיכום — מושגים שאסור לבלבל

טעות נפוצה	משמעות מדויקת	מונח
לכלול את התווית עצמה בתוך X.	תכונה זמינה לחיזוי	Feature
לחשוב שכל בעיית y היא סיווג.	ערך המטרה הידוע בדוגמאות האימון	Target / Label
לבצע fit על Test.	התאמת המודל על נתוני אימון	fit
לחשוב שתחזית היא תמיד אמת.	הפקת תחזית לדוגמה	predict
להסתמך רק על Accuracy של Train.	הצלחה יחסית על מידע חדש	Generalization
להתרשם מציון גבוה ללא נקודת ייחוס.	פתרון פשוט להשוואה	Baseline
להתאים תקנון על כל המאגר	מידע לא ראוי שחדר לתהליך	Leakage

טעות נפוצה	משמעות מדויקת	מונח
לפני חלוקה.		

17. דוגמה פתורה מסכמת — מנתונים עד מסקנה

נתון מאגר של 80 מחשבים ללמידה מונחית. המאפיינים הם גיל ותקלות מדווחות לפני הבדיקה, ותווית המטרה היא האם היה צורך בטיפול. הצוות מחלק את הנתונים ל־60 רשומות אימון ו־20 רשומות בדיקה. לאחר אימון המודל הוא מנבא נכון 16 מתוך 20 תוויות בדיקה. כעת ננתח את המצב בלי להמציא מידע נוסף.

ראשית, סוג הבעיה הוא סיווג בינארי משום שהתוצאה היא כן או לא. שנית, גודל מטריצת האימון הוא 60×2, וגודל מטריצת הבדיקה הוא 20×2, בתנאי שאין מאפיינים נוספים. שלישית, Accuracy של סט הבדיקה הוא 16/20=0.8, כלומר 80%. רביעית, איננו יודעים מה שיעור המחלקות ומה טיב שני סוגי הטעויות. לכן אין בסיס להכריז שהמודל טוב; עלינו להשוות לקו בסיס, לבדוק מטריצת בלבול ולבחון אם החלוקה עצמה הייתה נכונה.

$$\text{Accuracy(test)} = 16 / 20 = 0.80 = 80\%$$

מסקנה מדעית תקינה: ״בניסוי הנוכחי התקבלו 80% תחזיות נכונות על 20 דוגמאות בדיקה. לשם הערכת שימושיות המודל דרושים נתונים על איזון המחלקות, סוגי הטעויות, מאפייני המאגר וניסויים חוזרים בתהליך חלוקה מתאים״. זו תשובה שמראה הבנה, ולא רק הצגת מספר.

18. שאלות תרגול — רמת יסוד

תרגיל 1 — זיהוי X ו־y

למערכת המלצת ספרים יש שדות של מספר עמודים, שנת פרסום, סוגה ודירוג משתמש. רוצים לחזות את הדירוג של ספר חדש. מהו y? אילו שדות יכולים לשמש X? האם המשימה היא סיווג או רגרסיה, ומה תלוי באופן הצגת הדירוג?

תרגיל 2 — סיווג או רגרסיה

מיינו חמש משימות: חיזוי טמפרטורה מחר; זיהוי סוג פרח; חיזוי מחיר מכונית; חיזוי האם הודעה היא דואר זבל; חיזוי מספר דקות המתנה. נמקו את הבחירה בלי להסתפק בשם המשימה.

תרגיל 3 — גודל מטריצה

יש 120 תצפיות, ולכל אחת ארבעה מאפיינים מספריים ותווית אחת. מה גודלה של X? מה גודלו של y? הסבירו מה מייצגת השורה החמישית ואת מה מייצגת העמודה השלישית.

תרגיל 4 — השלמת תהליך

השלימו במילים שלכם: מגדירים בעיה → אוספים דוגמאות ותוויות → מבצעים ___ → משווים ___ → מודעו חשוב סדר השלבים? ___. לתוויות אמת →

19. שאלות תרגול — רמה בינונית

תרגיל 5 — חישוב שכנות

מחשב חדש מיוצג על ידי (10,2). נתונות תצפיות P=(9,2) ותוויתה 0, Q=(13,4) ותוויתה 1, R=(10,6) ותוויתה 1. חשבו מרחקים אוקלידיים לכל תצפית, מצאו את השכן הקרוב וציינו מה אינה מבטיחה התוצאה.

תרגיל 6 — מדדי ביצועים

על סט בדיקה של 40 דוגמאות התקבלו 30 תחזיות נכונות. חשבו Accuracy. כעת נאמר לכם ש־36 מתוך 40 דוגמאות הן ממחלקה 0. מדוע צריך לבדוק את קו הבסיס ואת טעויות המחלקה 1 לפני הסקת מסקנות?

תרגיל 7 — שגיאת רגרסיה

הערכים האמיתיים הם 10, 20, 30, 40 והתחזיות הן 12, 18, 25, 38. חשבו שגיאה מוחלטת לכל דוגמה ואת MAE. כתבו באילו יחידות יש לפרש את התוצאה.

תרגיל 8 — Data Leakage

תלמיד ביצע StandardScaler על כלל המאגר לפני train_test_split. תלמיד אחר הוסיף את העמודה "תאריך התיקון שהושלם" כמאפיין לחיזוי האם יידרש תיקון. הסבירו את שתי הבעיות והציעו תיקון לכל אחת.

20. שאלות חשיבה — רמת בגרות והכנה לפרויקט

תרגיל 9 — ניתוח קוד

נתון קטע הקוד model.fit(X_train,y_train); y_pred=model.predict(X_train). תלמיד טוען שהתוצאה מוכיחה שהמודל עובד על נתונים חדשים. האם הטענה נכונה? נמקו, והציעו תיקון לתהליך ההערכה.

תרגיל 10 — שינוי הנחות

חברת מחשבים אימנה מודל על מחשבי מעבדה משרדיים, אך מתכוונת להפעיל אותו על מחשבי גיימינג חדשים. אילו הנחות משתנות? מה צריך לבדוק לפני שימוש במודל?

תרגיל 11 — זיהוי טעויות

מודל סיווג משיג Accuracy של 95%, אבל אינו מזהה כמעט אף מחשב שנדרש עבורו תיקון דחוף. נסחו שני הסברים אפשריים והציעו מדדי בדיקה שישמשו בפרק 5.1.

תרגיל 12 — תכנון פרויקט

בחרו בעיה אחרת מעולם בית הספר או הקהילה. כתבו שאלת חיזוי מדויקת, הגדירו y ושני מאפיינים אפשריים, הסבירו כיצד תתקבל תווית אמת, והציעו שיטת הערכה שלא חושפת את התשובה למודל.

21. משימת סיכום מקורית בסגנון בגרות

דודו אסף 200 תצפיות של מחשבים. לכל מחשב נשמרו גיל בחודשים, מספר תקלות קודמות וסוג המעבד. לאחר בחירת מאפיינים מתאימים נוצרו X ו־y; המטרה היא לחזות אם המחשב יזדקק לטיפול בחודש הקרוב. התלמיד חילק את הנתונים ל־160 לאימון ו־40 לבדיקה. המודל חזה נכון 34 דוגמאות בדיקה. בזמן הכנת הנתונים הוא חישב ממוצעים מתוך כל 200 התצפיות.

א. הסבירו מדוע מדובר בסיווג בינארי ולא בקיבוץ נתונים. ב. מהו גודל X אם כל שלושת המאפיינים מיוצגים לאחר קידוד כשלושה מאפיינים מספריים? ג. מהו Accuracy של הבדיקה? ד. זהו את בעיית ההכנה. ה. הציעו שיטת השוואה לקו בסיס ומדד נוסף אם טעות "לא נדרש טיפול" יקרה במיוחד. ו. הסבירו מדוע אי אפשר להבטיח ביצועים על מחשבים מסוג שונה שלא הופיעו במאגר.

המשימה היא תרגיל מקורי שנבנה בסגנון מיומנויות הבחינה; היא אינה שאלה שנלקחה מבגרות רשמית.

22. מעבדת Colab — מה נבצע בלי לקבל פתרון מוכן?

במחברת הנלווית 12 משימות. התלמיד יבדוק גדלים של X ו־y, יפריד בין מאפיינים לתווית, יחשב מרחקים, יזהה סוג משימה, יחלק נתונים, יבנה מודל באמצעות תבנית שנלמדה, ינבא וינתח טעויות. המשימות בנויות כך שהדוגמה המלאה בספר מקנה את הידע הדרוש, אבל הקוד הפתור של המעבדה לא מופיע בספר.

במשימת הסיום התלמיד יתעד באופן מדעי: מה הייתה שאלת החקר? אילו מאפיינים היו זמינים בזמן החיזוי? מהו קו הבסיס? האם Test שימש לבחירת הגדרות? אילו מגבלות נותרו? אין צורך בתוצאה "מושלמת"; יש צורך בניסוי שחזור והסבר תקין.

23. גשר ללמידה עמוקה ולפרויקט הגמר

למידה מונחית איננה אלגוריתם יחיד; זו מסגרת עבודה. אותה מסגרת — X, y, אימון, חיזוי, תפקיד סט הבדיקה, מדד שגיאה — תחזור ברגרסיה, ב־Nearest Centroid, ב־KNN, בפרספטרון וברשתות עצביות עמוקות. מה שמשתנה הוא מנגנון הלמידה, צורת הייצוג וסוג החיזוי. לכן יש חשיבות להבנת המונחים לעומק כבר כעת.

לפרויקט גמר, התוצר המינימלי של הפרק הוא כרטיס בעיה: משפט אחד שמגדיר בדיוק מה המערכת תחזה, טבלה שמפרטת מאפיינים ותווית, ותיאור של נתוני אימון ונתוני בדיקה. בלי הגדרה זו עלול הפרויקט להפוך לאוסף קבצים וקוד שאין דרך להעריך את הצלחתו.

מסלול	הנושא בפרק	איך ממשיכים
יחידה 3 — בסיס קודם	טבלת מאפיינים, Train/Test, מודל ראשון	חזרה ממוקדת בלבד; אין לשכפל את כל הקורס.
יחידה 5 — נושאי ליבה	4.1 מבוא ללמידה מונחית	בהמשך: 4.2 Nearest Centroid, 4.3 KNN, 4.4 מבט משווה.
התמחות בלמידה עמוקה	אימון מודלים, הערכה והכללה	בהמשך: רגרסיה, פרספטרון ורשתות עמוקות.

24. סיכום המושגים והקשרים

למידה מונחית משתמשת בתצפיות שיש להן ערך מטרה ידוע בעת האימון. כאשר ערך המטרה הוא מחלקה, אנו פותרים משימת סיווג; כאשר הוא כמות רציפה, אנו פותרים משימת רגרסיה. אנו מייצגים את המאפיינים במטריצה X ואת ערכי המטרה בווקטור y, מתאימים מודל ל־Train, מפיקים תחזיות לנתונים חדשים ומשווים אותן לערכי אמת שלא שימשו לאימון.

הצלחה אמיתית נמדדת על היכולת להכליל, ולא על שינון דוגמאות. יש להיזהר מהתאמת־יתר, מדד הערכה לא מתאים, קו בסיס חלש וזליגת מידע. בשלב הבא נכיר מודל סיווג פשוט הנשען על מרכזי המחלקות — Nearest Centroid — ונראה כיצד כל המושגים של פרק זה הופכים לתהליך חישובי מדויק.

25. מקורות ועוגן בתכנית הלימודים

עוגן מיפוי: יחידה 5, פרק 4 ״מודלים מבוססי מרחק״, סעיף 4.1 ״מבוא ללמידה מונחית״, על פי מפת הנושאים השמורה במאגר Data-Science-ML-Course. יש לאשר את היקף החובה המדויק מול המהדורה הרשמית התקפה בעת ההוראה. הרחבות כמו קו בסיס, דליפת מידע, MAE והשימוש ב־Pipeline נכללו כאן כגשרים פדגוגיים למודלים ולפרויקט גמר ואינן מתויגות אוטומטית כחומר חובה בבגרות.

מקורות השראה והעמקה: Luis Serrano, Grokking Machine Learning (Manning); Aurélien Géron, Hands-On Machine Learning (O’Reilly); Gareth James ואחרים, An Introduction to Statistical Learning with Applications in Python (Springer); Josh Starmer, The StatQuest Illustrated Guide to Machine Learning. ניסוחי ההסבר, הסיפור, הטבלאות, האיורים והשאלות בפרק זה נכתבו מחדש כמקור חינוכי עצמאי.

תיעוד מקצועי: scikit-learn, User Guide ו־Common Pitfalls — הפרדה בין אימון לבדיקה, טיפול ב־Data Leakage והעדפה לשימוש ב־Pipeline. לצורך ההוראה נדרש לעקוב אחר גרסאות הספריות המעודכנות.

זהו ספר התלמיד. פתרונות תרגילי ההערכה ומעבדת התלמיד מיועדים למהדורת מורה נפרדת. אין לפרסם אותם בגוף חוברת התלמיד.

26. מילון מושגים ממוקד — עברית, אנגלית ומשמעות

מונח בעברית	English	מה חשוב להסביר
למידה מונחית	Supervised Learning	אימון על דוגמאות עם תשובת מטרה ידועה.
מאפיין	Feature	קלט הזמין בזמן החיזוי.
משתנה מטרה	Target / Label	הדבר שמבקשים לחזות.
אימון	Training / fit	התאמת המודל לפי דוגמאות אימון.
חיזוי	Prediction	הפקת פלט על סמך מודל מותאם.
הכללה	Generalization	הצלחה על נתונים חדשים מעבר לאימון.
התאמת־יתר	Overfitting	היצמדות רבה מדי לפרטי נתוני האימון.
קו בסיס	Baseline	פתרון פשוט שנותן נקודת השוואה.
זליגת מידע	Data Leakage	מידע בלתי מותר המשפיע על לימוד/הערכה.
בדיקה סופית	Test Set	דוגמאות השמורות להערכה לאחר בחירת המודל.

שימו לב להבחנה בין מונח מתמטי למונח בעולם הנתונים: הסימן y עשוי להיות מדידה מספרית ברגרסיה או קוד מספרי של מחלקה בסיווג. עלינו לפרש את הערכים לפי השאלה שניסחנו ולא לפי הצורה שבה הם שמורים במחשב.

27. כרטיס יציאה — בדיקת הבנה ללא מחשב

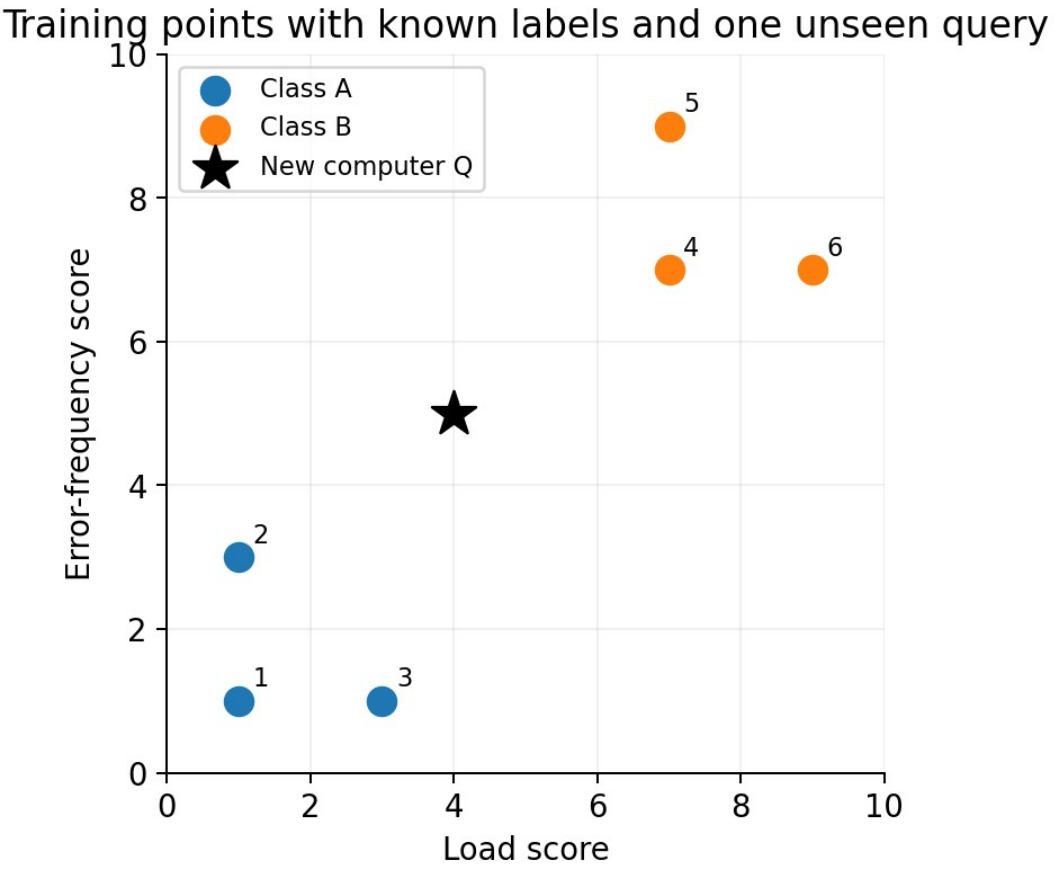
נסו לענות על השאלות הבאות בלי לפתוח מחברת או לקרוא מחדש את הפרק. רק לאחר מכן השוו להסברים בספר. השאלות קצרות אך דורשות נימוק ולא רק מילה אחת.

- למה כרטיס תלמיד עם גיל ושעות תרגול אינו בהכרח דוגמת אימון מונחית?
- מה ההבדל בין חיזוי מחלקה ״תקין״ לחיזוי מחיר תיקון של 250 שקלים?
- מה חייב להתאים בין שורות X לבין איברי y?
- מדוע 100% הצלחה על Train אינם מעידים על הצלחה בעולם האמיתי?
- איזו טעות עלולה להיווצר אם תקנון מותאם על כל המאגר לפני חלוקתו?
- איך הייתם מסבירים למנהל בית ספר מה פירוש Accuracy של 80% על 20 דוגמאות?
- מהו הדבר הראשון שתבדקו אם מודל חדש מגיע ל־95% Accuracy בבעיה עם מעט דוגמאות חיוביות?
- מהו המידע שעל תלמיד לציין לפני שהוא טוען שהמודל בפרויקט הגמר ״עובד״?

יעד ההצלחה בפרק זה אינו לשנן את שמות הפקודות בלבד. היעד הוא להסביר בעל־פה ולכתוב בצורה מדויקת כיצד דוגמה מתויגת הופכת לתחזית, מדוע נדרשת בדיקה בלתי תלויה, ומה מגביל את המסקנה. זהו הבסיס לפרקים 4.2–4.4 ולכל מודל מתקדם שנלמד בהמשך.

פרק 4.2 — אלגוריתם Nearest Centroid

כיצד מסווגים דוגמה חדשה באמצעות המרכז המייצג של כל מחלקה?



איור 1. שש דוגמאות מתויגות ושאלה חדשה. כל דוגמה היא וקטור של שתי תכונות; הסיווג של הנקודה החדשה עדיין אינו ידוע.

מהדורת תלמיד • הסבר מושגי מפורט • חישובים ידניים מלאים • גרפים מקוריים • NumPy ו־scikit-learn • תרגול עצמאי ללא פתרונות

בפרקים הקודמים למדנו לייצג תצפית כווקטור, לחשב מרחק אוקלידי ולמצוא מרכז של קבוצת נתונים. בפרק זה נחבר את שלושת הרעיונות כדי לבנות מסווג ראשון שאפשר לחשב כולו ביד. נלמד גם מתי דרך פשוטה זו מצליחה, מתי היא מטעה ומדוע אסור להעריך מודל רק על הדוגמאות ששימשו לאימונו.

מטרות למידה וידע קודם

לפני שמתחילים: צריך לדעת לחשב ממוצע, לייצג נקודה בשני ממדים, לחשב מרחק בין וקטורים ולזהות תווית ידועה. בפרק 4.1 למדנו מהי למידה מונחית, מהן X ו־y ומה פירוש Train ו־Test. בפרק 3.2 למדנו לחשב מרכזים בתהליך K-Means; כעת נשתמש במרכזים לצורך סיווג כאשר התוויות ידועות מראש.

- לזהות מהי מחלקה מתויגת ומהו המרכז (Centroid) המייצג אותה.
- לחשב ידנית מרכז דו־ממדי וגם מרכז של וקטורים בעלי n רכיבים.
- לחשב את מרחק התצפית החדשה מכל מרכז ולהכריע איזו מחלקה קרובה יותר.
- להסביר כיצד נקבע גבול החלטה וכיצד קנה מידה וחריגים עשויים לשנות את ההכרעה.

- להשוות בין Nearest Centroid, K-Means ו־KNN מבלי לבלבל ביניהם.
- לממש מסווג ב־NumPy וב־scikit-learn ולבדוק אותו על נתונים שלא השתתפו באימון.

1. סיפור פתיחה — דודו ומעבדת המחשבים

דודו אחראי על ציוד המחשוב בבית הספר. לכל מחשב נמדדים שני ציונים מספריים בתחום 0–10: עומס פעילות ותדירות שגיאות. טכנאי סיווג בעבר מחשבים למחלקה A — עבודה תקינה לפי הבדיקה שלו — או למחלקה B — מחשב הדורש בדיקה נוספת. דודו רוצה להעריך לאיזו מחלקה משתייך מחשב חדש, שעבורו טרם התקבלה תשובת הטכנאי.

חשוב להבחין: התווית אינה עובדה שהמחשב "יודע" על עצמו; היא החלטה שניתנה לדוגמאות העבר. המודל ילמד מדוגמאות אלו. הנתונים בפרק מומצאים לצורכי הוראה ולא נועדו להחליף בדיקה טכנית ממשית. העובדה שניתן לנבא תווית אינה מוכיחה שאנו יודעים את סיבת התקלה.

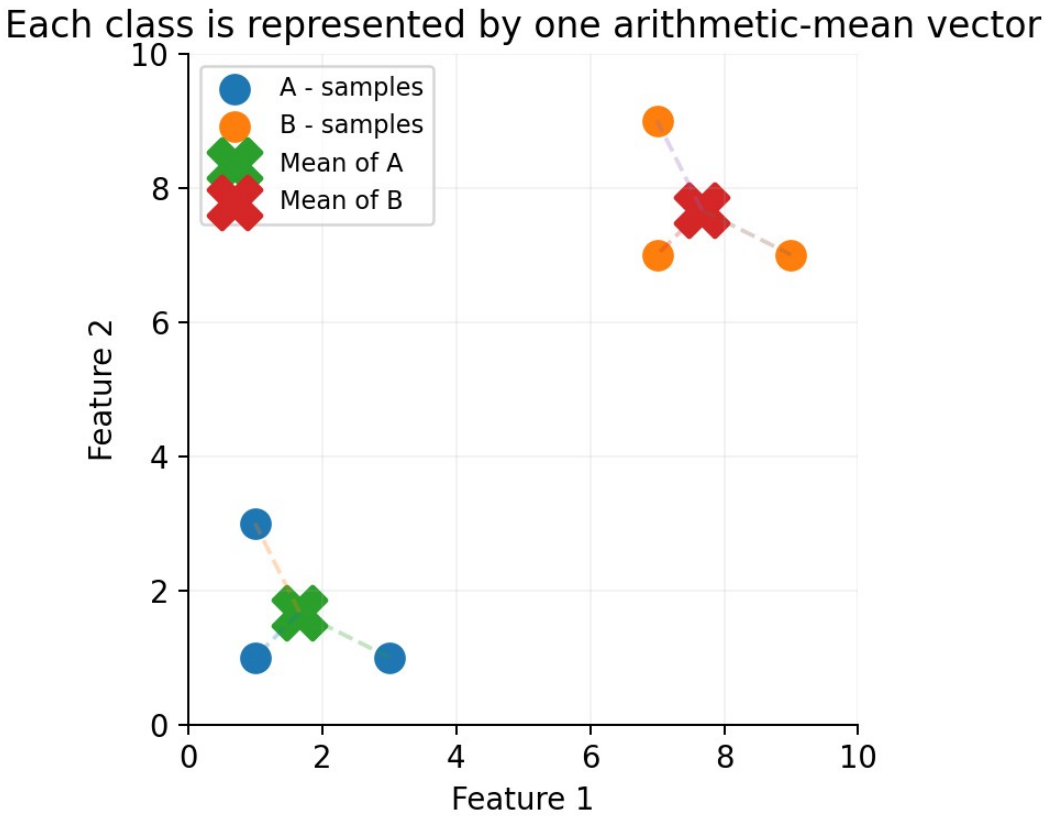
מחשב	מאפיין 1: עומס	מאפיין 2: שגיאות	תווית ידועה
A1	1	1	A
A2	1	3	A
A3	3	1	A
B1	7	7	B
B2	7	9	B
B3	9	7	B

נתוני האימון הם שש שורות ושלוש עמודות מידע. שתי העמודות המספריות נכנסות למטריצת X; התוויות נכנסות בנפרד לווקטור y. המחשב החדש ייוצג כ־$Q=(4,5)$. כדי לחזות את תוויתו, נחשב תחילה מה מאפיין בממוצע כל מחלקה.

2. הרעיון המרכזי: מחלקה מיוצגת על ידי מרכז אחד

כיצד נוכל לקבל החלטה בלי להשוות את המחשב החדש לכל אחת ממאות התצפיות? במקום לשמור לצורך ההכרעה את המיקומים של כל התצפיות, נחשב לכל מחלקה וקטור ממוצע. וקטור זה נקרא "מרכז המחלקה" (Centroid). נמדוד את מרחק המחשב החדש מכל מרכז, ונבחר בתווית של המרכז הקרוב ביותר.

זהו מסווג פשוט מסוג למידה מונחית: התוויות של הדוגמאות משמשות כדי לדעת אילו דוגמאות יש למצע יחד. בשלב האימון מחושבים מרכזי המחלקות; בשלב החיזוי משווים אליהם תצפית חדשה. המחלקות אינן מתגלות מאליהן, משום שהן ניתנו מראש.



איור 2. מרכז כל מחלקה מסומן ב־X גדול. המודל מחליף את קבוצת הדוגמאות של כל תווית במרכז אחד המתאר את הממוצע הרכיבי.

3. מהו מרכז מחלקה, ומדוע מחשבים ממוצע לכל רכיב?

אם כל מחשב מתואר על ידי שני מספרים, גם המרכז שלו חייב להכיל שני מספרים. המספר הראשון במרכז הוא ממוצע הערכים של המאפיין הראשון בקרב מחשבי אותה מחלקה; המספר השני הוא ממוצע המאפיין השני. איננו מחשבים ממוצע יחיד לכל הטבלה, משום שממוצע אחד היה מוחק את ההבדל בין המחלקות.

$$c_k = (1 / n_k) \cdot \Sigma_{(i : y_i = k)} x_i$$

כאן k הוא שם המחלקה; n_k הוא מספר הדוגמאות המסומנות במחלקה זו; x_i הוא וקטור המאפיינים של דוגמה מספר i; ו־c_k הוא וקטור המרכז שנחשב. סימן הסכימה אומר: מחברים רק את הווקטורים שהתווית שלהם שייכת למחלקה k, ומחלקים במספר הדוגמאות באותה מחלקה.

3.1 חישוב ידני מלא של מרכז A

למחלקה A שייכות שלוש התצפיות (1,1), (1,3), (3,1). כל רכיב מחושב בנפרד. עבור המאפיין הראשון נחבר 1+1+3 ונחלק ב־3; עבור המאפיין השני נחבר 1+3+1 ונחלק ב־3. במקרה זה שני הממוצעים זהים, אך זו תכונה של הנתונים הספציפיים ולא דרישה של האלגוריתם.

$$c_A = ((1+1+3)/3, (1+3+1)/3) = (5/3, 5/3)$$

כלומר מרכז A נמצא בקירוב בנקודה (1.67,1.67). המרכז אינו חייב להיות אחת מדוגמאות האימון ואינו חייב להיות מחשב ממשי. הוא סיכום מספרי של שלוש הדוגמאות.

3.2 חישוב ידני מלא של מרכז B

$$c_B = ((7+7+9)/3, (7+9+7)/3) = (23/3, 23/3)$$

מרכז B נמצא בקירוב בנקודה (7.67,7.67). כעת יש לנו שני וקטורים מייצגים: המרכז של A והמרכז של B. שימו לב שפעולת האימון לא מחקה את התוויות; כל מרכז עדיין קשור למחלקה שממנה חושב.

3.3 יותר משני מאפיינים

אם לרשומה היו גם גיל המחשב, טמפרטורה ממוצעת ותדירות כיבוי לא צפוי, המרכז היה וקטור בעל חמישה רכיבים. עדיין מחשבים ממוצע לכל עמודה בנפרד, ורק בתוך הדוגמאות בעלות אותה תווית. לכן נוסחת המרכז עובדת גם במרחב בעל ממדים רבים, שאי אפשר לצייר על דף.

4. שלב החיזוי — מודדים מרחק מכל מרכז

המחשב החדש מוצג כ־Q=(4,5). לפני החישוב ננחש: המרכז של A מצוי בסביבות (1.67,1.67), והמרכז של B בסביבות (7.67,7.67). נקודה Q נמצאת ביניהם, ולכן עלינו לחשב במדויק איזה מרכז קרוב יותר. הניחוש החזותי לבדו אינו מספיק כדי להכריע במקרים גבוליים.

$$d(Q,c) = \sqrt{[(Q_1 - c_1)^2 + (Q_2 - c_2)^2]}$$

המרחק האוקלידי מחושב כך: מחסרים את הרכיב המתאים בכל ממד, מעלים כל הפרש בריבוע כדי שלא יהיו ביטולים בין סימנים חיוביים ושליליים, מחברים ולבסוף מוציאים שורש. ככל שהמרחק קטן יותר, הנקודה קרובה יותר למרכז.

4.1 המרחק מ־Q למרכז A

$$d(Q,c_A) = \sqrt{[(4-5/3)^2 + (5-5/3)^2]}$$

$$= \sqrt{[(7/3)^2 + (10/3)^2]} = \sqrt{149 / 3} \approx 4.0697$$

ההפרש במאפיין הראשון הוא 7/3, ובשני 10/3. ריבועיהם הם 49/9 ו־100/9; הסכום הוא 149/9; השורש הוא √149/3. זה המרחק מ־Q אל המרכז של מחלקה A.

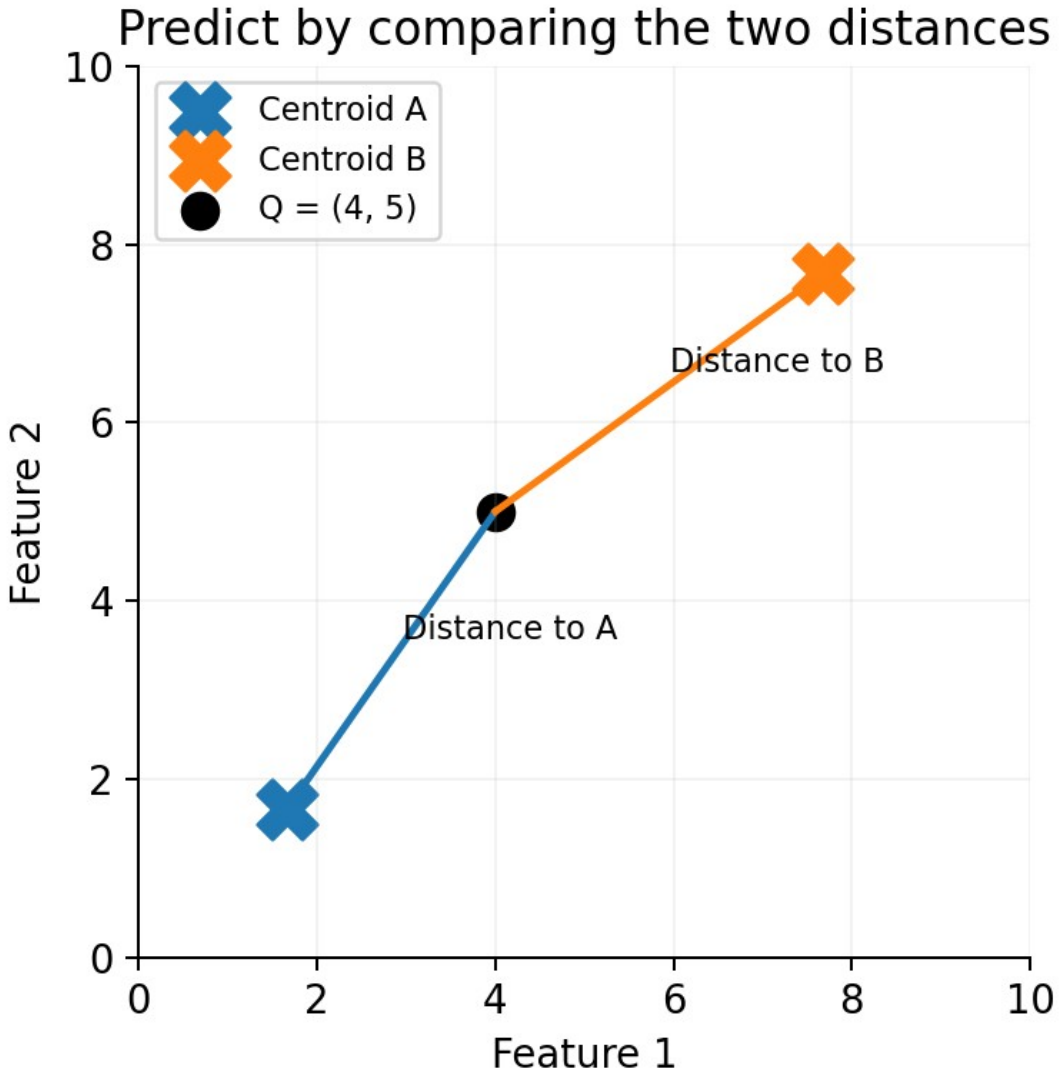
4.2 המרחק מ־Q למרכז B

$$d(Q,c_B) = \sqrt{[(4-23/3)^2 + (5-23/3)^2]}$$

$$= \sqrt{[(-11/3)^2 + (-8/3)^2]} = \sqrt{185 / 3} \approx 4.5338$$

מאחר ש־4.0697 קטן מ־4.5338, מרכז A קרוב יותר. לכן תחזית האלגוריתם עבור המחשב החדש היא A. זוהי תחזית של המודל, לא בדיקת טכנאי ולא הבטחה שהמחשב אכן תקין.

למי מודדים?	מרחק מדויק	קירוב	משמעות
למרכז A	√149 / 3	4.0697	קרוב יותר
למרכז B	√185 / 3	4.5338	רחוק יותר



איור 3. הקטעים מחברים את Q לשני המרכזים. ההחלטה נשענת על השוואת מרחקים, ולא על השוואת Q לכל דוגמת אימון בנפרד.

4.3 אפשר להשוות מרחקים גם בלי לחשב שורשים

אם שני המרחקים אוקלידיים, אפשר להשוות את ריבועי המרחקים. הסיבה פשוטה: פונקציית השורש עולה עבור מספרים לא־שליליים. לכן מי שיש לו סכום ריבועים קטן יותר, יהיה גם הקרוב יותר לאחר הוצאת השורש. זה חוסך עבודה כאשר צריך רק לבחור מחלקה, אך אינו משנה את הגדרת המרחק.

$$149/9 < 185/9 \;\Rightarrow\; d(Q,c_A) < d(Q,c_B)$$

5. תהליך האלגוריתם כולו — אימון, חיזוי והערכה

בשלב האימון מקבלים את מטריצת X ואת התוויות y, מקבצים לפי תווית ומחשבים לכל תווית את ממוצע הווקטורים שלה. התוצאה של האימון היא מערך מרכזים ומפת התוויות שלהם. בשלב החיזוי מקבלים וקטור חדש, מחשבים את מרחקו לכל מרכז, ומחזירים את התווית של המרכז הקרוב ביותר. בהערכה משווים תחזיות לתוויות אמיתיות של דוגמאות שלא שימשו בחישוב המרכזים.

האלגוריתם אינו מכוון משקולות באמצעות מאות צעדים כמו רשת עצבית. במקרה הבסיסי שלו אין צורך לבחור מספר שכנים ואין איטרציות של עדכון מרכזים: המרכזים מחושבים פעם אחת מהתוויות הידועות. הפשטות מאפשרת להשתמש בו כקו בסיס, כלומר נקודת השוואה למודל מורכב יותר.

פסאודו־קוד — מובן גם בלי ספרייה

```
TRAIN(X, y):
    for each known class k:
        center[k] = mean of X rows with label k
    save centers

PREDICT(q):
    for each class k:
        distance[k] = euclidean_distance(q, center[k])
    return class with smallest distance
```

שימו לב שהפסאודו־קוד משתמש בשמות קבועים וברעיון מתמטי, ולא דורש לזכור פקודת ספרייה מסוימת. זוהי מיומנות חשובה בבגרות: להסביר מה האלגוריתם מבצע גם אם שפת התכנות או הספרייה משתנה.

6. גבול ההחלטה — היכן המודל משנה את תחזיתו?

דמיינו שמזיזים את המחשב החדש על פני הגרף. בכל מקום מחשבים מחדש שני מרחקים. בצד אחד של המישור המרכז A קרוב יותר; בצד השני המרכז B קרוב יותר. קו הנקודות שבהן שני המרחקים שווים נקרא גבול ההחלטה (Decision Boundary).

כאשר משתמשים במרחק אוקלידי רגיל ובשני מרכזים שונים, גבול זה הוא האנך האמצעי לקטע המחבר ביניהם. במקרה שלנו שני המרכזים נמצאים על האלכסון שבו שני הרכיבים שווים, ולכן גבול ההחלטה פשוט במיוחד.

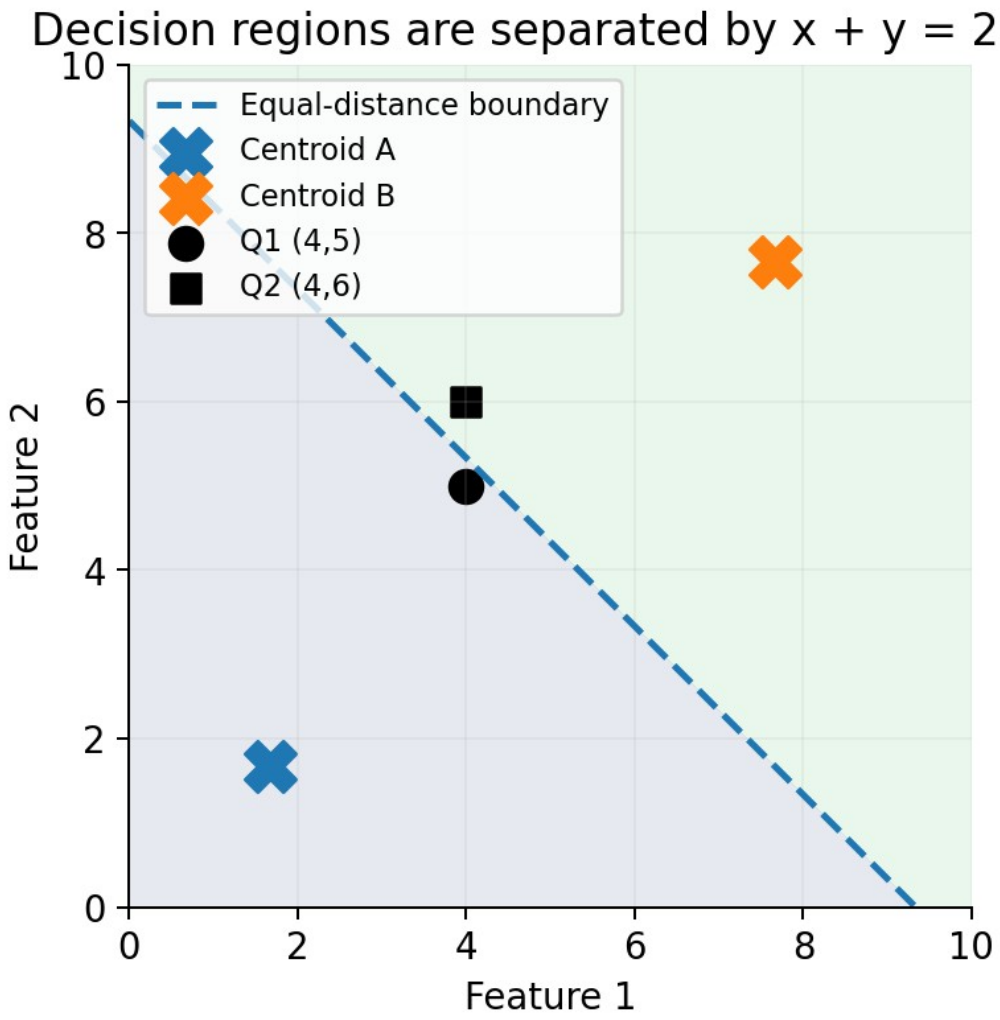
$$d((x,y),c_A)^2 = d((x,y),c_B)^2$$

$$(x-5/3)^2+(y-5/3)^2 = (x-23/3)^2+(y-23/3)^2$$

$$x+y = 28/3 \approx 9.3333$$

למי שרוצה להבין את המעבר: פותחים סוגריים, מבטלים את x^2 ואת y^2 שמופיעים בשני הצדדים, מסדרים את האיברים ומקבלים משוואת ישר. אין צורך לזכור בעל־פה את המשוואה המסוימת; צריך להבין מדוע היא נוצרת משוויון מרחקים.

למשל, עבור $Q=(4,5)$ סכום הרכיבים הוא 9, ולכן הנקודה בצד של A. עבור $R=(4,6)$ הסכום הוא 10, ולכן הנקודה בצד של B. על הגבול עצמו יש שוויון מרחקים; במקרה כזה יש להגדיר כלל הכרעה עקבי או לציין שהמקרה אינו מוכרע על ידי הקרבה בלבד.



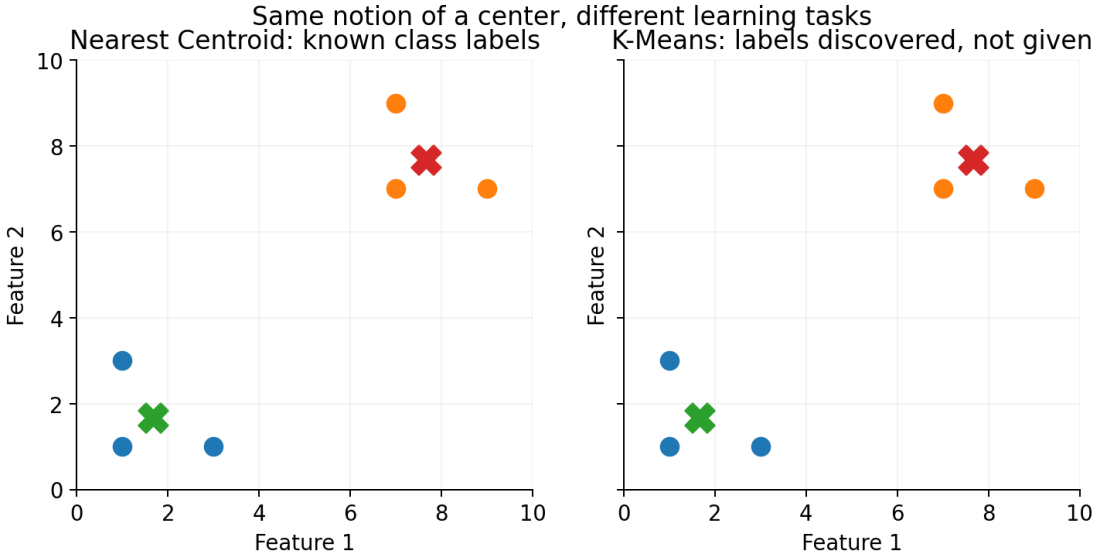
איור 4. גבול ההחלטה הוא הישר $x+y=28/3$. משני צדי הקו מתקבלות תחזיות שונות; בדיוק על הקו המרחקים שווים.

7. מה ההבדל בין Nearest Centroid לבין K-Means?

בשני האלגוריתמים משתמשים במרכזים, אך השאלה שהם פותרים שונה. ב־K-Means אין תוויות מחלקה נכונות מראש; האלגוריתם מנסה לגלות חלוקה על פי קרבה, ומעדכן את המרכזים והשייכות שוב ושוב. ב־Nearest Centroid התוויות ידועות בשלב האימון, ולכן מחשבים את מרכז מחלקה A מתוך הדוגמאות שכבר קיבלו A, ואת מרכז B מתוך הדוגמאות שקיבלו B.

המילה ״מרכז״ אינה מספיקה כדי לקבוע באיזו שיטת למידה משתמשים. שאלת הבדיקה היא האם המידע על המחלקות ניתן מראש והאם אנו רוצים לחזות תווית ידועה, או לגלות קבוצות שאיש לא סימן.

מאפיין	Nearest Centroid	K-Means
סוג הלמידה	מונחית	לא מונחית
תוויות בשלב האימון	ידועות	אינן נדרשות
מספר מרכזים	לפי מספר המחלקות	נבחר כ־K
עדכון חוזר של שיוך	לא בגרסה הבסיסית	כן, עד להתכנסות
התוצר	מסווג לתוויות	חלוקה לקבוצות



איור 5. מבחינה גיאומטרית המרכזים עשויים להיראות דומים, אבל המשימה והמידע שהאלגוריתם מקבל שונים.

8. מה ההבדל בין Nearest Centroid לבין KNN?

שיטת KNN, שתילמד במלואה בפרק 4.3, בוחנת את השכנים הקרובים לדוגמה החדשה מתוך דוגמאות האימון, ומשתמשת בתוויותיהם להצבעה. Nearest Centroid אינו בוחר שכנים פרטיים: הוא דוחס כל מחלקה למרכז אחד. לכן שני המודלים עשויים לתת תוצאות שונות על אותה נקודה.

דוגמה אינטואיטיבית: ייתכן שמחלקה A כוללת מאה דוגמאות הפרושות על שטח גדול, וביניהן דוגמה אחת קרובה מאוד ל־Q, בעוד שהמרכז הממוצע של A רחוק. שיטת 1-NN עשויה לבחור A בגלל אותה נקודה; Nearest Centroid עשוי לבחור B כי המרכז של B קרוב יותר. אף ששתיהן משתמשות במרחקים, אין להחליף את חוק ההחלטה של האחת בזה של האחרת.

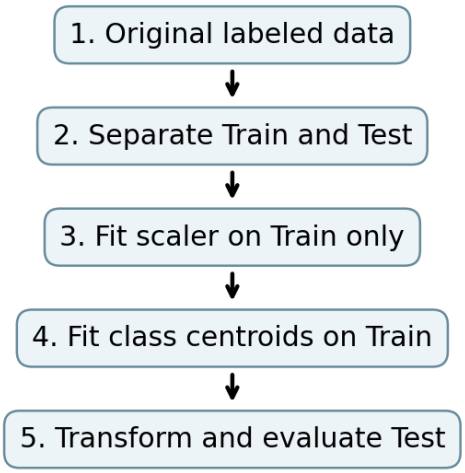
9. השפעת קנה המידה: מרחק תלוי ביחידות

נניח שבמקום שני ציונים בתחום 0–10 היינו מודדים תדירות שגיאות כאחוז, וגיל המחשב בימים. ההפרשים ברכיב הגיל היו עלולים להיות גדולים בהרבה מההפרשים ברכיב השגיאות, אף שאינם בהכרח חשובים יותר. במצב כזה המרחק האוקלידי עלול לתת בפועל משקל גדול יותר למאפיין שערכיו בעלי טווח מספרי גדול.

הפתרון המקובל לבדיקת השפעה זו הוא תקנון או נרמול, כפי שלמדנו בפרק 2.3. חשוב לזכור: אין להתאים את ה־Scaler לכל הנתונים לפני חלוקה ל־Train ול־Test. מחשבים את פרמטרי התקנון מתוך נתוני האימון בלבד, ולאחר מכן מפעילים את אותה טרנספורמציה על נתוני הבדיקה.

$$z = (x - \mu_{train}) / \sigma_{train}$$

למשל, אם ממוצע מאפיין באימון הוא 20 וסטיית התקן שלו היא 5, הערך 30 יתוקנן ל־2. גם אם במדגם הבדיקה מתקבל ממוצע אחר, אין לחשב ממנו תקנון חדש. אחרת הבדיקה חושפת למודל מידע שאינו אמור להיות זמין בזמן אימון.



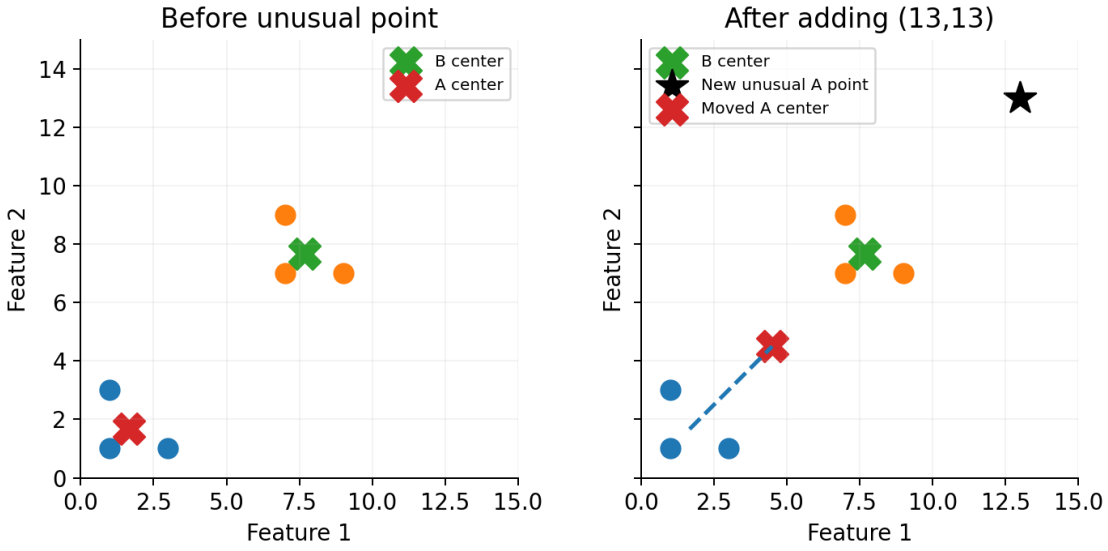
איור 6. חלוקת נתונים לפני התאמת התקנון. כל שלב שמשתמש בסטטיסטיקה של נתונים חייב ללמוד אותה מה־Train בלבד.

10. מה קורה כשמופיעה דוגמה חריגה?

מרכז המבוסס על ממוצע רגיש לערכים חריגים. אם נכניס למחלקה A מחשב חריג בעל מאפיינים (13,13), המרכז של A ישתנה אף על פי ששאר שלושת המחשבים לא השתנו כלל. זה מפני שהממוצע סופר כל ערך, גם ערך שנמצא רחוק מאוד מהשאר.

$$c_A(new) = ((1+1+3+13)/4,\ (1+3+1+13)/4) = (4.5,4.5)$$

מרכז A עבר מ־(1.67,1.67) ל־(4.5,4.5). שינוי זה עלול להזיז את גבול ההחלטה ולשנות תחזיות של דוגמאות חדשות. לכן לפני אימון יש לבחון חריגים, להבין אם הם טעויות, תופעות אמיתיות או תת־קבוצה חשובה, ולא למחוק אותם באופן אוטומטי.

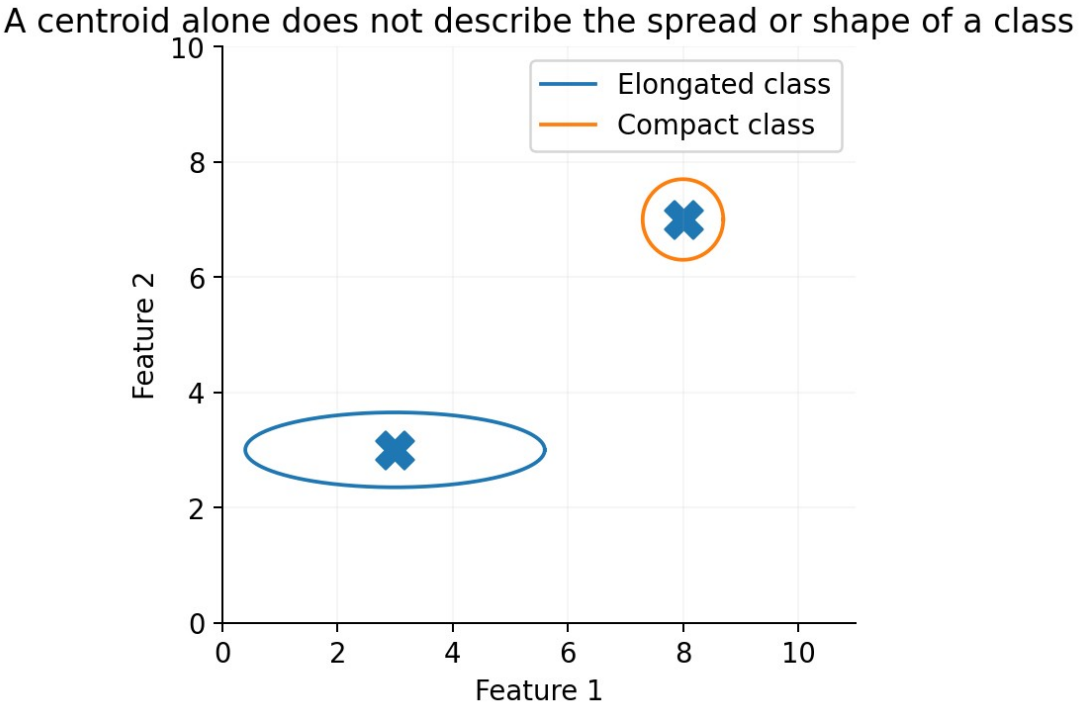


איור 7. דוגמה חריגה שנוספה למחלקה A מזיזה את המרכז הממוצע באופן משמעותי, אף כי שלוש התצפיות המקוריות נשארו במקומן.

11. מגבלות נוספות של מרכז מחלקה אחד

מרכז אחד מסכם את מקום המחלקה, אך אינו מתאר את צורתה ואת הפיזור שלה. שתי מחלקות יכולות להיות בעלות מרכזים דומים אך פיזור שונה מאוד; מחלקה יכולה להיראות כמו טבעת, כשמרכזה נמצא באזור שבו אין כלל תצפיות מהמחלקה. במצבים כאלה מרחק למרכז עשוי להיות כלי חלש לקבלת החלטה.

בנוסף, שתי מחלקות אינן חייבות להיות מופרדות בקו ישר. Nearest Centroid עם מרחק אוקלידי יוצר גבולות החלטה ליניאריים בין כל זוג מרכזים, ולכן אינו מתאים לכל דפוס. האלגוריתם אינו מודד ביטחון סטטיסטי רק מפני שהמרחק קטן יותר; פער המרחקים לבדו אינו הסתברות שהתחזית נכונה.



איור 8. שתי מחלקות יכולות להיות בעלות פיזור וצורה שונים מאוד. מיקומי המרכזים לבדם אינם מתארים את כל המבנה.

11.1 האם מחלקה גדולה בהכרח תועדף?

בחוק ההחלטה הבסיסי, המרכז של כל מחלקה מחושב רק מדוגמאותיה, ולאחר מכן נבחר המרכז הקרוב ביותר. גודל המחלקה משפיע על יציבות אומדן המרכז, אך אין זה אומר שבאופן אוטומטי המחלקה הגדולה מקבלת יותר קולות. זהו הבדל חשוב לעומת שיטת הצבעת שכנים. בהרחבות מקצועיות קיימים מסווגים המשקללים מידע נוסף, אך אין צורך בכך כדי להבין את חוק Nearest Centroid הבסיסי.

12. מימוש ראשון ב־Python: בלי להשתמש במסווג מוכן

נכתוב תחילה את החישוב באופן גלוי, כדי לראות שהספרייה אינה עושה קסם. נבחר את השורות של כל מחלקה, נחשב את הממוצע לפי עמודות ונמדוד מרחק מכל מרכז. לאחר שנבין את התוצאה, נשתמש באותה דוגמה בספריית scikit-learn.

```
import numpy as np

X = np.array([[1,1], [1,3], [3,1],
              [7,7], [7,9], [9,7]], dtype=float)
y = np.array(["A", "A", "A", "B", "B", "B"])

center_a = X[y == "A"].mean(axis=0)
center_b = X[y == "B"].mean(axis=0)

q = np.array([4, 5], dtype=float)
d_a = np.linalg.norm(q - center_a)
d_b = np.linalg.norm(q - center_b)

print("A:", center_a, "distance:", d_a)
print("B:", center_b, "distance:", d_b)
print("Prediction:", "A" if d_a < d_b else "B")
```

הביטוי X[y == "A"] בוחר רק את השורות המסומנות A. הפקודה mean(axis=0) מחשבת ממוצע לכל עמודה, לא ממוצע אחד לכל המספרים. הביטוי q - center_a יוצר את וקטור ההפרשים, ו־np.linalg.norm מחשבת את המרחק האוקלידי, בדיוק כמו בחישוב הידני. משפט התנאי בוחר את המרחק הקטן. בנתונים אחרים עלול להיות שוויון, ולכן רצוי להגדיר גם טיפול בתיקו.

13. מימוש מקצועי ב־scikit-learn

```
from sklearn.neighbors import NearestCentroid

model = NearestCentroid(metric="euclidean")
model.fit(X, y)

print("Labels:", model.classes_)
print("Centers:\n", model.centroids_)
print("Predictions:", model.predict([[4, 5], [4, 6]]))
```

הפעולה f i t(X,y) מחשבת את מרכזי המחלקות מתוך נתוני האימון והתוויות. התכונה classes_ מציגה את התוויות שלמד המסווג, והתכונה centroids_ מציגה את המרכזים לפי אותו סדר. הפעולה predict מקבלת מטריצה של דוגמאות חדשות: שתי השורות [[4,5],[4,6]] הן שתי בקשות חיזוי שונות. התוצאות בדוגמה הן A ואז B.

שימו לב: הצירוף metric="euclidean" מבהיר באיזה מרחק משתמשים. אין להסיק שכל מרחק תמיד משתמש בממוצע. בתיעוד עדכני של הספרייה, כאשר בוחרים Manhattan, מרכז המחלקה עשוי להיות החציון הרכיבי. בפרק זה נלמד את המקרה הבסיסי במרחק אוקלידי ובממוצע.

14. כיצד בודקים אם המודל מכליל?

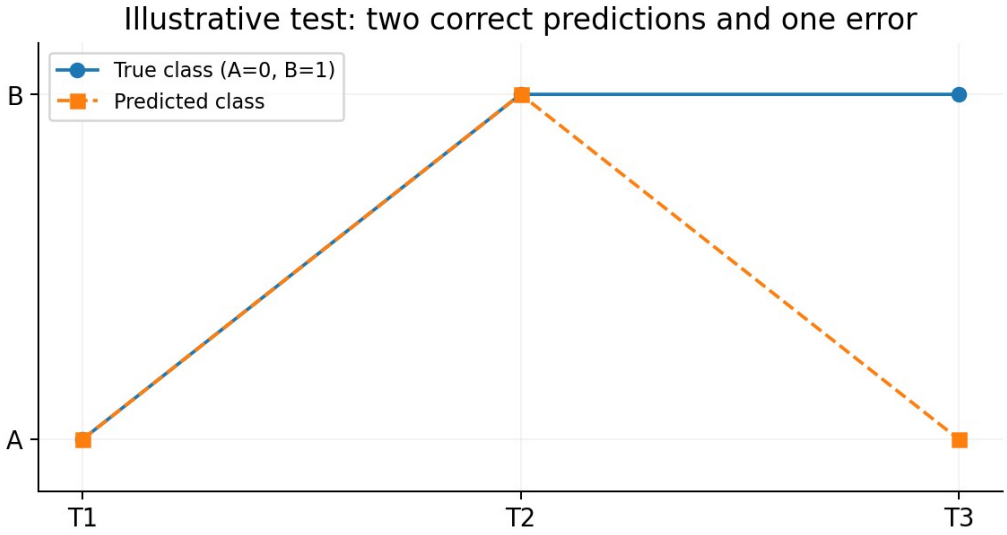
לא מספיק לבדוק שהמודל מסווג היטב את שש דוגמאות האימון, מפני שמרכזיו חושבו מהן. נצטרך דוגמאות עם תוויות ידועות שלא השתתפו באימון. לצורך הדגמה נניח שקיבלנו שלוש תצפיות בדיקה חדשות, שכבר נבדקו על ידי הטכנאי. התוויות האמיתיות בטבלה הן חלק מנתוני התרגיל המומצאים.

מחשב בדיקה	מאפיינים	תווית אמת	תחזית המודל	נכון?
T1	(3,4)	A	A	כן
T2	(6,5)	B	B	כן
T3	(6,3)	B	A	לא

יש שתי תחזיות נכונות מתוך שלוש. לכן Accuracy הוא 2/3, כלומר כ־66.7%. אין להסיק מכך מה תהיה מידת הצלחתו של המודל בעולם האמיתי: שלוש דוגמאות בדיקה הן מעטות מאוד, והמספרים מומצאים לצורך לימוד. הן מספיקות רק כדי להדגים כיצד מחשבים את המדד ואת ההבדל בין אימון לבין בדיקה.

$$\text{Accuracy} = \text{number_of_correct_predictions} / \text{number_of_test_examples}$$

$$\text{Accuracy} = 2 / 3 \approx 0.6667 = 66.67\%$$



איור 9. שלוש דוגמאות בדיקה: שתי תחזיות תואמות לתווית הידועה ואחת שגויה. מספר התצפיות קטן מאוד ולכן זו המחשה בלבד.

14.1 עבודה נכונה על מאגר אמיתי

בפרויקט משתמשים בנתונים רבים יותר, מפרידים Train ו־Test לפני תקנון וחקר פרמטרים ומשווים למודל בסיס. בכל פעם שמנסים תכונה אחרת או מחליפים שיטת הכנה, צריך להקפיד שלא להסתכל שוב ושוב על מדד ה־Test ולהתאים אליו את ההחלטות. הרחבה מסודרת של ולידציה וקרוס־ולידציה תופיע בפרק 5.2.

15. הקשר לפרויקט הגמר ולשאלות הבגרות

בשאלות בגרות עשויים לבקש להפעיל את האלגוריתם על טבלת נקודות, לחשב מרכזים, להשוות מרחקים, לזהות טעות בחישוב או להסביר את השפעתו של שינוי קנה מידה. המבחן אינו רק שליפת נוסחה: התלמיד צריך להסביר מדוע משתנה מסוים הוא מאפיין, מדוע נתון אחר הוא תווית, ומדוע מרכז מחושב מתוך דוגמאות האימון בלבד.

בפרויקט גמר Nearest Centroid יכול לשמש קו בסיס פשוט למערכת סיווג. לדוגמה, לפני שאנו בונים רשת עמוקה לזיהוי מצבי מחשב, נרצה לדעת האם מרכזי המחלקות כבר נותנים תוצאה מספקת. אם לא, ננתח האם הבעיה היא בחירת מאפיינים, קנה מידה, מבנה קבוצות, חריגים או איכות התוויות. כך המודל מלמד אותנו לשאול שאלות מקצועיות, ולא רק להפעיל ספרייה.

16. טעויות נפוצות וכיצד לזהות אותן

טעות נפוצה	מדוע היא בעייתית?	מה עושים?
מחשבים מרכז אחד לכל הנתונים	מוחקים את ההבדל בין התוויות	מחשבים מרכז נפרד לכל מחלקה
מחשבים ממוצע על כל הערכים	מאבדים את מבנה התכונות	מחשבים ממוצע לפי עמודות
משווים לנתוני אימון במקום	מחליפים את שיטת Nearest	משווים את q למרכז כל מחלקה

טעות נפוצה	מדוע היא בעייתית?	מה עושים?
למרכזים	Centroid ב־NN	
מתקננים לפי כל המאגר	דליפת מידע מנתוני בדיקה	מתאימים תקנון על Train בלבד
טוענים שמרחק קטן הוא הסתברות	מרחק אינו מודל הסתברותי מכויל	מתארים קרבה, לא ביטחון מובטח
מניחים שהמרכז חייב להיות דוגמה קיימת	הממוצע יכול להימצא בין הדוגמאות	מפרשים את המרכז כוקטור סיכום
מוחקים כל חריג	חריג עשוי להיות תופעה אמיתית	בודקים הקשר ומשמעות קודם

17. תרגול מדורג — רמה א׳: מושגים וחישובים

1. הסבירו במילים שלכם מה פירוש המונח ״מרכז מחלקה״, ומדוע נדרשות תוויות של דוגמאות האימון.
2. למחלקה C הנקודות (2,2), (4,2), (6,5). חשבו ידנית את מרכז המחלקה וכתבו את פירוש כל רכיב.
3. למחלקה D הנקודות (8,1), (8,3), (10,2). חשבו את המרכז והשוו את התוצאה למרכז C.
4. עבור $Q=(5,3)$, חשבו את שני המרחקים, החליטו מהי התחזית ונמקו כיצד בדקתם שהחישוב סביר.
5. מדוע ניתן להשוות סכומי ריבועי הפרשים בלי לחשב שורשים, כאשר משתמשים במרחק אוקלידי?

18. תרגול מדורג — רמה ב׳: הסקה וביקורת

6. הסבירו מה ההבדל בין Nearest Centroid, K-Means ו־1-NN. כתבו תשובה שאינה מסתפקת במילים ״משתמשים במרחקים״.
7. בדוגמה שלנו $R=(4,6)$ ו־$Q=(4,5)$. הסבירו מדוע הם מקבלים תחזיות שונות אף שההפרש ביניהם במאפיין אחד הוא 1 בלבד.
8. תלמיד הוסיף לדוגמאות מחלקה A את הנקודה (13,13). חשבו את המרכז החדש והסבירו האם נדרש לעדכן גם את מרכז B.
9. נמצאה דוגמת בדיקה שמרחקה לשני המרכזים זהה. הציעו כלל הכרעה עקבי וכתבו מה מגבלת ההחלטה במקרה זה.
10. הציעו מקרה שבו מחלקה גדולה ומפוזרת תסווג בצורה חלשה על ידי מרכז יחיד, גם אם יש בה דוגמה קרובה מאוד לתצפית החדשה.

19. תרגול מדורג — רמה ג׳: שאלות מקוריות בסגנון בגרות

שאלה 1 — טבלת מרכזים. נתונות שתי מחלקות בתלת־ממד: $A=\{(1,1,2),(3,1,2),(2,4,2)\}$ וכן $B=\{(7,6,7),(8,9,7),(9,6,7)\}$. נתונה הדוגמה $T=(5,5,4)$. חשבו את מרכז כל מחלקה, את ריבועי המרחקים והכריעו מהי התחזית. הראו את סדר פעולותיכם והסבירו מדוע אין צורך להוציא שורש בשלב האחרון.

שאלה 2 — ניתוח תקלה בקוד. תלמיד חישב center = X.mean() במקום חישוב מרכז לכל מחלקה ולכל מאפיין. הסבירו שתי טעויות נפרדות בפתרונו, וכתבו בקוד או בפסאודו־קוד כיצד נכון לבצע את החישוב.

שאלה 3 — תהליך מחקר. תלמידה התאימה StandardScaler לכל נתוני המאגר ורק אז חילקה ל־Train ול־Test. הסבירו מהי הבעיה, תארו את סדר העבודה התקין והציעו כיצד לבדוק האם Nearest Centroid מתאים לשאלת המחקר.

שאלה 4 — חשיבה על גבולות החלטה. הראו כיצד בדוגמה המקורית מתקבלת משוואת גבול ההחלטה $x+y=28/3$. הסבירו מה המשמעות של נקודה הנמצאת על הישר, וכיצד תשפיע הזזת מרכז A על הגבול.

20. מעבדת Colab — ממחשבון מרכזים למסווג בדיקה

לפרק מצורפת מחברת תלמיד נפרדת. המחברת אינה מכילה פתרונות למשימות: בכל משימה תופיע מטרת הפעולה והנחיה לשמור על הפרדה בין חישוב ידני לבין פקודת ספרייה. מתחילים מבניית X ו־y, עוברים לחישוב מרכזים בשתי שיטות, בודקים חיזוי של מחשב חדש, משווים החלטות לאורך גבול ההחלטה, מוסיפים חריג ומבצעים הערכה על דוגמאות בדיקה שלא השתתפו באימון.

לאחר המעבדה התלמיד צריך להיות מסוגל להסביר, בלי להסתכל בקוד, מה עושה fit, מה נשמר במרכזים, מדוע predict מחזירה תווית ומהי המגבלה של מדד הצלחה המבוסס על מעט תצפיות.

21. שאלות רפלקציה וקריאה מקצועית

לפני סיום הפרק חזרו לשאלה שבה פתחנו: כיצד דודו יכול להסיק את הצורך בבדיקה של מחשב חדש? עתה אנו יכולים לענות במדויק: הוא משתמש בדוגמאות מתויגות מן העבר, מסכם כל מחלקה באמצעות ממוצע רכיבי, מודד קרבה ומפיק תחזית. אבל אם התוויות אינן איכותיות או אם צורת המחלקות מורכבת, ייתכן שהמודל הפשוט יטעה. הכוח המקצועי הוא בידיעה מתי השיטה מועילה ומתי נדרש מודל אחר.

כדי לבסס את ההבנה, השוו את הפרק לתיאור Nearest Centroid בתיעוד scikit-learn, שבו האלגוריתם משמש מסווג בסיסי הנשען על מרכזי מחלקות. בספרי לימוד מעשיים כמו Hands-On Machine Learning ובספרים אינטואיטיביים כמו Grokking Machine Learning יש דגש על מעבר מדוגמה מחושבת למודל שניתן לבחון באופן ביקורתי. ההסברים והתרגילים בפרק זה נכתבו מחדש עבור תלמידי תיכון.

22. מה חשוב לזכור לקראת הבחינה?

- ב־Nearest Centroid יש תוויות ידועות מראש; לכל מחלקה מחשבים מרכז בנפרד.
- במרחק אוקלידי רגיל, מרכז המחלקה הוא וקטור הממוצעים הרכיביים של דוגמאות האימון מאותה מחלקה.
- תצפית חדשה משויכת למרכז הקרוב ביותר — לאו דווקא לדוגמת האימון הקרובה ביותר.
- המרחק תלוי בסולמות המדידה, ולכן הכנת נתונים עשויה לשנות החלטה.
- גבול ההחלטה בין שני מרכזים הוא מקום הנקודות בעלות מרחק שווה משני המרכזים.
- אין לבלבל בין ביצועים על נתוני אימון לבין יכולת הכללה, ואין לפרש מרחק כהסתברות.

23. מיקום הפרק בתכנית ובספר

עוגן תכני: נושאי ליבה בבינה מלאכותית, יחידה 5, פרק 4 — מודלים מבוססי מרחק, סעיף 4.2 Nearest Centroid, לפי מפת הסילבוס השמורה במאגר ההוראה. ידע קודם: פרק 1.2 מרחק בין וקטורים, פרק 2.1 ממוצע, פרק 2.3 תקנון, פרק 3.2 מרכזים ב־K-Means ופרק 4.1 למידה מונחית. הפרק הבא: 4.3 KNN.

הערה להבחנה בין מקור להרחבה: חישוב מרכזים, מרחקים וסיווג שייכים לליבת הנושא. גזירת משוואת גבול החלטה, ניתוח חריגים מתקדם, שיקולי פיזור ומניעת זליגת מידע משולבים כדי לחזק הבנה ולהכין לפרויקט גמר. אין להציג כל הרחבה כחובה לבגרות בלי הצלבה עם הנוסח הרשמי התקף.

מקורות להמשך לימוד

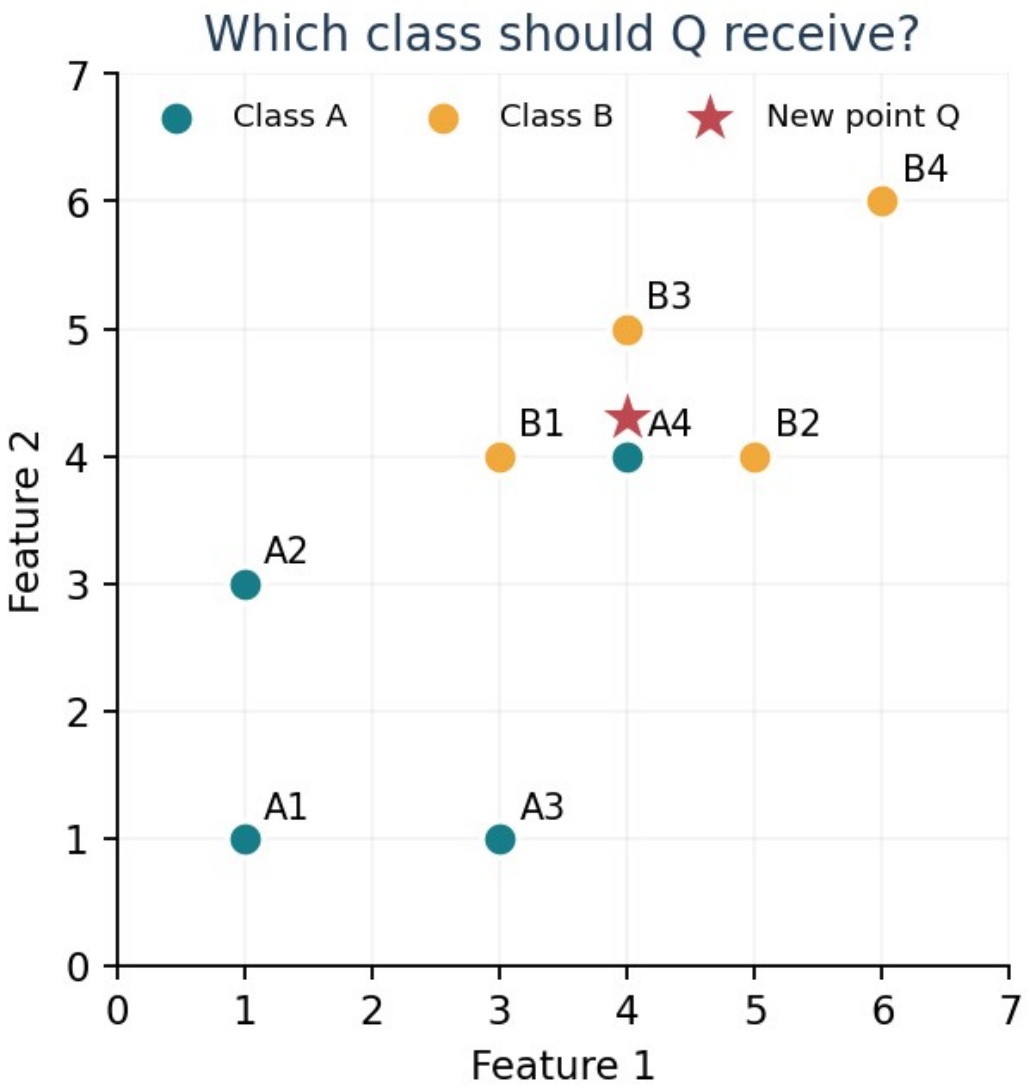
משרד החינוך: https://github.com/samiabo75/Data-Science-ML-Course/tree/main/02-Unit5-AI-Core/04-Distance-Based-Models

תיעוד ; מדריך: Nearest Centroid: https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.NearestCentroid.html https://scikit-learn.org/stable/modules/neighbors.html

ספרי יסוד להעמקה: Luis G. Serrano, Grokking Machine Learning (Manning); Aurélien Géron, Hands-On Machine Learning (O’Reilly); Deisenroth, Faisal & Ong, Mathematics for Machine Learning (Cambridge).

פרק 4.3 — אלגוריתם KNN

כיצד מחליטים לאיזו מחלקה שייכת דוגמה חדשה על פי השכנים הקרובים לה?



איור 1. נקודות בעלות תוויות ידועות ושאלה חדשה Q. המטרה אינה לנחש מהציור אלא ללמוד דרך החלטה מדידה ושחזורית.

חוברת תלמיד עצמאית • הסברים מלאים • חישובים ידניים • 12 המחשות מקוריות • NumPy ו־scikit-learn • תרגול ללא פתרונות

בפרק 4.2 השתמשנו במרכז אחד עבור כל מחלקה. בפרק זה נלמד שיטה אחרת: במקום לשאול איזה מרכז קרוב לדוגמה החדשה, נבחן אילו דוגמאות אימון עצמן נמצאות סביבה. נראה כיצד החלטה שנראית פשוטה — ״הצביעו לפי השכנים״ — תלויה בבחירת מספר השכנים, באופן המדידה, בהכנת הנתונים ובבדיקה הוגנת של המודל.

מפת הפרק — מה נדע לבצע בסיומו?

- להסביר בשפה מדויקת מהו K-Nearest Neighbors ולזהות תצפית, מאפיין, תווית, מרחק ופרמטר k.

- לחשב ידנית מרחקים, למיין שכנים, לשמור על ההתאמה בין מרחק לתווית ולהפעיל כלל הצבעת רוב.
- להסביר כיצד k=1, k=3 או k גדול יותר עשויים להוביל לתחזיות שונות; לנסח מדיניות ברורה לשוויון.
- לפרש את השפעת קנה המידה, החריגים, המשקולות והגבולות הגיאומטריים על התחזית.
- לממש את הרעיון בלולאה פשוטה וב־NumPy, ולאחר מכן להשתמש ב־KNeighborsClassifier.
- לבצע הפרדה תקינה בין אימון, אימות ובדיקה, ולנמק את בחירת k בלי להסתמך על Test לצורך בחירה.

ידע קודם: וקטורים ו־L1/L2 (פרקים 1.1–1.2), מדדי פיזור ותקנון (2.1 ו־2.3), למידה מונחית ומרכז מחלקה (4.1–4.2).

1. סיפור פתיחה — דודו ומעבדת המחשבים

דודו אחראי על מעבדת מחשבים בבית הספר. עבור כל מחשב נשמרים שני ציונים בתחום 0–10: עומס המערכת ותדירות תקלות. עבור מחשבים ישנים קיימת גם תווית שנתן הטכנאי לאחר בדיקה: A — תקין בדרך כלל, או B — דרוש טיפול. כעת נכנס מחשב חדש שהמאפיינים שלו ידועים, אך התווית טרם נקבעה. כיצד ניתן להשתמש בניסיון המצטבר כדי להציע תווית?

נייצג כל מחשב בנקודה במישור. במערכת אמיתית אלה יכולים להיות ערכים שנמדדו, אך נתוני הסיפור מומצאים לצורך למידה ואינם תחליף לבדיקה טכנית. המודל אינו קובע אמת רפואית או הנדסית; הוא מספק חיזוי על סמך תצפיות שסומנו בעבר.

מזהה	מאפיין 1 — עומס	מאפיין 2 — תקלות	תווית
A1	1	1	A
A2	1	3	A
A3	3	1	A
A4	4	4	A
B1	3	4	B
B2	5	4	B
B3	4	5	B
B4	6	6	B

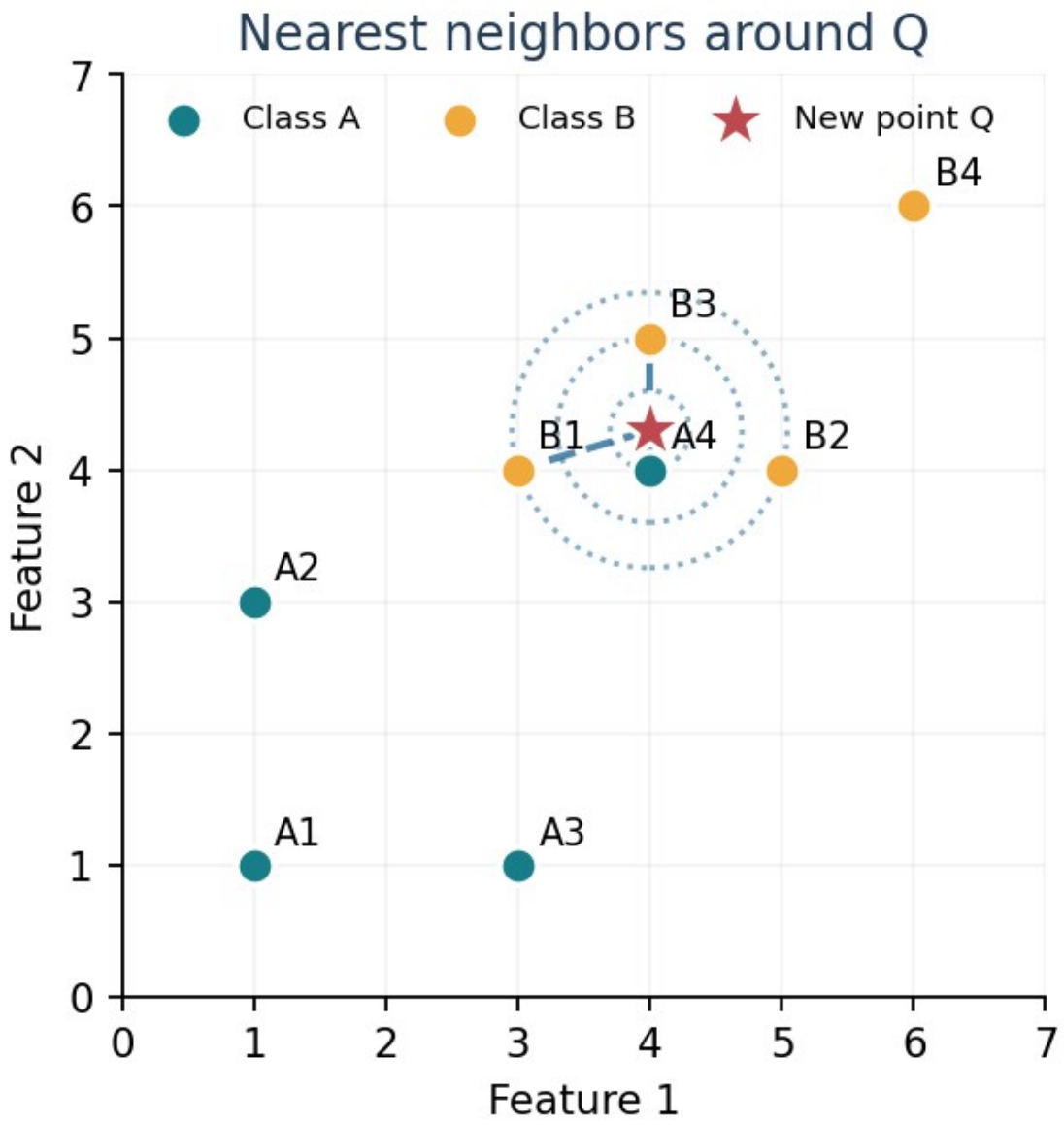
$$Q = (4, 4.3)$$

בדוגמה החדשה Q, עומס המערכת הוא 4 ותדירות התקלות 4.3. את התווית של Q אין להכניס לטבלת האימון: היא התוצאה שאותה נרצה לחזות. שימו לב: שמונה תצפיות האימון כבר מסומנות; Q היא נקודת הבדיקה שלנו.

2. אינטואיציה: השכן הקרוב ביותר לעומת רוב שכנים

אם התלמיד צריך לבחור לחבר חדש קבוצת פעילות, ייתכן שיסתכל על תלמיד אחד שדומה לו מאוד; ייתכן גם שיבדוק שלושה או חמישה תלמידים דומים ויאמץ את הבחירה הנפוצה ביניהם. זוהי בדיוק האינטואיציה מאחורי KNN. הסיווג אינו נוצר מתוך משוואה אחת המתארת גבול מראש: עבור כל דוגמה חדשה מחשבים את קרבתה לדוגמאות האימון וקובעים את התווית לפי השכנים שנבחרו.

האות k מייצגת את מספר השכנים הקרובים שנשתמש בהם לצורך תחזית. אם k=1 מסתכלים על השכן הקרוב ביותר. אם k=3 מסתכלים על שלושה. אין הכרח ששלושת השכנים יהיו מאותה מחלקה; כאשר הם חלוקים, משתמשים בכלל ההכרעה שהוגדר מראש.



איור 2. קווי העזר מייצגים מרחקים מן הנקודה Q. שכנים קרובים נקבעים לפי מספרים מחושבים — לא רק לפי התרשמות חזותית.

3. מהו KNN מבחינה מדעית?

KNN הוא אלגוריתם ללמידה מונחית המבוסס על דוגמאות אימון מתויגות. בעת פעולת fit של המסווג נשמר מידע על דוגמאות האימון באופן המאפשר חיפוש שכנים בהמשך. בגרסה הבסיסית שאנו לומדים כאן, אין שלב שבו מחושבים משקלים של רשת עצבית או ישר חיזוי; עיקר העבודה נעשית בזמן שאילתת החיזוי. לכן מכנים לעיתים את KNN ״למידה עצלה״ (Lazy Learning).

אין להסיק מכך שאין כאן למידה כלל: המודל מייצג קשר בין מאפיינים לתוויות באמצעות מאגר הדוגמאות המתויגות ומנגנון קבלת החלטה. התוצאה תלויה בנתוני האימון, במדד המרחק ובערך k.

$$\hat{y}(Q) = \text{majority_label}(y_i \text{ for } i \text{ among } k \text{ nearest neighbors of } Q)$$

הסימון $\hat{y}$ מתאר תווית מנובאת; y_i היא תווית ידועה של דוגמת אימון; הביטוי k nearest neighbors הוא קבוצת השכנים שנבחרו אחרי חישוב ומיון המרחקים. זוהי תמצית הפעולה, אך כדי להבין אותה באמת נבצע כל שלב ידנית.

4. השלב הראשון — מדידת מרחק אוקלידי

נחשב את המרחק של Q מכל אחת משמונה התצפיות. מאחר שיש שני מאפיינים מספריים, נוכל להשתמש בנוסחת פיתגורס שלמדנו בפרק 1.2. נחשב הפרש בכל מאפיין, נעלה בריבוע, נחבר ולבסוף נוציא שורש. סדר הפעולות חשוב; אי אפשר לחבר מרחקים רק לפי העין.

$$d(Q, X_i) = \sqrt{[(Q_1 - X_{i1})^2 + (Q_2 - X_{i2})^2]}$$

4.1 דוגמה פתורה: מרחק אל A4

$$A4 = (4,4) \qquad Q = (4,4.3)$$

$$d(Q,A4) = \sqrt{[(4-4)^2 + (4.3-4)^2]} = \sqrt{0.09} = 0.30$$

הפרש המאפיין הראשון הוא אפס, ולכן רק ההפרש האנכי משפיע על המרחק. מרחק של 0.30 הוא קטן יחסית למרחקים האחרים בטבלה שלנו. מכאן נחשוד ש־A4 יהיה אחד השכנים הקרובים ביותר.

4.2 דוגמה פתורה נוספת: מרחק אל B3

$$B3 = (4,5)$$

$$d(Q,B3) = \sqrt{[(4-4)^2 + (4.3-5)^2]} = \sqrt{0.49} = 0.70$$

השכן B3 רחוק יותר מ־A4, אך עדיין קרוב מאוד. כעת נחשב את שאר המרחקים. זכרו: אנו מחשבים מרחקים בין וקטורי המאפיינים בלבד; אין משמעות למרחק מספרי בין תוויות A ו־B.

בטבלה נרשמים הערכים המוחלטים של ההפרשים בכל מאפיין ($|\Delta x|$ ו־$|\Delta y|$), כדי שהריבוע בנוסחת המרחק יהיה פשוט לבדיקה. ערך המוחלט אינו משנה את המרחק האוקלידי, אך חשוב לדעת לחשב גם את ההפרש עם סימנו.

תצפית	וקטור	$\|\Delta x\|$	$\|\Delta y\|$	מרחק	מחלקה
A1	(1, 1)	3.0	3.3	4.460	A
A2	(1, 3)	3.0	1.3	3.270	A
A3	(3, 1)	1.0	3.3	3.448	A
A4	(4, 4)	0.0	0.3	0.300	A
B1	(3, 4)	1.0	0.3	1.044	B
B2	(5, 4)	1.0	0.3	1.044	B
B3	(4, 5)	0.0	0.7	0.700	B
B4	(6, 6)	2.0	1.7	2.625	B

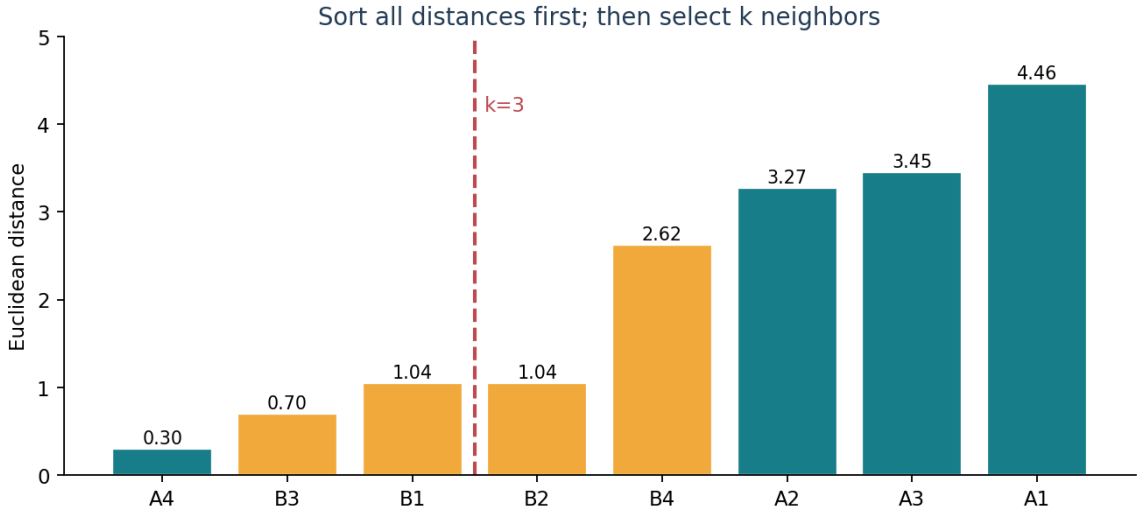
לצורך ההצבעה אפשר לעבוד גם עם ריבועי המרחקים, מפני ששורש ריבועי הוא פונקציה עולה: סדר המרחקים לא ישתנה. אבל כאשר מציגים מרחק אמיתי בסולם הנתונים, יש להחזיר את השורש.

5. השלב השני — מיון השכנים ושמירת התוויות

כעת נדרג את שמונה התצפיות מן הקרובה ביותר לרחוקה ביותר. אין די במיון המרחקים לבדם: חייבים לשמור יחד עם כל מרחק את מזהה התצפית ואת תווית המחלקה שלה. אחרת עלולים לבצע הצבעה על תוויות שאינן שייכות לשכנים שנבחרו.

מקום בדירוג	תצפית	מרחק מעוגל	תווית
1	A4	0.300	A
2	B3	0.700	B
3	B1	1.044	B
4	B2	1.044	B

מקום בדירוג	תצפית	מרחק מעוגל	תווית
5	B4	2.625	B
6	A2	3.270	A
7	A3	3.448	A
8	A1	4.460	A



איור 3. סדר המרחקים בדוגמת דודו. הקו המקווקו מסמן את שלושת השכנים הראשונים. עבור B1 ו־B2 מתקבל אותו מרחק בקירוב וגם בדיוק; הם מאותה מחלקה ולכן אין לכך השפעה על ההכרעה כאן.

בכל מקרה שבו כמה תצפיות נמצאות במרחק זהה בדיוק בגבול רשימת השכנים, יש להגדיר מדיניות הכרעה לפני החיזוי. בדוגמאות הידניות בספר נשתמש בשמירת סדר מזהי התצפיות בתיקו מרחקים; תוכנות שונות עשויות לטפל בתיקו בדרכים אחרות, ואין להניח שזו המדיניות האוטומטית של scikit-learn.

6. השלב השלישי — הצבעת רוב עבור ערכי k שונים

6.1 כאשר k = 1

השכן הקרוב ביותר הוא A4, שמרחקו 0.30 והתווית שלו A. לכן תחזית המסווג כאשר k=1 היא A. תצפית אחת בלבד יכולה להשפיע מאוד על ההכרעה, גם אם היא חריגה מבחינת המיקום שלה ביחס לשאר מחלקה A.

k = 1 → votes: A = 1, B = 0 → prediction A

6.2 כאשר k = 3

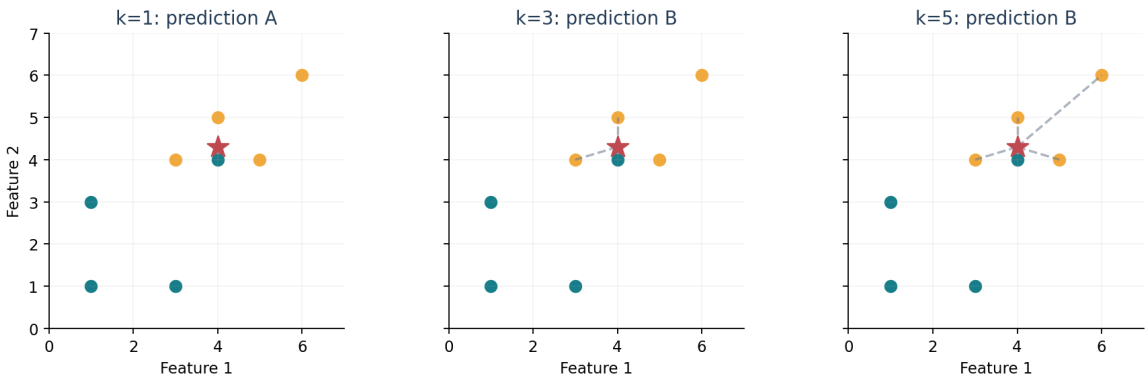
שלושת השכנים הקרובים הם A4, B3 ו־B1. התוויות שלהם הן A, B ו־B. למחלקה B יש שני קולות מול קול אחד של A, ולכן התחזית משתנה ל־B. זה אינו באג: שינוי k משנה את הכלל שבו משתמש המודל כדי להסיק מהסביבה המקומית.

k = 3 → votes: A = 1, B = 2 → prediction B

6.3 כאשר k = 5

בין חמשת השכנים הקרובים נמצאות ארבע תצפיות B ותצפית A אחת. גם כאן המחלקה המנבאת היא B. אין כלל מדעי שמבטיח כי k=5 יהיה מדויק יותר מ־k=3; הדוגמה מדגימה רק את החיזוי על נקודה אחת, לא את שיעור ההצלחה של המודל על אוכלוסייה.

k = 5 → votes: A = 1, B = 4 → prediction B



איור 4. אותה נקודה Q מקבלת תווית A עבור שכן יחיד, ותווית B עבור שלושה וחמישה שכנים. הקווים המקווקווים מציינים את השכנים המשתתפים בהחלטה.

7. כיצד בוחרים k, ומה קורה כאשר ההצבעה מסתיימת בתיקו?

k הוא היפר־פרמטר: ערכו נבחר על ידי מפתח המודל לפני הפעלת החיזוי, ולא נלמד אוטומטית על ידי פעולת fit של KNN הרגיל. ערך קטן מאוד עשוי להיות רגיש לרעש ולתצפיות יוצאות דופן; ערך גדול מדי עלול לערבב אזורים בעלי תכונות שונות ולהחליש החלטות מקומיות. עם זאת, אין נוסחה אחת הקובעת את k הנכון לכל מאגר.

כאשר יש שתי מחלקות, ערך k אי־זוגי יכול לצמצם תיקו במספר הקולות, בתנאי שכל השכנים שנבחרו מוגדרים באופן חד־משמעי; הוא אינו פותר כל מקרה של תיקו, ובעיה מרובת מחלקות יכולה להסתיים בתיקו גם ב־k אי־זוגי.

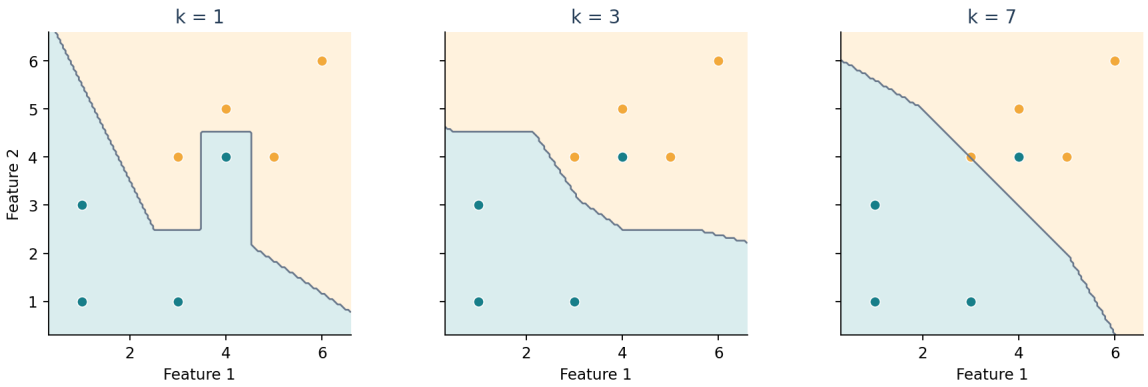
7.1 מדיניות השוויון בספר

כאשר נקבל מספר קולות זהה לשתי מחלקות, נפעיל מדיניות לימודית שהוגדרה מראש: נבדוק מהי המחלקה של השכן הקרוב ביותר מבין המחלקות שנקשרו. אם גם המרחקים שווים בדיוק, נציג ״אין הכרעה״ ונדרוש כלל נוסף שהוגדר מראש. הבחירה הזאת היא לצורכי דוגמאות ידניות בלבד ואינה הבטחה לגבי המימוש של ספרייה מסוימת.

אפשר להגדיר מדיניות אחרת, כגון בחירת התווית הראשונה בסדר קבוע, החזרת אי־הכרעה או שימוש בהצבעה משוקללת. העיקר הוא לכתוב אותה במפורש וליישם אותה באופן עקבי, במיוחד בבחינת בגרות או בהשוואת מימוש ידני לספרייה.

8. גבולות החלטה — כיצד k משנה את המפה?

אם נשאל את המסווג על כל נקודה אפשרית במישור, נקבל חלוקה לאזורים: בכל אזור המודל ינבא את אותה מחלקה. קו המפריד בין אזורים הוא גבול החלטה. כאשר k קטן, הגבול עשוי להתפתל סביב נקודות בודדות; כאשר k גדול, הוא עשוי להיות חלק יותר, אך עלול לאבד מידע מקומי.



איור 5. גבולות החלטה על אותן שמונה דוגמאות עבור $k=1$, $k=3$ ו־$k=7$. האזורים מחושבים במימוש scikit-learn וממחישים השפעה אפשרית של k על צורת ההחלטה.

לא בכל מאגר הגדלת k מחליקה את הגבול באותו אופן, ומורכבות הגבול אינה הוכחה בפני עצמה למודל מוצלח. נבחן את איכות החיזוי על דוגמאות שלא השתתפו באימון ועל פי המדדים המתאימים לבעיה.

9. מדוע תקנון ונרמול חשובים כל כך ל־KNN?

למדנו בפרק 2.3 כי מרחקים מושפעים מהסולם המספרי של כל מאפיין. למשל, אם מאפיין אחד נמדד בשקלים בטווח של אלפים והאחר בציון 1–5, השינוי במאפיין הראשון עלול להשתלט על המרחק הכולל גם אם מבחינה מקצועית שני המאפיינים חשובים. KNN אינו ״יודע״ להבחין בין יחידות מדידה: הוא מחשב את הנוסחה שהגדרנו.

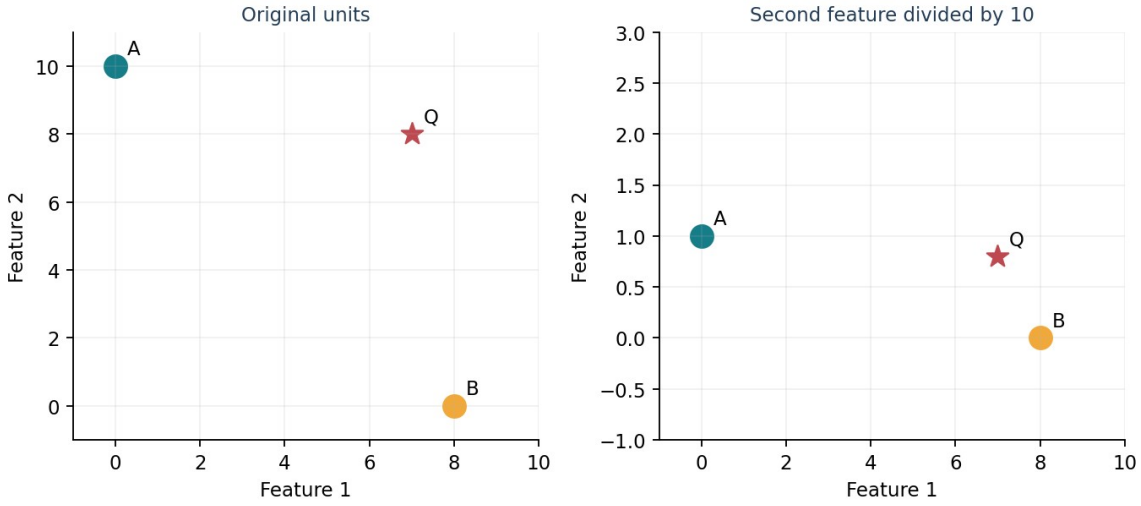
9.1 דוגמה שבה השכן הקרוב מתחלף

$$A=(0,10) \quad B=(8,0) \quad Q=(7,8)$$

$$\text{Before scaling: } d(Q,A)=\sqrt{53}\approx 7.28; \quad d(Q,B)=\sqrt{65}\approx 8.06$$

לפני שינוי קנה המידה A קרוב יותר. נניח שכעת מבטאים את המאפיין השני ביחידות קטנות פי עשרה, כלומר כל ערכי y מחולקים ב־10, גם בנתוני האימון וגם בדוגמה החדשה. המרחקים החדשים שונים, והקרוב מבין השניים מתחלף.

$$\text{After } y/10: d(Q,A)=\sqrt{49.04}\approx 7.003; \quad d(Q,B)=\sqrt{1.64}\approx 1.281$$



איור 6. אותו מצב גיאומטרי המיוצג בשני סולמות מספריים. אחרי שינוי סולם המאפיין השני השכן הקרוב מתחלף מ־A ל־B. זו הדגמת רגישות, לא המלצה לחלק מאפיין באופן שרירותי.

בפרויקט אמיתי לא בוחרים סקיילינג באופן שרירותי כדי לקבל תשובה רצויה. מגדירים שיטת הכנה מקצועית, כגון StandardScaler או MinMaxScaler, מתאימים אותה אך ורק לנתוני האימון ומיישמים את אותה טרנספורמציה על נתוני אימות, בדיקה ושימוש עתידי.

10. הצבעת רוב רגילה לעומת הצבעה משוקללת לפי מרחק

ב־KNN הבסיסי לכל אחד מ־k השכנים יש קול אחד, ללא קשר למרחק. לעומת זאת, בגישה משוקללת שכנים קרובים מקבלים השפעה גדולה יותר. אחת השיטות האפשריות נותנת משקל הפוך למרחק: שכן במרחק 0.3 משפיע יותר משכן במרחק 1.0. כאשר יש שכן במרחק אפס, נדרשת מדיניות מספרית מתאימה כדי להימנע מחלוקה באפס; ספריות מקצועיות מטפלות בכך לפי המימוש שלהן.

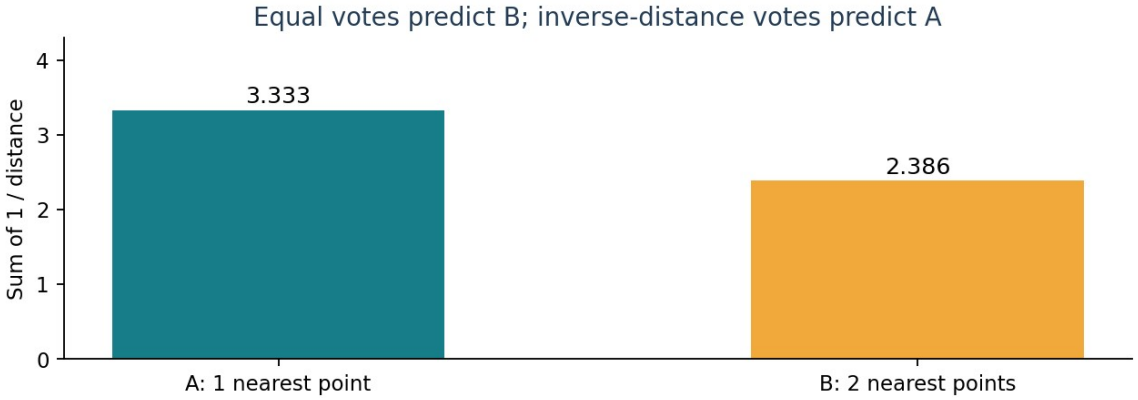
$$\text{weight}_i = 1 / d(Q, X_i), \quad \text{for } d > 0$$

בדוגמה שלנו, עבור k=3 יש שכן אחד ממחלקה A במרחק 0.30 ושני שכנים ממחלקה B במרחקים 0.70 ו־1.09√. בהצבעת רוב רגילה B זוכה ב־2 קולות. נחשב כעת את סכום המשקולות במקום את מספר הקולות.

$$A: 1 / 0.30 \approx 3.333$$

$$B: 1 / 0.70 + 1 / \sqrt{1.09} \approx 1.429 + 0.958 = 2.387$$

לכן בשיטת המשקולות הזו דווקא A מקבלת את הציון המשוקלל הגבוה יותר. זו דוגמה לכך שהגדרת המודל כוללת לא רק את k אלא גם את אופן הצבעת השכנים. בספריית scikit-learn משתמשים בפרמטר weights; ברירת המחדל היא uniform, ואילו distance מעניקה משקל גדול יותר לשכנים הקרובים יותר.



איור 7. סכום משקלים עבור k=3. המודל המשוקלל בוחר A, בעוד שהצבעת הרוב הרגילה בוחרת B. אין לפרש את סכום המשקלים כהסתברות מכוילת.

11. השוואה: KNN מול Nearest Centroid ומול K-Means

שלושת האלגוריתמים שייכים לעולם של וקטורים ומרחקים, אך פותרים שאלות שונות. חשוב לא להתבלבל בגלל השימוש במילים דומות כגון "שכן", "מרכז" ו"קבוצה".

תכונה	KNN	Nearest Centroid	K-Means
סוג למידה	מונחית	מונחית	לא־מונחית
תוויות ידועות באימון	כן	כן	לא
הייצוג להחלטה	k דוגמאות קרובות	מרכז אחד למחלקה	k מרכזי קבוצות
פלט עיקרי	מחלקה מנובאת	מחלקה מנובאת	שיוך לקבוצה
תלות ב־k	מספר שכנים	אין k של שכנים	מספר קבוצות

KNN לא מחליף את K-Means, ו־K-Means אינו מסווג המנבא תווית חיצונית ידועה אלא אם מוסיפים לו שלב פרשנות או מיפוי נפרד. בפרויקט גמר בחירת האלגוריתם צריכה להתחיל בשאלת המחקר: האם יש תוויות אמת ידועות? מהו הפלט המבוקש?

12. איך מממשים KNN ביד באמצעות Python?

תחילה נשתמש בקוד פשוט שמחקה בדיוק את פעולת החישוב הידני. המטרה אינה כתיבת התוכנה המהירה ביותר אלא להבין את רצף הפעולות. בקוד ההדגמה המלא נשתמש במקרה שאין בו תיקו קולות; במדיניות לשוויון עסקנו קודם.

```
import numpy as np

X = np.array([[1,1], [1,3], [3,1], [4,4],
              [3,4], [5,4], [4,5], [6,6]], dtype=float)
y = np.array(["A", "A", "A", "A", "B", "B", "B", "B"])
```

```
q = np.array([4.0, 4.3])

distances = []
for i in range(len(X)):
    distances.append(np.linalg.norm(X[i] - q))

order = np.argsort(distances, kind="stable")
nearest = order[:3]
labels = y[nearest]
print(labels)             # ['A' 'B' 'B']
print(np.unique(labels, return_counts=True))
```

בשורה np.linalg.norm מחשבים אורך של וקטור ההפרשים — כלומר מרחק אוקלידי. np.argsort מחזירה אינדקסים של ערכים בסדר עולה, ולא את המרחקים עצמם. בחירת אינדקסים מאפשרת לשלוף את אותן תוויות בדיוק. כאן ביקשנו מיון יציב כדי לשמור על סדר מקורי כאשר המרחקים שווים; זהו כלל מיון, לא מדיניות כללית להכרעה בתיקו הצבעות.

במחברת התלמיד תכתבו חלק מן הפעולות בעצמכם, ותבדקו כל שלב בעזרת print. לא נתקדם להפעלת ספרייה לפני שהתלמיד מסוגל להסביר מה יש בכל אחת מן המשתנים distances, order, nearest ו־labels.

13. מימוש מקצועי באמצעות scikit-learn

בפרויקט מעשי משתמשים לרוב במימוש שנבדק ונכתב על ידי מומחים. ב־scikit-learn מחלקת KNeighborsClassifier מאפשרת להגדיר את k, את מדד המרחק ואת משקל ההצבעה. החיבור בין הידע המתמטי לספרייה חשוב: fit אינו מבצע את אותו אימון של רשת עצבית; predict מבצע חיפוש שכנים והחלטה.

```
from sklearn.neighbors import KNeighborsClassifier

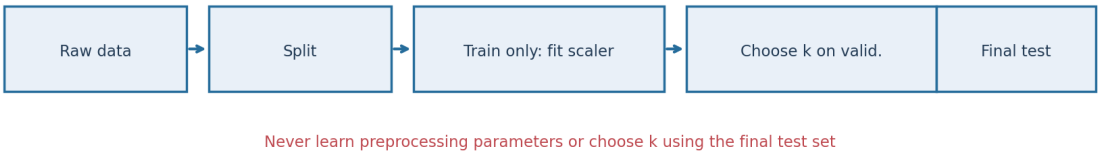
model = KNeighborsClassifier(n_neighbors=3,
                             weights="uniform",
                             metric="euclidean")
model.fit(X, y)
print(model.predict([[4.0, 4.3]]))  # ['B']

neighbor_distances, neighbor_indices = model.kneighbors([[4.0, 4.3]])
print(neighbor_distances)
print(neighbor_indices)
```

שימו לב למבנה [[4.0, 4.3]]: הספרייה מקבלת מטריצה בעלת שורה אחת ושתי עמודות, ולא וקטור חד־ממדי בלבד. בפונקציה kneighbors הפלט כולל את המרחקים ואת האינדקסים של השכנים. כאשר יש מספר שכנים בדיוק באותו מרחק, סדרם עשוי להיות תלוי במימוש; לפיכך אין לבסס מסקנה על סדר שכנים קשורים בלי להגדיר כללים מתאימים.

14. איך בודקים ביצועים ומונעים זליגת מידע?

דוגמה ידנית אחת אינה מספיקה כדי להחליט שהמודל טוב. יש למדוד את הצלחתו על תצפיות שלא השתתפו בפיתוח ובבחירת k. אנחנו משתמשים בשלוש קבוצות: Train, Validation ו־Test. תחילה נפריד Test; לאחר מכן נבחר k ונבדוק גישות עיבוד על בסיס Train ו־Validation. רק לאחר שהגדרנו את כללי העבודה נבחן את התוצאה על Test פעם אחת לצורכי הערכה מסכמת.



איור 8. רצף הערכה מקצועי: הפרדה, התאמת טרנספורמציות על Train, בחירת k באמצעות Validation והערכה סופית על Test שלא שימש לבחירה.

דוגמת הקוד הבאה משתמשת ב־Iris שבספריית scikit-learn. המחלקה החיובית אינה נדרשת לחישוב Accuracy פשוט, אך בהמשך נלמד מדדים המתייחסים במפורש ל־TP, FP ו־FN. הקוד מדגים תהליך נכון: הוא אינו מבטיח מראש תוצאה מספרית מסוימת.

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

iris = load_iris()
X = iris.data
y = iris.target

X_dev, X_test, y_dev, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y)
X_train, X_val, y_train, y_val = train_test_split(
    X_dev, y_dev, test_size=0.25, random_state=42, stratify=y_dev)

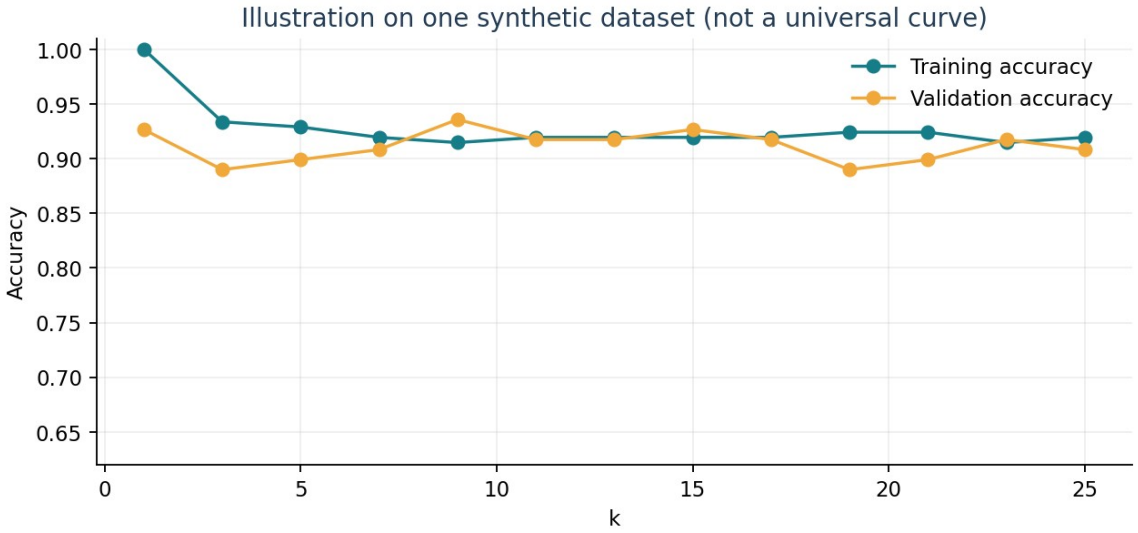
scores = {}
for k in [1, 3, 5, 7]:
    model = make_pipeline(StandardScaler(),
                          KNeighborsClassifier(n_neighbors=k))
    model.fit(X_train, y_train)
    scores[k] = accuracy_score(y_val, model.predict(X_val))

best_k = max(scores, key=scores.get)
final_model = make_pipeline(StandardScaler(),
                            KNeighborsClassifier(n_neighbors=best_k))
final_model.fit(X_dev, y_dev)
print("Chosen k:", best_k)
print("Final test accuracy:", final_model.score(X_test, y_test))
```

הקוד בוחר k לפי Validation בלבד. לאחר הבחירה אפשר לאמן מחדש את הצינור על כל נתוני הפיתוח X_dev, שבהם כלולות גם תצפיות ה־Validation, ואז להעריך פעם אחת על X_test. אם מתקיינים גרסת ספרייה אחרת, יש לוודא שהקוד פועל בגרסה הנוכחית ולהכיר את משמעות הפרמטרים.

15. למה לא לבחור k לפי Accuracy על נתוני האימון?

מודל עם k=1 יכול לעיתים להגיע לדיוק אימון גבוה מאוד כי כל דוגמת אימון היא השכנה הקרובה ביותר לעצמה. אך אין בכך הבטחה שהוא יסווג נכון נתונים חדשים. בניסוי הבא יצרנו נתונים סינתטיים בעלי שתי קבוצות עם רעש, ואימנו מודלים עבור ערכי k שונים. בגרף מציגים בנפרד את דיוק האימון ואת דיוק האימות.



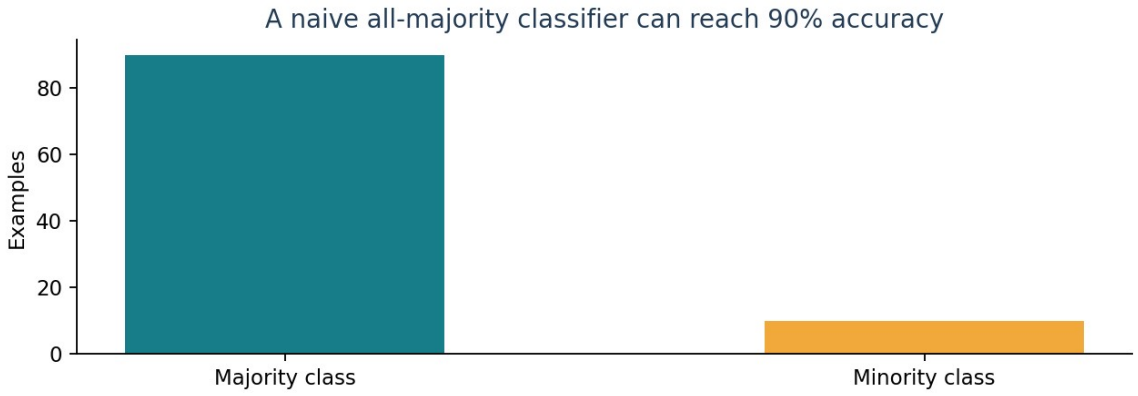
איור 9. דוגמה מניסוי אחד בנתונים סינתטיים: דיוק האימון והאימות עשויים להשתנות אחרת כאשר k משתנה. צורת העקומה תלויה במאגר, בחלוקה ובזרע האקראי; אין להסיק ממנה k מיטבי לכל בעיה.

כאשר k קטן מדי, המודל עלול להיות רגיש לרעש (Overfitting). כאשר k גדול מדי, הוא עלול להחמיץ מבנה מקומי חשוב (Underfitting). השאלה אינה מה נראה טוב יותר בגרף האימון, אלא איזה מודל מכליל בצורה אמינה לדוגמאות חדשות. בחירה בעזרת Cross-validation תילמד לעומק בפרק 5.2.

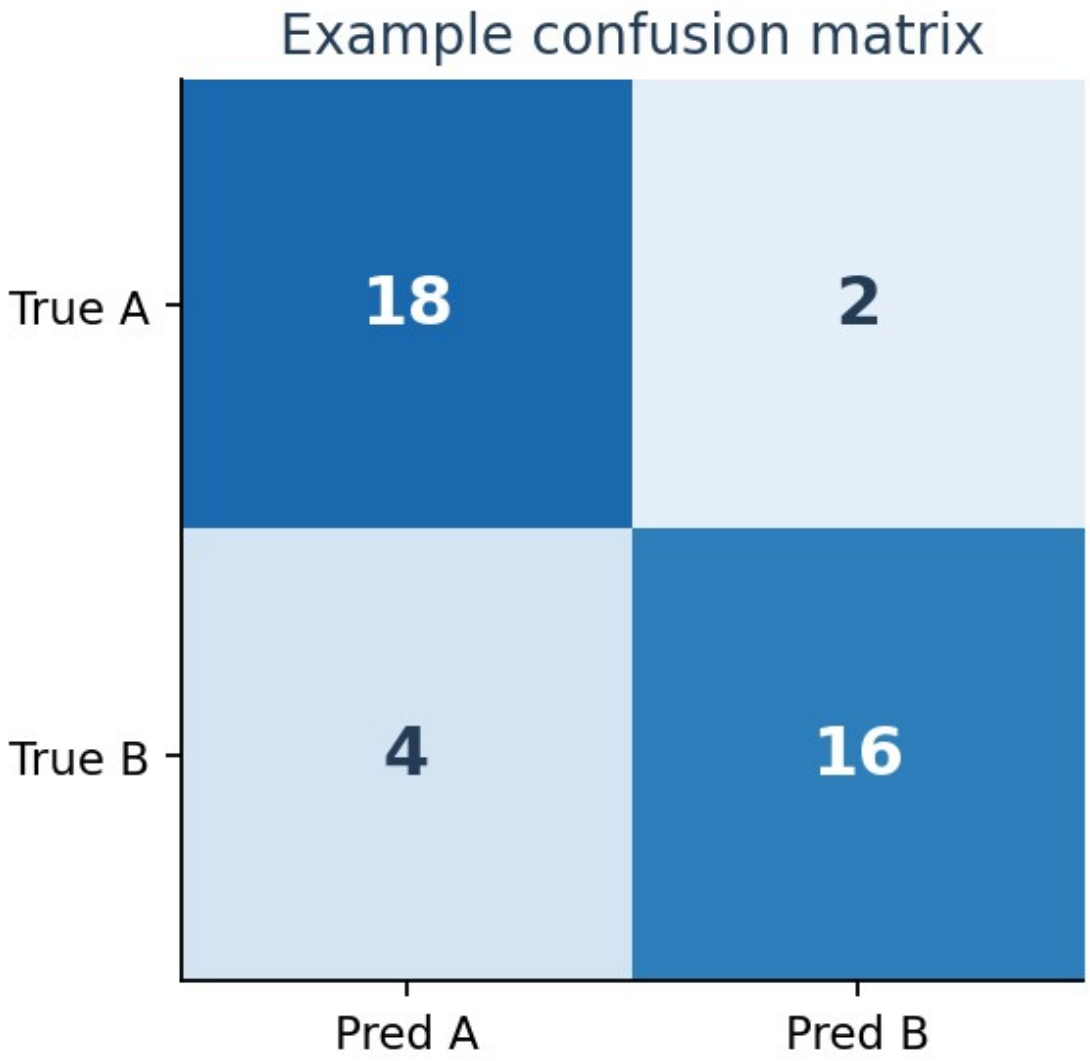
16. מדדי הערכה — למה Accuracy לא מספיק תמיד?

Accuracy הוא אחוז התחזיות הנכונות מתוך כלל התחזיות. אם יש 90 תצפיות מסוג A ורק 10 מסוג B, מסווג שמנבא תמיד A יקבל Accuracy של 90%, ובכל זאת יפספס את כל דוגמאות B. לכן צריך לשאול מה מחיר הטעויות ולבדוק את פירוטן באמצעות מטריצת בלבול ומדדים נוספים.

$$\text{Accuracy} = (\text{number of correct predictions}) / (\text{number of predictions})$$



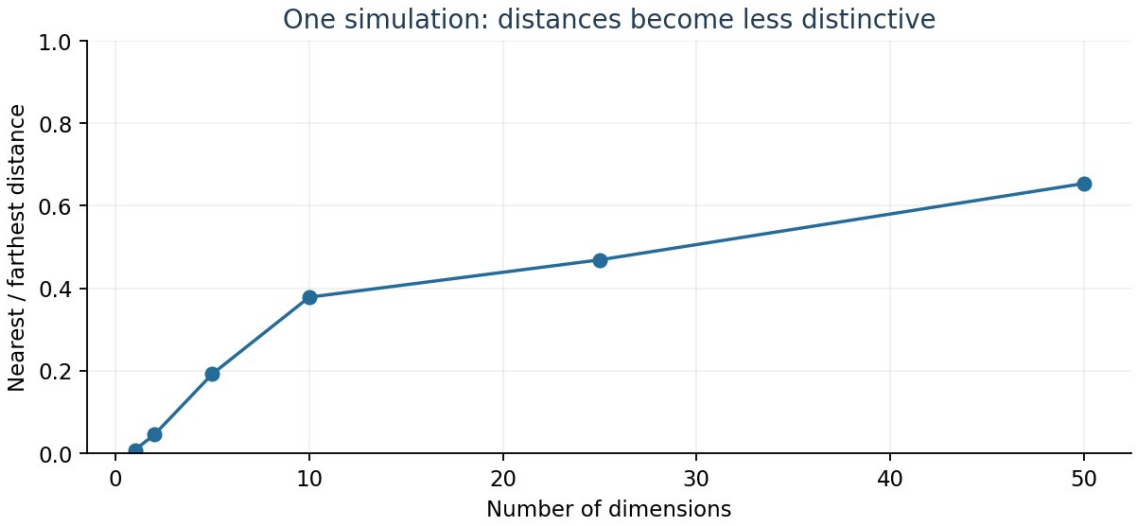
איור 10. מאגר לא מאוזן: 90 דוגמאות מסוג אחד ו־10 מהשני. דיוק של 90% לבדו אינו מעיד על איתור מוצלח של המחלקה הנדירה.



איור 11. מטריצת בלבול לדוגמה: 18 סיווגי A נכונים, 16 סיווגי B נכונים, ושש טעויות בסך הכול. Accuracy הוא 34/40 = 0.85; יש לנתח גם את סוגי הטעויות.

17. הקשר לממדים רבים — מתי מרחק מפסיק להיות מבחין?

כאשר מוסיפים עוד ועוד מאפיינים, המרחקים בין נקודות אקראיות עשויים להפוך דומים יחסית, והרעיון של ״שכנים קרובים״ נעשה פחות אינפורמטיבי. זו אחת התופעות המוכרות בשם קללת הממדיות. היא אינה אומרת ש־KNN תמיד נכשל כאשר מספר המאפיינים גדול, אלא מזכירה לנו שצריך לבדוק בחירת מאפיינים, תקנון, כמות דוגמאות ואיכות ההערכה.



איור 12. ניסוי סימולציה יחיד: היחס בין מרחק הנקודה הקרובה ביותר לבין הרחוקה ביותר עולה כאשר מוסיפים ממדים אקראיים. הנתונים סינתטיים, והמגמה אינה הבטחה לכל מאגר.

18. טעויות נפוצות — וכיצד מתקנים אותן

הטעות	מדוע היא בעייתית	דרך בדיקה
בחירת k=1 כי דיוק האימון גבוה	עלולה להעדיף זכירת נקודות על הכללה	בדיקת Validation נפרדת
מיון מרחקים וניתוקם מהתוויות	התווית של שכן כבר אינה מתאימה למרחקו	שמירת אינדקסים עם argsort
תקנון לפני חלוקת Train/Test	ממוצע ומידע מ־Test משפיעים על אימון	Pipeline מותאם על Train בלבד
הנחה שהצבעת 2 מתוך 3 היא הסתברות 66%	זהו שיעור קולות, לא הסתברות מכוילת	לציין הצבעה ולא ביטחון מוכח
הנחה ש־k אי־זוגי תמיד פותר תיקו	בתיקו מרחקים או בריבוי מחלקות ייתכנו בעיות	להגדיר כלל הכרעה מראש
שימוש במזהה תלמיד כמאפיין	מזהה לא בהכרח מתאר תופעה מנובאת	לבדוק משמעות כל עמודה

19. שאלות תרגול — רמה א: הבנה וביצוע

19.1 הגדרו במילים שלכם את המונח ״שכן״ ב־KNN. כיצד הוא שונה משכן במציאות הגיאוגרפית?

19.2 חשבו את המרחק בין Q=(2,5) לבין A=(5,9). כתבו כל שלב והסבירו את יחידות המדידה.

19.3 נתונות תוויות של חמשת השכנים הקרובים: A,B,A,A,B. מה ינבא המסווג עבור k=3 ועבור k=5?

19.4 מה מחזירה np.argsort? הסבירו מדוע אסור למיין את רשימת המרחקים בלי לשמור אינדקסים.

19.5 האם KNN הוא אלגוריתם ללמידה מונחית? ציינו איזה מידע מוכרח להופיע בדוגמאות האימון.

20. שאלות תרגול — רמה ב: ניתוח ושינוי תנאים

20.1 בתרגיל שבו k=4 התקבלו שני קולות לכל מחלקה. הציעו מדיניות הכרעה שקופה ובחנו כיצד היא תשפיע על אמינות המודל.

20.2 מצאו בדוגמת דודו את תוויות k=1, k=3 ו־k=5. הסבירו מדוע ההחלטה יכולה להשתנות גם בלי לשנות אף תצפית אימון.

20.3 שני מאפיינים נמדדים בסולמות שונים. הסבירו כיצד תקנון עלול לשנות את השכן הקרוב — ומדוע הוא אינו ״משנה את המציאות״.

20.4 הסבירו מדוע הערכת k על Test ואז דיווח אותו Test כציון סופי אינה בדיקה הוגנת.

20.5 יומן ניסוי כולל Accuracy אימון 1.0 ו־Accuracy אימות 0.74 עבור k=1. הציעו שני כיווני חקירה זהירים.

21. שאלות תרגול — רמה ג: חשיבה בסגנון בגרות

שאלה 21.1 — נתונות שמונה דוגמאות האימון בתחילת הפרק והנקודה החדשה Q=(4,4.3). א. בחרו את שלושת השכנים הקרובים על סמך חישוב מלא. ב. נבאו את המחלקה. ג. נמקו מדוע k=1 משנה את התוצאה. ד. הציעו בדיקה אחת שתסייע לבחור k בפרויקט אמיתי, בלי להשתמש בתוצאות Test לצורך בחירה.

שאלה 21.2 — תלמיד כתב את השורות: `distances=np.linalg.norm(X-q,axis=1); y_neighbors=y[np.sort(distances)[:3]]`. הסבירו מהי השגיאה המושגית במשפט השני, ומהו סוג המידע הנחוץ במקום ערכי המרחק עצמם.

שאלה 21.3 — מערכת זיהתה 980 מתוך 1000 דוגמאות מסוג A, אך רק 1 מתוך 20 דוגמאות מסוג B. חשבו Accuracy, והסבירו מדוע מערכת כזאת עלולה להיות לא מספקת למרות תוצאה כוללת גבוהה.

כל השאלות מקוריות ונכתבו בסגנון מיומנויות הנדרשות בלמידה עיונית; אין להציג אותן כשאלות בגרות רשמיות.

22. פעילות מעבדה — מתרגול מודרך לעצמאות

מחברת Colab המלווה את הפרק כוללת 12 משימות. התלמידים עובדים על מערך נתונים חדש של העדפות למידה, ולא על אותם מספרים מהספר, כדי לוודא העברת ידע. תחילה משלימים חישובי מרחקים ובחירת שכנים; בהמשך בונים מסווג, בודקים השפעת k, מבצעים תקנון תקין ומשווים לתוצאה בסיסית. חלק מן התאים כוללים הנחיות או סימני השלמה; פתרונות אינם נכללים במחברת התלמיד.

23. הקשר לפרויקט הגמר בלמידה עמוקה

גם אם בפרויקט הגמר נשתמש בהמשך ברשת עצבית, חשוב לבנות מודל בסיס להשוואה. KNN יכול לשמש Baseline למאגר טבלאי מתאים: אם רשת עמוקה אינה משפרת את התוצאה ביחס ל־KNN פשוט, יש לבדוק האם מורכבותה מוצדקת. על התלמיד לתעד בחירת מאפיינים, חלוקת נתונים, שיטת תקנון, k, מדד הערכה, תוצאות שגיאה ומגבלות.

עם זאת, KNN אינו תחליף אוטומטי לדרישת בנייה ואימון של מודל למידה עמוקה בפרויקט גמר שבו דרישה כזאת קיימת במחוון. הוא משמש נקודת השוואה ותשתית לחשיבה מדעית. אפשר גם להשוות אותו לשיטות של פרק 4.2, אך רק על אותם נתונים, באותה חלוקה ובהכנת מאפיינים הוגנת.

24. סיכום מושגים ונוסחאות

מושג / נוסחה	מה התלמיד חייב להסביר
k	מספר השכנים שבוחרים לצורך חיזוי; היפר־פרמטר
$d_2(u,v)=\Sigma(u_i-v_i)^2$	מרחק אוקלידי בין דוגמאות במרחב המאפיינים
`np.argsort(distances)`	מיון אינדקסים תוך שמירה על ההתאמה בין דוגמה

מה התלמיד חייב להסביר	מושג / נוסחה
לתווית	
בחירת המחלקה בעלת מספר הקולות הגבוה מבין השכנים k	הצבעת רוב
הענקת השפעה גבוהה יותר לשכנים קרובים	weights=distance
אימון, בחירת פרמטרים והערכה עצמאית	Train / Validation / Test
שיעור תחזיות נכונות — לא תמיד מספיק בנוכחות חוסר איזון	Accuracy

כרטיס יציאה: האם תלמיד מסוגל להסביר בקול כיצד הוחלטה תווית עבור Q, למה k=1 ו־k=3 שונים, מה ישתנה בעת תקנון ומה נדרש כדי לטעון שהמודל מכליל? אם אחת התשובות אינה ברורה, חזרו לטבלת השכנים לפני מעבר לפרק הבא.

25. עוגן בתכניות הלימודים ומקורות ללמידה נוספת

מיקום בתכנית: יחידה 5, פרק 4 — מודלים מבוססי מרחק, סעיף 4.3 KNN, על פי מפת התכנית השמורה במאגר שלנו. יחידה 3 מספקת בסיס מעשי של KNN ואילו במסלול הלמידה העמוקה נלמד גם מימוש מפורט עם NumPy ובדיקת נתונים. ציון המיקום במפת המאגר אינו לבדו אישור שכל הרחבה בפרק נדרשת בבגרות; ההיקף המחייב נקבע במסמך משרד החינוך ובמפרט הבחינה התקף.

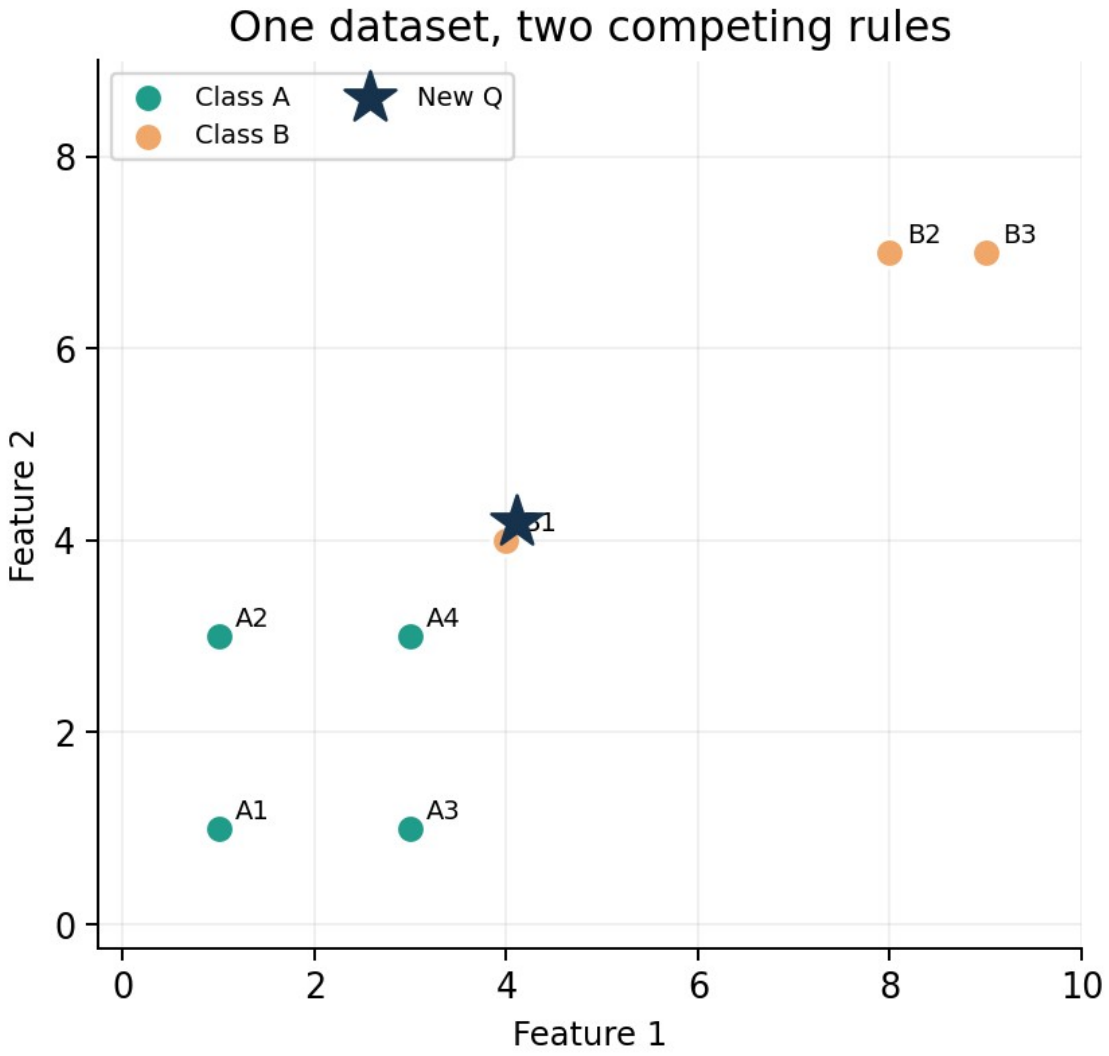
מקורות השראה מקצועיים (ללא העתקת טקסט או איורים): Luis Serrano, Grokking Machine Learning; Aurélien Géron, Hands-On Machine Learning; James An Introduction to Statistical Learning ועמיתיו, Mathematics for Machine Learning ועמיתיו, with Applications in Python; Deisenroth Josh Starmer, ו־ StatQuest. הגדרות פרמטרים והתנהגות הספרייה: התיעוד הרשמי של scikit-learn.

קישורים: https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.KNeighborsClassifier.html | https://scikit-learn.org/stable/common_pitfalls.html | https://github.com/samiabo75/Data-Science-ML-Course/tree/main/02-Unit5-AI-Core/04-Distance-Based-Models

פרק זה הוא תוכן לימודי מקורי בעברית. איורים וסיפורי הנתונים נוצרו לצורכי הוראה; הנתונים הסינתטיים אינם ראיה אמפירית. פתרונות המעבדה המלאים יישמרו בגרסת המורה בלבד.

פרק 4.4 — השוואת מסווגים מבוססי מרחק

כיצד בוחרים בין KNN ל־Nearest Centroid, ומה הופך השוואה לניסוי מדעי הוגן?



איור 1. אותו מאגר נתונים יכול להניב כללי החלטה שונים. נבחן מדוע, ולא רק נשווה פלט של ספריות.

בפרקים 4.2 ו־4.3 למדנו שני מסווגים המבוססים על מרחק. כעת נעבור משאלת ״איך האלגוריתם פועל?״ לשאלה עמוקה יותר: ״מתי כדאי להשתמש בכל אלגוריתם, ואיך נדע שהבחירה אכן מתאימה לנתונים ולמטרה?״

הפרק עוגן בסעיף 4.4 של מפת נושאי הליבה ביחידה 5, ומכין לפרק 5 על הערכת ביצועים וחקר פרמטרים. הדוגמאות, הגרפים והשאלות נכתבו במיוחד לספר זה. ההעמקות על מורכבות חישובית, תהליך ניסוי ועלויות טעות הן הרחבות להכנה לפרויקט הגמר.

מטרות ביצועיות וידע קודם

בסיום הפרק תוכלו להסביר ולחשב החלטת Nearest Centroid והחלטת KNN על אותם נתונים; לזהות מתי התחזיות שונות ולנמק מדוע; לפרש גבול החלטה, השפעת קנה מידה, חריגים ופיזור המחלקות; לערוך השוואה הוגנת תוך מניעת זליגת מידע; ולהציע בחירה מנומקת למערכת AI.

ידע קודם: וקטורים ומרחק אוקלידי (1.1–1.2), מדדי מרכז ופיזור (2.1), תקנון (2.3), למידה מונחית (4.1), Nearest Centroid (4.2) ו־KNN (4.3).

1. סיפור מסגרת — דודו בוחר מסווג למעבדת המחשבים

דודו מפתח מערכת שתסייע לצוות הטכני להחליט האם מחשב חדש דומה יותר למחשבים שנזקקו לטיפול מסוג A או מסוג B. לכל מחשב שני מאפיינים מדידים: עומס עבודה ותדירות תקלות; לצורך הדוגמה שני המאפיינים מוצגים באותו סולם מספרי. התווית A או B נקבעה בעבר בבדיקה מקצועית, ולכן זו למידה מונחית.

יש בידינו שבע דוגמאות מתויגות. הנתונים מומצאים והם קטנים בכוונה: באמצעותם ניתן לבצע כל פעולה ביד, לוודא את התוצאה, ורק אז לעבור למימוש תוכנתי.

דוגמה	מאפיין 1	מאפיין 2	מחלקה
A1	1	1	A
A2	1	3	A
A3	3	1	A
A4	3	3	A
B1	4	4	B
B2	8	7	B
B3	9	7	B

מחשב חדש מתואר באמצעות הווקטור Q. אין לנו תווית ידועה עבורו: זוהי התוצאה שאנו מנסים לחזות.

```
Q = (4.1, 4.2)
```

2. שני האלגוריתמים — שתי שאלות שונות

Nearest Centroid מסכם את כל הדוגמאות של כל מחלקה באמצעות ממוצע רכיבי אחד. לאחר מכן הוא שואל: ״לאיזה מרכז מחלקה הנקודה החדשה קרובה יותר?״ זהו מודל דחוס, משום שברגע שחישבנו את המרכזים אין צורך בכל נקודות האימון לצורך החיזוי הרגיל.

KNN אינו מצמצם כל מחלקה למרכז אחד. לצורך חיזוי הוא בוחן את הדוגמאות המתויגות הקרובות ביותר ל־Q. בגרסת הסיווג הרגילה הוא שואל: ״איזו תווית שכיחה יותר בקרב k השכנים הקרובים?״ עבור k=1 מספיק שכן אחד, אך הוא עשוי להיות חריג או נקודה לא מייצגת.

שאלה	Nearest Centroid	KNN
מה נשמר לצורך חיזוי?	מרכז אחד לכל מחלקה	דוגמאות אימון מתויגות
מהו כלל החלטה?	המרכז הקרוב ביותר	הצבעת שכנים קרובים

שאלה	Nearest Centroid	KNN
פרמטר חשוב	בחירת מרחק ותקנון	k, מרחק, משקולות ותקנון
נטייה לגבול החלטה	ליניארי באוקלידי רגיל	עשוי להיות מורכב ולא־ליניארי

3. חישוב Nearest Centroid ביד — בלי קיצורי דרך

נחשב קודם את מרכז מחלקה A. יש בה ארבע נקודות. מרכז מחושב באמצעות ממוצע של כל עמודה בנפרד: ממוצע המאפיין הראשון וממוצע המאפיין השני. זו בדיוק פעולת הממוצע שלמדנו בפרק 2.1, אך כעת מבצעים אותה לווקטורים.

$$C_A = ((1+1+3+3)/4,\ (1+3+1+3)/4) = (2,2)$$

במחלקה B שלוש נקודות. גם כאן מחשבים ממוצע בכל ממד, ואסור לשכוח שהמחלקות אינן חייבות להכיל מספר דוגמאות זהה.

$$C_B = ((4+8+9)/3,\ (4+7+7)/3) = (7,6)$$

כדי לסווג את Q נחשב את המרחק האוקלידי לכל מרכז. חשוב להפריד בין שני השלבים: שלב יצירת המרכזים מתוך נתוני האימון, ושלב החיזוי עבור דוגמה חדשה.

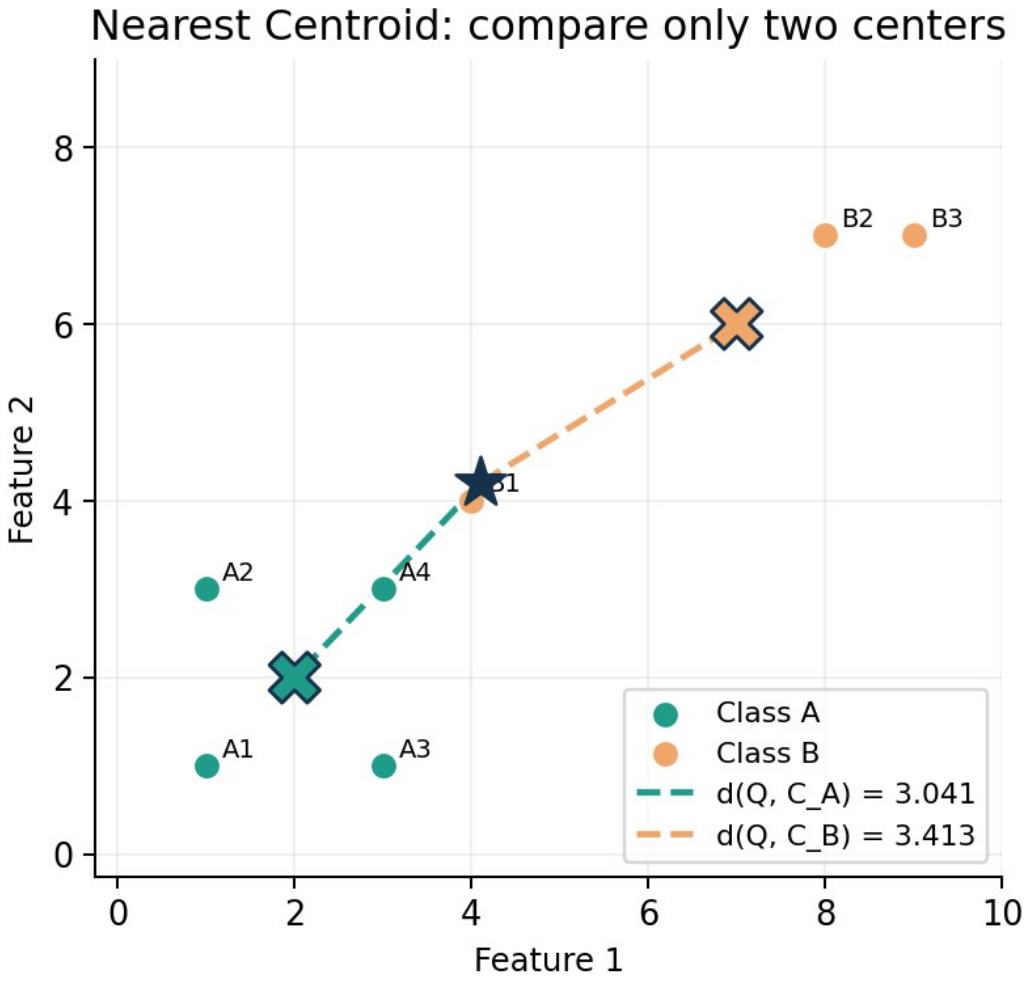
$$d(Q,C_A) = \sqrt{[(4.1-2)^2 + (4.2-2)^2]}$$

$$= \sqrt{(4.41 + 4.84)} = \sqrt{9.25} \approx 3.0414$$

$$d(Q,C_B) = \sqrt{[(4.1-7)^2 + (4.2-6)^2]}$$

$$= \sqrt{(8.41 + 3.24)} = \sqrt{11.65} \approx 3.4132$$

המרחק למרכז A קטן מן המרחק למרכז B. לכן התחזית של Nearest Centroid בדוגמה זו היא A. זכרו: זוהי החלטת המודל לפי כלל מסוים, ולא הוכחה שהתווית האמיתית של Q היא A.



איור 2. מרכזי שתי המחלקות מסומנים ב־X. המסווג משווה שני מרחקים בלבד.

4. חישוב KNN על אותם נתונים — מדוע התשובה עשויה להשתנות?

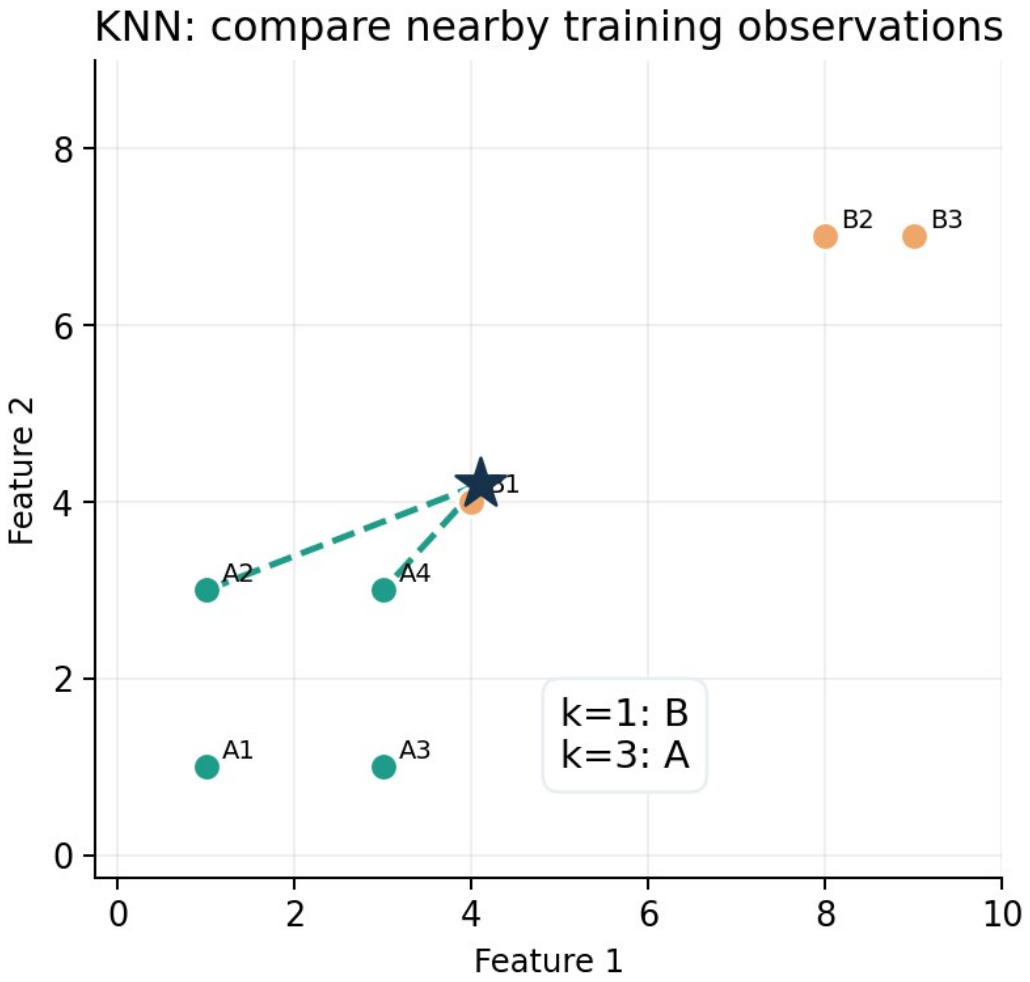
ב־KNN אין מחשבים ממוצע מחלקה כתנאי לחיזוי. מחשבים את המרחק של Q לכל אחת משבע דוגמאות האימון, ומסדרים אותן בסדר עולה. יש לשמור את המרחק יחד עם התווית כדי לא לבלבל בין שכנים.

מחשב	וקטור	מרחק מ־Q	תווית
A1	(1,1)	4.4553	A
A2	(1,3)	3.3242	A
A3	(3,1)	3.3838	A
A4	(3,3)	1.6279	A
B1	(4,4)	0.2236	B
B2	(8,7)	4.8010	B
B3	(9,7)	5.6436	B

הקרובה ביותר היא B1=(4,4), במרחק של כ־0.2236. לכן כאשר k=1 מסווג KNN מנבא B — בניגוד ל־Nearest Centroid, שניבא A. זו אינה טעות חישוב: שני האלגוריתמים פשוט משתמשים במידע אחר.

$$d(Q,B1) = \sqrt{[(4.1-4)^2 + (4.2-4)^2]} = \sqrt{0.05} \approx 0.2236$$

כאשר k=3, שלושת השכנים הקרובים ביותר הם B1, A4 ו־A2. ההצבעה היא שתי תוויות A ותווית B אחת, ולכן כעת KNN מנבא A. כך גם בתוך KNN עצמו הבחירה בערך k משנה את ההחלטה.



איור 3. הקווים מסמנים את שלושת השכנים הראשונים. אצל k=1 מתקבלת B, ואצל k=3 מתקבלת A.

5. מה למדנו מן הסתירה בין התחזיות?

הסתירה אינה מלמדת מי מהמסווגים ״צודק״. ללא תווית אמיתית של מחשב חדש, אי אפשר לחשב כאן דיוק. היא מלמדת שמרכז מחלקה מבטא התנהגות ממוצעת של כל המחלקה, ואילו שכן קרוב מתאר דמיון מקומי.

בנתונים שלנו, המחלקה B כוללת נקודה אחת הקרובה מאוד ל־Q ושתי נקודות רחוקות ממנה. שתי הנקודות הרחוקות מושכות את מרכז המחלקה B לכיוון ימין ולמעלה. המסווג שמסתמך על המרכז עשוי לפספס נקודת B מקומית, בעוד KNN עם k קטן עשוי לזהות אותה — או להאמין לה גם אם היא חריגה. כל שיטה משלמת מחיר אחר.

6. מהי משמעות גבול ההחלטה?

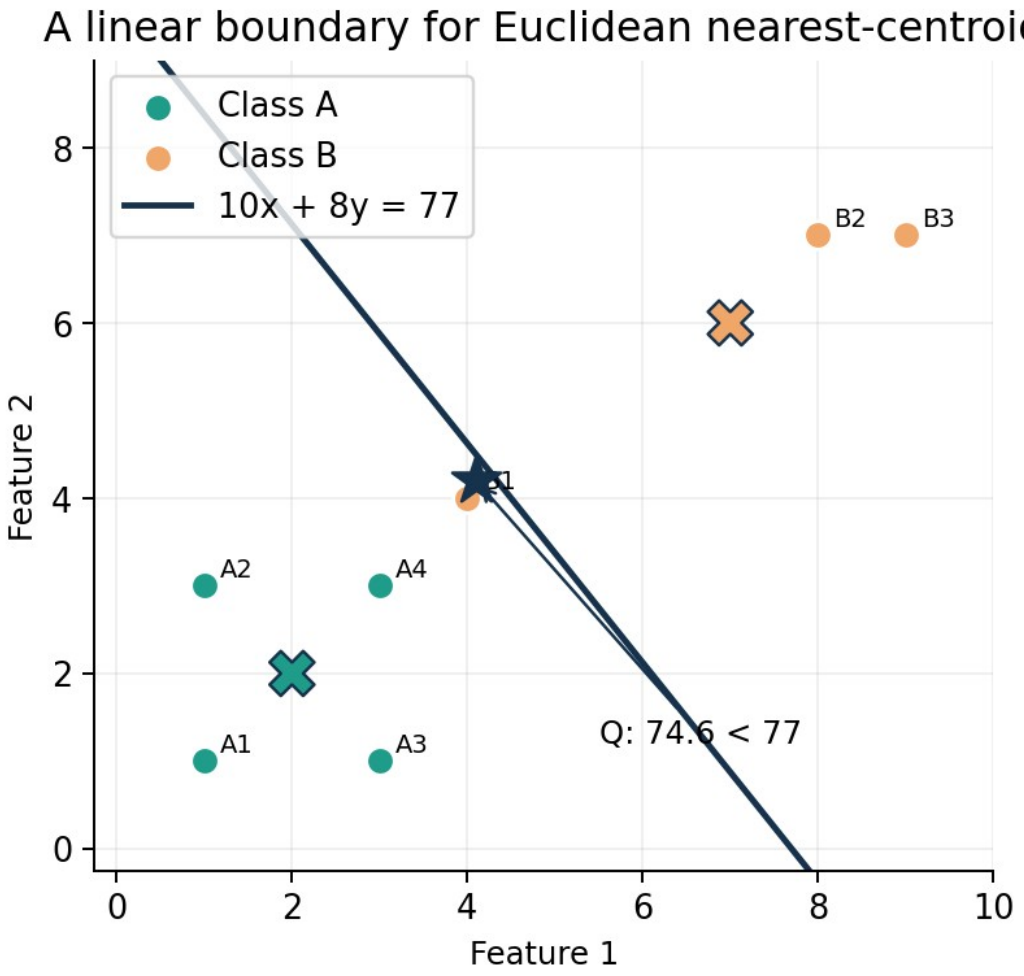
גבול ההחלטה הוא המקום שבו מודל עומד לשנות את תחזית המחלקה שלו. עבור Nearest Centroid עם שני מרכזים ומרחק אוקלידי, הגבול מורכב מכל הנקודות שמרחקן לשני המרכזים שווה. זהו האנך האמצעי לקטע המחבר את המרכזים; בשני ממדים מתקבל ישר.

$$(x-2)^2 + (y-2)^2 = (x-7)^2 + (y-6)^2$$

נפתח סוגריים ונחסר משני הצדדים את x^2 ואת y^2. איברי הריבוע מתבטלים, ולכן נותרת משוואה ליניארית. זה מראה מדוע גבול ההחלטה ישר ולא עקומה במקרה הזה.

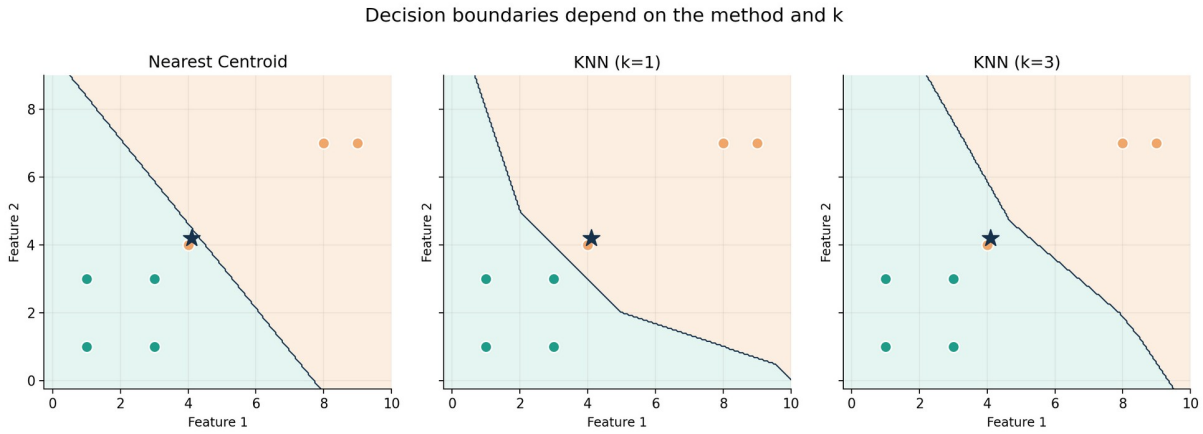
$$10x + 8y = 77$$

עבור Q נקבל באגף שמאל $10 \cdot 4.1 + 8 \cdot 4.2 = 74.6$. מאחר ש־74.6 קטן מ־77, הנקודה נמצאת בצד של המרכז A. אפשר לראות שהתוצאה תואמת את השוואת המרחקים הקודמת.



איור 4. גבול ההחלטה הליניארי של Nearest Centroid; התחזית נקבעת לפי הצד שבו נמצאת Q.

ב־KNN, לעומת זאת, נקודות ההכרעה תלויות במיקום התצפיות עצמן ובערך k. כאשר k=1 אזורי ההחלטה קשורים לחלוקת Voronoi של הדוגמאות; עבור k גדול יותר נוצר גבול שעשוי להיות חלק יותר אך עדיין לא־ליניארי.



איור 5. שלושה גבולות החלטה על אותם נתונים: מרכזי מחלקה, KNN עם שכן אחד ו־KNN עם שלושה שכנים.

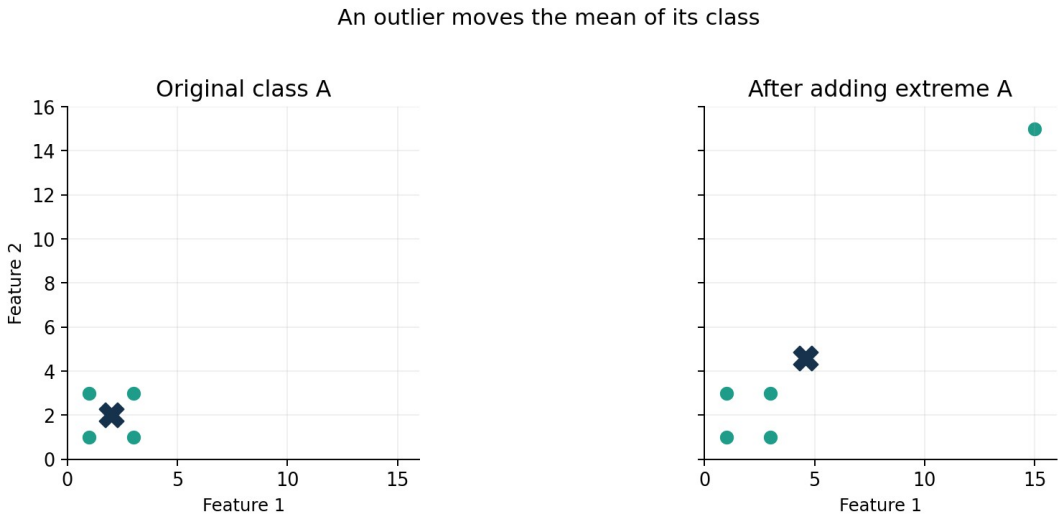
7. האם מסווג מורכב יותר תמיד טוב יותר?

לא. גבול החלטה מפורט יכול להתאים טוב מאוד לנתוני האימון, אך להגיב בעוצמה לרעש ולחריגים. מנגד, גבול פשוט עשוי להתעלם מצורת נתונים מורכבת. המושגים Underfitting ו־Overfitting מתארים שני סיכונים שונים: פשטות מופרזת לעומת התאמה רבה מדי לפרטים מקריים.

בחירה בין המודלים דורשת הסתכלות על דוגמאות שלא שימשו להתאמת המודל, ולא רק הערצה של גרף יפה או דיוק אימון גבוה. מודל פשוט יותר הוא לעיתים נקודת פתיחה טובה, אך אין כלל שאומר שהוא תמיד עדיף.

8. חריגים, פיזור מחלקות והגבלות של ממוצע

נניח שבמחלקה A מתקבלת מדידה חריגה מאוד של מחשב נוסף. מאחר שמרכז המחלקה הוא ממוצע, המדידה החריגה עשויה להזיז אותו באופן משמעותי. לעומת זאת, KNN קטן לא בהכרח יושפע מן החריג אם הוא רחוק מהדוגמה שאנו מסווגים; אולם אם החריג נמצא קרוב, הוא עלול דווקא להכריע את התוצאה.

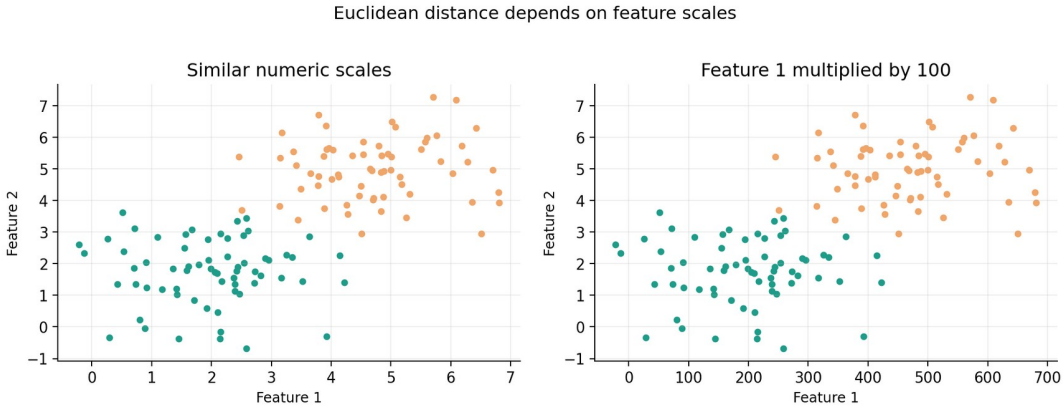


איור 6. תצפית חריגה משנה את מרכז המחלקה, גם אם מרבית נקודותיה אינן זזות.

יש גם בעיה של צורת הפיזור: שתי מחלקות יכולות להיות בעלות מרכזים קרובים או זהים, אף שהנקודות מסודרות בצורות שונות. מרכז יחיד אינו מתאר אורך, כיוון, צפיפות או תת־קבוצות. לכן לפני בחירת אלגוריתם חשוב לצייר את הנתונים ולבדוק מה משמעות המאפיינים.

9. השפעת קנה המידה — חזרה חיונית מפרק 2.3

מרחק אוקלידי מחבר את ריבועי ההפרשים בין המאפיינים. אם מאפיין אחד נמדד באלפים ומאפיין אחר ביחידות, הראשון עשוי לשלוט בחישוב גם כאשר אינו חשוב יותר מבחינה מקצועית. הדבר משפיע על שני המסווגים, מפני ששניהם משתמשים במרחקים.



איור 7. שינוי יחידות של מאפיין משנה את צורת המרחב המספרי, בלי לשנות את המידע המהותי על התצפיות.

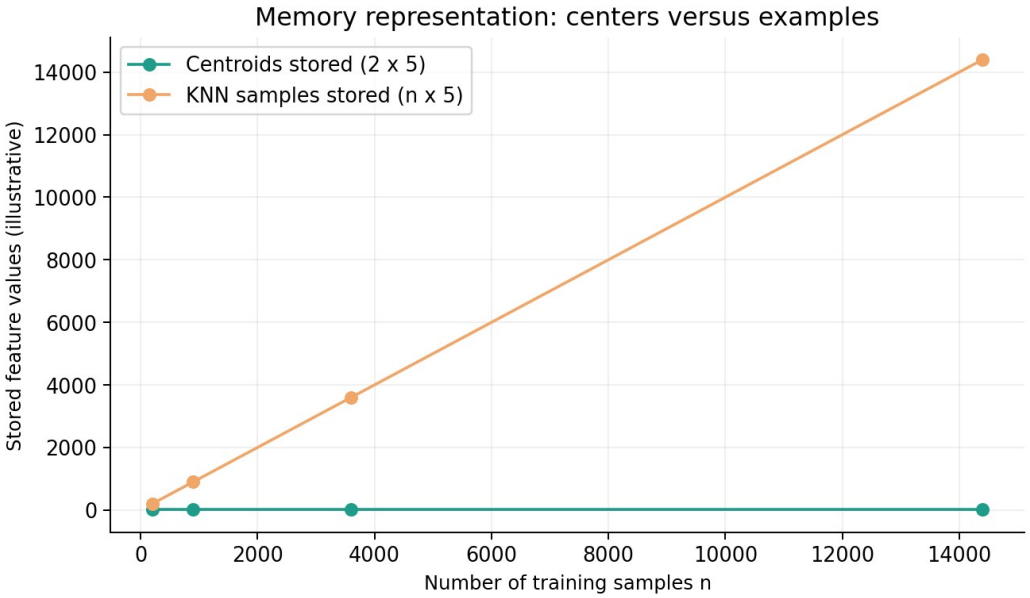
תקנון מתאים לכל מאפיין ממוצע וסטיית תקן על נתוני האימון, ואז מפעיל את אותם פרמטרים על נתוני האימות והבדיקה. אסור לחשב תקנון מחדש לכל תצפית בדיקה, משום שכך אנו מאבדים את משמעות הסולם המשותף; וגם אסור להתאים אותו מראש על כל המאגר, משום שהדבר חושף למודל מידע מן הבדיקה.

$$z = (x - \mu_{train}) / \sigma_{train}$$

10. מתי המסווג צפוי להיות נוח ומהיר?

ל־Nearest Centroid די לשמור מרכז אחד לכל מחלקה לצורך חיזוי, ולכן הייצוג שלו קומפקטי. עבור C מחלקות ו־d מאפיינים מספר הערכים המאוחסנים במרכזים הוא בערך $C \cdot d$. השיטה נוחה כקו בסיס פשוט ופרשני.

KNN שומר בדרך כלל את דוגמאות האימון לצורך חיזוי, ולכן מספר ערכי המאפיינים הוא בערך $n \cdot d$ עבור n דוגמאות. בחיפוש כוח גס של כל השכנים מחשבים מרחק לדוגמאות רבות עבור כל חיזוי; מבני חיפוש או שיטות קירוב יכולים לשפר את הביצועים במקרים מסוימים, אך לא בכל ממד ובכל מאגר.



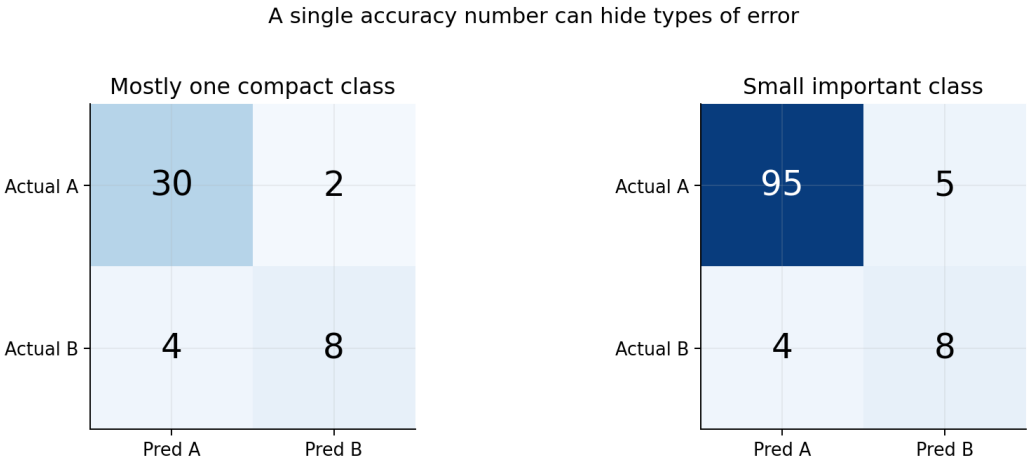
איור 8. המחשה של כמות נתוני המאפיינים הנשמרים: שני מרכזים לעומת כל תצפיות האימון; אינה מדידת זיכרון מדויקת של ספרייה.

אין להסיק מההשוואה ש־Nearest Centroid תמיד מהיר יותר בכל שלב: זמן יצירת המרכזים, גודל המאגר, מבנה החיפוש, החומרה ואופן המימוש משפיעים על המדידה. בפרויקט נבדוק מדדים אמיתיים ולא נסתפק בהבטחות תאורטיות.

11. בעיות נוספות שמסווג נדרש להתמודד איתן

11.1 חוסר איזון בין מחלקות

כאשר רוב מחשבי האימון שייכים למחלקה A, מסווג יכול להשיג Accuracy גבוה אפילו אם אינו מזהה היטב את B. לכן Accuracy לבדו אינו תמיד מספיק. בחירת המדד צריכה להתאים למחיר הטעות: האם החמצת תקלה חמורה גרועה יותר מאזעקת שווא?



איור 9. מטריצות לדוגמה בלבד: מספר ההצלחות הכולל מסתיר לעיתים סוגי טעות בעלי משמעות שונה.

11.2 ריבוי מאפיינים ומרחקים פחות מבדילים

כאשר מספר המאפיינים גדל, חלק משיטות המרחק עלולות לאבד יכולת להבחין היטב בין דוגמאות, במיוחד כשיש מאפיינים לא רלוונטיים או רעש. התופעה קשורה לקללת הממדיות. אין פירוש הדבר ש־KNN תמיד נכשל בממד גבוה; נדרשות בדיקות, בחירת מאפיינים ולעיתים הפחתת ממדיות.

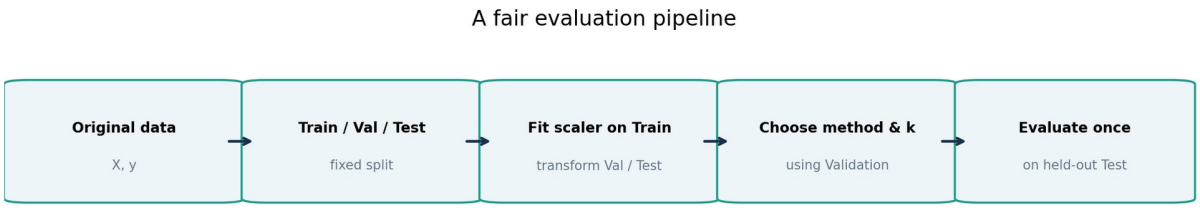
11.3 מספר דוגמאות קטן וחוסר ייצוג

כאשר יש מעט דוגמאות, גם מרכזי המחלקות וגם שכנים קרובים עשויים להיות בלתי יציבים. ההבחנה בין נתוני אימון לנתוני בדיקה הופכת חשובה במיוחד, משום שאותה תצפית יכולה להשפיע מאוד על מדד ההצלחה. גם טעות בתיוג עלולה לשנות תחזית.

12. השוואה הוגנת — איך מתכננים ניסוי?

הצעד הראשון אינו הפעלת שני מסווגים. קודם מגדירים את שאלת החקר ואת מדד ההצלחה. רק אחר כך יוצרים חלוקה אחת לנתוני Train, Validation ו־Test, ומשתמשים באותה חלוקה לצורך השוואה הוגנת. בחירת k או בחירת שיטת תקנון תיעשה על Validation או באמצעות Cross-Validation בתוך נתוני האימון בלבד.

בדיקת Test הסופית צריכה להישאר מבודדת עד בחירת המודל. אם מסתכלים שוב ושוב על ציון Test ומשנים את האלגוריתם בהתאם, Test הופך למעשה לכלי בחירה, ונקבל הערכת ביצועים אופטימית מדי.



איור 10. ניסוי תקין: חלוקה, התאמת עיבוד על אימון בלבד, בחירה באמצעות אימות, ורק לבסוף בדיקת Test.

13. שתי טבלאות עזר להחלטה

מאפיין	יתרון יחסי	מגבלה שיש לבדוק
מספר דוגמאות גדול	שומר מעט Nearest Centroid מרכזים	המרכז עלול להסתיר מבנה מורכב
גבול לא־ליניארי	KNN יכול ללכוד מבנה מקומי	רגיש לבחירת k ולרעש מקומי
נתונים עם חריגים	אין יתרון גורף	הממוצע מושפע; k קטן עשוי להאמין לחריג
מעט זמן לחיזוי	לרוב נוח לבדוק מסווג מרכזים	יש למדוד זמני אמת
צורך בהסבר לבוחן	Nearest Centroid פשוט להסבר	גם KNN אפשר להסביר דרך שכנים והצבעה

הטבלה אינה כלל אוטומטי לבחירת מסווג. היא מסייעת לנסח השערות שאותן נבחן באמצעות נתונים. תוצאה של ניסוי הוגן חשובה יותר מהעדפה אישית או מהנחה שמודל מסוים ״מתקדם״ יותר.

14. מימוש Python — מחשבים את אותם דברים שראינו ביד

נריץ תחילה מימוש קצר ללא עיבוד מקדים, משום שהדוגמה המלאכותית בנויה משני מאפיינים בעלי סולם דומה. הקוד מציג את שתי התחזיות ואת המרחקים למרכזים. ראינו את הערכים בחישוב הידני, ולכן אין כאן ״קופסה שחורה״.

```
import numpy as np
from sklearn.neighbors import NearestCentroid, KNeighborsClassifier
X = np.array([[1,1],[1,3],[3,1],[3,3],
              [4,4],[8,7],[9,7]], dtype=float)
y = np.array(['A','A','A','A','B','B','B'])
q = np.array([[4.1, 4.2]])
centroid = NearestCentroid().fit(X, y)
knn1 = KNeighborsClassifier(n_neighbors=1).fit(X, y)
knn3 = KNeighborsClassifier(n_neighbors=3).fit(X, y)
print('Centroid:', centroid.predict(q)[0])
print('KNN, k=1:', knn1.predict(q)[0])
print('KNN, k=3:', knn3.predict(q)[0])
```

ב־NearestCentroid הערכים של centroids_ מחושבים מתוך המחלקות. ב־KNeighborsClassifier פעולת fit מכינה את נתוני האימון לחיפוש שכנים. ההוראה predict מבצעת את כללי ההחלטה על הדוגמה החדשה; היא אינה חושפת את התווית האמיתית שלה.

15. מימוש ניסוי אמיתי: מדוע Pipeline חשוב?

בניסוי עם נתונים אמיתיים, המאפיינים עשויים להיות בסולמות שונים. לכן נעטוף כל מסווג ב־Pipeline עם StandardScaler, ונדאג להתאים את התקנון רק לאימון. הקטע הבא הוא תבנית לימודית לפיצול ולהפעלה, לא ראיה להצלחת מסווג מסוים על כל מאגר.

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import NearestCentroid, KNeighborsClassifier
from sklearn.metrics import accuracy_score
X, y = load_iris(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y)
models = {
    'centroid': make_pipeline(StandardScaler(), NearestCentroid()),
    'knn3': make_pipeline(StandardScaler(), KNeighborsClassifier(n_neighbors=3))
}
for name, model in models.items():
    model.fit(X_tr, y_tr)
    y_pred = model.predict(X_te)
    print(name, accuracy_score(y_te, y_pred))
```

לתרגול בחירת k, נפריד Validation מתוך נתוני האימון או נשתמש ב־Cross-Validation. מבחינה מדעית אין להכריז שמודל אחד טוב יותר על סמך חלוקה אחת וקטנה בלבד; בפרויקט נבחן גם מדדי מחלקות, יציבות, פערי ביצועים ומגבלות.

16. כיצד נכתוב מסקנה מדעית אמינה?

מסקנה טובה מפרידה בין תוצאה לבין הסבר אפשרי. לדוגמה: ״בחלוקה שבדקנו, KNN עם k=5 השיג מדד גבוה יותר מ־Nearest Centroid. ייתכן שמבנה הנתונים המקומי תרם לכך. עם זאת, נדרשת בדיקה נוספת בחלוקות אחרות, משום שהמדגם קטן.״ אין לכתוב ״KNN תמיד טוב יותר״ ללא ראיה.

בדוח פרויקט נרשום מהי שאלת המחקר, אילו מאפיינים נבחרו, כיצד חולקו הנתונים, באיזו שיטת מרחק השתמשנו, איזו טעות יקרה במיוחד, אילו ניסויים ביצענו ומהן מגבלות הבחירה. הסבר מדעי הוא חלק מן התוצר, לא נספח קישוטי.

17. תרגול מדורג — רמה א׳: שליטה ביסודות

שאלה 1 — זיהוי כלל ההחלטה

לכל משפט ציינו לאיזה מסווג הוא מתאים, והסבירו במשפט אחד: א. משווה את הנקודה החדשה למרכזי המחלקות; ב. מושפע מבחירת k; ג. משתמש בתוויות ידועות בזמן התאמת המודל; ד. עשוי להיות מושפע מקנה המידה.

מיומנות נבדקת: הבנה מושגית ונימוק

שאלה 2 — חישוב מרכזים

נתונות המחלקות A: (0,0), (0,4) והמחלקה B: (6,2), (8,2). חשבו את שני המרכזים וכתבו לכל מרכז את משמעותו בהקשר לנתונים.

מיומנות נבדקת: ממוצע רכיבי והסבר

שאלה 3 — בחירת שכן

לנקודה חדשה Q=(3,2) נתונות הדוגמאות A=(2,2), B=(4,3) ו־C=(7,2). חשבו מרחקים וסווגו בעזרת k=1.

מיומנות נבדקת: מרחק, מיון והכרעה

18. תרגול מדורג — רמה ב׳: העברה והסקת מסקנות

שאלה 4 — שני מסווגים, תחזיות שונות

השתמשו בשבע התצפיות של דודו מתחילת הפרק. שחזרו את מרכזי המחלקות ואת שלושת השכנים הקרובים ל־Q. הסבירו מדוע Nearest Centroid ו־KNN עם k=1 אינם חייבים להפיק אותה תחזית.

מיומנות נבדקת: נימוק המבוסס על חישוב

שאלה 5 — מה משתנה כשמוסיפים חריג?

למחלקה A נוספה תצפית חריגה (15,15). חשבו מחדש את המרכז; הסבירו מדוע אין להסיק מיד ש־KNN על Q ישנה את סיווגו. בדקו איזו תחזית דורשת חישוב מחדש.

מיומנות נבדקת: ממוצע, חריגים והכללה

שאלה 6 — זליגת מידע

תלמיד חילק מאגר ל־Train ול־Test רק לאחר שהפעיל StandardScaler על כל השורות. הסבירו מה בעייתי בתהליך, הציעו סדר פעולות מתוקן והבהירו מה נשמר מן האימון לצורך הבדיקה.

מיומנות נבדקת: תהליך ניסוי תקין

19. תרגול מדורג — רמה ג׳: חשיבה ביקורתית ובגרות

שאלה 7 — תשובה מנומקת בסגנון בחינה

במאגר מסוים Nearest Centroid קיבל Accuracy 91% ו־KNN קיבל 94% באימון. האם אפשר לקבוע ש־KNN עדיף? כתבו לפחות שלושה נתונים נוספים הדרושים לקבלת החלטה, והסבירו כיצד תמדדו אותם.

מיומנות נבדקת: הערכת מודלים והנמקה

שאלה 8 — ניתוח גבול החלטה

במערכת דו־ממדית נתונים המרכזים $C_A=(2,2)$ ו־$C_B=(7,6)$. פתחו משוואה המתארת שוויון מרחקים, קבעו לאיזה צד שייכת $Q=(4.1,4.2)$, והסבירו מדוע אצל KNN הגבול אינו חייב להיות ישר.

מיומנות נבדקת: אלגברה, גיאומטריה ופרשנות

שאלה 9 — שאלת פרויקט

למערכת המזהה מחשבים בעלי תקלה חמורה יש שיעור נמוך של מקרים חיוביים. כיצד ייתכן ששני מסווגים בעלי Accuracy דומה שונים מאוד ביכולת לזהות תקלות? הציעו מדד נוסף ותארו מהו מחיר הטעות החשוב במקרה זה.

מיומנות נבדקת: מחלקות לא מאוזנות ועלות טעות

20. מעבדה עצמאית ב־Google Colab

מחברת התלמיד הנלווית כוללת 12 משימות המתחילות בבדיקה של הדוגמה הידנית, ממשיכות למימוש והשוואה בין שני מסווגים ומסתיימות בניסוי מבוקר. המשימות כוללות מקומות TODO להשלמה אך אינן כוללות פתרונות מלאים. לפני הרצה יש לציין השערה ולרשום אחריה מסקנה.

21. טבלת בדיקה עצמית לסיום הפרק

אופן בדיקה	כעת אני יודע/ת...
פתרון מלא של שאלה 2	**לחשב מרכזי מחלקות ולנבא לפי מרחק**
טבלת מרחקים והצבעה	**לסווג בעזרת k שכנים**
הנמקה על Q מהסיפור	**להסביר סתירה בין תחזיות**
תרשים תהליך ללא Leakage	**להבחין בין אימון, אימות ובדיקה**
דוגמת תקלה חמורה	**להסביר מדוע Accuracy אינו מספיק תמיד**
פסקת מסקנה עם מגבלה	**לנמק בחירה במודל לפרויקט**

22. מפת ההמשכיות בספר

הפרק סוגר את יחידת הלמידה של מודלים מבוססי מרחק: הוא נשען על וקטורים, מרחקים, ממוצעים, תקנון, Nearest Centroid ו־KNN. בהמשך נלמד בפרק 5.1 באופן שיטתי מטריצת בלבול, Precision, Recall ו־F1; ובפרק 5.2 נרחיב על Validation, חקר פרמטרים, Cross-Validation והכללה.

למידה עמוקה משתמשת גם היא בייצוגים וקטוריים ובבחירת מדדים מתאימים, אך כאן איננו טוענים כי KNN או Nearest Centroid הם רשתות עצביות. ההשוואה ביניהם היא הזדמנות לתרגל חשיבה מדעית שתשרת אחר כך מודלים מתקדמים.

23. מקורות, עוגן לתכנית והרחבות

עוגן נושאי הליבה: יחידה 5, פרק 4 — מודלים מבוססי מרחק, סעיף 4.4 ״אלגוריתמי סיווג מבוסס מרחק — מבט על״, כפי שממופה במאגר Data-Science-ML-Course. החיבור לפרקי 5.1–5.2 הוא גשר הוראתי. להכרעה על חומר חובה יש להצליב עם המסמך הרשמי התקף לשנת ההיבחנות.

מקורות השראה מקצועיים: Luis G. Serrano, Grokking Machine Learning (Manning, 2021) — למידה דרך דוגמאות; Aurélien Géron, Hands-On Machine Learning (O’Reilly) — השוואת מודלים ו־Pipeline; Deisenroth, Faisal & Ong, Mathematics for Machine Learning (Cambridge, 2020) — הסבר גיאומטרי ומתמטי; James et al., An Introduction to Statistical Learning with Applications in Python (Springer, 2023) — אימון, בדיקה והסקה. ההסברים, התרגילים והאיורים כאן מקוריים.

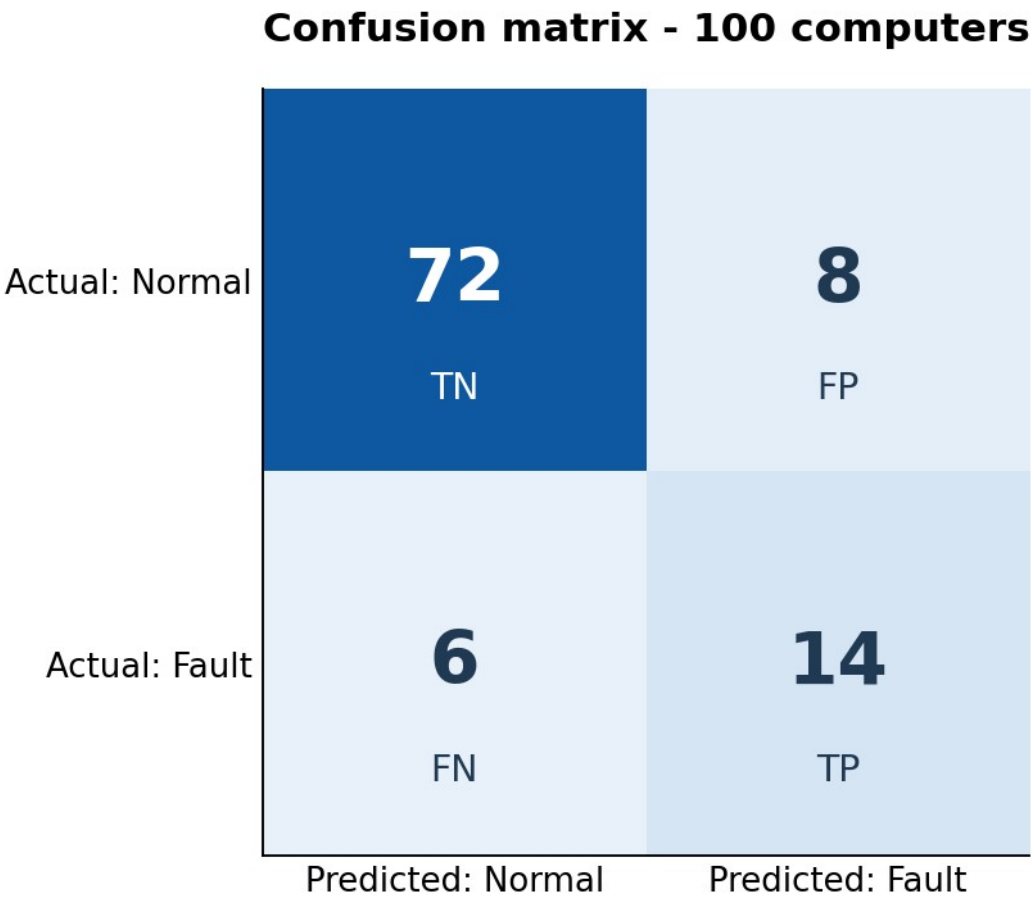
תיעוד תוכנה מומלץ: scikit-learn NearestCentroid, KNeighborsClassifier, Pipeline, StandardScaler, train_test_split. תיעוד המימוש אינו מחליף את ההבנה המתמטית, והניסוי על נתונים מומצאים אינו קובע ביצועים כלליים של מסווג כלשהו.

ספר מדעי הבינה המלאכותית

יחידה 5 — נושאי ליבה בבינה מלאכותית | פרק 5.1

הערכת ביצועי מודלים: כיצד יודעים שהחיזוי באמת מועיל?

מהדורת תלמיד מורחבת · הבנה מדעית, בגרות ויישום בפרויקט גמר



איור: מטריצת בלבול היא מפת השגיאות של מודל סיווג. היא מספרת יותר ממספר דיוק בודד.

הבסיס לפרק והמטרות

הפרק ממשיך את KNN ואת Nearest Centroid שנלמדו בפרקים 4.2–4.4. עד עכשיו הצלחנו לייצר תחזיות. כעת נשאל שאלה עמוקה יותר: איך בודקים אם התחזיות מועילות? השאלה אינה טכנית בלבד, משום שטעויות שונות עשויות לגרום נזקים שונים.

בסוף הפרק התלמיד יוכל לקרוא ולבנות מטריצת בלבול; לחשב ידנית Accuracy, Precision, Recall, Specificity ו־F1; להסביר את כשל הדיוק בנתונים לא מאוזנים; להשוות מודלים על בסיס מחיר הטעויות; להשתמש ב־scikit-learn כדי לבדוק חישובים; ולנמק בחירה של מדד הערכה בפרויקט גמר.

ידע קודם

משתנה מטרה ותווית, למידה מונחית, חלוקת Train/Test, אחוזים ושברים, כתיבת תנאי ב־Python, חישוב ממוצע, סיווג באמצעות KNN או Nearest Centroid.

תוכן עניינים

א. בעיית ההערכה • ב. מטריצת בלבול • ג. מדדי איכות • ד. נתונים לא מאוזנים • ה. ספי החלטה ועלויות • ו. מימוש וניתוח • ז. הערכה בבגרות ובפרויקט גמר

1. למה בכלל צריך להעריך מודל?

דודו עובד על מערכת המסייעת לאחראי המחשבים בבית הספר לאתר מחשבים שעלולים לפתח תקלה חמורה. בכל בוקר המערכת מקבלת מאפיינים כגון גיל המחשב, תדירות תקלות קודמות ומדידות עומס. לכל מחשב היא מנבאת אחת משתי תוויות: "דרוש טיפול" או "אין צורך בטיפול".

דמיינו שני מחשבים: באחד התקלה אמיתית והמודל מזהה אותה; בשני התקלה אמיתית אך המודל מתעלם ממנה. לא די לשאול כמה תשובות היו נכונות. צריך לשאול גם אילו טעויות בוצעו ובאילו מקרים. מכאן נובעים מדדי הערכה שונים.

מהו מודל טוב?

אין מדד יחיד המתאים לכל בעיה. בפרויקט לזיהוי תקלות מסוכנות חשוב לא להחמיץ מחשבים תקולים. בפרויקט לשליחת הודעות אזהרה יקרות חשוב גם שלא להטריד משתמשים ללא הצדקה. לכן איכות המודל נקבעת ביחס למטרת המערכת, למחיר הטעויות, להתפלגות הנתונים ולאופן ביצוע הבדיקה.

עצור וחשוב: אם מודל מזהה 95 מתוך 100 דוגמאות נכון, האם הוא בהכרח טוב? מה חסר לנו כדי לענות?

2. בונים מטריצת בלבול — צעד אחר צעד

2.1 הגדרת המחלקה החיובית

בבעיית סיווג בינארי יש שתי מחלקות. נגדיר מראש איזו מהן היא "חיובית" (Positive). בדוגמה שלנו המחלקה החיובית היא "דרוש טיפול". המילה חיובית אינה מציינת דבר טוב; היא רק שם טכני למחלקה שמעניינת אותנו.

הערך האמיתי של כל מחשב נקבע לאחר בדיקה, ואילו הערך החזוי הוא התשובה שהמודל נתן. צירוף שתי התשובות יוצר אחת מארבע אפשרויות: זיהוי נכון של תקלה, אזעקת שווא, החמצת תקלה או זיהוי נכון של מצב תקין.

2.2 ארבעת המקרים

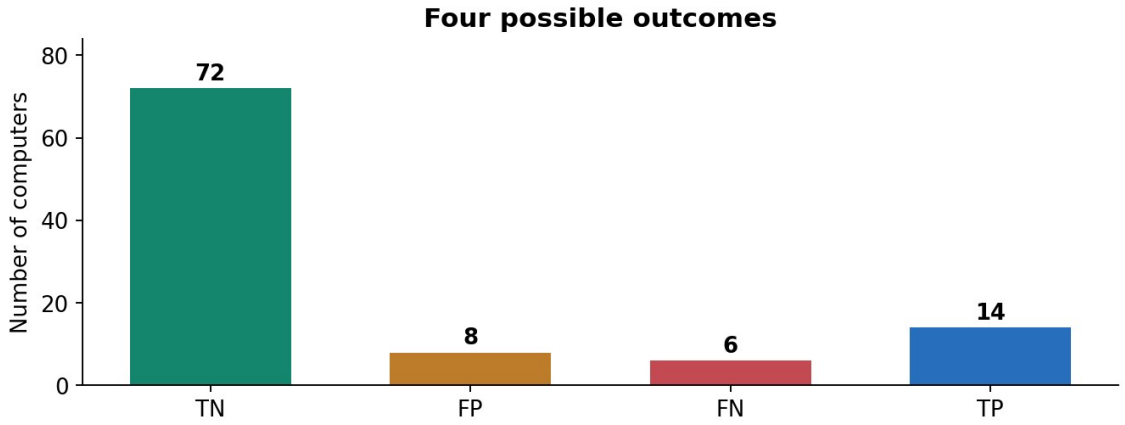
מה המודל אמר?	מה קרה במציאות?	שם באנגלית	קיצור
דרוש טיפול	המחשב דורש טיפול	True Positive	TP
דרוש טיפול	המחשב תקין	False Positive	FP
תקין	המחשב דורש טיפול	False Negative	FN
תקין	המחשב תקין	True Negative	TN

בלבול נפוץ: False Positive אינו "מחלקה חיובית שגויה" במציאות. פירושו שהמודל הכריז על חיובי, אך במציאות הדוגמה הייתה שלילית. באותו אופן False Negative הוא מקרה חיובי שהמודל החמיץ.

2.3 דוגמת 100 המחשבים

המערכת נבדקה על 100 מחשבים שלא שימשו לאימון. בבדיקה בפועל התגלו 20 מחשבים תקולים ו־80 תקינים. המודל סימן 22 מחשבים כזקוקים לטיפול: 14 מהם אכן תקולים, ואילו שמונה תקינים. ששת המחשבים התקולים הנותרים לא זוהו.

מצב אמיתי / תחזית	נחזה תקין (0)	נחזה תקול (1)	סך הכול
תקין בפועל (0)	TN = 72	FP = 8	80
תקול בפועל (1)	FN = 6	TP = 14	20
סך הכול	78	22	100



איור: ארבע קבוצות התוצאות — שני סוגי הצלחה ושני סוגי שגיאה.

בדיקת הסכום: 72 + 8 + 6 + 14 = 100. תמיד כדאי לוודא שמספר התצפיות במטריצה שווה למספר הדוגמאות שנבדקו. בדיקת שוליים נוספת: TP + FN = 20, כלומר כל המחשבים התקולים; TN + FP = 80, כלומר כל התקינים.

כלל קריאה: בפרק זה שורות המטריצה הן המצב האמיתי ועמודותיה הן התחזית. יש תרשימים שמחליפים ביניהן; תמיד קוראים תחילה את כותרות הצירים.

3. Accuracy — שיעור התשובות הנכונות

דיוק כולל, המכונה Accuracy, עונה על השאלה: איזה חלק מכלל הדוגמאות סווג נכון? כדי לענות, נחבר את שני המצבים שבהם המודל צדק: TP ו־TN. נחלק במספר הדוגמאות הכולל.

$$\text{Accuracy} = (TP + TN) / (TP + TN + FP + FN)$$

$$\text{Accuracy} = (14 + 72) / 100 = 0.86 = 86\%$$

המשמעות: מתוך כל 100 המחשבים שנבדקו, 86 סווגו נכון. זו תוצאה תיאורית על קבוצת הבדיקה שנבחרה; אין בה התחייבות לכך שבכל יום עתידי נקבל בדיוק 86%.

מדוע Accuracy אינו מספיק?

אם קבוצת הנתונים מכילה בעיקר מחשבים תקינים, קל לקבל Accuracy גבוה על ידי ניבוי "תקין" כמעט לכל מחשב. לכן נבחן בדוגמאות הבאות מדדים הממוקדים במחלקה התקולה ובטעויות שעלולות להיות יקרות.

4. Precision — עד כמה אפשר לסמוך על אזעקה?

נניח שדודו מקבל רשימה של מחשבים שהמערכת סימנה כתקולים. הוא רוצה לדעת: מתוך כל המחשבים שסומנו כתקולים, כמה באמת תקולים? זוהי בדיוק השאלה שעליה עונה Precision.

$$\text{Precision} = TP / (TP + FP)$$

$$\text{Precision} = 14 / (14 + 8) = 14 / 22 \approx 0.6364$$

כלומר, מתוך 22 האזהרות שהמודל הפיק, 14 היו מוצדקות. בהקשר שלנו Precision הוא כ־63.6%. שימו לב: המכנה הוא מספר התחזיות החיוביות, ולא מספר המקרים החיוביים במציאות.

Precision נמוך משמעותו שמופיעות אזעקות שווא רבות יחסית לאזהרות הנכונות. אם כל אזעקת שווא גורמת למעבדה יקרה או להפרעה משמעותית, המדד הזה חשוב במיוחד.

שאלת שליפה: המודל הפיק 50 התראות, מהן 40 נכונות. אילו ערכים דרושים כדי לחשב Precision? ומה לא ניתן ללמוד מן הנתון הזה על מחשבים שהוחמצו?

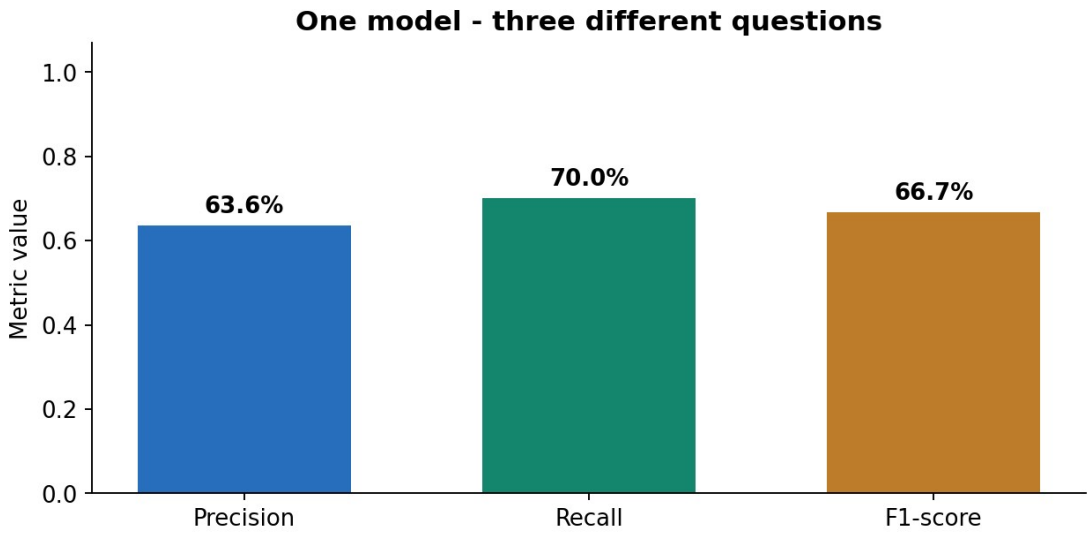
5. Recall — כמה מהתקלות הצלחנו למצוא?

האחראי על בטיחות המחשבים שואל שאלה שונה: מתוך כל המחשבים שבאמת דרשו טיפול, כמה אותרו בזמן? המדד שמודד זאת הוא Recall, שנקרא גם רגישות (Sensitivity).

$$\text{Recall} = TP / (TP + FN)$$

$$\text{Recall} = 14 / (14 + 6) = 14 / 20 = 0.70$$

המשמעות: המערכת זיהתה 70% מהמחשבים התקולים באמת, והחמיצה 30% מהם. ההבדל בין Recall ל־Precision הוא הבדל בין שתי קבוצות ייחוס: המקרים החיוביים האמיתיים לעומת התחזיות החיוביות.



איור: לשלושת המדדים ערכים שונים, משום שכל מדד עונה על שאלה אחרת.

5.1 Specificity — זיהוי נכון של המחלקה השלילית

מדי פעם נרצה לדעת עד כמה המודל מזהה נכון מחשבים תקינים. לכך משמש Specificity: מתוך כל המחשבים שבאמת תקינים, איזה חלק זוהה כתקין?

$$\text{Specificity} = TN / (TN + FP) = 72 / 80 = 0.90$$

Specificity בשיעור 90% פירושו ש־90% מהמחשבים התקינים זוהו נכון. שיעור אזעקות השווא מבין כל התקינים הוא $1 - \text{Specificity} = 10\%$.

6. F1-score — איזון בין Precision ל־Recall

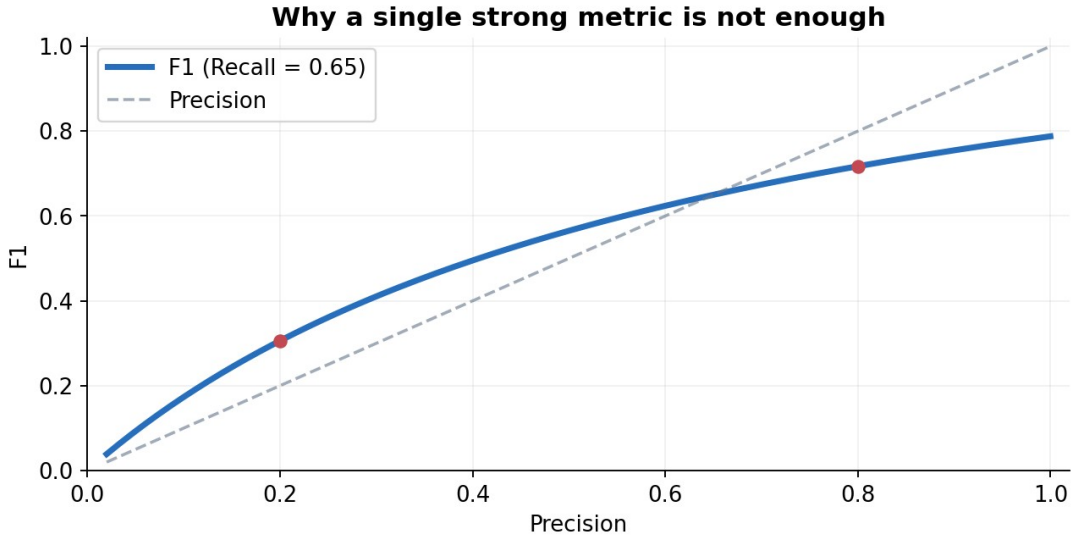
מצבים רבים דורשים איזון בין אי־החמצת מקרים חיוביים לבין צמצום אזעקות שווא. לשם כך משתמשים לעיתים במדד F1, שהוא הממוצע ההרמוני של Precision ושל Recall. בניגוד לממוצע חשבוני פשוט, הממוצע ההרמוני מעניש ערך נמוך במיוחד באחד משני המדדים.

$$F1 = 2 \times Precision \times Recall / (Precision + Recall)$$

$$F1 = 2TP / (2TP + FP + FN)$$

$$F1 = 28 / (28 + 8 + 6) = 28 / 42 \approx 0.6667$$

במקרה שלנו F1 הוא כ־66.7%. המדד אינו הסתברות לכך שהמודל "צודק". הוא דרך אחת לשקלל שני סוגים של איכות במודל סיווג בינארי. יש בעיות שבהן נעדיף Recall גבוה גם במחיר F1 נמוך יותר, מפני שהחמצה מסוכנת במיוחד.



איור: כשה־Recall נשאר קבוע, Precision נמוך מגביל את F1.

6.1 למה לא להסתפק בממוצע רגיל?

נניח Precision של 20% ו־Recall של 80%. ממוצעם החשבוני הוא 50%, אך F1 יהיה 32% בלבד. האיזון במערכת אינו טוב: ריבוי אזעקות השווא אינו נעלם רק מפני שה־Recall גבוה. זהו היתרון של ממוצע הרמוני כמדד המגיב באופן חזק לחוסר איזון בין שני המדדים.

$$P = 0.20, R = 0.80 \rightarrow F1 = (2 \times 0.2 \times 0.8)/(0.2+0.8) = 0.32$$

7. מסכמים את המדדים — מה לבחור?

מדד	שאלת המפתח	חישוב בדוגמה	תוצאה
Accuracy	כמה מהתחזיות נכונות בכלל?	100/(14+72)	86%
Precision	כמה מההתראות מוצדקות?	(14+8)/14	63.6%
Recall	כמה מהתקלות נמצאו?	(14+6)/14	70%
Specificity	כמה מהתקינים זוהו	(72+8)/72	90%

מדד	שאלת המפתח	חישוב בדוגמה	תוצאה
	נכון?		
F1	מהו האיזון בין Precision ו־Recall?	(28+8+6)/28	66.7%

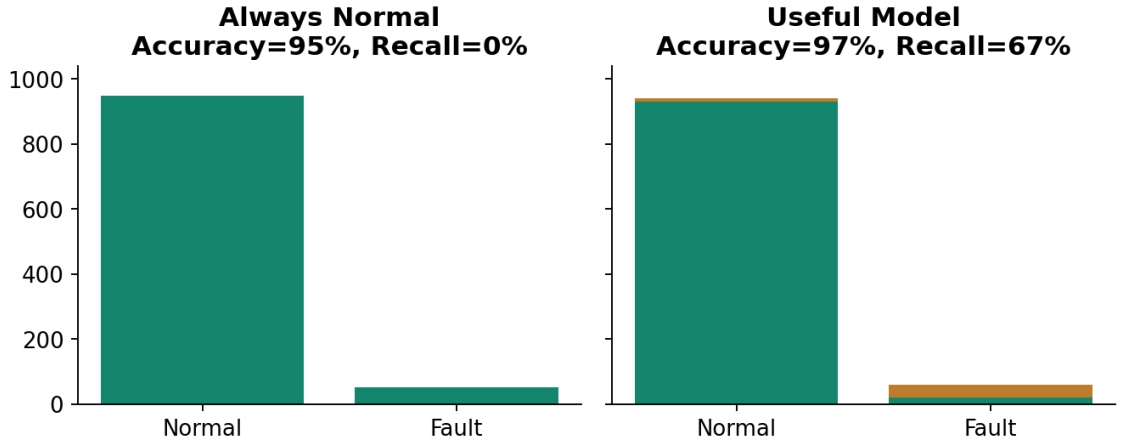
כדי לפרש מודל כראוי, אין לדווח רק על השורה שבה הוא נראה הכי טוב. יש להציג את המטריצה עצמה, את המדדים הרלוונטיים ואת הסיבה לבחירה בהם. במדידות אמיתיות כדאי לציין גם את גודל קבוצת הבדיקה ואת תדירות המחלקות.

8. מלכודת חוסר האיזון בין מחלקות

נחשוב על מאגר של 1,000 מחשבים, שרק 50 מהם תקולים. אם מודל יחזה תמיד "תקין", הוא יסווג נכון 950 מחשבים. ה־Accuracy יהיה 95%, אבל המודל יחמיץ את כל 50 התקלות. זו דוגמה שבה מספר מרשים עשוי להסתיר כישלון מוחלט במטרה המרכזית.

$$\text{Accuracy} = 950/1000 = 95\%; \quad \text{Recall} = 0/50 = 0\%$$

בדוגמה זו אין אף תחזית חיובית, ולכן המכנה של Precision הוא אפס. מבחינה מתמטית Precision אינו מוגדר במקרה זה; בספריות תוכנה נהוג לבחור מדיניות מיוחדת לערכים לא מוגדרים, ולעיתים להחזיר 0. יש לדעת להבחין בין התופעה המתמטית לבין האופן שבו הספרייה בוחרת לייצג אותה.



A 95% accuracy can still miss every real fault.

איור: מודל שאומר תמיד "תקין" עשוי להראות דיוק גבוה אך נכשל בזיהוי המחלקה החשובה.

8.1 Baseline — מודל בסיס להשוואה

לפני שמפתחים אלגוריתם מורכב יש לבנות קו בסיס פשוט, למשל ניבוי המחלקה השכיחה ביותר. אם KNN קיבל Accuracy של 92% אך מודל קו בסיס כבר קיבל 95%, צריך לברר מה ההצדקה לבחירת KNN. לעיתים היתרון של המודל נמצא דווקא ב־Recall, ולא ב־Accuracy.

8.2 Balanced Accuracy — הרחבת עומק

דרך נוספת לטפל בפער בין שכיחויות היא לחשב את ממוצע הרגישות של שתי המחלקות. בסיווג בינארי, Balanced Accuracy הוא הממוצע של Recall ו־Specificity. בדוגמה הראשית:

$$\text{Balanced Accuracy} = (\text{Recall} + \text{Specificity}) / 2 = (0.70+0.90)/2 = 0.80$$

זהו מדד מועיל לדיון, אך הוא אינו תחליף אוטומטי לכל שאר המדדים. נלמד להציגו כהרחבה ולא לטעון שהוא מחויב בבגרות ללא בדיקת המסמך התקף.

9. מחיר הטעות — מדוע המודל המדויק יותר לא תמיד עדיף?

נניח שכל תקלה מוחמצת (FN) גורמת בממוצע לעלות של 500 ₪, ואילו אזעקת שווא (FP) גורמת לבדיקת מעבדה בעלות 20 ₪. נבחן שני מודלים שנבדקו על אותם 100 מחשבים, באותו זמן ובאותם תנאים.

מודל	TP	FN	FP	TN	Accuracy
A	14	6	8	72	86%
B	18	2	22	58	76%

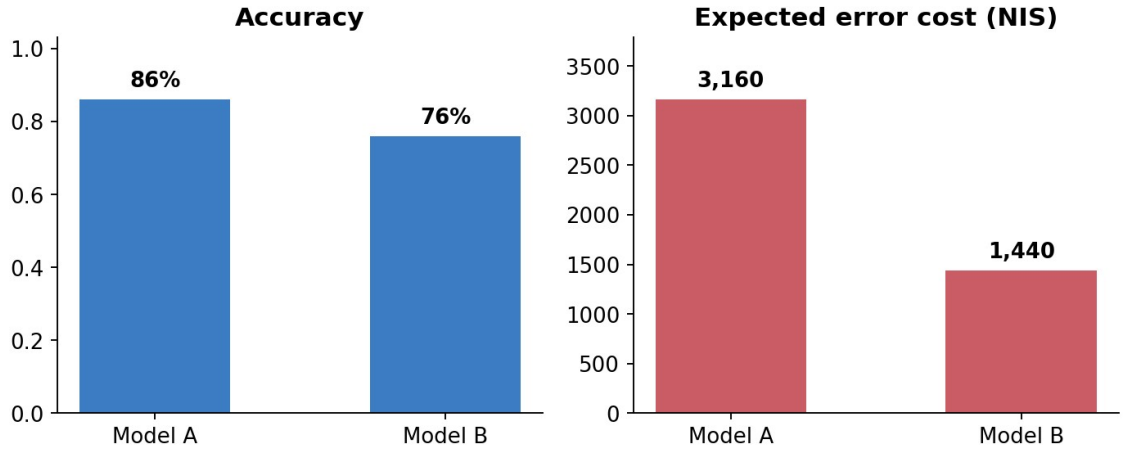
מודל A מדויק יותר על פי Accuracy. אך מודל B מחמיץ רק שתי תקלות במקום שש, ומפיק יותר אזעקות שווא. כדי להחליט לפי מטרת המערכת, נחשב עלות טעויות משוערת:

$$\text{Cost} = (\text{FN} \times 500) + (\text{FP} \times 20)$$

$$\text{Cost(A)} = 6\times500 + 8\times20 = 3{,}160 \text{ NIS}$$

$$\text{Cost(B)} = 2\times500 + 22\times20 = 1{,}440 \text{ NIS}$$

לפי הנחות העלות הנתונות, מודל B עדיף מבחינת העלות המשוערת, אף ש־Accuracy שלו נמוך יותר. זהו נימוק מקצועי לבחירת מודל. אם מחירי הטעות היו אחרים, ייתכן שהמסקנה הייתה מתהפכת.



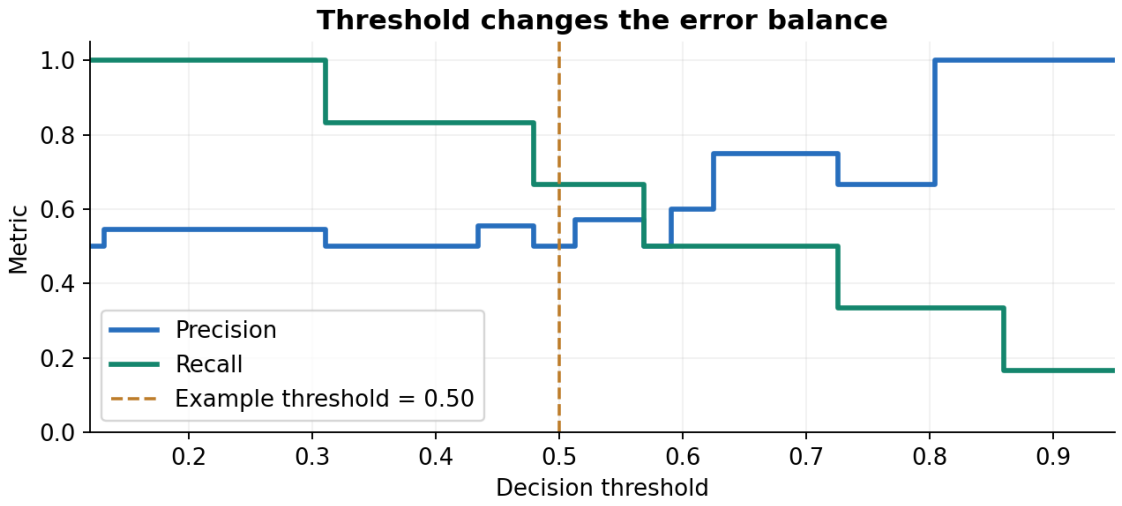
איור: דוגמה לכך שמדד דיוק ועלות טעויות עשויים להעדיף מודלים שונים.

הרחבה מקצועית: העלות בדוגמה היא אומדן לימודי פשוט. בפרויקט אמיתי יש לשקול גם זמני טיפול, סיכונים, משאבים, אי־ודאות ושיקולים אתיים.

10. סף החלטה — Threshold

חלק מהמסווגים מחזירים ציון מספרי או הסתברות מוערכת למחלקה חיובית. כדי להמיר ציון לתווית נבחר סף: למשל, אם ציון התקלה לפחות 0.50, נתריע. שינוי הסף משנה את מספר ההתראות ולכן גם את TP, FP, FN ו־TN.

הורדת הסף עשויה להגדיל את Recall משום שיותר מקרים אמיתיים מזוהים, אך לעיתים גם מגדילה את כמות אזעקות השווא ופוגעת ב־Precision. העלאת הסף לרוב תצמצם התראות, אך עלולה להחמיץ מקרים. אין הבטחה ששני המדדים ישתנו בקצב אחיד.



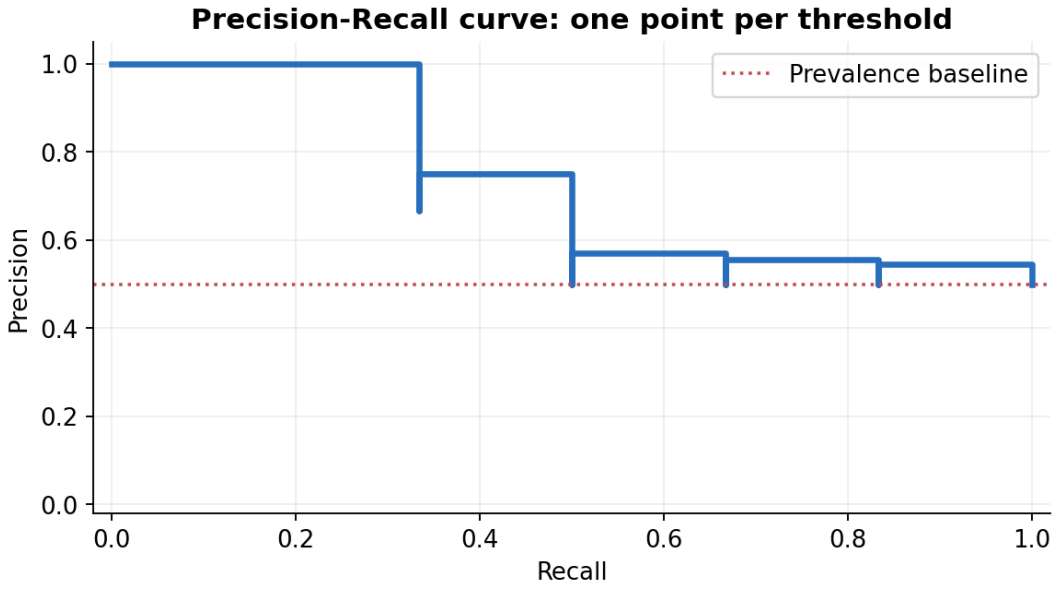
איור: Precision ו־Recall משתנים כאשר משנים את סף ההחלטה באותה קבוצת דוגמאות.

10.1 כיצד בוחרים סף נכון?

אין לבחור סף על סמך קבוצת ה־Test הסופית שוב ושוב. בחירה כזאת מתאימה את ההחלטה לנתונים שנועדו לבדיקה בלתי תלויה. את הסף בוחרים בעזרת קבוצת Validation או Cross-validation, בהתחשב במחיר הטעויות ובמטרת השימוש; את הביצוע הסופי מדווחים על נתוני Test שלא שימשו לבחירה.

10.2 Precision-Recall Curve — העמקה

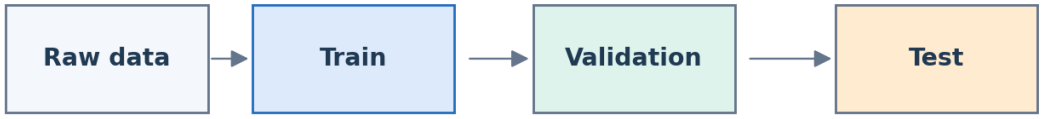
אפשר לחשב Precision ו־Recall עבור ספים רבים ולשרטט גרף המראה את הפשרה ביניהם. כל נקודה מייצגת סף אחר. הקו אינו גרף של הסתברויות תקינות ללא בדיקה; הוא מסכם את התנהגות המודל ביחס לספים על נתוני ההערכה.



איור: עקומת Precision–Recall; שיעור החיוביים הוא קו בסיס שימושי לקריאה ביקורתית.

11. תהליך הערכה הוגן: Train, Validation, Test

אם מאמנים מודל ובודקים אותו על אותן דוגמאות, הוא עשוי להצליח מפני שכבר ראה אותן. אם מתאימים ממוצע, תקנון או בחירת מאפיינים לפי קבוצת ה־Test, חלק מן המידע מהבדיקה דולף אל תהליך הפיתוח. לכן יש להפריד בבירור בין שלבי העבודה.



Fit transformations on training data; tune on validation; report once on test.

איור: מסלול עבודה מומלץ: אימון, בחירה ובדיקה סופית; אין להשתמש ב־Test לצורך כוונון חוזר.

קבוצה	מטרתה	מה אסור לעשות?
Train	ללמוד את פרמטרי המודל ואת פעולות ההכנה	להכניס לתוכה מידע עתידי
Validation	להשוות מודלים, לבחור k או סף	להציג את ציונה כאילו הוא Test בלתי תלוי
Test	להעריך ביצוע סופי על נתונים שלא שימשו לבחירות	לכוונן שוב את המודל לפי תוצאותיה

בהגשה לפרויקט גמר חשוב להציג כיצד בוצעה החלוקה, באילו פעולות נעשה שימוש ב־Pipeline, וכיצד נשמרה עצמאות קבוצת ה־Test. כאשר יש כמה רשומות לאותו תלמיד או מחשב, או כאשר הנתונים מסודרים בזמן, יש לשקול חלוקה קבוצתית או כרונולוגית כדי להימנע מדליפת מידע.

12. חישוב מדדים ב־Python — בדיקה מול היד

לפני שנריץ את הספרייה, עלינו לדעת מה אנו מצפים לקבל. נכין מערכי תוויות באורך 100 המתאימים לארבע קבוצות התוצאות ונבדוק שהמטריצה שחוזרת מקיימת את הסדר שבחרנו: שורות אמת, עמודות תחזית.

```
from sklearn.metrics import (
    confusion_matrix, accuracy_score,
    precision_score, recall_score, f1_score
)

# 72 TN, 8 FP, 6 FN, 14 TP
y_true = [0]*72 + [0]*8 + [1]*6 + [1]*14
y_pred = [0]*72 + [1]*8 + [0]*6 + [1]*14

print(confusion_matrix(y_true, y_pred, labels=[0, 1]))
print("Accuracy:", accuracy_score(y_true, y_pred))
print("Precision:", precision_score(y_true, y_pred))
print("Recall:", recall_score(y_true, y_pred))
print("F1:", f1_score(y_true, y_pred))
```

הפלט הצפוי למטריצה הוא [[72, 8], [6, 14]]. התוצאות המספריות אמורות לשחזר את החישובים הידניים. אם אינן תואמות, יש לבדוק האם החלפנו את תווית המחלקה החיובית, את סדר הצירים או את מערכי האמת והתחזית.

12.1 דוח סיווג מפורט

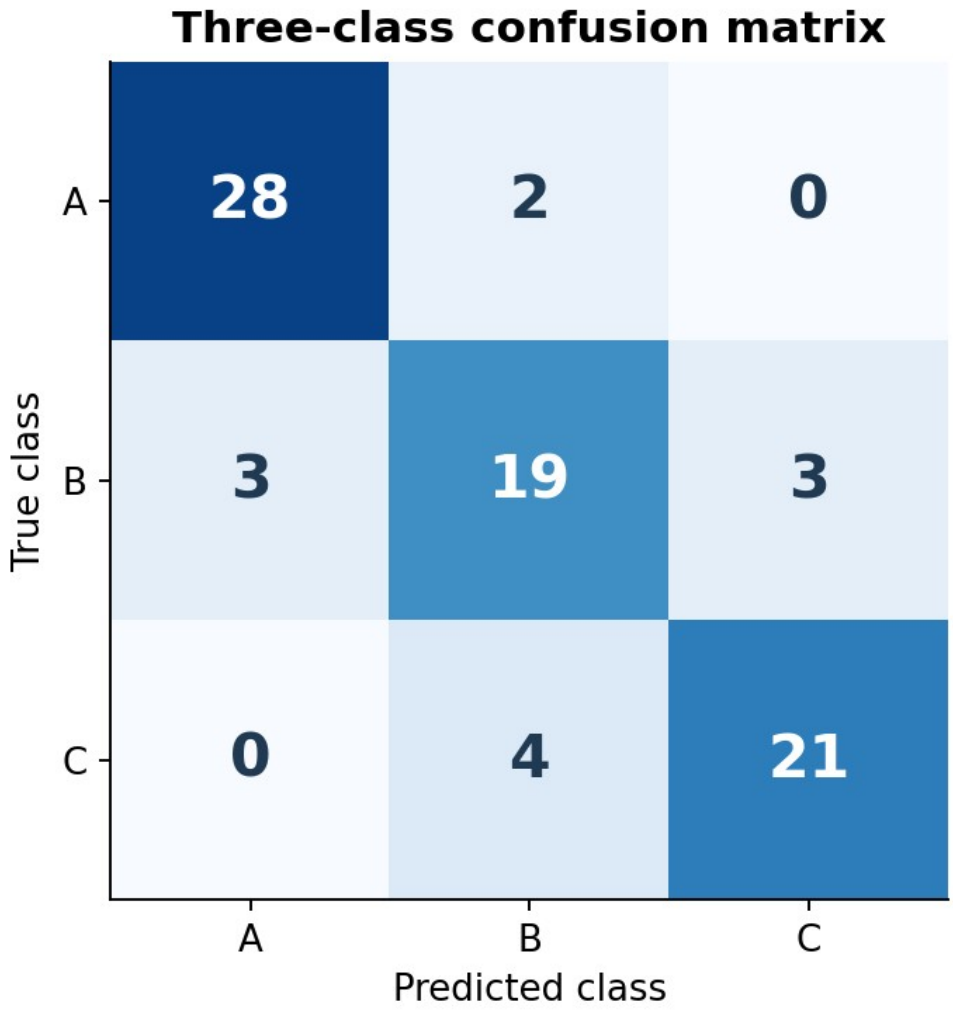
```
from sklearn.metrics import classification_report

print(classification_report(
    y_true, y_pred,
    labels=[0, 1],
    target_names=["Normal", "Fault"],
    zero_division=0
))
```

בדוח מוצג מדד לכל מחלקה. המונח support מתייחס למספר הדוגמאות האמיתיות השייכות לאותה מחלקה. למשל, בדוגמה שלנו support של Fault הוא 20. חשוב לא לבלבל בין support לבין מספר התחזיות שהמודל נתן למחלקה.

13. הרחבה — סיווג לשלוש מחלקות ומעלה

עד כה סיווגנו לשתי מחלקות. בעולם האמיתי ייתכן שנסווג תקלה ל"רכיב תקין", "תקלה בחומרה" ו"תקלה בתוכנה". במקרה כזה מטריצת הבלבול מכילה יותר משתי שורות ועמודות. הערכים על האלכסון הראשי הם התחזיות הנכונות; יתר התאים מתארים בלבול בין מחלקות.

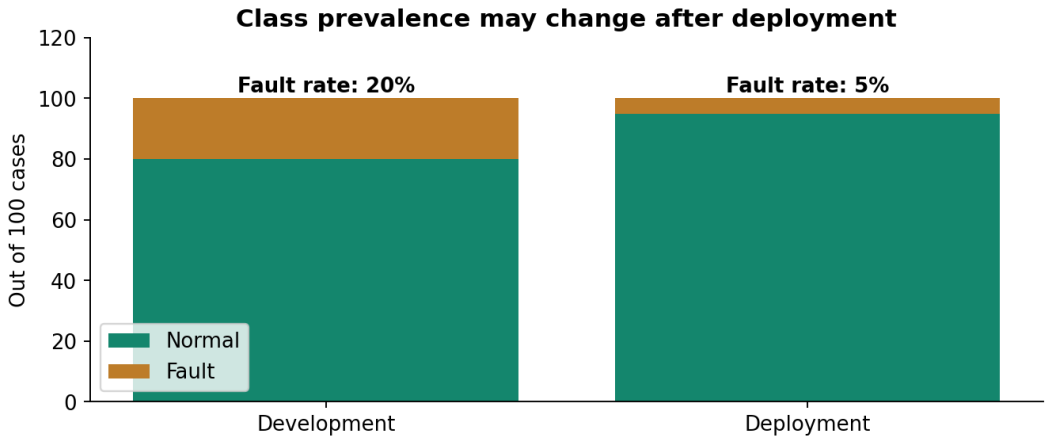


איור: מטריצת בלבול לשלוש מחלקות: תאים אלכסוניים מתארים החלטות נכונות.

כאשר מדווחים Precision, Recall ו־F1 למספר מחלקות, מקובל להציג מדד לכל מחלקה. Macro average הוא ממוצע פשוט של המדדים לפי מחלקה, ו־weighted average נותן לכל מחלקה משקל לפי מספר התצפיות שלה. אין להשתמש בממוצע בלבד בלי לקרוא את המדדים לכל מחלקה בנפרד.

14. שיעור חיוביים משתנה — מה קורה אחרי פרסום המערכת?

נניח שבתקופת הפיתוח היו 20% מחשבים תקולים, אך לאחר שדרוג כל המחשבים רק 5% מהם תקולים. שיעור המחלקות השתנה. גם אם חלק מתכונות המסווג נותרו דומות, ערכי Precision ותדירות ההתראות עשויים להשתנות. לכן תוצאות בדיקה ישנות אינן מבטיחות רמת שירות זהה בעתיד.

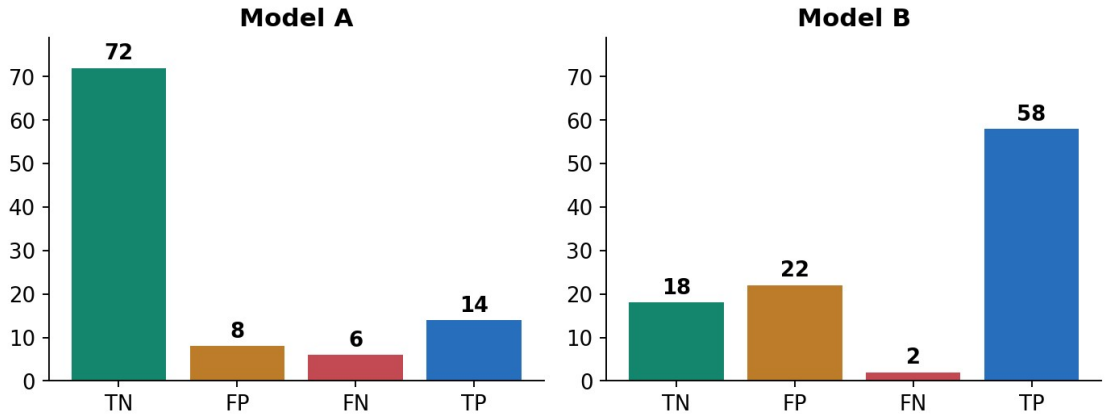


איור: לפני ואחרי פריסה: אותו סוג משימה, אך שיעור המחלקה החיובית שונה.

בפרויקט גמר כדאי לתעד את מועד איסוף הנתונים, את דרך הדגימה ואת הרכב קבוצות ההערכה. במקרה של שינוי משמעותי יש לשקול בדיקה חוזרת, ולא להציג מודל כ"מדויק תמיד".

15. השוואת KNN ו־Nearest Centroid — סוגרים מעגל

כעת אפשר לחזור למודלים שלמדנו. מדוע לא מספיק להדפיס accuracy_score עבור KNN ועבור Nearest Centroid? ייתכן ששני המודלים מציגים אותו Accuracy אך טועים במחשבים שונים לחלוטין. השוואה הוגנת תשתמש באותם נתוני Test, באותו צינור הכנה עקרוני, ובמדדים שנבחרו לפני הבדיקה לפי מטרת המערכת.



איור: שני מסווגים עם הרכב טעויות שונה: בחירה מקצועית מחייבת לראות מי מוחמץ.

למשל, אם מטרת המערכת היא לצמצם החמצות, נבחן Recall ואת מספר ה־FN. אם העלות של התרעות שווא גדולה, נבחן Precision ומספר FP. נדווח גם על יתרונות המימוש: זמן חיזוי, משאבי מחשוב, יכולת להסביר את ההחלטה וגודל נתוני האימון.

16. שאלות מקוריות בסגנון בגרות

רמה א׳ — ידע וחישוב

שאלה 1. מודל סיווג החזיר TP=24, FN=6, FP=12, TN=58. כתבו מטריצת בלבול, חשבו Accuracy, Precision, Recall ו־F1, והסבירו במילים מהו FN בהקשר של גילוי תקלות.

שאלה 2. בהינתן מספר TP ומספר FP, אפשר לחשב Precision. איזה נתון נוסף דרוש כדי לחשב Recall? נמקו מתוך משמעות המכנה בנוסחה.

שאלה 3. מודל חזה "אין תקלה" לכל 200 המחשבים, מתוכם 15 תקולים. חשבו את Accuracy ואת Recall, והסבירו כיצד יש לטפל ב־Precision שאינו מוגדר.

רמה ב׳ — פרשנות והשוואה

שאלה 4. שני מודלים נותנים Accuracy של 90%. לראשון Recall של 40%, ולשני Recall של 85%. הציעו מצב שבו נעדיף את הראשון ומצב שבו נעדיף את השני, והסבירו אילו מדדים נוספים חסרים.

שאלה 5. תלמיד כיוון את k ב־KNN לפי תוצאות קבוצת ה־Test, ואז דיווח את אותו ציון. הסבירו מה הבעיה בתהליך והציעו דרך מדויקת יותר.

שאלה 6. מדוע Precision גבוה אינו מבטיח Recall גבוה? המציאו דוגמה מספרית קטנה שתומכת בתשובה.

רמה ג׳ — חקר וחשיבה ביקורתית

שאלה 7. לבית ספר 1,000 מכשירים. 3% מהם תקולים. הציעו קו בסיס, בחרו מדד הערכה מוביל והסבירו איזו טעות מסוכנת יותר במקרה המתואר.

שאלה 8. חברה בוחרת בין מודל A למודל B. למודל A יש FP=5 ו־FN=20; למודל B יש FP=20 ו־FN=6. עלות FP היא 40 ₪ ועלות FN היא 300 ₪. חשבו עלויות, הכריעו ונמקו את ההנחות של ההכרעה.

שאלה 9. מערכת צמצמה את סף ההחלטה. ה־Recall גדל וה־Precision קטן. הסבירו מדוע הדבר אפשרי, ומה אפשר לבדוק כדי לבחור את הסף בסביבה שבה החמצת אירוע מסוכנת.

17. אתגר פרויקט גמר: כרטיס הערכת מודל

בחרו בעיה מפרויקט AI שלכם או מקרה לימודי ממאגר ציבורי. תעדו את מטרת החיזוי ואת המחלקה החיובית. הסבירו מי משתמש בפלט, מתי הוא מתקבל ומה עשויה להיות משמעות של אזעקת שווא לעומת החמצה.

מה התלמיד צריך להסביר	סעיף שחייב להופיע בדוח
מה המערכת מנסה לזהות ולמה?	בעיה ותווית חיובית
מה הוגדר Train, Validation, Test?	חלוקת נתונים
מהו פתרון פשוט להשוואה?	קו בסיס
כמה TP, FP, FN, TN התקבלו?	מטריצת בלבול
מדוע נבחרו Precision, Recall או F1?	בחירת מדדים
באילו דוגמאות המודל נכשל ומה ייבדק בהמשך?	שגיאות ומגבלות
האם יש חוסר איזון, דליפה או שינוי התפלגות?	יושר מדעי

בבחינה בעל־פה רצוי שהתלמיד יוכל להצביע על כל תא במטריצה, להסביר מי המקרים שהוחמצו, לתאר כיצד נבחר הסף ולהשיב מה יקרה אם שיעור החיוביים ישתנה. טבלה שנוצרה בלחיצה אחת אינה מחליפה הבנה.

18. בדיקת הבנה — לפני שממשיכים

1. מה ההבדל במשמעות של TP ו־TN?

2. איזה מכנה משמש Precision ואיזה Recall?

3. מתי אפשר לקבל Accuracy גבוה מאוד ו־Recall אפס?

4. מדוע שני מודלים בעלי Accuracy זהה עשויים להיות שונים באיכותם?

5. מדוע אסור לבחור סף באמצעות קבוצת Test ואז לדווח על אותה קבוצה?

6. מה כולל דיווח מדעי אחראי על ביצועי מסווג?

סיכום מהותי: אין להכריע על איכות מודל על סמך מספר יחיד. מטריצת הבלבול, בחירת המדדים, עלויות השגיאה, חלוקת הנתונים ומטרת השימוש יוצרים יחד הערכה מקצועית.

19. מפת הקשר לשלוש תכניות הלימוד

תפקיד בפרק	מה כבר יודעים / מה נלמד	מסלול
חזרה תמציתית ודיוק המושגים	KNN, Accuracy, מטריצת בלבול, מושגי אימון ובדיקה	יחידה 3 — כיתה י׳
העומק העיוני והחישובי של פרק זה	הערכת ביצועים והבנת מדדי איכות	יחידה 5 — פרק 5.1
ניתוח שגיאות, דיווח ותיעוד ניסויים	הערכת מודלים ומערכות כחלק מפרויקט גמר	התמחות למידה עמוקה

עוגן תכנית: יחידה 5, פרק 5 — מושגי יסוד בפיתוח אלגוריתמי למידת מכונה, סעיף 5.1 "הערכת ביצועים". המיפוי מבוסס על קובצי הסילבוס וההוראה במאגר GitHub של הקורס. ההרחבות על Precision–Recall, Balanced Accuracy עקומת ועלות טעויות מוצגות כאן כהעמקה פדגוגית; חובה פורמלית לכל פרט מחייבת הצלבה עם נוסח התכנית התקף.

20. מקורות להעמקה ואחריות אקדמית

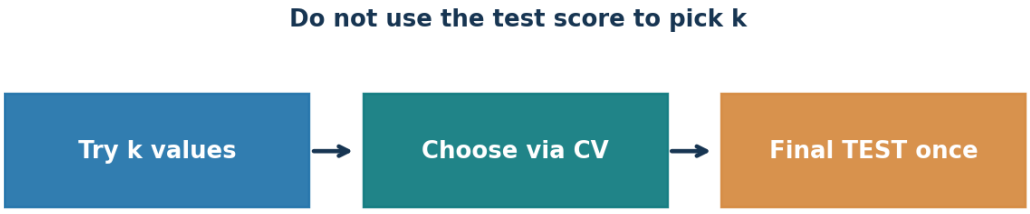
- משרד החינוך — נושאי ליבה בבינה מלאכותית, סעיף 5.1; מפת הקורס השמורה במאגר: https://github.com/samiabo75/Data-Science-ML-Course/tree/main/02-Unit5-AI-Core/05-ML-Development-Fundamentals
- Géron, A. Hands-On Machine Learning — תהליך עבודה, בחירת מדדים, אימון והערכת מודלים. ספר השראה בלבד; הדוגמאות והטקסט כאן מקוריים.
- James et al. An Introduction to Statistical Learning with Applications in Python — מסגרות השוואת מודלים, טעויות והכללה.
- Starmer, J. The StatQuest Illustrated Guide to Machine Learning — פירוק אינטואיטיבי של מדדים ומטריצות.
- scikit-learn User Guide, Model Evaluation: https://scikit-learn.org/stable/modules/model_evaluation.html

ספר מדעי הבינה המלאכותית

יחידה 5 — נושאי ליבה | פרק 5.2

ולידציה וחקר היפר־פרמטרים: איך בוחרים מודל שאפשר לסמוך עליו?

מהדורת תלמיד מורחבת · הסבר מדורג, חישוב ידני, Python, התנסות עצמאית והכנה לבגרות ולפרויקט גמר



איור: שלושה תפקידים שונים לשלוש קבוצות הנתונים. את קבוצת הבדיקה שומרים לסיום.

מעמד בתכנית: סעיף 5.2 ״ולידציה וחקר פרמטרים״ במפת נושאי הליבה שבמאגר הקורס. תכנות GridSearchCV, ניתוח דליפת מידע מתקדם ומקרי זמן/קבוצות מוצגים גם כהעמקות לפרויקט; אין לייחס להם דרישת חובה פורמלית בלי הצלבה עם נוסח הבחינה התקף.

לאן נגיע בסוף הפרק?

תוכלו להבחין בין פרמטר של מודל להיפר־פרמטר; להסביר מדוע דיוק האימון אינו אומדן אמין להצלחה בעולם האמיתי; לחלק נתונים ל־Train/Validation/Test; לבצע K-Fold באופן ידני; לחשב ממוצע ציונים על פני קיפולים; לבחור k ב־KNN בלי להסתכל על Test; להבין כיצד Pipeline מונע זליגת מידע; ולבצע ניסוי מתועד שתוכלו להצדיק בפני בוחן.

ידע קודם

KNN, Accuracy ומטריצת בלבול מפרקים 4.3 ו־5.1; ממוצע וסטיית תקן מפרק 2.1; תקנון מפרק 2.3; קריאת טבלאות, לולאות ו־Pandas/NumPy. בגרסה המודרכת של הקוד נדרשת היכרות עם fit, predict ועם המושגים X ו־y.

תוכן הפרק

1. בעיית הבחירה • 2. פרמטר והיפר־פרמטר • 3. הכללה והטיות • 4. חלוקת נתונים • 5. Holdout. חישוב ציונים • 6. K-Fold • 7. בחירת k • 8. StratifiedKFold • 9. Pipeline • 10. GridSearchCV. הערכה סופית • 11. ניסויים מיוחדים • 12. תרגול בגרות • 13. פרויקט גמר.

1. סיפור הפתיחה — המודל שהצליח יותר מדי

דודו מפתח מערכת שמתריעה על מחשבים שיש סיכון שיזדקקו לתיקון. הוא אסף נתוני עבר: גיל המחשב, מספר תקלות קודמות וטמפרטורה בזמן בדיקה. לכל מחשב קיימת תווית: האם התגלתה בו תקלה משמעותית. דודו בונה מסווג KNN ובודק אותו על אותם מחשבים שהכניס למערכת. הוא מקבל דיוק גבוה מאוד וממהר להכריז: ״המערכת מוכנה!״

אבל כשהוא מפעיל אותה על מחשבים חדשים, חלק מהתחזיות אינן נכונות. כיצד זה ייתכן? כאשר בודקים מודל על הדוגמאות שמהן למד, הבדיקה אינה מדמה באמת את העולם החדש. ב־KNN עם $k=1$, לדוגמה, כל דוגמת אימון היא בדרך כלל השכנה הקרובה ביותר של עצמה ולכן המודל יכול להראות 100% דיוק אימון בלי שהוכיח יכולת הכללה. השאלה המדעית איננה רק אם המודל זוכר את הנתונים, אלא אם הוא מסוגל להתמודד עם מקרים שטרם נראו.

שאלת הבנה: מדוע דיוק של 100% על נתוני האימון אינו הוכחה לכך שמודל יצליח על מחשבים חדשים?

2. מה המודל לומד ומה אנחנו בוחרים מראש?

2.1 פרמטר לעומת היפר־פרמטר

פרמטר (Parameter) הוא ערך שמוערך או נלמד מתוך נתוני האימון, למשל מקדם ברגרסיה ליניארית או מרכז מחלקה במסווג Nearest Centroid. לעומתו, היפר־פרמטר (Hyperparameter) הוא בחירה של מתכנן הניסוי או של תהליך חיפוש חיצוני. הוא קובע כיצד האלגוריתם יפעל או מה תהיה מידת הגמישות שלו. ב־KNN ההיפר־פרמטר המוכר הוא k — מספר השכנים המשתתפים בהצבעה. במודל רשת עצבית דוגמאות להיפר־פרמטרים הן קצב למידה, מספר שכבות וגודל אצווה.

KNN אינו מתאים במהלך fit משקולות דוגמת רגרסיה: הוא שומר את דוגמאות האימון ואת תוויותיהן, והחיזוי מתבצע לפי מרחקים לנתונים השמורים. לכן הבחירה ב־k והגדרת מדד המרחק חשובות במיוחד. איננו רוצים לבחור אותן לפי המקרים שאותם נשמור לבדיקה הסופית.

מושג	דוגמה	מי קובע?
פרמטר נלמד	מקדם w ברגרסיה	תהליך האימון
היפר־פרמטר	k ב־KNN	המפתח או חיפוש CV
מדד הערכה	Accuracy / F1	בחירה לפי מטרת היישום

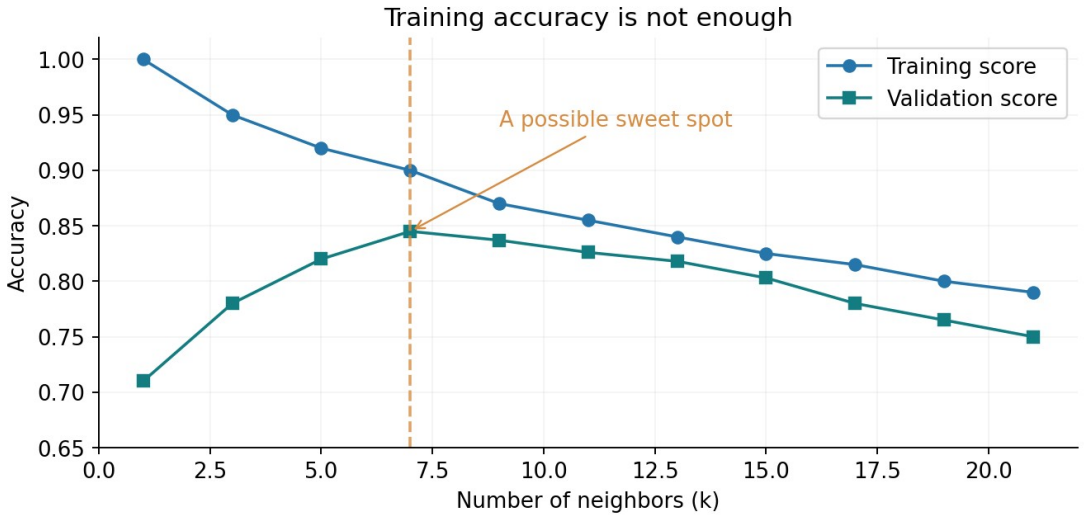
שאלת הבנה: האם מספר התכונות שנבחרו למודל הוא פרמטר שנלמד ב־**KNN**, או החלטה בתהליך פיתוח המודל? נמקו.

3. הכללה, התאמת חסר והתאמת יתר

הכללה (Generalization) פירושה הצלחה על דוגמאות חדשות שנדגמו מתנאים דומים לאלו שלגביהם אנו רוצים להשתמש במודל. כאשר מודל פשוט מדי ואינו מצליח לקלוט קשרים חשובים, אנו מדברים על התאמת חסר (Underfitting). כאשר מודל נעשה רגיש מאוד לפרטים מקריים של דוגמאות האימון ופחות יציב במצבים חדשים, מתעוררת התאמת יתר (Overfitting).

ב־KNN, ערך k קטן מאוד יוצר לרוב החלטות מקומיות ורגישות, בעוד ערך גדול מחליק את גבול ההחלטה. אולם אלה נטיות ולא חוק מוחלט: איכות המודל תלויה גם בנתונים, במספר התצפיות, בחפיפה בין

המחלקות ובמאפיינים. לכן לא נקבע מראש ש־k=5 תמיד טוב יותר מ־k=3. נבדוק זאת באמצעות נתונים שלא שימשו להתאמת המודל.



איור: איור סכמטי: דיוק האימון עשוי לרדת ככל ש־k גדל, בעוד דיוק הולידציה מגיע לשיא ואז יורד. הערכים להמחשה בלבד.

שאלת הבנה: מה תעדיפו: מודל בעל דיוק אימון 99% ודיוק ולידציה 70%, או מודל בעל דיוק אימון 88% ודיוק ולידציה 84%? הסבירו בלי להתייחס רק למספר הגדול ביותר.

4. שלושה תפקידים שונים לנתונים

כדי לבדוק מודל באופן הוגן מפרידים בין הנתונים המשמשים ללמידה ובחירה לבין הנתונים המיועדים להערכה סופית. אפשר לעבוד עם שלוש קבוצות: נתוני אימון (Train) לשם התאמת המודל; נתוני ולידציה (Validation) לבחירת הגדרות כמו k; ונתוני בדיקה (Test) שמוחזקים בצד ואינם משפיעים על החלטות הפיתוח. אין יחס חלוקה יחיד שהוא חובה: 60/20/20 היא דוגמה שימושית בלבד, ובמאגר קטן ייתכן שנרצה לבצע Cross-Validation בתוך קבוצת האימון.



איור: חלוקה אפשרית ל־60% אימון, 20% ולידציה ו־20% בדיקה. החלוקה אינה תקן קבוע לכל פרויקט.

4.1 חישוב ידני של חלוקה

נניח שיש ברשותנו 100 רשומות של מחשבים. בחלוקה המתוארת נקבל 60 רשומות אימון, 20 רשומות ולידציה ו־20 רשומות בדיקה. תחילה מאמנים על 60, לאחר מכן משווים אפשרויות באמצעות 20 רשומות הוולידציה, ולבסוף — פעם אחת לאחר קיבוע ההחלטות — מעריכים על 20 הרשומות שלא השתתפו בבחירה. ההפרדה הכרחית גם אם האלגוריתם עצמו פשוט.

100 × 0.60 = 60 | 100 × 0.20 = 20 | 100 × 0.20 = 20

בנתוני סיווג חשוב גם לשים לב לשכיחות המחלקות. אם יש רק מעט מחשבים תקולים, חלוקה אקראית רגילה עלולה להשאיר חלק מן הקבוצות עם מעט מדי דוגמאות חיוביות. חלוקה שכבתית (stratified split) מנסה לשמר את היחסים בין המחלקות. כשאותו מחשב מופיע כמה פעמים או כשהנתונים מתוארים לאורך זמן, יש כללים נוספים שנכיר בהמשך.

5. שיטת Holdout — בחירה באמצעות קבוצת ולידציה אחת

השיטה הפשוטה ביותר היא להפריד קבוצה אחת לבדיקה במהלך הפיתוח. דודו מאמן שלושה מודלים עם k שונים על אותו Train ומודד את מספר הסיווגים הנכונים על 20 דוגמאות Validation. תוצאת ולידציה היא אומדן לביצועי המודל על נתונים שלא ראה בזמן האימון; עם זאת, היא תלויה בחלוקה המסוימת. אם נחלק מחדש את הנתונים, ייתכן שנקבל דירוג אחר בין המודלים.

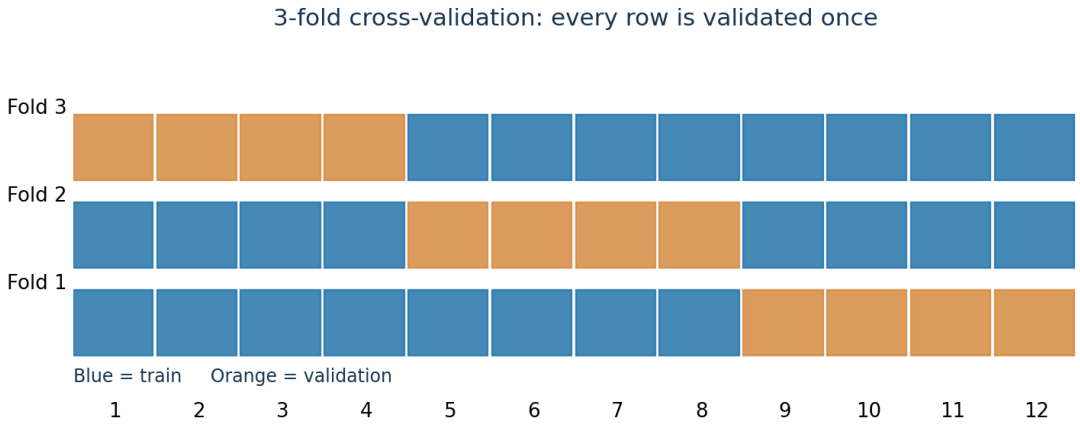
ערך **k**	סיווגים נכונים מתוך 20	**Accuracy** בוולידציה
1	15	75% = 15/20
3	17	85% = 17/20
7	16	80% = 16/20

בדוגמה זו נבחר k=3, מפני שקיבל את התוצאה הגבוהה ביותר בקבוצת הוולידציה. עדיין לא נשתמש ב־Test. לאחר הבחירה נוכל להתאים מחדש את המודל על כל הנתונים שהיו מותרים לפיתוח — לדוגמה Train ו־Validation יחד — ורק אז לבצע הערכת Test אחת. חשוב: ההחלטה על גודל k כבר התקבלה לפני בדיקת ה־Test.

שאלת הבנה: אם נחליף את קבוצת הוולידציה ו־k=7 יקבל 90%, האם הדבר מוכיח ש־k=7 טוב יותר בכל מצב?

6. Cross-Validation — במקום להסתמך על חלוקה אחת

Cross-Validation הוא שם למשפחה של שיטות בדיקה חוזרת. בשיטת K-Fold מחלקים את קבוצת הפיתוח ל־K חלקים דומים בגודלם, הקרויים קיפולים (Folds). בכל סבב משאירים קיפול אחד לוולידציה ומאמנים על יתר הקיפולים. לאחר K סבבים לכל דוגמה הייתה הזדמנות אחת להיות בוולידציה, ונקבל K ציוני הערכה. הממוצע שלהם נותן תמונה רחבה יותר מציון של חלוקה בודדת; הוא אינו מבטל את כל אי־הוודאות.



איור: ב־3-Fold כל אחת מ־12 התצפיות נמצאת פעם אחת בוולידציה ופעמיים באימון. זו הדגמה של קבוצת פיתוח בלבד; Test נפרד אינו מופיע באיור.

6.1 מימוש ביד עם 12 תצפיות

נמספר את התצפיות מ־1 עד 12. בקיפול הראשון תצפיות 1–4 הן ולידציה ותצפיות 5–12 הן אימון. בשני תצפיות 5–8 הן ולידציה, ובשלישי תצפיות 9–12. כך כל סבב כולל 8 תצפיות אימון ו־4 תצפיות ולידציה. למרות שבכל פעם מתבצע אימון על פחות נתונים מאשר בבניית המודל הסופי, ניתן להשוות באופן הוגן יחסית בין אפשרויות כאשר משתמשים באותם קיפולים ובאותו מדד.

קיפול	אימון	ולידציה	תוצאה לדוגמה
1	12–5	4–1	3/4
2	12–9 ,4–1	8–5	4/4
3	8–1	12–9	2/4

CV mean = (0.75 + 1.00 + 0.50) / 3 = 0.75 = 75%

שימו לב: חישוב הממוצע פשוט, אך הפרשנות מורכבת יותר. שלושה ציונים 75%, 100% ו־50% ממוצעם 75%, וההבדל הגדול בין הסבבים עשוי לרמוז על חוסר יציבות או על מדגם קטן. לא נסתפק באמירה ״המודל הצליח ב־75%״, אלא נציין שמדובר בממוצע Cross-Validation על קבוצת הפיתוח.

שאלת הבנה: בדוגמה של שלושת הקיפולים, מהו מספר הפעמים הכולל שבהן תצפית מסוימת מופיעה בקבוצת ולידציה? כמה פעמים היא מופיעה באימון?

7. ממוצע וסטיית תקן של ציוני הקיפולים

ממוצע הציונים עוזר להשוות מודלים, אך לפעמים שני מודלים בעלי אותו ממוצע נותנים תוצאות שונות מאוד מקיפול לקיפול. אפשר להציג את ציוני הקיפולים עצמם או את סטיית התקן שלהם כדי לתאר את הפיזור. אין לפרש סטיית תקן קטנה של קיפולים כהבטחה ליציבות מושלמת בעולם האמיתי: הקיפולים אינם ניסויים בלתי תלויים לגמרי, ונתונים חדשים עשויים להגיע מהתפלגות שונה.

$$\text{mean}(s_1,\ldots,s_k) = (s_1 + \ldots + s_k) / K$$

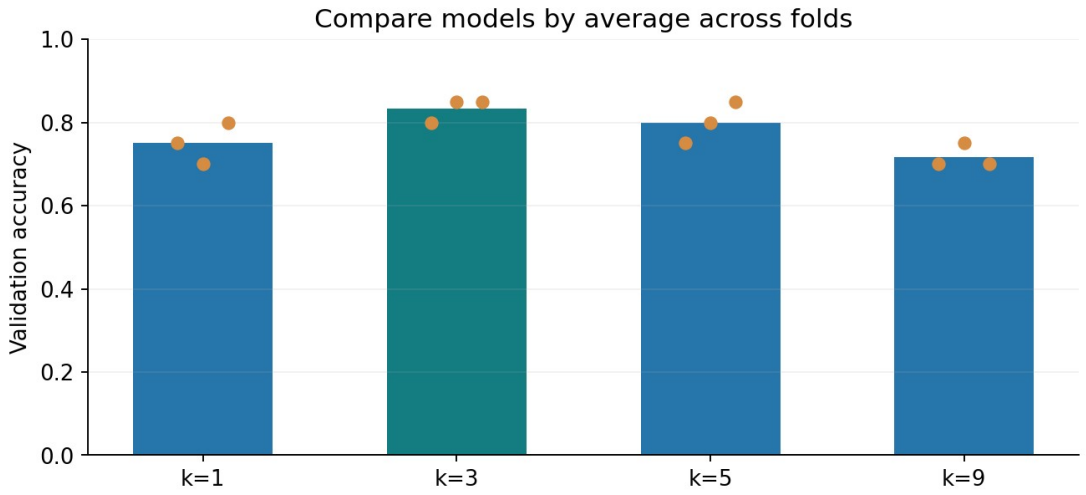
אם בציונים 0.75, 1.00, 0.50 נקבל ממוצע 0.75, הסטיות הן 0, 0.25, מינוס 0.25. בסכום ריבועי הסטיות נקבל 0.125. שונות האוכלוסייה של שלושת המספרים היא 0.125/3≈0.04167, וסטיית התקן ≈0.2041. זהו תיאור של שלושת ציוני הוולידציה, לא מדידה של הסיכון הסטטיסטי האמיתי במדויק.

$$\text{SD(scores)} = \sqrt{[(0^2 + 0.25^2 + (-0.25)^2)/3]} \approx 0.2041$$

8. חקר הפרמטר k — בוחרים על פי מספר קיפולים

נניח שדודו בודק ארבעה ערכים: k=1, k=3, k=5, k=9. הוא מקפיד שכל ערך ייבדק באותם שלושה קיפולים. עבור כל ערך אנו מחשבים ממוצע Accuracy של שלוש הבדיקות, ואז בוחרים את האפשרות בעלת הממוצע הגבוה ביותר. אם התוצאות כמעט שוות, כדאי לשקול גם מורכבות, יציבות, כמות הנתונים ואופי השגיאות. אין להציג הבדל קטן במספרים כהוכחה חד־משמעית לעליונותו של מודל אחד.

k	קיפול 1	קיפול 2	קיפול 3	ממוצע
1	0.75	0.70	0.80	0.750
3	0.80	0.85	0.85	0.833
5	0.75	0.80	0.85	0.800
9	0.70	0.75	0.70	0.717



איור: השוואת ממוצעי קיפולים. הנקודות הכתומות הן ציוני הקיפולים עצמם, והעמודות מציגות ממוצע.

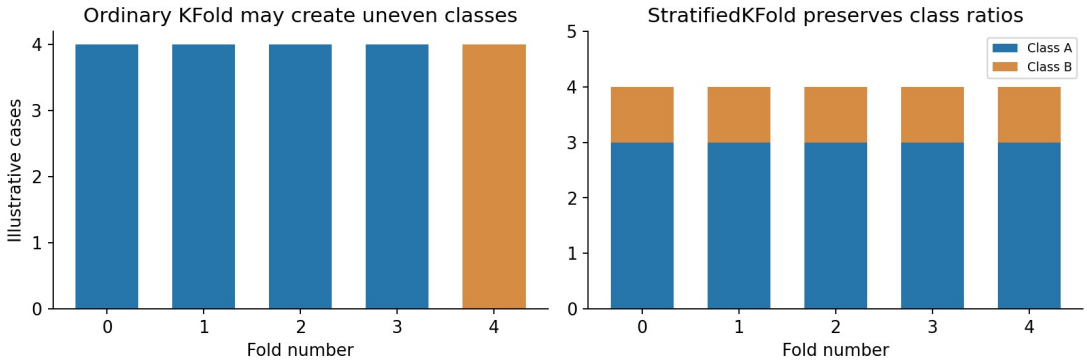
בדוגמה זו k=3 מוביל עם ממוצע של כ־0.833. הקפידו להשוות את התוצאה למדד שמתאים לבעיה. אם 5% מהמחשבים בלבד תקולים, ייתכן ש־Accuracy אינו המדד המרכזי: לעיתים Recall או F1 למחלקה החיובית יהיה רלוונטי יותר, כפי שלמדנו בפרק 5.1.

שאלת הבנה: מדוע חובה להשתמש באותו מדד ובאותה חלוקת קיפולים לצורך השוואה הוגנת ככל האפשר?

9. מתי צריך StratifiedKFold?

בסיווג יש לעיתים מחלקות נדירות. אם קיפול ולידציה מכיל כמעט רק מחשבים תקינים, הציון המתקבל אינו בוחן היטב את היכולת לזהות תקלות. StratifiedKFold מחלק את הדוגמאות כך שיחסי המחלקות יישמרו

בקירוב בכל קיפול. זו שיטת חלוקה שימושית במיוחד כאשר ההתפלגות בין המחלקות אינה מאוזנת, אך היא אינה מבטיחה שכל סבב יהיה זהה בדרגת הקושי, ואינה פותרת כשלעצמה את בעיית חוסר האיזון.

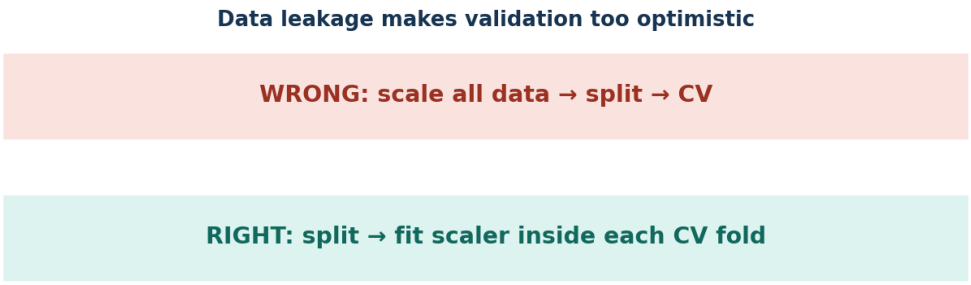


איור: המחשה סכמטית של שמירת פרופורציות מחלקות. בנתונים אמיתיים התוצאה תלויה בכמות התצפיות בכל מחלקה.

חשוב להכיר גם מגבלה: מספר הקיפולים אינו יכול להיות גדול באופן סביר ממספר הדוגמאות במחלקה הנדירה ביותר אם אנו מצפים לייצוג שלה בכל קיפול. כמו כן, כאשר יש רשומות חוזרות של אותו מחשב, שימוש בחלוקה שכבתית בלבד לא מונע מעבר של רשומות קשורות בין אימון לוולידציה. שם נדרשת חלוקה לפי קבוצות.

10. Data Leakage — כשמידע מגיע למודל מוקדם מדי

זליגת מידע היא מצב שבו תהליך הלמידה או בחירת המודל משתמש במידע שלא היה אמור להיות זמין באותו שלב. למשל, אם נחשב ממוצע וסטיית תקן של כל המאגר לפני Cross-Validation, אפילו ערכי הוולידציה השפיעו על המספרים שבהם השתמשנו לתקנון נתוני האימון. במקרה כזה ההערכה יכולה להיות אופטימית מדי. גם אם המודל אינו מקבל את התוויות של הוולידציה, הוא כבר קיבל מידע על התפלגות המאפיינים שלה.



איור: תקנון כל המאגר לפני חלוקה הוא תהליך לא תקין; יש להתאים את התקנון לנתוני האימון בכל קיפול.

לדוגמה, נניח שבקיפול מסוים גילי המחשבים באימון הם 2, 4, 6 שנים, ואילו בוולידציה יש מחשב שגילו 20. ממוצע האימון הוא 4, אבל ממוצע כל הארבע הוא 8. התלמיד מבין מיד שכאשר מחשבים ממוצע 8 לפני החלוקה, התצפית החדשה השפיעה על הייצוג המספרי של נתוני האימון. זה בדיוק המידע שהוולידציה אמורה לשמור לעצמה.

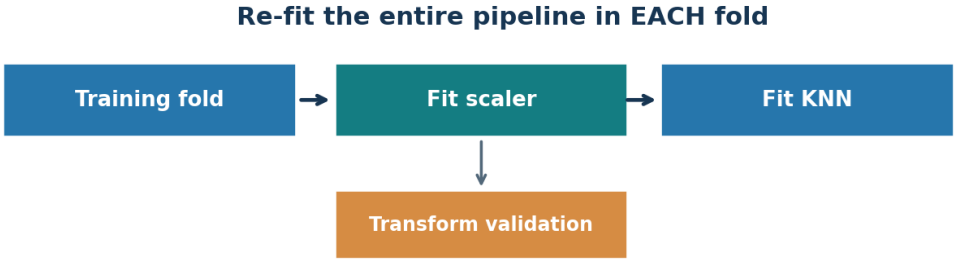
Train mean = (2 + 4 + 6) / 3 = 4

Wrong all-data mean = (2 + 4 + 6 + 20) / 4 = 8

שאלת הבנה: מדוע אסור לחשב את ממוצע כל הדוגמאות מראש, אפילו אם איננו משתמשים בתווית של דוגמת הוולידציה?

11. Pipeline — סדר פעולות שאינו ״מציץ״ לוולידציה

Pipeline הוא כלי המחבר כמה צעדי עיבוד למהלך אחד. אם נכניס לתוכו StandardScaler ומסווג KNN, תהליך Cross-Validation יוכל להתאים מחדש את הסקיילר ואת המסווג בתוך כל קיפול אימון, ורק אז להפעיל את אותו סקיילר על קיפול הוולידציה. בדרך זו איננו צריכים לכתוב ידנית את רצף הפעולות בכל סבב, והסיכוי לטעות קטן. Pipeline אינו פותר אוטומטית כל סוג זליגה; האחריות להפרדה נכונה ולבחירת מאפיינים מותרת נשארת בידי המפתח.



איור: בכל קיפול מחשבים מחדש את פרמטרי התקנון על חלק האימון בלבד; לאחר מכן מעבירים את קיפול הוולידציה ב־transform.

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import StratifiedKFold, cross_val_score

model = Pipeline([
    ("scale", StandardScaler()),
    ("knn", KNeighborsClassifier(n_neighbors=3))
])
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(model, X_train, y_train, cv=cv, scoring="accuracy")
print(scores)
print(scores.mean())
```

פירוש הקוד: Pipeline מגדיר תחילה כיצד לתקנן ואחר כך כיצד לסווג. StratifiedKFold קובע חמישה קיפולים ששומרים בקירוב על היחס בין המחלקות. cross_val_score מתאים בכל סבב עותק של צינור העיבוד על תת־קבוצת האימון ומחשב Accuracy על תת־קבוצת הוולידציה. הפלט הוא מערך של חמישה ציונים, והממוצע מסכם את הביצועים על פני הסבבים. המשתנים X_train ו־y_train מתקבלים מחלוקת הנתונים כפי שנדגים בהמשך.

12. דוגמה מלאה ב־Python — מנתונים ועד בחירת k

לשם הדגמה נשתמש במאגר Iris המובנה ב־scikit-learn. המאגר מכיל מדידות של פרחי אירוס ותווית של מין הפרח. נפריד 20% מן הרשומות ל־Test ונשמור אותן בצד. על 80% הרשומות האחרות נבצע Cross-

Validation למציאת k. הקוד מדגים תהליך מקצועי; במשימות המעבדה נדרשת השלמה עצמאית על וריאציות שלא נפתרו כאן.

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split, StratifiedKFold
from sklearn.model_selection import GridSearchCV
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier

X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)
pipe = Pipeline([
    ("scale", StandardScaler()),
    ("knn", KNeighborsClassifier())
])
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
search = GridSearchCV(
    estimator=pipe,
    param_grid={"knn__n_neighbors": [1, 3, 5, 7, 9]},
    scoring="accuracy", cv=cv, refit=True
)
search.fit(X_train, y_train)
print("Selected k:", search.best_params_)
print("Mean CV score:", search.best_score_)
print("Final test score:", search.score(X_test, y_test))
```

נפרק את המשמעות: הפרמטר stratify=y שומר בקירוב על שיעור מיני האירוס בחלוקה החיצונית; random_state מאפשר לשחזר את אותה חלוקה; `knn__n_neighbors` מציין פרמטר פנימי של שלב `knn` בתוך ה־Pipeline; `best_score_` הוא ממוצע ציון ה־Cross-Validation של ההגדרה שנבחרה — לא ציון Test; ובגלל refit=True, בסיום החיפוש המודל המועדף מותאם מחדש על כל X_train. רק לאחר מכן אנו פונים ל־X_test פעם אחת כדי להעריך תוצאה סופית.

13. GridSearchCV — חקר שיטתי של אפשרויות

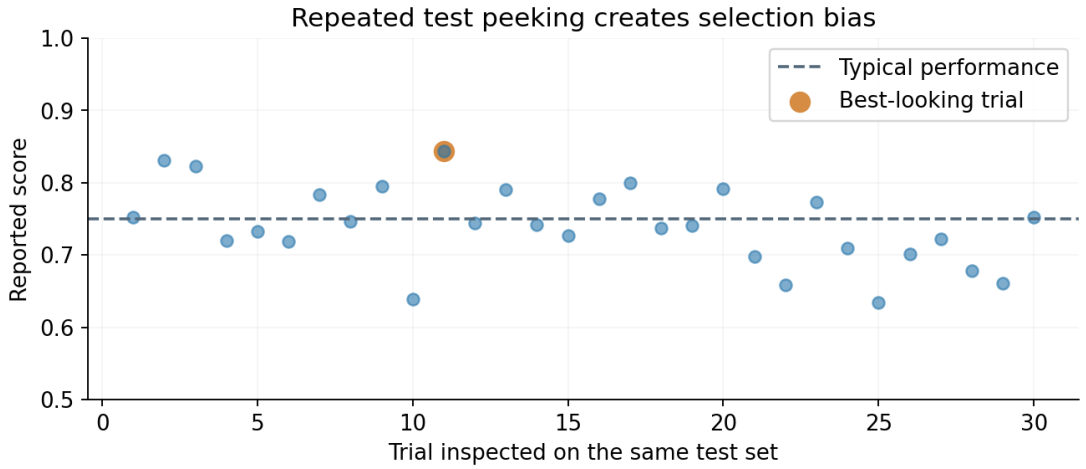
במקום לכתוב ידנית לולאה לכל k, אפשר לבקש מ־GridSearchCV לנסות רשימת ערכים מוגדרת מראש. בכל ערך הכלי מבצע Cross-Validation, מרכז ציונים ובוחר את ההגדרה הטובה ביותר לפי המדד שביקשנו. זה אינו חיפוש קסום: הכלי בודק רק את הערכים שהגדרנו, והבחירה עדיין תלויה באיכות הקיפולים, במדד ובמאפיינים. בחירת טווח החיפוש ונימוקו הם חלק מתכנון הניסוי.

שדה/פעולה	מה המשמעות?
param_grid	אוסף ערכי היפר־פרמטרים לבדיקה
cv	אסטרטגיית הקיפולים
scoring	המדד שבאמצעותו משווים
_best_params	ההגדרה שהובילה בחיפוש
_best_score	ממוצע ציוני CV עבור ההגדרה המובילה
_cv_results	טבלת כל הניסויים והציונים
refit=True	התאמה מחדש של המודל הנבחר על קבוצת הפיתוח

שימו לב שאין סיבה להפעיל GridSearchCV אם התלמיד אינו מסוגל להסביר קודם מדוע צריך לבצע חלוקת נתונים, מה הוא מודד בכל קיפול וכיצד נבחרת התוצאה. לפיכך כלי החיפוש נלמד כאן כהרחבת מימוש מקצועית שמשרתת את פרויקט הגמר, נוסף על ההבנה העיונית של תהליך הוולידציה.

14. מדוע קבוצת Test נבדקת רק לאחר בחירה?

אם דודו בודק k=1 על Test, ואז k=3 על אותו Test, ואז משנה את המאפיינים כדי לשפר את Test, קבוצת הבדיקה חדלה להיות עצמאית. בפועל היא שימשה לוולידציה נוספת. גם אם לא אימן על תוויות Test ישירות, הבחירות הושפעו מתוצאותיו. לאחר מאות ניסויים אפשר לקבל תוצאה גבוהה במקרה, משום שבחרנו את הניסוי שהיה לו מזל בנתונים המסוימים של Test.



איור: המחשה סימולטיבית: צפייה חוזרת ב־Test ובחירת התוצאה הגבוהה יכולה ליצור אומדן מוטה.

ההליך הנכון הוא להפריד Test מוקדם, לקבוע מדד מתאים, לבחור מאפיינים והיפר־פרמטרים רק בתוך קבוצת הפיתוח, ואז להעריך פעם אחת על Test שלא השתתף בבחירה. אם לאחר תוצאת Test מחליטים לשנות את המודל, יש לתעד שה־Test כבר השפיע על הפיתוח, ולצורך הערכה בלתי תלויה נזדקק בעתיד לנתונים חדשים נוספים.

15. מקרים שבהם חלוקה אקראית אינה נכונה

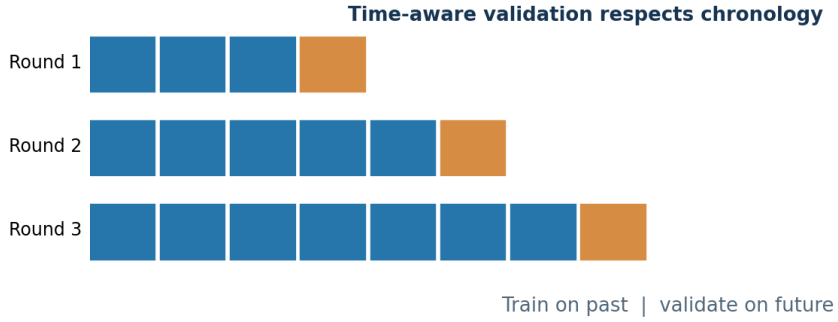
15.1 אותו אדם, תלמיד או מחשב בכמה רשומות

אם אותו מחשב מופיע עשר פעמים במאגר, וחלק מן המדידות שלו נמצאות באימון וחלק בוולידציה, המודל עשוי לזהות דפוס מיוחד לאותו מחשב. האם זו הכללה למחשבים חדשים? לא בהכרח. אם מטרת הפרויקט היא לחזות לגבי מחשבים חדשים, יש להפריד את כל הרשומות של אותו מחשב לאותה קבוצת אימון או ולידציה. לכך נועדו שיטות חלוקה לפי קבוצות, למשל GroupKFold — הרחבה מעשית לפרויקט.

15.2 נתונים שנאספו לאורך זמן

אם אנחנו רוצים לנבא תקלות שיתרחשו בחודש הבא, נעדיף לאמן על חודשים קודמים ולבדוק על חודשים מאוחרים. חלוקה אקראית של עבר ועתיד עלולה לתת למודל מידע שלא היה זמין בזמן ההחלטה.

או חלוקה כרונולוגית ידנית יכולים להיות מתאימים; יש לשים לב גם למאפיינים שנוצרים TimeSeriesSplit מהעתיד ולפערי זמן הנדרשים כדי למנוע זליגה.



איור: בדיקה המדמה חיזוי לעתיד: כל סבב מתאמן על עבר ומעריך על תקופה מאוחרת יותר.

16. האם תמיד משתמשים ב־Accuracy?

לא. פרק 5.1 לימד שמדד ההערכה תלוי במטרה. אם מערכת מחשבים אמורה לאתר תקלות נדירות אבל מסוכנות, בחירה לפי Accuracy יכולה להעדיף מודל שמנבא ״תקין״ כמעט תמיד. אפשר לבחור Recall, או מדד עלות מותאם, בתנאי שמחליטים מראש מה חשוב ומשתמשים באותו מדד לכל המודלים. F1 בפרויקט גמר על התלמיד להסביר את הבחירה וגם להציג את יתר המדדים הרלוונטיים, משום שמדד יחיד אינו מספר תמיד את כל הסיפור.

גם ציון CV גבוה אינו חסין לשינויי מציאות: תקלות מסוג חדש, חיישן חדש, בית ספר אחר או שנתון מחשבים אחר עלולים לשנות את התפלגות הנתונים. לכן לצד בדיקות כמותיות צריך להציג תיאור של תנאי השימוש, מגבלות המאגר ואפשרויות לניטור לאחר הפריסה.

17. סיכום מדעי: מה מוכיח תהליך בדיקה תקין?

Cross-Validation אינו מוכיח שמודל לעולם לא יטעה. הוא מספק דרך מסודרת יותר להשוות אפשרויות על נתוני פיתוח. Test עצמאי מסייע להעריך את המודל לאחר שקיבלנו את החלטותינו. כאשר החלוקה מייצגת את תנאי השימוש העתידיים, והמדד מתאים למטרת הפרויקט, ההערכה הופכת לאמינה יותר. כאשר יש דליפת מידע, נתונים לא מייצגים או שימוש חוזר ב־Test, גם מספר מרשים עלול להטעות.

השאלה שצריך לענות עליה	הפעולה	שלב
מה המודל למד?	fit על נתונים מותרים	Train
איזה k ומדד לבחור?	השוואת אפשרויות	Validation / CV
כיצד מנצלים את נתוני הפיתוח?	התאמה מחדש אחרי הבחירה	Refit
כמה טוב המודל על דוגמאות שלא שימשו לבחירה?	הערכה סופית אחת	Test
האם הבדיקה משקפת את תנאי השימוש האמיתיים?	תיעוד, מגבלות וביקורת	פרויקט

18. משימות הבנה מדורגות

רמה א׳ — בסיס

1. הסבירו במילים שלכם מה ההבדל בין Train, Validation ו־Test. 2. יש 80 דוגמאות ומחלקים 25% מהן ל־Test; כמה יישארו לקבוצת פיתוח? 3. מדוע k הוא היפר־פרמטר ולא פרמטר שנלמד על־ידי KNN? 4. בארבעה קיפולים התקבלו 0.7, 0.8, 0.9, 0.8; חשבו ממוצע ופרשו את המספר.

רמה ב׳ — יישום

5. נתונים 15 ציוני ולידציה? הציגו בתרשים K-Fold בעל חמישה קיפולים מה תפקידו של כל חלק בסבב הראשון והשני. 6. קיבלתם בטבלת CV עבור k=1 ממוצע 0.91 ועבור k=5 ממוצע 0.89. הציעו שתי בדיקות נוספות לפני קביעה שהראשון עדיף. 7. הסבירו מדוע פעולה `StandardScaler().fit_transform(X)` לפני חלוקת Train/Test עלולה להטעות. 8. כתבו משפט שיבדיל בין `_best_score` לבין `score(X_test,y_test)`.

רמה ג׳ — חשיבה מדעית ופרויקט

9. מאגר מכיל מדידות חודשיות של אותם 35 מחשבים. איזו חלוקת ולידציה מתאימה יותר אם רוצים לחזות על מחשבים שלא נראו בעבר? נמקו. 10. תלמיד בדק עשרה ערכי k על Test, בחר את הטוב ביותר ודיווח עליו כציון הסופי. הגדירו את הכשל והציעו תכנון חלופי. 11. הסבירו מדוע גם CV בעל ממוצע גבוה לא מבטיח ביצועים דומים על בית ספר חדש. 12. תכננו ניסוי לבחירה בין KNN ו־Nearest Centroid: אילו נתונים, אילו מדדים, כיצד תחלקו, ובאיזה שלב תדווחו את Test?

19. שאלת הערכה מקורית בסגנון בגרות

בבית ספר נבדקו 120 מחשבים. הוחלט לשמור 24 מחשבים לקבוצת Test, וב־96 האחרים לבצע חיפוש אחר k במודל KNN. התלמיד בדק k=1, k=3 ו־k=5 באמצעות ארבעה קיפולים, והתוצאות מוצגות בטבלה. כל מספר הוא שיעור הסיווגים הנכונים בקיפול ולידציה.

k	קיפול 1	קיפול 2	קיפול 3	קיפול 4
1	0.75	0.79	0.71	0.75
3	0.83	0.88	0.79	0.83
5	0.79	0.83	0.83	0.79

א. תארו כיצד פועל תהליך ארבעה קיפולים על 96 המחשבים. ב. חשבו לכל ערך k את הממוצע, וציינו איזו אפשרות תבחרו ונמקו. ג. הסבירו מדוע יש לשמור את 24 מחשבי Test מחוץ לבחירת k. ד. התלמיד ביצע תקנון על כל 120 המחשבים לפני החלוקה; הסבירו מה הבעיה והציעו פתרון מעשי. ה. לאחר הבחירה התקבל Accuracy של 0.83 על Test; האם ניתן להסיק שהמערכת תזהה היטב תקלות נדירות? מה עוד חשוב לבחון?

המשימה היא שאלה מקורית שנכתבה לצורכי לימוד ומדמה מיומנויות בחינה; אין לייחס אותה לשאלון בגרות רשמי.

20. מעבדת Colab — לא מחליפים את החשיבה בפעולת כפתור

לצד הפרק מצורפת מחברת תלמיד עצמאית. בחלק הראשון תזהו את X ואת y, תחלקו נתונים ותחשבו ציוני קיפולים. בהמשך תבדקו KNN בכמה ערכי k, תוסיפו StandardScaler באמצעות Pipeline ותפיקו טבלת השוואה. המשימות מסומנות להשלמה והפתרונות המלאים אינם מופיעים במחברת. ההדגמה המלאה בספר משמשת מקור ללמידת השיטה, אך המעבדה מחייבת להפעיל אותה על חלוקה וניסויים משלכם.

21. חיבור לפרויקט הגמר — מה מתעדים?

פרויקט גמר איכותי כולל לא רק קוד ותוצאה, אלא יומן ניסויים. לכל ניסוי על התלמיד לרשום מקור נתונים, חלוקת Train/Test, שיטת הולידציה, הכנת נתונים, ערכי ההיפר־פרמטרים שנבדקו, המדד שנבחר, תוצאות CV, התוצאה הסופית על Test ומגבלות התהליך. יומן כזה מאפשר לשחזר החלטות ולהציג בפני בוחן לא רק מה המודל עושה, אלא מדוע אפשר לסמוך על דרך הבדיקה שלו.

שאלה לתיעוד	רכיב ביומן
איזו תחזית נדרשת ולמי?	שאלת מחקר
מה היה זמין בזמן החיזוי?	מאפיינים ותווית
כיצד נשמרו Test, זמן וקבוצות?	חלוקה
מה נבדק ובאילו קיפולים?	בחירת מודל
למה דווקא Accuracy / Recall / F1?	מדד מרכזי
מהו CV, מהו Test ומה ההבדל ביניהם?	תוצאות
אילו מקרים עלולים להכשיל את המודל?	מגבלות

22. מילון מושגים

מונח	משמעות
Generalization	יכולת לפעול היטב על נתונים חדשים
Overfitting	התאמת יתר: תלות מוגזמת בפרטי האימון
Underfitting	התאמת חסר: מודל שאינו קולט מבנה חשוב
Hyperparameter	הגדרה הנבחרת לפני/מסביב לתהליך האימון
Validation	בדיקה המשמשת לבחירת מודל או הגדרות
K-Fold	חלוקת נתוני פיתוח לקיפולים וביצוע אימון ובדיקה חוזרים
StratifiedKFold	קיפולים השומרים בקירוב על שיעור המחלקות
Pipeline	רצף עיבוד ומודל המותאם כיחידה אחת
Data Leakage	חדירת מידע לא מותר לתהליך הבחירה/האימון
GridSearchCV	חיפוש על פני ערכים מוגדרים עם Cross-Validation

23. מקורות מקצועיים ומעמד התוכן

עוגן תכנית: מאגר Data-Science-ML-Course, יחידה 5, פרק 5.2 — ולידציה וחקר פרמטרים. סיכום ההשתלמות במאגר יחידה 3, מפגש 8, משמש גשר דידקטי לנושא. יש לבדוק בנפרד את הדרישות המחייבות של הבחינה העדכנית. תוכן זה הוא כתיבה מקורית לתלמידים, על בסיס הידע המקצועי שבספרים הבאים ובתיעוד המפתחים:

- James, Witten, Hastie, Tibshirani & Taylor, An Introduction to Statistical Learning with Applications in Python (2023), פרק Resampling Methods. https://www.statlearning.com/
- Aurélien Géron, Hands-On Machine Learning — עקרונות עבודה מעשית לבחירת מודלים, ניסויים ו־Pipelines. • Luis Serrano, Grokking Machine Learning — הסבר אינטואיטיבי של אימון והכללה. • Deisenroth, Faisal & Ong, Mathematics for Machine Learning — יסודות מתמטיים לסטטיסטיקה ולהערכת מודלים.
- scikit-learn User Guide: https://scikit-learn.org/stable/modules/cross_validation.html ; Common pitfalls: https://scikit-learn.org/stable/common_pitfalls.html ; GridSearchCV: https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.GridSearchCV.html

הרחבות כגון GroupKFold, TimeSeriesSplit, GridSearchCV וניהול ניסויים מעמיק הן בעלות ערך גבוה לפרויקט, אך מעמדן הפורמלי בחומר הבחינה טעון הצלבה עם מסמך ההנחיות העדכני של משרד החינוך.

חלק ב׳

התמחות בלמידת מכונה ולמידה עמוקה

מרגרסיה ומערכות תמונה, דרך פרספטרון ורשתות עמוקות, ועד לפרויקט גמר.

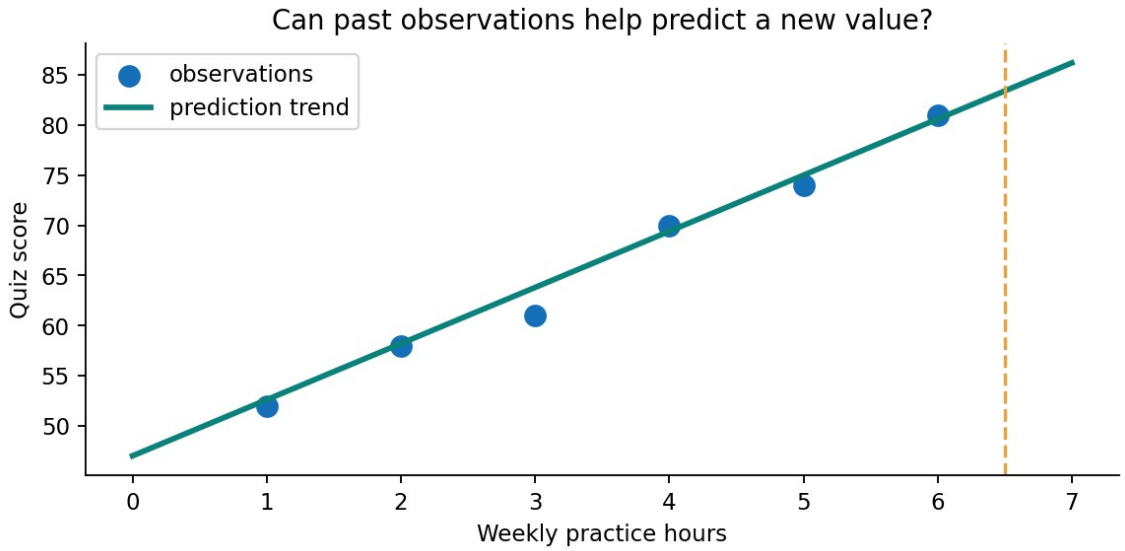
החלק כולל את פרקי ההתמחות 5–16; פרקים 1–4 מפורטים במפת הכיסוי בפתח הספר.

ספר מדעי הבינה המלאכותית

מסלול הלמידה העמוקה | פרק 5 בתכנית ההתמחות

רגרסיה ליניארית — מהחיזוי הראשון ועד לאימון מודל

מהדורת תלמיד מורחבת · מתמטיקה מוסברת · חישוב ידני · תרשימים · Python · תרגול לבגרות ולפרויקט גמר



איור: לומדים לזהות קשר בין מאפיין לתוצאה ולבדוק אם אפשר לחזות דוגמה חדשה.

מיקום בתכנית: פרק 5 — רגרסיה ליניארית (15 שעות) במסלול ההתמחות בלמידת מכונה ולמידה עמוקה, על פי מפת התכנית השמורה במאגר. הפרק נשען על מצגות ההשתלמות 4.1 ו־4.2, על תרגיל Iris ועל מחברת KC House. חלקים כגון גזירה של נוסחת הריבועים הפחותים ו־Gradient Descent מסומנים כהעמקה דידקטית ולא כדרישת בחינה מאומתת.

מטרות הפרק

בסיום הפרק תוכלו להבחין בין סיווג לרגרסיה, לקרוא את המשוואה $\hat{y} = wx + b$ ולהסביר כל סימן, לבצע תחזיות ידניות, לחשב שאריות ושגיאות, להבין כיצד נקבע הישר המתאים לנתונים, לממש LinearRegression ב־scikit-learn, לפרש MAE, MSE, RMSE ו־R^2, להרחיב למספר מאפיינים, ולהציע ניסוי הוגן על נתונים שלא שימשו לאימון.

ידע קודם והנחיות קריאה

לפני הקריאה נדרשת היכרות עם צירים, משוואת ישר, ממוצע, תרשים פיזור, וקטורים, טבלת DataFrame ופעולות בסיסיות ב־Python. אם פעולת המכפלה הפנימית אינה מוכרת, חזרו לפרק 1.1; אם מטריצת מאפיינים אינה ברורה, חזרו לפרק 4.1. תחילה נסביר כל פעולה בעברית, ורק אחר כך נשתמש בספרייה מקצועית.

מפת הדרך

בעיה אמיתית → משתנה מטרה → משוואת חיזוי → שיפוע וחיתוך → שארית → מדדי שגיאה → מציאת ישר מיטבי → Python → אימון/בדיקה → מספר מאפיינים → גשר לאופטימיזציה → תרגול עצמאי.

1. סיפור הפתיחה — האם ניתן לנבא תוצאה שטרם נמדדה?

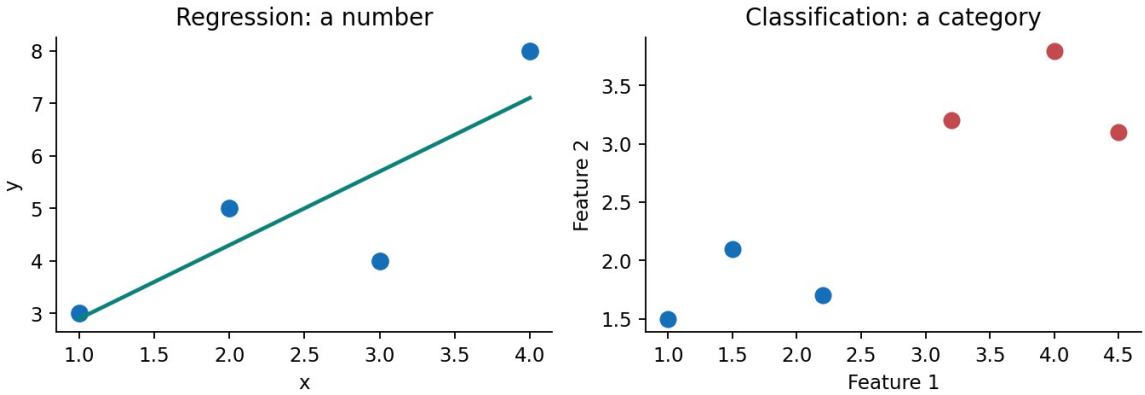
דודו מדריך תלמידים במעבדת תכנות. הוא מבחין שתלמידים המקדישים יותר שעות לתרגול מקבלים בדרך כלל ציונים גבוהים יותר בבוחן קצר. הוא אינו רוצה לנחש את הציון של כל תלמיד מחדש: הוא מחפש כלל מספרי שיוכל להשתמש בו לתלמידים נוספים. בידיו טבלה עם שעות תרגול והציונים שהתקבלו בעבר. משימתנו היא לבחון האם קיים קשר מועיל, כיצד לבטא אותו במספרים, וכיצד לדעת אם הכלל מצליח גם על תלמיד שלא השתתף באימון המודל.

שימו לב להבדל בין שאלה תיאורית לשאלת חיזוי: ״מהו ממוצע הציונים?״ היא שאלה סטטיסטית תיאורית. ״איזה ציון צפוי לתלמיד שתרגל שלוש שעות?״ היא שאלת חיזוי. גם אם התשובה המספרית נראית מדויקת, היא תחזית ולא הבטחה.

בדיקת הבנה: אם שני תלמידים למדו אותו מספר שעות וקיבלו ציונים שונים — האם פירוש הדבר שרגרסיה אינה אפשרית? נמקו.

2. מתי משתמשים ברגרסיה ומתי בסיווג?

למידה מונחית מתחילה בדוגמאות שבהן גם המאפיינים וגם התוצאה ידועים. ברגרסיה (Regression) התוצאה היא קטגוריה: ״תקין/תקול״, ״עיר/יער״, או סוג הפרח. אין לבחור אלגוריתם רק משום שהוא מוכר לנו; הבחירה מתחילה מהגדרת התוצאה.



איור: ברגרסיה מנבאים מספר על ציר רציף; בסיווג מנבאים תווית של מחלקה.

משפחה	הפלט	תכונות הקלט	שאלת חיזוי
רגרסיה	מספר דקות	גיל המחשב, סוג תקלה	כמה דקות יידרש לתיקון?
סיווג	כן / לא	גיל, תקלות עבר	האם המחשב יתקלקל?
רגרסיה	סכום כסף	שטח, מספר חדרים	מהו מחיר דירה?

חשוב: העובדה שמספר קטגוריות מסומנות בקוד כמספרים 0 ו־1 אינה הופכת אותן אוטומטית למשתנה רציף.

3. השפה של הנתונים: מאפיינים, מטרה, תחזית

לכל דוגמה יש מאפיינים (Features), שהם נתונים הידועים בזמן החיזוי. את התוצאה המבוקשת נקרא משתנה המטרה (Target), ומסמנים לרוב y. את כלל המאפיינים של הדוגמאות מסדרים במטריצה X.

כאשר יש רק מאפיין אחד, אפשר לצייר את הדוגמאות כנקודות במישור. כאשר משתמשים בשני מאפיינים, כל דוגמה היא וקטור בעל שני רכיבים, גם אם התוצאה המבוקשת נשארת מספר אחד.

Features (X) versus target (y)

Example	Practice hours	Breaks	Result
A	1	2	3
B	2	1	5
C	3	2	4
D	4	1	8

איור: ב־X שומרים את נתוני הקלט; בעמודת התוצאה y שומרים את הערך שאותו רוצים לחזות.

X: shape = (number of examples, number of features)

אם יש ארבע דוגמאות ושני מאפיינים לכל אחת, המטריצה X מכילה ארבע שורות ושתי עמודות. לעומת זאת, וקטור y מכיל ארבעה ערכים — ערך מטרה אחד לכל דוגמה. חשוב להתאים תמיד בין מספר השורות ב־X לבין אורך y.

```
import numpy as np
X = np.array([[1, 2], [2, 1], [3, 2], [4, 1]])
y = np.array([3, 5, 4, 8])
print(X.shape)     # (4, 2)
print(y.shape)     # (4,)
```

בדיקת הבנה: כאשר תוסיפו מאפיין שלישי לארבע הדוגמאות, מה ישתנה בממדי X, ומה לא ישתנה ב־y?

4. משוואת הישר: להפוך דפוס לכלל חיזוי

נפתח במודל הפשוט ביותר: ישר שקושר בין מאפיין אחד x לבין תחזית מספרית. הרעיון אינו שהנתונים עצמם חייבים להימצא על קו ישר, אלא שאנו משתמשים בקו כקירוב מסודר למגמה הכללית. סימן הגג מעל y נועד להבחין בין התוצאה שנמדדה בפועל לבין ערך שהמודל חישב.

$$\hat{y} = w\,x + b$$

סימן	השם	מה הוא אומר?
x	מאפיין	הנתון הידוע בעת החיזוי
w	מקדם / שיפוע	כמה תשתנה התחזית כאשר x גדל ביחידה אחת
b	הטיה / חיתוך	התחזית כאשר x שווה אפס
$\hat{y}$	תחזית	הערך שהמודל מחזיר; לא בהכרח הערך האמיתי

דוגמה: נניח שהמודל הוא $\hat{y} = 5x + 48$, כאשר x הוא מספר שעות תרגול והתחזית היא ציון. אם תלמיד מתרגל שלוש שעות, התחזית היא 63 נקודות: מכפילים שלוש בחמש ומוסיפים 48. אם השעות גדלות מארבע לחמש, התחזית עולה בחמש נקודות. זוהי משמעות השיפוע. את $b=48$ אין לפרש בהכרח כציון אמיתי ללא תרגול; ייתכן שערך $x=0$ כלל אינו בטווח הנתונים שעליו למדנו.

$$\hat{y}(3) = 5 \cdot 3 + 48 = 63$$

בדיקת הבנה: אם מחליפים את $w=5$ ב־$w=-5$, כיצד משתנה כיוון הקשר? מהי המשמעות של b שלילי?

5. חיזוי ידני: שלוש דוגמאות ללא Python

נבחר מודל לדוגמה: $\hat{y} = 1.4x + 1.5$. בשלב זה עדיין איננו טוענים שזהו הישר הטוב ביותר. מטרתנו להבין את פעולת החיזוי. נציב ערכי x שונים ונראה שהחישוב תמיד מתבצע באותו סדר: מכפילים את המאפיין בשיפוע, מוסיפים את החיתוך ומקבלים תחזית.

x	כפל בשיפוע	הוספת חיתוך	תחזית $\hat{y}$
1	$1.4 \times 1 = 1.4$	$1.4 + 1.5$	2.9
2	$1.4 \times 2 = 2.8$	$2.8 + 1.5$	4.3
3	$1.4 \times 3 = 4.2$	$4.2 + 1.5$	5.7
4	$1.4 \times 4 = 5.6$	$5.6 + 1.5$	7.1

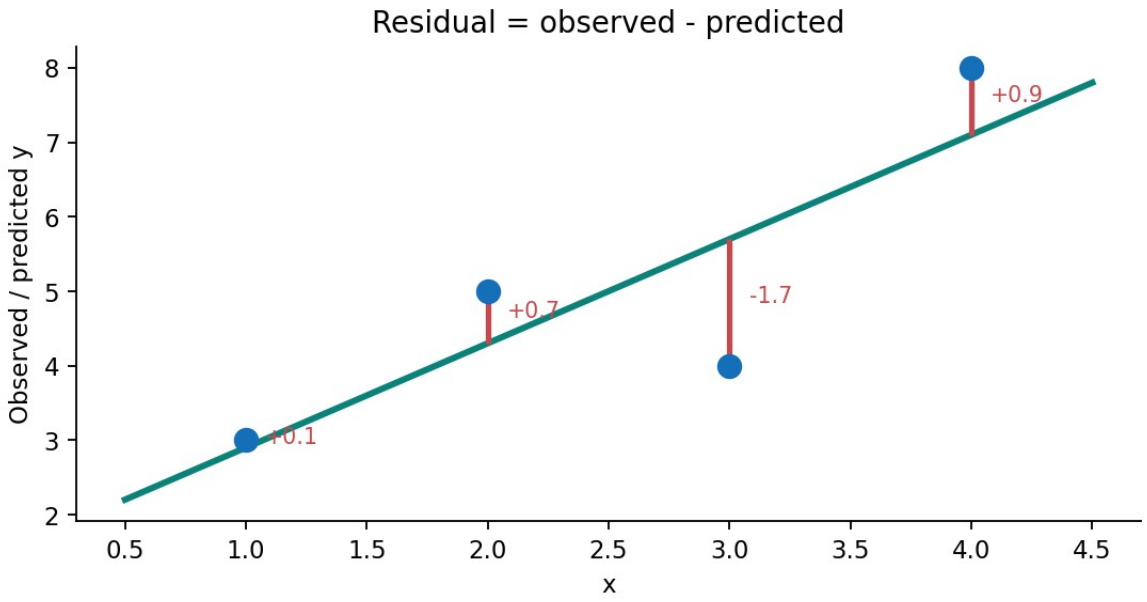
המודל מחזיר גם ערכים עשרוניים. אם אנו חוזים מחיר במאות שקלים זו התנהגות סבירה; אם אנו חוזים מספר תלמידים שאינם יכולים להתחלק לשבר, ייתכן שמודל אחר יתאים יותר. בחירת המודל אינה מסתיימת בפעולת ההצבה.

6. מהי שארית ומדוע היא חשובה?

נשתמש בארבע דוגמאות פשוטות. הערכים שנמדדו בפועל הם 3, 5, 4, 8 עבור $x=1,2,3,4$. התחזיות מהישר הקודם הן 2.9, 4.3, 5.7, 7.1. שארית (Residual) היא ההפרש בין הערך האמיתי לבין התחזית. שארית חיובית פירושה שהתחזית נמוכה מהערך האמיתי; שארית שלילית פירושה שהמודל העריך יותר מדי.

$$e_i = y_i - \hat{y}_i$$

x	ערך נמדד y	תחזית $\hat{y}$	שארית $y-\hat{y}$
1	3	2.9	0.1
2	5	4.3	0.7
3	4	5.7	−1.7
4	8	7.1	0.9



איור: הקווים האנכיים מייצגים את הפער בין הנקודות הנמדדות לקו החיזוי.

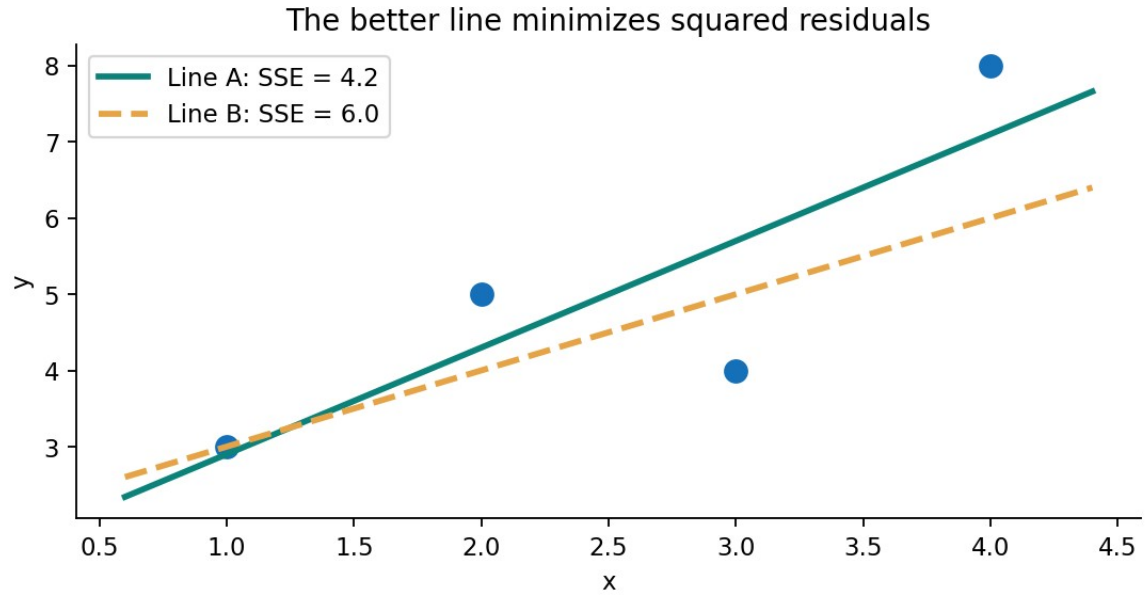
אם נחבר שאריות חיוביות ושליליות, הן עלולות לבטל זו את זו. לכן לא משתמשים בדרך כלל בסכום שאריות רגיל כמדד יחיד לטיב המודל. בהמשך נמדוד את גודל השגיאות באמצעות ערך מוחלט או באמצעות ריבוע.

7. מדוע אנו מחפשים את "הקו הטוב ביותר"?

אפשר להעביר אינסוף ישרים שונים דרך אותו אזור של נקודות. השאלה היא כיצד לבחור מתוכם באופן עקבי ולא רק לפי מה שנראה יפה בגרף. שיטת הריבועים הפחותים (Ordinary Least Squares, OLS) בוחרת שיפוע וחיתוך הממזערים את סכום ריבועי השאריות. ריבוע מבטיח ששגיאות חיוביות ושליליות יגדילו את הסכום, וששגיאות גדולות מקבלות משקל כבד יותר.

$$\mathbf{SSE = \Sigma\,(y_i - \hat{y}_i)^2}$$

לישר A מהדוגמה, סכום הריבועים הוא $0.7^2+0.1^2+(-1.7)^2+0.9^2 = 0.49+0.01+2.89+0.81 = 4.20$. נבדוק ישר חלופי $\hat{y} = x+2$:B. התחזיות הן 3,4,5,6 ולכן השאריות הן 1,2−,0,1 וסכום הריבועים $4+1+1+0=6$. בהשוואה בין שני הקווים על אותן ארבע דוגמאות, קו A מתאים טוב יותר לפי SSE.



איור: הישר בעל סכום ריבועי השאריות הקטן יותר נבחר לפי קריטריון *OLS*.

חשוב: SSE הוא מדד לשגיאה על הנתונים שנבדקו. העובדה שקו מסוים מנצח על נתוני האימון אינה מוכיחה שהוא ינבא טוב יותר נתונים חדשים.

8. כיצד מחשבים שיפוע וחיתוך מתוך הנתונים?

כעת נעבור מהצבה במודל קיים ללמידה של המודל עצמו. עבור רגרסיה ליניארית פשוטה, כאשר ערכי x אינם כולם זהים, ניתן לחשב את השיפוע שממזער את SSE ישירות מממוצעי x ו־y. אל תנסו לשנן את הנוסחה לפני שתבינו את מבנה המונה והמכנה: המונה בודק האם x ו־y משתנים יחד, והמכנה מודד כמה x משתנה סביב הממוצע שלו.

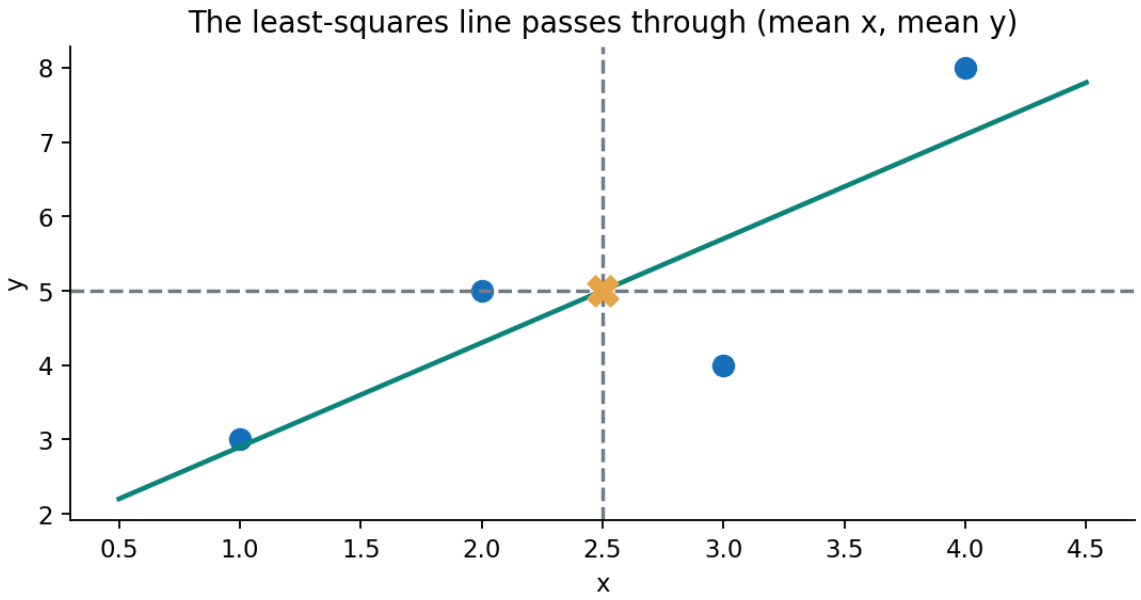
$$w = \Sigma[(x_i - \bar{x})(y_i - \bar{y})] \,/\, \Sigma[(x_i - \bar{x})^2]$$

$$b = \bar{y} - w\,\bar{x}$$

לנתונים $x=(1,2,3,4)$, $y=(3,5,4,8)$ נקבל $\bar{x}=2.5$, $\bar{y}=5$. נרכז את הסטיות בטבלה. כל ערך במונה נוצר ממכפלת שתי סטיות המתייחסות לאותה דוגמה. כל ערך במכנה הוא ריבוע הסטייה של x בלבד.

x	y	x−2.5	y−5	מכפלת סטיות	ריבוע סטיית x
1	3	−1.5	−2	3	2.25
2	5	−0.5	0	0	0.25
3	4	+0.5	−1	−0.5	0.25
4	8	+1.5	+3	4.5	2.25
סכום				7	5

$$w = 7/5 = 1.4 \qquad b = 5 - 1.4 \cdot 2.5 = 1.5$$



איור: ישר הריבועים הפחותים ברגרסיה פשוטה עם חיתוך עובר דרך נקודת הממוצעים $(\bar{x},\bar{y})$.

כעת אנו יודעים מדוע השתמשנו בישר $\hat{y}=1.4x+1.5$: הוא התקבל מחישוב המתאים לארבע הדוגמאות. אם נחליף תצפית או נוסיף נתונים, השיפוע והחיתוך עשויים להשתנות. זו משמעות ה״למידה״ ברגרסיה ליניארית: התאמת מקדמים מנתוני האימון.

בדיקת הבנה: מדוע הנוסחה אינה ניתנת לחישוב אם כל ערכי x זהים? מה מלמד אותנו המכנה במקרה זה?

9. מדדי שגיאה — לא כל מדד אומר אותו דבר

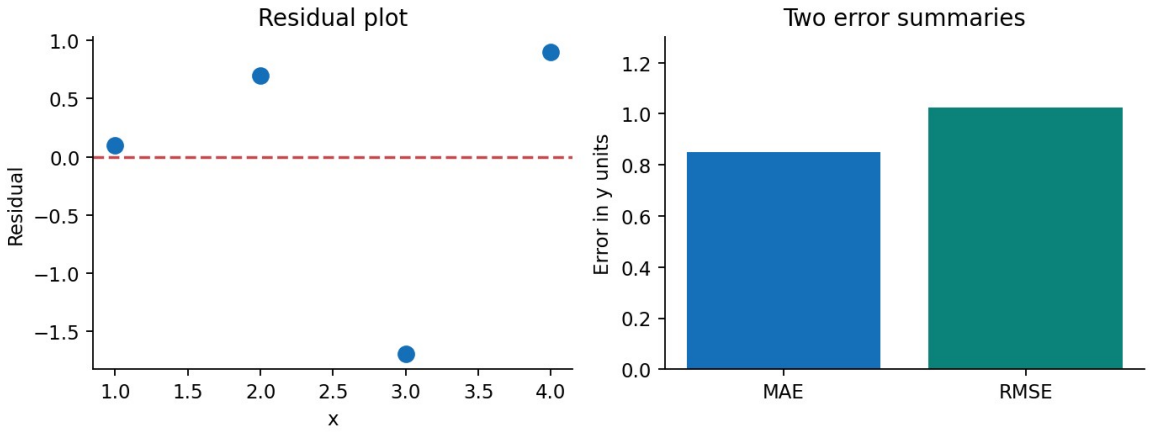
עד כה השתמשנו ב־SSE, אך לסכום שגיאות יש בעיה: ככל שמספר הדוגמאות גדל, גם הסכום נוטה לגדול. לכן משתמשים לעיתים בממוצע שגיאות. MAE מחשב את ממוצע הערכים המוחלטים של השאריות; MSE מחשב את ממוצע ריבועי השאריות; RMSE הוא שורש MSE ומחזיר אותנו ליחידות של משתנה המטרה.

$$\mathbf{MAE = (1/n)\ \Sigma\ |y_i - \hat{y}_i|}$$

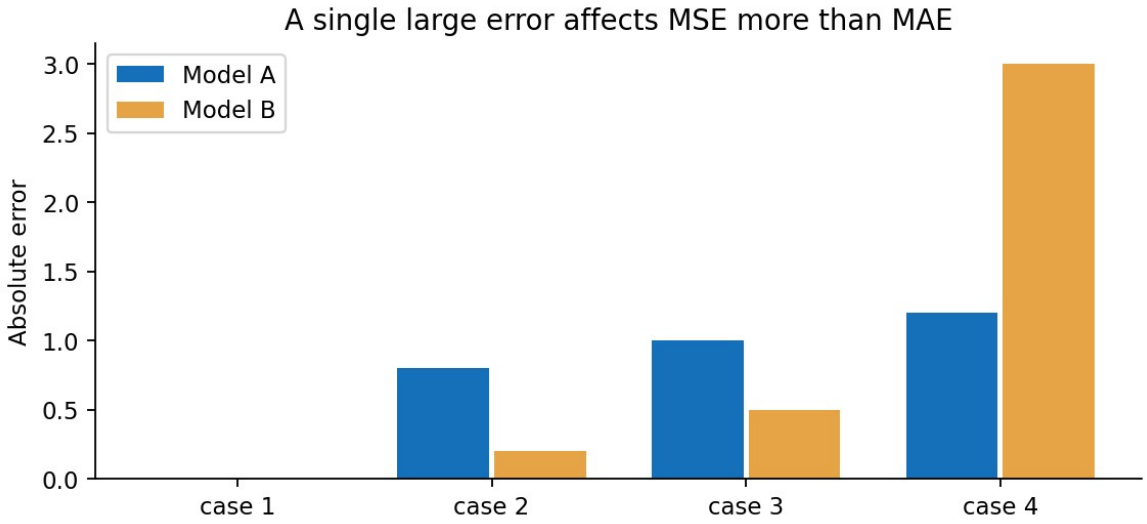
$$\mathbf{MSE = (1/n)\ \Sigma\ (y_i - \hat{y}_i)^2}$$

$$\mathbf{RMSE = \sqrt{MSE}}$$

בחישוב הידני שלנו: MAE=(0.1+0.7+1.7+0.9)/4=0.85. MSE=(0.01+0.49+2.89+0.81)/4=1.05. RMSE=1.051.0247. אם y נמדד בנקודות ציון, גם MAE ו־RMSE נמדדים בנקודות ציון, ואילו MSE נמדד בנקודות ציון בריבוע.



איור: תרשים שאריות ומדדי שגיאה: אין די בחישוב מספר אחד; חשוב להבין את יחידותיו.



איור: *MSE* מעניק משקל רב יותר לשגיאה גדולה בודדת, לעומת *MAE*.

בדיקת הבנה: אם שגיאה של 1 הוכפלה לשגיאה של 2, בכמה גדלה תרומתה ל־MAE ובכמה ל־MSE?

10. R^2 — כמה טוב יותר המודל מתחזית ממוצע?

מדד R^2 (מקדם ההסבר, Coefficient of Determination) בוחן את השיפור היחסי של המודל לעומת תחזית קבועה של הממוצע בנתונים הנבדקים. במונה נמצא סכום ריבועי השאריות של המודל; במכנה נמצא סכום ריבועי הסטיות של הערכים האמיתיים מממוצעם. זהו מדד יחסי ואינו מודד ישירות מספר שקלים או דקות של שגיאה.

$$R^2 = 1 - \Sigma(y_i - \hat{y}_i)^2 / \Sigma(y_i - \bar{y})^2$$

בדוגמה שלנו הממוצע הוא 5. כדי לחשב את המכנה, מחסרים 5 מכל אחד מארבעת הערכים האמיתיים, מעלים בריבוע ומחברים. מתקבלים ארבעה רכיבים: 4, 0, 1 ו־9. סכום 14. סכום שגיאות המודל הוא 4.2. לאחר הצבת הנתונים בנוסחה מקבלים R^2 השווה ל־0.70. פירוש זה מתייחס להתאמת המודל לנתונים הללו, לא להבטחה כי על כל תלמיד חדש יושג ציון דומה.

$$R^2 = 1 - 4.2/14 = 0.70$$

R^2 יכול להיות גם שלילי: נניח שבקבוצת בדיקה הערכים האמיתיים הם 3,5,7, אך המודל חוזה לכולם. סכום ריבועי השאריות הוא 83, וסכום ריבועי הסטיות ממוצע הבדיקה (5) הוא 8. לאחר הצבה בנוסחה נקבל R^2 שלילי שערכו מינוס 9.375. הוא אינו מוגבל לטווח 0 עד 1 כאשר בודקים תחזיות כלליות; המשמעות היא שבאותה קבוצת בדיקה המודל גרוע בהרבה מהתחזית הקבועה של ממוצע אותה קבוצה.

> חשוב: אם כל ערכי y זהים, המכנה בנוסחת R^2 הוא אפס והמדד דורש טיפול מיוחד. אל תציגו מקרה זה כמדד רגיל. הממוצע בביטוי R^2 של Test הוא אמת מידה מתמטית, ולא שימוש מותר ב־Test לשם אימון מודל.

11. מימוש ראשון ב־NumPy — לשחזר את המתמטיקה

לפני שמבקשים מספרייה לבחור את הישר, נוכל לשחזר את הנוסחה בעצמנו. חשוב להבחין בין חישוב מקדמים ובין חיזוי: בשלב הראשון משתמשים בנתוני האימון כדי למצוא w ו־b; בשלב השני מפעילים את אותו זוג מקדמים על x חדש. הנה קוד קצר שמדגים בדיוק את החישוב הידני.

```
import numpy as np
x = np.array([1., 2., 3., 4.])
y = np.array([3., 5., 4., 8.])
x_bar = x.mean()
y_bar = y.mean()
w = np.sum((x - x_bar) * (y - y_bar)) / np.sum((x - x_bar)**2)
b = y_bar - w * x_bar
predictions = w * x + b
print('w:', w, 'b:', b)
print('predictions:', predictions)
```

התוצאה הצפויה היא $w=1.4$ ו־$b=1.5$, והתחזיות 2.9, 4.3, 5.7, 7.1. זהו לא טריק של Python: כל פקודה מקבילה לשלב מסוים בטבלת החישוב הידני. אם תשנו את הנתונים ותבצעו את החישוב מחדש, תוכלו לבדוק כיצד הקו משתנה.

12. מימוש מקצועי ב־scikit-learn

בעבודה מקצועית משתמשים במחלקה LinearRegression. הפעולה fit מתאימה את מקדמי המודל לנתוני האימון; תכונות coef_ ו־intercept_ מאפשרות לקרוא את המקדמים שנלמדו; הפעולה predict מחשבת תחזיות. חשוב: גם כשיש רק מאפיין אחד, X צריך להיות מטריצה דו־ממדית עם שורה לכל דוגמה ועמודה אחת לפחות.

```
import numpy as np
from sklearn.linear_model import LinearRegression

X = np.array([[1.], [2.], [3.], [4.]])
y = np.array([3., 5., 4., 8.])
model = LinearRegression()
model.fit(X, y)
print(model.coef_, model.intercept_)
print(model.predict(np.array([[2.5], [3.5]])))
```

שימו לב לצורה של המערך החדש: [[2.5],[3.5]] פירושו שתי דוגמאות, שלכל אחת מהן מאפיין אחד. כתיבת [2.5,3.5] בלבד אינה מייצגת את אותו מבנה. אם יש שני מאפיינים לדוגמה חדשה אחת, צורת הקלט דווקא תהיה [[x1,x2]].

בדיקת הבנה: מה ההבדל בין x אחד עם שני מאפיינים לבין שתי דוגמאות שלכל אחת מאפיין אחד? תארו את ממדי X בשני המקרים.

13. איך מציגים את קו הרגרסיה בגרף?

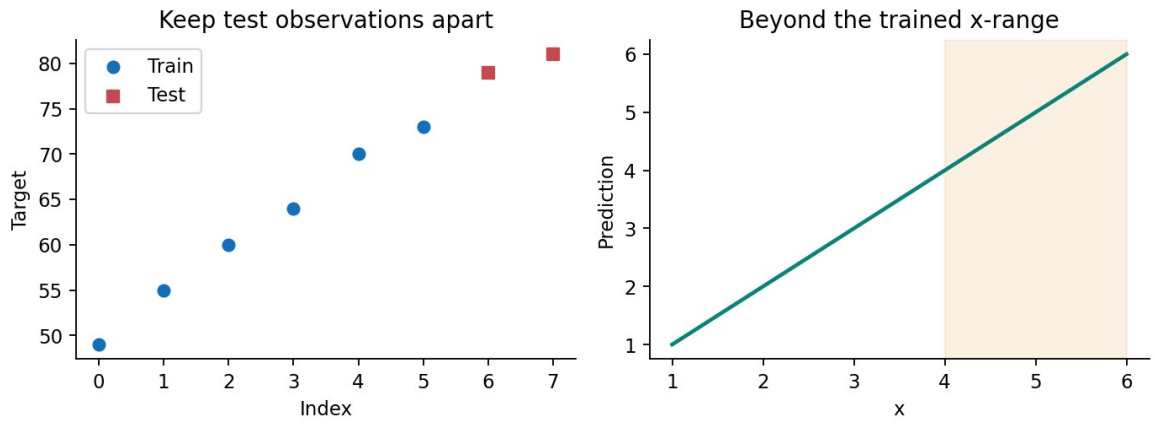
מטרתו של הגרף היא לאפשר לנו להשוות בצורה חזותית את הנקודות שנמדדו לקו התחזיות. לכן על אותו תרשים נציג את דוגמאות האימון כנקודות ואת התחזית של המודל עבור ערכי x רציפים כקו. כך ניתן לזהות נקודות שמרוחקות מהקו, מגמה שאינה ליניארית או אזור שבו אין לנו נתונים מספיקים.

```
import matplotlib.pyplot as plt
x_line = np.linspace(1, 4, 100).reshape(-1, 1)
y_line = model.predict(x_line)
plt.scatter(X[:, 0], y, label='Observed')
plt.plot(x_line[:, 0], y_line, label='Prediction')
plt.xlabel('Input feature')
plt.ylabel('Target')
plt.legend()
plt.show()
```

אל תציירו קו דרך הנקודות לפי סדר הופעתן בטבלה: כך עלול להתקבל ״זיגזג״ במקום ישר המודל. כדי לצייר קו תחזית, השתמשו ברצף מסודר של ערכי x והעבירו אותו דרך predict.

14. נתוני אימון ונתוני בדיקה — לא בודקים על מה שלמדנו בלבד

בדיקת ביצועים על נתוני האימון עשויה להסתיר בעיות. כדי לדעת כיצד המודל צפוי לעבוד על דוגמאות חדשות, נפריד מראש נתוני Train ו־Test. נלמד מקדמים מתוך Train בלבד, ונחשב מדדים על Test שלא השתתף באימון. אין להמשיך לשנות את המודל שוב ושוב לפי ציון Test ולהציג אותו אחר כך כהערכה בלתי תלויה.



איור: משמאל: הפרדה בין נתוני אימון ובדיקה. מימין: חיזוי מחוץ לטווח הנתונים הוא אקסטרפולציה.

```
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_error, r2_score

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42)
model = LinearRegression()
model.fit(X_train, y_train)
y_test_pred = model.predict(X_test)
print('Test MAE:', mean_absolute_error(y_test, y_test_pred))
```

חשוב: הדוגמה עם ארבע תצפיות בלבד ממחישה תחביר, אך היא קטנה מדי להערכת מודל אמינה. בפרויקט נעבוד עם מאגר משמעותי יותר, ונשווה לקו בסיס מתאים.

15. קו בסיס — האם צריך בכלל מודל מורכב?

לפני שמכריזים על הצלחה, כדאי להשוות למודל פשוט מאוד: תמיד לחזות את ממוצע ערכי y של נתוני האימון. זהו Baseline. אם הרגרסיה כמעט אינה משפרת את מדדי השגיאה ביחס לקו הבסיס, יש לשאול האם המאפיינים באמת מועילים, האם הנתונים מתאימים לבעיה והאם רמת הרעש גבוהה מדי. קו הבסיס אינו תחליף למודל אלא מבחן מינימלי לכך שהמודל החדש מביא ערך.

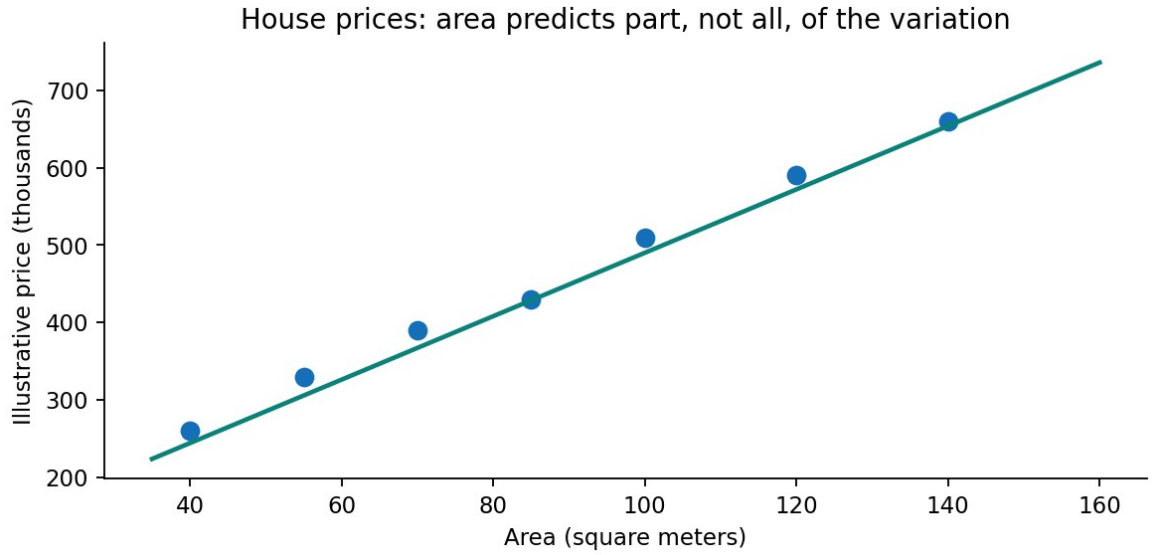
```
import numpy as np
baseline_value = np.mean(y_train)
baseline_predictions = np.full(len(y_test), baseline_value)
model_error = mean_absolute_error(y_test, y_test_pred)
base_error = mean_absolute_error(y_test, baseline_predictions)
print('Model MAE:', model_error)
print('Baseline MAE:', base_error)
```

16. רגרסיה עם יותר ממאפיין אחד

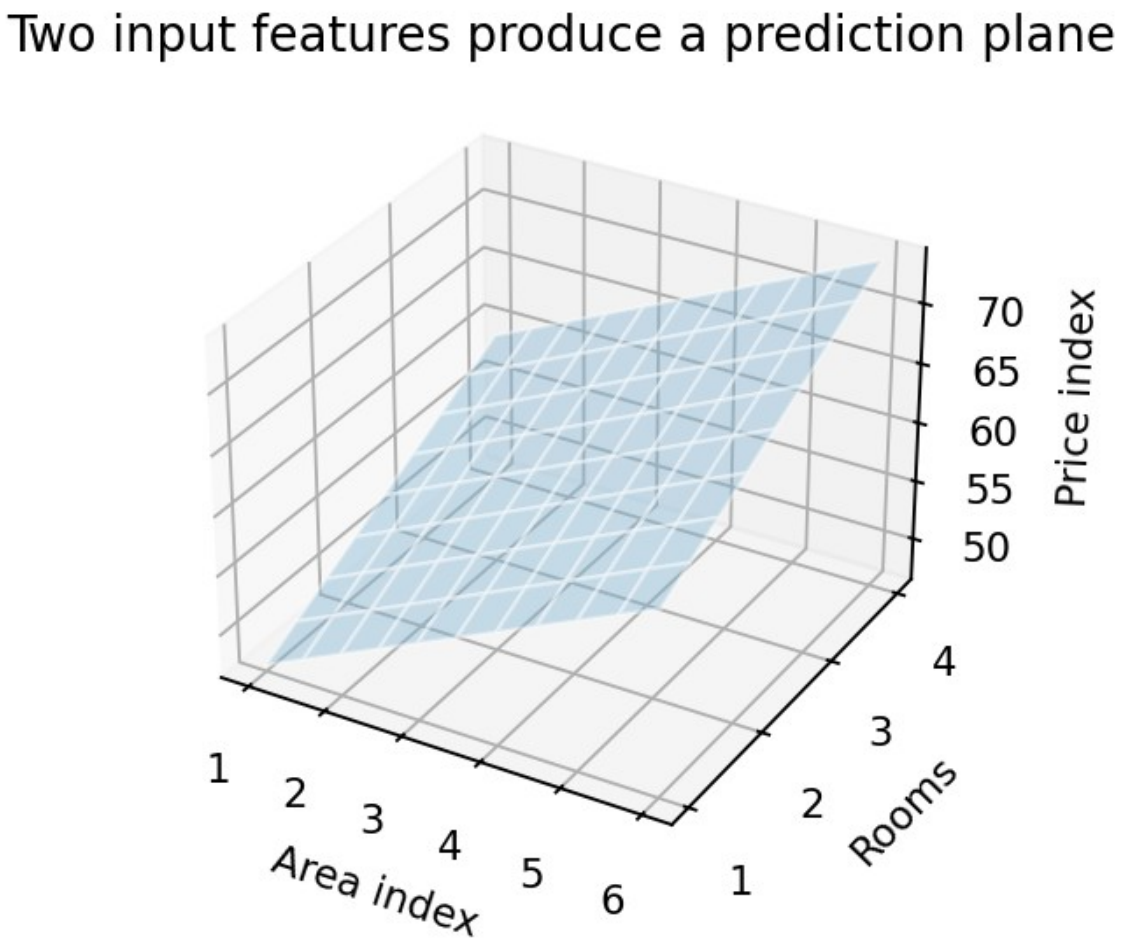
בחיזוי מחירי דירות, שטח לבדו אינו מתאר את כל ההבדלים בין דירות. נוכל להוסיף מספר חדרים, גיל בניין או מרחק ממרכז עיר. כאשר יש p מאפיינים, המודל מחשב מכפלה פנימית בין וקטור המאפיינים לווקטור המקדמים, ומוסיף b. זוהי אותה פעולה מתמטית שכבר למדנו בפרק 1.1 ושתופיע שוב במודל של נוירון.

$$\hat{y} = w_1x_1 + w_2x_2 + \ldots + w_px_p + b$$

נניח מודל המחזיר אומדן באלפי שקלים: $\hat{y}$=4×שטח+25×חדרים+80. לדירה בעלת שטח 70 מ״ר ו־3 חדרים מתקבלת תחזית 4×70+25×3+80=435 (אלפי שקלים). אלה מספרים מומצאים להמחשת פעולת המודל, ולא הערכת שוק אמיתית בישראל. השפעת המקדם 4 נמדדת באלפי שקלים למ״ר, בהינתן שמספר החדרים נשאר קבוע.



איור: קשר מומצא בין שטח לבין מחיר דירה — דוגמה להמחשה, לא נתוני שוק אמיתיים.



איור: כאשר יש שני מאפיינים, המודל מייצר מישור חיזוי במקום קו יחיד.

```
import pandas as pd
from sklearn.linear_model import LinearRegression

df = pd.DataFrame({
    'area_m2': [40, 55, 65, 80, 95, 110],
    'rooms':   [1, 2, 2, 3, 4, 4],
    'price_k': [250, 310, 360, 460, 550, 610]
})
X = df[['area_m2', 'rooms']]
y = df['price_k']
```

```
reg = LinearRegression().fit(X, y)
print(reg.coef_, reg.intercept_)
```

אם המקדמים משתנים לאחר הוספת מאפיין, אין בכך תקלה: כעת כל מקדם מתאר שינוי בתחזית כאשר יתר המאפיינים במודל מוחזקים קבועים. אין להסיק ישירות שקשר שנמצא בנתונים הוא קשר סיבתי. כמו כן, שני מאפיינים דומים מאוד עשויים להקשות על פירוש יציב של המקדמים.

17. תרגיל מבוסס Iris — ממדגם אמיתי לחיזוי רציף

במאגר Iris נמדדות ארבע תכונות של פרחים. אמנם רבים מכירים אותו מסיווג מינים, אך אפשר להשתמש באותן מדידות גם למשימת רגרסיה: לנבא אורך עלה כותרת מתוך אורך עלה הגביע. כך מתברר שאותו מאגר יכול לשמש שאלות חקר שונות, והמשתנה שאנו בוחרים לחזות קובע אם מדובר ברגרסיה או בסיווג.

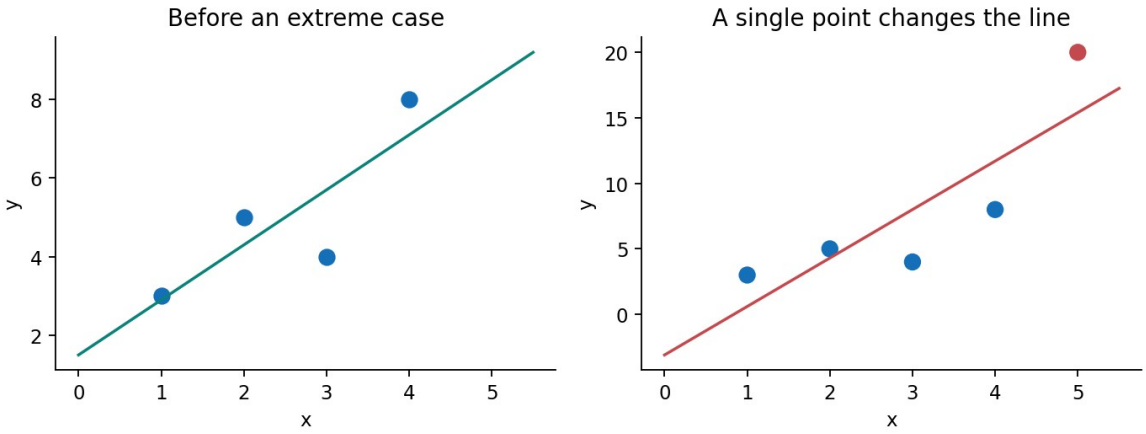
```
from sklearn.datasets import load_iris
from sklearn.linear_model import LinearRegression

iris = load_iris(as_frame=True).frame
X = iris[['sepal length (cm)']]
y = iris['petal length (cm)']
reg = LinearRegression().fit(X, y)
print('Slope:', reg.coef_[0])
print('Intercept:', reg.intercept_)
new_iris = X.iloc[[0]].copy()
new_iris.iloc[0, 0] = 6.0
print('Prediction:', reg.predict(new_iris))
```

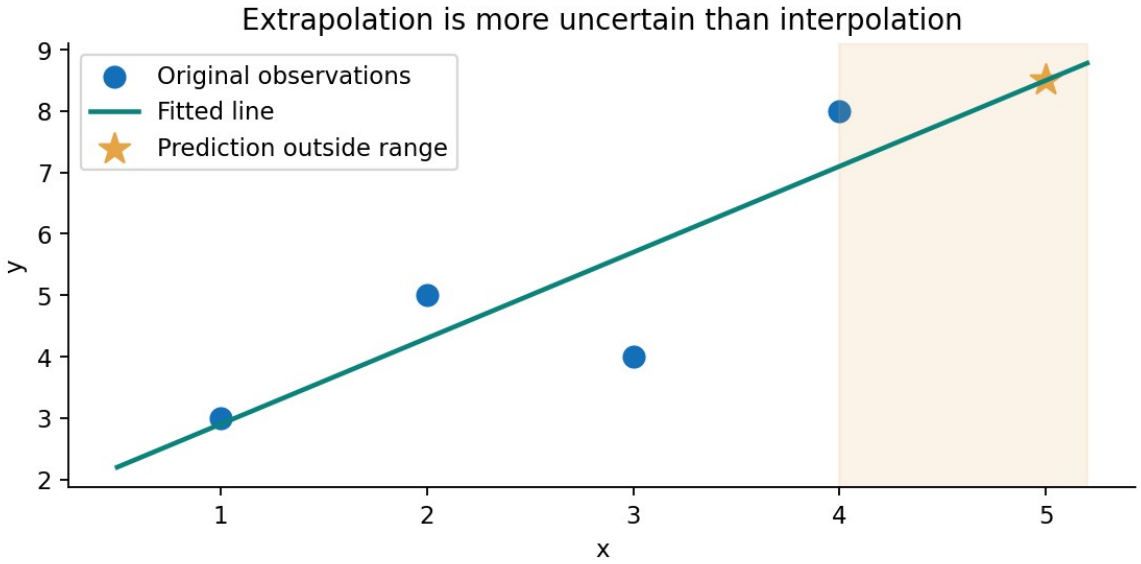
שאלת חקר: האם ייתכן שהקשר שונה בין מיני פרחים שונים? אם נבדוק את ההנחה הזו, חשוב להחליט מראש אם אנו מנתחים את המאגר כולו או תת־קבוצה מסוימת ולציין על אילו נתונים אומן המודל. מדידות פרחים אינן מודל לכל תופעה ביולוגית; זו דוגמה לימודית מוגדרת.

18. טעויות שיטתיות, חריגים והנחות

רגרסיה ליניארית היא מודל פשוט, אך אין פירושו שכל קשר אמיתי חייב להיות ישר. אם השאריות יוצרות צורת קשת בגרף, הדבר עשוי להעיד שהקשר אינו ליניארי. ערך חריג יכול למשוך את הישר לכיוונו. משתנה שנשמט מהמודל יכול ליצור הטיה, ומידע המופיע רק לאחר האירוע שאנו מנסים לחזות אינו מאפיין חוקי בזמן החיזוי.



איור: נקודת קיצון אחת יכולה להזיז משמעותית את הישר המותאם — אין למחוק חריג באופן אוטומטי.



איור: חיזוי ב־x שלא הופיע בתחום האימון הוא אקסטרפולציה; יש להתייחס אליו בזהירות.

בדקו גם אם המשתנה y מוגבל לטווח מסוים. ישר יכול להחזיר תחזית שלילית למחיר או תחזית מעל 100 לציון, אף שאין לתוצאות כאלה משמעות בבעיה. לעיתים נדרש מודל אחר, שינוי ייצוג או הגדרת טווח שימוש. התאמה סטטיסטית טובה אינה מבטלת צורך בביקורת מקצועית.

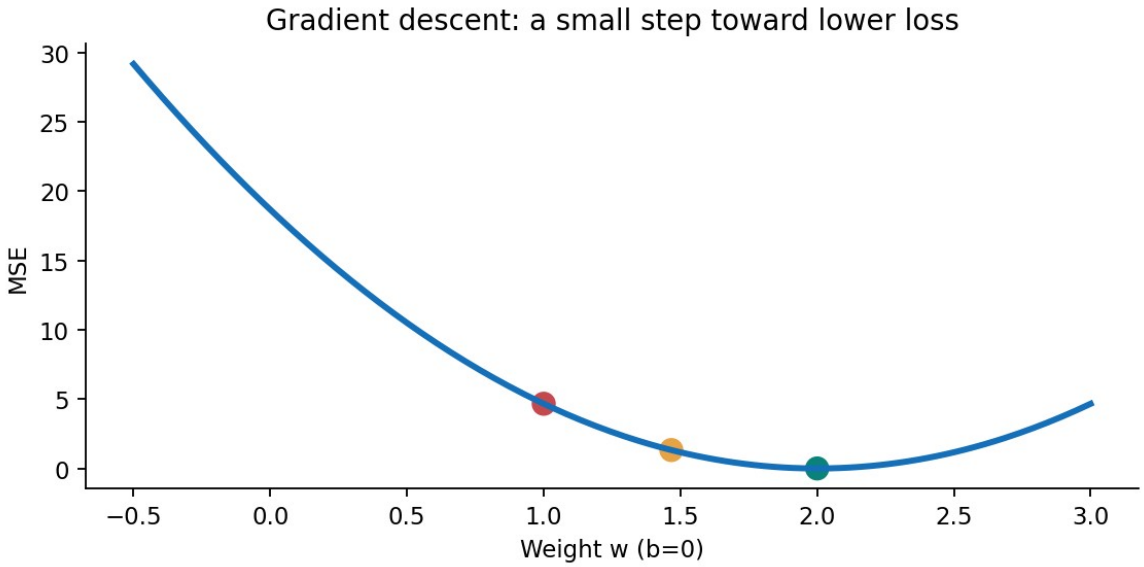
19. גשר ללמידה עמוקה — איך משקולות משתנות באימון?

עד כה מצאנו את השיפוע והחיתוך באמצעות נוסחה ישירה או ספריית LinearRegression. בהמשך הספר נלמד שיטה כללית יותר: להתחיל ממשקולות כלשהן, לחשב את השגיאה, להעריך כיצד השגיאה משתנה כשמשנים משקולות, ולעדכן אותן בכיוון שמקטין אותה. זהו הרעיון שמאחורי Gradient Descent, ומשם נמשיך לכלל השרשרת ולאימון רשתות עצביות.

$$w_{new} = w_{old} - \alpha \cdot (\partial L / \partial w)$$

להמחשה בלבד ניקח שלוש דוגמאות שבהן $x=(1,2,3)$ ו־$y=(2,4,6)$, ונגביל את המודל ל־$\hat{y}=wx$ (ללא b). אם מתחילים במשקל 1, סכום ריבועי השגיאות הוא 14, ולכן MSE הוא 14 חלקי 3. הנגזרת של MSE לפי המשקל היא מינוס 28 חלקי 3. בקצב למידה 0.05 מתקבל משקל מעודכן של כ־1.4667; הוא מקרב אותנו אל המשקל האופטימלי, שהוא 2.

$$w_{new} = 1 - 0.05 \cdot (-28/3) \approx 1.4667$$



איור: הנוסחה מתארת צעד קטן בכיוון ירידת השגיאה; זהו גשר לפרק האופטימיזציה הבא.

חשוב: גזירת הנגזרת וניתוח קצב הלמידה יילמדו בהרחבה בפרקי המתמטיקה ללמידה עמוקה.
המטרה כאן היא להבין מדוע המושג ״משקל״ ברגרסיה יחזור גם בפרספטרון וברשת עצבית.

20. טעויות נפוצות — כיצד מזהים ומתקנים אותן?

טעות שכיחה	למה היא טעות?	תיקון מומלץ
לקרוא לתחזית $\hat{y}$ ״התוצאה האמיתית״	המודל מחזיר אומדן	להבחין בכל טבלה בין y לבין $\hat{y}$
לחשוב שהישר עובר בכל הנקודות	רגרסיה מתאימה מגמה כללית	לשרטט שאריות
למסור X חד־ממדי ל־sklearn	הספרייה מצפה למטריצת מאפיינים	להשתמש ב־reshape(-1,1) או [[x]]
לדווח R^2 אימון כתוצאה סופית	אין הערכת הכללה	לשמור Test חיצוני
לפרש שיפוע כקשר סיבתי	מתאם אינו הוכחת סיבתיות	לדון במשתנים שלא נמדדו
לפרש R^2 שלילי כשגיאת קוד	ייתכן מודל גרוע מבסיס הממוצע	לחשב SSE ו־SST במפורש

21. מפת הקשרים לשלוש תכניות הלימוד

התוצר הנדרש	מה כבר ידענו / מה נוסף	מסלול
להסביר תחזית, שארית ומגבלה	גרף פיזור, X/y, מודל רגרסיה בסיסי ומדדים	יחידה 3 — כיתה י׳
להבין את התשתית המתמטית; רגרסיה היא גשר ולא פרק חובה מזוהה במפה	וקטורים, מכפלה פנימית, מדדי מרכז, מתאם והכללה	יחידה 5 — נושאי ליבה
להתאים מודל, לפרש מקדמים, להעריך ולתעד החלטות	רגרסיה עם מאפיין אחד ורבים, sklearn, ניסויים ו־R^2	למידה עמוקה — מסלול התמחות

22. תרגול מדורג — לפני המעבדה

רמה א׳ — הבנה וחישוב

1. עבור $\hat{y} = 4x + 12$ חשבו תחזיות ל־$x=2,5,8$. הסבירו את משמעות השיפוע ואת מגבלת פירוש החיתוך.

2. נתונות התוצאות $y=(10,13,14)$ והתחזיות $\hat{y}=(11,11,15)$. חשבו שארית לכל דוגמה וציינו מתי הייתה הערכת יתר.

3. הסבירו במילים מדוע MAE ומדד MSE אינם שווים בהכרח, אף ששניהם מבוססים על אותן שאריות.

4. תלמיד הזין למודל sklearn את $X=[1,2,3,4]$. מדוע עלולה להתקבל שגיאה ואיך תתקנו את צורת המערך?

רמה ב׳ — חקר ופרשנות

5. ניתנו $x=(1,2,3,4)$ ו־$y=(4,4,7,9)$. חשבו ממוצעים, שיפוע וחיתוך בשיטת OLS, ואז חשבו שתי תחזיות.

6. השוו שתי קבוצות תחזיות באמצעות MAE ו־MSE והציעו מתי חשוב במיוחד להעניש שגיאות גדולות.

7. קיבלתם $R^2=0.91$ ו־MAE=3.2 על Train, אך MAE=13 על Test. הציעו שני הסברים אפשריים ובדיקה נוספת.

8. התבוננו בגרף פיזור עם קשר מעוקל: האם רגרסיה ליניארית תהיה תמיד חסרת ערך? נמקו מתי היא קירוב סביר ומתי לא.

רמה ג׳ — שאלות מקוריות בסגנון בגרות ובהגנת פרויקט

9. נתונה טבלת ארבע דוגמאות, שני מאפיינים וערך מטרה. כתבו את ממדי X ואת אורך y, ואת צורת הקלט לחיזוי דוגמה חדשה אחת.

10. מודל א׳ מניב $R^2=0.72$ על Test ומודל ב׳ מניב $R^2=-0.4$. הסבירו מה מותר ומה אסור להסיק מההשוואה.

11. תלמיד טוען שהמודל שלו ״מדויק ב־95%״ משום ש־$R^2=0.95$. הסבירו מה שגוי בניסוח והציעו ניסוח מדעי מתאים.

12. בפיתוח מערכת מחירי דירות, תלמיד הוסיף כמאפיין את ״מחיר המכירה הסופי לאחר העסקה״ כדי לנבא את מחיר המכירה. זהו את הבעיה והציעו תיקון.

13. תכננו ניסוי המשווה מודל שטח בלבד למודל שטח+חדרים. פרטו את מדדי ההצלחה, חלוקת הנתונים והסיכון להסקת מסקנות מטעות.

14. הסבירו כיצד הנוסחה $\hat{y}=w\cdot x+b$ מופיעה הן ברגרסיה והן כשלב חישוב לפני פונקציית הפעלה של נוירון.

23. משימת יישום לקראת פרויקט הגמר

בחרו שאלה כמותית מעולם העניין שלכם: חיזוי משך טיפול בקריאת שירות, צריכת אנרגיה, זמן הגעה או מחיר מוצר. כתבו כרטיס פרויקט קצר הכולל: מהו y, אילו מאפיינים יהיו זמינים בזמן החיזוי, מקור הנתונים והזכויות להשתמש בהם, כיצד תחלקו Train/Test, מהו קו הבסיס, מהם שני המדדים שתשוו, ואילו בעיות פרטיות או הטיה עלולות לעלות. אל תתחילו מהקוד: קודם כתבו את שאלת המחקר ואת הקריטריונים להצלחה.

24. סיכום מושגים

מושג	הגדרה במשפט אחד
רגרסיה	ניבוי משתנה מטרה כמותי
שיפוע w	שינוי בתחזית ליחידת מאפיין אחת, בהינתן יתר המשתנים
חיתוך b	התחזית בנקודת המוצא של כל המאפיינים
שארית	הערך האמיתי פחות התחזית
OLS	בחירת מקדמים הממזערים סכום ריבועי שאריות
MAE	ממוצע גדלי השגיאות
MSE	ממוצע ריבועי השגיאות
RMSE	שורש MSE ביחידות משתנה המטרה
R^2	השוואת SSE לסכום סטיות מ־ממוצע ערכי הבדיקה
Baseline	תחזית פשוטה שמולה משווים את ערך המודל

25. מקורות והערות עריכה

עוגני משרד החינוך וההשתלמות: מפת התמחות למידת מכונה ולמידה עמוקה — פרק 5, 15 שעות; מצגות ההשתלמות 4.1 ו־4.2; מחברות Iris ו־KC House כפי שסוכמו במאגר GitHub של הקורס. תוכן הפרק הוא עיבוד דידקטי מקורי; אינו העתקה של מצגות או ספרים מסחריים. אין להסיק מהרחבות הנגזרות והבדיקות המתקדמות שהן דרישות חובה לבגרות ביחידה 5.

מקורות מקצועיים להעמקה: Aurélien Géron — Hands-On Machine Learning; James, Witten, Hastie, Tibshirani & Taylor — An Introduction to Statistical Learning with Applications in Python, פרק Linear Regression; Deisenroth, Faisal & Ong — Mathematics for Machine Learning; Luis Serrano — Grokking Machine Learning; התיעוד הרשמי של scikit-learn ול־LinearRegression ולמדדי רגרסיה.

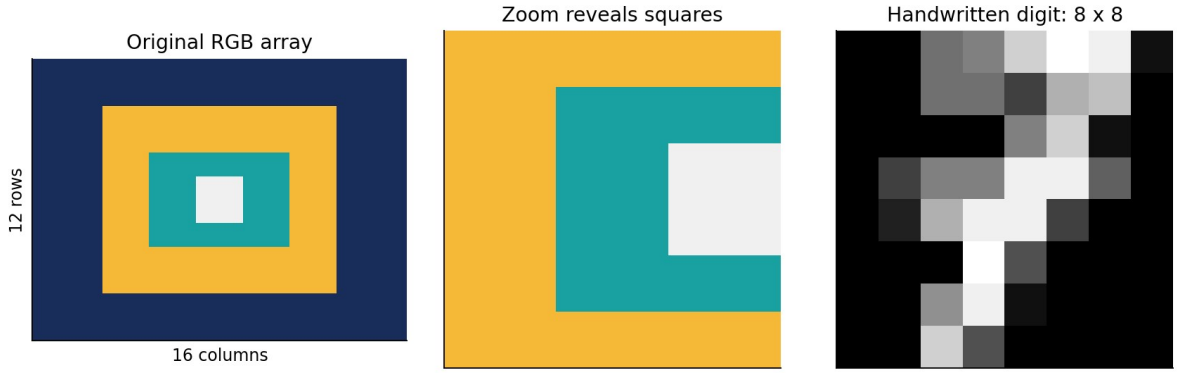
קישורים: https://www.statlearning.com/ ; https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LinearRegression.html ; https://scikit-learn.org/stable/modules/model_evaluation.html ; https://github.com/samiabo75/Data-Science-ML-Course/tree/main/03-Deep-Learning-5-Units/05-Linear-Regression

מדעי הבינה המלאכותית

התמחות למידת מכונה ולמידה עמוקה | פרק 6 בתכנית ההתמחות

עיבוד תמונה — מפיקסל בודד ועד לייצוג תמונה במודל AI

מהדורת תלמיד מורחבת · הסברים דידקטיים · נוסחאות וחישובים ידניים · 12 המחשות מקוריות · Python · תרגול עצמאי



איור: התמונה שאנחנו רואים כעצם שלם שמורה במחשב כמערך של ערכי פיקסלים. גם ספרות בכתב יד הן מערכים מספריים.

מיקום בסילבוס: פרק 6 — עיבוד תמונה, 15 שעות, בתכנית ההתמחות בלמידת מכונה ולמידה עמוקה. שיוך זה נבדק מול מפת הסילבוס השמורה במאגר הלימוד. רצף ההוראה והניסויים שלהלן הם עיבוד דידקטי מקורי; אין בידינו ברגע זה אישור שכל תת־נושא הוא דרישה פורמלית נפרדת בבחינה.

מה תדעו לעשות לאחר לימוד הפרק?

להסביר מהו פיקסל וכיצד מיוצגת תמונה כמערך; לזהות גובה, רוחב וערוצי צבע; לחשב ידנית פעולות על פיקסלים; לשנות בהירות וניגודיות בלי גלישה מטווח הערכים; לקרוא היסטוגרמת עוצמות; להבין חיתוך, שינוי גודל וסיכון לאיבוד מידע; לחשב מסנן ממוצע על חלון בגודל 3×3; לזהות את תפקיד סינון הקצוות; לכתוב קוד NumPy ו־Pillow; לבנות מעבדה משחזרת ולתאר כיצד קונבולוציה משמשת בסיס ל־CNN.

מפת דרכים לפרק

בעיה אמיתית → פיקסלים ומטריצות → צבע ותמונה אפורה → קואורדינטות ו־dtype → פעולות על תמונה → היסטוגרמה → חיתוך ושינוי גודל → רעש ומסננים → קונבולוציה → מבוא ל־CNN → הערכה, הטיות ופרויקט גמר.

1. סיפור הפתיחה — כיצד מכונה יכולה לקרוא כתב יד?

דודו בונה יישומון לתלמידים: מצלמים מספר שנכתב על דף, והיישומון מזהה את הספרה. לנו קל להבין שתמונה מציגה את הספרה 7, אך למחשב אין מושג טבעי של ״ספרה״ או ״קו״. הוא מקבל אוסף ערכים מספריים. כדי להגיע למערכת שמזהה כתב יד, עלינו להבין קודם מה נשמר בקובץ תמונה, כיצד משנים את הערכים וכיצד אפשר להפיק מהם דפוסים משמעותיים.

נניח שהמצלמה צילמה מספר בתאורה חלשה. התלמיד רואה ספרה כהה, אבל מכונה עלולה לקבל ערכים שההבדל בינם לבין הרקע קטן. לפני שנבנה מסווג, נצטרך לדעת לבדוק בהירות, להתאים גודל, להסיר רעש ולשמור על המידע החשוב. עיבוד תמונה אינו ״קישוט״: הוא עשוי לשנות את הקלט ואת איכות החיזוי של המודל.

שימו לב: שלב עיבוד התמונה יכול להתבצע בעזרת כללים קבועים שהמתכנת הגדיר, גם בלי למידת מכונה. רשת CNN תלמד בהמשך מסננים מתאימים מתוך נתוני האימון; ההבחנה הזאת היא אחת ממטרות הפרק.

בדיקת הבנה: אם נבהיר תמונה כהה, האם נוסיף מידע חדש לתמונה או נשנה את אופן הייצוג של המידע שכבר קיים?

2. מהו פיקסל ומהי רזולוציה?

פיקסל הוא יחידת מדידה במבנה של תמונה דיגיטלית. בתמונה אפורה, כל תא במטריצה מחזיק מספר המתאר עוצמת אור או בהירות. בתמונה צבעונית, לכל מיקום קיימים בדרך כלל כמה ערוצים — למשל אדום, ירוק וכחול. כאשר מציגים את הערכים יחד על מסך, העין שלנו רואה תמונה רציפה, אך במבט מוגדל אפשר להבחין בריבועים הקטנים.

רזולוציה מתארת את מספר הפיקסלים: גובה H כפול רוחב W. תמונה ברוחב 16 פיקסלים ובגובה 12 פיקסלים מכילה 192 מיקומי פיקסלים. אם זו תמונת RGB בת שלושה ערוצים, אפשר לייצג את הצבעים באמצעות 576 ערכי ערוץ. ככל שיש יותר פיקסלים אפשר לשמור פרטים מרחביים קטנים יותר, אך נדרש גם יותר זיכרון וחישוב.

$$N_{pixels} = H \times W \quad | \quad N_{RGB_values} = H \times W \times 3$$

נתון	גובה H	רוחב W	מספר פיקסלים	ערכי RGB
תמונה קטנה	12	16	192	576
תמונה רגילה	32	32	1,024	3,072
קלט גדול	224	224	50,176	150,528

הדוגמה האחרונה גם מבהירה למה קלט תמונה עשוי להיות יקר יותר מקלט טבלאי: במקום כמה מאפיינים, המודל מקבל מאות אלפי מספרים. המספרים אינם בלתי תלויים: פיקסלים סמוכים נוטים להשתייך לאותו קו, קצה או מרקם.

בדיקת הבנה: אם נכפיל את הגובה ואת הרוחב פי שניים, פי כמה יגדל מספר הפיקסלים?

3. תמונה אפורה היא מטריצה

נתחיל מתמונה בגווני אפור. נהוג להשתמש ב־uint8, סוג נתונים של מספר שלם בלתי שלילי שמיוצג בשמונה סיביות. לכן כל ערך יכול לנוע בין 0 ל־255: 0 הוא שחור, 255 הוא לבן, וערכי הביניים הם גוונים שונים של אפור. לא כל תמונה נשמרת כך: קיימות תמונות עם עומק צבע אחר, עם ערכים עשרוניים ועם קידוד אחר. אנו משתמשים ב־uint8 כדי להכיר את העיקרון בצורה ברורה.

עמודה 0	עמודה 1	עמודה 2	עמודה 3
0	0	0	0
0	80	160	0
0	160	255	0
0	0	0	0

כדי להציג את התמונה, הספרייה מתרגמת כל מספר לגוון מתאים. שימו לב שהערכים שבתוך המטריצה הם הנתונים עצמם, בעוד שצבעי התרשים הם דרך ההצגה שלהם. שינוי מפת הצבעים בתצוגה אינו בהכרח משנה את המטריצה.

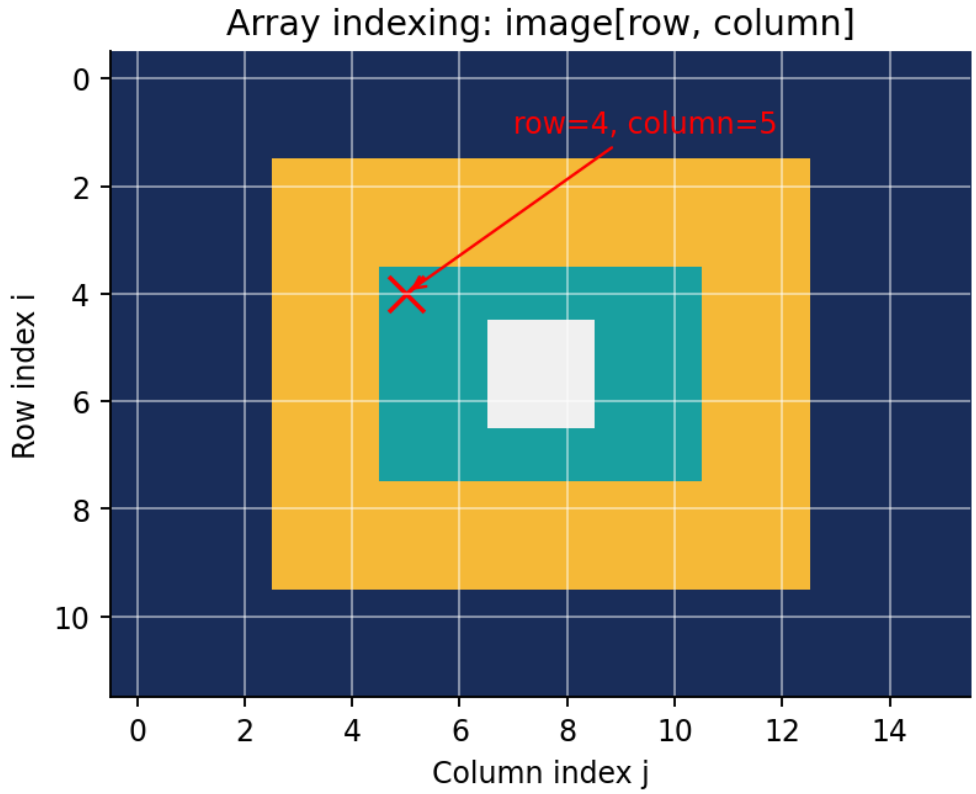
```
import numpy as np
import matplotlib.pyplot as plt
pixels = np.array([[0, 0, 0, 0],
                   [0, 80, 160, 0],
                   [0, 160, 255, 0],
```

```
                  [0, 0, 0, 0]], dtype=np.uint8)
print(pixels.shape)   # (4, 4)
plt.imshow(pixels, cmap="gray", vmin=0, vmax=255)
plt.show()
```

חשוב לזכור: מטריצה היא ייצוג של התמונה, אך אינה יודעת בעצמה מה מופיע בה. פירוש הסימנים שבתמונה הוא כבר משימה ברמה גבוהה יותר.

4. היכן נמצא פיקסל? שורות, עמודות ואינדקסים

בעבודה עם NumPy מציינים קודם שורה ואחר כך עמודה: image[row, column]. השורה הראשונה היא שורה 0; העמודה הראשונה היא עמודה 0. ציר השורות מתקדם בדרך כלל מלמעלה למטה בתמונה, וציר העמודות משמאל לימין. הדבר שונה במידת מה מהאינטואיציה של מערכות צירים במתמטיקה, שבהן ציר y נוטה לעלות כלפי מעלה.



איור: אינדקס של פיקסל: תחילה מספר שורה ואז מספר עמודה. נקודת (4,5) כאן מייצגת שורה 4 ועמודה 5.

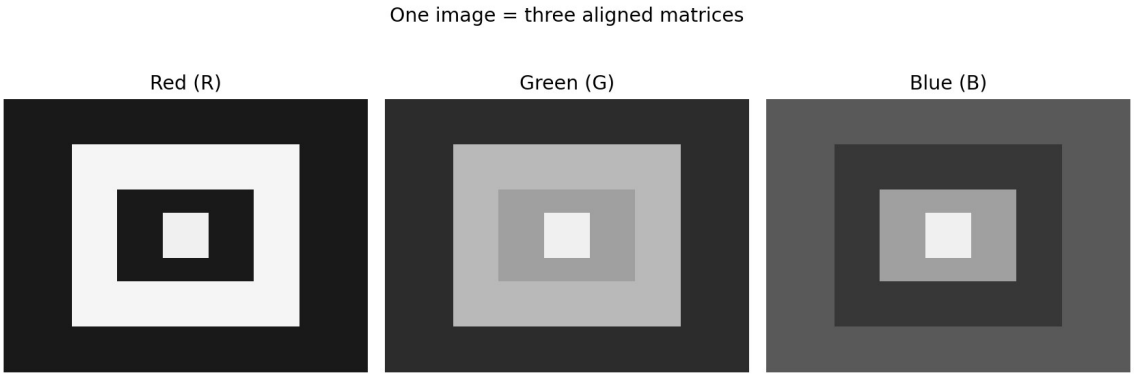
```
rgb = np.zeros((12, 16, 3), dtype=np.uint8)
rgb[:] = [25, 45, 90]
rgb[2:10, 3:13] = [245, 185, 55]
rgb[4:8, 5:11] = [25, 160, 160]
pixel = rgb[4, 5]       # three channel values
red_value = rgb[4, 5, 0]
print(rgb.shape)         # (12, 16, 3)
```

במרבית הפעולות נרצה לשנות אזור שלם ולא תא אחד. בעזרת חיתוך מערך אפשר לבחור טווח שורות וטווח עמודות. הביטוי image[2:8, 3:12] כולל שורות 2 עד 7 ועמודות 3 עד 11, מפני שהגבול העליון אינו נכלל בחיתוך.

בדיקת הבנה: בתמונה שממדיה 12×16, האם image[12,0] חוקי? ומהו אינדקס השורה האחרון?

5. תמונה צבעונית: שלושה ערוצי RGB

בייצוג RGB שומרים לכל פיקסל שלושה ערכים: עוצמת האדום (Red), הירוק (Green) והכחול (Blue). שלושת הערכים יוצרים צבע אחד על המסך. לדוגמה, RGB=(255,0,0) ייתן אדום מלא, RGB=(0,255,0) ירוק מלא, ו־RGB=(255,255,255) לבן. מערך RGB סטנדרטי ב־NumPy בנוי בגודל H×W×3.



איור: כל ערוץ צבע הוא מטריצה דו־ממדית נפרדת. שילוב שלושת הערוצים יוצר את התמונה הצבעונית.

חשוב להבחין בין גודל התמונה לבין מספר הערכים בכל פיקסל. תמונה בגודל 12×16 RGB עדיין מכילה 192 מיקומי פיקסלים, אך כל מיקום כולל שלושה מספרים. כאשר מנתחים משקל קובץ צריך גם לזכור שקובצי PNG/JPEG משתמשים בדחיסה, ולכן מספר הערכים בזיכרון אינו בהכרח שווה לגודל הקובץ על הדיסק.

צבע	R	G	B
אדום	255	0	0
ירוק	0	255	0
כחול	0	0	255
צהוב	255	255	0
אפור בינוני	128	128	128

```
from PIL import Image
color_image = Image.fromarray(rgb)
print(np.asarray(color_image).shape)
red_channel = np.asarray(color_image)[:, :, 0]
plt.imshow(red_channel, cmap="gray", vmin=0, vmax=255)
plt.show()
```

חשוב לזכור: ב־Pillow וב־Matplotlib הצבעים מטופלים לרוב כסדר RGB. פונקציות מסוימות ב־OpenCV משתמשות ב־BGR; בעת מעבר בין ספריות חובה לבדוק את סדר הערוצים.

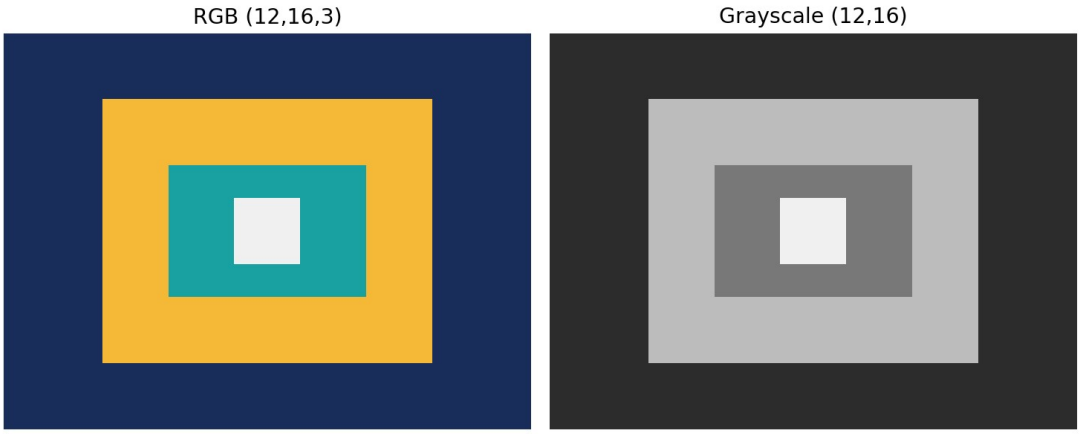
6. המרה לגווני אפור: מדוע לא עושים רק ממוצע פשוט?

לעיתים נרצה לצמצם תמונה צבעונית לערוץ אחד, למשל כדי להתמקד בקווי מתאר של ספרה. אפשר היה לחשב ממוצע רגיל של R,G,B, אך העין האנושית אינה רגישה לכל שלושת הצבעים באותה מידה. לכן מקובל לחשב קירוב לבהירות בעזרת משקולות שונות לערוצים. זו אחת משיטות ההמרה המקובלות, ואינה כל השיטות האפשריות.

$$\text{Gray} \approx 0.299R + 0.587G + 0.114B$$

נחשב ידנית פיקסל שערכיו R=200, G=100, B=50. התרומה של האדום היא 59.8, של הירוק 58.7 ושל הכחול 5.7. הסכום הוא 124.2, ולכן אם נדרוש ערך שלם נקבל בערך 124. לעומת זאת, ממוצע פשוט של שלושת הערכים הוא 116.67. ההבדל ממחיש שהמרה לגווני אפור היא כלל חישוב ולא היעלמות קסומה של הצבע.

0.299×200 + 0.587×100 + 0.114×50 = 124.2



איור: אותה תמונה ב־RGB ובגווני אפור: הממדים עוברים מ־$H \times W \times 3$ ל־$H \times W$.

```
rgb_float = rgb.astype(np.float32)
gray = 0.299 * rgb_float[:, :, 0]      + 0.587 * rgb_float[:, :, 1]      + 0.114 *
rgb_float[:, :, 2]
gray_u8 = np.clip(np.rint(gray), 0, 255).astype(np.uint8)
print(gray_u8.shape)
```

בדיקת הבנה: מדוע המרה לגווני אפור עלולה לפגוע במסווג שמבחין בין תפוחים אדומים לירוקים?

7. בהירות וניגודיות — לשנות פיקסלים בצורה מבוקרת

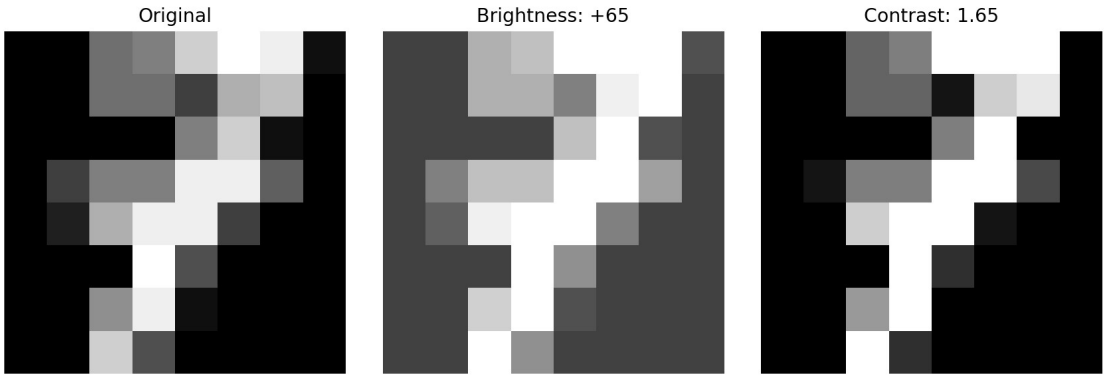
בהירות מתארת את הרמה הכללית של ערכי העוצמה; ניגודיות מתארת את ההבדל בין אזורים בהירים לכהים. אם תמונה חשוכה מדי אפשר לנסות להוסיף לכל ערך קבוע חיובי. אבל אם הפיקסל היה כבר 240 ונוסיף לו 40, התוצאה 280 חורגת מתחום 0–255. אין לשמור אותה ישירות כ־uint8, מפני שפעולות על מספרים בתחום מוגבל עלולות לגרום לגלישה או לליפוף ערכים במקום לרוויה לבנה. לכן מחשבים תחילה בסוג מספרי רחב וחותכים לטווח התקין.

brightness: I_new = clip(I_old + β, 0, 255)

פיקסל מקורי	תוספת β	תוצאה לפני clip	תוצאה סופית
60	40	100	100
220	40	260	255
0	40	40	40

ניגודיות אפשר לשנות באמצעות מתיחה של המרחק מהערך 128: ככל שמקדם α גדול מ־1, ערכים שמעל 128 מתרחקים כלפי מעלה וערכים שמתחתיו מתרחקים כלפי מטה. כאשר α בין 0 ל־1 ההבדלים נחלשים. נצטרך תמיד לחתוך את התוצאות לתחום החוקי. הגדלת ניגודיות עשויה להבליט קווים, אך עלולה גם לאבד פרטים אם אזורים רבים מגיעים ל־0 או ל־255.

contrast: I_new = clip(α(I_old − 128) + 128, 0, 255)



איור: אותה ספרה מוצגת עם בהירות וניגודיות שונים; התמונה נשמרת כמערך, אך הערכים משתנים.

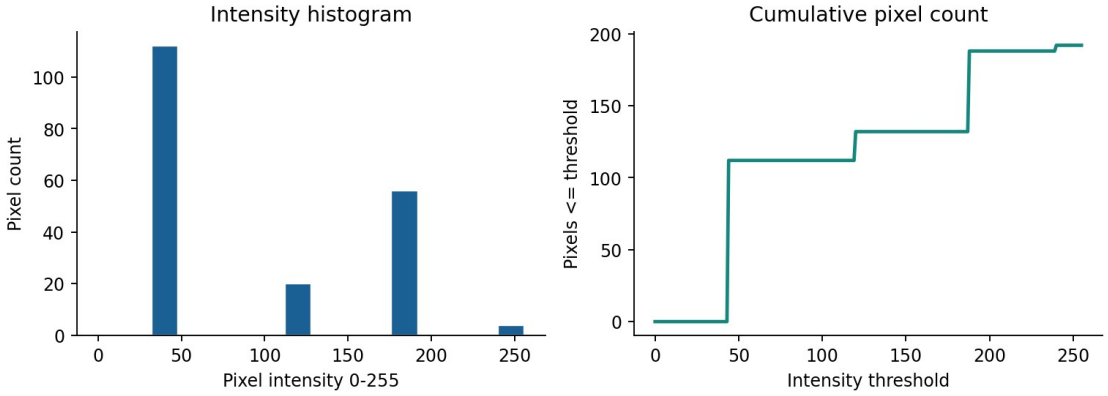
```
from sklearn.datasets import load_digits
digit8 = np.rint(load_digits().images[7] * 255 / 16).astype(np.uint8)
img_float = digit8.astype(np.float32)
brighter = np.clip(img_float + 65, 0, 255).astype(np.uint8)
stronger = np.clip(1.65 * (img_float - 128) + 128,
                   0, 255).astype(np.uint8)
```

חשוב לזכור: חשוב לשמור עותק של התמונה המקורית, כדי שאפשר יהיה לשחזר ולבדוק את השפעת כל שינוי בנפרד.

8. היסטוגרמת בהירות — לקרוא את התפלגות הפיקסלים

היסטוגרמה של תמונת אפור סופרת כמה פיקסלים נמצאים בכל טווח עוצמות. כאשר רוב הפיקסלים נמצאים בטווח נמוך, ייתכן שהתמונה כהה. כאשר הם מרוכזים בטווח גבוה, ייתכן שהיא בהירה. עם זאת, המשמעות תלויה בתוכן התמונה: תמונת לילה אמורה להכיל פיקסלים כהים רבים ואין בכך בהכרח תקלה.

נניח שלתמונה קטנה שישה פיקסלים: 0,0,50,50,50,255. ערך 0 מופיע פעמיים, 50 שלוש פעמים ו־255 פעם אחת. היסטוגרמה היא סיכום של עוצמות ולא של מיקומים. שתי תמונות שונות עשויות לקבל בדיוק אותה היסטוגרמה אם סידרנו מחדש את הפיקסלים, אף שהן מציגות צורות אחרות לחלוטין.



איור: היסטוגרמה סופרת עוצמות, אך אינה שומרת מידע על מיקום הפיקסלים.

```
pixel_values = gray_u8.ravel()
plt.hist(pixel_values, bins=np.arange(0, 257, 16),
         edgecolor="white")
plt.xlabel("Pixel intensity")
plt.ylabel("Count")
plt.show()
```

בדיקת הבנה: האם היסטוגרמה זהה של שתי תמונות מוכיחה שהתמונות זהות? הציגו דוגמה נגדית.

9. חיתוך תמונה ושינוי גודל — מדוע סדר הפעולות חשוב?

חיתוך (Crop) בוחר רק אזור מסוים מתוך התמונה. בתמונה עם מספר ועם רקע גדול מסביב, חיתוך יכול להסיר מידע שאינו מועיל ולהשאיר את הספרה במרכז. אבל חיתוך אגרסיבי עלול לחתוך גם חלק מהספרה. שינוי גודל (Resize) מתאים את ממדי התמונה לגודל הקלט הנדרש במודל, למשל 28×28. פעולה זו אינה מוסיפה מחדש פרטים שנעלמו בצילום המקורי.

שינוי הגודל מחייב כלל להערכת הערכים החדשים. באינטרפולציה הקרובה ביותר בוחרים ערך של פיקסל סמוך; בשיטה ביליניארית מחשבים שילוב משוקלל של כמה שכנים. לכן אותה תמונה יכולה להיראות משוננת או חלקה יותר לפי שיטת שינוי הגודל. חשוב לשמור על יחס רוחב־גובה, או להחליט במודע אם מוסיפים שוליים (Padding), כדי שלא לעוות את האובייקט הנלמד.



איור: אותו מערך קטן לאחר הגדלה בשתי שיטות: הפרטים החדשים הם אומדנים ולא מידע שנמדד במצלמה.

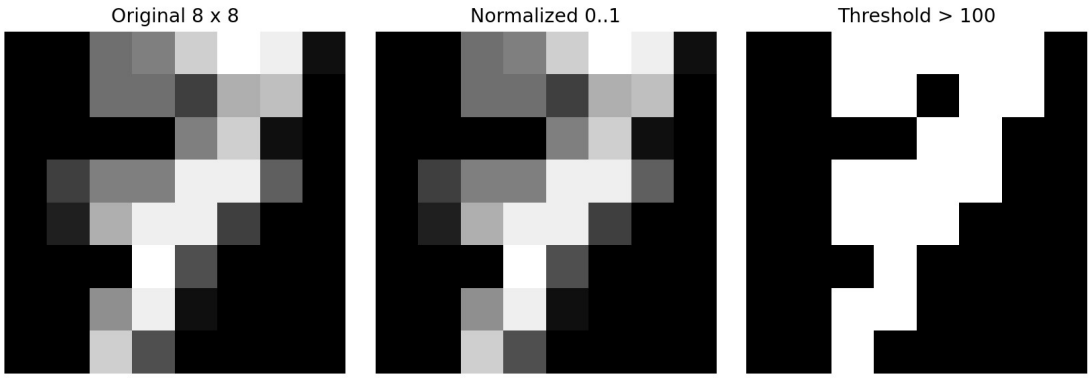
```
from PIL import Image
im = Image.fromarray(rgb)
crop = im.crop((3, 2, 13, 10))  # left, top, right, bottom
resized = im.resize((80, 60), Image.Resampling.BILINEAR)
print(np.array(crop).shape)
print(np.array(resized).shape)
```

חשוב לזכור: שימו לב להבדל: בחיתוך NumPy מסדרים [שורות, עמודות], ואילו Pillow.crop מקבל (left, top, right, bottom).

10. נרמול עוצמות — מ־0 עד 255 לטווח 0 עד 1

מודלים מסוימים נוחים יותר לאימון כאשר ערכי הקלט נמצאים בטווח מוגבל ומוכר. שינוי פשוט הוא חלוקת כל ערכי הפיקסלים ב־255. הפעולה אינה משנה את הסדר היחסי של הערכים ואף אינה משפרת אוטומטית את איכות התמונה; היא רק משנה את יחידות המדידה. פיקסל שהיה 128 יהפוך ל־0.50196 בקירוב, ופיקסל שהיה 255 יהפוך ל־1.

I_normalized = I_uint8 / 255.0



איור: הדגמה של אותם פיקסלים כערכי 0–255, כערכים עשרוניים בטווח 0–1 וכסף בינארי.

סף בינארי (Thresholding) הוא פעולה אחרת לחלוטין: למשל לקבוע שכל ערך מעל 100 יהפוך ל־1, וכל ערך אחר ל־0. הסף יכול לסייע בהפרדת ספרה מרקע, אבל הוא מוחק את רוב המידע על רמות הבהירות. לכן אין לבלבל בין נרמול לבין בינאריזציה.

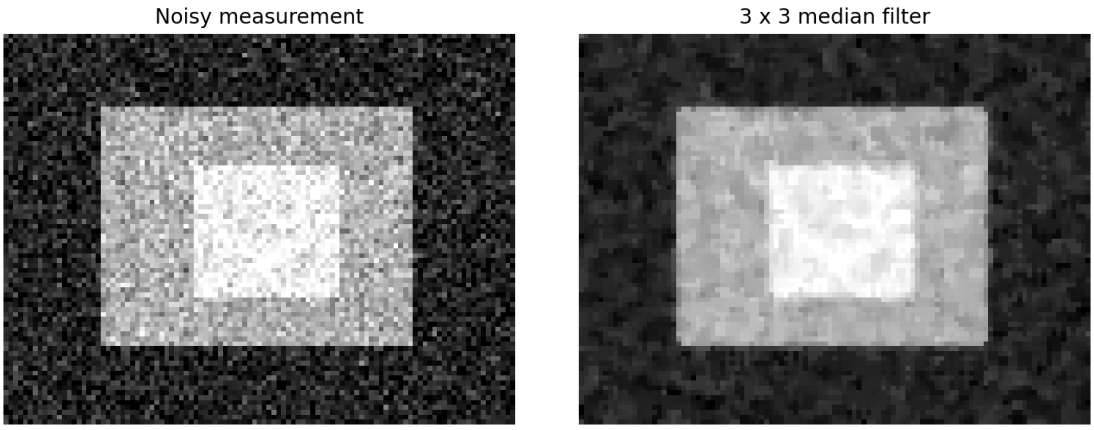
```
normalized = digit8.astype(np.float32) / 255.0
binary = (digit8 > 100).astype(np.uint8)
print(normalized.min(), normalized.max())
print(np.unique(binary))
```

חשוב לזכור: חלוקה ב־255 היא שינוי קבוע שאינו לומד פרמטר מנתוני האימון. לעומת זאת, תקנון לפי ממוצע וסטיית תקן מחייב התאמת הפרמטרים רק על Train, כדי למנוע דליפת מידע.

11. רעש — מה הוא עושה לתמונה?

בצילום אמיתי מופיעים לפעמים הפרעות: הבדלי תאורה, גרעיניות, נקודות בודדות או פיקסלים שאינם מייצגים היטב את המציאות. נוח לחשוב על רעש כהפרעה שמתווספת לאות הרצוי, אך לא כל שינוי קטן הוא רעש — לעיתים זהו פרט חשוב, כגון נקודה מעל אות או קצה של ספרה.

כדי להמחיש זאת נייצר תמונה פשוטה ונוסיף רעש אקראי. אפשר לנסות להקטין רעש באמצעות חציון מקומי: לוקחים את ערכי הסביבה, ממיינים ובוחרים את הערך האמצעי. מסנן חציון מועיל במיוחד לרעש של נקודות חריגות, אבל אינו מתאים בהכרח לכל סוג רעש. חשוב להשוות לתמונה המקורית ולוודא שלא נמחקו פרטים רלוונטיים.



איור: מסנן חציון מטפל ברעש מקומי, אך לעיתים גם מסיר פרטים קטנים.

בדיקת הבנה: אם מסנן מסיר נקודה מיותרת אך גם מסיר נקודה השייכת לאות ״ו״ — האם שיפרנו את התמונה? לפי איזו מטרה צריך להחליט?

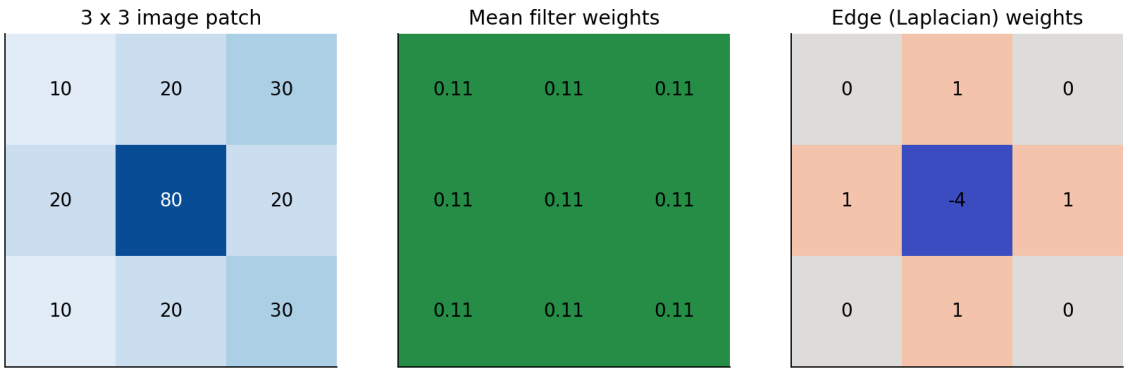
12. חלון שכנים ומסנן ממוצע — חישוב ידני

כעת נגיע לרעיון מרכזי בעיבוד תמונה: פיקסל אינו בהכרח מנותח לבדו. לעיתים נרצה לחקור את סביבתו הקרובה בעזרת חלון קטן, למשל שלוש שורות ושלוש עמודות. מסנן ממוצע מחליף את ערך הפיקסל המרכזי בממוצע של תשעת הערכים שבחלון. התוצאה נוטה להיות חלקה יותר, כי הבדלים חדים בין שכנים מצטמצמים.

10	20	30
20	80	20
10	20	30

בחלון שלפנינו סכום הערכים הוא 240. מחלקים בתשע מפני שיש בו תשעה תאים, ומקבלים 26.67 בקירוב. כלומר הפיקסל המרכזי שערכו 80 מקבל ערך חדש הקרוב ל־27. מה הרווחנו? שיטחנו שינוי מקומי חד. מה איבדנו? ייתכן שערך 80 היה סימן חשוב ולא רעש.

mean(3×3 patch) = 240 / 9 ≈ 26.67



איור: חלון 3×3, מסנן ממוצע ומשקולות מסנן להדגשת שינויים.

מסנן ממוצע הוא מקרה פשוט של פעולה משוקללת. אם לכל תא במשקל יש ערך 1/9, מכפילים כל פיקסל במשקל המתאים ומחברים. אפשר לבחור משקולות אחרות כדי להדגיש קצוות, לטשטש או לבצע פעולות נוספות.

```
patch = np.array([[10,20,30],
                  [20,80,20],
                  [10,20,30]], dtype=float)
weights = np.ones((3,3)) / 9
result = np.sum(patch * weights)
print(round(result, 2))   # 26.67
```

13. קונבולוציה ומסננים: הרעיון המתמטי

קונבולוציה היא פעולה שבה מעבירים מסנן קטן על פני מערך ומחשבים בכל מיקום סכום של מכפלות בין ערכי הסביבה למשקולות המסנן. כדי להבין את המנגנון, דמיינו חלון של 3×3 שמונח על תשעה פיקסלים. בכל צעד מכפילים תא מול תא, מחברים את תשע המכפלות ורושמים ערך בפלט; לאחר מכן מזיזים את החלון למיקום הבא.

$$S(i,j) = \Sigma_a \Sigma_b \; I(i+a, j+b) \cdot K(a,b)$$

הנוסחה שלפנינו היא למעשה פעולת Cross-Correlation, כפי שממומשת בדרך כלל בשכבות CNN. בקונבולוציה המתמטית הקלאסית יש גם היפוך של גרעין הסינון. להדגמת מסנן ממוצע סימטרי שתי הפעולות נותנות את אותה תוצאה; במסננים לא־סימטריים חשוב להבחין ביניהן. בפרק CNN נעסוק גם במשקולות שנלמדות מתוך נתונים.

מה נעשה בשוליים? חלון של 3×3 אינו יכול להיות ממורכז בפינה בלי לצאת מגבולות התמונה. אחת האפשרויות היא לחשב רק במיקומים שיש להם חלון מלא (Valid), ואז גובה ורוחב הפלט קטנים בשניים. אפשרות אחרת היא להוסיף שכבת שוליים מלאכותית (Padding), למשל אפסים, ולשמור על הגודל. הבחירה יכולה להשפיע על התוצאות ליד גבולות התמונה.

$$\text{valid 3×3 filter: } H_{out} = H - 2 \quad ; \quad W_{out} = W - 2$$

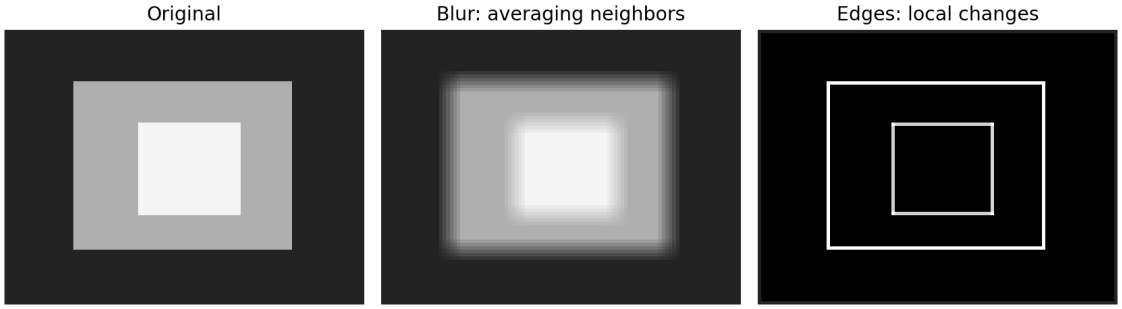
חשוב לזכור: כאשר stride = 1 וללא padding. בהמשך, עבור stride גדול יותר או padding אחר, נוסחת גודל הפלט משתנה.

14. זיהוי קצוות — מה אפשר ללמוד מהפרשים בין שכנים?

קצה בתמונה הוא אזור שבו ערכי הפיקסלים משתנים בחדות במרחב. למשל, במעבר מקיר כהה לחלון מואר נראה הבדל משמעותי בין פיקסלים סמוכים. מסננים מסוימים נועדו להבליט שינויים כאלה. קצה אינו ״שם של עצם״; הוא רק רמז מקומי שעשוי לסייע לזיהוי צורות.

נבחן מסנן סימטרי המדגיש הבדלים בין המרכז לארבעת שכניו הישירים. מקדמי השכנים הם 1, מקדם המרכז הוא מינוס 4, ובפינות מקדם 0. עבור חלון שבמרכזו 80 ובארבעת השכנים הישירים 20 נקבל 80 פחות 320, כלומר מינוס 240. לעיתים מציגים את גודל התגובה או ממירים אותה לטווח מותר לשם תצוגה. הסימן והערך המקורי עשויים להיות חשובים בחישוב.

response = 20+20+20+20 − 4×80 = −240



איור: טשטוש מפחית פרטים חדים; מסנן קצוות מדגיש שינויים מקומיים בעוצמה.

ב־CNN המסננים יכולים ללמוד להבליט דפוסים מתוך תמונות אימון, במקום שהמתכנת יבחר מראש את כל המשקולות. אולם תהליך הסכימה המקומית של ערכים כפול משקולות הוא אותו רעיון מרכזי.

15. מ־Pillow ו־NumPy אל ייצוג המתאים ללמידת מכונה

כאשר נטען תמונה ל־NumPy נקבל מערך מספרי. בשלב הבא יכול להיות שנרצה להפוך אותו לווקטור אחד ארוך, למשל 8×8 לאוסף של 64 תכונות. כך אפשר להפעיל מסווג רגיל כגון KNN. החיסרון הוא שהמיקום המרחבי פחות מפורש: פיקסלים צמודים בשורות שונות עלולים להתרחק במערך השטוח מבחינה אינדקסית, וקשרים מקומיים אינם נשמרים כשכבה נפרדת של תהליך הלמידה.

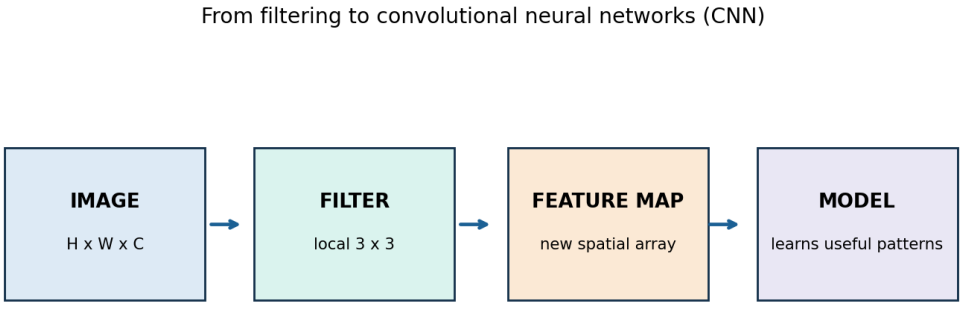
image shape (8,8) → flatten → feature vector length 64

```
from sklearn.datasets import load_digits
digits = load_digits()
X = digits.images.reshape(len(digits.images), -1)
y = digits.target
print(digits.images.shape)   # (n, 8, 8)
print(X.shape)               # (n, 64)
```

הספרייה load_digits מספקת דוגמאות אמיתיות של ספרות בכתב יד ברזולוציה של 8×8, עם ערכי עוצמה בין 0 ל־16. כדי להציגן בטווח 0–255 אפשר להכפיל ב־255/16, אבל זאת המרה לייצוג תצוגה; אין לבלבל בינה לבין העובדה שהערכים המקוריים של המאגר אינם שמונה־ביט מלאים.

בדיקת הבנה: מדוע תמונה של ספרה מתאימה גם ל־KNN וגם ל־CNN, אך לרשת CNN יש יתרון מבני בעבודה על שכונות פיקסלים?

16. גשר לפרק CNN: מסנן לומד לעומת מסנן קבוע



עד כה בחרנו ביד את משקולות המסננים: למשל כל משקל 1/9 לצורך מיצוע או מקדמים קבועים להדגשת קצוות. ברשת קונבולוציה, לעומת זאת, שכבות הקונבולוציה מכילות משקולות שנלמדות מן הנתונים בזמן האימון. בשכבות הראשונות עשויים להופיע מסננים המגיבים לקצוות או למרקמים; בשכבות עמוקות יותר יכולים להיווצר ייצוגים מורכבים. זהו תיאור רעיוני, לא הבטחה שכל רשת תלמד בדיוק אותו סדר של תכונות.

דימוי מועיל הוא ההבדל בין מסננת קבועה לבין כלי שמכוונים את המשקולות שלו לפי טעויות החיזוי. כעת אנו לומדים את הפעולה על חלון קטן; בפרק הרשתות נלמד כיצד השגיאה משנה את המשקולות בעזרת תהליך אימון.

למידה עמוקה אינה מחליפה חשיבה על איכות הנתונים. גם רשת עוצמתית עלולה להיכשל אם התמונות אינן מייצגות מצבי תאורה שונים, אם יש טעויות בתוויות, או אם נתוני Test הושפעו מן התהליך שבו בנינו את המודל.

17. הנדסת ניסוי נכון — מה בודקים לפני שמשווים מודלים?

נניח שמודל A אומן על תמונות בהירות ומודל B על תמונות כהות שנוצרו באמצעות עיבוד שונה. אם גם הנתונים, גם סוג העיבוד וגם המודל השתנו במקביל, קשה לדעת מה גרם לשינוי בביצועים. לכן ניסוי הוגן משנה גורם אחד בכל פעם, משאיר את נתוני הבדיקה מופרדים ומדווח לא רק על ציון אחד אלא גם על סוגי הטעויות.

לצורך פרויקט גמר רצוי לתעד תהליך קבוע: מקור התמונות והרשאות השימוש, גודל הקלט, סדר ערוצי הצבע, המרה לאפור או שמירת RGB, שיטת Resize, טווח נרמול והדרך שבה חולקו הדוגמאות ל־Train ול־Test. אם אותה תמונת מקור צולמה כמה פעמים בזוויות דומות, כדאי לבדוק שלא פיזרנו צילומים כמעט זהים בין אימון לבדיקה ויצרנו הערכה אופטימית מדי.

תיעוד מוצע	שאלת בדיקה	שלב
מקור, רישיון, סיכונים	האם התמונות דומות למצבים בעולם האמיתי?	נתונים
H×W×C, RGB/gray	מהם גודל התמונה וסדר הערוצים?	קלט
Crop, Resize, Normalize	האם שינינו את המידע?	עיבוד
חלוקה, מקור, מניעת כפילות	האם נשמר Test עצמאי?	בדיקה
מטריצת בלבול ודוגמאות	היכן המודל נכשל?	תוצאה

18. התנסות מודרכת מלאה — תמונה, מערך וגרף

הקוד הבא יוצר תמונה קטנה מקורית, מציג את צורת המערך ומחשב תמונה אפורה. הוא אינו דורש הורדת קובץ חיצוני, ולכן ניתן להפעילו במחברת Colab חדשה. לפני ההרצה נסו לנבא מה יהיו ממדי המערכים.

```
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image

rgb = np.zeros((12, 16, 3), dtype=np.uint8)
rgb[:] = [25, 45, 90]
rgb[2:10, 3:13] = [245, 185, 55]
rgb[4:8, 5:11] = [25, 160, 160]

r, g, b = [rgb[:, :, i].astype(float) for i in range(3)]
gray = np.clip(0.299*r + 0.587*g + 0.114*b,
               0, 255).astype(np.uint8)
```

```
print('RGB shape:', rgb.shape)
print('Gray shape:', gray.shape)
fig, ax = plt.subplots(1, 2, figsize=(7, 3))
ax[0].imshow(rgb)
ax[1].imshow(gray, cmap='gray', vmin=0, vmax=255)
for a in ax: a.axis('off')
plt.show()
```

לאחר ההרצה הסבירו איזו פעולה שינתה את מספר הערוצים, ואיזו פעולה רק הציגה אותם על המסך. את התשובה נסחו בעברית ולא רק באמצעות הדפסת הצורות.

19. משימות תרגול מדורגות

רמה א׳ — הבנת ייצוג ופעולות

1. תמונה אפורה בגובה 20 ורוחב 30: כמה פיקסלים יש בה? מהי צורת המערך? מה יהיה ההבדל אם היא תמונת RGB?

2. הציגו שלושה פיקסלים שערכיהם 0, 128, 255 והסבירו כיצד כל אחד ייראה בתמונה אפורה.

3. עבור הפיקסל (200,100,50), בצעו חישוב ידני של ההמרה לגווני אפור. הסבירו מדוע הממוצע הפשוט נותן תוצאה אחרת.

4. כתבו את טווח השורות והעמודות בביטוי image[1:4, 2:6]. כמה תאים ייבחרו?

5. קבעו אם שינוי colormap בעת plt.imshow משנה את נתוני התמונה או רק את התצוגה.

רמה ב׳ — חשיבה מתמטית וניתוח קוד

6. לפיקסלים 30, 210, 250 מוסיפים β=40. חשבו את הערכים לפני clip ואחריו והסבירו מדוע דרושה פעולת clipping.

7. קבלו חלון 3×3 שבו במרכז 90 וכל שמונת השכנים הם 10. מצאו ממוצע וחציון, והשוו מה כל מסנן יעשה לערך המרכזי.

8. תמונה בגודל 10×12 עוברת מסנן 3×3, ללא Padding ובּ־Stride של 1. מה יהיו ממדי הפלט? נמקו.

9. השוו בין Resize בשיטת Nearest Neighbor לבין Bilinear. איזה מידע לא ניתן לשחזר מתוך תמונה קטנה במיוחד?

10. התלמיד טוען ש־Normalization ו־Thresholding הן אותה פעולה. כתבו דוגמה מספרית הסותרת את הטענה.

רמה ג׳ — העברה לבגרות ולפרויקט גמר

11. מערכת לזיהוי ספרות מצליחה באימון ונכשלת בצילומים בתאורה חלשה. הציעו ניסוי מבוקר שיבדוק את השפעת עיבוד הבהירות בלי לשנות את נתוני Test לצורך בחירת הפרמטרים.

12. שתי קבוצות תמונות הן בעלות היסטוגרמת בהירות זהה, אך שונות במיקום קווי המתאר. הסבירו מדוע לא כדאי לבסס סיווג ספרות על היסטוגרמה בלבד.

13. הציעו מדוע רשת CNN שמקבלת תמונה דו־ממדית עשויה להיות עדיפה על מסווג רגיל המקבל וקטור שטוח, גם כשמספר ערכי הקלט זהה.

14. תארו שלושה פרטים שמוכרחים להופיע בתיעוד של שלב עיבוד התמונה בפרויקט גמר, ונמקו את חשיבותם.

15. תכננו בדיקת שגיאות למסווג תמונות: אילו מקרים תציגו בפני הבוחן כדי להוכיח שאתם מבינים גם את מגבלות המערכת?

20. שאלת הערכה מקורית בסגנון בגרות

מערכת למיון מוצרים מצלמת כרטיס שעליו מופיע מספר סידורי. כל כרטיס נחתך לתמונה אפורה בגודל 6×6 פיקסלים, ומסווג באמצעות מודל שנלמד מנתוני אימון מסומנים. נתונה המטריצה הבאה המתארת חלון מקומי:

10	20	10
20	100	20
10	20	10

א. הסבירו מהו ערך הפיקסל המרכזי ומהו טווח הערכים האפשרי בתמונה מטיפוס uint8. ב. חשבו את ערך המסנן הממוצע על החלון והראו את שלבי הסכימה. ג. הסבירו כיצד סינון עשוי גם לפגוע בזיהוי הספרה. ד. אם נבצע חיתוך תמונה ו־Resize, באיזה סדר הייתם פועלים ומדוע? ה. הסבירו מדוע צריך לשמור קבוצת Test נפרדת בעת בחירת שיטת עיבוד תמונה.

דרישת ההערכה: נימוק מלא במילים, חישוב מנומק והבחנה בין פעולת עיבוד קבועה לבין מודל שלומד מנתונים. זו שאלה מקורית שנכתבה לצורכי ההוראה, ולא שאלה מקורית ממאגר בחינות משרד החינוך.

21. מעבדת Google Colab ומדדי הצלחה

המעבדה המצורפת כוללת 12 משימות המתקדמות מזיהוי ממדים וערכי פיקסלים אל יצירת מסננים וניתוח ניסוי. בתאי התרגול מופיעים מקומות להשלמת קוד, ולא פתרונות מלאים. לפני כל תא יש לכתוב תחזית לגבי התוצאה; אחרי ההרצה יש להסביר אותה. מעבדה תיחשב מוצלחת רק אם התלמיד מסוגל להסביר את צורות המערכים, את שינויי הערכים ואת הסיכון לאיבוד מידע.

מה התלמיד מציג	יכולת
פירוש צורה (H,W) או (H,W,3)	ייצוג
המרה לאפור ומסנן ממוצע באופן ידני	חישוב
פעולות NumPy/Pillow מוסברות	קוד
השוואת שיטות עיבוד תוך שמירה על אותם נתונים	חקירה
הסבר מגבלה והשפעתה על AI	הגנה

22. הקשר בשלוש תכניות הלימוד

תכנית	תפקיד התוכן	הסטטוס
יחידה 3 — מדעי הנתונים	חזרה על מערכים, מאפיינים, KNN והמחשה בסיסית	ידע קודם / גשר פדגוגי
יחידה 5 — נושאי ליבה	וקטורים, מטריצות, מרחקים, תקנון, חקר נתונים	קישור מושגי; לא פרק תמונה נפרד
התמחות למידה עמוקה	עיבוד תמונה שיטתי, הכנת קלט וגשר ל־CNN	פרק 6: 15 שעות לפי מפת התכנית

הספר נשען על מפת ההתמחות השמורה במאגר GitHub שלנו. הגדרות, דוגמאות וניסויים נוספו כדי להנגיש לתלמידים רעיונות המופיעים בספרי יסוד כגון Hands-On Machine Learning, Mathematics for Machine Learning, Dive into Deep Learning ו־Grokking Deep Learning. כל הדוגמאות והאיורים בפרק נוצרו עבור החוברת; לא הועתקו מהם פרקים או איורים מוגנים.

23. הצעת חלוקת 15 שעות הוראה

תוצר	נושאים	שעות
קריאת מערכים ותיאור תמונה	מבוא, פיקסלים, ממדים, RGB ואפור	1–3
חישוב ידני ותרשים	אינדקסים, בהירות, ניגודיות והיסטוגרמה	4–6
השוואת עיבודים מודרכת	רעש, Crop, Resize, Normalize	7–9
מימוש מסנן וניתוח פלט	מסננים, קונבולוציה וזיהוי קצוות	10–12
הסבר פרויקט ומגבלות	הכנה ל־CNN, מעבדה, משימת הערכה	13–15

חלוקת השעות היא הצעה פדגוגית ל־15 שעות הסילבוס, ולא טבלה רשמית של משרד החינוך. ניתן להאריך או לקצר פעילויות בהתאם לרמת הכיתה, תוך שמירת תוצרי החובה שהמורה קובע על פי המסמך התקף.

מקורות והמשך קריאה

תכנית ההתמחות במאגר הקורס:
https://github.com/samiabo75/Data-Science-ML-Course/tree/main/03-Deep-Learning-5-Units/06-Image-Processing

Aurélien Géron, Hands-On Machine Learning with Scikit-Learn and PyTorch — פרקי ייצוג והכנת נתונים ומודלי ראייה.

Zhang, Lipton, Li & Smola, Dive into Deep Learning — פרקי image processing ו־Convolutional Neural Networks; https://d2l.ai

Deisenroth, Faisal & Ong, Mathematics for Machine Learning — אלגברה ליניארית ופעולות מערכיות; https://mml-book.github.io

Pillow documentation: https://pillow.readthedocs.io

/NumPy documentation: https://numpy.org/doc/stable

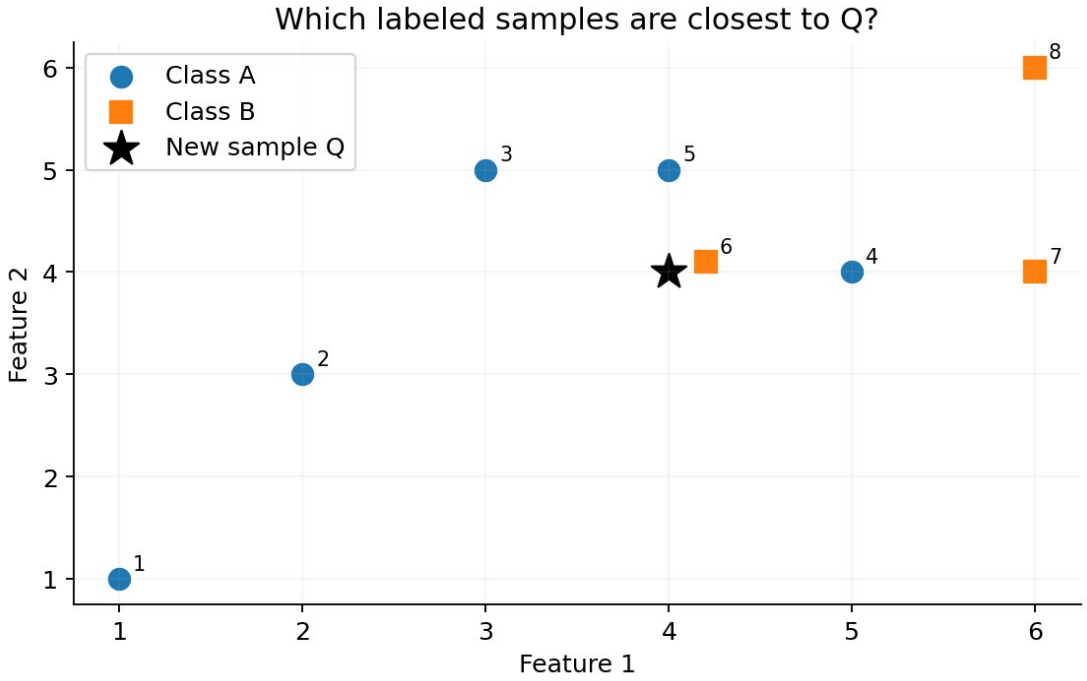
scikit-learn load_digits:
https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_digits.html

הערת מיפוי: הפרק מבוסס על כותרת פרק 6 והיקפו במאגר הסילבוס, אך ללא בדיקה חוזרת של כל שורות ה־PDF המקורי. ההרחבות בנושא קונבולוציה, הטיה וניהול ניסוי הוצגו כהכנה מקצועית ל־CNN ולפרויקט ולא כטענה אוטומטית לחובת בחינה.

למידה עמוקה | פרק 7 בסילבוס ההתמחות | 30 שעות מסגרת

פרק 7 — סיווג וKNN מתקדם

מהפעלת ספרייה למימוש אלגוריתם, חשיבה וקטורית ובדיקת הכללה



איור 1. אותו רעיון מוכר של שכנים קרובים — הפעם לומדים כיצד מממשים אותו בעצמנו.

פרק זה הוא המשך מעמיק לפרקים 4.1–4.4 של נושאי הליבה. במקום לשכפל את ההגדרות, נפרק כאן את השלבים שמבצע המסווג ונבנה אותם בהדרגה בקוד. תחילה נדע להסביר כל שורה, אחר כך נחשב ב־NumPy כמה מרחקים במקביל, ולבסוף נשווה את המימוש שלנו לכלי מקצועי. ההצלחה בפרק אינה נמדדת רק בקבלת תחזית נכונה, אלא ביכולת לזהות שגיאות, לבדוק נכונות ולהצדיק החלטות של מפתח AI.

עוגן: תכנית ההתמחות בלמידת מכונה ולמידה עמוקה, פרק 7 — בעיית הסיווג ו־KNN (30 שעות); חומרי השתלמות 5.1–5.3, כולל מימוש עצמאי ב־NumPy ותרגול Iris. חלוקת התרגול וההעמקה כאן היא הצעה דידקטית מקורית, לא נוסח רשמי של המחוון.

מפת הלמידה — מה נדע לעשות?

- להסביר כיצד אלגוריתם KNN מקבל את מטריצת המאפיינים X, את וקטור התוויות y ואת נקודת השאילתה q.
- לממש חישוב מרחקים, מיון אינדקסים, שמירת תוויות והצבעת רוב בעזרת לולאות, ואז בעזרת NumPy.
- להסביר Broadcasting, את המושג shape ואת משמעות axis בחישוב מרחקים לשאילתה אחת ולמספר שאילתות.
- לקבוע מראש מדיניות שוויון, להבחין בין תווית חזויה לציון הצבעה, ולהשוות הצבעה רגילה להצבעה משוקללת.

- לבדוק התאמת ממדים, תקנון ללא זליגת נתונים, חלוקת Train/Test, בחירת k באמצעות Cross-Validation והשוואה הוגנת.
- לנתח מורכבות חישובית, מגבלות ממדיות ולנסח החלטות מקצועיות בפרויקט גמר.

רצף מוצע ל־30 שעות ההתמחות

תוצר	מיקוד	שעות
טבלת מרחקים + הנמקת סיווג	ריענון סיווג וחישוב ידני	1–5
מימוש שקוף עם לולאות	פיתוח פונקציות Python	6–10
מימוש וקטורי ותיעוד shape	NumPy, Broadcasting וצורות	11–15
בדיקות נכונות והשוואה לספרייה	שוויון, משקולות ובדיקות	16–20
ניסוי Iris מתועד	Pipeline, CV ומדדים	21–25
דוח חקר קצר והגנה בעל־פה	ניתוח עלויות, העברה ותרגול	26–30

1. סיפור פתיחה — מדוע אי אפשר להסתפק ב־fit ו־predict?

דודו בונה מערכת המסייעת לסווג תקלות במעבדת מחשבים. לכל מחשב נשמרים שני מאפיינים מספריים: עומס במעבד ומדד התרעות. לכל מחשב שכבר נבדק קיימת גם תווית: A — תקלה קלה, B — דורש בדיקה נוספת. הדוגמה מומצאת להוראה, ואין להשתמש בה לקבלת החלטות תחזוקה אמיתיות ללא בדיקה מקצועית.

דודו הפעיל KNeighborsClassifier וקיבל תשובה — אבל המורה שאל: כיצד נדע שהמודל בחר את השכנים הנכונים? מה יקרה אם שני מחשבים יהיו באותו מרחק בדיוק? כיצד נבדוק שהנוסחה שהמרנו לקוד מחזירה את אותו המרחק שחישבנו על הנייר? אלה הן שאלות פיתוח תוכנה ומדע נתונים, ולא רק שאלות על פקודות.

$$q = (4,\ 4)$$

לצורך לימוד נשתמש בשמונה דוגמאות אימון ובשאילתה אחת q. כל שורה במטריצה היא מחשב; כל עמודה היא תכונה. התוויות נשמרות בנפרד, משום שאסור למודל להשתמש בתשובה הרצויה בתור מאפיין של הדוגמה החדשה.

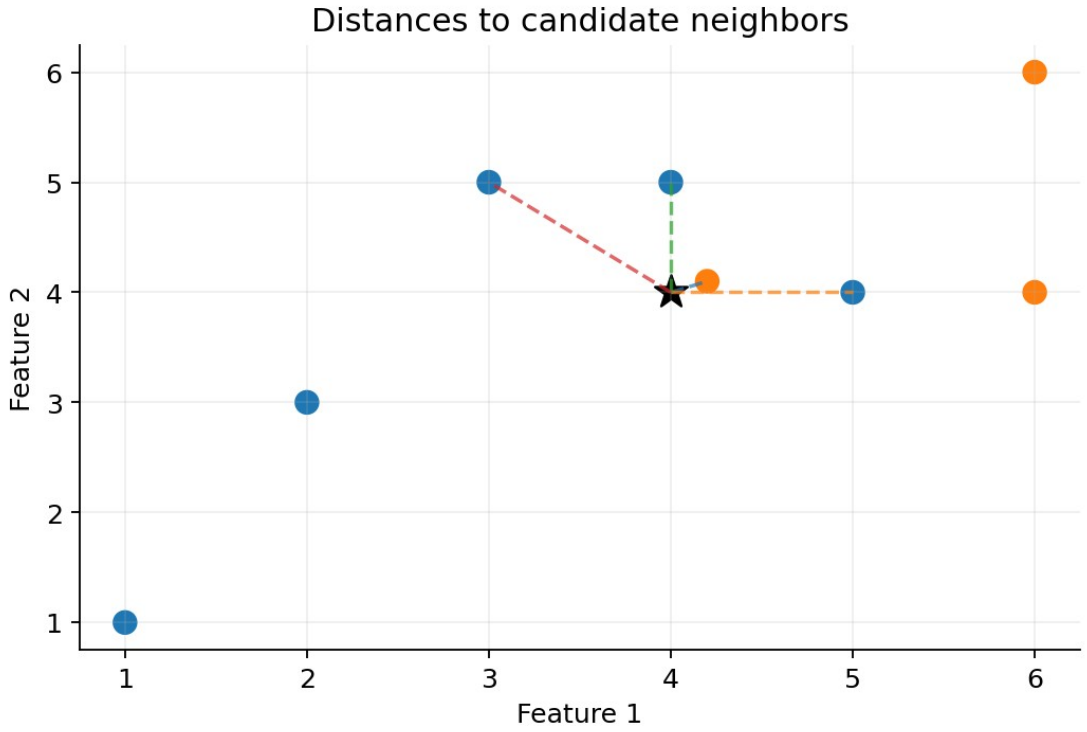
מספר	x_1	x_2	מחלקה
1	1	1	A
2	2	3	A
3	3	5	A
4	5	4	A
5	4	5	A
6	4.2	4.1	B
7	6	4	B
8	6	6	B

2. חזרה קצרה — האלגוריתם בחמש פעולות

תיאור האלגוריתם במילים חשוב לא פחות מקוד. התלמיד צריך להיות מסוגל לספר כיצד הופכת שורה חדשה לתווית חזויה: מודדים את המרחק לכל דוגמת אימון, מסדרים את הדוגמאות לפי מרחק, בוחרים k שכנים, סופרים את התוויות ומחזירים את המחלקה שנבחרה לפי כלל ההכרעה.

$$d(x, q) = \sqrt{\left[(x_1 - q_1)^2 + (x_2 - q_2)^2 \right]}$$

זהו מרחק אוקלידי. כבר למדנו אותו בפרק 1.2; כאן נשתמש בו שוב כדי לתכנן מבנה נתונים נכון. הדבר החשוב הוא שמרחק ותווית שייכים לאותה שורה: כאשר ממיינים מרחקים, אין למיין את התוויות בנפרד.



איור 2. הקווים המקווקווים מדגישים מרחקים של שכנים אפשריים אל השאילתה.

3. חישוב ידני מלא של סיווג אחד

נחשב קודם כמה מרחקים ללא מחשב. לתצפית מספר 6, הווקטור הוא (4.2,4.1). מרחקו מן השאילתה $q=(4,4)$ הוא השורש של סכום ריבועי ההפרשים. לתצפית מספר 4, הווקטור (5,4), ולכן המרחק שווה בדיוק ל־1.

$$d_6 = \sqrt{[(4.2 - 4)^2 + (4.1 - 4)^2]} = \sqrt{0.05} \approx 0.2236$$

$$d_4 = \sqrt{[(5 - 4)^2 + (4 - 4)^2]} = 1$$

$$d_5 = \sqrt{[(4 - 4)^2 + (5 - 4)^2]} = 1$$

לאחר חישוב כל המרחקים נרשום אותם לצד מספר התצפית והתווית. טבלה זו היא כלי בקרה חשוב: היא מאפשרת לאתר טעויות בקוד כגון שימוש בעמודה הלא נכונה או השמטת השורש.

סדר קרבה	תצפית	מרחק משוער	תווית
1	6	0.2236	B
2	4	1.0000	A

סדר קרבה	תצפית	מרחק משוער	תווית
3	5	1.0000	A
4	3	1.4142	A
5	7	2.0000	B
6	2	2.2361	A
7	8	2.8284	B
8	1	4.2426	A

התחזית כאשר k=1 היא B, מפני שהשכן הקרוב ביותר הוא תצפית 6. כאשר k=3, שלוש התוויות הראשונות הן B, A, A ולכן תחזית הרוב היא A. זוהי דוגמה לכך ששינוי k יכול להפוך החלטה גם כאשר כל הנתונים נשארו קבועים.



איור 3. סדר המרחקים לאחר המיון. יש לשמור את אינדקס השורה יחד עם המרחק שלה.

4. ממימוש רעיוני למימוש בפייתון עם לולאות

כדי להבין כיצד המחשב עובד, נתחיל בקוד שקוף ולא בפקודה אחת מוכנה. נשמור מערך של זוגות: המרחק ולידו אינדקס השורה. לאחר המיון נוכל למצוא את התווית מתוך y באותו אינדקס. זהו עיקרון תכנותי חשוב: אינדקס מחבר בין שתי טבלאות שמתארות את אותה תצפית.

```
import numpy as np
X = np.array([[1,1], [2,3], [3,5], [5,4],
              [4,5], [4.2,4.1], [6,4], [6,6]], dtype=float)
y = np.array(["A", "A", "A", "A", "A", "B", "B", "B"])
q = np.array([4.0, 4.0])
rows = []
for i in range(len(X)):
    total = 0.0
    for j in range(X.shape[1]):
        total += (X[i, j] - q[j]) ** 2
    rows.append((np.sqrt(total), i))
rows.sort(key=lambda pair: (pair[0], pair[1]))
print(rows[:3])
```

שאלו את עצמכם: מדוע הלולאה החיצונית עוברת על שורות והפנימית על מאפיינים? מדוע שומרים את i ולא רק את המרחק? מדוע במיון בחרנו צמד (מרחק, אינדקס)? הבחירה באינדקס כקריטריון משני הופכת את סדר השורות לשחזורי במקרה של מרחקים שווים, אבל היא עדיין אינה פתרון מלא לשוויון בהצבעה בין מחלקות.

5. פיתוח פונקציה עם חוזה ברור

לפני מימוש פונקציה נגדיר מה היא מקבלת ומה היא מחזירה. פונקציה לחיזוי אחת מקבלת X בגודל $n \times d$, תוויות y באורך n, נקודת שאילתה באורך d ו־k שלם בין 1 ל־n. תוצאתה היא תווית חזויה. פונקציה מקצועית גם צריכה לזהות קלט לא תקין, אבל לצורך ההיכרות נתחיל במקרה התקין ונסח בדיקות מתאימות בהמשך.

`X.shape = (n, d)` `y.shape = (n,)` `q.shape = (d,)`

```
def distances_one_query(X, q):
    result = np.zeros(X.shape[0], dtype=float)
    for i in range(X.shape[0]):
        total = 0.0
        for j in range(X.shape[1]):
            total += (X[i, j] - q[j]) ** 2
        result[i] = np.sqrt(total)
    return result

print(np.round(distances_one_query(X, q), 4))
```

הפרדת חישוב המרחקים מן ההצבעה מאפשרת לבדוק כל רכיב בנפרד. אם התקבלה תחזית שגויה, אפשר לשאול האם המרחקים שגויים, האם המיון שגוי, או האם ההצבעה שגויה. כך עובדים גם בפרויקט תוכנה גדול: מפרקים בעיה לבדיקות קטנות.

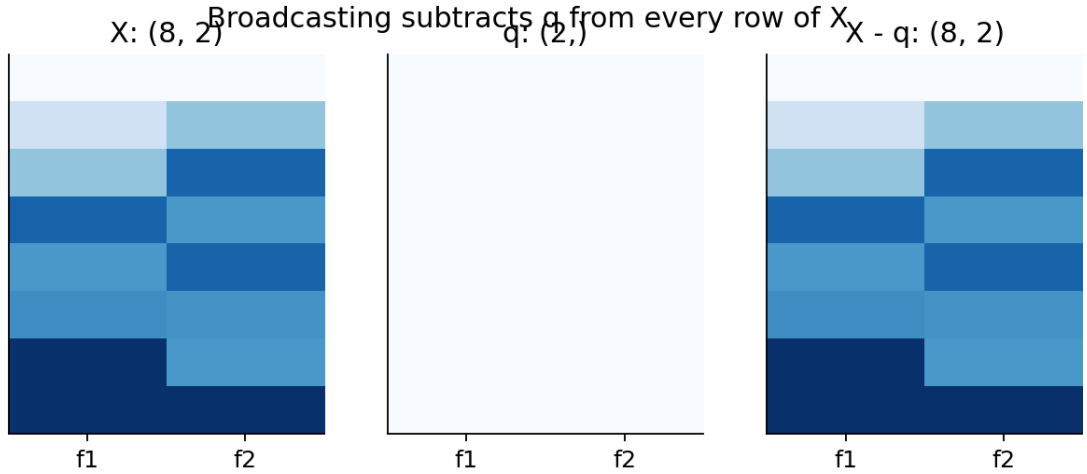
6. NumPy — מדוע לפעמים אפשר להחליף לולאות בפעולת מערכים?

למערך NumPy יש צורה (shape) שמתארת כמה שורות ועמודות יש בו. כאשר X היא מטריצה בעלת שתי עמודות ו־q הוא וקטור בן שני רכיבים, ניתן להחסיר את q מכל שורה ב־X. פעולה זו נקראת Broadcasting: NumPy מתאים מערכים בעלי צורות תואמות ומפעיל את אותה פעולה על קבוצת רכיבים. אין צורך לכתוב בעצמנו לולאה לכל שורה.

```
(8, 2) − (2,)  →  (8, 2)
```

```
differences = X - q
squared = differences ** 2
sums = np.sum(squared, axis=1)
distances = np.sqrt(sums)
print(differences.shape)
print(np.round(distances, 4))
```

ב־axis=1 אנחנו מסכמים כל שורה בנפרד, משום שכל שורה מתארת תצפית. אם בטעות נשתמש ב־axis=0, נקבל שני סכומים — אחד לכל מאפיין — במקום שמונה מרחקים. זו שגיאת תוכן ולא שגיאת תחביר, ולכן חשוב לנבא מראש מה אמורה להיות צורת התוצאה.



איור 4. הפחתת q מכל שורה ב־X ויצירת מטריצת הפרשים. שמירת shape נכונה קריטית למימוש.

7. מיון אינדקסים — מדוע argsort עדיף על sort?

הפקודה np.sort מחזירה את המרחקים כשהם מסודרים, אך בדרך זו נאבד את הקשר אל התווית של התצפית. לעומת זאת, np.argsort מחזירה את האינדקסים שמסדרים את המרחקים. לכן נוכל להשתמש באותם אינדקסים כדי לקרוא מתוך y את המחלקות של השכנים שנבחרו.

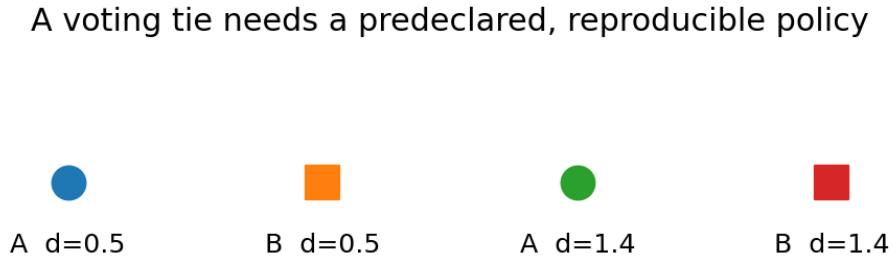
```
order = np.argsort(distances, kind="stable")
k = 3
neighbor_indices = order[:k]
neighbor_labels = y[neighbor_indices]
print(neighbor_indices)
print(neighbor_labels)
```

np.argsort היא אחת הפקודות החשובות בפרק: היא מחזירה מידע על סדר, לא את התוויות ולא את הערכים עצמם. בחרנו stable כדי לשמור על סדרם המקורי של פריטים בעלי מרחק שווה; עדיין צריך להגדיר כיצד מוכרעת הצבעה בין שתי מחלקות עם אותו מספר קולות.

8. הכרעת רוב ומדיניות שוויון — חלק מחוזה האלגוריתם

ב־k אי־זוגי ובבעיה שיש בה שתי מחלקות בלבד, הצבעה ברוב אינה יכולה להסתיים בשוויון במספר הקולות — בתנאי שנבחרו בדיוק k שכנים. אבל עם שלוש מחלקות, או כאשר k זוגי, ייתכן שוויון. נוסף לכך, ייתכן שוויון מרחק בגבול שבין השכן ה־k לבין השכן ה־k+1. אם לא נכתוב כלל מראש, שתי הרצות או שני מימושים עשויים לתת תשובות שונות.

לצורך המימוש הלימודי שלנו נקבע מדיניות מפורשת: במרחקים שווים נשמור תחילה על סדר האינדקסים המקורי; בהצבעת רוב שווה נבחר את המחלקה של השכן הקרוב ביותר מבין המחלקות הקשורות; ואם גם המרחק המינימלי שווה בדיוק, נבחר את שם המחלקה לפי סדר אלפביתי. זוהי מדיניות לימודית שבחרנו לצורך שחזוריות, ולא טענה שהמימוש של scikit-learn מפעיל בהכרח אותו כלל.



איור 5. שתי מחלקות עשויות לקבל מספר קולות זהה. יש לתעד כלל הכרעה לפני הפעלת המודל.

```
  def vote_with_tie_policy(labels, distances):
      classes, counts = np.unique(labels, return_counts=True)
      max_count = np.max(counts)
      tied = classes[counts == max_count]
      if len(tied) == 1:
          return tied[0]
      closest = []
      for label in tied:
          closest.append(np.min(distances[labels == label]))
      min_distance = np.min(closest)
      remaining = tied[np.isclose(closest, min_distance)]
      return sorted(remaining)[0]
```

דוגמה לבדיקה: עבור שכנים עם תוויות A,B,A,B ועם מרחקים 0.2,0.5,0.7,1.1, לכל מחלקה שני קולות. המדיניות שלנו בוחרת A, משום שהשכן הקרוב ביותר מבין המחלקות הקשורות שייך ל־A. אם השכן הקרוב ביותר היה B, הבחירה הייתה משתנה. תלמיד צריך להיות מסוגל להסביר את ההחלטה ללא הרצת הקוד.

9. בניית מסווג אחד קריא ומפורש

נשלב את רכיבי התהליך בפונקציה אחת לצורך הדגמה. נשתמש בפעולת NumPy לחישוב המרחקים, ב־argsort לשמירת אינדקסים ובפונקציית ההכרעה שכתבנו. השמות שנבחרו בקוד מבטאים תפקיד; הם אינם קיצור מסתורי שמחייב לזכור את גוף הפונקציה.

```
def predict_one(X, y, q, k=3):
    if X.ndim != 2 or len(y) != len(X):
        raise ValueError("Expected X: 2D and y: matching rows")
    if q.shape != (X.shape[1],):
        raise ValueError("Query feature count does not match")
    if not 1 <= k <= len(X):
        raise ValueError("k must be between 1 and n")
    distances = np.sqrt(np.sum((X - q) ** 2, axis=1))
    order = np.argsort(distances, kind="stable")[:k]
    return vote_with_tie_policy(y[order], distances[order])

print(predict_one(X, y, q, k=1))
print(predict_one(X, y, q, k=3))
```

שימו לב: עבור ערכי k הנבדקים בדוגמה זו אין שוויון בין מחלקות לאחר ההצבעה. כאשר משווים מימושים שונים על נתונים שיש בהם שוויון גבולי, אין לצפות בהכרח לאותה תוצאה ללא יישור של מדיניות השוויון.

10. מעבר לשאילתות רבות — שלושה ממדים במערך

עד כה חישבנו מרחקים בין שאילתה אחת לבין כל שורות האימון. אבל באפליקציה נחזה בדרך כלל כמה דוגמאות חדשות. נרצה מטריצת מרחקים שבה שורה אחת לכל שאילתה ועמודה לכל דוגמת אימון. כדי לאפשר זאת, נוסיף ממד בגודל 1 לכל אחד מהמערכים וניתן ל־Broadcasting ליצור את כל זוגות ההפרשים.

$$\texttt{Q.shape = (m, d) ; X.shape = (n, d)}$$

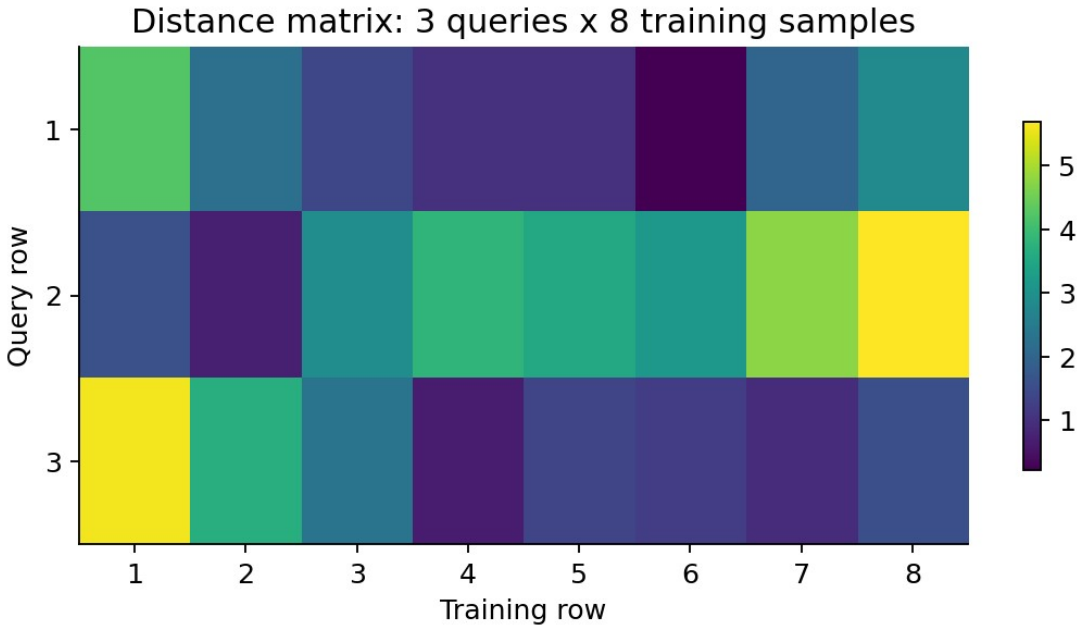
$$\texttt{Q[:, None, :].shape = (m, 1, d)}$$

$$\texttt{X[None, :, :].shape = (1, n, d)}$$

$$\texttt{differences.shape = (m, n, d)} \;\rightarrow\; \texttt{distances.shape = (m, n)}$$

```
Q = np.array([[4., 4.], [1.5, 2.5], [5.3, 4.6]])
all_diffs = Q[:, None, :] - X[None, :, :]
all_distances = np.linalg.norm(all_diffs, axis=2)
print(all_diffs.shape)
print(all_distances.shape)
```

המשמעות של axis=2 כאן שונה מ־axis=1 קודם: במערך החדש ציר 0 מייצג שאילתות, ציר 1 מייצג דוגמאות אימון, וציר 2 מייצג מאפיינים. סכימה על הציר האחרון מחשבת מרחק לכל זוג. אין לשנן axis ללא בדיקת shape.

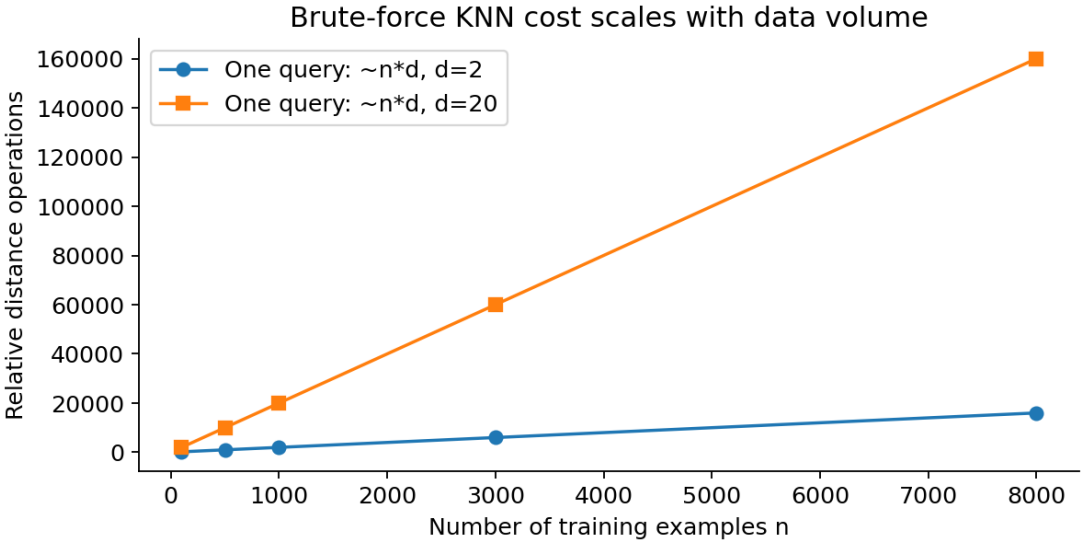


איור 6. מטריצת המרחקים מאפשרת להשוות כמה שאילתות מול כל דוגמאות האימון.

11. האם קוד קצר יותר בהכרח יעיל יותר?

שימוש ב־Broadcasting מקטין את הצורך בלולאות פייתון ויכול להיות מהיר מאוד. אולם עבור m שאילתות, n דוגמאות אימון ו־d מאפיינים, מערך הביניים של ההפרשים עשוי להכיל m×n×d מספרים. כאשר שלושת הגדלים גדולים, צריכת הזיכרון עלולה להפוך לבעיה ממשית. לכן לא נוכל להסיק ש״וקטוריזציה תמיד עדיפה״; נצטרך לשאול גם כמה זיכרון נדרש.

פתרונות אפשריים לפרויקט גדול הם חישוב בקבוצות קטנות (Batches), עבודה עם ספריות שמבצעות חישוב מותאם, או שימוש בשיטות חיפוש שכנים כגון KD-Tree ו־Ball-Tree כאשר תנאי הנתונים מתאימים. אין חובה לממש עצים בפרק זה; חשוב להבין שהמימוש הפשוט אינו הפתרון היחיד.



איור 7. במימוש brute-force, מספר חישובי המרחק גדל עם מספר דוגמאות האימון ומספר המאפיינים.

12. ניתוח מורכבות חישובית ברמת תיכון

נניח שיש n דוגמאות אימון ו־d מאפיינים. כדי לחשב את מרחקה של שאילתה אחת לכל הדוגמאות נבצע בסדר גודל של $n \times d$ פעולות חשבון. כאשר ממיינים את כל n המרחקים, המיון מוסיף עלות בקירוב $n \log n$. לכן עבור המימוש הלימודי עם argsort עלות החיזוי היא בקירוב $O(nd + n \log n)$ לשאילתה אחת, מעבר לעלות האחסון של הנתונים. קיימות שיטות לבחירת k איברים קטנים בלי למיין הכול, אך לא נכניס אותן לליבת המימוש הראשוני.

```
Training storage ≈ O(n d)    |    Distance calculation ≈ O(n d)
Full sort ≈ O(n log n)       |    m-query broadcast memory ≈ O(m n d)
```

יש להבדיל בין זמן חישוב לבין זיכרון. גם אם המחשב מצליח ליצור מערך עצום, אין פירוש הדבר שהפעולה יעילה או מתאימה להפעלה באפליקציה בזמן אמת. שאלה כזו נכנסת לפרק הגנת פרויקט: אילו מגבלות צפיתם, ואיזה שינוי עשיתם כשנפח הנתונים גדל?

13. הצבעה משוקללת לפי מרחק

במנגנון הצבעת רוב כל אחד מן השכנים הנבחרים מקבל קול אחד, גם אם אחד קרוב מאוד ואחר רחוק יותר. בשיטה משוקללת ניתן לתת לשכן קרוב משקל גבוה יותר, למשל באמצעות משקל הפוך למרחק. כך מקרה שבו יש שני קולות למרחקים גדולים וקול אחד קרוב מאוד עשוי להתהפך.

$$w_i = 1 / d_i \qquad (d_i > 0)$$

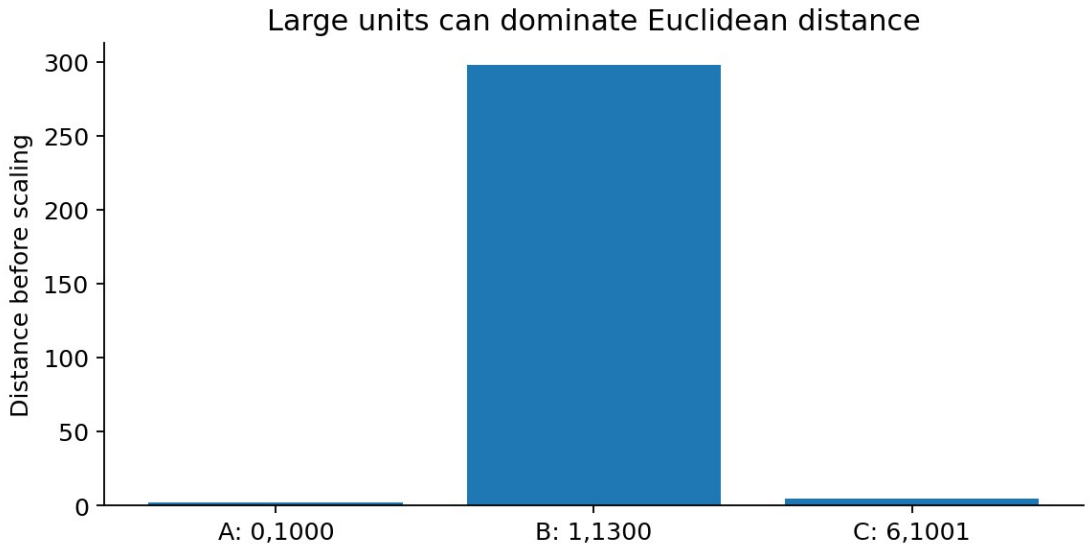
כאשר מרחק שווה לאפס, הביטוי $1/d$ אינו מוגדר. לכן יש צורך במדיניות נפרדת: אם יש שכנים זהים לשאילתה, ניתן להכריע רק ביניהם על פי התוויות שלהם, במקום לחלק באפס. בספריית scikit-learn קיימת אפשרות "weights="distance", אך היא מטפלת במקרים מיוחדים לפי המימוש שלה; אין לייחס לה באופן אוטומטי את כללי המימוש הלימודי שלנו.

```
from sklearn.neighbors import KNeighborsClassifier
model = KNeighborsClassifier(n_neighbors=3, weights="distance")
model.fit(X, y)
print(model.predict(q.reshape(1, -1)))
```

חשוב לא לבלבל משקל בהצבעת KNN עם משקל שנלמד ברגרסיה או ברשת עצבית: ב־KNN ההצבעה המשוקללת מבוססת על מרחקים בזמן החיזוי, ולא על עדכון הדרגתי של וקטור משקולות באמצעות Gradient Descent.

14. קנה מידה ותקנון — בדיקת שגיאה מקצועית

מרחק אוקלידי רגיש ליחידות המדידה. אם מאפיין אחד נע בין 0 ל־10 והשני בין 0 ל־10,000, ההפרש המספרי במאפיין השני עלול לשלוט במרחק. לפיכך, ייתכן שנקודה תיראה רחוקה בעיקר בגלל סולם המדידה ולא בגלל חשיבות אמיתית של המאפיין. כבר למדנו תקנון בפרק 2.3; כאן ניישם אותו באופן מקצועי.



איור 8. הבדל גדול ביחידות של מאפיין יכול להשתלט על המרחק הכולל.

את ממוצע האימון וסטיית התקן של האימון יש לחשב רק על נתוני האימון. נתוני Test עוברים transform באמצעות אותם פרמטרים; אין להשתמש בהם כדי להתאים את הממיר. בוולידציה צולבת יש להתאים מחדש את הממיר בתוך כל קיפול אימון, ולכן Pipeline הוא כלי העבודה המועדף.

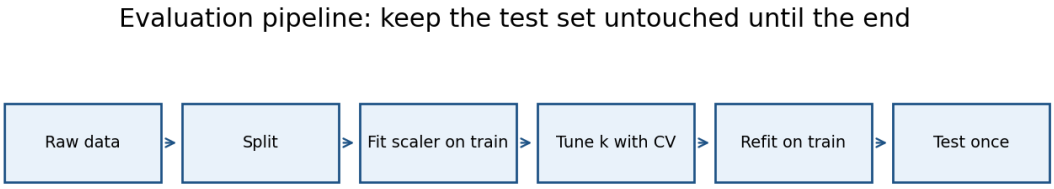
15. בניית ניסוי אמיתי על Iris בלי זליגת מידע

נעבור לדוגמה מקצועית מתוך Iris: קבוצת נתונים מוכרת בעלת ארבע תכונות מדידה ושלושה סוגי אירוסים. כאן נבחר לצורך גרפים שתי תכונות בלבד, ונשמור את יחס המחלקות בעת החלוקה. בנתוני פרויקט אמיתי חשוב לתאר גם מקור נתונים, איכותם, ערכים חסרים ומגבלות ייצוג האוכלוסייה.

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.neighbors import KNeighborsClassifier

iris = load_iris()
X = iris.data[:, [0, 2]]
y = iris.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.30, random_state=19, stratify=y
)
pipe = Pipeline([
    ("scale", StandardScaler()),
    ("knn", KNeighborsClassifier(n_neighbors=5))
])
pipe.fit(X_train, y_train)
print("Test accuracy:", pipe.score(X_test, y_test))
```

למה לא להפעיל scaler.fit_transform על כל X לפני החלוקה? משום שאז ממוצעי נתוני הבדיקה ישפיעו על אופן הכנת נתוני האימון. זוהי זליגת מידע. שימו לב גם ש־pipe.fit אינו ״מלמד״ את KNN משקולות כמו ברשת עצבית; הוא מתאים את הממיר ושומר את דוגמאות האימון שהמסווג צריך בזמן החיזוי.

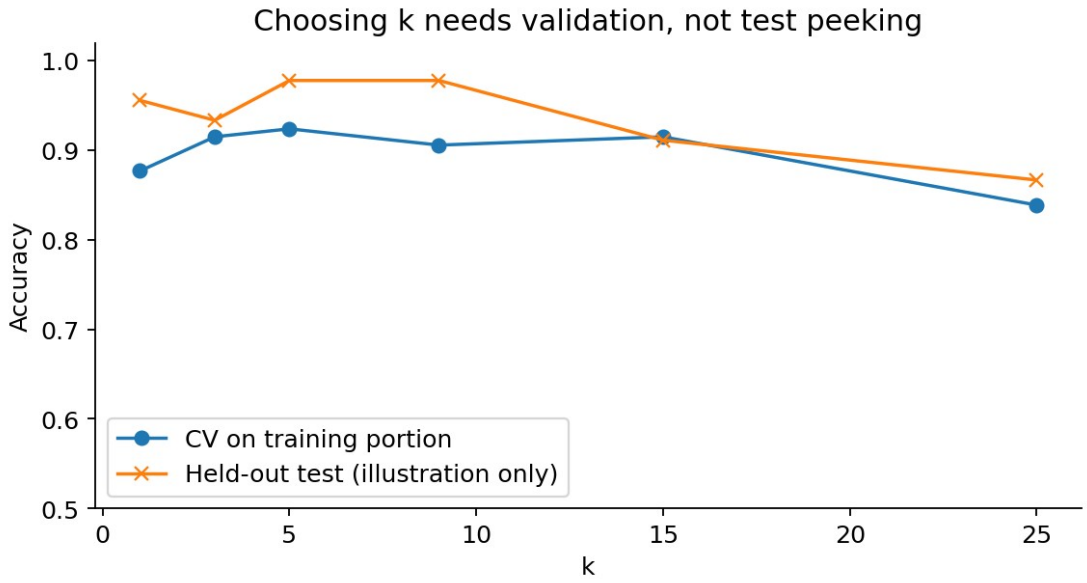


איור 9. סדר עבודה תקין: חלוקה, התאמת הכנה בתוך אימון, בחירת k בוולידציה ובדיקת Test בסוף.

16. בחירת k באמצעות Cross-Validation

היפר־פרמטר k משפיע על רגישות המודל. ערך קטן מאוד יכול לגרום להחלטה רגישה לרעש, וערך גדול מדי עשוי להחליק גבולות החלטה יתר על המידה. איננו בוחרים k לפי Test; בוחרים אותו באמצעות נתוני האימון, למשל באמצעות K-Fold או StratifiedKFold. קיפולים מספקים כמה אומדנים, אך אין לראות בממוצע שלהם הבטחה לתוצאה זהה בנתוני אמת.

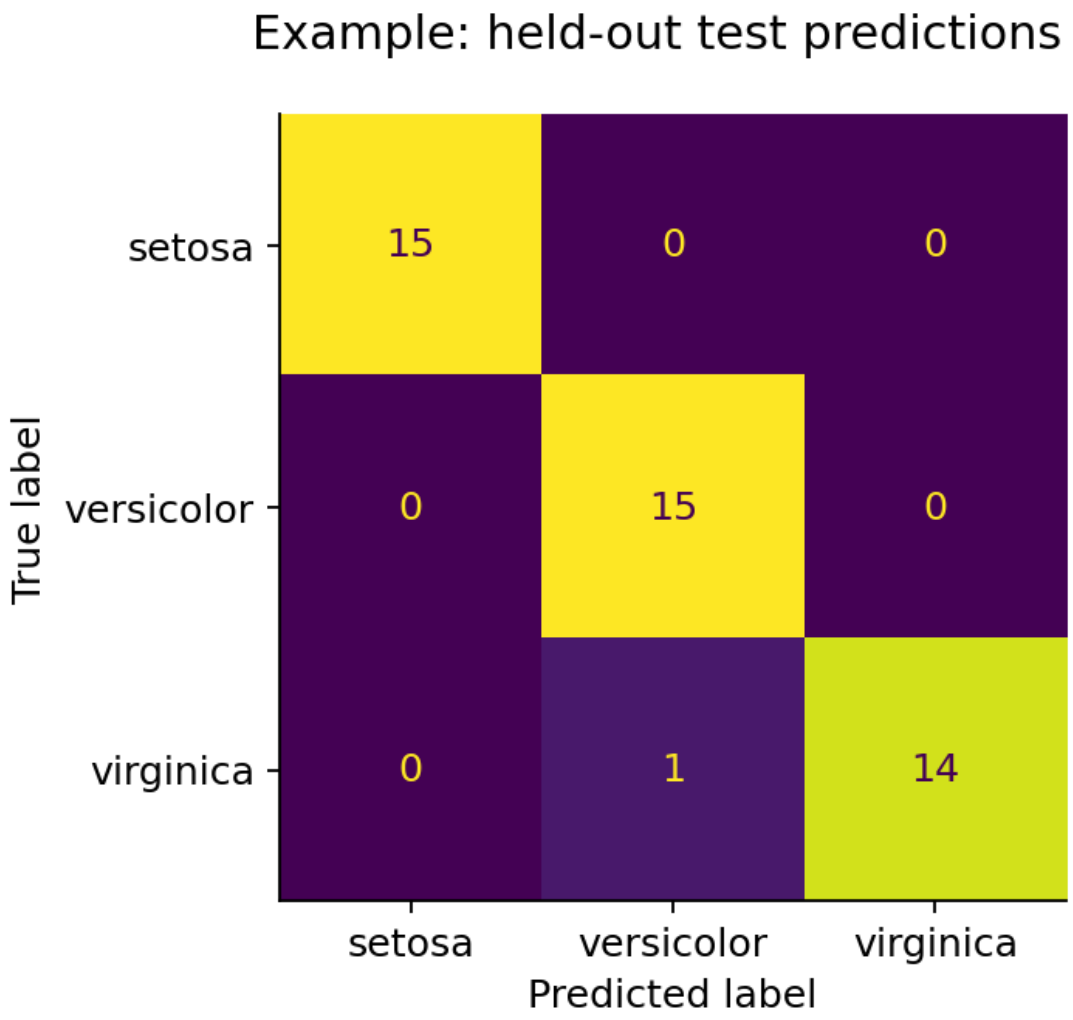
```
from sklearn.model_selection import StratifiedKFold, cross_val_score
cv = StratifiedKFold(n_splits=4, shuffle=True, random_state=19)
for k in [1, 3, 5, 9, 15]:
    candidate = Pipeline([
        ("scale", StandardScaler()),
        ("knn", KNeighborsClassifier(n_neighbors=k))
    ])
    scores = cross_val_score(candidate, X_train, y_train,
                             cv=cv, scoring="accuracy")
    print(k, round(scores.mean(), 3), round(scores.std(), 3))
```



איור 10. דוגמה מדגמית על Iris: עקומות CV ו־Test עשויות להשתנות עם החלוקה. בוחרים k מתוך CV בלבד; Test מוצג לצורך דיון לאחר הבחירה.

17. מדדי ביצוע — מעבר למספר Accuracy בודד

לפעמים מודל מסווג היטב את המחלקה הגדולה אך מחמיץ מחלקה נדירה וחשובה. כדי להבין מה קרה נבחן מטריצת בלבול, Precision, Recall ו־F1 לפי הצורך. במקרה של כמה מחלקות נדווח גם כיצד אוחדו המדדים בין מחלקות — למשל macro או weighted — ולא רק מספר ממוצע חסר הסבר.



איור 11. דוגמה למטריצת בלבול מתוך קבוצת בדיקה מופרדת; האלכסון מציין תחזיות נכונות.

בחינת הבגרות דורשת יכולת להסביר תוצאה, ולא להסתפק בשורת פלט. אם מודל קיבל Accuracy גבוה, התלמיד צריך לדעת לשאול: בכמה דוגמאות נבדק? מה יחס המחלקות? האם התבצעה התאמת תקנון על נתוני הבדיקה? מהו מחיר טעות מסוג מסוים?

18. בדיקות יחידה למימוש KNN שלנו

לפני שמציגים פרויקט גמר בודקים גם מקרים קטנים שביכולתנו לחשב ביד. בדיקת יחידה טובה אינה דורשת מאגר גדול: לעיתים דווקא שלוש נקודות מאפשרות לזהות שגיאה. רצוי לתכנן מראש בדיקה למרחק אפס, לשתי נקודות שוות מרחק, לאינדקסי מיון, לממדים לא תואמים ולמקרה שבו k גדול ממספר דוגמאות האימון.

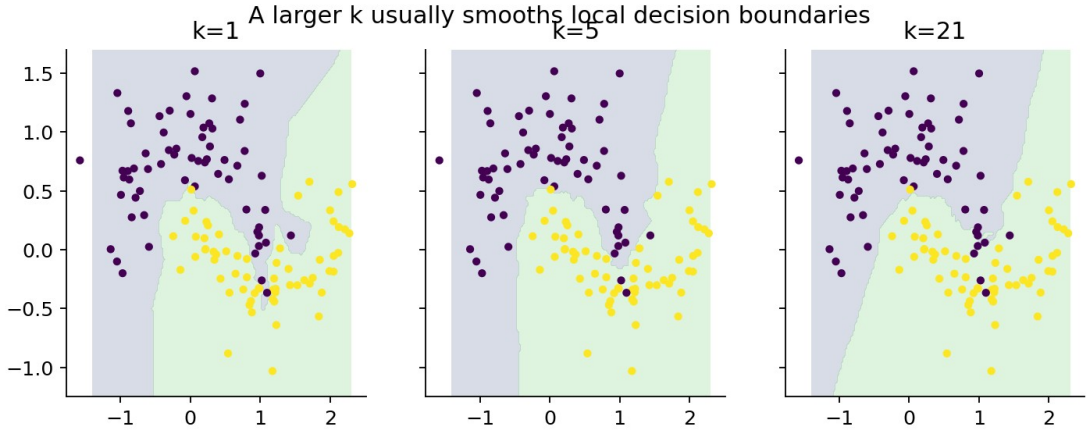
```
# Tests for our teaching implementation
assert predict_one(X=np.array([[0.,0.],[3.,0.]]),
                   y=np.array(["A", "B"]),
                   q=np.array([.2,0.]), k=1) == "A"

assert np.isclose(
    np.linalg.norm(np.array([3.,4.]) - np.array([0.,0.])), 5.0
)
print("Small deterministic tests passed")
```

חשוב להבדיל בין בדיקת נכונות לבין הערכת דיוק על מאגר: בדיקת יחידה אומרת שהמימוש פועל לפי הכללים שכתבנו במקרה ידוע; Accuracy על Test מודד כיצד המודל מכליל לדוגמאות חדשות. האחת אינה מחליפה את האחרת.

19. על אילו מקרים KNN עלול להיכשל?

KNN יכול להיכשל בשל בחירת מאפיינים לא מתאימים, קנה מידה, רעש, חוסר איזון במחלקות או מספר גדול של ממדים. במרחב עתיר ממדים, מרחקים בין נקודות עלולים להיות פחות מבחינים, תופעה המכונה לעיתים "קללת הממדיות". אין פירוש הדבר ש־KNN לעולם אינו עובד עם הרבה מאפיינים, אלא שיש לבדוק האם המרחק שנבחר משמעותי עבור המשימה והנתונים.



איור 12. שינוי k משפיע על צורת גבולות ההחלטה; הגבול החלק ביותר אינו בהכרח המתאים ביותר.

בנוסף, מאגר שיש בו שתי קבוצות של אותה מחלקה עשוי להיות מבלבל עבור Nearest Centroid, אך KNN יכול לעיתים לזהות את המבנה המקומי. מצד שני, כאשר יש מספר עצום של נקודות אימון ומערכת חייבת לענות במהירות רבה, העלויות של KNN יכולות להקשות על שימוש ישיר. בחירת המודל תמיד תלויה בבעיה.

20. כיצד מחברים KNN לעיבוד תמונה?

בפרק 6 למדנו שתמונה היא מערך מספרי. אפשר לשטח תמונת גווני אפור לווקטור ולהגדיר מרחק בין תמונות. אז לכאורה ניתן להפעיל KNN על פיקסלים: תמונה חדשה תסווג לפי התמונות הקרובות אליה. זהו קו בסיס לימודי שימושי, אך מרחק פיקסלים רגיש להזזה, תאורה וסיבוב, ולכן אינו מבטיח לזהות צורות סמנטיות דומות.

הקשר העמוק לפרקי CNN יהיה ברור בהמשך: במקום להשוות רק ערכי פיקסלים גולמיים, רשת עצבית יכולה ללמוד ייצוגים של מאפיינים חזותיים. לאחר לימוד רשתות נוכל לשאול אם KNN על ייצוגים נלמדים נותן תוצאות טובות יותר מ־KNN על פיקסלים. זו הרחבת גשר לפרויקט, ולא תחליף לפרק CNN.

21. יישום בפרויקט גמר — דף החלטות הנדסי

תלמיד שבוחר להשתמש ב־KNN בפרויקט צריך לתעד את שאלת המחקר, מקור הנתונים, שמות המאפיינים, יחידותיהם, דרך הטיפול בחסרים, אופן החלוקה, שיטת התקנון, רשימת ערכי k שנבדקו, מדדי ההערכה, מגבלות הביצועים והסיבה לבחירת המודל. יש להימנע מתיעוד טכני שנכתב לאחר מעשה ואינו משקף ניסוי שבוצע באמת.

מה צריך לנמק?	החלטה
האם הם זמינים בזמן חיזוי ובעלי משמעות?	מאפיינים
מדוע Euclidean או מדד אחר מתאים לנתונים?	מרחק
כיצד הותאם רק על Train?	תקנון
כיצד נבחר באמצעות Validation/CV?	k
מהו מחיר הטעות ומהי המחלקה החשובה?	מדד
מה עלול להשתבש בנתונים חדשים?	מגבלה

22. טעויות נפוצות שכדאי לעצור בזמן

- סידור distances במיון ושכחת לעדכן באותו סדר את התוויות המתאימות.
- שימוש ב־axis=0 במקום axis=1 בחישוב מרחקי שאילתה אחת, או במקום axis=2 בחישוב כמה שאילתות.
- הפעלת fit_transform על כל הדוגמאות לפני חלוקת Test או לפני CV.
- הנחה שהמימוש הידני ו־sklearn יכריעו כל שוויון באותה דרך.
- חשיבה ש־predict_proba בהכרח נותנת הסתברות מכוילת היטב לשימוש בהחלטה רגישה.
- שימוש ב־Accuracy אימון כהוכחה להכללה, או בחירת k לאחר הסתכלות ב־Test.
- בניית מטריצת Broadcasting ענקית בלי להבין את מחיר הזיכרון.
- תיאור k בתור פרמטר שנלמד מן הנתונים; k הוא היפר־פרמטר שהמפתח בוחר או מכוון.

23. סיכום מושגים, תפקידם והקשר ל־AI

מושג	מה צריך לדעת להסביר
shape	מה המשמעות של כל ציר במערך NumPy
Broadcasting	מתי ניתן לבצע פעולת מערך ללא לולאות Python מפורשות
argsort	כיצד לשמור קשר בין מרחק לאינדקס ולתווית
k	השפעת מספר השכנים על החלטת סיווג
Tie policy	מה קורה במקרה של שוויון מרחק או קולות

מה צריך לדעת להסביר	מושג
מדוע הכנת הנתונים חייבת להיכלל בתהליך האימון	Pipeline
כיצד בוחרים היפר־פרמטר בלי לגעת ב־Test	CV
מה מחיר מרחקים, מיון ואחסון בנתונים גדולים	Complexity

24. תרגול מדורג — ללא פתרונות

רמת יסוד: הבנת מבנה המימוש

- שאלה 1. נתונה מטריצה X בעלת 20 שורות ו־4 עמודות ושאילתה q בעלת 4 רכיבים. כתבו את shape של X, של q, של X-q ושל וקטור המרחקים.
- שאלה 2. לפניכם מרחקים [2.0,0.5,1.0] ותוויות [A,B,A]. קבעו אילו אינדקסים תחזיר argsort, ומה תהיה התחזית כאשר k=1 וכאשר k=3.
- שאלה 3. הסבירו במילים מדוע אי־אפשר פשוט למיין את מערך התוויות כאשר ממיינים את המרחקים.
- שאלה 4. תלמיד כתב np.sum((X-q)**2, axis=0). הסבירו מה הקוד מחשב ולמה זו אינה התוצאה הנדרשת.

רמת העמקה: מתמטיקה, ניסוי ותכנון

- שאלה 5. בנו מערכת של ארבע דוגמאות ושתי מחלקות שבה מספר הקולות ב־k=4 שווה. הגדירו כלל הכרעה ודונו בהשפעתו.
- שאלה 6. נתון מאגר עם 2,000 דוגמאות ו־30 מאפיינים. העריכו כמה פעולות הבדל־ריבוע נדרשות לשאילתה אחת במימוש brute-force.
- שאלה 7. הסבירו מדוע שימוש ב־StandardScaler לפני cross_val_score יכול לגרום לזליגת מידע, והציעו תיקון באמצעות Pipeline.
- שאלה 8. תכננו השוואה בין k=1, k=5 ו־k=15 על Iris שתשתמש ב־CV לבחירה וב־Test להערכה סופית בלבד.

רמת בגרות ופרויקט: נימוק מקצועי

- שאלה 9. תלמיד טוען כי ״כל מימוש NumPy מהיר יותר כי אין בו לולאות״. הסבירו מדוע הטענה גורפת מדי תוך התייחסות לזיכרון.
- שאלה 10. מערכת לזיהוי חריגות בנתוני תלמידים מקבלת Accuracy של 97%, אך כמעט אינה מזהה את המחלקה הנדירה. הציעו לפחות שני מדדים או ניסויים נוספים.
- שאלה 11. שני מימושים של KNN מחזירים תוצאה שונה רק כאשר השכן ה־k והשכן ה־k+1 נמצאים באותו מרחק. הציעו בדיקת יחידה והסבירו מקור אפשרי להבדל.
- שאלה 12. כתבו פסקת הגנה בעל־פה: מדוע בחרתם ב־KNN, מה יתרונו, מה חולשתו ומה תרצו לשנות אם המאגר יגדל פי מאה.

25. משימת סיכום — ממימוש למערכת קטנה

בנו מחברת Colab המשווה בין המימוש העצמאי שלכם לבין KNeighborsClassifier על אותה חלוקת נתונים. צרו טבלת השוואה של חיזויים, נתחו מקרים שבהם התחזיות שונות, בדקו תחילה את מדיניות השוויון, והשוו מדדי בדיקה. לאחר מכן כתבו דוח קצר שבו מוסברים מקורות הנתונים, השיטה, החלטת k, שיטת התקנון, מגבלה אחת והמלצה להמשך. הפעילו את כל התאים מהתחלה לפי סדרם כדי לוודא שחזוריות.

26. מפת הקשרים בין שלוש תכניות הלימודים

מעמד	מה נלמד או נדרש	מסלול
ידע קודם	הפעלה והבנה ראשונית של KNN ומדדי ביצוע	יחידה 3 — בסיס
עוגן בליבה	מרחקים, גבולות החלטה, k, ניתוח וסיווג	יחידה 5 — נושאי ליבה
עוגן התמחות	מימוש עצמי, NumPy, Broadcasting וניסויים על Iris	התמחות DL — פרק 7
העמקה דידקטית	בדיקות יחידה, זיכרון, מגבלות ומהירות	הרחבת פרויקט

27. קריאה משלימה ומקורות

עוגן תכנית: משרד החינוך, התמחות למידת מכונה ולמידה עמוקה, פרק 7 — בעיית הסיווג ו־KNN, 30 שעות (לפי מפת המאגר). מצגות ההשתלמות KNN 5.1, KNN with sklearn 5.2 ו־KNN with code 5.3; מחברות Iris ותרגולי המימוש העצמאי.

ספרי יסוד ששיטות ההסבר שלהם משמשות השראה לכתיבה מקורית: Luis Serrano, Grokking Machine Learning; Aurélien Géron, Hands-On Machine Learning; Deisenroth, Faisal & Ong, Mathematics for Machine Learning; James et al., An Introduction to Statistical Learning with Applications in Python; Zhang et al., Dive into Deep Learning.

תיעוד טכני: https://numpy.org/doc/stable/user/basics.broadcasting ; https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.KNeighborsClassifier.html ; https://scikit-learn.org/stable/modules/cross_validation.html

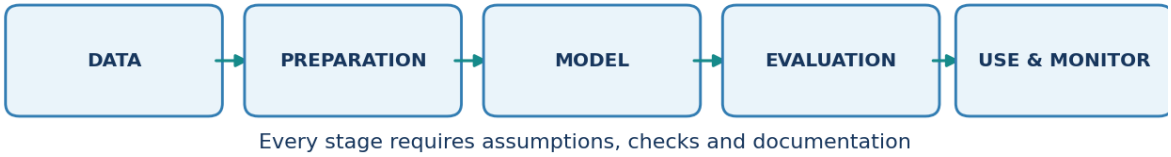
הערה: ההסברים, הדוגמאות, הטבלאות והאיורים נכתבו ויוצרו לפרק זה באופן מקורי. התאמה לנוסח המחייב של כל סעיף בבגרות טעונה בדיקה מול מסמכי משרד החינוך התקפים לשנת הלימודים.

פרק 8 — מושגי יסוד בלמידת מכונה

ממודל שמחשב תחזית למערכת שלומדת, נבדקת ומשתפרת

ספר תלמיד | התמחות למידת מכונה ולמידה עמוקה — 5 יחידות לימוד | 10 שעות על פי מפת התכנית שבמאגר.

מעמד הפרק: פרק 8 בתכנית ההתמחות. דוגמאות על Gradient Descent ורגולריזציה מובאות כהכנה לפרקי הפרספטרון והמתמטיקה של למידה עמוקה; אין לייחס להן מעמד של תת־סעיף חובה בפרק זה ללא הצלבת נוסח הסילבוס הרשמי המלא.



איור: מסלול פיתוח מודל: עבודה עם נתונים, הכנה, התאמת מודל, הערכה ושימוש אחראי לאורך זמן.

איך לומדים בעזרת הספר?

בכל מושג נתחיל משאלה מחיי היום־יום, נציג דוגמה שניתן לחשב ביד, ננסח את העיקרון המתמטי, נבדוק אותו ב־Python ולבסוף ננתח מקרה שבו התוצאה עלולה להטעות. אין צורך לזכור את שמות הפונקציות בעל פה: החשוב הוא להבין מה כל פעולה מקבלת, מה היא מחזירה ומדוע בחרנו לבצע אותה.

מטרות ביצועיות

- להבחין בין נתונים, מאפיינים, משתנה מטרה, תחזית, פרמטר והיפר־פרמטר.
- להסביר כיצד משתמשים בפונקציית שגיאה כדי להשוות תחזיות, ולחשב MAE ו־MSE ביד.
- להבין את רעיון שינוי הפרמטרים בעת אימון, כולל דוגמת צעד אחת של Gradient Descent כהכנה.
- לזהות Overfitting ו־Underfitting בעזרת גרפים והבדלים בין אימון לוולידציה.
- לתכנן ניסוי הוגן עם חלוקת נתונים, קו בסיס ומניעת זליגה.
- לנסח החלטה מקצועית מנומקת מתוך מדדים, מגבלות ויומן ניסויים.

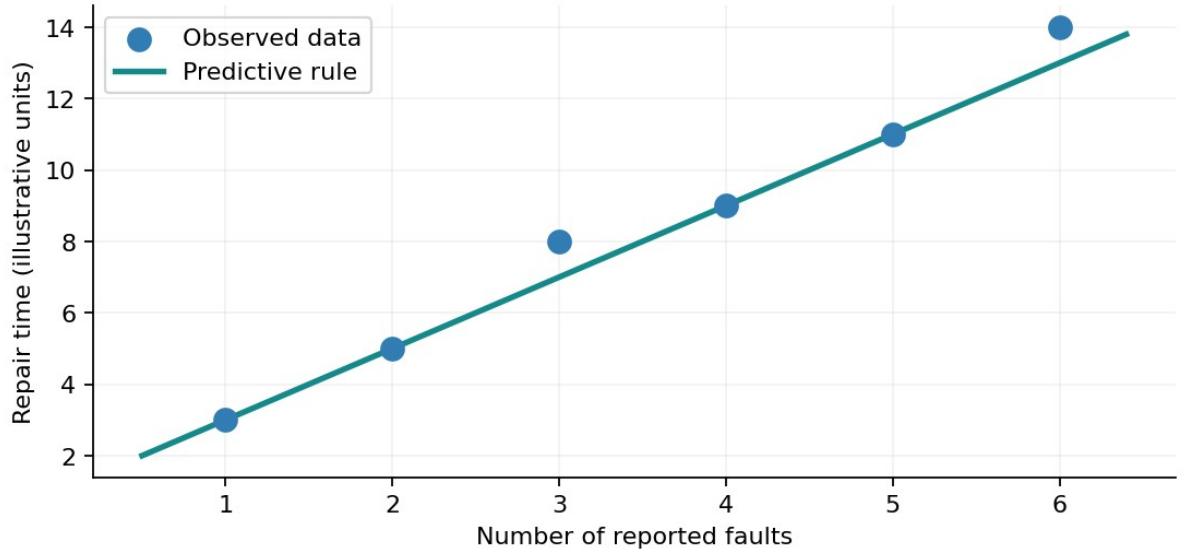
ידע קודם

הקורא כבר למד פונקציות בסיסיות, וקטורים, טבלאות וגרפים, רגרסיה ליניארית, KNN, מדדי הערכה ו־Train/Validation/Test. אין צורך ללמוד אותם מחדש. במקום זאת נשתמש בהם כדי להבין את תהליך האימון והבחירה במודל ברמה גבוהה יותר.

1. סיפור פתיחה: מחשב שחוזה זמן תיקון — האם הוא באמת לומד?

דודו אחראי על מעבדת המחשבים בבית הספר. לאחר כל תיקון הוא רושם כמה תקלות אותרו וכמה זמן ארך התיקון. הוא רוצה לבנות מערכת שתעריך מראש את משך הטיפול במחשב חדש, כדי לתכנן את עבודת הטכנאים. בתחילה הוא כותב כלל קבוע: ״כל תקלה מוסיפה עשר דקות״. אבל בבדיקת נתוני עבר הוא מגלה שהכלל מתאים לחלק מהמקרים ולאחרים פחות.

המעבר האמיתי ללמידת מכונה אינו עצם כתיבת נוסחה. בלמידת מכונה אנחנו בוחרים משפחה של כללים אפשריים, משתמשים בנתונים כדי להתאים את הפרמטרים של הכלל, ובודקים האם הכלל שנבחר עוזר לנו לחזות מקרים שלא שימשו להתאמתו.



איור: דוגמה לימודית של מספר תקלות מול זמן תיקון. הנקודות הן מדידות; הקו הוא כלל חיזוי. הוא אינו חייב לעבור בכל נקודה.

1.1 מה אנחנו יודעים ומה המודל צריך ללמוד?

תפקיד	דוגמה במעבדה	מרכיב
דוגמה אחת ללמידה	מחשב שטופל בעבר	תצפית
מידע ידוע בזמן החיזוי	מספר התקלות	מאפיין x
מה שרוצים לחזות	משך התיקון בפועל	מטרה y
תוצאת המודל	משך תיקון משוער	תחזית ŷ
ערכים המותאמים לנתונים	שיפוע וחיתוך	פרמטרים

שאלת הבנה: האם "משך התיקון בפועל" הוא מאפיין שמותר להזין למודל כאשר הוא מנבא משך תיקון שטרם התרחש? לא. זהו בדיוק המשתנה שאנחנו מנסים לחזות. הכנסת התוצאה האמיתית לתכונות בזמן האימון תייצר זליגת מידע חמורה.

2. מהו מודל וכיצד הוא שונה מאלגוריתם האימון?

מודל הוא הפונקציה שמקבלת נתונים ומפיקה תחזית. אלגוריתם האימון הוא התהליך שמחפש ערכי פרמטרים מתאימים. בסיום האימון שומרים את המודל המותאם, ולא את כל סדר השאלות והטעויות שביצענו בדרך. לדוגמה, כאשר עובדים עם רגרסיה ליניארית, המודל הוא ישר; שיטת התאמת המקדמים עשויה להיות חישוב מתמטי ישיר או תהליך איטרטיבי.

$$\hat{y} = wx + b$$

במשוואה זו x הוא מאפיין הקלט, w הוא המקדם שמכפיל אותו, b הוא החיתוך עם ציר y, ו־ŷ היא התחזית. שים לב להבדל בין y, הערך האמיתי, לבין ŷ, הניחוש של המודל. ההבדל ביניהם עומד בבסיס פונקציית השגיאה.

2.1 פרמטר לעומת היפר־פרמטר

פרמטר הוא ערך שנקבע מתוך נתוני האימון בהתאם לאלגוריתם. היפר־פרמטר הוא בחירה שמגדירה כיצד ייבנה או יופעל תהליך הלמידה. הוא עשוי להיבחר בידי המפתח או באמצעות חיפוש על נתוני ולידציה. חשוב שלא לבלבל היפר־פרמטר עם תכונה של דוגמה אחת.



איור: הבחנה בין ערכים שהמודל מתאים מנתונים לבין הגדרות תהליך שהמפתח בוחר או מכוון.

מודל	פרמטר שנלמד	דוגמה להיפר־פרמטר
רגרסיה ליניארית	משקולות w והטיה b	מידת רגולריזציה ב־Ridge
KNN	אין משקולות גלובליות מהסוג הזה; דוגמאות האימון נשמרות	מספר השכנים k
רשת עצבית	משקולות והטיות של הנוירונים	קצב למידה, מספר שכבות

שאלת חשיבה: נניח שבחרנו ב־KNN עם k=5. האם המודל "למד" את המספר 5? לא. זהו ערך שנקבע בתהליך תכנון או כוונון. לעומת זאת, ברשת עצבית המשקולות בדרך כלל משתנות תוך כדי האימון.

3. איך מודדים עד כמה התחזית טובה? פונקציית השגיאה

כדי לשפר מודל דרושה דרך כמותית לתאר עד כמה הוא טועה. לא מספיק לומר שהתחזית "קרובה" לערך האמיתי, משום שקשה להשוות בין מאות תחזיות ללא כלל חישוב משותף. פונקציית שגיאה (Loss) הופכת את הפער בין התחזית לערך האמיתי למספר. כשמקבלים מספר כזה אפשר להשוות בין מודלים או בין שלבי אימון שונים.

3.1 שארית — הצעד הקטן הראשון

$$e_i = y_i - \hat{y}_i$$

אם זמן התיקון בפועל הוא 8 יחידות זמן והמודל חזה 7, השארית היא 1. אם הוא חזה 9, השארית היא 1−. הסימן מספר לנו לאיזה כיוון טעה המודל, אבל כאשר מסכמים שאריות חיוביות ושליליות הן עלולות לבטל זו את זו. לכן משתמשים לעיתים בערך מוחלט או בריבוע של הטעות.

3.2 MAE — ממוצע השגיאות המוחלטות

$$\mathrm{MAE} = \frac{1}{n}\sum |y_i - \hat{y}_i|$$

MAE מודד בכמה יחידות, בממוצע, התחזית מתרחקת מן התוצאה האמיתית. היתרון הדידקטי שלו הוא שהיחידות נשמרות: אם אנחנו חוזים דקות, גם MAE נמדד בדקות. מדד זה נוח להסבר לאנשים שאינם מפתחים, אך הוא אינו מעניש שגיאות גדולות באותה עוצמה כמו ריבוע השגיאה.

3.3 MSE — ממוצע ריבועי השגיאות

$$\mathbf{MSE = (1/n)\ \Sigma\ (y_i - \hat{y}_i)^2}$$

MSE נותן משקל גדול יותר לשגיאות גדולות כי הוא מעלה כל שגיאה בריבוע. הוא שכיח מאוד בפונקציות אובדן לאימון מודלי רגרסיה, משום שהוא נוח לניתוח מתמטי. אולם היחידות שלו הן יחידות המדידה בריבוע, ולכן פירוש גודל התוצאה פחות מיידי. שורש ה־MSE, כלומר RMSE, מחזיר אותנו ליחידות המקור.

4. חישוב ידני מלא: איזה מודל עדיף?

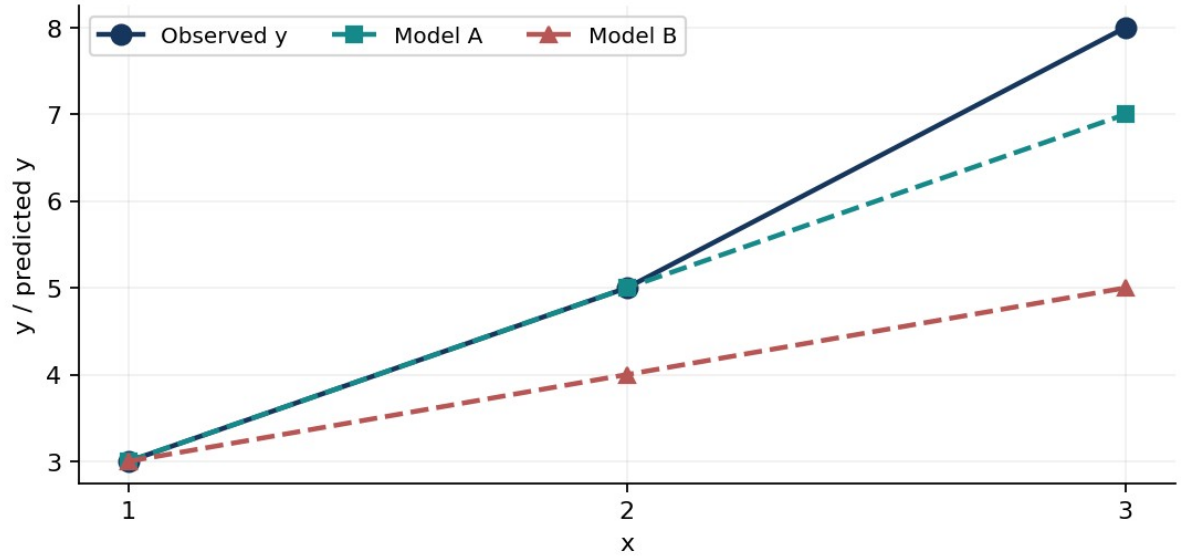
נשתמש בשלוש דוגמאות פשוטות. ערכי הקלט הם 1, 2, 3; ערכי המטרה הם 3, 5, 8. ננסה שני כללים: מודל A עם שיפוע 2 וחיתוך 1, ומודל B עם שיפוע 1 וחיתוך 2. נחשב תחזית לכל תצפית ולאחר מכן את השארית ואת ריבועה.

x	ערך אמיתי y	תחזית A: 2x+1	$(y-\hat{y}A)^2$	תחזית B: x+2	$(y-\hat{y}B)^2$
1	3	3	0	3	0
2	5	5	0	4	1
3	8	7	1	5	9
סכום			1		10

$$\mathbf{MSE(A) = (0 + 0 + 1) / 3 = 1/3 \approx 0.3333}$$

$$\mathbf{MSE(B) = (0 + 1 + 9) / 3 = 10/3 \approx 3.3333}$$

לכן על שלוש הדוגמאות האלה מודל A השיג MSE קטן יותר. אבל איננו יודעים מכאן שהוא יהיה טוב יותר לכל מחשב חדש. ייתכן שהנתונים קטנים מדי, וייתכן שהדוגמאות נבחרו באופן שאינו מייצג את האוכלוסייה. הבחנה זו תקפה לכל מדד: המדד מתאר את הביצועים על הנתונים שעליהם חושב.



איור: השוואת התחזיות של שני מודלים לאותן שלוש תצפיות. הפער בין קו לנקודת אמת הוא השגיאה.

```
import numpy as np
x = np.array([1, 2, 3])
y = np.array([3, 5, 8])
pred_a = 2 * x + 1
pred_b = x + 2
mse_a = np.mean((y - pred_a) ** 2)
mse_b = np.mean((y - pred_b) ** 2)
print("A:", mse_a)
print("B:", mse_b)
```

לפני ההרצה, נסו לחזות את המספרים שיתקבלו. לאחר מכן השוו בין הפלט לחישוב הידני. אם התוצאות אינן זהות, אל תחליפו מיד את החישוב: בדקו אילו ערכים נכנסו למערך ובאיזה שלב.

5. מה פירוש "אימון"? התאמת הפרמטרים

לאחר שבחרנו דרך למדוד שגיאה, אפשר לנסות לשנות את פרמטרי המודל כך שהשגיאה תקטן. יש אלגוריתמים שמוצאים את הפתרון באמצעות נוסחה; אחרים מתחילים מערכים כלשהם ומעדכנים אותם בהדרגה. לכן המילה "אימון" אינה מציינת בהכרח אותה שיטת חישוב בכל המודלים.

ברגרסיה ליניארית אפשר למצוא מקדמים באמצעות שיטת ריבועים פחותים. בהכנה ללמידה עמוקה חשוב להבין גם את הרעיון האיטרטיבי: מתחילים מניחוש, מחשבים שגיאה, בודקים באיזה כיוון שינוי הפרמטרים צפוי להקטין את השגיאה ומבצעים צעד קטן. ברשתות עצביות התהליך יהיה עשיר יותר ונלמד אותו בהמשך.

5.1 הדגמת צעד אחד של Gradient Descent — הרחבת גשר

נתחיל בדוגמה עם נתון יחיד כדי להתמקד במשמעות הפעולה. נניח שמודל פשוט מנבא $\hat{y}=wx$, כאשר $x=2$ והערך האמיתי $y=5$. נבחר תחילה $w=1$ ונחשב את השגיאה הריבועית.

$$L(w) = (2w - 5)^2$$

$$L(1) = (2 - 5)^2 = 9$$

כדי לדעת כיצד לשנות את המשקל נבחן את נגזרת פונקציית השגיאה. אין צורך בשלב זה לגזור פונקציות רבות; נשתמש בתוצאה ונפרש אותה. בפרק המתמטיקה של הלמידה העמוקה נלמד את הגזירה בצורה שיטתית.

$$dL/dw = 4(2w - 5)$$

$$dL/dw \mid (w=1) = -12$$

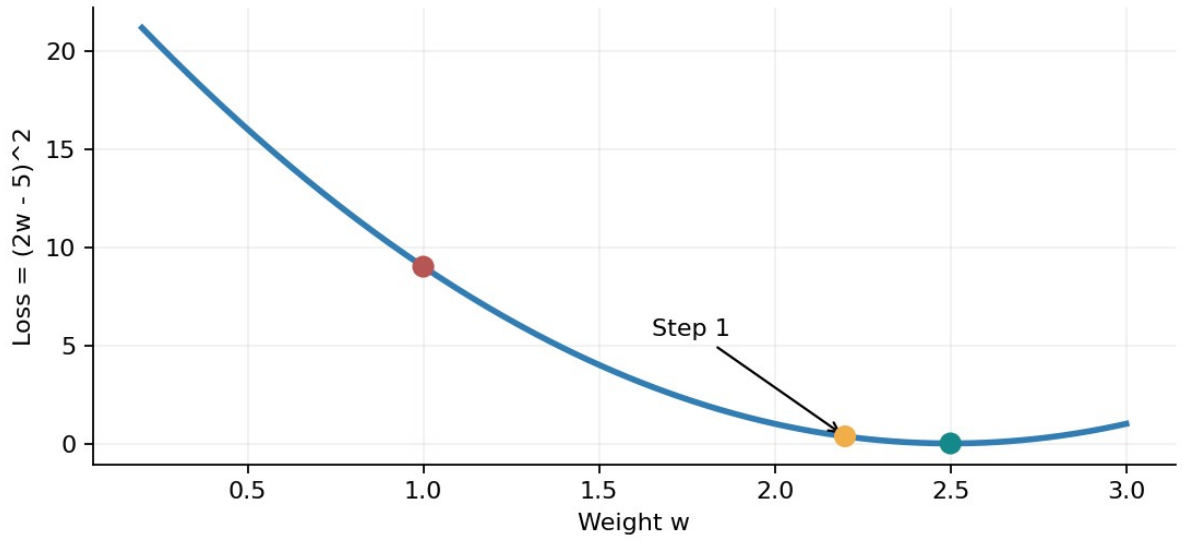
הנגזרת שלילית בנקודה זו. פירוש הדבר: אם נגדיל מעט את w, השגיאה אמורה לקטון. כלל העדכון מחסר מן המשקל את קצב הלמידה כפול הנגזרת; במקרה של נגזרת שלילית, החיסור גורם להגדלת המשקל.

$$w_{new} = w_{old} - \eta \, (dL/dw)$$

$$w_{new} = 1 - 0.1 \times (-12) = 2.2$$

$$L(2.2) = (4.4 - 5)^2 = 0.36$$

השגיאה ירדה מ־9 ל־0.36. זו דוגמה אחת בלבד, ולא הבטחה שכל גודל צעד ישפר תמיד את התוצאה. אם קצב הלמידה גדול מדי, נוכל לדלג מעל המינימום ואף להגדיל את השגיאה. זהו בדיוק אחד הנושאים שנחקור בהמשך הלמידה העמוקה.



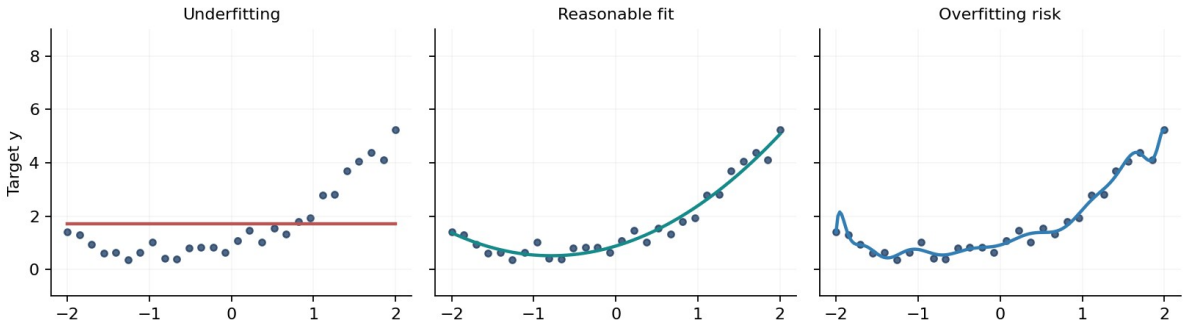
איור: פונקציית שגיאה כפונקציה של משקל יחיד. צעד לעבר המינימום עשוי להקטין את השגיאה.

```
w = 1.0
x = 2.0
y = 5.0
learning_rate = 0.1
loss_before = (w * x - y) ** 2
gradient = 2 * (w * x - y) * x
w = w - learning_rate * gradient
loss_after = (w * x - y) ** 2
print(w, loss_before, loss_after)
```

בדיקת הבנה: מה ישתנה אם קצב הלמידה יהיה 1 במקום 0.1? חשבו את הצעד ביד לפני שתריצו את הקוד, ונסו להסביר את התוצאה בעזרת גרף הפונקציה.

6. הצלחה באימון אינה הצלחה בעולם האמיתי

מודל עשוי לשנן דפוסים שמופיעים בנתוני האימון, כולל רעש, מקרים חריגים או פרטים ייחודיים שאינם חוזרים באוכלוסייה. אם הוא מצליח מאוד על נתוני האימון אבל נכשל על דוגמאות חדשות, יש חשד להתאמת יתר — Overfitting. מצב הפוך, שבו המודל פשוט מדי ואינו מזהה אפילו את הדפוס הבסיסי בנתונים, נקרא Underfitting.

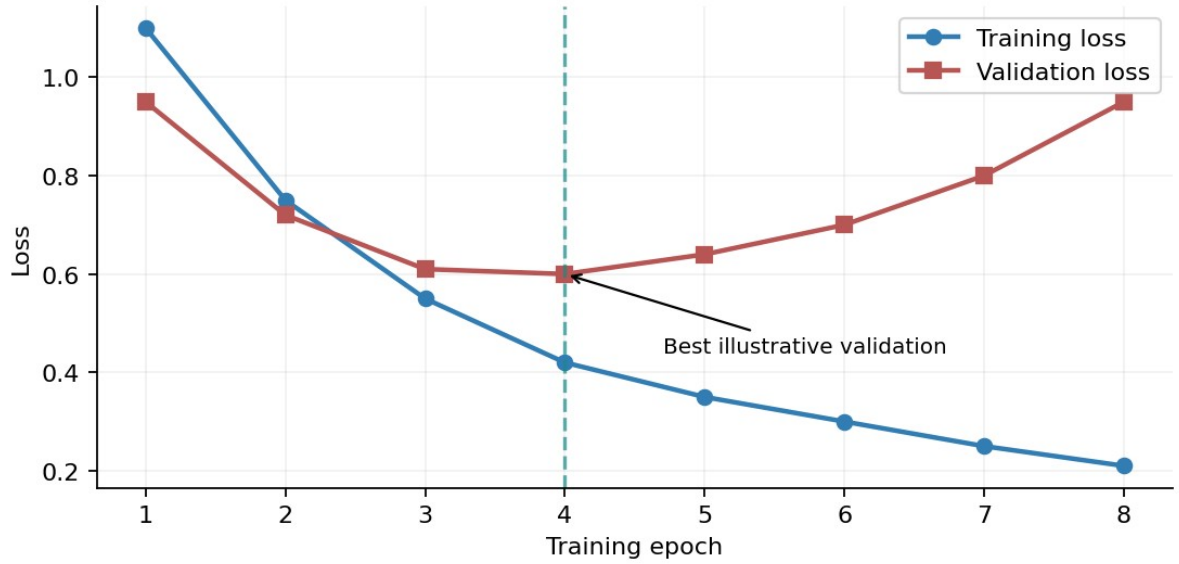


איור: שלוש דרגות התאמה: מודל פשוט מדי, מודל סביר ומודל מורכב מאוד שיכול לעקוב אחרי רעש המדגם.

חשוב להיזהר מהכללה: מודל מורכב אינו בהכרח מודל גרוע, ומודל פשוט אינו בהכרח טוב. איכות ההתאמה תלויה במורכבות הבעיה, בכמות הנתונים, ברמת הרעש ובאופן ההערכה. ההחלטה צריכה להסתמך על נתונים שאינם משמשים לבחירת הפרמטרים.

6.1 כיצד קוראים עקומות אימון?

במודלים מאומנים בתהליך איטרטיבי אפשר לצייר את השגיאה לאחר כל תקופת אימון (Epoch). כאשר שגיאת האימון יורדת ושגיאת הוולידציה יורדת יחד איתה, מתקבלת עדות לכך שהתהליך משפר את הכללת המודל. כאשר שגיאת האימון ממשיכה לרדת אך שגיאת הוולידציה מתחילה לעלות באופן עקבי, ייתכן שהמודל מתאים את עצמו לפרטי האימון מעבר לרצוי.



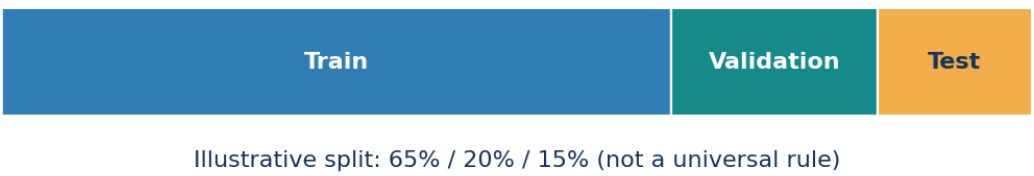
איור: עקומות אימון לדוגמה בלבד: לאחר תקופת אימון מסוימת, שגיאת הוולידציה עולה אף ששגיאת האימון יורדת.

הערה חשובה: עקומת אימון חריגה אחת אינה הוכחה מתמטית ל־Overfitting. יש לבחון כיצד נוצרו הקבוצות, האם חל שינוי בהתפלגות, האם ההבדלים יציבים בניסויים חוזרים והאם המדד מתאים לבעיה.

איזו בדיקה נבצע?	מה אפשר לחשוד?	דפוס
בדיקת מאפיינים, איכות נתונים ומורכבות	Underfitting או בעיית נתונים	שגיאת אימון גבוהה ושגיאת ולידציה גבוהה
בדיקה עצמאית, רגולריזציה או יותר נתונים	Overfitting אפשרי	אימון נמוך מאוד, ולידציה גבוהה
בדיקת Test חיצונית ובחינת מגבלות	הכללה טובה על הדגימה שנבדקה	אימון וולידציה דומים ונמוכים

7. מדוע מחלקים נתונים? אימון, בחירה ובדיקה

בפרק 5.2 של נושאי הליבה הכרנו Train, Validation ו־Test ואת K-Fold. כאן נשתמש בידע כדי לתכנן מערכת שלמה. קבוצת Train מאפשרת להתאים פרמטרים; Validation מסייעת לבחור מודל והיפר־פרמטרים; Test נועדה לתת הערכה סופית לאחר שהבחירות כבר התקבלו. אם נשתמש ב־Test שוב ושוב כדי לבחור מודלים, היא כבר לא תהיה מבחן עצמאי אמיתי.

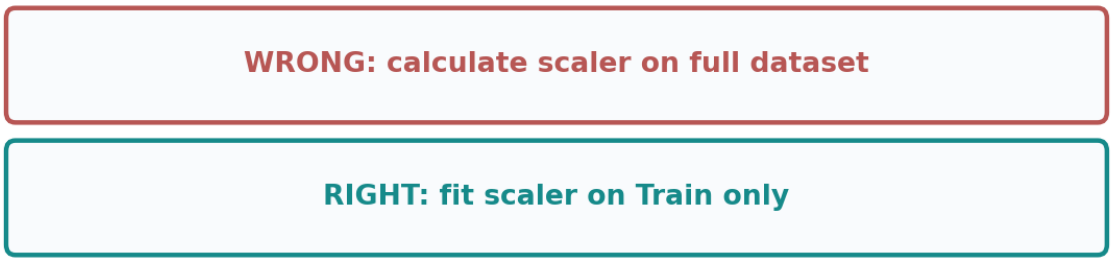


איור: חלוקת נתונים לדוגמה בלבד. אחוז החלוקה תלוי בכמות הנתונים, באיזון ובמבנה הזמן או הקבוצות.

אם יש כמה רשומות מאותו מחשב, צריך לשקול חלוקה לפי מזהה מחשב כדי שלא יהיו רשומות כמעט זהות גם באימון וגם בבדיקה. כאשר המטרה היא לחזות אירועים בעתיד, לעיתים נדרשת חלוקה לפי זמן: עבר לאימון ותקופה מאוחרת לבדיקה. חלוקה אקראית עלולה להיות בלתי מתאימה במקרה זה.

7.1 זליגת מידע — טעות שמציגה מודל מוצלח מדי

זליגת מידע מתרחשת כאשר מידע שלא היה זמין במועד החיזוי משפיע על בניית המודל או על הערכתו. לדוגמה, אם נשתמש במשך התיקון בפועל כאחד המאפיינים של המחשב, המודל יקבל במישרין או בעקיפין את התשובה. גם חישוב ממוצע וסטיית תקן על כל המאגר לפני חלוקה ל־Train/Test עלול לשלב מידע מן הבדיקה בתהליך האימון.



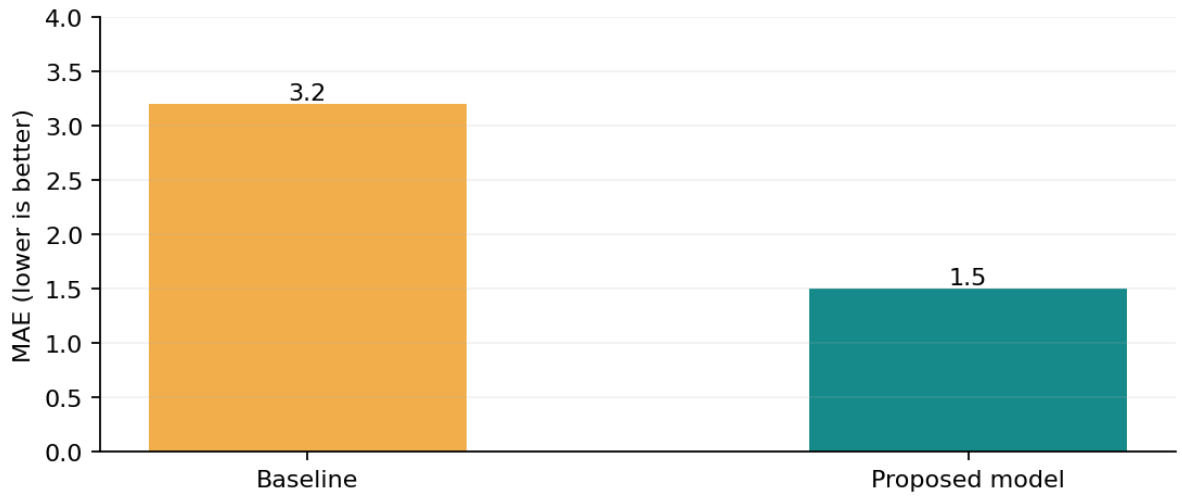
איור: התאמת פעולות עיבוד צריכה להישען על נתוני האימון בלבד; הנתונים האחרים עוברים באמצעות *transform* ההתאמה הקיימת.

הפתרון המקצועי הוא להפריד תחילה קבוצות ולהכניס פעולות עיבוד נלמדות ל־Pipeline. בתוך Cross-Validation הפעולות נלמדות מחדש על חלק האימון של כל קיפול. כך תוצאות ההערכה קרובות יותר למצב שבו המודל יראה דוגמה חדשה באמת.

8. מהו קו בסיס (Baseline) ומדוע הוא הכרחי?

מודל שמקבל ציון מסוים יכול להיראות מרשים, אך ללא השוואה אין לנו נקודת ייחוס. בבעיית רגרסיה אפשר לבנות קו בסיס שמנבא תמיד את ממוצע ערכי המטרה באימון. בבעיית סיווג אפשר לנבא תמיד את המחלקה הנפוצה ביותר, או להשתמש בכלל עסקי פשוט. קו הבסיס אינו אמור להיות חכם; הוא אמור להגדיר את הרף שהמודל החדש צריך להתעלות עליו.

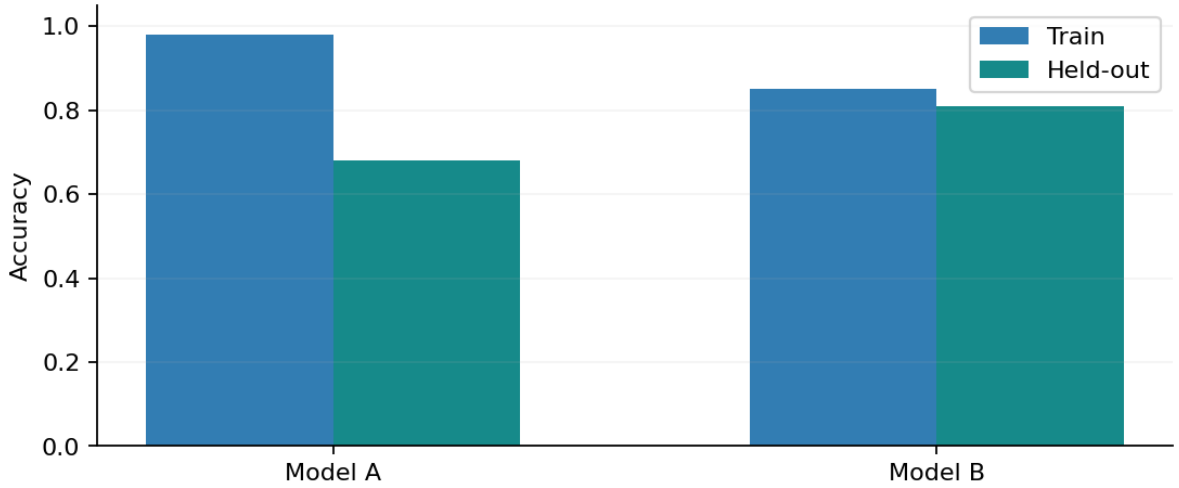
נניח ש־MAE של קו הבסיס הוא 3.2 יחידות זמן, ושל המודל שלנו הוא 1.5. זו ראיה לכך שהמודל עדיף לפי המדד ועל מערך הבדיקה שנבחר. אבל אם מודל חדש מסוכן יותר או יקר יותר להפעלה, השיפור המספרי לבדו אינו מבטיח שזו הבחירה הנכונה.



איור: השוואת *MAE* לדוגמה: תוצאת מודל חייבת להיבחן ביחס לקו בסיס ולמגבלות המדידה.

8.1 מקרה השוואה: דיוק אימון מול יכולת הכללה

לשני מודלים יש ציוני אימון ובדיקה שונים. מודל A משיג Accuracy של 98% באימון אך רק 68% בנתוני בדיקה חיצוניים; מודל B משיג 85% באימון ו־81% בבדיקה. אם שניהם נבחנו על אותן דוגמאות ובאותם תנאים, מודל B עדיף לפי Accuracy של הבדיקה. אך לפני שמכריעים סופית נבדוק אילו טעויות כל מודל עושה ואיזה מחיר יש לכל טעות.



איור: דוגמה להבחנה בין הצלחה באימון לביצועי הכללה. הנתונים באיור מומצאים לצורכי הוראה.

9. ניסוי מדעי במודל: איך משפרים בלי להטעות את עצמנו?

חקר מודלים אינו תחרות של הרצת כמה שיותר אלגוריתמים. ניסוי טוב מתחיל בהשערה ברורה: "אולי קנה המידה גורם לכך שמאפיין אחד משתלט על מרחקי KNN". נשנה גורם אחד, נמדוד תוצאה על אותם קיפולים או אותה קבוצת ולידציה, נתעד את התוצאה ונשקול הסברים חלופיים. שינוי כמה גורמים בבת אחת יקשה עלינו לדעת מה השפיע.



איור: מחזור ניסוי: השערה, שינוי מוגדר, מדידה, תיעוד והסבר מגבלות.

ניסוי	מה משנים?	מה מחזיקים קבוע?	מה מודדים?
1 — קו בסיס	סוג המודל	חלוקת הנתונים	MAE ומשך אימון
2 — תקנון	עם/בלי StandardScaler	אותו מודל ואותם קיפולים	MAE או Accuracy
3 — מורכבות	ערך k או מורכבות אחרת	אותם מאפיינים וקיפולים	Train/Validation
4 — הוגנות	בדיקת קבוצות נפרדות	מדד וסף החלטה	שגיאות לפי קבוצות

יומן ניסויים קצר צריך לכלול תאריך, שם מאגר, מספר דוגמאות, מאפיינים, שיטת חלוקה, seed, היפר־פרמטרים, מדדים והסבר למה למדנו מהתוצאה. בפרויקט גמר דרישות התיעוד אינן קישוט: הן מאפשרות לשחזר את הניסוי ולהגן על החלטות מקצועיות.

10. מעבדת Python מודרכת — קו בסיס לעומת רגרסיה

להלן נתוני הוראה סינתטיים. הקוד מפריד ל־Train/Test, בונה קו בסיס של ממוצע ומודל ליניארי, ומחשב MAE על אותה קבוצת בדיקה. הספר מסביר את התהליך במלואו; במחברת התלמיד הנלווית יהיו ניסויים חדשים להשלים ללא פתרונות.

```
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.dummy import DummyRegressor
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error

X = np.arange(1, 21).reshape(-1, 1)
y = 12 + 4 * X.ravel() + np.array([
    0, 1, -2, 1, 0, -1, 2, -2, 1, 0,
    -1, 1, 2, -1, 0, 1, -2, 2, 0, -1
])
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)
baseline = DummyRegressor(strategy="mean")
model = LinearRegression()
baseline.fit(X_train, y_train)
model.fit(X_train, y_train)
for name, estimator in [("Baseline", baseline), ("Linear", model)]:
    prediction = estimator.predict(X_test)
    print(name, mean_absolute_error(y_test, prediction))
```

הסבר: np.arange יוצר ערכי מאפיין מ־1 עד 20; reshape הופך אותם למטריצה של עשרים שורות ועמודה אחת, כי scikit-learn מצפה שמאפיינים יוגשו במבנה דו־ממדי. מערך y מכיל ערך בקירוב ליניארי עם רעש קטן. שתי השיטות מותאמות רק על Train ונבחנות על אותם X_test ו־y_test. השוואת שתי התוצאות היא הוגנת מבחינת קבוצת הבדיקה — אך אם נבחר מודל בעקבותיה, אסור לדווח על אותו Test כאילו היה מבחן סופי עצמאי.

למה השתמשנו במאגר קטן ומומצא? כדי לבודד את הרעיון המתמטי ולבדוק תחילה שקוד הבסיס פועל. בניסוי אמיתי חייבים להבין כיצד הנתונים נאספו, האם דוגמאות עצמאיות זו מזו, מה מייצג כל מאפיין ומהן ההשלכות של טעות חיזוי.

11. מתי מודל מצליח מספרית אך לא מתאים לפרויקט?

נניח שמודל מסווג מחשבים פגומים ומחמיץ מעט תקלות חמורות, אך מסמן כמעט כל מחשב כתקול. Recall עשוי להיות גבוה, אך השירות לא יהיה שימושי. לחלופין, מודל יכול להצליח בבדיקה פנימית אך להיכשל בשבוע הבא בגלל החלפת חומרה, שינויים בסביבת העבודה או מדידות חדשות. לכן פיתוח מערכת AI כולל גם בדיקת איכות הנתונים, שיקולי שימוש, פרטיות ובקרה לאחר ההטמעה.

11.1 ארבע שאלות שמפתחים חייבים לשאול

- האם הנתונים מייצגים את מי או את מה שהמערכת תפגוש בעתיד?
- האם המאפיינים יהיו ידועים באמת בזמן החיזוי?
- מהי עלות טעות מהסוג החמור יותר, והאם המדד שבחרנו משקף אותה?
- האם אדם יוכל להבין מתי המודל אינו בטוח או מתי צריך להעביר החלטה לבדיקה אנושית?

12. קשר ישיר לפרק הבא: פרספטרון ורשתות עצביות

בפרקים הקודמים השתמשנו במסווגים שמנבאים על סמך קרבה של דוגמאות או מרכזי מחלקות. עכשיו אנחנו מוכנים להכיר מודל שמשתמש במשקולות: עבור וקטור מאפיינים x ומשקולות w, מחשבים סכום משוקלל ומוסיפים הטיה b. במסווג בינארי אפשר להעביר את התוצאה דרך כלל החלטה.

$$z = w \cdot x + b$$

הרעיון החדש אינו עצם המכפלה הפנימית, שכבר למדנו בפרק הווקטורים. הרעיון החדש הוא כיצד משנים משקולות בעקבות שגיאות. בפרספטרון נלמד כלל עדכון המיועד לסיווג; בהמשך נלמד אופטימיזציה המבוססת על נגזרות, ובפרקים על רשתות נרחיב את העיקרון למספר שכבות ול־Backpropagation. כך כל נושא חדש נשען על מושגים שכבר הבנו.

13. טעויות נפוצות ותיקונן

כיצד מתקנים?	טענה שגויה
הביצועים באימון אינם מוכיחים הכללה; דרושה הערכה חיצונית.	אם Accuracy באימון הוא 100%, המודל מצוין.
בחלק מהמודלים פרמטרים נלמדים מתוך נתונים; היפר־פרמטרים מכוונים בנפרד.	כל פרמטר במודל נבחר מראש.
שניהם מתארים שגיאה אך מענישים גדלים שונים באופן שונה.	MSE ו־MAE מודדים אותו הדבר.
שימוש ב־Test לבחירה פוגע בעצמאותה של ההערכה הסופית.	המודל שניצח על Test הוא הסופי.
מורכבות עשויה גם להתאים רעש ולפגוע בביצועים מחוץ לאימון.	ככל שמוסיפים מורכבות, המודל תמיד משתפר.
ייתכן ששגיאת ולידציה לא תרד או אף תעלה.	כשיש ירידה בשגיאת אימון, יש שיפור אמיתי.
התאמת התקנון חייבת להתבצע על Train בלבד.	אפשר לתקנן את כל המאגר ואז לחלק.
יש להתחשב גם בעלות, יציבות, הוגנות ודרישות המערכת.	שיפור מספרי קטן מצדיק שימוש במודל.

14. תרגול מדורג — ללא פתרונות

רמה א׳ — מושגים וחישובים

- א1. הסבירו בשתי פסקאות את ההבדל בין מודל לבין אלגוריתם אימון, ובחרו דוגמה מתוך רגרסיה.
- א2. עבור $y=(4,6,8)$ ותחזיות $\hat{y}=(5,5,8)$, חשבו את השאריות, MAE ו־MSE, וציינו את יחידות המדדים.
- א3. סווגו את המונחים הבאים לפרמטר, היפר־פרמטר או מאפיין: מספר שכנים, מספר התקלות, שיפוע רגרסיה, קצב למידה.
- א4. מה ההבדל בין תחזית של מודל לבין הערך האמיתי? מדוע איננו משתמשים בתווית האמיתית בזמן חיזוי של מקרה חדש?

רמה ב׳ — פרשנות, ניסויים וגרפים

- ב1. נתונות שתי עקומות: שגיאת אימון יורדת מ־1.2 ל־0.2, ושגיאת ולידציה יורדת תחילה ל־0.6 ואז עולה ל־0.9. הסבירו מה ייתכן שהתרחש.
- ב2. במאגר יש כמה רשומות לכל מחשב. הציעו חלוקה ל־Train ול־Test שמצמצמת זליגה.

• ב3. הציעו קו בסיס לבעיה של חיזוי זמן תיקון, והסבירו מדוע ממוצע האימון הוא בחירה סבירה להשוואה.

• ב4. מדוע אין לכוון עשרים היפר־פרמטרים על בסיס תוצאה חוזרת של אותה קבוצת Test?

• ב5. תכננו ניסוי המבודד את השפעת התקנון על KNN. אילו דברים חייבים להיות זהים בשתי ההרצות?

רמה ג׳ — תכנון מערכת וחשיבה ביקורתית

• ג1. מערכת חיזוי תקלות נבדקה על מחשבים משנת 2024, וכעת מותקנים מחשבים מדגם חדש. איזה סיכון להכללה נוצר? תכננו בדיקת ניטור.

• ג2. מודל A משיג Accuracy 0.90 אך מחמיץ יותר תקלות קשות ממודל B שמשיג 0.85. מה עוד צריך לדעת כדי לבחור?

• ג3. קוד מתקנן את כל X לפני train_test_split. זהו את הבעיה וכתבו בפסאודו־קוד את סדר הפעולות הנכון.

• ג4. בנו טבלת ניסויים של שלוש גרסאות מודל. כללו השערה, שינוי מבוקר, מדדים, מסקנה ומגבלה.

15. שאלת הערכה מקורית בסגנון בגרות

דודו בונה מערכת שמנבאת את משך התיקון של מחשב חדש. הוא בדק מודל ליניארי על שלוש תצפיות: הערכים האמיתיים הם (10, 12, 16), והתחזיות הן (9, 14, 15). המודל אומן על נתונים אחרים, אך במהלך בנייתו השתמש דודו בקבוצת הבדיקה כדי לבחור את קצב הלמידה.

• א. חשבו את שאריות החיזוי ואת MAE. הציגו את שלבי החישוב.

• ב. חשבו את MSE והסבירו מדוע הוא מעניש יותר שגיאה גדולה.

• ג. הסבירו מדוע הבחירה בקצב למידה בעזרת Test יוצרת בעיה בהערכת המודל.

• ד. הציעו תהליך חלוקה ובחירה מתוקן, וציינו מה ידווח בסוף.

• ה. ציינו סיכון אחד שעשוי להתעורר כאשר משתמשים במודל בבית ספר אחר.

הערה: זו שאלה מקורית שנכתבה עבור הספר לצורכי הכנה, ואינה מועתקת מבחינת בגרות רשמית. תשובות וקריטריונים לניקוד יופיעו בגרסת המורה.

16. מעבדת Colab — מהספר אל ניסוי עצמאי

מחברת התלמיד הנלווית מכילה 12 משימות: זיהוי מאפיינים ומטרות; השוואת תחזיות; חישוב MAE ו־MSE; שינוי משקל; ניסוי בקצב למידה; מודל קו בסיס; הפרדת Train/Test; בדיקת Leakage; השוואת מודלים; בניית יומן ניסויים; ניתוח עקומות; וסיכום להערכה בעל־פה. במקומות המיועדים לעבודה יופיע TODO או תא חסר. אין במחברת פתרונות מלאים למשימות.

התוצר הסופי של המעבדה אינו רק מחברת שרצה ללא שגיאות. על התלמיד להגיש שני גרפים, טבלה אחת של ניסויים ופסקה שבה הוא מסביר מדוע בחר במודל מסוים, מה הראיות לכך ומה אינו יכול להסיק מהנתונים.

17. מפת קשרים בין שלוש התכניות

שימוש עתידי	מה כבר נלמד ומה העומק כאן	מסלול
ניתוח בסיסי והסבר אלגוריתמים	היכרות עם אימון, חיזוי, מדדים והכללה	יחידה 3 — כיתה י׳
הצדקת תהליך מדעי והכנה	פונקציות מרחק, מדדי הצלחה,	נושאי ליבה — יחידה 5

שימוש עתידי	מה כבר נלמד ומה העומק כאן	מסלול
לבגרות	ולידציה וחקר היפר־פרמטרים	
פרספטרון, Gradient Descent, רשתות ופרויקט גמר	תהליך האימון כמערכת של פרמטרים, שגיאה, בחירה ובדיקה	התמחות למידה עמוקה — פרק 8

עוגן התכנית: במפת ההתמחות שבמאגר GitHub מופיע פרק 8 ״מושגי יסוד בלמידת מכונה״ בהיקף 10 שעות, בין KNN לפרספטרון. סדר תתי־הנושאים בפרק זה הוא עיבוד דידקטי שלנו, ולא העתק מאומת של תתי־סעיפי ה־PDF הרשמי.

18. הצעת חלוקת זמן להוראה — 10 שעות

שעות	מוקד	שיעור
2	מודל, נתונים, תחזית, פרמטר והיפר־פרמטר	1
2	שאריות, MAE, MSE, קו בסיס וחישוב ידני	2
2	אימון, שינוי פרמטרים, העמקת Overfit/Underfit וקריאת עקומות	3
2	הכללה, זליגת מידע, ניסוי מבוקר ופרשנות	4
2	מעבדת Python, שאלת הערכה, יומן ניסויים והכנה לפרספטרון	5

זוהי חלוקת שעות מוצעת למורה (שעה לימודית = 45 דקות), לא חלוקה רשמית שהועתקה מהסילבוס.

19. מילון מושגים

מונח	פירוש קצר
Model	פונקציה המפיקה תחזית מקלט
Training	תהליך התאמת פרמטרים מנתוני אימון
Parameter	ערך שהמודל מתאים בהתאם לנתונים
Hyperparameter	הגדרה המכוונת את תהליך האימון או צורת המודל
Loss	פונקציה המודדת את הפער בין תחזית לתוצאה
Epoch	מעבר אחד על נתוני האימון, לפי שיטת האימון
Generalization	יכולת לתפקד היטב על נתונים חדשים
Overfitting	התאמת יתר שעלולה לפגוע בהכללה
Baseline	כלל פשוט המשמש אמת מידה להשוואה
Data Leakage	שימוש במידע שלא אמור להיות זמין בעת האימון או החיזוי
Reproducibility	יכולת לשחזר ניסוי בתנאים מתועדים

20. חקר מקרה מסכם — האם המודל החדש באמת עדיף?

דודו אסף שישה מחשבים ותיעד עבור כל אחד מהם את מספר התקלות ואת זמן התיקון בפועל. כדי להמחיש את הבדיקה בלי להסתבך במאגר גדול, נתוני האימון הם ארבעת המחשבים הראשונים, ונתוני

הבדיקה הם שני מחשבים חדשים. הערכים פשוטים ומומצאים לצורך המחשה. בניסוי אמיתי אסור לבחור חלוקה כזו רק מפני שהיא גורמת למודל להצליח.

קבוצה	מספר תקלות x	זמן תיקון בפועל y
Train	1	20
Train	2	24
Train	3	28
Train	4	32
Test	5	34
Test	6	38

20.1 קו הבסיס — תחזית אחת לכולם

נגדיר Baseline שמנבא תמיד את ממוצע ערכי המטרה של האימון. שימו לב שאיננו משתמשים כלל בשני נתוני Test לחישוב הממוצע. הם שמורים לבדיקת הכללה.

mean(Train) = (20 + 24 + 28 + 32) / 4 = 26

לכן שתי תחזיות קו הבסיס הן 26. עבור המחשב הראשון ב־Test, שהזמן האמיתי שלו הוא 34, הטעות המוחלטת היא 8. עבור המחשב השני, שהזמן האמיתי שלו הוא 38, הטעות היא 12. ממוצע הטעויות המוחלטות הוא:

MAE_baseline = (8 + 12) / 2 = 10

20.2 מודל שלומד את הקשר בנתוני Train

כעת נסתכל על ארבע דוגמאות האימון בלבד. כל תוספת של תקלה אחת מלווה בעלייה של ארבע יחידות זמן. ישר שעובר בכל ארבע הנקודות מקיים את המשוואה הבאה:

ŷ = 4x + 16

בבדיקה החיצונית המודל מנבא 36 למחשב עם חמש תקלות ו־40 למחשב עם שש תקלות. הערכים האמיתיים הם 34 ו־38. לכן בכל אחת משתי הדוגמאות הטעות המוחלטת היא 2:

MAE_model = (2 + 2) / 2 = 2

המודל הצליח יותר מקו הבסיס על שתי דוגמאות הבדיקה האלה. זהו ממצא, אך לא הוכחה שהוא תמיד יצליח. המדגם קטן מאוד, שתי נקודות ה־Test הן מחוץ לטווח ערכי x של Train — כלומר יש כאן גם אקסטרפולציה — והקשר במחשבים אחרים עלול להיות שונה. ההבחנה בין ממצא לבין מסקנה כללית היא חלק מהחשיבה המדעית שאנחנו רוצים לפתח.

20.3 שלוש שאלות של בוחן פרויקט גמר

- מדוע אינך יכול לקבוע שהמודל שלך מדויק תמיד, אף ש־MAE שלו קטן?
- מה היה קורה לו היית משתמש גם בשני מחשבי Test לצורך בחירת השיפוע?
- איזה נתון אמיתי היית אוסף כדי לבדוק אם מספר התקלות מספיק לחיזוי משך התיקון?

21. דף תכנון ניסוי לתלמיד

לקראת מיני־פרויקט או פרויקט גמר, כל תלמיד יוכל להשתמש בטבלה הבאה כדי לתכנן ניסוי. לפני שרצים לקוד, ממלאים את העמודה הראשונה והשלישית. לאחר ההרצה ממלאים את התוצאות והפירוש. כך שומרים על ניסוי שניתן לשחזר ועל שיקול דעת שקוף.

מה אכתוב ביומן הניסוי?	השאלה
למשל: האם StandardScaler משפר את KNN על הנתונים שלי?	מהי מטרת הניסוי?
מקור המאגר, מספר רשומות, מאפיינים, דרך איסוף ורישוי	מהם הנתונים?
קבוצת Test, אותם קיפולים, אותו מדד ואותם מאפיינים	מה קבוע?
גורם יחיד — למשל תקנון עם/בלי	מה אני משנה?
מדד איכות ובמידת הצורך זמן חישוב או עלות	מה אני מודד?
מסקנה מוגבלת לנתונים שנבדקו והסבר חלופי אפשרי	מה למדתי?
בדיקה נוספת, איסוף נתונים, שיפור או דחיית הרעיון	מה הצעד הבא?

דוגמה לניסוח מסקנה מקצועית: ״בשלושה קיפולי ולידציה על נתוני האימון, התקנון שיפר את ממוצע הדיוק של KNN. לכן בחרנו להמשיך לבדוק מודל עם תקנון. עם זאת, איננו יודעים עדיין אם השיפור יישמר עבור נתונים ממחשבים מדגם חדש, ונידרש לבחון זאת במועד מאוחר יותר.״

דוגמה לניסוח שאינו מקצועי: ״התקנון תמיד משפר KNN, ולכן כל המודלים צריכים תקנון״. משפט כזה הופך תוצאה מקומית לכלל אוניברסלי בלי נתונים שמצדיקים זאת.

22. סיכום מושגי — מה חשוב לקחת לפרק הפרספטרון?

בפרק זה ראינו שמודל מפיק תחזית באמצעות פרמטרים, שתהליך האימון מתאים את הפרמטרים לנתונים, ושפונקציית השגיאה מאפשרת להגדיר מה נחשב שיפור. למדנו ששיפור על נתוני האימון אינו מספיק, ולכן מפרידים בין אימון, בחירה והערכה סופית. בדקנו כיצד זליגת מידע, חלוקה לא נכונה וקנה מידה עלולים לגרום לנו להעריך מודל באופן מוטעה.

בפרק הבא נפגוש את הפרספטרון. במקום לזכור רק נוסחת סיווג חדשה, נחפש בו את אותם מרכיבים: מהו וקטור הקלט? מהם המשקולות וההטיה? מהי התחזית? מתי מתרחשת טעות? כיצד מתקנים את המשקולות? באילו נתונים בודקים שהשינוי הועיל? הקישור בין השאלות הללו הופך את הלמידה ממאגר פקודות להבנה של מערכת לומדת.

מקורות מקצועיים ועוגני קריאה

עוגן תכנית: משרד החינוך — התמחות בלמידת מכונה ולמידה עמוקה, פרק 8 (10 שעות), על פי מפת התכנית השמורה במאגר Data-Science-ML-Course; נדרש אימות סופי של תתי־הסעיפים מול PDF הרשמי המעודכן.

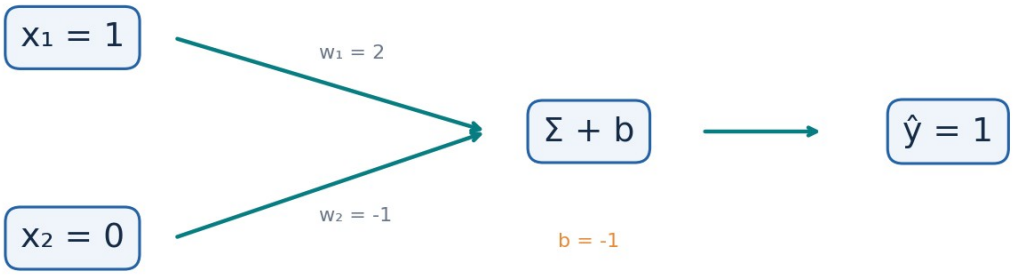
מקורות מרכזיים להשראה מקצועית: Luis Serrano, Grokking Machine Learning; Aurélien Géron, Hands-On Machine Learning; James et al., An Introduction to Statistical Learning with Applications in Python; Deisenroth et al., Mathematics for Machine Learning; Goodfellow, Bengio & Courville, Deep Learning. ההסברים והדוגמאות כאן מקוריים ולא הועתקו מהספרים.

לבחינת המימושים והטעויות: scikit-learn User Guide (Common Pitfalls); Google Machine Learning Crash Course; מאגר הקורס: github.com/samiabo75/Data-Science-ML-Course.

פרק 9

הפרספטרון — ממכפלה פנימית למודל שלומד

ספר לימוד לתלמיד | התמחות בלמידת מכונה ולמידה עמוקה (5 יחידות) | 30 שעות לפי מפת התכנית



איור: הפרספטרון מקבל וקטור מאפיינים, מחשב סכום משוקלל ומחליט על מחלקה.

המטרה שלנו אינה רק לדעת להפעיל פונקציה. נלמד לתאר את ההחלטה שהמודל מקבל, לחשב אותה ביד, להבין מדוע הוא טועה, לעקוב אחר עדכון המשקולות ולהסביר כיצד הוא משתפר לאורך האימון.

מפת למידה

מה אבין ומה אבצע	צעד
מהו נוירון מלאכותי ומה ההבדל בין חיזוי לאימון	1
כיצד פועלים משקולות, הטיה ופונקציית מדרגה	2
כיצד מחשבים z = 0 ותחזית, כולל מקרה z	3
כיצד מעדכנים משקולות בעקבות שגיאה	4
כיצד שינוי משקולות מזיז את קו ההחלטה	5
לא נלמד XOR נלמד ו־ AND למה שער בפרספטרון יחיד	6
כיצד מממשים אלגוריתם עצמאי ומשווים ל־scikit-learn	7
כיצד בודקים יכולת הכללה ומציגים ניסוי בפרויקט	8

ידע קודם: וקטורים, מכפלה פנימית, משוואת ישר, סיווג, NumPy, נתוני X ו־y, Train/Test. ידע על נגזרות אינו תנאי לפרק זה.

1. בעיית הפתיחה: מערכת פשוטה מקבלת החלטה

דודו רוצה לבנות מערכת שתסייע במיון פניות למעבדת המחשבים. לכל פנייה יש שני מאפיינים בינריים: האם המחשב אינו נדלק, והאם הופיעה הודעת שגיאה קריטית. המערכת תחזיר תווית אחת מתוך שתי אפשרויות: 1 — נדרש טיפול דחוף; 0 — אין סימון אוטומטי כדחוף. אלה נתונים מדומים לצורכי לימוד, ולא מדיניות בטיחות אמיתית.

אפשר להתחיל בכתיבת כללים ידניים: אם שני הסימנים מופיעים יחד, סמן דחוף. אבל מערכת למידת מכונה אמורה לקבל דוגמאות שכבר סווגו ולכוונן את הכללים המספריים שלה. הפרספטרון הוא אחד המודלים הפשוטים ביותר שממחישים את העיקרון הזה.

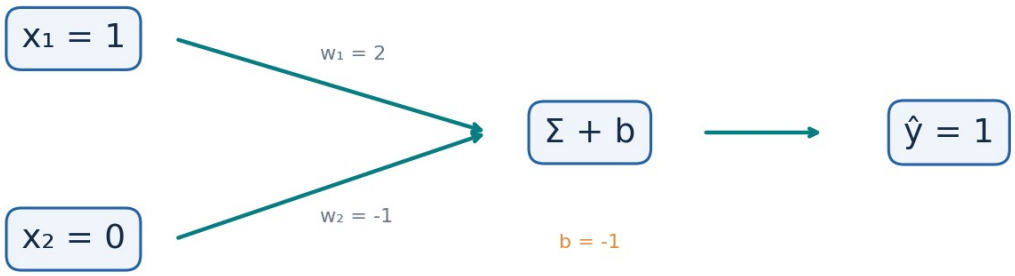
x_1 מחשב לא נדלק	x_2 שגיאה קריטית	y תווית
0	0	0
0	1	0
1	0	0
1	1	1

הטבלה ממחישה את שער AND: רק כאשר שתי התכונות שוות 1, התווית היא 1. בהמשך נשתמש דווקא בדוגמה הפשוטה הזאת כדי לראות כל חישוב וכל תיקון — בלי נתונים רבים שמסתירים את הרעיון.

2. מהו פרספטרון ומה הוא אינו?

פרספטרון הוא מסווג ליניארי בינרי: הוא יוצר ציון על סמך שילוב ליניארי של המאפיינים, ואחר כך ממיר אותו להחלטת מחלקה. חשוב להבחין בין שלושה מושגים: המודל הוא מבנה החישוב; המשקולות וההטיה הם הפרמטרים; אלגוריתם האימון הוא הכלל שמשנה את הפרמטרים לאחר טעויות.

פרספטרון אינו מודל שמפיק הסתברות רק משום שהתוצאה שלו היא 0 או 1. גם אם הציון גדול מאוד, אין משמעות הדבר שהמודל בטוח ב־99%. ציון z הוא ערך של פונקציית החלטה, לא הסתברות מכוילת.



איור: תרשים קלט–חישוב–החלטה; ההטיה מאפשרת להזיז את גבול ההחלטה.

3. מהתכונות לווקטור המאפיינים

בפרק הווקטורים למדנו שלכל תצפית אפשר להצמיד רשימת מספרים באותו סדר. בפרספטרון המספרים נקראים מאפיינים, ולכל מאפיין מוצמד משקל. אם יש שני מאפיינים, נכתוב את הקלט

בתור וקטור $x=(x_1,x_2)$ ואת המשקולות בתור $w=(w_1,w_2)$. אם יש d מאפיינים, שני הווקטורים הם באורך d.

$$x = (x_1, x_2, \ldots, x_d) \qquad w = (w_1, w_2, \ldots, w_d)$$

המשקל מתאר את התרומה הליניארית של המאפיין לציון ההחלטה, כאשר שאר הדברים נשארים קבועים. משקל חיובי מגדיל את הציון כשהמאפיין גדל; משקל שלילי מקטין אותו. אך אין לפרש את המשקל כסיבה בעולם האמיתי או להשוות גדלי משקולות בלי להביא בחשבון את יחידות המדידה.

4. חישוב הציון z — פירוק הנוסחה

$$z = w \cdot x + b = \Sigma(i=1..d)\ w_i x_i + b$$

הסימן · הוא מכפלה פנימית: כופלים רכיבים מתאימים ומחברים את כל המכפלות. האות b היא ההטיה (bias), כלומר מספר שמוסיפים לסכום המשוקלל. ההטיה מאפשרת להזיז את קו ההחלטה גם אם שני המשקולות נשארות ללא שינוי.

דוגמה פתורה: נניח $x=(1,0)$, $w=(2,-1)$, $b=-1$. מציבים כל רכיב במקום שלו:

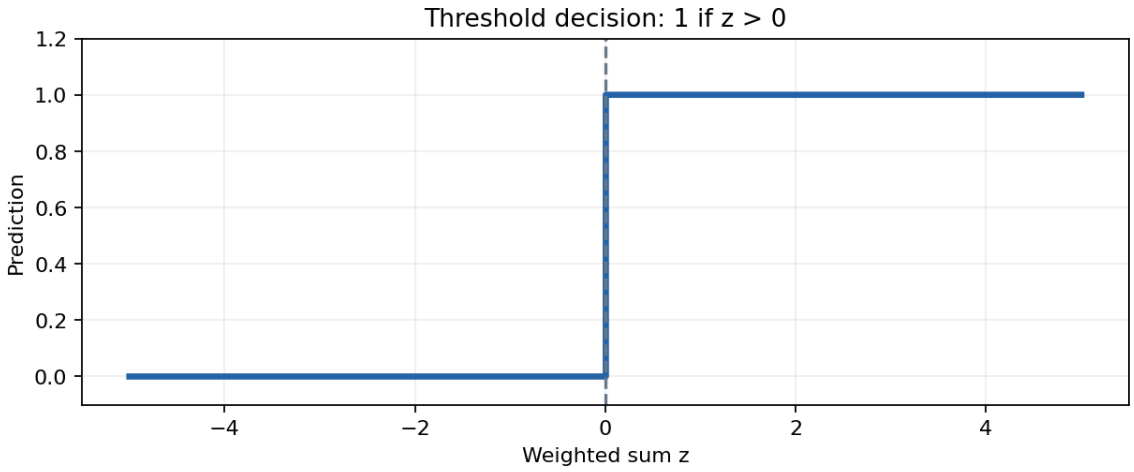
$$z = 2\cdot 1 + (-1)\cdot 0 - 1 = 1$$

קיבלנו $z=1$. זהו ציון חיובי, ולכן לפי כלל ההחלטה שנאמץ בפרק, התחזית תהיה 1. שימו לב: לא שינינו אף משקל. ביצענו חיזוי בלבד; אימון הוא תהליך נפרד.

5. פונקציית מדרגה: איך ציון הופך לתווית?

$$\hat{y} = 1 \ \text{ if } z > 0; \qquad \hat{y} = 0 \ \text{ if } z \le 0$$

בחרנו כלל חד־משמעי גם למקרה הגבולי $z=0$: במקרה זה התחזית היא 0. בחירת הכלל חשובה במיוחד בטבלאות מעקב, משום שהיא משפיעה על השאלה אם נדרשת פעולת עדכון. קיימות הגדרות אחרות בספרים ובספריות, ולכן יש לציין בכל דוגמה איזה כלל משמש אותנו.



איור: פונקציית ההחלטה קופצת מ־0 ל־1 כאשר z עובר מעל אפס.

בדיקת הבנה: אם נשנה רק את ההטיה מ־-1 ל־-3, מה יקרה לציון בדוגמה הקודמת? האם משתנים המאפיינים? האם התווית האמיתית השתנתה?

6. גבול ההחלטה — המשמעות הגיאומטרית

במישור של שני מאפיינים, כל הזוגות המקיימים $z=0$ נמצאים על ישר. מצד אחד של הישר נקבל $z>0$, ומצדו השני $z\le 0$. לכן פרספטרון יחיד יוצר גבול החלטה ליניארי. במרחב של שלושה מאפיינים מתקבל מישור; בממד גבוה יותר — על־מישור.

$$w_1x_1 + w_2x_2 + b = 0$$

$$x_2 = -(w_1/w_2)x_1 - b/w_2 \qquad (\text{when } w_2 \neq 0)$$

כדי לשרטט את הקו, מוצאים שתי נקודות שמקיימות את המשוואה ומחברים ביניהן. אם $w_2=0$, אין לחלק בו: הגבול הוא ישר אנכי בתנאי $w_1\neq 0$. השיפוע הוא $-w_1/w_2$ והחיתוך עם ציר x_2 הוא $-b/w_2$.

עיקרון חשוב: קו ההחלטה אינו חייב לעבור דרך הנקודות של הדוגמאות. תפקידו להפריד מחלקות ולא לבצע רגרסיה על ערכי התווית.

7. מהי טעות סיווג וכיצד מודדים אותה?

לכל דוגמת אימון יש תווית אמיתית y ותחזית $\hat{y}$. טעות מתרחשת כאשר שני הערכים שונים. אפשר לספור את מספר הטעויות בכל מעבר על נתוני האימון. בשונה מרגרסיה, כאן אין מנסים לקרב ערך מספרי רציף אל y באמצעות ישר; מטרתנו היא לשנות את ההחלטה של המודל עבור דוגמאות שסווגו לא נכון.

$$\text{error} = y - \hat{y}$$

פעולה	error	חזוי $\hat{y}$	אמיתי y
לא מעדכנים	0	0	0
לא מעדכנים	0	1	1
מגדילים ציון	+1	0	1
מקטינים ציון	−1	1	0

הטבלה קושרת בין סוג השגיאה לכיוון התיקון. כאשר החמצנו דוגמה חיובית, יש צורך להגדיל את הציון שלה. כאשר סימנו בטעות דוגמה שלילית כחיובית, צריך להקטין את הציון שלה.

8. כלל הלמידה של הפרספטרון

$$w_{new} = w_{old} + \eta(y - \hat{y})x$$

$$b_{new} = b_{old} + \eta(y - \hat{y})$$

האות היוונית η (אֶטָה) מייצגת קצב למידה — מספר חיובי שקובע את גודל הצעד. אם התחזית נכונה, ההפרש $y-\hat{y}$ הוא אפס ולכן לא יהיה שינוי. אם התחזית שגויה, נכפיל את וקטור הקלט ב־η ונוסיף או נחסיר אותו בהתאם לכיוון הטעות.

מדוע מעדכנים גם את b? משום שהגבול אינו תלוי רק בזווית הישר אלא גם במיקומו. שינוי המשקולות לבדו עלול להשאיר את הקו במקום לא מתאים. בהדגמות נשתמש ב־$\eta=1$ כדי לפשט את החישובים; אין פירוש הדבר שזה תמיד קצב הלמידה הטוב ביותר.

דוגמת עדכון יחיד: משקולות התחלתיות $w=(0,0)$, הטיה $b=0$, דוגמה $x=(1,1)$ עם $y=1$. הציון $z=0$ ולכן $\hat{y}=0$, ונוצרה שגיאה $+1$.

$$w_{new} = (0,0) + 1\cdot(+1)\cdot(1,1) = (1,1)$$

$$b_{new} = 0 + 1\cdot(+1) = 1$$

כעת אותה דוגמה תקבל $z=1+1+1=3$ ותסווג כחיובית. אך אסור להסיק מכך שכל הדוגמאות מסווגות נכון: שיפור בדוגמה אחת עלול לפגוע בסיווג דוגמה אחרת.

9. טבלת מעקב מלאה — איך לומדים שער AND?

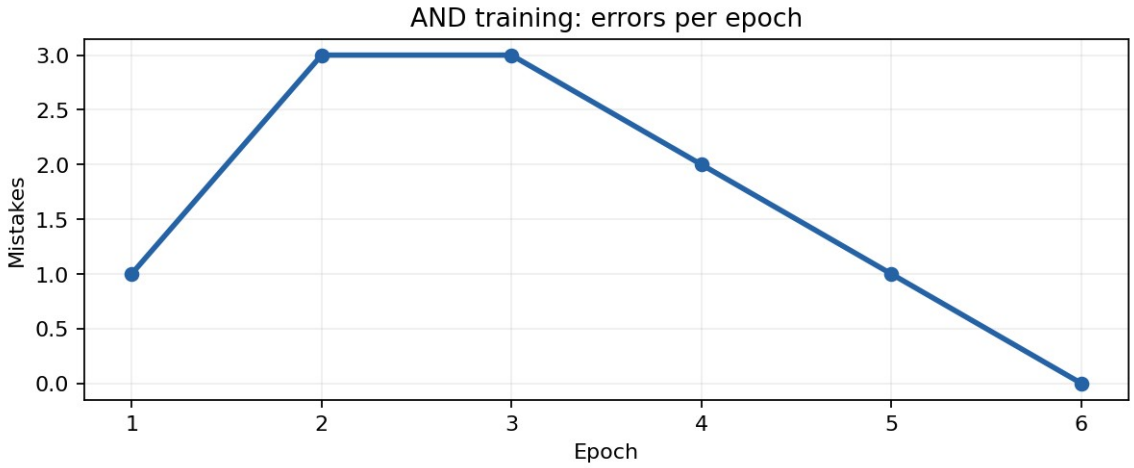
נאמץ סדר מעבר קבוע על ארבע הדוגמאות: (0,0), (0,1), (1,0), (1,1). מתחילים עם w=(0,0), b=0 ו־η=1. בכל שורה מחשבים z באמצעות הערכים הנוכחיים, מפיקים תחזית לפי z>0, מחשבים y−ŷ ורק אז מעדכנים את המשקולות וההטיה. את הערכים החדשים מעבירים לשורה הבאה.

הטבלה הבאה מציגה במלואם את שני המעברים הראשונים. השורה שמסומנת ״לפני״ היא המשקולות וההטיה לפני חיזוי הדוגמה; ״אחרי״ היא המצב לאחר העדכון. המשך התהליך מופיע באיור ובניסוי הקוד.

מעבר	x	y	z	ŷ	y−ŷ	לפני (w₁,w₂,b)	אחרי (w₁,w₂,b)
1	(0, 0)	0	0	0	0	(np.int64(0), np.int64(0), 0)	(0, 0, 0)
1	(0, 1)	0	0	0	0	(np.int64(0), np.int64(0), 0)	(0, 0, 0)
1	(1, 0)	0	0	0	0	(np.int64(0), np.int64(0), 0)	(0, 0, 0)
1	(1, 1)	1	0	0	1	(np.int64(0), np.int64(0), 0)	(1, 1, 1)
2	(0, 0)	0	1	1	-1	(np.int64(1), np.int64(1), 1)	(1, 1, 0)
2	(0, 1)	0	1	1	-1	(np.int64(1), np.int64(1), 0)	(1, 0, -1)
2	(1, 0)	0	0	0	0	(np.int64(1), np.int64(0), -1)	(1, 0, -1)
2	(1, 1)	1	0	0	1	(np.int64(1), np.int64(0), -1)	(2, 1, 0)

פענוח השורה האחרונה במעבר הראשון: מכיוון ש־(1,1) אמורה להיות חיובית אך קיבלה תחזית 0, מעדכנים ל־w=(1,1), b=1. בתחילת המעבר השני הדוגמה (0,0) מקבלת בטעות תחזית 1, ולכן מתקנים את ההטיה ל־0.

חשוב לראות שעדכון אינו מבטיח ירידה מיידית במספר הטעויות בכל שורה או בכל מעבר. האלגוריתם מגיב למקרים בזה אחר זה; כאשר הנתונים ניתנים להפרדה ליניארית, קיימת לו הבטחת התכנסות בתנאים המתאימים.



איור: מספר שגיאות במעברים 1–6 עבור נתוני AND בסדר קבוע; במעבר השישי לא היו טעויות.

תוצאה נבדקת: בסוף התהליך מתקבלות משקולות w=(2,1), הטיה b=−2; כל ארבע הדוגמאות מקבלות את התוויות הנכונות לפי כלל z>0.

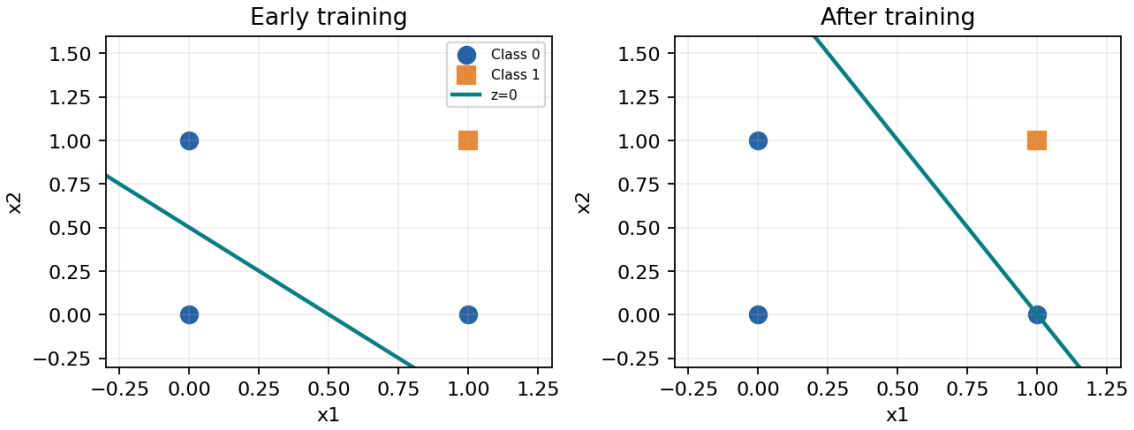
10. מהו מעבר (Epoch), ומה קובע סדר הנתונים?

Epoch הוא מעבר אחד על כל דוגמאות האימון. התהליך יכול לדרוש מספר מעברים, משום שעדכון שבוצע בדוגמה מאוחרת עשוי לשנות את הסיווג של דוגמה שכבר נבדקה במעבר הנוכחי. לכן בודקים שוב את כל הדוגמאות.

סדר הדוגמאות יכול להשפיע על סדר העדכונים ועל המשקולות הסופיות. לשם שחזור מדויק של ניסוי חשוב לשמור סדר קבוע או לקבוע seed כאשר מערבבים את הדוגמאות. הפרספטרון אינו מבטיח למצוא את אותם משקולות בכל ריצה, גם כשקיים יותר מקו אחד שמפריד את המחלקות.

11. כיצד רואים את הלמידה על הגרף?

כל עדכון משנה את המשוואה z=0 ולכן משנה את המיקום או הכיוון של גבול ההחלטה. בתחילת האימון הקו יכול לעבור בצד הלא נכון של חלק מהנקודות. ככל שמתקבלות טעויות ומבוצעים תיקונים, הקו יכול לנוע עד שכל הדוגמאות מופרדות כראוי.



איור: שתי נקודות זמן בהתקדמות האימון. הגבול משתנה עם w ו־b; קו ההפרדה הסופי אינו יחיד בהכרח.

דיון: אם נכפיל את כל המשקולות ואת ההטיה בקבוע חיובי, האם התחזיות ישתנו? לא — סימן z נשמר. אבל אם נכפיל בקבוע שלילי, הסימן מתהפך והמחלקות עלולות להתחלף. זהו חיבור ישיר ללימוד וקטורים וכיוונים.

12. חישוב ידני מאפס: אימון עם שתי תכונות ממשיות

נחזור לסיפור מעבדת המחשבים, אך הפעם נשתמש במאפיינים מספריים שאינם בינריים: שעות מאז התקלה ומספר הודעות שגיאה. נניח שהנתונים הותאמו לקנה מידה סביר, ויש לנו משקולות נתונות w=(0.5,−0.25) והטיה b=−0.5.

עבור מחשב שתכונותיו x=(2,1), הציון הוא 0.5×2 − 0.25×1 − 0.5 = 0.25 ולכן התחזית היא 1. אם התווית האמיתית דווקא 0, טעות החיזוי היא −1. עבור η=0.2 נעדכן את כל רכיבי המשקל באופן יחסי למאפיינים.

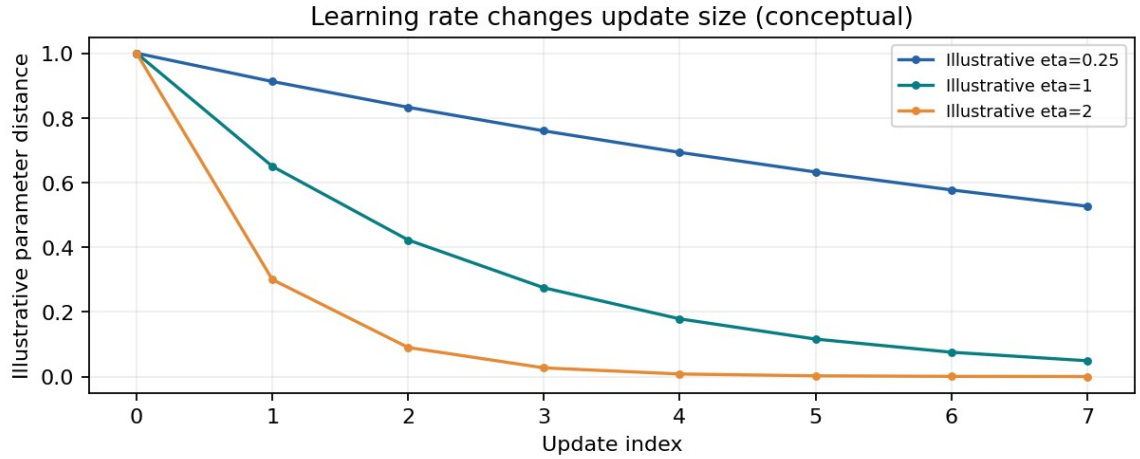
$$w_{new} = (0.5, -0.25) + 0.2(-1)(2,1) = (0.1, -0.45)$$

$$b_{new} = -0.5 + 0.2(-1) = -0.7$$

אם נחשב שוב את הציון לאותה דוגמה, נקבל 0.1×2 − 0.45×1 − 0.7 = −0.95, ולכן התחזית תשתנה ל־0. התיקון בכיוון הנדרש הצליח בדוגמה הזאת; עדיין יש לבדוק את יתר הדוגמאות ואת נתוני הבדיקה.

13. קצב למידה η — מה משפיע עליו?

קצב למידה קטן יוצר צעדים קטנים יותר בכל תיקון, וקצב למידה גדול יוצר צעדים גדולים יותר. חשוב לא להחיל באופן אוטומטי את ההסבר של Gradient Descent על הפרספטרון הקלאסי: כאן התיקון מופעל בדרך כלל בעקבות טעות סיווג, ולא על ידי חישוב נגזרת של MSE.



איור: המחשה רעיונית בלבד של גודל צעד; התרשים אינו ניסוי אמיתי של אימון פרספטרון.

בנתונים הניתנים להפרדה ליניארית, שינוי קצב הלמידה הקבוע והחיובי יכול לשנות את מסלול האימון ואת מקדמי ההפרדה. במימוש מעשי בודקים זאת בניסוי; לא קיימת הבטחה שכל הגדלה של η משפרת את המודל.

14. מבט לתוך הקוד: אימון פרספטרון ללא ספרייה

הקוד הבא מממש בדיוק את כלל ההחלטה שקבענו: תחזית 1 רק כאשר z>0. הוא משתמש ב־NumPy, אבל אינו קורא לפונקציית אימון מוכנה. בכל מעבר סופרים את מספר הטעויות; אם לא היו טעויות, עוצרים.

```
import numpy as np
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]], dtype=float)
y = np.array([0, 0, 0, 1])
w = np.zeros(X.shape[1], dtype=float)
b, eta = 0.0, 1.0

for epoch in range(20):
```

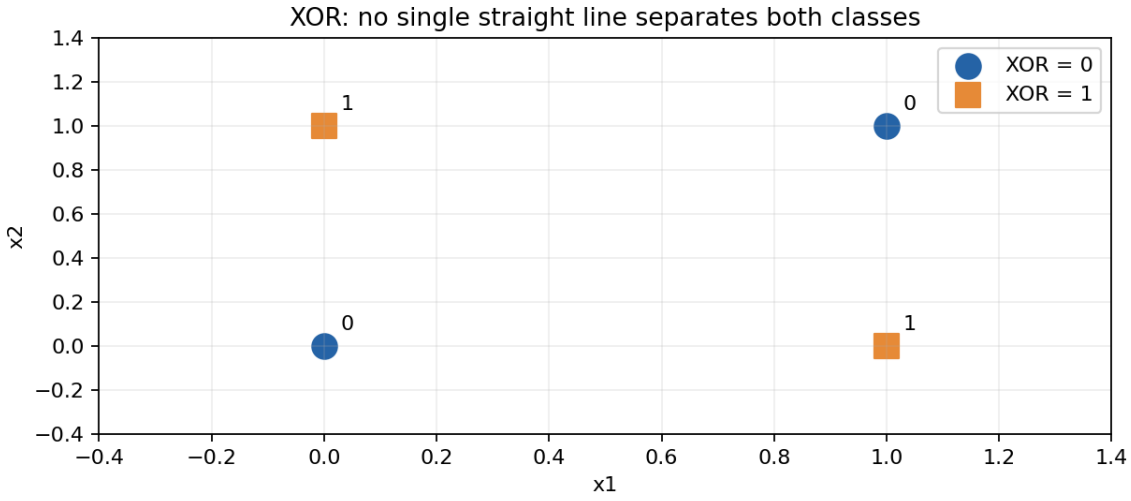
```
    errors = 0
    for i in range(len(X)):
        z = np.dot(w, X[i]) + b
        prediction = int(z > 0)
        delta = int(y[i]) - prediction
        w = w + eta * delta * X[i]
        b = b + eta * delta
        if delta != 0:
            errors += 1
    print(epoch + 1, errors, w, b)
    if errors == 0:
        break
```

שימו לב למיקום השורות: תחילה `z`, אחריו `prediction`, ורק אחר כך שינוי `w` ו־`b`. אם מעדכנים לפני שחושבו התוצאה והטעות, משנים את משמעות האלגוריתם. `X.shape[1]` מחזיר את מספר המאפיינים, ולכן אותו קוד מתאים גם ליותר משני מאפיינים.

תוצאה צפויה לנתוני AND ולסדר שנקבע: שישה מעברים; מספר הטעויות בכל מעבר הוא 1, 3, 3, 2, 1, 0. נסו להסביר את המספרים בעזרת טבלת המעקב, ולא רק לראות את הפלט.

15. למה אי אפשר ללמוד XOR באמצעות פרספטרון יחיד?

הביטוי XOR מחזיר 1 כאשר בדיוק אחד משני הקלטים הוא 1, ומחזיר 0 כאשר שניהם שווים. בארבע הנקודות במישור, שתי המחלקות מסודרות באלכסונים מנוגדים. שום קו ישר אחד אינו מסוגל להפריד את שתי הנקודות החיוביות משתי הנקודות השליליות.



איור: נקודות XOR משני סוגים באלכסונים מנוגדים; אין קו ישר שיפריד את המחלקות.

x_1	x_2	AND	OR	XOR
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

אפשר גם להוכיח את הבעיה בעזרת אי־שוויונים, בלי להסתמך רק על הציור. נסמן $z = w_1x_1 + w_2x_2 + b$ ונשתמש בהכרעה 1 כאשר $z > 0$. עבור XOR תנאי ארבע הדוגמאות מחייבים:

$$b \le 0 \;;\quad w_1 + b > 0 \;;\quad w_2 + b > 0 \;;\quad w_1 + w_2 + b \le 0$$

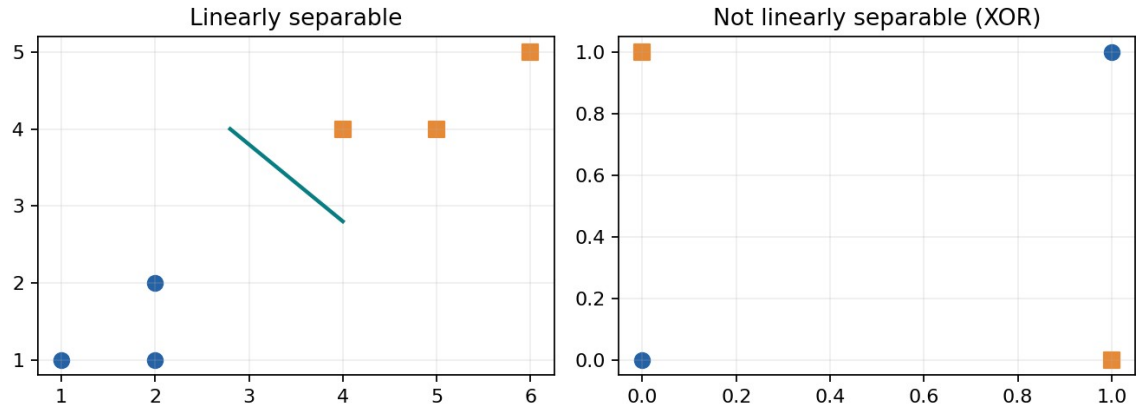
משני האי־שוויונים האמצעיים נקבל $w_1 > -b$ וגם $w_2 > -b$, ולכן $w_1 + w_2 + b > -b$. מאחר ש־$b \leq 0$, נקבל $-b \geq 0$, ולכן $w_1 + w_2 + b > 0$. זו סתירה לתנאי האחרון. לכן לא קיימים משקולות והטיה שיסווגו נכון את ארבע הדוגמאות באמצעות פרספטרון יחיד.

זו הסיבה שבפרק הרשתות העצביות נלמד לשלב מספר יחידות חישוב, כולל שכבות חבויות ואי־ליניאריות. יותר שכבות לבדן אינן מספיקות אם אין פעולות לא־ליניאריות בין השכבות.

16. מה אומר משפט ההתכנסות — ומה לא?

משפט ההתכנסות של הפרספטרון קובע שבתנאים מתאימים, עבור קבוצת אימון סופית שניתן להפריד ליניארית עם שוליים חיוביים ודוגמאות בעלות נורמה חסומה, אלגוריתם הפרספטרון הקלאסי יגיע לאחר מספר סופי של טעויות לגבול שמפריד נכון את דוגמאות האימון.

המשפט אינו מבטיח שהמודל יצליח על דוגמאות חדשות, אינו מחייב שהמשקולות יהיו יחידות ואינו חל על XOR כבעיה דו־ממדית ללא הרחבת מאפיינים. אם הנתונים אינם ניתנים להפרדה ליניארית, האימון עשוי להמשיך לטעות ולכן נהוג לקבוע מספר מעברים מרבי ולעקוב אחרי ביצועים על נתוני ולידציה.



איור: הפרדה ליניארית לעומת דפוס שאינו ניתן להפרדה בקו יחיד.

17. מה ההבדל בין פרספטרון לבין רגרסיה ליניארית?

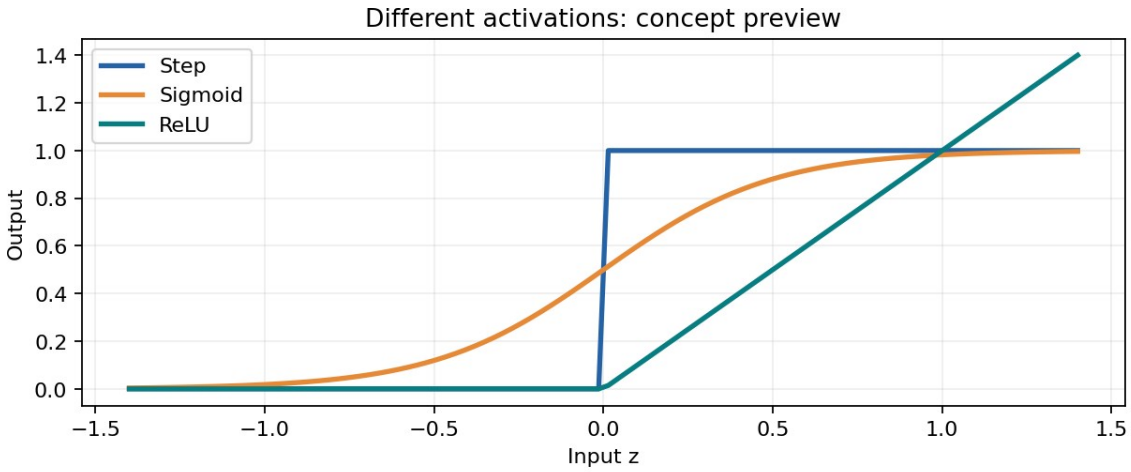
פרספטרון	רגרסיה ליניארית	נושא
סיווג למחלקה בינרית	חיזוי ערך מספרי	**מטרה**
z = w·x + b; threshold(z)	$\hat{y} = w \cdot x + b$	**ביטוי מרכזי**
תווית 0 או 1	מספר רציף	**פלט**
תיקון בעקבות סיווג שגוי	מזעור שגיאה מספרית	**אימון**
Accuracy, Precision, Recall, F1	MAE, MSE, R^2	**מדדי הערכה**

שני המודלים משתמשים באותו סכום משוקלל, אך שונים במשמעות הפלט ובמטרת האימון. זו נקודת חיבור חשובה בין פרק הרגרסיה שכבר למדנו לבין הרשתות העצביות שנלמד בהמשך.

18. פונקציית הפעלה לעומת פונקציית שגיאה

פונקציית הפעלה הופכת ציון ביניים לפלט של יחידת החישוב. בפרספטרון הבסיסי השתמשנו בפונקציית מדרגה. פונקציית שגיאה, לעומת זאת, מודדת עד כמה תוצאת המודל אינה עומדת במטרת האימון. אלו שני תפקידים שונים.

בפרקי הלמידה העמוקה נלמד פעולות כמו Sigmoid ו־ReLU, ונעמיק באימון באמצעות נגזרות ו־Backpropagation. אין לטעון שהפרספטרון הבינרי הקלאסי מתאמן באמצעות נגזרת של פונקציית המדרגה: הפונקציה הזאת אינה גזירה בנקודת המעבר והנגזרת שלה אפס כמעט בכל מקום אחר.



איור: השוואה חזותית של מדרגה, Sigmoid ו־ReLU — טעימה לקראת הרשת העמוקה.

19. מימוש מקצועי בעזרת scikit-learn

אחרי שכתבנו את האלגוריתם בעצמנו נוכל להשתמש במימוש הספרייה. מטרת ההשוואה אינה לשנן פקודות, אלא להבחין בין מה שאנחנו שולטים בו בניסוי לבין מה שהספרייה עושה אוטומטית: אתחול, ערבוב דוגמאות, מספר מעברים ועצירה.

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.linear_model import Perceptron
from sklearn.metrics import accuracy_score, confusion_matrix

iris = load_iris()
X = iris.data[iris.target != 2, :2]
y = iris.target[iris.target != 2]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=42
)
model = make_pipeline(StandardScaler(), Perceptron(
    max_iter=1000, tol=1e-3, random_state=42
))
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print('Accuracy:', accuracy_score(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))
```

מדוע בחרנו שתי מחלקות בלבד? מפני שהדוגמה מסבירה פרספטרון בינרי; ספריות יכולות להרחיב מסווגים ליניאריים גם לריבוי מחלקות, אבל ההרחבה אינה זהה לפרספטרון יחיד בעל פלט בינרי. מדוע שמנו `StandardScaler` בתוך Pipeline? משום שחישוב הממוצע וסטיית התקן חייב להתבצע על נתוני האימון בלבד.

שימו לב: אין לצפות שזהות מלאה של מקדמי המודל או סדר התיקונים תתקבל בין המימוש הידני שלנו לבין `Perceptron` של scikit-learn. חלק מהגדרות האימון שונות, ולכן נשווה תחזיות וביצועים באותו פרוטוקול ניסוי, לא את כל צעדיו הפנימיים.

20. מדוע Accuracy גבוה אינו הוכחה למודל מוצלח?

אם רוב הדוגמאות שייכות למחלקה 0, מודל שמנבא תמיד 0 יכול להשיג Accuracy גבוה, אך להחמיץ את כל האירועים החשובים של מחלקה 1. לכן עלינו לשמור נתוני Test שלא השתתפו באימון או בבחירת ההגדרות ולבדוק מטריצת בלבול, Precision, Recall ו־F1 לפי משמעות סוגי השגיאה.



Select hyperparameters without using Test labels

איור: הפרדה בין התאמת המודל, בחירת הגדרות והערכה סופית על נתונים שלא נחשפו בתהליך הפיתוח.

בתכנון פרויקט גמר יש לתעד את מקור המידע, סוגי המאפיינים, מי סיפק תוויות, כיצד התחלקו הנתונים ומה מחיר טעות מכל סוג. שיפור דיוק על אותם נתוני אימון בלבד אינו הוכחה לשיפור המודל בעולם האמיתי.

21. שלושה כשלים נפוצים ודרכי זיהוי

- בלבול בין חיזוי לאימון: מחשבים z בעזרת w,b קיימים וחושבים שהמודל כבר למד. הפתרון: לתעד לפחות שינוי אחד בפרמטרים בעקבות שגיאה.
- בלבול במקרה z=0: משתמשים פעם אחת ב־0≤ ופעם אחרת ב־0<. הפתרון: לנסח כלל אחד בתחילת השאלה ולדבוק בו לאורך טבלת המעקב.
- שינוי קנה מידה ללא תיעוד: הכפלת מאפיין באלף משנה את תרומתו לציון. הפתרון: להבין יחידות, לבחור תקנון כאשר מתאים ולבצע fit רק על Train.
- הנחה ש־XOR יילמד עם מספיק מעברים: עוד מעבר לא משנה את העובדה שאין גבול ליניארי מתאים. הפתרון: בדקו תחילה היתכנות הפרדה ליניארית.
- הצגת תוצאת Test כמדד לשיפור לאחר שכבר בחרנו לפיה מודל: זו זליגת החלטות. הפתרון: Train/Validation לבחירה ו־Test לבדיקה הסופית.

22. תרגול מדורג — ללא פתרונות

רמה א: הבנה וחישוב

- א. נתון x=(2,1), w=(−1,3), b=−1. חשבו z ואת התחזית לפי הכלל z>0.
- ב. לאותה דוגמה התווית האמיתית היא 0. עבור η=0.5 חשבו w_new ו־b_new.
- ג. הסבירו במילים את ההבדל בין משקל לבין הטיה, וציירו שני גבולות החלטה השונים רק בהטיה.
- ד. נתון z=0. מה תהיה התחזית בספר שלנו? מדוע חייבים לציין זאת מראש?

רמה ב: טבלת מעקב וניתוח

- ה. התחילו ב־$w=(0,0), b=0, \eta=1$ ועברו על OR בסדר הדוגמאות הנתון. רשמו שתי איטרציות לפחות.
- ו. עבור הנקודות $(1,1)$, $(2,2)$ במחלקה 1 ו־$(-1,-1)$,$(-2,-2)$ במחלקה 0, הציעו קו הפרדה ונמקו.
- ז. תלמיד כתב רק $w=w+\eta(y-\hat{y})x$ והשאיר את b קבוע. כיצד זה משפיע על אפשרויות ההפרדה?
- ח. בדקו קוד שבו מבוצע `np.argsort` על z במקום על תחזיות. האם יש כאן בכלל שלב הנחוץ לאימון פרספטרון? נמקו.

רמה ג: חשיבה מדעית ופרויקט

- ט. נסו להציע קו יחיד שמפריד XOR. הסבירו מדוע כל ניסיון נכשל, ובנו טיעון מתמטי.
- י. השוו ניסיון עם נתוני אימון בלבד לניסיון עם Train/Test ושימרו את כל התוצאות. מה ההבדל בין שתי הטענות שאפשר להסיק?
- יא. תכננו ניסיון על נתוני Iris עם שתי מחלקות: אילו מאפיינים תבחרו, איך תבדקו הכללה ואילו טעויות חשוב לנתח?
- יב. תלמיד מצא Accuracy של 97% אך Recall למחלקה החיובית הוא 30%. תארו מקרה שימוש שבו המודל אינו מתאים.

23. שאלת הערכה מקורית בסגנון בגרות

נתונות ארבע דוגמאות של שער AND בסדר שבטבלה שבתחילת הפרק. מתחילים עם $w=(0,0), b=0, \eta=1$. לפי כלל $z>0$ מתקבלת תחזית 1; אחרת 0. השיבו על הסעיפים הבאים בלי להריץ קוד לפני השלמת החישוב הידני:

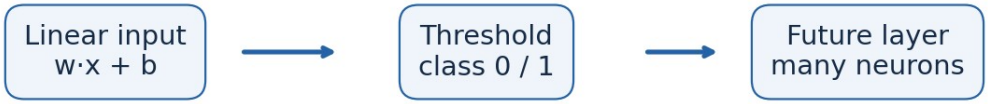
- א. חשבו את z, התחזית והשינוי בפרמטרים עבור הדוגמה הראשונה והדוגמה הרביעית במעבר הראשון.
- ב. הסבירו מדוע תחילת המעבר השני עשויה לכלול שגיאות אף שבסוף המעבר הראשון תוקנה הדוגמה החיובית.
- ג. ציינו מה פירוש Epoch וכתבו כלל עצירה מתאים לנתוני אימון שניתנים להפרדה.
- ד. עבור $w=(2,1), b=-2$, בדקו את כל ארבעת הקלטים והסבירו מדוע התקבלה הפרדה נכונה.
- ה. תלמיד טען שאם נוסיף עוד 100 מעברים אפשר יהיה ללמד את XOR באמצעות פרספטרון יחיד. נמקו מדוע זו טעות.
- ו. הציעו בדיקה שמאפשרת להעריך האם מודל שלמד נתוני אימון מתמודד גם עם דוגמאות שלא ראה.

הערה: זוהי שאלה מקורית לצורכי אימון בסגנון בגרות, לא שאלה שהופיעה בבחינה רשמית.

24. משימת פרויקט גמר: ממודל פשוט למערכת ניתנת לבדיקה

בפרויקט AI אמיתי, הפרספטרון יכול לשמש Baseline — מודל פשוט שמולו בודקים פתרון מורכב יותר. מומלץ להתחיל בבעיה שבה יש תוויות אמינות ומספר קטן של מאפיינים מספריים, להשתמש בחלוקת נתונים הוגנת ולהסביר את משמעות המשקולות ומגבלות הקו הליניארי.

כדי להגיש ניסוי שאפשר לשחזר, הכינו כרטיס ניסוי: תיאור הבעיה, מקור הנתונים וזכויות שימוש, גודל Train/Test, מאפיינים, קנה מידה, הגדרות המודל, מדדים, מטריצת בלבול, דוגמאות שגיאה ותובנות. אל תחשפו פרטי תלמידים אמיתיים או מידע מזהה במאגר ציבורי.



איור: הפרספטרון הוא יחידת בניין רעיונית בדרך לרשתות בעלות מספר נוירונים ושכבות.

25. סיכום מושגים וידע נדרש להמשך

מושג	מה חייבים להסביר
Feature	מספר המייצג מידע קיים על תצפית
Weight	מקדם הקובע תרומה לציון ליניארי
Bias	הטיה שמשנה את מיקום הגבול
z	ציון לפני ההחלטה; לא הסתברות
Threshold	כלל הממיר ציון לתווית
η	גודל צעד בתיקון בעקבות טעות
Epoch	מעבר מלא על כל דוגמאות האימון
Linear separability	קיום גבול ליניארי שמפריד את דוגמאות האימון
Generalization	ביצוע על נתונים שלא שימשו לאימון

אם אתם יכולים לחשב תחזית ידנית, לעדכן משקולות, לשרטט את הגבול ולהסביר את בעיית XOR — אתם מוכנים לעבור לשאלות המרכזיות בפרקי מתמטיקה ללמידה עמוקה ורשתות עצביות: מהי פונקציית שגיאה גזירה, כיצד מחשבים נגזרת, ואיך מאמנים כמה שכבות ביחד?

26. מקורות, עוגן תכנית והבחנה בין חובה להרחבה

עוגן תכנית: התמחות למידת מכונה ולמידה עמוקה, פרק 9 — פרספטרון, 30 שעות לפי מפת ההתמחות במאגר הקורס. חיבור קודם: יחידה 3, שיעור פרספטרון ו־SVM; פרקים 1.1–1.2 (וקטורים ומרחקים); פרקים 4.1–4.4 (סיווג); פרקים 5.1–5.2 (הערכה ולידציה).

ספרי עזר מקצועיים ששימשו השראה לאופן ההצגה: Grokking Machine Learning מאת Luis Serrano; Mathematics for Machine Learning מאת Deisenroth, Faisal ו־Dive into Deep Learning מאת Zhang, Lipton, Li ו־Understanding Deep Learning מאת Simon Prince. הקוד המוכן וההתנהגות של Perceptron נבדקו גם מול התיעוד הרשמי של scikit-learn.

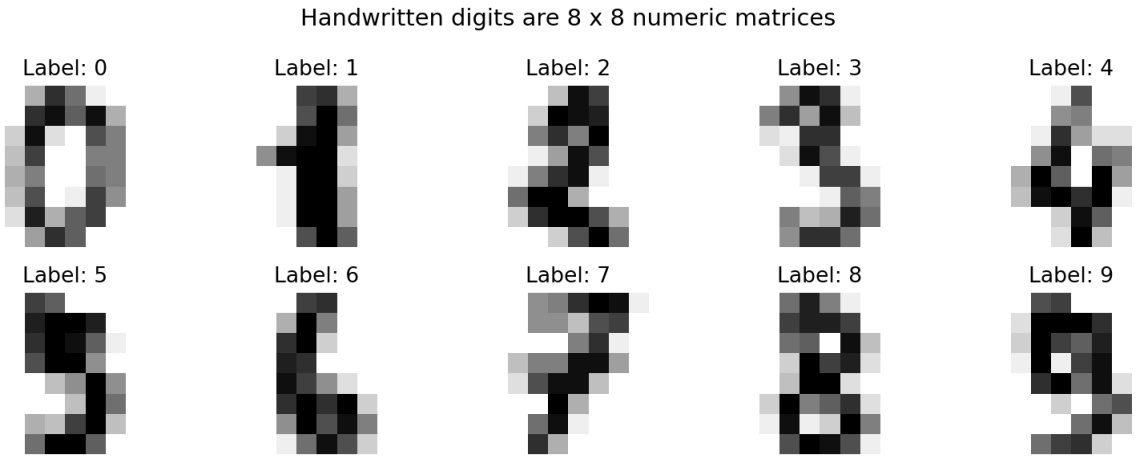
הדוגמאות המספריות, הסברים, איורים ושאלות בפרק נכתבו מחדש לצורכי הוראה; אין כאן העתקה של קטעים מספרים מסחריים. הרחבות כמו הוכחת XOR, משפט ההתכנסות והבחנת פונקציות הפעלה מוצגות כהעמקה פדגוגית. יש להצליב מול מסמך משרד החינוך התקף כדי לקבוע אילו תתי־נושאים מחויבים בפועל בבחינה או בהגשת הפרויקט.

פרק 10

מיני־פרויקט בלמידת מכונה

מספר לימוד לרעיון מחקר ועד למודל שאפשר להסביר, לבדוק ולשפר

התמחות בלמידת מכונה ולמידה עמוקה • 5 יחידות לימוד • פרק 10 • 30 שעות לפי מפת התכנית



איור: האתגר המלווה בפרק: המחשב מקבל מטריצות פיקסלים ומנסה לזהות ספרות בכתב יד.

בפרקים הקודמים למדנו את הכלים לחוד: מטריצות, עיבוד תמונה, KNN, מרכזי מחלקות, מדדי הערכה וולידציה. הפעם נשתמש בהם יחד. לא נשאל רק אם הקוד רץ, אלא האם ביצענו ניסוי אמין, מה למדנו מהטעויות ומה ניתן לומר בביטחון על המודל.

מפת למידה ומטרות ביצועיות

בסיום הפרק תוכלו להגדיר שאלת פרויקט, לתאר את הנתונים, לבחור מדד מתאים, להשוות מודל לקו בסיס, להפריד בין אימון לבחינה, לבצע ולידציה, לאתר טעויות, לתעד ניסוי ולהציג מסקנה זהירה בעל־פה.

הקשר לחומר קודם	התוצר המצופה	שלב
פרק 4.1	שאלה מדידה וקהל יעד	שאלת מחקר
פרקים 2.1–2.5	כרטיס מאגר, EDA וגרפים	הבנת נתונים
4.2, 4.3, 9	Baseline ושני מועמדים	בניית מודל
5.1–5.2	CV, Test, מדדים ושגיאות	בדיקה ושיפור
הכנה לפרויקט הגמר	דוח, מחברת ושחזור ניסוי	תוצר והצגה

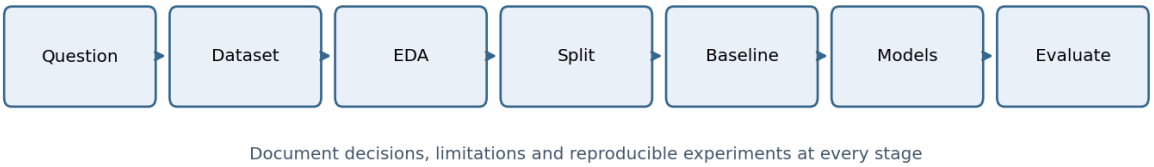
1. מה הופך תרגיל בכיתה למיני־פרויקט?

בתרגיל קצר בדרך כלל ידוע לנו איזו פקודה להפעיל ומה התוצאה שאמורה להתקבל. במיני־פרויקט חלק מההחלטות אינן מוכתבות מראש. עלינו להבין מה הבעיה, לבדוק אם הנתונים מתאימים, להצדיק את האלגוריתם, לבחון חלופות ולכתוב מה למדנו. זו עבודה מחקרית בקנה מידה קטן: המטרה אינה רק ציון טוב אלא תהליך שאפשר לשחזר ולבקר.

דודו רוצה לסייע למורה לקלוט טפסים שבהם תלמידים כתבו ספרות. במקום להקליד את הספרה בכל פעם, הוא רוצה לבדוק אם מערכת למידת מכונה תוכל להציע ספרה משוערת. הוא יודע כבר איך לייצג

תמונה כמטריצה ואיך מסווג KNN משתמש בשכנים קרובים. אך האם המודל יצליח גם על כתב יד שלא ראה? אילו ספרות יתבלבלו זו בזו? ומה תהיה הדרך הנכונה לבדוק זאת?

שאלת המחקר המנחה: עד כמה מודלים המבוססים על מרחק מצליחים לזהות ספרות בכתב יד במאגר digits, ומהם דפוסי הטעות שלהם? השאלה אינה מבטיחה הצלחה מראש. היא מזמינה ניסוי שבו נדע מה מדדנו, על אילו נתונים ומול איזה קו בסיס.



איור: תהליך העבודה הוא מחזורי: לאחר ניתוח שגיאות חוזרים לשאלת המחקר ולשיטות עיבוד הנתונים.

2. ניסוח בעיה טובה: מי המשתמש, מהו הקלט ומהי התחזית?

מערכת שאינה יודעת לומר במדויק מה היא מנבאת, אינה מוכנה לאימון. במקרה שלנו כל דוגמה היא תמונה אחת של ספרה; הקלט הוא 64 עוצמות פיקסלים, והיעד הוא ספרה אחת בין 0 ל־9. זהו סיווג רב־מחלקתי, ולא רגרסיה: הפלט 8 אינו כמות גדולה יותר מהפלט 3, אלא שם של מחלקה.

צריך להגדיר גם את תנאי השימוש. תמונות המאגר קטנות ומסודרות באותו פורמט; תמונות של טלפון נייד באור משתנה שונות מאוד. לכן הצלחה על המאגר לא מוכיחה יכולת לזהות כל כתב יד בעולם. כבר בשלב הגדרת הבעיה יש לציין את גבולות המערכת.

מה על התלמיד להחליט בפרויקט שלו	תשובה בדוגמת הספר	שאלה
מה מייצגת כל שורה?	תמונה אחת	מה יחידת התצפית?
אילו תכונות זמינות לפני התחזית?	64 ערכי פיקסלים	מהו X?
מה רוצים לדעת?	תווית הספרה 0–9	מהו y?
מה תיחשב תוצאה ראשונית?	ניחוש המחלקה הנפוצה באימון	מהו קו בסיס?
אילו טעויות חשובות?	Accuracy וגם Macro F1	מהו מבחן ההצלחה?

3. כרטיס מאגר — לומדים להכיר את הנתונים לפני אימון

מאגר digits הזמין בתוך scikit-learn מכיל 1,797 תמונות של ספרות בכתב יד. כל תמונה היא בגודל 8×8 פיקסלים, כלומר 64 תכונות מספריות, וערכי הפיקסלים נעים בין 0 ל־16. זהו פורמט המאפיינים של המאגר המסוים הזה; אין לבלבל אותו עם תמונות RGB רגילות שבהן מקובל להשתמש בטווח 0–255 לכל ערוץ.

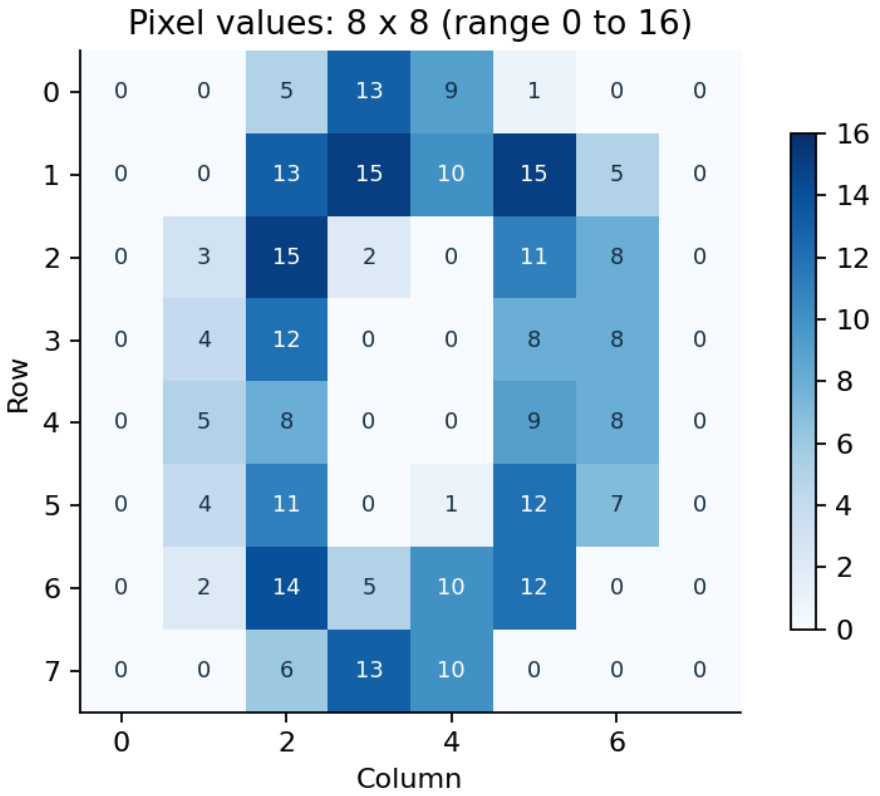
כרטיס המאגר צריך לתעד שם ומקור, משמעות התכונות, סוג המטרה, מספר הדוגמאות, מגבלות הייצוג והרשאות שימוש. במחקר אמיתי נוסיף גם מי אסף את הנתונים, באילו תנאים נאספו ומי אינו מיוצג בהם. מידע זה עוזר לקבוע אם המודל מתאים לשימוש שאליו אנו מייעדים אותו.

```
import numpy as np
from sklearn.datasets import load_digits

digits = load_digits()
X = digits.data
```

```
Y = digits.target
print(X.shape)      # (1797, 64)
print(digits.images.shape)  # (1797, 8, 8)
```

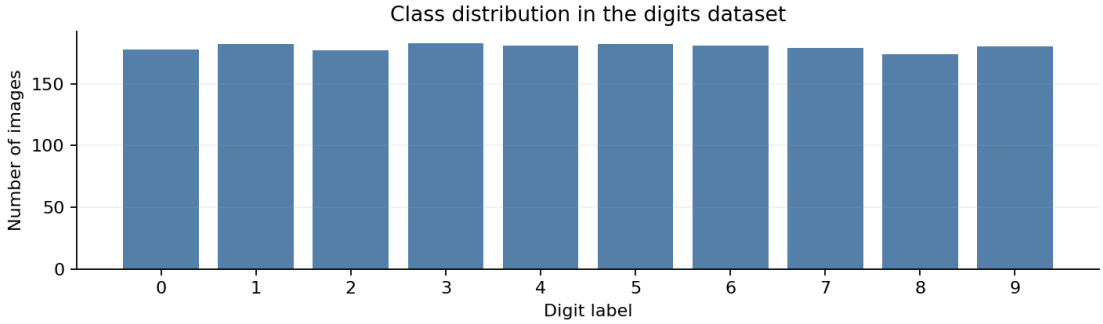
מבחינה מתמטית, המטריצה X מכילה 1,797 שורות ו־64 עמודות. כל שורה היא תמונה ש״נפרשה״ לווקטור; אותה תמונה עצמה נמצאת גם ב־digits.images כמטריצה של 8 שורות ו־8 עמודות. התוויות Y נשמרות בנפרד, משום שהתלמיד צריך להבין שהמודל לא רשאי לקבל את התשובה כחלק מהקלט.



איור: דוגמה למטריצת פיקסלים: מיקום התא ועוצמתו הם חלק מהתיאור המספרי של התמונה.

4. חקר נתונים — מה שואלים לפני שמאמנים?

שלב חקר הנתונים אינו קישוט. התלמיד בודק אם כל הספרות מיוצגות, האם קיימות דוגמאות חריגות, מהו טווח עוצמות הפיקסלים והאם הדוגמאות נראות ברורות לעין. באמצעות גרף שכיחויות אפשר לזהות אם אחת המחלקות נדירה מאוד; באמצעות תצוגה של מספר תמונות אפשר להבין את רמת השונות בכתב היד.



איור: שכיחות הספרות במאגר לדוגמה: התפלגות כמעט מאוזנת אינה פוטרת מבדיקת ביצועים לכל מחלקה.

אין להסיק מהיסטוגרמה בלבד שאין הטיה בנתונים. ייתכן שהמאגר מאוזן בין התוויות אבל פחות מייצג סגנונות כתיבה מסוימים או תנאי צילום חדשים. צריך להבחין בין איזון מספרי לבין ייצוג הולם של מצב שימוש אמיתי. בדוגמה שלפנינו לא נבצע טענות כלליות על אוכלוסיות של כותבים בלי מידע שמבסס זאת.

5. נתונים חסרים, כפולים ונתונים חשודים

לפני עיבוד נתונים יש להבחין בין ערך אפס בפיקסל לבין ערך חסר. במאגר digits אפס הוא עוצמת פיקסל תקינה; אין להחליפו בממוצע כאילו פירושו ״לא נמדד״. אם מאגר אחר מכיל NaN, נסביר כיצד נוצר החסר ורק אז נחליט על טיפול. גם ״תמונה שנראית מוזרה״ אינה בהכרח טעות — ייתכן שזה כתב יד שקשה לסווג.

יש לבדוק כפילויות או כמעט־כפילויות: אם צילומים של אותו טופס נמצאים גם באימון וגם בבדיקה, המודל עלול להצליח בזכות דמיון חריג בין הדוגמאות. בפרויקט עם כמה תמונות לכל אדם או מסמך, החלוקה הנכונה עשויה להיות לפי אדם או מסמך, ולא לפי תמונה בודדת. נתונים מקבוצות או מתאריכים שונים דורשים חלוקה המותאמת לשאלת הפרויקט.

6. קובעים מבחן הוגן: Train, Validation ו־Test

נתוני האימון הם הדוגמאות שמותר למודל ללמוד מהן. קבוצת הוולידציה משמשת לבדוק החלטות במהלך הפיתוח, למשל איזה k מתאים ל־KNN. קבוצת הבדיקה נשמרת מחוץ לתהליך הבחירה כדי לאפשר הערכה מסכמת אמינה ככל האפשר. אם נשנה את המודל שוב ושוב לפי ציוני Test, אותה קבוצה מפסיקה להיות מבחן חיצוני אמיתי.

בפרויקט הכיתה נשתמש בפיצול חד־פעמי ובשכבות לפי התווית כדי לשמור בקירוב על שיעור הספרות בכל חלק. בתוך נתוני האימון נוכל לבצע Stratified K-Fold לבחירת היפר־פרמטרים. כך אין צורך בקבוצת Validation נפרדת וקבועה: כל קיפול ממלא את תפקידה בתורו.

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, Y, test_size=0.20,
    random_state=42, stratify=Y
)
print(X_train.shape, X_test.shape)
```

חשוב לזכור: קביעת **random_state** מאפשרת לשחזר את החלוקה, אך אינה מוכיחה שהחלוקה מייצגת כל שימוש עתידי. תמיד צריך לבדוק את מבנה המאגר והקשר שלו לשאלת המחקר.

7. מדוע חייבים להתחיל ב־Baseline?

לפני השקעה במודל מורכב קובעים ניבוי פשוט שמולו משווים. בסיווג אפשר למשל לבחור תמיד את המחלקה הנפוצה בנתוני האימון. זהו מודל בסיס חלש מאוד, אך מועיל: אם האלגוריתם החדש אינו משיג תוצאה טובה יותר, ייתכן שיש שגיאה בקוד, בנתונים או בעצם ההנחה שהמאפיינים מאפשרים את החיזוי.

חשוב להתאים גם את קו הבסיס על Train בלבד. חיזוי ״הספרה הנפוצה בכל המאגר״ משתמש גם במידע מקבוצת הבדיקה ולכן אינו נקודת ייחוס נקייה. כמו כן, נתון Accuracy גבוה אינו בהכרח מספיק כאשר המחלקות אינן מאוזנות או כאשר מחיר הטעות שונה ממקרה למקרה.

```
from sklearn.dummy import DummyClassifier
from sklearn.metrics import accuracy_score

baseline = DummyClassifier(strategy="most_frequent")
baseline.fit(X_train, y_train)
base_pred = baseline.predict(X_test)
print(accuracy_score(y_test, base_pred))
```

8. מאמנים מועמדים — לא רק את המודל האהוב עלינו

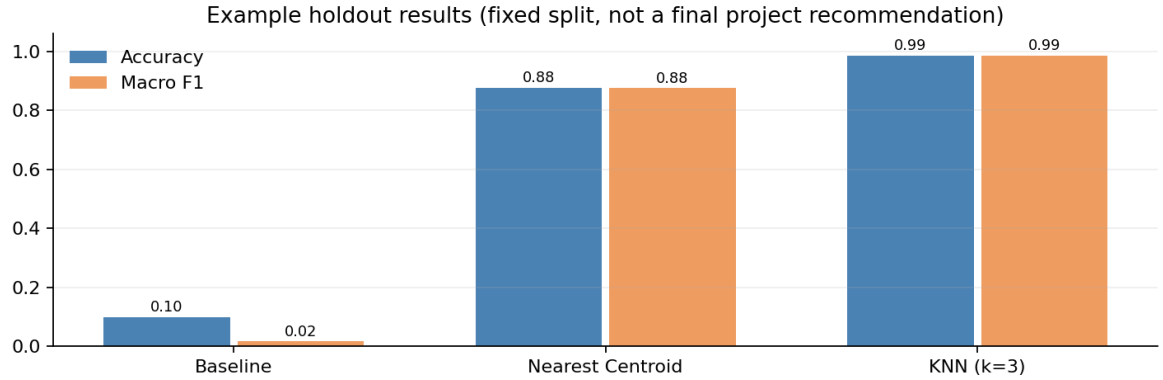
לאחר שבנינו קו בסיס, נבחן שני אלגוריתמים שכבר למדנו: Nearest Centroid ו־KNN. הראשון מייצג כל מחלקה על ידי מרכז יחיד ומסווג לפי מרחק למרכזים. השני משתמש בדוגמאות אימון הקרובות לנקודת

השאילתה. זהו ניסוי מעניין משום ששני המודלים מבוססי מרחקים אך מניחים דברים שונים על מבנה המחלקות.

על אותן דוגמאות אימון ועל אותן דוגמאות בדיקה אפשר להדגים השוואה ראשונית. יש לזכור שזו הדגמה אחת בחלוקה קבועה; לפני בחירת מודל סופית נרצה להגדיר את הליך הבחירה באמצעות Validation או Cross-Validation, ולא להתבסס על טבלת ה־Test כדי להמשיך לשפר את הפרמטרים.

```
from sklearn.neighbors import KNeighborsClassifier, NearestCentroid

model_centroid = NearestCentroid()
model_knn = KNeighborsClassifier(n_neighbors=3)
model_centroid.fit(X_train, y_train)
model_knn.fit(X_train, y_train)
```

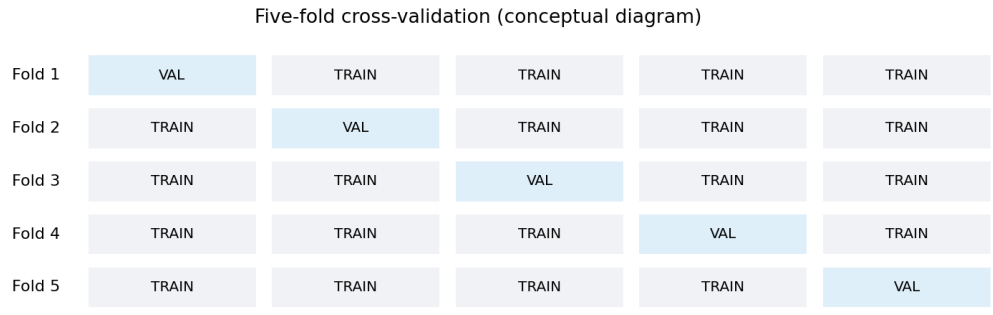


איור: השוואת הדגמה על פיצול יחיד: הבסיס, *Nearest Centroid* ו־*KNN*. זו אינה תחרות מחקרית סופית.

המספרים בגרף מבוססים על דוגמה ניתנת לשחזור מתוך מאגר digits עם random_state=42. גם כאשר KNN נראה טוב יותר כאן, אסור להסיק שמודל זה מנצח תמיד. השוואה מקצועית מחייבת לבדוק באילו נתונים, איזה מדד חושב, כיצד נבחרו ההיפר־פרמטרים ומהי רמת אי־הוודאות סביב התוצאה.

9. בוחרים היפר־פרמטרים בתוך נתוני האימון

מספר השכנים k אינו נלמד אוטומטית מתוך הנתונים על ידי KNN; אנחנו קובעים אותו לפני ההתאמה. לכן הוא היפר־פרמטר. הבחירה ב־k=3 בדוגמה הקודמת נועדה להמחשה בלבד. בפרויקט אמיתי נבדוק כמה ערכים בעזרת קיפולי ולידציה, ונשתמש באותו מדד בכל ההשוואות.



איור: בכל קיפול מאמנים על חלק אחר מהנתונים ובודקים על החלק הנותר; את *Test* החיצוני שומרים בצד.

נניח שקיבלנו עבור k=1 שלושה ציוני ולידציה: 0.92, 0.90, 0.94. ממוצעם הוא 0.92. עבור k=3 התקבלו 0.95, 0.93, 0.94, ולכן הממוצע הוא 0.94. לפי דוגמה מספרית זו k=3 עדיף, אך ההבדל הקטן דורש זהירות: בסט נתונים קטן גם חלוקה אחרת עשויה לשנות את המסקנה.

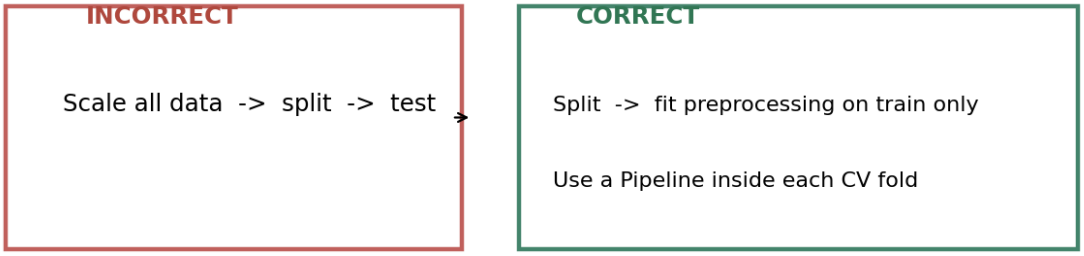
$$\text{mean}(k=1) = (0.92 + 0.90 + 0.94) / 3 = 0.92$$

$$\text{mean}(k=3) = (0.95 + 0.93 + 0.94) / 3 = 0.94$$

נוח להשתמש ב־GridSearchCV או בלולאה עם cross_val_score, אך חשוב להבין את ההיגיון לפני הפקודה. בשני המקרים מקפידים שהמודל מאומן מחדש בתוך כל קיפול, ואין מעבר מידע מהוולידציה אל נתוני האימון של אותו קיפול.

10. מניעת Data Leakage — שגיאה שמייצרת אשליה של הצלחה

זליגת מידע מתרחשת כאשר מידע שאינו אמור להיות זמין בעת החיזוי משפיע על אימון המודל או על בחירתו. דוגמה פשוטה: חישוב ממוצע וסטיית תקן לכל המאגר לפני חלוקת Train/Test. גם אם אין שימוש ישיר בתוויות, ערכי Test השפיעו על שלב ההכנה. לכן תקנון נלמד על Train בלבד ומוחל על Test באמצעות transform.



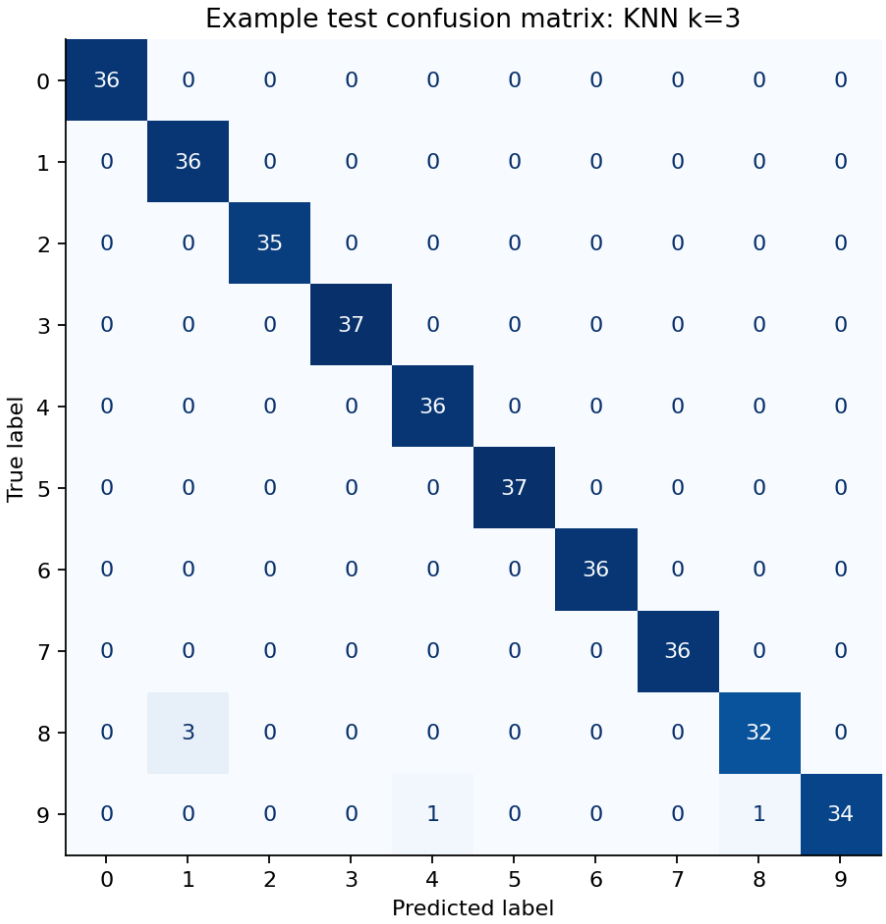
איור: סדר הפעולות קובע: קודם מפרידים נתונים, ורק אחר כך מתאימים טרנספורמציות למידע שמותר ללמוד ממנו.

בתהליך Cross-Validation חשוב עוד יותר לכלול את שלבי ההכנה בתוך Pipeline, כדי שהתקנון יתאים את עצמו מחדש לנתוני האימון של כל קיפול. בפרויקט ספרות שבו כל הפיקסלים כבר באותו סולם, אפשר לבדוק אם תקנון אכן משפר ביצועים; אין להניח שכל שלב הכנה משפר כל מודל.

11. כיצד מודדים הצלחה בסיווג עם עשר מחלקות?

בבעיה בינרית למדנו TP, FP, FN ו־TN. כאן יש עשר ספרות אפשריות, ולכן מטריצת בלבול בגודל 10×10: שורה מייצגת תווית אמיתית ועמודה תווית חזויה. התא באלכסון הראשי מציג חיזוי נכון, ותאים שמחוץ לאלכסון מציגים בלבולים בין ספרות.

Accuracy הוא אחוז התחזיות הנכונות מכלל הדוגמאות. Macro F1 מחשב F1 נפרד לכל ספרה וממצע את עשרת הערכים במשקל שווה, ולכן הוא מסייע כאשר מעוניינים לתת לכל מחלקה חשיבות דומה. Weighted F1 שוקל לפי גודל מחלקה; אלה מדדים שונים, ואין להחליף ביניהם בלי הסבר.



איור: דוגמת מטריצת בלבול ל־*KNN*. חשוב להצביע על ספרות שמבלבלות את המודל, ולא רק לדווח על מספר סופי.

12. חישוב ידני של מדדים לפרויקט — "האם זו הספרה 8?"

כדי לחבר את מטריצת הבלבול הרב־מחלקתית למונחים שלמדנו, נבחר ספרה אחת בתור מחלקה חיובית: 8. נניח שבדוגמה לימודית של 40 תמונות התקבלו 7 זיהויים נכונים של 8, שלוש אזעקות שווא, שתי החמצות ו־28 דוגמאות של ספרות אחרות שזוהו כלא־8. זו טבלה מומצאת לצורך חישוב, לא תוצאה של הניסוי על digits.

פירוש	מספר	מצב
הספרה 8 זוהתה נכון	7	TP
ספרה אחרת זוהתה בטעות כ־8	3	FP
הספרה 8 הוחמצה	2	FN
ספרה אחרת לא זוהתה כ־8	28	TN

$$\text{Accuracy} = (7 + 28) / 40 = 0.875$$

$$\text{Precision} = 7 / (7 + 3) = 0.70$$

$$\text{Recall} = 7 / (7 + 2) = 0.778$$

$$F1 = 2 * 0.70 * 0.778 / (0.70 + 0.778) \approx 0.737$$

למרות Accuracy של 87.5%, מדד Precision כאן הוא 70% בלבד. במערכת שבה סיווג שגוי של טופס עלול לגרום לאיבוד מידע, גם מקרה כזה דורש בדיקה. המדד החשוב אינו רק מספר גדול, אלא מדד שמתאים לנזק של שגיאה ולמטרת השימוש.

13. ניתוח טעויות — החלק שבו הפרויקט הופך למחקר

לאחר ההערכה נציג תמונות שהמודל טעה בהן, נבדוק מה הייתה התווית האמיתית ומה המודל חזה, ונשאל מה משותף למקרים. האם מדובר בצורה חריגה של הספרה? האם קיימות תכונות שאינן מספיקות להבדיל בין 1 ל־7? האם מגבלת הרזולוציה של 8×8 פיקסלים מקשה על הזיהוי?



איור: מדגם טעויות מתוך קבוצת הבדיקה בדוגמה — כל מקרה הוא הזדמנות להציע השערה חדשה.

אסור לבחור רק את הדוגמאות שהמודל הצליח בהן כדי להציגו בצורה מחמיאה. חשוב גם לא לשנות את המודל לפי אותן טעויות Test שוב ושוב ולדווח על אותה קבוצה כהערכה בלתי תלויה. דפוסי טעויות יכולים להוליד רעיון למחקר המשך; לצורך בחירה נוספת נידרש למסגרת הערכה חדשה או נשתמש בנתוני האימון והוולידציה.

14. ניתוח משאבים ומגבלות — מה עובד בכיתה ומה יעבוד במערכת?

מיני־פרויקט אינו מסתיים בבחירת אלגוריתם בעל ציון גבוה. עלינו לשאול גם מה זמן החיזוי, האם כל דוגמאות האימון חייבות להישמר, כמה זיכרון נדרש, ומה יקרה אם יתקבלו תמונות בגדלים שונים. KNN שומר דוגמאות אימון לצורך חיזוי; Nearest Centroid שומר לכל מחלקה ייצוג תמציתי. ההבדל עשוי להיות משמעותי בסביבת עבודה עם הרבה נתונים.

יש לשאול גם מה לא אפשר להסיק. מאגר digits הוא מאגר לימודי של תמונות קטנות, ולכן הוא מתאים לבדיקת תהליך אך לא מהווה אישור למערכת לזיהוי מסמכים משפטיים, פיננסיים או אישיים. במערכת אמיתית נידרש לבדיקת דוגמאות חדשות, פרטיות, אבטחה ודרכי טיפול כאשר המודל אינו בטוח בתשובתו.

15. כיצד הופכים מחברת ניסויים לתוצר שאפשר להדגים?

הפרויקט יכול להסתיים במחברת מסודרת שמקבלת תמונה אחת ומציגה את הספרה שהמודל בחר, לצד התמונה עצמה. אפשר לבנות ממשק פשוט שמאפשר לתלמיד לבחור דוגמה מתוך מאגר הבדיקה. אין חובה בשלב זה לבנות יישום נייד או שירות אינטרנט; אלה יהיו אפשרויות להעמקה בפרויקט הגמר.

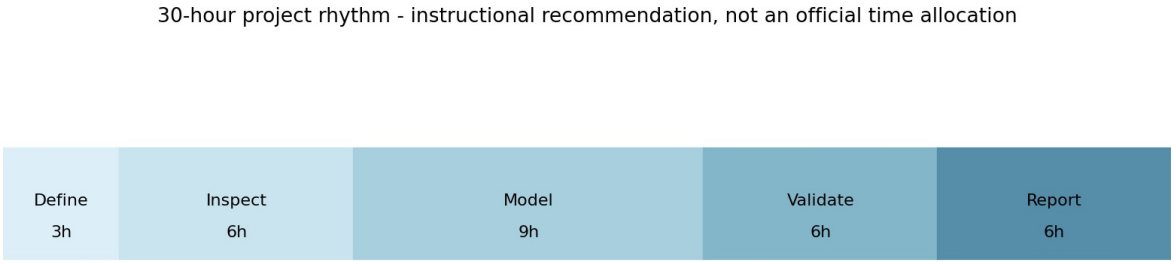
הדגמה אמינה צריכה להראות גם מקרה שבו המודל טועה, להסביר מה ההשלכות של הטעות ולתאר איזה מידע נוסף יהיה דרוש לפני שימוש אמיתי. במסגרת הדגמה לא צריך להמציא ״רמת ביטחון״ אם המודל אינו מספק הסתברות מכוילת. מספר השכנים שהצביעו למחלקה אינו תמיד הסתברות מהימנה.

16. לוח ביצוע: 30 שעות, עשרה מפגשים של שלוש שעות

חלוקת השעות בטבלה היא הצעה דידקטית שלנו למימוש היקף 30 השעות בפרק ההתמחות. זו אינה חלוקת זמן רשמית של משרד החינוך. כל מפגש מסתיים בתוצר שאפשר לבדוק, ולא רק ב״התקדמנו בקוד״.

מפגש	מוקד	תוצר להגשה
1	בעיית מחקר ומטרת שימוש	שאלת מחקר + שלושה קריטריוני הצלחה
2	מקור, איכות ורישוי נתונים	כרטיס מאגר והצהרת מגבלות

מפגש	מוקד	תוצר להגשה
3	EDA, שכיחויות ותמונות	שלושה גרפים עם פרשנות
4	חלוקה ושימוש ב־Baseline	פרוטוקול פיצול וקו בסיס
5	בניית שני מודלים	קוד אימון ותיאור הבדלים
6	ולידציה ובחירת פרמטרים	טבלת ניסויים ומסקנה
7	הערכה חיצונית וניתוח מדדים	מטריצת בלבול ודוח מדדים
8	ניתוח שגיאות ומגבלות	שתי השערות וממצאים
9	הדגמה, ריפרודוקציה ובדיקות	מחברת עובדת והוראות הרצה
10	דוח, מצגת והגנה	דוח סופי, הצגה ומשוב עמיתים



איור: חלוקה מוצעת של 30 שעות לשלבי עבודה; בתכנית הכיתה המפורטת מפורטים עשרה מפגשים בני 3 שעות.

17. מחוון הערכה לימודי — כיצד יודעים שהפרויקט שלם?

כדי להעריך פרויקט באופן הוגן נגדיר מראש מה נבדק. המחוון הבא הוא הצעת הוראה עצמאית לספר ואינו מוצג כמחוון הרשמי של משרד החינוך. ניתן להתאים את האחוזים בהתאם לצורכי הכיתה, תוך שמירה על כך שהערכת המודל ותיעוד ההחלטות מקבלים משקל משמעותי.

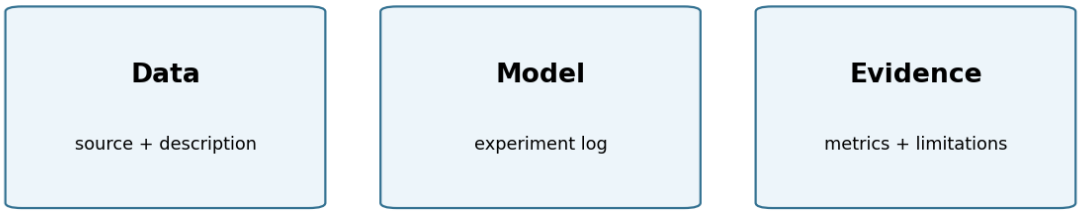
רכיב	ניקוד מוצע	ראיות נדרשות
הגדרת בעיה ונתונים	20	שאלה, כרטיס מאגר, בדיקות איכות
EDA והצגת ממצאים	15	גרפים, פירושים ומגבלות
מודלים ו־Baseline	20	שני מודלים לפחות וניסוי משוחזר
ולידציה והערכה	20	CV, Test ומדדים מותאמים
ניתוח שגיאות	15	מקרי טעות וניתוח מנומק
תיעוד והגנה	10	מחברת, מקורות, הצגה ומענה לשאלות
סה״כ	100	

18. דוח סופי מקצועי — מבנה אחיד להגשה

דוח טוב אינו תמליל של כל פקודה. הוא מספר מה הייתה הבעיה, מדוע הנתונים מתאימים באופן חלקי או מלא, אילו החלטות התקבלו ומה הראיות התומכות בהן. יש להבחין בין ממצא מדוד לבין הסבר

אפשרי. למשל "KNN" השיג ציון גבוה יותר בניסוי המסוים" הוא ממצא; "KNN" תמיד טוב יותר" אינו מסקנה מוצדקת.

מה צריך להופיע	פרק בדוח
שאלה, שיטה ותוצאה עיקרית בשלושה משפטים	תקציר
מה מתכוונים לחזות ולמי זה מועיל	בעיה ויעד
מקור, רישוי, יחידת תצפית ומגבלות	נתונים
פיצול, עיבוד נתונים, מודלים והיפר־פרמטרים	שיטה
טבלת החלטות, גרפים ונתוני ולידציה	ניסויים
Test, מדדים ומקרי שגיאה	תוצאות
מגבלות, סיכונים, רעיונות למחקר המשך	דיון
סביבת הרצה, גרסאות, זרע אקראי, קישורים	שחזור ומקורות



איור: שלושת סוגי הראיות המרכזיים: נתונים מתועדים, ניסויים משוחזרים ומסקנות מוגבלות למה שנמדד.

19. בדיקות עצמיות למפתח — אל תסמכו על קוד שרק "רץ"

פרויקט גמר מקצועי מחייב בדיקות גם לקוד. לפני אימון נבדוק שמספר התוויות שווה למספר השורות במטריצה, שאין ערכי NaN בלתי צפויים, שמספר המאפיינים מתאים לצורה שקבענו, ושאין חפיפה לא מכוונת בין דוגמאות האימון לבדיקה. לאחר חיזוי נבדוק שהפלט כולל תחזית לכל תמונה ושכל תווית נמצאת בטווח האפשרי.

```
assert X.shape[0] == len(Y)
assert X.shape[1] == 64
assert not np.isnan(X).any()
assert set(np.unique(Y)) == set(range(10))
```

הבדיקות הללו מתאימות למאגר המסוים שלנו ואינן "מתכון כללי" לכל מאגר. בפרויקט אחר ייתכן שמספר המאפיינים שונה, חסרים הם חלק מן הבעיה, או שהמחלקות הן טקסט ולא מספרים. המטרה היא ללמוד לכתוב בדיקות שמגנות על ההנחות של המודל, ולא להעתיק אותן בלי חשיבה.

20. אתיקה, פרטיות ושימוש אחראי

לפני שימוש בנתונים שנאספו בכיתה או באפליקציה יש לבחון אם יש מידע אישי, האם התקבלה הרשאה מתאימה, מה משך השמירה ומה עושים כאשר תלמיד מבקש להוציא נתונים מהמאגר. לצורך מיני־פרויקט לימודי עדיף להתחיל במאגר פומבי המיועד להוראה או בנתונים מדומים שאינם מזהים אנשים.

גם פרויקט בעל כוונה טובה עשוי להיכשל אם מתעלמים מהקשר שימוש. אין להסיק מתוצאות זיהוי ספרות שום דבר על יכולות לימודיות של תלמידים, ואין להשתמש בתחזית אוטומטית יחידה כדי לקבל החלטה בעלת השלכה משמעותית על אדם ללא בדיקה מתאימה.

21. שאלות מדורגות לתרגול — ללא פתרונות

רמה א׳ — הבנה וביצוע

1. מה ההבדל בין מאפיין לתווית במאגר digits? מדוע מספר התכונות הוא 64?

2. הסבירו מדוע בחירת הספרה הנפוצה בקבוצת האימון היא קו בסיס תקין, ומדוע אין לבחור את הספרה הנפוצה בכל המאגר.

3. למה אי אפשר להסיק ממספר תמונות כמעט שווה לכל ספרה שאין הטיה במאגר?

רמה ב׳ — חישוב, ניתוח ותכנון

1. בהערכה של 50 תמונות נמצאו 43 תחזיות נכונות. חשבו Accuracy והסבירו מה המדד אינו מספר לכם.

2. עבור ספרה 7 התקבלו TP=12, FP=3, FN=4. חשבו Precision, Recall ו־F1. הסבירו מה משמעות כל אחד.

3. הציעו כיצד לבחור k ב־KNN באמצעות Cross-Validation מבלי לגעת ב־Test.

4. נמצא שזמן החיזוי ב־KNN ארוך יותר מאשר ב־Nearest Centroid. הציעו שני הסברים אפשריים שמחייבים ניסוי נוסף.

רמה ג׳ — חשיבה ביקורתית בסגנון בגרות והגנה

1. תלמיד תקנן את כל הנתונים, בחר k לפי Test ואז דיווח על אותו Test כציון הסופי. זהו שתי שגיאות שונות: הסבירו כל אחת ותקנו את הליך המחקר.

2. מודל א׳ קיבל Accuracy גבוה יותר, אך נכשל במספר רב יותר של ספרות 9. אילו מדדים וראיות נוספים תדרשו לפני החלטה?

3. אחרי צפייה בחמש טעויות Test הוחלט לשנות את עיבוד התמונה. כיצד ניתן להמשיך במחקר בלי להציג באופן מטעה את תוצאת Test הראשונית?

4. דונו בהבדל בין הצלחה על תמונות 8×8 לבין הצלחה על תמונות שנלקחו במצלמת טלפון, והציעו תכנית בדיקה מתאימה.

22. מחברת המעבדה העצמאית — הוראות לתלמיד

לצד פרק זה מצורפת מחברת Colab עצמאית. התלמידים יטענו מאגר, יבדקו את מאפייניו, יתעדו שאלת מחקר, יבנו קו בסיס, ישוו מסווגים, יבצעו ולידציה ויכתבו מסקנות. תאי הקוד שיש להשלים מסומנים TODO; אין במחברת פתרון מלא מוכן להעתקה. הדוגמאות הפתורות שבספר מיועדות להבנת הרעיונות ולבדיקות חלקיות בלבד.

מומלץ לשמור גרסאות מסודרות של הניסוי, לא לשנות את מספר הזרע האקראי רק כדי לקבל ציון טוב יותר, ולרשום מה נוסה ומה לא. תוצאה שלא הצליחה יכולה להיות תוצר לימודי משמעותי, אם ברור כיצד בדקנו אותה ומה למדנו ממנה.

23. שאלות הגנה בעל־פה: האם אתם מבינים את מה שבניתם?

הבוחן יכול לבקש מהתלמיד להסביר את מהלך העבודה בלי לראות את הקוד. לכן נתרגל הצגה של שתי דקות: מה הבעיה, מה הנתונים, מה המודל, כיצד בחרנו אותו, מה תוצאת הבדיקה ומה המגבלה

החשובה ביותר. לאחר מכן נשיב לשאלות שינוי כמו ״מה יקרה אם יופיעו תמונות באיכות אחרת?״ או ״כיצד תזהו זליגת מידע?״.

משפט סיכום טוב לפרויקט הוא לא ״המערכת מזהה ספרות ב־99%״ ללא הקשר. ניסוח מקצועי יותר: ״בחלוקת הבדיקה המסוימת ובתנאים שתוארו התקבל ציון X; נדרשות בדיקות נוספות לפני שימוש בתמונות שמקורן מחוץ למאגר״. כך מציגים יכולת בלי להגזים בהסקת מסקנות.

24. מה נרצה לקחת מפרק 10 לפרויקט הגמר בכיתה י״ב?

כאן תרגלנו את השיטה השלמה: מגדירים בעיה, מתארים מאגר, בוחנים קו בסיס, בונים מודל, בודקים ולידציה, מנתחים טעויות ומציגים ראיות. בפרויקט הגמר נוסיף עומק רב יותר במרכיב המודל, במערכת התוכנה ובניהול הפרויקט. לכן התוצר החשוב של המיני־פרויקט הוא לא רק מודל שמחזיר תחזית, אלא תיק חקר מסודר שמוכיח כיצד הגענו אליה.

הפרק הבא, מתמטיקה ללמידה עמוקה, יאפשר לנו להבין כיצד מודלים מעדכנים משקולות בעזרת נגזרות וגרדיאנטים. את המיני־פרויקט הנוכחי נוכל לשמור כנקודת השוואה לרשת עצבית שנבנה בהמשך.

נספח א׳ — תיק הפרויקט: תבנית לתכנון והחלטות

גם תלמיד שיודע לתכנת עלול להתקשות בהסבר של החלטותיו כמה שבועות לאחר הניסוי. לכן נשמור תיק פרויקט קצר לצד המחברת. תיק זה אינו מוסיף אלגוריתם חדש; הוא מלמד להפוך עבודה חד־פעמית לתהליך שחוקר אחר יכול לשחזר. יש למלא אותו תוך כדי התקדמות, ולא לכתוב את כולו רק ערב ההגשה.

דוגמה לסוג תשובה	מה נרשום	שדה בתיק
סיווג ספרה בתמונה קטנה	איזו החלטה רוצים לקבל בעזרת המודל	גרסת השאלה
מאגר פתוח מתוך ספרייה מוכרת	מקור מדויק, רישוי והגבלות שימוש	מקור הנתונים
תמונה אחת, לא תלמיד שלם	מה כל שורה מייצגת	יחידת תצפית
חלוקה משוכבת לפי תווית	איך נוצרו Train ו־Test ומדוע	חלוקה לנתונים
ניבוי מחלקת הרוב באימון	מהי רמת הביצוע של פתרון נאיבי	קו בסיס
השוואה של מודל מרכזים ושכנים	מועמדים והסיבה לבחירתם	בחירת מודל
Accuracy־ו Macro F1	מדד, CV וקבוצת Test עצמאית	שיטת הערכה
תמונות ממצלמות אחרות	מה עוד לא בדקנו	החלטה פתוחה

נספח ב׳ — יומן ניסויים: דוגמה לפירוש תוצאות

נניח שתלמיד בדק שלוש גרסאות של מודל על אותה מסגרת ולידציה. כדי להבין אם נוצר שיפור, נרשום לא רק את הציון אלא גם שינוי אחד שנעשה בכל גרסה. אם משנים גם את הסולם, גם את סוג המודל וגם את מספר השכנים באותה ריצה, קשה לדעת מה תרם לשינוי בתוצאה. בניסוי מבוקר משנים משתנה אחד בכל פעם ככל האפשר, תוך התחשבות בכך שלפעמים יש תלות בין גורמים.

ניסוי	מה השתנה	Macro F1 ב־CV (דוגמה מומצאת)	מסקנה זהירה
A	KNN, k=1	0.91	נקודת התחלה

ניסוי	מה השתנה	Macro F1 ב־CV (דוגמה מומצאת)	מסקנה זהירה
B	KNN, k=3	0.93	נראה שיפור קטן, לא הוכחה כללית
C	KNN, k=5	0.92	נמוך מעט מ־B בחלוקה זו

המספרים בטבלה מומצאים לצורכי לימוד; הם אינם תוצאות הניסוי על digits שבאיורים. בחקר אמיתי כדאי לציין גם פיזור ציוני הקיפולים, תאריך ריצה, גרסאות ספריות וזרע אקראי. אם השינוי קטן מאוד, ייתכן שההבדל נובע מהחלוקה או ממספר הדוגמאות. אין לתאר הבדל זעיר כ״שיפור משמעותי״ בלי עדויות מתאימות.

בכל רשומת ניסוי יש לכתוב השערה לפני ההרצה, שינוי שבוצע, תוצאה שנמדדה, הסבר אפשרי ומסקנה לגבי הצעד הבא. בדרך זו גם ניסוי שלא שיפר את הציון תורם להבנת המודל ומונע חזרה על אותן טעויות.

נספח ג׳ — דוגמת הסבר לבוחן: ממצא לעומת טענה

בוחן שואל: ״מדוע בחרתם ב־KNN ולא במודל של מרכזי מחלקות?״ תשובה חלשה תהיה: ״כי הוא יצא הכי טוב״. תשובה מקצועית צריכה לתאר את ההשוואה, להראות איזה מדד נקבע מראש, להסביר כיצד הופרדו הנתונים, ולציין מגבלה. לדוגמה: ״בחרנו שני מודלים שכבר למדנו, ביצענו ולידציה על קבוצת האימון ובחרנו את ההגדרה המועדפת לפי המדד שהוגדר. בבדיקת Test שלא שימשה לבחירה קיבלנו את התוצאה המדווחת; איננו טוענים שהתוצאה תישמר באותה רמה על תמונות ממקורות אחרים.״

שאלת המשך אפשרית היא: ״איך תדעו שהמודל אינו פשוט זוכר את תמונות האימון?״ כאן יש להדגיש שהערכת Test משתמשת בדוגמאות שלא היו באימון, שזיהוי כפילויות עשוי להיות חשוב ושדוגמאות חדשות באמת יכולות להיות שונות יותר מנתוני המאגר. התלמיד צריך לדעת להבחין בין יכולת הכללה בתוך מאגר מסוים לבין יכולת ההעברה לסביבה חדשה.

נספח ד׳ — רשימת בדיקה לפני הגשה

בדקו שכל ההחלטות מתועדות: הגדרת משתנה המטרה; מקור נתונים חוקי; רישום מספר תצפיות ותכונות; גרפים עם כותרות ויחידות; חלוקת Train/Test שנעשתה לפני ההתאמות; השוואה ל־Baseline; בחירת פרמטרים שאינה משתמשת ב־Test; מדד מתאים; ניתוח לפחות כמה טעויות; תיאור מגבלות; הוראות הרצה ושחזור; והפרדה בין קוד תלמיד לבין מקורות שהסתמכתם עליהם.

מבחן עצמי לסיום: תלמיד אחר מקבל רק את קובצי הפרויקט וההוראות שלכם. האם הוא יכול להריץ את הניסוי, לזהות אילו נתונים היו זמינים למודל, לשחזר את ההערכה ולגלות כיצד התקבלה ההחלטה? אם התשובה אינה חיובית, הפרויקט עדיין אינו מוכן להגשה גם אם המספר הסופי מרשים.

25. עוגן בתכנית ומקורות מקצועיים

מיקום בתכנית: מסלול התמחות למידת מכונה ולמידה עמוקה, פרק 10 — מיני־פרויקט, 30 שעות, לפי מפת התכנית במאגר GitHub של הקורס. חלוקת 30 השעות לעשרת המפגשים, שאלת הפרויקט ומחוון ההערכה שבספר הם תכנון דידקטי מקורי; אין לראות בהם חלוקה רשמית או מחוון רשמי של משרד החינוך.

מקורות מקצועיים להעמקת ההסבר: Géron — Hands-On Machine Learning (עבודה שלמה עם מאגר, השוואת מודלים והערכה); James ואחרים — An Introduction to Statistical Learning with Applications in Python (רגרסיה, סיווג, הערכה וולידציה); Serrano — Grokking Machine Learning (הסברים אינטואיטיביים). ההמחשות, התרגילים והסיפור בספר נכתבו באופן עצמאי.

קריאה ותיעוד — קישורים למקורות:

```
https://github.com/samiabo75/Data-Science-ML-Course/tree/main/03-Deep-Learning-5-Units/10-Mini-Project
https://scikit-learn.org/stable/common_pitfalls.html
https://www.statlearning.com/
https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_digits.html
```

מפת שימוש במקורות — מה למדנו מכל ספר?

מה נכתב בספר שלנו	תרומה לפרק זה	מקור מקצועי
מקרה לימודי בעברית, ניסויים ואחריות תיעודית	מחזור עבודה: נתונים, אימון, מדידה ושיפור	Hands-On Machine Learning — Géron
חישובים ידניים וטבלאות שמתאימות לתיכון	השוואת מודלים, מדדים, ולידציה ופרשנות	An Introduction to Statistical Learning — James ואחרים
סיפור דודו, שאלות הבנה ודוגמה מפורקת	הסבר אינטואיטיבי של מושגים לפני קוד	Grokking Machine Learning — Serrano
קוד קצר שאפשר להריץ ומדיניות Test ברורה	הגדרות של API, מאגר digits ואזהרות Data Leakage	scikit-learn Documentation

השימוש במקורות מיועד לאימות מושגים, למידה של שיטות הוראה והפניית התלמיד להמשך העמקה. האיורים, התרגילים והדוגמאות המספריות בפרק נוצרו באופן מקורי; אין כאן תרגום של טקסט מוגן מספר מסחרי.

מעמד התוכן ביחס לתכניות הלימודים

פרק 10 מופיע בהיקף של 30 שעות במפת תכנית ההתמחות הרשומה במאגר הקורס. המטרות של תיעוד ניסויים, ניתוח טעויות ושחזור הן התאמות מקצועיות לקראת עבודת הגמר. חלוקת השעות הפנימית, המחוון המוצע והפרויקט על מאגר digits הם בחירות פדגוגיות מקוריות; אין להציגם כציטוט מחייב ממחוון משרד החינוך שלא נבדק במלואו.

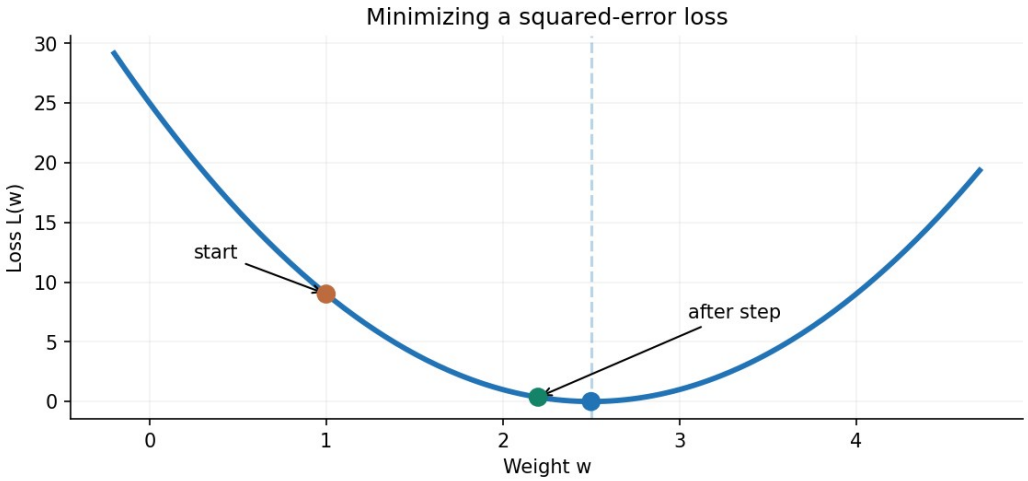
קישור רוחבי בספר המאוחד: יחידה 3 — קריאת נתונים ו־KNN בסיסי; יחידה 5 — וקטורים, מרחקים, הערכה ולידציה; התמחות למידה עמוקה — תהליך ניסוי מלא, תיעוד וביסוס לקראת רשתות עצביות ופרויקט גמר.

פרק י״א — מתמטיקה ללמידה עמוקה

כיצד מחשב לומד באמצעות שיפועים, נגזרות וגרדיאנטים?

מסלול התמחות בלמידת מכונה ולמידה עמוקה • כיתות י״א–י״ב • שלושים שעות לפי מפת תכנית ההתמחות

ספר תלמיד | פרק עצמאי בסדרה המאוחדת | נוסחאות, חישובים, גרפים, קוד ותרגול



מה נראה בפרק? מודל משנה משקל כדי להקטין את פונקציית השגיאה. נלמד לחשב ולהסביר את התהליך, ולא רק להריץ קוד.

הפרק בנוי לפי שיטת הספר שלנו: בעיה אמיתית → אינטואיציה → מתמטיקה מדורגת → חישוב ידני ב־Python → ניסוי → משמעות ל־AI → תרגול עצמאי.

מפת מטרות הלמידה

הוכחת הבנה	בסיום הפרק התלמיד יוכל…
לנתח מודל חיזוי ולקבוע מה נלמד ומה מחושב	לזהות קלט, פלט, פרמטר ופונקציית שגיאה
לחשב ולהסביר סימן וגודל נגזרת	לפרש שיפוע ונגזרת
לפתור תרגילי פונקציות בשני פרמטרים	לחשב נגזרות, נגזרות חלקיות וגרדיאנט
לבצע כמה צעדי עדכון ולפרש השפעת קצב למידה	לעקוב אחר Gradient Descent
לחשב את השפעת משקל על שגיאה	להפעיל כלל שרשרת במסלול חישוב
להשוות חישוב ידני, NumPy והפרשים סופיים	לממש ולבדוק חישובים ב־Python
לצייר גרף חישוב ולפרט מה מתקדם קדימה ומה אחורה	להסביר את הקשר ל־Backpropagation

ידע קודם וחיבור למסלול

דרוש: אלגברה בסיסית, חזקות, פונקציה ליניארית, משוואת ישר, מכפלה פנימית, NumPy, רגרסיה ליניארית ומושגי משקולת, הטיה ושגיאה מפרקים 5 , 8 ו־ 9 . איננו מניחים שהלומד כבר למד חשבון דיפרנציאלי באופן שיטתי; כל סימן יוסבר בעת הופעתו.

הפרק מעוגן בפרק י״א — מתמטיקה ללמידה עמוקה (0 3 שעות) שבמפת תכנית ההתמחות. החלוקה הפנימית, דוגמאות Gradient Descent ובדיקת נגזרות הן תכנון דידקטי שלנו. התוכן המדויק של כל תת־נושא יתוקף מול הסילבוס הרשמי המלא לפני קביעת מעמדו כחומר חובה.

11.1 סיפור מסגרת: דודו רוצה לשפר תחזית

דודו בונה מערכת המסייעת למעבדת המחשבים להעריך כמה דקות נדרשות לתיקון תקלה. בתחילה הוא יוצר נוסחה פשוטה המבוססת על מספר התקלות הקודמות. המודל שלו מפיק תחזית, אבל התחזית אינה זהה לזמן האמיתי. מה צריך לשנות כדי להשתפר?

דודו יכול לנסות משקל חדש באופן מקרי, לבדוק את השגיאה ולנסות שוב. אבל כאשר למודל יש אלפי משקולות, ניסוי מקרי לכל אחת מהן אינו דרך יעילה. עלינו להבין לאיזה כיוון לשנות כל משקל, וכמה לשנות אותו.

זה תפקיד המתמטיקה בפרק: הנגזרת מתארת שינוי מקומי, הגרדיאנט מאגד נגזרות לפי פרמטר, וכלל השרשרת מאפשר להעביר מידע על השגיאה דרך רצף החישובים.

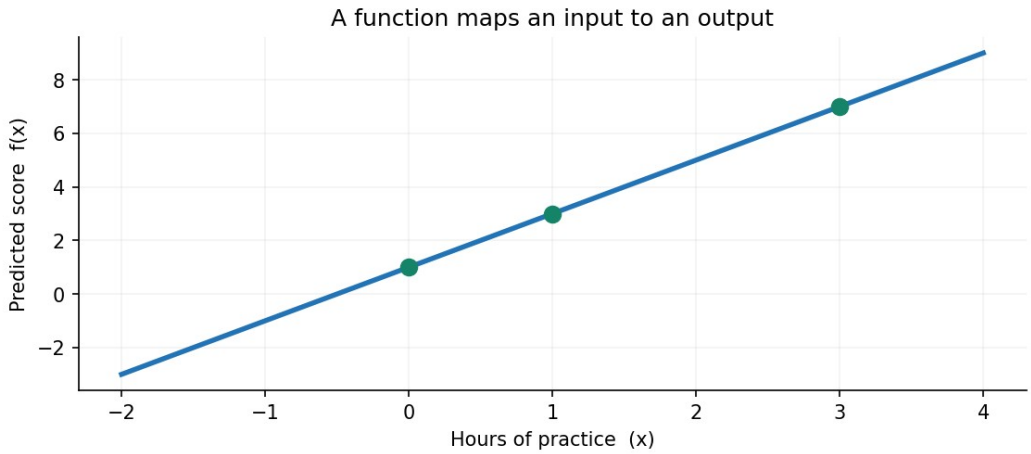
11.2 פונקציה — מכונה מתמטית המקבלת קלט ומחזירה פלט

פונקציה היא כלל המייחס לכל ערך קלט בתחום ערך פלט אחד. לדוגמה, 21 מתארת קשר בין משתנה x לערך מספרי מחושב. אם מציבים $x=3$ מקבלים 7 . אין פירוש הדבר שהעולם באמת מתנהג על פי הפונקציה הזאת; היא מודל אפשרי של הקשר.

במודל רגרסיה אנחנו כותבים $\hat{y}=wx+b$. הסימון $\hat{y}$ נקרא ״וואי כובע״ ומציין תחזית. x הוא מאפיין נתון, w הוא משקל ו־b הוא הטיה. במהלך האימון מנסים להתאים את w ואת b. בעת החיזוי, מאפייני הקלט אינם משקולות.

$$f(x)=2x+1 \quad ; \quad f(3)=2\cdot 3+1=7$$

כדי להבין מודל, עלינו תמיד לשאול: מה המשתנה שמשנים? מה מוחזק קבוע? ומה נמדד בציר האנכי?



דוגמה לקשר בין קלט לפלט. נקודות על הישר אינן הוכחה שהקשר בחיים האמיתיים הוא ליניארי.

11.3 שינוי ממוצע — השיפוע בין שתי נקודות

נניח שהתלמיד רוצה לדעת עד כמה התחזית משתנה כאשר מגדילים את ערך הקלט. נחשב תחילה שינוי ממוצע: לוקחים הפרש בין שני ערכי הפונקציה ומחלקים בהפרש בין ערכי הקלט.

$$\text{Average slope} = [f(x_2) - f(x_1)] \,/\, (x_2 - x_1)$$

עבור $f(x)=x^2$, בין $x=1$ לבין $x=2$ נקבל $f(1)=1$ ו־$f(2)=4$. לכן השיפוע הממוצע הוא $(4-1)/(2-1)=3$. משמעות התוצאה: לאורך הקטע הזה, הפונקציה גדלה בממוצע בשלוש יחידות פלט עבור יחידת קלט אחת.

אבל האם השיפוע הוא 3 גם ממש ליד $x=1$? לא. ככל שמקטינים את המרחק בין הנקודות מקבלים מידע מקומי מדויק יותר.

11.4 ממוצע שינוי לנגזרת — הרעיון של גבול

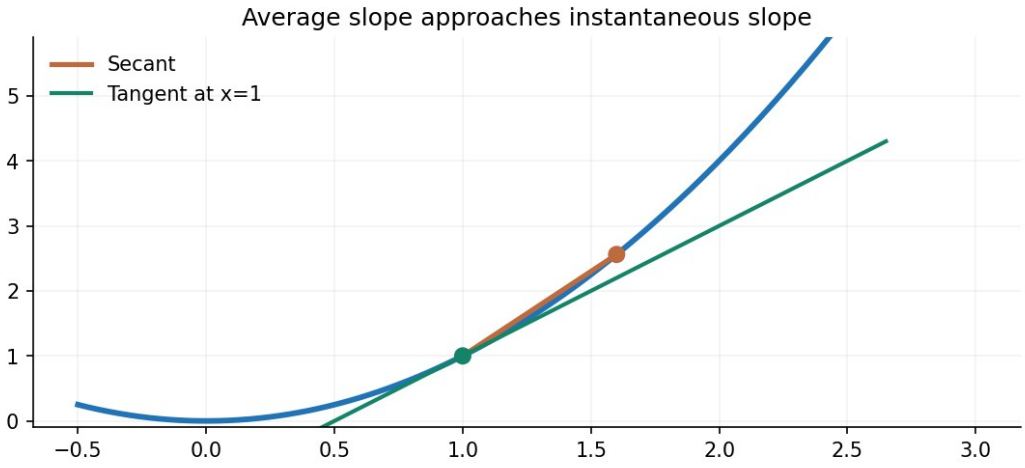
נרצה למדוד את קצב השינוי בנקודה אחת. נבחר נקודה x ונסתכל על נקודה קרובה $x+h$. כאשר h נעשה קטן יותר, מיתר המחבר את שתי הנקודות מתקרב למשיק לעקומה. אם הגבול קיים, שיפוע המשיק נקרא הנגזרת בנקודה.

$$f'(x) = \lim_{h \to 0} [f(x+h) - f(x)] \,/\, h$$

בסימון זה $\lim$ פירושו גבול; h הוא שינוי קטן בקלט; והסימן f' מבטא נגזרת. הגבול איננו הצבה ישירה של $h=0$, מפני שאז נקבל חילוק באפס. הוא מתאר למה מתקרב הביטוי כאשר h קרוב לאפס.

בדוגמה $f(x)=x^2$, כאשר $x=1$ נכתוב: $[(1+h)^2 - 1]/h = 2+h$, לכל h שאינו אפס. לכן כאשר h מתקרב לאפס, הנגזרת מתקרבת ל־2. זהו השיפוע המיידי בנקודה $x=1$.

שימו לב להבחנה: הפונקציה מחזירה גובה או ערך; הנגזרת מתארת את קצב השינוי של אותו ערך.



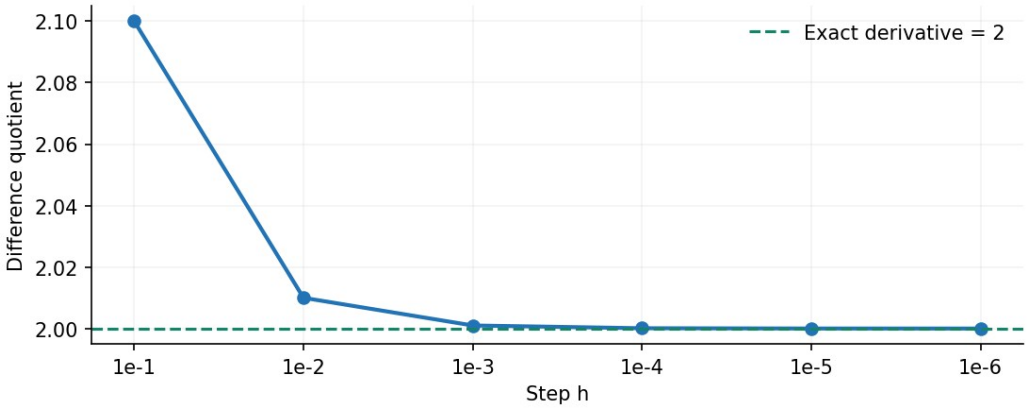
המיתר מודד שיפוע ממוצע; המשיק מתאר את השיפוע המקומי בנקודה $x=1$.

11.5 בדיקה מספרית — מאמתים נגזרת בעזרת Python

נחשב שוב את שיפוע ההפרש עבור $f(x)=x^2$ ליד $x=1$. כאשר $h=0.1$ נקבל 2.1, כאשר $h=0.01$ נקבל 2.01, וכאשר $h=0.001$ נקבל 2.001. הערכים מתקרבים לתוצאה המדויקת 2.

המחשב מאפשר לנו לחקור את הרעיון הנובע מן הנוסחה. עם זאת, מספרים במחשב מיוצגים בדיוק מוגבל. h קטן באופן קיצוני עלול דווקא ליצור שגיאת עיגול. לכן בדיקה מספרית מספקת ראיה מעשית, לא תחליף להוכחה מתמטית.

התנסות מודרכת: כתבו פונקציה המחזירה את $f(x)$, ואז חשבו את הביטוי עבור כמה ערכי h. לפני ההרצה נסו לנבא את כיוון ההתקרבות.



הפרש המנות מתקרב לערך הנגזרת המדויק. סדר הדוגמאות בגרף הוא מגודל $h=0.1$ ועד $h=0.000001$.

```
def f(x):
    return x ** 2

for h in [0.1, 0.01, 0.001, 0.0001]:
    slope = (f(1 + h) - f(1)) / h
    print(h, round(slope, 6))
```

11.6 איך מפרשים נגזרת חיובית, שלילית או אפס?

כאשר $f(x)>0$ הפונקציה עולה באופן מקומי. כאשר $f'(x)<0$ היא יורדת באופן מקומי. נגזרת אפס מתארת משיק אופקי, אך לא תמיד נקודת מינימום: היא יכולה להיות גם מקסימום או נקודה אחרת, למשל $f(x)=x^3$ בנקודה $x=0$.

גודל הנגזרת מלמד על רגישות מקומית: נגזרת 6 פירושה שינוי בקירוב של 6 יחידות פלט לכל שינוי קטן של יחידת קלט, כאשר נשארים מספיק קרובים לנקודה. זהו קירוב מקומי ולא הבטחה לשינוי לאורך מרחק גדול.

$$\Delta f \approx f'(x) \cdot \Delta x$$

למידת מכונה משתמשת בפירוש הזה כדי להעריך מה יקרה לשגיאה אם נשנה מעט את המשקל. אבל יש לבדוק את גודל הצעד: קירוב מקומי עשוי לא להספיק לצעד גדול.

11.7 כללי גזירה בסיסיים — כלי עבודה למודל

במקום לחשב גבול מההתחלה בכל פעם, אפשר להשתמש בכללי גזירה. חישוב הנגזרת של קבוע הוא אפס, משום ששינוי בקלט לא משנה אותו. הנגזרת של x היא 1, כי כל שינוי בקלט מועבר ישירות לפלט.

כלל החזקה קובע שהנגזרת של x בחזקת n היא n כפול x בחזקת $n-1$. אם יש מקדם קבוע, הוא נשאר כגורם כפל. סכום של פונקציות גוזרים רכיב אחר רכיב.

$$d(c)/dx=0 \;;\; d(x^n)/dx=n\cdot x^{n-1} \;;\; d[c\cdot f(x)]/dx=c\cdot f'(x)$$

דוגמה: אם $f(x)=3x-4x+7$, נגזור $3x^2$ לקבלת $6x$, נגזור $-4x$ לקבלת -4, ונגזור את הקבוע 7 לקבלת 0. לכן $f'(x)=6x-4$. בנקודה $x=2$, הנגזרת היא 8.

כללי הכפל והמנה יופיעו כהעשרה בסוף הפרק. לבניית ההבנה של אימון רשתות חשובה במיוחד שליטה בגזירת הרכבה באמצעות כלל השרשרת.

פונקציה	נגזרת	מה חשוב להבין
c	0	קבוע אינו משתנה עם הקלט
x	1	שינוי יחידה יוצר שינוי יחידה
x^2	$2x$	השיפוע תלוי במיקום
$3x^2-4x+7$	$6x-4$	גוזרים כל איבר בנפרד
e^x	e^x	פונקציה מעריכית שומרת צורה בנגזרת
$\ln(x)$	$1/x \ (x>0)$	לוגריתם טבעי בתחום החיובי

11.8 הקשר בין נגזרת לנקודות קיצון

במודל אנו מחפשים ערכי משקולות שמקטינים את השגיאה. כאשר מציירים את פונקציית השגיאה, מחפשים מקום שבו הערך שלה נמוך. בפונקציה חלקה, נקודת מינימום פנימית מקיימת לעיתים קרובות תנאי של נגזרת אפס.

אבל חשוב לא להפוך זאת לכלל שגוי: נגזרת אפס אינה מבטיחה מינימום, ולא כל פונקציה חלקה היא בעלת מינימום יחיד. לדוגמה, גם בשיא של גבעה הנגזרת יכולה להיות אפס.

בפונקציה $L(w)=(2w-5)^2$ נקבל מינימום כאשר $2w-5=0$, ולכן $w=2.5$. נגזור ונראה שהנגזרת מתאפסת בדיוק שם.

11.9 פונקציית שגיאה — מה מודדים במהלך האימון?

מודל מקבל מאפיין x ומחשב תחזית $\hat{y}$. להשוואה לערך הנכון y נקרא שארית או טעות חיזוי $e=\hat{y}-y$. כדי שהשגיאה החיובית והשלילית לא יתבטלו, אפשר לרבע אותה. פונקציית שגיאה אחת לדוגמה בודדת היא $L=(y-\hat{y})^2$.

כאשר יש כמה דוגמאות, מדד Mean Squared Error הוא הממוצע של ריבועי השגיאות. לעתים מגדירים הפסד עם גורם $1/2$ כדי לפשט את הנגזרת; יש לשמור על אותה הגדרה בזמן חישוב והשוואה.

$$e = \hat{y} - y \qquad ; \qquad L = e^2 \qquad ; \qquad MSE = (1/n)\ \Sigma_i(\hat{y}_i - y_i)^2$$

המטרה של אימון היא להפחית את השגיאה על נתוני האימון בלי לפגוע ביכולת ההכללה. הפחתה אינסופית של שגיאת אימון אינה יעד עצמאי: היא עלולה להוביל להתאמת יתר.

11.10 גוזרים את השגיאה ביחס למשקל אחד

נניח שבתרגיל מפושט $x=2$, התשובה הנכונה היא $y=5$, והמודל מנבא $\hat{y}=2w$ ללא הטיה. לכן הפונקציה המתארת את שגיאת המודל היא $L(w)=(2w-5)^2$.

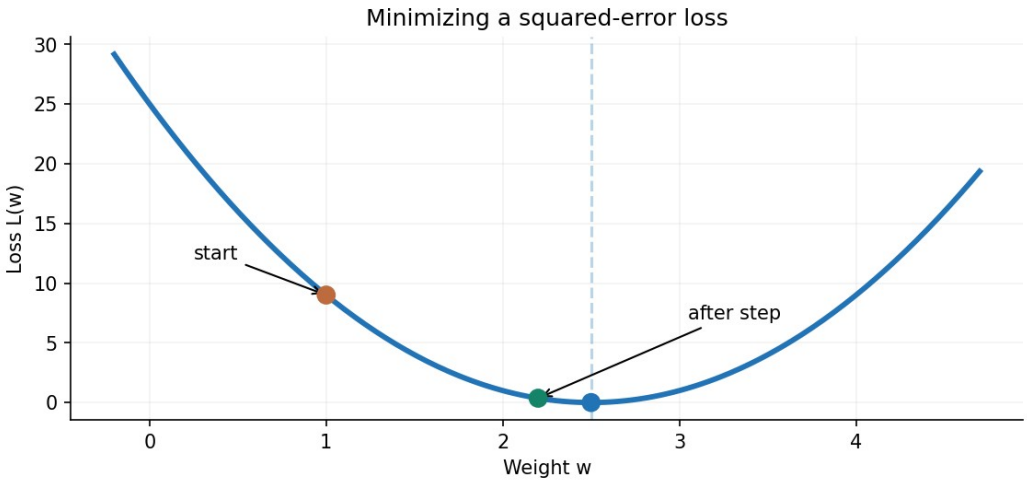
תחילה נגזור את הביטוי החיצוני: ריבוע של משהו. לפי כלל השרשרת, הנגזרת היא פעמיים הביטוי הפנימי כפול הנגזרת של הביטוי הפנימי.

$$L(w) = (2w - 5)^2$$

$$dL/dw = 2(2w-5)\cdot 2 = 8w-20$$

אם $w=1$, התחזית היא 2, והשגיאה היא 9. הנגזרת היא $8\cdot(1-20)=-12$. הסימן השלילי אומר שהגדלה קטנה של המשקל צפויה להקטין את השגיאה בנקודה הזאת. הוא לא אומר שהשגיאה עצמה שלילית.

אם $w=3$, הנגזרת היא 4, כלומר הגדלה קטנה של המשקל דווקא תעלה את השגיאה. כך ניתן לקבוע כיוון פעולה.



פונקציית השגיאה כתלות במשקל. הנגזרת מראה לאיזה כיוון לשנות אותו.

צעד עדכון מלא — Gradient Descent 11.11

ב־Gradient Descent משנים את המשקל בכיוון ההפוך לנגזרת, משום שזהו בדרך כלל הכיוון שבו השגיאה יורדת כאשר הצעד מספיק קטן. קצב הלמידה מסומן באות היוונית η (אֶטָה).

$$w_{new} = w_{old} - \eta \cdot (dL/dw)$$

בדוגמה שלנו $w=1$, $\eta=0.1$ והנגזרת היא -12. תחילה מכפילים קצב למידה בנגזרת: $0.1 \cdot (-12) = -1.2$. כעת מחסרים את הערך השלילי מהמשקל: $w_{new} = 1 - (-1.2) = 2.2$.

אחרי העדכון התחזית היא $2.2 \cdot 2 = 4.4$. השגיאה החדשה היא $(4.4-5)^2 = 0.36$, במקום 9. שימו לב: לא "בחרנו" מראש 2.2; הוא התקבל מכלל עדכון שהיה תלוי בנגזרת.

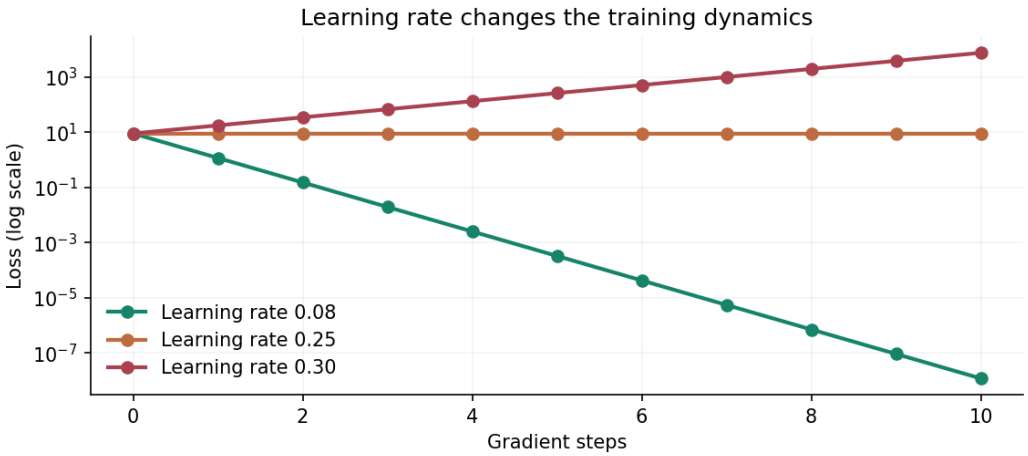
אין הבטחה שכל צעד ובכל קצב למידה יפחית את השגיאה. זוהי השערה שצריך לאמת על פונקציית השגיאה בפועל.

קצב למידה קטן מדי או גדול מדי 11.12

אם η קטן מאוד, ייתכן שהמודל יתקדם לעבר המינימום לאט. אם הוא גדול מדי, העדכון עשוי לדלג מעל המינימום ולהגדיל את השגיאה שוב ושוב. ערכי קצב הלמידה המתאימים תלויים בצורת פונקציית השגיאה.

בדוגמה הריבועית $L(w) = (2w-5)^2$, נגזרת השגיאה היא $8w - 20$. כאשר $\eta=0.1$, משקל התחלתי 1 מתקרב אל 2.5. לעומת זאת, כאשר $\eta=0.3$, צעד העדכון עלול להרחיק אותנו מן המינימום ולגרום לעלייה בשגיאה.

בתרגול נציג שני מסלולים ונבקש מתלמידים לנבא מה יקרה לאחר עוד שלושה צעדים לפני הרצת הקוד.



השוואת קצבי למידה באותה פונקציית שגיאה; הגרף משתמש בסולם לוגריתמי בציר האנכי.

```
def loss(w):
    return (2*w - 5)**2

def gradient(w):
    return 8*w - 20

w = 1.0
eta = 0.1
for step in range(6):
    print(step, round(w, 4), round(loss(w), 5))
    w = w - eta * gradient(w)
```

11.13 ממשקל אחד לשני פרמטרים: w ו־b

מודל אמיתי יכול להשתמש גם בהטיה b. כעת השגיאה תלויה בשני מספרים, ולכן נוכל לשאול בנפרד: כיצד היא משתנה כשמגדילים מעט את w ומשאירים את b קבוע? וכיצד היא משתנה כשמגדילים את b ומשאירים את w קבוע?

נגזרת ביחס למשתנה אחד כאשר האחרים קבועים נקראת נגזרת חלקית. סימונה ∂. אם $L(w,b)=(w+b-4)^2$, נגזור לפי w ו־b ונקבל בשני המקרים $2(w+b-4)$, מפני שכל אחד מהמשתנים מופיע כאן במקדם 1.

$$\partial L/\partial w = 2(w+b-4) \quad ; \quad \partial L/\partial b = 2(w+b-4)$$

אם w=1 ו־b=0, השגיאה היא 9, וכל אחת משתי הנגזרות שווה −6. בקצב למידה 0.1, שני הערכים גדלים ב־0.6. כעת w=1.6, b=0.6 והתחזית 2.2; השגיאה החדשה היא $(2.2-4)^2 = 3.24$.

בדוגמה הזאת יש יותר מזוג אחד של משקולת והטיה שנותנים אותה תחזית. כך אנו מבחינים בין הקטנת שגיאה לבין פתרון יחיד לפרמטרים.

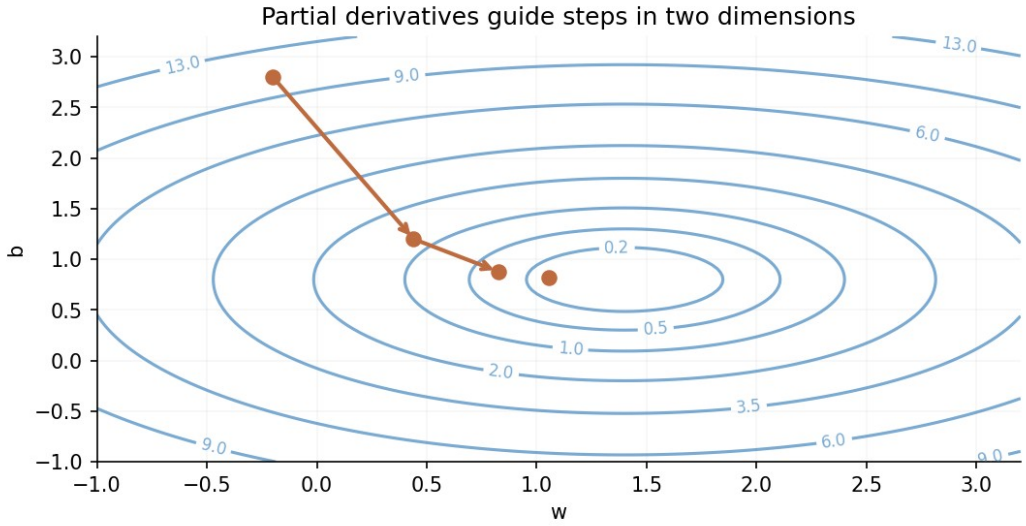
11.14 גרדיאנט — וקטור נגזרות חלקיות

כאשר לפונקציית השגיאה יש כמה פרמטרים, אוספים את הנגזרות החלקיות שלה לווקטור אחד הקרוי גרדיאנט (Gradient). כל רכיב בו מתאר רגישות של השגיאה לשינוי קטן בפרמטר מסוים.

$$\nabla L(w,b) = (\partial L/\partial w,\ \partial L/\partial b)$$

בכל איטרציה אפשר לעדכן את וקטור הפרמטרים כולו: $\theta_{new} = \theta_{old} - \eta \nabla L(\theta_{old})$. זהו אותו רעיון של צעד אחד, כעת במספר ממדים.

באזור חלק של הפונקציה, הגרדיאנט מצביע לכיוון העלייה המהירה ביותר לפי המרחק האוקלידי. לכן הכיוון השלילי שלו משמש לתנועה מקומית לכיוון ירידה. שוב: מדובר במידע מקומי, ולא בהבטחה למצוא מינימום גלובלי.



קווי גובה של פונקציית שגיאה בשני פרמטרים; החצים מדגימים תנועה לכיוון ערכי שגיאה קטנים.

11.15 דוגמה מלאה: גרדיאנט של MSE על שתי תצפיות

נשתמש בשתי דוגמאות אימון: $x=1$ עם $y=3$, ו־$x=2$ עם $y=5$. המודל הוא $\hat{y}=wx+b$. נתחיל במשקל $w=1$ ובהטיה $b=0$. התחזיות יהיו 1 ו־2, ולכן השאריות הן -2 ו־-3.

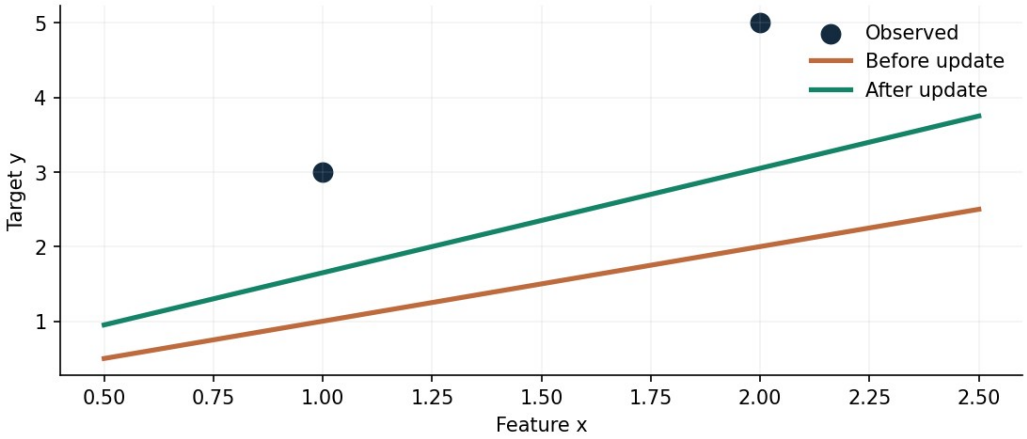
ממוצע ריבועי השגיאות הוא $(4+9)/2 = 6.5$. כאשר גוזרים את MSE ביחס למשקל, כל שארית מוכפלת גם בערך המאפיין שלה; ביחס להטיה מכפילים ב־1. כך מקבלים:

$$\partial \mathrm{MSE}/\partial w = (2/n)\,\Sigma_i\,(\hat{y}_i - y_i)\cdot x_i = -8$$

$$\partial \mathrm{MSE}/\partial b = (2/n)\,\Sigma_i\,(\hat{y}_i - y_i) = -5$$

בקצב למידה 0.05 המשקל החדש הוא 1.4 וההטיה החדשה 0.25. התחזיות הופכות ל־1.65 ול־0.35. השגיאה החדשה היא $(1.8225+3.8025)/2 = 2.8125$. ירדנו מ־$6.5$ ל־2.8125 בצעד אחד.

מדוע חישבנו את ממוצע השגיאות ולא את סכומן? משום שבדוגמה הזאת בחרנו במפורש ב־MSE. אם היינו מגדירים פונקציית שגיאה אחרת, גם הנגזרת הייתה משתנה.



שתי דוגמאות אימון, קו חיזוי לפני עדכון וקו חיזוי אחריו. הקווים הם המחשה לשינוי פרמטרים, לא להכללה.

שלב	לפני	אחרי עדכון אחד
משקל w	1.00	1.40
הטיה b	0.00	0.25
תחזית ל־$x=1$	1.00	1.65
תחזית ל־$x=2$	2.00	3.05
MSE	6.5000	2.8125

11.16 כלל השרשרת — איך נגזרת עוברת דרך שלבים?

חישוב של מודל אינו פעולה אחת. תחילה מחשבים $z=wx+b$; אחר כך ייתכן שמפעילים פונקציה; לאחר מכן מחשבים שגיאה. אם משנים את w, התוצאה הסופית משתנה דרך כל השלבים. כלל השרשרת מאפשר למדוד את ההשפעה הכוללת.

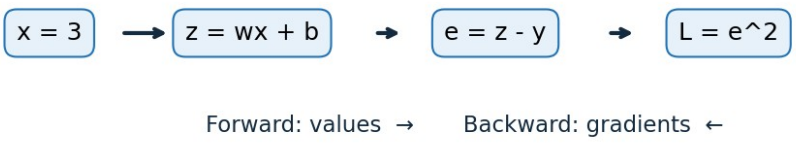
למשל, נניח $x=3,\ w=2,\ b=1$ ו־$y=10$. תחילה $z=2\cdot3+1=7$. השארית היא $e=z-y=-3$, ופונקציית השגיאה $L=e^2=9$.

$$\frac{dL}{dw} = \frac{dL}{de}\cdot\frac{de}{dz}\cdot\frac{dz}{dw}$$

$$\frac{dL}{de} = 2e = -6 \quad ; \quad \frac{de}{dz}=1 \quad ; \quad \frac{dz}{dw}=x=3$$

$$\frac{dL}{dw} = (-6)\cdot1\cdot3 = -18$$

בדומה, $dL/db = -6$ משום שנגזרת z ביחס ל־b היא 1. סימון קשתות בגרף החישוב מאפשר לעקוב אחרי ההשפעה בלי לאבד אף שלב.



גרף חישוב מקורי: הערכים מחושבים קדימה, והנגזרות מחושבות אחורה באמצעות כלל השרשרת.

11.17 עדכון משקולת והטיה אחרי כלל השרשרת

נמשיך את הדוגמה הקודמת ונבחר קצב למידה $\eta=0.01$. המשקל החדש הוא $2-0.01\cdot(-18)=2.18$. ההטיה החדשה היא $1-0.01\cdot(-6)=1.06$. עכשיו נחשב תחזית מחדש.

$$z_{new} = 2.18\cdot 3 + 1.06 = 7.60$$

$$L_{new} = (7.60 - 10)^2 = 5.76$$

השגיאה ירדה מ־9 ל־5.76. בניגוד לעדכון פרספטרון על טעות סיווג, כאן העדכון מבוסס על נגזרות של פונקציית שגיאה רציפה. ההבחנה הזאת חיונית בדרך להבנת אימון רשת עצבית.

בפונקציה מרובת שכבות, כל שכבה תלויה בקודמתה. לכן אי אפשר לחשב את השפעת המשקולות הראשונות על השגיאה הסופית בלי להעביר מידע לאורך כל שרשרת החישוב.

11.18 פונקציות הפעלה: מדוע לא מספיק חיבור ליניארי?

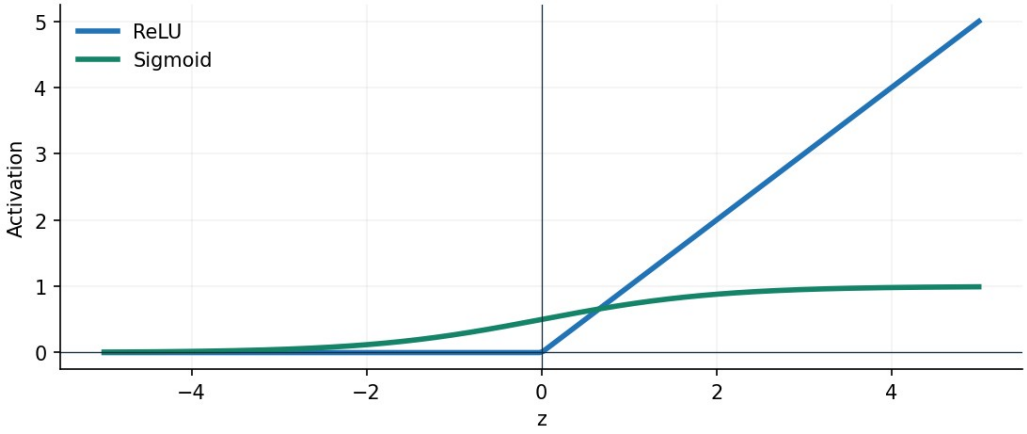
אם מחברים שכבות המבצעות רק צירופים ליניאריים, הרכב השכבות עדיין יכול להיות מוצג כפעולה אפינית אחת. כדי ללמוד דפוסים מורכבים כגון XOR, מוסיפים לרשת פונקציות הפעלה לא־ליניאריות.

פונקציית ReLU מחזירה 0 לקלט שלילי ואת הקלט עצמו לקלט חיובי. פונקציית Sigmoid מחזירה ערך בין 0 ל־1, ולכן משמשת לעתים כשלב פלט בסיווג בינארי. הפלט שלה לבדו אינו מבטיח הסתברות מכוילת.

$$\mathrm{ReLU}(z)=\max(0,z) \quad ; \quad \mathrm{sigmoid}(z)=1/(1+e^{-z})$$

נגזרת ReLU היא 0 לערכים שליליים ו־1 לערכים חיוביים; בנקודה 0 הנגזרת המתמטית אינה קיימת, ובמימושים נהוג לבחור ערך תת־נגזרת או מוסכמה. נגזרת Sigmoid היא $\sigma(z)\cdot[1-\sigma(z)]$.

בפרק זה אנו מבינים את הרעיון המתמטי. את אימון הרשת המלא נלמד בפרק 12.



שתי פונקציות הפעלה נפוצות: *ReLU* ו־*Sigmoid*. הן יוצרות אי־ליניאריות במודל.

11.19 דוגמת העמקה — כלל השרשרת דרך Sigmoid

נבנה יחידה אחת: $z=wx+b$, אחר כך $p=\sigma(z)$, ולבסוף $L=0.5(p-y)^2$. המספר 0.5 הוא בחירה טכנית שמפשטת את הנגזרת; בדוגמה זו איננו משתמשים ב־Cross Entropy.

ניקח $x=2, y=1, w=0, b=0$. נקבל $z=0$ ולכן $p=0.5$. השגיאה ההתחלתית היא $(0.5-1)^2=0.125$.

$$dL/dw = (p-y)\cdot p(1-p)\cdot x$$

כעת נציב: $dL/dw=(-0.5)\cdot 0.25\cdot 2=-0.25$. עבור ההטיה הנגזרת היא $(0.5-1)\cdot 0.25=-0.125$.

בקצב למידה 0.4 נקבל $w=0.1$ ו־$b=0.05$. הסכום המשוקלל החדש יהיה $z=0.25$, ולכן $p\approx 0.5622$. השגיאה החדשה תהיה בקירוב 0.0959, כלומר נמוכה מן השגיאה הקודמת.

זהו גשר מפורש אל Backpropagation, אך אינו מימוש מלא של רשת בעלת שכבות רבות.

11.20 Gradient Checking — כיצד בודקים נגזרת שחושבה ביד?

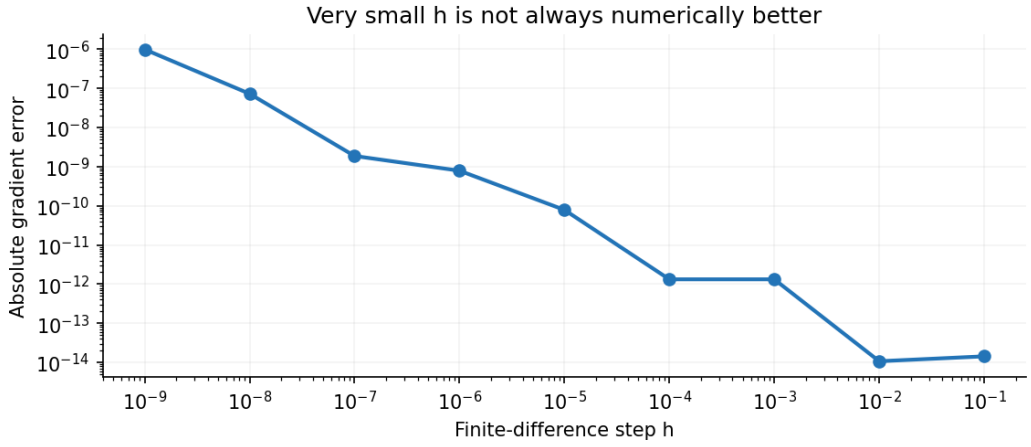
כאשר מפתחים פונקציית שגיאה מורכבת, טעות באחד מגורמי כלל השרשרת עלולה לגרום לאימון לא נכון. דרך אבחון מועילה היא להשוות את הנגזרת האנליטית לנגזרת מספרית המחושבת באמצעות הפרשים סופיים.

$$f'(w) \approx [f(w+h) - f(w-h)] / (2h)$$

הדוגמה נקראת הפרש מרכזי. נבחר h שאינו גדול מדי ואינו זעיר מדי. הנוסחה נותנת קירוב בדרך כלל מדויק יותר מהפרש חד־צדדי עבור פונקציה חלקה, אך אינה עובדת באותה צורה בנקודות לא־גזירות.

כדאי לבדוק הבדלים מוחלטים ויחסיים, להתייחס לסולם המספרים ולוודא שהפונקציה הדטרמיניסטית נבדקת באותם ערכי קלט.

בדיקת גרדיאנט היא כלי פיתוח ובקרה. במהלך אימון רגיל של רשת עמוקה נעדיף מנגנוני גזירה אוטומטיים ולא נחשב הפרשים סופיים לכל משקל.



דוגמה מספרית להשפעת גודל צעד ההפרש על שגיאת קירוב הנגזרת; מספרים זעירים מאוד עלולים לפגוע בדיוק.

```
def loss(w):
    return (2*w - 5)**2

w = 1.0
h = 1e-5
numerical = (loss(w+h) - loss(w-h)) / (2*h)
analytical = 8*w - 20
print('numerical:', numerical)
print('analytical:', analytical)
```

11.21 Gradient Descent במאגרי נתונים: Batch, Mini-batch ו־SGD

כאשר יש מאגר גדול, מחשבים גרדיאנט של שגיאה ממוצעת על מספר דוגמאות. ב־Batch Gradient Descent משתמשים בכל דוגמאות האימון לחישוב צעד אחד. ב־Stochastic Gradient Descent הקלאסי משתמשים בדוגמה אחת בכל צעד. ב־Mini-batch משתמשים בקבוצה קטנה של דוגמאות.

Mini-batch מאפשר עבודה וקטורית יעילה ויוצר גרדיאנט שהוא אומדן המבוסס על הקבוצה. גודל הקבוצה משפיע על עלות חישוב ועל תנודתיות העדכונים. אין מספר קסם אחד המתאים לכל משימה.

חשוב להבחין בין Epoch — מעבר על כלל דוגמאות האימון לפי שיטת האימון — לבין Step — פעולת עדכון אחת של הפרמטרים. בתקופה אחת עשויים להיות הרבה צעדים.

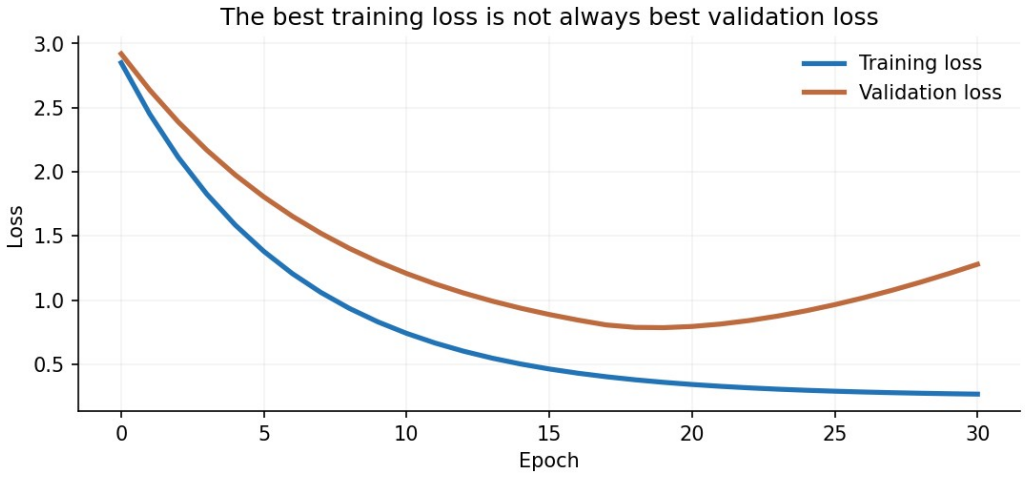
בדוגמה בעלת 100 תצפיות וקבוצות בגודל 20, מעבר מלא כולל בדרך כלל חמישה צעדי עדכון, אם אין דוגמה שנגרעת והקבוצות אינן חופפות.

11.22 האם ירידה בשגיאת אימון מספיקה?

לא. כמו בפרק 2 . 5 , מודל נדרש לבצע תחזיות גם על נתונים שלא שימשו להתאמת המשקולות. לכן נעקוב בנפרד אחרי שגיאת האימון ושגיאת הוולידציה. אם שגיאת האימון ממשיכה לרדת ושגיאת הוולידציה עולה, זו יכולה להיות אינדיקציה להתאמת יתר.

זו אינה הוכחה יחידה: צריך לבדוק איכות נתונים, מדדי הערכה, חלוקה תקינה, ייצוג אוכלוסיות ושונות ניסוי. לפעמים שגיאת ולידציה מתנדנדת בגלל מאגר קטן.

בפרויקטים תלמידים יתעדו ערכי קצב למידה, מספר Epochs, חלוקת הנתונים וניסוי השוואה. לא בוחרים פרמטרים באמצעות קבוצת Test שנועדה להערכה הסופית.



דוגמת עקומות אימון וולידציה סכמטיות. התלמיד צריך לזהות בעיית הכללה, לא רק את ערך השגיאה הנמוך ביותר.

11.23 מתי הנגזרת אינה מספר פשוט?

קיימות פונקציות שאין להן נגזרת בחלק מהנקודות. דוגמה מוכרת היא הערך המוחלט $|x|$ ב־$x=0$. בפונקציות בעלות פינות משתמשים לעיתים בכלים מתמטיים נוספים כגון תת־גרדיאנט, או בוחרים מוסכמה חישובית כפי שנהוג ב־ReLU.

יש גם אזורים שבהם הנגזרת קטנה מאוד או גדולה מאוד. ברשתות עמוקות תופעות של גרדיאנטים נעלמים ומתפוצצים עלולות להקשות על אימון. כאן נציג את הרעיון בלבד: אם גרדיאנט כמעט אפס, שינוי הפרמטר עשוי להיות זעיר; אם הוא עצום, צעד עלול להיות לא יציב.

את ההתמודדות המקצועית עם התופעות נבחן בפרקי הרשתות, לרבות בחירת פונקציות הפעלה, אתחול משקולות ואופטימייזרים מתקדמים.

11.24 הקשר המדויק בין פרק 11 לפרק 12

בפרק 11 למדנו כיצד מחשבים תחזית, שגיאה, נגזרת ושינוי משקולת. בפרק הבא נראה כיצד רשת שלמה מפעילה שכבות רבות: תחילה Forward Pass מחשב תוצאות, אחר כך Loss מודד שגיאה, ואז Backward Pass משתמש בכלל השרשרת כדי לחשב את הגרדיאנט לכל משקל.

יש להבדיל בין שלוש פעולות: Backpropagation מחשבת גרדיאנטים; אופטימייזר כגון SGD משתמש בהם לעדכון פרמטרים; והערכת ביצועים בודקת אם המודל מכליל לנתונים חדשים.

חשיבה במבנה הזה מונעת ערבוב בין פעולה מתמטית, כלי תוכנה ותוצאה לימודית.

שאלות עצירה ובדיקת הבנה

- אם הנגזרת של פונקציית שגיאה חיובית בנקודה מסוימת, מה המשמעות של הגדלה קטנה במשקל? האם ירידה בשגיאה מובטחת גם לצעד גדול?
- מדוע $f'(x)=0$ אינה הוכחה לכך שמצאנו מינימום?

- מה ההבדל בין פונקציית שגיאה לבין הגרדיאנט שלה?
- במודל $\hat{y}=wx+b$, מה מחזירה הנגזרת החלקית לפי w ומה מחזירה הנגזרת לפי b?
- כיצד יודעים מהו כיוון החישוב של הערכים ומהו כיוון החישוב של הגרדיאנטים בגרף חישוב?
- מה ההבדל בין Backpropagation לבין Gradient Descent?
- מדוע בדיקת ירידה בשגיאת אימון אינה מספיקה כדי להכריז על מודל מצליח?

תרגול מדורג — שלוש רמות

רמה א׳: מיומנויות יסוד

- הציבו $x=-2,\ x=0$ ו־$x=3$ בפונקציה $f(x)=3x-1$. פירשו משמעות של כל פלט.
- חשבו שיפוע ממוצע של $f(x)=x^{2}$ בין $x=2$ ל־$x=3$. השוו לנגזרת בנקודה $x=2$.
- גזרו $f(x)=4x-6x+5$ וחשבו $f'(1)$. הסבירו במילים את הסימן.
- נתון $L(w)=(w-4)^{2}$, $w=1$, $\eta=0.1$. חשבו נגזרת וצעד עדכון אחד.

רמה ב׳: חיבור מתמטיקה לאימון

- נתון $\hat{y}=wx+b$, $x=2,\ y=7,\ w=1,\ b=0$. חשבו תחזית, שגיאה וריבועה, ואז את שתי הנגזרות החלקיות.
- בדוגמה $L(w)=(2w-5)^{2}$ השוו צעד עדכון עם $\eta=0.05$ ו־$\eta=0.3$. איזו מסקנה אפשר להסיק מן התוצאה ואיזו לא?
- כתבו קוד המשווה בין נגזרת אנליטית לקירוב מספרי בשלוש נקודות שונות. הציגו טבלה ופרשנות.
- שרטטו גרף חישוב של $z=3w+1$ ושל $L=(z-10)^{2}$. ציינו מה ידוע בכל צומת ומהי הנגזרת המקומית.

רמה ג׳: חשיבה של מפתח AI

- במודל עם שני מאפיינים הגדירו MSE עבור שלוש דוגמאות. חשבו ידנית צעד שלם עבור שני משקולות והטיה.
- ניסוי מציג ירידה עקבית ב־Training Loss ועלייה ב־Validation Loss. הציעו שתי השערות ניתנות לבדיקה וניסוי מבוקר.
- מדוע לא נכון לפרש את פלט Sigmoid תמיד כהסתברות אמינה? הציעו דרך לבחון את האמינות.
- השוו חישוב גרדיאנט באמצעות הפרשים סופיים לבין גזירה אוטומטית: יתרון, מגבלה ושימוש מתאים לכל שיטה.

שאלת הערכה מקורית בסגנון בגרות

במערכת של דודו מתקבל מודל $\hat{y}=wx+b$. נתונים שני זוגות אימון: $(x=1,y=3)$, $(x=2,y=5)$. מגדירים MSE כממוצע ריבועי השאריות. בתחילת הניסוי $w=1$ ו־$b=0$, וקצב הלמידה הוא $\eta=0.05$.

סעיף	דרישה
א׳	חשבו את שתי התחזיות, השאריות וערך MSE ההתחלתי. (שמונה נקודות)
ב׳	גזרו את MSE לפי w ולפי b, והסבירו מה משמעות כל נגזרת. (שתים־עשרה נקודות)
ג׳	בצעו צעד Gradient Descent אחד וחשבו מחדש את שתי התחזיות. (עשר נקודות)
ד׳	חשבו את MSE החדש. נמקו אם הצעד הפחית את שגיאת האימון. (שש נקודות)
ה׳	הסבירו מדוע אין להסיק מכך שהמודל יצליח גם על מחשבים שלא נראו באימון. (ארבע נקודות)

זו שאלה מקורית לתרגול, ברוח ניתוח מתמטי ואלגוריתמי. אין מדובר בשאלת בגרות רשמית, והמחוון המוצע אינו מחוון משרד החינוך.

תכנית הוראה מוצעת — שלושים שעות

תוצר מרכזי	שעות	מפגש
תרשים פונקציה וחישוב שינוי ממוצע	3	1 — פונקציה ושיפוע
חישוב נגזרת ביד ובדיקה מספרית	3	2 — נגזרת וגבול
דף תרגול ונימוק סימן נגזרת	3	3 — כללי גזירה
טבלת עדכון משקולות	3	4 — Loss ו־Gradient Descent
גרף שגיאה עבור כמה קצבי למידה	3	5 — קצב למידה וניסויים
חישוב בשני פרמטרים	3	6 — נגזרות חלקיות וגרדיאנט
צעד אימון מלא מוקטָן	3	7 — MSE על כמה תצפיות
גרף חישוב והעברת נגזרות	3	8 — כלל השרשרת
בדיקת גרדיאנט וקשר ל־Sigmoid	3	9 — הפעלות ובדיקת נגזרות
מעבדה עצמאית ומבחן מסכם	3	10 — סיכום ויישום

מילון מונחים

מונח	הגדרה לתלמיד
נגזרת — Derivative	קצב השינוי המקומי של פונקציה
נגזרת חלקית — Partial derivative	קצב שינוי ביחס לפרמטר אחד כאשר אחרים קבועים
גרדיאנט — Gradient	וקטור הנגזרות החלקיות של פונקציה
פונקציית שגיאה — Loss	מספר המודד פער בין תחזית לערך מטרה לפי כלל מוגדר
קצב למידה — Learning rate	גודל צעד העדכון היחסי
Epoch	מעבר דרך נתוני האימון לפי שיטת האימון
כלל השרשרת — Chain rule	כלל לחישוב נגזרת של הרכבת פונקציות
Backpropagation	הליך יעיל לחישוב גרדיאנטים ברשת באמצעות כלל השרשרת
גזירה אוטומטית — Autodiff	מערכת המחשבת נגזרות של פעולות מתוך גרף חישוב

מקורות מקצועיים והבחנה בין חובה להרחבה

מקור מסגרת: מאגר Data-Science-ML-Course של מסלול ההתמחות בלמידת מכונה ולמידה עמוקה, פרק י״א — מתמטיקה ללמידה עמוקה, 30 שעות. שייכות הפרק וההיקף אומתו לפי קובץ מפת המסלול; סעיפי החובה המפורטים דורשים הצלבה נוספת עם מסמך המקור המלא של משרד החינוך.

מקורות השראה מתמטיים: Deisenroth, Faisal & Ong, Mathematics for Machine Learning (Cambridge University Press, 2020), בעיקר חשבון וקטורי ואופטימיזציה; Zhang, Lipton, Li & Smola, Dive into Deep Learning, פרקי Calculus ו־Automatic Differentiation; וכן Understanding Deep Learning של Simon J. D. Prince להעמקה של קשר הנגזרות לרשתות.

גישת הוראה מעשית: Hands-On Machine Learning של Aurélien Géron; Grokking Machine Learning של Luis G. Serrano. ההסברים, החישובים, השאלות והגרפים בפרק זה נכתבו ונוצרו באופן מקורי; לא הועתקו איורים או קטעי ספרים.

קישורים לקריאה נוספת: https://mml-book.github.io/ | https://d2l.ai/chapter_preliminaries/calculus.html | https://d2l.ai/chapter_preliminaries/autograd.html

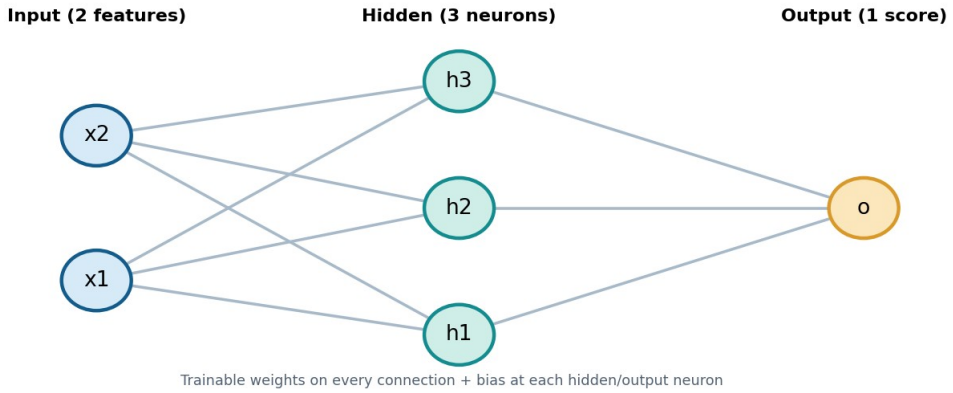
סימון פדגוגי: נגזרות וגרדיאנטים הם בסיס הפרק; Gradient Checking, בחינת יציבות מתמטית ופירוש מפורט של Sigmoid מופיעים כאן כהרחבה מקצועית וכהכנה לפרקים מתקדמים. מעמד החובה המדויק נקבע רק לפי תכנית הלימודים העדכנית.

התמחות למידת מכונה ולמידה עמוקה — 5 יחידות לימוד

פרק 12 | רשתות נוירונים עמוקות

מרשת של נוירונים עד לאימון מודל שמזהה דפוסים מורכבים

מהדורת תלמיד • ספר מאוחד יחידה 5 + התמחות • היקף פרק התמחות במפת התכנית: 20 שעות לימוד • כל הדוגמאות והאיורים מקוריים.



איור: מבנה רשת קדמית: שכבת קלט, שכבה חבויה ושכבת פלט.

כיצד להשתמש בפרק

הפרק בנוי כך שכל פעולה מתמטית מקבלת תחילה משמעות מילולית והמחשה. אחר כך נחשב ביד, נבדוק את התוצאה באמצעות NumPy ולבסוף נאמן רשת באמצעות ספריית למידת מכונה. ליד הנוסחאות נפרש כל סימן, ולא נצפה שתלמיד יבין רשת עצבית מתוך קוד בלבד.

הידע הקודם הדרוש: וקטורים ומטריצות, מכפלה פנימית, תקנון נתונים, רגרסיה ליניארית, הפרספטרון, נגזרות, כלל השרשרת ו־Gradient Descent. הפניות לפרקים קודמים מחליפות הוראה כפולה.

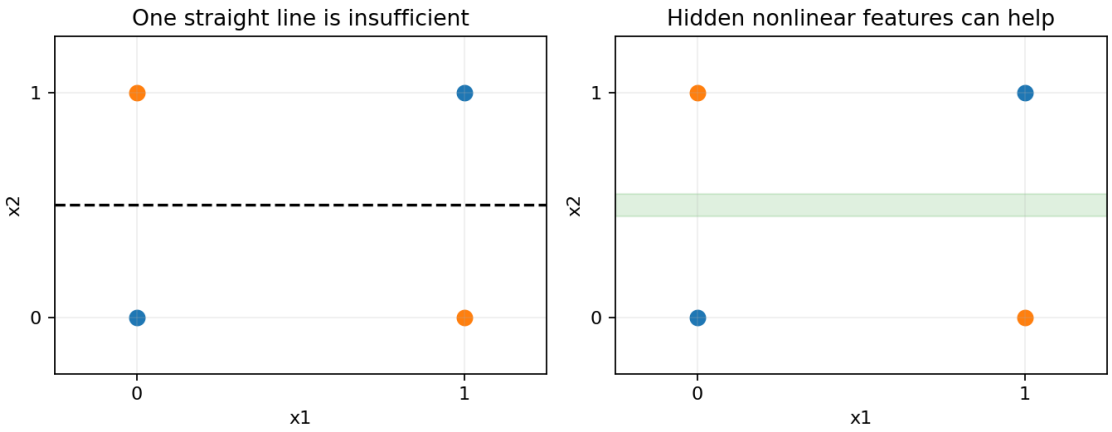
בסיום הפרק התלמיד יוכל...	ראיה לשליטה
להסביר מדוע רשת עם שכבות חבויות שונה ממסווג ליניארי	להמחיש אי־ליניאריות בעזרת XOR
לחשב מעבר קדימה של רשת קטנה	טבלת z, h ופלט עם משקולות נתונות
לחשב נגזרות בצעד Backpropagation	להציג נגזרות למשקולות ולהטיות
לאמן ולבדוק רשת על נתונים חדשים	מחברת עם חלוקה נכונה לנתוני אימון ובדיקה
לפרש גרף שגיאה ולהצביע על מגבלות	פסקת מסקנה מקצועית ומבוססת תוצאות

1. סיפור הפתיחה — מדוע פרספטרון יחיד אינו מספיק?

דודו בונה מערכת המזהה על סמך שני מאפיינים אם קיימת תופעה מסוימת במעבדה. בחלק מהמקרים אפשר להפריד את הדוגמאות בקו ישר: נתונים בצד אחד שייכים למחלקה א׳, ובצד השני למחלקה ב׳. אבל קיימים דפוסים שבהם אותו קו לא יצליח להפריד נכון את כל הנתונים. אם מציירים את ארבע

הדוגמאות של XOR, שתי הנקודות החיוביות נמצאות בפינות נגדיות. אין קו ישר יחיד שיפריד אותן משתי השליליות.

הבעיה איננה שהפרספטרון "התאמן מעט מדי". המגבלה נמצאת בצורה המתמטית של המודל: חישוב סכום משוקלל ולאחריו החלטה באמצעות סף. כדי לתאר גבולות מורכבים יותר נרצה ליצור ייצוגי ביניים חדשים, המתאפשרים באמצעות שכבות חבויות ופונקציות הפעלה לא־ליניאריות.



איור: דוגמת *XOR*: המבנה הגיאומטרי אינו ניתן להפרדה בקו ישר יחיד.

הבהרה מדעית: רשת עם כמה שכבות ליניאריות בלבד עדיין שקולה להעתקה ליניארית או אפינית אחת. אי־הליניאריות של פונקציית ההפעלה היא רכיב חיוני ביכולת לייצג דפוסים לא־ליניאריים.

2. מהי רשת נוירונים רב־שכבתית?

רשת נוירונים היא פונקציה שמורכבת מחישובים רבים המחוברים זה לזה. כל נוירון בשכבה מקבל מספרים מהשכבה הקודמת, מחשב סכום משוקלל ומוסיף הטיה. לאחר מכן הוא מפעיל פונקציית הפעלה. הפלטים של כל הנוירונים בשכבה נעשים לקלט של השכבה הבאה. כך נוצר רצף של ייצוגים מספריים של אותה תצפית.

השכבה הראשונה מקבלת את המאפיינים המקוריים. שכבה חבויה אינה מקבלת ישירות את התווית הנכונה, אלא יוצרת מאפייני ביניים הנלמדים במהלך האימון. שכבת הפלט מחזירה מספר לחיזוי ברגרסיה, או ציונים/הסתברויות בסיווג — בהתאם למבנה הרשת ולפונקציית הפלט.

מבחינת המונח "עמוקה": מקובל לתאר רשת כבעלת מספר שכבות חישוב נלמדות. רשת בעלת שכבה חבויה אחת היא דוגמת ההוראה הפשוטה ביותר לרשת רב־שכבתית; במהלך ההמשך נלמדות גם רשתות שבהן כמה שכבות חבויות.

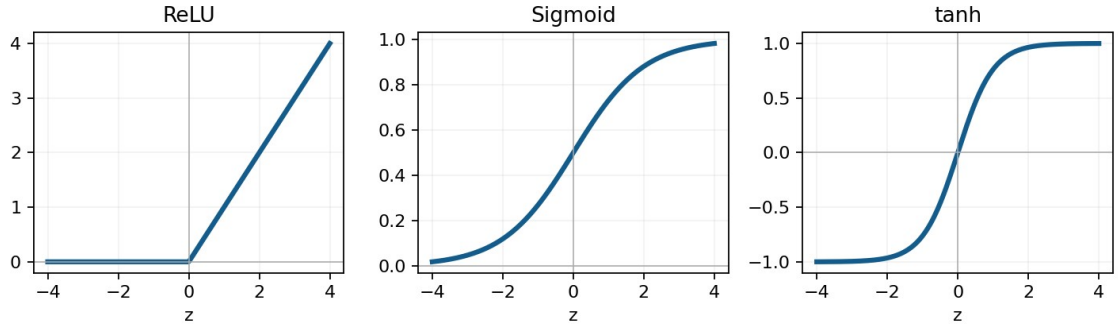
מונח	מה פירושו?	ממה חשוב להיזהר?
נוירון (Neuron)	יחידת חישוב המקבלת קלטים ומחזירה ערך	אין זה תא עצב ביולוגי
משקל (Weight)	מקדם נלמד המגדיל, מקטין או הופך השפעת מאפיין	המשקל אינו בהכרח חשיבות סיבתית
הטיה (Bias)	מספר נוסף המחושב בכל נוירון	אין לבלבל עם הטיה חברתית בנתונים
שכבה חבויה	ייצוג ביניים שנלמד מן הדוגמאות	אין הכרח שאפשר לתת לכל

מונח	מה פירושו?	ממה חשוב להיזהר?
		נוירון שם אנושי
Epoch	מעבר מלא על נתוני האימון	אינו שקול תמיד לצעד עדכון אחד
Batch	קבוצת דוגמאות לעדכון משקולות	גודל ה־Batch משפיע על חישוב ואימון

3. מדוע נדרשת פונקציית הפעלה?

נניח שהשכבה הראשונה מחשבת $z_1 = W_1x + b_1$, והשכבה השנייה מחשבת $z_2 = W_2z_1 + b_2$, בלי שום פעולה לא־ליניארית באמצע. אם מציבים את הביטוי הראשון בשני, מתקבלת עדיין נוסחה מהצורה $Wx + b$. למעשה אפשר לאחד את כל המשקולות למטריצה אחת ואת ההטיות לווקטור אחד. לכן הוספת שכבות ללא פונקציית הפעלה אינה משחררת אותנו מהמגבלה הליניארית.

פונקציית ההפעלה שוברת את הפשטות הזאת. ערכים מסוימים משתנים באופן שונה מאחרים, והרשת מסוגלת ליצור צורות החלטה מורכבות יותר. נדגים שלוש פונקציות נפוצות ונבחין היכן משתמשים בכל אחת.



איור: שלוש פונקציות הפעלה: *ReLU, Sigmoid* ו־*tanh*.

```
ReLU(z) = max(0, z)
```

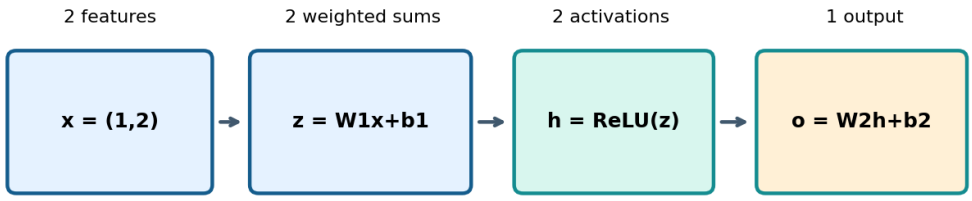
```
sigmoid(z) = 1 / (1 + exp(-z))
```

ReLU מחזירה אפס לכל קלט שלילי ומעבירה קלט חיובי כמו שהוא. Sigmoid ממפה מספר ממשי לערך בין אפס לאחד; כאשר משתמשים בה כפלט של מסווג בינארי ומתאימים פונקציית אובדן מתאימה, התוצאה יכולה להתפרש כהערכת הסתברות. tanh מפיקה ערכים בין מינוס אחד לאחד. הבחירה בפונקציית הפעלה קשורה לתפקיד השכבה וליציבות האימון, לא רק למראה הגרף.

ל־ReLU נגזרת אפס בחלק השלילי ונגזרת אחת בחלק החיובי; בנקודת האפס יש אי־גזירות במובן הקלאסי, וכלי תוכנה בוחרים שם כלל מעשי מתאים. ייתכן מצב שבו נוירונים נשארים כבויים זמן ממושך, ולכן קיימות גם גרסאות אחרות כגון Leaky ReLU — נושא העמקה.

4. כיצד מחשבים מעבר קדימה (Forward Pass)?

המעבר הקדמי הוא סדר החישובים שמפיק תחזית ממשקולות שכבר נקבעו. הוא איננו אימון: במהלך מעבר קדימה רגיל עדיין לא שינינו את המשקולות. נתחיל בדוגמה אחת עם שני מאפיינים, שני נוירונים חבויים ונוירון פלט יחיד. בדוגמה זו הפלט רציף, ולכן נשתמש בו להמחשת רגרסיה.



Forward pass: values flow from left to right

איור: סדר החישוב: קלט → סכום משוקלל → $ReLU$ → פלט.

```
z1 = W1 @ x + b1        h = ReLU(z1)
o = W2 @ h + b2
```

בכתיב זה x הוא וקטור קלט, W1 מטריצת משקולות לשכבה החבויה, b1 וקטור הטיות, z1 הסכומים לפני ההפעלה, h הפלטים אחרי ההפעלה, ו־o תחזית הרשת. סימן @ מציין מכפלה מטריציונית או מכפלה פנימית בהתאם לממדים.

5. דוגמה ידנית מלאה — שני נוירונים ושכבת פלט

דודו מודד שני מאפיינים של מחשב: $x_1=1$ ו־$x_2=2$ (ערכים שכבר הותאמו ליחידות המשמשות בניסוי). נבחר משקולות התחלתיות כדי ללמוד את הפעולה; במודל אמיתי הן נלמדות בזמן האימון ולא נקבעות ידנית לפי התוצאה הרצויה.

```
x = (1, 2)
W1 = [[0.2, 0.4], [-0.3, 0.5]]
b1 = (0.1, -0.2)
W2 = (0.6, -0.4)      b2 = 0.2
```

נוירון חבוי ראשון: מכפילים כל מאפיין במשקל המתאים, מחברים ומוסיפים הטיה. לכן $z_{11}=0.2\times1+0.4\times2+0.1=1.1$. מאחר שהתוצאה חיובית, ReLU מחזירה 1.1.

נוירון חבוי שני: $z_{12}=-0.3\times1+0.5\times2-0.2=0.5$. גם כאן התוצאה חיובית ולכן $h_2=0.5$.

שלב	פעולה	תוצאה
נוירון חבוי 1	0.1 + 2×0.4 + 1×0.2	1.1
נוירון חבוי 2	0.2 − 2×0.5 + 1×0.3−	0.5

שלב	פעולה	תוצאה
ReLU	$\max(0, 1.1), \max(0, 0.5)$	$h = (1.1, 0.5)$
שכבת פלט	$0.2 + 0.5 \times 0.4 - 1.1 \times 0.6$	$o = 0.66$

אם הערך האמיתי הוא $y=1$, פער החיזוי הוא $1-0.66=0.34$. הסימן השלילי אומר שהמודל חזה ערך נמוך מן הערך האמיתי; אין פירושו שמדובר בהסתברות שלילית, שכן בדוגמת הרגרסיה שלנו הפלט אינו מוגבל לטווח 0–1.

6. איך מודדים שגיאה? Loss לעומת מדד הערכה

לפני שנשנה משקולות, צריך להגדיר מה נחשב לטעות. ברגרסיה מקובל להשתמש, בין היתר, בשגיאה ריבועית. בחישוב ידני של תצפית אחת נוח לבחור במחצית ריבוע השגיאה; גורם המחצית מקל על הנגזרת ואינו משנה את נקודת המינימום.

```
L = 0.5 * (o - y)^2
```

```
L = 0.5 * (0.66 - 1)^2 = 0.0578
```

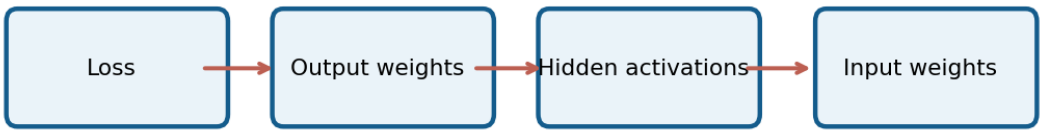
ה־Loss הוא מספר המשמש את תהליך האופטימיזציה. מדדי דיווח כגון MAE, Accuracy ו־F1 אינם בהכרח אותה פונקציה שעליה מתבצע עדכון המשקולות. בסיווג בינארי עם פלט Sigmoid נפוץ להשתמש ב־Binary Cross-Entropy, ולא רק בריבוע הפרש פשוט.

```
BCE = -[ y*ln(p) + (1-y)*ln(1-p) ]
```

בנוסחה זו p היא ההסתברות החזויה למחלקה החיובית, ו־y היא התווית האמיתית שערכה אפס או אחד. כאשר המודל בטוח בתחזית שגויה, פונקציית האובדן מטילה עליו קנס גדול. אין לחשב לוגריתם של אפס ללא טיפול מספרי מתאים; ספריות מקצועיות מבצעות חישובים יציבים יותר.

7. העברת נגזרות לאחור — Backpropagation

עד עכשיו ידענו לחשב תחזית, אבל עדיין לא ידענו אילו משקולות לשנות. Backpropagation הוא שיטה יעילה לחישוב הנגזרת של פונקציית האובדן ביחס לכל פרמטר ברשת. הוא מתקדם לאחור דרך פעולות הרשת תוך שימוש בכלל השרשרת, ומנצל תוצאות שנשמרו במעבר הקדמי. לאחר שחושבו הנגזרות, האלגוריתם המעדכן את המשקולות — למשל Gradient Descent — מבצע את השינוי בפועל.



Backward pass: chain-rule gradients propagate towards earlier layers

איור: כיוון חישוב הנגזרות: מן השגיאה אל שכבות הקלט.

בפרק 11 למדנו את כלל השרשרת: הנגזרת של פונקציה מורכבת היא מכפלת הנגזרות המקומיות לאורך המסלול. עכשיו נראה כיצד הכלל הופך לתהליך עבודה מסודר בנוירונים אחדים.

dL/do = o - y = -0.34

ראשית מחשבים את הנגזרת ביחס לפלט. במקרה שלנו היא שלילית, ולכן הגדלת הפלט מעט צפויה להקטין את השגיאה בסביבת המצב הנוכחי. נגזרת זו מועברת אל המשקולות בשכבת הפלט דרך הקשר בין הפלט לערכי השכבה החבויה.

dL/dW2 = (o-y) * h = (-0.374, -0.170)

dL/db2 = o-y = -0.34

המשקל הראשון בשכבת הפלט קיבל נגזרת −0.374 כי ערך הנוירון המתאים הוא 1.1. המשקל השני קיבל −0.170 כי ערך הנוירון השני 0.5. זוהי דוגמה לכך שהנוסחה נגזרת מהתלות של כל משקל בחישוב, ולא מכלל שרירותי של שינוי מספרים.

delta_hidden = (o-y) * W2 * ReLU'(z1)

delta_hidden = (-0.204, +0.136)

שני הערכים z_1 חיוביים, ולכן נגזרת ReLU היא אחת בכל אחד מהם. המשקל השני בשכבת הפלט שלילי, ולכן סימן הנגזרת המוחזרת לנוירון החבוי השני מתהפך. הדבר חשוב: לא כל משקל צריך להשתנות באותו כיוון.

dL/dW1 = outer(delta_hidden, x)

dL/dW1 = [[-0.204, -0.408], [0.136, 0.272]]

dL/db1 = (-0.204, 0.136)

סיימנו לחשב את הגרדיאנטים. יש לנו נגזרת לכל משקל ולכל הטיה, ומכאן ניתן לבצע עדכון. המילה outer מציינת כאן מכפלה חיצונית בין וקטור נגזרות הנוירונים לבין וקטור המאפיינים.

8. עדכון אחד מלא של המשקולות

נבחר קצב למידה $\eta = 0.1$. בכל פרמטר מחסרים את קצב הלמידה כפול הנגזרת שלו. אם הנגזרת שלילית — הערך יגדל; אם הנגזרת חיובית — הערך יקטן. העדכון אינו מבטיח ירידה בשגיאה בכל צעד ובכל קצב למידה, אך בדוגמה הקטנה שלנו נוכל לחשב ולבדוק את השפעתו.

parameter_new = parameter_old - 0.1 * gradient

פרמטר	לפני העדכון	נגזרת	לאחר העדכון
W2[0]	0.6000	-0.3740	0.6374
W2[1]	-0.4000	-0.1700	-0.3830
b2	0.2000	-0.3400	0.2340
W1[0,0]	0.2000	-0.2040	0.2204
W1[0,1]	0.4000	-0.4080	0.4408
W1[1,0]	-0.3000	+0.1360	-0.3136
W1[1,1]	0.5000	+0.2720	0.4728

פרמטר	לפני העדכון	נגזרת	לאחר העדכון
$b1[0]$	0.1000	-0.2040	0.1204
$b1[1]$	-0.2000	+0.1360	-0.2136

כאשר מחשבים מעבר קדימה נוסף אחרי העדכון, מקבלים תחזית בקירוב 0.8529, ושגיאה חדשה 0.01082 במקום 0.0578. זאת הדגמה של צעד למידה אחד על תצפית אחת בלבד. עדיין לא נוכל להסיק ממנה שהמודל מצליח על נתונים שלא ראה.

9. מעבר מחישוב ידני לקוד NumPy

נכתוב קוד הניתן להרצה רצופה הממחיש את אותם חישובים. הקוד מציג את הנגזרות ולא מסתיר אותן בפעולת fit. בהמשך, ספריית למידה עמוקה תחשב אותן אוטומטית, אבל ההבנה המתמטית שכאן מאפשרת להסביר מה הכלים עושים.

```
import numpy as np
x = np.array([1.0, 2.0])
y = 1.0
W1 = np.array([[0.2, 0.4], [-0.3, 0.5]])
b1 = np.array([0.1, -0.2])
W2 = np.array([0.6, -0.4])
b2 = 0.2
eta = 0.1

z1 = W1 @ x + b1
h = np.maximum(0, z1)
o = W2 @ h + b2
loss_before = 0.5 * (o - y)**2

delta_o = o - y
gW2 = delta_o * h
gb2 = delta_o
delta_h = delta_o * W2 * (z1 > 0)
gW1 = np.outer(delta_h, x)
gb1 = delta_h

W2 -= eta * gW2
b2 -= eta * gb2
W1 -= eta * gW1
b1 -= eta * gb1

h_after = np.maximum(0, W1 @ x + b1)
o_after = W2 @ h_after + b2
loss_after = 0.5 * (o_after - y)**2
print(round(loss_before, 5), round(loss_after, 5))
```

שימו לב לשלושה רעיונות בקוד: הווקטור z1 נוצר לפני ההפעלה; הביטוי `(z1 > 0)` מחזיר מסכה המתאימה לנגזרת ReLU במקרה הנלמד; ולבסוף מעדכנים את כל המשקולות רק אחרי שחושבו הנגזרות מתוך אותו מצב התחלתי. שינוי משקל מוקדם מדי במהלך החישוב עלול לערבב מצבים שונים ולשבש את התוצאה.

10. חישוב ממדים — כמה פרמטרים יש לרשת?

הצורה של מטריצת משקולות נקבעת לפי מספר הנוירונים בשכבות. אם בשכבה קודמת 3 נוירונים ובשכבה הבאה 4 נוירונים, דרושים 12 משקולות לחיבורים ועוד 4 הטיות. לכן מספר הפרמטרים של מעבר בין שכבות הוא מכפלת גדלי השכבות בתוספת מספר הנוירונים בשכבה הבאה.

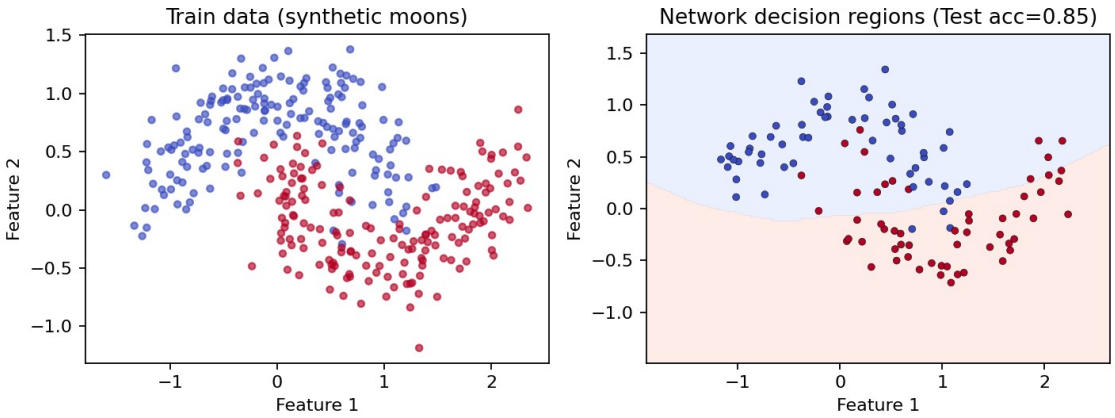
Parameters = n_in * n_out + n_out

לרשת בעלת 4 מאפייני קלט, 5 נוירונים חבויים, 3 נוירונים חבויים בשכבה השנייה ו־2 נוירוני פלט: בין קלט לשכבה הראשונה יש 25 פרמטרים; בין השכבות החבויות 18; ובין השנייה לפלט 8. סך הכול 51 פרמטרים. חשבו זאת ביד לפני הפעלת תוכנה.

יותר פרמטרים מאפשרים לייצג פונקציות מורכבות יותר, אך אינם מבטיחים ביצועים טובים יותר. רשת גדולה דורשת יותר נתונים, חישוב ואמצעי בקרה למניעת התאמת יתר.

11. מרשת קטנה למימוש מקצועי ב־scikit-learn

כדי לראות רשת לומדת על נתונים מורכבים מעט יותר, נשתמש במאגר סינתטי שנראה כשתי קשתות. זהו מאגר לימודי, ולא נתונים אמיתיים על בני אדם או על תהליך עסקי. נפריד את הנתונים לאימון ולבדיקה, ונבנה Pipeline המתקנן את המאפיינים באמצעות נתוני האימון בלבד.



איור: דוגמת שתי קשתות: רשת עם שכבות חבויות יכולה ללמוד אזורי החלטה שאינם קו ישר.

```
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score

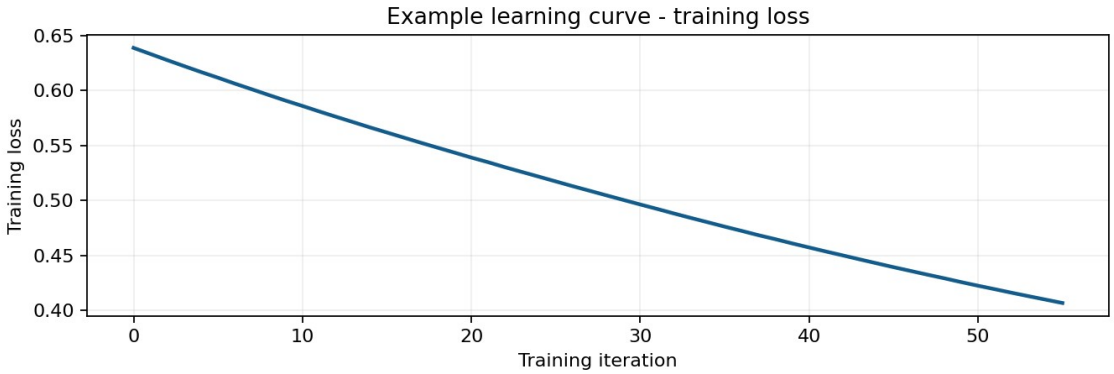
X, y = make_moons(n_samples=450, noise=0.22, random_state=8)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=15
)
model = make_pipeline(
    StandardScaler(),
    MLPClassifier(hidden_layer_sizes=(8, 8),
                  max_iter=600, early_stopping=True,
                  n_iter_no_change=25, random_state=12)
)
model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test)
print("Test accuracy:", accuracy_score(y_test, y_pred))
```

הארכיטקטורה `(8, 8)` פירושה שתי שכבות חבויות, שבכל אחת שמונה נוירונים. המשתנה `max_iter` מגביל את מספר איטרציות האימון לפי המימוש של הספרייה. `early_stopping=True` מאפשר עצירה מוקדמת על בסיס ולידציה פנימית. הדוגמה מיועדת להבנת הצינור המקצועי; היא אינה מחליפה בחירה מושכלת של ארכיטקטורה והערכת תוצאות על בעיה אמיתית.

12. כיצד קוראים עקומת למידה?

במהלך האימון המודל מציג לעיתים את שינוי פונקציית האובדן לאורך איטרציות. אם השגיאה באימון יורדת, סימן שהתהליך מצליח להתאים טוב יותר את הדוגמאות ששימשו לאימון. אך אין בכך הוכחה ליכולת הכללה. נדרשת גם בדיקה של ביצועים על נתוני ולידציה ובסוף על קבוצת Test שלא שימשה לבחירות.

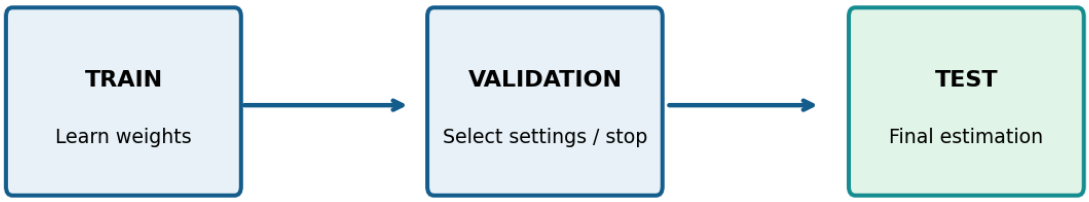


איור: עקומת שגיאת אימון מתוך הניסוי הסינתטי; לא להסיק ממנה לבדה על הכללה.

למידת חסר מתאפיינת לעיתים בשגיאה גבוהה גם באימון וגם בוולידציה, בעוד התאמת יתר יכולה להתבטא בפער גדל ביניהן. יש להיזהר מלהפוך סימנים אלה לכללים מוחלטים: אופי הנתונים, מדד השגיאה ומידת הרעש משפיעים על צורת העקומות.

13. Train, Validation ו־Test — מדוע לא משתמשים באותם נתונים לכל דבר?

אימון משמש לעדכון משקולות. ולידציה משמשת לבחירת ארכיטקטורה, קצב למידה, מספר Epochs או נקודת עצירה. Test מיועד להערכה סופית אחרי שהתקבלו ההחלטות. אם בוחרים את מבנה הרשת שוב ושוב לפי תוצאות Test, המבחן אינו בלתי תלוי עוד, והתוצאה המדווחת עלולה להיות אופטימית.



איור: שלוש מטרות שונות לשימוש בנתונים. *Pipeline* מפחית סיכון לדליפת מידע.

דוגמה לזליגה: מחשבים ממוצע וסטיית תקן על כלל המאגר לפני חלוקתו, ואז משתמשים בהם בתקנון נתוני האימון. כך מידע על נתוני הבדיקה השפיע בעקיפין על האימון. דרך נכונה היא להתאים את התקנון לנתוני האימון בלבד. בניסויי Cross-Validation יש להתאים את התקנון מחדש בתוך כל קיפול אימון.

14. אופטימיזציה, Batch ו־Epoch

כאשר מאגר הנתונים גדול, חישוב נגזרות על כל התצפיות בכל צעד עלול להיות יקר. לכן נהוג לעבד קבוצות קטנות (mini-batches). בכל Batch מחשבים תחזיות, אובדן, גרדיאנטים ועדכון. Epoch הוא מעבר מלא על דוגמאות האימון; אם גודל האימון 1,000 וגודל Batch הוא 100, יהיו בדרך כלל עשרה צעדי Batch בכל Epoch, אם לא משנים את דוגמת החלוקה.

קצב הלמידה קובע את אורך העדכון. אם הוא קטן מאוד, השיפור עשוי להיות איטי; אם הוא גדול מדי, המודל עלול להחמיץ אזורי שגיאה נמוכה או להפוך את האימון לבלתי יציב. שיטות כגון SGD ו־Adam משנות את אופן ביצוע העדכונים, אך אינן מחליפות את העיקרון של חישוב גרדיאנטים.

15. מה הקשר בין Backpropagation ל־Automatic Differentiation?

ברשתות מודרניות אין גוזרים ביד מאות אלפי משקולות. מסגרות כגון PyTorch בונות גרף חישובי ומשתמשות בגזירה אוטומטית לחישוב גרדיאנטים. הפעולה `loss.backward()` מחשבת נגזרות; היא אינה מעדכנת לבדה את המשקולות. לאחריה פועלה כגון `optimizer.step()` מבצעת עדכון. לפני צעד האימון הבא יש לאפס גרדיאנטים צבורים, למשל באמצעות `optimizer.zero_grad()`.

> הרחבה מקצועית: דוגמת *PyTorch* היא גשר להמשך ולא דרישה שכל תלמיד יכתוב את תשתית *Autograd* בעצמו. השימושים בקוד מקצועי מחייבים הסבר של מבנה המודל והחלטות האימון.

```
# A short professional pattern (PyTorch installed separately)
# optimizer.zero_grad()
# prediction = model(features)
# loss = loss_fn(prediction, target)
# loss.backward()       # compute gradients
# optimizer.step()      # update parameters
```

16. תקלות נפוצות וכיצד מאבחנים אותן

תצפית	מה ייתכן שקרה?	כיצד נבדוק?
שגיאת אימון אינה יורדת	קצב למידה, תקנון, נתונים או באג בקוד	בדיקת טווחים, גרדיאנטים וקו בסיס
אימון מצוין, Test חלש	התאמת יתר או זליגת מידע בבחירת מודל	השוואת עקומות ואופן חלוקת נתונים
פלט כמעט קבוע	ארכיטקטורה לא מתאימה או רשת שאינה לומדת	בדיקת משקולות, קלטים ופלטים
שגיאה מספרית NaN	ערכים קיצוניים, קצב למידה או לוגריתם אפס	ניטור חישובים ותקנון
מדדים משתנים בין הרצות	אקראיות באתחול ובחלוקה	זרעים קבועים וניסוי חוזר
פלט בינארי אינו 0/1	המודל מחזיר ציון או הסתברות לפני סף	הגדרת כלל החלטה מפורש

17. מדוע גם רשת מצליחה יכולה להיות בעייתית?

רשת עצבית היא כלי חיזוי סטטיסטי, ולא מכשיר הקובע אמת מוחלטת. אם הנתונים אינם מייצגים אוכלוסיות מסוימות, אם נאספו באופן לא מוסכם, או אם התוויות אינן אמינות — איכות גבוהה במדד מספרי יחיד אינה מספיקה כדי להצדיק שימוש. בפרויקט נדרש לתאר את מקור הנתונים, את מגבלותיהם, את השגיאות ואת הסיכון הנובע מן השימוש במודל.

בתחומים רגישים, כגון בריאות, מיון תלמידים או קבלת החלטות בעלות השלכות משמעותיות, יש צורך בבקרה מקצועית ובשמירה על פרטיות. את פרק 12 ניישם בבעיות לימודיות שניתן לבקר ולהסביר, ולא נציג תחזית אוטומטית כאבחנה או כפסיקה.

18. תרגול מדורג — מהבנה לעומק

רמה א — מושגים וחישוב: (א) הסבירו במילים שלכם את ההבדל בין שכבת קלט לשכבה חבויה. (ב) חשבו עבור ReLU $2, 0$− ו־3. (ג) ברשת בעלת 2 מאפיינים, 3 נוירונים חבויים ו־1 פלט — כמה משקולות וכמה הטיות יש? הציגו שלבי חישוב.

רמה ב — העברה ונימוק: (ד) נתונה רשת שבה כל השכבות ליניאריות. הסבירו למה הוספת שכבה לא בהכרח תפתור XOR. (ה) במודל שבדוגמה הידנית, מה יקרה לערך הנוירון השני אם נשנה את אחת המשקולות? נמקו בחישוב. (ו) מדוע אי־אפשר לפרש תחזית 0.66 בהכרח כהסתברות?

רמה ג — פיתוח וביקורת: (ז) גזרו מחדש נגזרת של משקל אחד בשכבה החבויה באמצעות כלל השרשרת. (ח) הציעו ניסוי שמשווה רשת בעלת שכבה חבויה אחת לרשת בעלת שתי שכבות, באותה חלוקת נתונים. (ט) תארו שלושה צעדים למניעת זליגת מידע בפרויקט אמיתי.

19. שאלת הערכה מקורית בסגנון בגרות / הגנה

מערכת AI מקבלת שני מאפיינים של מוצר ומפיקה תחזית מספרית. המודל כולל שכבה חבויה אחת בת שני נוירונים עם ReLU ושכבת פלט ליניארית. נתונים המשקולות, ההטיות והקלט מהדוגמה בסעיף 5. התלמיד מתבקש: (א) לחשב את פלט השכבה החבויה; (ב) לחשב תחזית ושגיאה; (ג) להסביר איזה שלב

נדרש כדי לעדכן משקולות; (ד) להציע בדיקה אחת שתבחן יכולת הכללה; (ה) לנמק מדוע הרשת עשויה להיות מתאימה יותר ממודל ליניארי פשוט עבור תופעה לא־ליניארית.

הערה: זוהי שאלה מקורית בדפוס הערכה עיוני והגנה בעל־פה, ואינה שאלה רשמית מתוך בחינת משרד החינוך.

20. רצף הוראה מוצע — 20 שעות

מפגש	שעות הוראה	מוקד	ראיה ללמידה
1	2	בעיה אמיתית, פרספטרון לעומת MLP, XOR	המחשת מגבלת ליניאריות
2	2	שכבות, משקולות, הטיות וממדים	שירטוט והערכת פרמטרים
3	2	פונקציות הפעלה ו־ReLU	חישוב ערכי הפעלה
4	2	Forward Pass ידני	טבלת עקיבה
5	2	פונקציות אובדן ורגרסיה/סיווג	חישוב Loss ופירוש
6	2	Backpropagation וכלל השרשרת	טבלת גרדיאנטים
7	2	עדכון משקולות ו־NumPy	מימוש חישוב ידני
8	2	אימון רשת וספריית sklearn	ניסוי עם שני ארכיטקטורות
9	2	Evaluation, Validation, Leakage	דוח ניסוי מנומק
10	2	תרגול עצמאי, שאלת הערכה והכנה לפרויקט	מחברת, מסקנה והסבר בעל־פה

חלוקת עשר יחידות לשתי שעות היא הצעה דידקטית מתוך היקף 20 השעות של פרק ההתמחות, ולא חלוקה רשמית של משרד החינוך.

21. מילון מושגים קצר

מושג	פירוש
MLP	רשת נוירונים קדמית בעלת כמה שכבות
Forward Pass	מעבר חישוב מהקלט אל הפלט
Backpropagation	חישוב גרדיאנטים לאחור באמצעות כלל השרשרת
Activation	פונקציה שמשנה את פלט הנוירון
Loss	פונקציית שגיאה לאימון
Gradient	וקטור הנגזרות של פונקציית האובדן
Optimizer	כלל עדכון פרמטרים

מושג	פירוש
Epoch	מעבר מלא על נתוני אימון
Batch	קבוצת תצפיות בתוך האימון
Early Stopping	עצירה לפי התקדמות מדד ולידציה

22. הקשר בין שלוש תכניות הלימוד

תכנית	מה כבר נלמד?	מה מוסיף פרק זה?
יחידה 3	Python, וקטורים בסיסיים, סיווג והערכה	חיבור לרשת כאמצעי ייצוג חדש — גשר בלבד
יחידה 5 נושאי ליבה	וקטורים, מרחקים, סטטיסטיקה ואימות מודלים	ניצול המתמטיקה שכבר נלמדה, ללא ייחוס רשתות כחובת יחידה 5
התמחות DL	רגרסיה, פרספטרון, נגזרות ו־Gradient Descent	מעבר קדימה, Backpropagation ורשת מאומנת

23. ספרות מקצועית ומקורות

עוגן תכנית: משרד החינוך, התמחות למידת מכונה ולמידה עמוקה, פרק 12 — רשת נוירונים עמוקה (20 שעות), על פי מפת התכנית השמורה במאגר GitHub. פירוט הדוגמאות, תרגילי ההעשרה והחלוקה למפגשים הם פיתוח מקורי; האימות הסופי של תתי־סעיפי חובה מחייב בדיקת מסמך התכנית הרשמי המעודכן.

Zhang, A., Lipton, Z., Li, M. & Smola, A. — Dive into Deep Learning, פרק Multilayer Perceptrons ובפרט Forward / Backpropagation: https://d2l.ai/chapter_multilayer-perceptrons/backprop.html

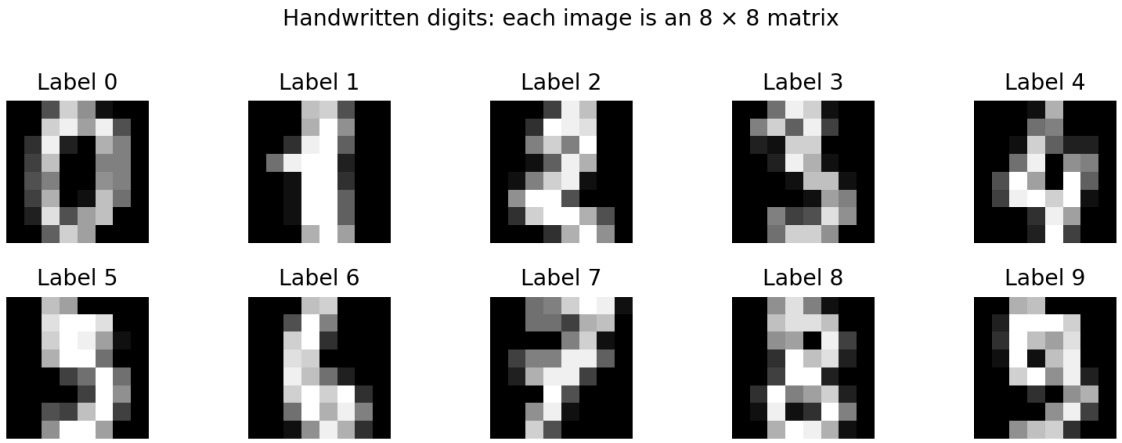
Prince, S. J. D. — Understanding Deep Learning, MIT Press, 2023. Deisenroth, Faisal & Ong — Mathematics for Machine Learning, Cambridge University Press, 2020. Géron — Hands-On Machine Learning. המקורות משמשים לעקרונות, לרמת ההסבר ולביקורת, ולא הועתקו מהם איורים או קטעי טקסט.

scikit-learn — User Guide: Neural Network Models (Supervised): https://scikit-learn.org/stable/modules/neural_networks_supervised.html. PyTorch — Neural Networks Tutorial: https://docs.pytorch.org/tutorials/beginner/blitz/neural_networks_tutorial

פרק 13 — רשתות קונבולוציה (CNN)

מפיקסלים ומסננים לרשת עצבית שמזהה דפוסים בתמונה

ספר הקורס המאוחד | מסלול התמחות למידת מכונה ולמידה עמוקה, 5 יחידות | היקף פרק בתכנית: 50 שעות | מהדורת תלמיד



איור: דוגמאות מתמונות ספרות. כל תמונה היא מערך של 64 עוצמות אפור.

לפני שמתחילים — מטרות, ידע קודם ומעמד התוכן

בסיום הפרק תוכלו להסביר מדוע רשת צפופה אינה מנצלת את המבנה המרחבי של תמונה באופן מובנה; לחשב מסנן על חלון תמונה; לחזות את ממדי התוצר של שכבות Convolution ו־Pooling; לתאר כיצד מסננים נלמדים באימון; ולבנות רשת CNN קטנה, להעריך אותה ולנתח טעויות.

הפרק ממשיך את פרק 6 (עיבוד תמונה), פרק 11 (נגזרות וגרדיאנטים) ופרק 12 (רשתות עצביות). מומלץ לחזור על מטריצה דו־ממדית, כפל וחיבור, מכפלה פנימית, ReLU, Cross-Entropy ונתוני Train/Validation/Test. לא נניח שידיעת Python לבדה מספיקה להבנת הרשת.

ידע קודם	איפה השתמשנו בו	איך ישמש בפרק
מטריצות ו־RGB	פרק 6	חלונות, מסננים וערוצים
מכפלה פנימית ומשקולות	פרקים 1 ו־9	כל תא במפת מאפיינים
כלל השרשרת ו־Gradient Descent	פרק 11	למידת משקולות המסננים
Forward / Backward	פרק 12	אימון רשת מקצה לקצה
הערכה וולידציה	פרקים 5.1 ו־5.2	בדיקה הוגנת של CNN

1. סיפור פתיחה — כיצד טלפון מזהה ספרה בכתב יד?

דודו מפתח יישומון לסריקת מספרי חדרים בבית הספר. לפעמים התמונה מוארת, לפעמים היא מעט מוזזת, ולפעמים הספרה נכתבת בקו עבה. אם נצמיד לכל פיקסל משקל שונה ברשת צפופה, הרשת

תצטרך ללמוד בנפרד שמאפיין מסוים יכול להופיע גם למעלה וגם למטה. אבל קצה, קו או פינה הם אותם דפוסים חזותיים גם כשהם זזים מעט.

מכאן עולה רעיון מרכזי: במקום לתת לכל מיקום בתמונה מערכת משקולות נפרדת, נסרוק את התמונה בעזרת מסנן קטן שמשקולותיו משותפות לכל המיקומים. כך נלמד לזהות דפוס מקומי, נשמור על קשרים בין פיקסלים שכנים ונקטין לעיתים קרובות את מספר הפרמטרים. זו אחת מאבני היסוד של CNN.

לבדיקה עצמית: אם אותו קו אנכי נמצא פעם בצד שמאל ופעם בצד ימין, מדוע כדאי להשתמש באותו מסנן בשני המקומות?

2. לפני הרשת: תמונה היא טנזור ולא ציור מסתורי

תמונת גווני אפור אפשר לראות כמטריצה בגובה H וברוחב W. ערך בכל תא מתאר עוצמת אור. בתמונת RGB יש שלוש מטריצות — אחת לכל ערוץ צבע. במהלך אימון הרשתות אנו עובדים בדרך כלל עם אוסף תמונות (Batch), ולכן למערך הנתונים יש ארבעה ממדים.

יש להבחין בין שמות הממדים לבין סדרם בספרייה. ב־PyTorch סדר ברירת המחדל הוא N,C,H,W: מספר תמונות, מספר ערוצים, גובה ורוחב. במערכות אחרות תיתכן צורה N,H,W,C. בלבול בסדר הערוצים גורם לאחת משגיאות הקוד השכיחות בתחום.

Grayscale: (H, W) | RGB: (H, W, 3) | PyTorch batch: (N, C, H, W)

למשל, אצווה של 32 תמונות צבע בגודל 64×64 תוצג ב־PyTorch בממדים (32,3,64,64). מספר הערכים הגולמיים הכולל הוא 32×3×64×64. רשת CNN לומדת להפוך את הפיקסלים למפות מאפיינים, ובהמשך להחלטה על מחלקת התמונה.

```
import numpy as np
from sklearn.datasets import load_digits

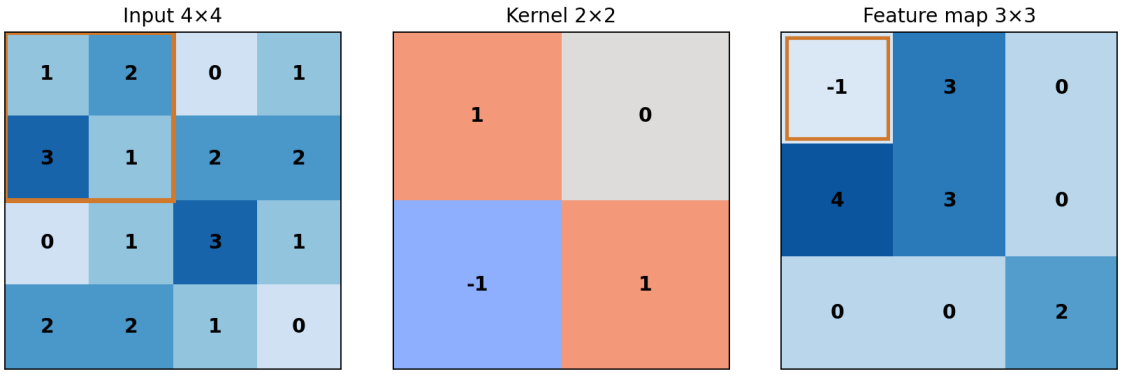
digits = load_digits()
image = digits.images[0]
print(image.shape)         # (8, 8)
print(image.min(), image.max())  # intensities in [0, 16]
print(image[0, :])         # first row of pixels
```

לבדיקה עצמית: מה ההבדל בין מספר הפיקסלים בתמונה לבין מספר המאפיינים המופקים בשכבת CNN?

3. מהו מסנן (Kernel) ומה הוא עושה?

מסנן הוא מטריצה קטנה של משקולות, לדוגמה 2×2 או 3×3. מניחים אותו מעל אזור מקומי בתמונה, מכפילים כל ערך תמונה במשקל המתאים במסנן ומחברים. התוצאה היא מספר יחיד: תגובת המסנן באזור הזה. לאחר מכן מזיזים את המסנן למיקום הבא וחוזרים על החישוב. כל התוצאות יחד יוצרות מפת מאפיינים (Feature Map).

במדעי התמונה נעשה שימוש במסננים קבועים, למשל להדגשת קצוות. ברשת CNN משקולות המסנן יכולות להשתנות בזמן האימון. לכן ההבדל החשוב אינו רק הפעולה של החלקת חלון, אלא העובדה שהמערכת לומדת איזה מסנן מועיל למשימה.



איור: חלון 2×2 עובר על מטריצה 4×4 ומפיק מפת מאפיינים 3×3.

4. חישוב ידני מלא — ערך ראשון במפת המאפיינים

ניקח חלון תמונה בגודל 2×2 והמסנן הבא. בכל תא במכפלה יש כפל אחד, ולאחריו מחברים את ארבע התוצאות. שימו לב לסימן השלילי במשקל השמאלי־תחתון של המסנן: הוא יכול להפחית מן התגובה כאשר הערך באזור הזה גבוה.

Window = [[1, 2], [3, 1]] Kernel = [[1, 0], [-1, 1]]

Output[0,0] = 1×1 + 2×0 + 3×(-1) + 1×1 = -1

כאשר המסנן זז פיקסל אחד ימינה, החלון הבא הוא [[2,0],[1,2]]. כעת מחשבים 2×0+1×1+0×(-1)+1×2=3. יש להשלים ידנית את יתר המיקומים כדי להבין מדוע גודל מפת התוצאה הוא 3×3. זו דוגמה של קורלציה צולבת (Cross-Correlation): המסנן אינו מתהפך לפני החישוב. בספריות CNN מקובל לקרוא לפעולה Convolution, אף שבמתמטיקה הקונבולוציה הפורמלית מבצעת היפוך של המסנן.

Y = [[-1, 3, 0], [4, 3, 0], [0, 0, 2]]

```
import numpy as np

image = np.array([[1, 2, 0, 1],
                  [3, 1, 2, 2],
                  [0, 1, 3, 1],
                  [2, 2, 1, 0]])
kernel = np.array([[1, 0], [-1, 1]])

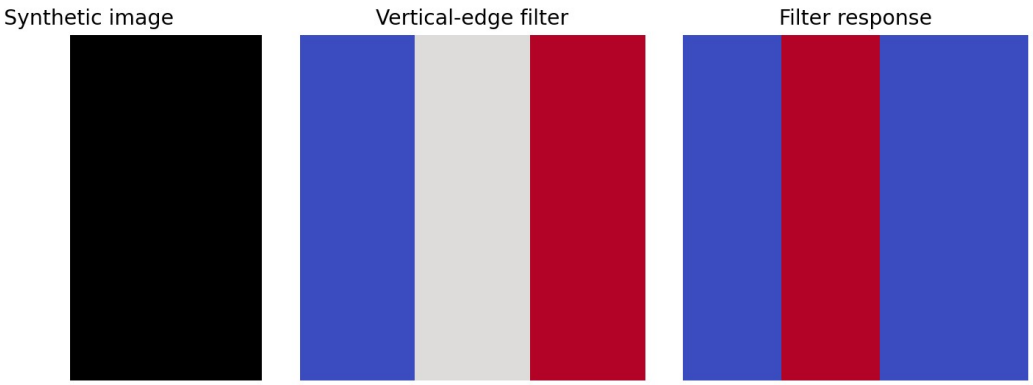
out = np.zeros((3, 3), dtype=int)
for row in range(3):
    for col in range(3):
        window = image[row:row+2, col:col+2]
        out[row, col] = np.sum(window * kernel)
print(out)
```

דיוק מתמטי: הקוד והדוגמה מבצעים *Cross-Correlation* ולא קונבולוציה עם היפוך מסנן. זו אותה מוסכמה שבה משתמשת *nn.Conv2d* ב*PyTorch*.

5. איך מסנן מזהה קצה או קו?

נניח שאזור שמאלי בתמונה כהה ואזור ימני בהיר. שינוי חד בין האזורים עשוי להצביע על קצה של עצם. מסנן שקובע משקלים שליליים בצד שמאל וחיוביים בצד ימין מגיב למעבר כזה. אין למסנן הבנה של המושג ״דלת״ או ״ספרה״; הוא רק מחבר הפרשים מקומיים על פי משקולות.

מסנן יחיד עשוי להגיב לקצה אנכי. מסננים אחרים עשויים להגיב לקווים אופקיים או לטקסטורות. בשכבות עמוקות יותר הרשת יכולה לשלב תגובות מקומיות ולהבחין בדפוסים מורכבים. התיאור הוא אינטואיטיבי: בפועל לא כל מסנן לומד תכונה שקל לתת לה שם.



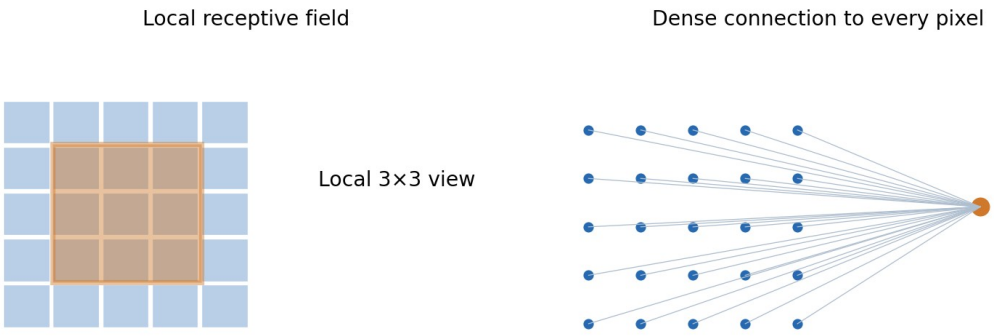
איור: דימוי של מסנן המדגיש מעבר בין אזור כהה ובהיר.

לבדיקה עצמית: אם נכפיל את כל ערכי התמונה ב־2, מה יקרה לתגובת המסנן הזה, בהנחה שאין **Bias** ואין **Clipping**?

6. מדוע שיתוף משקולות כל כך חשוב?

ברשת צפופה (Dense) כל נוירון יכול להיות מחובר לכל ערכי הקלט, ולכן ניתן ללמוד משקל שונה לכל מיקום. ב־CNN המסנן הקטן מחובר רק לסביבה מקומית, אך משתמש באותן משקולות כשהוא עובר על מיקומים שונים. השילוב בין קישוריות מקומית לשיתוף משקולות מאפשר לנצל את המבנה המרחבי.

לדוגמה, אם תמונה מכילה 64×64×3 ערכים ולנוירון צפוף יחיד יש משקל לכל ערך, דרושים 12,288 משקולות ועוד הטיה. לעומת זאת מסנן 3×3 המקבל שלושה ערוצי קלט דורש 3×3×3=27 משקולות ועוד הטיה; הוא מוחל שוב ושוב לאורך התמונה. ההשוואה היא בין נוירון צפוף יחיד למסנן יחיד, ולא בין שתי ארכיטקטורות שלמות.



איור: קישוריות מקומית לעומת קישוריות מלאה למאפייני הקלט.

One 3×3 RGB filter: 3 × 3 × 3 + 1 bias = 28 parameters

7. ממדי הפלט: Kernel Size, Stride, Padding

אחת המיומנויות החשובות לבגרות ולפרויקט היא היכולת לחזות את ממדי המערך לאחר שכבת עיבוד. יש שלושה גדלים מרכזיים: גודל המסנן K, גודל הצעד S (Stride) ומספר ערכי הריפוד P (Padding) שמוסיפים בכל צד.

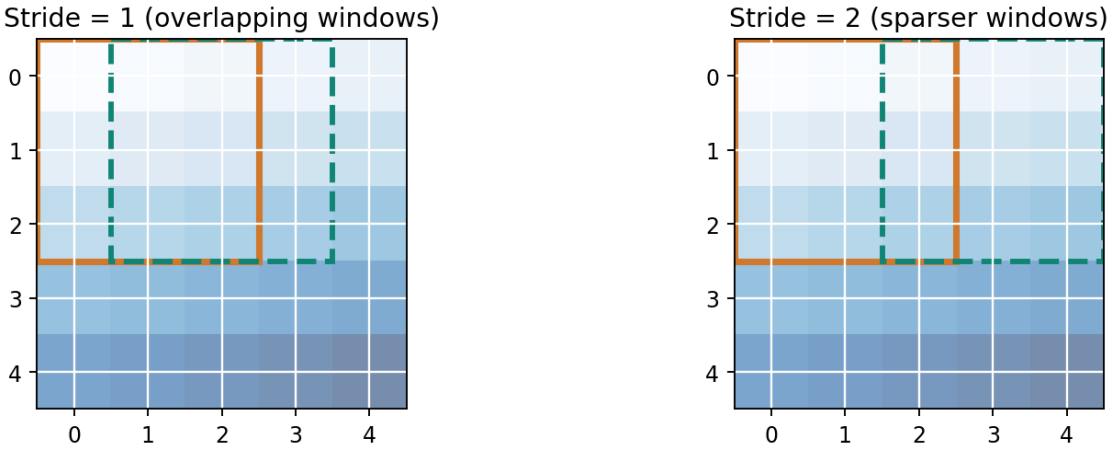
כאשר המסנן נע בצעדים גדולים יותר, מספר המקומות שבהם הוא נסרק קטן. כאשר מוסיפים ריפוד אפסים מסביב לתמונה, אפשר לחשב תגובות סמוך לשוליים ולשמר טוב יותר את הממדים. חשוב לזכור: Padding אינו משחזר מידע שלא היה בתמונה; הוא מוסיף ערכים לפי כלל שנקבע.

$$\mathbf{H_out = floor((H + 2P - K) / S) + 1 \quad [dilation = 1]}$$

$$\mathbf{W_out = floor((W + 2P - K) / S) + 1 \quad [dilation = 1]}$$

דוגמה 1: תמונה 8×8, מסנן 3×3, צעד 1, בלי ריפוד: H_out=(8−3)/1+1=6 ולכן הפלט 6×6. דוגמה 2: אותה תמונה עם ריפוד 1 בכל צד וצעד 1: H_out=(8+2−3)+1=8. דוגמה 3: תמונה 8×8, מסנן 3×3, ריפוד 1, צעד 2: 7214 . בכל שלוש הדוגמאות מספר ערוצי הפלט נקבע לפי מספר המסננים, לא לפי מספר ערוצי הקלט.

קלט H×W	K	Stride	Padding בכל צד	פלט H×W
8×8	3	1	0	6×6
8×8	3	1	1	8×8
8×8	3	2	1	4×4
28×28	5	1	0	24×24



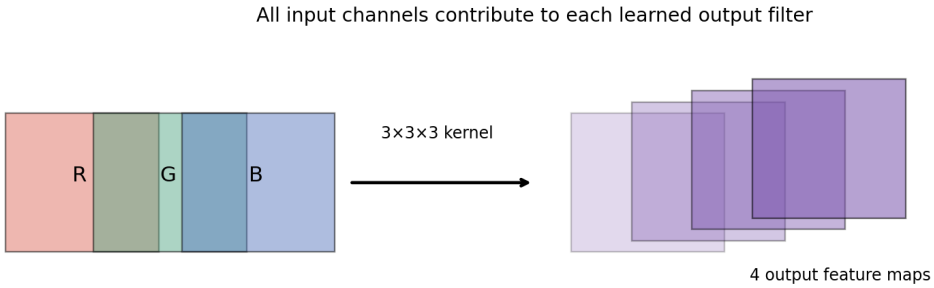
איור: *Stride* גדול יותר מדלג על מיקומי סריקה ומקטין את ממדי הפלט.

לבדיקה עצמית: חישבו את גודל הפלט של תמונה 32×32 עם מסנן 5×5, ריפוד 2 וצעד 2. נמקו את פעולת הרצפה.

8. לאן נעלמים ערוצי הצבע? Convolution רב־ערוצית

תמונת RGB מכילה שלושה ערוצים. מסנן CNN אחד אינו מסנן של ערוץ בודד בלבד: בדרך כלל הוא מכיל קבוצת משקולות לכל ערוץ קלט. מחשבים תגובה מכל ערוץ, מחברים את שלושת התרומות ומוסיפים Bias; התוצאה היא מפת מאפיינים אחת.

אם נבחר שמונה מסננים שונים, נקבל שמונה מפות מאפיינים. כל מסנן יכול ללמוד דפוס אחר. אם לקלט יש 3 ערוצים ולשכבה 8 מסננים בגודל 3×3, מספר הפרמטרים הוא 8×(3×3×3+1)=224. ממדי הפלט יהיו N,8,H_out,W_out.



איור: כל מסנן פועל על כל ערוצי הקלט ויוצר ערוץ פלט אחד.

Conv parameters = C_out × (C_in × K_h × K_w + 1 bias)

לצורך ההבנה, חשבו ידנית פיקסל פלט של מסנן 1×1 כאשר תרומת שלושת הערוצים היא R×0.2+G×0.5+B×(0.1)+0.3. אין משמעות הדבר שהרשת מפרשת כל ערוץ כמשתנה עצמאי ללא הקשר; המשקולות נלמדות יחד בתהליך האימון.

9. פונקציית הפעלה — מדוע Convolution לבדה אינה מספיקה?

שכבת קונבולוציה מחשבת צירוף ליניארי מקומי של הערכים. אם נחבר רק שכבות ליניאריות, ללא פעולות לא־ליניאריות ביניהן, המערכת עדיין תהיה צירוף ליניארי כולל (בהתעלם מפעולות כמו Pooling). לכן מוסיפים לרוב פונקציית הפעלה כגון ReLU: ערכים שליליים הופכים לאפס וערכים חיוביים נשארים כפי שהם.

ReLU מאפשרת לשלב דפוסים בצורה לא־ליניארית. בגרף, החצי השמאלי נשאר באפס והחצי הימני עולה בקו ישר. אולם ReLU לבדה אינה פותרת כל בעיה, ויש לדון גם במקרים שבהם נוירונים נשארים לא פעילים לאורך האימון.

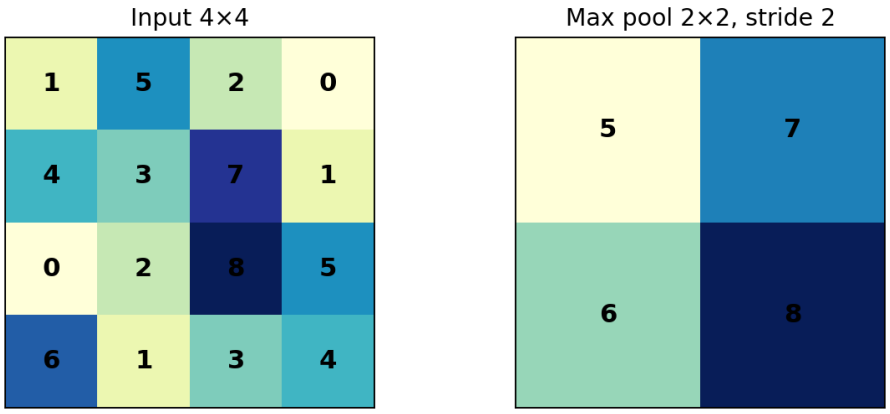
ReLU(z) = max(0, z)

ReLU(-2)=0 ReLU(0)=0 ReLU(3.5)=3.5

10. Pooling — כיצד מצמצמים מפה בלי לאבד כל דפוס מקומי?

Pooling מסכמת אזור קטן במפת מאפיינים לערך יחיד. ב־Max Pooling לוקחים את הערך המרבי בכל חלון. פעולה זו מקטינה את הרזולוציה ויכולה להקטין רגישות לשינויים מקומיים קטנים, אך עלולה גם למחוק מידע מרחבי מדויק. אין להתייחס אליה כאל פעולה שמשפרת כל מודל בכל מצב.

נחשב לדוגמה Max Pooling בגודל 2×2 עם Stride 2: נפרק מטריצה 4×4 לארבעה בלוקים לא חופפים. מכל בלוק נבחר את הערך הגבוה ביותר. התוצאה היא מטריצה 2×2. אין כאן משקולות שהשכבה לומדת; כלל הבחירה מוגדר מראש.



איור: *Max Pooling* בוחר את הערך הגדול בכל חלון 2×2.

MaxPool([[1,5],[4,3]]) = 5

```
import torch
import torch.nn.functional as F

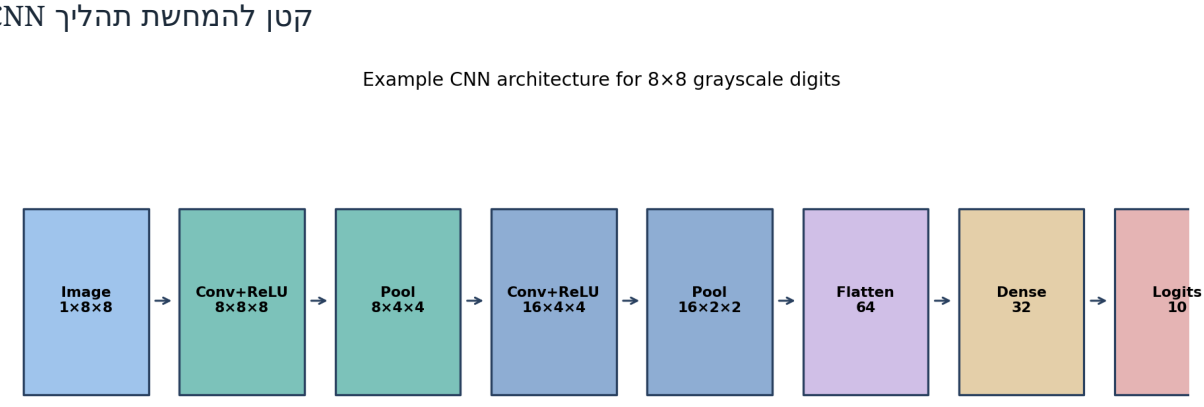
x = torch.tensor([[[[1., 5., 2., 0.],
                    [4., 3., 7., 1.],
                    [0., 2., 8., 5.],
                    [6., 1., 3., 4.]]]])
print(F.max_pool2d(x, kernel_size=2, stride=2))
```

לבדיקה עצמית: אם היינו משתמשים ב־**Average Pooling** במקום **Max Pooling**, איזה מידע היה מודגש יותר? חשבו את הבלוק הראשון.

11. מהי ארכיטקטורת CNN מלאה?

CNN היא לא שכבה אחת אלא שרשרת פעולות. בשלב הראשון הקלט הוא תמונה. שכבות Convolution לומדות דפוסים מקומיים, פונקציות ReLU מוסיפות אי־ליניאריות, ו־Pooling עשוי להקטין את המפות. בסוף ניתן לשטח את מפות המאפיינים לווקטור ולחבר שכבות Dense שמפיקות ערכי תחזית.

בדוגמה שנשתמש בה נבנה רשת קטנה לסיווג ספרות: 1×8×8 → קונבולוציה ל־8 מפות → Pooling ל־8×4×4 → קונבולוציה ל־16 מפות → Pooling ל־16×2×2 → Flatten באורך 64 → שכבה חבויה בגודל 32 → 10 ערכי פלט. המודל אינו נחשב בהכרח ״עמוק״ לפי כל סף מחקרי; זהו מבנה לימודי קטן להמחשת תהליך CNN.

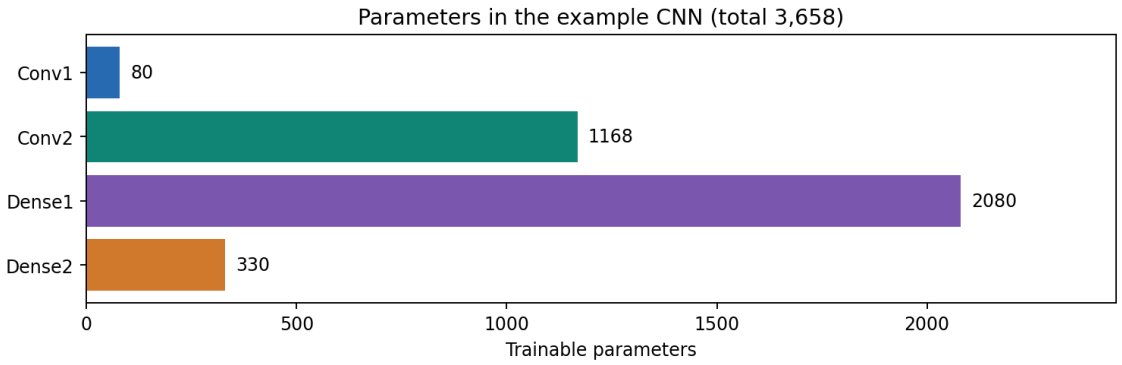


איור: ממדי המערכים ברשת *CNN* לימודית לספרות 8×8.

12. סופרים פרמטרים — יכולת שחזור והבנת הארכיטקטורה

ספירת פרמטרים מאפשרת לזהות טעויות בתכנון שכבות. בשכבת Conv2d המשקולות הן בגודל (מספר מסנני פלט, מספר ערוצי קלט, גובה המסנן, רוחב המסנן), ולכל מסנן יש בדרך כלל Bias אחד. בשכבת Dense מספר המשקולות הוא מספר הקלטים כפול מספר הנוירונים, ועוד Bias לכל נוירון.

שכבה	נוסחת חישוב	מספר פרמטרים
Conv1: 1→8, 3×3	(3+1×3×1)×8	80
Conv2: 816, 3×3	(3+1×3×8)×16	1,168
Dense1: 64→32	32+32×64	2,080
Dense2: 3210	10+10×32	330
סך הכל	80+1,168+2,080+330	3,658



איור: השוואת מספר הפרמטרים בארכיטקטורה הלימודית.

13. Forward Pass, Loss, Backpropagation — כיצד רשת לומדת?

במעבר קדימה (Forward Pass) התמונה עוברת דרך כל השכבות ונוצרים עשרה מספרים הנקראים Logits: מספר אחד לכל מחלקת ספרה. אלה אינם הסתברויות בפני עצמם. פונקציית Softmax יכולה להמיר אותם לערכים בין 0 ל־1 שסכומם 1, אך בזמן אימון מסווג רב־מחלקתי משתמשים לרוב ב־CrossEntropyLoss שמקבלת את ה־Logits ישירות.

לאחר השוואת ה־Logits לתווית האמיתית, פונקציית האובדן מספקת מספר המייצג עד כמה התחזית אינה מתאימה לתווית. באמצעות Backpropagation וכלל השרשרת מחשבים את נגזרת האובדן ביחס למשקל בכל שכבה. האופטימייזר משנה מעט את המשקולות בכיוון שנועד להקטין את השגיאה. זהו אותו עקרון מתמטי שתרגלנו בפרק 11, הפעם על מספר רב של מסננים ושכבות.

$$\text{Loss} = -\log\left(\frac{\exp(z_y)}{\sum_j \exp(z_j)}\right)$$

נוסחת Cross-Entropy לדוגמה אחת משתמשת ב־z_y שהוא ה־Logit של המחלקה האמיתית. אם למודל שלוש מחלקות וה־Logits הם [2,1,0], עבור תווית 0 ההסתברות אחרי Softmax היא בקירוב 0.6652 ולכן האובדן הוא בערך 0.4076. אם תווית האמת הייתה 2, האובדן היה גדול יותר. החישוב נועד לבנות אינטואיציה, לא לדרוש פיתוח של כל נגזרות Softmax בשלב זה.

$$P(y=0) = \frac{e^2}{e^2+e^1+e^0} \approx 0.6652$$

$$\text{CrossEntropy}(y=0) = -\ln(0.6652) \approx 0.4076$$

```
import torch
```

```
import torch.nn as nn

logits = torch.tensor([[2.0, 1.0, 0.0]])
labels = torch.tensor([0])
loss = nn.CrossEntropyLoss()(logits, labels)
print(round(loss.item(), 4))  # approximately 0.4076
```

14. מימוש רשת CNN אמיתית באמצעות PyTorch

במימוש הבא נשתמש ב־PyTorch, משום שהקוד מאפשר לזהות באופן מפורש את שכבות הרשת ואת תהליך האימון. זהו קוד לימודי, לא מתכון יחיד לפרויקט. נשתמש במאגר `digits` הזמין ב־scikit-learn ללא הורדה חיצונית. תמונה היא 8×8 בגווני אפור, הערכים המקוריים בין 0 ל־16, והתווית היא אחת מעשר הספרות.

נחלק את הנתונים באופן מדורג ל־Train, Validation ו־Test, עם שמירה בקירוב על התפלגות המחלקות. את קבוצת ה־Validation נשתמש כדי לבחור למשל מספר Epochs ואת Test נשמור להערכה סופית. הנרמול לפי תחום מדידה ידוע [0,16] אינו מחשב סטטיסטיקה מכל המאגר, ולכן אין כאן התאמת Scaler על Test.

```
import numpy as np
import torch
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split

torch.manual_seed(7)
np.random.seed(7)
digits = load_digits()
X = digits.images.astype(np.float32) / 16.0
y = digits.target.astype(np.int64)

X_dev, X_test, y_dev, y_test = train_test_split(
    X, y, test_size=0.15, stratify=y, random_state=7)
X_train, X_val, y_train, y_val = train_test_split(
    X_dev, y_dev, test_size=0.18, stratify=y_dev, random_state=7)

train_x = torch.tensor(X_train[:, None, :, :])
train_y = torch.tensor(y_train)
val_x = torch.tensor(X_val[:, None, :, :])
val_y = torch.tensor(y_val)
print(train_x.shape, val_x.shape)
```

השימוש בממד חדש `None` מוסיף ערוץ צבע יחיד, כדי לקבל את סדר הממדים המתאים ל־Conv2d: מספר הדוגמאות, מספר הערוצים, גובה ורוחב. אין לשטח תמונה לווקטור לפני שכבת הקונבולוציה, מפני שהמטרה היא לנצל את השכנות המרחבית.

```
import torch.nn as nn

class DigitCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(1, 8, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Conv2d(8, 16, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2)
        )
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(16 * 2 * 2, 32),
            nn.ReLU(),
            nn.Linear(32, 10)
```

```
        )

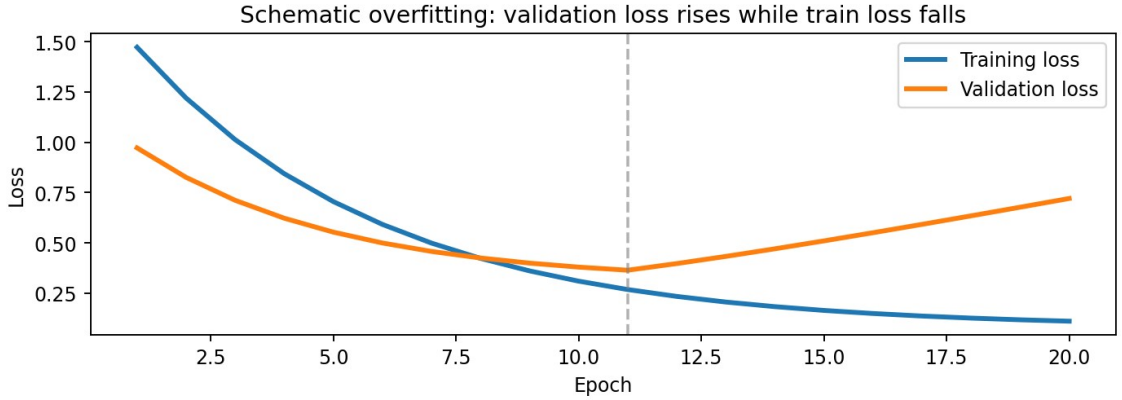
    def forward(self, x):
        return self.classifier(self.features(x))

model = DigitCNN()
print(model(torch.zeros(2, 1, 8, 8)).shape)  # (2, 10)
print(sum(p.numel() for p in model.parameters()))  # 3658
```

15. אימון מבוקר: מדוע צריך להפריד בין אימון לוולידציה?

בכל Epoch הרשת עוברת על דוגמאות האימון, מחשבת תחזיות, מודדת אובדן ומעדכנת משקולות. אחרי הסיום בודקים את המודל על קבוצת Validation ללא חישוב נגזרות. אם האובדן על Train ממשיך לרדת, אבל האובדן על Validation עולה, זו אינדיקציה אפשרית להתאמת יתר. אין להסיק מהתנהגות של Epoch יחיד מסקנה גורפת.

בדוגמת הקוד אנחנו בודקים את שתי הקבוצות אחרי כל Epoch. בספר נריץ מעט Epochs, כדי לשמור על זמן מעבדה סביר גם ללא GPU. בפרויקט גמר ניתן לשלב Batch קטן, Early Stopping ושמירת הגרסה הטובה ביותר לפי Validation. רצוי לא לבחור את המודל לפי Test, גם אם הדבר מפתה אחרי ניסוי ראשון.



איור: עקומות סכמטיות — אינן תוצאות אימון אמיתיות.

```
from torch.utils.data import TensorDataset, DataLoader

loader = DataLoader(TensorDataset(train_x, train_y),
                    batch_size=64, shuffle=True)
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.003)

for epoch in range(5):
    model.train()
    running = 0.0
    for xb, yb in loader:
        optimizer.zero_grad()
        loss = criterion(model(xb), yb)
        loss.backward()
        optimizer.step()
        running += loss.item() * len(yb)

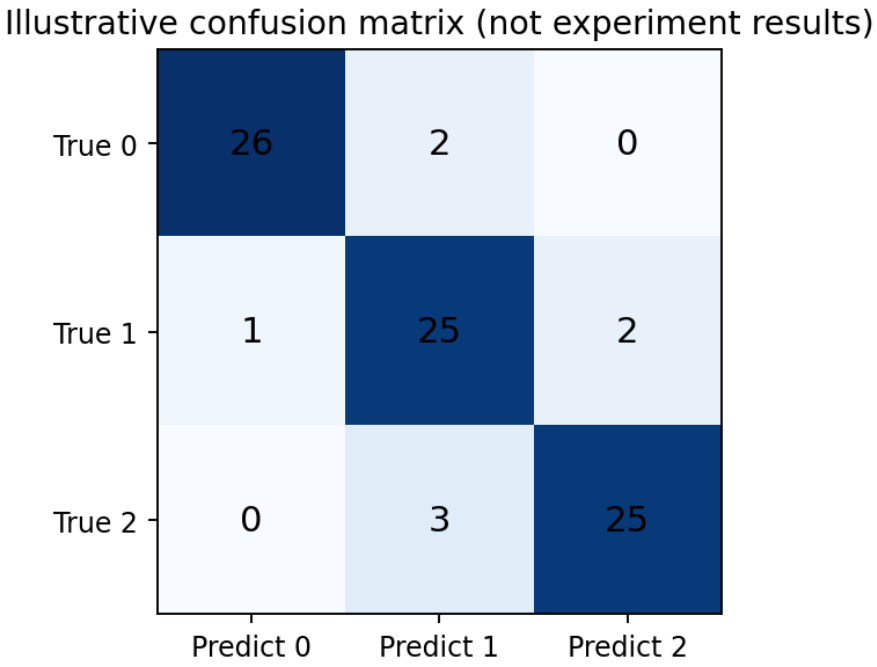
    model.eval()
    with torch.no_grad():
        val_logits = model(val_x)
        val_loss = criterion(val_logits, val_y).item()
        val_acc = (val_logits.argmax(1) == val_y).float().mean().item()
    print(epoch + 1, round(running / len(train_y), 4),
          round(val_loss, 4), round(val_acc, 3))
```

פלט המדדים תלוי באתחול, בסביבת הביצוע ובבחירות האימון; אין להבטיח תוצאה מסוימת. בדוגמת הפרק אין שימוש ב־*Test* לצורך בחירת *Epochs*.

16. כיצד נבדוק ביצועי CNN בצורה מדעית?

Accuracy מודד את שיעור התחזיות הנכונות, אבל אינו אומר אילו ספרות התבלבלו זו בזו. לכן נציג גם מטריצת בלבול, ונסתכל על כמה דוגמאות שסווגו בטעות. אם המודל מתקשה בספרות 3 ו־8, למשל, כדאי לשאול האם מדובר בכתב יד דומה, בעובי קו או בייצוג לא מספיק טוב.

מטריצת הבלבול שבתמונה הבאה היא דוגמה המחשה בלבד, ולא תוצאה שמדדנו במודל שלנו. במעבדה התלמיד יחשב מטריצה אמיתית מהתחזיות שלו, לאחר שהחליט על המודל. חשוב גם לחשוב על מדדי Macro F1 כאשר יש חוסר איזון משמעותי בין מחלקות.



איור: מטריצת בלבול מדגימה כיצד מנתחים את סוגי הטעויות.

```
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Inspect errors on validation during model development.
model.eval()
with torch.no_grad():
    val_pred = model(val_x).argmax(1).cpu().numpy()
print('CNN validation accuracy:', accuracy_score(y_val, val_pred))
print(confusion_matrix(y_val, val_pred))
print(classification_report(y_val, val_pred, zero_division=0))
```

17. השוואה הוגנת למודל בסיס — האם CNN תמיד מנצחת?

לא. על מאגרים קטנים מאוד, מודל KNN עשוי להיות תחרותי ואף להצליח יותר מרשת CNN מסוימת. CNN דורשת בחירות ארכיטקטורה, אימון ומשאבי חישוב, ואילו KNN מבוסס על דוגמאות שמורות ומרחקים. השוואה מקצועית אינה מתחילה במסקנה אלא בניסוי הוגן: אותן קבוצות נתונים, אותן דרכי מדידה, ושימוש ב־Validation לצורך בחירת פרמטרים.

בדוגמת הספר שלנו נשווה תחזיות KNN לרשת CNN רק אחרי שהרשת עברה אימון. נשווה ביצועים, זמן הרצה, מספר פרמטרים וקשיים בניתוח טעויות. נדגיש שאין למדוד את ביצועי המודלים על Train ולדווח עליהם כיכולת הכללה.

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X_train.reshape(len(X_train), -1), y_train)
knn_val_pred = knn.predict(X_val.reshape(len(X_val), -1))
knn_val_accuracy = accuracy_score(y_val, knn_val_pred)
cnn_val_accuracy = accuracy_score(y_val, val_pred)
print('KNN validation:', round(knn_val_accuracy, 3))
print('CNN validation:', round(cnn_val_accuracy, 3))

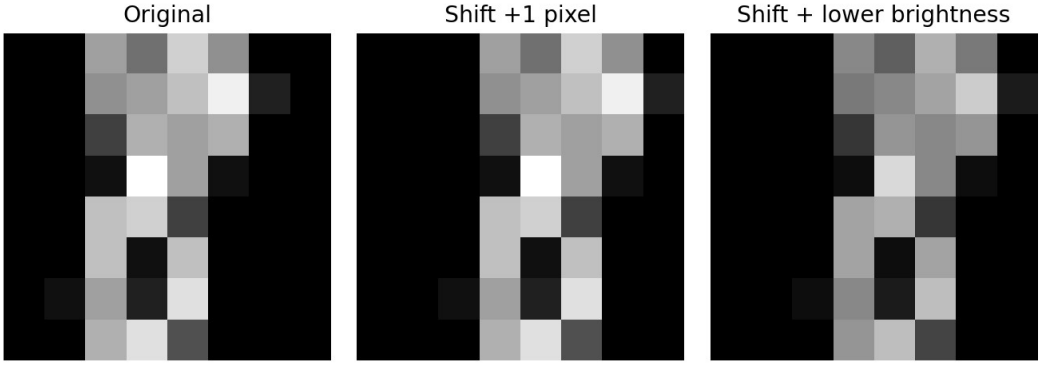
# Make the decision before looking at Test.
if knn_val_accuracy >= cnn_val_accuracy:
    selected = 'KNN'
    final_pred = knn.predict(X_test.reshape(len(X_test), -1))
else:
    selected = 'CNN'
    model.eval()
    with torch.no_grad():
        test_x = torch.tensor(X_test[:, None, :, :])
        final_pred = model(test_x).argmax(1).cpu().numpy()

print('Selected by validation:', selected)
print('Final test accuracy:', accuracy_score(y_test, final_pred))
print(confusion_matrix(y_test, final_pred))
```

18. Data Augmentation — למה להציג גרסאות נוספות של תמונה?

בפרויקט לזיהוי ספרות, צילום של אותה ספרה יכול להיות מעט מוזז או בעל בהירות שונה. אפשר להוסיף בזמן האימון שינויים ששומרים על התווית, כגון הזזה קטנה. זו שיטת Augmentation: יצירת דוגמאות אימון חדשות מתוך דוגמאות קיימות.

אסור להפעיל Augmentation על Test כדי להגדיל את מספר הדוגמאות המדווחות, ואסור לבחור שינויים שמשנים את משמעות הספרה. לדוגמה, סיבוב חד או שיקוף עלול ליצור ספרה אחרת או להפוך אותה ללא קריאה. על תלמיד בפרויקט להסביר מדוע הטרנספורמציות שבחר תואמות את המציאות הצפויה.



איור: המחשה של שינויי קלט אפשריים — לא כל שינוי מתאים לכל מאגר.

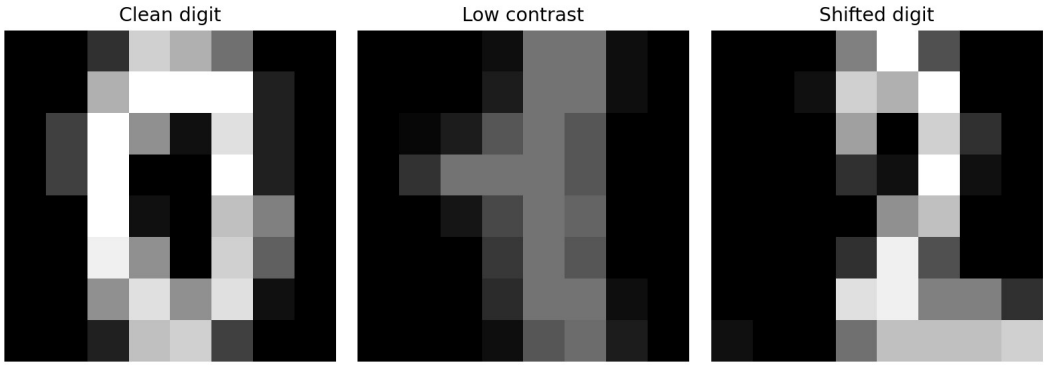
19. טעויות נפוצות שחייבים לזהות

טעות	מדוע זו טעות	כיצד מתקנים
שיטוח התמונה לפני Conv2d	נעלם המבנה המרחבי שהשכבה מנצלת	להשתמש ב־(N,C,H,W)
הוספת Softmax לפני CrossEntropyLoss	CrossEntropyLoss מקבלת Logits	להעביר ערכי Logits ישירים
שימוש ב־Test לבחירת Epoch	מידע מהבדיקה משמש לכיוון המודל	לבחור באמצעות Validation
בלבול בין מספר ערוצים לגודל התמונה	מספר מסננים קובע ערוצי פלט	לחשב בנפרד C_out ו־H_out,W_out
הנחה ש־CNN תמיד עדיפה על KNN	הצלחה תלויה בנתונים ובאימון	לבצע השוואה הוגנת
Augmentation שמשנה תווית	המודל לומד דוגמאות סותרות	לבחון כל טרנספורמציה בהקשר
שכחת model.eval() בזמן בדיקה	התנהגות של Dropout/BatchNorm יכולה להשתנות	להגדיר מצב בדיקה

20. חקר מקרה — מדוע המודל נכשל על תמונות שלא ראה?

נניח שרשת זיהתה תמונות ספרות מסריקות ברורות, אך במבחן על תמונות מטושטשות מאפליקציה חדשה הביצועים ירדו. לפני שמוסיפים שכבות יש לבדוק האם השוני נובע מתאורה, מרזולוציה, משיעור הספרות במאגר, מתהליך Crop, או מתוויות לא מדויקות. זהו שינוי התפלגות (Distribution Shift).

פתרון אפשרי אינו רק ״לאמן יותר״. ייתכן שצריך להגדיר מחדש תהליך עיבוד, להרחיב את נתוני האימון, לבצע הערכה לפי קבוצות או לשנות את הבעיה. בפרויקט גמר נדרשת יכולת לזהות את הסיבה האפשרית לכישלון ולהציע ניסוי מבוקר שיאמת אותה.



איור: שינויים בבהירות ובהזזה ממחישים מדוע יש לבדוק עמידות לשינויי קלט.

21. תרגול מדורג — הבנה, חישוב, ניתוח ונימוק

רמה א — הבנת מושגים

1. הסבירו בשפה שלכם מה ההבדל בין Kernel לבין Feature Map. מה מהם נלמד באימון?

2. הגדירו מהו Stride. הסבירו כיצד שינוי מ־1 ל־2 עשוי להשפיע על מספר תאי הפלט.

3. מהו ההבדל בין Max Pooling לבין Convolution מבחינת קיומן של משקולות נלמדות?

4. היכן מופיעה פונקציית ReLU ברשת, ומה קורה לפלט שלילי?

רמה ב — חישוב ידני

5. עבור מטריצה 5×5 ומסנן 3×3, Stride=1, Padding=0: חשבו את ממדי הפלט והסבירו את המספר.

6. עבור קלט 16×16×3 ו־12 מסננים בגודל 3×3, Padding=1: חשבו את מספר המשקולות ואת ממדי הפלט.

7. חשבו את ארבעת ערכי Max Pooling למטריצה 4×4 שבראש סעיף 10. השוו ל־Average Pooling.

8. לפלט Logits=[0,2,1] ולתווית אמת 1: בטאו את ה־Softmax של התווית ואת ה־Cross-Entropy. אפשר להיעזר במחשבון.

רמה ג — ניתוח מקצועי והכנה להגנה בעל־פה

9. תלמיד דיווח על Accuracy 99% בנתוני אימון ו־76% ב־Test. הציעו לפחות שלוש בדיקות שצריך לבצע לפני שינוי הארכיטקטורה.

10. השוו בין CNN ל־KNN בזיהוי ספרות: אילו יתרונות מעשיים יש לכל אחת, ומה אי אפשר להסיק ללא ניסוי?

11. מפתח הוסיף סיבוב של 180° לכל תמונת אימון. האם זה תמיד Augmentation תקין לספרות? נמקו.

12. בחרו בעיית תמונה לפרויקט גמר. הגדירו מאגר, תוויות, Baseline, חלוקת נתונים, שלוש בדיקות סיכון ושני מדדי הערכה.

22. תכנית הוראה מוצעת ל־50 שעות

זהו תכנון כיתתי מוצע של 10 יחידות בנות חמש שעות לימוד, לא חלוקה רשמית של מפגשי ההשתלמות.

יחידה	שעות	תוכן ותוצר
1	5	תמונות, טנזורים, קלט וידע קודם; מפת מושגים
2	5	Kernel וקורלציה צולבת; מפת מאפיינים בחישוב ידני
3	5	Padding, Stride, ערוצים וחישובי ממדים; בוחן קצר
4	5	Pooling, ReLU ואדריכלות; מעבדת NumPy
5	5	ארכיטקטורת CNN וספירת פרמטרים; פירוק שכבות
6	5	Forward, Loss, Backprop; ניסוי Logits
7	5	מימוש PyTorch, DataLoader ואימון מבוקר

תוכן ותוצר	שעות	יחידה
Validation, השוואה ל־KNN, מטריצת בלבול	5	**8**
Augmentation, ניתוח שגיאות ועמידות	5	**9**
מיני־פרויקט CNN, תיעוד והצגה בעל־פה	5	**10**
פרויקט למידה ותיק ניסויים	50	**סה״כ**

23. נספח לפרויקט הגמר — דף ניסוי CNN

מומלץ להדפיס או להעתיק את כרטיס הניסוי הבא לכל שינוי מהותי ברשת. מטרתו למנוע מצב שבו תלמיד משנה כמה רכיבים בבת אחת ואינו יודע מה גרם לשיפור. ניסוי איכותי מגדיר השערה, שומר את חלוקת הנתונים, מתעד את השינוי ומפריד בין תוצאות Validation לבין Test.

מה התלמיד מתעד	שדה
איזו בעיה אמיתית רוצים לפתור?	**שאלת חקר**
מקור, גודל, רישוי, איכות ואופן חלוקה	**גרסת מאגר**
שכבות, מסננים, Pooling ומספר פרמטרים	**ארכיטקטורה**
Learning rate, Epochs, Batch size, Seed	**Hyperparameters**
מודל פשוט להשוואה	**Baseline**
Accuracy, F1, Loss וניתוח טעויות	**מדדי Validation**
מה השתנה, מה נלמד ומה מגבלה	**החלטה מקצועית**
תוצאה לאחר סיום כל בחירות המודל	**בדיקת Test סופית**

24. הקשר לשלוש תכניות הלימוד

הקשר לחומר המקושר	מה התלמיד כבר יודע / לומד	מסלול
תשתית קוד ו־CNN; EDA אינה דרישת חובה מזוהה ביחידה 3	ייצוג נתונים, וקטורים, גרפים, KNN והערכת מודלים	**יחידה 3 — כיתה י׳**
תשתית מתמטית ומושגית; CNN אינה מוכרזת כאן כחומר חובה ביחידה 5	וקטורים, תקנון, מדדים ו־Validation	**יחידה 5 — נושאי ליבה**
יישום בתמונות, פיתוח מודל והכנה לפרויקט גמר	פרק 6 עיבוד תמונה; פרק 11 מתמטיקה; פרק 12 DNN; פרק 13 CNN	**התמחות למידה עמוקה**

25. מקורות מקצועיים והבחנה בין חובה להעשרה

מפת תכנית ההתמחות במאגר GitHub מגדירה את פרק 13 — CNN ואת היקפו (50 שעות). ההסברים מתבססים על רעיונות מקצועיים המופיעים ב־Dive into Deep Learning (פרקי CNN), Understanding

Deep Learning מאת Simon Prince (פרק 10), Hands-On Machine Learning מאת Aurélien Géron, ועל תיעוד PyTorch. הדוגמאות, המשימות והאיורים בספר מקוריים, ולא הועתקו מספרים מסחריים.

נושאים חיוניים ללמידה כאן: ייצוג טנזורים, מסננים, Convolution, Stride/Padding, Pooling, ארכיטקטורה, אימון בסיסי והערכת מודל. כהעמקה מוצעים חישובי Cross-Entropy ידניים, מורכבות פרמטרים, Augmentation וניתוח Distribution Shift. רק נוסח התכנית הרשמי המעודכן מוסמך לקבוע אילו פרטים הם חובה בבחינה או בפרויקט.

https://github.com/samiabo75/Data-Science-ML-Course/tree/main/03-Deep-Learning-5-Units/13-Convolutional-Neural-Networks

https://en.d2l.ai/chapter_convolutional-neural-networks/conv-layer.html

https://en.d2l.ai/chapter_convolutional-neural-networks/padding-and-strides.html

https://docs.pytorch.org/docs/stable/generated/torch.nn.Conv2d.html

https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_digits.html

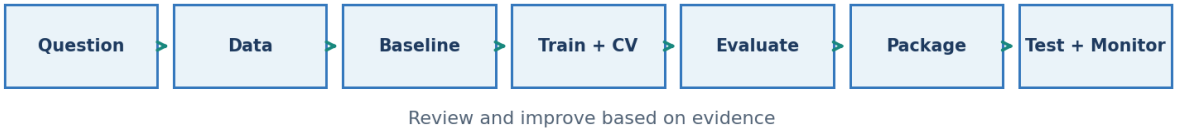
https://mitpress.mit.edu/9780262377102/understanding-deep-learning/

ספר הלימוד המאוחד במדעי הבינה המלאכותית

פרק 14 — ניהול פרויקט במדעי הנתונים

מן המחברת ב־Colab למערכת AI שאפשר לשחזר, לבדוק, להסביר ולהציג

מהדורת תלמיד | כיתות י״א–י״ב | התמחות למידת מכונה ולמידה עמוקה | תכנון לימודי ל־25 שעות



איור 1 — מחזור החיים של פרויקט למידת מכונה: התקדמות עם בדיקות וחזרה לתיקון.

מה נלמד בפרק?

עד כה למדנו כיצד לטעון נתונים, לבצע חקר, לאמן מסווגים ורשתות עצביות ולחשב מדדי הצלחה. אבל במעבדה אמיתית אין די בתא Colab שמפיק Accuracy גבוה. מפתח AI צריך לדאוג שהנתונים יישארו מזוהים, שהקוד ניתן להרצה חוזרת, שהמודל יעבוד גם מחוץ למחברת, ושהחלטות יהיו מתועדות ומבוססות ראיות.

בפרק זה נפתח את עבודתו של דודו: מערכת זיהוי ספרות בכתב יד. לא נבנה רשת עצבית חדשה — כבר למדנו לעשות זאת בפרקים קודמים. הפעם נלמד לעטוף מודל קיים בתהליך הנדסי ומחקרי אחראי. כל שלב ילווה בתוצר שאפשר להציג למורה ולבוחן.

עוגן התכנית וידע קודם

מפת ההתמחות במאגר הקורס מייעדת לפרק 14 — "ניהול פרויקט במדע הנתונים: היבטים מעשיים" — 25 שעות. תתי־הנושאים והמימושים המוצעים כאן הם הרחבה דידקטית מקצועית על סמך ספרי המקור והנחיות הנדסת ML; אין להציג את חלוקת השעות והכלים הספציפיים כציטוט מתוך הסילבוס הרשמי המלא, שעדיין מצריך הצלבה פרטנית.

ידע קודם: Python, NumPy, Pandas, תיעוד Notebook, KNN, חלוקת נתונים, Pipeline, Validation, Test, Accuracy, F1־ו, מטריצת בלבול ועקרונות עיבוד תמונה.

14.1 — סיפור המסגרת: למה קוד שעובד אינו תמיד מוצר?

דודו בנה מחברת שמזהה ספרות באימון. המודל קיבל ציון טוב, ולכן הוא שלח את המחברת לחברה שעובדת במחשב אחר. התברר שהקובץ לא מצא את נתוני האימון, המשתנים נוצרו בסדר הרצה לא ברור, ובמחשב החדש הותקנה גרסת ספרייה אחרת. בשלב הבא ניסו לחזות תמונה חדשה, אבל היא הייתה בגודל 28×28 ולא 8×8. שלוש בעיות שונות נחשפו: ניהול סביבת עבודה, חוזה קלט, ושחזור תוצאות.

הלקח הראשון הוא שיש להבחין בין אב־טיפוס מחקרי לבין מערכת אמינה. אב־טיפוס נועד להוכיח שרעיון עשוי לעבוד. מערכת מסודרת צריכה להגדיר מראש קלט ופלט, לבדוק כשלים, לתעד מגבלות ולאפשר לאדם אחר להפעיל אותה.

14.2 — ניסוח בעיה וקריטריון הצלחה

לפני כתיבת קוד מנסחים מה המערכת מקבלת, מה היא מחזירה, למי היא מיועדת וכיצד נחליט שהצליחה. בדוגמת הספר, הקלט הוא תמונה בגווני אפור של ספרה יחידה בגודל 8×8. הפלט הוא ספרה חזויה מ־0 עד 9. זו משימת סיווג רב־מחלקתי. הנתונים הם תמונות שכבר תויגו, ולכן מדובר בלמידה מונחית.

נגדיר את היעד במחקר: להשוות מסווג פשוט למודל מבוסס מרחק ולבדוק ביצועים על דוגמאות שלא שימשו לבחירת המודל. אחוז הצלחה לבדו אינו מספיק: נדרוש גם Macro F1, ניתוח דוגמאות שגויות וזמן סביר להפקת תחזית. ערכי סף מספריים יתאימו לצורכי הפרויקט ולא ייבחרו לאחר צפייה בתוצאות.

Success criteria = metric + threshold + test protocol + limitations

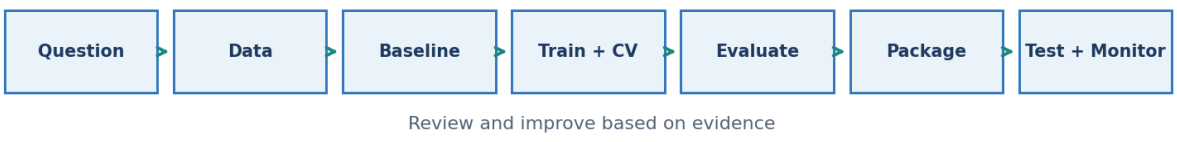
לדוגמה, אפשר להציע מראש כי המודל יעבור מבחן רק אם Macro F1 יהיה לפחות 0.90 על קבוצת Test שמורה, והמערכת תדחה קלט שאינו במבנה המוגדר. זהו קריטריון הוראה מוצע ולא תקן רשמי או התחייבות לביצועים.

14.3 — מסמך דרישות קצר (Project Brief)

למה צריך להגדירו?	החלטת הפרויקט	רכיב
מבהיר למי מיועדת המערכת	תלמיד המעוניין לבדוק ספרה כתובה	משתמש
מונע אי־התאמה למאגר	64 ערכים מספריים, טווח 0–16	קלט
מונע ערבוב עם הסתברות	תווית חזויה 0–9	פלט
מונע הסתמכות על מדד יחיד	Macro F1 לצד מטריצת בלבול	איכות
מבהיר אי־התאמה אפשרית לתמונות טלפון	תמונות 8×8 שדומות למקור	מגבלות
ניהול נתונים אחראי	לא משתמשים בתמונות אישיות ללא אישור	פרטיות

14.4 — שלבי הפרויקט וקריטריוני סיום

אפשר לראות בפרויקט רצף של תוצרים: כרטיס בעיה, כרטיס נתונים, מחברת EDA, Baseline, ניסוי ולידציה, מודל שמור, בדיקות קלט, דוח מסכם והדגמה. לכל שלב נגדיר תנאי סיום. כך התלמיד יודע מתי עבודה הושלמה, והמורה יכול לזהות בעיות לפני שלב מאוחר שבו קשה לתקן אותן.

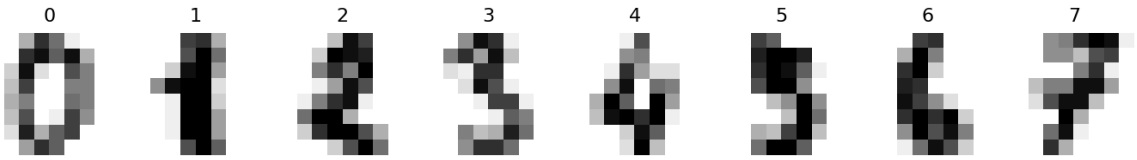


איור 2 — בכל מעבר שלב נדרש תוצר לבדיקה: נתון, קוד, מדד או הסבר.

14.5 — כרטיס נתונים (Data Card)

בכל פרויקט יש לתעד את מקור הנתונים, הרישוי, תאריך האיסוף, משמעות המאפיינים, אופן יצירת התוויות, נתונים חסרים, ייצוג קבוצות אפשרי וסיכונים. כרטיס כזה אינו בירוקרטיה: בלעדיו קשה לדעת אם אפשר להשתמש בנתונים באופן חוקי, האם מותר לפרסם אותם, והאם נתון מסוים יהיה זמין בזמן חיזוי אמיתי.

בדוגמת הספר נשתמש במאגר digits שמגיע עם scikit-learn: 1,797 תמונות, כל תמונה 8×8 פיקסלים בגווני אפור, והערכים במאגר נעים בין 0 ל־16. כל תמונה מיוצגת גם כווקטור באורך 64. המטרה היא ספרה מסומנת. מאגר זה מתאים להדגמת תהליך מבלי לאסוף מידע אישי על תלמידים.

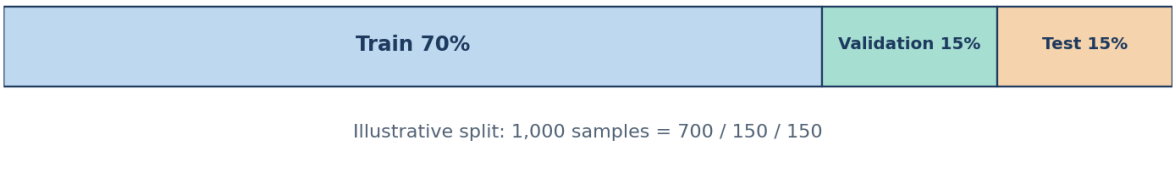


איור 3 — דוגמאות מתוך מאגר *digits* של *scikit-learn*. מספר מעל כל תמונה הוא התווית.

אם נרצה להחליף בהמשך את המאגר בצילומים מהטלפון, אי אפשר להניח שהמודל יעבוד באותה איכות. צילום כזה עשוי להוסיף צבע, צל, סיבוב ורקע, וקיים פער בין תחום נתוני האימון לבין הנתונים החדשים.

14.6 — הפרדת Train, Validation ו־Test

קבוצת Train משמשת ללמידת פרמטרים. קבוצת Validation או קיפולי Cross-Validation משמשים לבחירת מודל והיפר־פרמטרים. קבוצת Test נועדה לאומדן סופי של הכללה אחרי שכל ההחלטות התקבלו. שימוש ב־Test כדי לבחור k ואחר כך דיווח על אותה תוצאה כאילו הייתה עצמאית הוא שגיאה מתודולוגית.



איור 4 — חלוקה מספרית להמחשה בלבד: מתוך 1,000 דוגמאות, 700 לאימון, 150 לוולידציה ו־150 לבדיקה.

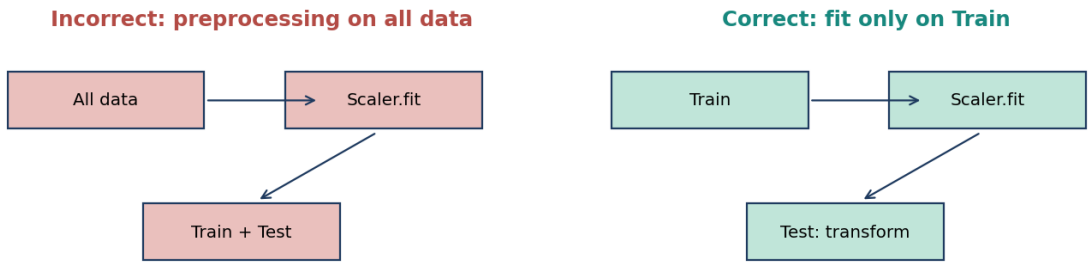
ב־digits נשתמש בדוגמה מודרכת בהפרדה חיצונית של 20% ל־Test, ולאחר מכן נבצע Cross-Validation רק על נתוני Train. נשמור random_state קבוע ונשתמש ב־stratify לשמירה בקירוב על חלקה של כל ספרה בחלוקה. במאגר זמנים או כשיש תמונות רבות מאותו אדם, חלוקה אקראית פשוטה עלולה להטעות; במקרה כזה יש לתכנן חלוקה לפי זמן או לפי קבוצות.

$$1{,}000 \times 0.70 = 700 \;;\; 1{,}000 \times 0.15 = 150 \;;\; 1{,}000 \times 0.15 = 150$$

14.7 — מניעת זליגת מידע כתנאי לאמינות

דוגמה לשגיאה: התלמיד מבצע תקנון על כל המאגר ורק אחר כך מחלק ל־Train ול־Test. כך אפילו ממוצע פשוט חושף לתהליך האימון מידע מנתוני הבדיקה. פתרון נכון: מחלקים קודם, וכל שלב שלומד

נתונים — למשל StandardScaler, בחירת מאפיינים או השלמת חסרים — מותאם ל־Train בלבד. בתוך Cross-Validation, שלבי ההכנה צריכים להילמד מחדש בכל קיפול אימון.



איור 5 — משמאל: פעולה לא נכונה על כל הנתונים; מימין: התאמה לנתוני האימון ושימוש ב־*transform* לנתונים חדשים.

Pipeline מסייע למנוע שגיאה זו משום שהוא כולל באותו אובייקט את שלבי ההכנה והמסווג. פעולה fit מתאימה את השלבים על נתוני האימון, ואילו predict מפעילה על קלט חדש את אותם שלבים בסדר הנכון. Pipeline אינה מתקנת אוטומטית כל דליפה, למשל תווית עתידית שהכנסנו לתכונה.

14.8 — ארגון תיקיות: מחברת אינה כל הפרויקט

כשכל הקוד מצוי במחברת אחת קשה לנהל שינוי, לבדוק רכיב בנפרד או להציג למפתח אחר מה צריך להריץ. לכן נפריד בין נתונים, מחברות חקר, קוד משותף, מודלים שמורים, בדיקות ותיעוד. מבנה מוצע לפרויקט תלמיד:

```
handwritten-digit-project/
    README.md
    requirements.txt
    .gitignore
    data/
        README.md
    notebooks/
        01_exploration.ipynb
        02_experiments.ipynb
    src/
        train.py
        predict.py
    models/
        model_info.json
    reports/
        experiment_log.csv
        model_card.md
    tests/
        test_prediction.py
```

העיקרון חשוב יותר מהשמות: תלמיד אחר צריך לדעת מה הקלטים, מה פותחים קודם, אילו קבצים נדרשים ומהו התוצר. README טוב יכלול את שאלת הפרויקט, המקורות, הוראות התקנה, שיטת הערכה ומגבלות ידועות.

14.9 — ניהול גרסאות עם Git

Git עוקב אחר שינויים בקוד ובתיעוד. כל Commit הוא צילום מצב מוסבר של העבודה. אם ניסוי חדש נשבר, אפשר להבין אילו קבצים השתנו. רצוי לכתוב הודעות Commit בעלות משמעות, כגון "Add train-test split" או "Document prediction input validation", במקום "final final 3".

אין להעלות אוטומטית נתונים אישיים, סיסמאות, מפתחות API וקובצי מודל גדולים ל־Git. ב־gitignore. נציין קבצים זמניים ורגישים. כאשר נדרשת גרסת נתונים גדולה, אפשר ללמוד על כלים כגון DVC כהרחבה — אך אין חובה להתקינם בפרויקט הראשון.

```
# .gitignore - example
__pycache__/
.ipynb_checkpoints/
.env
*.joblib
private_data/
```

14.10 — סביבת עבודה ושחזור (Reproducibility)

ניסוי משוחזר הוא ניסוי שאדם אחר מסוגל להריץ לפי אותן הוראות ולקבל תוצאה עקבית במידה סבירה. כדי לשחזר, יש לשמור את גרסאות הספריות, גרסת המאגר, קוד התהליך, פרמטרים שנבחרו וזרעים אקראיים. random_state מסייע לשלוט בחלק מהאקראיות, אך הוא לא מבטיח שתוצאה תהיה זהה על כל חומרה, גרסת ספרייה או מימוש מקבילי.

requirements.txt־ב מומלץ לתעד גרסאות שתואמות להרצה שנבדקה. אין להעתיק מספרי גרסאות באופן שרירותי; אפשר לייצר רשימת ספריות מתוך הסביבה הפעילה ולסנן אותה לתלויות הנדרשות. רצוי גם לציין גרסת Python ותאריך בדיקה.

14.11 — מודל בסיס (Baseline) לפני סבך של ניסויים

לפני KNN או CNN נבדוק מה משיגה שיטה פשוטה שאינה לומדת קשר בין פיקסלים לספרה, למשל תחזית של הספרה השכיחה ביותר ב־Train. אם יש עשר מחלקות בכמויות דומות, נקבל בערך עשירית מהתחזיות נכונות; התוצאה המדויקת תלויה בחלוקה. בלי Baseline אי אפשר לדעת אם המודל שלנו מוסיף ערך משמעותי.

לפעמים פתרון המבוסס על כללים פשוטים מתאים יותר מפרויקט מסובך. לכן על התלמיד להסביר לא רק איזה מודל טוב יותר, אלא גם מהו מחיר התחזוקה שלו, כיצד נבדק ומהן מגבלותיו.

14.12 — תהליך אימון מודרך ב־Python

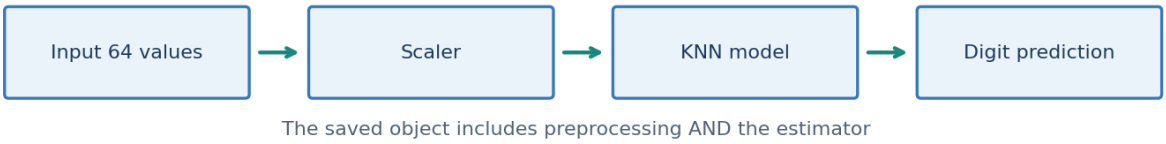
נבצע ניסוי בסיסי על digits. בחרנו להשתמש ב־StandardScaler וב־KNN בתוך Pipeline. הקוד שלהלן ניתן להרצה לפי הסדר; בהמשך הפרק נתרגל הפרדה בין קובץ אימון לקובץ חיזוי. אין לפרש את התוצאה כיכולת לזהות כל תמונה בעולם האמיתי.

```
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, f1_score


digits = load_digits()
X, y = digits.data, digits.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, stratify=y, random_state=42
)
model = make_pipeline(
    StandardScaler(), KNeighborsClassifier(n_neighbors=3)
)
model.fit(X_train, y_train)
```

```
pred_train = model.predict(X_train)
print('Train Accuracy:', accuracy_score(y_train, pred_train))
print('Train Macro F1:', f1_score(y_train, pred_train, average='macro'))
```

זהו ניסוי הדגמה לקוד, ולא פרוטוקול בחירת מודל סופי: לבחירת מספר השכנים נשתמש ב־Cross-Validation על Train בלבד, ורק אחריה נמדוד ביצועים סופיים על Test שמור.

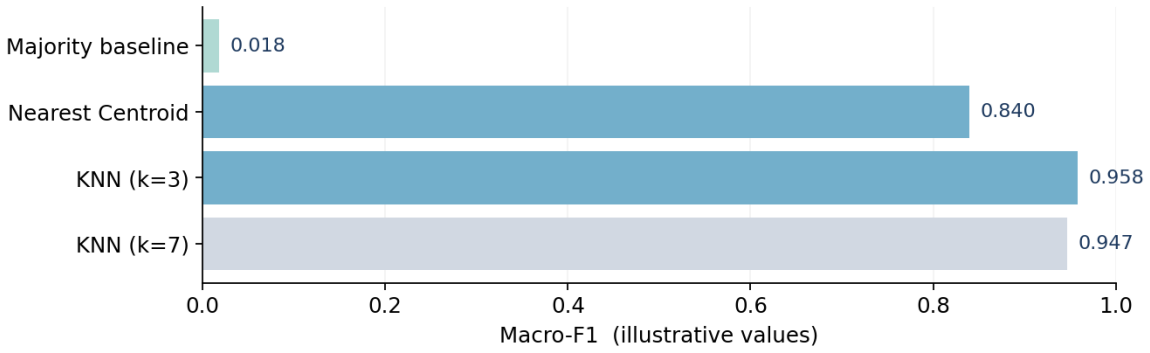


איור 6 — ה־*Pipeline* משמר את אותו עיבוד גם בזמן חיזוי של נתונים חדשים.

14.13 — יומן ניסויים: מה שינינו, ומה למדנו?

ניסוי טוב משנה גורם מתוכנן, מודד תוצאות ומסביר אותן. אם נחליף גם מאפיינים, גם מודל, גם שיטת חלוקה וגם מדד בבת אחת, לא נדע מה גרם לשינוי. לכן נתעד גרסת נתונים, מאפיינים, מודל, היפר־פרמטרים, שיטת ולידציה, מדדים, זמן ריצה ומסקנה.

ניסוי	מה נבדק	מדד CV	החלטה
E01	Baseline של מחלקת הרוב	המחשה — 0.018	נקודת ייחוס
E02	Nearest Centroid	המחשה — 0.840	השוואה ראשונית
E03	KNN עם k=3	המחשה — 0.958	מועמד מוביל
E04	KNN עם k=7	המחשה — 0.947	השוואה נוספת



איור 7 — ערכי הדגמה מומצאים להוראת יומן ניסויים; אינם תוצאות הרצה על *digits*.

בתוך מחברת התלמיד יתועדו תוצאות ההרצה האמיתיות. יש להבדיל בין מספר בדוגמה פדגוגית לבין מספר שהתקבל מריצה, ולשמור את שיטת המדידה כדי שנוכל לפרש פערים.

14.14 — ניתוח תוצאות ולא רק מספר אחד

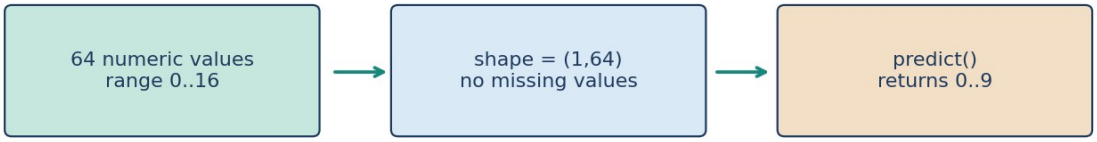
Accuracy שווה למספר התחזיות הנכונות מחולק במספר הבדיקות. אם מתוך 100 דוגמאות 92 סווגו נכון, נקבל 92/100 = 0.92. עם זאת, בסיווג כמה מחלקות חשוב לבדוק גם Macro F1, שמחשב את ממוצע ציוני F1 של המחלקות ללא שקלול לפי גודל המחלקה.

Accuracy = correct predictions / all predictions = 92 / 100 = 0.92

מטריצת בלבול מאפשרת לגלות אילו זוגות ספרות מתבלבלים. אם המודל טועה לעיתים בין 3 ל־8, ייתכן שהתמונות דומות או שמאגר האימון אינו מייצג דיו כתב יד מסוים. הממצא מחייב בדיקה של דוגמאות ולא הסקת סיבה חד־משמעית מתוך המטריצה לבדה.

14.15 — חוזה קלט ופלט (Inference Contract)

כשמעבירים מודל ממחברת למערכת, חובה להגדיר מה מותר להעביר אליו. לדוגמה: מערך של 64 מספרים בטווח 0 עד 16, ללא ערכים חסרים. לאחר הבדיקה נבנה שורה אחת במבנה (1,64) ונפעיל predict. התוצאה היא תווית ספרה ולא הסתברות. אם רוצים להציג הסתברות, נדרש לבדוק מה מחזיר המודל ואת מידת הכיול שלו; אין להפוך מרחק או הצבעת רוב אוטומטית ל"אחוז ביטחון" אמין.



Prediction service contract: validate before calling the model

איור 8 — חוזה הקלט והפלט מונע קריאות שגויות לפני שהן מגיעות למודל.

```
import numpy as np

def predict_digit(values, trained_model):
    arr = np.asarray(values, dtype=float)
    if arr.shape != (64,):
        raise ValueError('Expected exactly 64 values')
    if not np.isfinite(arr).all():
        raise ValueError('All values must be finite')
    if np.any(arr < 0) or np.any(arr > 16):
        raise ValueError('Pixel values must be in 0..16')
    return int(trained_model.predict(arr.reshape(1, -1))[0])
```

הקוד בודק את מבנה קלט הדוגמה, אך מערכת אמיתית זקוקה גם לטיפול מסודר בשגיאות ולהצגת הודעות מובנות למשתמש. שימו לב: תמונת טלפון אינה קלט תואם עד שתעבור תהליך עיבוד מוגדר ומאומת.

14.16 — שמירת מודל ושחזור תחזית

בסיום אימון נשמור את אובייקט ה־Pipeline, כדי שלא נשכח את שלב התקנון בעת הפעלת המודל. אפשר להשתמש ב־joblib עבור קובץ שנוצר על ידינו בסביבה מהימנה. חובה להזהיר כי טעינת קובצי joblib או pickle ממקור לא מהימן עלולה להפעיל קוד; אסור לטעון מודל אקראי שהגיע מאינטרנט או מאדם לא מוכר.

```
from pathlib import Path
import joblib

path = Path('digit_pipeline.joblib')
joblib.dump(model, path)       # save your own trusted model
restored = joblib.load(path)   # only from a trusted source
sample = X_train[0]
assert int(restored.predict([sample])[0]) == int(model.predict([sample])[0])
```

בנוסף למודל נשמור קובץ metadata בפורמט JSON ובו גרסת מודל, שם המאגר, רשימת המאפיינים, גרסאות תוכנה, תאריך ניסוי, שיטת בדיקה ומגבלות. קובץ JSON מכיל תיעוד, לא את המודל עצמו. אין להניח שקובץ מודל ישן יעבוד באופן נתמך אחרי שדרוג ספריות גדול.

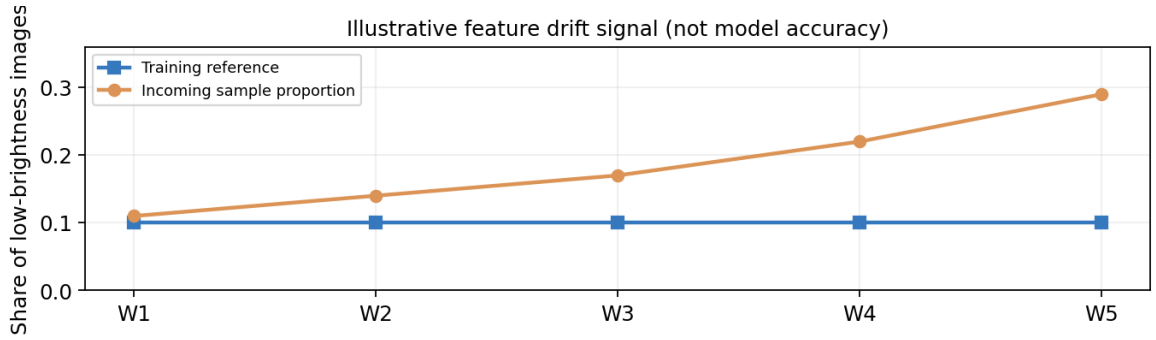
14.17 — בדיקות תוכנה: האם המערכת מתנהגת כמצופה?

בחינת מודל אינה זהה לבדיקת תוכנה. ייתכן שמודל משיג ציון מצוין אך היישום קורס אם משתמש שולח 63 ערכים במקום 64. נבנה בדיקות חיוביות ושליליות: קלט תקין, אורך שגוי, ערך חסר, ערך מחוץ לטווח, קובץ מודל חסר ואי־התאמה בין גרסאות. את הבדיקות נריץ מחדש לאחר שינוי בקוד.

בדיקה	קלט	תוצאה צפויה
חיזוי תקין	64 מספרים בטווח 0–16	ספרה בין 0 ל־9
צורה שגויה	63 מספרים	הודעת שגיאה מובנת
ערך לא חוקי	פיקסל שלילי	דחיית קלט
ערך חסר	NaN	דחיית קלט
שחזור	מודל שמור מהימן	אותה תחזית לדוגמת בדיקה

14.18 — ניטור, שינוי נתונים ותחזוקה

מודל שאומן על כתב יד מסוג אחד עלול לפעול אחרת כשהמשתמשים משתנים. יש להבחין בין שינוי בהתפלגות הקלט (Data Drift) לבין ירידה אמיתית באיכות (Performance Degradation). ניתן לזהות שינוי בסטטיסטיקות של הקלט גם לפני שיש לנו תוויות אמת, אך כדי למדוד Accuracy או F1 צריך תוויות אמת אמינות.



איור 9 — גרף מומצא של אות לשינוי נתונים, לא של איכות המודל. עלייה באות מחייבת בדיקה, לא מסקנה אוטומטית שהמודל כשל.

במערכת כיתתית אין צורך להקים תשתית ניטור עננית. מספיק להגדיר בדיקה תקופתית: האם טווחי הקלט השתנו? האם מופיעים ערכים חסרים? האם יש דוגמאות חדשות שעליהן המודל נכשל? כל שינוי במודל יחייב גרסה חדשה, תיעוד והערכה הוגנת. אין לשמור נתונים אישיים לצורך ניטור ללא בסיס ואישור מתאימים.

14.19 — פרטיות, רישוי ואחריות

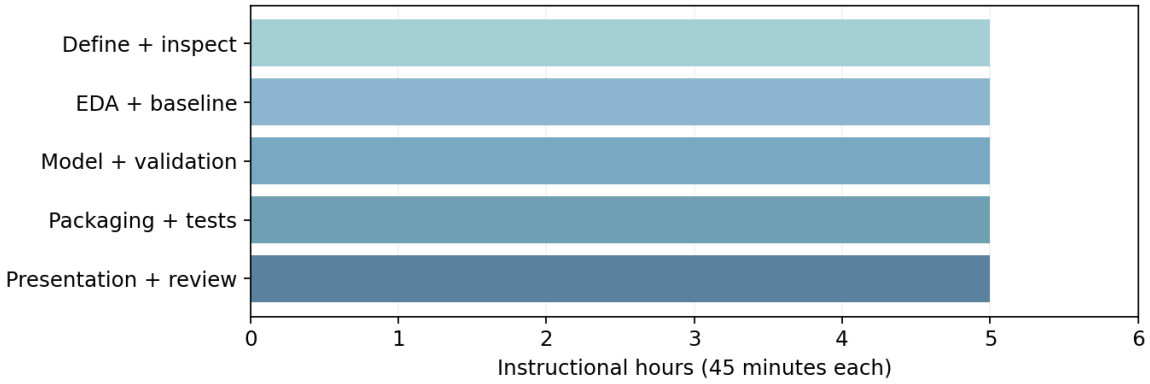
אם הפרויקט משתמש בתמונות של תלמידים, יש לשאול האם בכלל צריך לשמור אותן. אפשר לעבוד תחילה על מאגר ציבורי מתאים ולמנוע איסוף מיותר. כשהפרויקט נוגע לאנשים יש לבדוק ייצוג

אוכלוסיות, נזק אפשרי מטעות, זכויות שימוש ורישוי. אין להבטיח שהמערכת "נטולת הטיה" רק משום שהצליחה על Test אחד.

בפרויקט גמר יש לציין במפורש אם הנתונים מקוריים או פתוחים, מה מותר לעשות בהם, האם נדרש אישור צילום, אילו פרטים נמחקו או הוסוו ואילו מגבלות נשארו. תחזית מודל אינה החלטה מוסמכת בנושאים רפואיים, משפטיים או אחרים בעלי סיכון גבוה.

14.20 — תכנית עבודה מעשית ל־25 שעות

הטבלה שלהלן היא הצעת הוראה מקורית: 25 שעות לימוד לפי חלוקה לחמישה מקטעים בני חמש שעות (שעה לימודית היא 45 דקות). אפשר לפרוס את המקטעים לעשרה מפגשים קצרים יותר. בכל מקטע יש תוצר שמוגש ונבדק.



איור 10 — חלוקת שעות מוצעת, לא חלוקה רשמית של משרד החינוך.

שעות	משימות מובילות	תוצר להגשה
5–1	הגדרת בעיה, כרטיס נתונים, בדיקת שימושיות	Project Brief + Data Card
10–6	EDA, חלוקה, Baseline, הסברים לגרפים	מחברת EDA
15–11	Pipeline, CV, השוואת מודלים, ניתוח טעויות	יומן ניסויים
20–16	שמירת מודל, חוזה קלט, בדיקות, תיעוד	קובץ מודל + בדיקות
25–21	סקירת סיכונים, הצגת תוצר, שיפור והגנה	דוח ופרזנטציה

14.21 — חקר מקרה: האם המודל שבחרנו מתאים למשתמש?

נניח שמודל א׳ משיג Macro F1 של 0.95 אבל זמן חיזוי ממוצע של 400 מילישניות, בעוד מודל ב׳ משיג 0.94 וזמן חיזוי של 8 מילישניות. אין תשובה אוטומטית לשאלה מי טוב יותר. אם המערכת פועלת על טלפון חלש, זמן ההמתנה ומשאבי המכשיר עשויים להיות חשובים. אם טעות סיווג גוררת נזק גדול, הבדל המדד עשוי להיות משמעותי יותר. יש לבדוק שגם דרך המדידה זהה ושאותה קבוצת Test שימשה להשוואה לאחר סיום בחירת המודלים.

המספרים בסיפור זה מומצאים להמחשה. בשאלת הערכה, תלמיד צריך להציע בדיקה שתאפשר לבחור בצורה מקצועית ולא רק לבחור לפי תחושת בטן. אפשר לבקש מדד נוסף, בדיקת קבוצות משתמשים, ניסוי על נתונים חדשים או בדיקת זמן ריצה מדודה.

14.22 — תרגול מדורג בסגנון הערכה

רמה א׳ — הבנה ותיאור

משימה 1: הסבירו במילים שלכם את ההבדל בין תהליך אימון לבין תהליך חיזוי בסביבת משתמש.

משימה 2: ציינו ארבעה פרטים שחייבים להופיע בכרטיס נתונים, והסבירו מה עלול להשתבש כשאחד מהם חסר.

משימה 3: נתון פרויקט שבו כל הקוד נמצא במחברת אחת. הציעו חלוקה לתיקיות והסבירו את התועלת בכל תיקייה.

רמה ב׳ — חישוב, בדיקה והחלטה

משימה 4: מתוך 240 דוגמאות, המודל סיווג 216 נכון. חשבו Accuracy ופירשו את משמעות המדד והמגבלה שלו.

משימה 5: לפרויקט יש 1,000 דוגמאות. חלקו אותן ל־70% אימון, 15% ולידציה ו־15% בדיקה. הסבירו מתי מותר להשתמש בכל חלק.

משימה 6: תלמיד ביצע StandardScaler.fit על כל המאגר לפני החלוקה. הסבירו מדוע זה פוגע באמינות ואיך יש לתקן.

משימה 7: כתבו שלושה מקרי בדיקה לפונקציה שקולטת וקטור בן 64 ערכים. לפחות שניים צריכים להיות מקרי שגיאה.

רמה ג׳ — חקר והגנה על החלטות

משימה 8: בנו תכנית השוואה הוגנת בין KNN ל־Nearest Centroid על digits. פרטו Baseline, ולידציה ומדדי Test.

משימה 9: נסחו תכנית ניטור מצומצמת למערכת המזהה ספרות וכתבו כיצד תבחינו בין Data Drift לבין ירידה מוכחת באיכות.

משימה 10: הציעו שני סיכוני פרטיות או רישוי בפרויקט שמבוסס על תמונות של ילדים, והסבירו כיצד לצמצם אותם.

משימה 11: הגדירו קריטריון הצלחה מדיד לפרויקט אחר: זיהוי פגמים במוצרים או ניבוי זמני משלוח. נמקו בחירת מדד.

משימה 12: כתבו פסקת הגנה: מה הייתם משנים במערכת אם עלתה איכות האימון אבל איכות הבדיקה לא השתפרה?

14.23 — הכנה להסבר בעל־פה ולפרויקט גמר

בעת הגנה על פרויקט אין די במשפט "הרצתי מודל וקיבלתי 96%". יש לתאר את שאלת המחקר, מקור הנתונים, מה הוזן למודל, כיצד נבחרו הפרמטרים, אילו מדדים נמדדו, כיצד נמנעה זליגת מידע ומהן המגבלות. על התלמיד להיות מסוגל להדגים שינוי קטן בקוד, להציג בדיקה ולהסביר את תוצאתה.

דוגמאות לשאלות בוחן: מדוע בחרתם דווקא בנתונים האלה? כיצד אתם יודעים שמודלכם לא ראה את Test בזמן האימון? האם תוכלו לשחזר ניסוי ישן? מה קורה אם המשתמש שולח תמונה בפורמט אחר? האם שמירת מודל מבטיחה שהוא יפעל במחשב אחר? מה תעשו אם התפלגות הנתונים תשתנה?

מחוון כיתתי מוצע לפרק — שאינו מחוון משרד החינוך: הגדרת בעיה ונתונים 20 נקודות; תהליך ניסוי מבוקר 25; בדיקות וקוד מסודר 20; תיעוד ושחזור 15; ניתוח מגבלות ופרטיות 10; הצגה והגנה בעל־פה 10.

14.24 — מפת ההקשר בין שלוש תכניות הלימוד

תפקיד בהמשך	ידע שנדרש או נלמד	מסלול
בסיס לניתוח ולהסבר מודל	Pandas, גרפים, KNN ומדדי הצלחה	יחידה 3 — כיתה י׳
ביסוס מתמטי והערכה מדעית	מרחקים, K-Means, הערכה וולידציה	יחידה 5 — נושאי ליבה
הכנה ישירה לתוצר תוכנה ולעבודת גמר	ארגון קוד ונתונים, שמירת מודל, בדיקות ויישום	למידה עמוקה — התמחות

הקשר בין המסלולים הוא מיפוי דידקטי שלנו. כל דרישה פורמלית נבחנת בהתאם לתכנית ולמחוון הרלוונטיים, ואין להעתיק אוטומטית עומק של מסלול אחד לאחר.

14.25 — סיכום הפרק: מה הופך ניסוי לפרויקט מקצועי?

בשלב זה התלמיד כבר יודע לא רק לאמן מודל, אלא לחשוב כמו מפתח AI: להגדיר בעיה, לבחור נתונים, לתעד ניסויים, למנוע זליגה, לשמר את תהליך העיבוד, להגדיר חוזה קלט ולבדוק מקרים צפויים וחריגים. פרויקט איכותי אינו מדד יחיד ואינו רק Notebook ארוך; הוא רצף החלטות שאפשר לשחזר ולנמק.

בפרק 15 נלמד רשתות חוזרות (RNN). בהמשך, בפרק 16, נשתמש בכל תהליכי הניהול, התיעוד והבדיקה שנלמדו כאן לצורך עבודת גמר המבוססת על מודל למידה עמוקה ועל יישום תוכנה עצמאי.

נספח א׳ — דף תכנון פרויקט למילוי

זהו טופס עבודה לתלמיד. אין צורך למלא את כל הסעיפים לפני המפגש הראשון: יש לחזור אליו לאחר כל ניסוי ולתקן החלטות לפי הראיות. בכל שדה יש לכתוב תשובה קונקרטית שניתן לבדוק ולא סיסמה כללית.

מה נכתוב בפרויקט שלנו	השאלה שעלינו לתעד
_____	איזו בעיה נפתור, עבור מי ולמה?
_____	מה מקור הנתונים ומה תנאי השימוש?
_____	מהו מאפיין ומהי תווית המטרה?
_____	כיצד נחלק ל־Train, Validation ו־Test?
_____	מהו קו הבסיס ומה המדד הראשי?
_____	איזה ניסוי נבצע ראשון ומדוע?

מה נכתוב בפרויקט שלנו	השאלה שעלינו לתעד
_____	כיצד נבדוק קלט חדש למודל?
_____	מהם סיכון אחד ומגבלה אחת?

נספח ב׳ — יומן ניסויים למילוי

רשמו כל הרצה כראיה הניתנת לשחזור. אין למחוק תוצאה חלשה: גם כישלון מתועד עוזר לבחור ניסוי המשך. הקפידו לא לערבב ציוני Train, CV ו־Test באותה עמודה ללא ציון סוג המדידה.

מזהה ניסוי	מה שיניתי?	מדד + סוג בדיקה	מה הסקתי?
E01	_____	_____	_____
E02	_____	_____	_____
E03	_____	_____	_____
E04	_____	_____	_____

נספח ג׳ — רשימת בדיקה לפני הדגמה לבוחן

לפני ההגשה, הריצו את המחברת מההתחלה בסביבה נקייה וודאו שכל התאים החיוניים עובדים לפי הסדר. פתחו README במחשב אחר או בסביבה חדשה ובדקו שאפשר להבין כיצד מתקינים ומפעילים. הציגו דוגמה שהמודל מסווג נכון ודוגמה שנכשל בה, עם ניתוח ולא עם הסתרה. בדקו שניתן לשחזר חיזוי ממודל שמור ושבדיקת קלט לא חוקי מחזירה שגיאה מובנת.

מומלץ לשמור בפרויקט תמונת מסך של גרף בקרה, קובץ מדדים ברור, פירוט גרסאות ספריות, כרטיס נתונים וכרטיס מודל. אם הקובץ מכיל מידע רגיש יש להסירו לפני שיתוף. על כל תלמיד להיות מסוגל להצביע על קוד שחיבר בעצמו ולהסביר את ההיגיון שלו גם ללא עזרת כלי AI.

נספח ד׳ — כרטיס מודל קצר להגשה

כרטיס מודל הוא דף תיעוד שמסביר איזה מודל נבנה, לאיזו מטרה, על אילו נתונים, כיצד הוערך ומתי אסור להסתמך עליו. זהו כלי תיעוד מקצועי שמאפשר למישהו אחר להבין את ההחלטות שלכם, ולא קובץ שנוצר אוטומטית על ידי התוכנה.

שדה בכרטיס המודל	דוגמה שמראה מה נדרש
שם וגרסה	DigitClassifier, version 1.0
מטרת המודל	סיווג ספרה בודדת בתמונה 8×8
מקור נתונים	sklearn digits + תנאי שימוש
קלט / פלט	64 ערכי פיקסל 0–16 / ספרה 0–9
פרוטוקול בדיקה	Train 80% + CV פנימי, Test חיצוני 20%

דוגמה שמראה מה נדרש	שדה בכרטיס המודל
Accuracy, Macro F1 ומטריצת בלבול	מדדי איכות
לא נבדק על תמונות טלפון ברזולוציה שונה	מגבלות חשובות
גרסת Python ו־scikit-learn, קובץ Pipeline מהימן	גרסאות וקבצים

בוחן יכול לשאול: האם זו מערכת לזיהוי ספרות באופן כללי? התשובה המדויקת היא שהיא נבדקה על נתונים הדומים ל־digits; טענה רחבה יותר דורשת בדיקה עצמאית על אוכלוסיית תמונות חדשה. זהו ההבדל בין טענה מגובה לבין הבטחה שאיננה מבוססת ניסוי.

מקורות לקריאה ולהעמקה

עוגן מסלול: מאגר הקורס — פרק 14, 25 שעות (מפת התכנית; דורש הצלבה מול הסילבוס הרשמי המלא): https://github.com/samiabo75/Data-Science-ML-Course/tree/main/03-Deep-Learning-5-Units/14-Data-Science-Project-Management

ארגון ניסויים, אימון והערכה. — Géron, Hands-On Machine Learning with Scikit-Learn and PyTorch — James et al., An Introduction to Statistical Learning with Applications in Python — Zhang et al., Dive into Deep Learning. ניסויים ויישום. דוגמאות בחירת מודלים והערכת ביצועים. והסברים בפרק זה מקוריים.

scikit-learn — Common Pitfalls and Recommended Practices: https://scikit-learn.org/stable/common_pitfalls.html

scikit-learn — Model Persistence: https://scikit-learn.org/stable/model_persistence.html

Google for Developers — Rules of Machine Learning: https://developers.google.com/machine-learning/guides/rules-of-ml

Google for Developers — Productionization: https://developers.google.com/machine-learning/managing-ml-projects/production

DVC — User Guide (העשרה בלבד): https://doc.dvc.org/user-guide

הערת מקור: הספר אינו תרגום של ספר מסחרי. ההסברים, הדוגמאות, השאלות והתרשימים נוצרו לצורכי הוראה. אין לייחס לכלי כגון DVC, Git או joblib מעמד של דרישת חובה רשמית ללא אימות מול התכנית.

פרק 15 | רשתות עצביות חוזרות (RNN)

מהרצף והמצב החבוי ועד לאימון **RNN, GRU** ו־**LSTM** — הבנה מתמטית, קוד ופרויקט גמר

מסלול התמחות למידת מכונה ולמידה עמוקה | 5 יחידות לימוד | 30 שעות לפי מפת התכנית במאגר GitHub

גרסת תלמיד מורחבת • המחשות מקוריות • פתרונות הדוגמאות המודרכות בלבד • מעבדה עצמאית ללא פתרונות



אותם סוגי מדידות, אבל הסדר בזמן משנה את משמעותן.

מה נלמד ומדוע?

עד כה למדנו מודלים שמקבלים תמונה, טבלה או וקטור ומתייחסים אליהם כאל דוגמה אחת. בפרק זה נפגוש נתונים שהסדר שלהם חיוני: קריאות חיישן, מילים במשפט, תנועת אצבע על מסך ונתוני מכירות לאורך ימים. נלמד כיצד רשת עצבית יכולה לשמור סיכום פנימי של העבר ולעדכן אותו בכל צעד זמן.

הפרק נשען על הידע מהפרקים הקודמים: וקטורים ומטריצות, פונקציות הפעלה, נגזרות, גרדיאנטים, Backpropagation והערכת מודלים. כאן נוסיף את ממד הזמן ואת הבעיה של תלות בין צעדים רבים.

מטרות ביצועיות

- לזהות סדרה, צעד זמן, חלון קלט, אופק חיזוי ותלות בזמן; להבחין בין Sequence-to-One לבין Sequence-to-Sequence.
- לחשב ידנית שלושה מצבים חבויים, להסביר את השימוש החוזר במשקולות, ולזהות את ממדי המערכים.
- להסביר כיצד BPTT גוזר דרך זמן, לחשב עדכון קטן ולהבחין בין גרדיאנט נעלם לגרדיאנט מתפוצץ.
- לתאר את תפקידי השערים ב־LSTM וב־GRU, ולבחור מודל לפי בעיה וניסוי ולא לפי שם מרשים.
- לבנות ניסוי חיזוי סדרת זמן עם הפרדה כרונולוגית, קו בסיס, אימון, הערכה ותיעוד לקראת פרויקט גמר.

היכן הפרק משתלב בשלוש התכניות?

הקשר לפרויקט	ידע והיקף ההוראה	מסלול
פירוש סדרת מדידות והצעת חקר	קריאת טבלאות וגרפים, חלוקה ל־Train/Test ומודלים בסיסיים	**יחידה 3 — כיתה י׳**
הסבר מתמטי של קלט וגרדיאנט; אין לייחס RNN כחובת יחידה 5	וקטורים, תקנון, הערכה ולידציה כידע קודם	**יחידה 5 — נושאי ליבה**
בניית מסווג או חזאי סדרתי, תיעוד והגנה	RNN — פרק התמחות בהיקף 30 שעות במפת המאגר	**התמחות DL — פרק 15**

שימו לב: ה־GitHub. פירוש התוכן 30 שעות והשיוך לפרק 15 אומתו מול מפת תכנית ההתמחות ב־PDF. הרשמי התקף עדיין נדרש. והחלוקה הפנימית כאן הם עיבוד דידקטי; אימות מלא מול נוסח ה־

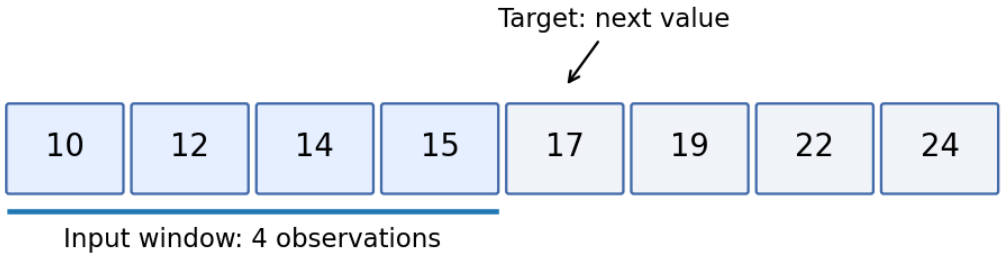
15.1 | מדוע הסדר חשוב? מסיפור על חיישן לשאלת חקר

דודו מנהל מעבדת מחשבים בבית הספר. בכל דקה חיישן מודד את טמפרטורת אחד המחשבים. אם נקבל את המדידות 28, 31, 34, 37 ו־31 מעלות, נחשוד בהתחממות מתמשכת. אך אם אותן מדידות מופיעות בסדר הפוך — 37, 34, 31, 28 — המחשב דווקא מתקרר. הממוצע, המינימום והמקסימום זהים, אולם הסיפור על התהליך שונה לחלוטין.

בנתון טבלאי רגיל, סדר השורות אינו בהכרח משנה את הדוגמה. בסדרה, כל מדידה תלויה במיקום שלה בזמן ובמה שקדם לה. לכן עלינו לשמר את הסדר, ולהימנע משימוש במידע מן העתיד כאשר אנו מבקשים לחזות אותו.

מושגים חדשים

מושג	פירוש	דוגמה
סדרה — Sequence	רצף תצפיות בעלות סדר משמעותי	קריאות חיישן על פני 20 דקות
צעד זמן — Time step	עמדה אחת ברצף, לא בהכרח שנייה אחת	דקה 5 ברצף
חלון — Window	מספר מדידות עוקבות המשמשות קלט	12 הדקות האחרונות
אופק חיזוי — Horizon	כמה צעדים קדימה חוזים	הטמפרטורה בדקה הבאה
Sequence-to-One	רצף קלט לתחזית אחת	האם תופיע תקלה?
Sequence-to-Sequence	פלט לכל צעד או רצף פלט	תווית לכל מילה במשפט



חלון של ארבע מדידות משמש לחיזוי המדידה הבאה. יש לשמור על סדר זמנים.

15.2 | כיצד הופכים סדרת מדידות לדוגמאות אימון?

נניח שקיבלנו שמונה מדידות ברצף. נבחר חלון באורך ארבע. דוגמת האימון הראשונה תכלול את המדידות במקומות 1 – 4 ותחזית היעד שלה תהיה המדידה במקום 5 . הדוגמה השנייה תכלול את המקומות 2 – 5 , והיעד יהיה המקום 6 . כך אנו יוצרים מאותו רצף מספר דוגמאות מבלי לשבור את סדר הזמן.

$$X_1 = [10, 12, 14, 15] \rightarrow y_1 = 17$$

$$X_2 = [12, 14, 15, 17] \rightarrow y_2 = 19$$

חלונות עוקבים חופפים זה לזה. עובדה זו חשובה מאוד בעת החלוקה לקבוצות אימון ובדיקה: אם נערבב חלונות שכנים בין הקבוצות, נתוני הבדיקה עלולים להיות דומים מדי לנתוני האימון ואף לשתף את רוב המדידות. הדרך הבטוחה בדוגמתנו היא לחלק קודם את הרצף לפי זמן, ואז ליצור חלונות בכל קבוצה בנפרד.

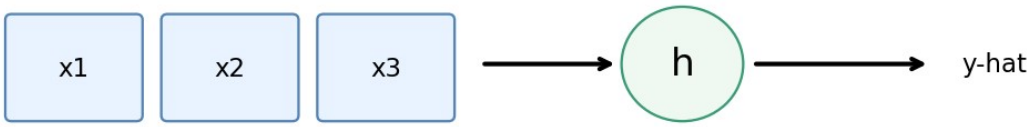
```
def create_windows(values, window=4):
    X, y = [], []
    for end in range(window, len(values)):
        X.append(values[end-window:end])
        y.append(values[end])
    return np.array(X), np.array(y)

values = np.array([10, 12, 14, 15, 17, 19, 22, 24])
X, y = create_windows(values, window=4)
print(X.shape, y.shape)   # (4, 4) (4,)
```

נסו להסביר מדוע מספר החלונות הוא מספר המדידות פחות אורך החלון, כאשר מנבאים את הערך הבא בצעד אחד. מה ישתנה אם נרצה לחזות שני צעדים קדימה?

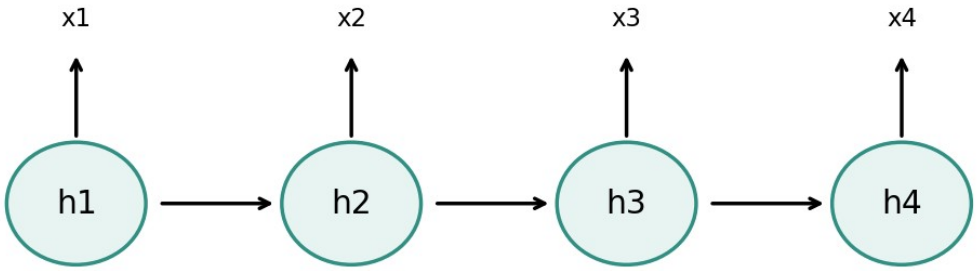
15.3 | מרשת רגילה לרשת עם זיכרון

ברשת Feedforward רגילה החישוב מתקדם מן הקלט לפלט, בלי שמצב פנימי עובר מדוגמה אחת לבאה. ברשת חוזרת קיימת יחידה שמקבלת בכל זמן גם קלט חדש וגם מצב חבוי מן הצעד הקודם. המצב החבוי אינו זיכרון מילולי כמו אצל אדם; זהו וקטור מספרי שנלמד לשמש סיכום שימושי של הרצף שנצפה עד כה.



Sequence model summarizes the ordered inputs

משימת *Sequence-to-One*: כל מדידות החלון מובילות לתחזית אחת.



The same W_x and W_h are reused at each time step

פריסה לאורך זמן: אותו תא *RNN* מופעל כמה פעמים עם אותן משקולות.

הנוסחה הכללית

$$h_t = \tanh(W_x\, x_t + W_h\, h_{t-1} + b_h)$$

$$\hat{y}_t = W_y\, h_t + b_y$$

לנוסחה יש שני חלקים. המכפלה הראשונה מודדת את תרומת הקלט הנוכחי; המכפלה השנייה מודדת את תרומת המצב הקודם; ההטיה מאפשרת הזזה של סכום התרומות. פונקציית tanh מוסיפה אי־ליניאריות ומחזירה ערכים בתחום שבין -1 לבין 1. אותם W_x, W_h ו־b_h משמשים בכל צעדי הזמן — זהו שיתוף משקולות לאורך הזמן.

שימו לב: שימוש באותן משקולות אינו אומר שאותו מצב חבוי מתקבל בכל פעם. הקלט והמצב הקודם משתנים, ולכן גם התוצאה משתנה.

15.4 | דוגמה פתורה: שלושה צעדים וחישוב כל מצב חבוי

כדי שנוכל לחשב ביד, נשתמש במצב חבוי מספרי יחיד. במערכת אמיתית הוא בדרך כלל וקטור בעל כמה רכיבים. נבחר את הפרמטרים הבאים:

$$h_0 = 0, \quad W_x = 0.5, \quad W_h = 0.4, \quad b_h = 0.1$$

$$x_1 = 0.2, \quad x_2 = 0.6, \quad x_3 = 0.4$$

בצעד הראשון אין זיכרון קודם ולכן h_0 שווה אפס. מחשבים את הסכום המשוקלל, מפעילים tanh ושומרים את התוצאה. בצעד השני משתמשים ב־h_1 שחושב בפועל; אין להתחיל בכל צעד מהתחלה.

$$z_1 = 0.5 \cdot 0.2 + 0.4 \cdot 0 + 0.1 = 0.2000$$

$$h_1 = \tanh(0.2000) \approx 0.1974$$

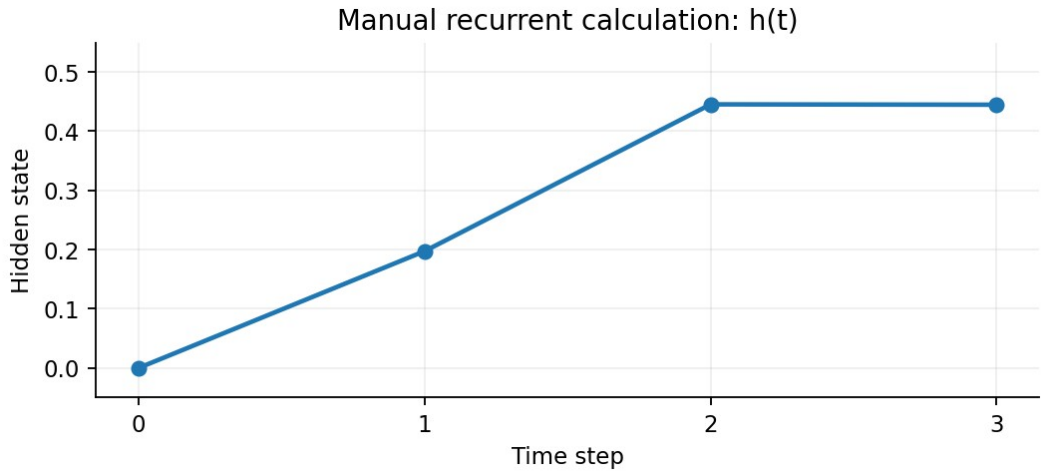
$$z_2 = 0.5 \cdot 0.6 + 0.4 \cdot 0.1974 + 0.1 \approx 0.4790$$

$$h_2 = \tanh(0.4790) \approx 0.4454$$

$$z_3 = 0.5 \cdot 0.4 + 0.4 \cdot 0.4454 + 0.1 \approx 0.4782$$

$$h_3 = \tanh(0.4782) \approx 0.4448$$

צעד	x_t	z_t לפני tanh	h_t אחרי tanh
1	0.2000	0.2000	0.1974
2	0.6000	0.4790	0.4454
3	0.4000	0.4782	0.4448



המצב החבוי משתנה בהדרגה בעקבות הקלט החדש והמצב הקודם.

נוסיף שכבת פלט ליניארית לחיזוי ערך מספרי. נגדיר משקל פלט, הטיה וערך מטרה, ונכתוב אותם בנפרד כדי למנוע בלבול בין הסימונים:

$$W_y = 0.8, \quad b_y = 0.05, \quad y = 0.35$$

$$\hat{y} = 0.8 \cdot h_3 + 0.05 \approx 0.4058$$

$$L = \tfrac{1}{2}(\hat{y} - 0.35)^2 \approx 0.00156$$

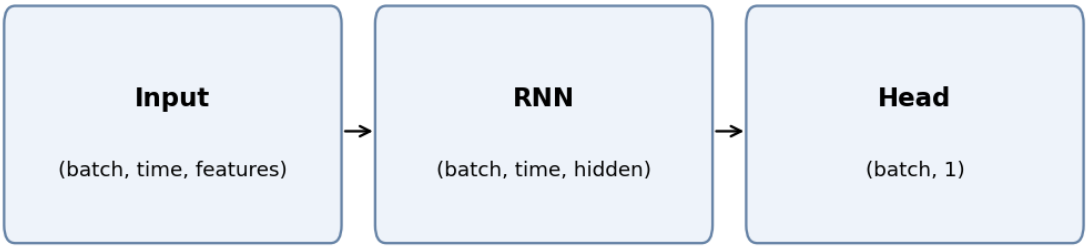
למדנו לבצע Forward Pass שלם. אך בשלב זה המשקולות עדיין קבועות. כדי שהרשת תלמד, נצטרך להבין כיצד שינוי במשקל בכל אחד מצעדי הזמן עשוי לשנות את השגיאה הסופית.

15.5 | ממדים וצורות מערכים — איך המחשב מארגן סדרה?

כאשר מאמנים רשת על כמה סדרות יחד, הקלט אינו וקטור יחיד ואינו מטריצה רגילה של תצפיות. הוא טנזור בשלושה ממדים: מספר דוגמאות באצווה, מספר צעדי זמן, ומספר המאפיינים בכל צעד. כאשר כל רצף מכיל שתים־עשרה דקות ובכל דקה נמדדים שני מאפיינים, דוגמה אחת היא מטריצה ואצווה של דוגמאות היא טנזור. הצורות המדויקות מופיעות בנוסחאות להלן.

$$\texttt{X.shape} = (\text{batch}, \text{sequence_length}, \text{input_size})$$

$$\text{One sample: } (12, 2) \qquad \text{Batch of 32: } (32, 12, 2)$$



כאשר *batch_first=True*, הקלט והשכבות פועלים בסדר (*batch, time, features*).

```
import torch
from torch import nn

X_batch = torch.randn(32, 12, 2)
rnn = nn.RNN(input_size=2, hidden_size=8, batch_first=True)
outputs, last_hidden = rnn(X_batch)
print(outputs.shape)       # (32, 12, 8)
print(last_hidden.shape)   # (1, 32, 8)
```

ממד `8` מציין את מספר הרכיבים במצב החבוי, ולא את מספר המחלקות. הממד הראשון של last_hidden מציין מספר שכבות כפול מספר הכיוונים; כאשר זו שכבה יחידה חד־כיוונית הוא שווה ל־ `1` . תכונת batch_first משנה את צורת הקלט והפלט הסדרתי, אך לא את סדר ממדי המצב החבוי.

15.6 | מהי למידה לאורך זמן? Backpropagation Through Time

ברשת רגילה עקבנו אחרי שגיאה מן הפלט לאחור דרך שכבות. ברשת חוזרת אותו פרמטר מופיע שוב ושוב בצעדי זמן שונים. לכן כאשר מחשבים נגזרת של השגיאה לפי משקל חוזר, יש להביא בחשבון את כל הדרכים שבהן המשקל השפיע על הפלט דרך הצעדים הקודמים. פעולת גזירה זו נקראת Backpropagation Through Time, בקיצור BPTT.

אין צורך לדמיין שיש לנו משקולות חדשות לכל צעד: בפריסה החזותית אנו מציירים עותקים של אותו תא לצורך הבנה, אך המשקולות משותפות. הגרדיאנטים מכל הופעותיהן מצטברים אל אותו פרמטר.

חישוב ידני מפושט של BPTT

כדי לחשב בלי נגזרות ארוכות של tanh, נשתמש בדוגמה לימודית עם הפעלה זהותית. זהו מודל ליניארי פשוט להמחשת כלל השרשרת, לא מודל מומלץ לאימון אמיתי.

$$h_1 = 0.5, \qquad h_2 = w \cdot h_1, \qquad h_3 = 0.5 + w \cdot h_2$$

$$\hat{y} = h_3 = 0.5 + 0.5w^2, \qquad L(w) = \tfrac{1}{2}(\hat{y} - 1)^2$$

נבחר משקל חוזר התחלתי $w=0.2$. נחשב קדימה: $h_2=0.1$, תחזית 0.52 והשגיאה 0.1152. כעת נגזור לאחור. הנגזרת של השגיאה לפי התחזית היא $\hat{y}-1$, והנגזרת של התחזית לפי w היא w. מכפילים אותן לפי כלל השרשרת.

$$dL/dw = (\hat{y} - 1) \cdot w = (0.52 - 1) \cdot 0.2 = -0.096$$

$$w_{new} = w - \eta(dL/dw) = 0.2 - 0.5(-0.096) = 0.248$$

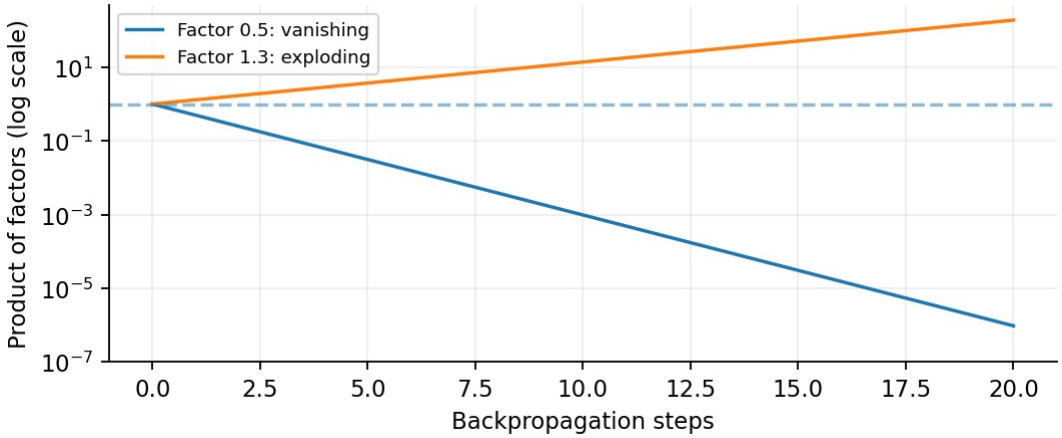
$$\hat{y}_{new} = 0.5 + 0.5 \cdot (0.248)^2 = 0.530752$$

$$L_{new} \approx 0.11010 < 0.11520$$

החישוב מלמד שעדכון פרמטר חוזר משפיע על כמה צעדי זמן יחד. הירידה בשגיאה תקפה בדוגמה ולצעד הזה; אין הבטחה שכל קצב למידה או כל צעד יקטינו תמיד את השגיאה.

15.7 | מדוע קשה לאמן רצפים ארוכים?

ב־BPTT מכפילים נגזרות רבות. כאשר הגורמים קטנים מ־ 1 לאורך צעדים רבים, המכפלה עשויה להתקרב לאפס: זהו גרדיאנט נעלם (Vanishing Gradient). במקרה כזה צעדים מוקדמים מקבלים אות עדכון חלש מאוד, וקשה ללמוד תלות רחוקה. כאשר המכפלות גדולות, הגרדיאנט עלול לגדול מאוד ולגרום לעדכונים לא יציבים: גרדיאנט מתפוצץ (Exploding Gradient).



המחשה מתמטית של מכפלות חוזרות: 0.5 בחזקת t דועך, ואילו 1.3 בחזקת t גדל.

$$0.5^{10} \approx 0.00098, \qquad 1.3^{10} \approx 13.79$$

הדוגמה המספרית מתארת מכפלות פשוטות, ולא טוענת שכל נגזרת בכל RNN היא בדיוק 0.5 או 1.3. ברשת אמיתית התמונה תלויה במשקולות, בפונקציות ההפעלה ובקלט לאורך זמן.

שתי דרכי התמודדות מרכזיות הן שימוש בתאי זיכרון עם שערים (LSTM/GRU), ובמקרה של גרדיאנט מתפוצץ שימוש ב־Gradient Clipping. קיצוץ גרדיאנטים יכול לייצב את הגודל שלהם, אך אינו פותר כשלעצמו בעיית גרדיאנט נעלם.

```
# Training step: clipping happens AFTER backward and BEFORE step.
optimizer.zero_grad()
prediction = model(X_train_batch)
loss = loss_fn(prediction, y_train_batch)
loss.backward()
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
optimizer.step()
```

15.8 | LSTM — תא זיכרון עם שערים

רשת LSTM נועדה לאפשר למידה יעילה יותר של תלות ארוכה. לצד המצב החבוי h_t היא מחזיקה מצב תא c_t המשמש מסלול זיכרון נפרד. שערים לומדים אילו חלקים מן הזיכרון הקודם לשמר, איזה מידע חדש להוסיף ומה לחשוף כפלט. המילה "שער" אינה מתג מכני; זהו וקטור של מספרים בין 0 ל־ 1 , המחושב באמצעות Sigmoid.



השוואה רעיונית: *RNN* פשוט, *GRU* עם שערים, *LSTM* עם מצב תא.

$$f_t = \sigma(W_f[x_t, h_{t-1}] + b_f) \quad \text{(forget gate)}$$

$$i_t = \sigma(W_i[x_t, h_{t-1}] + b_i) \quad \text{(input gate)}$$

$$g_t = \tanh(W_g[x_t, h_{t-1}] + b_g) \quad \text{(candidate)}$$

$$c_t = f_t \odot c_{t-1} + i_t \odot g_t$$

$$o_t = \sigma(W_o[x_t, h_{t-1}] + b_o) \quad \text{(output gate)}$$

$$h_t = o_t \odot \tanh(c_t)$$

הסימון ⊙ פירושו כפל רכיב ברכיב. הסימון [x,h] פירושו שרשור שני וקטורים. הנוסחאות כאן הן ייצוג חינוכי שמאחד קלט ומצב קודם; מימושי ספרייה מסוימים מחזיקים שתי מטריצות נפרדות והטיות נפרדות לכל שער. אין להסיק ש־LSTM תמיד מצליחה או שאין בה לעולם בעיות גרדיאנט.

דוגמה קצרה לשער שכחה

נניח שלמצב התא הקודם ערך 0.8, שער השכחה נתן 0.75, שער הקלט נתן 0.20 והמועמד החדש הוא 0.50. נחשב את הזיכרון המעודכן:

$$c_{new} = 0.75 \cdot 0.8 + 0.20 \cdot 0.5 = 0.60 + 0.10 = 0.70$$

הערך 0.70 משלב מידע שנשמר מן העבר עם מידע חדש. זהו חישוב של צעד אחד בלבד; בפועל הערכים מחושבים על ידי משקולות שנלמדות מהנתונים.

15.9 | GRU — שערים במבנה פשוט יותר

GRU משתמשת בדרך כלל במצב חבוי יחיד ובשני שערים מרכזיים: Reset, המשפיע על האופן שבו העבר נכנס למועמד למצב חדש, ו־Update, הקובע את היחס בין המצב הישן למצב החדש. המבנה פשוט יותר מן המבנה הסטנדרטי של LSTM מבחינת סוגי המצבים, אך אף אחת מהשיטות אינה מנצחת תמיד.

$$h_t = (1 - z_t) \odot n_t + z_t \odot h_{t-1}$$

שימו לב: בנוסחה של PyTorch, ערך z_t גדול נותן משקל גדול יותר לשימור המצב הישן. מקורות אחרים עשויים להגדיר את השער המשלים בכיוון הפוך; לכן יש לקרוא את הנוסחה ולא רק לזכור תיאור מילולי של שער עדכון.

מאפיין	RNN פשוט	GRU	LSTM
מצב עיקרי	h בלבד	h ושערים	h ו־c ושערים

מאפיין	RNN פשוט	GRU	LSTM
התמודדות עם תלות ארוכה	לעיתים קשה	משופרת לעיתים	משופרת לעיתים
מורכבות יחידה	נמוכה יחסית	בינונית	גבוהה יחסית
בחירה מקצועית	Baseline לימודי	ניסוי והשוואה	ניסוי והשוואה

15.10 | משימות סדרתיות נפוצות והחלטת מודל

בבעיית חיזוי סדרת זמן אפשר לקבל חלון של מדידות ולהחזיר ערך אחד — למשל טמפרטורה בדקה הבאה. בעיית Sequence-to-Sequence תחזיר פלט עבור כל זמן, למשל סימון קטעים לא תקינים ברצף. בניתוח טקסט אפשר לסווג משפט שלם או לתייג כל מילה. הבחירה בין משימות תלויה בשאלה שבאמת רוצים לענות עליה, ולא רק בטכנולוגיה הזמינה.

רשת דו־כיוונית יכולה להתחשב הן בעבר והן בעתיד *בתוך רצף שכבר זמין כולו*. אין להשתמש בה בלי זהירות בתחזית בזמן אמת כאשר בעת קבלת ההחלטה המידע מן העתיד עדיין אינו ידוע. זו דוגמה לשגיאה מערכתית שאינה נפתרת על ידי Accuracy גבוה.

15.11 | תהליך ניסוי הוגן לסדרות זמן

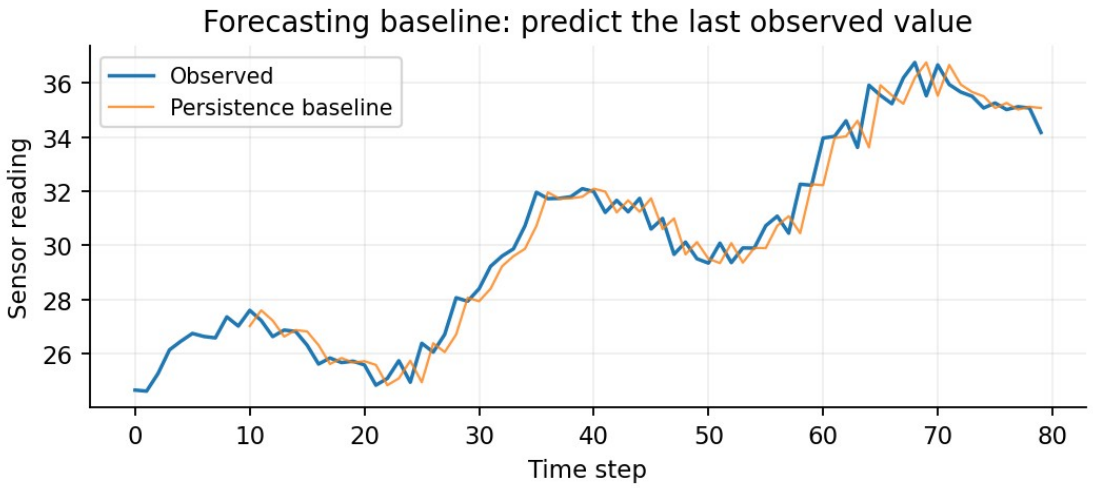
בסדרות זמן סדר החלוקה חשוב במיוחד. לדוגמה, מודל שאמור לפעול בחודש דצמבר צריך להיבחן על תקופה מאוחרת מזו ששימשה לאימונו. אם נערבב נתונים מכל השנה, הסטטיסטיקות של התקופה העתידית עלולות לחדור לאימון. גם תקנון, השלמת חסרים והתאמת פרמטרים צריכים להיעשות רק על נתוני האימון המתאימים.



דוגמה לחלוקה כרונולוגית: אימון בעבר, ולידציה לאחריו, בדיקה סופית בעתיד.

כאשר המערכת אמורה להשתמש במדידות העבר שקדמו לתחילת התקופה החדשה, אפשר לבנות חלונות בהקשר תפעולי מתאים; אך אסור להכניס לחלון קלט ערכים שלא היו ידועים בזמן התחזית. לצורך התרגול נבחר כלל פשוט וזהיר: בונים חלונות בנפרד בכל חלוקה כרונולוגית.

גם אחרי שבנינו RNN, חייבים להשוות אותה לקו בסיס. בחיזוי הערך הבא, אחת האפשרויות הפשוטות היא לנבא שהערך הבא יהיה דומה לערך האחרון שנצפה. מודל מורכב שאינו משפר באופן עקבי את ה־Baseline אינו מוצדק רק משום שהוא רשת עמוקה.



קו בסיס של התמדה (*Persistence*): חיזוי הערך הבא לפי המדידה הקודמת.

15.12 | מודל PyTorch מלא לחיזוי הערך הבא

ניצור נתוני חיישן מדומים בעזרת Python. הנתונים אינם קריאות אמיתיות ממחשבים; כך נוכל ללמוד את התהליך בלי צורך להוריד קובץ חיצוני. תחילה נחלק כרונולוגית, נתקנן באמצעות ממוצע וסטיית תקן מנתוני האימון בלבד, וניצור חלונות באורך 12. 1 2

```
import numpy as np
import torch
from torch import nn

np.random.seed(7)
torch.manual_seed(7)
t = np.arange(600)
sensor = 30 + 0.004*t + 2*np.sin(t/12) + np.random.normal(0, 0.25, len(t))
train_raw = sensor[:390]
val_raw = sensor[390:480]
test_raw = sensor[480:]
mu = train_raw.mean()
sigma = train_raw.std()

def make_windows(raw, window=12):
    scaled = (raw - mu) / sigma
    X, y = [], []
    for end in range(window, len(scaled)):
        X.append(scaled[end-window:end])
        y.append(scaled[end])
    X = torch.tensor(np.array(X), dtype=torch.float32).unsqueeze(-1)
    y = torch.tensor(np.array(y), dtype=torch.float32).unsqueeze(-1)
    return X, y

X_tr, y_tr = make_windows(train_raw)
X_va, y_va = make_windows(val_raw)
X_te, y_te = make_windows(test_raw)
print(X_tr.shape, y_tr.shape)
```

הפונקציה unsqueeze מוסיפה ממד אחד של מאפיינים. כך צורת הקלט הופכת מ־(n_windows,12) לצורה (n_windows,12,1), כפי ש־RNN עם batch_first=True מצפה לקבל.

הגדרת המודל ואימון בסיסי

```
class SensorRNN(nn.Module):
    def __init__(self, hidden_size=12):
        super().__init__()
        self.rnn = nn.RNN(input_size=1, hidden_size=hidden_size,
                          batch_first=True)
        self.head = nn.Linear(hidden_size, 1)
```

```
    def forward(self, x):
        sequence_output, hidden = self.rnn(x)
        last_step = sequence_output[:, -1, :]
        return self.head(last_step)

model = SensorRNN(hidden_size=12)
loss_fn = nn.MSELoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)

for epoch in range(25):
    model.train()
    optimizer.zero_grad()
    pred = model(X_tr)
    loss = loss_fn(pred, y_tr)
    loss.backward()
    torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
    optimizer.step()
    model.eval()
    with torch.no_grad():
        val_loss = loss_fn(model(X_va), y_va).item()
    if (epoch + 1) % 5 == 0:
        print(epoch + 1, round(loss.item(), 4), round(val_loss, 4))
```

אנחנו מאמנים דוגמה לימודית על אצווה אחת בכל Epoch כדי שקוד האימון יהיה קריא. בפרויקט גדול נשתמש לרוב ב־DataLoader ובאצוות קטנות. תוצאת המודל תלויה גם במימוש, בזרע האקראי ובנתונים; אסור לטעון מראש שהוא טוב יותר מקו בסיס.

הערכה ביחידות המקוריות

```
model.eval()
with torch.no_grad():
    pred_scaled = model(X_te).squeeze().numpy()
actual_scaled = y_te.squeeze().numpy()
pred_real = pred_scaled * sigma + mu
actual_real = actual_scaled * sigma + mu

rmse_model = np.sqrt(np.mean((pred_real - actual_real)**2))
baseline_real = X_te[:, -1, 0].numpy() * sigma + mu
rmse_baseline = np.sqrt(np.mean((baseline_real-actual_real)**2))
print('RNN RMSE:', round(rmse_model, 3))
print('Persistence RMSE:', round(rmse_baseline, 3))
```

מדוע מחזירים את התחזיות ליחידות המקוריות? משום שקל יותר להסביר לתלמיד או למשתמש שגיאת חיזוי של 0.6 מעלות מאשר 0.4 יחידות תקן. ההשוואה הוגנת רק אם שני המודלים נבדקו על אותם יעדים בזמן. אל תחזרו להתאים את המודל על Test לאחר צפייה בתוצאות.

15.13 | השוואת RNN, GRU ו־LSTM באופן מבוקר

כדי לבדוק האם תאי זיכרון משפרים את התוצאות, נשאיר את אותו מאגר, חלוקה, חלון, מדד ומספר Epochs. נשנה רק את סוג התא. כך נוכל להסביר מה הייתה השאלה הניסויית ואיזה גורם שינינו. מספר הפרמטרים עשוי להיות שונה ולכן אין לומר שהשוואה כזו ״משווה אותו מספר פרמטרים״; אם דרושה בדיקה מדויקת יותר יש להתאים את המודלים בהתאם.

```
def make_recurrent(kind, in_features=1, hidden=12):
    layers = {'RNN': nn.RNN, 'GRU': nn.GRU, 'LSTM': nn.LSTM}
    recurrent = layers[kind](input_size=in_features, hidden_size=hidden,
                             batch_first=True)
    return recurrent

for kind in ['RNN', 'GRU', 'LSTM']:
    net = make_recurrent(kind)
    print(kind, sum(p.numel() for p in net.parameters()))
```

הקטע מציג השוואה של מספר פרמטרים בתא בלבד. בפרויקט המלא צריך להוסיף גם שכבת פלט, לבצע אימון של כל מודל באותם תנאים ולשמור תוצאות מסודרות. המודל שהשיג את ציון הולידציה הטוב ביותר לא בהכרח ינצח בכל מאגר אחר.

15.14 | טעויות נפוצות, אתיקה והגנה בעל־פה

- לערבב את כל חלונות הזמן באקראי בלי לבחון תלות כרונולוגית; במקרה זה ייתכן שהערכת הביצועים אופטימית מדי.
- לחשב StandardScaler על כלל המדידות לפני חלוקת Train/Validation/Test; זו זליגת מידע.
- לבלבל בין ממד האצווה לממד הזמן, או לקחת את hidden האחרון בצורה שאינה תואמת למספר השכבות/הכיוונים.
- להניח ש־LSTM תמיד טובה יותר מ־RNN/GRU, בלי בדיקה מול Baseline וביצועי הולידציה.
- להשתמש בתווית או במדד מן העתיד כמפתח לחיזוי העבר; לעיתים המקור ל״מודל מצוין״ הוא מידע שלא היה זמין בפועל.
- לשמור רק Accuracy או RMSE בלי לתעד תאריכים, פרמטרים, גרסאות והחלטות מקצועיות.

בפרויקטים על נתוני אנשים יש להימנע מהסקת מסקנות לא מוצדקות על אדם מתוך דפוס התנהגות קצר. בסדרות ניטור רפואיות, למשל, הפלט צריך לעבור בחינה מקצועית והערכת סיכונים. גם במעבדת בית ספר יש להבהיר שהמודל מסייע לקבלת החלטה ואינו מחליף אבחון תקלה אמיתי.

15.15 | תרגול מדורג — ידע, חישוב וחשיבה

רמת יסוד

- 1 . הסבירו כיצד שינוי סדר המדידות יכול לשנות את המשמעות בלי לשנות את הממוצע.
- 2 . עבור רצף בן 5 1 מדידות וחלון באורך 4 , כמה דוגמאות לחיזוי הצעד הבא ניתן ליצור? נמקו.
- 3 . ציינו מה מייצג כל ממד בטנזור שצורתו (3 , 1 0 , 2 0), כאשר batch_first=True.
- 4 . הגדירו מהו מצב חבוי והסבירו מדוע הוא אינו שווה בהכרח לתצפית האחרונה.

רמת תרגול מתמטית

- 5 . חשבו שלושה צעדי RNN עם h0=0, Wx=0.2, Wh=0.5, b=0, x=[1,0,1] והפעלה זהותית; השוו ל־tanh.
- 6 . עבור $L(w)=\frac{1}{2}(0.5+0.5w^2-1)^2$ ו־w=0.4, מצאו נגזרת ובצעו עדכון בקצב 1 . 0 .
- 7 . חשבו 0 . 8 בחזקת 1 2 ופירשו מה היא ממחישה על מכפלות גרדיאנטים.
- 8 . ב־LSTM: c_prev=0.4, f=0.6, i=0.2, g=0.5. חשבו c_new ופרשו את שני האיברים.

רמת בגרות, חקר ופרויקט

- 9 . קיבלתם נתוני חיישן לפי תאריך. תכננו חלוקת Train/Validation/Test והסבירו היכן תקנון צריך להתבצע.

- 10. מודל RNN השיג RMSE=0.8 ו־Baseline השיג RMSE=0.6 על Test. האם ההחלטה לבחור ב־RNN מוצדקת? הציעו שני ניסויי המשך.
- 11. דוגמה מציגה ביצועי אימון מעולים אך ולידציה חלשה. הסבירו אפשרויות: התאמת יתר, שינוי התפלגות ומידע חסר.
- 12. הציעו תכנית למערכת לניטור מחשבים: מאפיינים, חלון, יעד, מדד הצלחה, תוצר תוכנה וקריטריון בטיחות.

15.16 | מעבדת Colab — משימות עצמאיות

לצד פרק זה קיימת מחברת Jupyter עם 12 משימות הדרגתיות. חלק מהתאים כוללים סימוני TODO או סימני שאלה כקוד להשלמה — אלה מקומות שבהם התלמיד אמור לכתוב בעצמו. פתרונות מלאים נשמרים לגרסת המורה בלבד.

תוצר ההגשה כולל מחברת פועלת, השוואת Baseline ו־RNN על Test כרונולוגי, גרף תחזיות, טבלת ניסויים ופסקת מסקנות הכוללת לפחות מגבלה אחת. אין חובה לקבל ציון מסוים; חובה להצדיק את התהליך.

15.17 | תכנית הוראה מומלצת ל־30 שעות

מפגש	מוקד	תוצר ביצועי
1 — 3 שעות	רצף, חלון, יעד והבדל מטבלה	טבלת חלונות ידנית
2 — 3 שעות	ארכיטקטורה, שיתוף משקולות ומצב חבוי	תרשים RNN מחושב
3 — 3 שעות	Forward Pass ו־NumPy	שלושה צעדים מחושבים
4 — 3 שעות	BPTT וכלל השרשרת	נגזרת ועדכון ידניים
5 — 3 שעות	Vanishing/Exploding ו־Clipping	גרפים ופירוש
6 — 3 שעות	LSTM ושערים	חישוב מצב תא
7 — 3 שעות	GRU והשוואת ארכיטקטורות	טבלת בחירה מנומקת
8 — 3 שעות	חלונות, חלוקה בזמן ו־Baseline	Pipeline נתונים
9 — 3 שעות	אימון RNN ב־PyTorch והערכה	מודל וגרף תחזיות
10 — 3 שעות	ניסוי מבוקר, כתיבה והגנה	מחברת ודוח קצר

זו חלוקה פדגוגית מוצעת לעשרה מפגשים בני שלוש שעות. את שעות ההתמחות והיקף הדרישה בפועל יש לקבוע על פי תכנית בית הספר והמסמך הרשמי העדכני.

15.18 | הכנה לעבודת הגמר — תיק חקר מינימלי

שאלה שהתלמיד חייב לענות עליה	רכיב
מהם הנתונים, למי הם שייכים ומתי נאספו?	מקור נתונים

שאלה שהתלמיד חייב לענות עליה	רכיב
איזו תחזית מבצעים ומה היה ידוע בזמן התחזית?	**הגדרת יעד**
מדוע נבחר אורך הרצף הזה?	**בחירת חלון**
איזה פתרון פשוט חייבים לנצח?	**Baseline**
מה נבדק: RNN, GRU או LSTM, ומה נשאר קבוע?	**מודלים**
כיצד נבחרו פרמטרים בלי לראות Test?	**ולידציה**
מה גודל הטעות במונחים של המשתמש?	**מדדים**
כיצד מזינים רצף חדש למערכת?	**תוכנה**
מה עלול להשתבש בפעילות בעולם האמיתי?	**מגבלות**

15.19 | מילון מושגים

הגדרה בעברית	מונח
רשת המעדכנת מצב חבוי לאורך רצף	**Recurrent Neural Network**
וקטור פנימי המסכם מידע מהעבר	**Hidden State**
פריסה חזותית של אותו תא לאורך זמן	**Unrolling**
גזירת השגיאה לאחור לאורך צעדי זמן	**BPTT**
היחלשות אות העדכון כאשר נגזרות מוכפלות	**Vanishing Gradient**
גדילת גרדיאנט עד לחוסר יציבות	**Exploding Gradient**
תא חוזר עם מצב זיכרון ושערים	**LSTM**
תא חוזר עם שערי איפוס ועדכון	**GRU**
חלון הקלט ואופק התחזית	**Window / Horizon**
זליגת מידע שאינו זמין בזמן התחזית	**Temporal Leakage**
ניבוי הערך הבא לפי הערך האחרון שנצפה	**Persistence Baseline**

15.20 | מבחן מסכם — האם אתם מבינים רשת סדרתית?

להלן מבחן עצמי קצר לפני המעבדה האחרונה. התשובות אינן מופיעות בגרסת התלמיד; מומלץ להסביר כל בחירה במילים ולא רק לכתוב מספר. אפשר להגיש תשובות בכתב או להציגן בהסבר בן חמש דקות מול תלמיד אחר.

כיצד תראו הבנה	שאלה מסכמת	מוקד
הסבר לפי מצב חבוי	שני רצפים מכילים אותם ערכים בסדר שונה. האם ה־RNN חייב	**סדר ונתונים**

כיצד תראו הבנה	שאלה מסכמת	מוקד
	להחזיר אותו פלט?	
נימוק לכל אחד מהממדים	.X.shape=(32,12,2), hidden_size=8 מה צורת הפלט הסדרתי?	**ממדיות**
חיבור ל־BPTT	מה המשמעות של שיתוף המשקולות לאורך 1 2 צעדים?	**אימון**
ניסוי הוגן ובסיס השוואה	מדוע לא נכון להבטיח ש־LSTM תמיד תנצח GRU?	**מודלים**
מסקנה מקצועית ומגבלה	הדיוק על Test היה חלש יותר מ־Baseline. מה תדווחו?	**בדיקה**

משימת מעבר לפרויקט: הכינו כרטיס ניסוי בן חצי עמוד הכולל הגדרת בעיה, סוג רצף, אורך חלון, חלוקה כרונולוגית, Baseline, מדד מרכזי ומשפט אחד המסביר מתי השימוש במודל לא יהיה אחראי.

15.21 | מקורות קריאה והבחנה בין חובה להרחבה

עוגן תכנית ההתמחות: מאגר Data-Science-ML-Course, פרק 1 5 — רשתות סדרתיות חוזרות (RNN), 3 0 שעות. הקישור למיקום בפרק מאמת את שם הפרק והיקפו במפה; הוא אינו מחליף קריאה של מסמך משרד החינוך הרשמי המלא.

ספרות מקצועית: Zhang, Lipton, Li & Smola — Dive into Deep Learning, פרקי RNN ו־BPTT; Simon Prince — Understanding Deep Learning; Goodfellow, Bengio & Courville — Deep Learning; Aurélien Géron — Hands-On Machine Learning. שיטות ההסבר הותאמו ונכתבו מחדש באופן מקורי לתלמידי תיכון.

תיעוד יישום: PyTorch nn.RNN, nn.GRU, nn.LSTM ו־torch.nn.utils.clip_grad_norm_. יש לקרוא את הגרסה המותקנת כאשר עובדים עם נתוני תלמידים וגרסאות תוכנה שונות.

קישור לחומרי המקור במאגר: https://github.com/samiabo75/Data-Science-ML-Course/tree/main/03-Deep-Learning-5-Units/15-Recurrent-Neural-Networks

D2L: https://en.d2l.ai/chapter_recurrent-neural-networks/index.html

PyTorch RNN: https://docs.pytorch.org/docs/stable/generated/torch.nn.RNN.html

PyTorch GRU: https://docs.pytorch.org/docs/stable/generated/torch.nn.GRU.html

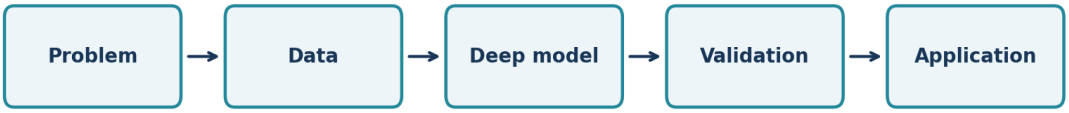
PyTorch LSTM: https://docs.pytorch.org/docs/stable/generated/torch.nn.LSTM.html

מדעי הבינה המלאכותית

פרק 16 — עבודת הגמר

אמיתית למערכת עובדת, תיק פרויקט והגנה בעל־פה AI מבעיית

התמחות למידת מכונה ולמידה עמוקה | 5 יחידות לימוד | מסגרת של 80 שעות



Document decisions, test assumptions and report limitations at every stage

הפרויקט הוא תהליך שלם. איכות התוצאה תלויה גם במידע, בבדיקות ובהסבר — לא רק במודל.

מהדורת תלמיד מורחבת. הדוגמאות, הגרפים והמשימות נכתבו לצורכי הוראה. נתונים מספריים לדוגמה אינם תוצאות אמתיות של פרויקט תלמיד. הבחנה בין הוראות משרד החינוך לבין המלצות פדגוגיות מפורטת בפרק.

מפת הפרק — מה צריך לדעת בסיום?

תוצר הלמידה איננו רק מחברת שפועלת. התלמיד יוכל להציג מערכת המשתמשת במודל למידה עמוקה שהוא בנה ואימן, להסביר את שלבי העבודה, להראות כיצד בדק ושיפר את הביצועים, להוכיח שהיישום מפיק תחזית תקינה, ולהגן על בחירותיו.

יחידה בפרק	שאלת מפתח	ראיה להצלחה
בחירת בעיה	מה בדיוק צריך לחזות, ולמי?	מסמך אפיון עם מדד הצלחה
נתונים	האם אפשר לסמוך על הנתונים?	כרטיס מאגר ודוח EDA
מודל	מה הרשת לומדת וכיצד?	ארכיטקטורה ויומן אימון
הערכה	האם ההצלחה אמיתית?	Test נפרד וניתוח טעויות
יישום	האם משתמש מסוגל להיעזר במודל?	מערכת ומבחני קלט
הגנה	האם התלמיד מבין את עבודתו?	הדגמה והסברים בעל־פה

16.1 מה ההבדל בין תרגיל, מיני־פרויקט ועבודת גמר?

תרגיל כיתה בוחן בדרך כלל מיומנות אחת: למשל חישוב Precision או אימון רשת שמספר שכבותיה נתון. במיני־פרויקט כבר מתקבלת בעיה רחבה יותר, אבל למורה עדיין יש תפקיד משמעותי בבחירת המאגר, הניסוי והארכיטקטורה. בעבודת הגמר התלמיד נדרש לקבל החלטות רבות בעצמו, לתעד אותן ולהסביר אותן. החלטות אלו הן חלק מן התוצר, ולא פעילות נסתרת שמתרחשת מאחורי הקלעים.

לדוגמה, כאשר רשת מזהה ספרות בדיוק של 96%, אין זה מספיק להציג צילום מסך של המספר. על התלמיד לדעת מאין הגיעו התמונות, כיצד חולקו לקבוצות, האם קיימות תמונות דומות מדי בין אימון לבדיקה, מה היה מודל הבסיס, מדוע נבחרו שכבות מסוימות, אילו טעויות בולטות קיימות, וכיצד היישום מטפל בתמונה חדשה שאינה מתאימה לקלט הצפוי.

16.2 העוגן הרשמי — שלושת רכיבי הפרויקט

בהנחיות משרד החינוך לכתיבת תיק פרויקט בחלופת למידת מכונה (סמל 883589) מופיעים שלושה רכיבים מרכזיים: איסוף, הכנה וניתוח הנתונים; בנייה ואימון של מודל למידה עמוקה; ויישום תוכנה המשתמש במודל. נדרשת גם הרחבה והעמקה באחד מן הרכיבים. פרק ההתמחות במפת המאגר שלנו מקצה לעבודת הגמר 80 שעות.

רכיב	מה התלמיד צריך להראות בפועל	דוגמת ראיות
נתונים	מקור, מבנה, חקר והכנה לאימון	Dataset Card, גרפים, פיצול, נרמול
מודל עמוק	רשת שנבנתה ואומנה, הסבר על שכבות ואופטימיזציה	קוד, תרשים שכבות, עקומות Loss, יומן ניסויים
יישום תוכנה	קלט חדש → קדם־עיבוד → חיזוי → תוצאה	מסכים, קוד יישום, בדיקות שגיאה
העמקה	עבודה משמעותית נוספת באחד	החלטת העמקה, השוואה

דוגמת ראיות	מה התלמיד צריך להראות בפועל	רכיב
ותוצאות מתועדות	משלושת הרכיבים	

ההנחיות הרשמיות שנמצאו מפורטות לגבי מבנה הספר, מדריך למפתח, מדריך למשתמש ורפלקציה. מסמך מחוון תשפ״ג שזמין לציבור מציג חלוקה היסטורית של 30% לתיק פרויקט ו־70% לבחינה בעל־פה. אין להניח שחלוקה זו עדיין תקפה במחזור ההגשה הנוכחי; לפני הגשה יש לבדוק מחוון עדכני דרך הפיקוח והמרחב הפדגוגי.

נקודה למחשבה — מסמך זה הוא ספר לימוד, לא תחליף למסמך ההנחיות או למחוון התקפים בשנת ההגשה. המבנים, הטבלאות והמשימות המוצעים כאן הם כלי הוראה, אלא אם צוין במפורש כי מקורם בהנחיות הרשמיות.

16.3 בחירת בעיה מציאותית — לפני שבוחרים רשת

בשלב הראשון מתארים בעיה של משתמש אמיתי ולא סוג אלגוריתם. המשפט "אני רוצה להשתמש ב־CNN" מתאר טכנולוגיה, אך אינו עונה על השאלה מי צריך אותה. לעומתו, המשפט "אני רוצה לאפשר לילדים להזין ספרה כתובה ביד ולקבל פענוח נגיש" מתאר צורך, קלט ותוצאה אפשרית. רק לאחר מכן נשאל אם CNN היא פתרון מתאים מבחינת נתונים, איכות ומאמץ פיתוח.

מסמך אפיון קצר חייב לענות על שש שאלות:

- מי המשתמש? מה הפעולה שהוא רוצה לבצע?
- מהו הקלט המדויק (יחידות, טווחים, פורמט, גודל)?
- מהו הפלט? מחלקה, מספר רציף, מסכה או סדרה?
- מהי טעות מסוכנת או יקרה ומהי טעות נסבלת?
- אילו נתונים נדרשים, והאם מותר להשתמש בהם?
- איזה מדד וערך יעד יסמנו הצלחה? מדוע הם סבירים?

בדוגמת זיהוי הספרות, משתמש מצייר תמונה ולא מעלה תמונה גדולה מן הטלפון. הקלט הצפוי עשוי להיות מטריצת 8×8, והפלט אחת מעשר ספרות. בהמשך נצטרך להגדיר מה קורה אם התמונה ריקה, מטושטשת או כתובה בפורמט אחר. החלטות מסוג זה משפיעות על האפליקציה ועל תוצאות המודל לא פחות מהארכיטקטורה שלו.

16.4 סינון רעיונות והתאמת היקף

לפני בחירת נושא בודקים היתכנות. רעיון עשוי להיות מעניין מבחינה חברתית אך בלתי אפשרי במסגרת שעות הלימוד בגלל קושי להשיג מאגר מתאים, פגיעה בפרטיות, דרישה לתשתיות כבדות או תלות בשירות חיצוני שאי אפשר להפעיל בבחינה. יש לבחור פרויקט שמאפשר למידה עמוקה אמיתית, בניית יישום ועבודה עצמאית.

סיכון שדורש פתרון	רשת אפשרית	נתונים	רעיון
פער בין ציור חדש לבין המאגר	MLP או CNN	תמונות ספרות מתויגות	**זיהוי ספרות בכתב יד**
רקע שמסגיר את	CNN	תמונות עם רשות שימוש	**זיהוי סוג פסולת**

סיכון שדורש פתרון	רשת אפשרית	נתונים	רעיון
התווית			
זליגת מידע ממדידות עתידיות	RNN / LSTM	מדידות מסודרות בזמן	**תחזית מדידות חיישן**
פרטיות, איכות הקלט ומורכבות	מודל רצפי / עמוק	הקלטות עם הסכמה ורישוי	**זיהוי קולי**

16.5 תכנון נתונים ובניית כרטיס מאגר

נתונים הם הבסיס של כל מודל. יש לתעד מי יצר אותם, מתי, מה משמעות כל שדה, מה מותר לעשות איתם ואילו אוכלוסיות או מצבים אינם מיוצגים. לדוגמה, מאגר של ספרות סרוקות וממורכזות לא בהכרח מייצג צילומים בתאורה חלשה או כתב יד על דף מחברת. איכות המודל נמדדת גם ביחס לאוכלוסיית השימוש, ולא רק ביחס למאגר נוח שנמצא באינטרנט.

כרטיס מאגר טוב מכיל שם, מקור וקישור, מספר דוגמאות, סוג התוויות, ממדי הקלט, תנאי איסוף, רישיון, תנאי שימוש, ערכים חסרים, חוסר איזון, סיכונים ופעולות עיבוד. בעת שימוש בנתונים אישיים מתבקשים לצמצם מידע מזהה, לבדוק רשות שימוש ולהימנע מהצגת תמונות, קולות או טקסטים פרטיים בדוח הפומבי.

דוגמה מלווה: **digits** מתוך **scikit-learn**

זהו מאגר לימודי מוכר ובו 1,797 תמונות של ספרות בכתב יד. כל תמונה בגודל 8×8 וכוללת ערכי עוצמה בטווח 0–16; לכן 64 מספרים מייצגים תמונה אחת. השימוש במאגר הזה מאפשר להתמקד בתהליך המחקר, האימון והיישום בלי להסתבך בהורדת קבצים. הוא אינו מייצג באופן מלא קלט חופשי שמגיע ממשתמשי מערכת אמיתית.

$$X \in R^{(1797\times64)} \qquad y \in \{0, 1, ..., 9\}^{(1797)}$$

16.6 חקר נתונים (EDA) אינו רק יצירת גרפים

חקר נתונים טוב מתחיל משאלה. לדוגמה: האם כל הספרות מיוצגות במידה דומה? האם חלק מהתמונות כהות במיוחד? האם יש תוויות שגויות? האם החלק המרכזי של התמונה נושא את רוב המידע? גרף שכיחויות וגלריית תמונות אינם קישוט לדוח אלא ראיות בעזרתן מחליטים כיצד לעבד נתונים ומהן מגבלותיהם.

יש לנסח לכל גרף ממצא, אפשרות לפרשנות ומגבלה. "בספרה 1 מופיעים פיקסלים בהירים באזור המרכז" הוא ממצא אפשרי; "לכן הרשת תזהה כל 1 בעולם" היא טענה שאין לה בסיס. בנוסף, כאשר מציגים ממוצעים או מתאמים, אסור להסיק סיבתיות מן הקשר בלבד.

16.7 חלוקת נתונים — שמירת מבחן אמיתי



Example split for illustration; adapt to dataset and dependence structure

חלוקה לדוגמה בלבד. חלוקה אקראית מתאימה רק אם אין תלות בין הדוגמאות שגורמת להדלפת מידע.

חלוקה בין Train, Validation ו־Test שומרת על שלוש מטרות שונות. נתוני האימון משמשים לעדכון משקולות. נתוני הולידציה משמשים להשוואת ארכיטקטורות, קצבי למידה וספי החלטה. נתוני המבחן נשמרים להערכת התוצר לאחר שהחלטות התכנון הוכרעו. אם בודקים שוב ושוב את Test ומשנים את הרשת בהתאם, הוא מפסיק להיות מבחן בלתי תלוי.

בנתוני סדרות זמן יש לשמור על הסדר הכרונולוגי; בנתונים של אותו אדם או אותו מוצר יש לשקול פיצול לפי קבוצות, כדי שדוגמאות קשורות לא יופיעו גם באימון וגם בבדיקה. קנה מידה כגון ממוצע וסטיית תקן חייב להילמד מקבוצת האימון בלבד, ולהיות מיושם באותו אופן בולידציה, במבחן ובאפליקציה.

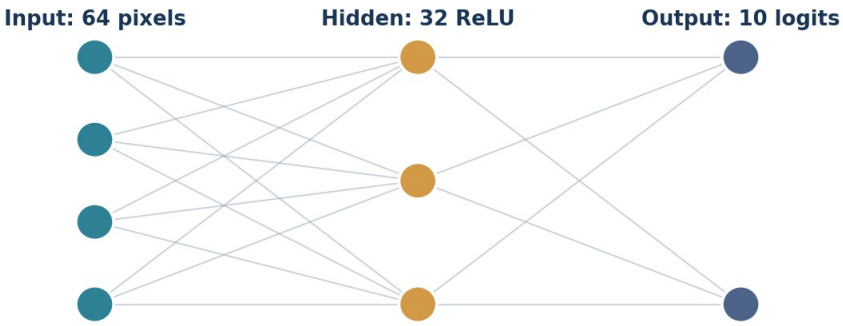
16.8 מהו Baseline ומדוע הוא חובה מקצועית?

מודל בסיס הוא פתרון פשוט שנבנה קודם למודל העמוק. בסיווג אפשר לבדוק ניבוי של המחלקה השכיחה ביותר, KNN שכבר למדנו, או כלל חישובי. בחיזוי סדרה אפשר לנבא שהערך הבא שווה לערך האחרון. אם רשת עמוקה אינה משפרת את הבסיס לפי המדד שנבחר, אין זה כישלון פדגוגי: זו ראיה שצריך לבחון את הנתונים, תהליך האימון או הצדקת מורכבות המודל.

נקודה למחשבה — יש להשוות מודלים על אותה חלוקה, באותו מדד ובתנאי קדם־עיבוד בני־השוואה. השוואה לפי *Accuracy* בלבד אינה מספיקה כאשר לטעויות מסוגים שונים יש מחיר שונה.

16.9 כיצד מתכננים ארכיטקטורת למידה עמוקה?

הארכיטקטורה מתרגמת את סוג הקלט לסדרה של פעולות. עבור 64 ערכי פיקסל ניתן להתחיל ברשת פשוטה בעלת שכבה חבויה של 32 יחידות ReLU ושכבת פלט של 10 ערכים. השכבה האחרונה מחזירה logits, כלומר ציונים גולמיים, ולא בהכרח הסתברויות. בעת אימון עם CrossEntropyLoss ב־PyTorch מוסרים את ה־logits ישירות; אין להחיל Softmax לפני פונקציית האובדן הזו.



ארכיטקטורה אפשרית לדוגמת *digits*. מספר העיגולים באיור הוא סכמטי; הממדים כתובים מעליו.

$$64 \times 32 + 32 + 32 \times 10 + 10 = 2410 \text{ parameters}$$

מספר הפרמטרים המחושבים הוא 2,410: 2,048 משקולות ו־32 הטיות בשכבה הראשונה, ו־320 משקולות ו־10 הטיות בשכבת הפלט. זהו תרגיל מתמטי חשוב משום שהוא מאפשר לתלמיד להעריך את מורכבות המודל ולבדוק את מספר הפרמטרים שמדפיסה הספרייה.

16.10 דוגמה מתמטית — מה פירוש Cross-Entropy?

נניח שהרשת מפיקה שלושה ציונים גולמיים עבור שלוש מחלקות: 2, 1 ו־0. פונקציית Softmax ממירה ציונים להסתברויות יחסיות שסכומן 1. אם המחלקה האמיתית היא הראשונה, השגיאה היא מינוס הלוגריתם של ההסתברות שהרשת נתנה למחלקה זו.

$$p_i = \exp(z_i) / \Sigma_j \exp(z_j)$$

$$p_{true} = \exp(2) / [\exp(2)+\exp(1)+\exp(0)] \approx 0.6652$$

$$L = -\ln(p_{true}) \approx 0.4076$$

המשמעות: כאשר הרשת מקצה הסתברות גבוהה למחלקה הנכונה, ערך האובדן קטן. עם זאת, ערך Loss של דוגמה בודדת אינו Accuracy ואינו מדד מספיק להצלחת המערכת. באימון יש לבחון גם התפלגות השגיאות, עקומות Train/Validation ומדדי הביצועים על קבוצות שלא שימשו לעדכון הפרמטרים.

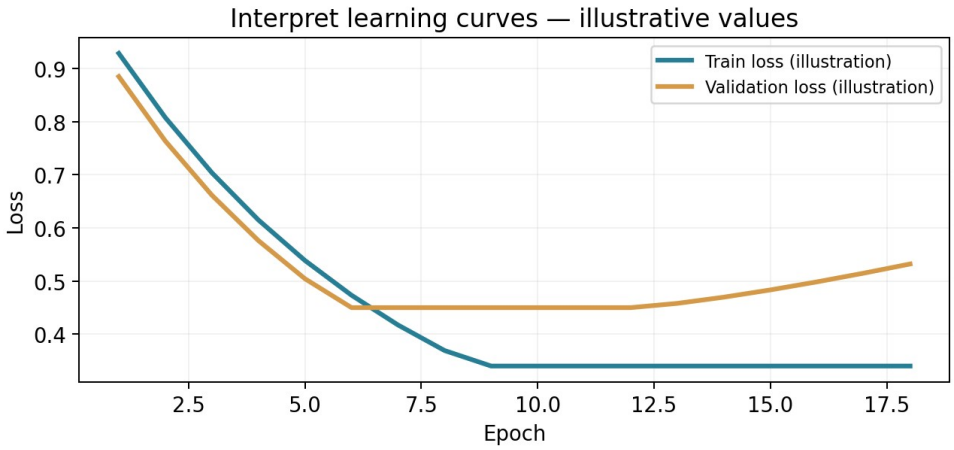
16.11 תכנון ניסוי: מה משתנה ומה נשאר קבוע?

ניסוי טוב משנה החלטת תכנון אחת בכל פעם כאשר הדבר מעשי. אפשר להתחיל ברשת 64–32–10, לשמור את החלוקה, את הזרע האקראי ואת מספר ה־Epochs, ואז לבדוק השפעת שינוי מספר היחידות החבויות. בניסוי אחר נשנה את קצב הלמידה. כך נוצרת יכולת לייחס הבדל שנצפה לבחירה מוגדרת, בלי להתבלבל בין שינויים רבים שבוצעו יחד.

מספר ניסוי	השערה	מה שינינו	מה מודדים
E01	רשת קטנה תספק בסיס הוגן	ארכיטקטורת התחלה	בולידציה Loss ו־Accuracy
E02	הגדלת השכבה עשויה לשפר יכולת ייצוג	64 → 32 יחידות	אותם מדדים ותנאי אימון
E03	קצב למידה נמוך ישפיע על התכנסות	η=0.01 → η=0.001	עקומת שגיאה לאורך Epochs

ביומן הניסויים נדרשים גם תאריך, גרסת קוד, נתוני אימון, חומרת הרצה אם רלוונטית, תוצאה, פירוש ומגבלה. מסקנה מקצועית אינה רק "מודל E02 יותר טוב", אלא "בחלוקה ובמספר Epochs שנבדקו, E02 השיג שיפור של כך וכך; יש לבדוק אם השיפור יציב".

16.12 עקומות אימון — מתי עוצרים?



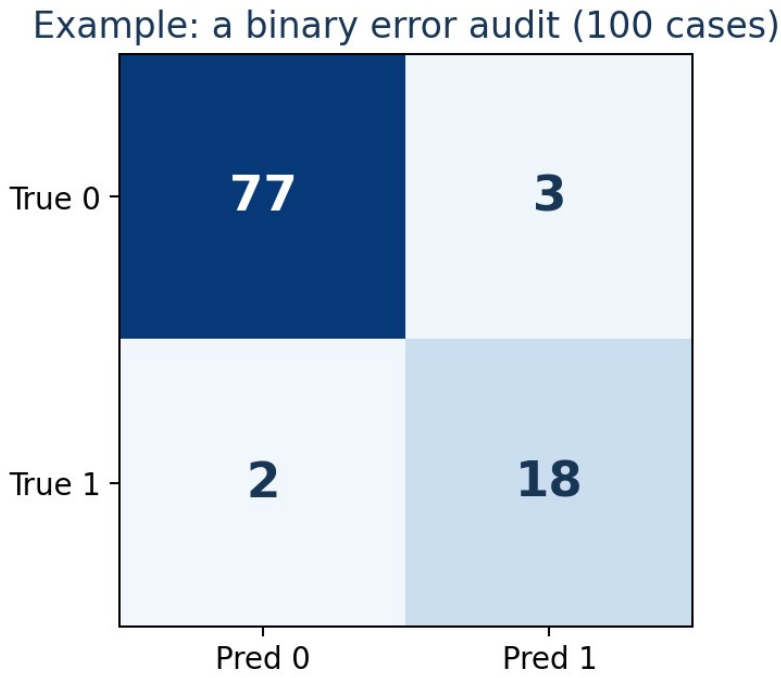
עקומות מומצאות להמחשת הרעיון; אין לפרש אותן כתוצאות ניסוי שבוצע.

עקומת Train Loss מלמדת אם המודל מצליח להתאים את עצמו לדוגמאות האימון. עקומת Validation Loss מלמדת אם השינוי משתפר גם על דוגמאות שלא שימשו לעדכון המשקולות. כאשר Train Loss ממשיך לרדת אבל Validation Loss מתחיל לעלות לאורך זמן, ייתכן Overfitting. זה אינו פסק דין אוטומטי: יש לבחון תנודתיות, גודל המדגם ואופן המדידה.

Early Stopping היא החלטה לעצור לפי תוצאות הולידציה ולהחזיר את מצב המשקולות הטוב שנשמר. אין להחליט על מועד העצירה לאחר הצצה לתוצאות Test. כמו כן חשוב להבדיל בין שימוש במדד האובדן לתהליך האימון לבין מדד העסקי או המחקרי שבו מקבלים החלטה על איכות הפרויקט.

16.13 הערכה סופית וניתוח טעויות

כאשר הוחלט על הארכיטקטורה ועל מדיניות האימון, מריצים הערכה סופית על Test. עבור סיווג רב־מחלקתי משתמשים במטריצת בלבול, במדדי Precision/Recall/F1 לפי צורך ובדוגמאות שגויות מייצגות. הדיוק הכללי חשוב, אבל אינו מסביר אילו מחלקות המודל מבלבל ומדוע.



דוגמה מספרית נפרדת של בעיית סיווג בינארית; היא אינה מטריצת הבלבול של מאגר הספרות.

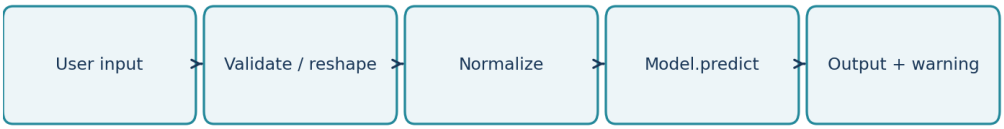
במטריצה זו TP=18, FN=2, FP=3, TN=77. לכן Accuracy הוא 95%, Precision הוא 18 חלקי 21 (כ־85.7%), ו־Recall הוא 18 חלקי 20 (90%). כאשר החלטה שגויה עשויה להזיק למשתמש, אין להסתפק ב־Accuracy. צריך גם לתעד דוגמאות שהמודל מחמיץ, ולבדוק האם יש תת־קבוצות שקשה לו במיוחד לזהות.

$$\text{Accuracy} = (TP + TN) / N = 95 / 100 = 0.95$$

$$\text{Precision} = 18/21 \approx 0.8571 \qquad \text{Recall} = 18/20 = 0.90$$

16.14 מדוע יישום תוכנה הוא רכיב נפרד מהמודל?

רשת שפועלת במחברת אינה בהכרח מערכת שמישהו מסוגל להשתמש בה. היישום נדרש לקבל נתון חדש, לבדוק שהוא חוקי, להתאים אותו לאותו פורמט ששימש באימון, להפעיל את המודל שהוכשר ולהציג פלט ברור. תלמיד שמגיע עם Notebook בלבד ועדיין לא שילב את המודל ביישום, לא הראה את מלוא מחזור החיים המתואר בהנחיות החלופה.



שרשרת היסק: אין לבצע באפליקציה עיבוד אחר מזה ששימש באימון.

לדוגמה: אם הרשת אומנה על תמונות 8×8 בעוצמות 0–16 המנורמלות לחלוקה ב־16, אין להזין ישירות צילום בגודל 224×224 ובטווח 0–255. צריך להגדיר תהליך המרה מפורש, לבדוק אותו ולתעד את

מגבלותיו. בלי התאמה זו המודל יכול להחזיר מספר, אבל המספר אינו תחזית אמינה באותה משמעות שבה אומן.

16.15 דוגמת קוד מונחית — שמירת מודל ובדיקת קלט

הדוגמה שלהלן מדגישה את מבנה האחריות של קוד היישום. היא אינה מהווה פרויקט מוגמר, ואינה מחליפה את תהליך האימון הנדרש לתלמיד. הקפדה על צורת הקלט מונעת שגיאות שניתן לאתר עוד לפני הרצת הרשת.

```
import numpy as np

def prepare_digit(image):
    arr = np.asarray(image, dtype=np.float32)
    if arr.shape != (8, 8):
        raise ValueError("Expected an 8x8 image")
    if not np.isfinite(arr).all():
        raise ValueError("Image contains invalid values")
    if np.any(arr < 0) or np.any(arr > 16):
        raise ValueError("Pixel values must be in [0, 16]")
    return (arr.reshape(1, 64) / 16.0)
```

יש להסביר מדוע פעולת reshape משנה את צורת הנתונים אך לא את ערכיהם, מדוע מחלקים ב־16 ולא ב־255 בדוגמה הזו, ומדוע חוסמים ערכים שאינם בתחום שהמודל ראה באימון. במערכת אמיתית יש להתאים את מדיניות החריגות לשימוש, ולא רק לזרוק שגיאה לא מוסברת למשתמש.

16.16 תכנון ארכיטקטורת מערכת

מומלץ להפריד בין שלושה רכיבים תוכנתיים: שירות קדם־עיבוד, שכבת חיזוי וממשק משתמש. ההפרדה מאפשרת לבדוק כל חלק בנפרד. אפליקציה שולחנית, Web או Android עשויות להתאים, בתנאי שהמודל אכן משולב ביישום המתאים ושאפשר להדגים את פעולתו באופן יציב בבחינה.

רכיב	אחריות	בדיקה חשובה
DataPreprocessor	מתאים קלט למבנה, לערכים ולסוג הנתונים	תמונה לא נכונה נדחית עם הודעה
ModelService	טוען מודל שמור ומבצע תחזית	אותו קלט מחזיר תחזית תקינה ועקבית
UserInterface	מקבל קלט ומציג תוצאה ומגבלות	המשתמש מבין איך להפעיל ולתקן
Logger (ללא נתונים רגישים)	מתעד תקלה בלי לשמור מידע אישי שלא לצורך	רישום מועיל ללא חשיפת פרטיות

16.17 בדיקות תוכנה שאינן קשורות למדד Accuracy

בדיקות המודל ובדיקות התוכנה משלימות זו את זו. Accuracy גבוה אינו מגן מפני קריסה של מסך, העלאת קובץ פגום, כשל בטעינת משקולות או הצגה שגויה של תווית. לכן נחלק את הבדיקות לבדיקות יחידה של פונקציות קדם־עיבוד, בדיקות אינטגרציה בין רכיבי המערכת ובדיקות שימוש עם קלט תקין ושגוי.

מקרה	קלט	התנהגות צפויה
קלט חוקי	מטריצה 8×8 עם ערכים 0–16	תוצג ספרה חזויה ומידע על מגבלות
גודל לא תקין	מערך 10×10	הודעה ידידותית; אין לבצע תחזית
ערך חסר	NaN בתוך תמונה	הודעת שגיאה והנחיה לתיקון
טווח לא תואם	פיקסל בעוצמה 255	דחייה או המרה מאומתת לפי האפיון
מודל חסר	קובץ משקולות לא קיים	הודעה ברורה ולא סגירה שקטה

16.18 בטיחות, פרטיות, ייחוס וכלי AI

פרויקט טוב אינו אוסף יותר מידע מכפי שנדרש. לפני שימוש בתמונות, הקלטות או נתונים אישיים, יש לברר הרשאת שימוש, רישיון ומגבלות שיתוף. אין להכניס סיסמאות או אסימוני גישה למחברות ציבוריות או ל־GitHub. בנוסף, אין לטעון באופן אוטומטי קובצי מודל ממקור לא מוכר בפורמט שמאפשר הרצת קוד לא מהימן.

מותר לשלב כלי AI לצורך למידה, ניסוח, חקר שגיאות ובדיקת הסברים, לפי מדיניות בית הספר. מבחינה פדגוגית חשוב לתעד איזו עזרה התקבלה, מה נבדק עצמאית ומה נכתב על ידי התלמיד. תיעוד כזה מגן על אמינות העבודה ומסייע לתלמיד להסביר באמת את הקוד שהוא מגיש. אין להציג פלט של כלי AI כהוכחה לתקינות ללא הרצה ובדיקה.

16.19 מבנה תיק הפרויקט

בהנחיות משרד החינוך מופיעים שער, תוכן עניינים, מבוא, ארכיטקטורה, מדריך למפתח, מדריך למשתמש, רפלקציה, מקורות ונספחים. להלן הצעת פירוק מעשית, השומרת על המבנה ובתוכה מסדרת את ראיות העבודה שלנו:

פרק בתיק	חומר שיש להכין
שער ותוכן עניינים	פרטי העבודה לפי הנחיות בית הספר, כותרות ומספור
מבוא	צורך, משתמשים, מטרות, מחקר מקדים ופתרונות חלופיים
ארכיטקטורה — נתונים	מקור, מבנה, EDA, חלוקה והכנה לאימון
ארכיטקטורה — מודל	תרשים רשת, אובדן, אימון, היפר־פרמטרים וניסויים
ארכיטקטורה — יישום	תרשים מערכת, מעבר נתונים, קוד קלט וחיזוי
מדריך למפתח	קבצים, מחלקות, פונקציות, התקנה וגרסאות
מדריך למשתמש	מסכים, הוראות, מגבלות והודעות
רפלקציה	מה למדתי, מה נכשל, ומה הייתי עושה אחרת

חומר שיש להכין	פרק בתיק
ראיות ניסוי, קוד, נתונים, בדיקות ומקורות	**נספחים**

16.20 כתיבת רפלקציה מקצועית

רפלקציה אינה המקום לכתוב רק "נהניתי מהפרויקט". היא מתארת את ההתפתחות של התלמיד: רעיון שהתברר כלא מתאים, ניסוי שהראה תוצאה מפתיעה, באג שנמצא בבדיקת קלט, שיחה עם משתמש ששינתה את המסך, או החלטה מקצועית שהתלמיד היה מקבל אחרת אילו התחיל מחדש.

דוגמה לניסוח מחקרי: "בתחילה בחרתי CNN מתוך הנחה שמודל עמוק ישיג תוצאה טובה יותר. לאחר השוואת קו בסיס גיליתי שהיתרון אינו מובטח במאגר הקטן. שיניתי את סדר הניסויים והוספתי בדיקות לנתוני קלט חדשים. מן התהליך למדתי שמורכבות אינה תחליף למדידה". אין להעתיק את הפסקה הזו כהצהרת תלמיד; היא מדגימה את סוג החשיבה המצופה.

16.21 הכנה להגנה בעל־פה — להראות הבנה ולא רק מצגת

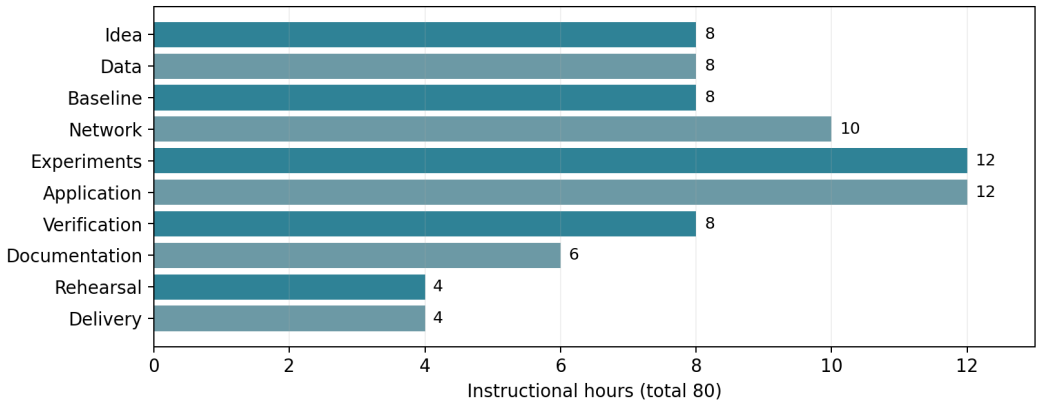
בוחן עשוי לבקש מן התלמיד להפעיל את התוכנה, להסביר את גבולות היכולת של המודל, לפתוח קובץ קוד ולתאר כיצד ישנה רכיב. תלמיד שמכיר רק את המצגת לא יוכל להגן על עבודתו. לכן כל תלמיד יכין הדגמה קצרה, יתאמן במענה לשאלות וידע להסביר נתיב מלא: מקלט חדש, דרך עיבוד וטעינת מודל, ועד הצגת תחזית.

שאלות לדוגמה לתרגול בעל־פה:

- מדוע בחרת בבעיה הזו, ולמה נדרש מודל למידה עמוקה?
- האם הדוגמאות במאגר דומות לקלט שמגיע מהמשתמש?
- כיצד מנעת Data Leakage? איך בחרת היפר־פרמטרים?
- מהו מספר הפרמטרים ברשת ומה תפקיד פונקציית ההפעלה?
- מה ההבדל בין Loss לבין Accuracy ולמה שניהם נמדדו?
- כיצד היישום ממיר קלט חדש למבנה מתאים למודל?
- איזה מקרה קצה עלול להכשיל את המערכת? איך תטפל בו?
- אם אבקש להוסיף מחלקה חדשה, אילו חלקים במערכת תצטרך לשנות?

16.22 תכנון עבודה ל־80 שעות

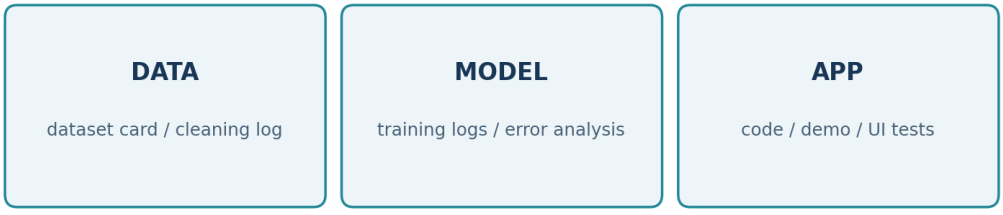
החלוקה להלן היא תכנית הוראה מוצעת של הספר, לא חלוקת שעות רשמית פנימית. זמני הפרויקט האמיתיים משתנים לפי רמת הקבוצה, סוג הנתונים ויכולת התלמידים. בכל שער־שלב נדרש תוצר מוחשי שישמר בתיק הפרויקט.



חלוקה מוצעת ל־80 שעות, עם בקרה ותוצר מסכם בסיום כל שלב.

שלב	שעות	תוצר מחייב בכיתה
בחירת בעיה ואפיון	8	מסמך אפיון ותבחיני הצלחה
נתונים ואישורים	8	כרטיס מאגר ואישור מקור
EDA וקו בסיס	8	ניתוח נתונים והשוואת Baseline
ארכיטקטורה	10	תרשים רשת וקוד אימון ראשון
ניסויים ושיפור	12	יומן ניסויים, גרפים והחלטת מודל
יישום תוכנה	12	ממשק פועל המשתמש במודל
בדיקות וניתוח כשלים	8	בדיקות תוכנה, הערכה ודיון במגבלות
כתיבת תיק	6	תיק פרויקט ומדריכי שימוש
הכנה להגנה	4	הדגמה ומענה מנומק לשאלות
אריזה ובקרה סופית	4	קבצים מסודרים, גרסאות ובדיקת הפעלה

16.23 מטריצת ראיות — כיצד נוכיח שהעבודה בוצעה?



לכל רכיב יש תוצרים שאפשר לבדוק. צילום מסך יחיד של תוצאה אינו מחליף רצף ראיות.

מטריצת ראיות מסייעת להימנע מהמצב שבו תלמיד עושה עבודה איכותית אך אינו מתעד אותה. הדגש אינו על כמות התמונות בדוח אלא על התאמה בין טענה לבין בדיקה. למשל, אם כתוב "שיפרתי את המודל", יש לצרף טבלת ניסויים שמראה את מצב ההתחלה ואת מצב הסיום באותן שיטות מדידה.

16.24 מחוון הוראה מוצע — לא מחוון משרד החינוך

לצורך משוב שוטף בכיתה מוצע להשתמש במפת 100 נקודות הבאה. היא אינה קובעת את ציון הבגרות ואינה מחליפה את משקלי המחוון התקף של הפיקוח; מטרתה לאפשר לתלמיד לזהות מוקדם חולשות מקצועיות ולתקנן.

שאלה מנחה	משקל מוצע	תחום
האם הבעיה ברורה והנתונים מתאימים ומותרים?	20	**בעיה, נתונים ואתיקה**
האם הרשת והאימון מוסברים ונבדקו היטב?	25	**מודל ואימון**
האם יש Baseline, Test וניתוח שגיאות?	20	**הערכה וניסוי**
האם מערכת פועלת עם קלט חדש ומקרי קצה?	20	**יישום תוכנה**
האם אפשר לשחזר ולהסביר עצמאית?	15	**תיעוד והגנה**

16.25 פרויקט מודרך: זיהוי ספרות — תרגיל העברה

כעת נחבר את החלקים לתכנון ראשוני של פרויקט אחד. בחרו את מאגר digits. נסחו מטרת מוצר ברורה, בנו כרטיס נתונים, השוו קו בסיס ורשת MLP, בחרו מדדי הערכה, שמרו את המודל, ובנו ממשק שמקבל תמונה בפורמט שהוגדר. אם תשתמשו בקלט של ציור חופשי, תצטרכו לתעד ולהוכיח את שלב המרת הקלט — לא די לשנות את גודל המערך ללא בדיקה.

המשימות במחברת Colab מלוות את תהליך החשיבה. אינן מספקות פתרון מלא לעבודת הגמר, משום שהפרויקט דורש גם בחירת נושא עצמאית, פיתוח יישום, תיעוד ועמידה בהנחיות העדכניות. ניתן להשתמש בדוגמה המודרכת כדי ללמוד כיצד לנהל ראיות ולהסביר החלטות, אבל אין להגיש אותה כשלעצמה כפרויקט מקורי ללא פיתוח עצמאי מתאים.

16.26 משימות הערכה מדורגות

רמה א׳ — שליטה ביסודות

משימה 1: הגדירו מהי אוכלוסיית השימוש, מהו Dataset ואילו תנאים דרושים לכך שביצועי Test יתנו הערכה הגיונית. משימה 2: חשבו כמה משקולות והטיות קיימות ב־MLP בגודל 64–32–10 והסבירו את שלבי החישוב. משימה 3: הסבירו מדוע בעת שימוש ב־CrossEntropyLoss אין להפעיל Softmax פעמיים.

רמה ב׳ — שימוש בידע

משימה 4: תוכננו שתי ארכיטקטורות. אחת טובה יותר באימון אך פחות טובה בולידציה. נתחו שלוש סיבות אפשריות והציעו ניסוי מבוקר לכל אחת. משימה 5: חשבו Accuracy, Precision ו־Recall מטבלה שבה

TP=18, FN=2, FP=3, TN=77, ובחרו מדד מתאים לבעיה שבה החמצה יקרה יותר מאזעקת שווא. משימה 6: תארו שלושה מקרי בדיקה ליישום שאינו קשור כלל לדיוק המודל.

משימה 7: תכננו מערך נתונים שבו חלוקה אקראית תיצור זליגת מידע, והציעו חלופה מתאימה. משימה 8: בחרו סוג העמקה אחד — נתונים, מודל או יישום — והגדירו כיצד תמדדו את תרומתו. משימה 9: כתבו מכתב קצר למשתמש המסביר מתי אסור להסתמך על המערכת. משימה 10: תכננו חקר של שגיאות בפרויקט שלכם. משימה 11: הציגו תרחיש שבו Baseline עדיף על הרשת. משימה 12: הכינו סרטוט ותסריט הדגמה להגנה בעל־פה שכולל תקלה מתוכננת אחת וכיצד מאתרים אותה.

16.27 כרטיסי עבודה מוכנים למילוי

א. כרטיס אפיון פרויקט

שם הפרויקט: _____

צורך של המשתמש: _____

קלט, פלט ומדד הצלחה: _____

מגבלה או סיכון ידוע: _____

ב. יומן ניסויים

מספר וגרסת ניסוי: _____

השערה והפרמטר ששונה: _____

תוצאת Validation: _____

מסקנה והניסוי הבא: _____

ג. כרטיס מודל

ארכיטקטורה וגרסה: _____

Dataset, רישוי וחלוקה: _____

מדדים ומגבלות שימוש: _____

קישור לראיות בדיקה: _____

ד. רשימת הגשה

האם קיים קובץ שמפעיל את היישום? _____

האם כל הנתיבים והגרסאות מתועדים? ____

האם ניתן להציג קלט חדש? _____

האם התיק כולל רפלקציה ומקורות? _____

16.28 הקשר בין שלוש תכניות הלימוד

תכנית	מה היא תורמת	המשך בספר המאוחד
יחידה 3 — כיתה י׳	קריאת נתונים, Python, EDA, מודלים בסיסיים	חשיבה מחקרית וקו בסיס
יחידה 5 — נושאי ליבה	וקטורים, סטטיסטיקה, מרחקים, KNN, הערכה	הצדקה מתמטית ומדידת איכות
התמחות DL — י״א-י״ב	רשתות עמוקות, אימון, CNN/RNN ופיתוח	מערכת AI, תיעוד ועבודת גמר

מפת ההקשרים היא בחירה פדגוגית שלנו. אין פירושה שכל מיומנות המוזכרת בפרק זה היא יעד חובה ביחידה 5 העיונית. הדרישה לעבודה הכוללת מודל למידה עמוקה ויישום תוכנה משויכת לחלופת ההתמחות בלבד, לפי ההנחיות שנסקרו.

16.29 מקורות והערת גרסה

- משרד החינוך — הנחיות לכתיבת תיק פרויקט בחלופה למידת מכונה, סמל 883589: https://meyda.education.gov.il/files/CSIT/MachinLearning-Project.pdf
- משרד החינוך — מחוון חלופת למידת מכונה 5 יחידות, תשפ״ג (לצורך עיון היסטורי; יש לבדוק עדכון): https://meyda.education.gov.il/files/CSIT/machinLearningProject_Finalt-5.pdf
- משרד החינוך — המרחב הפדגוגי של הנדסת תוכנה, בינה מלאכותית וסייבר: https://pop.education.gov.il/tchumey_daat/computer-science-software-engineering-cyber/high-school/pedagogy-of/high-school-software-cyber-engineering/
- Aurélien Géron — Hands-On Machine Learning, כבסיס לשיטה הנדסית ולניסויים.
- Zhang, Lipton, Li & Smola — Dive into Deep Learning, כבסיס להסבר מודלים ואימון.
- Simon J. D. Prince — Understanding Deep Learning, כבסיס לחשיבה על רשתות ואובדן.
- Google — Rules of Machine Learning: https://developers.google.com/machine-learning/guides/rules-of-ml
- Mitchell et al. — Model Cards for Model Reporting (2019): https://research.google/pubs/model-cards-for-model-reporting/
- מאגר ההוראה שלנו: https://github.com/samiabo75/Data-Science-ML-Course/tree/main/03-Deep-Learning-5-Units/16-Final-Project

נקודה למחשבה — מהדורת ההוראה הזו נכתבה כפרק עצמאי בספר המאוחד. לפני הגשה מעשית יש לוודא שהנחיות הפיקוח, מחוון ההערכה, מועדי ההגשה ודרישות הטכנולוגיה נכונים לשנת ההגשה הספציפית.